SOURCE CODE DEPOSIT (plain text)
Work: Point-of-sale (POS) application (web, backend, mobile)
Total source files: 231
Generated: 2026-04-17

Comments removed. Secrets/credentials replaced with [REDACTED]. Paths below are relative to pos/.

==========

FILE: pos/backend/analytics_response.json
==========

```
{
  "success": true,
  "data": {
    "kpis": {
      "totalSales": 17923,
      "totalTransactions": 12,
      "totalUnitsSold": 49,
      "cogs": 12955,
      "totalProfit": 4968,
      "profitMargin": 27.72,
      "inventoryValue": 62077,
      "costValue": 35877,
      "totalStockUnits": 184
    },
    "stats": {
      "totalItems": 5,
      "inStockCount": 4,
      "lowStockCount": 1,
      "outOfStockCount": 0,
      "stockInCount": 74,
      "stockOutCount": 50,
      "pullOutCount": 12
    },
    "inventoryChartData": [
      {
        "period": "2022",
        "stockIn": 0,
        "stockOut": 0,
        "pullOut": 0
      },
      {
        "period": "2023",
        "stockIn": 0,
        "stockOut": 0,
        "pullOut": 0
      },
      {
        "period": "2024",
        "stockIn": 0,
        "stockOut": 0,
        "pullOut": 0
      },
      {
        "period": "2025",
        "stockIn": 0,
        "stockOut": 0,
        "pullOut": 0
      },
      {
        "period": "2026",
        "stockIn": 74,
        "stockOut": 50,
        "pullOut": 12
      }
    ],
    "profitChartData": [
      {
        "period": "2022",
        "revenue": 0,
        "cogs": 0,
        "profit": 0
      },
      {
        "period": "2023",
        "revenue": 0,
        "cogs": 0,
```

```
        "profit": 0
      },
      {
        "period": "2024",
        "revenue": 0,
        "cogs": 0,
        "profit": 0
      },
      {
        "period": "2025",
        "revenue": 1200,
        "cogs": 800,
        "profit": 400
      },
      {
        "period": "2026",
        "revenue": 16723,
        "cogs": 12155,
        "profit": 4568
      }
    ],
    "damagedExpired": {
      "items": [
        {
          "itemName": "BKK DRESS",
          "sku": "TOP-JHMEEO-RED",
          "category": "Tops",
          "type": "Damaged",
          "quantity": 5,
          "date": "2026-02-15T06:55:08.250Z",
          "handledBy": "Erika Bonifacio",
          "notes": ""
        },
        {
          "itemName": "MB1",
          "sku": "BOT-JWX1KO",
          "category": "Bottoms",
          "type": "Expired",
          "quantity": 3,
          "date": "2026-02-15T06:55:08.250Z",
          "handledBy": "Erika Bonifacio",
          "notes": ""
        },
        {
          "itemName": "running shoes",
          "sku": "SHO-149EJC-BLU",
          "category": "Shoes",
          "type": "Damaged",
          "quantity": 4,
          "date": "2026-02-15T06:55:08.250Z",
          "handledBy": "Erika Bonifacio",
          "notes": ""
        }
      ],
      "summary": {
        "damaged": 9,
        "expired": 3,
        "lost": 0,
        "total": 12
      }
    }
  }
}
```


FILE: pos/backend/check_db.js
==========

```
const mongoose = require("mongoose");
const dotenv = require("dotenv");
const path = require("path");
const fs = require("fs");
dotenv.config({ path: path.join(__dirname, ".env") });
const uri = process.env.MONGODB_URI;
async function check() {
  let output = "";
  const log = (msg) => {
    console.log(msg);
```

```
    output += msg + "\n";
  };
  try {
    await mongoose.connect(uri);
    log("Connected!");
    const Product = require("./models/Product");
    const SalesTransaction = require("./models/SalesTransaction");
    const StockMovement = require("./models/StockMovement");
    const VoidLog = require("./models/VoidLog");
    const productCount = await Product.countDocuments();
    const txnCount = await SalesTransaction.countDocuments();
    const movementCount = await StockMovement.countDocuments();
    const voidLogCount = await VoidLog.countDocuments();
    log(`Products: ${productCount}`);
    log(`Transactions: ${txnCount}`);
    log(`Stock Movements: ${movementCount}`);
    log(`Void Logs: ${voidLogCount}`);
    if (txnCount > 0) {
      const txns = await SalesTransaction.find().sort({ createdAt: -1 }).limit(5);
      log("--- Last 5 Transactions ---");
      txns.forEach((t) => {
        log(`ID: ${t._id}, Created: ${t.createdAt}, Amount: ${t.totalAmount}`);
      });
    }
    if (movementCount > 0) {
      const movs = await StockMovement.find().sort({ createdAt: -1 }).limit(5);
      log("--- Last 5 Stock Movements ---");
      movs.forEach((m) => {
        log(`ID: ${m._id}, Created: ${m.createdAt}, Type: ${m.type}, Qty: ${m.quantity}`);
      });
    }
    if (voidLogCount > 0) {
      const voids = await VoidLog.find().sort({ voidedAt: -1 }).limit(5);
      log("--- Last 5 Void Logs ---");
      voids.forEach((v) => {
        log(`ID: ${v.voidId}, Date: ${v.voidedAt}, By: ${v.voidedByName}, Total: ${v.totalAmount}`);
      });
    }
    fs.writeFileSync("db_check_result.txt", output);
    console.log("Output written to db_check_result.txt");
  } catch (err) {
    console.error("Error:", err);
    fs.writeFileSync("db_check_result.txt", "Error: " + err.message);
  } finally {
    await mongoose.disconnect();
    process.exit(0);
  }
}
check();
```

FILE: pos/backend/check_voidlogs.js
==========

```
const mongoose = require("mongoose");
const dotenv = require("dotenv");
const path = require("path");
dotenv.config({ path: path.join(__dirname, ".env") });
const uri = process.env.MONGODB_URI;
async function checkVoidLogs() {
  try {
    await mongoose.connect(uri);
    console.log("Connected to DB");
    const VoidLog = require("./models/VoidLog");
    const count = await VoidLog.countDocuments();
    console.log(`Total Void Logs: ${count}`);
    if (count > 0) {
      const logs = await VoidLog.find().sort({ voidedAt: -1 }).limit(5);
      console.log(JSON.stringify(logs, null, 2));
    }
  } catch (err) {
    console.error("Error:", err);
  } finally {
    await mongoose.disconnect();
    process.exit(0);
  }
}
```

```
checkVoidLogs();
```

FILE: pos/backend/config/database.js
==========

```
const dbManager = require("./databaseManager");
const connectDB = async () => {
  try {
    await dbManager.initialize();
    console.log("Database connected successfully");
  } catch (error) {
    console.error(`Error connecting to database: ${error.message}`);
    process.exit(1);
  }
};
module.exports = connectDB;
```

FILE: pos/backend/config/databaseManager.js
==========

```
const mongoose = require("mongoose");
const dns = require("dns").promises;
class DatabaseManager {
  constructor() {
    this.connection = null;
    this.cloudURI = process.env.MONGODB_URI || "";
    this.localURI = "mongodb://[REDACTED]";
    this.currentURI = null;
    this.isOnline = false;
    this.connectionCheckInterval = null;
    this.isReconnecting = false;
    this.cloudOnlyMode = process.env.CLOUD_ONLY === "true";
  }
  async checkInternetConnection() {
    try {
      await dns.resolve("google.com");
      return true;
    } catch (error) {
      return false;
    }
  }
  async initialize() {
    try {
      await this.connect();
      this.startConnectionMonitoring();
    } catch (error) {
      console.error("Failed to initialize database:", error.message);
      throw error;
    }
  }
  startConnectionMonitoring() {
    if (this.cloudOnlyMode) return;
    this.connectionCheckInterval = setInterval(async () => {
      if (this.isReconnecting) return;
      const wasOnline = this.isOnline;
      const isNowOnline = await this.checkInternetConnection();
      if (wasOnline !== isNowOnline) {
        this.isOnline = isNowOnline;
        console.log(
          `
[Connection Status Changed] Online: ${wasOnline} -> ${isNowOnline}`
        );
        const shouldUseCloud = isNowOnline && this.cloudURI;
        const currentlyUsingCloud = this.currentURI === this.cloudURI;
        if (shouldUseCloud && !currentlyUsingCloud) {
          console.log("[Switching] Moving from LOCAL to CLOUD database...");
          await this.reconnect();
        } else if (!shouldUseCloud && currentlyUsingCloud) {
          console.log("[Switching] Moving from CLOUD to LOCAL database...");
          await this.reconnect();
        }
      }
    }, 3e4);
  }
  stopConnectionMonitoring() {
```

```
    if (this.connectionCheckInterval) {
      clearInterval(this.connectionCheckInterval);
      this.connectionCheckInterval = null;
    }
  }
  async reconnect() {
    if (this.isReconnecting) {
      console.log("[Reconnect] Already reconnecting, skipping...");
      return;
    }
    try {
      this.isReconnecting = true;
      if (mongoose.connection.readyState !== 0) {
        await mongoose.connection.close();
        console.log("[Reconnect] Previous connection closed");
      }
      await this.connect();
    } catch (error) {
      console.error("[Reconnect] Failed:", error.message);
    } finally {
      this.isReconnecting = false;
    }
  }
  async connect() {
    try {
      if (process.env.MONGODB_URI) {
        this.cloudURI = process.env.MONGODB_URI;
      }
      if (mongoose.connection.readyState === 1 && !this.isReconnecting) {
        return mongoose.connection;
      }
      if (this.cloudOnlyMode) {
        this.currentURI = this.cloudURI;
        this.isOnline = true;
        console.log("[Connection] Cloud-Only Mode - Connecting to Cloud DB...");
      } else {
        this.isOnline = await this.checkInternetConnection();
        console.log("DEBUG: isOnline =", this.isOnline);
        console.log(
          "DEBUG: cloudURI =",
          this.cloudURI ? "Configured" : "Not Configured"
        );
        if (this.isOnline && this.cloudURI) {
          this.currentURI = this.cloudURI;
          console.log("[Connection] Internet ONLINE - Forcing CLOUD database");
        } else {
          this.currentURI = this.localURI;
          if (!this.cloudURI) {
            console.log(
              "[Connection] No cloud URI configured - Using LOCAL database"
            );
          } else {
            console.log("[Connection] Internet OFFLINE - Using LOCAL database");
          }
        }
      }
      await mongoose.connect(this.currentURI, {
        maxPoolSize: 10,
        // Max connections in pool (default is 5, 10 handles concurrent mobile+web)
        minPoolSize: 2,
        // Keep 2 warm connections ready
        socketTimeoutMS: 45e3,
        // Close sockets after 45s of inactivity
        serverSelectionTimeoutMS: 1e4
        // Fail fast if no server available (10s vs default 30s)
      });
      console.log(`\u2713 MongoDB Connected: ${mongoose.connection.host}`);
      console.log(
        `\u2713 Database mode: ${this.isOnline && this.cloudURI ? "CLOUD" : "LOCAL"}`
      );
      this.connection = mongoose.connection;
      mongoose.connection.on("disconnected", () => {
        console.log("[Warning] Database disconnected");
      });
      mongoose.connection.on("error", (err) => {
        console.error("[Error] Database connection error:", err.message);
      });
```

```
      return mongoose.connection;
    } catch (error) {
      console.error("[Error] Failed to connect to MongoDB:", error.message);
      if (this.isOnline && this.currentURI === this.cloudURI) {
        console.log(
          "[Fallback] Cloud database failed - Attempting local database..."
        );
        try {
          this.currentURI = this.localURI;
          await mongoose.connect(this.currentURI);
          console.log(
            `\u2713 MongoDB Connected (Local Fallback): ${mongoose.connection.host}`
          );
          this.connection = mongoose.connection;
          return mongoose.connection;
        } catch (fallbackError) {
          console.error(
            "[Error] Local database fallback failed:",
            fallbackError.message
          );
          throw fallbackError;
        }
      }
      throw error;
    }
  }
  getConnection() {
    return mongoose.connection;
  }
  isConnected() {
    return mongoose.connection.readyState === 1;
  }
  getCurrentMode() {
    return this.currentURI === this.cloudURI ? "CLOUD" : "LOCAL";
  }
  async disconnect() {
    this.stopConnectionMonitoring();
    if (mongoose.connection.readyState !== 0) {
      await mongoose.connection.close();
      console.log("Database connection closed");
    }
  }
}
const dbManager = new DatabaseManager();
module.exports = dbManager;
```

FILE: pos/backend/controllers/archiveController.js
==========

```
const Archive = require("../models/Archive");
const Product = require("../models/Product");
exports.createArchiveItem = async (req, res) => {
  try {
    const archiveData = { ...req.body };
    if (!archiveData.productId) {
      return res.status(400).json({
        success: false,
        message: "Product ID is required"
      });
    }
    if (archiveData.source === "return" && !archiveData.originalTransactionId) {
      return res.status(400).json({
        success: false,
        message: "Original transaction ID is required for returns"
      });
    }
    if (!archiveData.itemName || !archiveData.sku) {
      return res.status(400).json({
        success: false,
        message: "Item name and SKU are required"
      });
    }
    if (!archiveData.reason) {
      return res.status(400).json({
        success: false,
        message: "Reason is required"
```

```
      });
    }
    if (!archiveData.archivedBy) {
      archiveData.archivedBy = "System";
    }
    if (!archiveData.source) {
      archiveData.source = archiveData.originalTransactionId ? "return" : "stock-out";
    }
    const archiveItem = await Archive.create(archiveData);
    res.status(201).json({
      success: true,
      message: "Item archived successfully",
      data: archiveItem
    });
  } catch (error) {
    console.error("Error archiving item:", error);
    res.status(500).json({
      success: false,
      message: "Error archiving item",
      error: error.message
    });
  }
};
exports.getAllArchiveItems = async (req, res) => {
  try {
    const {
      search = "",
      itemType,
      category,
      sortBy = "date-desc",
      page = 1,
      limit = 50
    } = req.query;
    const query = {};
    if (search) {
      query.$or = [
        { "itemData.itemName": { $regex: search, $options: "i" } },
        { "itemData.sku": { $regex: search, $options: "i" } }
      ];
    }
    if (itemType && itemType !== "All") {
      query.itemType = itemType;
    }
    if (category && category !== "All") {
      query["itemData.category"] = category;
    }
    let sort = { archivedAt: -1 };
    switch (sortBy) {
      case "date-asc":
        sort = { archivedAt: 1 };
        break;
      case "name-asc":
        sort = { "itemData.itemName": 1 };
        break;
      case "name-desc":
        sort = { "itemData.itemName": -1 };
        break;
    }
    const skip = (parseInt(page) - 1) * parseInt(limit);
    const [archiveItems, total] = await Promise.all([
      Archive.find(query).sort(sort).skip(skip).limit(parseInt(limit)).lean(),
      Archive.countDocuments(query)
    ]);
    res.json({
      success: true,
      data: archiveItems,
      pagination: {
        page: parseInt(page),
        limit: parseInt(limit),
        total,
        pages: Math.ceil(total / parseInt(limit))
      }
    });
  } catch (error) {
    console.error("Error fetching archive items:", error);
    res.status(500).json({
      success: false,
```

```
      message: "Error fetching archive items",
      error: error.message
    });
  }
};
exports.getArchiveItemById = async (req, res) => {
  try {
    const archiveItem = await Archive.findById(req.params.id).lean();
    if (!archiveItem) {
      return res.status(404).json({
        success: false,
        message: "Archive item not found"
      });
    }
    res.json({
      success: true,
      data: archiveItem
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: "Error fetching archive item",
      error: error.message
    });
  }
};
exports.deleteArchiveItem = async (req, res) => {
  try {
    const archiveItem = await Archive.findByIdAndDelete(req.params.id);
    if (!archiveItem) {
      return res.status(404).json({
        success: false,
        message: "Archive item not found"
      });
    }
    res.json({
      success: true,
      message: "Archive item deleted permanently"
    });
  } catch (error) {
    console.error("Error deleting archive item:", error);
    res.status(500).json({
      success: false,
      message: "Error deleting archive item",
      error: error.message
    });
  }
};
exports.deleteAllArchiveItems = async (req, res) => {
  try {
    await Archive.deleteMany({});
    res.json({
      success: true,
      message: "All archive items deleted permanently"
    });
  } catch (error) {
    console.error("Error deleting all archive items:", error);
    res.status(500).json({
      success: false,
      message: "Error deleting all archive items",
      error: error.message
    });
  }
};
exports.restoreArchiveItem = async (req, res) => {
  try {
    const archiveItem = await Archive.findById(req.params.id);
    if (!archiveItem) {
      return res.status(404).json({
        success: false,
        message: "Archive item not found"
      });
    }
    const product = await Product.findById(archiveItem.productId);
    if (!product) {
      return res.status(404).json({
        success: false,
```

```
        message: "Product no longer exists. Remove this archive entry with Clear instead."
      });
    }
    const qty = Math.max(0, Number(archiveItem.quantity) || 0);
    const wasArchived = product.isArchived === true;
    const prevDisplay = product.displayInTerminal;
    const hasSizes = product.sizes && typeof product.sizes === "object" &&
Object.keys(product.sizes).length > 0;
    if (wasArchived) {
      product.isArchived = false;
      product.displayInTerminal = true;
      product.lastUpdated = Date.now();
      if (!hasSizes) {
        product.currentStock = qty;
      }
      await product.save();
    }
    const deleteResult = await Archive.deleteOne({ _id: archiveItem._id });
    if (deleteResult.deletedCount === 0) {
      if (wasArchived) {
        try {
          product.isArchived = true;
          product.displayInTerminal = prevDisplay;
          await product.save();
        } catch (rollbackErr) {
          console.error("Restore rollback failed:", rollbackErr);
        }
      }
      return res.status(500).json({
        success: false,
        message: "Could not remove archive row. Nothing was changed. Try again."
      });
    }
    res.json({
      success: true,
      message: wasArchived ? "Item restored to inventory" : "Archive entry removed (product was already
active)"
    });
  } catch (error) {
    console.error("Error restoring archive item:", error);
    res.status(500).json({
      success: false,
      message: "Error restoring archive item",
      error: error.message
    });
  }
};
```

FILE: pos/backend/controllers/brandPartnerController.js
==========

```
const BrandPartner = require("../models/BrandPartner");
const formatValidationError = (message) => ({
  success: false,
  message
});
exports.getBrandPartners = async (req, res) => {
  try {
    const partners = await BrandPartner.find().sort({ createdAt: -1 });
    res.json({
      success: true,
      data: partners
    });
  } catch (error) {
    console.error("Error fetching brand partners:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch brand partners."
    });
  }
};
exports.createBrandPartner = async (req, res) => {
  try {
    const { brandName, email, contactNumber, contactPerson, logo } = req.body;
    if (!brandName || !brandName.trim()) {
      return res.status(400).json(formatValidationError("Brand name is required."));
```

```
    }
    const normalizedBrandName = brandName.trim();
    const existing = await BrandPartner.findOne({ brandName: normalizedBrandName });
    if (existing) {
      return res.status(409).json(formatValidationError("Brand name already exists."));
    }
    const newPartner = await BrandPartner.create({
      brandName: normalizedBrandName,
      email: email?.trim() || "",
      contactNumber: contactNumber?.trim() || "",
      contactPerson: contactPerson?.trim() || "",
      logo: logo || "",
      status: "active"
      // Default status
    });
    res.status(201).json({
      success: true,
      data: newPartner
    });
  } catch (error) {
    console.error("Error creating brand partner:", error);
    res.status(500).json({
      success: false,
      message: "Failed to create brand partner."
    });
  }
};
exports.updateBrandPartner = async (req, res) => {
  try {
    const { id } = req.params;
    const { brandName, email, contactNumber, contactPerson, logo, status } = req.body;
    const partner = await BrandPartner.findById(id);
    if (!partner) {
      return res.status(404).json({
        success: false,
        message: "Brand partner not found."
      });
    }
    if (brandName) {
      const normalizedBrandName = brandName.trim();
      if (normalizedBrandName !== partner.brandName) {
        const existing = await BrandPartner.findOne({ brandName: normalizedBrandName });
        if (existing) {
          return res.status(409).json(formatValidationError("Brand name already exists."));
        }
        partner.brandName = normalizedBrandName;
      }
    }
    if (email !== void 0) partner.email = email.trim();
    if (contactNumber !== void 0) partner.contactNumber = contactNumber.trim();
    if (contactPerson !== void 0) partner.contactPerson = contactPerson.trim();
    if (logo !== void 0) partner.logo = logo;
    if (status !== void 0) partner.status = status;
    await partner.save();
    res.json({
      success: true,
      data: partner
    });
  } catch (error) {
    console.error("Error updating brand partner:", error);
    res.status(500).json({
      success: false,
      message: "Failed to update brand partner."
    });
  }
};
exports.deleteBrandPartner = async (req, res) => {
  try {
    const { id } = req.params;
    const partner = await BrandPartner.findByIdAndDelete(id);
    if (!partner) {
      return res.status(404).json({
        success: false,
        message: "Brand partner not found."
      });
    }
    res.json({
```

```
      success: true,
      message: "Brand partner deleted successfully."
    });
  } catch (error) {
    console.error("Error deleting brand partner:", error);
    res.status(500).json({
      success: false,
      message: "Failed to delete brand partner."
    });
  }
};
```

FILE: pos/backend/controllers/cartController.js
==========

```
const mongoose = require("mongoose");
const Cart = require("../models/Cart");
exports.getCartByUser = async (req, res) => {
  try {
    const { userId } = req.params;
    if (!userId) {
      return res.status(400).json({
        success: false,
        message: "User ID is required"
      });
    }
    const cart = await Cart.findOne({ userId }).lean();
    res.json({
      success: true,
      data: cart || { userId, items: [] }
    });
  } catch (error) {
    console.error("Error fetching cart:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch cart",
      error: error.message
    });
  }
};
exports.saveCart = async (req, res) => {
  try {
    const { userId } = req.params;
    const { items } = req.body;
    if (!userId) {
      return res.status(400).json({
        success: false,
        message: "User ID is required"
      });
    }
    if (!Array.isArray(items)) {
      return res.status(400).json({
        success: false,
        message: "Items must be an array"
      });
    }
    const sanitizedItems = items.map((item) => ({
      productId: item._id || item.productId,
      itemName: item.itemName || "",
      sku: item.sku || "",
      itemPrice: item.itemPrice || 0,
      quantity: item.quantity || 1,
      selectedSize: item.selectedSize || "",
      variant: item.selectedVariation || item.variant || "",
      itemImage: item.itemImage || ""
    }));
    const cart = await Cart.findOneAndUpdate(
      { userId },
      { items: sanitizedItems },
      { upsert: true, new: true, setDefaultsOnInsert: true }
    ).lean();
    res.json({
      success: true,
      message: "Cart saved successfully",
      data: cart
    });
```

```
  } catch (error) {
    console.error("Error saving cart:", error);
    res.status(500).json({
      success: false,
      message: "Failed to save cart",
      error: error.message
    });
  }
};



FILE: pos/backend/controllers/categoryController.js
==============================================================================
const Category = require("../models/Category");
const Product = require("../models/Product");
exports.getAllCategories = async (req, res) => {
  try {
    const categories = await Category.find({}).sort({ dateCreated: -1 }).lean();
    const productCounts = {};
    const detailedSubcategoryCounts = {};
    try {
      const aggregationResults = await Product.aggregate([
        {
          $facet: {
            byCategory: [
              { $match: { category: { $nin: [null, ""] } } },
              { $group: { _id: "$category", count: { $sum: 1 } } }
            ],
            bySubCategoryDetailed: [
              { $match: { subCategory: { $nin: [null, ""] }, category: { $nin: [null, ""] } } },
              { $group: { _id: { category: "$category", subCategory: "$subCategory" }, count: { $sum: 1 } } }
            ],
            bySubCategory: [
              { $match: { subCategory: { $nin: [null, ""] } } },
              { $group: { _id: "$subCategory", count: { $sum: 1 } } }
            ]
          }
        }
      ]);
      if (aggregationResults && aggregationResults[0]) {
        aggregationResults[0].byCategory.forEach((cat) => {
          if (cat._id) {
            productCounts[cat._id] = (productCounts[cat._id] || 0) + cat.count;
          }
        });
        aggregationResults[0].bySubCategoryDetailed.forEach((sub) => {
          if (sub._id && sub._id.category && sub._id.subCategory) {
            if (!detailedSubcategoryCounts[sub._id.category]) {
              detailedSubcategoryCounts[sub._id.category] = {};
            }
            detailedSubcategoryCounts[sub._id.category][sub._id.subCategory] = sub.count;
          }
        });
        aggregationResults[0].bySubCategory.forEach((sub) => {
          if (sub._id) {
            productCounts[sub._id] = (productCounts[sub._id] || 0) + sub.count;
          }
        });
      }
    } catch (error) {
      console.warn("Error computing product counts aggregation:", error.message);
    }
    const formattedCategories = categories.map((category) => ({
      ...category,
      _id: category._id.toString(),
      productCount: productCounts[category.name] || 0
    }));
    res.json({
      success: true,
      count: formattedCategories.length,
      detailedSubcategoryCounts,
      data: formattedCategories
    });
  } catch (error) {
    console.error("Error fetching categories:", error);
    res.status(500).json({
```

```
      success: false,
      message: "Error fetching categories",
      error: error.message
    });
  }
};
exports.getCategoryById = async (req, res) => {
  try {
    const category = await Category.findById(req.params.id).lean();
    if (!category) {
      return res.status(404).json({
        success: false,
        message: "Category not found"
      });
    }
    const productCount = await Product.countDocuments({
      $or: [
        { category: category.name },
        { subCategory: category.name }
      ]
    });
    category.productCount = productCount;
    res.json({
      success: true,
      data: category
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: "Error fetching category",
      error: error.message
    });
  }
};
exports.createCategory = async (req, res) => {
  try {
    const categoryData = { ...req.body };
    if (!categoryData.name) {
      return res.status(400).json({
        success: false,
        message: "Category name is required"
      });
    }
    if (!categoryData.type) {
      categoryData.type = "category";
    }
    if (categoryData.type === "category") {
      categoryData.parentCategory = null;
    }
    if (categoryData.type === "subcategory" && !categoryData.parentCategory) {
      return res.status(400).json({
        success: false,
        message: "Parent category is required for subcategories"
      });
    }
    if (!categoryData.status) {
      categoryData.status = "active";
    }
    const savedCategory = await Category.create(categoryData);
    res.status(201).json({
      success: true,
      data: savedCategory
    });
  } catch (error) {
    if (error.code === 11e3) {
      return res.status(400).json({
        success: false,
        message: "Category with this name already exists"
      });
    }
    console.error("Error creating category:", error);
    res.status(500).json({
      success: false,
      message: "Error creating category",
      error: error.message
    });
  }
```

```
};
exports.updateCategory = async (req, res) => {
  try {
    const categoryId = req.params.id;
    const updateData = { ...req.body };
    updateData.lastUpdated = Date.now();
    const updatedCategory = await Category.findByIdAndUpdate(
      categoryId,
      updateData,
      { new: true }
    ).lean();
    if (!updatedCategory) {
      return res.status(404).json({
        success: false,
        message: "Category not found"
      });
    }
    const productCount = await Product.countDocuments({ category: updatedCategory.name });
    updatedCategory.productCount = productCount;
    res.json({
      success: true,
      data: updatedCategory
    });
  } catch (error) {
    if (error.code === 11e3) {
      return res.status(400).json({
        success: false,
        message: "Category with this name already exists"
      });
    }
    console.error("Error updating category:", error);
    res.status(500).json({
      success: false,
      message: "Error updating category",
      error: error.message
    });
  }
};
exports.deleteCategory = async (req, res) => {
  try {
    const categoryId = req.params.id;
    const category = await Category.findById(categoryId);
    if (!category) {
      return res.status(404).json({
        success: false,
        message: "Category not found"
      });
    }
    const productCount = await Product.countDocuments({ category: category.name });
    if (productCount > 0) {
      return res.status(400).json({
        success: false,
        message: `Cannot delete category. There are ${productCount} products in this category. Please
reassign or delete products first.`
      });
    }
    await Category.findByIdAndDelete(categoryId);
    res.json({
      success: true,
      message: "Category deleted successfully"
    });
  } catch (error) {
    console.error("Error deleting category:", error);
    res.status(500).json({
      success: false,
      message: "Error deleting category",
      error: error.message
    });
  }
};
exports.archiveCategory = async (req, res) => {
  try {
    const categoryId = req.params.id;
    const category = await Category.findById(categoryId);
    if (!category) {
      return res.status(404).json({
        success: false,
```

```
        message: "Category not found"
      });
    }
    const newStatus = category.status === "active" ? "inactive" : "active";
    const updatedCategory = await Category.findByIdAndUpdate(
      categoryId,
      { status: newStatus, lastUpdated: Date.now() },
      { new: true }
    ).lean();
    const productCount = await Product.countDocuments({ category: updatedCategory.name });
    updatedCategory.productCount = productCount;
    res.json({
      success: true,
      data: updatedCategory
    });
  } catch (error) {
    console.error("Error archiving category:", error);
    res.status(500).json({
      success: false,
      message: "Error archiving category",
      error: error.message
    });
  }
};
```

FILE: pos/backend/controllers/dataManagementController.js
==========

```
const mongoose = require("mongoose");
const Product = require("../models/Product");
const SalesTransaction = require("../models/SalesTransaction");
const StockMovement = require("../models/StockMovement");
const Employee = require("../models/Employee");
const Category = require("../models/Category");
const Discount = require("../models/Discount");
const BrandPartner = require("../models/BrandPartner");
const VoidLog = require("../models/VoidLog");
const Archive = require("../models/Archive");
const Cart = require("../models/Cart");
const MerchantSettings = require("../models/MerchantSettings");
const Remittance = require("../models/Remittance");
const SyncLog = require("../models/SyncLog");
const bcrypt = require("bcryptjs");
const DATA_COLLECTIONS = {
  products: { model: Product, name: "Inventory / Products" },
  transactions: { model: SalesTransaction, name: "Sales Transactions" },
  stockMovements: { model: StockMovement, name: "Stock Movements" },
  categories: { model: Category, name: "Categories" },
  discounts: { model: Discount, name: "Discounts" },
  brandPartners: { model: BrandPartner, name: "Brand Partners" },
  voidLogs: { model: VoidLog, name: "Void Logs" },
  archives: { model: Archive, name: "Archives" },
  carts: { model: Cart, name: "Cart Data" },
  remittances: { model: Remittance, name: "Remittance Logs" },
  employees: { model: Employee, name: "Employees" },
  merchantSettings: { model: MerchantSettings, name: "Merchant Settings" }
};
exports.getCollections = async (req, res) => {
  try {
    const collections = await Promise.all(
      Object.entries(DATA_COLLECTIONS).map(async ([key, { model, name }]) => {
        try {
          const count = await model.countDocuments();
          return { key, name, count };
        } catch {
          return { key, name, count: 0 };
        }
      })
    );
    res.json({ success: true, data: collections });
  } catch (error) {
    console.error("Error fetching collections:", error);
    res.status(500).json({ success: false, message: "Failed to fetch collections" });
  }
};
exports.clearData = async (req, res) => {
```

```
  try {
    const { collections, pin } = req.body;
    if (!pin) {
      return res.status(400).json({ success: false, message: "PIN is required" });
    }
    if (!collections || !Array.isArray(collections) || collections.length === 0) {
      return res.status(400).json({
        success: false,
        message: "Please select at least one data type to clear"
      });
    }
    const employees = await Employee.find({
      status: "Active",
      role: { $in: ["Owner", "Manager"] }
    }).select("+pin").lean();
    let verified = false;
    for (const emp of employees) {
      if (!emp.pin) continue;
      try {
        const isMatch = await bcrypt.compare(pin, emp.pin);
        if (isMatch) {
          verified = true;
          break;
        }
      } catch {
        continue;
      }
    }
    if (!verified) {
      return res.status(401).json({
        success: false,
        message: "Invalid PIN. Only Owner or Manager can perform this action."
      });
    }
    const invalidKeys = collections.filter((key) => !DATA_COLLECTIONS[key]);
    if (invalidKeys.length > 0) {
      return res.status(400).json({
        success: false,
        message: `Invalid collections: ${invalidKeys.join(", ")}`
      });
    }
    const results = [];
    for (const key of collections) {
      const { model, name } = DATA_COLLECTIONS[key];
      try {
        if (key === "employees") {
          const result = await model.deleteMany({ role: { $ne: "Owner" } });
          results.push({
            key,
            name,
            deleted: result.deletedCount,
            note: "Owner accounts were preserved"
          });
        } else {
          const result = await model.deleteMany({});
          results.push({ key, name, deleted: result.deletedCount });
        }
      } catch (err) {
        results.push({ key, name, deleted: 0, error: err.message });
      }
    }
    try {
      await SyncLog.deleteMany({});
    } catch {
    }
    const totalDeleted = results.reduce((sum, r) => sum + r.deleted, 0);
    res.json({
      success: true,
      message: `Successfully cleared ${totalDeleted} records from ${results.length} collection(s)`,
      data: results
    });
  } catch (error) {
    console.error("Error clearing data:", error);
    res.status(500).json({ success: false, message: "Failed to clear data" });
  }
};
exports.exportData = async (req, res) => {
```

```
  try {
    const { collections } = req.body;
    if (!collections || !Array.isArray(collections) || collections.length === 0) {
      return res.status(400).json({
        success: false,
        message: "Please select at least one data type to export"
      });
    }
    const invalidKeys = collections.filter((key) => !DATA_COLLECTIONS[key]);
    if (invalidKeys.length > 0) {
      return res.status(400).json({
        success: false,
        message: `Invalid collections: ${invalidKeys.join(", ")}`
      });
    }
    const exportedData = {};
    const summary = [];
    for (const key of collections) {
      const { model, name } = DATA_COLLECTIONS[key];
      try {
        let data;
        if (key === "employees") {
          data = await model.find().select("-pin").lean();
        } else if (key === "merchantSettings") {
          data = await model.find().select("-encryptedPrivateKey -privateKeyIV -privateKeyAuthTag").lean();
        } else {
          data = await model.find().lean();
        }
        exportedData[key] = data;
        summary.push({ key, name, count: data.length });
      } catch (err) {
        exportedData[key] = [];
        summary.push({ key, name, count: 0, error: err.message });
      }
    }
    res.json({
      success: true,
      data: exportedData,
      summary,
      exportedAt: (/* @__PURE__ */ new Date()).toISOString(),
      version: "1.0"
    });
  } catch (error) {
    console.error("Error exporting data:", error);
    res.status(500).json({ success: false, message: "Failed to export data" });
  }
};
exports.importData = async (req, res) => {
  try {
    const { data, pin } = req.body;
    if (!pin) {
      return res.status(400).json({ success: false, message: "PIN is required" });
    }
    if (!data || typeof data !== "object") {
      return res.status(400).json({ success: false, message: "Invalid import data format" });
    }
    const employees = await Employee.find({
      status: "Active",
      role: { $in: ["Owner", "Manager"] }
    }).select("+pin").lean();
    let verified = false;
    for (const emp of employees) {
      if (!emp.pin) continue;
      try {
        const isMatch = await bcrypt.compare(pin, emp.pin);
        if (isMatch) {
          verified = true;
          break;
        }
      } catch {
        continue;
      }
    }
    if (!verified) {
      return res.status(401).json({
        success: false,
        message: "Invalid PIN. Only Owner or Manager can perform this action."
```

```
  });
}
const results = [];
const collectionKeys = Object.keys(data).filter(
  (key) => DATA_COLLECTIONS[key]
);
if (collectionKeys.length === 0) {
  return res.status(400).json({
    success: false,
    message: "No valid collection data found in import file"
  });
}
const uniqueKeyFields = {
  products: "sku",
  employees: "email",
  categories: "name",
  brandPartners: "name",
  discounts: "name",
  merchantSettings: "merchantName"
};
for (const key of collectionKeys) {
  const { model, name } = DATA_COLLECTIONS[key];
  const records = data[key];
  if (!Array.isArray(records) || records.length === 0) {
    results.push({ key, name, imported: 0, skipped: true });
    continue;
  }
  try {
    const cleanRecords = records.map((record) => {
      const { _id, __v, ...rest } = record;
      return rest;
    });
    const uniqueField = uniqueKeyFields[key];
    let insertedCount = 0;
    let updatedCount = 0;
    if (uniqueField) {
      const bulkOps = cleanRecords.map((record) => ({
        updateOne: {
          filter: { [uniqueField]: record[uniqueField] },
          update: { $set: record },
          upsert: true
        }
      }));
      const bulkResult = await model.bulkWrite(bulkOps, { ordered: false });
      insertedCount = bulkResult.upsertedCount || 0;
      updatedCount = bulkResult.modifiedCount || 0;
      results.push({
        key,
        name,
        imported: insertedCount,
        updated: updatedCount,
        total: insertedCount + updatedCount
      });
    } else {
      const result = await model.insertMany(cleanRecords, { ordered: false }).catch((err) => {
        if (err.insertedDocs) return err.insertedDocs;
        if (err.result && err.result.nInserted)
          return { length: err.result.nInserted };
        throw err;
      });
      insertedCount = Array.isArray(result) ? result.length : result?.length || 0;
      results.push({ key, name, imported: insertedCount });
    }
  } catch (err) {
    console.error(`Error importing ${key}:`, err);
    results.push({ key, name, imported: 0, error: err.message });
  }
}
const totalImported = results.reduce(
  (sum, r) => sum + (r.imported || 0),
  0
);
const totalUpdated = results.reduce(
  (sum, r) => sum + (r.updated || 0),
  0
);
let message = `Successfully imported ${totalImported} new records`;
```

```
    if (totalUpdated > 0) {
      message += ` and updated ${totalUpdated} existing records`;
    }
    message += ` across ${results.filter((r) => (r.imported || 0) + (r.updated || 0) > 0).length}
collection(s)`;
    res.json({
      success: true,
      message,
      data: results
    });
  } catch (error) {
    console.error("Error importing data:", error);
    res.status(500).json({ success: false, message: "Failed to import data" });
  }
};
```

FILE: pos/backend/controllers/discountController.js
==========

```
const Discount = require("../models/Discount");
exports.getAllDiscounts = async (req, res) => {
  try {
    const discounts = await Discount.find({}).sort({ dateCreated: -1 }).lean();
    for (let i = 0; i < discounts.length; i++) {
      let discount = discounts[i];
      if (discount.usageLimit && discount.usageLimit > 0 && (discount.usageCount || 0) >= discount.usageLimit
&& discount.status === "active") {
        await Discount.findByIdAndUpdate(discount._id, { status: "inactive" });
        discounts[i].status = "inactive";
      }
    }
    const formattedDiscounts = discounts.map((discount) => {
      const discountValue = discount.discountType === "percentage" ? `${discount.discountValue}% OFF` :
`\u20B1${discount.discountValue} OFF`;
      const appliesToText = discount.appliesTo === "all" ? "All Products" : discount.appliesTo ===
"category" ? `Category: ${discount.category}${discount.subCategory ? ` > ${discount.subCategory}` : ""}` :
"Specific Products";
      return {
        ...discount,
        _id: discount._id.toString(),
        discountValue,
        appliesTo: appliesToText,
        appliesToType: discount.appliesTo,
        usage: discount.usageLimit ? { used: discount.usageCount || 0, total: discount.usageLimit } : null,
        validFrom: discount.noExpiration ? "Permanent" : discount.validFrom ? new
Date(discount.validFrom).toISOString().split("T")[0] : null,
        validTo: discount.noExpiration ? null : discount.validTo ? new
Date(discount.validTo).toISOString().split("T")[0] : null
      };
    });
    res.json({
      success: true,
      count: formattedDiscounts.length,
      data: formattedDiscounts
    });
  } catch (error) {
    console.error("Error fetching discounts:", error);
    res.status(500).json({
      success: false,
      message: "Error fetching discounts",
      error: error.message
    });
  }
};
exports.getDiscountById = async (req, res) => {
  try {
    const discount = await Discount.findById(req.params.id).lean();
    if (!discount) {
      return res.status(404).json({
        success: false,
        message: "Discount not found"
      });
    }
    res.json({
      success: true,
      data: discount
```

```
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: "Error fetching discount",
      error: error.message
    });
  }
};
exports.createDiscount = async (req, res) => {
  try {
    const discountData = { ...req.body };
    if (!discountData.title || !discountData.discountType || discountData.discountValue === void 0) {
      return res.status(400).json({
        success: false,
        message: "Title, discount type, and discount value are required"
      });
    }
    if (!discountData.status) {
      discountData.status = "active";
    }
    const discount = await Discount.create(discountData);
    res.status(201).json({
      success: true,
      data: discount
    });
  } catch (error) {
    if (error.code === 11e3) {
      return res.status(400).json({
        success: false,
        message: "Discount with this name already exists"
      });
    }
    console.error("Error creating discount:", error);
    res.status(500).json({
      success: false,
      message: "Error creating discount",
      error: error.message
    });
  }
};
exports.updateDiscount = async (req, res) => {
  try {
    const updateData = { ...req.body };
    updateData.lastUpdated = Date.now();
    const currentDiscount = await Discount.findById(req.params.id);
    if (!currentDiscount) {
      return res.status(404).json({
        success: false,
        message: "Discount not found"
      });
    }
    if (updateData.usageLimit && updateData.usageLimit > (currentDiscount.usageCount || 0) &&
currentDiscount.status === "inactive") {
      updateData.status = "active";
    } else if (updateData.usageLimit && updateData.usageLimit <= (currentDiscount.usageCount || 0) &&
currentDiscount.status === "active") {
      updateData.status = "inactive";
    }
    const discount = await Discount.findByIdAndUpdate(
      req.params.id,
      updateData,
      { new: true }
    );
    if (!discount) {
      return res.status(404).json({
        success: false,
        message: "Discount not found"
      });
    }
    res.json({
      success: true,
      data: discount
    });
  } catch (error) {
    if (error.code === 11e3) {
      return res.status(400).json({
```

```
        success: false,
        message: "Discount with this name already exists"
      });
    }
    console.error("Error updating discount:", error);
    res.status(500).json({
      success: false,
      message: "Error updating discount",
      error: error.message
    });
  }
};
exports.deleteDiscount = async (req, res) => {
  try {
    const discount = await Discount.findByIdAndDelete(req.params.id);
    if (!discount) {
      return res.status(404).json({
        success: false,
        message: "Discount not found"
      });
    }
    res.json({
      success: true,
      message: "Discount deleted successfully"
    });
  } catch (error) {
    console.error("Error deleting discount:", error);
    res.status(500).json({
      success: false,
      message: "Error deleting discount",
      error: error.message
    });
  }
};
exports.toggleDiscountStatus = async (req, res) => {
  try {
    const discount = await Discount.findById(req.params.id);
    if (!discount) {
      return res.status(404).json({
        success: false,
        message: "Discount not found"
      });
    }
    const newStatus = discount.status === "active" ? "inactive" : "active";
    const updatedDiscount = await Discount.findByIdAndUpdate(
      req.params.id,
      { status: newStatus, lastUpdated: Date.now() },
      { new: true }
    );
    res.json({
      success: true,
      data: updatedDiscount
    });
  } catch (error) {
    console.error("Error toggling discount status:", error);
    res.status(500).json({
      success: false,
      message: "Error toggling discount status",
      error: error.message
    });
  }
};
exports.getActiveDiscounts = async (req, res) => {
  try {
    const now = /* @__PURE__ */ new Date();
    let discounts = await Discount.find({
      status: "active",
      $or: [
        { noExpiration: true },
        { validFrom: { $lte: now }, validTo: { $gte: now } }
      ]
    }).lean();
    discounts = discounts.filter((discount) => {
      if (discount.usageLimit && discount.usageLimit > 0) {
        return (discount.usageCount || 0) < discount.usageLimit;
      }
      return true;
```

```
    });
    res.json({
      success: true,
      count: discounts.length,
      data: discounts
    });
  } catch (error) {
    console.error("Error fetching active discounts:", error);
    res.status(500).json({
      success: false,
      message: "Error fetching active discounts",
      error: error.message
    });
  }
};



FILE: pos/backend/controllers/employeeController.js
================================================================================
const Employee = require("../models/Employee");
const bcrypt = require("bcryptjs");
const crypto = require("crypto");
const { sendEmail } = require("../utils/emailService");
const pinCache = /* @__PURE__ */ new Map();
const getFastPinHash = (rawPin) => {
  const hmacSecret = process.env.PIN_SECRET || "fallback-secret-for-pos-pin";
  return crypto.createHmac("sha256", hmacSecret).update(String(rawPin)).digest("hex");
};
const isPinAlreadyUsedByOther = async (rawPin, excludeEmployeeId = null) => {
  const fastPinHash = getFastPinHash(rawPin);
  const query = excludeEmployeeId ? { fastPinHash, _id: { $ne: excludeEmployeeId } } : { fastPinHash };
  const existing = await Employee.exists(query);
  return !!existing;
};
exports.getAllEmployees = async (req, res) => {
  try {
    const employees = await Employee.find({}).sort({ dateJoined: -1 }).lean();
    const employeesWithoutPin = employees.map((emp) => {
      const { pin, ...rest } = emp;
      return rest;
    });
    res.json({
      success: true,
      count: employeesWithoutPin.length,
      data: employeesWithoutPin
    });
  } catch (error) {
    console.error("Error fetching employees:", error);
    res.status(500).json({
      success: false,
      message: "Error fetching employees",
      error: error.message
    });
  }
};
exports.getEmployeeById = async (req, res) => {
  try {
    const employee = await Employee.findById(req.params.id).lean();
    if (!employee) {
      return res.status(404).json({
        success: false,
        message: "Employee not found"
      });
    }
    const { pin, ...employeeWithoutPin } = employee;
    res.json({
      success: true,
      data: employeeWithoutPin
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: "Error fetching employee",
      error: error.message
    });
  }
```

```
};
exports.getEmployeeImage = async (req, res) => {
  try {
    const employee = await Employee.findById(req.params.id).select("profileImage").lean();
    if (!employee || !employee.profileImage) {
      return res.status(404).send("No image found");
    }
    const base64Data = employee.profileImage.replace(/^data:image\/\w+base64,/, "");
    const imgBuffer = Buffer.from(base64Data, "base64");
    res.setHeader("Content-Type", "image/jpeg");
    res.setHeader("Cache-Control", "public, max-age=86400");
    res.send(imgBuffer);
  } catch (error) {
    console.error("Error fetching employee image:", error);
    res.status(500).send("Error loading image");
  }
};
exports.createEmployee = async (req, res) => {
  try {
    const {
      firstName,
      middleInitial,
      lastName,
      name,
      contactNo,
      email,
      role,
      pin,
      dateJoined,
      status,
      profileImage,
      permissions,
      dateJoinedActual,
      requiresPinReset,
      sendPinEmail
      // Flag to send PIN via email
    } = req.body;
    if (!email || !role || !pin) {
      return res.status(400).json({
        success: false,
        message: "Email, role, and PIN are required"
      });
    }
    const existingEmployee = await Employee.findOne({ email });
    if (existingEmployee) {
      return res.status(400).json({
        success: false,
        message: "Employee with this email already exists"
      });
    }
    const rawPin = pin;
    if (await isPinAlreadyUsedByOther(rawPin)) {
      return res.status(400).json({
        success: false,
        message: "Please use other pin"
      });
    }
    const employeeData = {
      firstName: firstName || "",
      middleInitial: middleInitial || "",
      lastName: lastName || "",
      name: name || [firstName || "", middleInitial || "", lastName || ""].map((p) =>
String(p).trim()).filter(Boolean).join(" ").trim(),
      contactNo: contactNo || "",
      email,
      role,
      pin,
      // Pass raw PIN - model will hash it
      dateJoined: dateJoined || /* @__PURE__ */ new Date(),
      dateJoinedActual: dateJoinedActual || /* @__PURE__ */ new Date(),
      status: status || "Active",
      profileImage: profileImage || "",
      permissions: permissions || {},
      requiresPinReset: requiresPinReset !== void 0 ? requiresPinReset : true
      // Default to true for new employees
    };
    const employee = await Employee.create(employeeData);
```

```
    const { pin: _, profileImage: __, ...employeeWithoutPin } = employee.toObject();
    let emailSent = false;
    console.log("[Employee] Attempting to send welcome email to:", email);
    if (email && sendPinEmail !== false) {
      try {
        const storeName = process.env.STORE_NAME || "Create Your Style";
        console.log("[Employee] Sending PIN email with storeName:", storeName);
        const emailResult = await sendEmail(
          email,
          `Welcome to ${storeName} - Your Temporary PIN`,
          `Welcome to ${storeName}!

Your temporary PIN is: ${rawPin}

Please change this PIN after your first login for security purposes.

Best regards,
${storeName} Team`,
          `
            <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px;">
              <div style="background: linear-gradient(135deg, #AD7F65 0%, #76462B 100%); color: white;
padding: 20px; border-radius: 10px 10px 0 0; text-align: center;">
                <h1 style="margin: 0;">Welcome to ${storeName}!</h1>
              </div>
              <div style="background: #f9f9f9; padding: 30px; border: 1px solid #ddd; border-top: none;">
                <p style="font-size: 16px; color: #333;">Hello <strong>${firstName || name || "Team
Member"}</strong>,</p>
                <p style="font-size: 16px; color: #333;">Your account has been created successfully. Here is
your temporary PIN:</p>
                <div style="background: white; border: 2px dashed #AD7F65; padding: 20px; text-align: center;
margin: 20px 0; border-radius: 10px;">
                  <p style="font-size: 32px; font-weight: bold; color: #AD7F65; margin: 0; letter-spacing:
8px;">${rawPin}</p>
                </div>
                <p style="font-size: 14px; color: #666;">\u26A0\uFE0F <strong>Important:</strong> Please
change this PIN after your first login for security purposes.</p>
                <p style="font-size: 14px; color: #666;">Your role: <strong>${role}</strong></p>
              </div>
              <div style="background: #f3f4f6; padding: 15px; text-align: center; font-size: 12px; color:
#666; border-radius: 0 0 10px 10px;">
                <p style="margin: 0;">This is an automated message from ${storeName} POS System</p>
              </div>
            </div>
          `
        );
        emailSent = emailResult.success;
        console.log("[Employee] Email send result:", emailResult);
        if (!emailResult.success) {
          console.error("[Employee] Failed to send welcome email:", emailResult.error);
        } else {
          console.log("[Employee] Welcome email sent successfully to:", email);
        }
      } catch (emailError) {
        console.error("[Employee] Error sending welcome email:", emailError);
      }
    } else {
      console.log("[Employee] Skipping email - email:", email, "sendPinEmail:", sendPinEmail);
    }
    res.status(201).json({
      success: true,
      message: emailSent ? "Employee created successfully. Temporary PIN sent to email." : "Employee created
successfully.",
      data: employeeWithoutPin,
      emailSent
    });
  } catch (error) {
    if (error.code === 11e3) {
      return res.status(400).json({
        success: false,
        message: "Employee with this email already exists"
      });
    }
    console.error("Error creating employee:", error);
    res.status(500).json({
      success: false,
      message: "Error creating employee",
      error: error.message
```

```
    });
  }
};
exports.updateEmployee = async (req, res) => {
  try {
    const { id } = req.params;
    const updateData = { ...req.body };
    if (updateData.pin) {
      const rawPin = updateData.pin.toString();
      if (await isPinAlreadyUsedByOther(rawPin, id)) {
        return res.status(400).json({
          success: false,
          message: "Please use other pin"
        });
      }
      const salt = await bcrypt.genSalt(10);
      updateData.pin = await bcrypt.hash(rawPin, salt);
      updateData.fastPinHash = getFastPinHash(rawPin);
    }
    updateData.lastUpdated = Date.now();
    const employee = await Employee.findByIdAndUpdate(
      id,
      updateData,
      { new: true, runValidators: true }
    );
    if (!employee) {
      return res.status(404).json({
        success: false,
        message: "Employee not found"
      });
    }
    const { pin, ...employeeWithoutPin } = employee.toObject();
    res.json({
      success: true,
      message: "Employee updated successfully",
      data: employeeWithoutPin
    });
  } catch (error) {
    console.error("Error updating employee:", error);
    res.status(500).json({
      success: false,
      message: "Error updating employee",
      error: error.message
    });
  }
};
exports.deleteEmployee = async (req, res) => {
  try {
    const employee = await Employee.findByIdAndDelete(req.params.id).select("_id");
    if (!employee) {
      return res.status(404).json({
        success: false,
        message: "Employee not found"
      });
    }
    res.json({
      success: true,
      message: "Employee deleted successfully"
    });
  } catch (error) {
    console.error("Error deleting employee:", error);
    res.status(500).json({
      success: false,
      message: "Error deleting employee",
      error: error.message
    });
  }
};
exports.verifyPin = async (req, res) => {
  try {
    const { pin } = req.body;
    if (!pin) {
      return res.status(400).json({
        success: false,
        message: "PIN is required"
      });
    }
```

```
    const hmacSecret = process.env.PIN_SECRET || "fallback-secret-for-pos-pin";
    const computedFastHash = crypto.createHmac("sha256", hmacSecret).update(pin.toString()).digest("hex");
    const fastEmployee = await Employee.findOne({
      fastPinHash: computedFastHash,
      status: "Active"
    }).lean();
    if (fastEmployee) {
      await Employee.findByIdAndUpdate(fastEmployee._id, {
        isOnline: true,
        lastLogin: /* @__PURE__ */ new Date(),
        lastActive: /* @__PURE__ */ new Date()
      });
      const { pin: unusedPin, fastPinHash: unusedFastHash, ...employeeWithoutPin } = fastEmployee;
      return res.json({
        success: true,
        message: "PIN verified successfully (fast path)",
        data: employeeWithoutPin,
        requiresPinReset: fastEmployee.requiresPinReset || false
      });
    }
    const employees = await Employee.find({
      status: "Active"
    }).select("+pin").lean();
    if (employees.length === 0) {
      return res.status(401).json({
        success: false,
        message: "Invalid PIN or insufficient permissions"
      });
    }
    const matchPromises = employees.map(async (emp) => {
      if (!emp.pin) return null;
      try {
        const isMatch = await bcrypt.compare(pin.toString(), emp.pin);
        return isMatch ? emp : null;
      } catch (err) {
        console.error("Bcrypt error on", emp.email, err);
        return null;
      }
    });
    const results = await Promise.all(matchPromises);
    const found = results.find((emp) => emp !== null);
    if (found) {
      await Employee.findByIdAndUpdate(found._id, {
        fastPinHash: computedFastHash,
        isOnline: true,
        lastLogin: /* @__PURE__ */ new Date(),
        lastActive: /* @__PURE__ */ new Date()
      });
      const { pin: unusedPin, fastPinHash: unusedFastHash, ...employeeWithoutPin } = found;
      return res.json({
        success: true,
        message: "PIN verified successfully (migrated)",
        data: employeeWithoutPin,
        requiresPinReset: found.requiresPinReset || false
      });
    }
    return res.status(401).json({
      success: false,
      message: "Invalid PIN or insufficient permissions"
    });
  } catch (error) {
    console.error("Error verifying PIN:", error);
    res.status(500).json({
      success: false,
      message: "Error verifying PIN",
      error: error.message
    });
  }
};
exports.resetPin = async (req, res) => {
  try {
    const { id } = req.params;
    const { newPin, requiresPinReset } = req.body;
    if (!newPin) {
      return res.status(400).json({
        success: false,
        message: "New PIN is required"
```

```
      });
    }
    if (await isPinAlreadyUsedByOther(newPin.toString(), id)) {
      return res.status(400).json({
        success: false,
        message: "Please use other pin"
      });
    }
    const salt = await bcrypt.genSalt(10);
    const hashedPin = await bcrypt.hash(newPin.toString(), salt);
    const fastPinHash = getFastPinHash(newPin.toString());
    const employee = await Employee.findByIdAndUpdate(
      id,
      {
        pin: hashedPin,
        fastPinHash,
        requiresPinReset: requiresPinReset !== void 0 ? requiresPinReset : false,
        lastUpdated: Date.now()
      },
      { new: true }
    );
    if (!employee) {
      return res.status(404).json({
        success: false,
        message: "Employee not found"
      });
    }
    res.json({
      success: true,
      message: "PIN reset successfully"
    });
  } catch (error) {
    console.error("Error resetting PIN:", error);
    res.status(500).json({
      success: false,
      message: "Error resetting PIN",
      error: error.message
    });
  }
};
exports.toggleStatus = async (req, res) => {
  try {
    const { id } = req.params;
    const employee = await Employee.findById(id);
    if (!employee) {
      return res.status(404).json({
        success: false,
        message: "Employee not found"
      });
    }
    const newStatus = employee.status === "Active" ? "Inactive" : "Active";
    const updatedEmployee = await Employee.findByIdAndUpdate(
      id,
      { status: newStatus, lastUpdated: Date.now() },
      { new: true }
    );
    const { pin, ...employeeWithoutPin } = updatedEmployee.toObject();
    res.json({
      success: true,
      message: `Employee ${newStatus === "Active" ? "activated" : "deactivated"} successfully`,
      data: employeeWithoutPin
    });
  } catch (error) {
    console.error("Error toggling employee status:", error);
    res.status(500).json({
      success: false,
      message: "Error toggling employee status",
      error: error.message
    });
  }
};
exports.getEmployeesByRole = async (req, res) => {
  try {
    const { role } = req.params;
    const employees = await Employee.find({ role }).sort({ dateJoined: -1 }).lean();
    const employeesWithoutPin = employees.map((emp) => {
      const { pin, ...rest } = emp;
```

```
      return rest;
    });
    res.json({
      success: true,
      count: employeesWithoutPin.length,
      data: employeesWithoutPin
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: "Error fetching employees",
      error: error.message
    });
  }
};
exports.sendTemporaryPin = async (req, res) => {
  try {
    const { id } = req.params;
    const employee = await Employee.findById(id);
    if (!employee) {
      return res.status(404).json({
        success: false,
        message: "Employee not found"
      });
    }
    const tempPin = Math.floor(1e5 + Math.random() * 9e5).toString();
    const salt = await bcrypt.genSalt(10);
    const hashedPin = await bcrypt.hash(tempPin.toString(), salt);
    const hmacSecret = process.env.PIN_SECRET || "fallback-secret-for-pos-pin";
    const fastPinHash = crypto.createHmac("sha256", hmacSecret).update(tempPin.toString()).digest("hex");
    await Employee.findByIdAndUpdate(id, {
      pin: hashedPin,
      fastPinHash,
      requiresPinReset: true,
      lastUpdated: Date.now()
    });
    let emailSent = false;
    if (employee.email) {
      try {
        const storeName = process.env.STORE_NAME || "Create Your Style";
        const emailResult = await sendEmail(
          employee.email,
          `${storeName} - Your New Temporary PIN`,
          `Your new temporary PIN is: ${tempPin}

Please change this PIN after logging in for security purposes.

Best regards,
${storeName} Team`,
          `
            <div style="font-family: Arial, sans-serif; max-width: 600px; margin: 0 auto; padding: 20px;">
              <div style="background: linear-gradient(135deg, #AD7F65 0%, #76462B 100%); color: white;
padding: 20px; border-radius: 10px 10px 0 0; text-align: center;">
                <h1 style="margin: 0;">PIN Reset</h1>
              </div>
              <div style="background: #f9f9f9; padding: 30px; border: 1px solid #ddd; border-top: none;">
                <p style="font-size: 16px; color: #333;">Hello <strong>${employee.firstName || employee.name
|| "Team Member"}</strong>,</p>
                <p style="font-size: 16px; color: #333;">Your PIN has been reset. Here is your new temporary
PIN:</p>
                <div style="background: white; border: 2px dashed #AD7F65; padding: 20px; text-align: center;
margin: 20px 0; border-radius: 10px;">
                  <p style="font-size: 32px; font-weight: bold; color: #AD7F65; margin: 0; letter-spacing:
8px;">${tempPin}</p>
                </div>
                <p style="font-size: 14px; color: #666;">\u26A0\uFE0F <strong>Important:</strong> Please
change this PIN after logging in for security purposes.</p>
              </div>
              <div style="background: #f3f4f6; padding: 15px; text-align: center; font-size: 12px; color:
#666; border-radius: 0 0 10px 10px;">
                <p style="margin: 0;">This is an automated message from ${storeName} POS System</p>
              </div>
            </div>
          `
        );
        emailSent = emailResult.success;
        if (!emailResult.success) {
```

```
          console.error("Failed to send PIN reset email:", emailResult.error);
        }
      } catch (emailError) {
        console.error("Error sending email:", emailError);
      }
    }
    res.json({
      success: true,
      message: emailSent ? "Temporary PIN sent to email successfully" : "Temporary PIN generated (email not
sent)",
      emailSent
    });
  } catch (error) {
    console.error("Error sending temporary PIN:", error);
    res.status(500).json({
      success: false,
      message: "Error sending temporary PIN",
      error: error.message
    });
  }
};
exports.searchEmployees = async (req, res) => {
  try {
    const { query } = req.params;
    if (!query) {
      return res.status(400).json({
        success: false,
        message: "Search query is required"
      });
    }
    const employees = await Employee.find({
      $or: [
        { name: { $regex: query, $options: "i" } },
        { firstName: { $regex: query, $options: "i" } },
        { middleInitial: { $regex: query, $options: "i" } },
        { lastName: { $regex: query, $options: "i" } },
        { email: { $regex: query, $options: "i" } },
        { role: { $regex: query, $options: "i" } }
      ]
    }).sort({ dateJoined: -1 }).lean();
    const employeesWithoutPin = employees.map((emp) => {
      const { pin, ...rest } = emp;
      return rest;
    });
    res.json({
      success: true,
      count: employeesWithoutPin.length,
      data: employeesWithoutPin
    });
  } catch (error) {
    console.error("Error searching employees:", error);
    res.status(500).json({
      success: false,
      message: "Error searching employees",
      error: error.message
    });
  }
};
exports.updatePin = async (req, res) => {
  try {
    const { id } = req.params;
    const { pin, newPin, requiresPinReset } = req.body;
    const pinToSet = newPin || pin;
    if (!pinToSet) {
      return res.status(400).json({
        success: false,
        message: "PIN is required"
      });
    }
    if (await isPinAlreadyUsedByOther(pinToSet.toString(), id)) {
      return res.status(400).json({
        success: false,
        message: "Please use other pin"
      });
    }
    const salt = await bcrypt.genSalt(10);
    const hashedPin = await bcrypt.hash(pinToSet.toString(), salt);
```

```
    const fastPinHash = getFastPinHash(pinToSet.toString());
    const employee = await Employee.findByIdAndUpdate(
      id,
      {
        pin: hashedPin,
        fastPinHash,
        requiresPinReset: requiresPinReset !== void 0 ? requiresPinReset : false,
        lastUpdated: Date.now()
      },
      { new: true }
    );
    if (!employee) {
      return res.status(404).json({
        success: false,
        message: "Employee not found"
      });
    }
    const { pin: _, ...employeeWithoutPin } = employee.toObject();
    res.json({
      success: true,
      message: "PIN updated successfully",
      data: employeeWithoutPin
    });
  } catch (error) {
    console.error("Error updating PIN:", error);
    res.status(500).json({
      success: false,
      message: "Error updating PIN",
      error: error.message
    });
  }
};
exports.logoutEmployee = async (req, res) => {
  try {
    const { employeeId } = req.body;
    if (!employeeId) {
      return res.status(400).json({ success: false, message: "Employee ID is required" });
    }
    await Employee.findByIdAndUpdate(employeeId, { isOnline: false });
    res.json({ success: true, message: "Logged out successfully" });
  } catch (error) {
    console.error("Error logging out employee:", error);
    res.status(500).json({ success: false, message: "Error logging out", error: error.message });
  }
};
exports.employeeHeartbeat = async (req, res) => {
  try {
    const { employeeId } = req.body;
    if (!employeeId) {
      return res.status(400).json({ success: false, message: "Employee ID is required" });
    }
    await Employee.findByIdAndUpdate(employeeId, {
      isOnline: true,
      lastActive: /* @__PURE__ */ new Date()
    });
    res.json({ success: true, message: "Heartbeat received" });
  } catch (error) {
    console.error("Error processing heartbeat:", error);
    res.status(500).json({ success: false, message: "Server error" });
  }
};
exports.getOnlineEmployees = async (req, res) => {
  try {
    const twoMinsAgo = new Date(Date.now() - 2 * 60 * 1e3);
    await Employee.updateMany(
      {
        isOnline: true,
        role: { $ne: "Owner" },
        $or: [
          { lastActive: { $lt: twoMinsAgo } },
          // For legacy sessions that logged in before we added lastActive
          { lastActive: null, lastLogin: { $lt: twoMinsAgo } }
        ]
      },
      { $set: { isOnline: false } }
    );
    const onlineEmployees = await Employee.find({ isOnline: true, role: { $ne: "Owner" } }).select("-pin -
```

```
fastPinHash -profileImage").lean();
    res.json({ success: true, data: onlineEmployees });
  } catch (error) {
    console.error("Error fetching online employees:", error);
    res.status(500).json({ success: false, message: "Error fetching online employees", error:
error.message });
  }
};
```

FILE: pos/backend/controllers/gcashPaymentController.js
==========

```
const apicache = require("apicache");
const SalesTransaction = require("../models/SalesTransaction");
const Product = require("../models/Product");
const Discount = require("../models/Discount");
const gcashService = require("../services/gcashPaymentService");
const clearTransactionAnalyticsCache = () => {
  try {
    apicache.clear();
  } catch (_) {
  }
};
let wsClients = /* @__PURE__ */ new Map();
exports.setWsClients = (clients) => {
  wsClients = clients;
};
const notifyPaymentUpdate = (merchantOrderId, data) => {
  const clients = wsClients.get(merchantOrderId);
  if (clients && clients.size > 0) {
    const message = JSON.stringify({
      type: "PAYMENT_UPDATE",
      merchantOrderId,
      ...data
    });
    clients.forEach((ws) => {
      try {
        if (ws.readyState === 1) {
          ws.send(message);
        }
      } catch (err) {
        console.error("[WS] Error sending to client:", err.message);
      }
    });
    console.log(
      `[WS] Notified ${clients.size} client(s) for order ${merchantOrderId}`
    );
  }
};
const generateUniqueReceiptNumber = async () => {
  let attempts = 0;
  while (attempts < 10) {
    const receiptNo = Math.floor(1e5 + Math.random() * 9e5).toString();
    const existing = await SalesTransaction.findOne({ receiptNo });
    if (!existing) {
      return receiptNo;
    }
    attempts++;
  }
  const timestamp = Date.now().toString().slice(-4);
  const random = Math.floor(10 + Math.random() * 90).toString();
  return `${timestamp}${random}`;
};
exports.createGCashPayment = async (req, res) => {
  try {
    const {
      items,
      totalAmount,
      userId,
      performedById,
      performedByName,
      subtotal,
      discount,
      discountType,
      discountValue,
      appliedDiscountIds
```

```
} = req.body;
if (!items || !Array.isArray(items) || items.length === 0) {
  return res.status(400).json({
    success: false,
    message: "Items are required"
  });
}
if (!totalAmount || totalAmount <= 0) {
  return res.status(400).json({
    success: false,
    message: "Valid total amount is required"
  });
}
const merchantOrderId = gcashService.generateMerchantOrderId();
const receiptNo = await generateUniqueReceiptNumber();
const MerchantSettings = require("../models/MerchantSettings");
const config = await MerchantSettings.getActiveConfig();
const expiryMinutes = config?.paymentExpiryMinutes || 15;
const expiresAt = new Date(Date.now() + expiryMinutes * 60 * 1e3);
const transactionData = {
  receiptNo,
  userId: userId || performedById || "system",
  performedById: performedById || "",
  performedByName: performedByName || "",
  items: items.map((item) => ({
    productId: item.productId || item._id,
    itemName: item.itemName || "Unknown Item",
    sku: item.sku || "",
    variant: item.variant || null,
    selectedSize: item.selectedSize || null,
    quantity: item.quantity || 1,
    price: item.price || item.itemPrice || 0,
    itemImage: item.itemImage || ""
  })),
  subtotal: subtotal || totalAmount,
  discount: discount || 0,
  discountType: discountType || null,
  discountValue: discountValue || 0,
  totalAmount,
  paymentMethod: "gcash",
  amountReceived: 0,
  changeGiven: 0,
  status: "Pending",
  // Will be updated to Completed when payment is confirmed
  merchantOrderId,
  gcashStatus: "PENDING",
  gcashExpiresAt: expiresAt,
  checkedOutAt: /* @__PURE__ */ new Date()
};
const transaction = await SalesTransaction.create(transactionData);
let paymentSource;
try {
  paymentSource = await gcashService.createPaymentSource({
    merchantOrderId,
    amount: totalAmount,
    description: `POS Order ${receiptNo}`
  });
} catch (gatewayError) {
  transaction.gcashStatus = "FAILED";
  transaction.status = "Failed";
  await transaction.save();
  console.error(
    "[GCash] Gateway error, transaction marked FAILED:",
    gatewayError.message
  );
  return res.status(502).json({
    success: false,
    message: gatewayError.message || "Payment gateway unavailable",
    merchantOrderId
  });
}
transaction.gcashSourceId = paymentSource.sourceId;
transaction.gcashCheckoutUrl = paymentSource.checkoutUrl;
await transaction.save();
console.log("[GCash] Payment created:", {
  merchantOrderId,
  receiptNo,
```

```
      sourceId: paymentSource.sourceId,
      amount: totalAmount,
      expiresAt: expiresAt.toISOString()
    });
    res.status(201).json({
      success: true,
      data: {
        merchantOrderId,
        receiptNo,
        sourceId: paymentSource.sourceId,
        checkoutUrl: paymentSource.checkoutUrl,
        amount: totalAmount,
        currency: "PHP",
        status: "PENDING",
        expiresAt: expiresAt.toISOString(),
        expiryMinutes
      }
    });
    if (appliedDiscountIds && Array.isArray(appliedDiscountIds) && appliedDiscountIds.length > 0) {
      Promise.all(
        appliedDiscountIds.map(
          (id) => Discount.findByIdAndUpdate(id, { $inc: { usageCount: 1 } })
        )
      ).catch(
        (err) => console.error("Error updating discount usage counts:", err)
      );
    }
  } catch (error) {
    console.error("[GCash] Error creating payment:", error);
    res.status(500).json({
      success: false,
      message: "Failed to create GCash payment",
      error: error.message
    });
  }
};
exports.checkPaymentStatus = async (req, res) => {
  try {
    const { merchantOrderId } = req.params;
    const transaction = await SalesTransaction.findOne({ merchantOrderId }).select(
      "merchantOrderId gcashStatus gcashPaidAt gcashReference receiptNo totalAmount status gcashExpiresAt"
    ).lean();
    if (!transaction) {
      return res.status(404).json({
        success: false,
        message: "Payment not found"
      });
    }
    if (transaction.gcashStatus === "PENDING" && transaction.gcashExpiresAt && /* @__PURE__ */ new Date() >
new Date(transaction.gcashExpiresAt)) {
      await SalesTransaction.findOneAndUpdate(
        { merchantOrderId, gcashStatus: "PENDING" },
        { gcashStatus: "EXPIRED" }
      );
      transaction.gcashStatus = "EXPIRED";
    }
    res.json({
      success: true,
      data: {
        merchantOrderId: transaction.merchantOrderId,
        status: transaction.gcashStatus,
        paidAt: transaction.gcashPaidAt,
        gcashReference: transaction.gcashReference,
        receiptNo: transaction.receiptNo,
        totalAmount: transaction.totalAmount,
        transactionStatus: transaction.status,
        expiresAt: transaction.gcashExpiresAt
      }
    });
  } catch (error) {
    console.error("[GCash] Error checking status:", error);
    res.status(500).json({
      success: false,
      message: "Failed to check payment status"
    });
  }
};
```

```
exports.handleWebhook = async (req, res) => {
  try {
    const rawBody = JSON.stringify(req.body);
    const signatureHeader = req.headers["paymongo-signature"];
    const webhookSecret = process.env.PAYMONGO_WEBHOOK_SECRET;
    if (!webhookSecret) {
      console.error(
        "[GCash Webhook] PAYMONGO_WEBHOOK_SECRET is not configured \u2014 rejecting webhook for security"
      );
      return res.status(500).json({ message: "Webhook secret not configured" });
    }
    const isValid = gcashService.verifyWebhookSignature(
      rawBody,
      signatureHeader,
      webhookSecret
    );
    if (!isValid) {
      console.warn("[GCash Webhook] Invalid signature \u2014 rejecting");
      return res.status(401).json({ message: "Invalid signature" });
    }
    const event = req.body?.data;
    if (!event) {
      return res.status(400).json({ message: "Invalid webhook payload" });
    }
    const eventId = event.id;
    const eventType = event.attributes?.type;
    const sourceData = event.attributes?.data;
    console.log("[GCash Webhook] Received event:", {
      eventId,
      eventType,
      sourceId: sourceData?.id
    });
    if (eventType !== "source.chargeable") {
      console.log("[GCash Webhook] Ignoring event type:", eventType);
      return res.status(200).json({ message: "Event acknowledged" });
    }
    const sourceId = sourceData?.id;
    const merchantOrderId = sourceData?.attributes?.metadata?.merchant_order_id;
    const paidAmount = sourceData?.attributes?.amount;
    if (!sourceId || !merchantOrderId) {
      console.warn("[GCash Webhook] Missing sourceId or merchantOrderId");
      return res.status(400).json({ message: "Missing required fields" });
    }
    const existingProcessed = await SalesTransaction.findOne({
      merchantOrderId,
      gcashWebhookEventId: eventId
    });
    if (existingProcessed) {
      console.log(
        "[GCash Webhook] Duplicate event, already processed:",
        eventId
      );
      return res.status(200).json({ message: "Already processed" });
    }
    const transaction = await SalesTransaction.findOne({
      merchantOrderId,
      gcashSourceId: sourceId,
      gcashStatus: { $in: ["PENDING", "CHARGEABLE"] }
    });
    if (!transaction) {
      console.warn(
        "[GCash Webhook] No matching pending transaction for:",
        merchantOrderId
      );
      return res.status(404).json({ message: "Transaction not found" });
    }
    const expectedAmountCentavos = Math.round(transaction.totalAmount * 100);
    if (paidAmount && Math.abs(paidAmount - expectedAmountCentavos) > 1) {
      console.error("[GCash Webhook] Amount mismatch:", {
        expected: expectedAmountCentavos,
        received: paidAmount,
        merchantOrderId
      });
      transaction.gcashStatus = "FAILED";
      transaction.status = "Failed";
      await transaction.save();
      return res.status(400).json({ message: "Amount mismatch" });
```

```
    }
    transaction.gcashStatus = "CHARGEABLE";
    transaction.gcashWebhookEventId = eventId;
    await transaction.save();
    let paymentResult;
    try {
      paymentResult = await gcashService.createPayment(
        sourceId,
        transaction.totalAmount
      );
    } catch (paymentError) {
      console.error(
        "[GCash Webhook] Failed to create payment:",
        paymentError.message
      );
      transaction.gcashStatus = "FAILED";
      transaction.status = "Failed";
      await transaction.save();
      notifyPaymentUpdate(merchantOrderId, {
        status: "FAILED",
        error: "Payment processing failed"
      });
      return res.status(500).json({ message: "Payment creation failed" });
    }
    transaction.gcashPaymentId = paymentResult.paymentId;
    transaction.gcashPaidAt = paymentResult.paidAt || /* @__PURE__ */ new Date();
    transaction.gcashReference = paymentResult.gcashReference;
    transaction.gcashFeeAmount = paymentResult.feeAmount;
    transaction.gcashNetAmount = paymentResult.netAmount;
    transaction.amountReceived = transaction.totalAmount;
    transaction.referenceNo = paymentResult.gcashReference || paymentResult.paymentId;
    console.log("[GCash Webhook] \u2705 Payment confirmed:", {
      merchantOrderId,
      paymentId: paymentResult.paymentId,
      amount: transaction.totalAmount,
      gcashReference: paymentResult.gcashReference
    });
    try {
      await updateStockAfterPayment(transaction);
    } catch (stockErr) {
      console.error("[GCash] Stock update error:", stockErr.message);
    }
    transaction.gcashStatus = "PAID";
    transaction.status = "Completed";
    await transaction.save();
    notifyPaymentUpdate(merchantOrderId, {
      status: "PAID",
      paidAt: transaction.gcashPaidAt,
      gcashReference: transaction.gcashReference,
      receiptNo: transaction.receiptNo,
      totalAmount: transaction.totalAmount
    });
    res.status(200).json({ message: "Payment processed successfully" });
  } catch (error) {
    console.error("[GCash Webhook] Unhandled error:", error);
    res.status(200).json({ message: "Acknowledged with error" });
  }
};
exports.cancelPayment = async (req, res) => {
  try {
    const { merchantOrderId } = req.params;
    const transaction = await SalesTransaction.findOne({
      merchantOrderId,
      gcashStatus: "PENDING"
    });
    if (!transaction) {
      return res.status(404).json({
        success: false,
        message: "No pending payment found to cancel"
      });
    }
    transaction.gcashStatus = "EXPIRED";
    transaction.status = "Voided";
    transaction.voidReason = "GCash payment cancelled by cashier";
    transaction.voidedAt = /* @__PURE__ */ new Date();
    await transaction.save();
    console.log("[GCash] Payment cancelled:", merchantOrderId);
```

```
    res.json({
      success: true,
      message: "Payment cancelled successfully"
    });
  } catch (error) {
    console.error("[GCash] Error cancelling payment:", error);
    res.status(500).json({
      success: false,
      message: "Failed to cancel payment"
    });
  }
};
exports.getConfigStatus = async (req, res) => {
  try {
    const MerchantSettings = require("../models/MerchantSettings");
    const config = await MerchantSettings.getActiveConfig();
    res.json({
      success: true,
      data: {
        isConfigured: !!config,
        environment: config?.environment || null,
        merchantName: config?.merchantName || null
      }
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: "Failed to check configuration"
    });
  }
};
async function updateStockAfterPayment(transaction) {
  if (!transaction || !transaction.items || transaction.items.length === 0)
    return;
  const getSizeQty = (sizeData) => {
    if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
      return sizeData.quantity;
    }
    return typeof sizeData === "number" ? sizeData : 0;
  };
  const getSizePrice = (sizeData) => {
    if (typeof sizeData === "object" && sizeData !== null && sizeData.price !== void 0) {
      return sizeData.price;
    }
    return null;
  };
  const findSizeKey = (sizes, targetSize) => {
    if (!sizes || !targetSize) return null;
    const keys = typeof sizes.keys === "function" ? Array.from(sizes.keys()) : Object.keys(sizes);
    return keys.find((k) => k.toLowerCase() === targetSize.toLowerCase()) || null;
  };
  for (const item of transaction.items) {
    try {
      const product = await Product.findById(item.productId);
      if (!product) {
        console.warn(`[GCash] Product not found: ${item.productId}`);
        continue;
      }
      const stockBefore = product.currentStock || 0;
      const qty = item.quantity || 1;
      const size = item.selectedSize || null;
      const variant = item.variant || null;
      if (size && product.sizes && typeof product.sizes === "object") {
        const sizeKey = findSizeKey(product.sizes, size);
        if (sizeKey) {
          const currentSizeData = product.sizes.get ? product.sizes.get(sizeKey) : product.sizes[sizeKey];
          const currentQty = getSizeQty(currentSizeData);
          const currentPrice = getSizePrice(currentSizeData);
          if (variant && typeof currentSizeData === "object" && currentSizeData !== null &&
currentSizeData.variants) {
            const variantData = currentSizeData.variants[variant];
            let currentVariantQty = 0;
            if (typeof variantData === "number") {
              currentVariantQty = variantData;
            } else if (typeof variantData === "object" && variantData !== null) {
              currentVariantQty = variantData.quantity || 0;
            }
```

```
          const newVariantQty = Math.max(0, currentVariantQty - qty);
          if (typeof variantData === "object" && variantData !== null) {
            currentSizeData.variants[variant] = {
              ...variantData,
              quantity: newVariantQty
            };
          } else {
            currentSizeData.variants[variant] = newVariantQty;
          }
          let sizeTotal = 0;
          for (const [varKey, varValue] of Object.entries(currentSizeData.variants)) {
            if (typeof varValue === "number") {
              sizeTotal += varValue;
            } else if (typeof varValue === "object" && varValue !== null) {
              sizeTotal += varValue.quantity || 0;
            }
          }
          currentSizeData.quantity = sizeTotal;
          if (product.sizes.set) {
            product.sizes.set(sizeKey, currentSizeData);
          } else {
            product.sizes[sizeKey] = currentSizeData;
          }
          product.markModified("sizes");
        } else {
          const newQty = Math.max(0, currentQty - qty);
          if (currentPrice !== null || typeof currentSizeData === "object" && currentSizeData !== null) {
            if (product.sizes.set) {
              product.sizes.set(sizeKey, {
                ...currentSizeData,
                quantity: newQty,
                price: currentPrice !== null ? currentPrice : product.itemPrice || 0
              });
            } else {
              product.sizes[sizeKey] = {
                ...currentSizeData,
                quantity: newQty,
                price: currentPrice !== null ? currentPrice : product.itemPrice || 0
              };
            }
          } else {
            if (product.sizes.set) {
              product.sizes.set(sizeKey, newQty);
            } else {
              product.sizes[sizeKey] = newQty;
            }
          }
          product.markModified("sizes");
        }
      } else {
        console.warn(
          `[GCash] Size "${size}" not found for product ${product.itemName}`
        );
      }
      const sizeEntries = product.sizes.entries ? Array.from(product.sizes.entries()) :
Object.entries(product.sizes);
      let totalStock = 0;
      for (const [, value] of sizeEntries) {
        totalStock += getSizeQty(value);
      }
      product.currentStock = totalStock;
    } else {
      product.currentStock = Math.max(0, (product.currentStock || 0) - qty);
    }
    if (product.currentStock === 0) {
      product.displayInTerminal = false;
    } else if (stockBefore === 0 && product.currentStock > 0) {
      product.displayInTerminal = true;
    }
    product.lastUpdated = Date.now();
    await product.save();
  } catch (err) {
    console.error(
      `[GCash] Stock update failed for product ${item.productId}:`,
      err.message
    );
  }
```

```
  }
  console.log("[GCash] Stock updated for", transaction.items.length, "items");
}



FILE: pos/backend/controllers/globalSettingsController.js
================================================================================
const GlobalSettings = require("../models/GlobalSettings");
const startOfLocalDay = (value = /* @__PURE__ */ new Date()) => {
  const date = new Date(value);
  return new Date(date.getFullYear(), date.getMonth(), date.getDate());
};
const toNumericFloat = (value, fallback = 0) => {
  const parsed = Number(value);
  return Number.isFinite(parsed) && parsed >= 0 ? parsed : fallback;
};
const normalizeOpeningFloatEntry = (entry = {}) => {
  const id = String(entry._id || entry.id || "").trim();
  const employeeId = String(entry.employeeId || "").trim();
  const employeeName = String(entry.employeeName || "").trim();
  const employeeRole = String(entry.employeeRole || "").trim();
  const amount = toNumericFloat(entry.amount, NaN);
  if (!employeeId || !employeeName || !Number.isFinite(amount)) {
    return null;
  }
  return {
    ...id ? { _id: id } : {},
    employeeId,
    employeeName,
    employeeRole,
    amount,
    createdAt: entry.createdAt ? new Date(entry.createdAt) : /* @__PURE__ */ new Date(),
    businessDate: entry.businessDate ? startOfLocalDay(entry.businessDate) : startOfLocalDay(entry.createdAt
|| /* @__PURE__ */ new Date()),
    updatedAt: entry.updatedAt ? new Date(entry.updatedAt) : null
  };
};
exports.getGlobalSettings = async (req, res) => {
  try {
    let settings = await GlobalSettings.findOne();
    if (!settings) {
      settings = await GlobalSettings.create({});
    }
    res.json({ success: true, data: settings });
  } catch (error) {
    console.error("Error fetching global settings:", error);
    res.status(500).json({ success: false, message: "Server error" });
  }
};
exports.updateGlobalSettings = async (req, res) => {
  try {
    const { openingFloat, openingFloats, addOpeningFloat, updateOpeningFloat, removeOpeningFloat } = req.body;
    let settings = await GlobalSettings.findOne();
    if (!settings) {
      settings = await GlobalSettings.create({});
    }
    if (openingFloat !== void 0) {
      settings.openingFloat = toNumericFloat(openingFloat, settings.openingFloat || 2e3);
    }
    if (Array.isArray(openingFloats)) {
      settings.openingFloats = openingFloats.map(normalizeOpeningFloatEntry).filter(Boolean);
    }
    if (addOpeningFloat) {
      const normalizedEntry = normalizeOpeningFloatEntry(addOpeningFloat);
      if (!normalizedEntry) {
        return res.status(400).json({
          success: false,
          message: "Employee and amount are required for opening float entry"
        });
      }
      const today = startOfLocalDay(/* @__PURE__ */ new Date());
      const employeeIdStr = String(normalizedEntry.employeeId || "").trim();
      const existing = (settings.openingFloats || []).find((row) => {
        const rowEmp = String(row?.employeeId || "").trim();
        if (!rowEmp || rowEmp !== employeeIdStr) return false;
        const rowDate = row?.businessDate || row?.createdAt;
```

```
        if (!rowDate) return false;
        return startOfLocalDay(rowDate).getTime() === today.getTime();
      });
      if (existing) {
        existing.amount = normalizedEntry.amount;
        existing.employeeName = normalizedEntry.employeeName;
        existing.employeeRole = normalizedEntry.employeeRole;
        existing.businessDate = today;
        existing.updatedAt = /* @__PURE__ */ new Date();
      } else {
        normalizedEntry.businessDate = today;
        settings.openingFloats.push(normalizedEntry);
      }
    }
    if (updateOpeningFloat) {
      const entryId = String(updateOpeningFloat.entryId || updateOpeningFloat._id || "").trim();
      const nextAmount = toNumericFloat(updateOpeningFloat.amount, NaN);
      if (!entryId || !Number.isFinite(nextAmount)) {
        return res.status(400).json({
          success: false,
          message: "Entry id and amount are required to update opening float"
        });
      }
      const entry = settings.openingFloats.id(entryId);
      if (!entry) {
        return res.status(404).json({
          success: false,
          message: "Opening float entry not found"
        });
      }
      entry.amount = nextAmount;
      if (updateOpeningFloat.employeeName) {
        entry.employeeName = String(updateOpeningFloat.employeeName).trim();
      }
      if (updateOpeningFloat.employeeRole !== void 0) {
        entry.employeeRole = String(updateOpeningFloat.employeeRole || "").trim();
      }
      entry.businessDate = startOfLocalDay(/* @__PURE__ */ new Date());
      entry.updatedAt = /* @__PURE__ */ new Date();
    }
    if (removeOpeningFloat) {
      const entryId = String(removeOpeningFloat.entryId || removeOpeningFloat._id || "").trim();
      if (!entryId) {
        return res.status(400).json({
          success: false,
          message: "Entry id is required to remove opening float"
        });
      }
      const entry = settings.openingFloats.id(entryId);
      if (!entry) {
        return res.status(404).json({
          success: false,
          message: "Opening float entry not found"
        });
      }
      entry.deleteOne();
    }
    if (openingFloat === void 0 && !Array.isArray(openingFloats) && !addOpeningFloat && !updateOpeningFloat
&& !removeOpeningFloat) {
      return res.status(400).json({
        success: false,
        message: "No settings update payload provided"
      });
    } else {
    }
    await settings.save();
    res.json({ success: true, data: settings, message: "Settings updated successfully" });
  } catch (error) {
    console.error("Error updating global settings:", error);
    res.status(500).json({ success: false, message: "Server error" });
  }
};
```

FILE: pos/backend/controllers/merchantSettingsController.js
==========

```
const MerchantSettings = require("../models/MerchantSettings");
const { encrypt } = require("../utils/encryption");
exports.getSettings = async (req, res) => {
  try {
    const settings = await MerchantSettings.getActiveConfig();
    if (!settings) {
      return res.json({
        success: true,
        data: null,
        message: "No payment gateway configured"
      });
    }
    res.json({
      success: true,
      data: settings.toSafeJSON()
    });
  } catch (error) {
    console.error("[MerchantSettings] Error fetching settings:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch payment settings",
      error: error.message
    });
  }
};
exports.saveSettings = async (req, res) => {
  try {
    const {
      appId,
      privateKey,
      publicKey,
      environment,
      merchantName,
      paymentExpiryMinutes,
      configuredBy,
      configuredByName
    } = req.body;
    if (!appId || !privateKey || !publicKey) {
      return res.status(400).json({
        success: false,
        message: "app_id, private_key, and public_key are all required"
      });
    }
    if (environment && !["sandbox", "production"].includes(environment)) {
      return res.status(400).json({
        success: false,
        message: 'Environment must be "sandbox" or "production"'
      });
    }
    const { encrypted, iv, authTag } = encrypt(privateKey);
    const baseUrl = process.env.WEBHOOK_BASE_URL || process.env.BASE_URL ||
`http://localhost:${process.env.PORT || 5e3}`;
    const webhookUrl = `${baseUrl}/api/payments/gcash/webhook`;
    await MerchantSettings.updateMany({ isActive: true }, { isActive: false });
    const settings = await MerchantSettings.create({
      isActive: true,
      appId: appId.trim(),
      encryptedPrivateKey: encrypted,
      privateKeyIV: iv,
      privateKeyAuthTag: authTag,
      publicKey: publicKey.trim(),
      environment: environment || "sandbox",
      webhookUrl,
      merchantName: merchantName?.trim() || "POS System",
      paymentExpiryMinutes: paymentExpiryMinutes || 15,
      configuredBy: configuredBy || "",
      configuredByName: configuredByName || ""
    });
    console.log("[MerchantSettings] Configuration saved:", {
      id: settings._id,
      appId: settings.appId,
      environment: settings.environment,
      configuredBy: configuredByName
    });
    res.status(201).json({
      success: true,
      message: "Payment gateway configured successfully",
```

```
      data: settings.toSafeJSON()
    });
  } catch (error) {
    console.error("[MerchantSettings] Error saving settings:", error);
    res.status(500).json({
      success: false,
      message: "Failed to save payment settings",
      error: error.message
    });
  }
};
exports.deleteSettings = async (req, res) => {
  try {
    const result = await MerchantSettings.updateMany(
      { isActive: true },
      { isActive: false }
    );
    res.json({
      success: true,
      message: result.modifiedCount > 0 ? "Payment gateway configuration removed" : "No active configuration
to remove"
    });
  } catch (error) {
    console.error("[MerchantSettings] Error deleting settings:", error);
    res.status(500).json({
      success: false,
      message: "Failed to remove payment settings",
      error: error.message
    });
  }
};
exports.testConnection = async (req, res) => {
  try {
    const settings = await MerchantSettings.getActiveConfig();
    if (!settings) {
      return res.status(404).json({
        success: false,
        message: "No payment gateway configured"
      });
    }
    const { decrypt: decryptKey } = require("../utils/encryption");
    const privateKey = decryptKey(
      settings.encryptedPrivateKey,
      settings.privateKeyIV,
      settings.privateKeyAuthTag
    );
    const baseUrl = settings.environment === "production" ? "https://api.paymongo.com/v1" :
"https://api.paymongo.com/v1";
    const response = await fetch(`${baseUrl}/payment_methods?limit=1`, {
      method: "GET",
      headers: {
        Authorization: `Basic ${Buffer.from(privateKey + ":").toString("base64")}`,
        Accept: "application/json"
      }
    });
    if (response.ok || response.status === 404) {
      res.json({
        success: true,
        message: `Connected to PayMongo (${settings.environment}) successfully`,
        environment: settings.environment
      });
    } else {
      const errorData = await response.json().catch(() => ({}));
      res.status(400).json({
        success: false,
        message: "Connection failed. Please verify your credentials.",
        error: errorData?.errors?.[0]?.detail || `HTTP ${response.status}`
      });
    }
  } catch (error) {
    console.error("[MerchantSettings] Test connection error:", error);
    res.status(500).json({
      success: false,
      message: "Connection test failed",
      error: error.message
    });
  }
```

```
};



FILE: pos/backend/controllers/printController.js
================================================================================
const { ThermalPrinter, PrinterTypes, CharacterSet, BreakLine } = require("node-thermal-printer");
const escpos = require("escpos");
let windowsPrinterDriver = null;
let UsbAdapter = null;
try {
  windowsPrinterDriver = require("printer");
  if (windowsPrinterDriver && typeof windowsPrinterDriver.getPrinters === "function") {
    console.log("Windows printer driver module loaded successfully");
  } else {
    console.warn("Windows printer driver module loaded but appears to be invalid");
    windowsPrinterDriver = null;
  }
} catch (err) {
  console.warn('Windows printer driver module "printer" failed to load:', err.message);
  console.warn('Install it with "npm install printer" or use TCP/IP printing instead.');
  windowsPrinterDriver = null;
}
try {
  const usb = require("usb");
  if (usb && typeof usb.on === "undefined") {
    const EventEmitter = require("events");
    const usbEmitter = new EventEmitter();
    usb.on = function(event, callback) {
      usbEmitter.on(event, callback);
    };
    usb.emit = function(event, ...args) {
      usbEmitter.emit(event, ...args);
    };
    usb.removeListener = function(event, callback) {
      usbEmitter.removeListener(event, callback);
    };
    usb.removeAllListeners = function(event) {
      usbEmitter.removeAllListeners(event);
    };
  }
  UsbAdapter = require("escpos-usb");
  escpos.USB = UsbAdapter;
} catch (err) {
  console.warn('Optional module "escpos-usb" is not installed. Run "npm install escpos escpos-usb" to enable
direct USB thermal printing.');
}
const RECEIPT_CHAR_WIDTH = parseInt(process.env.RECEIPT_CHAR_WIDTH || "32", 10);
if (RECEIPT_CHAR_WIDTH !== 32) {
  console.warn(`\u26A0\uFE0F  WARNING: RECEIPT_CHAR_WIDTH is set to ${RECEIPT_CHAR_WIDTH}, but 58mm paper
requires 32 characters per line.`);
  console.warn(`   Current setting will work for ${RECEIPT_CHAR_WIDTH === 48 ? "80mm" : "custom"} paper
width.`);
} else {
  console.log("\u2705 Receipt configured for 58mm thermal paper (32 characters per line)");
}
const DEFAULT_STORE_NAME = process.env.STORE_NAME || "CreateYourStyle";
const DEFAULT_ADDRESS_LINES = (process.env.STORE_ADDRESS || "TC USHS - #831100254488|Pasoanca, Zambonaga
City").split("|");
const CURRENCY_SYMBOL = process.env.PRINTER_CURRENCY_SYMBOL || "\u20B1";
const formatCurrency = (value = 0) => `${CURRENCY_SYMBOL}${Number(value || 0).toFixed(2)}`;
const lineBreak = (char = "-") => char.repeat(RECEIPT_CHAR_WIDTH);
const padLine = (left = "", right = "") => {
  const lhs = String(left);
  const rhs = String(right);
  const spacing = Math.max(RECEIPT_CHAR_WIDTH - lhs.length - rhs.length, 0);
  return `${lhs}${" ".repeat(spacing)}${rhs}`.slice(0, RECEIPT_CHAR_WIDTH);
};
const parseUsbId = (value) => {
  if (!value) return null;
  if (typeof value === "number") return value;
  const trimmed = value.trim();
  if (!trimmed) return null;
  const radix = trimmed.toLowerCase().startsWith("0x") ? 16 : 10;
  const parsed = parseInt(trimmed, radix);
  return Number.isNaN(parsed) ? null : parsed;
};
```

```
const printViaEscposUsb = (receiptData = {}) => new Promise((resolve, reject) => {
  if (!UsbAdapter) {
    reject(new Error('escpos-usb adapter is not installed. Run "npm install escpos escpos-usb" inside
backend/.'));
    return;
  }
  const vendorId = parseUsbId(process.env.PRINTER_VENDOR_ID);
  const productId = parseUsbId(process.env.PRINTER_PRODUCT_ID);
  console.log("USB Printer Configuration:");
  console.log(`  Vendor ID: ${process.env.PRINTER_VENDOR_ID} -> ${vendorId} (${vendorId ? "0x" +
vendorId.toString(16) : "not set"})`);
  console.log(`  Product ID: ${process.env.PRINTER_PRODUCT_ID} -> ${productId} (${productId ? "0x" +
productId.toString(16) : "not set"})`);
  let device;
  try {
    if (vendorId && productId) {
      console.log(`Attempting to connect to USB printer with VID: 0x${vendorId.toString(16)}, PID:
0x${productId.toString(16)}`);
      device = new escpos.USB(vendorId, productId);
    } else {
      console.log("No vendor/product ID specified, attempting auto-detect...");
      device = new escpos.USB();
    }
  } catch (usbError) {
    const errorMsg = usbError.message || "Unknown USB error";
    console.error("USB Error Details:", errorMsg);
    console.error("\n\u26A0\uFE0F  IMPORTANT: USB printing requires WinUSB driver to be installed.");
    console.error("   Steps to fix:");
    console.error("   1. Download Zadig: https://zadig.akeo.ie/");
    console.error("   2. Open Zadig \u2192 Options \u2192 List All Devices");
    console.error(`   3. Select your printer (VID: 0x${vendorId ? vendorId.toString(16).padStart(4, "0") :
"XXXX"}, PID: 0x${productId ? productId.toString(16).padStart(4, "0") : "YYYY"})`);
    console.error('   4. Select "WinUSB" as the driver');
    console.error('   5. Click "Install Driver" or "Replace Driver"');
    console.error("   6. Restart your computer if prompted");
    console.error("\n   The system will automatically fall back to file printing until the driver is
installed.\n");
    reject(new Error(`USB device initialization failed: ${errorMsg}. WinUSB driver required. See console for
installation instructions.`));
    return;
  }
  if (!device || typeof device.open !== "function") {
    reject(new Error("USB device object is invalid. The escpos-usb library may be incompatible. Try using the
thermal printer interface instead."));
    return;
  }
  device.open((error) => {
    if (error) {
      reject(error);
      return;
    }
    const printer = new escpos.Printer(device, {
      encoding: process.env.PRINTER_ENCODING || "GB18030"
    });
    const closePrinter = () => {
      try {
        printer.close();
      } catch (closeErr) {
        console.error("Failed to close printer connection", closeErr);
      }
    };
    const storeName = receiptData.storeName || DEFAULT_STORE_NAME;
    const contactNumber = receiptData.contactNumber || "+631112224444";
    const location = receiptData.location || "Pasonanca, Zamboanga City";
    const receiptNo = receiptData.receiptNo || "000000";
    const paymentMethod = receiptData.paymentMethod || "CASH";
    const referenceNo = receiptData.referenceNo || receiptData.reference || "";
    const issueDate = receiptData.date || (/* @__PURE__ */ new Date()).toLocaleDateString();
    const issueTime = receiptData.time || (/* @__PURE__ */ new Date()).toLocaleTimeString();
    try {
      printer.align("ct").size(1, 0).text(storeName).size(0, 0).feed(1);
      printer.align("lt");
      const timeLine = padLine(issueTime, contactNumber);
      printer.text(timeLine);
      printer.align("ct");
      printer.text(location);
      printer.feed(1);
```

```
      printer.text(lineBreak());
      printer.align("ct");
      printer.text("Receipt No:");
      printer.style("B");
      printer.text(`#${receiptNo}`);
      printer.style("NORMAL");
      printer.text(lineBreak());
      printer.align("lt");
      const itemCol = "Item".padEnd(14);
      const qtyCol = "Qty".padStart(5);
      const priceCol = "Price".padStart(13);
      printer.text(`${itemCol}${qtyCol}${priceCol}`);
      printer.text(lineBreak());
      (receiptData.items || []).forEach((item) => {
        const itemName = (item.name || item.itemName || "Item").toString();
        const qty = item.qty || item.quantity || 1;
        const price = item.price || item.itemPrice || 0;
        const itemNameLine = itemName.substring(0, 14).padEnd(14);
        const qtyStr = qty.toString().padStart(5);
        const priceStr = `PHP ${price.toFixed(2)}`.padStart(13);
        printer.text(`${itemNameLine}${qtyStr}${priceStr}`);
      });
      if ((receiptData.items || []).length === 0) {
        printer.text("No line items provided.");
      }
      printer.text(lineBreak());
      printer.align("lt");
      printer.text(padLine("Payment Method", paymentMethod));
      printer.text(padLine("Subtotal", `PHP ${(receiptData.subtotal || 0).toFixed(2)}`));
      printer.text(lineBreak());
      printer.text(padLine("Discount", `PHP ${(receiptData.discount || 0).toFixed(2)}`));
      printer.text(lineBreak());
      printer.style("B");
      printer.text(padLine("Total", `PHP ${(receiptData.total || 0).toFixed(2)}`));
      printer.style("NORMAL");
      if (receiptData.cash !== void 0) {
        printer.text(padLine("Cash", `PHP ${receiptData.cash.toFixed(2)}`));
      }
      if (receiptData.change !== void 0) {
        printer.text(padLine("Change", `PHP ${receiptData.change.toFixed(2)}`));
      }
      printer.text(lineBreak());
      printer.align("ct");
      printer.style("B");
      printer.text("This is not an official receipt");
      printer.style("NORMAL");
      printer.feed(2);
      printer.cut("full");
      closePrinter();
      resolve();
    } catch (printErr) {
      closePrinter();
      reject(printErr);
    }
  });
});
const printReceipt = async (req, res) => {
  try {
    const { receiptData } = req.body;
    if (!receiptData) {
      return res.status(400).json({ success: false, message: "Receipt data is required" });
    }
    const printerType = process.env.PRINTER_TYPE || "EPSON";
    const defaultInterface = !windowsPrinterDriver ? "file" : "printer";
    let printerInterface = process.env.PRINTER_INTERFACE || defaultInterface;
    let printerName = process.env.PRINTER_NAME || (defaultInterface === "file" ? "./receipt.txt" : "");
    const printerIP = process.env.PRINTER_IP || "192.168.1.100";
    const printerPort = process.env.PRINTER_PORT || "9100";
    const printerTransport = (process.env.PRINTER_TRANSPORT || "").toLowerCase();
    if (printerTransport === "escpos-usb") {
      try {
        await printViaEscposUsb(receiptData);
        return res.json({
          success: true,
          message: "Receipt sent to USB thermal printer successfully"
        });
      } catch (usbError) {
```

```
        console.error("USB print error:", usbError);
        printerInterface = "file";
        printerName = process.env.PRINTER_NAME || "./receipt.txt";
      }
    }
    let printer;
    let printerTypeEnum = PrinterTypes.EPSON;
    if (printerType === "STAR") {
      printerTypeEnum = PrinterTypes.STAR;
    } else if (printerType === "BEPOST") {
      printerTypeEnum = PrinterTypes.BEPOST;
    }
    let interfaceString;
    if (printerInterface === "tcp" || printerInterface === "network") {
      interfaceString = `tcp://${printerIP}:${printerPort}`;
    } else if (printerInterface === "usb") {
      interfaceString = "usb";
    } else if (printerInterface === "file") {
      interfaceString = `file:${printerName || "./receipt.txt"}`;
    } else {
      interfaceString = printerName ? `printer:${printerName}` : "printer";
    }
    if ((interfaceString.startsWith("printer:") || interfaceString === "printer") && !windowsPrinterDriver) {
      console.warn('Windows printer driver module "printer" is not available. Falling back to file
printing...');
      interfaceString = `file:${printerName || "./receipt.txt"}`;
      console.log(`Receipt will be saved to: ${interfaceString.replace("file:", "")}`);
    }
    const printerConfig = {
      type: printerTypeEnum,
      interface: interfaceString,
      characterSet: CharacterSet.PC852_LATIN2,
      removeSpecialCharacters: false,
      lineCharacter: "-",
      breakLine: BreakLine.WORD,
      options: {
        timeout: 3e4
      }
    };
    if ((interfaceString.startsWith("printer:") || interfaceString === "printer") && windowsPrinterDriver) {
      printerConfig.driver = windowsPrinterDriver;
    }
    printer = new ThermalPrinter(printerConfig);
    if (!printer) {
      throw new Error("Failed to initialize printer. Please check printer configuration.");
    }
    let isConnected = true;
    const shouldCheckConnection = printerInterface === "tcp" || printerInterface === "network";
    if (shouldCheckConnection) {
      isConnected = false;
      try {
        isConnected = await printer.isPrinterConnected();
      } catch (connectionError) {
        console.error("Printer connection check failed:", connectionError);
        isConnected = true;
      }
      if (!isConnected) {
        return res.status(500).json({
          success: false,
          message: "Printer not connected. Please check printer connection and configuration. See
README_PRINTER_SETUP.md for setup instructions."
        });
      }
    }
    const storeName = receiptData.storeName || DEFAULT_STORE_NAME;
    const contactNumber = receiptData.contactNumber || "+631112224444";
    const location = receiptData.location || "Pasonanca, Zamboanga City";
    const referenceNo = receiptData.referenceNo || receiptData.reference || "";
    const issueTime = receiptData.time || (/* @__PURE__ */ new Date()).toLocaleTimeString();
    printer.alignCenter();
    printer.setTextSize(1, 0);
    printer.bold(true);
    printer.println(storeName);
    printer.bold(false);
    printer.setTextSize(0, 0);
    printer.alignLeft();
    const timeContactLine = `${issueTime}${" ".repeat(Math.max(0, RECEIPT_CHAR_WIDTH - issueTime.length -
```

```
contactNumber.length))}${contactNumber}`;
    printer.println(timeContactLine.substring(0, RECEIPT_CHAR_WIDTH));
    printer.alignCenter();
    printer.println(location);
    printer.drawLine();
    printer.alignCenter();
    printer.println("Receipt No:");
    printer.bold(true);
    printer.println(`#${receiptData.receiptNo || "000000"}`);
    printer.bold(false);
    printer.drawLine();
    printer.alignLeft();
    printer.tableCustom([
      { text: "Item", align: "LEFT", width: 0.55 },
      { text: "Qty", align: "CENTER", width: 0.15 },
      { text: "Price", align: "RIGHT", width: 0.3 }
    ]);
    printer.drawLine();
    if (receiptData.items && receiptData.items.length > 0) {
      receiptData.items.forEach((item) => {
        const itemName = item.name || item.itemName || "Item";
        const qty = item.qty || item.quantity || 1;
        const price = item.price || item.itemPrice || 0;
        printer.tableCustom([
          { text: itemName.substring(0, 20), align: "LEFT", width: 0.55 },
          { text: qty.toString(), align: "CENTER", width: 0.15 },
          { text: `PHP${price.toFixed(2)}`, align: "RIGHT", width: 0.3 }
        ]);
      });
    }
    printer.drawLine();
    printer.alignLeft();
    printer.println(`Transaction/Reference: ${referenceNo || "-"}`);
    printer.println(`Payment Method: ${receiptData.paymentMethod || "CASH"}`);
    printer.println(`Subtotal: PHP ${(receiptData.subtotal || 0).toFixed(2)}`);
    printer.drawLine();
    printer.println(`Discount: PHP ${(receiptData.discount || 0).toFixed(2)}`);
    printer.drawLine();
    printer.bold(true);
    printer.println(`Total: PHP ${(receiptData.total || 0).toFixed(2)}`);
    printer.bold(false);
    if (receiptData.cash) {
      printer.println(`Cash: PHP ${receiptData.cash.toFixed(2)}`);
      printer.println(`Change: PHP ${(receiptData.change || 0).toFixed(2)}`);
    }
    printer.drawLine();
    printer.alignCenter();
    printer.bold(true);
    printer.println("This is not an official receipt");
    printer.bold(false);
    printer.newLine();
    printer.cut();
    const isFilePrinting = interfaceString.startsWith("file:");
    if (isFilePrinting) {
      console.warn("WARNING: Using file printing mode. Receipt will be saved to file, not printed to physical
printer.");
      console.warn(`File location: ${printerName || "./receipt.txt"}`);
    } else {
      console.log(`Attempting to print to: ${interfaceString}`);
    }
    let printSuccess = false;
    try {
      console.log("Sending print job to printer...");
      await printer.execute();
      printSuccess = true;
      console.log("Print job sent successfully");
    } catch (executeError) {
      console.error("Printer execute error:", executeError);
      console.error("Error details:", {
        message: executeError.message,
        stack: executeError.stack,
        interface: interfaceString,
        printerType
      });
      const errorMessage = executeError.message || "Unknown printer error";
      const isTimeout = errorMessage.toLowerCase().includes("timeout") ||
errorMessage.toLowerCase().includes("timed out");
```

```
      const isConnection = errorMessage.toLowerCase().includes("connect") ||
errorMessage.toLowerCase().includes("connection");
      return res.status(500).json({
        success: false,
        message: `Print execution failed: ${errorMessage}. ${isTimeout ? "Printer may be slow or
unresponsive." : isConnection ? "Please check printer connection." : "Please check printer configuration and
connection."}`
      });
    }
    if (!printSuccess) {
      return res.status(500).json({
        success: false,
        message: "Print job did not complete successfully"
      });
    }
    if (isFilePrinting) {
      return res.json({
        success: false,
        message: `Receipt saved to file (${printerName || "./receipt.txt"}) but not printed. Please configure
a physical printer. See PRINTER_SETUP.md for instructions.`,
        fileSaved: true,
        filePath: printerName || "./receipt.txt"
      });
    }
    console.log("Print completed successfully");
    res.json({
      success: true,
      message: "Receipt printed successfully"
    });
  } catch (error) {
    console.error("Print error:", error);
    res.status(500).json({
      success: false,
      message: "Failed to print receipt: " + error.message
    });
  }
};
module.exports = { printReceipt };
```

FILE: pos/backend/controllers/productController.js
==========

```
const Product = require("../models/Product");
const StockMovement = require("../models/StockMovement");
const Archive = require("../models/Archive");
const SalesTransaction = require("../models/SalesTransaction");
const ARCHIVE_CATEGORY_ENUM = /* @__PURE__ */ new Set([
  "Tops",
  "Bottoms",
  "Dresses",
  "Makeup",
  "Accessories",
  "Essentials",
  "Shoes",
  "Head Wear",
  "Foods",
  "Others"
]);
const mapProductCategoryToArchiveCategory = (category) => {
  const c = String(category || "").trim();
  if (ARCHIVE_CATEGORY_ENUM.has(c)) return c;
  return "Others";
};
exports.getAllProducts = async (req, res) => {
  try {
    const page = parseInt(req.query.page) || 1;
    let limit = parseInt(req.query.limit);
    let skip = 0;
    const isMinimal = req.query.fields === "minimal";
    const projection = isMinimal ? "-stockHistory -itemImage" : "-stockHistory";
    let query = Product.find({ isArchived: { $ne: true } }, projection).sort({ dateAdded: -1 });
    if (limit) {
      skip = (page - 1) * limit;
      query = query.skip(skip).limit(limit);
    }
    const [products, totalCount] = await Promise.all([
```

```
      query.lean(),
      Product.countDocuments()
    ]);
    const formattedProducts = products.map((product) => {
      let currentStock = product.currentStock || 0;
      if (product.sizes && typeof product.sizes === "object" && Object.keys(product.sizes).length > 0) {
        currentStock = Object.values(product.sizes).reduce((sum, sizeData) => {
          if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
            return sum + (sizeData.quantity || 0);
          }
          return sum + (typeof sizeData === "number" ? sizeData : 0);
        }, 0);
      }
      return {
        ...product,
        _id: product._id.toString(),
        variant: product.variant || "",
        size: product.size || "",
        brandName: product.brandName || "",
        costPrice: product.costPrice || 0,
        currentStock,
        reorderNumber: product.reorderNumber || 0,
        expirationThresholdDays: product.expirationThresholdDays || 0,
        supplierName: product.supplierName || "",
        supplierContact: product.supplierContact || "",
        displayInTerminal: product.displayInTerminal !== void 0 ? product.displayInTerminal : true,
        terminalStatus: product.displayInTerminal !== false ? "shown" : "not shown"
      };
    });
    res.json({
      success: true,
      count: formattedProducts.length,
      total: totalCount,
      totalPages: limit ? Math.ceil(totalCount / limit) : 1,
      currentPage: page,
      data: formattedProducts
    });
  } catch (error) {
    console.error("Error fetching products:", error);
    res.status(500).json({
      success: false,
      message: "Error fetching products",
      error: error.message
    });
  }
};
exports.getProductById = async (req, res) => {
  try {
    const product = await Product.findById(req.params.id).lean();
    if (!product) {
      return res.status(404).json({
        success: false,
        message: "Product not found"
      });
    }
    let currentStock = product.currentStock || 0;
    if (product.sizes && typeof product.sizes === "object" && Object.keys(product.sizes).length > 0) {
      currentStock = Object.values(product.sizes).reduce((sum, sizeData) => {
        if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
          return sum + (sizeData.quantity || 0);
        }
        return sum + (typeof sizeData === "number" ? sizeData : 0);
      }, 0);
    }
    const productResponse = {
      ...product,
      currentStock,
      expirationThresholdDays: product.expirationThresholdDays || 0,
      displayInTerminal: product.displayInTerminal !== void 0 ? product.displayInTerminal : true,
      terminalStatus: product.displayInTerminal !== false ? "shown" : "not shown"
    };
    res.json({
      success: true,
      data: productResponse
    });
  } catch (error) {
    res.status(500).json({
```

```
      success: false,
      message: "Error fetching product",
      error: error.message
    });
  }
};
exports.createProduct = async (req, res) => {
  try {
    const productData = { ...req.body };
    const creatorName = req.body?.handledBy || req.body?.performedByName || req.user?.name ||
(req.user?.firstName || req.user?.lastName ? `${req.user?.firstName || ""} ${req.user?.lastName ||
""}`.trim() : "") || "System";
    const creatorId = req.body?.handledById || req.body?.performedById || req.user?._id || req.user?.id || "";
    productData.expirationThresholdDays = productData.expirationDate ? Math.max(0,
parseInt(productData.expirationThresholdDays, 10) || 30) : 0;
    let openingStockTotal = 0;
    if (!productData.sizes && productData.selectedSizes) {
      if (productData.selectedSizes.length > 0) {
        productData.sizes = {};
        productData.selectedSizes.forEach((size) => {
          const hasDifferentPricesPerVariant = productData.differentPricesPerVariant?.[size];
          if (hasDifferentPricesPerVariant && productData.variantQuantities?.[size]) {
            const variants = {};
            const variantQtys = productData.variantQuantities[size] || {};
            const variantPrices = productData.variantPrices?.[size] || {};
            const variantCostPrices = productData.variantCostPrices?.[size] || {};
            Object.keys(variantQtys).forEach((variantName) => {
              const qty = parseInt(variantQtys[variantName]) || 0;
              if (qty > 0 || variantPrices[variantName] > 0) {
                variants[variantName] = {
                  quantity: qty,
                  price: parseFloat(variantPrices[variantName]) || productData.itemPrice || 0,
                  costPrice: parseFloat(variantCostPrices[variantName]) || productData.costPrice || 0
                };
              }
            });
            const totalQty = Object.values(variants).reduce((sum, v) => sum + (v.quantity || 0), 0);
            productData.sizes[size] = {
              quantity: totalQty,
              variants,
              hasDifferentPricesPerVariant: true
            };
          } else if (productData.differentPricesPerSize || productData.differentVariantsPerSize) {
            const sizeData = {
              quantity: productData.sizeQuantities[size] || 0,
              price: productData.differentPricesPerSize ? productData.sizePrices?.[size] ||
productData.itemPrice : productData.itemPrice,
              variant: productData.differentVariantsPerSize ? productData.sizeVariants?.[size] ||
productData.variant : productData.variant
            };
            if (productData.differentPricesPerSize && productData.sizeCostPrices?.[size]) {
              sizeData.costPrice = productData.sizeCostPrices[size];
            }
            if (productData.variantQuantities?.[size]) {
              const variants = {};
              const variantQtys = productData.variantQuantities[size];
              Object.keys(variantQtys).forEach((variantName) => {
                const qty = parseInt(variantQtys[variantName]) || 0;
                if (qty > 0) {
                  variants[variantName] = {
                    quantity: qty,
                    price: sizeData.price,
                    costPrice: sizeData.costPrice || productData.costPrice || 0
                  };
                }
              });
              if (Object.keys(variants).length > 0) {
                sizeData.variants = variants;
                sizeData.quantity = Object.values(variants).reduce((sum, v) => sum + (v.quantity || 0), 0);
              }
            }
            productData.sizes[size] = sizeData;
          } else {
            if (productData.variantQuantities?.[size]) {
              const variants = {};
              const variantQtys = productData.variantQuantities[size];
              Object.keys(variantQtys).forEach((variantName) => {
```

```
            const qty = parseInt(variantQtys[variantName]) || 0;
            if (qty > 0) {
              variants[variantName] = {
                quantity: qty,
                price: productData.itemPrice || 0,
                costPrice: productData.costPrice || 0
              };
            }
          });
          const totalQtySimple = Object.values(variants).reduce(
            (sum, v) => sum + (v.quantity || 0),
            0
          );
          productData.sizes[size] = {
            quantity: totalQtySimple,
            variants
          };
        } else {
          productData.sizes[size] = productData.sizeQuantities?.[size] || 0;
        }
      }
    });
  }
}
if (productData.sizes && typeof productData.sizes === "object") {
  const dr = productData.dateReceived;
  const openingTs = dr && String(dr).trim() ? (/* @__PURE__ */ new Date(
    `${String(dr).slice(0, 10)}T12:00:00`
  )).toISOString() : (/* @__PURE__ */ new Date()).toISOString();
  const openingBatchCode = (() => {
    if (productData.batchCode && String(productData.batchCode).trim()) {
      return String(productData.batchCode).trim();
    }
    const d = /* @__PURE__ */ new Date();
    const y = d.getFullYear();
    const mo = String(d.getMonth() + 1).padStart(2, "0");
    const day = String(d.getDate()).padStart(2, "0");
    const h = String(d.getHours()).padStart(2, "0");
    const min = String(d.getMinutes()).padStart(2, "0");
    return `B${y}${mo}${day}-${h}${min}`;
  })();
  const openingExp = productData.expirationDate ? String(productData.expirationDate).slice(0, 10) : "";
  const attachOpeningBatches = (sizes) => {
    for (const sizeKey of Object.keys(sizes)) {
      const sd = sizes[sizeKey];
      if (typeof sd === "number") continue;
      if (!sd || typeof sd !== "object") continue;
      if (sd.variants && typeof sd.variants === "object") {
        for (const vk of Object.keys(sd.variants)) {
          const v = sd.variants[vk];
          if (typeof v !== "object" || v === null) continue;
          const q = parseInt(v.quantity, 10) || 0;
          if (q <= 0) continue;
          if (Array.isArray(v.batches) && v.batches.length > 0) continue;
          sd.variants[vk] = {
            ...v,
            batches: [
              {
                qty: q,
                price: parseFloat(v.price) || 0,
                costPrice: parseFloat(v.costPrice) || 0,
                createdAt: openingTs,
                ...openingBatchCode ? { batchCode: openingBatchCode } : {},
                ...openingExp ? { expirationDate: openingExp } : {}
              }
            ]
          };
        }
      } else if (sd.quantity !== void 0 && !sd.variants) {
        const q = parseInt(sd.quantity, 10) || 0;
        if (q > 0 && (!Array.isArray(sd.batches) || sd.batches.length === 0)) {
          sizes[sizeKey] = {
            ...sd,
            batches: [
              {
                qty: q,
                price: parseFloat(sd.price) || 0,
```

```
                costPrice: parseFloat(sd.costPrice) || 0,
                createdAt: openingTs,
                ...openingBatchCode ? { batchCode: openingBatchCode } : {},
                ...openingExp ? { expirationDate: openingExp } : {}
              }
            ]
          };
        }
      }
    }
  };
  attachOpeningBatches(productData.sizes);
  openingStockTotal = Object.values(productData.sizes).reduce((sum, sizeData) => {
    if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
      return sum + (parseInt(sizeData.quantity, 10) || 0);
    }
    return sum + (typeof sizeData === "number" ? parseInt(sizeData, 10) || 0 : 0);
  }, 0);
} else {
  openingStockTotal = parseInt(productData.currentStock, 10) || 0;
}
productData.currentStock = openingStockTotal;
if (openingStockTotal <= 0 && productData.displayInTerminal === void 0) {
  productData.displayInTerminal = false;
}
delete productData.selectedSizes;
delete productData.sizeQuantities;
delete productData.sizePrices;
delete productData.sizeCostPrices;
delete productData.sizeVariants;
delete productData.differentPricesPerSize;
delete productData.differentVariantsPerSize;
delete productData.differentPricesPerVariant;
delete productData.variantQuantities;
delete productData.variantPrices;
delete productData.variantCostPrices;
delete productData.sizeMultiVariants;
delete productData.multipleVariantsPerSize;
delete productData.dateReceived;
delete productData.handledBy;
delete productData.handledById;
delete productData.performedByName;
delete productData.performedById;
if (!productData.dateAdded) {
  productData.dateAdded = Date.now();
}
if (!productData.sku) {
  const category = productData.category || "Foods";
  const variant = productData.variant || "";
  const categoryCode = category.substring(0, 3).toUpperCase();
  const variantCode = variant ? `-${variant.substring(0, 3).toUpperCase()}` : "";
  const generateRandomCode = () => {
    const chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
    let result = "";
    for (let i = 0; i < 6; i++) {
      result += chars.charAt(Math.floor(Math.random() * chars.length));
    }
    return result;
  };
  let attempts = 0;
  let uniqueSku = "";
  while (attempts < 10) {
    const randomCode = generateRandomCode();
    uniqueSku = `${categoryCode}-${randomCode}${variantCode}`;
    const existing = await Product.findOne({ sku: uniqueSku });
    if (!existing) break;
    attempts++;
  }
  productData.sku = uniqueSku;
}
const product = await Product.create(productData);
const productResponse = {
  ...product.toObject(),
  displayInTerminal: product.displayInTerminal !== void 0 ? product.displayInTerminal : true,
  terminalStatus: product.displayInTerminal !== false ? "shown" : "not shown"
};
if (openingStockTotal > 0) {
```

```
      let openingSizeQuantities = null;
      if (productData.sizes && typeof productData.sizes === "object") {
        const perSize = {};
        Object.entries(productData.sizes).forEach(([size, sizeData]) => {
          let qty = 0;
          if (typeof sizeData === "number") {
            qty = parseInt(sizeData, 10) || 0;
          } else if (sizeData && typeof sizeData === "object" && sizeData.quantity !== void 0) {
            qty = parseInt(sizeData.quantity, 10) || 0;
          }
          if (qty > 0) {
            perSize[size] = qty;
          }
        });
        if (Object.keys(perSize).length > 0) {
          openingSizeQuantities = perSize;
        }
      }
      await logStockMovement(
        product,
        0,
        openingStockTotal,
        "Stock-In",
        "Initial Stock",
        creatorName,
        creatorId,
        openingSizeQuantities
      );
    }
    res.status(201).json({
      success: true,
      message: "Product created successfully",
      data: productResponse
    });
  } catch (error) {
    if (error.code === 11e3) {
      return res.status(400).json({
        success: false,
        message: "Product with this SKU already exists"
      });
    }
    res.status(400).json({
      success: false,
      message: "Error creating product",
      error: error.message
    });
  }
};
const logStockMovement = async (product, stockBefore, stockAfter, type, reason, handledBy, handledById,
sizeQuantities = null) => {
  try {
    const quantity = Math.abs(stockAfter - stockBefore);
    if (quantity === 0) return;
    await StockMovement.create({
      productId: product._id,
      sku: product.sku,
      itemName: product.itemName,
      itemImage: product.itemImage || "",
      category: product.category,
      brandName: product.brandName || "",
      type: type || (stockAfter > stockBefore ? "Stock-In" : "Stock-Out"),
      quantity,
      stockBefore,
      stockAfter,
      reason: reason || "Other",
      handledBy: handledBy || "System",
      handledById: handledById || "",
      notes: "",
      sizeQuantities: sizeQuantities || null
    });
  } catch (error) {
    console.error("Error logging stock movement:", error);
  }
};
const hasZeroStock = (productData) => {
  if (productData.sizes && typeof productData.sizes === "object" && Object.keys(productData.sizes).length > 0)
{
```

```
    return Object.values(productData.sizes).every((sizeData) => {
      if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
        return (sizeData.quantity || 0) === 0;
      }
      return (typeof sizeData === "number" ? sizeData : 0) === 0;
    });
  }
  return (productData.currentStock || 0) === 0;
};
const nowIso = () => (/* @__PURE__ */ new Date()).toISOString();
const safeNum = (v, fallback = 0) => {
  const n = typeof v === "number" ? v : parseFloat(v);
  return Number.isFinite(n) ? n : fallback;
};
const toSizeObject = (sizeData, fallbackPrice, fallbackCostPrice) => {
  if (typeof sizeData === "object" && sizeData !== null) {
    const qty2 = safeNum(sizeData.quantity, 0);
    return {
      ...sizeData,
      quantity: qty2,
      price: safeNum(sizeData.price, safeNum(fallbackPrice, 0)),
      costPrice: safeNum(sizeData.costPrice, safeNum(fallbackCostPrice, 0))
    };
  }
  const qty = typeof sizeData === "number" ? sizeData : 0;
  return {
    quantity: qty,
    price: safeNum(fallbackPrice, 0),
    costPrice: safeNum(fallbackCostPrice, 0)
  };
};
const ensureBatches = (obj, fallbackPrice, fallbackCostPrice) => {
  const next = typeof obj === "object" && obj !== null ? { ...obj } : { ...toSizeObject(obj, fallbackPrice,
fallbackCostPrice) };
  delete next.expirationDate;
  delete next.batchCode;
  next.quantity = safeNum(next.quantity, 0);
  next.price = safeNum(next.price, safeNum(fallbackPrice, 0));
  next.costPrice = safeNum(next.costPrice, safeNum(fallbackCostPrice, 0));
  if (!Array.isArray(next.batches) || next.batches.length === 0) {
    next.batches = next.quantity > 0 ? [{ qty: next.quantity, price: next.price, costPrice: next.costPrice,
createdAt: nowIso() }] : [];
  }
  return next;
};
const addBatch = (batches, addQty, price, costPrice, meta = {}) => {
  const qty = safeNum(addQty, 0);
  const next = Array.isArray(batches) ? batches.map((b) => ({ ...b })) : [];
  if (qty > 0) {
    next.push({
      qty,
      price: safeNum(price, 0),
      costPrice: safeNum(costPrice, 0),
      createdAt: nowIso(),
      ...meta.batchCode ? { batchCode: String(meta.batchCode) } : {},
      ...meta.expirationDate ? { expirationDate: String(meta.expirationDate) } : {}
    });
  }
  return next;
};
const addToExistingBatchSlot = (batches, slotIndex, addQty, meta, padPrice, padCost) => {
  const next = Array.isArray(batches) ? batches.map((b) => ({ ...b })) : [];
  const S = Math.floor(safeNum(slotIndex, -1));
  const q = safeNum(addQty, 0);
  if (S < 0 || q <= 0) return next;
  while (next.length <= S) {
    next.push({
      qty: 0,
      price: safeNum(padPrice, 0),
      costPrice: safeNum(padCost, 0),
      createdAt: nowIso(),
      batchSlotPadding: true
    });
  }
  const entry = { ...next[S] };
  entry.qty = safeNum(entry.qty, 0) + q;
  if (entry.batchSlotPadding && entry.qty > 0) {
```

```
    delete entry.batchSlotPadding;
  }
  if (meta.expirationDate) {
    entry.expirationDate = String(meta.expirationDate);
  }
  next[S] = entry;
  return next;
};
const padBatchesToLengthBeforeAdd = (batches, targetLenBeforePush, price, costPrice) => {
  const next = Array.isArray(batches) ? batches.map((b) => ({ ...b })) : [];
  while (next.length < targetLenBeforePush) {
    next.push({
      qty: 0,
      price: safeNum(price, 0),
      costPrice: safeNum(costPrice, 0),
      createdAt: nowIso(),
      batchSlotPadding: true
    });
  }
  return next;
};
const consumeBatches = (batches, removeQty) => {
  let remaining = safeNum(removeQty, 0);
  const next = Array.isArray(batches) ? batches.map((b) => ({ ...b })) : [];
  for (let i = 0; i < next.length && remaining > 0; i++) {
    const take = Math.min(safeNum(next[i].qty, 0), remaining);
    next[i].qty = safeNum(next[i].qty, 0) - take;
    remaining -= take;
  }
  return next;
};
const sumBatchesQty = (batches) => (Array.isArray(batches) ? batches : []).reduce((sum, b) => sum +
safeNum(b.qty, 0), 0);
exports.stockInProduct = async (req, res) => {
  try {
    const productId = req.params.id;
    const stockData = req.body || {};
    const product = await Product.findById(productId);
    if (!product) {
      return res.status(404).json({ success: false, message: "Product not found" });
    }
    const stockBefore = product.currentStock || 0;
    const handledBy = stockData.handledBy || "System";
    const handledById = stockData.handledById || "";
    const reason = stockData.reason || "Restock";
    const batchMeta = {
      batchCode: stockData.batchCode ? String(stockData.batchCode).trim() : "",
      expirationDate: stockData.expirationDate ? String(stockData.expirationDate) : ""
    };
    const hasVariantPayload = !!(stockData.hasVariants && stockData.variantQuantities && typeof
stockData.variantQuantities === "object" && Object.keys(stockData.variantQuantities).length > 0);
    const rawTargetSlot = stockData.targetBatchSlotIndex;
    const parsedTargetSlot = Number.isFinite(Number(rawTargetSlot)) ? Math.floor(Number(rawTargetSlot)) : -1;
    const shouldAddToExistingSlot = parsedTargetSlot >= 0;
    if ((!product.sizes || typeof product.sizes !== "object" || Object.keys(product.sizes).length === 0)
&& !hasVariantPayload) {
      const qty = safeNum(stockData.quantity, 0);
      if (qty <= 0) {
        return res.status(400).json({ success: false, message: "Invalid quantity" });
      }
      const hadZeroStockBefore2 = hasZeroStock(product.toObject());
      const hasZeroStockNow2 = hasZeroStock({ ...product.toObject(), currentStock: stockBefore + qty });
      const updatePayload2 = {
        $set: {
          currentStock: stockBefore + qty,
          lastUpdated: Date.now()
        }
      };
      if (hasZeroStockNow2 && stockData.displayInTerminal === void 0) {
        updatePayload2.$set.displayInTerminal = false;
      } else if (!hasZeroStockNow2 && hadZeroStockBefore2 && stockData.displayInTerminal === void 0) {
        updatePayload2.$set.displayInTerminal = true;
      }
      const updatedProduct2 = await Product.findByIdAndUpdate(productId, updatePayload2, { new: true,
runValidators: true });
      await logStockMovement(updatedProduct2, stockBefore, updatedProduct2.currentStock, "Stock-In", reason,
handledBy, handledById, null);
```

```
      return res.json({ success: true, message: "Stock added successfully", data:
updatedProduct2.toObject() });
    }
    const updatedSizes = { ...product.sizes || {} };
    const selectedSizes = Array.isArray(stockData.selectedSizes) ? stockData.selectedSizes : [];
    if (selectedSizes.length === 0) {
      return res.status(400).json({ success: false, message: "No sizes selected" });
    }
    const hasVariants = !!(stockData.hasVariants && stockData.variantQuantities);
    const sizeQuantitiesAdded = {};
    if (hasVariants) {
      let globalMaxBatchLenBeforeAdd = 0;
      if (!shouldAddToExistingSlot) {
        selectedSizes.forEach((size) => {
          const currentSizeData = ensureBatches(
            toSizeObject(updatedSizes[size] || {}, product.itemPrice || 0, product.costPrice || 0),
            product.itemPrice || 0,
            product.costPrice || 0
          );
          const currentVariants = currentSizeData.variants && typeof currentSizeData.variants === "object" ?
currentSizeData.variants : {};
          const addVariantQtys = stockData.variantQuantities?.[size] || {};
          Object.entries(addVariantQtys).forEach(([variant, addQty]) => {
            if (safeNum(addQty, 0) <= 0) return;
            const fallbackExistingPrice = safeNum(currentSizeData.variantPrices?.[variant],
safeNum(currentSizeData.price, product.itemPrice || 0));
            const fallbackExistingCost = safeNum(currentSizeData.variantCostPrices?.[variant],
safeNum(currentSizeData.costPrice, product.costPrice || 0));
            const normalized = ensureBatches(currentVariants[variant] || {}, fallbackExistingPrice,
fallbackExistingCost);
            globalMaxBatchLenBeforeAdd = Math.max(globalMaxBatchLenBeforeAdd, normalized.batches.length);
          });
        });
      }
      selectedSizes.forEach((size) => {
        const currentSizeData = ensureBatches(
          toSizeObject(updatedSizes[size] || {}, product.itemPrice || 0, product.costPrice || 0),
          product.itemPrice || 0,
          product.costPrice || 0
        );
        const currentVariants = currentSizeData.variants && typeof currentSizeData.variants === "object" ?
currentSizeData.variants : {};
        const addVariantQtys = stockData.variantQuantities?.[size] || {};
        const newVariants = { ...currentVariants };
        Object.entries(addVariantQtys).forEach(([variant, addQty]) => {
          const qty = safeNum(addQty, 0);
          if (qty <= 0) return;
          const fallbackExistingPrice = safeNum(currentSizeData.variantPrices?.[variant],
safeNum(currentSizeData.price, product.itemPrice || 0));
          const fallbackExistingCost = safeNum(currentSizeData.variantCostPrices?.[variant],
safeNum(currentSizeData.costPrice, product.costPrice || 0));
          const normalizedVariant = ensureBatches(newVariants[variant] || {}, fallbackExistingPrice,
fallbackExistingCost);
          let incomingPrice = fallbackExistingPrice;
          let incomingCost = fallbackExistingCost;
          if (stockData.stockVariantPrices?.[size]?.[variant]) {
            incomingPrice = safeNum(stockData.stockVariantPrices[size][variant].price, incomingPrice);
            incomingCost = safeNum(stockData.stockVariantPrices[size][variant].costPrice, incomingCost);
          } else if (stockData.newVariantPrices?.[variant]) {
            incomingPrice = safeNum(stockData.newVariantPrices[variant].price, incomingPrice);
            incomingCost = safeNum(stockData.newVariantPrices[variant].costPrice, incomingCost);
          } else if (stockData.newSizePrices?.[size]) {
            incomingPrice = safeNum(stockData.newSizePrices[size].price, incomingPrice);
            incomingCost = safeNum(stockData.newSizePrices[size].costPrice, incomingCost);
          }
          let nextBatches;
          if (shouldAddToExistingSlot) {
            nextBatches = addToExistingBatchSlot(
              normalizedVariant.batches,
              parsedTargetSlot,
              qty,
              batchMeta,
              incomingPrice,
              incomingCost
            );
          } else {
            const padLen = globalMaxBatchLenBeforeAdd;
```

```
            const alignedBatches = padBatchesToLengthBeforeAdd(
              normalizedVariant.batches,
              padLen,
              incomingPrice,
              incomingCost
            );
            nextBatches = addBatch(
              alignedBatches,
              qty,
              incomingPrice,
              incomingCost,
              batchMeta
            );
          }
          newVariants[variant] = {
            ...normalizedVariant,
            batches: nextBatches,
            quantity: sumBatchesQty(nextBatches),
            price: incomingPrice,
            costPrice: incomingCost
          };
        });
        const newTotalQty = Object.values(newVariants).reduce((sum, v) => {
          if (typeof v === "number") return sum + safeNum(v, 0);
          if (typeof v === "object" && v !== null) return sum + safeNum(v.quantity, 0);
          return sum;
        }, 0);
        updatedSizes[size] = {
          ...currentSizeData,
          variants: newVariants,
          quantity: newTotalQty,
          hasDifferentPricesPerVariant: stockData.diffPricesPerVariant?.[size] ? true :
currentSizeData.hasDifferentPricesPerVariant
        };
        if (stockData.newSizePrices?.[size]) {
          updatedSizes[size].price = safeNum(stockData.newSizePrices[size].price, product.itemPrice || 0);
          updatedSizes[size].costPrice = safeNum(stockData.newSizePrices[size].costPrice, product.costPrice
|| 0);
        }
        const totalForSize = Object.values(addVariantQtys).reduce((s, q) => s + safeNum(q, 0), 0);
        if (totalForSize > 0) sizeQuantitiesAdded[size] = totalForSize;
      });
    } else {
      let maxBatchLenBeforeAddNoVar = 0;
      selectedSizes.forEach((size) => {
        const addQty = safeNum(stockData.sizes?.[size], 0);
        if (addQty <= 0) return;
        const cur = ensureBatches(
          toSizeObject(updatedSizes[size] || {}, product.itemPrice || 0, product.costPrice || 0),
          product.itemPrice || 0,
          product.costPrice || 0
        );
        maxBatchLenBeforeAddNoVar = Math.max(maxBatchLenBeforeAddNoVar, cur.batches.length);
      });
      selectedSizes.forEach((size) => {
        const addQty = safeNum(stockData.sizes?.[size], 0);
        if (addQty <= 0) return;
        const currentSizeData = ensureBatches(
          toSizeObject(updatedSizes[size] || {}, product.itemPrice || 0, product.costPrice || 0),
          product.itemPrice || 0,
          product.costPrice || 0
        );
        let incomingPrice = currentSizeData.price;
        let incomingCost = currentSizeData.costPrice;
        if (stockData.newSizePrices?.[size]) {
          incomingPrice = safeNum(stockData.newSizePrices[size].price, incomingPrice);
          incomingCost = safeNum(stockData.newSizePrices[size].costPrice, incomingCost);
        }
        let nextBatches;
        if (shouldAddToExistingSlot) {
          nextBatches = addToExistingBatchSlot(
            currentSizeData.batches,
            parsedTargetSlot,
            addQty,
            batchMeta,
            incomingPrice,
            incomingCost
```

```
          );
        } else {
          const padLenNoVar = maxBatchLenBeforeAddNoVar;
          const alignedBatches = padBatchesToLengthBeforeAdd(
            currentSizeData.batches,
            padLenNoVar,
            incomingPrice,
            incomingCost
          );
          nextBatches = addBatch(
            alignedBatches,
            addQty,
            incomingPrice,
            incomingCost,
            batchMeta
          );
        }
        const nextQty = sumBatchesQty(nextBatches);
        updatedSizes[size] = {
          ...currentSizeData,
          batches: nextBatches,
          quantity: nextQty,
          price: incomingPrice,
          costPrice: incomingCost
        };
        sizeQuantitiesAdded[size] = addQty;
      });
    }
    const totalStock = Object.values(updatedSizes).reduce((sum, sizeData) => {
      if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
        return sum + safeNum(sizeData.quantity, 0);
      }
      return sum + (typeof sizeData === "number" ? safeNum(sizeData, 0) : 0);
    }, 0);
    const hadZeroStockBefore = hasZeroStock(product.toObject());
    const hasZeroStockNow = hasZeroStock({ ...product.toObject(), sizes: updatedSizes, currentStock:
totalStock });
    const updatePayload = {
      $set: {
        sizes: updatedSizes,
        currentStock: totalStock,
        lastUpdated: Date.now()
      }
    };
    if (hasZeroStockNow && stockData.displayInTerminal === void 0) {
      updatePayload.$set.displayInTerminal = false;
    } else if (!hasZeroStockNow && hadZeroStockBefore && stockData.displayInTerminal === void 0) {
      updatePayload.$set.displayInTerminal = true;
    }
    const updatedProduct = await Product.findByIdAndUpdate(productId, updatePayload, { new: true,
runValidators: true });
    await logStockMovement(
      updatedProduct,
      stockBefore,
      updatedProduct.currentStock,
      "Stock-In",
      reason,
      handledBy,
      handledById,
      Object.keys(sizeQuantitiesAdded).length > 0 ? sizeQuantitiesAdded : null
    );
    res.json({ success: true, message: "Stock added successfully", data: updatedProduct.toObject() });
  } catch (error) {
    console.error("Error stock-in:", error);
    if (error.errors) {
      Object.keys(error.errors).forEach((key) => console.error("Validation error:",
error.errors[key].message));
    }
    res.status(400).json({ success: false, message: "Error stocking in", error: error.message, stack:
error.stack });
  }
};
exports.stockOutProduct = async (req, res) => {
  try {
    const productId = req.params.id;
    const stockData = req.body || {};
    const product = await Product.findById(productId);
```

```
    if (!product) {
      return res.status(404).json({ success: false, message: "Product not found" });
    }
    const stockBefore = product.currentStock || 0;
    const handledBy = stockData.handledBy || "System";
    const handledById = stockData.handledById || "";
    const reason = stockData.reason || "Sold";
    const movementType = reason === "Damaged" || reason === "Lost" || reason === "Expired" ? "Pull-Out" :
"Stock-Out";
    if (!product.sizes || typeof product.sizes !== "object" || Object.keys(product.sizes).length === 0) {
      const qty = safeNum(stockData.quantity, 0);
      if (qty <= 0) return res.status(400).json({ success: false, message: "Invalid quantity" });
      if (qty > stockBefore) return res.status(400).json({ success: false, message: "Insufficient stock" });
      const hadZeroStockBefore2 = hasZeroStock(product.toObject());
      const newStock = Math.max(0, stockBefore - qty);
      const hasZeroStockNow2 = hasZeroStock({ ...product.toObject(), currentStock: newStock });
      const updatePayload2 = {
        $set: {
          currentStock: newStock,
          lastUpdated: Date.now()
        }
      };
      if (hasZeroStockNow2 && stockData.displayInTerminal === void 0) {
        updatePayload2.$set.displayInTerminal = false;
      } else if (!hasZeroStockNow2 && hadZeroStockBefore2 && stockData.displayInTerminal === void 0) {
        updatePayload2.$set.displayInTerminal = true;
      }
      const updatedProduct2 = await Product.findByIdAndUpdate(productId, updatePayload2, { new: true,
runValidators: true });
      await logStockMovement(updatedProduct2, stockBefore, updatedProduct2.currentStock, movementType, reason,
handledBy, handledById, null);
      return res.json({ success: true, message: "Stock removed successfully", data:
updatedProduct2.toObject() });
    }
    const updatedSizes = { ...product.sizes || {} };
    const selectedSizes = Array.isArray(stockData.selectedSizes) ? stockData.selectedSizes : [];
    if (selectedSizes.length === 0) {
      return res.status(400).json({ success: false, message: "No sizes selected" });
    }
    const hasVariants = !!(stockData.hasVariants && stockData.variantQuantities);
    const sizeQuantitiesRemoved = {};
    let totalQuantityRemoved = 0;
    if (hasVariants) {
      selectedSizes.forEach((size) => {
        const currentSizeData = toSizeObject(updatedSizes[size] || {}, product.itemPrice || 0,
product.costPrice || 0);
        const currentVariants = currentSizeData.variants && typeof currentSizeData.variants === "object" ?
currentSizeData.variants : {};
        const removeVariantQtys = stockData.variantQuantities?.[size] || {};
        const newVariants = { ...currentVariants };
        Object.entries(removeVariantQtys).forEach(([variant, removeQty]) => {
          const qty = safeNum(removeQty, 0);
          if (qty <= 0 || newVariants[variant] === void 0) return;
          const fallbackExistingPrice = safeNum(currentSizeData.variantPrices?.[variant],
safeNum(currentSizeData.price, product.itemPrice || 0));
          const fallbackExistingCost = safeNum(currentSizeData.variantCostPrices?.[variant],
safeNum(currentSizeData.costPrice, product.costPrice || 0));
          const normalizedVariant = ensureBatches(newVariants[variant] || {}, fallbackExistingPrice,
fallbackExistingCost);
          const nextBatches = consumeBatches(normalizedVariant.batches, qty);
          newVariants[variant] = {
            ...normalizedVariant,
            batches: nextBatches,
            quantity: sumBatchesQty(nextBatches)
          };
        });
        const newTotalQty = Object.values(newVariants).reduce((sum, v) => {
          if (typeof v === "number") return sum + safeNum(v, 0);
          if (typeof v === "object" && v !== null) return sum + safeNum(v.quantity, 0);
          return sum;
        }, 0);
        updatedSizes[size] = {
          ...currentSizeData,
          variants: newVariants,
          quantity: newTotalQty
        };
        const totalForSize = Object.values(removeVariantQtys).reduce((s, q) => s + safeNum(q, 0), 0);
```

```
        if (totalForSize > 0) {
          sizeQuantitiesRemoved[size] = totalForSize;
          totalQuantityRemoved += totalForSize;
        }
      });
    } else {
      selectedSizes.forEach((size) => {
        const removeQty = safeNum(stockData.sizes?.[size], 0);
        if (removeQty <= 0) return;
        const currentSizeData = ensureBatches(
          toSizeObject(updatedSizes[size] || {}, product.itemPrice || 0, product.costPrice || 0),
          product.itemPrice || 0,
          product.costPrice || 0
        );
        const nextBatches = consumeBatches(currentSizeData.batches, removeQty);
        const newQty = sumBatchesQty(nextBatches);
        updatedSizes[size] = {
          ...currentSizeData,
          batches: nextBatches,
          quantity: newQty
        };
        sizeQuantitiesRemoved[size] = removeQty;
        totalQuantityRemoved += removeQty;
      });
    }
    const totalStock = Object.values(updatedSizes).reduce((sum, sizeData) => {
      if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
        return sum + safeNum(sizeData.quantity, 0);
      }
      return sum + (typeof sizeData === "number" ? safeNum(sizeData, 0) : 0);
    }, 0);
    const hadZeroStockBefore = hasZeroStock(product.toObject());
    const hasZeroStockNow = hasZeroStock({ ...product.toObject(), sizes: updatedSizes, currentStock:
totalStock });
    const updatePayload = {
      $set: {
        sizes: updatedSizes,
        currentStock: totalStock,
        lastUpdated: Date.now()
      }
    };
    if (hasZeroStockNow && stockData.displayInTerminal === void 0) {
      updatePayload.$set.displayInTerminal = false;
    } else if (!hasZeroStockNow && stockBefore === 0 && totalStock > 0 && stockData.displayInTerminal ===
void 0) {
      updatePayload.$set.displayInTerminal = true;
    }
    const updatedProduct = await Product.findByIdAndUpdate(productId, updatePayload, { new: true,
runValidators: true });
    await logStockMovement(
      updatedProduct,
      stockBefore,
      updatedProduct.currentStock,
      movementType,
      reason,
      handledBy,
      handledById,
      Object.keys(sizeQuantitiesRemoved).length > 0 ? sizeQuantitiesRemoved : null
    );
    res.json({ success: true, message: "Stock removed successfully", data: updatedProduct.toObject() });
  } catch (error) {
    console.error("Error stock-out:", error);
    res.status(400).json({ success: false, message: "Error stocking out", error: error.message, stack:
error.stack });
  }
};
exports.updateProduct = async (req, res) => {
  try {
    const productId = req.params.id;
    const updateData = { ...req.body };
    if (Object.prototype.hasOwnProperty.call(updateData, "expirationDate") ||
Object.prototype.hasOwnProperty.call(updateData, "expirationThresholdDays")) {
      updateData.expirationThresholdDays = updateData.expirationDate ? Math.max(0,
parseInt(updateData.expirationThresholdDays, 10) || 30) : 0;
    }
    const stockMovementType = updateData.stockMovementType;
    const stockMovementReason = updateData.stockMovementReason;
```

```
    const handledBy = updateData.handledBy;
    const handledById = updateData.handledById;
    const stockMovementSizeQuantities = updateData.stockMovementSizeQuantities;
    delete updateData.stockMovementType;
    delete updateData.stockMovementReason;
    delete updateData.handledBy;
    delete updateData.handledById;
    delete updateData.stockMovementSizeQuantities;
    if (!updateData.sizes && updateData.selectedSizes) {
      if (updateData.selectedSizes.length > 0) {
        updateData.sizes = {};
        updateData.selectedSizes.forEach((size) => {
          let sizeData = updateData.sizeQuantities?.[size] || 0;
          if (updateData.differentPricesPerSize || updateData.differentVariantsPerSize) {
            sizeData = {
              quantity: updateData.sizeQuantities?.[size] || 0,
              price: updateData.differentPricesPerSize ? updateData.sizePrices?.[size] ||
updateData.itemPrice : updateData.itemPrice,
              variant: updateData.differentVariantsPerSize ? updateData.sizeVariants?.[size] ||
updateData.variant : updateData.variant
            };
            if (updateData.differentPricesPerSize && updateData.sizeCostPrices?.[size]) {
              sizeData.costPrice = updateData.sizeCostPrices[size];
            }
          }
          updateData.sizes[size] = sizeData;
        });
      } else if (updateData.selectedSizes.length === 0) {
        updateData.sizes = null;
      }
    }
    delete updateData.selectedSizes;
    delete updateData.sizeQuantities;
    delete updateData.sizePrices;
    delete updateData.sizeVariants;
    delete updateData.differentPricesPerSize;
    delete updateData.differentVariantsPerSize;
    updateData.lastUpdated = Date.now();
    const productBefore = await Product.findById(productId);
    if (!productBefore) {
      return res.status(404).json({
        success: false,
        message: "Product not found"
      });
    }
    const stockBefore = productBefore.currentStock;
    const hadZeroStockBefore = hasZeroStock(productBefore);
    const hasZeroStockNow = hasZeroStock(updateData);
    if (hasZeroStockNow && req.body.displayInTerminal === void 0) {
      updateData.displayInTerminal = false;
    } else if (!hasZeroStockNow && hadZeroStockBefore && req.body.displayInTerminal === void 0) {
      updateData.displayInTerminal = true;
    } else if (updateData.displayInTerminal === void 0) {
      updateData.displayInTerminal = productBefore.displayInTerminal !== void 0 ?
productBefore.displayInTerminal : true;
    }
    const product = await Product.findByIdAndUpdate(productId, updateData, {
      new: true,
      runValidators: true
    });
    const stockAfter = product.currentStock;
    if (stockBefore !== stockAfter && stockMovementType && stockMovementReason && handledBy) {
      await logStockMovement(
        product,
        stockBefore,
        stockAfter,
        stockMovementType,
        stockMovementReason,
        handledBy,
        handledById,
        stockMovementSizeQuantities
      );
    }
    const productResponse = {
      ...product.toObject(),
      displayInTerminal: product.displayInTerminal !== void 0 ? product.displayInTerminal : true,
      terminalStatus: product.displayInTerminal !== false ? "shown" : "not shown"
```

```
    };
    res.json({
      success: true,
      message: "Product updated successfully",
      data: productResponse
    });
  } catch (error) {
    res.status(400).json({
      success: false,
      message: "Error updating product",
      error: error.message
    });
  }
};
exports.deleteProduct = async (req, res) => {
  try {
    const product = await Product.findByIdAndDelete(req.params.id);
    if (!product) {
      return res.status(404).json({
        success: false,
        message: "Product not found"
      });
    }
    res.json({
      success: true,
      message: "Product deleted successfully"
    });
  } catch (error) {
    console.error("Error deleting product:", error);
    res.status(500).json({
      success: false,
      message: "Error deleting product",
      error: error.message
    });
  }
};
exports.archiveProduct = async (req, res) => {
  try {
    const product = await Product.findById(req.params.id);
    if (!product) {
      return res.status(404).json({
        success: false,
        message: "Product not found"
      });
    }
    if (product.isArchived) {
      return res.json({
        success: true,
        message: "Product already archived"
      });
    }
    const qty = Math.max(
      1,
      Number(product.currentStock) >= 0 ? Number(product.currentStock) : 0
    );
    const prevDisplayInTerminal = product.displayInTerminal;
    product.isArchived = true;
    product.displayInTerminal = false;
    product.lastUpdated = Date.now();
    await product.save();
    try {
      await Archive.create({
        productId: product._id,
        itemName: product.itemName,
        sku: product.sku || "N/A",
        variant: product.variant || "",
        selectedSize: product.size || "",
        category: mapProductCategoryToArchiveCategory(product.category),
        brandName: product.brandName || "",
        itemPrice: product.itemPrice ?? 0,
        costPrice: product.costPrice ?? 0,
        quantity: qty,
        itemImage: product.itemImage || "",
        reason: "Other",
        returnReason: "",
        originalTransactionId: null,
        source: "stock-out",
```

```
        archivedBy: req.body?.archivedByName || "Inventory",
        archivedById: req.body?.archivedById || "",
        notes: "Product archived from inventory"
      });
    } catch (archiveErr) {
      product.isArchived = false;
      product.displayInTerminal = prevDisplayInTerminal;
      await product.save();
      throw archiveErr;
    }
    res.json({
      success: true,
      message: "Product archived successfully"
    });
  } catch (error) {
    console.error("Error archiving product:", error);
    res.status(500).json({
      success: false,
      message: "Error archiving product",
      error: error.message
    });
  }
};
exports.getProductsByCategory = async (req, res) => {
  try {
    const products = await Product.find({ category: req.params.category }).sort({ dateAdded: -1 }).lean();
    res.json({
      success: true,
      count: products.length,
      data: products
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: "Error fetching products",
      error: error.message
    });
  }
};
const findSizeKey = (sizes = {}, size = "") => {
  const normalized = size?.toLowerCase();
  return Object.keys(sizes).find((key) => key?.toLowerCase() === normalized);
};
const findVariantKey = (variants, name) => {
  if (!name || typeof variants !== "object" || variants === null) {
    return null;
  }
  if (Object.prototype.hasOwnProperty.call(variants, name)) {
    return name;
  }
  const target = String(name).trim().toLowerCase();
  return Object.keys(variants).find(
    (k) => String(k).trim().toLowerCase() === target
  ) || null;
};
const getSizeKeys = (sizes) => {
  if (!sizes || typeof sizes !== "object") {
    return [];
  }
  return sizes instanceof Map ? Array.from(sizes.keys()) : Object.keys(sizes);
};
const getSizeQuantity = (sizeData) => {
  if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
    return sizeData.quantity;
  }
  return typeof sizeData === "number" ? sizeData : 0;
};
const getSizePrice = (sizeData) => {
  if (typeof sizeData === "object" && sizeData !== null && sizeData.price !== void 0) {
    return sizeData.price;
  }
  return null;
};
exports.updateStockAfterTransaction = async (req, res) => {
  try {
    const { items, performedByName, performedById, type, reason, linkTransactionId } = req.body;
    if (!items || !Array.isArray(items)) {
```

```
  return res.status(400).json({
    success: false,
    message: "Invalid items data"
  });
}
const isStockIn = type === "Stock-In";
const isStockOut = type === "Stock-Out" || type === "Pull-Out";
const movementType = type || "Stock-Out";
const movementReason = reason || (isStockIn ? "Returned Item" : "Sold");
const nowIso2 = () => (/* @__PURE__ */ new Date()).toISOString();
const safeNum2 = (v, fallback = 0) => {
  const n = typeof v === "number" ? v : parseFloat(v);
  return Number.isFinite(n) ? n : fallback;
};
const ensureBatches2 = (obj, fallbackPrice, fallbackCostPrice) => {
  const quantity = safeNum2(obj?.quantity, safeNum2(obj, 0));
  const price = safeNum2(obj?.price, safeNum2(fallbackPrice, 0));
  const costPrice = safeNum2(obj?.costPrice, safeNum2(fallbackCostPrice, 0));
  const next = typeof obj === "object" && obj !== null ? { ...obj } : { quantity, price, costPrice };
  delete next.expirationDate;
  delete next.batchCode;
  next.quantity = quantity;
  next.price = price;
  next.costPrice = costPrice;
  if (!Array.isArray(next.batches)) {
    next.batches = quantity > 0 ? [{ qty: quantity, price, costPrice, createdAt: nowIso2() }] : [];
  } else if (next.batches.length === 0 && quantity > 0) {
    next.batches = [{ qty: quantity, price, costPrice, createdAt: nowIso2() }];
  }
  return next;
};
const addBatch2 = (batches, addQty, price, costPrice, meta = {}) => {
  const qty = safeNum2(addQty, 0);
  const next = Array.isArray(batches) ? batches.map((b) => ({ ...b })) : [];
  if (qty > 0) {
    next.push({
      qty,
      price: safeNum2(price, 0),
      costPrice: safeNum2(costPrice, 0),
      createdAt: nowIso2(),
      ...meta.batchCode ? { batchCode: String(meta.batchCode) } : {},
      ...meta.expirationDate ? { expirationDate: String(meta.expirationDate) } : {}
    });
  }
  return next;
};
const consumeBatches2 = (batches, removeQty) => {
  let remaining = safeNum2(removeQty, 0);
  const next = Array.isArray(batches) ? batches.map((b) => ({ ...b })) : [];
  for (let i = 0; i < next.length && remaining > 0; i++) {
    const take = Math.min(safeNum2(next[i].qty, 0), remaining);
    next[i].qty = safeNum2(next[i].qty, 0) - take;
    remaining -= take;
  }
  return next;
};
const consumeBatchesWithAllocations = (batches, removeQty) => {
  let remaining = safeNum2(removeQty, 0);
  const next = Array.isArray(batches) ? batches.map((b) => ({ ...b })) : [];
  const allocations = [];
  for (let i = 0; i < next.length && remaining > 0; i++) {
    const avail = safeNum2(next[i].qty, 0);
    if (avail <= 0) continue;
    const take = Math.min(avail, remaining);
    if (take <= 0) continue;
    const createdAt = next[i].createdAt || nowIso2();
    allocations.push({
      createdAt: String(createdAt),
      qty: take,
      price: safeNum2(next[i].price, 0),
      costPrice: safeNum2(next[i].costPrice, 0)
    });
    next[i].qty = avail - take;
    remaining -= take;
  }
  return { batches: next, allocations };
};
```

```
const allocationsForReturnQty = (fullAllocations, returnQty) => {
  let rem = safeNum2(returnQty, 0);
  const out = [];
  for (const a of fullAllocations || []) {
    if (rem <= 0) break;
    const q = safeNum2(a.qty, 0);
    if (q <= 0) continue;
    const take = Math.min(q, rem);
    out.push({
      createdAt: a.createdAt,
      qty: take,
      price: safeNum2(a.price, 0),
      costPrice: safeNum2(a.costPrice, 0)
    });
    rem -= take;
  }
  return out;
};
const restoreBatchesFromAllocations = (batches, allocations) => {
  const next = Array.isArray(batches) ? batches.map((b) => ({ ...b })) : [];
  for (const alloc of allocations || []) {
    const q = safeNum2(alloc.qty, 0);
    if (q <= 0) continue;
    const target = String(alloc.createdAt || "");
    const idx = next.findIndex((b) => String(b.createdAt || "") === target);
    if (idx >= 0) {
      next[idx].qty = safeNum2(next[idx].qty, 0) + q;
    } else {
      next.push({
        qty: q,
        price: safeNum2(alloc.price, 0),
        costPrice: safeNum2(alloc.costPrice, 0),
        createdAt: alloc.createdAt || nowIso2()
      });
    }
  }
  next.sort(
    (a, b) => String(a.createdAt || "").localeCompare(String(b.createdAt || ""))
  );
  return next;
};
const restoreReturnQtyToOldestBatches = (batches, addQty, price, costPrice) => {
  const q = safeNum2(addQty, 0);
  if (q <= 0) {
    return Array.isArray(batches) ? batches.map((b) => ({ ...b })) : [];
  }
  const next = Array.isArray(batches) ? batches.map((b) => ({ ...b })) : [];
  if (next.length === 0) {
    return [
      {
        qty: q,
        price: safeNum2(price, 0),
        costPrice: safeNum2(costPrice, 0),
        createdAt: nowIso2()
      }
    ];
  }
  next[0] = {
    ...next[0],
    qty: safeNum2(next[0].qty, 0) + q,
    price: safeNum2(price, safeNum2(next[0].price, 0)),
    costPrice: safeNum2(costPrice, safeNum2(next[0].costPrice, 0))
  };
  return next;
};
const persistLineBatchAllocations = async (txId, lineIndex, allocations) => {
  if (!txId || lineIndex == null || lineIndex < 0 || !allocations?.length) {
    return;
  }
  await SalesTransaction.updateOne(
    { _id: txId },
    { $set: { [`items.${lineIndex}.batchAllocations`]: allocations } }
  );
};
const updatedProducts = [];
for (const item of items) {
  if (!item._id && !item.sku) {
```

```
        throw new Error("Item missing both _id and sku fields");
      }
      let product = null;
      if (item._id) {
        product = await Product.findById(item._id);
      }
      if (!product && item.sku) {
        product = await Product.findOne({ sku: item.sku });
      }
      if (!product) {
        throw new Error(
          `Product not found (ID: ${item._id || "N/A"}, SKU: ${item.sku || "N/A"})`
        );
      }
      const stockBefore = product.currentStock;
      const isReturnStockIn = isStockIn && (String(reason || "").includes("Returned") || movementReason ===
"Returned Item");
      let batchAllocationsForReturn = null;
      if (isReturnStockIn) {
        batchAllocationsForReturn = Array.isArray(item.batchAllocations) ? item.batchAllocations : null;
        if ((!batchAllocationsForReturn || !batchAllocationsForReturn.length) && item.originalTransactionId
&& item.originalLineIndex != null) {
          const origTx = await SalesTransaction.findById(
            item.originalTransactionId
          ).lean();
          const line = origTx?.items?.[item.originalLineIndex];
          batchAllocationsForReturn = line?.batchAllocations || null;
        }
        if (batchAllocationsForReturn?.length) {
          batchAllocationsForReturn = allocationsForReturnQty(
            batchAllocationsForReturn,
            item.quantity
          );
        }
      }
      const sizeKeys = getSizeKeys(product.sizes);
      const hasSizes = sizeKeys.length > 0;
      const rawSize = item.size || item.selectedSize || "";
      let sizeForStock = rawSize && String(rawSize).trim() ? String(rawSize).trim() : null;
      const rawVariant = item.variant || item.selectedVariation || "";
      let variantForStock = rawVariant && String(rawVariant).trim() ? String(rawVariant).trim() : null;
      if (hasSizes && !sizeForStock && isStockIn && sizeKeys.length === 1) {
        sizeForStock = sizeKeys[0];
      }
      if (hasSizes && !sizeForStock) {
        throw new Error(
          `Size is required to update stock for "${product.itemName}" (SKU: ${product.sku || "N/A"}).`
        );
      }
      if (hasSizes && sizeForStock) {
        const sizeKey = findSizeKey(product.sizes, sizeForStock);
        if (!sizeKey) {
          if (isStockIn) {
            const sizeVals = product.sizes instanceof Map ? Array.from(product.sizes.values()) :
Object.values(product.sizes);
            const hasPriceStructure = sizeVals.some(
              (s) => typeof s === "object" && s !== null && s.price !== void 0
            );
            if (hasPriceStructure) {
              product.sizes[sizeForStock] = {
                quantity: item.quantity,
                price: item.price || product.itemPrice || 0
              };
            } else {
              product.sizes[sizeForStock] = item.quantity;
            }
            product.markModified("sizes");
          } else {
            throw new Error(
              `Size ${sizeForStock} not found for product ${product.itemName}`
            );
          }
        } else {
          const currentSizeData = product.sizes.get ? product.sizes.get(sizeKey) : product.sizes[sizeKey];
          const currentQuantity = getSizeQuantity(currentSizeData);
          const currentPrice = getSizePrice(currentSizeData);
          const hasVariantBuckets = typeof currentSizeData === "object" && currentSizeData !== null &&
```

```
currentSizeData.variants && typeof currentSizeData.variants === "object" &&
Object.keys(currentSizeData.variants).length > 0;
          if (hasVariantBuckets && !variantForStock) {
            const vKeys = Object.keys(currentSizeData.variants);
            if (vKeys.length === 1) {
              variantForStock = vKeys[0];
            }
          }
          if (hasVariantBuckets && !variantForStock) {
            throw new Error(
              `Variant is required to update stock for ${product.itemName} (size ${sizeKey}).`
            );
          }
          if (variantForStock && hasVariantBuckets) {
            const variantKey = findVariantKey(
              currentSizeData.variants,
              variantForStock
            );
            if (!variantKey) {
              throw new Error(
                `Variant "${variantForStock}" not found for ${product.itemName} (size ${sizeKey}).`
              );
            }
            const variantData = currentSizeData.variants[variantKey];
            const fallbackVariantPrice = safeNum2(
              currentSizeData.variantPrices?.[variantKey],
              safeNum2(
                currentPrice,
                safeNum2(item.price, safeNum2(product.itemPrice, 0))
              )
            );
            const fallbackVariantCostPrice = safeNum2(
              currentSizeData.variantCostPrices?.[variantKey],
              safeNum2(currentSizeData.costPrice, safeNum2(product.costPrice, 0))
            );
            const normalizedVariant = ensureBatches2(
              variantData,
              fallbackVariantPrice,
              fallbackVariantCostPrice
            );
            const currentVariantQty = safeNum2(normalizedVariant.quantity, 0);
            if (isStockOut && currentVariantQty < item.quantity) {
              throw new Error(
                `Insufficient stock for ${product.itemName} (${sizeForStock}, ${variantForStock}). Available:
${currentVariantQty}, Requested: ${item.quantity}`
              );
            }
            let nextVariantBatches;
            if (isStockIn) {
              if (batchAllocationsForReturn?.length > 0) {
                nextVariantBatches = restoreBatchesFromAllocations(
                  normalizedVariant.batches,
                  batchAllocationsForReturn
                );
              } else if (isReturnStockIn) {
                nextVariantBatches = restoreReturnQtyToOldestBatches(
                  normalizedVariant.batches,
                  item.quantity,
                  safeNum2(
                    item.price,
                    safeNum2(fallbackVariantPrice, safeNum2(product.itemPrice, 0))
                  ),
                  safeNum2(
                    item.costPrice,
                    safeNum2(fallbackVariantCostPrice, safeNum2(product.costPrice, 0))
                  )
                );
              } else {
                nextVariantBatches = addBatch2(
                  normalizedVariant.batches,
                  item.quantity,
                  safeNum2(
                    item.price,
                    safeNum2(fallbackVariantPrice, safeNum2(product.itemPrice, 0))
                  ),
                  safeNum2(
                    item.costPrice,
```

```
                safeNum2(fallbackVariantCostPrice, safeNum2(product.costPrice, 0))
              ),
              {
                batchCode: item.batchCode,
                expirationDate: item.expirationDate
              }
            );
          }
        } else {
          const cons = consumeBatchesWithAllocations(
            normalizedVariant.batches,
            item.quantity
          );
          nextVariantBatches = cons.batches;
          if (linkTransactionId != null && item.lineIndex != null && cons.allocations?.length) {
            await persistLineBatchAllocations(
              linkTransactionId,
              item.lineIndex,
              cons.allocations
            );
          }
        }
        const nextVariantQty = nextVariantBatches.reduce(
          (sum, b) => sum + safeNum2(b.qty, 0),
          0
        );
        currentSizeData.variants[variantKey] = {
          ...normalizedVariant,
          batches: nextVariantBatches,
          quantity: nextVariantQty
        };
        let sizeTotal = 0;
        for (const [, varValue] of Object.entries(
          currentSizeData.variants
        )) {
          if (typeof varValue === "number") {
            sizeTotal += varValue;
          } else if (typeof varValue === "object" && varValue !== null) {
            sizeTotal += varValue.quantity || 0;
          }
        }
        currentSizeData.quantity = sizeTotal;
        if (product.sizes.set) {
          product.sizes.set(sizeKey, currentSizeData);
        } else {
          product.sizes[sizeKey] = currentSizeData;
        }
        product.markModified("sizes");
      } else {
        if (isStockOut && currentQuantity < item.quantity) {
          throw new Error(
            `Insufficient stock for ${product.itemName} (${sizeForStock}). Available: ${currentQuantity},
Requested: ${item.quantity}`
          );
        }
        const normalizedSize = ensureBatches2(
          currentSizeData,
          safeNum2(
            currentPrice,
            safeNum2(item.price, safeNum2(product.itemPrice, 0))
          ),
          safeNum2(currentSizeData?.costPrice, safeNum2(product.costPrice, 0))
        );
        let nextSizeBatches;
        if (isStockIn) {
          if (batchAllocationsForReturn?.length > 0) {
            nextSizeBatches = restoreBatchesFromAllocations(
              normalizedSize.batches,
              batchAllocationsForReturn
            );
          } else if (isReturnStockIn) {
            nextSizeBatches = restoreReturnQtyToOldestBatches(
              normalizedSize.batches,
              item.quantity,
              safeNum2(
                item.price,
                safeNum2(normalizedSize.price, safeNum2(product.itemPrice, 0))
```

```
                ),
                safeNum2(
                  item.costPrice,
                  safeNum2(
                    normalizedSize.costPrice,
                    safeNum2(product.costPrice, 0)
                  )
                )
              );
            } else {
              nextSizeBatches = addBatch2(
                normalizedSize.batches,
                item.quantity,
                safeNum2(
                  item.price,
                  safeNum2(normalizedSize.price, safeNum2(product.itemPrice, 0))
                ),
                safeNum2(
                  item.costPrice,
                  safeNum2(
                    normalizedSize.costPrice,
                    safeNum2(product.costPrice, 0)
                  )
                ),
                { batchCode: item.batchCode, expirationDate: item.expirationDate }
              );
            }
          } else {
            const cons = consumeBatchesWithAllocations(
              normalizedSize.batches,
              item.quantity
            );
            nextSizeBatches = cons.batches;
            if (linkTransactionId != null && item.lineIndex != null && cons.allocations?.length) {
              await persistLineBatchAllocations(
                linkTransactionId,
                item.lineIndex,
                cons.allocations
              );
            }
          }
          const newQuantity = nextSizeBatches.reduce(
            (sum, b) => sum + safeNum2(b.qty, 0),
            0
          );
          const updatedSizeData = {
            ...normalizedSize,
            batches: nextSizeBatches,
            quantity: newQuantity,
            price: safeNum2(
              currentPrice,
              safeNum2(item.price, safeNum2(product.itemPrice, 0))
            )
          };
          if (product.sizes.set) {
            product.sizes.set(sizeKey, updatedSizeData);
          } else {
            product.sizes[sizeKey] = updatedSizeData;
          }
          product.markModified("sizes");
        }
      }
      let totalStock = 0;
      const sizeEntries = product.sizes instanceof Map ? Array.from(product.sizes.entries()) :
Object.entries(product.sizes);
      for (const [, value] of sizeEntries) {
        totalStock += getSizeQuantity(value);
      }
      product.currentStock = totalStock;
    } else {
      if (isStockOut && product.currentStock < item.quantity) {
        throw new Error(
          `Insufficient stock for ${product.itemName}. Available: ${product.currentStock}, Requested:
${item.quantity}`
        );
      }
      product.currentStock = isStockIn ? product.currentStock + item.quantity : Math.max(0,
```

```
product.currentStock - item.quantity);
      }
      if (hasZeroStock(product)) {
        product.displayInTerminal = false;
      } else if (stockBefore === 0 && product.currentStock > 0) {
        product.displayInTerminal = true;
      }
      product.lastUpdated = Date.now();
      await product.save();
      const stockAfter = product.currentStock;
      await StockMovement.create({
        productId: product._id,
        sku: product.sku,
        itemName: product.itemName,
        itemImage: product.itemImage || "",
        category: product.category,
        brandName: product.brandName || "",
        type: movementType,
        quantity: item.quantity,
        stockBefore,
        stockAfter,
        reason: movementReason,
        handledBy: performedByName || "System",
        handledById: performedById || "",
        notes: [
          sizeForStock ? `Size: ${sizeForStock}` : null,
          variantForStock ? `Variant: ${variantForStock}` : null
        ].filter(Boolean).join(" \xB7 "),
        sizeQuantities: sizeForStock ? { [sizeForStock]: item.quantity } : null
      });
      updatedProducts.push(product);
    }
    res.json({
      success: true,
      message: "Stock updated successfully",
      data: updatedProducts
    });
  } catch (error) {
    console.error("Error updating stock:", error);
    res.status(400).json({
      success: false,
      message: error.message || "Error updating stock"
    });
  }
};
exports.toggleDisplayInTerminal = async (req, res) => {
  try {
    const { id } = req.params;
    const { displayInTerminal } = req.body;
    const product = await Product.findByIdAndUpdate(
      id,
      { displayInTerminal, lastUpdated: Date.now() },
      { new: true }
    );
    if (!product) {
      return res.status(404).json({
        success: false,
        message: "Product not found"
      });
    }
    res.json({
      success: true,
      message: `Product ${displayInTerminal ? "shown" : "hidden"} in terminal`,
      data: product
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: "Error updating product",
      error: error.message
    });
  }
};
exports.searchProducts = async (req, res) => {
  try {
    const { query } = req.params;
    if (!query) {
```

```
      return res.status(400).json({
        success: false,
        message: "Search query is required"
      });
    }
    let products = await Product.find(
      { $text: { $search: query } },
      { score: { $meta: "textScore" } }
    ).select("-sizes").sort({ score: { $meta: "textScore" } }).lean();
    if (products.length === 0) {
      products = await Product.find({
        $or: [
          { itemName: { $regex: query, $options: "i" } },
          { sku: { $regex: query, $options: "i" } },
          { category: { $regex: query, $options: "i" } },
          { brandName: { $regex: query, $options: "i" } }
        ]
      }).select("-sizes").sort({ dateAdded: -1 }).limit(50).lean();
    }
    res.json({
      success: true,
      count: products.length,
      data: products
    });
  } catch (error) {
    console.error("Error searching products:", error);
    res.status(500).json({
      success: false,
      message: "Error searching products",
      error: error.message
    });
  }
};
exports.getInventoryStats = async (req, res) => {
  try {
    const stats = await Product.aggregate([
      {
        $match: {
          isArchived: { $ne: true }
        }
      },
      {
        $group: {
          _id: null,
          totalItems: { $sum: 1 },
          inStock: {
            $sum: { $cond: [{ $gt: ["$currentStock", 0] }, 1, 0] }
          },
          lowStock: {
            $sum: {
              $cond: [
                {
                  $and: [
                    { $gt: ["$currentStock", 0] },
                    { $lte: ["$currentStock", 10] }
                  ]
                },
                1,
                0
              ]
            }
          },
          outOfStock: {
            $sum: { $cond: [{ $eq: ["$currentStock", 0] }, 1, 0] }
          },
          inventoryValue: {
            $sum: { $multiply: ["$currentStock", "$itemPrice"] }
          },
          totalCostValue: {
            $sum: { $multiply: ["$currentStock", "$costPrice"] }
          }
        }
      }
    ]);
    const result = stats[0] || {
      totalItems: 0,
      inStock: 0,
```

```
      lowStock: 0,
      outOfStock: 0,
      inventoryValue: 0,
      totalCostValue: 0
    };
    const grossProfit = result.inventoryValue - result.totalCostValue;
    const grossMargin = result.inventoryValue > 0 ? grossProfit / result.inventoryValue * 100 : 0;
    res.json({
      success: true,
      data: {
        ...result,
        grossProfit,
        grossMargin
      }
    });
  } catch (error) {
    console.error("Error getting inventory stats:", error);
    res.status(500).json({
      success: false,
      message: "Error getting inventory stats",
      error: error.message
    });
  }
};
```

FILE: pos/backend/controllers/remittanceController.js
==========

```
const mongoose = require("mongoose");
const Remittance = require("../models/Remittance");
const SalesTransaction = require("../models/SalesTransaction");
const GlobalSettings = require("../models/GlobalSettings");
const startOfLocalDay = (value = /* @__PURE__ */ new Date()) => {
  const date = new Date(value);
  return new Date(date.getFullYear(), date.getMonth(), date.getDate());
};
const isSameLocalDay = (left, right) => {
  const a = startOfLocalDay(left);
  const b = startOfLocalDay(right);
  return a.getTime() === b.getTime();
};
const getOpeningFloatForEmployee = async (employeeId) => {
  const settings = await GlobalSettings.findOne().lean();
  if (!settings) {
    return 0;
  }
  const entries = Array.isArray(settings.openingFloats) ? settings.openingFloats : [];
  const employeeIdStr = String(employeeId || "").trim();
  const employeeFloatTotal = entries.reduce((sum, entry) => {
    if (String(entry?.employeeId || "").trim() !== employeeIdStr) {
      return sum;
    }
    const effectiveDate = entry?.businessDate || entry?.createdAt;
    if (!effectiveDate || !isSameLocalDay(effectiveDate, /* @__PURE__ */ new Date())) {
      return sum;
    }
    return sum + (Number(entry?.amount) || 0);
  }, 0);
  if (employeeFloatTotal > 0) {
    return employeeFloatTotal;
  }
  return 0;
};
const buildTodayRemittanceQuery = (employeeId, startOfDay, endOfDay) => ({
  employeeId,
  $or: [
    { submittedAt: { $gte: startOfDay, $lt: endOfDay } },
    { shiftDate: { $gte: startOfDay, $lt: endOfDay } }
  ]
});
exports.getRemittanceSummary = async (req, res) => {
  try {
    const { employeeId } = req.query;
    if (!employeeId) {
      return res.status(400).json({ success: false, message: "Employee ID is required" });
    }
```

```
    const now = /* @__PURE__ */ new Date();
    const startOfDay = new Date(now.getFullYear(), now.getMonth(), now.getDate());
    const endOfDay = new Date(now.getFullYear(), now.getMonth(), now.getDate() + 1);
    const completedTransactions = await SalesTransaction.find({
      performedById: employeeId,
      status: { $in: ["Completed", "Partially Returned", "Returned"] },
      checkedOutAt: { $gte: startOfDay, $lt: endOfDay },
      paymentMethod: { $ne: "return" }
    }).select("totalAmount status").lean();
    const returnTransactions = await SalesTransaction.find({
      paymentMethod: "return",
      $or: [
        { performedById: employeeId },
        { returnedById: employeeId }
      ],
      checkedOutAt: { $gte: startOfDay, $lt: endOfDay }
    }).select("totalAmount").lean();
    const [existingTodayRemittance, openingFloatTotal] = await Promise.all([
      Remittance.findOne(
        buildTodayRemittanceQuery(employeeId, startOfDay, endOfDay)
      ).sort({ submittedAt: -1 }).lean(),
      getOpeningFloatForEmployee(employeeId)
    ]);
    const netSales = completedTransactions.reduce((sum, t) => sum + (t.totalAmount || 0), 0);
    const returns = returnTransactions.reduce((sum, t) => sum + Math.abs(t.totalAmount || 0), 0);
    const grossSales = netSales + returns;
    const noOfSales = completedTransactions.filter((t) => t.status !== "Returned").length;
    res.json({
      success: true,
      data: {
        shiftDate: startOfDay,
        grossSales,
        returns,
        netSales,
        noOfSales,
        openingFloatTotal,
        alreadyRemittedToday: !!existingTodayRemittance,
        remittedAmountToday: existingTodayRemittance?.cashToRemit || 0,
        remittedAtToday: existingTodayRemittance?.submittedAt || null
      }
    });
  } catch (error) {
    console.error("Error fetching remittance summary:", error);
    res.status(500).json({ success: false, message: "Error fetching remittance summary", error:
error.message });
  }
};
exports.createRemittance = async (req, res) => {
  try {
    const {
      employeeId,
      employeeName,
      shiftDate,
      grossSales,
      returns,
      netSales,
      noOfSales,
      denominations,
      totalCashOnHand,
      openingFloat,
      cashToRemit,
      expectedCash,
      variance,
      remarks,
      receivedBy
    } = req.body;
    if (!employeeId || !employeeName) {
      return res.status(400).json({ success: false, message: "Employee ID and name are required" });
    }
    const now = /* @__PURE__ */ new Date();
    const startOfDay = new Date(now.getFullYear(), now.getMonth(), now.getDate());
    const endOfDay = new Date(now.getFullYear(), now.getMonth(), now.getDate() + 1);
    const existingToday = await Remittance.findOne(
      buildTodayRemittanceQuery(employeeId, startOfDay, endOfDay)
    ).sort({ submittedAt: -1 }).lean();
    if (existingToday) {
      return res.status(409).json({
```

```
        success: false,
        code: "ALREADY_REMITTED_TODAY",
        message: `You already remitted today. Amount remitted: \u20B1${(existingToday.cashToRemit ||
0).toLocaleString("en-PH", {
          minimumFractionDigits: 2,
          maximumFractionDigits: 2
        })}`,
        data: {
          amountRemitted: existingToday.cashToRemit || 0,
          submittedAt: existingToday.submittedAt,
          remittanceId: existingToday._id
        }
      });
    }
    const remittance = await Remittance.create({
      employeeId,
      employeeName,
      shiftDate: shiftDate || /* @__PURE__ */ new Date(),
      grossSales: grossSales || 0,
      returns: returns || 0,
      netSales: netSales || 0,
      noOfSales: noOfSales || 0,
      denominations: denominations || {},
      totalCashOnHand: totalCashOnHand || 0,
      openingFloat: openingFloat != null ? openingFloat : 0,
      cashToRemit: cashToRemit || 0,
      expectedCash: expectedCash || 0,
      variance: variance || 0,
      remarks: remarks || "",
      receivedBy: receivedBy || "",
      status: "Submitted",
      submittedAt: /* @__PURE__ */ new Date()
    });
    res.status(201).json({
      success: true,
      message: "Remittance submitted successfully",
      data: remittance
    });
  } catch (error) {
    console.error("Error creating remittance:", error);
    res.status(500).json({ success: false, message: "Error creating remittance", error: error.message });
  }
};
exports.getRemittanceKpiStats = async (req, res) => {
  try {
    const { startMs, endMs, startDate, endDate, employeeId } = req.query;
    let lo;
    let hi;
    let allTime = false;
    if (startMs != null && endMs != null && String(startMs).trim() !== "" && String(endMs).trim() !== "") {
      const sm = Number(startMs);
      const em = Number(endMs);
      if (!Number.isFinite(sm) || !Number.isFinite(em)) {
        return res.status(400).json({ success: false, message: "Invalid startMs or endMs" });
      }
      lo = new Date(sm);
      hi = new Date(em);
    } else if (startDate || endDate) {
      lo = startDate ? new Date(startDate) : /* @__PURE__ */ new Date(0);
      hi = endDate ? new Date(endDate) : /* @__PURE__ */ new Date(864e13);
      if (Number.isNaN(lo.getTime())) {
        return res.status(400).json({ success: false, message: "Invalid startDate" });
      }
      if (Number.isNaN(hi.getTime())) {
        return res.status(400).json({ success: false, message: "Invalid endDate" });
      }
    } else {
      allTime = true;
    }
    const empIdStr = employeeId && String(employeeId).trim() ? String(employeeId).trim() : null;
    const transactionDateOr = allTime ? null : {
      $or: [
        { checkedOutAt: { $gte: lo, $lte: hi } },
        {
          $and: [
            {
              $or: [
```

```
            { checkedOutAt: { $exists: false } },
            { checkedOutAt: null }
          ]
        },
        { createdAt: { $gte: lo, $lte: hi } }
      ]
    }
  ]
};
const salesMatch = {
  status: { $in: ["Completed", "Partially Returned", "Returned"] },
  paymentMethod: { $ne: "return" },
  ...transactionDateOr || {}
};
if (empIdStr) {
  salesMatch.performedById = empIdStr;
}
const [completedTransactions, rawReturnTransactions, remitAgg] = await Promise.all([
  SalesTransaction.find(salesMatch).select("totalAmount").lean(),
  SalesTransaction.find({
    paymentMethod: "return",
    ...empIdStr ? {
      $or: [
        { performedById: empIdStr },
        { returnedById: empIdStr }
      ]
    } : {},
    ...transactionDateOr || {}
  }).select("totalAmount").lean(),
  (() => {
    const shiftMatch = {};
    if (!allTime) {
      shiftMatch.shiftDate = { $gte: lo, $lte: hi };
    }
    if (empIdStr && mongoose.Types.ObjectId.isValid(empIdStr)) {
      shiftMatch.employeeId = new mongoose.Types.ObjectId(empIdStr);
    }
    return Remittance.aggregate([
      { $match: shiftMatch },
      {
        $group: {
          _id: null,
          totalRemitted: { $sum: { $ifNull: ["$cashToRemit", 0] } },
          totalVariance: { $sum: { $ifNull: ["$variance", 0] } },
          slipNetSales: { $sum: { $ifNull: ["$netSales", 0] } }
        }
      }
    ]);
  })()
]);
const posNetSales = completedTransactions.reduce(
  (s, t) => s + (Number(t.totalAmount) || 0),
  0
);
const returns = rawReturnTransactions.reduce(
  (s, t) => s + Math.abs(Number(t.totalAmount) || 0),
  0
);
const grossSales = posNetSales + returns;
const netSalesAfterReturns = posNetSales;
const row = remitAgg[0] || {
  totalRemitted: 0,
  totalVariance: 0,
  slipNetSales: 0
};
const totalRemitted = row.totalRemitted || 0;
const totalVariance = row.totalVariance || 0;
const unremittedCash = Math.max(0, netSalesAfterReturns - totalRemitted);
res.json({
  success: true,
  data: {
    posNetSales: netSalesAfterReturns,
    grossSales,
    returns,
    totalRemitted,
    totalVariance,
    slipNetSalesSum: row.slipNetSales || 0,
```

```
        unremittedCash
      }
    });
  } catch (error) {
    console.error("Error fetching remittance KPI stats:", error);
    res.status(500).json({
      success: false,
      message: "Error fetching remittance KPI stats",
      error: error.message
    });
  }
};
exports.getRemittances = async (req, res) => {
  try {
    const remittances = await Remittance.find({}).sort({ submittedAt: -1 }).lean();
    res.json({
      success: true,
      count: remittances.length,
      data: remittances
    });
  } catch (error) {
    console.error("Error fetching remittances:", error);
    res.status(500).json({ success: false, message: "Error fetching remittances", error: error.message });
  }
};
```

FILE: pos/backend/controllers/reportController.js
==========

```
const SalesTransaction = require("../models/SalesTransaction");
const Product = require("../models/Product");
const StockMovement = require("../models/StockMovement");
const getDateRange = (timeframe, customStartDate, customEndDate) => {
  const now = /* @__PURE__ */ new Date();
  let kpiStart, chartStart, end;
  end = new Date(now);
  end.setHours(23, 59, 59, 999);
  if (timeframe === "custom" && customStartDate && customEndDate) {
    kpiStart = new Date(customStartDate);
    kpiStart.setHours(0, 0, 0, 0);
    chartStart = kpiStart;
    end = new Date(customEndDate);
    end.setHours(23, 59, 59, 999);
    return { kpiStart, chartStart, end };
  }
  switch (timeframe) {
    case "daily":
      kpiStart = new Date(now);
      kpiStart.setHours(0, 0, 0, 0);
      chartStart = new Date(now);
      chartStart.setDate(chartStart.getDate() - 6);
      chartStart.setHours(0, 0, 0, 0);
      break;
    case "weekly":
      kpiStart = new Date(now);
      const day = kpiStart.getDay();
      kpiStart.setDate(kpiStart.getDate() - day);
      kpiStart.setHours(0, 0, 0, 0);
      chartStart = new Date(now);
      chartStart.setDate(chartStart.getDate() - 12 * 7);
      chartStart.setHours(0, 0, 0, 0);
      break;
    case "monthly":
      kpiStart = new Date(now.getFullYear(), now.getMonth(), 1);
      kpiStart.setHours(0, 0, 0, 0);
      chartStart = new Date(now.getFullYear(), now.getMonth() - 11, 1);
      chartStart.setHours(0, 0, 0, 0);
      break;
    case "yearly":
      kpiStart = new Date(now.getFullYear(), 0, 1);
      kpiStart.setHours(0, 0, 0, 0);
      chartStart = new Date(now.getFullYear() - 4, 0, 1);
      chartStart.setHours(0, 0, 0, 0);
      break;
    default:
      kpiStart = new Date(now);
```

```
      kpiStart.setHours(0, 0, 0, 0);
      chartStart = new Date(now);
      chartStart.setDate(chartStart.getDate() - 6);
      chartStart.setHours(0, 0, 0, 0);
  }
  return { kpiStart, chartStart, end };
};
exports.getInventoryAnalytics = async (req, res) => {
  try {
    const {
      timeframe = "daily",
      startDate: customStart,
      endDate: customEnd
    } = req.query;
    const { kpiStart, chartStart, end } = getDateRange(timeframe, customStart, customEnd);
    const [
      inventoryAgg,
      salesAgg,
      movementAgg,
      movementTimeSeries,
      salesTimeSeries,
      damagedExpiredItems
    ] = await Promise.all([
      // 1. Product inventory aggregation (replaces Product.find({}).lean())
      Product.aggregate([
        {
          $match: {
            isArchived: { $ne: true }
          }
        },
        {
          $group: {
            _id: null,
            totalItems: { $sum: 1 },
            inventoryValue: {
              $sum: { $multiply: [{ $ifNull: ["$itemPrice", 0] }, { $ifNull: ["$currentStock", 0] }] }
            },
            costValue: {
              $sum: { $multiply: [{ $ifNull: ["$costPrice", 0] }, { $ifNull: ["$currentStock", 0] }] }
            },
            totalStockUnits: { $sum: { $ifNull: ["$currentStock", 0] } },
            inStockCount: {
              $sum: {
                $cond: [
                  { $gt: [{ $ifNull: ["$currentStock", 0] }, { $ifNull: ["$reorderNumber", 10] }] },
                  1,
                  0
                ]
              }
            },
            lowStockCount: {
              $sum: {
                $cond: [
                  {
                    $and: [
                      { $gt: [{ $ifNull: ["$currentStock", 0] }, 0] },
                      { $lte: [{ $ifNull: ["$currentStock", 0] }, { $ifNull: ["$reorderNumber", 10] }] }
                    ]
                  },
                  1,
                  0
                ]
              }
            },
            outOfStockCount: {
              $sum: {
                $cond: [{ $eq: [{ $ifNull: ["$currentStock", 0] }, 0] }, 1, 0]
              }
            }
          }
        }
      ]),
      // 2. Sales transaction aggregation with COGS calculation
      SalesTransaction.aggregate([
        {
          $match: {
            $or: [
```

```
      { checkedOutAt: { $gte: kpiStart, $lte: end } },
      {
        checkedOutAt: { $exists: false },
        createdAt: { $gte: kpiStart, $lte: end }
      }
    ],
    status: { $not: { $regex: /^voided$/i } },
    paymentMethod: { $ne: "return" }
  }
},
{ $unwind: { path: "$items", preserveNullAndEmptyArrays: true } },
{
  $lookup: {
    from: "products",
    localField: "items.productId",
    foreignField: "_id",
    as: "productInfo",
    pipeline: [{ $project: { costPrice: 1, sizes: 1 } }]
  }
},
{ $unwind: { path: "$productInfo", preserveNullAndEmptyArrays: true } },
{
  $addFields: {
    sizeData: {
      $let: {
        vars: {
          matchedSize: {
            $first: {
              $filter: {
                input: {
                  $objectToArray: { $ifNull: ["$productInfo.sizes", {}] } }
                },
                as: "sz",
                cond: { $eq: ["$$sz.k", "$items.selectedSize"] }
              }
            }
          }
        },
        in: "$$matchedSize.v"
      }
    }
  }
},
{
  $addFields: {
    variantCostPrice: {
      $let: {
        vars: {
          matchedVariantCostFromVariants: {
            $first: {
              $filter: {
                input: {
                  $objectToArray: { $ifNull: ["$sizeData.variants", {}] } }
                },
                as: "vn",
                cond: { $eq: ["$$vn.k", "$items.variant"] }
              }
            }
          },
          firstVariantCostFromVariants: {
            $arrayElemAt: [
              {
                $objectToArray: { $ifNull: ["$sizeData.variants", {}] } }
              },
              0
            ]
          },
          matchedVariantCostFromLegacyMap: {
            $first: {
              $filter: {
                input: {
                  $objectToArray: {
                    $ifNull: ["$sizeData.variantCostPrices", {}]
                  }
                },
                as: "lvc",
                cond: { $eq: ["$$lvc.k", "$items.variant"] }
```

```
          }
        }
      }
    },
    in: {
      $ifNull: [
        "$$matchedVariantCostFromVariants.v.costPrice",
        {
          $ifNull: [
            "$$firstVariantCostFromVariants.v.costPrice",
            "$$matchedVariantCostFromLegacyMap.v"
          ]
        }
      ]
    }
  }
},
sizeCostPrice: {
  $cond: [
    { $eq: [{ $type: "$sizeData" }, "object"] },
    { $ifNull: ["$sizeData.costPrice", null] },
    null
  ]
},
itemCostPrice: {
  $let: {
    vars: {
      computedCost: {
        $ifNull: [
          "$variantCostPrice",
          { $ifNull: ["$sizeCostPrice", { $ifNull: ["$productInfo.costPrice", 0] }] }
        ]
      },
      firstSizeEntry: {
        $arrayElemAt: [
          { $objectToArray: { $ifNull: ["$productInfo.sizes", {}] } },
          0
        ]
      }
    },
    in: {
      $cond: [
        { $gt: ["$$computedCost", 0] },
        "$$computedCost",
        {
          $let: {
            vars: {
              fallbackVariantCostPrice: {
                $let: {
                  vars: {
                    firstVariantEntry: {
                      $arrayElemAt: [
                        {
                          $objectToArray: {
                            $ifNull: ["$$firstSizeEntry.v.variants", {}]
                          }
                        },
                        0
                      ]
                    },
                    matchedVariantFromVariants: {
                      $first: {
                        $filter: {
                          input: {
                            $objectToArray: {
                              $ifNull: ["$$firstSizeEntry.v.variants", {}]
                            }
                          },
                          as: "vn",
                          cond: { $eq: ["$$vn.k", "$items.variant"] }
                        }
                      }
                    },
                    matchedVariantFromLegacyMap: {
                      $first: {
                        $filter: {
                          input: {
```

```
                                $objectToArray: {
                                  $ifNull: ["$$firstSizeEntry.v.variantCostPrices", {}]
                                }
                              },
                              as: "lvc",
                              cond: { $eq: ["$$lvc.k", "$items.variant"] }
                            }
                          }
                        }
                      },
                      in: {
                        $ifNull: [
                          "$$matchedVariantFromVariants.v.costPrice",
                          {
                            $ifNull: [
                              "$$firstVariantEntry.v.costPrice",
                              "$$matchedVariantFromLegacyMap.v"
                            ]
                          }
                        ]
                      }
                    }
                  }
                },
                in: {
                  $ifNull: [
                    "$$fallbackVariantCostPrice",
                    {
                      $ifNull: [
                        "$$firstSizeEntry.v.costPrice",
                        { $ifNull: ["$productInfo.costPrice", 0] }
                      ]
                    }
                  ]
                }
              }
            }
          ]
        }
      }
    }
  }
},
{
  $addFields: {
    effectiveSoldQty: {
      $cond: [
        { $eq: ["$items.returnStatus", "Returned"] },
        0,
        { $ifNull: ["$items.quantity", 1] }
      ]
    }
  }
},
{
  $group: {
    _id: "$_id",
    totalAmount: { $first: "$totalAmount" },
    totalUnitsSold: { $sum: "$effectiveSoldQty" },
    cogs: {
      $sum: {
        $multiply: [
          { $ifNull: ["$itemCostPrice", 0] },
          "$effectiveSoldQty"
        ]
      }
    }
  }
},
{
  $group: {
    _id: null,
    totalSales: { $sum: { $ifNull: ["$totalAmount", 0] } },
    totalTransactions: { $sum: 1 },
    totalUnitsSold: { $sum: "$totalUnitsSold" },
    cogs: { $sum: "$cogs" }
  }
```

```
  }
]),
// 3. Stock movement counts aggregation
StockMovement.aggregate([
  { $match: { createdAt: { $gte: kpiStart, $lte: end } } },
  {
    $group: {
      _id: "$type",
      totalQuantity: { $sum: { $abs: "$quantity" } }
    }
  }
]),
// 4. Stock movement time series (for inventory chart)
StockMovement.aggregate([
  { $match: { createdAt: { $gte: chartStart, $lte: end } } },
  {
    $group: {
      _id: getTimeSeriesGroupId(timeframe),
      stockIn: {
        $sum: {
          $cond: [{ $eq: ["$type", "Stock-In"] }, { $abs: "$quantity" }, 0]
        }
      },
      stockOut: {
        $sum: {
          $cond: [{ $eq: ["$type", "Stock-Out"] }, { $abs: "$quantity" }, 0]
        }
      },
      pullOut: {
        $sum: {
          $cond: [{ $eq: ["$type", "Pull-Out"] }, { $abs: "$quantity" }, 0]
        }
      },
      sortDate: { $min: "$createdAt" }
    }
  },
  { $sort: { sortDate: 1 } }
]),
// 5. Sales time series (for profit chart)
SalesTransaction.aggregate([
  {
    $match: {
      $or: [
        { checkedOutAt: { $gte: chartStart, $lte: end } },
        {
          checkedOutAt: { $exists: false },
          createdAt: { $gte: chartStart, $lte: end }
        }
      ],
      status: { $not: { $regex: /^voided$/i } },
      paymentMethod: { $ne: "return" }
    }
  },
  { $unwind: { path: "$items", preserveNullAndEmptyArrays: true } },
  {
    $lookup: {
      from: "products",
      localField: "items.productId",
      foreignField: "_id",
      as: "productInfo",
      pipeline: [{ $project: { costPrice: 1, sizes: 1 } }]
    }
  },
  { $unwind: { path: "$productInfo", preserveNullAndEmptyArrays: true } },
  {
    $addFields: {
      sizeData: {
        $let: {
          vars: {
            matchedSize: {
              $first: {
                $filter: {
                  input: {
                    $objectToArray: { $ifNull: ["$productInfo.sizes", {}] }
                  },
                  as: "sz",
                  cond: { $eq: ["$$sz.k", "$items.selectedSize"] }
```

```
                }
              }
            }
          },
          in: "$$matchedSize.v"
        }
      }
    }
  },
  {
    $addFields: {
      variantCostPrice: {
        $let: {
          vars: {
            matchedVariantCostFromVariants: {
              $first: {
                $filter: {
                  input: {
                    $objectToArray: { $ifNull: ["$sizeData.variants", {}] }
                  },
                  as: "vn",
                  cond: { $eq: ["$$vn.k", "$items.variant"] }
                }
              }
            },
            firstVariantCostFromVariants: {
              $arrayElemAt: [
                {
                  $objectToArray: { $ifNull: ["$sizeData.variants", {}] }
                },
                0
              ]
            },
            matchedVariantCostFromLegacyMap: {
              $first: {
                $filter: {
                  input: {
                    $objectToArray: {
                      $ifNull: ["$sizeData.variantCostPrices", {}]
                    }
                  },
                  as: "lvc",
                  cond: { $eq: ["$$lvc.k", "$items.variant"] }
                }
              }
            }
          },
          in: {
            $ifNull: [
              "$$matchedVariantCostFromVariants.v.costPrice",
              {
                $ifNull: [
                  "$$firstVariantCostFromVariants.v.costPrice",
                  "$$matchedVariantCostFromLegacyMap.v"
                ]
              }
            ]
          }
        }
      }
      },
      sizeCostPrice: {
        $cond: [
          { $eq: [{ $type: "$sizeData" }, "object"] },
          { $ifNull: ["$sizeData.costPrice", null] },
          null
        ]
      },
      itemCostPrice: {
        $let: {
          vars: {
            computedCost: {
              $ifNull: [
                "$variantCostPrice",
                { $ifNull: ["$sizeCostPrice", { $ifNull: ["$productInfo.costPrice", 0] }] }
              ]
            },
            firstSizeEntry: {
```

```
      $arrayElemAt: [
        { $objectToArray: { $ifNull: ["$productInfo.sizes", {}] } },
        0
      ]
    }
  },
  in: {
    $cond: [
      { $gt: ["$$computedCost", 0] },
      "$$computedCost",
      {
        $let: {
          vars: {
            fallbackVariantCostPrice: {
              $let: {
                vars: {
                  firstVariantEntry: {
                    $arrayElemAt: [
                      {
                        $objectToArray: {
                          $ifNull: ["$$firstSizeEntry.v.variants", {}]
                        }
                      },
                      0
                    ]
                  },
                  matchedVariantFromVariants: {
                    $first: {
                      $filter: {
                        input: {
                          $objectToArray: {
                            $ifNull: ["$$firstSizeEntry.v.variants", {}]
                          }
                        },
                        as: "vn",
                        cond: { $eq: ["$$vn.k", "$items.variant"] }
                      }
                    }
                  },
                  matchedVariantFromLegacyMap: {
                    $first: {
                      $filter: {
                        input: {
                          $objectToArray: {
                            $ifNull: ["$$firstSizeEntry.v.variantCostPrices", {}]
                          }
                        },
                        as: "lvc",
                        cond: { $eq: ["$$lvc.k", "$items.variant"] }
                      }
                    }
                  }
                },
                in: {
                  $ifNull: [
                    "$$matchedVariantFromVariants.v.costPrice",
                    {
                      $ifNull: [
                        "$$firstVariantEntry.v.costPrice",
                        "$$matchedVariantFromLegacyMap.v"
                      ]
                    }
                  ]
                }
              }
            }
          },
          in: {
            $ifNull: [
              "$$fallbackVariantCostPrice",
              {
                $ifNull: [
                  "$$firstSizeEntry.v.costPrice",
                  { $ifNull: ["$productInfo.costPrice", 0] }
                ]
              }
            ]
```

```
                    }
                  }
                }
              ]
            }
          }
        },
        effectiveSoldQty: {
          $cond: [
            { $eq: ["$items.returnStatus", "Returned"] },
            0,
            { $ifNull: ["$items.quantity", 1] }
          ]
        },
        dateField: { $ifNull: ["$checkedOutAt", "$createdAt"] }
      }
    },
    {
      $group: {
        _id: {
          txnId: "$_id",
          period: getTimeSeriesGroupIdForDate(timeframe, "$dateField")
        },
        revenue: { $first: "$totalAmount" },
        cogs: {
          $sum: {
            $multiply: [
              { $ifNull: ["$itemCostPrice", 0] },
              "$effectiveSoldQty"
            ]
          }
        },
        sortDate: { $min: "$dateField" }
      }
    },
    {
      $group: {
        _id: "$_id.period",
        revenue: { $sum: { $ifNull: ["$revenue", 0] } },
        cogs: { $sum: "$cogs" },
        sortDate: { $min: "$sortDate" }
      }
    },
    { $sort: { sortDate: 1 } }
  ]),
  // 6. Damaged & expired items (limited to 20)
  StockMovement.aggregate([
    {
      $match: {
        createdAt: { $gte: kpiStart, $lte: end },
        reason: { $in: ["Damaged", "Expired", "Lost"] }
      }
    },
    { $sort: { createdAt: -1 } },
    { $limit: 20 },
    {
      $project: {
        itemName: { $ifNull: ["$itemName", "Unknown"] },
        sku: { $ifNull: ["$sku", "-"] },
        category: { $ifNull: ["$category", "-"] },
        type: "$reason",
        quantity: { $abs: "$quantity" },
        date: "$createdAt",
        handledBy: { $ifNull: ["$handledBy", "Unknown"] },
        notes: { $ifNull: ["$notes", ""] }
      }
    }
  ])
]);
const inv = inventoryAgg[0] || {
  totalItems: 0,
  inventoryValue: 0,
  costValue: 0,
  totalStockUnits: 0,
  inStockCount: 0,
  lowStockCount: 0,
  outOfStockCount: 0
```

```
    };
    const sales = salesAgg[0] || {
      totalSales: 0,
      totalTransactions: 0,
      totalUnitsSold: 0,
      cogs: 0
    };
    const totalProfit = sales.totalSales - sales.cogs;
    const profitMargin = sales.totalSales > 0 ? totalProfit / sales.totalSales * 100 : 0;
    let stockInCount = 0, stockOutCount = 0, pullOutCount = 0;
    movementAgg.forEach((m) => {
      if (m._id === "Stock-In") stockInCount = m.totalQuantity;
      else if (m._id === "Stock-Out") stockOutCount = m.totalQuantity;
      else if (m._id === "Pull-Out") pullOutCount = m.totalQuantity;
    });
    const inventoryChartData = buildChartFromAggregation(
      movementTimeSeries,
      timeframe,
      chartStart,
      end,
      "inventory"
    );
    const profitChartData = buildChartFromAggregation(
      salesTimeSeries,
      timeframe,
      chartStart,
      end,
      "profit"
    );
    let damagedTotal = 0, expiredTotal = 0, lostTotal = 0;
    damagedExpiredItems.forEach((item) => {
      if (item.type === "Damaged") damagedTotal += item.quantity;
      else if (item.type === "Expired") expiredTotal += item.quantity;
      else if (item.type === "Lost") lostTotal += item.quantity;
    });
    res.json({
      success: true,
      data: {
        // KPI Cards
        kpis: {
          totalSales: sales.totalSales,
          totalTransactions: sales.totalTransactions,
          totalUnitsSold: sales.totalUnitsSold,
          cogs: sales.cogs,
          totalProfit,
          profitMargin: Math.round(profitMargin * 100) / 100,
          inventoryValue: inv.inventoryValue,
          costValue: inv.costValue,
          totalStockUnits: inv.totalStockUnits
        },
        // Stat Cards
        stats: {
          totalItems: inv.totalItems,
          inStockCount: inv.inStockCount,
          lowStockCount: inv.lowStockCount,
          outOfStockCount: inv.outOfStockCount,
          stockInCount,
          stockOutCount,
          pullOutCount
        },
        // Chart: Inventory Analysis (Stock-In vs Stock-Out bars)
        inventoryChartData,
        // Chart: Profit Analysis (Revenue, COGS, Profit bars)
        profitChartData,
        // Table: Damaged & Expired
        damagedExpired: {
          items: damagedExpiredItems,
          summary: {
            damaged: damagedTotal,
            expired: expiredTotal,
            lost: lostTotal,
            total: damagedTotal + expiredTotal + lostTotal
          }
        }
      }
    });
  } catch (error) {
```

```
    console.error("Error fetching inventory analytics:", error);
    res.status(500).json({
      success: false,
      message: "Error fetching inventory analytics",
      error: error.message
    });
  }
};
function getTimeSeriesGroupId(timeframe) {
  switch (timeframe) {
    case "daily":
      return {
        year: { $year: "$createdAt" },
        month: { $month: "$createdAt" },
        day: { $dayOfMonth: "$createdAt" }
      };
    case "weekly":
      return {
        year: { $isoWeekYear: "$createdAt" },
        week: { $isoWeek: "$createdAt" }
      };
    case "monthly":
      return {
        year: { $year: "$createdAt" },
        month: { $month: "$createdAt" }
      };
    case "yearly":
      return { year: { $year: "$createdAt" } };
    default:
      return {
        year: { $year: "$createdAt" },
        month: { $month: "$createdAt" },
        day: { $dayOfMonth: "$createdAt" }
      };
  }
}
function getTimeSeriesGroupIdForDate(timeframe, dateField) {
  switch (timeframe) {
    case "daily":
      return {
        year: { $year: dateField },
        month: { $month: dateField },
        day: { $dayOfMonth: dateField }
      };
    case "weekly":
      return {
        year: { $isoWeekYear: dateField },
        week: { $isoWeek: dateField }
      };
    case "monthly":
      return {
        year: { $year: dateField },
        month: { $month: dateField }
      };
    case "yearly":
      return { year: { $year: dateField } };
    default:
      return {
        year: { $year: dateField },
        month: { $month: dateField },
        day: { $dayOfMonth: dateField }
      };
  }
}
function buildChartFromAggregation(aggResults, timeframe, start, end, chartType) {
  const buckets = generateBuckets(timeframe, start, end);
  const dataMap = {};
  aggResults.forEach((item) => {
    const key = aggIdToKey(item._id, timeframe);
    dataMap[key] = item;
  });
  return buckets.map((bucket) => {
    const data = dataMap[bucket.key];
    if (chartType === "inventory") {
      return {
        period: bucket.label,
        stockIn: data ? data.stockIn : 0,
```

```
        stockOut: data ? data.stockOut : 0,
        pullOut: data ? data.pullOut : 0
      };
    } else {
      const revenue = data ? data.revenue : 0;
      const cogs = data ? data.cogs : 0;
      return {
        period: bucket.label,
        revenue: Math.round(revenue * 100) / 100,
        cogs: Math.round(cogs * 100) / 100,
        profit: Math.round((revenue - cogs) * 100) / 100
      };
    }
  });
}
function aggIdToKey(id, timeframe) {
  if (!id) return "unknown";
  switch (timeframe) {
    case "daily":
      return `day_${id.year}_${id.month - 1}_${id.day}`;
    case "weekly":
      return `week_${id.year}_${id.week}`;
    case "monthly":
      return `month_${id.year}_${id.month - 1}`;
    case "yearly":
      return `year_${id.year}`;
    default:
      if (id.day !== void 0) return `day_${id.year}_${id.month - 1}_${id.day}`;
      if (id.month !== void 0) return `month_${id.year}_${id.month - 1}`;
      return `year_${id.year}`;
  }
}
function generateBuckets(timeframe, start, end) {
  const buckets = [];
  const now = /* @__PURE__ */ new Date();
  switch (timeframe) {
    case "daily": {
      for (let i = 6; i >= 0; i--) {
        const d = new Date(now);
        d.setDate(d.getDate() - i);
        d.setHours(0, 0, 0, 0);
        const label = d.toLocaleDateString("en-US", {
          month: "short",
          day: "numeric"
        });
        buckets.push({
          key: `day_${d.getFullYear()}_${d.getMonth()}_${d.getDate()}`,
          label
        });
      }
      break;
    }
    case "weekly": {
      for (let i = 11; i >= 0; i--) {
        const weekStart = new Date(now);
        const dayOfWeek = weekStart.getDay();
        weekStart.setDate(weekStart.getDate() - dayOfWeek - i * 7);
        weekStart.setHours(0, 0, 0, 0);
        const tempDate = new Date(weekStart);
        tempDate.setHours(0, 0, 0, 0);
        tempDate.setDate(tempDate.getDate() + 3 - (tempDate.getDay() + 6) % 7);
        const week1 = new Date(tempDate.getFullYear(), 0, 4);
        const isoWeek = 1 + Math.round(
          ((tempDate.getTime() - week1.getTime()) / 864e5 - 3 + (week1.getDay() + 6) % 7) / 7
        );
        const label = `Week ${weekStart.getDate()}`;
        buckets.push({
          key: `week_${weekStart.getFullYear()}_${isoWeek}`,
          label
        });
      }
      break;
    }
    case "monthly": {
      for (let i = 11; i >= 0; i--) {
        const d = new Date(now.getFullYear(), now.getMonth() - i, 1);
        d.setHours(0, 0, 0, 0);
```

```
        const label = d.toLocaleDateString("en-US", { month: "short" });
        buckets.push({
          key: `month_${d.getFullYear()}_${d.getMonth()}`,
          label
        });
      }
      break;
    }
    case "yearly": {
      for (let i = 4; i >= 0; i--) {
        const year = now.getFullYear() - i;
        buckets.push({
          key: `year_${year}`,
          label: year.toString()
        });
      }
      break;
    }
    default: {
      const diffDays = Math.ceil((end - start) / (1e3 * 60 * 60 * 24));
      if (diffDays <= 31) {
        for (let i = 0; i <= diffDays; i++) {
          const d = new Date(start);
          d.setDate(d.getDate() + i);
          const label = d.toLocaleDateString("en-US", {
            month: "short",
            day: "numeric"
          });
          buckets.push({
            key: `day_${d.getFullYear()}_${d.getMonth()}_${d.getDate()}`,
            label
          });
        }
      } else if (diffDays <= 180) {
        const weeks = Math.ceil(diffDays / 7);
        for (let i = 0; i < weeks; i++) {
          const d = new Date(start);
          d.setDate(d.getDate() + i * 7);
          const label = `Week of ${d.toLocaleDateString("en-US", { month: "short", day: "numeric" })}`;
          buckets.push({
            key: `day_${d.getFullYear()}_${d.getMonth()}_${d.getDate()}`,
            label
          });
        }
      } else {
        let current = new Date(start.getFullYear(), start.getMonth(), 1);
        while (current <= end) {
          const label = current.toLocaleDateString("en-US", {
            month: "short",
            year: "numeric"
          });
          buckets.push({
            key: `month_${current.getFullYear()}_${current.getMonth()}`,
            label
          });
          current.setMonth(current.getMonth() + 1);
        }
      }
    }
  }
  return buckets;
}
```

FILE: pos/backend/controllers/stockMovementController.js
==========

```
const StockMovement = require("../models/StockMovement");
const Product = require("../models/Product");
exports.createStockMovement = async (req, res) => {
  try {
    const {
      productId,
      type,
      quantity,
      stockBefore,
      stockAfter,
```

```
      reason,
      handledBy,
      handledById,
      notes
    } = req.body;
    if (!productId || !type || !quantity || stockBefore === void 0 || stockAfter === void 0 || !reason
|| !handledBy) {
      return res.status(400).json({
        success: false,
        message: "Missing required fields"
      });
    }
    const product = await Product.findById(productId);
    if (!product) {
      return res.status(404).json({
        success: false,
        message: "Product not found"
      });
    }
    const stockMovement = await StockMovement.create({
      productId,
      sku: product.sku,
      itemName: product.itemName,
      itemImage: product.itemImage || "",
      category: product.category,
      brandName: product.brandName || "",
      type,
      quantity,
      stockBefore,
      stockAfter,
      reason,
      handledBy,
      handledById: handledById || "",
      notes: notes || ""
    });
    res.status(201).json({
      success: true,
      message: "Stock movement logged successfully",
      data: stockMovement
    });
  } catch (error) {
    console.error("Error creating stock movement:", error);
    res.status(500).json({
      success: false,
      message: "Error creating stock movement",
      error: error.message
    });
  }
};
exports.getStockMovements = async (req, res) => {
  try {
    const {
      search = "",
      category,
      type,
      brand,
      reason,
      date,
      sortBy = "date-desc",
      page = 1,
      limit = 60
    } = req.query;
    const query = {};
    if (search) {
      query.$or = [
        { itemName: { $regex: search, $options: "i" } },
        { sku: { $regex: search, $options: "i" } },
        { handledBy: { $regex: search, $options: "i" } }
      ];
    }
    if (category && category !== "All") {
      query.category = category;
    }
    if (type && type !== "All") {
      query.type = type;
    }
    if (brand && brand !== "All") {
```

```
      query.brandName = brand;
    }
    if (reason && reason !== "All") {
      query.reason = reason;
    }
    if (date && date !== "All") {
      const now = /* @__PURE__ */ new Date();
      let startDate, endDate;
      if (date === "Today") {
        startDate = new Date(now);
        startDate.setHours(0, 0, 0, 0);
        endDate = new Date(now);
        endDate.setHours(23, 59, 59, 999);
      } else if (date === "This Week") {
        startDate = new Date(now);
        startDate.setDate(now.getDate() - now.getDay());
        startDate.setHours(0, 0, 0, 0);
        endDate = new Date(now);
        endDate.setHours(23, 59, 59, 999);
      } else if (date === "This Month") {
        startDate = new Date(now.getFullYear(), now.getMonth(), 1);
        startDate.setHours(0, 0, 0, 0);
        endDate = new Date(now);
        endDate.setHours(23, 59, 59, 999);
      } else {
        startDate = new Date(date);
        startDate.setHours(0, 0, 0, 0);
        endDate = new Date(date);
        endDate.setHours(23, 59, 59, 999);
      }
      query.createdAt = { $gte: startDate, $lte: endDate };
    }
    let sort = { createdAt: -1 };
    switch (sortBy) {
      case "date-asc":
        sort = { createdAt: 1 };
        break;
      case "quantity-desc":
        sort = { quantity: -1 };
        break;
      case "quantity-asc":
        sort = { quantity: 1 };
        break;
      case "name-asc":
        sort = { itemName: 1 };
        break;
      case "name-desc":
        sort = { itemName: -1 };
        break;
      case "sku-asc":
        sort = { sku: 1 };
        break;
      case "sku-desc":
        sort = { sku: -1 };
        break;
    }
    const skip = (parseInt(page) - 1) * parseInt(limit);
    const [movements, total] = await Promise.all([
      StockMovement.find(query).sort(sort).skip(skip).limit(parseInt(limit)).lean(),
      StockMovement.countDocuments(query)
    ]);
    res.json({
      success: true,
      data: movements,
      pagination: {
        page: parseInt(page),
        limit: parseInt(limit),
        total,
        pages: Math.ceil(total / parseInt(limit))
      }
    });
  } catch (error) {
    console.error("Error fetching stock movements:", error);
    res.status(500).json({
      success: false,
      message: "Error fetching stock movements",
      error: error.message
```

```
    });
  }
};
exports.getTodayStats = async (req, res) => {
  try {
    const today = /* @__PURE__ */ new Date();
    today.setHours(0, 0, 0, 0);
    const tomorrow = new Date(today);
    tomorrow.setDate(tomorrow.getDate() + 1);
    const result = await StockMovement.aggregate([
      { $match: { createdAt: { $gte: today, $lt: tomorrow } } },
      {
        $group: {
          _id: "$type",
          total: { $sum: "$quantity" },
          count: { $sum: 1 }
        }
      }
    ]);
    const stats = {
      stockIn: 0,
      stockOut: 0,
      pullOut: 0,
      totalMovements: 0
    };
    result.forEach((r) => {
      if (r._id === "Stock-In") stats.stockIn = r.total;
      else if (r._id === "Stock-Out") stats.stockOut = r.total;
      else if (r._id === "Pull-Out") stats.pullOut = r.total;
      stats.totalMovements += r.count;
    });
    res.json({
      success: true,
      data: stats
    });
  } catch (error) {
    console.error("Error fetching today stats:", error);
    res.status(500).json({
      success: false,
      message: "Error fetching today stats",
      error: error.message
    });
  }
};
exports.getMovementsByProduct = async (req, res) => {
  try {
    const { productId } = req.params;
    const { limit = 50 } = req.query;
    const movements = await StockMovement.find({ productId }).sort({ createdAt: -
1 }).limit(parseInt(limit)).lean();
    res.json({
      success: true,
      count: movements.length,
      data: movements
    });
  } catch (error) {
    console.error("Error fetching product movements:", error);
    res.status(500).json({
      success: false,
      message: "Error fetching product movements",
      error: error.message
    });
  }
};
exports.getStockStatsOverTime = async (req, res) => {
  try {
    const { timeframe = "daily" } = req.query;
    const now = /* @__PURE__ */ new Date();
    let periods = [];
    let limit = 7;
    switch (timeframe.toLowerCase()) {
      case "daily":
        limit = 7;
        for (let i = limit - 1; i >= 0; i--) {
          const d = new Date(now);
          d.setDate(d.getDate() - i);
          const label = d.toLocaleDateString("en-US", { month: "short", day: "numeric" });
```

```
          const start = new Date(d);
          start.setHours(0, 0, 0, 0);
          const end = new Date(d);
          end.setHours(23, 59, 59, 999);
          periods.push({ start, end, label });
        }
        break;
      case "monthly":
        limit = 6;
        for (let i = limit - 1; i >= 0; i--) {
          const d = new Date(now.getFullYear(), now.getMonth() - i, 1);
          const label = d.toLocaleDateString("en-US", { month: "short" });
          const start = new Date(d);
          const end = new Date(d.getFullYear(), d.getMonth() + 1, 0, 23, 59, 59, 999);
          periods.push({ start, end, label });
        }
        break;
      case "yearly":
        limit = 5;
        for (let i = limit - 1; i >= 0; i--) {
          const year = now.getFullYear() - i;
          const label = year.toString();
          const start = new Date(year, 0, 1, 0, 0, 0, 0);
          const end = new Date(year, 11, 31, 23, 59, 59, 999);
          periods.push({ start, end, label });
        }
        break;
      default:
        limit = 7;
        for (let i = limit - 1; i >= 0; i--) {
          const d = new Date(now);
          d.setDate(d.getDate() - i);
          const label = d.toLocaleDateString("en-US", { month: "short", day: "numeric" });
          const start = new Date(d);
          start.setHours(0, 0, 0, 0);
          const end = new Date(d);
          end.setHours(23, 59, 59, 999);
          periods.push({ start, end, label });
        }
    }
    const stockIn = new Array(periods.length).fill(0);
    const pullOut = new Array(periods.length).fill(0);
    const labels = periods.map((p) => p.label);
    const globalStart = periods[0].start;
    const globalEnd = periods[periods.length - 1].end;
    const movements = await StockMovement.find({
      createdAt: { $gte: globalStart, $lte: globalEnd }
    }).lean();
    movements.forEach((movement) => {
      const date = new Date(movement.createdAt);
      const index = periods.findIndex((p) => date >= p.start && date <= p.end);
      if (index !== -1) {
        if (movement.type === "Stock-In" || movement.type === "in" || movement.movementType === "stock-in") {
          stockIn[index] += movement.quantity || 0;
        } else if (movement.type === "Pull-Out" || movement.type === "out" || movement.movementType ===
"stock-out" || movement.movementType === "pull-out") {
          pullOut[index] += movement.quantity || 0;
        }
      }
    });
    res.json({
      success: true,
      data: {
        labels,
        stockIn,
        pullOut
      }
    });
  } catch (error) {
    console.error("Error fetching stock stats over time:", error);
    res.status(500).json({
      success: false,
      message: "Error fetching stock stats over time",
      error: error.message
    });
  }
};
```

FILE: pos/backend/controllers/transactionController.js
==========

```
const mongoose = require("mongoose");
const SalesTransaction = require("../models/SalesTransaction");
const VoidLog = require("../models/VoidLog");
const Product = require("../models/Product");
const Discount = require("../models/Discount");
const toObjectId = (id) => {
  if (!id) return null;
  try {
    if (mongoose.Types.ObjectId.isValid(id)) {
      return new mongoose.Types.ObjectId(id);
    }
    return null;
  } catch (e) {
    console.error("Error converting to ObjectId:", e);
    return null;
  }
};
const isArchiveReturnReason = (reason) => {
  const head = String(reason || "").split(":")[0].trim().toLowerCase();
  return ["damaged", "defective", "expired"].includes(head);
};
const generateUniqueReceiptNumber = async () => {
  let attempts = 0;
  while (attempts < 10) {
    const receiptNo = Math.floor(1e5 + Math.random() * 9e5).toString();
    const existing = await SalesTransaction.findOne({ receiptNo });
    if (!existing) {
      return receiptNo;
    }
    attempts++;
  }
  const timestamp = Date.now().toString().slice(-4);
  const random = Math.floor(10 + Math.random() * 90).toString();
  return `${timestamp}${random}`;
};
const generateVoidId = async () => {
  const chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
  let voidId;
  let isUnique = false;
  let attempts = 0;
  while (!isUnique && attempts < 10) {
    voidId = "VOID-";
    for (let i = 0; i < 6; i++) {
      voidId += chars.charAt(Math.floor(Math.random() * chars.length));
    }
    const existing = await SalesTransaction.findOne({ voidId });
    if (!existing) {
      isUnique = true;
    }
    attempts++;
  }
  if (!isUnique) {
    const timestamp = Date.now().toString().slice(-4);
    voidId = `VOID-${timestamp}${Math.floor(Math.random() * 100).toString().padStart(2, "0")}`;
  }
  return voidId;
};
exports.getAllTransactions = async (req, res) => {
  try {
    const page = parseInt(req.query.page) || 1;
    const limit = parseInt(req.query.limit) || 20;
    const skip = (page - 1) * limit;
    const transactions = await SalesTransaction.find({}).select("-items.itemImage").sort({ createdAt: -
1 }).skip(skip).limit(limit).lean();
    const totalCount = await SalesTransaction.estimatedDocumentCount();
    res.json({
      success: true,
      count: transactions.length,
      total: totalCount,
      totalPages: Math.ceil(totalCount / limit),
      currentPage: page,
      data: transactions
```

```
    });
  } catch (error) {
    console.error("Error fetching transactions:", error);
    res.status(500).json({
      success: false,
      message: "Error fetching transactions",
      error: error.message
    });
  }
};
exports.getTransactionById = async (req, res) => {
  try {
    const transaction = await SalesTransaction.findById(req.params.id).lean();
    if (!transaction) {
      return res.status(404).json({
        success: false,
        message: "Transaction not found"
      });
    }
    res.json({
      success: true,
      data: transaction
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: "Error fetching transaction",
      error: error.message
    });
  }
};
exports.updateTransaction = async (req, res) => {
  try {
    const { id } = req.params;
    const updateData = req.body;
    const transaction = await SalesTransaction.findById(id);
    if (!transaction) {
      return res.status(404).json({
        success: false,
        message: "Transaction not found"
      });
    }
    if (updateData.status) {
      transaction.status = updateData.status;
    }
    if (updateData.items) {
      transaction.items = updateData.items;
    }
    if (updateData.totalAmount !== void 0) {
      transaction.totalAmount = updateData.totalAmount;
    }
    await transaction.save();
    res.json({
      success: true,
      message: "Transaction updated successfully",
      data: transaction
    });
  } catch (error) {
    console.error("Error updating transaction:", error);
    res.status(500).json({
      success: false,
      message: "Error updating transaction",
      error: error.message
    });
  }
};
exports.createTransaction = async (req, res) => {
  try {
    const {
      items,
      subtotal,
      discount,
      discountType,
      discountValue,
      totalAmount,
      paymentMethod,
      amountPaid,
```

```
      amountReceived,
      change,
      changeGiven,
      cashierName,
      cashierId,
      userId,
      performedById,
      performedByName,
      status,
      customerName,
      customerType,
      seniorCitizenId,
      pwdId,
      referenceNo,
      receiptNo: providedReceiptNo,
      originalTransactionId,
      checkedOutAt,
      appliedDiscountIds
      // Array of discount IDs used in this transaction
    } = req.body;
    if (!items || !Array.isArray(items) || items.length === 0) {
      return res.status(400).json({
        success: false,
        message: "Items are required"
      });
    }
    const receiptNo = providedReceiptNo || await generateUniqueReceiptNumber();
    const transactionData = {
      receiptNo,
      userId: userId || cashierId || performedById || "system",
      performedById: performedById || cashierId || "",
      performedByName: performedByName || cashierName || "",
      items: items.map((item) => {
        const productId = toObjectId(item._id || item.productId);
        if (!productId) {
          console.warn("Invalid productId for item:", item.itemName, "ID:", item._id || item.productId);
        }
        return {
          productId,
          itemName: item.itemName || "Unknown Item",
          sku: item.sku || "",
          variant: item.variant || item.selectedVariation || null,
          selectedSize: item.selectedSize || item.size || null,
          quantity: item.quantity || 1,
          price: item.itemPrice || item.price || 0,
          itemImage: item.itemImage || "",
          returnReason: item.returnReason || null
        };
      }).filter((item) => item.productId !== null),
      subtotal: subtotal || 0,
      discount: discount || 0,
      discountType: discountType || null,
      discountValue: discountValue || 0,
      totalAmount: totalAmount || 0,
      paymentMethod: paymentMethod || "cash",
      amountReceived: amountReceived || amountPaid || 0,
      changeGiven: changeGiven || change || 0,
      status: status || "Completed",
      customerName: customerName || null,
      customerType: customerType || "regular",
      seniorCitizenId: seniorCitizenId || null,
      pwdId: pwdId || null,
      referenceNo: referenceNo || null,
      originalTransactionId: originalTransactionId || null,
      checkedOutAt: checkedOutAt || /* @__PURE__ */ new Date(),
      appliedDiscountIds: Array.isArray(appliedDiscountIds) && appliedDiscountIds.length > 0 ?
appliedDiscountIds.map((id) => toObjectId(id)).filter(Boolean) : void 0
    };
    if (transactionData.items.length === 0) {
      return res.status(400).json({
        success: false,
        message: "No valid items to process. All product IDs were invalid."
      });
    }
    console.log("Creating transaction, receiptNo:", transactionData.receiptNo, "items:",
transactionData.items.length);
    const transaction = await SalesTransaction.create(transactionData);
```

```
    res.status(201).json({
      success: true,
      message: "Transaction created successfully",
      data: transaction
    });
    if (appliedDiscountIds && Array.isArray(appliedDiscountIds) && appliedDiscountIds.length > 0) {
      Promise.all(appliedDiscountIds.map(
        (id) => Discount.findByIdAndUpdate(id, { $inc: { usageCount: 1 } })
      )).catch((err) => console.error("Error updating discount usage counts:", err));
    }
  } catch (error) {
    console.error("Error creating transaction:", error);
    console.error("Error details:", error.errors || error.message);
    res.status(500).json({
      success: false,
      message: "Failed to create transaction",
      error: error.message,
      details: error.errors ? Object.keys(error.errors).map((key) => ({
        field: key,
        message: error.errors[key].message
      })) : null
    });
  }
};
exports.voidTransaction = async (req, res) => {
  try {
    const { id } = req.params;
    const { voidReason, voidedBy, voidedById, voidedByName } = req.body;
    if (!voidReason || !voidedByName) {
      return res.status(400).json({
        success: false,
        message: "Void reason and voided by name are required"
      });
    }
    const transaction = await SalesTransaction.findById(id);
    if (!transaction) {
      return res.status(404).json({
        success: false,
        message: "Transaction not found"
      });
    }
    if (transaction.status === "Voided") {
      return res.status(400).json({
        success: false,
        message: "Transaction is already voided"
      });
    }
    const voidId = await generateVoidId();
    transaction.status = "voided";
    transaction.voidId = voidId;
    transaction.voidReason = voidReason;
    transaction.voidedBy = voidedBy || voidedById;
    transaction.voidedById = voidedById || voidedBy;
    transaction.voidedByName = voidedByName;
    transaction.voidedAt = /* @__PURE__ */ new Date();
    await transaction.save();
    await VoidLog.create({
      voidId,
      items: transaction.items.map((item) => ({
        productId: item.productId,
        itemName: item.itemName,
        sku: item.sku,
        variant: item.variant,
        selectedSize: item.selectedSize,
        quantity: item.quantity,
        price: item.price,
        itemImage: item.itemImage,
        voidReason
      })),
      totalAmount: transaction.totalAmount,
      voidReason,
      voidedBy: voidedBy || voidedById,
      voidedById: voidedById || voidedBy,
      voidedByName,
      voidedAt: /* @__PURE__ */ new Date(),
      originalTransactionId: transaction._id,
      source: "transaction"
```

```
    });
    for (const item of transaction.items) {
      try {
        const product = await Product.findById(item.productId);
        if (product) {
          if (product.sizes && item.selectedSize) {
            const sizeKey = Object.keys(product.sizes).find(
              (key) => key.toLowerCase() === item.selectedSize.toLowerCase()
            );
            if (sizeKey) {
              const sizeData = product.sizes[sizeKey];
              if (typeof sizeData === "object" && sizeData.quantity !== void 0) {
                product.sizes[sizeKey].quantity += item.quantity;
              } else {
                product.sizes[sizeKey] = (sizeData || 0) + item.quantity;
              }
              product.markModified("sizes");
            }
          }
          product.currentStock += item.quantity;
          product.lastUpdated = Date.now();
          await product.save();
        }
      } catch (stockError) {
        console.error("Error restoring stock for item:", item.itemName, stockError);
      }
    }
    res.json({
      success: true,
      message: "Transaction voided successfully",
      data: transaction
    });
  } catch (error) {
    console.error("Error voiding transaction:", error);
    res.status(500).json({
      success: false,
      message: "Failed to void transaction",
      error: error.message
    });
  }
};
exports.returnItems = async (req, res) => {
  try {
    const { id } = req.params;
    const { items, returnReason, returnedBy, returnedById, returnedByName } = req.body;
    if (!items || !Array.isArray(items) || items.length === 0) {
      return res.status(400).json({
        success: false,
        message: "Items to return are required"
      });
    }
    const originalTransaction = await SalesTransaction.findById(id);
    if (!originalTransaction) {
      return res.status(404).json({
        success: false,
        message: "Original transaction not found"
      });
    }
    let returnAmount = 0;
    const returnItems = items.map((item) => {
      const originalItem = originalTransaction.items.find(
        (oi) => oi.productId?.toString() === item.productId?.toString() && oi.selectedSize ===
item.selectedSize
      );
      const price = originalItem?.price || item.price || 0;
      returnAmount += price * item.quantity;
      return {
        productId: item.productId,
        itemName: item.itemName,
        sku: item.sku,
        variant: item.variant,
        selectedSize: item.selectedSize,
        quantity: item.quantity,
        price,
        itemImage: item.itemImage,
        returnReason: item.returnReason || returnReason
      };
```

```
    });
    const receiptNo = await generateUniqueReceiptNumber();
    const returnTransaction = await SalesTransaction.create({
      userId: String(returnedById || returnedBy || originalTransaction.userId || "guest"),
      performedById: String(returnedById || returnedBy || ""),
      performedByName: returnedByName || "",
      receiptNo,
      items: returnItems,
      subtotal: -returnAmount,
      discount: 0,
      totalAmount: -returnAmount,
      paymentMethod: "return",
      amountPaid: 0,
      change: returnAmount,
      cashierName: returnedByName || "",
      cashierId: returnedById || returnedBy || "",
      status: "Completed",
      originalTransactionId: originalTransaction._id,
      returnReason,
      returnedBy: returnedBy || returnedById,
      returnedById: returnedById || returnedBy,
      returnedByName
    });
    originalTransaction.hasReturns = true;
    originalTransaction.returnTransactionIds = originalTransaction.returnTransactionIds || [];
    originalTransaction.returnTransactionIds.push(returnTransaction._id);
    await originalTransaction.save();
    for (const item of returnItems) {
      const rr = item.returnReason || returnReason;
      if (isArchiveReturnReason(rr)) {
        continue;
      }
      try {
        const product = await Product.findById(item.productId);
        if (product) {
          if (product.sizes && item.selectedSize) {
            const sizeKey = Object.keys(product.sizes).find(
              (key) => key.toLowerCase() === item.selectedSize.toLowerCase()
            );
            if (sizeKey) {
              const sizeData = product.sizes[sizeKey];
              if (typeof sizeData === "object" && sizeData.quantity !== void 0) {
                product.sizes[sizeKey].quantity += item.quantity;
              } else {
                product.sizes[sizeKey] = (sizeData || 0) + item.quantity;
              }
              product.markModified("sizes");
            }
          }
          product.currentStock += item.quantity;
          product.lastUpdated = Date.now();
          await product.save();
        }
      } catch (stockError) {
        console.error("Error restoring stock for returned item:", item.itemName, stockError);
      }
    }
    res.json({
      success: true,
      message: "Items returned successfully",
      data: {
        returnTransaction,
        originalTransaction,
        returnAmount
      }
    });
  } catch (error) {
    console.error("Error processing return:", error);
    res.status(500).json({
      success: false,
      message: "Failed to process return",
      error: error.message
    });
  }
};
exports.getTransactionStats = async (req, res) => {
  try {
```

```
    const { startDate, endDate } = req.query;
    const query = { status: "completed", paymentMethod: { $ne: "return" } };
    if (startDate || endDate) {
      query.createdAt = {};
      if (startDate) query.createdAt.$gte = new Date(startDate);
      if (endDate) query.createdAt.$lte = new Date(endDate);
    }
    const [totalTransactions, totalSales, byPaymentMethod] = await Promise.all([
      SalesTransaction.countDocuments(query),
      SalesTransaction.aggregate([
        { $match: query },
        { $group: { _id: null, total: { $sum: "$totalAmount" } } }
      ]),
      SalesTransaction.aggregate([
        { $match: query },
        { $group: { _id: "$paymentMethod", count: { $sum: 1 }, total: { $sum: "$totalAmount" } } }
      ])
    ]);
    res.json({
      success: true,
      data: {
        totalTransactions,
        totalSales: totalSales[0]?.total || 0,
        byPaymentMethod: byPaymentMethod.reduce((acc, item) => {
          acc[item._id] = { count: item.count, total: item.total };
          return acc;
        }, {})
      }
    });
  } catch (error) {
    console.error("Error fetching transaction stats:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch transaction statistics",
      error: error.message
    });
  }
};
exports.getDashboardStats = async (req, res) => {
  try {
    const { timeframe = "daily", startDate: customStartDate, endDate: customEndDate } = req.query;
    const now = /* @__PURE__ */ new Date();
    const getDateRange = (tf, date = /* @__PURE__ */ new Date()) => {
      let start, end;
      const d = new Date(date);
      if (tf.toLowerCase() === "custom" && customStartDate && customEndDate) {
        start = new Date(customStartDate);
        start.setHours(0, 0, 0, 0);
        end = new Date(customEndDate);
        end.setHours(23, 59, 59, 999);
        return { start, end };
      }
      switch (tf.toLowerCase()) {
        case "daily":
          start = new Date(d);
          start.setHours(0, 0, 0, 0);
          end = new Date(start);
          end.setDate(end.getDate() + 1);
          break;
        case "weekly":
          start = new Date(d);
          const day = start.getDay();
          start.setDate(start.getDate() - day);
          start.setHours(0, 0, 0, 0);
          end = new Date(start);
          end.setDate(end.getDate() + 7);
          break;
        case "monthly":
          start = new Date(d.getFullYear(), d.getMonth(), 1);
          start.setHours(0, 0, 0, 0);
          end = new Date(d.getFullYear(), d.getMonth() + 1, 1);
          break;
        case "yearly":
          start = new Date(d.getFullYear(), 0, 1);
          start.setHours(0, 0, 0, 0);
          end = new Date(d.getFullYear() + 1, 0, 1);
          break;
```

```
    default:
      start = new Date(d);
      start.setHours(0, 0, 0, 0);
      end = new Date(start);
      end.setDate(end.getDate() + 1);
  }
  return { start, end };
};
const current = getDateRange(timeframe, now);
let previousDate = new Date(now);
let previous = { start: /* @__PURE__ */ new Date(), end: /* @__PURE__ */ new Date() };
if (timeframe.toLowerCase() === "custom" && customStartDate && customEndDate) {
  const duration = current.end.getTime() - current.start.getTime();
  const previousEnd = new Date(current.start.getTime() - 1);
  const previousStart = new Date(previousEnd.getTime() - duration);
  previous = { start: previousStart, end: previousEnd };
} else {
  if (timeframe === "daily") previousDate.setDate(previousDate.getDate() - 1);
  else if (timeframe === "weekly") previousDate.setDate(previousDate.getDate() - 7);
  else if (timeframe === "monthly") previousDate.setMonth(previousDate.getMonth() - 1);
  else if (timeframe === "yearly") previousDate.setFullYear(previousDate.getFullYear() - 1);
  previous = getDateRange(timeframe, previousDate);
}
const buildMatchFilter = (start, end) => ({
  $or: [
    { checkedOutAt: { $gte: start, $lte: end } },
    { checkedOutAt: { $exists: false }, createdAt: { $gte: start, $lte: end } }
  ],
  status: { $not: { $regex: /^voided$/i } },
  paymentMethod: { $ne: "return" }
});
const getStatsAggregation = (start, end) => {
  return SalesTransaction.aggregate([
    { $match: buildMatchFilter(start, end) },
    // Compute total sales and count at transaction level
    {
      $facet: {
        summary: [
          {
            $group: {
              _id: null,
              totalSales: { $sum: { $ifNull: ["$totalAmount", 0] } },
              totalTransactions: { $sum: 1 }
            }
          }
        ],
        profit: [
          { $unwind: { path: "$items", preserveNullAndEmptyArrays: false } },
          {
            $lookup: {
              from: "products",
              localField: "items.productId",
              foreignField: "_id",
              as: "productInfo",
              pipeline: [{ $project: { costPrice: 1, sizes: 1 } }]
            }
          },
          { $unwind: { path: "$productInfo", preserveNullAndEmptyArrays: true } },
          {
            $addFields: {
              sizeData: {
                $let: {
                  vars: {
                    matchedSize: {
                      $first: {
                        $filter: {
                          input: {
                            $objectToArray: { $ifNull: ["$productInfo.sizes", {}] }
                          },
                          as: "sz",
                          cond: { $eq: ["$$sz.k", "$items.selectedSize"] }
                        }
                      }
                    }
                  },
                  in: "$$matchedSize.v"
                }
```

```
        }
      }
    },
    {
      $addFields: {
        variantCostPrice: {
          $let: {
            vars: {
              matchedVariantCostFromVariants: {
                $first: {
                  $filter: {
                    input: {
                      $objectToArray: { $ifNull: ["$sizeData.variants", {}] }
                    },
                    as: "vn",
                    cond: { $eq: ["$$vn.k", "$items.variant"] }
                  }
                }
              },
              firstVariantCostFromVariants: {
                $arrayElemAt: [
                  {
                    $objectToArray: { $ifNull: ["$sizeData.variants", {}] }
                  },
                  0
                ]
              },
              matchedVariantCostFromLegacyMap: {
                $first: {
                  $filter: {
                    input: {
                      $objectToArray: { $ifNull: ["$sizeData.variantCostPrices", {}] }
                    },
                    as: "lvc",
                    cond: { $eq: ["$$lvc.k", "$items.variant"] }
                  }
                }
              }
            },
            in: {
              $ifNull: [
                "$$matchedVariantCostFromVariants.v.costPrice",
                {
                  $ifNull: [
                    "$$firstVariantCostFromVariants.v.costPrice",
                    "$$matchedVariantCostFromLegacyMap.v"
                  ]
                }
              ]
            }
          }
        },
        sizeCostPrice: {
          $cond: [
            { $eq: [{ $type: "$sizeData" }, "object"] },
            { $ifNull: ["$sizeData.costPrice", null] },
            null
          ]
        },
        itemCostPrice: {
          $let: {
            vars: {
              computedCost: {
                $ifNull: [
                  "$variantCostPrice",
                  { $ifNull: ["$sizeCostPrice", { $ifNull: ["$productInfo.costPrice", 0] }] }
                ]
              },
              firstSizeEntry: {
                $arrayElemAt: [
                  { $objectToArray: { $ifNull: ["$productInfo.sizes", {}] } },
                  0
                ]
              }
            },
            in: {
              $cond: [
```

```
                    { $gt: ["$$computedCost", 0] },
                    "$$computedCost",
                    {
                      $let: {
                        vars: {
                          firstVariantEntry: {
                            $arrayElemAt: [
                              {
                                $objectToArray: {
                                  $ifNull: ["$$firstSizeEntry.v.variants", {}]
                                }
                              },
                              0
                            ]
                          }
                        },
                        in: {
                          $ifNull: [
                            "$$firstVariantEntry.v.costPrice",
                            {
                              $ifNull: [
                                "$$firstSizeEntry.v.costPrice",
                                { $ifNull: ["$productInfo.costPrice", 0] }
                              ]
                            }
                          ]
                        }
                      }
                    }
                  ]
                }
              }
            }
          }
          },
          {
            $addFields: {
              effectiveSoldQty: {
                $cond: [
                  { $eq: ["$items.returnStatus", "Returned"] },
                  0,
                  { $ifNull: ["$items.quantity", 1] }
                ]
              }
            }
          },
          {
            $group: {
              _id: null,
              totalProfit: {
                $sum: {
                  $multiply: [
                    { $subtract: [{ $ifNull: ["$items.price", 0] }, "$itemCostPrice"] },
                    "$effectiveSoldQty"
                  ]
                }
              }
            }
          }
        ]
      }
    }
  ]);
};
const [currentResult, previousResult, lowStockCountResult] = await Promise.all([
  getStatsAggregation(current.start, current.end),
  getStatsAggregation(previous.start, previous.end),
  Product.aggregate([
    {
      $match: {
        isArchived: { $ne: true },
        $expr: {
          $lte: ["$currentStock", { $max: [{ $ifNull: ["$reorderNumber", 0] }, 10] }]
        }
      }
    },
    { $count: "count" }
```

```
      ])
    ]);
    const currentSummary = currentResult[0]?.summary[0] || {};
    const currentProfit = currentResult[0]?.profit[0] || {};
    const totalSalesToday = currentSummary.totalSales || 0;
    const totalTransactions = currentSummary.totalTransactions || 0;
    const profit = currentProfit.totalProfit || 0;
    const previousSummary = previousResult[0]?.summary[0] || {};
    const totalSalesPrevious = previousSummary.totalSales || 0;
    const totalTransactionsPrevious = previousSummary.totalTransactions || 0;
    let growthRate = 0;
    if (totalSalesPrevious > 0) {
      growthRate = (totalSalesToday - totalSalesPrevious) / totalSalesPrevious * 100;
    } else if (totalSalesToday > 0) {
      growthRate = 100;
    }
    let transactionGrowthRate = 0;
    if (totalTransactionsPrevious > 0) {
      transactionGrowthRate = (totalTransactions - totalTransactionsPrevious) / totalTransactionsPrevious *
100;
    } else if (totalTransactions > 0) {
      transactionGrowthRate = 100;
    }
    const lowStockItems = lowStockCountResult.length > 0 ? lowStockCountResult[0].count : 0;
    res.json({
      success: true,
      data: {
        totalSalesToday,
        totalTransactions,
        profit,
        lowStockItems,
        growthRate: parseFloat(growthRate.toFixed(1)),
        totalSalesPrevious,
        totalTransactionsPrevious,
        transactionGrowthRate: parseFloat(transactionGrowthRate.toFixed(1))
      }
    });
  } catch (error) {
    console.error("Error fetching dashboard stats:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch dashboard statistics",
      error: error.message
    });
  }
};
exports.getSalesByCategory = async (req, res) => {
  try {
    const { startDate, endDate } = req.query;
    let matchStage = {
      status: { $nin: ["Voided", "voided"] },
      paymentMethod: { $nin: ["return", "void"] }
    };
    if (startDate && endDate) {
      matchStage.$or = [
        { checkedOutAt: { $gte: new Date(startDate), $lt: new Date(endDate) } },
        { checkedOutAt: { $exists: false }, createdAt: { $gte: new Date(startDate), $lt: new Date(endDate) } }
      ];
    }
    const salesByCategory = await SalesTransaction.aggregate([
      { $match: matchStage },
      { $unwind: "$items" },
      // Lookup product to get category
      {
        $lookup: {
          from: "products",
          localField: "items.productId",
          foreignField: "_id",
          as: "product"
        }
      },
      { $unwind: "$product" },
      {
        $group: {
          _id: "$product.category",
          totalSales: { $sum: { $multiply: ["$items.price", "$items.quantity"] } }
        }
```

```
      },
      { $sort: { totalSales: -1 } }
    ]);
    const result = salesByCategory.map((item) => ({
      name: item._id || "Uncategorized",
      value: item.totalSales
    }));
    res.json({
      success: true,
      data: result
    });
  } catch (error) {
    console.error("Error fetching sales by category:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch sales by category",
      error: error.message
    });
  }
};
exports.getTopSellingProducts = async (req, res) => {
  try {
    const { sort = "most", limit = 5, period = "daily" } = req.query;
    const now = /* @__PURE__ */ new Date();
    let startDate, endDate;
    if (period.toLowerCase() === "custom" && req.query.startDate && req.query.endDate) {
      startDate = new Date(req.query.startDate);
      startDate.setHours(0, 0, 0, 0);
      endDate = new Date(req.query.endDate);
      endDate.setHours(23, 59, 59, 999);
    } else {
      switch (period.toLowerCase()) {
        case "daily":
          startDate = new Date(now);
          startDate.setHours(0, 0, 0, 0);
          endDate = new Date(startDate);
          endDate.setDate(endDate.getDate() + 1);
          break;
        case "weekly":
          startDate = new Date(now);
          const dayOfWeek = startDate.getDay();
          startDate.setDate(startDate.getDate() - dayOfWeek);
          startDate.setHours(0, 0, 0, 0);
          endDate = new Date(startDate);
          endDate.setDate(endDate.getDate() + 7);
          break;
        case "monthly":
          startDate = new Date(now.getFullYear(), now.getMonth(), 1);
          startDate.setHours(0, 0, 0, 0);
          endDate = new Date(now.getFullYear(), now.getMonth() + 1, 1);
          break;
        case "yearly":
          startDate = new Date(now.getFullYear(), 0, 1);
          startDate.setHours(0, 0, 0, 0);
          endDate = new Date(now.getFullYear() + 1, 0, 1);
          break;
        default:
          startDate = new Date(now);
          startDate.setHours(0, 0, 0, 0);
          endDate = new Date(startDate);
          endDate.setDate(endDate.getDate() + 1);
      }
    }
    const salesData = await SalesTransaction.aggregate([
      // Only include completed transactions (not voided or returns) within the time period
      {
        $match: {
          status: { $nin: ["Voided", "voided"] },
          paymentMethod: { $nin: ["return", "void"] },
          $or: [
            { checkedOutAt: { $gte: startDate, $lt: endDate } },
            { checkedOutAt: { $exists: false }, createdAt: { $gte: startDate, $lt: endDate } }
          ]
        }
      },
      // Unwind items array to get individual product sales
      { $unwind: "$items" },
```

```
      // Group by product ID and sum quantities
      {
        $group: {
          _id: "$items.productId",
          itemName: { $first: "$items.itemName" },
          sku: { $first: "$items.sku" },
          itemImage: { $first: "$items.itemImage" },
          totalQuantitySold: { $sum: "$items.quantity" },
          totalRevenue: { $sum: { $multiply: ["$items.price", "$items.quantity"] } }
        }
      },
      // Sort by quantity sold
      { $sort: { totalQuantitySold: sort === "most" ? -1 : 1 } },
      // Limit results
      { $limit: parseInt(limit) }
    ]);
    const enrichedData = await Promise.all(
      salesData.map(async (item) => {
        let product = null;
        if (item._id) {
          product = await Product.findById(item._id).lean();
        }
        return {
          productId: item._id,
          itemName: item.itemName || product?.itemName || "Unknown Product",
          sku: item.sku || product?.sku || "-",
          itemImage: item.itemImage || product?.itemImage || "",
          category: product?.category || "-",
          totalQuantitySold: item.totalQuantitySold,
          totalRevenue: item.totalRevenue,
          currentStock: product?.currentStock || 0
        };
      })
    );
    res.json({
      success: true,
      data: enrichedData
    });
  } catch (error) {
    console.error("Error fetching top selling products:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch top selling products",
      error: error.message
    });
  }
};
exports.getSalesOverTime = async (req, res) => {
  try {
    const { timeframe = "daily", startDate: customStartDate, endDate: customEndDate } = req.query;
    let limit;
    let periodType = timeframe.toLowerCase();
    if (periodType === "custom" && customStartDate && customEndDate) {
      const start = new Date(customStartDate);
      const end = new Date(customEndDate);
      const diffTime = Math.abs(end - start);
      const diffDays = Math.ceil(diffTime / (1e3 * 60 * 60 * 24));
      if (diffDays <= 31) {
        periodType = "daily-custom";
        limit = diffDays;
      } else if (diffDays <= 180) {
        periodType = "weekly-custom";
        limit = Math.ceil(diffDays / 7);
      } else {
        periodType = "monthly-custom";
        limit = Math.ceil(diffDays / 30);
      }
    } else {
      switch (periodType) {
        case "daily":
          limit = 7;
          break;
        case "weekly":
          limit = 12;
          break;
        // 12 weeks
        case "monthly":
```

```
      limit = 6;
      break;
    // 6 months
    case "yearly":
      limit = 5;
      break;
    // 5 years
    default:
      limit = 7;
  }
}
const periods = [];
const now = /* @__PURE__ */ new Date();
if (periodType.includes("custom")) {
  const end = new Date(customEndDate);
  end.setHours(23, 59, 59, 999);
  for (let i = limit - 1; i >= 0; i--) {
    let startDate, endDate, label;
    if (periodType === "daily-custom") {
      startDate = new Date(end);
      startDate.setDate(startDate.getDate() - i);
      startDate.setHours(0, 0, 0, 0);
      endDate = new Date(startDate);
      endDate.setDate(endDate.getDate() + 1);
      label = startDate.toLocaleDateString("en-US", { month: "short", day: "numeric" });
    } else if (periodType === "weekly-custom") {
      startDate = new Date(end);
      startDate.setDate(startDate.getDate() - i * 7);
      startDate.setHours(0, 0, 0, 0);
      endDate = new Date(startDate);
      endDate.setDate(endDate.getDate() + 7);
      label = `Week of ${startDate.toLocaleDateString("en-US", { month: "short", day: "numeric" })}`;
    } else {
      startDate = new Date(end);
      startDate.setMonth(startDate.getMonth() - i);
      startDate.setDate(1);
      startDate.setHours(0, 0, 0, 0);
      endDate = new Date(startDate);
      endDate.setMonth(endDate.getMonth() + 1);
      label = startDate.toLocaleDateString("en-US", { month: "short", year: "numeric" });
    }
    periods.push({ startDate, endDate, label });
  }
} else {
  for (let i = limit - 1; i >= 0; i--) {
    let startDate, endDate, label;
    if (timeframe.toLowerCase() === "daily") {
      startDate = new Date(now);
      startDate.setDate(startDate.getDate() - i);
      startDate.setHours(0, 0, 0, 0);
      endDate = new Date(startDate);
      endDate.setDate(endDate.getDate() + 1);
      label = startDate.toLocaleDateString("en-US", { month: "short", day: "numeric" });
    } else if (timeframe.toLowerCase() === "weekly") {
      startDate = new Date(now);
      const dayOfWeek = startDate.getDay();
      startDate.setDate(startDate.getDate() - dayOfWeek - i * 7);
      startDate.setHours(0, 0, 0, 0);
      endDate = new Date(startDate);
      endDate.setDate(endDate.getDate() + 7);
      label = `Week ${startDate.getDate()}`;
    } else if (timeframe.toLowerCase() === "monthly") {
      startDate = new Date(now.getFullYear(), now.getMonth() - i, 1);
      startDate.setHours(0, 0, 0, 0);
      endDate = new Date(startDate.getFullYear(), startDate.getMonth() + 1, 1);
      label = startDate.toLocaleDateString("en-US", { month: "short" });
    } else {
      startDate = new Date(now);
      startDate.setFullYear(startDate.getFullYear() - i);
      startDate.setMonth(0, 1);
      startDate.setHours(0, 0, 0, 0);
      endDate = new Date(startDate);
      endDate.setFullYear(endDate.getFullYear() + 1);
      label = startDate.getFullYear().toString();
    }
    periods.push({ startDate, endDate, label });
  }
```

```
    }
    const globalStart = periods[0].startDate;
    const globalEnd = periods[periods.length - 1].endDate;
    const allTransactions = await SalesTransaction.find({
      $or: [
        { checkedOutAt: { $gte: globalStart, $lt: globalEnd } },
        { checkedOutAt: null, createdAt: { $gte: globalStart, $lt: globalEnd } },
        { checkedOutAt: { $exists: false }, createdAt: { $gte: globalStart, $lt: globalEnd } }
      ],
      status: { $not: { $regex: /^voided$/i } },
      paymentMethod: { $ne: "return" }
    }).select("totalAmount checkedOutAt createdAt").lean();
    const rawSalesData = periods.map(({ startDate, endDate, label }) => {
      let revenue = 0;
      for (const t of allTransactions) {
        const txDate = t.checkedOutAt || t.createdAt;
        if (txDate >= startDate && txDate < endDate) {
          revenue += t.totalAmount || 0;
        }
      }
      return { period: label, revenue };
    });
    const maxRevenue = Math.max(...rawSalesData.map((item) => item.revenue || 0));
    const dynamicTarget = maxRevenue > 0 ? maxRevenue : 1e4;
    const salesData = rawSalesData.map((item, index) => {
      let growth = 0;
      if (index > 0) {
        const prevRevenue = rawSalesData[index - 1].revenue;
        if (prevRevenue > 0) {
          growth = (item.revenue - prevRevenue) / prevRevenue * 100;
        } else if (item.revenue > 0) {
          growth = 100;
        }
      }
      return {
        ...item,
        target: Math.round(dynamicTarget),
        growth: Math.round(growth)
      };
    });
    res.json({
      success: true,
      data: salesData
    });
  } catch (error) {
    console.error("Error fetching sales over time:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch sales over time",
      error: error.message
    });
  }
};
```

FILE: pos/backend/controllers/voidLogController.js
==========

```
const VoidLog = require("../models/VoidLog");
const generateVoidId = async () => {
  const chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
  let voidId;
  let isUnique = false;
  let attempts = 0;
  const maxAttempts = 10;
  while (!isUnique && attempts < maxAttempts) {
    voidId = "VOID-";
    for (let i = 0; i < 6; i++) {
      voidId += chars.charAt(Math.floor(Math.random() * chars.length));
    }
    const existing = await VoidLog.findOne({ voidId });
    if (!existing) {
      isUnique = true;
    }
    attempts++;
  }
  if (!isUnique) {
```

```
    const timestamp = Date.now().toString().slice(-4);
    voidId = `VOID-${timestamp}${Math.floor(Math.random() * 100).toString().padStart(2, "0")}`;
  }
  return voidId;
};
exports.createVoidLog = async (req, res) => {
  try {
    const {
      items,
      totalAmount,
      voidReason,
      voidedBy,
      voidedById,
      voidedByName,
      approvedBy,
      approvedByRole,
      originalTransactionId,
      source,
      userId,
      notes
    } = req.body;
    if (!items || !Array.isArray(items) || items.length === 0) {
      return res.status(400).json({
        success: false,
        message: "Items are required"
      });
    }
    if (!totalAmount || !voidReason || !voidedByName) {
      return res.status(400).json({
        success: false,
        message: "Missing required fields: totalAmount, voidReason, voidedByName"
      });
    }
    const voidId = await generateVoidId();
    const voidLogData = {
      voidId,
      items: items.map((item) => ({
        productId: item.productId || item._id,
        itemName: item.itemName,
        sku: item.sku,
        variant: item.variant || item.selectedVariation || null,
        selectedSize: item.selectedSize || item.size || null,
        quantity: item.quantity,
        price: item.itemPrice || item.price || 0,
        itemImage: item.itemImage || "",
        voidReason: item.voidReason || voidReason
      })),
      totalAmount,
      voidReason,
      voidedBy: voidedBy || voidedById || "",
      voidedById: voidedById || voidedBy || "",
      voidedByName,
      approvedBy: approvedBy || null,
      approvedByRole: approvedByRole || null,
      voidedAt: /* @__PURE__ */ new Date(),
      originalTransactionId: originalTransactionId || null,
      source: source || "cart",
      userId: userId || null,
      notes: notes || ""
    };
    const voidLog = await VoidLog.create(voidLogData);
    res.json({
      success: true,
      message: "Void log created successfully",
      data: voidLog
    });
  } catch (error) {
    console.error("Error creating void log:", error);
    res.status(500).json({
      success: false,
      message: "Failed to create void log",
      error: error.message
    });
  }
};
exports.getVoidLogs = async (req, res) => {
  try {
```

```
    const {
      search,
      voidedBy,
      source,
      startDate,
      endDate,
      limit = 1e3,
      page = 1,
      sortBy = "voidedAt",
      sortOrder = "desc"
    } = req.query;
    const query = {};
    if (search) {
      query.$or = [
        { voidId: { $regex: search, $options: "i" } },
        { "items.itemName": { $regex: search, $options: "i" } },
        { "items.sku": { $regex: search, $options: "i" } },
        { voidedByName: { $regex: search, $options: "i" } }
      ];
    }
    if (voidedBy) {
      query.voidedById = voidedBy;
    }
    if (source) {
      query.source = source;
    }
    if (startDate || endDate) {
      query.voidedAt = {};
      if (startDate) {
        query.voidedAt.$gte = new Date(startDate);
      }
      if (endDate) {
        query.voidedAt.$lte = new Date(endDate);
      }
    }
    const skip = (parseInt(page) - 1) * parseInt(limit);
    const sort = { [sortBy]: sortOrder === "desc" ? -1 : 1 };
    const [voidLogs, total] = await Promise.all([
      VoidLog.find(query).sort(sort).skip(skip).limit(parseInt(limit)).lean(),
      VoidLog.countDocuments(query)
    ]);
    res.json({
      success: true,
      data: voidLogs,
      pagination: {
        page: parseInt(page),
        limit: parseInt(limit),
        total,
        pages: Math.ceil(total / parseInt(limit))
      }
    });
  } catch (error) {
    console.error("Error fetching void logs:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch void logs",
      error: error.message
    });
  }
};
exports.getVoidLogById = async (req, res) => {
  try {
    const { id } = req.params;
    const voidLog = await VoidLog.findOne({
      $or: [
        { _id: id },
        { voidId: id }
      ]
    }).lean();
    if (!voidLog) {
      return res.status(404).json({
        success: false,
        message: "Void log not found"
      });
    }
    res.json({
      success: true,
```

```
      data: voidLog
    });
  } catch (error) {
    console.error("Error fetching void log:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch void log",
      error: error.message
    });
  }
};
exports.getVoidLogStats = async (req, res) => {
  try {
    const { startDate, endDate } = req.query;
    const query = {};
    if (startDate || endDate) {
      query.voidedAt = {};
      if (startDate) {
        query.voidedAt.$gte = new Date(startDate);
      }
      if (endDate) {
        query.voidedAt.$lte = new Date(endDate);
      }
    }
    const [totalVoids, totalAmount, byReason, bySource] = await Promise.all([
      VoidLog.countDocuments(query),
      VoidLog.aggregate([
        { $match: query },
        { $group: { _id: null, total: { $sum: "$totalAmount" } } }
      ]),
      VoidLog.aggregate([
        { $match: query },
        { $group: { _id: "$voidReason", count: { $sum: 1 } } },
        { $sort: { count: -1 } }
      ]),
      VoidLog.aggregate([
        { $match: query },
        { $group: { _id: "$source", count: { $sum: 1 } } }
      ])
    ]);
    res.json({
      success: true,
      data: {
        totalVoids,
        totalAmount: totalAmount[0]?.total || 0,
        byReason: byReason.reduce((acc, item) => {
          acc[item._id] = item.count;
          return acc;
        }, {}),
        bySource: bySource.reduce((acc, item) => {
          acc[item._id] = item.count;
          return acc;
        }, {})
      }
    });
  } catch (error) {
    console.error("Error fetching void log stats:", error);
    res.status(500).json({
      success: false,
      message: "Failed to fetch void log statistics",
      error: error.message
    });
  }
};
```

FILE: pos/backend/debug_txn_dates.js
==========

```
const mongoose = require("mongoose");
const dotenv = require("dotenv");
const path = require("path");
dotenv.config({ path: path.join(__dirname, ".env") });
async function debug() {
  try {
    await mongoose.connect(process.env.MONGODB_URI);
    console.log("Connected!");
```

```
    const SalesTransaction = require("./models/SalesTransaction");
    const now = /* @__PURE__ */ new Date();
    console.log("Current Date:", now);
    console.log("Current Year:", now.getFullYear());
    const start = new Date(now.getFullYear() - 4, 0, 1);
    start.setHours(0, 0, 0, 0);
    const end = new Date(now);
    end.setHours(23, 59, 59, 999);
    console.log("\\n=== Date Range for yearly ===");
    console.log("Start:", start);
    console.log("End:", end);
    const allTxns = await SalesTransaction.find({}).sort({ createdAt: -1 }).limit(5).lean();
    console.log("\\n=== Last 5 Transactions (any) ===");
    allTxns.forEach((t) => {
      console.log("ID:", t._id);
      console.log("  createdAt:", t.createdAt);
      console.log("  checkedOutAt:", t.checkedOutAt || "NOT SET");
      console.log("  status:", t.status);
      console.log("  paymentMethod:", t.paymentMethod);
      console.log("  totalAmount:", t.totalAmount);
    });
    const query = {
      paymentMethod: { $ne: "return" },
      status: { $not: { $regex: /^void/i } }
    };
    console.log("\\n=== Simple Query ===");
    const matchingSimple = await SalesTransaction.find(query).lean();
    console.log("Matching (simple):", matchingSimple.length);
    query.$or = [
      { checkedOutAt: { $gte: start, $lte: end } },
      {
        checkedOutAt: { $exists: false },
        createdAt: { $gte: start, $lte: end }
      }
    ];
    console.log("\\n=== With Date Range ===");
    const matchingTxns = await SalesTransaction.find(query).lean();
    console.log("Matching (with dates):", matchingTxns.length);
    matchingTxns.slice(0, 3).forEach((t) => {
      console.log("ID:", t._id, "Amount:", t.totalAmount, "Created:", t.createdAt);
    });
  } catch (err) {
    console.error("Error:", err);
  } finally {
    await mongoose.disconnect();
    process.exit(0);
  }
}
debug();
```

FILE: pos/backend/middleware/networkDetection.js
==========

```
const dbManager = require("../config/databaseManager");
const mongoose = require("mongoose");
const networkDetection = async (req, res, next) => {
  try {
    if (mongoose.connection.readyState !== 1) {
      try {
        await dbManager.connect();
      } catch (error) {
        return res.status(503).json({
          success: false,
          message: "Database connection unavailable",
          error: error.message
        });
      }
    }
    req.dbManager = dbManager;
    next();
  } catch (error) {
    console.error("Database check error:", error);
    next();
  }
};
module.exports = networkDetection;
```

FILE: pos/backend/migrate_categories.js
==========

```
const mongoose = require("mongoose");
const dotenv = require("dotenv");
dotenv.config();
const Product = require("./models/Product");
const connectDB = async () => {
  try {
    const mongoUri = process.env.MONGODB_URI || process.env.MONGO_URI || "mongodb://[REDACTED]";
    if (!process.env.MONGODB_URI) {
      console.log("MongoDB URI empty, falling back to local database");
    }
    const conn = await mongoose.connect(mongoUri, {
      useNewUrlParser: true,
      useUnifiedTopology: true
    });
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error: ${error.message}`);
    process.exit(1);
  }
};
const migrate = async () => {
  await connectDB();
  console.log("Starting category migration...");
  try {
    const products = await Product.find({}).lean();
    let migratedCount = 0;
    for (const product of products) {
      if (product.genderCategory !== void 0 || product.subCategory === void 0) {
        let newCategory = product.genderCategory || "Unisex";
        if (newCategory === "Male") newCategory = "Apparel - Men";
        else if (newCategory === "Female") newCategory = "Apparel - Women";
        else if (newCategory === "Kids") newCategory = "Apparel - Kids";
        else if (newCategory === "Unisex") {
          if (product.category === "Foods") newCategory = "Foods";
          else if (product.category === "Makeup") newCategory = "Makeup";
          else newCategory = "Apparel - Unisex";
        }
        let newSubCategory = product.category || "Others";
        if (product.category === "Foods" && product.foodSubtype) {
          newSubCategory = product.foodSubtype;
        }
        console.log(`Migrating product SKU: ${product.sku} | Old: ${product.category} +
${product.genderCategory} -> New: ${newCategory} -> ${newSubCategory}`);
        await Product.updateOne({ _id: product._id }, {
          $set: { category: newCategory, subCategory: newSubCategory },
          $unset: { genderCategory: "", foodSubtype: "" }
        }, { strict: false });
        migratedCount++;
      }
    }
    console.log(`Migration complete! Successfully migrated ${migratedCount} products.`);
    process.exit(0);
  } catch (error) {
    console.error("Migration failed:", error);
    process.exit(1);
  }
};
migrate();
```

FILE: pos/backend/models/Archive.js
==========

```
const mongoose = require("mongoose");
const archiveItemSchema = new mongoose.Schema({
  productId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Product",
    required: true
  },
  itemName: {
    type: String,
```

```
    required: true,
    trim: true
  },
  sku: {
    type: String,
    required: true,
    trim: true
  },
  variant: {
    type: String,
    default: "",
    trim: true
  },
  selectedSize: {
    type: String,
    default: "",
    trim: true
  },
  category: {
    type: String,
    required: true,
    // Keep in sync with web categories (e.g. "Essentials")
    enum: ["Tops", "Bottoms", "Dresses", "Makeup", "Accessories", "Essentials", "Shoes", "Head Wear", "Foods",
"Others"]
  },
  brandName: {
    type: String,
    default: "",
    trim: true
  },
  itemPrice: {
    type: Number,
    required: true,
    min: 0
  },
  costPrice: {
    type: Number,
    default: 0,
    min: 0
  },
  quantity: {
    type: Number,
    required: true,
    min: 1
  },
  itemImage: {
    type: String,
    default: ""
  },
  reason: {
    type: String,
    required: true,
    enum: ["Damaged", "Defective", "Expired", "Other"]
  },
  returnReason: {
    type: String,
    default: ""
  },
  originalTransactionId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "SalesTransaction",
    default: null
  },
  source: {
    type: String,
    enum: ["return", "stock-out"],
    default: "return"
  },
  returnTransactionId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "SalesTransaction",
    default: null
  },
  archivedBy: {
    type: String,
    required: true
  },
```

```
  archivedById: {
    type: String,
    default: ""
  },
  notes: {
    type: String,
    default: ""
  },
  archivedAt: {
    type: Date,
    default: Date.now
  }
}, {
  timestamps: true
});
archiveItemSchema.index({ archivedAt: -1 });
archiveItemSchema.index({ productId: 1 });
archiveItemSchema.index({ originalTransactionId: 1 });
archiveItemSchema.index({ sku: 1 });
module.exports.schema = archiveItemSchema;
module.exports = mongoose.model("Archive", archiveItemSchema);
```

FILE: pos/backend/models/BrandPartner.js
==========

```
const mongoose = require("mongoose");
const brandPartnerSchema = new mongoose.Schema(
  {
    brandName: {
      type: String,
      required: true,
      trim: true
    },
    email: {
      type: String,
      trim: true,
      default: ""
    },
    contactNumber: {
      type: String,
      trim: true,
      default: ""
    },
    contactPerson: {
      type: String,
      trim: true,
      default: ""
    },
    logo: {
      type: String,
      default: ""
    },
    status: {
      type: String,
      enum: ["active", "archived"],
      default: "active"
    }
  },
  {
    timestamps: true
  }
);
brandPartnerSchema.index({ brandName: 1 }, { unique: true });
module.exports.schema = brandPartnerSchema;
module.exports = mongoose.model("BrandPartner", brandPartnerSchema);
```

FILE: pos/backend/models/Cart.js
==========

```
const mongoose = require("mongoose");
const cartItemSchema = new mongoose.Schema({
  productId: {
    type: String,
    required: true
  },
```

```
  itemName: String,
  sku: String,
  itemPrice: Number,
  quantity: {
    type: Number,
    default: 1
  },
  selectedSize: String,
  variant: String,
  itemImage: String
}, { _id: false });
const cartSchema = new mongoose.Schema({
  userId: {
    type: String,
    required: true,
    unique: true
  },
  items: {
    type: [cartItemSchema],
    default: []
  }
}, {
  timestamps: true
});
cartSchema.index({ updatedAt: -1 });
module.exports.schema = cartSchema;
module.exports = mongoose.model("Cart", cartSchema);
```

FILE: pos/backend/models/Category.js
==========

```
const mongoose = require("mongoose");
const categorySchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true
  },
  type: {
    type: String,
    enum: ["category", "subcategory"],
    default: "category",
    index: true
  },
  parentCategory: {
    type: String,
    default: null,
    trim: true,
    index: true
  },
  status: {
    type: String,
    enum: ["active", "inactive"],
    default: "active"
  },
  showOnPos: {
    type: Boolean,
    default: true
  },
  dateCreated: {
    type: Date,
    default: Date.now
  }
}, {
  timestamps: true
});
categorySchema.index({ status: 1 });
categorySchema.index({ name: 1 });
categorySchema.index({ type: 1, parentCategory: 1, status: 1 });
categorySchema.index({ dateCreated: -1 });
categorySchema.index({ name: 1, type: 1, parentCategory: 1 }, { unique: true });
module.exports.schema = categorySchema;
module.exports = mongoose.model("Category", categorySchema);
```

FILE: pos/backend/models/Discount.js
==========

```
const mongoose = require("mongoose");
const discountSchema = new mongoose.Schema({
  title: {
    type: String,
    required: true,
    trim: true
  },
  discountCode: {
    type: String,
    default: "",
    trim: true
  },
  discountType: {
    type: String,
    enum: ["percentage", "fixed"],
    required: true,
    default: "percentage"
  },
  discountValue: {
    type: Number,
    required: true,
    min: 0
  },
  appliesTo: {
    type: String,
    enum: ["all", "category", "products"],
    required: true,
    default: "all"
  },
  category: {
    type: String,
    default: null,
    trim: true
  },
  subCategory: {
    type: String,
    default: null,
    trim: true
  },
  productIds: {
    type: [mongoose.Schema.Types.ObjectId],
    ref: "Product",
    default: []
  },
  validFrom: {
    type: Date,
    default: null
  },
  validTo: {
    type: Date,
    default: null
  },
  noExpiration: {
    type: Boolean,
    default: false
  },
  minPurchaseAmount: {
    type: Number,
    default: 0,
    min: 0
  },
  maxPurchaseAmount: {
    type: Number,
    default: null,
    min: 0
  },
  usageLimit: {
    type: Number,
    default: null,
    min: 0
  },
  usageCount: {
    type: Number,
    default: 0,
    min: 0
```

```
  },
  status: {
    type: String,
    enum: ["active", "inactive"],
    default: "active"
  },
  description: {
    type: String,
    default: "",
    trim: true
  },
  dateCreated: {
    type: Date,
    default: Date.now
  }
}, {
  timestamps: true
});
discountSchema.index({ status: 1 });
discountSchema.index({ appliesTo: 1 });
discountSchema.index({ category: 1 });
discountSchema.index({ subCategory: 1 });
discountSchema.index({ dateCreated: -1 });
discountSchema.index({ discountCode: 1 });
module.exports.schema = discountSchema;
module.exports = mongoose.model("Discount", discountSchema);
```

FILE: pos/backend/models/Employee.js
==========

```
const mongoose = require("mongoose");
const bcrypt = require("bcryptjs");
const employeeSchema = new mongoose.Schema({
  firstName: {
    type: String,
    trim: true,
    required: true
  },
  middleInitial: {
    type: String,
    trim: true,
    default: ""
  },
  lastName: {
    type: String,
    trim: true,
    default: ""
  },
  name: {
    type: String,
    required: true,
    trim: true
  },
  contactNo: {
    type: String,
    required: true,
    trim: true
  },
  email: {
    type: String,
    required: true,
    unique: true,
    trim: true,
    lowercase: true
  },
  role: {
    type: String,
    required: true,
    enum: ["Sales Clerk", "Manager", "Cashier", "Supervisor", "Owner", "Stock Manager", "Subcashier", "Sub-
Cashier"],
    default: "Sales Clerk"
  },
  pin: {
    type: String,
    required: true
  },
```

```
  fastPinHash: {
    // O(1) instantaneous login lookup
    type: String,
    default: ""
  },
  dateJoined: {
    type: Date,
    default: Date.now
  },
  dateJoinedActual: {
    type: Date,
    default: null
  },
  status: {
    type: String,
    enum: ["Active", "Inactive"],
    default: "Active"
  },
  profileImage: {
    type: String,
    default: ""
  },
  // User Access Control Permissions
  permissions: {
    posTerminal: {
      type: Boolean,
      default: true
    },
    inventory: {
      type: Boolean,
      default: false
    },
    viewTransactions: {
      type: Boolean,
      default: true
    },
    generateReports: {
      type: Boolean,
      default: false
    }
  },
  requiresPinReset: {
    type: Boolean,
    default: true
  },
  isOnline: {
    type: Boolean,
    default: false
  },
  lastLogin: {
    type: Date,
    default: null
  },
  lastActive: {
    type: Date,
    default: null
  }
}, {
  timestamps: true
});
employeeSchema.pre("save", async function(next) {
  if (!this.isModified("pin")) {
    return next();
  }
  try {
    if (!/^\d{6}$/.test(this.pin)) {
      const error = new Error("PIN must be exactly 6 digits");
      return next(error);
    }
    const salt = await bcrypt.genSalt(10);
    this.pin = await bcrypt.hash(this.pin, salt);
    const crypto = require("crypto");
    const hmacSecret = process.env.PIN_SECRET || "fallback-secret-for-pos-pin";
    this.fastPinHash = crypto.createHmac("sha256", hmacSecret).update(this.pin.toString()).digest("hex");
    next();
  } catch (error) {
    next(error);
```

```
  }
});
employeeSchema.pre("validate", function(next) {
  const normalizeMiddleInitial = (mi) => {
    if (mi === void 0 || mi === null) return "";
    const t = String(mi).trim();
    if (!t) return "";
    const match = t.match(/^([A-Za-z])\.?$/);
    if (match) return match[1].toUpperCase();
    const firstAlpha = t.replace(/[^A-Za-z]/g, "").slice(0, 1);
    return firstAlpha ? firstAlpha.toUpperCase() : "";
  };
  if (this.name && (this.isModified("name") || !this.firstName || !this.lastName)) {
    const parts = this.name.trim().split(/\s+/).filter(Boolean);
    const first = parts[0] || "";
    let middle = "";
    let last = "";
    if (parts.length === 2) {
      last = parts[1] || "";
    } else if (parts.length >= 3) {
      const maybeMiddleNorm = normalizeMiddleInitial(parts[1]);
      if (maybeMiddleNorm) {
        middle = maybeMiddleNorm;
        last = parts.slice(2).join(" ") || "";
      } else {
        last = parts.slice(1).join(" ") || "";
      }
    }
    if (!this.firstName || this.isModified("name")) this.firstName = first;
    if (!this.middleInitial || this.isModified("name")) this.middleInitial = middle;
    if (!this.lastName || this.isModified("name")) this.lastName = last;
  }
  if (this.isModified("firstName") || this.isModified("middleInitial") || this.isModified("lastName")
|| !this.name) {
    this.middleInitial = normalizeMiddleInitial(this.middleInitial);
    const nameParts = [this.firstName || ""].concat(this.middleInitial ? [this.middleInitial] :
[]).concat(this.lastName ? [this.lastName] : []).map((p) => String(p).trim()).filter(Boolean);
    this.name = nameParts.join(" ").trim();
  }
  next();
});
employeeSchema.methods.comparePin = async function(candidatePin) {
  if (!this.pin) {
    console.warn("[comparePin] Employee has no PIN field");
    return false;
  }
  if (this.pin.startsWith("$2")) {
    return await bcrypt.compare(candidatePin, this.pin);
  } else {
    console.error("[comparePin] CRITICAL: PIN is stored as plain text for employee", this._id, "\u2014
rejecting login. Re-hash this PIN immediately.");
    return false;
  }
};
employeeSchema.index({ email: 1 });
employeeSchema.index({ status: 1 });
employeeSchema.index({ role: 1 });
employeeSchema.index({ dateJoined: -1 });
employeeSchema.index({ firstName: "text", middleInitial: "text", lastName: "text", name: "text", email:
"text" });
employeeSchema.index({ fastPinHash: 1 });
employeeSchema.index({ isOnline: 1 });
module.exports.schema = employeeSchema;
module.exports = mongoose.model("Employee", employeeSchema);
```

FILE: pos/backend/models/GlobalSettings.js
==========

```
const mongoose = require("mongoose");
const openingFloatEntrySchema = new mongoose.Schema(
  {
    employeeId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Employee",
      required: true
    },
```

```
    employeeName: {
      type: String,
      required: true,
      trim: true
    },
    employeeRole: {
      type: String,
      default: "",
      trim: true
    },
    amount: {
      type: Number,
      required: true,
      min: 0
    },
    createdAt: {
      type: Date,
      default: Date.now
    },
    businessDate: {
      type: Date,
      default: Date.now
    },
    updatedAt: {
      type: Date,
      default: null
    }
  },
  { _id: true }
);
const globalSettingsSchema = new mongoose.Schema(
  {
    storeName: {
      type: String,
      default: "Create Your Style"
    },
    openingFloat: {
      type: Number,
      default: 2e3
    },
    openingFloats: {
      type: [openingFloatEntrySchema],
      default: []
    }
  },
  {
    timestamps: true
  }
);
module.exports = mongoose.model("GlobalSettings", globalSettingsSchema);
```

FILE: pos/backend/models/MerchantSettings.js
==========

```
const mongoose = require("mongoose");
const merchantSettingsSchema = new mongoose.Schema(
  {
    // Only one active config per merchant - enforced by unique index
    isActive: {
      type: Boolean,
      default: true
    },
    // GCash for Business / Payment Gateway credentials
    appId: {
      type: String,
      required: true,
      trim: true
    },
    // Encrypted at rest using AES-256-GCM — never returned to frontend after save
    encryptedPrivateKey: {
      type: String,
      required: true
    },
    privateKeyIV: {
      type: String,
      required: true
```

```
    },
    privateKeyAuthTag: {
      type: String,
      required: true
    },
    publicKey: {
      type: String,
      required: true,
      trim: true
    },
    // sandbox | production
    environment: {
      type: String,
      enum: ["sandbox", "production"],
      default: "sandbox",
      required: true
    },
    // Auto-generated webhook URL (read-only on frontend)
    webhookUrl: {
      type: String,
      default: ""
    },
    // Merchant display name for QR labels
    merchantName: {
      type: String,
      default: "POS System",
      trim: true
    },
    // Payment expiry in minutes (default 15)
    paymentExpiryMinutes: {
      type: Number,
      default: 15,
      min: 5,
      max: 60
    },
    // Who configured this
    configuredBy: {
      type: String,
      default: ""
    },
    configuredByName: {
      type: String,
      default: ""
    }
  },
  {
    timestamps: true
  }
);
merchantSettingsSchema.index(
  { isActive: 1 },
  { unique: true, partialFilterExpression: { isActive: true } }
);
merchantSettingsSchema.methods.toSafeJSON = function() {
  const obj = this.toObject();
  delete obj.encryptedPrivateKey;
  delete obj.privateKeyIV;
  delete obj.privateKeyAuthTag;
  return obj;
};
merchantSettingsSchema.statics.getActiveConfig = async function() {
  return this.findOne({ isActive: true });
};
module.exports = mongoose.model("MerchantSettings", merchantSettingsSchema);
```

FILE: pos/backend/models/Product.js
==========

```
const mongoose = require("mongoose");
const productSchema = new mongoose.Schema({
  sku: {
    type: String,
    required: true,
    unique: true,
    trim: true
  },
```

```
itemName: {
  type: String,
  required: true,
  trim: true
},
category: {
  type: String,
  required: true
},
subCategory: {
  type: String,
  required: true,
  trim: true
},
unitOfMeasure: {
  type: String,
  default: "pcs",
  trim: true
},
brandName: {
  type: String,
  default: "",
  trim: true
},
variant: {
  type: String,
  default: "",
  trim: true
},
foodSubtype: {
  type: String,
  default: "",
  trim: true
},
size: {
  type: String,
  default: "",
  trim: true
},
sizes: {
  type: mongoose.Schema.Types.Mixed,
  default: null
},
itemPrice: {
  type: Number,
  default: 0,
  min: 0
},
costPrice: {
  type: Number,
  default: 0,
  min: 0
},
currentStock: {
  type: Number,
  required: true,
  default: 0,
  min: 0
},
reorderNumber: {
  type: Number,
  default: 0,
  min: 0
},
expirationDate: {
  type: Date,
  default: null
},
expirationThresholdDays: {
  type: Number,
  default: 0,
  min: 0
},
supplierName: {
  type: String,
  default: "",
  trim: true
```

```
  },
  supplierContact: {
    type: String,
    default: "",
    trim: true
  },
  dateAdded: {
    type: Date,
    default: Date.now
  },
  lastUpdated: {
    type: Date,
    default: Date.now
  },
  itemImage: {
    type: String,
    default: ""
  },
  productImages: {
    type: [String],
    default: []
  },
  displayInTerminal: {
    type: Boolean,
    default: true
  },
  discountIds: {
    type: [mongoose.Schema.Types.ObjectId],
    default: [],
    ref: "Discount"
  },
  isArchived: {
    type: Boolean,
    default: false
  }
}, {
  timestamps: true
});
productSchema.pre("save", function(next) {
  this.lastUpdated = Date.now();
  next();
});
productSchema.virtual("terminalStatus").get(function() {
  return this.displayInTerminal !== false ? "shown" : "not shown";
});
productSchema.set("toJSON", { virtuals: true });
productSchema.set("toObject", { virtuals: true });
productSchema.index({ category: 1 });
productSchema.index({ itemName: "text", sku: "text", brandName: "text" });
productSchema.index({ dateAdded: -1 });
productSchema.index({ currentStock: 1 });
productSchema.index({ lastUpdated: -1 });
productSchema.index({ sku: 1 });
module.exports.schema = productSchema;
module.exports = mongoose.model("Product", productSchema);
```

FILE: pos/backend/models/Remittance.js
==========

```
const mongoose = require("mongoose");
const remittanceSchema = new mongoose.Schema({
  employeeId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Employee",
    required: true
  },
  employeeName: {
    type: String,
    required: true
  },
  shiftDate: {
    type: Date,
    required: true
  },
  grossSales: {
    type: Number,
```

```
    required: true,
    default: 0
  },
  returns: {
    type: Number,
    default: 0
  },
  netSales: {
    type: Number,
    required: true,
    default: 0
  },
  noOfSales: {
    type: Number,
    default: 0
  },
  denominations: {
    p1000: { type: Number, default: 0 },
    p500: { type: Number, default: 0 },
    p200: { type: Number, default: 0 },
    p100: { type: Number, default: 0 },
    p50: { type: Number, default: 0 },
    p20: { type: Number, default: 0 },
    p10: { type: Number, default: 0 },
    p5: { type: Number, default: 0 },
    p1: { type: Number, default: 0 },
    c25: { type: Number, default: 0 },
    c10: { type: Number, default: 0 },
    c5: { type: Number, default: 0 }
  },
  totalCashOnHand: {
    type: Number,
    required: true,
    default: 0
  },
  openingFloat: {
    type: Number,
    default: 2e3
  },
  cashToRemit: {
    type: Number,
    required: true,
    default: 0
  },
  expectedCash: {
    type: Number,
    default: 0
  },
  variance: {
    type: Number,
    default: 0
  },
  remarks: {
    type: String,
    default: ""
  },
  receivedBy: {
    type: String,
    default: ""
  },
  status: {
    type: String,
    enum: ["Submitted", "Reviewed"],
    default: "Submitted"
  },
  submittedAt: {
    type: Date,
    default: Date.now
  }
}, {
  timestamps: true
});
remittanceSchema.index({ employeeId: 1, shiftDate: -1 });
remittanceSchema.index({ submittedAt: -1 });
remittanceSchema.index({ status: 1 });
module.exports.schema = remittanceSchema;
module.exports = mongoose.model("Remittance", remittanceSchema);
```

FILE: pos/backend/models/SalesTransaction.js
==========

```
const mongoose = require("mongoose");
const transactionItemSchema = new mongoose.Schema(
  {
    productId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: "Product",
      required: true
    },
    itemName: String,
    sku: String,
    variant: String,
    selectedSize: String,
    quantity: {
      type: Number,
      required: true
    },
    price: {
      type: Number,
      required: true
    },
    itemImage: {
      type: String,
      default: ""
    },
    returnStatus: {
      type: String,
      enum: ["Returned", "Partially Returned", null],
      default: null
    },
    returnedQuantity: {
      type: Number,
      default: null
    },
    returnReason: {
      type: String,
      default: null
    },
    voidReason: {
      type: String,
      default: null
    },
    // FIFO batches consumed on this line (filled when stock-out runs after sale) — used to return stock to
the same batch slots
    batchAllocations: {
      type: [
        {
          createdAt: { type: String, default: "" },
          qty: { type: Number, required: true },
          price: { type: Number, default: 0 },
          costPrice: { type: Number, default: 0 }
        }
      ],
      default: void 0
    }
  },
  { _id: false }
);
const salesTransactionSchema = new mongoose.Schema(
  {
    userId: {
      type: String,
      required: true
    },
    performedById: String,
    performedByName: String,
    items: {
      type: [transactionItemSchema],
      required: true
    },
    paymentMethod: {
      type: String,
      enum: ["cash", "gcash", "void", "return"],
```

```
  required: true
},
amountReceived: Number,
changeGiven: Number,
referenceNo: String,
receiptNo: String,
totalAmount: {
  type: Number,
  required: true
},
subtotal: {
  type: Number,
  default: null
},
discount: {
  type: Number,
  default: 0
},
discountType: {
  type: String,
  default: null
},
discountValue: {
  type: Number,
  default: 0
},
appliedDiscountIds: {
  type: [{ type: mongoose.Schema.Types.ObjectId, ref: "Discount" }],
  default: void 0
},
status: {
  type: String,
  enum: ["Completed", "Returned", "Partially Returned", "Voided", "Pending", "Failed"],
  default: "Completed"
},
checkedOutAt: {
  type: Date,
  default: Date.now
},
originalTransactionId: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "SalesTransaction",
  default: null
},
voidedBy: {
  type: String,
  default: null
},
voidedByName: {
  type: String,
  default: null
},
voidedAt: {
  type: Date,
  default: null
},
voidReason: {
  type: String,
  default: null
},
voidId: {
  type: String,
  unique: true,
  sparse: true,
  index: true
},
// === GCash Automated Payment Fields ===
// Unique merchant-generated order ID for GCash payments (GCASH-<ts>-<rand>)
merchantOrderId: {
  type: String,
  sparse: true,
  index: true,
  default: null
},
// Payment gateway source ID (e.g. PayMongo source_xxx)
gcashSourceId: {
  type: String,
```

```
      default: null
    },
    // Payment gateway payment ID (after source is charged)
    gcashPaymentId: {
      type: String,
      default: null
    },
    // GCash payment lifecycle status
    gcashStatus: {
      type: String,
      enum: ["PENDING", "CHARGEABLE", "PAID", "FAILED", "EXPIRED", null],
      default: null
    },
    // URL for customer to complete GCash payment / generate QR
    gcashCheckoutUrl: {
      type: String,
      default: null
    },
    // GCash reference number from payment gateway
    gcashReference: {
      type: String,
      default: null
    },
    // Timestamp when GCash payment was confirmed
    gcashPaidAt: {
      type: Date,
      default: null
    },
    // Payment expiry time for auto-expiration cron
    gcashExpiresAt: {
      type: Date,
      default: null
    },
    // Fee charged by payment gateway
    gcashFeeAmount: {
      type: Number,
      default: null
    },
    // Net amount after gateway fees
    gcashNetAmount: {
      type: Number,
      default: null
    },
    // Webhook event ID for idempotency
    gcashWebhookEventId: {
      type: String,
      default: null
    }
  },
  {
    timestamps: true
  }
);
salesTransactionSchema.index({ checkedOutAt: -1 });
salesTransactionSchema.index({ status: 1 });
salesTransactionSchema.index({ receiptNo: 1 });
salesTransactionSchema.index({ userId: 1 });
salesTransactionSchema.index({ paymentMethod: 1 });
salesTransactionSchema.index({ referenceNo: 1 });
salesTransactionSchema.index({ createdAt: -1 });
salesTransactionSchema.index({ "items.productId": 1 });
salesTransactionSchema.index({ status: 1, paymentMethod: 1, checkedOutAt: -1 });
salesTransactionSchema.index({ status: 1, paymentMethod: 1, createdAt: -1 });
salesTransactionSchema.index({ gcashSourceId: 1 }, { sparse: true });
salesTransactionSchema.index(
  { gcashStatus: 1, gcashExpiresAt: 1 },
  { sparse: true }
);
module.exports.schema = salesTransactionSchema;
module.exports = mongoose.model("SalesTransaction", salesTransactionSchema);



FILE: pos/backend/models/StockMovement.js
==============================================================================
const mongoose = require("mongoose");
const stockMovementSchema = new mongoose.Schema({
```

```
  productId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Product",
    required: true
  },
  sku: {
    type: String,
    required: true
  },
  itemName: {
    type: String,
    required: true
  },
  itemImage: {
    type: String,
    default: ""
  },
  category: {
    type: String,
    required: true
  },
  brandName: {
    type: String,
    default: ""
  },
  type: {
    type: String,
    enum: ["Stock-In", "Stock-Out", "Pull-Out"],
    required: true
  },
  quantity: {
    type: Number,
    required: true
  },
  stockBefore: {
    type: Number,
    required: true
  },
  stockAfter: {
    type: Number,
    required: true
  },
  reason: {
    type: String,
    required: true,
    enum: ["Restock", "Sold", "Returned Item", "Return", "Exchange", "Damaged", "Lost", "Expired",
"Adjustment", "Initial Stock", "Other"]
  },
  handledBy: {
    type: String,
    required: true
  },
  handledById: {
    type: String,
    default: ""
  },
  notes: {
    type: String,
    default: ""
  },
  sizeQuantities: {
    type: mongoose.Schema.Types.Mixed,
    default: null
  }
}, {
  timestamps: true
});
stockMovementSchema.index({ createdAt: -1 });
stockMovementSchema.index({ productId: 1 });
stockMovementSchema.index({ type: 1 });
stockMovementSchema.index({ category: 1 });
stockMovementSchema.index({ sku: 1 });
stockMovementSchema.index({ handledById: 1 });
stockMovementSchema.index({ reason: 1 });
module.exports.schema = stockMovementSchema;
module.exports = mongoose.model("StockMovement", stockMovementSchema);
```

FILE: pos/backend/models/SyncLog.js
==========

```
const mongoose = require("mongoose");
const syncLogSchema = new mongoose.Schema(
  {
    entity: {
      type: String,
      required: true,
      enum: [
        "SalesTransaction",
        "Product",
        "StockMovement",
        "Employee",
        "Category",
        "Discount",
        "BrandPartner",
        "VoidLog",
        "Archive",
        "Cart"
      ],
      unique: true
    },
    lastSyncedAt: {
      type: Date,
      default: /* @__PURE__ */ new Date(0)
      // Default to epoch to sync everything initially
    },
    status: {
      type: String,
      enum: ["Success", "Failed", "InProgress"],
      default: "Success"
    },
    message: String
  },
  {
    timestamps: true
  }
);
module.exports = mongoose.model("SyncLog", syncLogSchema);
```

FILE: pos/backend/models/VerificationCode.js
==========

```
const mongoose = require("mongoose");
const verificationCodeSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    trim: true,
    lowercase: true
  },
  code: {
    type: String,
    required: true
  },
  expiresAt: {
    type: Date,
    required: true,
    default: () => new Date(Date.now() + 10 * 60 * 1e3)
    // 10 minutes expiration
  },
  createdAt: {
    type: Date,
    default: Date.now,
    expires: 600
    // TTL index: document automatically deleted after 600 seconds (10 mins)
  }
});
module.exports = mongoose.model("VerificationCode", verificationCodeSchema);
```

FILE: pos/backend/models/VoidLog.js
==========

```
const mongoose = require("mongoose");
const voidedItemSchema = new mongoose.Schema({
  productId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Product",
    required: true
  },
  itemName: {
    type: String,
    required: true
  },
  sku: {
    type: String,
    required: true
  },
  variant: {
    type: String,
    default: null
  },
  selectedSize: {
    type: String,
    default: null
  },
  quantity: {
    type: Number,
    required: true
  },
  price: {
    type: Number,
    required: true
  },
  itemImage: {
    type: String,
    default: ""
  },
  voidReason: {
    type: String,
    required: true
  }
}, { _id: false });
const voidLogSchema = new mongoose.Schema({
  voidId: {
    type: String,
    required: true,
    unique: true,
    index: true
  },
  items: {
    type: [voidedItemSchema],
    required: true
  },
  totalAmount: {
    type: Number,
    required: true
  },
  voidReason: {
    type: String,
    required: true,
    enum: ["Customer cancellation", "Wrong transaction", "System error", "Payment issue", "Other"]
  },
  voidedBy: {
    type: String,
    required: true
  },
  voidedById: {
    type: String,
    default: ""
  },
  voidedByName: {
    type: String,
    required: true
  },
  approvedBy: {
    type: String,
    default: null
  },
  approvedByRole: {
```

```
    type: String,
    enum: ["Owner", "Manager", null],
    default: null
  },
  voidedAt: {
    type: Date,
    default: Date.now,
    required: true
  },
  originalTransactionId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "SalesTransaction",
    default: null
  },
  source: {
    type: String,
    enum: ["cart", "transaction"],
    default: "cart"
  },
  userId: {
    type: String,
    default: null
  },
  notes: {
    type: String,
    default: ""
  }
}, {
  timestamps: true
});
voidLogSchema.index({ voidedAt: -1 });
voidLogSchema.index({ voidId: 1 });
voidLogSchema.index({ voidedBy: 1 });
voidLogSchema.index({ voidedById: 1 });
voidLogSchema.index({ originalTransactionId: 1 });
voidLogSchema.index({ source: 1 });
voidLogSchema.index({ createdAt: -1 });
module.exports.schema = voidLogSchema;
module.exports = mongoose.model("VoidLog", voidLogSchema);
```

FILE: pos/backend/package.json
==========

```
{
  "name": "pos-system-backend",
  "version": "1.0.0",
  "description": "Backend for POS System",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "delete-data": "node scripts/deleteDatabaseData.js",
    "delete-all": "node scripts/deleteDatabaseData.js --yes",
    "delete-all-local": "node scripts/deleteDatabaseData.js --yes --local"
  },
  "keywords": [
    "pos",
    "backend",
    "mongodb"
  ],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "@getbrevo/brevo": "^4.0.1",
    "@sendgrid/mail": "^8.1.6",
    "apicache": "^1.6.3",
    "bcryptjs": "^3.0.3",
    "body-parser": "^1.20.2",
    "compression": "^1.8.1",
    "cors": "^2.8.5",
    "dotenv": "^16.3.1",
    "express": "^4.18.2",
    "mongodb": "^7.0.0",
    "mongoose": "^8.0.0",
    "node-cron": "^4.2.1",
    "node-thermal-printer": "^4.5.0",
```

```
    "nodemailer": "^7.0.11",
    "ws": "^8.19.0"
  },
  "optionalDependencies": {
    "escpos": "^3.0.0-alpha.6",
    "escpos-usb": "^3.0.0-alpha.4",
    "printer": "^0.4.0",
    "usb": "^2.16.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

FILE: pos/backend/products.json
==========
(empty)

FILE: pos/backend/routes/archiveRoutes.js
==========

```
const express = require("express");
const apicache = require("apicache");
const router = express.Router();
const {
  createArchiveItem,
  getAllArchiveItems,
  getArchiveItemById,
  deleteArchiveItem,
  deleteAllArchiveItems,
  restoreArchiveItem
} = require("../controllers/archiveController");
const clearCache = (req, res, next) => {
  apicache.clear();
  next();
};
router.get("/", getAllArchiveItems);
router.get("/:id", getArchiveItemById);
router.post("/", createArchiveItem);
router.delete("/all", clearCache, deleteAllArchiveItems);
router.delete("/:id", clearCache, deleteArchiveItem);
router.post("/:id/restore", clearCache, restoreArchiveItem);
module.exports = router;
```

FILE: pos/backend/routes/brandPartnerRoutes.js
==========

```
const express = require("express");
const router = express.Router();
const {
  getBrandPartners,
  createBrandPartner,
  updateBrandPartner,
  deleteBrandPartner
} = require("../controllers/brandPartnerController");
router.get("/", getBrandPartners);
router.post("/", createBrandPartner);
router.put("/:id", updateBrandPartner);
router.delete("/:id", deleteBrandPartner);
module.exports = router;
```

FILE: pos/backend/routes/cartRoutes.js
==========

```
const express = require("express");
const router = express.Router();
const { getCartByUser, saveCart } = require("../controllers/cartController");
router.get("/:userId", getCartByUser);
router.put("/:userId", saveCart);
module.exports = router;
```

FILE: pos/backend/routes/categoryRoutes.js

```
================================================================================
const express = require("express");
const router = express.Router();
const {
  getAllCategories,
  getCategoryById,
  createCategory,
  updateCategory,
  deleteCategory,
  archiveCategory
} = require("../controllers/categoryController");
router.route("/").get(getAllCategories).post(createCategory);
router.route("/:id").get(getCategoryById).put(updateCategory).delete(deleteCategory);
router.patch("/:id/archive", archiveCategory);
module.exports = router;
```

FILE: pos/backend/routes/dataManagementRoutes.js

```
================================================================================
const express = require("express");
const router = express.Router();
const {
  getCollections,
  clearData,
  exportData,
  importData
} = require("../controllers/dataManagementController");
router.get("/collections", getCollections);
router.post("/clear", clearData);
router.post("/export", exportData);
router.post("/import", importData);
module.exports = router;
```

FILE: pos/backend/routes/discountRoutes.js

```
================================================================================
const express = require("express");
const router = express.Router();
const {
  getAllDiscounts,
  getDiscountById,
  createDiscount,
  updateDiscount,
  deleteDiscount
} = require("../controllers/discountController");
router.route("/").get(getAllDiscounts).post(createDiscount);
router.route("/:id").get(getDiscountById).put(updateDiscount).delete(deleteDiscount);
module.exports = router;
```

FILE: pos/backend/routes/employeeRoutes.js

```
================================================================================
const express = require("express");
const router = express.Router();
const {
  getAllEmployees,
  getEmployeeById,
  getEmployeeImage,
  createEmployee,
  updateEmployee,
  deleteEmployee,
  verifyPin,
  searchEmployees,
  updatePin,
  sendTemporaryPin,
  logoutEmployee,
  getOnlineEmployees,
  employeeHeartbeat
} = require("../controllers/employeeController");
router.route("/").get(getAllEmployees).post(createEmployee);
router.get("/search/:query", searchEmployees);
router.post("/verify-pin", verifyPin);
router.post("/logout", logoutEmployee);
router.post("/heartbeat", employeeHeartbeat);
```

```
router.get("/online", getOnlineEmployees);
router.route("/:id").get(getEmployeeById).put(updateEmployee).delete(deleteEmployee);
router.get("/:id/image", getEmployeeImage);
router.put("/:id/pin", updatePin);
router.post("/:id/send-temporary-pin", sendTemporaryPin);
module.exports = router;
```

FILE: pos/backend/routes/globalSettingsRoutes.js
==========

```
const express = require("express");
const router = express.Router();
const {
  getGlobalSettings,
  updateGlobalSettings
} = require("../controllers/globalSettingsController");
router.get("/", getGlobalSettings);
router.put("/", updateGlobalSettings);
module.exports = router;
```

FILE: pos/backend/routes/merchantSettingsRoutes.js
==========

```
const express = require("express");
const router = express.Router();
const {
  getSettings,
  saveSettings,
  deleteSettings,
  testConnection
} = require("../controllers/merchantSettingsController");
router.get("/", getSettings);
router.post("/", saveSettings);
router.delete("/", deleteSettings);
router.post("/test", testConnection);
module.exports = router;
```

FILE: pos/backend/routes/paymentRoutes.js
==========

```
const express = require("express");
const router = express.Router();
const {
  createGCashPayment,
  checkPaymentStatus,
  handleWebhook,
  cancelPayment,
  getConfigStatus
} = require("../controllers/gcashPaymentController");
router.get("/gcash/config-status", getConfigStatus);
router.post("/gcash/create", createGCashPayment);
router.get("/gcash/status/:merchantOrderId", checkPaymentStatus);
router.post("/gcash/webhook", handleWebhook);
router.post("/gcash/cancel/:merchantOrderId", cancelPayment);
router.get("/gcash/redirect", async (req, res) => {
  const { status, order } = req.query;
  if (status === "failed" && order) {
    try {
      const SalesTransaction = require("../models/SalesTransaction");
      const transaction = await SalesTransaction.findOne({
        merchantOrderId: order,
        gcashStatus: { $in: ["PENDING", "CHARGEABLE"] }
      });
      if (transaction) {
        transaction.gcashStatus = "FAILED";
        transaction.status = "Failed";
        await transaction.save();
        console.log("[GCash Redirect] Payment failed/cancelled for order:", order);
      }
    } catch (err) {
      console.error("[GCash Redirect] Error updating failed transaction:", err.message);
    }
  }
  const message = status === "success" ? "Payment successful! You can close this page and return to the
```

```
store." : "Payment was not completed. Please return to the store counter.";
  res.send(`
    <!DOCTYPE html>
    <html>
    <head>
      <meta name="viewport" content="width=device-width, initial-scale=1">
      <title>Payment ${status === "success" ? "Complete" : "Failed"}</title>
      <style>
        body { font-family: -apple-system, sans-serif; display: flex; justify-content: center; align-items:
center; min-height: 100vh; margin: 0; background: #f5f5f5; }
        .card { background: white; border-radius: 16px; padding: 40px; text-align: center; box-shadow: 0 4px
20px rgba(0,0,0,0.1); max-width: 400px; }
        .icon { font-size: 64px; margin-bottom: 16px; }
        h1 { color: ${status === "success" ? "#22c55e" : "#ef4444"}; font-size: 24px; }
        p { color: #666; line-height: 1.6; }
      </style>
    </head>
    <body>
      <div class="card">
        <div class="icon">${status === "success" ? "\u2705" : "\u274C"}</div>
        <h1>${status === "success" ? "Payment Successful!" : "Payment Not Completed"}</h1>
        <p>${message}</p>
        <p style="color:#999; font-size:12px; margin-top:20px;">Order: ${order || "N/A"}</p>
      </div>
    </body>
    </html>
  `);
});
module.exports = router;
```

FILE: pos/backend/routes/printRoutes.js
==========

```
const express = require("express");
const router = express.Router();
const { printReceipt } = require("../controllers/printController");
router.post("/receipt", printReceipt);
module.exports = router;
```

FILE: pos/backend/routes/productRoutes.js
==========

```
const express = require("express");
const router = express.Router();
const apicache = require("apicache");
const {
  getAllProducts,
  getProductById,
  createProduct,
  updateProduct,
  deleteProduct,
  getProductsByCategory,
  searchProducts,
  updateStockAfterTransaction,
  stockInProduct,
  stockOutProduct,
  toggleDisplayInTerminal,
  getInventoryStats,
  archiveProduct
} = require("../controllers/productController");
const cache = apicache.middleware;
const clearCache = (req, res, next) => {
  apicache.clear();
  next();
};
router.route("/").get(cache("30 seconds"), getAllProducts).post(clearCache, createProduct);
router.get("/search/:query", searchProducts);
router.get("/inventory-stats", getInventoryStats);
router.get("/low-stock", async (req, res) => {
  try {
    const Product = require("../models/Product");
    const MS_PER_DAY = 24 * 60 * 60 * 1e3;
    const products = await Product.find({ isArchived: { $ne: true } }).select(
      "itemName sku currentStock reorderNumber itemImage category expirationDate expirationThresholdDays
sizes"
```

```
    ).lean();
    const normalizeDateStart = (value) => {
      if (!value) return null;
      const d = new Date(value);
      if (Number.isNaN(d.getTime())) return null;
      d.setHours(0, 0, 0, 0);
      return d;
    };
    const getNearestActiveExpiration = (product) => {
      const candidates = [];
      const stock = Number(product?.currentStock || 0);
      if (product?.expirationDate && stock > 0) {
        candidates.push(product.expirationDate);
      }
      const sizes = product?.sizes;
      if (sizes && typeof sizes === "object") {
        Object.values(sizes).forEach((sizeData) => {
          if (!sizeData || typeof sizeData !== "object") return;
          if (Array.isArray(sizeData.batches)) {
            sizeData.batches.forEach((batch) => {
              if ((batch?.qty || 0) > 0 && batch?.expirationDate) {
                candidates.push(batch.expirationDate);
              }
            });
          }
          if (sizeData.variants && typeof sizeData.variants === "object") {
            Object.values(sizeData.variants).forEach((variantData) => {
              if (!variantData || typeof variantData !== "object") return;
              if (!Array.isArray(variantData.batches)) return;
              variantData.batches.forEach((batch) => {
                if ((batch?.qty || 0) > 0 && batch?.expirationDate) {
                  candidates.push(batch.expirationDate);
                }
              });
            });
          }
        });
      }
      const dates = candidates.map((value) => normalizeDateStart(value)).filter(Boolean).sort((a, b) => a -
b);
      return dates[0] || null;
    };
    const today = /* @__PURE__ */ new Date();
    today.setHours(0, 0, 0, 0);
    const alertItems = [];
    products.forEach((product) => {
      const stock = Number(product.currentStock || 0);
      const effectiveThreshold = Math.max(Number(product.reorderNumber || 0), 10);
      const baseData = {
        _id: String(product._id),
        itemName: product.itemName || "Unnamed Product",
        sku: product.sku || "",
        currentStock: stock,
        itemImage: product.itemImage || "",
        category: product.category || ""
      };
      if (stock <= effectiveThreshold) {
        alertItems.push({
          ...baseData,
          alertKey: `${String(product._id)}:stock`,
          alertType: stock === 0 ? "out_of_stock" : "low_stock",
          effectiveThreshold
        });
      }
      const nearestExpiry = getNearestActiveExpiration(product);
      if (nearestExpiry && stock > 0) {
        const daysUntilExpiration = Math.ceil(
          (nearestExpiry.getTime() - today.getTime()) / MS_PER_DAY
        );
        const expirationThresholdDays = Math.max(
          0,
          parseInt(product.expirationThresholdDays, 10) || 30
        );
        if (daysUntilExpiration < 0 || daysUntilExpiration <= expirationThresholdDays) {
          alertItems.push({
            ...baseData,
            alertKey: `${String(product._id)}:expiry`,
```

```
            alertType: daysUntilExpiration < 0 ? "expired" : "expiring_soon",
            expirationDate: nearestExpiry.toISOString().slice(0, 10),
            expirationThresholdDays,
            daysUntilExpiration
          });
        }
      }
    });
    const alertPriority = {
      expired: 0,
      out_of_stock: 1,
      expiring_soon: 2,
      low_stock: 3
    };
    alertItems.sort((a, b) => {
      const pa = alertPriority[a.alertType] ?? 99;
      const pb = alertPriority[b.alertType] ?? 99;
      if (pa !== pb) return pa - pb;
      if (a.alertType === "expiring_soon" || a.alertType === "expired") {
        return (a.daysUntilExpiration ?? Number.MAX_SAFE_INTEGER) - (b.daysUntilExpiration ??
Number.MAX_SAFE_INTEGER);
      }
      return (a.currentStock ?? 0) - (b.currentStock ?? 0);
    });
    const limitedItems = alertItems.slice(0, 50);
    res.json({
      success: true,
      count: limitedItems.length,
      data: limitedItems
    });
  } catch (error) {
    console.error("Error fetching stock alert products:", error);
    res.status(500).json({
      success: false,
      message: "Error fetching stock alert products",
      error: error.message
    });
  }
});
router.post("/update-stock", clearCache, updateStockAfterTransaction);
router.post("/:id/stock-in", clearCache, stockInProduct);
router.post("/:id/stock-out", clearCache, stockOutProduct);
router.get("/sku-stats", cache("2 minutes"), async (req, res) => {
  try {
    const Product = require("../models/Product");
    const products = await Product.find({}).select("brandName currentStock").lean();
    const brandStats = {};
    products.forEach((product) => {
      const brand = product.brandName?.trim() || "Unbranded";
      if (!brandStats[brand]) {
        brandStats[brand] = {
          brand,
          skuCount: 0,
          totalStock: 0
        };
      }
      brandStats[brand].skuCount += 1;
      brandStats[brand].totalStock += product.currentStock || 0;
    });
    const statsArray = Object.values(brandStats).sort((a, b) => b.skuCount - a.skuCount);
    res.json({
      success: true,
      data: statsArray
    });
  } catch (error) {
    console.error("Error fetching SKU stats:", error);
    res.status(500).json({
      success: false,
      message: "Error fetching SKU stats",
      error: error.message
    });
  }
});
router.get("/category/:category", getProductsByCategory);
router.route("/:id").get(getProductById).put(clearCache, updateProduct).delete(clearCache, deleteProduct);
router.patch("/:id/toggle-display", clearCache, toggleDisplayInTerminal);
router.patch("/:id/archive", clearCache, archiveProduct);
```

```
module.exports = router;
```

FILE: pos/backend/routes/remittanceRoutes.js
==========

```
const express = require("express");
const router = express.Router();
const {
  getRemittanceSummary,
  getRemittanceKpiStats,
  createRemittance,
  getRemittances
} = require("../controllers/remittanceController");
router.get("/summary", getRemittanceSummary);
router.get("/kpi-stats", getRemittanceKpiStats);
router.post("/", createRemittance);
router.get("/", getRemittances);
module.exports = router;
```

FILE: pos/backend/routes/reportRoutes.js
==========

```
const express = require("express");
const router = express.Router();
const apicache = require("apicache");
const { getInventoryAnalytics } = require("../controllers/reportController");
const cache = apicache.middleware;
router.get("/inventory-analytics", cache("1 minute"), getInventoryAnalytics);
module.exports = router;
```

FILE: pos/backend/routes/stockMovementRoutes.js
==========

```
const express = require("express");
const router = express.Router();
const apicache = require("apicache");
const {
  createStockMovement,
  getStockMovements,
  getTodayStats,
  getMovementsByProduct,
  getStockStatsOverTime
} = require("../controllers/stockMovementController");
const cache = apicache.middleware;
const clearCache = (req, res, next) => {
  apicache.clear();
  next();
};
router.post("/", clearCache, createStockMovement);
router.get("/", cache("30 seconds"), getStockMovements);
router.get("/stats/today", cache("30 seconds"), getTodayStats);
router.get("/stats/over-time", cache("1 minute"), getStockStatsOverTime);
router.get("/product/:productId", getMovementsByProduct);
module.exports = router;
```

FILE: pos/backend/routes/syncRoutes.js
==========

```
const express = require("express");
const dataSyncService = require("../services/dataSyncService");
const router = express.Router();
router.post("/all", async (req, res) => {
  try {
    console.log("[Sync Route] Manual sync triggered...");
    const startTime = Date.now();
    await dataSyncService.sync();
    const duration = ((Date.now() - startTime) / 1e3).toFixed(1);
    console.log(`[Sync Route] Manual sync completed in ${duration}s`);
    return res.json({
      success: true,
      message: `Data synchronized successfully (${duration}s)`,
      syncedAt: (/* @__PURE__ */ new Date()).toISOString(),
      duration: `${duration}s`
```

```
    });
  } catch (error) {
    console.error("Sync /all failed:", error);
    return res.status(500).json({
      success: false,
      message: error.message || "Sync failed"
    });
  }
});
module.exports = router;
```

FILE: pos/backend/routes/transactionRoutes.js
==========

```
const express = require("express");
const router = express.Router();
const apicache = require("apicache");
const {
  getAllTransactions,
  getTransactionById,
  updateTransaction,
  createTransaction,
  voidTransaction,
  returnItems,
  getTransactionStats,
  getDashboardStats,
  getTopSellingProducts,
  getSalesOverTime,
  getSalesByCategory
} = require("../controllers/transactionController");
const cache = apicache.middleware;
const clearCache = (req, res, next) => {
  apicache.clear();
  next();
};
router.get("/", cache("30 seconds"), getAllTransactions);
router.get("/stats", cache("2 minutes"), getTransactionStats);
router.get("/dashboard/stats", cache("2 minutes"), getDashboardStats);
router.get("/top-selling", cache("2 minutes"), getTopSellingProducts);
router.get("/sales-over-time", cache("2 minutes"), getSalesOverTime);
router.get("/sales-by-category", cache("2 minutes"), getSalesByCategory);
router.post("/", clearCache, createTransaction);
router.get("/:id", getTransactionById);
router.put("/:id", clearCache, updateTransaction);
router.post("/:id/void", clearCache, voidTransaction);
router.post("/:id/return", clearCache, returnItems);
module.exports = router;
```

FILE: pos/backend/routes/verificationRoutes.js
==========

```
const express = require("express");
const router = express.Router();
const VerificationCode = require("../models/VerificationCode");
const { sendEmail } = require("../utils/emailService");
const generateCode = () => {
  return Math.floor(1e5 + Math.random() * 9e5).toString();
};
router.post("/send-code", async (req, res) => {
  const { email } = req.body;
  if (!email) {
    return res.status(400).json({ success: false, message: "Email is required" });
  }
  try {
    await VerificationCode.deleteMany({ email });
    const code = generateCode();
    await VerificationCode.create({
      email,
      code
    });
    const subject = "Your Verification Code";
    const text = `Your verification code is: ${code}. It will expire in 10 minutes.`;
    const html = `
      <div style="font-family: Arial, sans-serif; padding: 20px; color: #333;">
        <h2 style="color: #8B7355;">Verification Code</h2>
```

```
        <p>Your verification code is:</p>
        <h1 style="font-size: 32px; letter-spacing: 5px; color: #2D2D2D;">${code}</h1>
        <p>This code will expire in 10 minutes.</p>
        <p>If you didn't request this code, please ignore this email.</p>
      </div>
    `;
    const emailResult = await sendEmail(email, subject, text, html);
    if (emailResult.success) {
      res.json({ success: true, message: "Verification code sent successfully" });
    } else {
      await VerificationCode.deleteMany({ email });
      res.status(500).json({ success: false, message: "Failed to send email" });
    }
  } catch (error) {
    console.error("Error in send-code:", error);
    res.status(500).json({ success: false, message: "Server error" });
  }
});
router.post("/verify-code", async (req, res) => {
  const { email, code } = req.body;
  if (!email || !code) {
    return res.status(400).json({ success: false, message: "Email and code are required" });
  }
  try {
    const record = await VerificationCode.findOne({ email });
    if (!record) {
      return res.status(400).json({ success: false, message: "Invalid or expired verification code" });
    }
    if (record.code !== code) {
      return res.status(400).json({ success: false, message: "Invalid verification code" });
    }
    await VerificationCode.deleteOne({ _id: record._id });
    res.json({ success: true, message: "Email verified successfully" });
  } catch (error) {
    console.error("Error in verify-code:", error);
    res.status(500).json({ success: false, message: "Server error" });
  }
});
module.exports = router;
```

FILE: pos/backend/routes/voidLogRoutes.js
==========

```
const express = require("express");
const router = express.Router();
const {
  createVoidLog,
  getVoidLogs,
  getVoidLogById,
  getVoidLogStats
} = require("../controllers/voidLogController");
router.post("/", createVoidLog);
router.get("/", getVoidLogs);
router.get("/stats", getVoidLogStats);
router.get("/:id", getVoidLogById);
module.exports = router;
```

FILE: pos/backend/scripts/syncEmployees.js
==========

```
const http = require("http");
const options = {
  hostname: "localhost",
  port: 5e3,
  path: "/api/sync-employees",
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  }
};
console.log("Syncing employees from Atlas to local database...");
console.log("========================================");
const req = http.request(options, (res) => {
  let data = "";
  res.on("data", (chunk) => {
```

```
    data += chunk;
  });
  res.on("end", () => {
    try {
      const result = JSON.parse(data);
      if (result.success) {
        console.log("\n\u2713 Sync completed successfully!");
        console.log(`  - New employees synced: ${result.synced}`);
        console.log(`  - Employees updated: ${result.updated}`);
        console.log(`  - Errors: ${result.errors}`);
        console.log(`  - Total employees in Atlas: ${result.total}`);
        console.log(`  - Timestamp: ${result.timestamp}`);
      } else {
        console.error("\n\u2717 Sync failed:", result.message);
        if (result.error) {
          console.error("  Error:", result.error);
        }
      }
    } catch (error) {
      console.error("\n\u2717 Failed to parse response:", error.message);
      console.log("Raw response:", data);
    }
  });
});
req.on("error", (error) => {
  console.error("\n\u2717 Request failed:", error.message);
  console.error("\nMake sure the backend server is running on port 5000");
  console.error("Start the server with: npm run dev (or npm start)");
});
req.end();
```

FILE: pos/backend/server.js
==========

```
const express = require("express");
const dotenv = require("dotenv");
dotenv.config();
const http = require("http");
const { WebSocketServer } = require("ws");
const cors = require("cors");
const compression = require("compression");
const connectDB = require("./config/database");
const networkDetection = require("./middleware/networkDetection");
const { initStockAlertCron } = require("./services/stockAlertCron");
const {
  initPaymentExpiryCron,
  setWsClients: setExpiryCronWsClients
} = require("./services/paymentExpiryCron");
const {
  setWsClients: setPaymentControllerWsClients
} = require("./controllers/gcashPaymentController");
const app = express();
const server = http.createServer(app);
const wss = new WebSocketServer({ server, path: "/ws/payments" });
const wsPaymentClients = /* @__PURE__ */ new Map();
wss.on("connection", (ws, req) => {
  const url = new URL(req.url, `http://${req.headers.host}`);
  const merchantOrderId = url.searchParams.get("orderId");
  if (!merchantOrderId) {
    ws.close(4e3, "Missing orderId parameter");
    return;
  }
  if (!wsPaymentClients.has(merchantOrderId)) {
    wsPaymentClients.set(merchantOrderId, /* @__PURE__ */ new Set());
  }
  wsPaymentClients.get(merchantOrderId).add(ws);
  console.log(
    `[WS] Client connected for order: ${merchantOrderId} (${wsPaymentClients.get(merchantOrderId).size}
clients)`
  );
  ws.send(
    JSON.stringify({
      type: "CONNECTED",
      merchantOrderId,
      message: "Listening for payment updates"
    })
```

```
  );
  ws.on("close", () => {
    const clients = wsPaymentClients.get(merchantOrderId);
    if (clients) {
      clients.delete(ws);
      if (clients.size === 0) {
        wsPaymentClients.delete(merchantOrderId);
      }
    }
    console.log(`[WS] Client disconnected for order: ${merchantOrderId}`);
  });
  ws.on("error", (err) => {
    console.error(`[WS] Error for order ${merchantOrderId}:`, err.message);
  });
});
setPaymentControllerWsClients(wsPaymentClients);
setExpiryCronWsClients(wsPaymentClients);
connectDB();
const allowedOrigins = [
  "http://localhost:5173",
  // Vite dev server
  "http://localhost:3000",
  // Alternate dev port
  "http://localhost:8081",
  // Expo Web dev server (default)
  "http://localhost:19006",
  // Expo Web legacy dev port
  "http://127.0.0.1:5173",
  "http://127.0.0.1:3000",
  "http://127.0.0.1:8081",
  "http://127.0.0.1:19006",
  "https://createyourstyle.me",
  // Explicitly added to prevent CORS blocks
  process.env.FRONTEND_URL,
  // Production frontend URL (set in .env)
  process.env.WEBHOOK_BASE_URL
  // ngrok tunnel URL
].filter(Boolean);
const allowedOriginPatterns = [
  /\.vercel\.app$/,
  // All Vercel deployments
  /\.onrender\.com$/
  // Render deployments
];
app.use(
  cors((req, callback) => {
    const origin = req.header("Origin");
    const requestLabel = `${req.method} ${req.originalUrl}`;
    if (!origin) {
      console.log(`[CORS] Allowed (no origin): ${requestLabel}`);
      return callback(null, { origin: true, credentials: true });
    }
    if (allowedOrigins.some((allowed) => origin.startsWith(allowed))) {
      console.log(
        `[CORS] Allowed (exact match): ${origin} -> ${requestLabel}`
      );
      return callback(null, { origin: true, credentials: true });
    }
    if (allowedOriginPatterns.some((pattern) => pattern.test(origin))) {
      console.log(
        `[CORS] Allowed (pattern match): ${origin} -> ${requestLabel}`
      );
      return callback(null, { origin: true, credentials: true });
    }
    console.warn(`[CORS] Blocked: ${origin} -> ${requestLabel}`);
    return callback(new Error("Not allowed by CORS"), {
      origin: false,
      credentials: true
    });
  })
);
app.use(compression());
app.use(express.json({ limit: "50mb" }));
app.use(express.urlencoded({ extended: true, limit: "50mb" }));
app.set("etag", "strong");
app.use(networkDetection);
app.get("/", (req, res) => {
```

```
  const dbManager = require("./config/databaseManager");
  res.json({
    message: "Welcome to POS System API",
    database: `${dbManager.getCurrentMode()} MongoDB`
  });
});
app.get("/api/ping", (req, res) => {
  res.json({ ok: true });
});
app.use("/api/products", require("./routes/productRoutes"));
app.use("/api/print", require("./routes/printRoutes"));
const employeeRoutes = require("./routes/employeeRoutes");
const verificationRoutes = require("./routes/verificationRoutes");
app.use("/api/employees", employeeRoutes);
app.use("/api/verification", verificationRoutes);
app.use("/api/cart", require("./routes/cartRoutes"));
app.use("/api/transactions", require("./routes/transactionRoutes"));
app.use("/api/stock-movements", require("./routes/stockMovementRoutes"));
app.use("/api/archive", require("./routes/archiveRoutes"));
app.use("/api/void-logs", require("./routes/voidLogRoutes"));
app.use("/api/discounts", require("./routes/discountRoutes"));
app.use("/api/categories", require("./routes/categoryRoutes"));
app.use("/api/brand-partners", require("./routes/brandPartnerRoutes"));
app.use("/api/sync", require("./routes/syncRoutes"));
app.use("/api/reports", require("./routes/reportRoutes"));
app.use("/api/data-management", require("./routes/dataManagementRoutes"));
app.use("/api/merchant-settings", require("./routes/merchantSettingsRoutes"));
app.use("/api/payments", require("./routes/paymentRoutes"));
app.use("/api/remittances", require("./routes/remittanceRoutes"));
app.use("/api/global-settings", require("./routes/globalSettingsRoutes"));
const PORT = process.env.PORT || 5e3;
const HOST = "0.0.0.0";
server.listen(PORT, HOST, () => {
  console.log(`
========================================`);
  console.log(`  POS Backend Server Started`);
  console.log(`========================================`);
  console.log(`  Environment: ${process.env.NODE_ENV || "development"}`);
  console.log(`  Port: ${PORT}`);
  console.log(`  Host: ${HOST}`);
  console.log(`  Cloud-Only Mode: ${process.env.CLOUD_ONLY === "true" ? "Yes" : "No"}`);
  console.log(`  Sync Enabled: ${process.env.ENABLE_SYNC !== "false" ? "Yes" : "No"}`);
  console.log(`========================================
`);
  if (process.env.ENABLE_SYNC !== "false") {
    initStockAlertCron();
    const cron = require("node-cron");
    const dataSyncService = require("./services/dataSyncService");
    cron.schedule("*/5 * * * *", async () => {
      console.log("[Cron] Triggering Data Sync...");
      await dataSyncService.sync();
    });
    console.log("\u2713 Background services (Sync & Alerts) started");
  } else {
    console.log("\u2139 Background services (Sync & Alerts) disabled (ENABLE_SYNC=false)");
  }
  initPaymentExpiryCron();
});
process.on("SIGTERM", () => {
  console.log("SIGTERM received. Shutting down gracefully...");
  server.close(() => {
    console.log("HTTP server closed");
    const dbManager = require("./config/databaseManager");
    dbManager.disconnect().then(() => {
      console.log("Database connection closed");
      process.exit(0);
    });
  });
});
process.on("SIGINT", () => {
  console.log("SIGINT received. Shutting down gracefully...");
  server.close(() => {
    console.log("HTTP server closed");
    process.exit(0);
  });
});
```

FILE: pos/backend/services/dataSyncService.js
==========

```
const mongoose = require("mongoose");
const SyncLog = require("../models/SyncLog");
const dbManager = require("../config/databaseManager");
const SalesTransactionSchema = require("../models/SalesTransaction").schema;
const ProductSchema = require("../models/Product").schema;
const StockMovementSchema = require("../models/StockMovement").schema;
const EmployeeSchema = require("../models/Employee").schema;
const CategorySchema = require("../models/Category").schema;
const DiscountSchema = require("../models/Discount").schema;
const BrandPartnerSchema = require("../models/BrandPartner").schema;
const VoidLogSchema = require("../models/VoidLog").schema;
const ArchiveSchema = require("../models/Archive").schema;
const CartSchema = require("../models/Cart").schema;
class DataSyncService {
  constructor() {
    this.isSyncing = false;
    this.localURI = "mongodb://[REDACTED]";
    this.cloudURI = process.env.MONGODB_URI || "";
  }
  async sync() {
    if (this.isSyncing) {
      console.log("[Sync] Sync already in progress, skipping...");
      return;
    }
    const isOnline = await dbManager.checkInternetConnection();
    if (!isOnline) {
      console.log("[Sync] Offline, skipping sync.");
      return;
    }
    this.isSyncing = true;
    console.log("[Sync] Starting synchronization...");
    let localConn = null;
    let cloudConn = null;
    try {
      console.log("[Sync] Connecting to Local DB...");
      localConn = await mongoose.createConnection(this.localURI).asPromise();
      console.log("[Sync] Connecting to Cloud DB...");
      cloudConn = await mongoose.createConnection(this.cloudURI).asPromise();
      const models = {
        SalesTransaction: {
          Local: localConn.model("SalesTransaction", SalesTransactionSchema),
          Cloud: cloudConn.model("SalesTransaction", SalesTransactionSchema),
          uniqueField: "_id"
        },
        Product: {
          Local: localConn.model("Product", ProductSchema),
          Cloud: cloudConn.model("Product", ProductSchema),
          uniqueField: "sku"
        },
        StockMovement: {
          Local: localConn.model("StockMovement", StockMovementSchema),
          Cloud: cloudConn.model("StockMovement", StockMovementSchema),
          uniqueField: "_id"
        },
        Employee: {
          Local: localConn.model("Employee", EmployeeSchema),
          Cloud: cloudConn.model("Employee", EmployeeSchema),
          uniqueField: "email"
        },
        // Changed to email
        Category: {
          Local: localConn.model("Category", CategorySchema),
          Cloud: cloudConn.model("Category", CategorySchema),
          uniqueField: "_id"
        },
        Discount: {
          Local: localConn.model("Discount", DiscountSchema),
          Cloud: cloudConn.model("Discount", DiscountSchema),
          uniqueField: "_id"
        },
        BrandPartner: {
          Local: localConn.model("BrandPartner", BrandPartnerSchema),
          Cloud: cloudConn.model("BrandPartner", BrandPartnerSchema),
```

```
        uniqueField: "_id"
      },
      VoidLog: {
        Local: localConn.model("VoidLog", VoidLogSchema),
        Cloud: cloudConn.model("VoidLog", VoidLogSchema),
        uniqueField: "_id"
      },
      Archive: {
        Local: localConn.model("Archive", ArchiveSchema),
        Cloud: cloudConn.model("Archive", ArchiveSchema),
        uniqueField: "_id"
      },
      Cart: {
        Local: localConn.model("Cart", CartSchema),
        Cloud: cloudConn.model("Cart", CartSchema),
        uniqueField: "_id"
      }
    };
    for (const [entityName, config] of Object.entries(models)) {
      await this.syncBidirectional(
        entityName,
        config.Local,
        config.Cloud,
        config.uniqueField
      );
    }
    console.log("[Sync] Synchronization complete.");
  } catch (error) {
    console.error("[Sync] Error during synchronization:", error);
  } finally {
    this.isSyncing = false;
    if (localConn) await localConn.close();
    if (cloudConn) await cloudConn.close();
  }
}
async syncBidirectional(entityName, LocalModel, CloudModel, uniqueField = "_id") {
  try {
    let lastSync = await SyncLog.findOne({ entity: entityName });
    if (!lastSync) {
      lastSync = new SyncLog({
        entity: entityName,
        lastSyncedAt: /* @__PURE__ */ new Date(0)
      });
    }
    console.log(
      `[Sync] Syncing ${entityName} since ${lastSync.lastSyncedAt}...`
    );
    const localUpdates = await LocalModel.find({
      updatedAt: { $gt: lastSync.lastSyncedAt }
    });
    if (localUpdates.length > 0) {
      console.log(
        `[Sync] Pushing ${localUpdates.length} ${entityName} updates to Cloud...`
      );
      const bulkPush = localUpdates.map((doc) => {
        const docObj = doc.toObject ? doc.toObject() : { ...doc };
        delete docObj.__v;
        return {
          updateOne: {
            filter: { [uniqueField]: doc[uniqueField] },
            update: {
              $set: docObj,
              $setOnInsert: { createdAt: docObj.createdAt || doc.createdAt }
            },
            upsert: true,
            timestamps: false
            // Prevent Mongoose from overwriting createdAt on upsert
          }
        };
      });
      await CloudModel.bulkWrite(bulkPush, { timestamps: false });
    }
    const cloudUpdates = await CloudModel.find({
      updatedAt: { $gt: lastSync.lastSyncedAt }
    });
    if (cloudUpdates.length > 0) {
      console.log(
```

```
          `[Sync] Pulling ${cloudUpdates.length} ${entityName} updates from Cloud...`
        );
        const bulkPull = cloudUpdates.map((doc) => {
          const docObj = doc.toObject ? doc.toObject() : { ...doc };
          delete docObj.__v;
          return {
            updateOne: {
              filter: { [uniqueField]: doc[uniqueField] },
              update: {
                $set: docObj,
                $setOnInsert: { createdAt: docObj.createdAt || doc.createdAt }
              },
              upsert: true,
              timestamps: false
              // Prevent Mongoose from overwriting createdAt on upsert
            }
          };
        });
        await LocalModel.bulkWrite(bulkPull, { timestamps: false });
      }
      lastSync.lastSyncedAt = /* @__PURE__ */ new Date();
      lastSync.status = "Success";
      await lastSync.save();
      if (localUpdates.length === 0 && cloudUpdates.length === 0) {
        console.log(`[Sync] ${entityName} is up to date.`);
      }
    } catch (error) {
      console.error(`[Sync] Failed to sync ${entityName}:`, error);
      await SyncLog.findOneAndUpdate(
        { entity: entityName },
        { status: "Failed", message: error.message },
        { upsert: true }
      );
    }
  }
}
module.exports = new DataSyncService();
```

FILE: pos/backend/services/emailService.js
==========

```
const nodemailer = require("nodemailer");
const usesBrevo = () => !!process.env.BREVO_API_KEY;
const sendViaBrevoAPI = async (to, subject, htmlContent) => {
  const response = await fetch("https://api.brevo.com/v3/smtp/email", {
    method: "POST",
    headers: {
      "accept": "application/json",
      "api-key": process.env.BREVO_API_KEY,
      "content-type": "application/json"
    },
    body: JSON.stringify({
      sender: {
        name: process.env.STORE_NAME || "POS System",
        email: process.env.EMAIL_FROM
      },
      to: [{ email: to }],
      subject,
      htmlContent
    })
  });
  if (!response.ok) {
    const error = await response.json();
    throw new Error(JSON.stringify(error));
  }
  return await response.json();
};
const createGmailTransporter = () => {
  return nodemailer.createTransport({
    service: "gmail",
    auth: {
      user: process.env.EMAIL_USER,
      pass: process.env.EMAIL_PASS
    }
  });
};
```

```
const sendLowStockAlert = async (ownerEmail, lowStockItems) => {
  if (!ownerEmail || !lowStockItems || lowStockItems.length === 0) {
    console.log("[EmailService] No email to send - missing owner email or no low stock items");
    return { success: false, message: "No email to send" };
  }
  const transporter = createTransporter();
  const outOfStockItems = lowStockItems.filter((item) => item.currentStock === 0);
  const lowItems = lowStockItems.filter((item) => item.currentStock > 0);
  const htmlContent = `
    <!DOCTYPE html>
    <html>
    <head>
      <style>
        body { font-family: Arial, sans-serif; line-height: 1.6; color: #333; }
        .container { max-width: 600px; margin: 0 auto; padding: 20px; }
        .header { background: linear-gradient(135deg, #AD7F65 0%, #76462B 100%); color: white; padding: 20px;
border-radius: 10px 10px 0 0; }
        .header h1 { margin: 0; font-size: 24px; }
        .content { background: #f9f9f9; padding: 20px; border: 1px solid #ddd; }
        .section { margin-bottom: 20px; }
        .section-title { font-size: 16px; font-weight: bold; margin-bottom: 10px; padding: 10px; border-
radius: 5px; }
        .out-of-stock-title { background: #fee2e2; color: #dc2626; }
        .low-stock-title { background: #fef3c7; color: #d97706; }
        .item { background: white; padding: 12px; margin-bottom: 8px; border-radius: 5px; border-left: 4px
solid; display: flex; justify-content: space-between; }
        .item-out { border-left-color: #dc2626; }
        .item-low { border-left-color: #f59e0b; }
        .item-name { font-weight: bold; }
        .item-sku { color: #666; font-size: 12px; }
        .item-stock { font-weight: bold; text-align: right; }
        .stock-zero { color: #dc2626; }
        .stock-low { color: #f59e0b; }
        .footer { background: #f3f4f6; padding: 15px; text-align: center; font-size: 12px; color: #666;
border-radius: 0 0 10px 10px; }
        .summary { background: white; padding: 15px; border-radius: 5px; margin-bottom: 20px; }
        .summary-item { display: inline-block; margin-right: 20px; }
        .summary-count { font-size: 24px; font-weight: bold; }
        .summary-label { font-size: 12px; color: #666; }
      </style>
    </head>
    <body>
      <div class="container">
        <div class="header">
          <h1>\u{1F514} Stock Alert - ${process.env.STORE_NAME || "Create Your Style"}</h1>
        </div>
        <div class="content">
          <div class="summary">
            <div class="summary-item">
              <div class="summary-count" style="color: #dc2626;">${outOfStockItems.length}</div>
              <div class="summary-label">Out of Stock</div>
            </div>
            <div class="summary-item">
              <div class="summary-count" style="color: #f59e0b;">${lowItems.length}</div>
              <div class="summary-label">Low Stock</div>
            </div>
            <div class="summary-item">
              <div class="summary-count">${lowStockItems.length}</div>
              <div class="summary-label">Total Alerts</div>
            </div>
          </div>

          ${outOfStockItems.length > 0 ? `
          <div class="section">
            <div class="section-title out-of-stock-title">\u26D4 Out of Stock Items
(${outOfStockItems.length})</div>
            ${outOfStockItems.map((item) => `
              <div class="item item-out">
                <div>
                  <div class="item-name">${item.itemName}</div>
                  <div class="item-sku">SKU: ${item.sku}</div>
                </div>
                <div class="item-stock stock-zero">0 in stock</div>
              </div>
            `).join("")}
          </div>
          ` : ""}
```

```
          ${lowItems.length > 0 ? `
          <div class="section">
            <div class="section-title low-stock-title">\u26A0\uFE0F Low Stock Items
(${lowItems.length})</div>
            ${lowItems.map((item) => `
              <div class="item item-low">
                <div>
                  <div class="item-name">${item.itemName}</div>
                  <div class="item-sku">SKU: ${item.sku}</div>
                </div>
                <div class="item-stock stock-low">${item.currentStock} in stock</div>
              </div>
            `).join("")}
          </div>
          ` : ""}

          <p style="margin-top: 20px; color: #666;">
            Please restock these items to avoid running out of inventory.
          </p>
        </div>
        <div class="footer">
          <p>This is an automated notification from ${process.env.STORE_NAME || "Create Your Style"} POS
System</p>
          <p>You will receive this alert every 5 hours while items remain low on stock.</p>
        </div>
      </div>
    </body>
    </html>
  `;
  const fromEmail = process.env.EMAIL_FROM || process.env.EMAIL_USER;
  const storeName = process.env.STORE_NAME || "Create Your Style";
  const subject = `\u{1F514} Stock Alert: ${outOfStockItems.length} out of stock, ${lowItems.length} low
stock items`;
  try {
    if (usesBrevo()) {
      await sendViaBrevoAPI(ownerEmail, subject, htmlContent);
      console.log(`[EmailService] Low stock alert sent via Brevo API to ${ownerEmail}`);
      return { success: true };
    }
    const mailOptions = {
      from: `"${storeName} POS" <${fromEmail}>`,
      to: ownerEmail,
      subject,
      html: htmlContent
    };
    const transporter2 = createGmailTransporter();
    const info = await transporter2.sendMail(mailOptions);
    console.log(`[EmailService] Low stock alert sent via Gmail to ${ownerEmail}. Message ID:
${info.messageId}`);
    return { success: true, messageId: info.messageId };
  } catch (error) {
    console.error("[EmailService] Failed to send low stock alert:", error.message);
    return { success: false, error: error.message };
  }
};
module.exports = {
  sendLowStockAlert
};
```

FILE: pos/backend/services/gcashPaymentService.js
==========

```
const crypto = require("crypto");
const MerchantSettings = require("../models/MerchantSettings");
const { decrypt } = require("../utils/encryption");
const PAYMONGO_ENDPOINTS = {
  sandbox: "https://api.paymongo.com/v1",
  production: "https://api.paymongo.com/v1"
};
class GCashPaymentService {
  /**
   * Get active merchant configuration with decrypted private key.
   * Private key is decrypted in-memory only, never stored decrypted.
   */
  async getConfig() {
```

```
  const settings = await MerchantSettings.getActiveConfig();
  if (!settings) {
    throw new Error(
      "GCash payment not configured. Please set up merchant credentials in Admin Settings."
    );
  }
  let privateKey;
  try {
    privateKey = decrypt(
      settings.encryptedPrivateKey,
      settings.privateKeyIV,
      settings.privateKeyAuthTag
    );
  } catch (err) {
    console.error("[GCash] Failed to decrypt private key:", err.message);
    throw new Error(
      "Payment configuration error. Please re-enter merchant credentials."
    );
  }
  return {
    appId: settings.appId,
    privateKey,
    publicKey: settings.publicKey,
    environment: settings.environment,
    baseUrl: PAYMONGO_ENDPOINTS[settings.environment],
    merchantName: settings.merchantName,
    paymentExpiryMinutes: settings.paymentExpiryMinutes
  };
}
/**
 * Create a GCash payment source via PayMongo.
 *
 * This generates a checkout URL that produces a dynamic QR code.
 * Each call creates a unique payment source tied to the specific order.
 *
 * @param {Object} params
 * @param {string} params.merchantOrderId - Unique POS order ID
 * @param {number} params.amount - Amount in PHP (will be converted to centavos)
 * @param {string} params.description - Order description
 * @returns {Object} { sourceId, checkoutUrl, status }
 */
async createPaymentSource({ merchantOrderId, amount, description }) {
  const config = await this.getConfig();
  const amountInCentavos = Math.round(amount * 100);
  if (amountInCentavos < 1e4) {
    throw new Error("Minimum GCash payment amount is \u20B1100.00");
  }
  const webhookBaseUrl = process.env.WEBHOOK_BASE_URL || `http://localhost:${process.env.PORT || 5e3}`;
  const payload = {
    data: {
      attributes: {
        amount: amountInCentavos,
        currency: "PHP",
        type: "gcash",
        redirect: {
          success: `${webhookBaseUrl}/api/payments/gcash/redirect?status=success&order=${merchantOrderId}`,
          failed: `${webhookBaseUrl}/api/payments/gcash/redirect?status=failed&order=${merchantOrderId}`
        },
        billing: {
          name: config.merchantName || "POS Customer",
          email: "pos-customer@payment.local"
        },
        metadata: {
          merchant_order_id: merchantOrderId,
          description: description || `POS Order ${merchantOrderId}`
        }
      }
    }
  };
  try {
    const response = await fetch(`${config.baseUrl}/sources`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Basic ${Buffer.from(config.privateKey + ":").toString("base64")}`,
        Accept: "application/json"
      },
```

```
      body: JSON.stringify(payload)
    });
    if (!response.ok) {
      const errorData = await response.json().catch(() => ({}));
      console.error("[GCash] PayMongo source creation failed:", {
        status: response.status,
        error: errorData
      });
      throw new Error(
        errorData?.errors?.[0]?.detail || `Payment gateway error: ${response.status}`
      );
    }
    const data = await response.json();
    const source = data.data;
    console.log("[GCash] Payment source created:", {
      sourceId: source.id,
      merchantOrderId,
      amount,
      status: source.attributes.status
    });
    return {
      sourceId: source.id,
      checkoutUrl: source.attributes.redirect.checkout_url,
      status: source.attributes.status,
      amount,
      currency: "PHP"
    };
  } catch (error) {
    if (error.message.includes("Payment gateway") || error.message.includes("Minimum")) {
      throw error;
    }
    console.error(
      "[GCash] Network error creating payment source:",
      error.message
    );
    throw new Error(
      "Unable to connect to payment gateway. Please check your internet connection."
    );
  }
}
/**
 * Create a Payment object from a chargeable source.
 * Called when webhook notifies us the source is chargeable.
 *
 * @param {string} sourceId - The PayMongo source ID
 * @param {number} amount - Amount in PHP
 * @returns {Object} Payment confirmation
 */
async createPayment(sourceId, amount) {
  const config = await this.getConfig();
  const amountInCentavos = Math.round(amount * 100);
  const payload = {
    data: {
      attributes: {
        amount: amountInCentavos,
        currency: "PHP",
        source: {
          id: sourceId,
          type: "source"
        },
        description: `POS GCash Payment`
      }
    }
  };
  try {
    const response = await fetch(`${config.baseUrl}/payments`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        Authorization: `Basic ${Buffer.from(config.privateKey + ":").toString("base64")}`,
        Accept: "application/json"
      },
      body: JSON.stringify(payload)
    });
    if (!response.ok) {
      const errorData = await response.json().catch(() => ({}));
      console.error("[GCash] Payment creation failed:", errorData);
```

```
      throw new Error(
        errorData?.errors?.[0]?.detail || `Payment processing error: ${response.status}`
      );
    }
    const data = await response.json();
    const payment = data.data;
    console.log("[GCash] Payment created:", {
      paymentId: payment.id,
      sourceId,
      status: payment.attributes.status
    });
    return {
      paymentId: payment.id,
      status: payment.attributes.status,
      // 'paid'
      paidAt: payment.attributes.paid_at,
      amount: payment.attributes.amount / 100,
      currency: payment.attributes.currency,
      feeAmount: payment.attributes.fee / 100,
      netAmount: payment.attributes.net_amount / 100,
      gcashReference: payment.attributes.external_reference_number || payment.id
    };
  } catch (error) {
    if (error.message.includes("Payment processing")) {
      throw error;
    }
    console.error("[GCash] Network error creating payment:", error.message);
    throw new Error("Payment processing failed. Please try again.");
  }
}
/**
 * Check the status of a payment source.
 * Used for polling from the POS frontend.
 *
 * @param {string} sourceId - PayMongo source ID
 * @returns {Object} { status, checkoutUrl }
 */
async checkSourceStatus(sourceId) {
  const config = await this.getConfig();
  try {
    const response = await fetch(`${config.baseUrl}/sources/${sourceId}`, {
      method: "GET",
      headers: {
        Authorization: `Basic ${Buffer.from(config.privateKey + ":").toString("base64")}`,
        Accept: "application/json"
      }
    });
    if (!response.ok) {
      throw new Error(`Failed to check payment status: ${response.status}`);
    }
    const data = await response.json();
    const source = data.data;
    return {
      sourceId: source.id,
      status: source.attributes.status,
      // pending, chargeable, cancelled, expired, paid
      amount: source.attributes.amount / 100,
      checkoutUrl: source.attributes.redirect?.checkout_url
    };
  } catch (error) {
    console.error("[GCash] Error checking source status:", error.message);
    throw new Error("Unable to check payment status.");
  }
}
/**
 * Verify webhook signature from PayMongo.
 *
 * PayMongo signs webhooks with HMAC-SHA256 using the webhook secret key.
 * This prevents spoofed webhook calls.
 *
 * Signature header format: t=<timestamp>,te=<test_signature>,li=<live_signature>
 *
 * @param {string} rawBody - The raw request body string
 * @param {string} signatureHeader - The Paymongo-Signature header
 * @param {string} webhookSecretKey - The webhook signing secret
 * @returns {boolean} true if signature is valid
 */
```

```
  verifyWebhookSignature(rawBody, signatureHeader, webhookSecretKey) {
    if (!signatureHeader || !webhookSecretKey) {
      console.warn("[GCash] Missing signature header or webhook secret");
      return false;
    }
    try {
      const parts = {};
      signatureHeader.split(",").forEach((part) => {
        const [key, value] = part.split("=");
        parts[key.trim()] = value?.trim();
      });
      const timestamp = parts["t"];
      if (!timestamp) {
        console.warn("[GCash] No timestamp in signature header");
        return false;
      }
      const signatureAge = Math.abs(Date.now() / 1e3 - parseInt(timestamp));
      if (signatureAge > 300) {
        console.warn(
          "[GCash] Webhook signature too old:",
          signatureAge,
          "seconds"
        );
        return false;
      }
      const signedPayload = `${timestamp}.${rawBody}`;
      const expectedSignature = crypto.createHmac("sha256",
webhookSecretKey).update(signedPayload).digest("hex");
      const testSignature = parts["te"];
      const liveSignature = parts["li"];
      const isValid = testSignature && crypto.timingSafeEqual(
        Buffer.from(expectedSignature, "hex"),
        Buffer.from(testSignature, "hex")
      ) || liveSignature && crypto.timingSafeEqual(
        Buffer.from(expectedSignature, "hex"),
        Buffer.from(liveSignature, "hex")
      );
      if (!isValid) {
        console.warn("[GCash] Webhook signature verification failed");
      }
      return !!isValid;
    } catch (error) {
      console.error("[GCash] Signature verification error:", error.message);
      return false;
    }
  }
  /**
   * Generate a unique merchant order ID.
   * Format: GCASH-<timestamp>-<random>
   */
  generateMerchantOrderId() {
    const timestamp = Date.now().toString(36).toUpperCase();
    const random = crypto.randomBytes(3).toString("hex").toUpperCase();
    return `GCASH-${timestamp}-${random}`;
  }
}
module.exports = new GCashPaymentService();
```

FILE: pos/backend/services/paymentExpiryCron.js
==========

```
const cron = require("node-cron");
const SalesTransaction = require("../models/SalesTransaction");
let wsClients = null;
function setWsClients(clients) {
  wsClients = clients;
}
function notifyExpired(merchantOrderId) {
  if (!wsClients) return;
  const clients = wsClients.get(merchantOrderId);
  if (clients && clients.size > 0) {
    const message = JSON.stringify({
      type: "PAYMENT_UPDATE",
      merchantOrderId,
      status: "EXPIRED"
    });
```

```
    clients.forEach((ws) => {
      try {
        if (ws.readyState === 1) {
          ws.send(message);
        }
      } catch (err) {
      }
    });
  }
}
async function expireStalePayments() {
  try {
    const now = /* @__PURE__ */ new Date();
    const expiredTransactions = await SalesTransaction.find({
      gcashStatus: "PENDING",
      gcashExpiresAt: { $lte: now }
    }).select("merchantOrderId gcashStatus");
    if (expiredTransactions.length === 0) return;
    console.log(
      `[PaymentExpiry] Found ${expiredTransactions.length} expired GCash payment(s)`
    );
    const merchantOrderIds = expiredTransactions.map((t) => t.merchantOrderId);
    await SalesTransaction.updateMany(
      {
        merchantOrderId: { $in: merchantOrderIds },
        gcashStatus: "PENDING"
      },
      {
        $set: {
          gcashStatus: "EXPIRED",
          status: "Voided",
          voidReason: "GCash payment expired (timeout)",
          voidedAt: now
        }
      }
    );
    merchantOrderIds.forEach((orderId) => {
      notifyExpired(orderId);
    });
    console.log(
      `[PaymentExpiry] Expired ${merchantOrderIds.length} payment(s):`,
      merchantOrderIds
    );
  } catch (error) {
    console.error(
      "[PaymentExpiry] Error processing expired payments:",
      error.message
    );
  }
}
function initPaymentExpiryCron() {
  cron.schedule("* * * * *", async () => {
    await expireStalePayments();
  });
  console.log("\u2713 GCash payment expiry cron started (runs every minute)");
}
module.exports = {
  initPaymentExpiryCron,
  setWsClients,
  expireStalePayments
};
```

FILE: pos/backend/services/stockAlertCron.js
==========

```
const cron = require("node-cron");
const { sendLowStockAlert } = require("./emailService");
const Product = require("../models/Product");
const Employee = require("../models/Employee");
let lastEmailSentTime = null;
const MIN_EMAIL_INTERVAL_MS = 5 * 60 * 60 * 1e3;
const getLowStockItems = async () => {
  try {
    const products = await Product.find({});
    const lowStockItems = products.filter((p) => {
      if (p.isArchived) return false;
```

```
      const stock = p.currentStock || 0;
      const reorder = p.reorderNumber || 5;
      return stock <= reorder && stock >= 0;
    }).map((p) => ({
      _id: p._id,
      itemName: p.itemName,
      sku: p.sku,
      currentStock: p.currentStock || 0,
      reorderNumber: p.reorderNumber || 5,
      alertType: (p.currentStock || 0) === 0 ? "out_of_stock" : "low_stock"
    }));
    lowStockItems.sort((a, b) => {
      if (a.alertType === "out_of_stock" && b.alertType !== "out_of_stock") return -1;
      if (a.alertType !== "out_of_stock" && b.alertType === "out_of_stock") return 1;
      return a.currentStock - b.currentStock;
    });
    return lowStockItems;
  } catch (error) {
    console.error("[StockAlertCron] Error fetching low stock items:", error.message);
    return [];
  }
};
const getOwnerEmail = async () => {
  try {
    const employees = await Employee.find({ role: "Owner", status: "Active" });
    if (employees.length > 0 && employees[0].email) {
      return employees[0].email;
    }
    console.log("[StockAlertCron] No active owner found in database");
    return null;
  } catch (error) {
    console.error("[StockAlertCron] Error fetching owner email:", error.message);
    return null;
  }
};
const canSendEmail = () => {
  if (!lastEmailSentTime) {
    return true;
  }
  const timeSinceLastEmail = Date.now() - lastEmailSentTime;
  const canSend = timeSinceLastEmail >= MIN_EMAIL_INTERVAL_MS;
  if (!canSend) {
    const remainingMinutes = Math.ceil((MIN_EMAIL_INTERVAL_MS - timeSinceLastEmail) / (60 * 1e3));
    console.log(`[StockAlertCron] Email cooldown active. Next email allowed in ${remainingMinutes} minutes.`);
  }
  return canSend;
};
const runStockAlertCheck = async (forceCheck = false) => {
  console.log("[StockAlertCron] Running stock alert check...");
  if (!forceCheck && !canSendEmail()) {
    console.log("[StockAlertCron] Skipping - email was sent recently.");
    return;
  }
  try {
    const lowStockItems = await getLowStockItems();
    if (lowStockItems.length === 0) {
      console.log("[StockAlertCron] No low stock items found. Skipping email.");
      return;
    }
    const ownerEmail = await getOwnerEmail();
    if (!ownerEmail) {
      console.log("[StockAlertCron] No owner email found. Skipping email.");
      return;
    }
    console.log(`[StockAlertCron] Found ${lowStockItems.length} low stock items. Sending email to
${ownerEmail}...`);
    const result = await sendLowStockAlert(ownerEmail, lowStockItems);
    if (result.success) {
      lastEmailSentTime = Date.now();
      console.log("[StockAlertCron] Stock alert email sent successfully! Next email allowed after 5 hours.");
    } else {
      console.log("[StockAlertCron] Failed to send stock alert email:", result.error || result.message);
    }
  } catch (error) {
    console.error("[StockAlertCron] Error in stock alert check:", error.message);
  }
};
```

```
const initStockAlertCron = () => {
  cron.schedule("0 */5 * * *", async () => {
    console.log("[StockAlertCron] Scheduled cron triggered at", (/* @__PURE__ */ new Date()).toISOString());
    await runStockAlertCheck(true);
  });
  console.log("[StockAlertCron] Stock alert cron job initialized (runs every 5 hours at minute 0)");
  setTimeout(async () => {
    if (canSendEmail()) {
      console.log("[StockAlertCron] Running initial stock alert check (first run or cooldown expired)...");
      await runStockAlertCheck(false);
    } else {
      console.log("[StockAlertCron] Skipping initial check - email was sent recently.");
    }
  }, 1e4);
};
module.exports = {
  initStockAlertCron,
  runStockAlertCheck,
  getLowStockItems,
  getOwnerEmail
};
```

FILE: pos/backend/start-mongodb.bat
==========

```
@echo off
echo ========================================
echo Starting MongoDB for POS System...
echo ========================================

set MONGODB_PATH=%USERPROFILE%\Downloads\mongodb-windows-x86_64-8.2.1\mongodb-win32-x86_64-windows-8.2.1

if not exist "%MONGODB_PATH%\data\db" (
    echo Creating data directory...
    mkdir "%MONGODB_PATH%\data\db"
)

if not exist "%MONGODB_PATH%\logs" (
    echo Creating logs directory...
    mkdir "%MONGODB_PATH%\logs"
)

echo Starting MongoDB on port 27017...
echo.

cd /d "%MONGODB_PATH%\bin"
mongod.exe --dbpath "%MONGODB_PATH%\data\db" --logpath "%MONGODB_PATH%\logs\mongodb.log" --logappend --
journal

pause
```

FILE: pos/backend/stop-mongodb.bat
==========

```
@echo off
echo ========================================
echo Stopping MongoDB for POS System...
echo ========================================

taskkill /F /IM mongod.exe 2>nul

if %ERRORLEVEL%==0 (
    echo MongoDB stopped successfully!
) else (
    echo MongoDB was not running or already stopped.
)

timeout /t 2 >nul
echo.
```

FILE: pos/backend/test_cogs_output.json

==========
{
  "record": {
    "_id": "69ca6a650148f0a7a883e7b9",
    "userId": "69af9ab7a34c2c0b09de7af5",
    "performedById": "69af9ab7a34c2c0b09de7af5",
    "performedByName": "Ms. Owner",
    "items": [
      {
        "productId": "69c8dc644a33ecd2782a3a22",
        "itemName": "dress",
        "sku": "",
        "variant": null,
        "selectedSize": null,
        "quantity": 1,
        "price": 1499,
        "itemImage": "",
        "returnStatus": null,
        "returnedQuantity": null,
        "returnReason": "Customer Changed Mind",
        "voidReason": null
      }
    ],
    "paymentMethod": "return",
    "amountReceived": 0,
    "changeGiven": 0,
    "referenceNo": "RET-69ca69a80148-1774873189492-1",
    "receiptNo": "592698",
    "totalAmount": 1499,
    "status": "Returned",
    "checkedOutAt": "2026-03-30T12:19:49.492Z",
    "originalTransactionId": "69ca69a80148f0a7a883e6e4",
    "voidedBy": null,
    "voidedByName": null,
    "voidedAt": null,
    "voidReason": null,
    "merchantOrderId": null,
    "gcashSourceId": null,
    "gcashPaymentId": null,
    "gcashStatus": null,
    "gcashCheckoutUrl": null,
    "gcashReference": null,
    "gcashPaidAt": null,
    "gcashExpiresAt": null,
    "gcashFeeAmount": null,
    "gcashNetAmount": null,
    "gcashWebhookEventId": null,
    "createdAt": "2026-03-30T12:19:49.890Z",
    "updatedAt": "2026-03-30T12:19:49.890Z",
    "__v": 0
  },
  "productRootCostPrice": null,
  "productSizes": null,
  "sampleLookup": [
    {
      "_id": "69ca6a650148f0a7a883e7b9",
      "userId": "69af9ab7a34c2c0b09de7af5",
      "performedById": "69af9ab7a34c2c0b09de7af5",
      "performedByName": "Ms. Owner",
      "items": {
        "productId": "69c8dc644a33ecd2782a3a22",
        "itemName": "dress",
        "sku": "",
        "variant": null,
        "selectedSize": null,
        "quantity": 1,
        "price": 1499,
        "itemImage": "",
        "returnStatus": null,
        "returnedQuantity": null,
        "returnReason": "Customer Changed Mind",
        "voidReason": null
      },
      "paymentMethod": "return",
      "amountReceived": 0,
      "changeGiven": 0,
      "referenceNo": "RET-69ca69a80148-1774873189492-1",

```
      "receiptNo": "592698",
      "totalAmount": 1499,
      "status": "Returned",
      "checkedOutAt": "2026-03-30T12:19:49.492Z",
      "originalTransactionId": "69ca69a80148f0a7a883e6e4",
      "voidedBy": null,
      "voidedByName": null,
      "voidedAt": null,
      "voidReason": null,
      "merchantOrderId": null,
      "gcashSourceId": null,
      "gcashPaymentId": null,
      "gcashStatus": null,
      "gcashCheckoutUrl": null,
      "gcashReference": null,
      "gcashPaidAt": null,
      "gcashExpiresAt": null,
      "gcashFeeAmount": null,
      "gcashNetAmount": null,
      "gcashWebhookEventId": null,
      "createdAt": "2026-03-30T12:19:49.890Z",
      "updatedAt": "2026-03-30T12:19:49.890Z",
      "__v": 0,
      "productInfo": [],
      "itemCostPrice": 0
    }
  ]
}
```


FILE: pos/backend/test_products.json
==========


```
{
  "totalProducts": 6,
  "productsWithCostPrice": 0,
  "productsWithNestedCostPrice": 6,
  "sampleProduct": {
    "_id": "69c93277c829406df3978d28",
    "sku": "TOP-IUSV8F",
    "itemName": "compression shirt",
    "category": "Apparel - Men",
    "subCategory": "Tops",
    "unitOfMeasure": "pcs",
    "brandName": "Default",
    "variant": "Black",
    "foodSubtype": "",
    "size": "",
    "sizes": {
      "S": {
        "quantity": 15,
        "variant": "Black",
        "variants": {
          "Black": 15
        },
        "variantPrices": {
          "Black": 399
        },
        "variantCostPrices": {
          "Black": 299
        }
      },
      "M": {
        "quantity": 13,
        "variant": "Black",
        "variants": {
          "Black": {
            "quantity": 13,
            "price": 399,
            "costPrice": 299,
            "batches": [
              {
                "qty": 13,
                "price": 399,
                "costPrice": 299,
                "createdAt": "2026-03-29T14:47:44.235Z"
              }
            ]
```

```
          }
        },
        "variantPrices": {
          "Black": 399
        },
        "variantCostPrices": {
          "Black": 299
        }
      },
      "L": {
        "quantity": 15,
        "variant": "Black",
        "variants": {
          "Black": 15
        },
        "variantPrices": {
          "Black": 399
        },
        "variantCostPrices": {
          "Black": 299
        }
      }
    },
    "itemPrice": 399,
    "costPrice": 0,
    "currentStock": 43,
    "reorderNumber": 8,
    "expirationDate": null,
    "supplierName": "",
    "supplierContact": "",
    "dateAdded": "2026-03-29T14:08:55.326Z",
    "itemImage":
"
BolHhUXITEhJSkrLi4uFx8zODMtNygtLisBCgoKDg0OFxAQFSsdHR0tLS0tLS0tKystLS0tLS0tLS0tLTAtLS0tLS0tLS0tKystKy0tLS0tLS
srKys3LS0tK//AABEIAOEA4AMBIgACEQEDEQH/xAAcAAABBQEBAQAAAAAAAAAAAAAAAQIDBAcFBgj/xABHEAABAwICBgcDBwgKAwAAAAABAAI
DBBESIQUxQVFxkQYHEyJhgaEyscEjM0JSctHwFFNzgpKywuEIFSRDYmOTorPxJTTS/8QAGAEBAQEBAQAAAAAAAAAAAAAAAAAEDAgT/xAAfEQEA
AgMBAAMBAQAAAAAAAAAAAQIDETEhIkFCMhL/2gAMAwEAAhEDEQA/ANesksnWQgakTrIsgahLZFkCJE6yRAJEqECJE5CCJ0QJBLQSNRIFxfI2O
zJYnpca92RHlr9y3ADNfO1VpUNlfG491znFp3EnVwKOqqulYcUzrkgAfRNrhwBsfC6WldA0YX0hduc3I+ZBBKK2YOOIHZbxyVrRemYwO83NcW
bUmHQ7KGRmKOJ0NhsOEnLPEAbOBXc6OUIqJ44nXLXZusSDhsXHMZjiuKKyN/si1/HfuWg9XdE3A+f6RPZjwaA0m3G45KRG5W9vHq6GkbDG2Jt
8LBYXNz5nep7JUWWjzGoTrJECISoVCJLJyLIG2RZKhBZRZOskUDbITrJLIGoTrIQIkTrIsgZZFk6y52l9OU1I3FUTsj2gOPeP2WDvO8ggv2RZ
ZZ0h64Gtuyjgx H87NcN4tiabkcS3VqWbac6W1tWSZqqQg37jXFkYB2dm2wPE3PirpG99IOl1JSPMEk3yxYXNjaHOOru4iBZl/EjIEr5w09Jd1
lBDUFj8Zzve5256zxUlSO0N1J8d1japFUuw6zu35eKmow97gGi3AkKRujswvT6HoGtAyF/O64m2mlccz1a0TovBmST962DoPFho2n6znu9cP8
KzW9gAtR6JVEclJH2bgcAwOG1rwSHAjZnc+alNzO3WWIrERDsoSgJVo85qEqLIGoslshAlkichA1CVFlRaSJyFA2ySydZFkDUJ1lXr6xkET5p
HYWMaXOPgPigh0rpSGljMs8jWMG06ydzQM3HwCzHTXWxLjLaWFgZbuulDnPd42a4BvDP4LxHS7pNLX1BmeSGi4jjvkxm7idp2nyXBdLt3Ee9d
xCPU6U6wtITgtNSWNI1RAR/7m971Xl5pC8lziSSbkk3JO8k60hGaHGyoicEzApS3mkaEEPZp0UdtR8ipbIATWzbpwzRuAvdpG/V5ELpUlRG0j
5Ro8wF52yUrOcUNYzTD0tRplgccJx2yFtR89yoaO01NTT9tDIWPzvbU65vhc3U4eBXJBTmLqtIq4vebdbv0M6dw1wEUloqi3s37sltsZO3/AA
nPjrXsF8utdbMHUbg7jvC0Xoh1lvitFWYpGahKM5G/a+uPHWPHYmrnbXUKpozScNUztIJWyN3tOYO5w1tPgVbXKhIlQgSyLJUIG2QnJEFlCVC
BEJUiAWWddenC1sdC02xWlk8QCQxvME+TVqa+eutGq7TSU9zkHCMeGBoHvB5qwPH3ubbwbcQoS/IjwPuUb5TmNrTiH1rCSR+ZI1EEjzC62i65
33oadu1VXy524e4KaIqiWyAEFKFUInBIgIHISXQgWyUNSApwKBUJwSkILWidKzU0glgkdG4bRtH1XA5Ob4Fa/wBC+sNlY4QVDRFNkA4H5OQ7h
fNhO4347FitrHjl9ynp3WJ4BSY2Pp2ySy8b1a9JHVUToJXYpIgLOOt8ZyBO8g5E+IXs1xMadESJ1kWUDUJbIsgsISoQIhKkQC+b+sZlq+pH+a
/3r6KrKlsUb5XXwsa57rAk2aCTYDWcl8zdIK01Uz5iReRxcd3eN7eWryXVUeYlOfiNvwKje7K2648jsVyWPeAfxvVSZlggWI3zV2O+1V6H2Sd
t/RSlysCwHp2JVmPUrSukSoTQUqByEiVAoSgpqcCgkaVK0A5FQtKnjKBJGd3hq+Cax+fL3X+KsvbcfjauZDL+PT4IPZ9AtKfk9bE8nuuPZv8A
svy9DY+S3pfMlJJmM/P4r6VoZccTH/WY137TQVzZYTJE5IuFIhKhBYSJUIEQhCDzvTyu7GifY2L7MHA5u9AeawkRskcbsGvXqPote61XHsY2j
aXH0A+9ZLgwngjuseKWldFsaRgcRvBz5LjaS0eWNviB8iF3aqTE/kudpj2SuZtO3f8AiNbcGllwm2/JWZFRVmN9wtIYJGFSh6rF9kB11di2JE
/tFVaE+yqLAkS4/FV0XQWg5PDlRMiRtQmx0mlTxlc2KoV5ptmqJtIT4IyeAHG65FM4jL8fzVivnxkMGpuvikgpwfwP5qC5RuuV9PaPiwQxs+r
GxvJoC+dei1GH1kEZGTpYwdtwXi/ovpJSywRCVC4UiEIQToQhAJEqEHges43MI8HH1H3LK684SVqPWQ75aMboyebisn0pJmVJaVVIRd11S0xe
1vQK9TbV6jqz0Y2o0ozG3E2OOWRwOo3b2QB/1DyWcdaz5VlFk+NdXpZor8jraimF7Ryua2/1L3Z6ELlxhbPMkIQ0J4CLKomjCnDFWY6ysMeuk
OwI7NBcgSKhHQqJ8CtY7ppCgoujI1K0+cts0nWy45fyV2GIbRkclQ0qzCBvbkOBSVS6Hp2yG7iczfK23yXo4NCsJye4cbH3WXmtBPsbL2FI9e
a1rRPXppSs146nRvRzYauCTERhlZmNuY131bea3BYlG/URrFjyW1xuuAd4B5rSJ2yvWI4chCFXAQhCCZCEIBCEIMs61arDOBfPs2j1cfisqrZ
cV3L13TrSJqa2V2wOwt+y3Ie6/muDVUlh4WsuLWb0r5tz6V9wtY6k9HWZUVR+k5sTeDBidbzcP2VkQu15avpHobon8koYYPpBuJ/23953qbeS
lY9S8+aYH1xs/wDMVB/R/wDExeLiC0Drrp8OlJD9ZsbucbB/CV4CILZglIQE62SjBVEoCcHWULXKW1wqiwW4hcKsHJaWXCbFPrI7d4aigljUr
25XVWlfsV6D6pQS0Zu0jdmF0I9DOkpJKox/NzxsD8R2gl7cOr6mfiuQy8b77FqvQ+mZNoSriGbmvdIeIYxzbeTSEtxY6ySCLs53M3OyXqqJ3u
XB0zHgqsWxwaeYBXVo5dXArzZOvVinx36Z1wtupW2Y0bmtHIBYhoZmKRrPrOA5m3xW6WXdeM8vQhCVdMiIQhBMhCEAo6h1mOO5pPIKRVdKyYY
JXbo3n/aUHzxWuvKfVTudibYqpWn5UhTDUsbx69dOIX0ti0gbR719JNWBaBYH1dOw6jNED+2Fvy7pxlm7DEuv6jwzwzfXiLfNjifc8LJYta2L
+kLOL0se0CVx4EtA/dKxtpWsMFhqhkFipGJJm3CoTxUrCoYDfJSNySAk7Nqs0sge3CUxueRVdwMbrqge0scukHXaHhQzNEjLjWEzRkmuM7dXF
B1LB4Wq9S7QYKqMm/ejB+yWvH8vJZFTusVq3Uu75SpGwsjPJzvvPJJ4M86Y0pjlaDrZeM8WOLfgqkM5sAMyvYddNGIqoED535QcT3Xjm2/mvG
6P1sFtd9fBYXb0l6Xo1XdnLFI4HuPYXcGuB9y+gWm4uF860Lc771tXQbSRnpG4jd0fybuAHdPK3IpSTLXkvQoQhdsQhIlQSoQhALj9LpcFFMf
8IHNwHxXYXnOn0lqJwP0nNHK7v4UWGEaRPy342kq+BkubVHFKT4+mxdCndduays9NOLugD/bILa+3i/5GrflgfROIyV1O0fnmH9lwcfQLfF1T
jLL1hX9IJ39spx/kEn/Udb4rKmLWOv6L+2QHfTn0kd96ymJasUsSe5RR5FTuVgU74XK08bVBO1S0z7ixSA4FOJxCxTLWKAqhKaXA6x1J9ZHhI
kbxUc7dqmpJgRgdqKipzL9ManZ8DtW39SkcZoXzNN3vlLX+AYBhbydf9ZYXTtwOMTtRzb+PFaz1FylktVD9FzY5QNzmuLTzDm8kngf13Rh09H
fLuTWP60dx7uaz2RrWm4OwgcTZaP14DOmO0dpnxLPuWZvOLkFhbr0Unx06J11o/VdV2mkiJ9tgcOLD9zjyWdaMb7l7DoC4ivjttx34YHLmvWm
SPi15CEi2eQIQhBMhCEAvJdZLrUrft+uE2XrV5PrKYTSNI2SD9x6LHWIu+cN/JWSLNVSYWk/G9XHG7VnZ6K8et6qaHtK3tCMomOP6zu6PeeS2
BeI6p6HBSvmIzkksPssFveXcl7dd14xvO5Y119AflFKdpikBHhiFjzJ5LICML7LS+u+txaSbHf5uCMcC5z3H0LVms4uV39MxKyxuponXCbram
MyKoe9qrt7rlbIuoJGoJyb+aYUkBuLFOsqgChfGQp7JS26KWI9o3CcnD2T8FqfUfWMFTMx2Uj4hh8cDrvHHMHyKygRkZheg6L6RdDVQzAkOZI
wkja3EA4ebSR5oNM67W92mPjL6YPvWVYsx45LYuumkLqSOQD2JC0ncHjL1b6rEo5CCB+LrG0NqS9Towd1em6AutXxeJeObHLzOjnXbcbR/2vR
9Cu7Ww/btzBHxWdOtcn8tkQhC2eUIQhBMkSpECrhdNosVDL4BruTh8LruKlpuHHTTM3xPH+0oQ+dKtveJ8VO4WsotIHPI7bk8NikjaXYRvsOa
4s9FeN66JUvZUMDP8trjxf3z+8uuo4Y8LWt+qA3kLJZJA0FzjYNBJJ1AAXJK7eeXzN1jdo/SlU92Z7Zzbbmss1oH6oC8vKDe9l1dOaQ7SrqJL
```

lwfPK8E67Oe4jLgQqxz8Qu0VIhkggJ/YlBYUCxuSvZdMAUjSdyCu02KsDNI9oOdkxrcPBWBJqTgE0FOCIcpaWSzgdxuFBZPY3aivpzTlC2uoH
x6+0iDmfawhzT7ua+baiAskLSMwfUL6P6E1nbaOpZL/wByxp+0wYD6tKyTrZ0N+T1pkbk2YdoLbDqcOefms5d1lw9Ev+j5jzXpujb8NVCR+dj
9XAfFeR0XL6W5L1/RZuKqg/Sx+jgVlEet5n4trQhC1eYIQkQTIQkQKo6n2HX+q73FPUdXGXxvaDYua5oO4kEAoPnLSIFzzVnQbQZ4A7V2sd+B
eLqppKF7C5rsnNcWuG4tyI53S0TiC031WPIriz0V4+kSub0jpxLSTxl+AOieC617C2u23gr0Uoe0PBBDgHAjUQRcELk9LJ8FHId4Df2nALt53
zJpbRz45HA55625qm5zme0CN1wW35617vS7A6S+oW1ACyrsiAFregHou9Dxrasf9WKeKgL1clFE7WyM8WtuoHaHg/NM8iR7imkecEgO1KCu0/
QUWxpHB7viSojoEbJHjkfgg5mPwRfwXVg0Owg3kfcEi4w/Fqf/AFI0f3z+TfuVRxb+BT2uHjyXZ/qIHVM4cQ37k9vR1h1zX/Uag4waCntaF3W
dGIvrPdw7Mfwqwzo5FvkHmz/5RWldS9djoXwn+5lNvsyDF78Sl64NGdrQdqB3oXg/qP7p9cPJHVXo+OCGYMLiXPbcuOwN7uQAH1l7HSNG2eGS
F/syMcw+Ytfy1riVh8z6Jd3ive9BGYq6EHY4u/ZY4+8BeOpaXsp3sPtNe5p4gkWXtOgcobXxX2lw5scB6rL9N/w2JIhC0YBCEIJUIQgEIQEGC
dMWB1ZPhGXay88Zv63XEjFtS7XSOW88p3yPP7Ti74riFxtuvq4/gLO0vRRqfVhp0vBo3m9ml8V9gv3mcM7jzXX6w5SKVo3yi/k1xWb9XtVhr4
DvcW5f4mlvxuva9ZNYLMiB1XceJ1envXeNnkj1nNa8E61CB5JKplzdRCEHXfmtWR8ksY9pwUJrmfRYXHgpLRN+iL80oqzqYxAxssrvZiDfEoe
Hj2njgEPLzm52EeCoVVXGwXv5n4DaqLLJWjW62Z96l/KIzrmA4Bcemrb94APG4/cr8VfCfahA8lNi0JKbbIT5qVj6Xf6lMidSu+i0cRZWmUcJ
9lreQVD4BEfYfycV0IvtX4rnCmG8Hw1KzA0bviqjR+r/ANiTiz+JeuC8d1fO7sg+yfen9ZmmHU1FhYSHzvEQI1hpBL7bshb9ZZX66iNsw0wGv
ramRhu108jmneMRzB3K10dhc+rhazJxlZY7rOBJ8gCVyWOtkuroaYxzxSA2LZGHyDhccrhYb9emY+LdkIKRbPMEISIJ0iEIBCEIPnzTfzjuP3
KppP5hn6RnvSIWV+vVj4v9X3/v0/6Rv7q9X0/+fPH4NQhaY2WTrw1ZrCgKRC2YkZrV6HUlQkI5+kta8vpP2jwQhc2VPoL2lfrkIVjgrhdPRGt
CEgdSr1hXaLUEIXSNC6v9cn2R7wuf1zfNUn6Z37gQhZXd16zpvtcl06f228W+9Ihef7er8t7TUIW7yBCEIP/Z",
    "productImages": [

"
BolHhUXITEhJSkrLi4uFx8zODMtNygtLisBCgoKDg0OFxAQFSsdHR0tLS0tLS0tKystLS0tLS0tLS0tLTAtLS0tLS0tLS0tKystKy0tLS0tLS
srKys3LS0tK//AABEIAOEA4AMBIgACEQEDEQH/xAAcAAABBQEBAQAAAAAAAAAAAAAAAQIDBAcFBgj/xABHEAABAwICBgcDBwgKAwAAAAABAAI
DBBESIQUxQVFxkQYHEyJhgaEyscEjM0JSctHwFFNzgpKywuEIFSRDYmOTorPxJTTS/8QAGAEBAQEBAQAAAAAAAAAAAAAAAAEDAgT/xAAfEQEA
AgMBAAMBAQAAAAAAAAAAQIDETEhIkFCMhL/2gAMAwEAAhEDEQA/ANesksnWQgakTrIsgahLZFkCJE6yRAJEqECJE5CCJ0QJBLQSNRIFxfI2O
zJYnpca92RHlr9y3ADNfO1VpUNlfG491znFp3EnVwKOqqulYcUzrkgAfRNrhwBsfC6WldA0YX0hduc3I+ZBBKK2YOOIHZbxyVrRemYwO83NcW
bUmHQ7KGRmKOJ0NhsOEnLPEAbOBXc6OUIqJ44nXLXZusSDhsXHMZjiuKKyN/si1/HfuWg9XdE3A+f6RPZjwaA0m3G45KRG5W9vHq6GkbDG2Jt
8LBYXNz5nep7JUWWjzGoTrJECISoVCJLJyLIG2RZKhBZRZOskUDbITrJLIGoTrIQIkTrIsgZZFk6y52l9OU1I3FUTsj2gOPeP2WDvO8ggv2RZ
ZZ0h64Gtuyjgx H87NcN4tiabkcS3VqWbac6W1tWSZqqQg37jXFkYB2dm2wPE3PirpG99IOl1JSPMEk3yxYXNjaHOOru4iBZl/EjIEr5w09Jd1
lBDUFj8Zzve5256zxUlSO0N1J8d1japFUuw6zu35eKmow97gGi3AkKRujswvT6HoGtAyF/O64m2mlccz1a0TovBmST962DoPFho2n6znu9cP8
KzW9gAtR6JVEclJH2bgcAwOG1rwSHAjZnc+alNzO3WWIrERDsoSgJVo85qEqLIGoslshAlkichA1CVFlRaSJyFA2ySydZFkDUJ1lXr6xkET5p
HYWMaXOPgPigh0rpSGljMs8jWMG06ydzQM3HwCzHTXWxLjLaWFgZbuulDnPd42a4BvDP4LxHS7pNLX1BmeSGi4jjvkxm7idp2nyXBdLt3Ee9d
xCPU6U6wtITgtNSWNI1RAR/7m971Xl5pC8lziSSbkk3JO8k60hGaHGyoicEzApS3mkaEEPZp0UdtR8ipbIATWzbpwzRuAvdpG/V5ELpUlRG0j
5Ro8wF52yUrOcUNYzTD0tRplgccJx2yFtR89yoaO01NTT9tDIWPzvbU65vhc3U4eBXJBTmLqtIq4vebdbv0M6dw1wEUloqi3s37sltsZO3/AA
nPjrXsF8utdbMHUbg7jvC0Xoh1lvitFWYpGahKM5G/a+uPHWPHYmrnbXUKpozScNUztIJWyN3tOYO5w1tPgVbXKhIlQgSyLJUIG2QnJEFlCVC
BEJUiAWWddenC1sdC02xWlk8QCQxvME+TVqa+eutGq7TSU9zkHCMeGBoHvB5qwPH3ubbwbcQoS/IjwPuUb5TmNrTiH1rCSR+ZI1EEjzC62i65
33oadu1VXy524e4KaIqiWyAEFKFUInBIgIHISXQgWyUNSApwKBUJwSkILWidKzU0glgkdG4bRtH1XA5Ob4Fa/wBC+sNlY4QVDRFNkA4H5OQ7h
fNhO4347FitrHjl9ynp3WJ4BSY2Pp2ySy8b1a9JHVUToJXYpIgLOOt8ZyBO8g5E+IXs1xMadESJ1kWUDUJbIsgsISoQIhKkQC+b+sZlq+pH+a
/3r6KrKlsUb5XXwsa57rAk2aCTYDWcl8zdIK01Uz5iReRxcd3eN7eWryXVUeYlOfiNvwKje7K2648jsVyWPeAfxvVSZlggWI3zV2O+1V6H2Sd
t/RSlysCwHp2JVmPUrSukSoTQUqByEiVAoSgpqcCgkaVK0A5FQtKnjKBJGd3hq+Cax+fL3X+KsvbcfjauZDL+PT4IPZ9AtKfk9bE8nuuPZv8A
svy9DY+S3pfMlJJmM/P4r6VoZccTH/WY137TQVzZYTJE5IuFIhKhBYSJUIEQhCDzvTyu7GifY2L7MHA5u9AeawkRskcbsGvXqPote61XHsY2j
aXH0A+9ZLgwngjuseKWldFsaRgcRvBz5LjaS0eWNviB8iF3aqTE/kudpj2SuZtO3f8AiNbcGllwm2/JWZFRVmN9wtIYJGFSh6rF9kB11di2JE
/tFVaE+yqLAkS4/FV0XQWg5PDlRMiRtQmx0mlTxlc2KoV5ptmqJtIT4IyeAHG65FM4jL8fzVivnxkMGpuvikgpwfwP5qC5RuuV9PaPiwQxs+r
GxvJoC+dei1GH1kEZGTpYwdtwXi/ovpJSywRCVC4UiEIQToQhAJEqEHges43MI8HH1H3LK684SVqPWQ75aMboyebisn0pJmVJaVVIRd11S0xe
1vQK9TbV6jqz0Y2o0ozG3E2OOWRwOo3b2QB/1DyWcdaz5VlFk+NdXpZor8jraimF7Ryua2/1L3Z6ELlxhbPMkIQ0J4CLKomjCnDFWY6ysMeuk
OwI7NBcgSKhHQqJ8CtY7ppCgoujI1K0+cts0nWy45fyV2GIbRkclQ0qzCBvbkOBSVS6Hp2yG7iczfK23yXo4NCsJye4cbH3WXmtBPsbL2FI9e
a1rRPXppSs146nRvRzYauCTERhlZmNuY131bea3BYlG/URrFjyW1xuuAd4B5rSJ2yvWI4chCFXAQhCCZCEIBCEIMs61arDOBfPs2j1cfisqrZ
cV3L13TrSJqa2V2wOwt+y3Ie6/muDVUlh4WsuLWb0r5tz6V9wtY6k9HWZUVR+k5sTeDBidbzcP2VkQu15avpHobon8koYYPpBuJ/23953qbeS
lY9S8+aYH1xs/wDMVB/R/wDExeLiC0Drrp8OlJD9ZsbucbB/CV4CILZglIQE62SjBVEoCcHWULXKW1wqiwW4hcKsHJaWXCbFPrI7d4aigljUr
25XVWlfsV6D6pQS0Zu0jdmF0I9DOkpJKox/NzxsD8R2gl7cOr6mfiuQy8b77FqvQ+mZNoSriGbmvdIeIYxzbeTSEtxY6ySCLs53M3OyXqqJ3u
XB0zHgqsWxwaeYBXVo5dXArzZOvVinx36Z1wtupW2Y0bmtHIBYhoZmKRrPrOA5m3xW6WXdeM8vQhCVdMiIQhBMhCEAo6h1mOO5pPIKRVdKyYY
JXbo3n/aUHzxWuvKfVTudibYqpWn5UhTDUsbx69dOIX0ti0gbR719JNWBaBYH1dOw6jNED+2Fvy7pxlm7DEuv6jwzwzfXiLfNjifc8LJYta2L
+kLOL0se0CVx4EtA/dKxtpWsMFhqhkFipGJJm3CoTxUrCoYDfJSNySAk7Nqs0sge3CUxueRVdwMbrqge0scukHXaHhQzNEjLjWEzRkmuM7dXF
B1LB4Wq9S7QYKqMm/ejB+yWvH8vJZFTusVq3Uu75SpGwsjPJzvvPJJ4M86Y0pjlaDrZeM8WOLfgqkM5sAMyvYddNGIqoED535QcT3Xjm2/mvG
6P1sFtd9fBYXb0l6Xo1XdnLFI4HuPYXcGuB9y+gWm4uF860Lc771tXQbSRnpG4jd0fybuAHdPK3IpSTLXkvQoQhdsQhIlQSoQhALj9LpcFFMf
8IHNwHxXYXnOn0lqJwP0nNHK7v4UWGEaRPy342kq+BkubVHFKT4+mxdCndduays9NOLugD/bILa+3i/5GrflgfROIyV1O0fnmH9lwcfQLfF1T
jLL1hX9IJ39spx/kEn/Udb4rKmLWOv6L+2QHfTn0kd96ymJasUsSe5RR5FTuVgU74XK08bVBO1S0z7ixSA4FOJxCxTLWKAqhKaXA6x1J9ZHhI
kbxUc7dqmpJgRgdqKipzL9ManZ8DtW39SkcZoXzNN3vlLX+AYBhbydf9ZYXTtwOMTtRzb+PFaz1FylktVD9FzY5QNzmuLTzDm8kngf13Rh09H
fLuTWP60dx7uaz2RrWm4OwgcTZaP14DOmO0dpnxLPuWZvOLkFhbr0Unx06J11o/VdV2mkiJ9tgcOLD9zjyWdaMb7l7DoC4ivjttx34YHLmvWm
SPi15CEi2eQIQhBMhCEAvJdZLrUrft+uE2XrV5PrKYTSNI2SD9x6LHWIu+cN/JWSLNVSYWk/G9XHG7VnZ6K8et6qaHtK3tCMomOP6zu6PeeS2
BeI6p6HBSvmIzkksPssFveXcl7dd14xvO5Y119AflFKdpikBHhiFjzJ5LICML7LS+u+txaSbHf5uCMcC5z3H0LVms4uV39MxKyxuponXCbram
MyKoe9qrt7rlbIuoJGoJyb+aYUkBuLFOsqgChfGQp7JS26KWI9o3CcnD2T8FqfUfWMFTMx2Uj4hh8cDrvHHMHyKygRkZheg6L6RdDVQzAkOZI
wkja3EA4ebSR5oNM67W92mPjL6YPvWVYsx45LYuumkLqSOQD2JC0ncHjL1b6rEo5CCB+LrG0NqS9Towd1em6AutXxeJeObHLzOjnXbcbR/2vR
9Cu7Ww/btzBHxWdOtcn8tkQhC2eUIQhBMkSpECrhdNosVDL4BruTh8LruKlpuHHTTM3xPH+0oQ+dKtveJ8VO4WsotIHPI7bk8NikjaXYRvsOa
4s9FeN66JUvZUMDP8trjxf3z+8uuo4Y8LWt+qA3kLJZJA0FzjYNBJ1AAXJK7eeXzN1jdo/SlU92Z7Zzbbmss1oH6oC8vKDe9l1dOaQ7SrqJL
lwfPK8E67Oe4jLgQqxz8Qu0VIhkggJ/YlBYUCxuSvZdMAUjSdyCu02KsDNI9oOdkxrcPBWBJqTgE0FOCIcpaWSzgdxuFBZPY3aivpzTlC2uoH
x6+0iDmfawhzT7ua+baiAskLSMwfUL6P6E1nbaOpZL/wByxp+0wYD6tKyTrZ0N+T1pkbk2YdoLbDqcOefms5d1lw9Ev+j5jzXpujb8NVCR+dj
9XAfFeR0XL6W5L1/RZuKqg/Sx+jgVlEet5n4trQhC1eYIQkQTIQkQKo6n2HX+q73FPUdXGXxvaDYua5oO4kEAoPnLSIFzzVnQbQZ4A7V2sd+B
eLqppKF7C5rsnNcWuG4tyI53S0TiC031WPIriz0V4+kSub0jpxLSTxl+AOieC617C2u23gr0Uoe0PBBDgHAjUQRcELk9LJ8FHId4Df2nALt53
zJpbRz45HA55625qm5zme0CN1wW35617vS7A6S+oW1ACyrsiAFregHou9Dxrasf9WKeKgL1clFE7WyM8WtuoHaHg/NM8iR7imkecEgO1KCu0/
QUWxpHB7viSojoEbJHjkfgg5mPwRfwXVg0Owg3kfcEi4w/Fqf/AFI0f3z+TfuVRxb+BT2uHjyXZ/qIHVM4cQ37k9vR1h1zX/Uag4waCntaF3W
dGIvrPdw7Mfwqwzo5FvkHmz/5RWldS9djoXwn+5lNvsyDF78Sl64NGdrQdqB3oXg/qP7p9cPJHVXo+OCGYMLiXPbcuOwN7uQAH1l7HSNG2eGS
F/syMcw+Ytfy1riVh8z6Jd3ive9BGYq6EHY4u/ZY4+8BeOpaXsp3sPtNe5p4gkWXtOgcobXxX2lw5scB6rL9N/w2JIhC0YBCEIJUIQgEIQEGC
dMWB1ZPhGXay88Zv63XEjFtS7XSOW88p3yPP7Ti74riFxtuvq4/gLO0vRRqfVhp0vBo3m9ml8V9gv3mcM7jzXX6w5SKVo3yi/k1xWb9XtVhr4
DvcW5f4mlvxuva9ZNYLMiB1XceJ1envXeNnkj1nNa8E61CB5JKplzdRCEHXfmtWR8ksY9pwUJrmfRYXHgpLRN+iL80oqzqYxAxssrvZiDfEoe
Hj2njgEPLzm52EeCoVVXGwXv5n4DaqLLJWjW62Z96l/KIzrmA4Bcemrb94APG4/cr8VfCfahA8lNi0JKbbIT5qVj6Xf6lMidSu+i0cRZWmUcJ
9lreQVD4BEfYfycV0IvtX4rnCmG8Hw1KzA0bviqjR+r/ANiTiz+JeuC8d1fO7sg+yfen9ZmmHU1FhYSHzvEQI1hpBL7bshb9ZZX66iNsw0wGv
ramRhu108jmneMRzB3K10dhc+rhazJxlZY7rOBJ8gCVyWOtkuroaYxzxSA2LZGHyDhccrhYb9emY+LdkIKRbPMEISIJ0iEIBCEIPnzTfzjuP3
KppP5hn6RnvSIWV+vVj4v9X3/v0/6Rv7q9X0/+fPH4NQhaY2WTrw1ZrCgKRC2YkZrV6HUlQkI5+kta8vpP2jwQhc2VPoL2lfrkIVjgrhdPRGt
CEgdSr1hXaLUEIXSNC6v9cn2R7wuf1zfNUn6Z37gQhZXd16zpvtcl06f228W+9Ihef7er8t7TUIW7yBCEIP/Z"

```
    ],
    "displayInTerminal": true,
    "discountIds": [],
    "isArchived": false,
    "lastUpdated": "2026-03-30T12:17:11.329Z",
    "createdAt": "2026-03-29T14:08:55.328Z",
    "updatedAt": "2026-03-30T12:17:11.329Z",
    "__v": 0
  }
}
```


FILE: pos/backend/utils/emailService.js
==========

```
const nodemailer = require("nodemailer");
const usesBrevo = () => !!process.env.BREVO_API_KEY;
const sendViaBrevoAPI = async (to, subject, text, html) => {
  const response = await fetch("https://api.brevo.com/v3/smtp/email", {
    method: "POST",
    headers: {
      "accept": "application/json",
      "api-key": process.env.BREVO_API_KEY,
      "content-type": "application/json"
    },
    body: JSON.stringify({
      sender: {
        name: process.env.STORE_NAME || "POS System",
        email: process.env.EMAIL_FROM
      },
      to: [{ email: to }],
      subject,
      htmlContent: html || `<p>${text}</p>`
    })
  });
  if (!response.ok) {
    const error = await response.json();
    throw new Error(JSON.stringify(error));
  }
  return await response.json();
};
let gmailTransporter = null;
const getGmailTransporter = () => {
  if (!gmailTransporter) {
    gmailTransporter = nodemailer.createTransport({
      service: process.env.EMAIL_SERVICE || "gmail",
      auth: {
        user: process.env.EMAIL_USER,
        pass: process.env.EMAIL_PASS
      }
    });
    console.log("[EmailService] Using Gmail for emails");
  }
  return gmailTransporter;
};
const sendEmail = async (to, subject, text, html) => {
  try {
    if (usesBrevo()) {
      await sendViaBrevoAPI(to, subject, text, html);
      console.log("[EmailService] Email sent via Brevo API to:", to);
      return { success: true };
    }
    const fromEmail = process.env.EMAIL_FROM || process.env.EMAIL_USER;
    const storeName = process.env.STORE_NAME || "POS System";
    const mailOptions = {
      from: `"${storeName}" <${fromEmail}>`,
      to,
      subject,
      text,
      html
    };
    const info = await getGmailTransporter().sendMail(mailOptions);
    console.log("[EmailService] Email sent via Gmail to:", to);
    return { success: true, info };
  } catch (error) {
    console.error("Error sending email:", error);
    return { success: false, error };
  }
```

```
};
module.exports = { sendEmail, usesBrevo };
```

FILE: pos/backend/utils/encryption.js
==========

```
const crypto = require("crypto");
const ALGORITHM = "aes-256-gcm";
const IV_LENGTH = 16;
const KEY_LENGTH = 32;
function getEncryptionKey() {
  const secret = process.env.ENCRYPTION_SECRET;
  if (!secret) {
    throw new Error(
      `ENCRYPTION_SECRET environment variable is required. Generate one with: node -e
"console.log(require('crypto').randomBytes(32).toString('hex'))"`
    );
  }
  return crypto.createHash("sha256").update(secret).digest();
}
function encrypt(plaintext) {
  if (!plaintext || typeof plaintext !== "string") {
    throw new Error("Plaintext must be a non-empty string");
  }
  const key = getEncryptionKey();
  const iv = crypto.randomBytes(IV_LENGTH);
  const cipher = crypto.createCipheriv(ALGORITHM, key, iv);
  let encrypted = cipher.update(plaintext, "utf8", "hex");
  encrypted += cipher.final("hex");
  const authTag = cipher.getAuthTag().toString("hex");
  return {
    encrypted,
    iv: iv.toString("hex"),
    authTag
  };
}
function decrypt(encrypted, ivHex, authTagHex) {
  if (!encrypted || !ivHex || !authTagHex) {
    throw new Error(
      "All three parameters (encrypted, iv, authTag) are required"
    );
  }
  const key = getEncryptionKey();
  const iv = Buffer.from(ivHex, "hex");
  const authTag = Buffer.from(authTagHex, "hex");
  const decipher = crypto.createDecipheriv(ALGORITHM, key, iv);
  decipher.setAuthTag(authTag);
  let decrypted = decipher.update(encrypted, "hex", "utf8");
  decrypted += decipher.final("utf8");
  return decrypted;
}
module.exports = { encrypt, decrypt };
```

FILE: pos/backend/utils/getModel.js
==========

```
const mongoose = require("mongoose");
const getProductModel = () => {
  const ProductModule = require("../models/Product");
  if (mongoose.models.Product) {
    return mongoose.models.Product;
  }
  return mongoose.model("Product", ProductModule.schema);
};
const getStockMovementModel = () => {
  const StockMovementModule = require("../models/StockMovement");
  if (mongoose.models.StockMovement) {
    return mongoose.models.StockMovement;
  }
  return mongoose.model("StockMovement", StockMovementModule.schema);
};
const getEmployeeModel = () => {
  const EmployeeModule = require("../models/Employee");
  if (mongoose.models.Employee) {
    return mongoose.models.Employee;
```

```
  }
  return mongoose.model("Employee", EmployeeModule.schema);
};
const getTransactionModel = () => {
  const SalesTransactionModule = require("../models/SalesTransaction");
  if (mongoose.models.SalesTransaction) {
    return mongoose.models.SalesTransaction;
  }
  return mongoose.model("SalesTransaction", SalesTransactionModule.schema);
};
const getCartModel = () => {
  const CartModule = require("../models/Cart");
  if (mongoose.models.Cart) {
    return mongoose.models.Cart;
  }
  return mongoose.model("Cart", CartModule.schema);
};
const getArchiveModel = () => {
  const ArchiveModule = require("../models/Archive");
  if (mongoose.models.Archive) {
    return mongoose.models.Archive;
  }
  return mongoose.model("Archive", ArchiveModule.schema);
};
const getDiscountModel = () => {
  const DiscountModule = require("../models/Discount");
  if (mongoose.models.Discount) {
    return mongoose.models.Discount;
  }
  return mongoose.model("Discount", DiscountModule.schema);
};
const getCategoryModel = () => {
  const CategoryModule = require("../models/Category");
  if (mongoose.models.Category) {
    return mongoose.models.Category;
  }
  return mongoose.model("Category", CategoryModule.schema);
};
const getModel = (modelName) => {
  const modelMap = {
    "Product": getProductModel,
    "StockMovement": getStockMovementModel,
    "Employee": getEmployeeModel,
    "SalesTransaction": getTransactionModel,
    "Cart": getCartModel,
    "Archive": getArchiveModel,
    "Discount": getDiscountModel,
    "Category": getCategoryModel
  };
  const getter = modelMap[modelName];
  if (!getter) {
    throw new Error(`Unknown model: ${modelName}`);
  }
  return getter();
};
module.exports = {
  getProductModel,
  getStockMovementModel,
  getEmployeeModel,
  getTransactionModel,
  getCartModel,
  getArchiveModel,
  getDiscountModel,
  getCategoryModel,
  getModel
};
```

FILE: pos/backend/utils/modelFactory.js
==========

```
const dbManager = require("../config/databaseManager");
const getModel = (modelName, schema) => {
  const connection = dbManager.getConnection();
  if (!connection) {
    throw new Error("No database connection available");
  }
```

```
  if (connection.models[modelName]) {
    return connection.models[modelName];
  }
  return connection.model(modelName, schema);
};
const getModelForConnection = (connection, modelName, schema) => {
  if (!connection) {
    throw new Error("Connection not provided");
  }
  if (connection.models[modelName]) {
    return connection.models[modelName];
  }
  return connection.model(modelName, schema);
};
module.exports = {
  getModel,
  getModelForConnection
};
```

FILE: pos/frontend/eslint.config.js
==========

```
import js from "@eslint/js";
import globals from "globals";
import reactHooks from "eslint-plugin-react-hooks";
import reactRefresh from "eslint-plugin-react-refresh";
import { defineConfig, globalIgnores } from "eslint/config";
export default defineConfig([
  globalIgnores(["dist"]),
  {
    files: ["**/*.{js,jsx}"],
    extends: [
      js.configs.recommended,
      reactHooks.configs["recommended-latest"],
      reactRefresh.configs.vite
    ],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
      parserOptions: {
        ecmaVersion: "latest",
        ecmaFeatures: { jsx: true },
        sourceType: "module"
      }
    },
    rules: {
      "no-unused-vars": ["warn", { varsIgnorePattern: "^[A-Z_]" }],
      "react-refresh/only-export-components": "warn",
      "no-case-declarations": "warn",
      "no-empty": "warn"
    }
  }
]);
```

FILE: pos/frontend/package.json
==========

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "@tailwindcss/vite": "^4.1.16",
    "date-fns": "^4.1.0",
    "framer-motion": "^11.18.2",
    "html2canvas": "^1.4.1",
    "jspdf": "^3.0.4",
    "qrcode.react": "^4.2.0",
```

```
    "react": "^19.1.1",
    "react-datepicker": "^9.1.0",
    "react-dom": "^19.1.1",
    "react-hot-toast": "^2.6.0",
    "react-icons": "^5.5.0",
    "react-is": "^19.2.4",
    "react-router-dom": "^7.9.5",
    "recharts": "^3.3.0",
    "tailwindcss": "^4.1.16",
    "xlsx": "^0.18.5"
  },
  "overrides": {
    "react": "^19.1.1",
    "react-dom": "^19.1.1"
  },
  "devDependencies": {
    "@eslint/js": "^9.36.0",
    "@types/react": "^19.1.16",
    "@types/react-dom": "^19.1.9",
    "@vitejs/plugin-react": "^5.0.4",
    "eslint": "^9.36.0",
    "eslint-plugin-react-hooks": "^5.2.0",
    "eslint-plugin-react-refresh": "^0.4.22",
    "globals": "^16.4.0",
    "vite": "^7.1.7",
    "vite-plugin-pwa": "^1.2.0"
  }
}


FILE: pos/frontend/src/App.jsx
================================================================================
import { lazy, memo, Suspense, useCallback, useEffect, useState, Component } from "react";
import { Toaster } from "react-hot-toast";
import { Route, BrowserRouter as Router, Routes } from "react-router-dom";
import "./App.css";
import PageTitle from "./components/shared/PageTitle";
import ProtectedRoute from "./components/shared/ProtectedRoute";
import Sidebar from "./components/shared/Sidebar";
import { AuthProvider } from "./context/AuthContext";
import { DataCacheProvider } from "./context/DataCacheContext";
import { SidebarContext } from "./context/SidebarContext";
import { ThemeProvider, useTheme } from "./context/ThemeContext";
import { API_ENDPOINTS } from "./config/api";
const lazyWithRetry = (componentImport) => {
  return lazy(async () => {
    const pageHasAlreadyBeenForceRefreshed = JSON.parse(
      window.sessionStorage.getItem("page-has-been-force-refreshed") || "false"
    );
    try {
      const component = await componentImport();
      window.sessionStorage.setItem("page-has-been-force-refreshed", "false");
      return component;
    } catch (error) {
      if (!pageHasAlreadyBeenForceRefreshed) {
        window.sessionStorage.setItem("page-has-been-force-refreshed", "true");
        window.location.reload();
        return { default: () => null };
      }
      throw error;
    }
  });
};
class ErrorBoundary extends Component {
  constructor(props) {
    super(props);
    this.state = { hasError: false, error: null };
  }
  static getDerivedStateFromError(error) {
    return { hasError: true, error };
  }
  componentDidCatch(error, errorInfo) {
    console.error("ErrorBoundary caught an error:", error, errorInfo);
  }
  handleRetry = () => {
    window.sessionStorage.removeItem("page-has-been-force-refreshed");
    window.location.reload();
```

```
  };
  render() {
    if (this.state.hasError) {
      return /* @__PURE__ */ React.createElement("div", { className: "min-h-screen flex items-center justify-
center bg-gray-50" }, /* @__PURE__ */ React.createElement("div", { className: "text-center p-8" }, /*
@__PURE__ */ React.createElement("div", { className: "text-6xl mb-4" }, "\u26A0\uFE0F"), /* @__PURE__ */
React.createElement("h1", { className: "text-2xl font-bold text-gray-800 mb-2" }, "Something went wrong"), /*
@__PURE__ */ React.createElement("p", { className: "text-gray-600 mb-6" }, "The page failed to load. This
usually happens after an update."), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: this.handleRetry,
          className: "px-6 py-3 bg-[#AD7F65] text-white rounded-lg hover:bg-[#8B6B4F] transition-colors font-
semibold"
        },
        "Refresh Page"
      )));
    }
    return this.props.children;
  }
}
const StaffSelection = lazyWithRetry(() => import("./pages/StaffSelection"));
const PinEntry = lazyWithRetry(() => import("./pages/PinEntry"));
const Inventory = lazyWithRetry(() => import("./pages/Inventory"));
const Logs = lazyWithRetry(() => import("./pages/logs"));
const Terminal = lazyWithRetry(() => import("./pages/terminal"));
const Transaction = lazyWithRetry(() => import("./pages/transaction"));
const CashRemittance = lazyWithRetry(() => import("./pages/CashRemittance"));
const Settings = lazyWithRetry(() => import("./pages/Settings"));
const Dashboard = lazyWithRetry(() => import("./pages/owner/Dashboard"));
const Reports = lazyWithRetry(() => import("./pages/owner/Reports"));
const ManageEmployees = lazyWithRetry(() => import("./pages/owner/ManageEmployees"));
const DiscountManagement = lazyWithRetry(
  () => import("./pages/owner/DiscountManagement")
);
const BrandPartners = lazyWithRetry(() => import("./pages/owner/BrandPartners"));
const Categories = lazyWithRetry(() => import("./pages/owner/Categories"));
const SetNewPin = lazyWithRetry(() => import("./pages/SetNewPin"));
const OwnerOnboarding = lazyWithRetry(() => import("./pages/OwnerOnboarding"));
const ManageData = lazyWithRetry(() => import("./pages/ManageData"));
const PageLoader = () => /* @__PURE__ */ React.createElement("div", { className: "min-h-screen flex items-
center justify-center bg-[#FFFFFF]" }, /* @__PURE__ */ React.createElement("div", { className: "text-
center" }, /* @__PURE__ */ React.createElement("div", { className: "inline-block animate-spin rounded-full h-
12 w-12 border-b-2 border-[#8B7355] mb-4" }), /* @__PURE__ */ React.createElement("p", { className: "text-
gray-600" }, "Loading..."")));
const MainLayout = memo(({ children }) => {
  const [isExpanded, setIsExpanded] = useState(false);
  const { theme } = useTheme();
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "min-h-screen transition-colors duration-300",
      style: { backgroundColor: theme === "dark" ? "#1E1B18" : "#FFFFFF" }
    },
    /* @__PURE__ */ React.createElement(SidebarContext.Provider, { value: { isExpanded } }, /* @__PURE__ */
React.createElement(Sidebar, { isExpanded, setIsExpanded }), /* @__PURE__ */ React.createElement(
      "main",
      {
        className: `transition-all duration-300 px-6 py-4 ${isExpanded ? "ml-80" : "ml-20"}`
      },
      children
    ))
  );
});
const LandingGate = () => {
  const { theme } = useTheme();
  const [status, setStatus] = useState({
    loading: true,
    error: "",
    hasAccounts: false
  });
  const checkEmployees = useCallback(async () => {
    setStatus((prev) => ({
      ...prev,
      loading: true,
      error: ""
    }));
```

```
    try {
      const response = await fetch(API_ENDPOINTS.employees);
      const data = await response.json();
      if (!data.success) {
        throw new Error(data.message || "Failed to determine account status.");
      }
      setStatus({
        loading: false,
        error: "",
        hasAccounts: data.count > 0
      });
    } catch (error) {
      setStatus({
        loading: false,
        error: error.message || "Unable to reach the server. Please ensure the backend is running.",
        hasAccounts: false
      });
    }
  }, []);
  useEffect(() => {
    checkEmployees();
  }, [checkEmployees]);
  if (status.loading) {
    return /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `min-h-screen flex items-center justify-center ${theme === "dark" ? "bg-[#1F1F1F]" : "bg-
[#FFFFFF]"}`
      },
      /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `${theme === "dark" ? "text-white" : "text-[#8B7355]"} tracking-[0.3em] uppercase text-
sm`
        },
        "Preparing CYSPOS..."
      )
    );
  }
  if (status.error) {
    return /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `min-h-screen flex flex-col items-center justify-center ${theme === "dark" ? "bg-
[#1F1F1F]" : "bg-[#FFFFFF]"} px-4 text-center gap-6`
      },
      /* @__PURE__ */ React.createElement("div", { className: "max-w-md" }, /* @__PURE__ */
React.createElement(
        "p",
        {
          className: `${theme === "dark" ? "text-white" : "text-[#2D2D2D]"} text-lg font-semibold mb-2`
        },
        "Something went wrong"
      ), /* @__PURE__ */ React.createElement("p", { className: "text-gray-400 text-sm" }, status.error)),
      /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: checkEmployees,
          className: "px-6 py-3 rounded-2xl bg-white text-[#1F1F1F] font-semibold shadow-lg"
        },
        "Retry"
      )
    );
  }
  if (!status.hasAccounts) {
    return /* @__PURE__ */ React.createElement(Suspense, { fallback: /* @__PURE__ */
React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(OwnerOnboarding,
{ onSetupComplete: checkEmployees }));
  }
  return /* @__PURE__ */ React.createElement(Suspense, { fallback: /* @__PURE__ */
React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(PinEntry, null));
};
function App() {
  useEffect(() => {
    const handleWheel = (e) => {
      if (e.target.type === "number") {
        e.preventDefault();
```

```
      }
    };
    document.addEventListener("wheel", handleWheel, { passive: false });
    const handleNumberInputWheel = (e) => {
      if (document.activeElement.type === "number") {
        document.activeElement.blur();
      }
    };
    document.addEventListener("wheel", handleNumberInputWheel);
    return () => {
      document.removeEventListener("wheel", handleWheel);
      document.removeEventListener("wheel", handleNumberInputWheel);
    };
  }, []);
  return /* @__PURE__ */ React.createElement(ErrorBoundary, null, /* @__PURE__ */
React.createElement(ThemeProvider, null, /* @__PURE__ */ React.createElement(AuthProvider, null, /* @__PURE__
*/ React.createElement(DataCacheProvider, null, /* @__PURE__ */ React.createElement(Router, null, /*
@__PURE__ */ React.createElement(
    Toaster,
    {
      position: "top-center",
      reverseOrder: false,
      toastOptions: { style: { zIndex: 99999 } }
    }
  ), /* @__PURE__ */ React.createElement(
    Toaster,
    {
      toasterId: "terminal",
      position: "bottom-right",
      reverseOrder: false,
      toastOptions: { style: { zIndex: 99999 } }
    }
  ), /* @__PURE__ */ React.createElement(PageTitle, null), /* @__PURE__ */ React.createElement(Suspense,
{ fallback: /* @__PURE__ */ React.createElement(PageLoader, null) }, /* @__PURE__ */
React.createElement(Routes, null, /* @__PURE__ */ React.createElement(Route, { path: "/", element: /*
@__PURE__ */ React.createElement(LandingGate, null) }), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/pin",
      element: /* @__PURE__ */ React.createElement(Suspense, { fallback: /* @__PURE__ */
React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(PinEntry, null))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/staff",
      element: /* @__PURE__ */ React.createElement(Suspense, { fallback: /* @__PURE__ */
React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(StaffSelection, null))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/set-pin",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, null, /* @__PURE__ */
React.createElement(Suspense, { fallback: /* @__PURE__ */ React.createElement(PageLoader, null) }, /*
@__PURE__ */ React.createElement(SetNewPin, null)))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/dashboard",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, { ownerOnly: true }, /* @__PURE__ */
React.createElement(MainLayout, null, /* @__PURE__ */ React.createElement(Suspense, { fallback: /* @__PURE__
*/ React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(Dashboard, null))))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/reports",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, { requiredPermission: "generateReports" },
/* @__PURE__ */ React.createElement(MainLayout, null, /* @__PURE__ */ React.createElement(Suspense,
{ fallback: /* @__PURE__ */ React.createElement(PageLoader, null) }, /* @__PURE__ */
React.createElement(Reports, null))))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
```

```
      path: "/manage-employees",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, { ownerOnly: true }, /* @__PURE__ */
React.createElement(MainLayout, null, /* @__PURE__ */ React.createElement(Suspense, { fallback: /* @__PURE__
*/ React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(ManageEmployees, null))))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/inventory",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, { requiredPermission: "inventory" }, /*
@__PURE__ */ React.createElement(MainLayout, null, /* @__PURE__ */ React.createElement(Suspense, { fallback:
/* @__PURE__ */ React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(Inventory,
null))))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/stock-movement",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, { requiredPermission: "inventory" }, /*
@__PURE__ */ React.createElement(MainLayout, null, /* @__PURE__ */ React.createElement(Suspense, { fallback:
/* @__PURE__ */ React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(Logs, null))))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/terminal",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, { requiredPermission: "posTerminal" }, /*
@__PURE__ */ React.createElement(MainLayout, null, /* @__PURE__ */ React.createElement(Suspense, { fallback:
/* @__PURE__ */ React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(Terminal,
null))))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/transactions",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, { requiredPermission: "viewTransactions" },
/* @__PURE__ */ React.createElement(MainLayout, null, /* @__PURE__ */ React.createElement(Suspense,
{ fallback: /* @__PURE__ */ React.createElement(PageLoader, null) }, /* @__PURE__ */
React.createElement(Transaction, null))))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/cash-remittance",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, { requiredPermission: "viewTransactions" },
/* @__PURE__ */ React.createElement(MainLayout, null, /* @__PURE__ */ React.createElement(Suspense,
{ fallback: /* @__PURE__ */ React.createElement(PageLoader, null) }, /* @__PURE__ */
React.createElement(CashRemittance, null))))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/settings",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, { requiredPermission: null }, /* @__PURE__
*/ React.createElement(MainLayout, null, /* @__PURE__ */ React.createElement(Suspense, { fallback: /*
@__PURE__ */ React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(Settings, null))))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/manage-data",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, { requiredPermission: null }, /* @__PURE__
*/ React.createElement(MainLayout, null, /* @__PURE__ */ React.createElement(Suspense, { fallback: /*
@__PURE__ */ React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(ManageData, null))))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/discount-management",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, { requiredPermission: null }, /* @__PURE__
*/ React.createElement(MainLayout, null, /* @__PURE__ */ React.createElement(Suspense, { fallback: /*
@__PURE__ */ React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(DiscountManagement,
null))))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
```

```
      path: "/brand-partners",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, { requiredPermission: null }, /* @__PURE__
*/ React.createElement(MainLayout, null, /* @__PURE__ */ React.createElement(Suspense, { fallback: /*
@__PURE__ */ React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(BrandPartners,
null))))
    }
  ), /* @__PURE__ */ React.createElement(
    Route,
    {
      path: "/categories",
      element: /* @__PURE__ */ React.createElement(ProtectedRoute, { requiredPermission: null }, /* @__PURE__
*/ React.createElement(MainLayout, null, /* @__PURE__ */ React.createElement(Suspense, { fallback: /*
@__PURE__ */ React.createElement(PageLoader, null) }, /* @__PURE__ */ React.createElement(Categories, null))))
    }
  ))))))));
}
export default App;
```

FILE: pos/frontend/src/components/inventory/AddBrandModal.jsx
==========

```
import { useState } from "react";
import { useTheme } from "../../context/ThemeContext";
import { API_BASE_URL } from "../../config/api";
const AddBrandModal = ({ show, onClose, onAdd }) => {
  const { theme } = useTheme();
  const [newBrand, setNewBrand] = useState("");
  const [loading, setLoading] = useState(false);
  if (!show) return null;
  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!newBrand.trim()) return;
    setLoading(true);
    try {
      const response = await fetch(`${API_BASE_URL}/api/brand-partners`, {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({ brandName: newBrand.trim() })
      });
      const data = await response.json();
      if (data.success) {
        onAdd(data.data?.brandName || newBrand.trim());
        setNewBrand("");
        onClose();
      } else {
        alert(data.message || "Failed to add brand");
      }
    } catch (error) {
      console.error("Error adding brand:", error);
      alert("Failed to add brand. Please try again.");
    } finally {
      setLoading(false);
    }
  };
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10000] p-4 bg-black/50 backdrop-blur-sm pointer-events-auto" }, /* @__PURE__ */
React.createElement("div", { className: `rounded-2xl w-full max-w-md p-6 shadow-2xl transform transition-all
scale-100 opacity-100 ${theme === "dark" ? "bg-[#2A2724] text-white" : "bg-white"}` }, /* @__PURE__ */
React.createElement("h2", { className: "text-2xl font-bold mb-6" }, "Add Brand"), /* @__PURE__ */
React.createElement("form", { onSubmit: handleSubmit }, /* @__PURE__ */ React.createElement("div",
{ className: "mb-6" }, /* @__PURE__ */ React.createElement("label", { className: `block text-sm font-medium
mb-2 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Brand Name"), /* @__PURE__ */
React.createElement(
    "input",
    {
      type: "text",
      value: newBrand,
      onChange: (e) => setNewBrand(e.target.value),
      placeholder: "Enter brand name",
      className: `w-full px-4 py-3 rounded-xl border focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
focus:border-transparent transition-all ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white
placeholder-gray-300" : "border-gray-300"}`,
      autoFocus: true
    }
```

```
  )), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3 justify-end" }, /* @__PURE__ */
React.createElement(
    "button",
    {
      type: "button",
      onClick: onClose,
      className: `px-6 py-2.5 rounded-xl font-medium transition-colors ${theme === "dark" ? "bg-gray-700
text-gray-300 hover:bg-gray-600" : "bg-gray-100 text-gray-700 hover:bg-gray-200"}`
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "submit",
      disabled: loading || !newBrand.trim(),
      className: "px-6 py-2.5 rounded-xl bg-green-600 text-white font-medium hover:bg-green-700
disabled:opacity-50 disabled:cursor-not-allowed transition-all shadow-lg"
    },
    loading ? "Adding..." : "Add Brand"
  )))));
};
export default AddBrandModal;
```

FILE: pos/frontend/src/components/inventory/AddCategoryModal.jsx
==========

```
import { useState } from "react";
import { useTheme } from "../../context/ThemeContext";
const AddCategoryModal = ({ show, onClose, onAdd }) => {
  const { theme } = useTheme();
  const [newCategory, setNewCategory] = useState("");
  const [loading, setLoading] = useState(false);
  if (!show) return null;
  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!newCategory.trim()) return;
    setLoading(true);
    try {
      const response = await fetch("http://localhost:5000/api/categories", {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          name: newCategory,
          type: "category",
          parentCategory: null
        })
      });
      const data = await response.json();
      if (data.success) {
        onAdd(newCategory);
        setNewCategory("");
        onClose();
      } else {
        alert(data.message || "Failed to add category");
      }
    } catch (error) {
      console.error("Error adding category:", error);
      alert("Failed to add category. Please try again.");
    } finally {
      setLoading(false);
    }
  };
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-10000 p-4 bg-black/50 backdrop-blur-sm pointer-events-auto" }, /* @__PURE__ */
React.createElement("div", { className: `rounded-2xl w-full max-w-md p-6 shadow-2xl transform transition-all
scale-100 opacity-100 ${theme === "dark" ? "bg-[#2A2724] text-white" : "bg-white"}` }, /* @__PURE__ */
React.createElement("h2", { className: "text-2xl font-bold mb-6" }, "Add Category"), /* @__PURE__ */
React.createElement("form", { onSubmit: handleSubmit }, /* @__PURE__ */ React.createElement("div",
{ className: "mb-6" }, /* @__PURE__ */ React.createElement("label", { className: `block text-sm font-medium
mb-2 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Category Name"), /* @__PURE__ */
React.createElement(
    "input",
    {
      type: "text",
```

```
      value: newCategory,
      onChange: (e) => setNewCategory(e.target.value),
      placeholder: "Enter category name",
      className: `w-full px-4 py-3 rounded-xl border focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
focus:border-transparent transition-all ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white
placeholder-gray-300" : "border-gray-300"}`,
      autoFocus: true
    }
  )), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3 justify-end" }, /* @__PURE__ */
React.createElement(
    "button",
    {
      type: "button",
      onClick: onClose,
      className: `px-6 py-2.5 rounded-xl font-medium transition-colors ${theme === "dark" ? "bg-gray-700
text-gray-300 hover:bg-gray-600" : "bg-gray-100 text-gray-700 hover:bg-gray-200"}`
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "submit",
      disabled: loading || !newCategory.trim(),
      className: "px-6 py-2.5 rounded-xl bg-green-600 text-white font-medium hover:bg-green-700
disabled:opacity-50 disabled:cursor-not-allowed transition-all shadow-lg"
    },
    loading ? "Adding..." : "Add Category"
  )))));
};
export default AddCategoryModal;
```

FILE: pos/frontend/src/components/inventory/AddProductModal.jsx
==========

```
import { useEffect, useState } from "react";
import { useTheme } from "../../context/ThemeContext";
import AddBrandModal from "./AddBrandModal";
import AddCategoryModal from "./AddCategoryModal";
import AddSubcategoryModal from "./AddSubcategoryModal";
const COMMON_COLORS = [
  "Black",
  "White",
  "Red",
  "Blue",
  "Navy Blue",
  "Green",
  "Yellow",
  "Orange",
  "Pink",
  "Purple",
  "Brown",
  "Gray",
  "Beige",
  "Cream",
  "Maroon",
  "Olive",
  "Teal",
  "Coral",
  "Lavender",
  "Mint",
  "Gold",
  "Silver",
  "Rose Gold",
  "Custom"
];
const BRAND_ADD_SENTINEL = "__add_new_brand__";
const UNIT_OF_MEASURE_OPTIONS = [
  { value: "pcs", label: "Pieces (pcs)" },
  { value: "kg", label: "Kilograms (kg)" },
  { value: "g", label: "Grams (g)" },
  { value: "L", label: "Liters (L)" },
  { value: "ml", label: "Milliliters (ml)" },
  { value: "mg", label: "Milligrams (mg)" },
  { value: "packs", label: "Packs" },
  { value: "boxes", label: "Boxes" },
  { value: "bundle", label: "Bundle" }
```

```
];
const AddProductModal = ({
  showAddModal,
  setShowAddModal,
  editingProduct,
  newProduct,
  setNewProduct,
  handleAddProduct,
  handleInputChange,
  handleSizeToggle,
  handleSizeQuantityChange,
  handleSizePriceChange,
  resetProductForm,
  loading,
  categories = [],
  brandPartners = [],
  onCategoryAdd,
  onBrandAdd
}) => {
  const { theme } = useTheme();
  const VARIANT_ONLY_KEY = "__VARIANT_ONLY__";
  const categoryStructure = {
    "Apparel - Men": ["Tops", "Bottoms", "Outerwear"],
    "Apparel - Women": ["Tops", "Bottoms", "Dresses", "Outerwear"],
    "Apparel - Kids": ["Tops", "Bottoms", "Dresses", "Outerwear"],
    "Apparel - Unisex": ["Tops", "Bottoms", "Dresses", "Outerwear"],
    "Foods": ["Beverages", "Snacks", "Meals", "Desserts", "Ingredients", "Other"],
    "Makeup": ["Face", "Eyes", "Lips", "Nails", "SkinCare", "Others"],
    "Accessories": ["Jewelry", "Bags", "Head Wear", "Glasses/Sunglasses", "Others"],
    "Shoes": ["Sneakers", "Boots", "Sandals", "Others"],
    "Essentials": ["Daily Essentials", "Personal Care", "Home Essentials", "Others"],
    "Others": ["Others"]
  };
  const defaultParentCategories = Object.keys(categoryStructure);
  const allKnownDefaultSubs = new Set(Object.values(categoryStructure).flat());
  const legacyParentCategories = ["Apparel", "Shoes", "Foods", "Accessories", "Makeup", "Head Wear",
"Essentials"];
  const customParentCategories = categories.filter(
    (c) => c?.name && c?.name !== "All" && c?.name !== "Others" && !defaultParentCategories.includes(c?.name)
&& !allKnownDefaultSubs.has(c?.name) && !legacyParentCategories.includes(c?.name) && (!c?.type || c?.type ===
"category") && !c?.parentCategory
  ).map((c) => c?.name);
  const parentCategories = [...defaultParentCategories, ...customParentCategories];
  const orphanCustomSubCategories = categories.filter(
    (c) => c?.name !== "All" && c?.name !== "Others" && c?.parentCategory
&& !defaultParentCategories.includes(c?.name) && !allKnownDefaultSubs.has(c?.name)
&& !legacyParentCategories.includes(c?.name)
  ).map((c) => c?.name);
  const getSubcategories = (parentCat) => {
    const defaultSubs = categoryStructure[parentCat] || [];
    const mappedCustomSubCategories = categories.filter((c) => c?.parentCategory === parentCat).map((c) =>
c?.name).filter(Boolean);
    const subs = [...defaultSubs, ...mappedCustomSubCategories, ...orphanCustomSubCategories];
    if (newProduct.subCategory && newProduct.subCategory !== "__add_new__"
&& !subs.includes(newProduct.subCategory)) {
      if (newProduct.category === parentCat) {
        subs.push(newProduct.subCategory);
      }
    }
    return [...new Set(subs)];
  };
  const [showDraftNotice, setShowDraftNotice] = useState(false);
  const [showCategoryModal, setShowCategoryModal] = useState(false);
  const [showSubcategoryModal, setShowSubcategoryModal] = useState(false);
  const [showBrandModal, setShowBrandModal] = useState(false);
  const [showCustomSizeInput, setShowCustomSizeInput] = useState(false);
  const [customSizeValue, setCustomSizeValue] = useState("");
  const [showSizeDropdown, setShowSizeDropdown] = useState(false);
  const [productImages, setProductImages] = useState([]);
  const [imageUrlInput, setImageUrlInput] = useState("");
  const [customSizes, setCustomSizes] = useState([]);
  const [multipleVariantsPerSize, setMultipleVariantsPerSize] = useState({});
  const [sizeMultiVariants, setSizeMultiVariants] = useState({});
  const [selectedVariants, setSelectedVariants] = useState([]);
  const [showVariantDropdown, setShowVariantDropdown] = useState(false);
  const [customColorInput, setCustomColorInput] = useState("");
  const hasVariants = selectedVariants.length > 0 || newProduct.selectedSizes?.length > 0;
```

```
  const sortedParentCategories = (() => {
    const unique = Array.from(new Set(parentCategories.filter(Boolean)));
    const withoutOthers = unique.filter((c) => c !== "Others").sort((a, b) => a.localeCompare(b));
    return [...withoutOthers, ...unique.includes("Others") ? ["Others"] : []];
  })();
  const effectiveParentCategories = sortedParentCategories.includes(newProduct.category) ?
sortedParentCategories : newProduct.category ? [...sortedParentCategories, newProduct.category] :
sortedParentCategories;
  const formatStockInStyleBatchCode = (d = /* @__PURE__ */ new Date()) => {
    const y = d.getFullYear();
    const mo = String(d.getMonth() + 1).padStart(2, "0");
    const day = String(d.getDate()).padStart(2, "0");
    const h = String(d.getHours()).padStart(2, "0");
    const min = String(d.getMinutes()).padStart(2, "0");
    return `B${y}${mo}${day}-${h}${min}`;
  };
  const todayISO = (/* @__PURE__ */ new Date()).toISOString().slice(0, 10);
  const getOpeningStockTotal = () => {
    const combos = [];
    const rvariants = selectedVariants.length > 0 ? selectedVariants : [null];
    const rsizes = newProduct.selectedSizes?.length > 0 ? newProduct.selectedSizes : [VARIANT_ONLY_KEY];
    rvariants.forEach((v) => {
      rsizes.forEach((s) => {
        combos.push({ variant: v, size: s });
      });
    });
    const hasAnyCombos = hasVariants && combos.length > 0 && (combos[0].variant || combos[0].size);
    let total = 0;
    if (hasAnyCombos) {
      combos.forEach(({ variant: v, size: s }) => {
        if (v && s) total += parseInt(newProduct.variantQuantities?.[s]?.[v], 10) || 0;
        else if (s) total += parseInt(newProduct.sizeQuantities?.[s], 10) || 0;
        else total += parseInt(newProduct.currentStock, 10) || 0;
      });
    } else {
      total = parseInt(newProduct.currentStock, 10) || 0;
    }
    return total;
  };
  const [optionGroup1Name, setOptionGroup1Name] = useState("Color");
  const [optionGroup2Name, setOptionGroup2Name] = useState("Size");
  const [variantQuantities, setVariantQuantities] = useState({});
  const [differentPricesPerVariant, setDifferentPricesPerVariant] = useState({});
  const [variantPrices, setVariantPrices] = useState({});
  const [variantCostPrices, setVariantCostPrices] = useState({});
  const [fillAllCost, setFillAllCost] = useState("");
  const [fillAllPrice, setFillAllPrice] = useState("");
  const [fillAllQty, setFillAllQty] = useState("");
  const [currentStep, setCurrentStep] = useState(1);
  const [reviewImgIdx, setReviewImgIdx] = useState(0);
  const STEPS = [
    { id: 1, label: "Basic Info" },
    { id: 2, label: "Variants" },
    { id: 3, label: "Stock and Price" },
    { id: 4, label: "Batch" },
    { id: 5, label: "Review" }
  ];
  const getVariantsForSize = (size) => {
    if (!size) return [];
    if (newProduct.differentVariantsPerSize) {
      if (multipleVariantsPerSize[size]) {
        return (sizeMultiVariants[size] || []).filter(Boolean);
      }
      const single = (newProduct.sizeVariants?.[size] || "").trim();
      return single ? [single] : [];
    }
    return (selectedVariants || []).filter(Boolean);
  };
  const getTotalStockFromInputs = () => {
    if (!hasVariants || !newProduct.selectedSizes || newProduct.selectedSizes.length === 0) {
      return parseInt(newProduct.currentStock) || 0;
    }
    const sizeTotal = Object.values(newProduct.sizeQuantities || {}).reduce(
      (sum, q) => sum + (parseInt(q) || 0),
      0
    );
    const variantTotal = Object.values(newProduct.variantQuantities || {}).reduce(
```

```
      (sum, perSize) => sum + (perSize && typeof perSize === "object" ? Object.values(perSize).reduce((s, q)
=> s + (parseInt(q) || 0), 0) : 0),
      0
    );
    return Math.max(sizeTotal, variantTotal);
  };
  const isStepValid = (step) => {
    if (editingProduct) return true;
    switch (step) {
      case 1:
        if (!newProduct.itemName || !newProduct.itemName.trim()) return false;
        if (!newProduct.category) return false;
        if (!newProduct.subCategory || newProduct.subCategory === "__add_new__") return false;
        return true;
      case 2:
        return true;
      case 3:
        return true;
      case 4: {
        const opening = getOpeningStockTotal();
        if (opening > 0 && !String(newProduct.dateReceived || "").trim()) {
          return false;
        }
        if (opening > 0 && !String(newProduct.reorderNumber ?? "").trim()) {
          return false;
        }
        if (opening > 0 && String(newProduct.expirationDate || "").trim()
&& !String(newProduct.expirationThresholdDays ?? "").trim()) {
          return false;
        }
        return true;
      }
      default:
        return true;
    }
  };
  const [editableSizePrices, setEditableSizePrices] = useState({});
  useEffect(() => {
    if (editingProduct && editingProduct.sizes && typeof editingProduct.sizes === "object") {
      const sizePrices = {};
      Object.entries(editingProduct.sizes).forEach(([size, sizeData]) => {
        if (typeof sizeData === "object" && sizeData !== null) {
          if (sizeData.variants && typeof sizeData.variants === "object") {
            sizePrices[size] = {
              hasVariants: true,
              basePrice: sizeData.price || editingProduct.itemPrice || 0,
              baseCostPrice: sizeData.costPrice || editingProduct.costPrice || 0,
              variants: {}
            };
            Object.entries(sizeData.variants).forEach(([variant, variantData]) => {
              if (typeof variantData === "object" && variantData !== null) {
                sizePrices[size].variants[variant] = {
                  price: variantData.price || sizeData.variantPrices?.[variant] || sizeData.price ||
editingProduct.itemPrice || 0,
                  costPrice: variantData.costPrice || sizeData.variantCostPrices?.[variant] ||
sizeData.costPrice || editingProduct.costPrice || 0,
                  quantity: variantData.quantity || 0
                };
              } else {
                sizePrices[size].variants[variant] = {
                  price: sizeData.variantPrices?.[variant] || sizeData.price || editingProduct.itemPrice || 0,
                  costPrice: sizeData.variantCostPrices?.[variant] || sizeData.costPrice ||
editingProduct.costPrice || 0,
                  quantity: typeof variantData === "number" ? variantData : 0
                };
              }
            });
          } else {
            sizePrices[size] = {
              hasVariants: false,
              price: sizeData.price || editingProduct.itemPrice || 0,
              costPrice: sizeData.costPrice || editingProduct.costPrice || 0,
              quantity: sizeData.quantity || 0
            };
          }
        } else if (typeof sizeData === "number") {
          sizePrices[size] = {
```

```
          hasVariants: false,
          price: editingProduct.itemPrice || 0,
          costPrice: editingProduct.costPrice || 0,
          quantity: sizeData
        };
      }
    });
    setEditableSizePrices(sizePrices);
  } else {
    setEditableSizePrices({});
  }
}, [editingProduct]);
const handleEditableSizePriceChange = (size, price) => {
  setEditableSizePrices((prev) => ({
    ...prev,
    [size]: {
      ...prev[size],
      price: parseFloat(price) || 0
    }
  }));
};
const handleEditableSizeCostPriceChange = (size, costPrice) => {
  setEditableSizePrices((prev) => ({
    ...prev,
    [size]: {
      ...prev[size],
      costPrice: parseFloat(costPrice) || 0
    }
  }));
};
const handleEditableVariantPriceChange = (size, variant, price) => {
  setEditableSizePrices((prev) => ({
    ...prev,
    [size]: {
      ...prev[size],
      variants: {
        ...prev[size]?.variants,
        [variant]: {
          ...prev[size]?.variants?.[variant],
          price: parseFloat(price) || 0
        }
      }
    }
  }));
};
const handleEditableVariantCostPriceChange = (size, variant, costPrice) => {
  setEditableSizePrices((prev) => ({
    ...prev,
    [size]: {
      ...prev[size],
      variants: {
        ...prev[size]?.variants,
        [variant]: {
          ...prev[size]?.variants?.[variant],
          costPrice: parseFloat(costPrice) || 0
        }
      }
    }
  }));
};
const handleVariantPriceChange = (size, variant, price) => {
  const priceValue = parseFloat(price) || 0;
  setVariantPrices((prev) => ({
    ...prev,
    [size]: {
      ...prev[size] || {},
      [variant]: priceValue
    }
  }));
};
const handleVariantCostPriceChange = (size, variant, price) => {
  const priceValue = parseFloat(price) || 0;
  setVariantCostPrices((prev) => ({
    ...prev,
    [size]: {
      ...prev[size] || {},
      [variant]: priceValue
```

```
      }
    }));
  };
  const handleVariantToggle = (color) => {
    setSelectedVariants((prev) => {
      const newVariants = prev.includes(color) ? prev.filter((v) => v !== color) : [...prev, color];
      const variantString = newVariants.join(", ");
      setNewProduct((prevProduct) => ({ ...prevProduct, variant: variantString }));
      return newVariants;
    });
  };
  const addCustomColor = () => {
    const trimmedColor = customColorInput.trim();
    if (trimmedColor && !selectedVariants.includes(trimmedColor)) {
      const newVariants = [...selectedVariants, trimmedColor];
      setSelectedVariants(newVariants);
      const variantString = newVariants.join(", ");
      setNewProduct((prevProduct) => ({ ...prevProduct, variant: variantString }));
      setCustomColorInput("");
    }
  };
  const handleCustomColorKeyDown = (e) => {
    if (e.key === "Enter") {
      e.preventDefault();
      addCustomColor();
    }
  };
  const removeVariant = (color) => {
    setSelectedVariants((prev) => {
      const newVariants = prev.filter((v) => v !== color);
      const variantString = newVariants.join(", ");
      setNewProduct((prevProduct) => ({ ...prevProduct, variant: variantString }));
      return newVariants;
    });
  };
  useEffect(() => {
    if (!showAddModal) {
      setSelectedVariants([]);
      setShowVariantDropdown(false);
      setCustomColorInput("");
      setVariantQuantities({});
      setShowCustomSizeInput(false);
      setCustomSizeValue("");
      setCustomSizes([]);
      setCurrentStep(1);
      setProductImages([]);
      setImageUrlInput("");
      return;
    }
    if (editingProduct) {
      setShowDraftNotice(false);
      const imgs = Array.isArray(editingProduct.productImages) && editingProduct.productImages.length > 0 ?
editingProduct.productImages.filter(Boolean) : editingProduct.itemImage &&
String(editingProduct.itemImage).trim() ? [editingProduct.itemImage] : [];
      setProductImages(imgs);
      return;
    }
    const hasData = newProduct.itemName || newProduct.variant || newProduct.itemPrice || newProduct.costPrice
|| newProduct.currentStock || newProduct.itemImage || newProduct.selectedSizes &&
newProduct.selectedSizes.length > 0;
    setShowDraftNotice(hasData);
    if (newProduct.variant) {
      setSelectedVariants(newProduct.variant.split(", ").filter((v) => v.trim()));
    } else {
      setSelectedVariants([]);
    }
    setProductImages([]);
  }, [showAddModal, editingProduct]);
  useEffect(() => {
    if (!showAddModal) return;
    if (newProduct.brandName === BRAND_ADD_SENTINEL) {
      setNewProduct((prev) => ({ ...prev, brandName: "" }));
      setShowBrandModal(true);
    }
  }, [showAddModal, newProduct.brandName, setNewProduct]);
  if (!showAddModal) return null;
  const partnerNames = Array.from(
```

```
    new Set(brandPartners.map((partner) => partner.brandName).filter(Boolean))
  ).sort((a, b) => a.localeCompare(b));
  const legacyBrandSelected = newProduct.brandName && newProduct.brandName !== "Default"
&& !partnerNames.includes(newProduct.brandName);
  const handleBrandSelectChange = (e) => {
    const { value } = e.target;
    if (value === BRAND_ADD_SENTINEL) {
      setNewProduct((prev) => ({ ...prev, brandName: "" }));
      setShowBrandModal(true);
      return;
    }
    handleInputChange(e);
  };
  const handleFormSubmit = (e) => {
    e.preventDefault();
    const normalizedBrandName = newProduct.brandName && newProduct.brandName !== BRAND_ADD_SENTINEL ?
newProduct.brandName : "Default";
    const completeProductData = {
      ...newProduct,
      brandName: normalizedBrandName,
      productImages,
      ...newProduct.variant === "Custom" && { variant: "Custom" },
      ...Object.keys(sizeMultiVariants).length > 0 && {
        sizeMultiVariants,
        multipleVariantsPerSize
      },
      ...Object.keys(variantQuantities).length > 0 && {
        variantQuantities
      },
      differentPricesPerVariant,
      ...Object.keys(variantPrices).length > 0 && { variantPrices },
      ...Object.keys(variantCostPrices).length > 0 && { variantCostPrices },
      ...editingProduct && Object.keys(editableSizePrices).length > 0 && {
        editableSizePrices
      },
      ...hasVariants && {
        optionGroup1Name: optionGroup1Name || "Color",
        ...newProduct.selectedSizes?.length > 0 && { optionGroup2Name: optionGroup2Name || "Size" }
      }
    };
    setNewProduct(completeProductData);
    requestAnimationFrame(() => {
      handleAddProduct(e);
    });
  };
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-9999 p-4 backdrop-blur-sm pointer-events-none" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-2xl w-full max-w-2xl relative pointer-events-auto shadow-2xl flex flex-col ${theme
=== "dark" ? "bg-[#2A2724] text-white" : "bg-white text-gray-900"}`,
      style: {
        height: "90vh",
        boxShadow: theme === "dark" ? "0 25px 50px -12px rgba(0, 0, 0, 0.7)" : "0 25px 50px -12px rgba(0, 0,
0, 0.5), 0 0 0 1px rgba(0, 0, 0, 0.1)"
      }
    },
    /* @__PURE__ */ React.createElement("div", { className: "flex justify-between items-center px-6 py-4 pb-
2" }, /* @__PURE__ */ React.createElement("h2", { className: "text-xl font-bold" }, editingProduct ? "Edit
Product Details" : "Add New Product"), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => {
          setShowAddModal(false);
        },
        className: `text-2xl hover:text-gray-500 transition-colors ${theme === "dark" ? "text-gray-400" :
"text-gray-400"}`
      },
      "\xD7"
    )),
    showDraftNotice && !editingProduct && /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `mx-6 mt-2 p-3 border rounded-lg flex items-center justify-between ${theme === "dark" ?
"bg-amber-900/20 border-amber-800" : "bg-amber-50 border-amber-200"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-2" }, /* @__PURE__ */
```

```
React.createElement("svg", { className: `w-5 h-5 ${theme === "dark" ? "text-amber-500" : "text-amber-600"}`,
fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path",
{ strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M13 16h-1v-4h-1m1-4h.01M21 12a9 9 0
11-18 0 9 9 0 0118 0z" })), /* @__PURE__ */ React.createElement("span", { className: `text-sm ${theme ===
"dark" ? "text-amber-200" : "text-amber-800"}` }, "Draft recovered from your previous session")),
      /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: () => {
            resetProductForm();
            setShowDraftNotice(false);
          },
          className: `text-xs underline hover:opacity-80 ${theme === "dark" ? "text-amber-300" : "text-amber-
700"}`
        },
        "Clear Draft"
      )
    ),
    /* @__PURE__ */ React.createElement("form", { onSubmit: handleFormSubmit, className: "flex flex-col flex-
1 min-h-0" }, !editingProduct && /* @__PURE__ */ React.createElement("div", { className: "px-8 pt-3 pb-4
flex-shrink-0" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center" },
STEPS.map((step, index) => /* @__PURE__ */ React.createElement("div", { key: step.id, className: "flex items-
center flex-1 last:flex-none" }, /* @__PURE__ */ React.createElement("div", { className: "flex flex-col
items-center", style: { minWidth: 50 } }, /* @__PURE__ */ React.createElement(
      "div",
      {
        onClick: () => {
          if (currentStep > step.id) setCurrentStep(step.id);
        },
        className: `w-9 h-9 rounded-full flex items-center justify-center text-sm font-bold border-2
transition-all duration-200 ${currentStep > step.id ? "bg-[#09A046] border-[#09A046] text-white cursor-
pointer hover:shadow-md" : currentStep === step.id ? "bg-[#09A046] border-[#09A046] text-white shadow-md" :
theme === "dark" ? "bg-transparent border-gray-600 text-gray-500" : "bg-transparent border-gray-300 text-
gray-400"}`
      },
      currentStep > step.id ? /* @__PURE__ */ React.createElement("svg", { className: "w-4 h-4", fill: "none",
stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap:
"round", strokeLinejoin: "round", strokeWidth: 3, d: "M5 13l4 4L19 7" })) : step.id
    ), /* @__PURE__ */ React.createElement("span", { className: `text-[11px] mt-1 font-semibold whitespace-
nowrap ${currentStep === step.id ? "text-[#09A046]" : currentStep > step.id ? "text-[#09A046]" : theme ===
"dark" ? "text-gray-500" : "text-gray-400"}` }, step.label)), index < STEPS.length - 1 && /* @__PURE__ */
React.createElement("div", { className: `flex-1 h-[2px] mx-1 mt-[-16px] rounded transition-all duration-200
${currentStep > step.id ? "bg-[#09A046]" : theme === "dark" ? "bg-gray-700" : "bg-gray-200"}` })))))), /*
@__PURE__ */ React.createElement("div", { className: `flex-1 px-8 pb-4 ${currentStep === 5 ? "flex flex-col
overflow-hidden" : "overflow-y-auto"}` }, editingProduct && /* @__PURE__ */ React.createElement("div",
{ className: "space-y-5" }, /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("label", { className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${theme ===
"dark" ? "text-gray-300" : "text-gray-700"}` }, "Product Name ", /* @__PURE__ */ React.createElement("span",
{ className: "text-red-500" }, "*")), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "text",
        name: "itemName",
        value: newProduct.itemName,
        onChange: handleInputChange,
        required: true,
        placeholder: "Product name",
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white placeholder-
gray-300" : "bg-white border-gray-300 placeholder-gray-400"}`
      }
    )), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-4" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: `block text-xs
font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` },
"Category ", /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__
*/ React.createElement(
      "select",
      {
        name: "category",
        value: newProduct.category || "",
        onChange: (e) => {
          if (e.target.value === "__add_new_category__") {
            setShowCategoryModal(true);
            return;
          }
          handleInputChange(e);
```

```
          setNewProduct((prev) => ({ ...prev, subCategory: "" }));
        },
        required: true,
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent appearance-none bg-no-repeat bg-[length:16px] bg-[center_right_12px]
${!newProduct.category ? "text-gray-400" : ""} ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-
white" : "bg-white border-gray-300"}`,
        style: { backgroundImage: `url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%239CA3AF' stroke-width='2'%3E%3Cpath d='M6
9l6 6 6-6'/%3E%3C/svg%3E")` }
      },
      /* @__PURE__ */ React.createElement("option", { value: "", disabled: true, className: theme === "dark" ?
"bg-[#2A2724]" : "" }, "Select Category"),
      sortedParentCategories.filter((c) => c !== "Others").map((cat) => /* @__PURE__ */
React.createElement("option", { key: cat, value: cat, className: theme === "dark" ? "bg-[#2A2724]" : "" },
cat)),
      /* @__PURE__ */ React.createElement("option", { value: "__add_new_category__", className: "font-
semibold text-[#09A046]" }, "+ Add Category")
    )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" :
"text-gray-700"}` }, "Sub Category ", /* @__PURE__ */ React.createElement("span", { className: "text-red-
500" }, "*")), /* @__PURE__ */ React.createElement(
      "select",
      {
        name: "subCategory",
        value: newProduct.subCategory || "",
        onChange: (e) => {
          if (e.target.value === "__add_new__") {
            setShowSubcategoryModal(true);
            return;
          }
          handleInputChange(e);
          setSelectedVariants([]);
          setCustomColorInput("");
          setVariantQuantities({});
          setVariantPrices({});
          setVariantCostPrices({});
          setDifferentPricesPerVariant({});
        },
        required: true,
        disabled: !newProduct.category,
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent appearance-none bg-no-repeat bg-[length:16px] bg-[center_right_12px]
${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white" : "bg-white border-gray-300"}
${!newProduct.category ? "opacity-50 cursor-not-allowed" : ""}`,
        style: { backgroundImage: `url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%239CA3AF' stroke-width='2'%3E%3Cpath d='M6
9l6 6 6-6'/%3E%3C/svg%3E")` }
      },
      /* @__PURE__ */ React.createElement("option", { value: "", disabled: true, className: theme === "dark" ?
"bg-[#2A2724]" : "" }, "Select SubCategory"),
      getSubcategories(newProduct.category).map((sub) => /* @__PURE__ */ React.createElement("option", { key:
sub, value: sub, className: theme === "dark" ? "bg-[#2A2724]" : "" }, sub)),
      /* @__PURE__ */ React.createElement("option", { value: "__add_new__", className: "font-semibold text-
[#09A046]" }, "+ Add Subcategory")
    ))), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-4" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: `block text-xs
font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Brand
Partner ", /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */
React.createElement(
      "select",
      {
        name: "brandName",
        value: newProduct.brandName || "",
        onChange: handleBrandSelectChange,
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent appearance-none bg-no-repeat bg-[length:16px] bg-[center_right_12px]
${!newProduct.brandName ? "text-gray-400" : ""} ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-
white" : "bg-white border-gray-300"}`,
        style: { backgroundImage: `url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%239CA3AF' stroke-width='2'%3E%3Cpath d='M6
9l6 6 6-6'/%3E%3C/svg%3E")` }
      },
      /* @__PURE__ */ React.createElement("option", { value: "", disabled: true, className: theme === "dark" ?
"bg-[#2A2724]" : "", style: { color: "#9CA3AF" } }, "Select Brand Partner"),
      /* @__PURE__ */ React.createElement("option", { value: "Default", className: theme === "dark" ? "bg-
[#2A2724]" : "" }, "Default"),
```

```
      partnerNames.map((name) => /* @__PURE__ */ React.createElement("option", { key: name, value: name,
className: theme === "dark" ? "bg-[#2A2724]" : "" }, name)),
      /* @__PURE__ */ React.createElement("option", { value: BRAND_ADD_SENTINEL, className: "font-semibold
text-[#09A046]" }, "+ Add Brand"),
      legacyBrandSelected && /* @__PURE__ */ React.createElement("option", { value: newProduct.brandName },
newProduct.brandName, " (Inactive)")
    )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" :
"text-gray-700"}` }, "Unit of Measure ", /* @__PURE__ */ React.createElement("span", { className: "text-red-
500" }, "*")), /* @__PURE__ */ React.createElement(
      "select",
      {
        name: "unitOfMeasure",
        value: newProduct.unitOfMeasure || "pcs",
        onChange: handleInputChange,
        required: true,
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent appearance-none bg-no-repeat bg-[length:16px] bg-[center_right_12px]
${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white" : "bg-white border-gray-300"}`,
        style: { backgroundImage: `url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%239CA3AF' stroke-width='2'%3E%3Cpath d='M6
9l6 6 6-6'/%3E%3C/svg%3E")` }
      },
      UNIT_OF_MEASURE_OPTIONS.map(
        (option) => /* @__PURE__ */ React.createElement(
          "option",
          {
            key: option.value,
            value: option.value,
            className: theme === "dark" ? "bg-[#2A2724]" : ""
          },
          option.label
        )
      )
    ))), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-4" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: `block text-xs
font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Reorder
Level"), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "number",
        min: "0",
        name: "reorderNumber",
        value: newProduct.reorderNumber || "",
        onChange: handleInputChange,
        placeholder: "eg. 10",
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white placeholder-
gray-500" : "bg-white border-gray-300 placeholder-gray-400"}`
      }
    )), /* @__PURE__ */ React.createElement("div", null, newProduct.expirationDate && /* @__PURE__ */
React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("label", { className: `block
text-xs font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` },
"Expiration Threshold (days)"), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "number",
        min: "0",
        name: "expirationThresholdDays",
        value: newProduct.expirationThresholdDays || "",
        onChange: handleInputChange,
        placeholder: "eg. 30",
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white placeholder-
gray-500" : "bg-white border-gray-300 placeholder-gray-400"}`
      }
    )))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `block text-xs font-bold uppercase tracking-wide mb-2 ${theme === "dark" ? "text-gray-300" :
"text-gray-700"}` }, "Product Image"), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3
items-start flex-wrap" }, productImages.map((img, index) => /* @__PURE__ */ React.createElement("div", { key:
index, className: `relative group rounded-xl overflow-hidden border-2 border-dashed flex-shrink-0 ${index ===
0 ? "ring-2 ring-[#09A046]" : ""} ${theme === "dark" ? "border-gray-600" : "border-gray-300"}`, style:
{ width: "120px", height: "120px" } }, /* @__PURE__ */ React.createElement("img", { src: img, alt: `Product
${index + 1}`, className: "w-full h-full object-cover" }), index === 0 && /* @__PURE__ */
React.createElement("span", { className: "absolute top-1 left-1 bg-[#BAE4CB] text-white text-[10px] px-1.5
py-0.5 rounded-full font-medium" }, "Main"), /* @__PURE__ */ React.createElement(
      "button",
```

```
      {
        type: "button",
        onClick: (e) => {
          e.stopPropagation();
          setProductImages((prev) => {
            const updated = prev.filter((_, i) => i !== index);
            setNewProduct((p) => ({ ...p, itemImage: updated[0] || "" }));
            return updated;
          });
        },
        className: "absolute top-1 right-1 w-5 h-5 bg-red-500 text-white rounded-full flex items-center
justify-center opacity-0 group-hover:opacity-100 transition-opacity text-xs font-bold hover:bg-red-600"
      },
      "\xD7"
    ))), /* @__PURE__ */ React.createElement(
      "div",
      {
        onClick: () => document.getElementById("editFileInput").click(),
        className: `rounded-xl border-2 border-dashed flex flex-col items-center justify-center cursor-
pointer transition-all flex-shrink-0 ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 hover:border-
[#09A046]" : "bg-gray-50 border-gray-300 hover:border-[#09A046]"}`,
        style: { width: "120px", height: "120px" }
      },
      /* @__PURE__ */ React.createElement("input", { id: "editFileInput", type: "file", accept: "image/*",
multiple: true, onChange: (e) => {
        const files = Array.from(e.target.files);
        files.forEach((file) => {
          const reader = new FileReader();
          reader.onloadend = () => {
            setProductImages((prev) => {
              const updated = [...prev, reader.result];
              setNewProduct((p) => ({ ...p, itemImage: updated[0] }));
              return updated;
            });
          };
          reader.readAsDataURL(file);
        });
        e.target.value = "";
      }, className: "hidden" }),
      /* @__PURE__ */ React.createElement("svg", { className: "w-6 h-6 text-gray-400 mb-1", fill: "none",
stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap:
"round", strokeLinejoin: "round", strokeWidth: 2, d: "M12 4v16m8-8H4" })),
      /* @__PURE__ */ React.createElement("p", { className: `text-[10px] text-center ${theme === "dark" ?
"text-gray-500" : "text-gray-400"}` }, "Upload an image")
    ))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `block text-xs font-bold uppercase tracking-wide mb-2 ${theme === "dark" ? "text-gray-300" :
"text-gray-700"}` }, "Display Settings"), (() => {
      const hasZeroStock = () => {
        if (editingProduct.sizes && typeof editingProduct.sizes === "object" &&
Object.keys(editingProduct.sizes).length > 0) {
          return Object.values(editingProduct.sizes).every((sd) => {
            if (typeof sd === "object" && sd !== null && sd.quantity !== void 0) return (sd.quantity || 0)
=== 0;
            return (typeof sd === "number" ? sd : 0) === 0;
          });
        }
        return (editingProduct.currentStock || 0) === 0;
      };
      const isStockZero = hasZeroStock();
      const isDisabled = isStockZero;
      return /* @__PURE__ */ React.createElement("label", { className: `flex items-center gap-3 ${isDisabled ?
"cursor-not-allowed opacity-60" : "cursor-pointer"}` }, /* @__PURE__ */ React.createElement("div",
{ className: "relative" }, /* @__PURE__ */ React.createElement("input", { type: "checkbox", name:
"displayInTerminal", checked: newProduct.displayInTerminal !== false, onChange: handleInputChange, disabled:
isDisabled, className: "sr-only" }), /* @__PURE__ */ React.createElement("div", { className: `w-14 h-7
rounded-full transition-colors duration-200 ${newProduct.displayInTerminal !== false ? "bg-[#09A046]" : "bg-
gray-300"} ${isDisabled ? "opacity-50" : ""}` }, /* @__PURE__ */ React.createElement("div", { className: `w-6
h-6 bg-white rounded-full shadow-md transform transition-transform duration-200
${newProduct.displayInTerminal !== false ? "translate-x-7" : "translate-x-1"} mt-0.5` }))), /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("span", { className: `text-sm font-
medium ${isDisabled ? "text-gray-400" : theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Display in
Terminal"), /* @__PURE__ */ React.createElement("p", { className: `text-xs ${isDisabled ? "text-gray-500" :
"text-gray-500"}` }, isDisabled ? "Add stock to enable this option" : "Show this product in POS/terminal")));
    })())), !editingProduct && /* @__PURE__ */ React.createElement(React.Fragment, null, currentStep === 1 &&
/* @__PURE__ */ React.createElement("div", { className: "space-y-5" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: `block text-xs
font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Product
```

```
Name ", /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */
React.createElement(
      "input",
      {
        type: "text",
        name: "itemName",
        value: newProduct.itemName,
        onChange: handleInputChange,
        required: true,
        placeholder: "Product name",
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white placeholder-
gray-300" : "bg-white border-gray-300 placeholder-gray-400"}`
      }
    )), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-4" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: `block text-xs
font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` },
"Category ", /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__
*/ React.createElement(
      "select",
      {
        name: "category",
        value: newProduct.category || "",
        onChange: (e) => {
          if (e.target.value === "__add_new_category__") {
            setShowCategoryModal(true);
            return;
          }
          handleInputChange(e);
          setNewProduct((prev) => ({ ...prev, subCategory: "" }));
        },
        required: true,
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent appearance-none bg-no-repeat bg-[length:16px] bg-[center_right_12px]
${!newProduct.category ? "text-gray-400" : ""} ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-
white" : "bg-white border-gray-300"}`,
        style: { backgroundImage: `url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%239CA3AF' stroke-width='2'%3E%3Cpath d='M6
9l6 6 6-6'/%3E%3C/svg%3E")` }
      },
      /* @__PURE__ */ React.createElement("option", { value: "", disabled: true, className: theme === "dark" ?
"bg-[#2A2724]" : "" }, "Select Category"),
      sortedParentCategories.filter((c) => c !== "Others").map((cat) => /* @__PURE__ */
React.createElement("option", { key: cat, value: cat, className: theme === "dark" ? "bg-[#2A2724]" : "" },
cat)),
      /* @__PURE__ */ React.createElement("option", { value: "__add_new_category__", className: "font-
semibold text-[#09A046]" }, "+ Add Category")
    )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" :
"text-gray-700"}` }, "Sub Category ", /* @__PURE__ */ React.createElement("span", { className: "text-red-
500" }, "*")), /* @__PURE__ */ React.createElement(
      "select",
      {
        name: "subCategory",
        value: newProduct.subCategory || "",
        onChange: (e) => {
          if (e.target.value === "__add_new__") {
            setShowSubcategoryModal(true);
            return;
          }
          handleInputChange(e);
          setSelectedVariants([]);
          setCustomColorInput("");
          setVariantQuantities({});
          setVariantPrices({});
          setVariantCostPrices({});
          setDifferentPricesPerVariant({});
        },
        required: true,
        disabled: !newProduct.category,
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent appearance-none bg-no-repeat bg-[length:16px] bg-[center_right_12px]
${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white" : "bg-white border-gray-300"}
${!newProduct.category ? "opacity-50 cursor-not-allowed" : ""}`,
        style: { backgroundImage: `url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%239CA3AF' stroke-width='2'%3E%3Cpath d='M6
9l6 6 6-6'/%3E%3C/svg%3E")` }
```

```
      },
      /* @__PURE__ */ React.createElement("option", { value: "", disabled: true, className: theme === "dark" ?
"bg-[#2A2724]" : "" }, "Select SubCategory"),
      getSubcategories(newProduct.category).map((sub) => /* @__PURE__ */ React.createElement("option", { key:
sub, value: sub, className: theme === "dark" ? "bg-[#2A2724]" : "" }, sub)),
      /* @__PURE__ */ React.createElement("option", { value: "__add_new__", className: "font-semibold text-
[#09A046]" }, "+ Add Subcategory")
    ))), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-4" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: `block text-xs
font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Brand
Partner ", /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */
React.createElement(
      "select",
      {
        name: "brandName",
        value: newProduct.brandName || "",
        onChange: handleBrandSelectChange,
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent appearance-none bg-no-repeat bg-[length:16px] bg-[center_right_12px]
${!newProduct.brandName ? "text-gray-400" : ""} ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-
white" : "bg-white border-gray-300"}`,
        style: { backgroundImage: `url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%239CA3AF' stroke-width='2'%3E%3Cpath d='M6
9l6 6 6-6'/%3E%3C/svg%3E")` }
      },
      /* @__PURE__ */ React.createElement("option", { value: "", disabled: true, className: theme === "dark" ?
"bg-[#2A2724]" : "", style: { color: "#9CA3AF" } }, "Select Brand Partner"),
      /* @__PURE__ */ React.createElement("option", { value: "Default", className: theme === "dark" ? "bg-
[#2A2724]" : "" }, "Default"),
      partnerNames.map((name) => /* @__PURE__ */ React.createElement("option", { key: name, value: name,
className: theme === "dark" ? "bg-[#2A2724]" : "" }, name)),
      /* @__PURE__ */ React.createElement("option", { value: BRAND_ADD_SENTINEL, className: "font-semibold
text-[#09A046]" }, "+ Add Brand"),
      legacyBrandSelected && /* @__PURE__ */ React.createElement("option", { value: newProduct.brandName },
newProduct.brandName, " (Inactive)")
    )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" :
"text-gray-700"}` }, "Unit of Measure ", /* @__PURE__ */ React.createElement("span", { className: "text-red-
500" }, "*")), /* @__PURE__ */ React.createElement(
      "select",
      {
        name: "unitOfMeasure",
        value: newProduct.unitOfMeasure || "",
        onChange: handleInputChange,
        required: true,
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent appearance-none bg-no-repeat bg-[length:16px] bg-[center_right_12px]
${!newProduct.unitOfMeasure ? "text-gray-400" : ""} ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-
white" : "bg-white border-gray-300"}`,
        style: { backgroundImage: `url("data:image/svg+xml,%3Csvg xmlns='http://www.w3.org/2000/svg'
width='16' height='16' viewBox='0 0 24 24' fill='none' stroke='%239CA3AF' stroke-width='2'%3E%3Cpath d='M6
9l6 6 6-6'/%3E%3C/svg%3E")` }
      },
      /* @__PURE__ */ React.createElement("option", { value: "", disabled: true, className: theme === "dark" ?
"bg-[#2A2724]" : "", style: { color: "#9CA3AF" } }, "Select Unit"),
      UNIT_OF_MEASURE_OPTIONS.map(
        (option) => /* @__PURE__ */ React.createElement(
          "option",
          {
            key: option.value,
            value: option.value,
            className: theme === "dark" ? "bg-[#2A2724]" : ""
          },
          option.label
        )
      )
    ))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `block text-xs font-bold uppercase tracking-wide mb-2 ${theme === "dark" ? "text-gray-300" :
"text-gray-700"}` }, "Product Image"), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3
items-start flex-wrap" }, productImages.map((img, index) => /* @__PURE__ */ React.createElement("div", { key:
index, className: `relative group rounded-xl overflow-hidden border-2 border-dashed flex-shrink-0 ${index ===
0 ? "ring-2 ring-[#09A046]" : ""} ${theme === "dark" ? "border-gray-600" : "border-gray-300"}`, style:
{ width: "120px", height: "120px" } }, /* @__PURE__ */ React.createElement("img", { src: img, alt: `Product
${index + 1}`, className: "w-full h-full object-cover" }), index === 0 && /* @__PURE__ */
React.createElement("span", { className: "absolute top-1 left-1 bg-[#BAE4CB] text-white text-[10px] px-1.5
py-0.5 rounded-full font-medium" }, "Main"), /* @__PURE__ */ React.createElement(
      "button",
```

```
      {
        type: "button",
        onClick: (e) => {
          e.stopPropagation();
          setProductImages((prev) => {
            const updated = prev.filter((_, i) => i !== index);
            setNewProduct((p) => ({ ...p, itemImage: updated[0] || "" }));
            return updated;
          });
        },
        className: "absolute top-1 right-1 w-5 h-5 bg-red-500 text-white rounded-full flex items-center
justify-center opacity-0 group-hover:opacity-100 transition-opacity text-xs font-bold hover:bg-red-600"
      },
      "\xD7"
    ))), /* @__PURE__ */ React.createElement(
      "div",
      {
        onClick: () => document.getElementById("fileInput").click(),
        className: `rounded-xl border-2 border-dashed flex flex-col items-center justify-center cursor-
pointer transition-all flex-shrink-0 ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 hover:border-
[#09A046]" : "bg-gray-50 border-gray-300 hover:border-[#09A046]"}`,
        style: { width: "120px", height: "120px" }
      },
      /* @__PURE__ */ React.createElement("input", { id: "fileInput", type: "file", accept: "image/*",
multiple: true, onChange: (e) => {
        const files = Array.from(e.target.files);
        files.forEach((file) => {
          const reader = new FileReader();
          reader.onloadend = () => {
            setProductImages((prev) => {
              const updated = [...prev, reader.result];
              setNewProduct((p) => ({ ...p, itemImage: updated[0] }));
              return updated;
            });
          };
          reader.readAsDataURL(file);
        });
        e.target.value = "";
      }, className: "hidden" }),
      /* @__PURE__ */ React.createElement("svg", { className: "w-8 h-8 text-gray-400 mb-1", fill: "none",
stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap:
"round", strokeLinejoin: "round", strokeWidth: 1.5, d: "M12 4v16m8-8H4" })),
      /* @__PURE__ */ React.createElement("p", { className: `text-[10px] text-center px-1 ${theme === "dark" ?
"text-gray-500" : "text-gray-400"}` }, "Upload an image")
    )), productImages.length > 3 && /* @__PURE__ */ React.createElement("div", { className: "flex items-
center gap-2 mt-2 px-3 py-2 rounded-lg bg-amber-50 border border-amber-200 text-amber-700" }, /* @__PURE__ */
React.createElement("svg", { className: "w-4 h-4 flex-shrink-0", fill: "currentColor", viewBox: "0 0 20 20" },
/* @__PURE__ */ React.createElement("path", { fillRule: "evenodd", d: "M8.257 3.099c.765-1.36 2.722-1.36
3.486 0l5.58 9.92c.75 1.334-.213 2.98-1.742 2.98H4.42c-1.53 0-2.493-1.646-1.743-2.98l5.58-9.92zM11 13a1 1 0
11-2 0 1 1 0 012 0zm-1-8a1 1 0 00-1 1v3a1 1 0 002 0V6a1 1 0 00-1-1z", clipRule: "evenodd" })), /* @__PURE__
*/ React.createElement("span", { className: "text-xs font-medium" }, "Adding more than 3 photos may slow down
the system."))))), currentStep === 2 && /* @__PURE__ */ React.createElement("div", { className: "space-y-6" },
/* @__PURE__ */ React.createElement("div", { className: `p-4 rounded-xl border ${theme === "dark" ? "bg-
[#1E1B18] border-gray-700" : "bg-gray-50 border-gray-200"}` }, /* @__PURE__ */ React.createElement("div",
{ className: "flex items-center justify-between mb-3" }, /* @__PURE__ */ React.createElement("div",
{ className: "flex items-center gap-2" }, /* @__PURE__ */ React.createElement("label", { className: `text-xs
font-bold uppercase tracking-wide ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Option Group
1"), /* @__PURE__ */ React.createElement("span", { title: "Press Enter to add", className: `inline-flex
items-center justify-center w-4 h-4 rounded-full text-[10px] font-bold border ${theme === "dark" ? "text-
gray-300 border-gray-600" : "text-gray-600 border-gray-300"}` }, "i")), /* @__PURE__ */
React.createElement("span", { className: `text-[10px] italic ${theme === "dark" ? "text-gray-500" : "text-
gray-400"}` }, "e.g. Color \xB7 Flavor \xB7 Shade \xB7 Style")), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "text",
        value: optionGroup1Name,
        onChange: (e) => setOptionGroup1Name(e.target.value),
        placeholder: "e.g. Color",
        className: `w-48 px-3 py-2 text-sm border-b-2 border-t-0 border-l-0 border-r-0 bg-transparent
focus:outline-none focus:border-[#09A046] mb-3 font-medium ${theme === "dark" ? "border-gray-600 text-white
placeholder-gray-500" : "border-gray-300 text-gray-800 placeholder-gray-400"}`
      }
    ), /* @__PURE__ */ React.createElement("div", { className: "relative flex items-start gap-3 mb-3" }, /*
@__PURE__ */ React.createElement("div", { className: "flex-1 relative" }, /* @__PURE__ */
React.createElement(
      "div",
      {
```

```
        onClick: () => setShowVariantDropdown(!showVariantDropdown),
        className: `w-full px-3 py-2.5 text-sm border rounded-lg cursor-pointer flex items-center justify-
between ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white hover:border-[#09A046]" : "bg-white
border-gray-300 hover:border-[#09A046]"}`
      },
      /* @__PURE__ */ React.createElement("span", { className: selectedVariants.length === 0 ? "text-gray-
400" : "" }, selectedVariants.length === 0 ? `Select ${optionGroup1Name.toLowerCase() || "options"}...` :
`${selectedVariants.length} selected`),
      /* @__PURE__ */ React.createElement("svg", { className: `w-4 h-4 transition-transform
${showVariantDropdown ? "rotate-180" : ""}`, fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /*
@__PURE__ */ React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d:
"M19 9l-7 7-7-7" }))
    ), showVariantDropdown && /* @__PURE__ */ React.createElement("div", { className: `absolute z-50 w-full
mt-1 max-h-48 overflow-y-auto border rounded-lg shadow-lg ${theme === "dark" ? "bg-[#2A2724] border-gray-
600" : "bg-white border-gray-200"}` }, COMMON_COLORS.filter((c) => c !== "Custom").map((color) => /*
@__PURE__ */ React.createElement(
      "div",
      {
        key: color,
        onClick: () => handleVariantToggle(color),
        className: `px-3 py-2 text-sm cursor-pointer flex items-center justify-between
${selectedVariants.includes(color) ? theme === "dark" ? "bg-[#09A046]/15 text-[#09A046]" : "bg-[#09A046]/15
text-[#09A046]" : theme === "dark" ? "hover:bg-[#09A046]/10" : "hover:bg-[#09A046]/10"}`
      },
      /* @__PURE__ */ React.createElement("span", null, color),
      selectedVariants.includes(color) && /* @__PURE__ */ React.createElement("svg", { className: "w-4 h-4",
fill: "currentColor", viewBox: "0 0 20 20" }, /* @__PURE__ */ React.createElement("path", { fillRule:
"evenodd", d: "M16.707 5.293a1 1 0 010 1.414l-8 8a1 1 0 01-1.414 0l-4-4a1 1 0 011.414-1.414L8 12.586l7.293-
7.293a1 1 0 011.414 0z", clipRule: "evenodd" }))
    ))), showVariantDropdown && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-40",
onClick: () => setShowVariantDropdown(false) })), /* @__PURE__ */ React.createElement("div", { className:
"flex flex-wrap gap-1.5 flex-1 min-h-[38px] items-center" }, selectedVariants.map((variant) => /* @__PURE__
*/ React.createElement("span", { key: variant, className: `inline-flex items-center gap-1 px-2.5 py-1 text-xs
rounded-full ${theme === "dark" ? "bg-[#09A046]/15 text-[#09A046] border border-[#09A046]/20" : "bg-
[#09A046]/12 text-[#09A046] border border-[#09A046]/20"}` }, variant, /* @__PURE__ */
React.createElement("button", { type: "button", onClick: (e) => {
      e.stopPropagation();
      removeVariant(variant);
    }, className: "hover:text-red-500 transition-colors" }, "\xD7"))), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "text",
        value: customColorInput,
        onChange: (e) => setCustomColorInput(e.target.value),
        onKeyDown: (e) => {
          if (e.key === "Enter") {
            e.preventDefault();
            addCustomColor();
          }
        },
        placeholder: "Add +",
        className: `w-16 px-2 py-1 text-xs border-2 border-dashed rounded-full focus:outline-none
focus:border-[#09A046] text-center ${theme === "dark" ? "bg-transparent border-gray-600 text-white
placeholder-gray-500" : "bg-transparent border-gray-300 text-gray-700 placeholder-gray-400"}`
      }
    )))), /* @__PURE__ */ React.createElement("div", { className: `p-4 rounded-xl border ${theme === "dark" ?
"bg-[#1E1B18] border-gray-700" : "bg-gray-50 border-gray-200"}` }, /* @__PURE__ */ React.createElement("div",
{ className: "flex items-center justify-between mb-3" }, /* @__PURE__ */ React.createElement("div",
{ className: "flex items-center gap-2" }, /* @__PURE__ */ React.createElement("label", { className: `text-xs
font-bold uppercase tracking-wide ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Option Group
2"), /* @__PURE__ */ React.createElement("span", { title: "Press Enter to add", className: `inline-flex
items-center justify-center w-4 h-4 rounded-full text-[10px] font-bold border ${theme === "dark" ? "text-
gray-300 border-gray-600" : "text-gray-600 border-gray-300"}` }, "i")), /* @__PURE__ */
React.createElement("span", { className: `text-[10px] italic ${theme === "dark" ? "text-gray-500" : "text-
gray-400"}` }, "e.g. Size \xB7 Weight \xB7 Volume \xB7 Pack")), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "text",
        value: optionGroup2Name,
        onChange: (e) => setOptionGroup2Name(e.target.value),
        placeholder: "e.g. Size",
        className: `w-48 px-3 py-2 text-sm border-b-2 border-t-0 border-l-0 border-r-0 bg-transparent
focus:outline-none focus:border-[#09A046] mb-3 font-medium ${theme === "dark" ? "border-gray-600 text-white
placeholder-gray-500" : "border-gray-300 text-gray-800 placeholder-gray-400"}`
      }
    ), /* @__PURE__ */ React.createElement("div", { className: "relative flex items-start gap-3 mb-3" }, /*
@__PURE__ */ React.createElement("div", { className: "flex-1 relative" }, /* @__PURE__ */
```

```
React.createElement(
      "div",
      {
        onClick: () => setShowSizeDropdown(!showSizeDropdown),
        className: `w-full px-3 py-2.5 text-sm border rounded-lg cursor-pointer flex items-center justify-
between ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white hover:border-[#09A046]" : "bg-white
border-gray-300 hover:border-[#09A046]"}`
      },
      /* @__PURE__ */ React.createElement("span", { className: (newProduct.selectedSizes?.length || 0) === 0 ?
"text-gray-400" : "" }, (newProduct.selectedSizes?.length || 0) === 0 ? `Select
${optionGroup2Name.toLowerCase() || "options"}...` : `${newProduct.selectedSizes.length} selected`),
      /* @__PURE__ */ React.createElement("svg", { className: `w-4 h-4 transition-transform
${showSizeDropdown ? "rotate-180" : ""}`, fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /*
@__PURE__ */ React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d:
"M19 9l-7 7-7-7" }))
    ), showSizeDropdown && /* @__PURE__ */ React.createElement("div", { className: `absolute z-50 w-full mt-1
max-h-40 overflow-y-auto border rounded-lg shadow-lg ${theme === "dark" ? "bg-[#2A2724] border-gray-600" :
"bg-white border-gray-200"}` }, (() => {
      const category = newProduct.category;
      const subCategory = newProduct.subCategory || "";
      let sizes = [];
      const parentHasSizes = categoryStructure[category] !== void 0;
      if (!parentHasSizes) {
        sizes = ["Free Size"];
      } else if (category === "Foods") {
        switch (subCategory) {
          case "Beverages":
            sizes = ["Small", "Medium", "Large", "Extra Large", "Free Size"];
            break;
          case "Snacks":
            sizes = ["Small Pack", "Medium Pack", "Large Pack", "Family Pack", "Free Size"];
            break;
          case "Meals":
            sizes = ["Regular", "Large", "Family Size", "Free Size"];
            break;
          case "Desserts":
            sizes = ["Small", "Medium", "Large", "Free Size"];
            break;
          case "Ingredients":
            sizes = ["100g", "250g", "500g", "1kg", "Free Size"];
            break;
          default:
            sizes = ["Small", "Medium", "Large", "Free Size"];
        }
      } else if (["Tops", "Bottoms", "Dresses", "Outerwear"].includes(subCategory)) {
        sizes = ["XS", "S", "M", "L", "XL", "XXL", "Free Size"];
      } else if (category === "Shoes") {
        sizes = ["5", "6", "7", "8", "9", "10", "11", "12"];
      } else if (category === "Accessories" || category === "Makeup" || category === "Essentials") {
        sizes = ["Free Size"];
      } else {
        sizes = ["Free Size"];
      }
      return [...sizes, ...customSizes].map((size) => /* @__PURE__ */ React.createElement(
        "div",
        {
          key: size,
          onClick: () => handleSizeToggle(size),
          className: `px-3 py-2 text-sm cursor-pointer flex items-center justify-between
${newProduct.selectedSizes?.includes(size) ? theme === "dark" ? "bg-[#09A046]/15 text-[#09A046]" : "bg-
[#09A046]/15 text-[#09A046]" : theme === "dark" ? "hover:bg-[#09A046]/10" : "hover:bg-[#09A046]/10"}`
        },
        /* @__PURE__ */ React.createElement("span", null, size),
        /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-1" },
customSizes.includes(size) && /* @__PURE__ */ React.createElement(
          "button",
          {
            type: "button",
            title: "Remove custom size",
            onClick: (e) => {
              e.stopPropagation();
              setCustomSizes((prev) => prev.filter((s) => s !== size));
              if (newProduct.selectedSizes?.includes(size)) {
                handleSizeToggle(size);
              }
            },
            className: "text-red-400 hover:text-red-600 text-xs px-1"
```

```
          },
          "\xD7"
        ), newProduct.selectedSizes?.includes(size) && /* @__PURE__ */ React.createElement("svg", { className:
"w-4 h-4", fill: "currentColor", viewBox: "0 0 20 20" }, /* @__PURE__ */ React.createElement("path",
{ fillRule: "evenodd", d: "M16.707 5.293a1 1 0 010 1.414l-8 8a1 1 0 01-1.414 0l-4-4a1 1 0 011.414-1.414L8
12.586l7.293-7.293a1 1 0 011.414 0z", clipRule: "evenodd" })))
      ));
    })()), showSizeDropdown && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-40",
onClick: () => setShowSizeDropdown(false) })), /* @__PURE__ */ React.createElement("div", { className: "flex
flex-wrap gap-1.5 flex-1 min-h-[38px] items-center" }, (newProduct.selectedSizes || []).map((size) => /*
@__PURE__ */ React.createElement("span", { key: size, className: `inline-flex items-center gap-1 px-2.5 py-1
text-xs rounded-full ${theme === "dark" ? "bg-[#09A046]/15 text-[#09A046] border border-[#09A046]/20" : "bg-
[#09A046]/12 text-[#09A046] border border-[#09A046]/20"}` }, size, /* @__PURE__ */
React.createElement("button", { type: "button", onClick: (e) => {
      e.stopPropagation();
      handleSizeToggle(size);
    }, className: "hover:text-red-500 transition-colors" }, "\xD7"))), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "text",
        value: customSizeValue,
        onChange: (e) => setCustomSizeValue(e.target.value),
        onKeyDown: (e) => {
          if (e.key === "Enter") {
            e.preventDefault();
            const trimmed = customSizeValue.trim();
            if (trimmed && !customSizes.includes(trimmed)) {
              setCustomSizes((prev) => [...prev, trimmed]);
              handleSizeToggle(trimmed);
              setCustomSizeValue("");
            }
          }
        },
        placeholder: "Add +",
        className: `w-16 px-2 py-1 text-xs border-2 border-dashed rounded-full focus:outline-none
focus:border-[#09A046] text-center ${theme === "dark" ? "bg-transparent border-gray-600 text-white
placeholder-gray-500" : "bg-transparent border-gray-300 text-gray-700 placeholder-gray-400"}`
      }
    )))), currentStep === 3 && /* @__PURE__ */ React.createElement("div", { className: "space-y-5" }, (() =>
{
      const combos = [];
      const variants = selectedVariants.length > 0 ? selectedVariants : [null];
      const sizes = newProduct.selectedSizes?.length > 0 ? newProduct.selectedSizes : [VARIANT_ONLY_KEY];
      variants.forEach((v) => {
        sizes.forEach((s) => {
          combos.push({ variant: v, size: s, key: `${v || ""}-${s || ""}` });
        });
      });
      const hasAnyCombos = hasVariants && combos.length > 0 && (combos[0].variant || combos[0].size);
      const handleFillAll = () => {
        if (!hasAnyCombos) return;
        const cost = fillAllCost;
        const price = fillAllPrice;
        const qty = fillAllQty;
        combos.forEach(({ variant: v, size: s }) => {
          if (v && s) {
            if (cost) handleVariantCostPriceChange(s, v, cost);
            if (price) handleVariantPriceChange(s, v, price);
            if (qty) {
              setVariantQuantities((prev) => {
                const updated = { ...prev, [s]: { ...prev[s] || {}, [v]: parseInt(qty) || 0 } };
                setNewProduct((p) => ({ ...p, variantQuantities: updated }));
                return updated;
              });
            }
          } else if (s && !v) {
            if (cost) setNewProduct((p) => ({ ...p, sizeCostPrices: { ...p.sizeCostPrices || {}, [s]:
cost } }));
            if (price) handleSizePriceChange(s, price);
            if (qty) handleSizeQuantityChange(s, qty);
          } else if (v && !s) {
            if (cost) setNewProduct((p) => ({ ...p, costPrice: cost }));
            if (price) setNewProduct((p) => ({ ...p, itemPrice: price }));
            if (qty) setNewProduct((p) => ({ ...p, currentStock: qty }));
          }
        });
      };
```

```
      return hasAnyCombos ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement("div", { className: `flex items-end gap-2 p-3 rounded-xl border ${theme === "dark" ? "bg-
[#1E1B18] border-gray-700" : "bg-gray-50 border-gray-200"}` }, /* @__PURE__ */ React.createElement("div",
{ className: "flex-1" }, /* @__PURE__ */ React.createElement("label", { className: `block text-[10px] font-
bold uppercase tracking-wide mb-1 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Cost Price
\u20B1"), /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "number",
          step: "0.01",
          min: "0",
          value: fillAllCost,
          onChange: (e) => setFillAllCost(e.target.value),
          placeholder: "\u20B1",
          className: `w-full px-2 py-2 text-sm border rounded-lg focus:outline-none focus:ring-1 focus:ring-
[#09A046] ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white" : "bg-white border-gray-300"}`
        }
      )), /* @__PURE__ */ React.createElement("div", { className: "flex-1" }, /* @__PURE__ */
React.createElement("label", { className: `block text-[10px] font-bold uppercase tracking-wide mb-1 ${theme
=== "dark" ? "text-gray-400" : "text-gray-500"}` }, "Selling Price \u20B1"), /* @__PURE__ */
React.createElement(
        "input",
        {
          type: "number",
          step: "0.01",
          min: "0",
          value: fillAllPrice,
          onChange: (e) => setFillAllPrice(e.target.value),
          placeholder: "\u20B1",
          className: `w-full px-2 py-2 text-sm border rounded-lg focus:outline-none focus:ring-1 focus:ring-
[#09A046] ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white" : "bg-white border-gray-300"}`
        }
      )), /* @__PURE__ */ React.createElement("div", { className: "flex-1" }, /* @__PURE__ */
React.createElement("label", { className: `block text-[10px] font-bold uppercase tracking-wide mb-1 ${theme
=== "dark" ? "text-gray-400" : "text-gray-500"}` }, "Qty"), /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "number",
          min: "0",
          value: fillAllQty,
          onChange: (e) => setFillAllQty(e.target.value),
          placeholder: "0",
          className: `w-full px-2 py-2 text-sm border rounded-lg focus:outline-none focus:ring-1 focus:ring-
[#09A046] ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white" : "bg-white border-gray-300"}`
        }
      )), /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: handleFillAll,
          disabled: !fillAllCost && !fillAllPrice && !fillAllQty,
          className: `px-4 py-2 text-xs font-semibold rounded-lg whitespace-nowrap transition-colors
${fillAllCost || fillAllPrice || fillAllQty ? "bg-[#09A046] text-white hover:bg-[#078a3b]" : theme ===
"dark" ? "bg-gray-700 text-gray-500 cursor-not-allowed" : "bg-gray-200 text-gray-400 cursor-not-allowed"}`
        },
        "Fill all"
      )), /* @__PURE__ */ React.createElement("div", { className: `rounded-xl border overflow-hidden ${theme
=== "dark" ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */ React.createElement("div",
{ className: `grid grid-cols-[1.5fr_1fr_1fr_0.8fr] gap-0 text-[10px] font-bold uppercase tracking-wider
${theme === "dark" ? "bg-[#2A2724] text-gray-400" : "bg-gray-100 text-gray-500"}` }, /* @__PURE__ */
React.createElement("div", { className: "px-3 py-2.5" }, "Variant Combo"), /* @__PURE__ */
React.createElement("div", { className: "px-3 py-2.5" }, "Cost Price \u20B1"), /* @__PURE__ */
React.createElement("div", { className: "px-3 py-2.5" }, "Selling Price \u20B1"), /* @__PURE__ */
React.createElement("div", { className: "px-3 py-2.5" }, "Qty")), combos.map(({ variant: v, size: s, key })
=> /* @__PURE__ */ React.createElement("div", { key, className: `grid grid-cols-[1.5fr_1fr_1fr_0.8fr] gap-0
items-center border-t ${theme === "dark" ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */
React.createElement("div", { className: "px-3 py-2 flex flex-wrap gap-1" }, s && s !== VARIANT_ONLY_KEY && /*
@__PURE__ */ React.createElement("span", { className: `inline-block px-2 py-0.5 text-[11px] rounded-full
font-medium ${theme === "dark" ? "bg-blue-500/20 text-blue-400" : "bg-blue-100 text-blue-700"}` }, s), v &&
/* @__PURE__ */ React.createElement("span", { className: `inline-block px-2 py-0.5 text-[11px] rounded-full
font-medium ${theme === "dark" ? "bg-pink-500/20 text-pink-400" : "bg-pink-100 text-pink-700"}` }, v)), /*
@__PURE__ */ React.createElement("div", { className: "px-2 py-1.5" }, /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "number",
          step: "0.01",
          min: "0",
```

```
          value: v && s ? variantCostPrices[s]?.[v] ?? "" : s ? newProduct.sizeCostPrices?.[s] ?? "" :
newProduct.costPrice ?? "",
          onChange: (e) => {
            if (v && s) handleVariantCostPriceChange(s, v, e.target.value);
            else if (s) setNewProduct((p) => ({ ...p, sizeCostPrices: { ...p.sizeCostPrices || {}, [s]:
e.target.value } }));
            else setNewProduct((p) => ({ ...p, costPrice: e.target.value }));
          },
          placeholder: "\u20B1",
          className: `w-full px-2 py-1.5 text-sm border rounded-lg focus:outline-none focus:ring-1
focus:ring-[#09A046] ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white" : "bg-white border-gray-
300"}`
        }
      )), /* @__PURE__ */ React.createElement("div", { className: "px-2 py-1.5" }, /* @__PURE__ */
React.createElement(
        "input",
        {
          type: "number",
          step: "0.01",
          min: "0",
          value: v && s ? variantPrices[s]?.[v] ?? "" : s ? newProduct.sizePrices?.[s] ?? "" :
newProduct.itemPrice ?? "",
          onChange: (e) => {
            if (v && s) handleVariantPriceChange(s, v, e.target.value);
            else if (s) handleSizePriceChange(s, e.target.value);
            else setNewProduct((p) => ({ ...p, itemPrice: e.target.value }));
          },
          placeholder: "\u20B1",
          className: `w-full px-2 py-1.5 text-sm border rounded-lg focus:outline-none focus:ring-1
focus:ring-[#09A046] ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white" : "bg-white border-gray-
300"}`
        }
      )), /* @__PURE__ */ React.createElement("div", { className: "px-2 py-1.5" }, /* @__PURE__ */
React.createElement(
        "input",
        {
          type: "number",
          min: "0",
          value: v && s ? newProduct.variantQuantities?.[s]?.[v] ?? "" : s ? newProduct.sizeQuantities?.[s] ??
"" : newProduct.currentStock ?? "",
          onChange: (e) => {
            const val = e.target.value;
            if (v && s) {
              setVariantQuantities((prev) => {
                const updated = { ...prev, [s]: { ...prev[s] || {}, [v]: parseInt(val) || 0 } };
                setNewProduct((p) => ({ ...p, variantQuantities: updated }));
                return updated;
              });
            } else if (s) {
              handleSizeQuantityChange(s, val);
            } else {
              setNewProduct((p) => ({ ...p, currentStock: val }));
            }
          },
          placeholder: "0",
          className: `w-full px-2 py-1.5 text-sm border rounded-lg focus:outline-none focus:ring-1
focus:ring-[#09A046] ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white" : "bg-white border-gray-
300"}`
        }
      )))))) : (
        /* No variants — simple pricing + qty */
        /* @__PURE__ */ React.createElement("div", { className: "space-y-4" }, /* @__PURE__ */
React.createElement("div", { className: "grid grid-cols-2 gap-3" }, /* @__PURE__ */ React.createElement("div",
null, /* @__PURE__ */ React.createElement("label", { className: `block text-xs font-bold uppercase tracking-
wide mb-1.5 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Cost Price \u20B1"), /* @__PURE__ */
React.createElement(
          "input",
          {
            type: "number",
            step: "0.01",
            name: "costPrice",
            value: newProduct.costPrice,
            onChange: handleInputChange,
            placeholder: "Enter cost price",
            className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2
focus:ring-[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white" :
"bg-white border-gray-300"}`
```

```
          }
        )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" :
"text-gray-700"}` }, "Selling Price \u20B1"), /* @__PURE__ */ React.createElement(
          "input",
          {
            type: "number",
            step: "0.01",
            name: "itemPrice",
            value: newProduct.itemPrice,
            onChange: handleInputChange,
            placeholder: "Enter selling price",
            className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2
focus:ring-[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white" :
"bg-white border-gray-300"}`
          }
        ))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" :
"text-gray-700"}` }, "Quantity"), /* @__PURE__ */ React.createElement(
          "input",
          {
            type: "number",
            min: "0",
            name: "currentStock",
            value: newProduct.currentStock || "",
            onChange: handleInputChange,
            placeholder: "Enter quantity",
            className: `w-full max-w-xs px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2
focus:ring-[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white" :
"bg-white border-gray-300"}`
          }
        )))
      );
    })(), /* @__PURE__ */ React.createElement("div", { className: `mt-4 pt-4 border-t ${theme === "dark" ?
"border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */ React.createElement("label", { className: "flex
items-center gap-3 cursor-pointer" }, /* @__PURE__ */ React.createElement("div", { className: "relative" },
/* @__PURE__ */ React.createElement("input", { type: "checkbox", name: "displayInTerminal", checked:
newProduct.displayInTerminal !== false, onChange: handleInputChange, className: "sr-only" }), /* @__PURE__ */
React.createElement("div", { className: `w-14 h-7 rounded-full transition-colors duration-200
${newProduct.displayInTerminal !== false ? "bg-[#09A046]" : "bg-gray-300"}` }, /* @__PURE__ */
React.createElement("div", { className: `w-6 h-6 bg-white rounded-full shadow-md transform transition-
transform duration-200 ${newProduct.displayInTerminal !== false ? "translate-x-7" : "translate-x-1"} mt-
0.5` }))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("span",
{ className: `text-sm font-medium ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Display in
Terminal"), /* @__PURE__ */ React.createElement("p", { className: `text-xs ${theme === "dark" ? "text-gray-
500" : "text-gray-500"}` }, "Show this product in POS/terminal"))))), currentStep === 4 && /* @__PURE__ */
React.createElement("div", { className: "space-y-5 pt-10" }, (() => {
      const openingStockTotal = getOpeningStockTotal();
      const hasOpeningStock = openingStockTotal > 0;
      return /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: `block text-xs
font-bold uppercase tracking-wide mb-0.5 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Initial
stock batch"), /* @__PURE__ */ React.createElement("p", { className: `text-[10px] mb-3 ${theme === "dark" ?
"text-gray-500" : "text-gray-400"}` }, hasOpeningStock ? "Batch 1 for this opening stock. Add more lots via
Stock In." : "No batch until you receive stock \u2014 use Stock In.")), hasOpeningStock ? /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: `block text-xs
font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` },
"Batch/Lot No."), /* @__PURE__ */ React.createElement("p", { className: `text-lg font-bold ${theme ===
"dark" ? "text-white" : "text-gray-900"}` }, formatStockInStyleBatchCode(/* @__PURE__ */ new Date()))) : null,
hasOpeningStock ? /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-4 w-full max-
w-xl" }, /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" :
"text-gray-700"}` }, "Date received ", /* @__PURE__ */ React.createElement("span", { className: "text-red-
500" }, "*")), /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "date",
          name: "dateReceived",
          value: newProduct.dateReceived || "",
          onChange: handleInputChange,
          className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2
focus:ring-[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white" :
"bg-white border-gray-300"}`
        }
      )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" :
"text-gray-700"}` }, "Expiration Date"), /* @__PURE__ */ React.createElement(
```

```
        "input",
        {
          type: "date",
          name: "expirationDate",
          value: newProduct.expirationDate || "",
          min: todayISO,
          onChange: handleInputChange,
          className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2
focus:ring-[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white" :
"bg-white border-gray-300"}`
        }
      ))) : null, hasOpeningStock && /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__
*/ React.createElement("div", { className: `h-px w-full ${theme === "dark" ? "bg-gray-700" : "bg-gray-
200"}` }), /* @__PURE__ */ React.createElement("div", { className: "flex items-start justify-between gap-6" },
/* @__PURE__ */ React.createElement("div", { className: "flex-1" }, /* @__PURE__ */
React.createElement("label", { className: `block text-xs font-bold uppercase tracking-wide mb-0.5 ${theme ===
"dark" ? "text-gray-300" : "text-gray-700"}` }, "Reorder Level (Per SKU) ", /* @__PURE__ */
React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */ React.createElement("p",
{ className: `text-[10px] mb-2 ${theme === "dark" ? "text-gray-500" : "text-gray-400"}` }, "System alerts
when any SKU falls below this level.")), /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "number",
          min: "0",
          name: "reorderNumber",
          value: newProduct.reorderNumber || "",
          onChange: handleInputChange,
          placeholder: "eg. 23",
          className: `w-36 px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white placeholder-
gray-500" : "bg-white border-gray-300 placeholder-gray-400"}`
        }
      )), newProduct.expirationDate && /* @__PURE__ */ React.createElement("div", { className: "flex items-
start justify-between gap-6" }, /* @__PURE__ */ React.createElement("div", { className: "flex-1" }, /*
@__PURE__ */ React.createElement("label", { className: `block text-xs font-bold uppercase tracking-wide mb-
0.5 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Expiration Threshold (days) ", /* @__PURE__
*/ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */ React.createElement("p",
{ className: `text-[10px] mb-2 ${theme === "dark" ? "text-gray-500" : "text-gray-400"}` }, "Alert users when
this product is nearing expiration.")), /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "number",
          min: "0",
          name: "expirationThresholdDays",
          value: newProduct.expirationThresholdDays || "",
          onChange: handleInputChange,
          placeholder: "eg. 30",
          className: `w-36 px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white placeholder-
gray-500" : "bg-white border-gray-300 placeholder-gray-400"}`
        }
      ))));
    })()), currentStep === 5 && (() => {
      const combos = [];
      const rvariants = selectedVariants.length > 0 ? selectedVariants : [null];
      const rsizes = newProduct.selectedSizes?.length > 0 ? newProduct.selectedSizes : [VARIANT_ONLY_KEY];
      rvariants.forEach((v) => {
        rsizes.forEach((s) => {
          combos.push({ variant: v, size: s });
        });
      });
      const hasAnyCombos = hasVariants && combos.length > 0 && (combos[0].variant || combos[0].size);
      const totalSkus = hasAnyCombos ? combos.length : 1;
      const variantType = hasVariants ? `${optionGroup1Name || "Color"}${newProduct.selectedSizes?.length >
0 ? ` x ${optionGroup2Name || "Size"}` : ""}` : "None";
      let totalStock = 0;
      if (hasAnyCombos) {
        combos.forEach(({ variant: v, size: s }) => {
          if (v && s) totalStock += parseInt(newProduct.variantQuantities?.[s]?.[v]) || 0;
          else if (s) totalStock += parseInt(newProduct.sizeQuantities?.[s]) || 0;
          else totalStock += parseInt(newProduct.currentStock) || 0;
        });
      } else {
        totalStock = parseInt(newProduct.currentStock) || 0;
      }
      return /* @__PURE__ */ React.createElement("div", { className: "flex flex-col gap-4 pt-2 h-full min-h-
0" }, /* @__PURE__ */ React.createElement("div", { className: `flex gap-4 rounded-xl border p-4 ${theme ===
```

```
"dark" ? "bg-[#1E1B18] border-gray-700" : "bg-gray-50 border-gray-200"}` }, /* @__PURE__ */
React.createElement("div", { className: "w-32 h-32 rounded-xl overflow-hidden flex-shrink-0 relative group" },
productImages.length > 0 ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement("img", { src: productImages[reviewImgIdx] || productImages[0], alt: "Product", className:
"w-full h-full object-cover" }), productImages.length > 1 && /* @__PURE__ */
React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: () => setReviewImgIdx((i) => (i - 1 + productImages.length) % productImages.length),
          className: "absolute left-0.5 top-1/2 -translate-y-1/2 w-6 h-6 rounded-full bg-black/40 text-white
flex items-center justify-center text-xs opacity-0 group-hover:opacity-100 transition-opacity"
        },
        "\u2039"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: () => setReviewImgIdx((i) => (i + 1) % productImages.length),
          className: "absolute right-0.5 top-1/2 -translate-y-1/2 w-6 h-6 rounded-full bg-black/40 text-white
flex items-center justify-center text-xs opacity-0 group-hover:opacity-100 transition-opacity"
        },
        "\u203A"
      ), /* @__PURE__ */ React.createElement("div", { className: "absolute bottom-1 left-1/2 -translate-x-1/2
flex gap-1" }, productImages.map((_, i) => /* @__PURE__ */ React.createElement("span", { key: i, className:
`w-1.5 h-1.5 rounded-full transition-colors ${i === reviewImgIdx ? "bg-white" : "bg-white/40"}` }))))) :
newProduct.itemImage && newProduct.itemImage.trim() !== "" ? /* @__PURE__ */ React.createElement("img", { src:
newProduct.itemImage, alt: "Product", className: "w-full h-full object-cover" }) : /* @__PURE__ */
React.createElement("div", { className: `w-full h-full flex items-center justify-center ${theme === "dark" ?
"bg-[#2A2724]" : "bg-gray-200"}` }, /* @__PURE__ */ React.createElement("span", { className: "text-gray-400
text-xs" }, "No image"))), /* @__PURE__ */ React.createElement("div", { className: "flex-1 min-w-0" }, /*
@__PURE__ */ React.createElement("p", { className: `text-lg font-bold leading-tight ${theme === "dark" ?
"text-white" : "text-gray-900"}` }, newProduct.itemName || "\u2014"), /* @__PURE__ */ React.createElement("p",
{ className: `text-[10px] uppercase tracking-wider mb-3 ${theme === "dark" ? "text-gray-500" : "text-gray-
400"}` }, "Product Name"), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-x-4
gap-y-2" }, /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p",
{ className: `text-sm font-semibold ${theme === "dark" ? "text-white" : "text-gray-800"}` },
newProduct.category || "\u2014"), /* @__PURE__ */ React.createElement("p", { className: `text-[10px]
uppercase tracking-wider ${theme === "dark" ? "text-gray-500" : "text-gray-400"}` }, "Category")), /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: `text-sm
font-semibold ${theme === "dark" ? "text-white" : "text-gray-800"}` }, newProduct.subCategory || "\u2014"),
/* @__PURE__ */ React.createElement("p", { className: `text-[10px] uppercase tracking-wider ${theme ===
"dark" ? "text-gray-500" : "text-gray-400"}` }, "Subcategory")), /* @__PURE__ */ React.createElement("div",
null, /* @__PURE__ */ React.createElement("p", { className: `text-sm font-semibold ${theme === "dark" ?
"text-white" : "text-gray-800"}` }, variantType), /* @__PURE__ */ React.createElement("p", { className:
`text-[10px] uppercase tracking-wider ${theme === "dark" ? "text-gray-500" : "text-gray-400"}` }, "Variant
Type")), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p",
{ className: `text-sm font-semibold ${theme === "dark" ? "text-white" : "text-gray-800"}` }, totalSkus), /*
@__PURE__ */ React.createElement("p", { className: `text-[10px] uppercase tracking-wider ${theme === "dark" ?
"text-gray-500" : "text-gray-400"}` }, "Total SKUs")), /* @__PURE__ */ React.createElement("div", null, /*
@__PURE__ */ React.createElement("p", { className: `text-sm font-semibold ${theme === "dark" ? "text-white" :
"text-gray-800"}` }, totalStock > 0 ? formatStockInStyleBatchCode(/* @__PURE__ */ new Date()) : /* @__PURE__
*/ React.createElement("span", { className: `text-xs font-normal ${theme === "dark" ? "text-gray-400" :
"text-gray-600"}` }, "No batch until you receive stock \u2014 use Stock In")), /* @__PURE__ */
React.createElement("p", { className: `text-[10px] uppercase tracking-wider ${theme === "dark" ? "text-gray-
500" : "text-gray-400"}` }, "Initial stock batch")), /* @__PURE__ */ React.createElement("div", null, /*
@__PURE__ */ React.createElement("p", { className: `text-sm font-semibold ${theme === "dark" ? "text-white" :
"text-gray-800"}` }, totalStock), /* @__PURE__ */ React.createElement("p", { className: `text-[10px]
uppercase tracking-wider ${theme === "dark" ? "text-gray-500" : "text-gray-400"}` }, "Total Stock"))))),
hasAnyCombos && /* @__PURE__ */ React.createElement("div", { className: `rounded-xl border overflow-hidden
flex flex-col min-h-0 flex-1 ${theme === "dark" ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */
React.createElement("div", { className: `grid grid-cols-[1fr_auto_auto_auto] gap-2 px-4 py-2 text-[10px]
font-bold uppercase tracking-wider flex-shrink-0 ${theme === "dark" ? "bg-[#2A2724] text-gray-400 border-b
border-gray-700" : "bg-gray-100 text-gray-500 border-b border-gray-200"}` }, /* @__PURE__ */
React.createElement("span", null, "Variant Combination"), /* @__PURE__ */ React.createElement("span",
{ className: "w-20 text-right" }, "Cost Price"), /* @__PURE__ */ React.createElement("span", { className: "w-
20 text-right" }, "Selling Price"), /* @__PURE__ */ React.createElement("span", { className: "w-14 text-
right" }, "Qty")), /* @__PURE__ */ React.createElement("div", { className: "overflow-y-auto flex-1" },
combos.map(({ variant: v, size: s }, idx) => {
        const sell = v && s ? variantPrices[s]?.[v] ?? newProduct.itemPrice ?? "" : s ?
newProduct.sizePrices?.[s] ?? newProduct.itemPrice ?? "" : newProduct.itemPrice ?? "";
        const cost = v && s ? variantCostPrices[s]?.[v] ?? newProduct.costPrice ?? "" : s ?
newProduct.sizeCostPrices?.[s] ?? newProduct.costPrice ?? "" : newProduct.costPrice ?? "";
        const qty = v && s ? newProduct.variantQuantities?.[s]?.[v] ?? "" : s ?
newProduct.sizeQuantities?.[s] ?? "" : newProduct.currentStock ?? "";
        const uom = newProduct.unitOfMeasure || "pcs";
        return /* @__PURE__ */ React.createElement("div", { key: idx, className: `grid grid-cols-
```

```
[1fr_auto_auto_auto] gap-2 items-center px-4 py-2.5 ${idx > 0 ? theme === "dark" ? "border-t border-gray-
700" : "border-t border-gray-100" : ""}` }, /* @__PURE__ */ React.createElement("div", { className: "flex
items-center gap-1.5" }, s && s !== VARIANT_ONLY_KEY && /* @__PURE__ */ React.createElement("span",
{ className: `inline-block px-2.5 py-0.5 text-[11px] rounded-full font-medium ${theme === "dark" ? "bg-
[#09A046]/15 text-[#09A046]" : "bg-[#09A046]/10 text-[#09A046]"}` }, s), v && s && /* @__PURE__ */
React.createElement("span", { className: `text-xs ${theme === "dark" ? "text-gray-500" : "text-gray-400"}` },
"\xD7"), v && /* @__PURE__ */ React.createElement("span", { className: `inline-block px-2.5 py-0.5 text-[11px]
rounded-full font-medium ${theme === "dark" ? "bg-pink-500/15 text-pink-400" : "bg-pink-100 text-pink-
700"}` }, v)), /* @__PURE__ */ React.createElement("span", { className: `w-20 text-right text-sm ${theme ===
"dark" ? "text-gray-300" : "text-gray-600"}` }, cost ? `\u20B1${cost}` : "\u2014"), /* @__PURE__ */
React.createElement("span", { className: `w-20 text-right text-sm font-semibold text-[#09A046]` }, sell ?
`\u20B1${sell}` : "\u2014"), /* @__PURE__ */ React.createElement("span", { className: `w-14 text-right text-
sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, qty ? `${qty} ${uom}` : "\u2014"));
      }))));
    })())), !editingProduct ? /* @__PURE__ */ React.createElement("div", { className: `px-8 py-4 flex
justify-between items-center border-t flex-shrink-0 ${theme === "dark" ? "border-gray-700" : "border-gray-
200"}` }, /* @__PURE__ */ React.createElement(
      "button",
      {
        type: "button",
        onClick: () => currentStep === 1 ? setShowAddModal(false) : setCurrentStep((prev) => prev - 1),
        className: `px-8 py-2.5 text-sm font-semibold rounded-xl border-2 transition-colors ${theme ===
"dark" ? "text-gray-300 border-gray-600 hover:bg-gray-700" : "text-gray-600 border-gray-300 hover:bg-gray-
100"}`
      },
      currentStep === 1 ? "Cancel" : "\u2190 Back"
    ), currentStep < 5 ? /* @__PURE__ */ React.createElement(
      "button",
      {
        key: "btn-continue",
        type: "button",
        onClick: (e) => {
          e.preventDefault();
          setCurrentStep((prev) => prev + 1);
        },
        disabled: !isStepValid(currentStep),
        className: `px-8 py-2.5 text-sm font-semibold rounded-xl text-white transition-all shadow-md
hover:opacity-90 ${!isStepValid(currentStep) ? "opacity-50 cursor-not-allowed" : ""}`,
        style: { background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)" }
      },
      "Continue"
    ) : /* @__PURE__ */ React.createElement(
      "button",
      {
        key: "btn-submit",
        type: "submit",
        disabled: loading,
        className: "px-8 py-2.5 text-sm font-semibold rounded-xl text-white transition-all shadow-md
hover:opacity-90 disabled:opacity-50",
        style: { background: "#09A046" }
      },
      loading ? "Adding..." : "Add Product"
    )) : /* @__PURE__ */ React.createElement("div", { className: `px-8 py-4 flex justify-between items-center
border-t flex-shrink-0 ${theme === "dark" ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */
React.createElement(
      "button",
      {
        type: "button",
        onClick: () => setShowAddModal(false),
        className: `px-8 py-2.5 text-sm font-semibold rounded-xl border-2 transition-colors ${theme ===
"dark" ? "text-gray-300 border-gray-600 hover:bg-gray-700" : "text-gray-600 border-gray-300 hover:bg-gray-
100"}`
      },
      "Cancel"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        type: "submit",
        disabled: loading,
        className: "px-8 py-2.5 text-sm font-semibold rounded-xl text-white transition-all shadow-md
hover:opacity-90 disabled:opacity-50",
        style: { background: "#1B89CD" }
      },
      loading ? "Updating..." : "Update Product"
    )))
  ), /* @__PURE__ */ React.createElement(
    AddCategoryModal,
```

```
    {
      show: showCategoryModal,
      onClose: () => setShowCategoryModal(false),
      onAdd: (newCategoryName) => {
        if (onCategoryAdd) {
          onCategoryAdd();
        }
        setNewProduct((prev) => ({ ...prev, category: newCategoryName }));
      }
    }
  ), /* @__PURE__ */ React.createElement(
    AddSubcategoryModal,
    {
      show: showSubcategoryModal,
      parentCategory: newProduct.category,
      onClose: () => setShowSubcategoryModal(false),
      onAdd: (newSubcategoryName) => {
        if (onCategoryAdd) {
          onCategoryAdd();
        }
        setNewProduct((prev) => ({ ...prev, subCategory: newSubcategoryName }));
      }
    }
  ), /* @__PURE__ */ React.createElement(
    AddBrandModal,
    {
      show: showBrandModal,
      onClose: () => setShowBrandModal(false),
      onAdd: (newBrandName) => {
        if (onBrandAdd) {
          onBrandAdd();
        }
        setNewProduct((prev) => ({ ...prev, brandName: newBrandName }));
      }
    }
  ));
};
export default AddProductModal;
```

FILE: pos/frontend/src/components/inventory/AddSubcategoryModal.jsx
==========

```
import { useState } from "react";
import { useTheme } from "../../context/ThemeContext";
const AddSubcategoryModal = ({ show, parentCategory, onClose, onAdd }) => {
  const { theme } = useTheme();
  const [newSubcategory, setNewSubcategory] = useState("");
  const [loading, setLoading] = useState(false);
  if (!show) return null;
  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!newSubcategory.trim()) return;
    setLoading(true);
    try {
      const response = await fetch("http://localhost:5000/api/categories", {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          name: newSubcategory,
          type: "subcategory",
          parentCategory
        })
      });
      const data = await response.json();
      if (data.success) {
        onAdd(newSubcategory);
        setNewSubcategory("");
        onClose();
      } else {
        alert(data.message || "Failed to add subcategory");
      }
    } catch (error) {
      console.error("Error adding subcategory:", error);
      alert("Failed to add subcategory. Please try again.");
```

```
    } finally {
      setLoading(false);
    }
  };
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-10000 p-4 bg-black/50 backdrop-blur-sm pointer-events-auto" }, /* @__PURE__ */
React.createElement("div", { className: `rounded-2xl w-full max-w-md p-6 shadow-2xl transform transition-all
scale-100 opacity-100 ${theme === "dark" ? "bg-[#2A2724] text-white" : "bg-white"}` }, /* @__PURE__ */
React.createElement("h2", { className: "text-2xl font-bold mb-6" }, "Add Subcategory"), parentCategory && /*
@__PURE__ */ React.createElement("p", { className: `text-sm mb-4 ${theme === "dark" ? "text-gray-400" :
"text-gray-500"}` }, "Parent Category: ", /* @__PURE__ */ React.createElement("span", { className: "font-
semibold" }, parentCategory)), /* @__PURE__ */ React.createElement("form", { onSubmit: handleSubmit }, /*
@__PURE__ */ React.createElement("div", { className: "mb-6" }, /* @__PURE__ */ React.createElement("label",
{ className: `block text-sm font-medium mb-2 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` },
"Subcategory Name"), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      value: newSubcategory,
      onChange: (e) => setNewSubcategory(e.target.value),
      placeholder: "Enter subcategory name",
      className: `w-full px-4 py-3 rounded-xl border focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
focus:border-transparent transition-all ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white
placeholder-gray-300" : "border-gray-300"}`,
      autoFocus: true
    }
  )), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3 justify-end" }, /* @__PURE__ */
React.createElement(
    "button",
    {
      type: "button",
      onClick: onClose,
      className: `px-6 py-2.5 rounded-xl font-medium transition-colors ${theme === "dark" ? "bg-gray-700
text-gray-300 hover:bg-gray-600" : "bg-gray-100 text-gray-700 hover:bg-gray-200"}`
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "submit",
      disabled: loading || !newSubcategory.trim(),
      className: "px-6 py-2.5 rounded-xl bg-green-600 text-white font-medium hover:bg-green-700
disabled:opacity-50 disabled:cursor-not-allowed transition-all shadow-lg"
    },
    loading ? "Adding..." : "Add Subcategory"
  )))));
};
export default AddSubcategoryModal;
```

FILE: pos/frontend/src/components/inventory/ArchiveConfirmationModal.jsx
==========

```
import React from "react";
import { FaTimes, FaArchive } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const ArchiveConfirmationModal = ({ isOpen, onClose, onConfirm, itemName }) => {
  const { theme } = useTheme();
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10002] p-4 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-2xl w-full max-w-md relative shadow-2xl ${theme === "dark" ? "bg-[#1E1B18]" : "bg-
white"}`,
      style: { boxShadow: "0 25px 50px -12px rgba(0, 0, 0, 0.5)" }
    },
    /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: `absolute top-4 right-4 transition-colors z-10 ${theme === "dark" ? "text-gray-400
hover:text-gray-200" : "text-gray-400 hover:text-gray-600"}`
      },
      /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
    ),
    /* @__PURE__ */ React.createElement("div", { className: "p-8" }, /* @__PURE__ */
```

```
React.createElement("div", { className: "flex justify-center mb-6" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "w-20 h-20 rounded-full flex items-center justify-center relative",
        style: {
          backgroundColor: theme === "dark" ? "#352F2A" : "#F3F4F6"
        }
      },
      /* @__PURE__ */ React.createElement(
        FaArchive,
        {
          className: `text-3xl ${theme === "dark" ? "text-gray-300" : "text-gray-600"}`
        }
      )
    )), /* @__PURE__ */ React.createElement("h3", { className: `text-xl font-bold text-center mb-3 ${theme
=== "dark" ? "text-white" : "text-gray-800"}` }, "Are you sure you want to archive this item?"), /* @__PURE__
*/ React.createElement("p", { className: `text-sm text-center mb-8 ${theme === "dark" ? "text-gray-400" :
"text-gray-500"}` }, "This action will hide the item from the active inventory.", /* @__PURE__ */
React.createElement("br", null), "Item: ", /* @__PURE__ */ React.createElement("span", { className: "font-
semibold" }, itemName)), /* @__PURE__ */ React.createElement("div", { className: "flex gap-4" }, /* @__PURE__
*/ React.createElement(
      "button",
      {
        onClick: onConfirm,
        className: "flex-1 py-3 px-6 rounded-lg font-bold text-white transition-all shadow-md hover:shadow-
lg",
        style: {
          background: "linear-gradient(135deg, #D4A59A 0%, #AD7F65 50%, #76462B 100%)"
        }
      },
      "Confirm"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: `flex-1 py-3 px-6 rounded-lg font-bold transition-all shadow-sm ${theme === "dark" ? "bg-
[#2A2724] text-gray-300 hover:bg-[#322f2c]" : "text-gray-700 bg-gray-200 hover:bg-gray-300"}`
      },
      "Cancel"
    )))
  ));
};
export default ArchiveConfirmationModal;
```

FILE: pos/frontend/src/components/inventory/ConfirmAddProductModal.jsx
==========

```
import React from "react";
import { FaTimes } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const ConfirmAddProductModal = ({ isOpen, onClose, onConfirm, productName, noStockWarning }) => {
  const { theme } = useTheme();
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-10000 p-4 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-2xl w-full max-w-md relative shadow-2xl ${theme === "dark" ? "bg-[#1E1B18]" : "bg-
white"}`,
      style: { boxShadow: "0 25px 50px -12px rgba(0, 0, 0, 0.5)" }
    },
    /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: `absolute top-4 right-4 transition-colors z-10 ${theme === "dark" ? "text-gray-400
hover:text-gray-200" : "text-gray-400 hover:text-gray-600"}`
      },
      /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
    ),
    /* @__PURE__ */ React.createElement("div", { className: "p-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-center mb-6" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "w-20 h-20 rounded-full flex items-center justify-center",
        style: { backgroundColor: "#E8D5C4" }
```

```
      },
      /* @__PURE__ */ React.createElement("span", { className: "text-4xl font-bold text-[#76462B]" }, "?")
    )), /* @__PURE__ */ React.createElement("h3", { className: `text-xl font-bold text-center mb-3 ${theme
=== "dark" ? "text-white" : "text-gray-800"}` }, "Are you sure you want to add this item?"), /* @__PURE__ */
React.createElement("div", { className: "mb-8" }, /* @__PURE__ */ React.createElement("p", { className:
`text-sm text-center ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "This action will be apply
to the inventory."), noStockWarning && /* @__PURE__ */ React.createElement("p", { className: "text-sm text-
center mt-3 font-semibold text-amber-600 dark:text-amber-400" }, "No stock yet \u2014 use Stock In")), /*
@__PURE__ */ React.createElement("div", { className: "flex gap-4" }, /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onConfirm,
        className: "flex-1 py-3 px-6 rounded-lg font-bold text-white transition-all shadow-md hover:shadow-
lg",
        style: {
          background: "linear-gradient(135deg, #D4A59A 0%, #AD7F65 50%, #76462B 100%)"
        }
      },
      "Confirm"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: `flex-1 py-3 px-6 rounded-lg font-bold transition-all shadow-sm ${theme === "dark" ? "bg-
[#2A2724] text-gray-300 hover:bg-[#322f2c]" : "text-gray-700 bg-gray-200 hover:bg-gray-300"}`
      },
      "Cancel"
    )))
  ));
};
export default ConfirmAddProductModal;
```

FILE: pos/frontend/src/components/inventory/DeleteConfirmationModal.jsx
==========

```
import React from "react";
import { FaTimes, FaTrash } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const DeleteConfirmationModal = ({
  isOpen,
  onClose,
  onConfirm,
  itemName,
  title = "Are you sure you want to delete this item?",
  description = "This action will be apply to the inventory.",
  confirmLabel = "Confirm"
}) => {
  const { theme } = useTheme();
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10002] p-4 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-2xl w-full max-w-md relative shadow-2xl ${theme === "dark" ? "bg-[#1E1B18]" : "bg-
white"}`,
      style: { boxShadow: "0 25px 50px -12px rgba(0, 0, 0, 0.5)" }
    },
    /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: `absolute top-4 right-4 transition-colors z-10 ${theme === "dark" ? "text-gray-400
hover:text-gray-200" : "text-gray-400 hover:text-gray-600"}`
      },
      /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
    ),
    /* @__PURE__ */ React.createElement("div", { className: "p-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-center mb-6" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "w-20 h-20 rounded-full flex items-center justify-center relative",
        style: {
          backgroundColor: "#FFE5E5",
          boxShadow: "0 4px 12px rgba(220, 38, 38, 0.2)"
        }
      },
```

```
      /* @__PURE__ */ React.createElement(
        FaTrash,
        {
          className: "text-3xl text-red-600",
          style: {
            animation: "trashShake 0.5s ease-out"
          }
        }
      ),
      /* @__PURE__ */ React.createElement("style", null, `
                @keyframes trashShake {
                  0% {
                    opacity: 0;
                    transform: scale(0) rotate(-10deg);
                  }
                  50% {
                    opacity: 1;
                    transform: scale(1.1) rotate(5deg);
                  }
                  100% {
                    opacity: 1;
                    transform: scale(1) rotate(0deg);
                  }
                }
              `)
    )), /* @__PURE__ */ React.createElement("h3", { className: `text-xl font-bold text-center mb-3 ${theme
=== "dark" ? "text-white" : "text-gray-800"}` }, title), /* @__PURE__ */ React.createElement("p", { className:
`text-sm text-center mb-8 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, description), /*
@__PURE__ */ React.createElement("div", { className: "flex gap-4" }, /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onConfirm,
        className: "flex-1 py-3 px-6 rounded-lg font-bold text-white transition-all shadow-md hover:shadow-
lg",
        style: {
          background: "linear-gradient(135deg, #D4A59A 0%, #AD7F65 50%, #76462B 100%)"
        }
      },
      confirmLabel
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: `flex-1 py-3 px-6 rounded-lg font-bold transition-all shadow-sm ${theme === "dark" ? "bg-
[#2A2724] text-gray-300 hover:bg-[#322f2c]" : "text-gray-700 bg-gray-200 hover:bg-gray-300"}`
      },
      "Cancel"
    )))
  ));
};
export default DeleteConfirmationModal;
```

FILE: pos/frontend/src/components/inventory/EditConfirmationModal.jsx
==========

```
import React from "react";
import { FaTimes } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const EditConfirmationModal = ({ isOpen, onClose, onConfirm, itemName }) => {
  const { theme } = useTheme();
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10002] p-4 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-2xl w-full max-w-md relative shadow-2xl ${theme === "dark" ? "bg-[#1E1B18]" : "bg-
white"}`,
      style: { boxShadow: "0 25px 50px -12px rgba(0, 0, 0, 0.5)" }
    },
    /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: `absolute top-4 right-4 transition-colors z-10 ${theme === "dark" ? "text-gray-400
hover:text-gray-200" : "text-gray-400 hover:text-gray-600"}`
      },
```

```
      /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
    ),
    /* @__PURE__ */ React.createElement("div", { className: "p-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-center mb-6" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "w-20 h-20 rounded-full flex items-center justify-center relative",
        style: {
          backgroundColor: "#E8D5C4",
          boxShadow: "0 4px 12px rgba(118, 70, 43, 0.2)"
        }
      },
      /* @__PURE__ */ React.createElement(
        "span",
        {
          className: "text-4xl font-bold text-[#76462B]",
          style: {
            animation: "questionPulse 0.6s ease-out"
          }
        },
        "?"
      ),
      /* @__PURE__ */ React.createElement("style", null, `
                @keyframes questionPulse {
                  0% {
                    opacity: 0;
                    transform: scale(0) rotate(-10deg);
                  }
                  50% {
                    opacity: 1;
                    transform: scale(1.2) rotate(5deg);
                  }
                  100% {
                    opacity: 1;
                    transform: scale(1) rotate(0deg);
                  }
                }
              `)
    )), /* @__PURE__ */ React.createElement("h3", { className: `text-xl font-bold text-center mb-3 ${theme
=== "dark" ? "text-white" : "text-gray-800"}` }, "Are you sure you want to edit this item?"), /* @__PURE__ */
React.createElement("p", { className: `text-sm text-center mb-8 ${theme === "dark" ? "text-gray-400" : "text-
gray-500"}` }, "This action will be apply to the inventory."), /* @__PURE__ */ React.createElement("div",
{ className: "flex gap-4" }, /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onConfirm,
        className: "flex-1 py-3 px-6 rounded-lg font-bold text-white transition-all shadow-md hover:shadow-
lg",
        style: {
          background: "linear-gradient(135deg, #D4A59A 0%, #AD7F65 50%, #76462B 100%)"
        }
      },
      "Confirm"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: `flex-1 py-3 px-6 rounded-lg font-bold transition-all shadow-sm ${theme === "dark" ? "bg-
[#2A2724] text-gray-300 hover:bg-[#322f2c]" : "text-gray-700 bg-gray-200 hover:bg-gray-300"}`
      },
      "Cancel"
    )))
  ));
};
export default EditConfirmationModal;
```

FILE: pos/frontend/src/components/inventory/Pagination.jsx
==========

```
import React from "react";
import { useTheme } from "../../context/ThemeContext";
const Pagination = ({ currentPage, totalPages, onPageChange }) => {
  const { theme } = useTheme();
  if (totalPages <= 1) return null;
  const getPageNumbers = () => {
    const pages = [];
```

```
    const maxVisible = 5;
    if (totalPages <= maxVisible) {
      for (let i = 1; i <= totalPages; i++) {
        pages.push(i);
      }
    } else {
      if (currentPage <= 3) {
        for (let i = 1; i <= 4; i++) {
          pages.push(i);
        }
        pages.push("...");
        pages.push(totalPages);
      } else if (currentPage >= totalPages - 2) {
        pages.push(1);
        pages.push("...");
        for (let i = totalPages - 3; i <= totalPages; i++) {
          pages.push(i);
        }
      } else {
        pages.push(1);
        pages.push("...");
        for (let i = currentPage - 1; i <= currentPage + 1; i++) {
          pages.push(i);
        }
        pages.push("...");
        pages.push(totalPages);
      }
    }
    return pages;
  };
  return /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-center mt-6" },
/* @__PURE__ */ React.createElement("div", { className: `rounded-lg shadow-md px-6 py-3 flex items-center
gap-4 ${theme === "dark" ? "bg-[#2A2724] border border-[#4A4037]" : "bg-white"}` }, /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: () => onPageChange(currentPage - 1),
      disabled: currentPage === 1,
      className: `transition-colors ${currentPage === 1 ? theme === "dark" ? "text-gray-600 cursor-not-
allowed" : "text-gray-300 cursor-not-allowed" : theme === "dark" ? "text-gray-400 hover:text-white cursor-
pointer" : "text-gray-700 hover:text-gray-900 cursor-pointer"}`
    },
    /* @__PURE__ */ React.createElement("svg", { className: "w-5 h-5", fill: "none", stroke: "currentColor",
viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap: "round", strokeLinejoin:
"round", strokeWidth: 2, d: "M15 19l-7-7 7-7" }))
  ), /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" },
getPageNumbers().map((page, index) => {
    if (page === "...") {
      return /* @__PURE__ */ React.createElement("span", { key: `ellipsis-${index}`, className: theme ===
"dark" ? "text-gray-400" : "text-gray-900" }, "...");
    }
    return /* @__PURE__ */ React.createElement(
      "button",
      {
        key: page,
        onClick: () => onPageChange(page),
        className: `${currentPage === page ? theme === "dark" ? "bg-[#AD7F65] text-white shadow-md" : "bg-
[#DDC9BC] text-gray-900 rounded shadow-sm" : theme === "dark" ? "text-gray-300 hover:text-white" : "text-
gray-900 hover:text-gray-700"} w-8 h-8 flex items-center justify-center font-medium rounded transition-
colors`
      },
      page
    );
  })), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => onPageChange(currentPage + 1),
      disabled: currentPage === totalPages,
      className: `transition-colors ${currentPage === totalPages ? theme === "dark" ? "text-gray-600 cursor-
not-allowed" : "text-gray-300 cursor-not-allowed" : theme === "dark" ? "text-gray-400 hover:text-white
cursor-pointer" : "text-gray-700 hover:text-gray-900 cursor-pointer"}`
    },
    /* @__PURE__ */ React.createElement("svg", { className: "w-5 h-5", fill: "none", stroke: "currentColor",
viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap: "round", strokeLinejoin:
"round", strokeWidth: 2, d: "M9 5l7 7-7 7" }))
  )));
};
```

```
export default Pagination;



FILE: pos/frontend/src/components/inventory/ProductTable.jsx
==============================================================================
import { useEffect, useRef } from "react";
import { MdCategory } from "react-icons/md";
import { useTheme } from "../../context/ThemeContext";
const ProductTable = ({
  loading,
  filteredProducts,
  handleEditProduct,
  handleDeleteProduct,
  handleArchiveProduct,
  handleViewProduct,
  handleStockUpdate,
  selectedProductIds = [],
  onToggleSelect,
  onToggleSelectAll,
  showSelection = false,
  allVisibleSelected = false,
  someVisibleSelected = false
}) => {
  const { theme } = useTheme();
  const getSizePrice = (sizeData) => {
    if (typeof sizeData === "object" && sizeData !== null && sizeData.price !== void 0) {
      return sizeData.price;
    }
    return null;
  };
  const getSizeQuantity = (sizeData) => {
    if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
      return sizeData.quantity;
    }
    return typeof sizeData === "number" ? sizeData : 0;
  };
  const selectAllRef = useRef(null);
  useEffect(() => {
    if (selectAllRef.current) {
      selectAllRef.current.indeterminate = showSelection && !allVisibleSelected && someVisibleSelected;
    }
  }, [showSelection, allVisibleSelected, someVisibleSelected]);
  const getPriceRange = (product) => {
    let prices = [];
    if (product.sizes && typeof product.sizes === "object") {
      Object.values(product.sizes).forEach((sizeData) => {
        if (typeof sizeData === "object" && sizeData !== null) {
          if (sizeData.variants && typeof sizeData.variants === "object") {
            Object.entries(sizeData.variants).forEach(([vName, v]) => {
              const vPrice = v?.price ?? sizeData.variantPrices?.[vName] ?? sizeData.price;
              if (vPrice !== void 0 && vPrice !== null) prices.push(Number(vPrice));
            });
          } else if (sizeData.variantPrices && typeof sizeData.variantPrices === "object") {
            Object.values(sizeData.variantPrices).forEach((p) => {
              if (p !== void 0 && p !== null) prices.push(Number(p));
            });
          } else if (sizeData.price !== void 0 && sizeData.price !== null) {
            prices.push(Number(sizeData.price));
          }
        }
      });
    }
    if (prices.length === 0) {
      prices.push(Number(product.itemPrice || 0));
    }
    const validPrices = prices.filter((p) => !isNaN(p));
    if (validPrices.length > 0) {
      const minPrice = Math.min(...validPrices);
      const maxPrice = Math.max(...validPrices);
      return {
        min: minPrice,
        max: maxPrice,
        isRange: minPrice !== maxPrice
      };
    }
    return {
```

```
      min: 0,
      max: 0,
      isRange: false
    };
  };
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-lg shadow ${theme === "dark" ? "bg-[#2A2724] border border-[#4A4037]" : "bg-
white"}`
    },
    /* @__PURE__ */ React.createElement("div", { className: "overflow-x-auto p-6" }, loading ? /* @__PURE__
*/ React.createElement("div", { className: "text-center py-10" }, "Loading...") : filteredProducts.length ===
0 ? /* @__PURE__ */ React.createElement("div", { className: "text-center py-10 text-gray-500" }, 'No products
found. Click "Add New Item" to add your first product.') : /* @__PURE__ */ React.createElement(
      "table",
      {
        className: "w-full relative border-separate",
        style: { borderSpacing: "0" }
      },
      /* @__PURE__ */ React.createElement(
        "thead",
        {
          className: `border-b ${theme === "dark" ? "border-[#4A4037]" : "border-gray-200"}`
        },
        /* @__PURE__ */ React.createElement(
          "tr",
          {
            className: `text-left text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-600"}`
          },
          showSelection && /* @__PURE__ */ React.createElement("th", { className: "pb-3 pr-4 w-10" }, /*
@__PURE__ */ React.createElement(
            "input",
            {
              ref: selectAllRef,
              type: "checkbox",
              className: "w-4 h-4 text-[#AD7F65] border-gray-300 rounded focus:ring-[#AD7F65]",
              onChange: onToggleSelectAll,
              checked: showSelection ? allVisibleSelected : false
            }
          )),
          /* @__PURE__ */ React.createElement("th", { className: "pb-3 pr-4" }, "Item Image"),
          /* @__PURE__ */ React.createElement("th", { className: "pb-3 px-4" }, "SKU"),
          /* @__PURE__ */ React.createElement("th", { className: "pb-3 px-4" }, "Item Name"),
          /* @__PURE__ */ React.createElement("th", { className: "pb-3 px-4" }, "Category"),
          /* @__PURE__ */ React.createElement("th", { className: "pb-3 px-4" }, "Brand"),
          /* @__PURE__ */ React.createElement("th", { className: "pb-3 px-4" }, "Item Price"),
          /* @__PURE__ */ React.createElement("th", { className: "pb-3 px-4 text-center" }, "Current Stock"),
          /* @__PURE__ */ React.createElement("th", { className: "pb-3 px-4 text-center" }, "Terminal"),
          /* @__PURE__ */ React.createElement("th", { className: "pb-3 pl-4" }, "Actions")
        )
      ),
      /* @__PURE__ */ React.createElement("tbody", { className: "relative" }, filteredProducts.map(
        (product) => /* @__PURE__ */ React.createElement(
          "tr",
          {
            key: product._id,
            className: `border-b transition-colors cursor-pointer ${theme === "dark" ? "border-[#4A4037]
hover:bg-[#352F2A] text-gray-300" : "border-gray-100 hover:bg-gray-50 text-gray-800"}`,
            onClick: () => handleViewProduct(product)
          },
          showSelection && /* @__PURE__ */ React.createElement(
            "td",
            {
              className: "py-3 pr-4",
              onClick: (e) => e.stopPropagation()
            },
            /* @__PURE__ */ React.createElement(
              "input",
              {
                type: "checkbox",
                className: "w-4 h-4 text-[#AD7F65] border-gray-300 rounded focus:ring-[#AD7F65]",
                checked: selectedProductIds.includes(product._id),
                onChange: () => onToggleSelect && onToggleSelect(product._id)
              }
            )
          ),
```

```
          /* @__PURE__ */ React.createElement("td", { className: "py-3 pr-4" }, product.itemImage &&
product.itemImage.trim() !== "" ? /* @__PURE__ */ React.createElement(
            "img",
            {
              src: product.itemImage,
              alt: product.itemName,
              className: "w-12 h-12 object-cover rounded"
            }
          ) : /* @__PURE__ */ React.createElement(
            "div",
            {
              className: `w-12 h-12 rounded flex items-center justify-center ${theme === "dark" ? "bg-
[#352F2A] text-gray-500" : "bg-gray-200 text-gray-400"}`
            },
            /* @__PURE__ */ React.createElement(MdCategory, null)
          )),
          /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4" }, product.sku),
          /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4" }, product.itemName),
          /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4" }, product.category),
          /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4" }, product.brandName || "-"),
          /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4" }, (() => {
            const priceRange = getPriceRange(product);
            return priceRange.isRange ? `PHP ${priceRange.min.toFixed(2)} - ${priceRange.max.toFixed(2)}` :
`PHP ${priceRange.min.toFixed(2)}`;
          })()),
          /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4 text-center" }, (() => {
            const totalStock = product.sizes && typeof product.sizes === "object" &&
Object.keys(product.sizes).length > 0 ? Object.values(product.sizes).reduce(
              (sum, sd) => sum + getSizeQuantity(sd),
              0
            ) : product.currentStock || 0;
            return /* @__PURE__ */ React.createElement(
              "span",
              {
                className: `px-2 py-1 rounded font-semibold ${totalStock === 0 ? "bg-red-100 text-red-700" :
totalStock <= (product.reorderNumber || 10) ? "bg-yellow-100 text-yellow-700" : "bg-green-100 text-green-
700"}`
              },
              totalStock
            );
          })()),
          /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4 text-center" }, /* @__PURE__ */
React.createElement(
            "span",
            {
              className: `px-2 py-1 rounded text-sm font-medium ${product.displayInTerminal !== false ? "bg-
green-100 text-green-700" : "bg-gray-100 text-gray-600"}`
            },
            product.terminalStatus || (product.displayInTerminal !== false ? "shown" : "not shown")
          )),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: "py-3 pl-4",
              onClick: (e) => e.stopPropagation()
            },
            /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-2" }, /* @__PURE__
*/ React.createElement(
              "button",
              {
                onClick: (e) => {
                  e.stopPropagation();
                  handleStockUpdate(product, "in");
                },
                className: `p-2 rounded-lg transition-colors group relative ${theme === "dark" ? "hover:bg-
[#352F2A]" : "hover:bg-green-50"}`,
                title: "Stock In"
              },
              /* @__PURE__ */ React.createElement("div", { className: "relative w-6 h-6 flex items-center
justify-center" }, /* @__PURE__ */ React.createElement(
                "svg",
                {
                  className: `w-6 h-6 ${theme === "dark" ? "text-gray-400 group-hover:text-green-500" :
"text-gray-600 group-hover:text-green-600"}`,
                  fill: "none",
                  stroke: "currentColor",
                  viewBox: "0 0 24 24"
```

```
                },
                /* @__PURE__ */ React.createElement(
                  "path",
                  {
                    strokeLinecap: "round",
                    strokeLinejoin: "round",
                    strokeWidth: 2,
                    d: "M20 7l-8-4-8 4m16 0l-8 4m8-4v10l-8 4m0-10L4 7m8 4v10M4 7v10l8 4"
                  }
                )
              ), /* @__PURE__ */ React.createElement("span", { className: "absolute -bottom-0.5 -right-0.5 w-
3 h-3 bg-green-500 rounded-full flex items-center justify-center border border-white" }, /* @__PURE__ */
React.createElement("span", { className: "text-white text-[8px] font-bold leading-none" }, "+")))
            ), /* @__PURE__ */ React.createElement(
              "button",
              {
                onClick: (e) => {
                  e.stopPropagation();
                  handleStockUpdate(product, "out");
                },
                className: `p-2 rounded-lg transition-colors group relative ${theme === "dark" ? "hover:bg-
[#352F2A]" : "hover:bg-red-50"}`,
                title: "Stock Out"
              },
              /* @__PURE__ */ React.createElement("div", { className: "relative w-6 h-6 flex items-center
justify-center" }, /* @__PURE__ */ React.createElement(
                "svg",
                {
                  className: `w-6 h-6 ${theme === "dark" ? "text-gray-400 group-hover:text-red-500" : "text-
gray-600 group-hover:text-red-600"}`,
                  fill: "none",
                  stroke: "currentColor",
                  viewBox: "0 0 24 24"
                },
                /* @__PURE__ */ React.createElement(
                  "path",
                  {
                    strokeLinecap: "round",
                    strokeLinejoin: "round",
                    strokeWidth: 2,
                    d: "M20 7l-8-4-8 4m16 0l-8 4m8-4v10l-8 4m0-10L4 7m8 4v10M4 7v10l8 4"
                  }
                )
              ), /* @__PURE__ */ React.createElement("span", { className: "absolute -bottom-0.5 -right-0.5 w-
3 h-3 bg-red-500 rounded-full flex items-center justify-center border border-white" }, /* @__PURE__ */
React.createElement("span", { className: "text-white text-[8px] font-bold leading-none" }, "-")))
            ), /* @__PURE__ */ React.createElement(
              "button",
              {
                onClick: (e) => {
                  e.stopPropagation();
                  handleEditProduct(product);
                },
                className: `p-2 rounded-lg transition-colors group ${theme === "dark" ? "hover:bg-[#352F2A]" :
"hover:bg-blue-50"}`,
                title: "Update"
              },
              /* @__PURE__ */ React.createElement(
                "svg",
                {
                  className: "w-6 h-6 text-blue-500 group-hover:text-blue-600",
                  fill: "none",
                  stroke: "currentColor",
                  viewBox: "0 0 24 24"
                },
                /* @__PURE__ */ React.createElement(
                  "path",
                  {
                    strokeLinecap: "round",
                    strokeLinejoin: "round",
                    strokeWidth: 2,
                    d: "M11 5H6a2 2 0 00-2 2v11a2 2 0 002 2h11a2 2 0 002-2v-5m-1.414-9.414a2 2 0 112.828
2.828L11.828 15H9v-2.828l8.586-8.586z"
                  }
                )
              )
            ), /* @__PURE__ */ React.createElement(
```

```
              "button",
              {
                onClick: (e) => {
                  e.stopPropagation();
                  if (handleArchiveProduct) handleArchiveProduct(product);
                },
                className: `p-2 rounded-lg transition-colors group ${theme === "dark" ? "hover:bg-[#352F2A]" :
"hover:bg-orange-50"}`,
                title: "Archive"
              },
              /* @__PURE__ */ React.createElement(
                "svg",
                {
                  className: "w-6 h-6 text-orange-400 group-hover:text-orange-500",
                  fill: "none",
                  stroke: "currentColor",
                  viewBox: "0 0 24 24"
                },
                /* @__PURE__ */ React.createElement(
                  "path",
                  {
                    strokeLinecap: "round",
                    strokeLinejoin: "round",
                    strokeWidth: 2,
                    d: "M5 8h14M5 8a2 2 0 110-4h14a2 2 0 110 4M5 8v10a2 2 0 002 2h10a2 2 0 002-2V8m-9 4h4"
                  }
                )
              )
            ))
          )
        )
      ))
    ))
  );
};
export default ProductTable;
```

FILE: pos/frontend/src/components/inventory/StockInModal.jsx
==========

```
import { useEffect, useMemo, useState } from "react";
import { useTheme } from "../../context/ThemeContext";
const VARIANT_ONLY_SIZE_KEY = "__VARIANT_ONLY__";
const buildPickerCombos = (v1Tags, v2Tags, existingSet, addedSet) => {
  const out = [];
  if (v1Tags.length > 0 && v2Tags.length > 0) {
    v1Tags.forEach((v1) => {
      v2Tags.forEach((v2) => {
        const key = `${v2}|${v1}`;
        if (!existingSet.has(key) && !addedSet.has(key)) out.push({ size: v2, variant: v1 });
      });
    });
  } else if (v1Tags.length > 0) {
    v1Tags.forEach((v1) => {
      const key = `${VARIANT_ONLY_SIZE_KEY}|${v1}`;
      if (!existingSet.has(key) && !addedSet.has(key)) out.push({ size: VARIANT_ONLY_SIZE_KEY, variant: v1 });
    });
  }
  return out;
};
const StockInModal = ({ isOpen, onClose, product, onConfirm, loading, brandPartners = [] }) => {
  const [selectedSizes, setSelectedSizes] = useState([]);
  const [sizeQuantities, setSizeQuantities] = useState({});
  const [variantQuantities, setVariantQuantities] = useState({});
  const [quantity, setQuantity] = useState("");
  const [batchCode, setBatchCode] = useState("");
  const [batchExpirationDate, setBatchExpirationDate] = useState("");
  const [reason, setReason] = useState("Restock");
  const [otherReason, setOtherReason] = useState("");
  const [newVariantInputs, setNewVariantInputs] = useState({});
  const [addedVariants, setAddedVariants] = useState([]);
  const [newVariantPrices, setNewVariantPrices] = useState({});
  const [showNewSizes, setShowNewSizes] = useState(false);
  const [customNewSizeInput, setCustomNewSizeInput] = useState("");
  const [addedNewSizes, setAddedNewSizes] = useState([]);
  const [newSizePrices, setNewSizePrices] = useState({});
```

```
  const [diffPricesPerVariant, setDiffPricesPerVariant] = useState({});
  const [stockVariantPrices, setStockVariantPrices] = useState({});
  const [currentStep, setCurrentStep] = useState(1);
  const [checkedCombos, setCheckedCombos] = useState({});
  const [showAddNewSection, setShowAddNewSection] = useState(false);
  const [newV1Tags, setNewV1Tags] = useState([]);
  const [newV2Tags, setNewV2Tags] = useState([]);
  const [newV1Input, setNewV1Input] = useState("");
  const [newV2Input, setNewV2Input] = useState("");
  const [addedNewCombos, setAddedNewCombos] = useState([]);
  const [newComboData, setNewComboData] = useState({});
  const [fillAllCostSI, setFillAllCostSI] = useState("");
  const [fillAllSellSI, setFillAllSellSI] = useState("");
  const [fillAllQtySI, setFillAllQtySI] = useState("");
  const [stockInBrandPartner, setStockInBrandPartner] = useState("");
  const [dateReceived, setDateReceived] = useState("");
  const [stockInBatchChoice, setStockInBatchChoice] = useState("new");
  const { theme } = useTheme();
  const todayISO = useMemo(() => (/* @__PURE__ */ new Date()).toISOString().slice(0, 10), []);
  const brandPartnerNames = useMemo(
    () => [...new Set(brandPartners.map((bp) => bp.brandName).filter(Boolean))].sort(),
    [brandPartners]
  );
  const reasons = ["Restock", "Returned Item", "Exchange", "Other"];
  const hasSizes = product.sizes && typeof product.sizes === "object" && Object.keys(product.sizes).length >
0;
  const getSizeQuantity = (sizeData) => {
    if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
      return sizeData.quantity;
    }
    return typeof sizeData === "number" ? sizeData : 0;
  };
  const getSizeVariants = (size) => {
    if (hasSizes && product.sizes[size] && typeof product.sizes[size] === "object" &&
product.sizes[size].variants) {
      return product.sizes[size].variants;
    }
    return null;
  };
  const getAllVariants = () => {
    const variantSet = /* @__PURE__ */ new Set();
    if (hasSizes) {
      Object.values(product.sizes).forEach((sizeData) => {
        if (typeof sizeData === "object" && sizeData?.variants) {
          Object.keys(sizeData.variants).forEach((v) => variantSet.add(v));
        }
      });
    }
    addedVariants.forEach((v) => variantSet.add(v));
    return Array.from(variantSet);
  };
  const allVariants = getAllVariants();
  const hasVariants = allVariants.length > 0;
  const shouldShowVariantBuilder = hasSizes && hasVariants || !hasSizes && (product?.currentStock || 0) === 0;
  const stockInBatchList = useMemo(() => {
    if (!hasSizes || !product) return [];
    const getVariantQty = (vData) => {
      if (typeof vData === "number") return parseInt(vData, 10) || 0;
      if (vData && typeof vData === "object") {
        const q = vData.qty ?? vData.quantity ?? 0;
        return parseInt(q, 10) || 0;
      }
      return 0;
    };
    if (hasVariants) {
      const maxBatchDepth2 = (() => {
        let max = 0;
        Object.values(product.sizes).forEach((sd) => {
          if (!sd || typeof sd !== "object" || !sd?.variants) return;
          Object.values(sd.variants).forEach((vData) => {
            const batches = typeof vData === "object" && Array.isArray(vData.batches) ? vData.batches : [];
            max = Math.max(max, batches.length);
          });
        });
        return max;
      })();
      if (maxBatchDepth2 <= 0) {
```

```
        const openingBatchCode = product?.batchNumber || product?.openingBatchNumber || "B1";
        let totalQty = 0;
        Object.entries(product.sizes).forEach(([_, sd]) => {
          if (!sd || typeof sd !== "object" || !sd?.variants) return;
          Object.entries(sd.variants).forEach(([__, vData]) => {
            totalQty += getVariantQty(vData);
          });
        });
        if (totalQty <= 0) return [];
        return [{ slotIndex: 0, code: openingBatchCode, totalQty }];
      }
      const slots2 = Array.from({ length: maxBatchDepth2 }, (_, slotIndex) => ({
        slotIndex,
        code: "",
        totalQty: 0
      }));
      Object.entries(product.sizes).forEach(([_, sd]) => {
        if (!sd || typeof sd !== "object" || !sd?.variants) return;
        Object.entries(sd.variants).forEach(([__, vData]) => {
          const batches = typeof vData === "object" && Array.isArray(vData.batches) ? vData.batches : [];
          const fallbackQty = (idx) => idx === 0 ? getVariantQty(vData) : 0;
          for (let slotIndex = 0; slotIndex < maxBatchDepth2; slotIndex++) {
            const b = batches[slotIndex];
            const qty = b ? parseInt(b.qty, 10) || 0 : fallbackQty(slotIndex);
            if (qty <= 0) continue;
            const slot = slots2[slotIndex];
            slot.totalQty += qty;
            if (!slot.code) {
              slot.code = b?.batchCode || (slotIndex === 0 ? product?.batchNumber ||
product?.openingBatchNumber || "" : "");
            }
          }
        });
      });
      return slots2.filter((s) => s.totalQty > 0);
    }
    const maxBatchDepth = Math.max(
      0,
      ...Object.values(product.sizes).map((sd) => {
        if (!sd || typeof sd !== "object") return 0;
        return Array.isArray(sd.batches) ? sd.batches.length : 0;
      })
    );
    if (maxBatchDepth <= 0) {
      const openingBatchCode = product?.batchNumber || product?.openingBatchNumber || "B1";
      let totalQty = 0;
      Object.values(product.sizes).forEach((sd) => {
        if (typeof sd === "object" && sd !== null) totalQty += getSizeQuantity(sd);
      });
      if (totalQty <= 0) return [];
      return [{ slotIndex: 0, code: openingBatchCode, totalQty }];
    }
    const slots = Array.from({ length: maxBatchDepth }, (_, slotIndex) => ({
      slotIndex,
      code: "",
      totalQty: 0
    }));
    Object.values(product.sizes).forEach((sd) => {
      if (!sd || typeof sd !== "object") return;
      const batches = Array.isArray(sd.batches) ? sd.batches : [];
      for (let slotIndex = 0; slotIndex < maxBatchDepth; slotIndex++) {
        const b = batches[slotIndex];
        const qty = b ? parseInt(b.qty, 10) || 0 : slotIndex === 0 ? getSizeQuantity(sd) : 0;
        if (qty <= 0) continue;
        const slot = slots[slotIndex];
        slot.totalQty += qty;
        if (!slot.code) {
          slot.code = b?.batchCode || (slotIndex === 0 ? product?.batchNumber || product?.openingBatchNumber
|| "" : "");
        }
      }
    });
    return slots.filter((s) => s.totalQty > 0);
  }, [product, hasSizes, hasVariants]);
  const existingSizes = hasSizes ? Object.keys(product.sizes) : [];
  const availableSizes = [...existingSizes, ...addedNewSizes];
  const allPossibleSizes = (() => {
```

```
    const category = product.category || "";
    let sizes = [];
    if (["Tops", "Bottoms", "Dresses"].includes(category)) {
      sizes = ["XS", "S", "M", "L", "XL", "XXL", "XXXL", "Free Size"];
    } else if (category === "Shoes") {
      sizes = ["5", "6", "7", "8", "9", "10", "11", "12"];
    } else if (["Accessories", "Head Wear", "Makeup"].includes(category)) {
      sizes = ["Free Size"];
    } else if (category === "Foods") {
      const subtype = product.foodSubtype || "";
      if (["Beverages", "Drinks"].includes(subtype)) {
        sizes = ["Small", "Medium", "Large", "Family Size", "Free Size"];
      } else {
        sizes = ["Small", "Medium", "Large", "Free Size"];
      }
    } else {
      sizes = ["XS", "S", "M", "L", "XL", "XXL", "XXXL", "Small", "Medium", "Large", "Free Size"];
    }
    return sizes.filter((s) => !existingSizes.includes(s) && !addedNewSizes.includes(s));
  })();
  const existingCombos = (() => {
    const combos = [];
    if (hasSizes) {
      existingSizes.forEach((size) => {
        const sizeVariants = getSizeVariants(size);
        if (sizeVariants) {
          Object.keys(sizeVariants).forEach((variant) => {
            const vData = sizeVariants[variant];
            const stock = typeof vData === "object" ? vData.quantity || 0 : typeof vData === "number" ?
vData : 0;
            combos.push({ size, variant, stock });
          });
        }
      });
    }
    return combos;
  })();
  const getExistingPrice = (size, variant) => {
    const sizeData = hasSizes && product.sizes[size];
    if (!sizeData || typeof sizeData !== "object") return { cost: product.costPrice || 0, sell:
product.itemPrice || 0 };
    const vts = sizeData.variants;
    if (!vts || !vts[variant]) return { cost: sizeData.costPrice || product.costPrice || 0, sell:
sizeData.price || product.itemPrice || 0 };
    const vData = vts[variant];
    if (typeof vData === "object" && vData !== null) {
      return {
        cost: vData.costPrice || sizeData.variantCostPrices?.[variant] || sizeData.costPrice ||
product.costPrice || 0,
        sell: vData.price || sizeData.variantPrices?.[variant] || sizeData.price || product.itemPrice || 0
      };
    }
    return { cost: sizeData.costPrice || product.costPrice || 0, sell: sizeData.price || product.itemPrice ||
0 };
  };
  useEffect(() => {
    if (isOpen && product) {
      setSizeQuantities({});
      setVariantQuantities({});
      setQuantity("");
      const now = /* @__PURE__ */ new Date();
      const y = now.getFullYear();
      const m = String(now.getMonth() + 1).padStart(2, "0");
      const d = String(now.getDate()).padStart(2, "0");
      const h = String(now.getHours()).padStart(2, "0");
      const min = String(now.getMinutes()).padStart(2, "0");
      setBatchCode(`B${y}${m}${d}-${h}${min}`);
      setBatchExpirationDate("");
      setReason("Restock");
      setOtherReason("");
      setNewVariantInputs({});
      setAddedVariants([]);
      setNewVariantPrices({});
      setShowNewSizes(false);
      setCustomNewSizeInput("");
      setAddedNewSizes([]);
      setNewSizePrices({});
```

```
      setCurrentStep(1);
      setShowAddNewSection(false);
      setNewV1Tags([]);
      setNewV2Tags([]);
      setNewV1Input("");
      setNewV2Input("");
      setAddedNewCombos([]);
      setNewComboData({});
      setFillAllCostSI("");
      setFillAllSellSI("");
      setFillAllQtySI("");
      const preferredBrand = String(product?.brandName || "").trim();
      const explicitDefaultBrand = brandPartnerNames.find((name) => String(name).trim().toLowerCase() ===
"default") || "";
      const defaultBrand = explicitDefaultBrand || preferredBrand && brandPartnerNames.find((name) =>
String(name).trim().toLowerCase() === preferredBrand.toLowerCase()) || brandPartnerNames[0] || "";
      setStockInBrandPartner(defaultBrand);
      setDateReceived((/* @__PURE__ */ new Date()).toISOString().split("T")[0]);
      setStockInBatchChoice("new");
      const initChecked = {};
      const initPrices = {};
      const initDiff = {};
      if (product.sizes && typeof product.sizes === "object") {
        const sizes = Object.keys(product.sizes);
        setSelectedSizes(sizes);
        sizes.forEach((size) => {
          const sd = product.sizes[size];
          if (typeof sd === "object" && sd?.variants) {
            initDiff[size] = true;
            initPrices[size] = {};
            Object.entries(sd.variants).forEach(([variant, vData]) => {
              initChecked[`${size}|${variant}`] = true;
              const cost = typeof vData === "object" ? vData.costPrice || sd.variantCostPrices?.[variant] ||
sd.costPrice || product.costPrice || 0 : sd.variantCostPrices?.[variant] || sd.costPrice || product.costPrice
|| 0;
              const sell = typeof vData === "object" ? vData.price || sd.variantPrices?.[variant] || sd.price
|| product.itemPrice || 0 : sd.variantPrices?.[variant] || sd.price || product.itemPrice || 0;
              initPrices[size][variant] = { price: sell, costPrice: cost };
            });
          }
        });
      } else {
        setSelectedSizes([]);
      }
      setCheckedCombos(initChecked);
      setStockVariantPrices(initPrices);
      setDiffPricesPerVariant(initDiff);
    }
  }, [isOpen, product?._id, product?.brandName, brandPartnerNames]);
  useEffect(() => {
    if (!isOpen || stockInBrandPartner) return;
    if (brandPartnerNames.length === 0) return;
    setStockInBrandPartner(brandPartnerNames[0]);
  }, [isOpen, stockInBrandPartner, brandPartnerNames]);
  if (!isOpen || !product) return null;
  const handleClose = () => {
    setSelectedSizes([]);
    setSizeQuantities({});
    setVariantQuantities({});
    setQuantity("");
    setBatchCode("");
    setBatchExpirationDate("");
    setStockInBrandPartner("");
    setDateReceived("");
    setStockInBatchChoice("new");
    setReason("Restock");
    setOtherReason("");
    setNewVariantInputs({});
    setAddedVariants([]);
    setNewVariantPrices({});
    setShowNewSizes(false);
    setCustomNewSizeInput("");
    setAddedNewSizes([]);
    setNewSizePrices({});
    setDiffPricesPerVariant({});
    setStockVariantPrices({});
    setCurrentStep(1);
```

```
  setCheckedCombos({});
  setShowAddNewSection(false);
  setNewV1Tags([]);
  setNewV2Tags([]);
  setNewV1Input("");
  setNewV2Input("");
  setAddedNewCombos([]);
  setNewComboData({});
  setFillAllCostSI("");
  setFillAllSellSI("");
  setFillAllQtySI("");
  onClose();
};
const handleAddNewSize = (size) => {
  if (!size || addedNewSizes.includes(size) || existingSizes.includes(size)) return;
  setAddedNewSizes((prev) => [...prev, size]);
  setNewSizePrices((prev) => ({
    ...prev,
    [size]: {
      price: product.itemPrice || 0,
      costPrice: product.costPrice || 0
    }
  }));
};
const handleRemoveNewSize = (size) => {
  setAddedNewSizes((prev) => prev.filter((s) => s !== size));
  setNewSizePrices((prev) => {
    const updated = { ...prev };
    delete updated[size];
    return updated;
  });
  if (selectedSizes.includes(size)) {
    handleSizeToggle(size);
  }
};
const handleNewSizePriceChange = (size, field, value) => {
  setNewSizePrices((prev) => ({
    ...prev,
    [size]: {
      ...prev[size] || { price: product.itemPrice || 0, costPrice: product.costPrice || 0 },
      [field]: value
    }
  }));
};
const handleSizeToggle = (size) => {
  const isSelected = selectedSizes.includes(size);
  if (isSelected) {
    setSelectedSizes((prev) => prev.filter((s) => s !== size));
    setSizeQuantities((prev) => {
      const newQuantities = { ...prev };
      delete newQuantities[size];
      return newQuantities;
    });
    setVariantQuantities((prev) => {
      const newQuantities = { ...prev };
      delete newQuantities[size];
      return newQuantities;
    });
  } else {
    setSelectedSizes((prev) => [...prev, size]);
    setSizeQuantities((prev) => ({
      ...prev,
      [size]: ""
    }));
    if (hasVariants) {
      setVariantQuantities((prev) => ({
        ...prev,
        [size]: {}
      }));
    }
  }
};
const handleSizeQuantityChange = (size, qty) => {
  setSizeQuantities((prev) => ({
    ...prev,
    [size]: qty
  }));
```

```
  };
  const handleVariantQuantityChange = (size, variant, qty) => {
    setVariantQuantities((prev) => ({
      ...prev,
      [size]: {
        ...prev[size] || {},
        [variant]: qty
      }
    }));
  };
  const getVariantCurrentQty = (size, variant) => {
    const variants = getSizeVariants(size);
    if (variants && variants[variant] !== void 0) {
      if (typeof variants[variant] === "number") {
        return variants[variant];
      }
      return variants[variant].quantity || 0;
    }
    return 0;
  };
  const handleNewVariantInputChange = (size, value) => {
    setNewVariantInputs((prev) => ({
      ...prev,
      [size]: value
    }));
  };
  const handleAddNewVariant = (size) => {
    const variantName = (newVariantInputs[size] || "").trim();
    if (!variantName) {
      alert("Please enter a variant name");
      return;
    }
    if (allVariants.includes(variantName)) {
      alert("This variant already exists");
      return;
    }
    setAddedVariants((prev) => [...prev, variantName]);
    setNewVariantPrices((prev) => ({
      ...prev,
      [variantName]: {
        price: product.itemPrice || 0,
        costPrice: product.costPrice || 0
      }
    }));
    setNewVariantInputs((prev) => ({
      ...prev,
      [size]: ""
    }));
  };
  const handleNewVariantPriceChange = (variantName, field, value) => {
    setNewVariantPrices((prev) => ({
      ...prev,
      [variantName]: {
        ...prev[variantName] || { price: product.itemPrice || 0, costPrice: product.costPrice || 0 },
        [field]: value
      }
    }));
  };
  const handleDiffPricesToggle = (size) => {
    const willEnable = !diffPricesPerVariant[size];
    setDiffPricesPerVariant((prev) => ({
      ...prev,
      [size]: willEnable
    }));
    if (willEnable) {
      const sizeVariants = getSizeVariants(size);
      const sizeData = hasSizes && product.sizes[size] && typeof product.sizes[size] === "object" ?
product.sizes[size] : {};
      const initialPrices = {};
      allVariants.forEach((variant) => {
        let variantPrice = product.itemPrice || 0;
        let variantCost = product.costPrice || 0;
        if (sizeVariants && sizeVariants[variant]) {
          const vData = sizeVariants[variant];
          if (typeof vData === "object" && vData !== null) {
            variantPrice = vData.price || sizeData.variantPrices?.[variant] || sizeData.price || variantPrice;
            variantCost = vData.costPrice || sizeData.variantCostPrices?.[variant] || sizeData.costPrice ||
```

```
variantCost;
          } else {
            variantPrice = sizeData.variantPrices?.[variant] || sizeData.price || variantPrice;
            variantCost = sizeData.variantCostPrices?.[variant] || sizeData.costPrice || variantCost;
          }
        }
        initialPrices[variant] = {
          price: variantPrice,
          costPrice: variantCost
        };
      });
      setStockVariantPrices((prev) => ({
        ...prev,
        [size]: {
          ...prev[size] || {},
          ...initialPrices
        }
      }));
    }
  };
  const handleStockVariantPriceChange = (size, variant, field, value) => {
    setStockVariantPrices((prev) => ({
      ...prev,
      [size]: {
        ...prev[size] || {},
        [variant]: {
          ...prev[size]?.[variant] || { price: product.itemPrice || 0, costPrice: product.costPrice || 0 },
          [field]: value
        }
      }
    }));
  };
  const handleComboCheck = (size, variant) => {
    const key = `${size}|${variant}`;
    const wasChecked = checkedCombos[key] !== false;
    setCheckedCombos((prev) => ({ ...prev, [key]: !wasChecked }));
    if (wasChecked) {
      setVariantQuantities((prev) => {
        const updated = { ...prev };
        if (updated[size]) {
          delete updated[size][variant];
        }
        return updated;
      });
    }
  };
  const handleAddNewCombosFromPicker = () => {
    const existingSet = new Set(existingCombos.map((c) => `${c.size}|${c.variant}`));
    const addedSet = new Set(addedNewCombos.map((c) => `${c.size}|${c.variant}`));
    const previewCombos = buildPickerCombos(newV1Tags, newV2Tags, existingSet, addedSet);
    if (previewCombos.length === 0) return;
    setAddedNewCombos((prev) => [...prev, ...previewCombos]);
    const newVars = newV1Tags.filter((v) => !allVariants.includes(v));
    const newSzs = newV2Tags.filter((s) => !existingSizes.includes(s) && !addedNewSizes.includes(s));
    if (newVars.length > 0) setAddedVariants((prev) => [.../* @__PURE__ */ new Set([...prev, ...newVars])]);
    if (newSzs.length > 0) {
      setAddedNewSizes((prev) => [.../* @__PURE__ */ new Set([...prev, ...newSzs])]);
      setSelectedSizes((prev) => [.../* @__PURE__ */ new Set([...prev, ...newSzs])]);
    }
    newV2Tags.filter((s) => existingSizes.includes(s) && !selectedSizes.includes(s)).forEach((s) => {
      setSelectedSizes((prev) => prev.includes(s) ? prev : [...prev, s]);
    });
    if (previewCombos.some((c) => c.size === VARIANT_ONLY_SIZE_KEY)) {
      setSelectedSizes((prev) => prev.includes(VARIANT_ONLY_SIZE_KEY) ? prev : [...prev,
VARIANT_ONLY_SIZE_KEY]);
    }
    const checkedUpdates = {};
    const priceUpdates = {};
    const qtyUpdates = {};
    previewCombos.forEach(({ size, variant }) => {
      const key = `${size}|${variant}`;
      const data = newComboData[key] || {};
      checkedUpdates[key] = true;
      if (!priceUpdates[size]) priceUpdates[size] = {};
      priceUpdates[size][variant] = { price: data.sell || product.itemPrice || 0, costPrice: data.cost ||
product.costPrice || 0 };
      if (data.qty) {
```

```
        if (!qtyUpdates[size]) qtyUpdates[size] = {};
        qtyUpdates[size][variant] = data.qty;
      }
    });
    setCheckedCombos((prev) => ({ ...prev, ...checkedUpdates }));
    setStockVariantPrices((prev) => {
      const u = { ...prev };
      Object.entries(priceUpdates).forEach(([s, vs]) => {
        u[s] = { ...u[s] || {}, ...vs };
      });
      return u;
    });
    if (Object.keys(qtyUpdates).length > 0) {
      setVariantQuantities((prev) => {
        const u = { ...prev };
        Object.entries(qtyUpdates).forEach(([s, vs]) => {
          u[s] = { ...u[s] || {}, ...vs };
        });
        return u;
      });
    }
    setNewV1Tags([]);
    setNewV2Tags([]);
    setNewComboData({});
    setShowAddNewSection(false);
  };
  const handleFillAllSI = () => {
    const allCombosList = [...existingCombos.map((c) => ({ size: c.size, variant:
c.variant })), ...addedNewCombos];
    const qFill = String(fillAllQtySI ?? "").trim();
    const cFill = String(fillAllCostSI ?? "").trim();
    const sFill = String(fillAllSellSI ?? "").trim();
    if (cFill !== "" || sFill !== "") {
      setStockVariantPrices((prev) => {
        const u = { ...prev };
        allCombosList.forEach(({ size, variant }) => {
          if (checkedCombos[`${size}|${variant}`] !== false) {
            if (!u[size]) u[size] = {};
            u[size][variant] = {
              ...u[size]?.[variant] || {},
              ...cFill !== "" ? { costPrice: cFill } : {},
              ...sFill !== "" ? { price: sFill } : {}
            };
          }
        });
        return u;
      });
    }
    if (qFill !== "") {
      setVariantQuantities((prev) => {
        const u = { ...prev };
        allCombosList.forEach(({ size, variant }) => {
          if (checkedCombos[`${size}|${variant}`] !== false) {
            if (!u[size]) u[size] = {};
            u[size][variant] = qFill;
          }
        });
        return u;
      });
    }
  };
  const isStepValid = (step) => {
    if (step === 1) {
      if (!hasSizes) {
        if (hasVariants) {
          const allCombosList = [...existingCombos.map((c) => ({ size: c.size, variant:
c.variant })), ...addedNewCombos];
          return allCombosList.some(({ size, variant }) => {
            return checkedCombos[`${size}|${variant}`] !== false &&
(parseInt(variantQuantities[size]?.[variant]) || 0) > 0;
          });
        }
        return (parseInt(quantity) || 0) > 0;
      }
      if (hasVariants) {
        const allCombosList = [...existingCombos.map((c) => ({ size: c.size, variant:
c.variant })), ...addedNewCombos];
```

```
        return allCombosList.some(({ size, variant }) => {
          return checkedCombos[`${size}|${variant}`] !== false &&
(parseInt(variantQuantities[size]?.[variant]) || 0) > 0;
        });
      }
      return selectedSizes.some((s) => (parseInt(sizeQuantities[s]) || 0) > 0);
    }
    if (step === 2) {
      if (!stockInBrandPartner) return false;
      if (!dateReceived) return false;
      if (reason === "Other" && !otherReason.trim()) return false;
      return true;
    }
    return true;
  };
  const handleSubmit = () => {
    if (currentStep !== 3) return;
    if (reason === "Other" && !otherReason.trim()) {
      alert("Please specify the reason");
      return;
    }
    const finalReason = reason === "Other" ? `Other: ${otherReason.trim()}` : reason;
    const stockInBatchPayload = !hasSizes ? {
      ...batchCode.trim() ? { batchCode: batchCode.trim() } : {},
      ...batchExpirationDate ? { expirationDate: batchExpirationDate } : {}
    } : stockInBatchChoice !== "new" ? {
      targetBatchSlotIndex: parseInt(stockInBatchChoice, 10),
      ...batchExpirationDate ? { expirationDate: batchExpirationDate } : {}
    } : {
      ...batchCode.trim() ? { batchCode: batchCode.trim() } : {},
      ...batchExpirationDate ? { expirationDate: batchExpirationDate } : {}
    };
    if (!hasSizes && !hasVariants) {
      const qty = parseInt(quantity) || 0;
      if (qty <= 0) {
        alert("Please enter a valid quantity");
        return;
      }
      onConfirm({
        quantity: qty,
        noSizes: true,
        reason: finalReason,
        ...stockInBatchPayload,
        ...stockInBrandPartner ? { brandPartner: stockInBrandPartner } : {},
        ...dateReceived ? { dateReceived } : {}
      });
      return;
    }
    if (!hasVariants && selectedSizes.length === 0) {
      alert("Please select at least one size");
      return;
    }
    if (hasVariants) {
      const filteredVarQtys = {};
      const checkedSizesSet = /* @__PURE__ */ new Set();
      [...existingCombos.map((c) => ({ size: c.size, variant:
c.variant })), ...addedNewCombos].forEach(({ size, variant }) => {
        const key = `${size}|${variant}`;
        const qty = parseInt(variantQuantities[size]?.[variant]) || 0;
        if (checkedCombos[key] !== false && qty > 0) {
          if (!filteredVarQtys[size]) filteredVarQtys[size] = {};
          filteredVarQtys[size][variant] = variantQuantities[size][variant];
          checkedSizesSet.add(size);
        }
      });
      if (Object.keys(filteredVarQtys).length === 0) {
        alert("Please enter quantities for at least one checked variant");
        return;
      }
      const filteredSizes = Array.from(checkedSizesSet);
      onConfirm({
        sizes: sizeQuantities,
        variantQuantities: filteredVarQtys,
        selectedSizes: filteredSizes,
        reason: finalReason,
        hasVariants: true,
        newVariantPrices: addedVariants.length > 0 ? newVariantPrices : null,
```

```
        newSizePrices: addedNewSizes.length > 0 ? newSizePrices : null,
        diffPricesPerVariant: Object.keys(diffPricesPerVariant).some((k) => diffPricesPerVariant[k]) ?
diffPricesPerVariant : null,
        stockVariantPrices: Object.keys(stockVariantPrices).length > 0 ? stockVariantPrices : null,
        ...stockInBatchPayload,
        ...stockInBrandPartner ? { brandPartner: stockInBrandPartner } : {},
        ...dateReceived ? { dateReceived } : {}
      });
      return;
    }
    const hasValidQuantities = selectedSizes.some((size) => {
      const qty = sizeQuantities[size] || 0;
      return qty > 0;
    });
    if (!hasValidQuantities) {
      alert("Please enter quantities for at least one selected size");
      return;
    }
    onConfirm({
      sizes: sizeQuantities,
      selectedSizes,
      reason: finalReason,
      newSizePrices: addedNewSizes.length > 0 ? newSizePrices : null,
      ...stockInBatchPayload,
      ...stockInBrandPartner ? { brandPartner: stockInBrandPartner } : {},
      ...dateReceived ? { dateReceived } : {}
    });
  };
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-9999 p-4 backdrop-blur-sm pointer-events-none" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-2xl w-full max-w-3xl relative pointer-events-auto overflow-hidden ${theme ===
"dark" ? "bg-[#1E1B18]" : "bg-white"}`,
      style: {
        boxShadow: "0 25px 50px -12px rgba(0, 0, 0, 0.5), 0 0 0 1px rgba(0, 0, 0, 0.1)"
      }
    },
    /* @__PURE__ */ React.createElement("div", { className: "flex justify-between items-center px-6 py-4" },
/* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement("div", { className: "relative w-6 h-6 flex items-center justify-center" }, /* @__PURE__
*/ React.createElement(
      "svg",
      {
        className: "w-6 h-6",
        fill: "none",
        stroke: "currentColor",
        viewBox: "0 0 24 24"
      },
      /* @__PURE__ */ React.createElement(
        "path",
        {
          strokeLinecap: "round",
          strokeLinejoin: "round",
          strokeWidth: 2,
          d: "M20 7l-8-4-8 4m16 0l-8 4m8-4v10l-8 4m0-10L4 7m8 4v10M4 7v10l8 4"
        }
      )
    ), /* @__PURE__ */ React.createElement("span", { className: "absolute -top-0.5 -right-0.5 w-3 h-3 bg-
green-500 rounded-full flex items-center justify-center border-2 border-white" }, /* @__PURE__ */
React.createElement("span", { className: "text-white text-[8px] font-bold leading-none" }, "+"))), /*
@__PURE__ */ React.createElement(
      "h2",
      {
        className: `text-xl font-bold ${theme === "dark" ? "text-white" : "text-gray-900"}`
      },
      product?.itemName || "Stock In"
    )), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: handleClose,
        className: `text-2xl font-light ${theme === "dark" ? "text-gray-400 hover:text-gray-200" : "text-
gray-400 hover:text-gray-600"}`
      },
      "\xD7"
    )),
    /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("div", { className:
```

```
"flex", style: { maxHeight: "calc(100vh - 150px)" } }, /* @__PURE__ */ React.createElement("div", { className:
"w-full p-6 flex flex-col justify-between overflow-y-auto", style: { maxHeight: "calc(100vh - 150px)" } }, /*
@__PURE__ */ React.createElement("div", { className: "space-y-6" }, /* @__PURE__ */ React.createElement("div",
{ className: "mb-2" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-
center gap-0" }, [{ id: 1, label: "Add Items" }, { id: 2, label: "Details" }, { id: 3, label:
"Review" }].map((s, idx) => /* @__PURE__ */ React.createElement("div", { key: s.id, className: "flex items-
center" }, /* @__PURE__ */ React.createElement("div", { className: "flex flex-col items-center" }, /*
@__PURE__ */ React.createElement("div", { className: `w-8 h-8 rounded-full flex items-center justify-center
text-xs font-bold border-2 ${currentStep === s.id ? "bg-[#09A046] border-[#09A046] text-white" :
currentStep > s.id ? "bg-[#09A046] border-[#09A046] text-white" : theme === "dark" ? "border-gray-600 text-
gray-400" : "border-gray-300 text-gray-400"}` }, currentStep > s.id ? "\u2713" : s.id), /* @__PURE__ */
React.createElement("span", { className: `text-[10px] mt-1 ${currentStep >= s.id ? "text-[#09A046] font-
semibold" : "text-gray-400"}` }, s.label)), idx < 2 && /* @__PURE__ */ React.createElement("div", { className:
`w-14 h-[2px] mx-2 mb-4 ${currentStep > s.id ? "bg-[#09A046]" : theme === "dark" ? "bg-gray-700" : "bg-gray-
200"}`, style: { borderStyle: "dashed" } })))), currentStep === 1 && /* @__PURE__ */
React.createElement("div", { className: "space-y-3" }, /* @__PURE__ */ React.createElement("div", null, /*
@__PURE__ */ React.createElement("h4", { className: `text-sm font-bold uppercase tracking-wide ${theme ===
"dark" ? "text-white" : "text-gray-900"}` }, "Items to Stock In"), /* @__PURE__ */ React.createElement("p",
{ className: `text-xs ${theme === "dark" ? "text-gray-500" : "text-gray-400"}` }, "check variants that
arrived, unchecked = skip")), shouldShowVariantBuilder ? /* @__PURE__ */ React.createElement(React.Fragment,
null, /* @__PURE__ */ React.createElement("p", { className: `text-xs ${theme === "dark" ? "text-gray-400" :
"text-gray-500"}` }, existingCombos.length > 0 ? "Existing variants" : "No existing variants yet"), /*
@__PURE__ */ React.createElement("div", { className: `rounded-xl border overflow-hidden ${theme === "dark" ?
"border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */ React.createElement("div", { className: `grid
grid-cols-[28px_1fr_1fr_48px_68px_88px_88px] gap-1 items-center px-3 py-2 text-[10px] font-bold uppercase
tracking-wider ${theme === "dark" ? "bg-[#2A2724] text-gray-400 border-b border-gray-700" : "bg-gray-50 text-
gray-500 border-b border-gray-200"}` }, /* @__PURE__ */ React.createElement("span", null), /* @__PURE__ */
React.createElement("span", null, "V1"), /* @__PURE__ */ React.createElement("span", null, "V2"), /*
@__PURE__ */ React.createElement("span", null, "Stock"), /* @__PURE__ */ React.createElement("span", null,
"Qty In"), /* @__PURE__ */ React.createElement("span", null, "Cost"), /* @__PURE__ */
React.createElement("span", null, "Sell Price")), /* @__PURE__ */ React.createElement("div", { className:
"max-h-72 overflow-y-auto" }, existingCombos.map(({ size, variant, stock }) => {
      const key = `${size}|${variant}`;
      const checked = checkedCombos[key] !== false;
      return /* @__PURE__ */ React.createElement("div", { key, className: `grid grid-cols-
[28px_1fr_1fr_48px_68px_88px_88px] gap-1 items-center px-3 py-2 ${theme === "dark" ? "border-t border-gray-
700" : "border-t border-gray-100"}` }, /* @__PURE__ */ React.createElement("input", { type: "checkbox",
checked, onChange: () => handleComboCheck(size, variant), className: "w-4 h-4 rounded cursor-pointer", style:
{ accentColor: "#09A046" } }), /* @__PURE__ */ React.createElement("span", { className: `inline-block w-fit
px-2 py-0.5 text-[11px] rounded-full font-medium ${theme === "dark" ? "bg-pink-500/15 text-pink-400" : "bg-
pink-100 text-pink-700"}` }, variant), /* @__PURE__ */ React.createElement("span", { className: `inline-block
w-fit px-2 py-0.5 text-[11px] rounded-full font-medium min-w-[1.25rem] ${theme === "dark" ? "bg-[#09A046]/15
text-[#09A046]" : "bg-[#09A046]/10 text-[#09A046]"}` }, size === VARIANT_ONLY_SIZE_KEY ? "\u2014" : size), /*
@__PURE__ */ React.createElement("span", { className: "text-xs font-semibold text-[#09A046]" }, stock), /*
@__PURE__ */ React.createElement(
        "input",
        {
          type: "number",
          min: "0",
          placeholder: "qty",
          disabled: !checked,
          value: variantQuantities[size]?.[variant] || "",
          onChange: (e) => handleVariantQuantityChange(size, variant, e.target.value),
          className: `w-full px-2 py-1.5 text-xs border rounded-lg focus:outline-none focus:ring-1
focus:ring-[#09A046] ${!checked ? "opacity-40" : ""} ${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-
white" : "bg-white border-gray-300"}`
        }
      ), /* @__PURE__ */ React.createElement("div", { className: "flex items-center" }, /* @__PURE__ */
React.createElement(
        "input",
        {
          type: "number",
          min: "0",
          step: "0.01",
          disabled: !checked,
          value: stockVariantPrices[size]?.[variant]?.costPrice ?? "",
          onChange: (e) => handleStockVariantPriceChange(size, variant, "costPrice", e.target.value),
          className: `w-full px-2 py-1.5 text-xs border rounded-lg focus:outline-none focus:ring-1
focus:ring-[#09A046] ${!checked ? "opacity-40" : ""} ${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-
white" : "bg-white border-gray-300"}`
        }
      ), /* @__PURE__ */ React.createElement("span", { className: "text-[10px] text-gray-400 ml-0.5" },
"\u20B1")), /* @__PURE__ */ React.createElement("div", { className: "flex items-center" }, /* @__PURE__ */
React.createElement(
        "input",
        {
          type: "number",
```

```
          min: "0",
          step: "0.01",
          disabled: !checked,
          value: stockVariantPrices[size]?.[variant]?.price ?? "",
          onChange: (e) => handleStockVariantPriceChange(size, variant, "price", e.target.value),
          className: `w-full px-2 py-1.5 text-xs border rounded-lg focus:outline-none focus:ring-1
focus:ring-[#09A046] ${!checked ? "opacity-40" : ""} ${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-
white" : "bg-white border-gray-300"}`
        }
      ), /* @__PURE__ */ React.createElement("span", { className: "text-[10px] text-gray-400 ml-0.5" },
"\u20B1")));
    }), addedNewCombos.length > 0 && /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__
*/ React.createElement("div", { className: `px-3 py-1.5 ${theme === "dark" ? "border-t border-gray-700" :
"border-t border-gray-200"}` }, /* @__PURE__ */ React.createElement("span", { className: "text-xs font-bold
text-[#09A046]" }, "NEW")), addedNewCombos.map(({ size, variant }) => {
      const key = `${size}|${variant}`;
      const checked = checkedCombos[key] !== false;
      return /* @__PURE__ */ React.createElement("div", { key: `new-${key}`, className: `grid grid-cols-
[28px_1fr_1fr_48px_68px_88px_88px] gap-1 items-center px-3 py-2 ${theme === "dark" ? "border-t border-gray-
700" : "border-t border-gray-100"}` }, /* @__PURE__ */ React.createElement("input", { type: "checkbox",
checked, onChange: () => handleComboCheck(size, variant), className: "w-4 h-4 rounded cursor-pointer", style:
{ accentColor: "#09A046" } }), /* @__PURE__ */ React.createElement("span", { className: `inline-block w-fit
px-2 py-0.5 text-[11px] rounded-full font-medium ${theme === "dark" ? "bg-pink-500/15 text-pink-400" : "bg-
pink-100 text-pink-700"}` }, variant), /* @__PURE__ */ React.createElement("span", { className: `inline-block
w-fit px-2 py-0.5 text-[11px] rounded-full font-medium min-w-[1.25rem] ${theme === "dark" ? "bg-[#09A046]/15
text-[#09A046]" : "bg-[#09A046]/10 text-[#09A046]"}` }, size === VARIANT_ONLY_SIZE_KEY ? "\u2014" : size), /*
@__PURE__ */ React.createElement("span", { className: "text-xs text-gray-400" }, "\u2014"), /* @__PURE__ */
React.createElement(
        "input",
        {
          type: "number",
          min: "0",
          placeholder: "qty",
          disabled: !checked,
          value: variantQuantities[size]?.[variant] || "",
          onChange: (e) => handleVariantQuantityChange(size, variant, e.target.value),
          className: `w-full px-2 py-1.5 text-xs border rounded-lg focus:outline-none focus:ring-1
focus:ring-[#09A046] ${!checked ? "opacity-40" : ""} ${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-
white" : "bg-white border-gray-300"}`
        }
      ), /* @__PURE__ */ React.createElement("div", { className: "flex items-center" }, /* @__PURE__ */
React.createElement(
        "input",
        {
          type: "number",
          min: "0",
          step: "0.01",
          disabled: !checked,
          value: stockVariantPrices[size]?.[variant]?.costPrice ?? "",
          onChange: (e) => handleStockVariantPriceChange(size, variant, "costPrice", e.target.value),
          className: `w-full px-2 py-1.5 text-xs border rounded-lg focus:outline-none focus:ring-1
focus:ring-[#09A046] ${!checked ? "opacity-40" : ""} ${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-
white" : "bg-white border-gray-300"}`
        }
      ), /* @__PURE__ */ React.createElement("span", { className: "text-[10px] text-gray-400 ml-0.5" },
"\u20B1")), /* @__PURE__ */ React.createElement("div", { className: "flex items-center" }, /* @__PURE__ */
React.createElement(
        "input",
        {
          type: "number",
          min: "0",
          step: "0.01",
          disabled: !checked,
          value: stockVariantPrices[size]?.[variant]?.price ?? "",
          onChange: (e) => handleStockVariantPriceChange(size, variant, "price", e.target.value),
          className: `w-full px-2 py-1.5 text-xs border rounded-lg focus:outline-none focus:ring-1
focus:ring-[#09A046] ${!checked ? "opacity-40" : ""} ${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-
white" : "bg-white border-gray-300"}`
        }
      ), /* @__PURE__ */ React.createElement("span", { className: "text-[10px] text-gray-400 ml-0.5" },
"\u20B1")));
    }))), /* @__PURE__ */ React.createElement("div", { className: `px-3 py-2 border-t ${theme === "dark" ?
"border-gray-700 bg-[#1E1B18]/80" : "border-gray-200 bg-gray-50/80"}` }, /* @__PURE__ */
React.createElement("div", { className: "grid grid-cols-[28px_1fr_1fr_48px_68px_88px_88px] gap-1 items-
center" }, /* @__PURE__ */ React.createElement("span", { className: "text-[9px] font-semibold uppercase text-
[#09A046] col-span-4 text-right pr-1" }, "Fill"), /* @__PURE__ */ React.createElement(
      "input",
```

```
      {
        type: "number",
        min: "0",
        value: fillAllQtySI,
        onChange: (e) => setFillAllQtySI(e.target.value),
        placeholder: "Qty",
        className: `w-full px-2 py-1.5 text-xs border rounded-lg focus:outline-none focus:ring-1 focus:ring-
[#09A046] ${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-300"}`
      }
    ), /* @__PURE__ */ React.createElement("div", { className: "flex items-center" }, /* @__PURE__ */
React.createElement(
      "input",
      {
        type: "number",
        min: "0",
        step: "0.01",
        value: fillAllCostSI,
        onChange: (e) => setFillAllCostSI(e.target.value),
        placeholder: "Cost",
        className: `w-full px-2 py-1.5 text-xs border rounded-lg focus:outline-none focus:ring-1 focus:ring-
[#09A046] ${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-300"}`
      }
    ), /* @__PURE__ */ React.createElement("span", { className: "text-[10px] text-gray-400 ml-0.5 shrink-0" },
"\u20B1")), /* @__PURE__ */ React.createElement("div", { className: "flex items-center" }, /* @__PURE__ */
React.createElement(
      "input",
      {
        type: "number",
        min: "0",
        step: "0.01",
        value: fillAllSellSI,
        onChange: (e) => setFillAllSellSI(e.target.value),
        placeholder: "Sell",
        className: `w-full px-2 py-1.5 text-xs border rounded-lg focus:outline-none focus:ring-1 focus:ring-
[#09A046] ${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-300"}`
      }
    ), /* @__PURE__ */ React.createElement("span", { className: "text-[10px] text-gray-400 ml-0.5 shrink-0" },
"\u20B1"))), /* @__PURE__ */ React.createElement("div", { className: "flex justify-end mt-1.5" }, /*
@__PURE__ */ React.createElement("button", { type: "button", onClick: handleFillAllSI, className: "text-xs
font-semibold text-[#09A046] hover:underline" }, "Fill All")))), /* @__PURE__ */ React.createElement("div",
{ className: `border-2 border-dashed rounded-xl overflow-hidden ${theme === "dark" ? "border-[#09A046]/30 bg-
[#09A046]/5" : "border-[#09A046]/40 bg-[#09A046]/5"}` }, /* @__PURE__ */ React.createElement("button", { type:
"button", onClick: () => setShowAddNewSection(!showAddNewSection), className: "w-full flex items-center
justify-between px-4 py-2.5" }, /* @__PURE__ */ React.createElement("span", { className: "text-sm font-
semibold text-[#09A046]" }, "+ add NEW Variants"), /* @__PURE__ */ React.createElement("span", { className:
"text-xs text-[#09A046]" }, showAddNewSection ? "collapse" : "expand")), showAddNewSection && /* @__PURE__ */
React.createElement("div", { className: `px-4 pb-4 space-y-4 border-t ${theme === "dark" ? "border-
[#09A046]/20" : "border-[#09A046]/20"}` }, /* @__PURE__ */ React.createElement("div", { className: "grid
grid-cols-2 gap-4 pt-3" }, /* @__PURE__ */ React.createElement("div", { className: `p-3 rounded-lg border
${theme === "dark" ? "border-gray-700 bg-[#1E1B18]" : "border-gray-200 bg-white"}` }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-2 mb-2" }, /* @__PURE__ */
React.createElement("p", { className: `text-xs font-bold ${theme === "dark" ? "text-white" : "text-gray-
800"}` }, "VARIANT 1 \u2013 Colors"), /* @__PURE__ */ React.createElement("span", { title: "Press Enter to
add", className: `inline-flex items-center justify-center w-4 h-4 rounded-full text-[10px] font-bold border
${theme === "dark" ? "text-gray-300 border-gray-600" : "text-gray-600 border-gray-300"}` }, "i")), /*
@__PURE__ */ React.createElement(
      "select",
      {
        value: "",
        onChange: (e) => {
          if (e.target.value && !newV1Tags.includes(e.target.value)) setNewV1Tags((prev) => [...prev,
e.target.value]);
        },
        className: `w-full px-2 py-1.5 text-xs border rounded-lg mb-2 appearance-none cursor-pointer ${theme
=== "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-300"}`
      },
      /* @__PURE__ */ React.createElement("option", { value: "" }, "Select color..."),
      ["White", "Black", "Red", "Blue", "Green", "Yellow", "Pink", "Purple", "Orange", "Brown", "Gray",
"Beige", "Navy", "Maroon", "Cream", "Teal"].filter((c) => !newV1Tags.includes(c)).map((c) => /* @__PURE__ */
React.createElement("option", { key: c, value: c }, c))
    ), /* @__PURE__ */ React.createElement("div", { className: "flex flex-wrap gap-1 items-center" },
newV1Tags.map((tag) => /* @__PURE__ */ React.createElement("span", { key: tag, className: "inline-flex items-
center gap-1 px-2 py-0.5 text-[11px] rounded-full bg-pink-100 text-pink-700 font-medium" }, tag, " ", /*
@__PURE__ */ React.createElement("button", { type: "button", onClick: () => setNewV1Tags((prev) =>
prev.filter((t) => t !== tag)), className: "hover:text-pink-900" }, "\xD7"))), /* @__PURE__ */
React.createElement(
      "input",
```

```
      {
        type: "text",
        value: newV1Input,
        onChange: (e) => setNewV1Input(e.target.value),
        onKeyDown: (e) => {
          if (e.key === "Enter") {
            e.preventDefault();
            const v = newV1Input.trim();
            if (v && !newV1Tags.includes(v)) {
              setNewV1Tags((prev) => [...prev, v]);
              setNewV1Input("");
            }
          }
        },
        placeholder: "Add +",
        className: `w-16 px-1 py-0.5 text-[11px] border-b border-dashed outline-none ${theme === "dark" ?
"bg-transparent border-gray-600 text-white placeholder-gray-500" : "bg-transparent border-gray-400
placeholder-gray-400"}`
      }
    ))), /* @__PURE__ */ React.createElement("div", { className: `p-3 rounded-lg border ${theme === "dark" ?
"border-gray-700 bg-[#1E1B18]" : "border-gray-200 bg-white"}` }, /* @__PURE__ */ React.createElement("div",
{ className: "flex items-center gap-2 mb-2" }, /* @__PURE__ */ React.createElement("p", { className: `text-xs
font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}` }, "VARIANT 2 \u2013 Size ", /* @__PURE__ */
React.createElement("span", { className: "font-normal text-gray-400" }, "(optional)")), /* @__PURE__ */
React.createElement("span", { title: "Press Enter to add", className: `inline-flex items-center justify-
center w-4 h-4 rounded-full text-[10px] font-bold border ${theme === "dark" ? "text-gray-300 border-gray-
600" : "text-gray-600 border-gray-300"}` }, "i")), /* @__PURE__ */ React.createElement(
      "select",
      {
        value: "",
        onChange: (e) => {
          if (e.target.value && !newV2Tags.includes(e.target.value)) setNewV2Tags((prev) => [...prev,
e.target.value]);
        },
        className: `w-full px-2 py-1.5 text-xs border rounded-lg mb-2 appearance-none cursor-pointer ${theme
=== "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-300"}`
      },
      /* @__PURE__ */ React.createElement("option", { value: "" }, "Select size..."),
      ["XS", "S", "M", "L", "XL", "XXL", "XXXL", "Free Size", "Small", "Medium", "Large"].filter((s)
=> !newV2Tags.includes(s)).map((s) => /* @__PURE__ */ React.createElement("option", { key: s, value: s }, s))
    ), /* @__PURE__ */ React.createElement("div", { className: "flex flex-wrap gap-1 items-center" },
newV2Tags.map((tag) => /* @__PURE__ */ React.createElement("span", { key: tag, className: "inline-flex items-
center gap-1 px-2 py-0.5 text-[11px] rounded-full bg-[#09A046]/10 text-[#09A046] font-medium" }, tag, " ", /*
@__PURE__ */ React.createElement("button", { type: "button", onClick: () => setNewV2Tags((prev) =>
prev.filter((t) => t !== tag)), className: "hover:text-green-800" }, "\xD7"))), /* @__PURE__ */
React.createElement(
      "input",
      {
        type: "text",
        value: newV2Input,
        onChange: (e) => setNewV2Input(e.target.value),
        onKeyDown: (e) => {
          if (e.key === "Enter") {
            e.preventDefault();
            const v = newV2Input.trim();
            if (v && !newV2Tags.includes(v)) {
              setNewV2Tags((prev) => [...prev, v]);
              setNewV2Input("");
            }
          }
        },
        placeholder: "Add +",
        className: `w-16 px-1 py-0.5 text-[11px] border-b border-dashed outline-none ${theme === "dark" ?
"bg-transparent border-gray-600 text-white placeholder-gray-500" : "bg-transparent border-gray-400
placeholder-gray-400"}`
      }
    )))), (() => {
      const existingSet = new Set(existingCombos.map((c) => `${c.size}|${c.variant}`));
      const addedSet = new Set(addedNewCombos.map((c) => `${c.size}|${c.variant}`));
      const previewCombos = buildPickerCombos(newV1Tags, newV2Tags, existingSet, addedSet);
      if (previewCombos.length === 0) return null;
      return /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement("div", { className: "flex items-center justify-between" }, /* @__PURE__ */
React.createElement("span", { className: `text-xs font-semibold ${theme === "dark" ? "text-gray-400" : "text-
gray-500"}` }, previewCombos.length, " COMBINATIONS GENERATED"), /* @__PURE__ */ React.createElement("span",
{ className: "text-xs text-gray-400 cursor-pointer hover:underline", onClick: () => {
        const qFill = String(fillAllQtySI ?? "").trim();
```

```
        const cFill = String(fillAllCostSI ?? "").trim();
        const sFill = String(fillAllSellSI ?? "").trim();
        const u = { ...newComboData };
        previewCombos.forEach(({ size, variant }) => {
          const k = `${size}|${variant}`;
          u[k] = {
            ...u[k] || {},
            ...cFill !== "" ? { cost: cFill } : {},
            ...sFill !== "" ? { sell: sFill } : {},
            ...qFill !== "" ? { qty: qFill } : {}
          };
        });
        setNewComboData(u);
      } }, "Fill all")), /* @__PURE__ */ React.createElement("div", { className: `rounded-lg border overflow-
hidden ${theme === "dark" ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */
React.createElement("div", { className: `grid grid-cols-[1fr_88px_88px_68px] gap-1 items-center px-3 py-2
text-[10px] font-bold uppercase tracking-wider ${theme === "dark" ? "bg-[#2A2724] text-gray-400 border-b
border-gray-700" : "bg-gray-50 text-gray-500 border-b border-gray-200"}` }, /* @__PURE__ */
React.createElement("span", null, "New Combo"), /* @__PURE__ */ React.createElement("span", null, "Cost Price
\u20B1"), /* @__PURE__ */ React.createElement("span", null, "Selling Price \u20B1"), /* @__PURE__ */
React.createElement("span", null, "Qty In")), previewCombos.map(({ size, variant }) => {
        const k = `${size}|${variant}`;
        return /* @__PURE__ */ React.createElement("div", { key: k, className: `grid grid-cols-
[1fr_88px_88px_68px] gap-1 items-center px-3 py-2 ${theme === "dark" ? "border-t border-gray-700" : "border-t
border-gray-100"}` }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-1" },
size !== VARIANT_ONLY_SIZE_KEY && /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement("span", { className: `px-2 py-0.5 text-[11px] rounded-full font-medium ${theme ===
"dark" ? "bg-[#09A046]/15 text-[#09A046]" : "bg-[#09A046]/10 text-[#09A046]"}` }, size), /* @__PURE__ */
React.createElement("span", { className: `text-xs ${theme === "dark" ? "text-gray-500" : "text-gray-400"}` },
"\xD7")), /* @__PURE__ */ React.createElement("span", { className: `px-2 py-0.5 text-[11px] rounded-full
font-medium ${theme === "dark" ? "bg-pink-500/15 text-pink-400" : "bg-pink-100 text-pink-700"}` }, variant)),
/* @__PURE__ */ React.createElement("div", { className: "flex items-center" }, /* @__PURE__ */
React.createElement(
          "input",
          {
            type: "number",
            min: "0",
            step: "0.01",
            value: newComboData[k]?.cost || "",
            onChange: (e) => setNewComboData((prev) => ({ ...prev, [k]: { ...prev[k] || {}, cost:
e.target.value } })),
            placeholder: "0",
            className: `w-full px-2 py-1.5 text-xs border rounded-lg focus:outline-none focus:ring-1
focus:ring-[#09A046] ${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-
300"}`
          }
        ), /* @__PURE__ */ React.createElement("span", { className: "text-[10px] text-gray-400 ml-0.5" },
"\u20B1")), /* @__PURE__ */ React.createElement("div", { className: "flex items-center" }, /* @__PURE__ */
React.createElement(
          "input",
          {
            type: "number",
            min: "0",
            step: "0.01",
            value: newComboData[k]?.sell || "",
            onChange: (e) => setNewComboData((prev) => ({ ...prev, [k]: { ...prev[k] || {}, sell:
e.target.value } })),
            placeholder: "0",
            className: `w-full px-2 py-1.5 text-xs border rounded-lg focus:outline-none focus:ring-1
focus:ring-[#09A046] ${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-
300"}`
          }
        ), /* @__PURE__ */ React.createElement("span", { className: "text-[10px] text-gray-400 ml-0.5" },
"\u20B1")), /* @__PURE__ */ React.createElement(
          "input",
          {
            type: "number",
            min: "0",
            value: newComboData[k]?.qty || "",
            onChange: (e) => setNewComboData((prev) => ({ ...prev, [k]: { ...prev[k] || {}, qty:
e.target.value } })),
            placeholder: "qty",
            className: `w-full px-2 py-1.5 text-xs border rounded-lg focus:outline-none focus:ring-1
focus:ring-[#09A046] ${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-
300"}`
          }
        ));
```

```
      })), /* @__PURE__ */ React.createElement("button", { type: "button", onClick:
handleAddNewCombosFromPicker, className: "w-full py-2 text-sm font-semibold rounded-lg text-white transition-
all hover:opacity-90", style: { background: "#09A046" } }, "Add +"));
    })()))) : hasSizes ? /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("p", { className: `text-xs mb-2 ${theme === "dark" ? "text-gray-400" : "text-gray-
500"}` }, "Select sizes to add stock to"), /* @__PURE__ */ React.createElement("div", { className: "grid
grid-cols-4 gap-2 mb-3" }, availableSizes.map((size) => {
      const currentQty = hasSizes && product.sizes[size] ? getSizeQuantity(product.sizes[size]) : 0;
      return /* @__PURE__ */ React.createElement("label", { key: size, className: "flex items-center gap-2
cursor-pointer" }, /* @__PURE__ */ React.createElement("input", { type: "checkbox", checked:
selectedSizes.includes(size), onChange: () => handleSizeToggle(size), className: "w-4 h-4 rounded cursor-
pointer", style: { accentColor: "#09A046" } }), /* @__PURE__ */ React.createElement("span", { className:
`text-sm ${theme === "dark" ? "text-gray-200" : "text-gray-900"}` }, size, " ", /* @__PURE__ */
React.createElement("span", { className: "text-xs text-gray-500" }, "(", currentQty, ")")));
    })), selectedSizes.length > 0 && /* @__PURE__ */ React.createElement("div", { className: `space-y-2 p-3
rounded-lg ${theme === "dark" ? "bg-[#2A2724]" : "bg-gray-50"}` }, /* @__PURE__ */
React.createElement("label", { className: `block text-xs font-semibold mb-2 ${theme === "dark" ? "text-gray-
300" : "text-gray-700"}` }, "Quantity per Size:"), /* @__PURE__ */ React.createElement("div", { className:
"grid grid-cols-2 gap-3" }, selectedSizes.map((size) => /* @__PURE__ */ React.createElement("div", { key:
size }, /* @__PURE__ */ React.createElement("label", { className: `block text-xs mb-1 ${theme === "dark" ?
"text-gray-400" : "text-gray-600"}` }, size), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "number",
        min: "0",
        value: sizeQuantities[size] || "",
        onChange: (e) => handleSizeQuantityChange(size, e.target.value),
        placeholder: "Qty",
        className: `w-full px-3 py-2 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#1E1B18] border-gray-700 text-white" : "bg-white
border-gray-300"}`
      }
    )))))) : /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p",
{ className: `text-xs mb-2 ${theme === "dark" ? "text-gray-500" : "text-gray-400"}` }, "Current Stock: ",
product.currentStock || 0), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "number",
        min: "1",
        value: quantity,
        onChange: (e) => setQuantity(e.target.value),
        placeholder: "Enter quantity to add",
        className: `w-full px-4 py-3 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#09A046]
focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white placeholder-gray-500" :
"bg-white border-gray-300"}`
      }
    )), currentStep === 2 && /* @__PURE__ */ React.createElement("div", { className: "space-y-5 pt-2" }, /*
@__PURE__ */ React.createElement("h3", { className: `text-sm font-bold uppercase tracking-wide ${theme ===
"dark" ? "text-white" : "text-gray-900"}` }, "Entry Details"), /* @__PURE__ */ React.createElement("div",
{ className: "grid grid-cols-2 gap-4" }, /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("label", { className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${theme ===
"dark" ? "text-gray-300" : "text-gray-700"}` }, "Brand Partner/Supplier ", /* @__PURE__ */
React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */ React.createElement("div",
{ className: "relative" }, /* @__PURE__ */ React.createElement(
      "select",
      {
        value: stockInBrandPartner,
        onChange: (e) => setStockInBrandPartner(e.target.value),
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent appearance-none cursor-pointer ${theme === "dark" ? "bg-[#2A2724] border-
gray-600 text-white" : "bg-white border-gray-300 text-gray-900"}`
      },
      /* @__PURE__ */ React.createElement("option", { value: "", disabled: true, style: { color:
"#9CA3AF" } }, "Select Brand Partner"),
      !brandPartnerNames.some((name) => String(name).trim().toLowerCase() === "default") ? /* @__PURE__ */
React.createElement("option", { value: "Default" }, "Default") : null,
      brandPartnerNames.map((name) => /* @__PURE__ */ React.createElement("option", { key: name, value:
name }, name))
    ), /* @__PURE__ */ React.createElement("div", { className: "absolute inset-y-0 right-0 flex items-center
pr-3 pointer-events-none" }, /* @__PURE__ */ React.createElement("svg", { className: "w-4 h-4 text-gray-400",
fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path",
{ strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M19 9l-7 7-7-7" }))))), /* @__PURE__
*/ React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: `block text-xs
font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Date
Received ", /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */
React.createElement(
      "input",
```

```
      {
        type: "date",
        value: dateReceived,
        onChange: (e) => setDateReceived(e.target.value),
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white" : "bg-white
border-gray-300 text-gray-900"}`
      }
    ))), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-4" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: `block text-xs
font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Batch
Number"), hasSizes ? /* @__PURE__ */ React.createElement("div", { className: "relative" }, /* @__PURE__ */
React.createElement(
      "select",
      {
        value: stockInBatchChoice,
        onChange: (e) => {
          const v = e.target.value;
          setStockInBatchChoice(v);
          if (v === "new") {
            const now = /* @__PURE__ */ new Date();
            const y = now.getFullYear();
            const mo = String(now.getMonth() + 1).padStart(2, "0");
            const d = String(now.getDate()).padStart(2, "0");
            const h = String(now.getHours()).padStart(2, "0");
            const min = String(now.getMinutes()).padStart(2, "0");
            setBatchCode(`B${y}${mo}${d}-${h}${min}`);
          } else {
            const slot = stockInBatchList.find((b) => String(b.slotIndex) === v);
            setBatchCode(slot?.code || "");
          }
        },
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent appearance-none cursor-pointer ${theme === "dark" ? "bg-[#2A2724] border-
gray-600 text-white" : "bg-white border-gray-300 text-gray-900"}`
      },
      /* @__PURE__ */ React.createElement("option", { value: "new" }, "New batch", stockInBatchChoice ===
"new" && batchCode ? ` (${batchCode})` : ""),
      stockInBatchList.map((b) => /* @__PURE__ */ React.createElement("option", { key: b.slotIndex, value:
String(b.slotIndex) }, "Batch ", b.slotIndex + 1, b.code ? ` (${b.code})` : "", " \xB7 ", b.totalQty, " pcs"))
    ), /* @__PURE__ */ React.createElement("div", { className: "absolute inset-y-0 right-0 flex items-center
pr-3 pointer-events-none" }, /* @__PURE__ */ React.createElement("svg", { className: "w-4 h-4 text-gray-400",
fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path",
{ strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M19 9l-7 7-7-7" }))))) : /* @__PURE__
*/ React.createElement("p", { className: `text-lg font-bold ${theme === "dark" ? "text-white" : "text-gray-
900"}` }, batchCode || "\u2014")), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("label", { className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${theme ===
"dark" ? "text-gray-300" : "text-gray-700"}` }, "Expiring Date (if applicable)"), /* @__PURE__ */
React.createElement(
      "input",
      {
        type: "date",
        value: batchExpirationDate,
        min: todayISO,
        onChange: (e) => setBatchExpirationDate(e.target.value),
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white" : "bg-white
border-gray-300 text-gray-900"}`
      }
    ))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${theme === "dark" ? "text-gray-300" :
"text-gray-700"}` }, "Reason"), /* @__PURE__ */ React.createElement("div", { className: "relative" }, /*
@__PURE__ */ React.createElement(
      "select",
      {
        value: reason,
        onChange: (e) => {
          setReason(e.target.value);
          if (e.target.value !== "Other") setOtherReason("");
        },
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#09A046] focus:border-transparent appearance-none cursor-pointer ${theme === "dark" ? "bg-[#2A2724] border-
gray-600 text-white" : "bg-white border-gray-300 text-gray-900"}`
      },
      reasons.map((r) => /* @__PURE__ */ React.createElement("option", { key: r, value: r }, r))
    ), /* @__PURE__ */ React.createElement("div", { className: "absolute inset-y-0 right-0 flex items-center
pr-3 pointer-events-none" }, /* @__PURE__ */ React.createElement("svg", { className: "w-4 h-4 text-gray-400",
```

```
fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path",
{ strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M19 9l-7 7-7-7" }))))), reason ===
"Other" && /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "text",
        value: otherReason,
        onChange: (e) => setOtherReason(e.target.value),
        placeholder: "Please specify the reason",
        className: `w-full mt-2 px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2
focus:ring-[#09A046] focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white
placeholder-gray-500" : "bg-white border-gray-300 text-gray-900"}`
      }
    ))), currentStep === 3 && (() => {
      const reviewRows = [];
      if (hasVariants) {
        const allCombos = [...existingCombos.map((c) => ({ size: c.size, variant:
c.variant })), ...addedNewCombos];
        allCombos.forEach(({ size, variant }) => {
          const key = `${size}|${variant}`;
          if (checkedCombos[key] === false) return;
          const qty = parseInt(variantQuantities[size]?.[variant]) || 0;
          if (qty <= 0) return;
          const sizeData = product.sizes?.[size];
          const existingStock = typeof sizeData?.variants?.[variant] === "object" ?
sizeData.variants[variant].quantity || 0 : typeof sizeData?.variants?.[variant] === "number" ?
sizeData.variants[variant] : 0;
          const cost = stockVariantPrices[size]?.[variant]?.costPrice || 0;
          const sell = stockVariantPrices[size]?.[variant]?.price || 0;
          reviewRows.push({ variant, size, stock: existingStock, qtyIn: qty, cost, sell });
        });
      } else if (hasSizes) {
        selectedSizes.forEach((size) => {
          const qty = parseInt(sizeQuantities[size]) || 0;
          if (qty <= 0) return;
          const sizeData = product.sizes?.[size];
          const existingStock = sizeData?.quantity || 0;
          reviewRows.push({ variant: null, size, stock: existingStock, qtyIn: qty, cost: 0, sell: 0 });
        });
      } else {
        const qty = parseInt(quantity) || 0;
        if (qty > 0) reviewRows.push({ variant: null, size: null, stock: product.currentStock || 0, qtyIn:
qty, cost: 0, sell: 0 });
      }
      return /* @__PURE__ */ React.createElement("div", { className: "space-y-3" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("h3", { className: `text-sm font-bold
uppercase tracking-wide ${theme === "dark" ? "text-white" : "text-gray-900"}` }, "Review Before Saving"), /*
@__PURE__ */ React.createElement("p", { className: `text-xs mt-0.5 ${theme === "dark" ? "text-gray-500" :
"text-gray-400"}` }, "Double-check everything. Once confirmed, inventory will be updated immediately.")), /*
@__PURE__ */ React.createElement("div", { className: `rounded-xl border overflow-hidden ${theme === "dark" ?
"border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */ React.createElement("div", { className: "overflow-
y-auto", style: { maxHeight: "340px" } }, /* @__PURE__ */ React.createElement("table", { className: "w-full
text-xs" }, /* @__PURE__ */ React.createElement("thead", { className: `sticky top-0 z-10 ${theme === "dark" ?
"bg-[#2A2724]" : "bg-gray-50"}` }, /* @__PURE__ */ React.createElement("tr", { className: `border-b ${theme
=== "dark" ? "border-gray-700" : "border-gray-200"}` }, hasVariants && /* @__PURE__ */
React.createElement("th", { className: `px-3 py-2 text-left font-bold uppercase tracking-wide ${theme ===
"dark" ? "text-gray-400" : "text-gray-500"}` }, "V1"), hasVariants && /* @__PURE__ */
React.createElement("th", { className: `px-3 py-2 text-left font-bold uppercase tracking-wide ${theme ===
"dark" ? "text-gray-400" : "text-gray-500"}` }, "V2"), /* @__PURE__ */ React.createElement("th", { className:
`px-3 py-2 text-center font-bold uppercase tracking-wide ${theme === "dark" ? "text-gray-400" : "text-gray-
500"}` }, "Stock"), /* @__PURE__ */ React.createElement("th", { className: `px-3 py-2 text-center font-bold
uppercase tracking-wide ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Qty In"), /* @__PURE__
*/ React.createElement("th", { className: `px-3 py-2 text-right font-bold uppercase tracking-wide ${theme ===
"dark" ? "text-gray-400" : "text-gray-500"}` }, "Cost"), /* @__PURE__ */ React.createElement("th",
{ className: `px-3 py-2 text-right font-bold uppercase tracking-wide ${theme === "dark" ? "text-gray-400" :
"text-gray-500"}` }, "Sell Price"))), /* @__PURE__ */ React.createElement("tbody", null, reviewRows.map((row,
i) => /* @__PURE__ */ React.createElement("tr", { key: i, className: `border-b last:border-b-0 ${theme ===
"dark" ? "border-gray-700/50" : "border-gray-100"}` }, hasVariants && /* @__PURE__ */
React.createElement("td", { className: "px-3 py-2.5" }, /* @__PURE__ */ React.createElement("span",
{ className: `inline-block px-2.5 py-0.5 text-[11px] rounded-full font-medium ${theme === "dark" ? "bg-pink-
500/15 text-pink-400" : "bg-pink-100 text-pink-700"}` }, row.variant)), hasVariants && /* @__PURE__ */
React.createElement("td", { className: "px-3 py-2.5" }, row.size && row.size !== VARIANT_ONLY_SIZE_KEY ? /*
@__PURE__ */ React.createElement("span", { className: `inline-block px-2.5 py-0.5 text-[11px] rounded-full
font-medium ${theme === "dark" ? "bg-blue-500/15 text-blue-400" : "bg-blue-100 text-blue-700"}` }, row.size) :
/* @__PURE__ */ React.createElement("span", { className: "text-gray-400" }, "\u2014")), /* @__PURE__ */
React.createElement("td", { className: `px-3 py-2.5 text-center ${theme === "dark" ? "text-blue-400" : "text-
blue-600"}` }, row.stock), /* @__PURE__ */ React.createElement("td", { className: "px-3 py-2.5 text-center
```

```
font-semibold text-[#09A046]" }, "+", row.qtyIn), /* @__PURE__ */ React.createElement("td", { className: `px-
3 py-2.5 text-right ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "\u20B1 ",
Number(row.cost).toLocaleString()), /* @__PURE__ */ React.createElement("td", { className: `px-3 py-2.5 text-
right ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "\u20B1 ",
Number(row.sell).toLocaleString()))), reviewRows.length === 0 && /* @__PURE__ */ React.createElement("tr",
null, /* @__PURE__ */ React.createElement("td", { colSpan: hasVariants ? 6 : 4, className: "px-3 py-6 text-
center text-gray-400 italic text-xs" }, "No items to review")))))));
    })(), false), /* @__PURE__ */ React.createElement("div", { className: `flex justify-between items-center
mt-6 pt-4 border-t ${theme === "dark" ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */
React.createElement(
      "button",
      {
        type: "button",
        onClick: currentStep === 1 ? handleClose : () => setCurrentStep((prev) => prev - 1),
        className: `px-8 py-2.5 text-sm font-semibold rounded-xl border-2 transition-colors ${theme ===
"dark" ? "text-gray-300 border-gray-600 hover:bg-gray-700" : "text-gray-600 border-gray-300 hover:bg-gray-
100"}`
      },
      currentStep === 1 ? "Cancel" : "\u2190 Back"
    ), currentStep < 3 ? /* @__PURE__ */ React.createElement(
      "button",
      {
        type: "button",
        disabled: !isStepValid(currentStep),
        onClick: () => {
          if (isStepValid(currentStep)) setCurrentStep((prev) => prev + 1);
        },
        className: "px-8 py-2.5 text-sm font-semibold rounded-xl text-white transition-all shadow-md
hover:opacity-90 disabled:opacity-50 disabled:cursor-not-allowed",
        style: { background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)" }
      },
      "Continue"
    ) : /* @__PURE__ */ React.createElement(
      "button",
      {
        type: "button",
        disabled: loading,
        onClick: handleSubmit,
        className: "px-8 py-2.5 text-sm font-semibold rounded-xl text-white transition-all shadow-md
hover:opacity-90 disabled:opacity-50",
        style: { background: "#09A046" }
      },
      loading ? "Adding..." : "Stock In"
    )))))
  ));
};
export default StockInModal;
```

FILE: pos/frontend/src/components/inventory/StockMovementIcons.jsx
==========

```
import React from "react";
export const StockInIcon = ({ size = 20, className = "" }) => {
  return /* @__PURE__ */ React.createElement(
    "svg",
    {
      width: size,
      height: size,
      viewBox: "0 0 24 24",
      fill: "none",
      xmlns: "http://www.w3.org/2000/svg",
      className
    },
    /* @__PURE__ */ React.createElement(
      "path",
      {
        d: "M8 8L12 6L16 8L12 10L8 8Z",
        fill: "white",
        stroke: "#22C55E",
        strokeWidth: "1.5"
      }
    ),
    /* @__PURE__ */ React.createElement(
      "path",
      {
        d: "M8 8V14L12 16V10L8 8Z",
```

```
        fill: "white",
        stroke: "#22C55E",
        strokeWidth: "1.5"
      }
    ),
    /* @__PURE__ */ React.createElement(
      "path",
      {
        d: "M16 8V14L12 16V10L16 8Z",
        fill: "white",
        stroke: "#22C55E",
        strokeWidth: "1.5"
      }
    ),
    /* @__PURE__ */ React.createElement(
      "circle",
      {
        cx: "19",
        cy: "19",
        r: "3.5",
        fill: "#22C55E"
      }
    ),
    /* @__PURE__ */ React.createElement(
      "path",
      {
        d: "M19 16.5V21.5M16.5 19H21.5",
        stroke: "white",
        strokeWidth: "1.2",
        strokeLinecap: "round"
      }
    )
  );
};
export const StockOutIcon = ({ size = 24, className = "" }) => {
  return /* @__PURE__ */ React.createElement(
    "svg",
    {
      width: size,
      height: size,
      viewBox: "0 0 24 24",
      fill: "none",
      xmlns: "http://www.w3.org/2000/svg",
      className
    },
    /* @__PURE__ */ React.createElement(
      "path",
      {
        d: "M8 8L12 6L16 8L12 10L8 8Z",
        fill: "white",
        stroke: "#4B5563",
        strokeWidth: "1.5"
      }
    ),
    /* @__PURE__ */ React.createElement(
      "path",
      {
        d: "M8 8V14L12 16V10L8 8Z",
        fill: "white",
        stroke: "#4B5563",
        strokeWidth: "1.5"
      }
    ),
    /* @__PURE__ */ React.createElement(
      "path",
      {
        d: "M16 8V14L12 16V10L16 8Z",
        fill: "white",
        stroke: "#4B5563",
        strokeWidth: "1.5"
      }
    ),
    /* @__PURE__ */ React.createElement(
      "circle",
      {
        cx: "19",
        cy: "19",
```

```
        r: "3.5",
        fill: "#EF4444"
      }
    ),
    /* @__PURE__ */ React.createElement(
      "path",
      {
        d: "M16.5 19H21.5",
        stroke: "white",
        strokeWidth: "1.2",
        strokeLinecap: "round"
      }
    )
  );
};
export const PullOutIcon = ({ size = 24, className = "" }) => {
  return /* @__PURE__ */ React.createElement(StockOutIcon, { size, className });
};
```

FILE: pos/frontend/src/components/inventory/StockOutModal.jsx
==========

```
import { useEffect, useState, useMemo } from "react";
import { useTheme } from "../../context/ThemeContext";
const VARIANT_ONLY_SIZE_KEY = "__VARIANT_ONLY__";
const StockOutModal = ({ isOpen, onClose, product, onConfirm, loading }) => {
  const { theme } = useTheme();
  const [selectedBatch, setSelectedBatch] = useState("");
  const [selectedCombos, setSelectedCombos] = useState([]);
  const [comboQuantities, setComboQuantities] = useState({});
  const [quantity, setQuantity] = useState("");
  const [reason, setReason] = useState("");
  const [otherReason, setOtherReason] = useState("");
  const reasons = ["Sold", "Damaged", "Defective", "Returned Item", "Lost", "Expired", "Other"];
  const hasSizes = product?.sizes && typeof product.sizes === "object" && Object.keys(product.sizes).length >
0;
  const hasVariants = useMemo(() => {
    if (!hasSizes) return false;
    return Object.values(product.sizes).some((sd) => typeof sd === "object" && sd?.variants &&
Object.keys(sd.variants).length > 0);
  }, [product, hasSizes]);
  const batchList = useMemo(() => {
    if (!hasSizes || !hasVariants) return [];
    const getVariantQty = (vData) => {
      if (typeof vData === "number") return parseInt(vData, 10) || 0;
      if (vData && typeof vData === "object") {
        const qty = vData.qty ?? vData.quantity ?? 0;
        return parseInt(qty, 10) || 0;
      }
      return 0;
    };
    const maxBatchDepth = (() => {
      let max = 0;
      Object.entries(product.sizes).forEach(([_, sd]) => {
        if (!sd || typeof sd !== "object" || !sd?.variants) return;
        Object.values(sd.variants).forEach((vData) => {
          const batches = typeof vData === "object" && Array.isArray(vData.batches) ? vData.batches : [];
          max = Math.max(max, batches.length);
        });
      });
      return max;
    })();
    if (maxBatchDepth <= 0) {
      const openingBatchCode = product?.batchNumber || product?.openingBatchNumber || "B1";
      const combos = {};
      Object.entries(product.sizes).forEach(([size, sd]) => {
        if (!sd || typeof sd !== "object" || !sd?.variants) return;
        Object.entries(sd.variants).forEach(([variant, vData]) => {
          const qty = getVariantQty(vData);
          if (qty > 0) combos[`${size}|${variant}`] = { size, variant, qty };
        });
      });
      const comboArr = Object.values(combos);
      const totalQty = comboArr.reduce((sum, c) => sum + (c.qty || 0), 0);
      if (totalQty <= 0) return [];
      return [{ slotIndex: 0, code: openingBatchCode, totalQty, combos }];
```

```
    }
    const slots = Array.from({ length: maxBatchDepth }, (_, slotIndex) => ({
      slotIndex,
      code: "",
      totalQty: 0,
      combos: {}
    }));
    Object.entries(product.sizes).forEach(([size, sd]) => {
      if (!sd || typeof sd !== "object" || !sd?.variants) return;
      Object.entries(sd.variants).forEach(([variant, vData]) => {
        const batches = typeof vData === "object" && Array.isArray(vData.batches) ? vData.batches : [];
        const fallbackQty = (slotIndexForFallback) => slotIndexForFallback === 0 ? getVariantQty(vData) : 0;
        for (let slotIndex = 0; slotIndex < maxBatchDepth; slotIndex++) {
          const b = batches[slotIndex];
          const qty = b ? parseInt(b.qty, 10) || 0 : fallbackQty(slotIndex);
          if (qty <= 0) continue;
          const slot = slots[slotIndex];
          const key = `${size}|${variant}`;
          if (!slot.combos[key]) slot.combos[key] = { size, variant, qty: 0 };
          slot.combos[key].qty += qty;
          slot.totalQty += qty;
          if (!slot.code) {
            slot.code = b?.batchCode || (slotIndex === 0 ? product?.batchNumber ||
product?.openingBatchNumber || "B1" : "");
          }
        }
      });
    });
    return slots.filter((s) => s.totalQty > 0);
  }, [product, hasSizes, hasVariants]);
  const batchCombos = useMemo(() => {
    if (!selectedBatch) return [];
    const slot = batchList.find((b) => String(b.slotIndex) === selectedBatch);
    if (!slot) return [];
    return Object.values(slot.combos).filter((c) => c.qty > 0);
  }, [selectedBatch, batchList]);
  useEffect(() => {
    if (isOpen && product) {
      setSelectedBatch("");
      setSelectedCombos([]);
      setComboQuantities({});
      setQuantity("");
      setReason("");
      setOtherReason("");
    }
  }, [isOpen, product?._id]);
  if (!isOpen || !product) return null;
  const handleClose = () => {
    setSelectedBatch("");
    setSelectedCombos([]);
    setComboQuantities({});
    setQuantity("");
    setReason("");
    setOtherReason("");
    onClose();
  };
  const comboKey = (size, variant) => `${size}|${variant}`;
  const toggleCombo = (size, variant) => {
    const key = comboKey(size, variant);
    const isSelected = selectedCombos.includes(key);
    if (isSelected) {
      setSelectedCombos((prev) => prev.filter((k) => k !== key));
      setComboQuantities((prev) => {
        const n = { ...prev };
        delete n[key];
        return n;
      });
    } else {
      setSelectedCombos((prev) => [...prev, key]);
    }
  };
  const handleQtyChange = (key, val) => {
    setComboQuantities((prev) => ({ ...prev, [key]: parseInt(val) || 0 }));
  };
  const isValid = () => {
    if (!reason) return false;
    if (reason === "Other" && !otherReason.trim()) return false;
```

```
    if (!hasSizes) return (parseInt(quantity) || 0) > 0;
    if (hasVariants) {
      if (!selectedBatch) return false;
      return selectedCombos.some((key) => (comboQuantities[key] || 0) > 0);
    }
    return (parseInt(quantity) || 0) > 0;
  };
  const handleSubmit = () => {
    const finalReason = reason === "Other" ? `Other: ${otherReason.trim()}` : reason;
    if (!hasSizes || !hasVariants) {
      const qty = parseInt(quantity) || 0;
      if (qty > (product.currentStock || 0)) {
        alert(`Cannot remove more than available stock (${product.currentStock || 0})`);
        return;
      }
      onConfirm({ quantity: qty, noSizes: true, reason: finalReason });
      return;
    }
    const variantQuantities = {};
    const selectedSizes = /* @__PURE__ */ new Set();
    const overLimitItems = [];
    selectedCombos.forEach((key) => {
      const qty = comboQuantities[key] || 0;
      if (qty <= 0) return;
      const [size, variant] = key.split("|");
      const combo = batchCombos.find((c) => c.size === size && c.variant === variant);
      if (combo && qty > combo.qty) {
        overLimitItems.push(`${variant}${size !== VARIANT_ONLY_SIZE_KEY ? ` \xD7 ${size}` : ""} (max:
${combo.qty})`);
      }
      if (!variantQuantities[size]) variantQuantities[size] = {};
      variantQuantities[size][variant] = qty;
      selectedSizes.add(size);
    });
    if (overLimitItems.length > 0) {
      alert(`Cannot remove more than available stock for:
${overLimitItems.join("\n")}`);
      return;
    }
    onConfirm({
      sizes: {},
      variantQuantities,
      selectedSizes: Array.from(selectedSizes),
      reason: finalReason,
      hasVariants: true
    });
  };
  const isDark = theme === "dark";
  const inputCls = `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
red-400 focus:border-transparent appearance-none cursor-pointer ${isDark ? "bg-[#2A2724] border-gray-600
text-white" : "bg-white border-gray-300 text-gray-900"}`;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-9999 p-4 backdrop-blur-sm pointer-events-none" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-2xl w-full max-w-lg relative pointer-events-auto overflow-hidden ${isDark ? "bg-
[#1E1B18]" : "bg-white"}`,
      style: { boxShadow: "0 25px 50px -12px rgba(0,0,0,0.5), 0 0 0 1px rgba(0,0,0,0.1)" }
    },
    /* @__PURE__ */ React.createElement("div", { className: "flex justify-between items-center px-6 py-4" },
/* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement("div", { className: "relative w-6 h-6 flex items-center justify-center" }, /* @__PURE__
*/ React.createElement("svg", { className: "w-6 h-6", fill: "none", stroke: "currentColor", viewBox: "0 0 24
24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round",
strokeWidth: 2, d: "M20 7l-8-4-8 4m16 0l-8 4m8-4v10l-8 4m0-10L4 7m8 4v10M4 7v10l8 4" })), /* @__PURE__ */
React.createElement("span", { className: "absolute -top-0.5 -right-0.5 w-3 h-3 bg-red-500 rounded-full flex
items-center justify-center border-2 border-white" }, /* @__PURE__ */ React.createElement("span", { className:
"text-white text-[8px] font-bold leading-none" }, "\u2212"))), /* @__PURE__ */ React.createElement("h2",
{ className: `text-xl font-bold ${isDark ? "text-white" : "text-gray-900"}` }, "Stock Out")), /* @__PURE__ */
React.createElement("button", { onClick: handleClose, className: `text-2xl font-light ${isDark ? "text-gray-
400 hover:text-gray-200" : "text-gray-400 hover:text-gray-600"}` }, "\xD7")),
    /* @__PURE__ */ React.createElement("div", { className: "px-6 pb-4" }, /* @__PURE__ */
React.createElement("div", { className: "space-y-5" }, /* @__PURE__ */ React.createElement("div", { className:
"grid grid-cols-2 gap-4" }, /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("label", { className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${isDark ?
"text-gray-300" : "text-gray-700"}` }, "Product Name ", /* @__PURE__ */ React.createElement("span",
{ className: "text-red-500" }, "*")), /* @__PURE__ */ React.createElement("p", { className: `text-lg font-
```

```
bold ${isDark ? "text-white" : "text-gray-900"}` }, product.itemName)), hasVariants && batchList.length > 0
&& /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className:
`block text-xs font-bold uppercase tracking-wide mb-1.5 ${isDark ? "text-gray-300" : "text-gray-700"}` },
"Batch Number ", /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /*
@__PURE__ */ React.createElement("div", { className: "relative" }, /* @__PURE__ */ React.createElement(
      "select",
      {
        value: selectedBatch,
        onChange: (e) => {
          setSelectedBatch(e.target.value);
          setSelectedCombos([]);
          setComboQuantities({});
        },
        className: inputCls
      },
      /* @__PURE__ */ React.createElement("option", { value: "", disabled: true, style: { color:
"#9CA3AF" } }, "Select Batch"),
      batchList.map((b) => /* @__PURE__ */ React.createElement("option", { key: b.slotIndex, value:
String(b.slotIndex) }, "Batch ", b.slotIndex + 1, b.code ? ` (${b.code})` : "", " (", b.totalQty, ")"))
    ), /* @__PURE__ */ React.createElement("div", { className: "absolute inset-y-0 right-0 flex items-center
pr-3 pointer-events-none" }, /* @__PURE__ */ React.createElement("svg", { className: "w-4 h-4 text-gray-400",
fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path",
{ strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M19 9l-7 7-7-7" }))))))), hasVariants
&& selectedBatch && batchCombos.length > 0 && /* @__PURE__ */ React.createElement("div", { className:
`rounded-xl border p-4 ${isDark ? "border-gray-700 bg-[#2A2724]" : "border-gray-200 bg-gray-50"}` }, /*
@__PURE__ */ React.createElement("p", { className: `text-xs font-bold uppercase tracking-wide mb-3 ${isDark ?
"text-gray-400" : "text-gray-500"}` }, "Select Variants to Remove"), /* @__PURE__ */
React.createElement("div", { className: "flex flex-wrap gap-2" }, batchCombos.map((c) => {
      const key = comboKey(c.size, c.variant);
      const isSel = selectedCombos.includes(key);
      const label = c.size !== VARIANT_ONLY_SIZE_KEY ? `${c.variant} x ${c.size}` : c.variant;
      return /* @__PURE__ */ React.createElement(
        "button",
        {
          key,
          type: "button",
          onClick: () => toggleCombo(c.size, c.variant),
          className: `px-3 py-1.5 text-xs font-medium rounded-full border transition-all ${isSel ? "bg-red-50
border-red-400 text-red-700" : isDark ? "border-gray-600 text-gray-300 hover:border-gray-400" : "border-gray-
300 text-gray-600 hover:border-gray-400"}`
        },
        label
      );
    }))), hasVariants && selectedCombos.length > 0 && /* @__PURE__ */ React.createElement("div", { className:
`rounded-xl border overflow-hidden ${isDark ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */
React.createElement("div", { className: "overflow-y-auto", style: { maxHeight: "220px" } }, /* @__PURE__ */
React.createElement("table", { className: "w-full text-xs" }, /* @__PURE__ */ React.createElement("thead",
{ className: `sticky top-0 z-10 ${isDark ? "bg-[#2A2724]" : "bg-gray-50"}` }, /* @__PURE__ */
React.createElement("tr", { className: `border-b ${isDark ? "border-gray-700" : "border-gray-200"}` }, /*
@__PURE__ */ React.createElement("th", { className: `px-4 py-2 text-left font-bold uppercase tracking-wide
${isDark ? "text-gray-400" : "text-gray-500"}` }, "Variant"), /* @__PURE__ */ React.createElement("th",
{ className: `px-4 py-2 text-center font-bold uppercase tracking-wide ${isDark ? "text-gray-400" : "text-
gray-500"}` }, "Stock"), /* @__PURE__ */ React.createElement("th", { className: `px-4 py-2 text-center font-
bold uppercase tracking-wide ${isDark ? "text-gray-400" : "text-gray-500"}` }, "Qty Out"))), /* @__PURE__ */
React.createElement("tbody", null, selectedCombos.map((key) => {
      const combo = batchCombos.find((c) => comboKey(c.size, c.variant) === key);
      if (!combo) return null;
      const [size, variant] = key.split("|");
      const maxQty = combo.qty;
      const val = comboQuantities[key] || "";
      return /* @__PURE__ */ React.createElement("tr", { key, className: `border-b last:border-b-0 ${isDark ?
"border-gray-700/50" : "border-gray-100"}` }, /* @__PURE__ */ React.createElement("td", { className: "px-4
py-2.5" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-1.5" }, size !==
VARIANT_ONLY_SIZE_KEY && /* @__PURE__ */ React.createElement("span", { className: `inline-block px-2.5 py-0.5
text-[11px] rounded-full font-medium border border-dashed ${isDark ? "border-blue-400/40 text-blue-400" :
"border-blue-300 text-blue-700"}` }, size), size !== VARIANT_ONLY_SIZE_KEY && /* @__PURE__ */
React.createElement("span", { className: `text-[10px] ${isDark ? "text-gray-500" : "text-gray-400"}` },
"\xD7"), /* @__PURE__ */ React.createElement("span", { className: `inline-block px-2.5 py-0.5 text-[11px]
rounded-full font-medium border border-dashed ${isDark ? "border-pink-400/40 text-pink-400" : "border-pink-
300 text-pink-700"}` }, variant))), /* @__PURE__ */ React.createElement("td", { className: `px-4 py-2.5 text-
center font-medium ${isDark ? "text-gray-300" : "text-gray-600"}` }, maxQty), /* @__PURE__ */
React.createElement("td", { className: "px-4 py-2.5 text-center" }, /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "number",
          min: "0",
          max: maxQty,
```

```
          value: val,
          onChange: (e) => handleQtyChange(key, e.target.value),
          placeholder: "0",
          className: `w-20 mx-auto px-2 py-1.5 text-sm text-center border rounded-lg focus:outline-none
focus:ring-2 focus:ring-red-400 ${isDark ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" :
"bg-white border-gray-300 text-gray-900 placeholder-gray-400"}`
        }
      )));
    ))))), !hasVariants && /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("label", { className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${isDark ?
"text-gray-300" : "text-gray-700"}` }, "Quantity to Remove"), /* @__PURE__ */ React.createElement("p",
{ className: `text-xs mb-2 ${isDark ? "text-gray-500" : "text-gray-400"}` }, "Current Stock: ",
product.currentStock || 0), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "number",
        min: "1",
        max: product.currentStock || 0,
        value: quantity,
        onChange: (e) => setQuantity(e.target.value),
        placeholder: "Enter quantity",
        className: `w-full px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
red-400 ${isDark ? "bg-[#2A2724] border-gray-600 text-white placeholder-gray-500" : "bg-white border-gray-300
text-gray-900"}`
      }
    )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `block text-xs font-bold uppercase tracking-wide mb-1.5 ${isDark ? "text-gray-300" : "text-gray-
700"}` }, "Reason ", /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /*
@__PURE__ */ React.createElement("div", { className: "relative" }, /* @__PURE__ */ React.createElement(
      "select",
      {
        value: reason,
        onChange: (e) => {
          setReason(e.target.value);
          if (e.target.value !== "Other") setOtherReason("");
        },
        className: inputCls
      },
      /* @__PURE__ */ React.createElement("option", { value: "", disabled: true, style: { color:
"#9CA3AF" } }, "EG. Damaged"),
      reasons.map((r) => /* @__PURE__ */ React.createElement("option", { key: r, value: r }, r))
    ), /* @__PURE__ */ React.createElement("div", { className: "absolute inset-y-0 right-0 flex items-center
pr-3 pointer-events-none" }, /* @__PURE__ */ React.createElement("svg", { className: "w-4 h-4 text-gray-400",
fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path",
{ strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M19 9l-7 7-7-7" }))))), reason ===
"Other" && /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "text",
        value: otherReason,
        onChange: (e) => setOtherReason(e.target.value),
        placeholder: "Please specify the reason",
        className: `w-full mt-2 px-3 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2
focus:ring-red-400 ${isDark ? "bg-[#2A2724] border-gray-600 text-white placeholder-gray-500" : "bg-white
border-gray-300 text-gray-900"}`
      }
    )))),
    /* @__PURE__ */ React.createElement("div", { className: `flex justify-between items-center px-6 py-4
border-t ${isDark ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */ React.createElement(
      "button",
      {
        type: "button",
        onClick: handleClose,
        className: `px-8 py-2.5 text-sm font-semibold rounded-xl border-2 transition-colors ${isDark ? "text-
gray-300 border-gray-600 hover:bg-gray-700" : "text-gray-600 border-gray-300 hover:bg-gray-100"}`
      },
      "Cancel"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        type: "button",
        disabled: loading || !isValid(),
        onClick: handleSubmit,
        className: "px-8 py-2.5 text-sm font-semibold rounded-xl text-white transition-all shadow-md
hover:opacity-90 disabled:opacity-50 disabled:cursor-not-allowed bg-red-600"
      },
      loading ? "Removing..." : "Remove"
```

```
    ))
  ));
};
export default StockOutModal;
```

FILE: pos/frontend/src/components/inventory/SuccessModal.jsx
==========

```
import React from "react";
import { FaTimes, FaCheck } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const SuccessModal = ({ isOpen, onClose, message = "The item was added successfully!" }) => {
  const { theme } = useTheme();
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10001] p-4 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-2xl w-full max-w-md relative shadow-2xl ${theme === "dark" ? "bg-[#1E1B18]" : "bg-
white"}`,
      style: { boxShadow: "0 25px 50px -12px rgba(0, 0, 0, 0.5)" }
    },
    /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: `absolute top-4 right-4 transition-colors z-10 ${theme === "dark" ? "text-gray-400
hover:text-gray-200" : "text-gray-400 hover:text-gray-600"}`
      },
      /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
    ),
    /* @__PURE__ */ React.createElement("div", { className: "p-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-center mb-6" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "w-20 h-20 rounded-full flex items-center justify-center relative",
        style: {
          backgroundColor: "#D4EDDA",
          boxShadow: "0 4px 12px rgba(40, 167, 69, 0.3)"
        }
      },
      /* @__PURE__ */ React.createElement(
        FaCheck,
        {
          className: "text-4xl text-green-600",
          style: {
            animation: "checkmarkAnimation 0.6s ease-out"
          }
        }
      ),
      /* @__PURE__ */ React.createElement("style", null, `
                @keyframes checkmarkAnimation {
                  0% {
                    opacity: 0;
                    transform: scale(0) rotate(-45deg);
                  }
                  50% {
                    opacity: 1;
                    transform: scale(1.2) rotate(5deg);
                  }
                  100% {
                    opacity: 1;
                    transform: scale(1) rotate(0deg);
                  }
                }
              `)
    )), /* @__PURE__ */ React.createElement("h3", { className: `text-2xl font-bold text-center mb-3 ${theme
=== "dark" ? "text-white" : "text-gray-800"}` }, "Success!"), /* @__PURE__ */ React.createElement("p",
{ className: `text-sm text-center mb-8 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, message),
/* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: "w-full py-3 px-6 rounded-lg font-bold text-white transition-all shadow-md hover:shadow-
lg",
        style: {
```

```
          background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)"
        }
      },
      "OK"
    ))
  ));
};
export default SuccessModal;
```

FILE: pos/frontend/src/components/inventory/ViewProductModal.jsx
==========

```
import { useEffect, useMemo, useState } from "react";
import { useTheme } from "../../context/ThemeContext";
import { generateDynamicSku } from "../../utils/skuUtils";
import filterIcon from "../../assets/filter.svg";
const VARIANT_ONLY_SIZE_KEY = "__VARIANT_ONLY__";
const ViewProductModal = ({
  showViewModal,
  setShowViewModal,
  viewingProduct,
  formatDate
}) => {
  const { theme } = useTheme();
  const [batchTab, setBatchTab] = useState("totals");
  const [stockViewOpen, setStockViewOpen] = useState(false);
  const [stockViewQuery, setStockViewQuery] = useState("");
  const [batchSortOrder, setBatchSortOrder] = useState("oldest");
  const [productImgIdx, setProductImgIdx] = useState(0);
  const toNum = (v) => {
    const n = typeof v === "number" ? v : parseInt(v);
    return Number.isFinite(n) ? n : 0;
  };
  const hasRealBatchHistory = (batches) => {
    const list = Array.isArray(batches) ? batches : [];
    if (list.length === 0) return false;
    return list.some((b) => {
      if (!b) return false;
      if (toNum(b.qty) > 0) return true;
      if (b.batchSlotPadding) return false;
      return true;
    });
  };
  const shouldShowStockRow = (lineQty, batches) => {
    if (typeof batchTab === "number") {
      const b = Array.isArray(batches) ? batches[batchTab] : null;
      if (!b) return false;
      if (b.batchSlotPadding && toNum(b.qty) === 0) return false;
      return true;
    }
    if (toNum(lineQty) > 0) return true;
    return hasRealBatchHistory(batches);
  };
  const formatMoneyRange = (nums, fallbackStr) => {
    const arr = (nums || []).filter((n) => typeof n === "number" && Number.isFinite(n));
    if (arr.length === 0) return fallbackStr;
    const lo = Math.min(...arr);
    const hi = Math.max(...arr);
    return lo !== hi ? `\u20B1${lo.toFixed(2)} - \u20B1${hi.toFixed(2)}` : `\u20B1${lo.toFixed(2)}`;
  };
  const expirationAlertThresholdDays = Math.max(
    0,
    Number(viewingProduct?.expirationThresholdDays ?? 30)
  );
  const showExpirationColumns = Boolean(viewingProduct?.expirationDate);
  const getDaysUntilExpiration = (dateInput) => {
    if (!dateInput) return null;
    const d = new Date(String(dateInput).slice(0, 10));
    if (Number.isNaN(d.getTime())) return null;
    const now = /* @__PURE__ */ new Date();
    return Math.ceil((d - now) / (1e3 * 60 * 60 * 24));
  };
  const getFallbackExpirationDate = () => {
    const fallbackHasStock = toNum(viewingProduct?.currentStock) > 0;
    if (viewingProduct?.expirationDate && fallbackHasStock) {
      return String(viewingProduct.expirationDate).slice(0, 10);
```

```
    }
    return null;
  };
  const getSingleBatchExpirationDate = (batch) => {
    if (batch?.expirationDate) return String(batch.expirationDate).slice(0, 10);
    return getFallbackExpirationDate();
  };
  const getAggregateExpirationDate = (batches) => {
    const active = (Array.isArray(batches) ? batches : []).filter((b) => toNum(b?.qty) > 0);
    if (active.length === 0) return getFallbackExpirationDate();
    const withExp = active.filter((b) => b?.expirationDate);
    const withoutExp = active.filter((b) => !b?.expirationDate);
    if (withExp.length > 0 && withoutExp.length > 0) return null;
    if (withExp.length === 0) return getFallbackExpirationDate();
    const dates = [...new Set(withExp.map((b) => String(b.expirationDate).slice(0, 10)))].sort();
    return dates[0] || null;
  };
  const renderDaysLeftCell = (dateInput) => {
    const daysUntil = getDaysUntilExpiration(dateInput);
    if (daysUntil === null) {
      return /* @__PURE__ */ React.createElement("span", { className: `text-xs ${theme === "dark" ? "text-
gray-500" : "text-gray-400"}` }, "\u2014");
    }
    if (daysUntil < 0) {
      return /* @__PURE__ */ React.createElement("span", { className: "text-xs font-semibold text-red-500" },
Math.abs(daysUntil), " day(s) overdue");
    }
    return /* @__PURE__ */ React.createElement("span", { className: `text-xs font-semibold ${theme ===
"dark" ? "text-gray-300" : "text-gray-700"}` }, daysUntil, " day(s)");
  };
  const renderExpirationDateWithStatus = (dateInput) => {
    if (!dateInput) {
      return /* @__PURE__ */ React.createElement("span", { className: `text-xs ${theme === "dark" ? "text-
gray-500" : "text-gray-400"}` }, "\u2014");
    }
    const dateStr = String(dateInput).slice(0, 10);
    const now = /* @__PURE__ */ new Date();
    const expDate = new Date(dateStr);
    const daysUntil = Math.ceil((expDate - now) / (1e3 * 60 * 60 * 24));
    const colorClass = daysUntil < 0 ? "text-red-500 font-semibold" : daysUntil <=
expirationAlertThresholdDays ? "text-yellow-500 font-semibold" : theme === "dark" ? "text-gray-300" : "text-
gray-600";
    return /* @__PURE__ */ React.createElement("div", { className: "space-y-0.5" }, /* @__PURE__ */
React.createElement("div", { className: `text-xs ${colorClass}` }, dateStr), daysUntil < 0 && /* @__PURE__ */
React.createElement("div", { className: "text-[10px] text-red-400" }, "Expired"), daysUntil >= 0 && daysUntil
<= expirationAlertThresholdDays && /* @__PURE__ */ React.createElement("div", { className: "text-[10px] text-
yellow-400" }, "Expiring soon"));
  };
  const getBatchList = (data) => {
    if (!data || typeof data !== "object") return [];
    return Array.isArray(data.batches) ? data.batches : [];
  };
  const renderAggregateExpiration = (batches) => {
    const resolvedDate = getAggregateExpirationDate(batches);
    if (!resolvedDate) {
      return /* @__PURE__ */ React.createElement("span", { className: `text-xs ${theme === "dark" ? "text-
gray-500" : "text-gray-400"}` }, "\u2014");
    }
    const active = (Array.isArray(batches) ? batches : []).filter((b) => toNum(b?.qty) > 0);
    const withExp = active.filter((b) => b?.expirationDate);
    const nearest = resolvedDate;
    const now = /* @__PURE__ */ new Date();
    const expDate = new Date(nearest);
    const daysUntil = Math.ceil((expDate - now) / (1e3 * 60 * 60 * 24));
    const colorClass = daysUntil < 0 ? "text-red-500 font-semibold" : daysUntil <=
expirationAlertThresholdDays ? "text-yellow-500 font-semibold" : theme === "dark" ? "text-gray-300" : "text-
gray-600";
    return /* @__PURE__ */ React.createElement("div", { className: "space-y-0.5" }, /* @__PURE__ */
React.createElement("div", { className: `text-xs ${colorClass}` }, nearest), withExp.length > 1 && /*
@__PURE__ */ React.createElement("div", { className: `text-[10px] ${theme === "dark" ? "text-gray-500" :
"text-gray-500"}` }, "Multiple expiration dates"), withExp.length <= 1 && daysUntil < 0 && /* @__PURE__ */
React.createElement("div", { className: "text-[10px] text-red-400" }, "Expired"), withExp.length <= 1 &&
daysUntil >= 0 && daysUntil <= expirationAlertThresholdDays && /* @__PURE__ */ React.createElement("div",
{ className: "text-[10px] text-yellow-400" }, "Expiring soon"));
  };
  const renderBatchSlotCell = (batch, slotIndex, expirationInOwnColumn = false) => {
    if (!batch) {
```

```
      return /* @__PURE__ */ React.createElement("span", { className: `text-xs ${theme === "dark" ? "text-
gray-500" : "text-gray-400"}` }, "\u2014");
    }
    if (batch.batchSlotPadding && toNum(batch.qty) === 0) {
      return /* @__PURE__ */ React.createElement("span", { className: `text-xs ${theme === "dark" ? "text-
gray-500" : "text-gray-400"}` }, "\u2014");
    }
    const qty = toNum(batch.qty);
    const isZero = qty === 0;
    const pillClass = slotIndex === 0 ? isZero ? "bg-red-100 text-red-700" : "bg-green-100 text-green-700" :
slotIndex === 1 ? isZero ? "bg-red-100 text-red-700" : "bg-blue-100 text-blue-700" : isZero ? "bg-red-100
text-red-700" : theme === "dark" ? "bg-violet-900/50 text-violet-200" : "bg-violet-100 text-violet-900";
    const showLotOrInlineExp = batch.batchCode || !expirationInOwnColumn && batch.expirationDate;
    return /* @__PURE__ */ React.createElement("div", { className: "space-y-0.5" }, /* @__PURE__ */
React.createElement("span", { className: `inline-flex items-center px-2 py-0.5 rounded-full text-xs font-
medium ${pillClass}` }, qty, " ", viewingProduct?.unitOfMeasure || "pcs"), /* @__PURE__ */
React.createElement("div", { className: `text-[11px] ${theme === "dark" ? "text-gray-500" : "text-gray-
500"}` }, "Sell: \u20B1", (batch.price ?? 0).toFixed ? batch.price.toFixed(2) : Number(batch.price ||
0).toFixed(2), " \xB7 Buy: \u20B1", (batch.costPrice ?? 0).toFixed ? batch.costPrice.toFixed(2) :
Number(batch.costPrice || 0).toFixed(2), showLotOrInlineExp && /* @__PURE__ */ React.createElement("span",
{ className: "ml-2" }, batch.batchCode ? `\xB7 Lot: ${batch.batchCode}` : "", !expirationInOwnColumn &&
batch.expirationDate ? ` \xB7 Exp: ${String(batch.expirationDate).slice(0, 10)}` : "")));
  };
  const renderSingleBatchExpirationCell = (batch) => {
    const resolvedDate = getSingleBatchExpirationDate(batch);
    if (!resolvedDate) {
      return /* @__PURE__ */ React.createElement("span", { className: `text-xs ${theme === "dark" ? "text-
gray-500" : "text-gray-400"}` }, "\u2014");
    }
    return renderExpirationDateWithStatus(resolvedDate);
  };
  const renderSingleBatchDateAddedCell = (batch) => {
    if (!batch || !batch.createdAt) {
      return /* @__PURE__ */ React.createElement("span", { className: `text-xs ${theme === "dark" ? "text-
gray-500" : "text-gray-400"}` }, "\u2014");
    }
    const dateStr = String(batch.createdAt).slice(0, 10);
    return /* @__PURE__ */ React.createElement("span", { className: `text-xs ${theme === "dark" ? "text-gray-
300" : "text-gray-600"}` }, dateStr);
  };
  const renderBatchNumberCell = (value, kind = "neutral") => {
    const n = Number(value || 0);
    const colorClass = kind === "stock" ? n === 0 ? "bg-red-100 text-red-700" : "bg-green-100 text-green-
700" : theme === "dark" ? "bg-[#2A2724] text-gray-200" : "bg-gray-100 text-gray-700";
    return /* @__PURE__ */ React.createElement("span", { className: `inline-flex items-center px-2 py-0.5
rounded-full text-xs font-medium ${colorClass}` }, n, " ", viewingProduct?.unitOfMeasure || "pcs");
  };
  const maxBatchDepth = useMemo(() => {
    if (!viewingProduct?.sizes || typeof viewingProduct.sizes !== "object") return 0;
    let max = 0;
    Object.values(viewingProduct.sizes).forEach((sizeData) => {
      if (!sizeData || typeof sizeData !== "object") return;
      if (sizeData.variants && typeof sizeData.variants === "object") {
        Object.values(sizeData.variants).forEach((variantData) => {
          if (variantData && typeof variantData === "object" && Array.isArray(variantData.batches)) {
            max = Math.max(max, variantData.batches.length);
          }
        });
      }
      if (Array.isArray(sizeData.batches)) {
        max = Math.max(max, sizeData.batches.length);
      }
    });
    return max;
  }, [viewingProduct]);
  const selectedBatchSlotTotal = useMemo(() => {
    if (!viewingProduct?.sizes || typeof viewingProduct.sizes !== "object") return null;
    if (batchTab === "totals" || typeof batchTab !== "number") return null;
    const slot = batchTab;
    let sum = 0;
    Object.values(viewingProduct.sizes).forEach((sizeData) => {
      if (!sizeData || typeof sizeData !== "object") return;
      if (sizeData.variants && typeof sizeData.variants === "object") {
        Object.values(sizeData.variants).forEach((variantData) => {
          if (variantData && typeof variantData === "object" && Array.isArray(variantData.batches)) {
            const b = variantData.batches[slot];
            if (b) sum += toNum(b.qty);
```

```
          }
        });
        return;
      }
      if (Array.isArray(sizeData.batches)) {
        const b = sizeData.batches[slot];
        if (b) sum += toNum(b.qty);
      }
    });
    return sum;
  }, [viewingProduct, batchTab]);
  const batchSlotLots = useMemo(() => {
    if (!viewingProduct?.sizes || typeof viewingProduct.sizes !== "object") return [];
    const perSlot = Array.from({ length: maxBatchDepth }, () => /* @__PURE__ */ new Set());
    Object.values(viewingProduct.sizes).forEach((sizeData) => {
      if (!sizeData || typeof sizeData !== "object") return;
      if (sizeData.variants && typeof sizeData.variants === "object") {
        Object.values(sizeData.variants).forEach((variantData) => {
          if (!variantData || typeof variantData !== "object" || !Array.isArray(variantData.batches)) return;
          variantData.batches.forEach((batch, slotIndex) => {
            if (!batch || batch.batchSlotPadding) return;
            const lot = String(batch.batchCode || "").trim();
            if (lot && perSlot[slotIndex]) perSlot[slotIndex].add(lot);
          });
        });
        return;
      }
      if (Array.isArray(sizeData.batches)) {
        sizeData.batches.forEach((batch, slotIndex) => {
          if (!batch || batch.batchSlotPadding) return;
          const lot = String(batch.batchCode || "").trim();
          if (lot && perSlot[slotIndex]) perSlot[slotIndex].add(lot);
        });
      }
    });
    return perSlot.map((slotSet) => [...slotSet].sort());
  }, [viewingProduct, maxBatchDepth]);
  const getBatchLabel = (slotIndex) => {
    const lots = batchSlotLots[slotIndex] || [];
    const lotLabel = lots.length === 1 ? ` (${lots[0]})` : lots.length > 1 ? " (multiple lots)" : "";
    if (slotIndex === 0) {
      return `Batch 1${lotLabel}`;
    }
    return `Batch ${slotIndex + 1}${lotLabel}`;
  };
  const stockViewOptions = useMemo(() => {
    const options = [{ value: "totals", label: "Totals" }];
    const batchSlots = Array.from({ length: maxBatchDepth }, (_, i) => i);
    const sortedBatchSlots = batchSortOrder === "latest" ? [...batchSlots].reverse() : batchSlots;
    sortedBatchSlots.forEach((i) => {
      options.push({ value: i, label: getBatchLabel(i) });
    });
    return options;
  }, [maxBatchDepth, batchSlotLots, batchSortOrder]);
  const filteredStockViewOptions = useMemo(() => {
    const q = stockViewQuery.trim().toLowerCase();
    if (!q) return stockViewOptions;
    return stockViewOptions.filter((opt) => opt.label.toLowerCase().includes(q));
  }, [stockViewOptions, stockViewQuery]);
  const selectedStockViewLabel = batchTab === "totals" ? "Totals" : getBatchLabel(batchTab);
  const productImages = Array.isArray(viewingProduct?.productImages) && viewingProduct.productImages.length >
0 ? viewingProduct.productImages : viewingProduct?.itemImage ? [viewingProduct.itemImage] : [];
  const selectedBatchInsights = useMemo(() => {
    if (!viewingProduct?.sizes || typeof viewingProduct.sizes !== "object") return null;
    if (batchTab === "totals" || typeof batchTab !== "number") return null;
    const slot = batchTab;
    const sellPrices = [];
    const costs = [];
    const variantsWithStock = [];
    const variantsWithSlot = [];
    const lotCodes = /* @__PURE__ */ new Set();
    const exps = /* @__PURE__ */ new Set();
    Object.entries(viewingProduct.sizes).forEach(([size, sizeData]) => {
      if (!sizeData || typeof sizeData !== "object") return;
      if (sizeData.variants && typeof sizeData.variants === "object") {
        Object.entries(sizeData.variants).forEach(([vName, v]) => {
          if (!v || typeof v !== "object") return;
```

```
          const batches2 = Array.isArray(v.batches) ? v.batches : [];
          const b2 = batches2[slot];
          if (!b2) return;
          const label = size === VARIANT_ONLY_SIZE_KEY ? vName : `${vName} / ${size}`;
          variantsWithSlot.push(label);
          if (toNum(b2.qty) > 0) variantsWithStock.push(label);
          const p2 = Number(b2.price);
          const c2 = Number(b2.costPrice);
          if (Number.isFinite(p2)) sellPrices.push(p2);
          if (Number.isFinite(c2)) costs.push(c2);
          if (b2.batchCode) lotCodes.add(String(b2.batchCode).trim());
          if (b2.expirationDate) exps.add(String(b2.expirationDate).slice(0, 10));
        });
        return;
      }
      const batches = Array.isArray(sizeData.batches) ? sizeData.batches : [];
      const b = batches[slot];
      if (!b) return;
      variantsWithSlot.push(size);
      if (toNum(b.qty) > 0) variantsWithStock.push(size);
      const p = Number(b.price);
      const c = Number(b.costPrice);
      if (Number.isFinite(p)) sellPrices.push(p);
      if (Number.isFinite(c)) costs.push(c);
      if (b.batchCode) lotCodes.add(String(b.batchCode).trim());
      if (b.expirationDate) exps.add(String(b.expirationDate).slice(0, 10));
    });
    const uniq = (list) => [...new Set(list)];
    return {
      sellPrices,
      costs,
      variantsWithStock: uniq(variantsWithStock),
      variantsWithSlot: uniq(variantsWithSlot),
      lotCodes: [...lotCodes],
      exps: [...exps].sort()
    };
  }, [viewingProduct, batchTab]);
  useEffect(() => {
    setBatchTab("totals");
    setStockViewOpen(false);
    setStockViewQuery("");
    setBatchSortOrder("oldest");
    setProductImgIdx(0);
  }, [viewingProduct?._id]);
  if (!showViewModal || !viewingProduct) return null;
  const totalStock = viewingProduct.sizes && typeof viewingProduct.sizes === "object" &&
Object.keys(viewingProduct.sizes).length > 0 ? Object.values(viewingProduct.sizes).reduce((sum, sizeData) =>
{
    if (typeof sizeData === "object" && sizeData !== null && sizeData.variants && typeof sizeData.variants
=== "object") {
      const vSum = Object.values(sizeData.variants).reduce((s, v) => {
        if (typeof v === "number") return s + (parseInt(v, 10) || 0);
        if (v && typeof v === "object" && v.quantity !== void 0) return s + (parseInt(v.quantity, 10) || 0);
        return s;
      }, 0);
      if (vSum > 0) return sum + vSum;
    }
    const qty = typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0 ?
sizeData.quantity : typeof sizeData === "number" ? sizeData : 0;
    return sum + qty;
  }, 0) : viewingProduct.currentStock || 0;
  const showPerBatchColumn = maxBatchDepth > 0 && typeof batchTab === "number";
  const tableFooterTotal = selectedBatchSlotTotal !== null ? selectedBatchSlotTotal : totalStock;
  const tableFooterLabel = selectedBatchSlotTotal !== null ? "Stock in this batch:" : "Total Options Stock:";
  const collectAggregateSellPrices = () => {
    const prices = [];
    if (viewingProduct.sizes && typeof viewingProduct.sizes === "object") {
      Object.values(viewingProduct.sizes).forEach((sizeData) => {
        if (typeof sizeData === "object" && sizeData !== null) {
          if (sizeData.variants && typeof sizeData.variants === "object") {
            Object.entries(sizeData.variants).forEach(([vName, v]) => {
              const vPrice = v?.price ?? sizeData.variantPrices?.[vName] ?? sizeData.price;
              if (vPrice !== void 0 && vPrice !== null) prices.push(Number(vPrice));
            });
          } else if (sizeData.variantPrices && typeof sizeData.variantPrices === "object") {
            Object.values(sizeData.variantPrices).forEach((p) => {
              if (p !== void 0 && p !== null) prices.push(Number(p));
```

```
            });
          } else if (sizeData.price !== void 0 && sizeData.price !== null) {
            prices.push(Number(sizeData.price));
          }
        }
      });
    }
    return prices;
  };
  const collectAggregateCosts = () => {
    const costs = [];
    if (viewingProduct.sizes && typeof viewingProduct.sizes === "object") {
      Object.values(viewingProduct.sizes).forEach((sizeData) => {
        if (typeof sizeData === "object" && sizeData !== null) {
          if (sizeData.variants && typeof sizeData.variants === "object") {
            Object.entries(sizeData.variants).forEach(([vName, v]) => {
              const vCost = v?.costPrice ?? sizeData.variantCostPrices?.[vName] ?? sizeData.costPrice;
              if (vCost !== void 0 && vCost !== null) costs.push(Number(vCost));
            });
          } else if (sizeData.costPrice !== void 0 && sizeData.costPrice !== null) {
            costs.push(Number(sizeData.costPrice));
          }
        }
      });
    }
    return costs;
  };
  const sellingPriceDisplay = selectedBatchInsights ? formatMoneyRange(selectedBatchInsights.sellPrices,
`\u20B1${Number(viewingProduct.itemPrice ?? 0).toFixed(2)}`) : formatMoneyRange(collectAggregateSellPrices(),
`\u20B1${Number(viewingProduct.itemPrice ?? 0).toFixed(2)}`);
  const purchasePriceDisplay = selectedBatchInsights ? formatMoneyRange(selectedBatchInsights.costs,
`\u20B1${Number(viewingProduct.costPrice ?? 0).toFixed(2)}`) : formatMoneyRange(collectAggregateCosts(),
`\u20B1${Number(viewingProduct.costPrice ?? 0).toFixed(2)}`);
  const overviewVariantDisplay = selectedBatchInsights ? selectedBatchInsights.variantsWithStock.length > 0 ?
selectedBatchInsights.variantsWithStock.join(", ") : selectedBatchInsights.variantsWithSlot.length > 0 ?
selectedBatchInsights.variantsWithSlot.join(", ") : "\u2014" : viewingProduct.variant || "N/A";
  const displayHeaderStock = selectedBatchSlotTotal !== null ? selectedBatchSlotTotal : totalStock;
  const batchMetaLine = selectedBatchInsights ? (() => {
    const parts = [];
    if (selectedBatchInsights.lotCodes.length === 1) parts.push(`Lot ${selectedBatchInsights.lotCodes[0]}`);
    else if (selectedBatchInsights.lotCodes.length > 1) parts.push("Multiple lots");
    if (selectedBatchInsights.exps.length === 1) parts.push(`Exp ${selectedBatchInsights.exps[0]}`);
    else if (selectedBatchInsights.exps.length > 1) parts.push("Multiple expirations");
    return parts.length ? parts.join(" \xB7 ") : null;
  })() : null;
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 flex items-center justify-center z-9999 p-4 bg-black/40 backdrop-blur-sm",
      onClick: () => setShowViewModal(false)
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-2xl w-full max-w-6xl max-h-[90vh] relative overflow-hidden flex flex-col ${theme
=== "dark" ? "bg-[#1E1B18] text-white" : "bg-white text-gray-900"}`,
        style: {
          boxShadow: "0 25px 50px -12px rgba(0, 0, 0, 0.5), 0 0 0 1px rgba(0, 0, 0, 0.1)"
        },
        onClick: (e) => e.stopPropagation()
      },
      /* @__PURE__ */ React.createElement("div", { className: `px-6 py-4 border-b shrink-0 ${theme ===
"dark" ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */ React.createElement("div", { className:
"flex items-start justify-between gap-4" }, /* @__PURE__ */ React.createElement("div", { className: "min-w-
0" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement("div", { className: "w-10 h-10 rounded-xl bg-[#AD7F65] flex items-center justify-center
shadow-sm" }, /* @__PURE__ */ React.createElement("svg", { className: "w-6 h-6 text-white", fill: "none",
stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap:
"round", strokeLinejoin: "round", strokeWidth: 2, d: "M20 7l-8-4-8 4m16 0l-8 4m8-4v10l-8 4m0-10L4 7m8 4v10M4
7v10l8 4" }))), /* @__PURE__ */ React.createElement("div", { className: "min-w-0" }, /* @__PURE__ */
React.createElement("h2", { className: "text-lg font-bold truncate" }, viewingProduct.itemName), /* @__PURE__
*/ React.createElement("div", { className: `mt-0.5 text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-
500"}` }, "SKU: ", /* @__PURE__ */ React.createElement("span", { className: `font-semibold ${theme ===
"dark" ? "text-gray-200" : "text-gray-700"}` }, viewingProduct.sku || "\u2014"), viewingProduct.category ? /*
@__PURE__ */ React.createElement("span", { className: "ml-2" }, "\xB7 ", viewingProduct.category) : null)))),
/* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-2 shrink-0" }, /* @__PURE__ */
React.createElement(
```

```
        "button",
        {
          onClick: () => setShowViewModal(false),
          className: `w-10 h-10 rounded-xl flex items-center justify-center border transition-colors ${theme
=== "dark" ? "border-gray-700 text-gray-300 hover:bg-gray-800" : "border-gray-200 text-gray-500 hover:bg-
gray-50"}`,
          "aria-label": "Close modal"
        },
        /* @__PURE__ */ React.createElement("span", { className: "text-2xl leading-none" }, "\xD7")
      )))),
      /* @__PURE__ */ React.createElement("div", { className: "p-6 overflow-y-auto flex-1" }, /* @__PURE__ */
React.createElement("div", { className: "space-y-6" }, maxBatchDepth > 0 && /* @__PURE__ */
React.createElement(
        "div",
        {
          className: `rounded-xl border px-4 py-3 flex flex-col gap-2 sm:flex-row sm:items-center sm:justify-
between ${theme === "dark" ? "border-gray-700 bg-[#2A2724]" : "border-gray-200 bg-gray-50"}`
        },
        /* @__PURE__ */ React.createElement("div", { className: `text-[10px] font-semibold uppercase
tracking-wider ${theme === "dark" ? "text-gray-500" : "text-gray-500"}` }, "Stock view"),
        /* @__PURE__ */ React.createElement("div", { className: "relative w-full sm:w-[340px]" }, /*
@__PURE__ */ React.createElement(
          "button",
          {
            type: "button",
            onClick: () => setStockViewOpen((prev) => !prev),
            className: `w-full inline-flex items-center justify-between px-3 py-2 rounded-lg text-[12px]
font-semibold border ${theme === "dark" ? "bg-[#1E1B18] text-gray-200 border-gray-600" : "bg-white text-gray-
700 border-gray-300"}`
          },
          /* @__PURE__ */ React.createElement("span", { className: "truncate pr-3" }, selectedStockViewLabel),
          /* @__PURE__ */ React.createElement("span", { className: "text-xs" }, stockViewOpen ? "\u25B2" :
"\u25BC")
        ), stockViewOpen ? /* @__PURE__ */ React.createElement(
          "div",
          {
            className: `absolute z-50 mt-2 w-full rounded-lg border shadow-lg ${theme === "dark" ? "bg-
[#1E1B18] border-gray-600" : "bg-white border-gray-200"}`
          },
          /* @__PURE__ */ React.createElement("div", { className: `p-2 border-b ${theme === "dark" ? "border-
gray-700" : "border-gray-100"}` }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center
gap-2" }, /* @__PURE__ */ React.createElement(
            "input",
            {
              type: "text",
              value: stockViewQuery,
              onChange: (e) => setStockViewQuery(e.target.value),
              placeholder: "Search batch...",
              className: `flex-1 px-2.5 py-2 rounded-md text-xs border outline-none ${theme === "dark" ? "bg-
[#2A2724] border-gray-600 text-gray-200 placeholder:text-gray-500" : "bg-white border-gray-300 text-gray-700
placeholder:text-gray-400"}`
            }
          ), /* @__PURE__ */ React.createElement(
            "button",
            {
              type: "button",
              onClick: () => setBatchSortOrder((prev) => prev === "oldest" ? "latest" : "oldest"),
              title: `Sort: ${batchSortOrder === "oldest" ? "Oldest first" : "Latest first"}`,
              className: `inline-flex items-center justify-center w-9 h-9 rounded-md border text-xs font-
semibold ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-gray-200 hover:border-[#AD7F65]" : "bg-white
border-gray-300 text-gray-700 hover:border-[#AD7F65]"}`
            },
            /* @__PURE__ */ React.createElement(
              "img",
              {
                src: filterIcon,
                alt: "Sort",
                className: `w-4 h-4 ${theme === "dark" ? "opacity-90 invert" : "opacity-90"}`
              }
            )
          ))),
          /* @__PURE__ */ React.createElement("div", { className: "max-h-56 overflow-y-auto p-1.5" },
filteredStockViewOptions.length > 0 ? filteredStockViewOptions.map((option) => /* @__PURE__ */
React.createElement(
            "button",
            {
              key: String(option.value),
```

```
              type: "button",
              onClick: () => {
                setBatchTab(option.value);
                setStockViewOpen(false);
              },
              className: `w-full text-left px-2.5 py-2 rounded-md text-xs font-medium ${batchTab ===
option.value ? "bg-[#AD7F65] text-white" : theme === "dark" ? "text-gray-200 hover:bg-[#2A2724]" : "text-
gray-700 hover:bg-gray-50"}`
            },
            option.label
          )) : /* @__PURE__ */ React.createElement("div", { className: `px-2.5 py-3 text-xs ${theme ===
"dark" ? "text-gray-500" : "text-gray-500"}` }, "No matching stock view."))
        ) : null)
      ), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-1 lg:grid-cols-12 gap-6" },
/* @__PURE__ */ React.createElement("div", { className: "lg:col-span-5" }, /* @__PURE__ */
React.createElement("div", { className: `rounded-2xl border overflow-hidden h-full ${theme === "dark" ?
"border-gray-700 bg-[#2A2724]" : "border-gray-200 bg-gray-50"}` }, /* @__PURE__ */ React.createElement("div",
{ className: "p-4" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-
between gap-3" }, /* @__PURE__ */ React.createElement("div", { className: `text-xs font-semibold uppercase
tracking-wider ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Preview"), /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-2" }, /* @__PURE__ */
React.createElement("span", { className: `px-2.5 py-1 rounded-full text-xs font-semibold border ${theme ===
"dark" ? "border-gray-700 text-gray-200" : "border-gray-200 text-gray-700"}` }, viewingProduct.category ||
"\u2014"), viewingProduct.genderCategory && /* @__PURE__ */ React.createElement("span", { className: `px-2.5
py-1 rounded-full text-xs font-semibold border ${theme === "dark" ? "border-gray-700 text-gray-200" :
"border-gray-200 text-gray-700"}` }, viewingProduct.genderCategory)))), /* @__PURE__ */
React.createElement("div", { className: `p-0 min-h-[440px] ${theme === "dark" ? "bg-[#1E1B18]" : "bg-
white"}` }, productImages.length > 0 && productImages[productImgIdx] ? /* @__PURE__ */
React.createElement("div", { className: "relative w-full h-[440px]" }, /* @__PURE__ */ React.createElement(
        "img",
        {
          src: productImages[productImgIdx],
          alt: viewingProduct.itemName,
          className: "w-full h-full object-cover"
        }
      ), productImages.length > 1 && /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__
*/ React.createElement(
        "button",
        {
          type: "button",
          onClick: () => setProductImgIdx((i) => (i - 1 + productImages.length) % productImages.length),
          className: "absolute left-2 top-1/2 -translate-y-1/2 w-8 h-8 rounded-full bg-black/40 text-white
flex items-center justify-center"
        },
        "\u2039"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: () => setProductImgIdx((i) => (i + 1) % productImages.length),
          className: "absolute right-2 top-1/2 -translate-y-1/2 w-8 h-8 rounded-full bg-black/40 text-white
flex items-center justify-center"
        },
        "\u203A"
      ), /* @__PURE__ */ React.createElement("div", { className: "absolute bottom-2 left-0 right-0 flex
justify-center gap-1" }, productImages.map((_, idx) => /* @__PURE__ */ React.createElement(
        "button",
        {
          key: idx,
          type: "button",
          onClick: () => setProductImgIdx(idx),
          className: `w-2 h-2 rounded-full ${idx === productImgIdx ? "bg-white" : "bg-white/40"}`
        }
      ))))) : /* @__PURE__ */ React.createElement("div", { className: "text-center text-gray-400" }, /*
@__PURE__ */ React.createElement("svg", { className: "w-24 h-24 mx-auto mb-4", fill: "none", stroke:
"currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap: "round",
strokeLinejoin: "round", strokeWidth: 2, d: "M4 16l4.586-4.586a2 2 0 012.828 0L16 16m-2-2l1.586-1.586a2 2 0
012.828 0L20 14m-6-6h.01M6 20h12a2 2 0 002-2V6a2 2 0 00-2-2H6a2 2 0 00-2 2v12a2 2 0 002 2z" })), /* @__PURE__
*/ React.createElement("p", { className: "text-sm" }, "No Image Available"))))), /* @__PURE__ */
React.createElement("div", { className: "lg:col-span-7 space-y-4" }, /* @__PURE__ */
React.createElement("div", { className: `rounded-2xl border p-5 ${theme === "dark" ? "border-gray-700 bg-
[#2A2724]" : "border-gray-200 bg-white"}` }, /* @__PURE__ */ React.createElement("div", { className: "flex
items-start justify-between gap-4" }, /* @__PURE__ */ React.createElement("div", { className: "min-w-0" }, /*
@__PURE__ */ React.createElement("div", { className: `text-xs font-semibold uppercase tracking-wider ${theme
=== "dark" ? "text-gray-400" : "text-gray-500"}` }, "Overview"), /* @__PURE__ */ React.createElement("div",
{ className: "mt-2 text-lg font-bold truncate" }, viewingProduct.itemName), /* @__PURE__ */
React.createElement("div", { className: `mt-1 text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-
```

```
500"}` }, viewingProduct.category, " ", viewingProduct.subCategory ? `> ${viewingProduct.subCategory}` : "",
viewingProduct.foodSubtype ? ` \xB7 ${viewingProduct.foodSubtype}` : ""), selectedBatchInsights && /*
@__PURE__ */ React.createElement("div", { className: "mt-2 flex flex-wrap items-center gap-2" }, /* @__PURE__
*/ React.createElement(
        "span",
        {
          className: `inline-flex items-center px-2 py-0.5 rounded-lg text-[10px] font-bold uppercase
tracking-wide ${theme === "dark" ? "bg-[#AD7F65]/25 text-[#D4A88A]" : "bg-[#AD7F65]/15 text-[#76462B]"}`
        },
        getBatchLabel(batchTab)
      ), batchMetaLine ? /* @__PURE__ */ React.createElement("span", { className: `text-[11px] ${theme ===
"dark" ? "text-gray-400" : "text-gray-600"}` }, batchMetaLine) : null)), /* @__PURE__ */
React.createElement("div", { className: "flex flex-col items-end gap-1" }, /* @__PURE__ */
React.createElement("div", { className: `text-xs font-semibold ${theme === "dark" ? "text-gray-400" : "text-
gray-500"}` }, "SKU"), /* @__PURE__ */ React.createElement("div", { className: `px-3 py-1.5 rounded-xl border
text-sm font-semibold ${theme === "dark" ? "border-gray-700 bg-[#1E1B18] text-gray-200" : "border-gray-200
bg-gray-50 text-gray-800"}` }, viewingProduct.sku || "\u2014"))), /* @__PURE__ */ React.createElement("div",
{ className: "mt-4 grid grid-cols-2 gap-3" }, /* @__PURE__ */ React.createElement("div", { className:
`rounded-xl border p-3 ${theme === "dark" ? "border-gray-700 bg-[#1E1B18]" : "border-gray-200 bg-gray-50"}` },
/* @__PURE__ */ React.createElement("div", { className: `text-[11px] font-semibold uppercase tracking-wider
${theme === "dark" ? "text-gray-500" : "text-gray-500"}` }, "Brand"), /* @__PURE__ */
React.createElement("div", { className: `mt-1 text-sm font-semibold ${theme === "dark" ? "text-gray-200" :
"text-gray-800"}` }, viewingProduct.brandName || "N/A")), /* @__PURE__ */ React.createElement("div",
{ className: `rounded-xl border p-3 ${theme === "dark" ? "border-gray-700 bg-[#1E1B18]" : "border-gray-200
bg-gray-50"}` }, /* @__PURE__ */ React.createElement("div", { className: `text-[11px] font-semibold uppercase
tracking-wider ${theme === "dark" ? "text-gray-500" : "text-gray-500"}` }, selectedBatchInsights ? "Options
(this batch)" : "Variant"), /* @__PURE__ */ React.createElement("div", { className: `mt-1 text-sm font-
semibold leading-snug ${theme === "dark" ? "text-gray-200" : "text-gray-800"}` }, overviewVariantDisplay)))),
/* @__PURE__ */ React.createElement("div", { className: `rounded-2xl border p-5 ${theme === "dark" ? "border-
gray-700 bg-[#2A2724]" : "border-gray-200 bg-white"}` }, /* @__PURE__ */ React.createElement("div",
{ className: `text-xs font-semibold uppercase tracking-wider ${theme === "dark" ? "text-gray-400" : "text-
gray-500"}` }, "Pricing"), selectedBatchInsights ? /* @__PURE__ */ React.createElement("div", { className:
`mt-1 text-[10px] ${theme === "dark" ? "text-gray-500" : "text-gray-500"}` }, "Sell and cost reflect this
batch layer across all options (range if prices differ).") : null, /* @__PURE__ */ React.createElement("div",
{ className: "mt-4 grid grid-cols-2 gap-3" }, /* @__PURE__ */ React.createElement("div", { className:
`rounded-xl border p-4 ${theme === "dark" ? "border-gray-700 bg-[#1E1B18]" : "border-gray-200 bg-gray-50"}` },
/* @__PURE__ */ React.createElement("div", { className: `text-[11px] font-semibold uppercase tracking-wider
${theme === "dark" ? "text-gray-500" : "text-gray-500"}` }, "Selling Price"), /* @__PURE__ */
React.createElement("div", { className: `mt-1 text-lg font-bold ${theme === "dark" ? "text-green-400" :
"text-green-600"}` }, sellingPriceDisplay)), /* @__PURE__ */ React.createElement("div", { className:
`rounded-xl border p-4 ${theme === "dark" ? "border-gray-700 bg-[#1E1B18]" : "border-gray-200 bg-gray-50"}` },
/* @__PURE__ */ React.createElement("div", { className: `text-[11px] font-semibold uppercase tracking-wider
${theme === "dark" ? "text-gray-500" : "text-gray-500"}` }, "Purchase Price"), /* @__PURE__ */
React.createElement("div", { className: `mt-1 text-lg font-bold ${theme === "dark" ? "text-red-400" : "text-
red-600"}` }, purchasePriceDisplay)))), /* @__PURE__ */ React.createElement("div", { className: "grid grid-
cols-3 gap-3" }, /* @__PURE__ */ React.createElement("div", { className: `rounded-xl border p-3 ${theme ===
"dark" ? "border-gray-700 bg-[#1E1B18]" : "border-gray-200 bg-gray-50"}` }, /* @__PURE__ */
React.createElement("div", { className: `text-[11px] font-semibold uppercase tracking-wider ${theme ===
"dark" ? "text-gray-500" : "text-gray-500"}` }, selectedBatchInsights ? "Stock (this batch)" : "Total Stock"),
/* @__PURE__ */ React.createElement("div", { className: "mt-1 flex items-center justify-between gap-2" }, /*
@__PURE__ */ React.createElement("div", { className: `text-lg font-bold ${theme === "dark" ? "text-gray-200" :
"text-gray-800"}` }, displayHeaderStock), /* @__PURE__ */ React.createElement("div", { className: `text-xs
font-semibold px-2 py-0.5 rounded-full ${displayHeaderStock === 0 ? "bg-red-100 text-red-700" :
displayHeaderStock <= (viewingProduct.reorderNumber || 10) ? "bg-yellow-100 text-yellow-700" : "bg-green-100
text-green-700"}` }, displayHeaderStock === 0 ? "Out" : displayHeaderStock <= (viewingProduct.reorderNumber
|| 10) ? "Low" : "In Stock"))), /* @__PURE__ */ React.createElement("div", { className: `rounded-xl border p-
3 ${theme === "dark" ? "border-gray-700 bg-[#1E1B18]" : "border-gray-200 bg-gray-50"}` }, /* @__PURE__ */
React.createElement("div", { className: `text-[11px] font-semibold uppercase tracking-wider ${theme ===
"dark" ? "text-gray-500" : "text-gray-500"}` }, "Reorder Level"), /* @__PURE__ */ React.createElement("div",
{ className: `mt-1 text-lg font-bold ${theme === "dark" ? "text-gray-200" : "text-gray-800"}` },
viewingProduct.reorderNumber || 0), /* @__PURE__ */ React.createElement("div", { className: `text-[10px]
${theme === "dark" ? "text-gray-500" : "text-gray-500"}` }, "Threshold for low-stock")), /* @__PURE__ */
React.createElement("div", { className: `rounded-xl border p-3 ${theme === "dark" ? "border-gray-700 bg-
[#1E1B18]" : "border-gray-200 bg-gray-50"}` }, /* @__PURE__ */ React.createElement("div", { className: `text-
[11px] font-semibold uppercase tracking-wider ${theme === "dark" ? "text-gray-500" : "text-gray-500"}` },
"Expiration Threshold"), /* @__PURE__ */ React.createElement("div", { className: `mt-1 text-lg font-bold
${theme === "dark" ? "text-gray-200" : "text-gray-800"}` }, viewingProduct.expirationDate ?
viewingProduct.expirationThresholdDays ?? 30 : "\u2014"), /* @__PURE__ */ React.createElement("div",
{ className: `text-[10px] ${theme === "dark" ? "text-gray-500" : "text-gray-500"}` }, "Days before expiration
alert"))))), /* @__PURE__ */ React.createElement("div", { className: `rounded-2xl border ${theme === "dark" ?
"border-gray-700 bg-[#2A2724]" : "border-gray-200 bg-white"}` }, /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "px-5 py-4 border-b flex flex-col gap-1",
          style: { borderColor: theme === "dark" ? "#374151" : "#E5E7EB" }
        },
        /* @__PURE__ */ React.createElement("div", { className: `text-xs font-semibold uppercase tracking-
```

```
wider ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Stock"),
        /* @__PURE__ */ React.createElement("div", { className: `text-xs ${theme === "dark" ? "text-gray-
500" : "text-gray-500"}` }, maxBatchDepth > 0 ? showPerBatchColumn ? batchTab === 0 ? "Table: oldest batch
(FIFO) per option" : `Table: batch ${batchTab + 1} per option` : "Table: total + price per option" : "Table:
total + price per option"),
        /* @__PURE__ */ React.createElement("div", { className: `text-[11px] ${theme === "dark" ? "text-gray-
500" : "text-gray-500"}` }, typeof batchTab === "number" ? `Only options that have data in ${batchTab === 0 ?
"Batch 1" : `Batch ${batchTab + 1}`} are listed.` : "Showing options that have stock or past batch history.
Never-stocked combinations are hidden.")
      ), /* @__PURE__ */ React.createElement("div", { className: "p-4" }, viewingProduct.sizes && typeof
viewingProduct.sizes === "object" && Object.keys(viewingProduct.sizes).length > 0 ? /* @__PURE__ */
React.createElement("div", { className: `overflow-x-auto rounded-xl border ${theme === "dark" ? "border-gray-
700 bg-[#1E1B18]" : "border-gray-200 bg-white"}` }, /* @__PURE__ */ React.createElement("table", { className:
"w-full text-left text-sm whitespace-nowrap" }, /* @__PURE__ */ React.createElement("thead", { className:
`text-xs uppercase bg-opacity-50 ${theme === "dark" ? "bg-gray-800 text-gray-400" : "bg-gray-50 text-gray-
600"}` }, /* @__PURE__ */ React.createElement("tr", null, showPerBatchColumn ? /* @__PURE__ */
React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("th", { className: "px-4 py-3
font-semibold" }, "Date Added"), /* @__PURE__ */ React.createElement("th", { className: "px-4 py-3 font-
semibold" }, "SKU"), /* @__PURE__ */ React.createElement("th", { className: "px-4 py-3 font-semibold" },
"Variant / Size"), /* @__PURE__ */ React.createElement("th", { className: "px-4 py-3 font-semibold" }, "Qty
Stocked In"), /* @__PURE__ */ React.createElement("th", { className: "px-4 py-3 font-semibold" }, "Current
Stock"), /* @__PURE__ */ React.createElement("th", { className: "px-4 py-3 font-semibold" }, "Cost Price"),
/* @__PURE__ */ React.createElement("th", { className: "px-4 py-3 font-semibold" }, "Selling Price"),
showExpirationColumns && /* @__PURE__ */ React.createElement("th", { className: "px-4 py-3 font-semibold" },
"Expiration"), showExpirationColumns && /* @__PURE__ */ React.createElement("th", { className: "px-4 py-3
font-semibold" }, "Days Left")) : /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement("th", { className: "px-4 py-3 font-semibold" }, "SKU"), /* @__PURE__ */
React.createElement("th", { className: "px-4 py-3 font-semibold" }, "Variant / Size"), /* @__PURE__ */
React.createElement("th", { className: "px-4 py-3 font-semibold" }, "Stock"), /* @__PURE__ */
React.createElement("th", { className: "px-4 py-3 font-semibold" }, "Price"), showExpirationColumns && /*
@__PURE__ */ React.createElement("th", { className: "px-4 py-3 font-semibold" }, "Expiration"),
showExpirationColumns && /* @__PURE__ */ React.createElement("th", { className: "px-4 py-3 font-semibold" },
"Days Left")))), /* @__PURE__ */ React.createElement("tbody", { className: `divide-y ${theme === "dark" ?
"divide-gray-700" : "divide-gray-100"}` }, (() => {
        const rows = [];
        const baseSku = viewingProduct.sku || "";
        Object.entries(viewingProduct.sizes).forEach(([size, sizeData]) => {
          const variants = sizeData && typeof sizeData === "object" && sizeData.variants ? sizeData.variants :
null;
          const variantPrices = sizeData && typeof sizeData === "object" && sizeData.variantPrices ?
sizeData.variantPrices : null;
          const sizeInitial = size === VARIANT_ONLY_SIZE_KEY ? "VO" : size === "Free Size" ? "FS" :
size.substring(0, 2).toUpperCase();
          if (variants && Object.keys(variants).length > 0 || variantPrices &&
Object.keys(variantPrices).length > 0) {
            const variantKeys = variants && Object.keys(variants).length > 0 ? Object.keys(variants) :
variantPrices ? Object.keys(variantPrices) : [];
            variantKeys.forEach((variantName) => {
              const variantData = variants?.[variantName];
              const variantQty = typeof variantData === "number" ? variantData : variantData && typeof
variantData === "object" ? variantData.quantity || 0 : 0;
              const batches = getBatchList(typeof variantData === "object" && variantData !== null ?
variantData : null);
              if (!shouldShowStockRow(variantQty, batches)) return;
              const dynamicSku = generateDynamicSku(baseSku, variantName, size);
              rows.push(
                /* @__PURE__ */ React.createElement("tr", { key: `${size}-${variantName}`, className:
`transition-colors ${theme === "dark" ? "hover:bg-gray-800" : "hover:bg-gray-50"}` }, showPerBatchColumn ? /*
@__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("td", { className:
"px-4 py-3" }, renderSingleBatchDateAddedCell(batches[batchTab] ?? null)), /* @__PURE__ */
React.createElement("td", { className: `px-4 py-3 font-medium ${theme === "dark" ? "text-gray-300" : "text-
gray-900"}` }, dynamicSku), /* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 ${theme ===
"dark" ? "text-gray-400" : "text-gray-600"}` }, size === VARIANT_ONLY_SIZE_KEY ? variantName : `${variantName}
/ ${size}`), /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3" },
renderBatchNumberCell(batches[batchTab]?.originalQty ?? batches[batchTab]?.qty ?? 0)), /* @__PURE__ */
React.createElement("td", { className: "px-4 py-3" }, renderBatchNumberCell(batches[batchTab]?.qty ?? 0,
"stock")), /* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 text-xs ${theme === "dark" ?
"text-red-400" : "text-red-500"}` }, "\u20B1", Number(batches[batchTab]?.costPrice ?? 0).toFixed(2)), /*
@__PURE__ */ React.createElement("td", { className: `px-4 py-3 text-xs font-medium ${theme === "dark" ?
"text-green-400" : "text-green-600"}` }, "\u20B1", Number(batches[batchTab]?.price ?? 0).toFixed(2)),
showExpirationColumns && /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3" },
renderSingleBatchExpirationCell(batches[batchTab] ?? null)), showExpirationColumns && /* @__PURE__ */
React.createElement("td", { className: "px-4 py-3" },
renderDaysLeftCell(getSingleBatchExpirationDate(batches[batchTab] ?? null)))) : /* @__PURE__ */
React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("td", { className: `px-4 py-3
font-medium ${theme === "dark" ? "text-gray-300" : "text-gray-900"}` }, dynamicSku), /* @__PURE__ */
React.createElement("td", { className: `px-4 py-3 ${theme === "dark" ? "text-gray-400" : "text-gray-600"}` },
```

```
size === VARIANT_ONLY_SIZE_KEY ? variantName : `${variantName} / ${size}`), /* @__PURE__ */
React.createElement("td", { className: "px-4 py-3" }, /* @__PURE__ */ React.createElement("span", { className:
`inline-flex items-center px-2 py-0.5 rounded-full text-xs font-medium ${variantQty === 0 ? "bg-red-100 text-
red-700" : variantQty <= (viewingProduct.reorderNumber || 10) ? "bg-yellow-100 text-yellow-700" : "bg-green-
100 text-green-700"}` }, variantQty, " ", viewingProduct.unitOfMeasure || "pcs")), /* @__PURE__ */
React.createElement("td", { className: "px-4 py-3" }, /* @__PURE__ */ React.createElement("div", { className:
"space-y-0.5" }, /* @__PURE__ */ React.createElement("div", { className: `text-xs font-medium ${theme ===
"dark" ? "text-green-400" : "text-green-600"}` }, "Sell: \u20B1", Number(variantData?.price ??
variantPrices?.[variantName] ?? sizeData?.price ?? viewingProduct.itemPrice ?? 0).toFixed(2)), /* @__PURE__
*/ React.createElement("div", { className: `text-xs ${theme === "dark" ? "text-red-400" : "text-red-500"}` },
"Cost: \u20B1", Number(variantData?.costPrice ?? sizeData?.variantCostPrices?.[variantName] ??
sizeData?.costPrice ?? viewingProduct.costPrice ?? 0).toFixed(2)))), showExpirationColumns && /* @__PURE__ */
React.createElement("td", { className: "px-4 py-3" }, renderAggregateExpiration(batches)),
showExpirationColumns && /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3" },
renderDaysLeftCell(getAggregateExpirationDate(batches)))))
              );
            });
          } else {
            const stock = typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0 ?
sizeData.quantity : typeof sizeData === "number" ? sizeData : 0;
            const batches = getBatchList(typeof sizeData === "object" && sizeData !== null ? sizeData : null);
            if (!shouldShowStockRow(stock, batches)) return;
            const dynamicSku = generateDynamicSku(baseSku, null, size);
            rows.push(
              /* @__PURE__ */ React.createElement("tr", { key: size, className: `transition-colors ${theme
=== "dark" ? "hover:bg-gray-800" : "hover:bg-gray-50"}` }, showPerBatchColumn ? /* @__PURE__ */
React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("td", { className: "px-4 py-
3" }, renderSingleBatchDateAddedCell(batches[batchTab] ?? null)), /* @__PURE__ */ React.createElement("td",
{ className: `px-4 py-3 font-medium ${theme === "dark" ? "text-gray-300" : "text-gray-900"}` }, dynamicSku),
/* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 ${theme === "dark" ? "text-gray-400" :
"text-gray-600"}` }, size), /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3" },
renderBatchNumberCell(batches[batchTab]?.originalQty ?? batches[batchTab]?.qty ?? 0)), /* @__PURE__ */
React.createElement("td", { className: "px-4 py-3" }, renderBatchNumberCell(batches[batchTab]?.qty ?? 0,
"stock")), /* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 text-xs ${theme === "dark" ?
"text-red-400" : "text-red-500"}` }, "\u20B1", Number(batches[batchTab]?.costPrice ?? 0).toFixed(2)), /*
@__PURE__ */ React.createElement("td", { className: `px-4 py-3 text-xs font-medium ${theme === "dark" ?
"text-green-400" : "text-green-600"}` }, "\u20B1", Number(batches[batchTab]?.price ?? 0).toFixed(2)),
showExpirationColumns && /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3" },
renderSingleBatchExpirationCell(batches[batchTab] ?? null)), showExpirationColumns && /* @__PURE__ */
React.createElement("td", { className: "px-4 py-3" },
renderDaysLeftCell(getSingleBatchExpirationDate(batches[batchTab] ?? null)))) : /* @__PURE__ */
React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("td", { className: `px-4 py-3
font-medium ${theme === "dark" ? "text-gray-300" : "text-gray-900"}` }, dynamicSku), /* @__PURE__ */
React.createElement("td", { className: `px-4 py-3 ${theme === "dark" ? "text-gray-400" : "text-gray-600"}` },
size), /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3" }, /* @__PURE__ */
React.createElement("span", { className: `inline-flex items-center px-2 py-0.5 rounded-full text-xs font-
medium ${stock === 0 ? "bg-red-100 text-red-700" : stock <= (viewingProduct.reorderNumber || 10) ? "bg-
yellow-100 text-yellow-700" : "bg-green-100 text-green-700"}` }, stock, " ", viewingProduct.unitOfMeasure ||
"pcs")), /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3" }, /* @__PURE__ */
React.createElement("div", { className: "space-y-0.5" }, /* @__PURE__ */ React.createElement("div",
{ className: `text-xs font-medium ${theme === "dark" ? "text-green-400" : "text-green-600"}` }, "Sell:
\u20B1", Number(typeof sizeData === "object" && sizeData !== null ? sizeData.price ??
viewingProduct.itemPrice ?? 0 : viewingProduct.itemPrice ?? 0).toFixed(2)), /* @__PURE__ */
React.createElement("div", { className: `text-xs ${theme === "dark" ? "text-red-400" : "text-red-500"}` },
"Cost: \u20B1", Number(typeof sizeData === "object" && sizeData !== null ? sizeData.costPrice ??
viewingProduct.costPrice ?? 0 : viewingProduct.costPrice ?? 0).toFixed(2)))), showExpirationColumns && /*
@__PURE__ */ React.createElement("td", { className: "px-4 py-3" }, renderAggregateExpiration(batches)),
showExpirationColumns && /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3" },
renderDaysLeftCell(getAggregateExpirationDate(batches)))))
            );
          }
        });
        if (rows.length === 0) {
          rows.push(
            /* @__PURE__ */ React.createElement("tr", { key: "no-visible-options" }, /* @__PURE__ */
React.createElement(
              "td",
              {
                colSpan: showPerBatchColumn ? showExpirationColumns ? 9 : 7 : showExpirationColumns ? 6 : 4,
                className: `px-4 py-6 text-center text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-
500"}`
              },
              "No options with stock or batch history yet. Stock in to add inventory for a variant."
            ))
          );
        }
        return rows;
```

```
      })()())), /* @__PURE__ */ React.createElement("div", { className: `px-4 py-3 border-t text-right flex
justify-end items-center gap-2 ${theme === "dark" ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__
*/ React.createElement("span", { className: `text-xs uppercase tracking-wider font-semibold ${theme ===
"dark" ? "text-gray-500" : "text-gray-500"}` }, tableFooterLabel), /* @__PURE__ */ React.createElement("span",
{ className: `text-sm font-bold ${theme === "dark" ? "text-white" : "text-gray-900"}` }, tableFooterTotal))) :
/* @__PURE__ */ React.createElement("div", { className: `inline-flex items-center gap-2 px-3 py-2 rounded-lg
border ${theme === "dark" ? "bg-[#1E1B18] border-gray-700" : "bg-gray-50 border-gray-200"}` }, /* @__PURE__
*/ React.createElement("span", { className: `text-sm font-medium ${theme === "dark" ? "text-gray-300" :
"text-gray-700"}` }, viewingProduct.sku || "N/A"), /* @__PURE__ */ React.createElement("span", { className:
`inline-flex items-center px-2.5 py-0.5 rounded-full text-xs font-medium ${totalStock === 0 ? "bg-red-100
text-red-700" : totalStock <= (viewingProduct.reorderNumber || 10) ? "bg-yellow-100 text-yellow-700" : "bg-
green-100 text-green-700"}` }, totalStock, " ", viewingProduct.unitOfMeasure || "pcs"))))))
    )
  );
};
export default ViewProductModal;
```

FILE: pos/frontend/src/components/logs/ViewVoidLogModal.jsx
==========

```
import { FaTimes } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const ViewVoidLogModal = ({ isOpen, onClose, voidLog }) => {
  const { theme } = useTheme();
  if (!isOpen || !voidLog) return null;
  const formatDateTime = (dateString) => {
    if (!dateString) return "-";
    const date = new Date(dateString);
    return date.toLocaleString("en-US", {
      year: "numeric",
      month: "long",
      day: "numeric",
      hour: "2-digit",
      minute: "2-digit",
      hour12: true
    });
  };
  const formatCurrency = (amount) => {
    return `\u20B1${(amount || 0).toFixed(2)}`;
  };
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 flex items-center justify-center z-9999 backdrop-blur-sm bg-black/30",
      onClick: onClose
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-2xl w-full max-w-lg shadow-2xl overflow-hidden ${theme === "dark" ? "bg-
[#2A2724]" : "bg-white"}`,
        onClick: (e) => e.stopPropagation()
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "px-6 py-5 relative",
          style: { background: "linear-gradient(135deg, #DC2626 0%, #991B1B 100%)" }
        },
        /* @__PURE__ */ React.createElement("h2", { className: "text-xl font-bold text-white" }, "This
Transaction is Voided"),
        /* @__PURE__ */ React.createElement("p", { className: "text-red-200 text-sm mt-1" }, "This entire
transaction has been cancelled"),
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: onClose,
            className: "absolute top-4 right-4 text-white/80 hover:text-white transition"
          },
          /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
        )
      ),
      /* @__PURE__ */ React.createElement("div", { className: "p-6" }, /* @__PURE__ */
React.createElement("div", { className: "mb-4" }, /* @__PURE__ */ React.createElement("h3", { className:
"text-2xl font-bold text-[#8B4513]" }, "Transaction Id: ", voidLog.voidId || "N/A"), /* @__PURE__ */
React.createElement("p", { className: `text-sm mt-1 ${theme === "dark" ? "text-gray-400" : "text-gray-
```

```
500"}` }, "Date: ", formatDateTime(voidLog.voidedAt)), /* @__PURE__ */ React.createElement("p", { className:
`text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Handled By: ", voidLog.voidedByName ||
"Unknown")), /* @__PURE__ */ React.createElement("div", { className: `rounded-lg overflow-hidden mb-4 ${theme
=== "dark" ? "bg-[#1E1B18]" : "bg-gray-50"}` }, /* @__PURE__ */ React.createElement("table", { className: "w-
full" }, /* @__PURE__ */ React.createElement("thead", null, /* @__PURE__ */ React.createElement("tr",
{ className: `border-b ${theme === "dark" ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */
React.createElement("th", { className: `px-4 py-3 text-left text-sm font-semibold ${theme === "dark" ? "text-
gray-400" : "text-gray-700"}` }, "Item"), /* @__PURE__ */ React.createElement("th", { className: `px-4 py-3
text-center text-sm font-semibold ${theme === "dark" ? "text-gray-400" : "text-gray-700"}` }, "Quantity"), /*
@__PURE__ */ React.createElement("th", { className: `px-4 py-3 text-right text-sm font-semibold ${theme ===
"dark" ? "text-gray-400" : "text-gray-700"}` }, "Price"), /* @__PURE__ */ React.createElement("th",
{ className: `px-4 py-3 text-right text-sm font-semibold ${theme === "dark" ? "text-gray-400" : "text-gray-
700"}` }, "Total"))), /* @__PURE__ */ React.createElement("tbody", null, voidLog.items &&
voidLog.items.length > 0 ? voidLog.items.map(
        (item, index) => /* @__PURE__ */ React.createElement("tr", { key: index, className: `border-b
last:border-0 ${theme === "dark" ? "border-gray-700" : "border-gray-100"}` }, /* @__PURE__ */
React.createElement("td", { className: `px-4 py-3 text-sm ${theme === "dark" ? "text-gray-300" : "text-gray-
600"}` }, item.itemName || "Unknown Item", item.selectedSize && /* @__PURE__ */ React.createElement("span",
{ className: `ml-1 ${theme === "dark" ? "text-gray-500" : "text-gray-400"}` }, "(", item.selectedSize, ")")),
/* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 text-sm text-center ${theme === "dark" ?
"text-gray-300" : "text-gray-600"}` }, "x", item.quantity || 1), /* @__PURE__ */ React.createElement("td",
{ className: `px-4 py-3 text-sm text-right ${theme === "dark" ? "text-gray-300" : "text-gray-600"}` },
formatCurrency(item.price)), /* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 text-sm text-
right ${theme === "dark" ? "text-gray-300" : "text-gray-600"}` }, formatCurrency((item.price || 0) *
(item.quantity || 1))))
      ) : /* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */ React.createElement("td", { colSpan:
"4", className: "px-4 py-3 text-sm text-gray-500 text-center" }, "No items found"))))), /* @__PURE__ */
React.createElement("div", { className: "space-y-2 mb-4" }, /* @__PURE__ */ React.createElement("div",
{ className: "flex justify-between text-sm" }, /* @__PURE__ */ React.createElement("span", { className:
`${theme === "dark" ? "text-gray-400" : "text-gray-600"}` }, "Subtotal:"), /* @__PURE__ */
React.createElement("span", { className: `${theme === "dark" ? "text-gray-200" : "text-gray-800"}` },
formatCurrency(voidLog.totalAmount))), /* @__PURE__ */ React.createElement("div", { className: "flex justify-
between text-sm" }, /* @__PURE__ */ React.createElement("span", { className: `${theme === "dark" ? "text-
gray-400" : "text-gray-600"}` }, "Discount:"), /* @__PURE__ */ React.createElement("span", { className:
`${theme === "dark" ? "text-gray-200" : "text-gray-800"}` }, formatCurrency(0))), /* @__PURE__ */
React.createElement("div", { className: `flex justify-between pt-2 border-t ${theme === "dark" ? "border-
gray-700" : "border-gray-200"}` }, /* @__PURE__ */ React.createElement("span", { className: `font-bold
${theme === "dark" ? "text-gray-200" : "text-gray-800"}` }, "Total:"), /* @__PURE__ */
React.createElement("span", { className: "font-bold text-red-600" }, formatCurrency(voidLog.totalAmount), " ",
/* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "(Voided)")))), /* @__PURE__ */
React.createElement("div", { className: "mb-4" }, /* @__PURE__ */ React.createElement("p", { className:
`text-sm mb-2 ${theme === "dark" ? "text-gray-400" : "text-gray-600"}` }, "Reason for Void:"), /* @__PURE__
*/ React.createElement("div", { className: `rounded-lg px-4 py-3 ${theme === "dark" ? "bg-[#1E1B18]" : "bg-
gray-100"}` }, /* @__PURE__ */ React.createElement("p", { className: `${theme === "dark" ? "text-gray-300" :
"text-gray-800"}` }, voidLog.voidReason || "No reason provided"))), /* @__PURE__ */ React.createElement("div",
{ className: "space-y-2 mb-6" }, /* @__PURE__ */ React.createElement("p", { className: "text-sm text-gray-
600" }, "Approved By: ", /* @__PURE__ */ React.createElement("span", { className: `font-medium
${voidLog.approvedByRole === "Owner" ? "text-purple-700" : voidLog.approvedByRole === "Manager" ? "text-blue-
700" : `${theme === "dark" ? "text-gray-300" : "text-gray-800"}`}` }, voidLog.approvedBy ?
`${voidLog.approvedBy}${voidLog.approvedByRole ? ` (${voidLog.approvedByRole})` : ""}` : "N/A")), /*
@__PURE__ */ React.createElement("div", { className: "flex items-center gap-2" }, /* @__PURE__ */
React.createElement("span", { className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-600"}` },
"Status:"), /* @__PURE__ */ React.createElement("span", { className: "inline-flex items-center gap-1 px-3 py-
1 rounded-full text-xs font-medium bg-red-100 text-red-600" }, /* @__PURE__ */ React.createElement("span",
{ className: "w-2 h-2 rounded-full bg-red-500" }), "Voided"))), /* @__PURE__ */ React.createElement("div",
{ className: "flex justify-end" }, /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onClose,
          className: `px-6 py-2 rounded-lg font-medium transition-all ${theme === "dark" ? "bg-gray-700 text-
gray-200 hover:bg-gray-600" : "bg-gray-200 text-gray-700 hover:bg-gray-300"}`
        },
        "Cancel"
      )))
    )
  );
};
export default ViewVoidLogModal;
```

FILE: pos/frontend/src/components/owner/AddBrandPartnerModal.jsx
==========

```
import { useEffect, useRef, useState } from "react";
import { FaTimes, FaUserFriends } from "react-icons/fa";
import circleIcon from "../../assets/owner/circle.svg";
import cameraIcon from "../../assets/owner/camera.svg";
```

```
import { useTheme } from "../../context/ThemeContext";
const initialFormState = {
  brandName: "",
  email: "",
  contactNumber: "+63",
  contactPerson: "",
  logo: ""
};
const AddBrandPartnerModal = ({ isOpen, onClose, onSuccess }) => {
  const { theme } = useTheme();
  const [formData, setFormData] = useState(initialFormState);
  const [submitting, setSubmitting] = useState(false);
  const [error, setError] = useState("");
  const fileInputRef = useRef(null);
  useEffect(() => {
    if (!isOpen) {
      setFormData(initialFormState);
      setError("");
    }
  }, [isOpen]);
  if (!isOpen) return null;
  const handleInputChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({
      ...prev,
      [name]: value
    }));
  };
  const handleImageChange = (e) => {
    const file = e.target.files?.[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onloadend = () => {
      setFormData((prev) => ({
        ...prev,
        logo: reader.result
      }));
    };
    reader.readAsDataURL(file);
  };
  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!formData.brandName.trim()) {
      setError("Please complete all required fields.");
      return;
    }
    setSubmitting(true);
    setError("");
    try {
      const response = await fetch("http://localhost:5000/api/brand-partners", {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          ...formData,
          brandName: formData.brandName.trim()
        })
      });
      const data = await response.json();
      if (!response.ok || !data.success) {
        throw new Error(data.message || "Failed to add brand partner.");
      }
      onSuccess?.(data.data);
      onClose();
    } catch (err) {
      console.error("Failed to add brand partner:", err);
      setError(err.message || "Failed to add brand partner.");
    } finally {
      setSubmitting(false);
    }
  };
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-10001 flex items-center
justify-center p-4 bg-black/60 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement("div", { className:
`rounded-3xl w-full max-w-lg relative shadow-2xl overflow-hidden ${theme === "dark" ? "bg-[#1E1B18]" : "bg-
white"}` }, /* @__PURE__ */ React.createElement(
    "button",
```

```
    {
      onClick: onClose,
      className: "absolute top-5 right-5 text-gray-400 hover:text-gray-600 transition-colors",
      "aria-label": "Close add brand partner modal"
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
  ), /* @__PURE__ */ React.createElement("form", { onSubmit: handleSubmit, className: "p-8 pt-10 space-y-6" },
/* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement("div", { className: "w-10 h-10 rounded-xl flex items-center justify-center", style:
{ background: "#E1D5CB" } }, /* @__PURE__ */ React.createElement(FaUserFriends, { className: "text-
[#76462B]" })), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("h2",
{ className: `text-2xl font-bold ${theme === "dark" ? "text-white" : "text-gray-900"}` }, "Add Brand
Partner"), /* @__PURE__ */ React.createElement("p", { className: `text-sm ${theme === "dark" ? "text-gray-
400" : "text-gray-500"}` }, "Create a new brand partner profile"))), /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `w-40 h-40 mx-auto rounded-full flex items-center justify-center cursor-pointer border
border-dashed overflow-hidden relative ${theme === "dark" ? "bg-linear-to-b from-[#2A2724] to-[#1E1B18]
border-gray-600 hover:border-[#AD7F65]" : "bg-linear-to-b from-gray-50 to-gray-200 border-gray-300
hover:border-[#AD7F65]"}`,
      onClick: () => fileInputRef.current?.click()
    },
    formData.logo ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement("img", { src: formData.logo, alt: "Brand logo preview", className: "w-full h-full object-
cover" }), /* @__PURE__ */ React.createElement("div", { className: "absolute bottom-4 right-4 w-14 h-14
rounded-full bg-white/95 flex items-center justify-center shadow-md border border-gray-200" }, /* @__PURE__
*/ React.createElement("img", { src: cameraIcon, alt: "Camera", className: "w-7 h-7 opacity-80" }))) : /*
@__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("img", { src:
circleIcon, alt: "Circle background", className: "w-full h-full object-cover" }), /* @__PURE__ */
React.createElement("div", { className: "absolute bottom-4 right-4 w-14 h-14 rounded-full bg-white/95 flex
items-center justify-center shadow-md border border-gray-200" }, /* @__PURE__ */ React.createElement("img",
{ src: cameraIcon, alt: "Camera", className: "w-7 h-7 opacity-80" })))
  ), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "file",
      ref: fileInputRef,
      accept: "image/*",
      onChange: handleImageChange,
      className: "hidden"
    }
  ), /* @__PURE__ */ React.createElement("div", { className: `p-5 ${theme === "dark" ? "text-white" : "text-
gray-900"}` }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-2 mb-4" }, /*
@__PURE__ */ React.createElement("span", { className: "text-gray font-semibold text-sm" }, "Brand Partner
Information")), /* @__PURE__ */ React.createElement("div", { className: "space-y-4" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: "text-xs font-bold
uppercase tracking-wide text-gray-500 mb-1 block" }, "Brand ", /* @__PURE__ */ React.createElement("span",
{ className: "text-red-500" }, "*")), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      name: "brandName",
      placeholder: "e.g., John Doe",
      value: formData.brandName,
      onChange: handleInputChange,
      required: true,
      className: `w-full px-3 py-2 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" : "bg-white
border-gray-200 text-gray-900"}`
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Email"), /* @__PURE__
*/ React.createElement(
    "input",
    {
      type: "email",
      name: "email",
      placeholder: "yourname12345@gmail.com",
      value: formData.email,
      onChange: handleInputChange,
      className: `w-full px-3 py-2 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" : "bg-white
border-gray-200 text-gray-900"}`
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Contact No."), /*
@__PURE__ */ React.createElement(
```

```
    "input",
    {
      type: "text",
      name: "contactNumber",
      placeholder: "+63",
      value: formData.contactNumber,
      onChange: handleInputChange,
      className: `w-full px-3 py-2 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" : "bg-white
border-gray-200 text-gray-900"}`
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Contact Person"), /*
@__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      name: "contactPerson",
      placeholder: "e.g., John Doe",
      value: formData.contactPerson,
      onChange: handleInputChange,
      className: `w-full px-3 py-2 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" : "bg-white
border-gray-200 text-gray-900"}`
    }
  )))), error && /* @__PURE__ */ React.createElement("div", { className: "text-sm text-red-600 bg-red-50
border border-red-100 rounded-lg px-3 py-2" }, error), /* @__PURE__ */ React.createElement("div", { className:
"flex justify-end gap-3" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: onClose,
      className: `px-5 py-2 rounded-[15px] text-sm font-medium transition-colors ${theme === "dark" ? "bg-
gray-700 text-gray-300 hover:bg-gray-600" : "bg-gray-100 text-gray-700 hover:bg-gray-200"}`
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "submit",
      disabled: submitting,
      className: "px-6 py-2 rounded-[15px] text-white text-sm font-semibold shadow-lg disabled:opacity-60",
      style: { background: "linear-gradient(135deg, #1DB954 0%, #0A8A3E 100%)" }
    },
    submitting ? "Adding..." : "Add Brand"
  )))));
};
export default AddBrandPartnerModal;
```

FILE: pos/frontend/src/components/owner/AddDiscountModal.jsx
==========

```
import { useCallback, useEffect, useMemo, useState } from "react";
import { FaBox, FaCheck, FaSearch, FaTimes } from "react-icons/fa";
import { API_ENDPOINTS } from "../../config/api";
import { useTheme } from "../../context/ThemeContext";
const AddDiscountModal = ({ isOpen, onClose, onAdd, onEdit, discountToEdit }) => {
  const { theme } = useTheme();
  const [formData, setFormData] = useState({
    discountName: "",
    discountCode: "",
    discountType: "percentage",
    discountValue: "",
    appliesTo: "all",
    category: "",
    subCategory: "",
    selectedProducts: [],
    validFrom: "",
    validUntil: "",
    noExpiration: false,
    minPurchaseAmount: "",
    usageLimit: "",
    description: ""
  });
  const [showProductPicker, setShowProductPicker] = useState(false);
  const [allProducts, setAllProducts] = useState([]);
```

```
  const [productsLoading, setProductsLoading] = useState(false);
  const [productSearch, setProductSearch] = useState("");
  const [productCategoryFilter, setProductCategoryFilter] = useState("All");
  const [tempSelectedProducts, setTempSelectedProducts] = useState([]);
  const [categoryOptions, setCategoryOptions] = useState([]);
  const fetchProducts = useCallback(async () => {
    setProductsLoading(true);
    try {
      const response = await fetch(API_ENDPOINTS.products);
      const data = await response.json();
      if (data.success && Array.isArray(data.data)) {
        setAllProducts(data.data);
      }
    } catch (error) {
      console.error("Failed to fetch products:", error);
    } finally {
      setProductsLoading(false);
    }
  }, []);
  const fetchCategories = useCallback(async () => {
    try {
      const response = await fetch(API_ENDPOINTS.categories);
      const data = await response.json();
      if (data.success && Array.isArray(data.data)) {
        setCategoryOptions(
          data.data.filter(
            (c) => c && typeof c?.name === "string" && c.name.trim() && c.name !== "All"
          )
        );
      }
    } catch (error) {
      console.error("Failed to fetch categories:", error);
    }
  }, []);
  const categoryStructure = {
    "Apparel - Men": ["Tops", "Bottoms", "Outerwear"],
    "Apparel - Women": ["Tops", "Bottoms", "Dresses", "Outerwear"],
    "Apparel - Kids": ["Tops", "Bottoms", "Dresses", "Outerwear"],
    "Apparel - Unisex": ["Tops", "Bottoms", "Dresses", "Outerwear"],
    "Foods": ["Beverages", "Snacks", "Meals", "Desserts", "Ingredients", "Other"],
    "Makeup": ["Face", "Eyes", "Lips", "Nails", "SkinCare", "Others"],
    "Accessories": ["Jewelry", "Bags", "Head Wear", "Glasses/Sunglasses", "Others"],
    "Shoes": ["Sneakers", "Boots", "Sandals", "Others"],
    "Essentials": ["Daily Essentials", "Personal Care", "Home Essentials", "Others"],
    "Others": ["Others"]
  };
  const defaultParentCategories = Object.keys(categoryStructure);
  const allKnownDefaultSubs = new Set(Object.values(categoryStructure).flat());
  const legacyParentCategories = ["Apparel", "Shoes", "Foods", "Accessories", "Makeup", "Head Wear",
"Essentials"];
  const categories = useMemo(() => {
    const set = /* @__PURE__ */ new Set();
    defaultParentCategories.forEach((cat) => set.add(cat));
    categoryOptions.forEach((c) => {
      const name = String(c?.name || "").trim();
      if (!name || name === "All" || name === "Others") return;
      if (defaultParentCategories.includes(name)) return;
      if (allKnownDefaultSubs.has(name)) return;
      if (legacyParentCategories.includes(name)) return;
      set.add(name);
    });
    allProducts.forEach((p) => {
      const cat = String(p?.category || "").trim();
      if (!cat) return;
      if (allKnownDefaultSubs.has(cat)) return;
      set.add(cat);
    });
    if (discountToEdit?.category) set.add(String(discountToEdit.category).trim());
    if (formData.category) set.add(String(formData.category).trim());
    return Array.from(set).filter(Boolean).sort((a, b) => a.localeCompare(b));
  }, [allProducts, categoryOptions, discountToEdit, formData.category]);
  const subCategoryOptions = useMemo(() => {
    const parentCat = String(formData.category || "").trim();
    if (!parentCat) return [];
    const defaultSubs = categoryStructure[parentCat] || [];
    const mappedCustomSubCategories = categoryOptions.filter((c) => c?.parentCategory === parentCat).map((c)
=> c?.name).filter((name) => typeof name === "string" && name.trim() && name !== "All");
```

```
    const orphanCustomSubCategories = categoryOptions.map((c) => c?.name).filter((name) => {
      if (typeof name !== "string") return false;
      const trimmed = name.trim();
      if (!trimmed || trimmed === "All" || trimmed === "Others") return false;
      if (defaultParentCategories.includes(trimmed)) return false;
      if (allKnownDefaultSubs.has(trimmed)) return false;
      if (legacyParentCategories.includes(trimmed)) return false;
      return true;
    });
    const subs = [...defaultSubs, ...mappedCustomSubCategories, ...orphanCustomSubCategories];
    const selectedSub = String(formData.subCategory || "").trim();
    if (selectedSub && selectedSub !== "__add_new__" && !subs.includes(selectedSub)) {
      subs.push(selectedSub);
    }
    const editSub = String(discountToEdit?.subCategory || "").trim();
    const editCat = String(discountToEdit?.category || "").trim();
    if (editSub && editCat === parentCat && !subs.includes(editSub)) {
      subs.push(editSub);
    }
    return [...new Set(subs)].filter(Boolean).sort((a, b) => a.localeCompare(b));
  }, [categoryOptions, defaultParentCategories, formData.category, formData.subCategory, discountToEdit,
allKnownDefaultSubs, legacyParentCategories]);
  useEffect(() => {
    if (isOpen) {
      fetchProducts();
      fetchCategories();
      if (discountToEdit) {
        setFormData({
          discountName: discountToEdit.title || "",
          discountCode: discountToEdit.discountCode || "",
          discountType: discountToEdit.discountType || "percentage",
          discountValue: discountToEdit.discountValue || "",
          appliesTo: discountToEdit.appliesTo || "all",
          category: discountToEdit.category || "",
          subCategory: discountToEdit.subCategory || "",
          selectedProducts: discountToEdit.selectedProducts || [],
          validFrom: discountToEdit.validFrom && discountToEdit.validFrom !== "Permanent" ? new
Date(discountToEdit.validFrom).toISOString().split("T")[0] : "",
          validUntil: discountToEdit.validTo && discountToEdit.validTo !== "Permanent" ? new
Date(discountToEdit.validTo).toISOString().split("T")[0] : "",
          noExpiration: discountToEdit.noExpiration || false,
          minPurchaseAmount: discountToEdit.minPurchaseAmount || "",
          usageLimit: discountToEdit.usageLimit || "",
          description: discountToEdit.description || ""
        });
      } else {
        setFormData({
          discountName: "",
          discountCode: "",
          discountType: "percentage",
          discountValue: "",
          appliesTo: "all",
          category: "",
          subCategory: "",
          selectedProducts: [],
          validFrom: "",
          validUntil: "",
          noExpiration: false,
          minPurchaseAmount: "",
          usageLimit: "",
          description: ""
        });
      }
    }
  }, [isOpen, discountToEdit, fetchProducts, fetchCategories]);
  if (!isOpen) return null;
  const handleSubmit = (e) => {
    e.preventDefault();
    if (formData.appliesTo === "category" && !formData.category) {
      alert("Please select a category");
      return;
    }
    if (formData.appliesTo === "products" && formData.selectedProducts.length === 0) {
      alert("Please select at least one product");
      return;
    }
    if (discountToEdit) {
```

```
      onEdit(discountToEdit._id, formData);
    } else {
      onAdd(formData);
    }
    onClose();
  };
  const handleChange = (e) => {
    const { name, value, type, checked } = e.target;
    setFormData((prev) => ({
      ...prev,
      ...name === "category" ? { subCategory: "" } : {},
      [name]: type === "checkbox" ? checked : value
    }));
  };
  const openProductPicker = () => {
    setTempSelectedProducts([...formData.selectedProducts]);
    setProductSearch("");
    setProductCategoryFilter("All");
    setShowProductPicker(true);
  };
  const toggleProductSelection = (product) => {
    setTempSelectedProducts((prev) => {
      const exists = prev.find((p) => p._id === product._id);
      if (exists) {
        return prev.filter((p) => p._id !== product._id);
      }
      return [...prev, {
        _id: product._id,
        itemName: product.itemName,
        sku: product.sku,
        itemImage: product.itemImage,
        category: product.category,
        brandName: product.brandName
      }];
    });
  };
  const confirmProductSelection = () => {
    setFormData((prev) => ({
      ...prev,
      selectedProducts: tempSelectedProducts
    }));
    setShowProductPicker(false);
  };
  const removeSelectedProduct = (productId) => {
    setFormData((prev) => ({
      ...prev,
      selectedProducts: prev.selectedProducts.filter((p) => p._id !== productId)
    }));
  };
  const isDark = theme === "dark";
  const inputClass = `w-full px-4 py-2 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
focus:border-transparent ${isDark ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" : "bg-
white border-gray-300 text-gray-900"}`;
  const labelClass = `text-xs font-bold uppercase tracking-wide mb-1 block ${isDark ? "text-gray-400" :
"text-gray-500"}`;
  return /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("div",
{ className: "fixed inset-0 flex items-center justify-center z-[10002] p-4 backdrop-blur-sm" }, /* @__PURE__
*/ React.createElement("div", { className: `rounded-2xl w-full max-w-5xl max-h-[90vh] overflow-hidden shadow-
2xl flex flex-col ${isDark ? "bg-[#2A2724]" : "bg-white"}` }, /* @__PURE__ */ React.createElement("div",
{ className: `px-6 py-4 border-b flex items-center justify-between ${isDark ? "border-gray-700" : "border-
gray-200"}` }, /* @__PURE__ */ React.createElement("h2", { className: `text-2xl font-bold ${isDark ? "text-
white" : "text-black"}` }, discountToEdit ? "Edit Discount" : "Create New Discount"), /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: onClose,
      className: "text-gray-400 hover:text-gray-600 transition-colors"
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
  )), /* @__PURE__ */ React.createElement("form", { onSubmit: handleSubmit, className: "flex-1 overflow-y-
auto" }, /* @__PURE__ */ React.createElement("div", { className: "p-6" }, /* @__PURE__ */
React.createElement("div", { className: "grid grid-cols-2 gap-6" }, /* @__PURE__ */ React.createElement("div",
null, /* @__PURE__ */ React.createElement("h3", { className: `text-lg font-bold mb-4 ${isDark ? "text-white" :
"text-black"}` }, "Basic Info"), /* @__PURE__ */ React.createElement("div", { className: "space-y-4" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className:
labelClass }, "Discount Name ", /* @__PURE__ */ React.createElement("span", { className: "text-red-500" },
"*")), /* @__PURE__ */ React.createElement(
```

```
    "input",
    {
      type: "text",
      name: "discountName",
      value: formData.discountName,
      onChange: handleChange,
      placeholder: "e.g ANNIVERSARY SALE",
      className: inputClass,
      required: true
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: labelClass }, /* @__PURE__ */ React.createElement("span", null, "Discount Code"), /* @__PURE__
*/ React.createElement("span", { className: "ml-2 text-[10px] font-normal normal-case tracking-normal text-
gray-400" }, "Optional")), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      name: "discountCode",
      value: formData.discountCode,
      onChange: handleChange,
      placeholder: "e.g DRESS10",
      className: inputClass
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: labelClass }, "Discount Type ", /* @__PURE__ */ React.createElement("span", { className: "text-
red-500" }, "*")), /* @__PURE__ */ React.createElement("div", { className: "flex gap-6" }, /* @__PURE__ */
React.createElement("label", { className: "flex items-center gap-2 cursor-pointer" }, /* @__PURE__ */
React.createElement(
    "input",
    {
      type: "radio",
      name: "discountType",
      value: "percentage",
      checked: formData.discountType === "percentage",
      onChange: handleChange,
      className: "w-4 h-4 text-[#AD7F65] focus:ring-[#AD7F65]"
    }
  ), /* @__PURE__ */ React.createElement("span", { className: `text-sm ${isDark ? "text-gray-300" : "text-
gray-700"}` }, "Percentage")), /* @__PURE__ */ React.createElement("label", { className: "flex items-center
gap-2 cursor-pointer" }, /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "radio",
      name: "discountType",
      value: "fixed",
      checked: formData.discountType === "fixed",
      onChange: handleChange,
      className: "w-4 h-4 text-[#AD7F65] focus:ring-[#AD7F65]"
    }
  ), /* @__PURE__ */ React.createElement("span", { className: `text-sm ${isDark ? "text-gray-300" : "text-
gray-700"}` }, "Fixed Amount")))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("label", { className: labelClass }, "Discount Value ", /* @__PURE__ */
React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */ React.createElement("div",
{ className: "flex items-center gap-2" }, /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "number",
      name: "discountValue",
      value: formData.discountValue,
      onChange: handleChange,
      placeholder: "e.g 15",
      className: `flex-1 px-4 py-2 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
focus:border-transparent ${isDark ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" : "bg-
white border-gray-300 text-gray-900"}`,
      required: true
    }
  ), /* @__PURE__ */ React.createElement("span", { className: `text-sm font-medium ${isDark ? "text-gray-
300" : "text-black"}` }, formData.discountType === "percentage" ? "% OFF" : "\u20B1 OFF"))), /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: labelClass },
"Applies to ", /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__
*/ React.createElement("div", { className: "space-y-2" }, /* @__PURE__ */ React.createElement("label",
{ className: "flex items-center gap-2 cursor-pointer" }, /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "radio",
      name: "appliesTo",
      value: "all",
```

```
      checked: formData.appliesTo === "all",
      onChange: handleChange,
      className: "w-4 h-4 text-[#AD7F65] focus:ring-[#AD7F65]"
    }
  ), /* @__PURE__ */ React.createElement("span", { className: `text-sm ${isDark ? "text-gray-300" : "text-
gray-700"}` }, "All Products")), /* @__PURE__ */ React.createElement("label", { className: "flex items-center
gap-2 cursor-pointer" }, /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "radio",
      name: "appliesTo",
      value: "category",
      checked: formData.appliesTo === "category",
      onChange: handleChange,
      className: "w-4 h-4 text-[#AD7F65] focus:ring-[#AD7F65]"
    }
  ), /* @__PURE__ */ React.createElement("span", { className: `text-sm ${isDark ? "text-gray-300" : "text-
gray-700"}` }, "Specific Category")), /* @__PURE__ */ React.createElement("label", { className: "flex items-
center gap-2 cursor-pointer" }, /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "radio",
      name: "appliesTo",
      value: "products",
      checked: formData.appliesTo === "products",
      onChange: handleChange,
      className: "w-4 h-4 text-[#AD7F65] focus:ring-[#AD7F65]"
    }
  ), /* @__PURE__ */ React.createElement("span", { className: `text-sm ${isDark ? "text-gray-300" : "text-
gray-700"}` }, "Specific Products"))), formData.appliesTo === "category" && /* @__PURE__ */
React.createElement("div", { className: "mt-3" }, /* @__PURE__ */ React.createElement("label", { className:
labelClass }, "Select Category ", /* @__PURE__ */ React.createElement("span", { className: "text-red-500" },
"*")), /* @__PURE__ */ React.createElement(
    "select",
    {
      name: "category",
      value: formData.category,
      onChange: handleChange,
      className: inputClass,
      required: formData.appliesTo === "category"
    },
    /* @__PURE__ */ React.createElement("option", { value: "" }, "Select a category"),
    categories.map(
      (cat) => /* @__PURE__ */ React.createElement("option", { key: cat, value: cat }, cat)
    )
  ), subCategoryOptions.length > 0 && /* @__PURE__ */ React.createElement("div", { className: "mt-3" }, /*
@__PURE__ */ React.createElement("label", { className: labelClass }, "Select Subcategory ", /* @__PURE__ */
React.createElement("span", { className: "text-[10px] font-normal normal-case tracking-normal text-gray-
400" }, "Optional")), /* @__PURE__ */ React.createElement(
    "select",
    {
      name: "subCategory",
      value: formData.subCategory || "",
      onChange: handleChange,
      className: inputClass
    },
    /* @__PURE__ */ React.createElement("option", { value: "" }, "All in selected category"),
    subCategoryOptions.map((sub) => /* @__PURE__ */ React.createElement("option", { key: sub, value: sub },
sub))
  ))), formData.appliesTo === "products" && /* @__PURE__ */ React.createElement("div", { className: "mt-3" },
/* @__PURE__ */ React.createElement("label", { className: labelClass }, "Products ", /* @__PURE__ */
React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: openProductPicker,
      className: `w-full px-4 py-3 border-2 border-dashed rounded-xl flex items-center justify-center gap-2
transition-all hover:border-[#AD7F65] ${isDark ? "border-gray-600 text-gray-300 hover:bg-[#352F2A]" :
"border-gray-300 text-gray-600 hover:bg-[#FDF7F1]"}`
    },
    /* @__PURE__ */ React.createElement(FaBox, { className: "text-[#AD7F65]" }),
    /* @__PURE__ */ React.createElement("span", { className: "font-medium" }, "Select Products",
formData.selectedProducts.length > 0 && /* @__PURE__ */ React.createElement("span", { className: "ml-2 px-2
py-0.5 text-xs font-bold text-white bg-[#AD7F65] rounded-full" }, formData.selectedProducts.length))
  ), formData.selectedProducts.length > 0 && /* @__PURE__ */ React.createElement("div", { className: "mt-3
flex flex-wrap gap-2 max-h-32 overflow-y-auto" }, formData.selectedProducts.map(
    (product) => /* @__PURE__ */ React.createElement(
```

```
      "span",
      {
        key: product._id,
        className: `inline-flex items-center gap-1.5 pl-2 pr-1 py-1 rounded-full text-xs font-medium
${isDark ? "bg-[#352F2A] text-gray-200 border border-gray-600" : "bg-[#FDF7F1] text-[#76462B] border border-
[#E8D5C8]"}`
      },
      product.itemImage && /* @__PURE__ */ React.createElement(
        "img",
        {
          src: product.itemImage,
          alt: "",
          className: "w-4 h-4 rounded-full object-cover"
        }
      ),
      /* @__PURE__ */ React.createElement("span", { className: "max-w-[120px] truncate" }, product.itemName),
      /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: () => removeSelectedProduct(product._id),
          className: "w-4 h-4 rounded-full flex items-center justify-center hover:bg-red-100 hover:text-red-
500 transition-colors"
        },
        /* @__PURE__ */ React.createElement(FaTimes, { className: "w-2.5 h-2.5" })
      )
    )
  )))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: labelClass }, "Validity Period ", !formData.noExpiration), /* @__PURE__ */
React.createElement("div", { className: "grid grid-cols-2 gap-3 mb-2" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: `text-xs font-
medium block mb-1 ${isDark ? "text-gray-400" : "text-gray-600"}` }, "Valid from ", !formData.noExpiration),
/* @__PURE__ */ React.createElement(
    "input",
    {
      type: "date",
      name: "validFrom",
      value: formData.validFrom,
      onChange: handleChange,
      disabled: formData.noExpiration,
      required: !formData.noExpiration,
      className: `w-full px-4 py-2 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
focus:border-transparent disabled:opacity-50 disabled:cursor-not-allowed ${isDark ? "bg-[#1E1B18] border-
gray-600 text-white" : "bg-white border-gray-300 text-gray-900 disabled:bg-gray-100"}`
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: `text-xs font-medium block mb-1 ${isDark ? "text-gray-400" : "text-gray-600"}` }, "Valid until
", !formData.noExpiration), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "date",
      name: "validUntil",
      value: formData.validUntil,
      onChange: handleChange,
      disabled: formData.noExpiration,
      required: !formData.noExpiration,
      className: `w-full px-4 py-2 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
focus:border-transparent disabled:opacity-50 disabled:cursor-not-allowed ${isDark ? "bg-[#1E1B18] border-
gray-600 text-white" : "bg-white border-gray-300 text-gray-900 disabled:bg-gray-100"}`
    }
  ))), /* @__PURE__ */ React.createElement("label", { className: "flex items-center gap-2 cursor-pointer" },
/* @__PURE__ */ React.createElement(
    "input",
    {
      type: "checkbox",
      name: "noExpiration",
      checked: formData.noExpiration,
      onChange: handleChange,
      className: "w-4 h-4 text-[#AD7F65] rounded focus:ring-[#AD7F65]"
    }
  ), /* @__PURE__ */ React.createElement("span", { className: `text-sm ${isDark ? "text-gray-300" : "text-
gray-700"}` }, "No expiration date")))))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("h3", { className: `text-lg font-bold mb-4 ${isDark ? "text-white" : "text-black"}` },
"Advanced Option"), /* @__PURE__ */ React.createElement("div", { className: "space-y-4" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: labelClass }, /*
@__PURE__ */ React.createElement("span", null, "Minimum Purchase Amount"), /* @__PURE__ */
React.createElement("span", { className: "ml-2 text-[10px] font-normal normal-case tracking-normal text-gray-
```

```
400" }, "Optional")), /* @__PURE__ */ React.createElement("div", { className: "relative" }, /* @__PURE__ */
React.createElement("span", { className: "absolute left-3 top-1/2 transform -translate-y-1/2 text-gray-600" },
"\u20B1"), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "number",
      name: "minPurchaseAmount",
      value: formData.minPurchaseAmount,
      onChange: handleChange,
      step: "0.01",
      className: `w-full pl-8 pr-4 py-2 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
focus:border-transparent ${isDark ? "bg-[#1E1B18] border-gray-600 text-white" : "border-gray-300 bg-white"}`
    }
  )), /* @__PURE__ */ React.createElement("p", { className: "text-xs text-gray-400 mt-1" }, "Customer must
spend at least this amount")), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("label", { className: labelClass }, /* @__PURE__ */ React.createElement("span", null,
"Usage Limit"), /* @__PURE__ */ React.createElement("span", { className: "ml-2 text-[10px] font-normal
normal-case tracking-normal text-gray-400" }, "Optional")), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "number",
      name: "usageLimit",
      value: formData.usageLimit,
      onChange: handleChange,
      min: "0",
      className: inputClass
    }
  ), /* @__PURE__ */ React.createElement("p", { className: "text-xs text-gray-400 mt-1" }, "Total number of
times this discount can be used")), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("label", { className: labelClass }, "Description / Notes ", /* @__PURE__ */
React.createElement("span", { className: "text-[10px] font-normal normal-case tracking-normal text-gray-
400" }, "Optional")), /* @__PURE__ */ React.createElement(
    "textarea",
    {
      name: "description",
      value: formData.description,
      onChange: handleChange,
      placeholder: "Notes about this discount...",
      rows: "4",
      className: `w-full px-4 py-2 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
focus:border-transparent resize-none ${isDark ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-
500" : "bg-white border-gray-300 text-gray-900"}`
    }
  )))))), /* @__PURE__ */ React.createElement("div", { className: `px-6 py-4 border-t flex justify-end
${isDark ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "submit",
      className: "px-8 py-3 text-white rounded-lg font-bold text-lg shadow-md hover:shadow-lg transition-all",
      style: {
        background: "linear-gradient(135deg, #10B981 0%, #059669 100%)"
      }
    },
    discountToEdit ? "Update Discount" : "Add New Discount"
  )))), showProductPicker && /* @__PURE__ */ React.createElement(
    ProductPickerModal,
    {
      isDark,
      allProducts,
      productsLoading,
      productSearch,
      setProductSearch,
      productCategoryFilter,
      setProductCategoryFilter,
      categories,
      tempSelectedProducts,
      toggleProductSelection,
      confirmProductSelection,
      onClose: () => setShowProductPicker(false)
    }
  ));
};
const ProductPickerModal = ({
  isDark,
  allProducts,
  productsLoading,
  productSearch,
```

```
  setProductSearch,
  productCategoryFilter,
  setProductCategoryFilter,
  categories,
  tempSelectedProducts,
  toggleProductSelection,
  confirmProductSelection,
  onClose
}) => {
  const filteredProducts = useMemo(() => {
    return allProducts.filter((product) => {
      const matchesSearch = !productSearch ||
product.itemName?.toLowerCase().includes(productSearch.toLowerCase()) ||
product.sku?.toLowerCase().includes(productSearch.toLowerCase()) ||
product.brandName?.toLowerCase().includes(productSearch.toLowerCase());
      const matchesCategory = productCategoryFilter === "All" || product.category === productCategoryFilter;
      return matchesSearch && matchesCategory;
    });
  }, [allProducts, productSearch, productCategoryFilter]);
  const isSelected = useCallback((productId) => {
    return tempSelectedProducts.some((p) => p._id === productId);
  }, [tempSelectedProducts]);
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10003] p-4 backdrop-blur-sm bg-black/30" }, /* @__PURE__ */ React.createElement("div", { className:
`rounded-2xl w-full max-w-4xl max-h-[85vh] overflow-hidden shadow-2xl flex flex-col ${isDark ? "bg-
[#2A2724]" : "bg-white"}` }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "h-2",
      style: { background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)" }
    }
  ), /* @__PURE__ */ React.createElement("div", { className: `px-6 py-4 border-b flex items-center justify-
between ${isDark ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */ React.createElement("div", null,
/* @__PURE__ */ React.createElement("h3", { className: `text-xl font-bold ${isDark ? "text-white" : "text-
black"}` }, "Select Products"), /* @__PURE__ */ React.createElement("p", { className: `text-sm mt-0.5
${isDark ? "text-gray-400" : "text-gray-500"}` }, tempSelectedProducts.length, " product",
tempSelectedProducts.length !== 1 ? "s" : "", " selected")), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onClose,
      className: "text-gray-400 hover:text-gray-600 transition-colors"
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
  )), /* @__PURE__ */ React.createElement("div", { className: `px-6 py-3 flex gap-3 border-b ${isDark ?
"border-gray-700 bg-[#1E1B18]" : "border-gray-100 bg-gray-50"}` }, /* @__PURE__ */ React.createElement("div",
{ className: "flex-1 relative" }, /* @__PURE__ */ React.createElement(FaSearch, { className: "absolute left-3
top-1/2 -translate-y-1/2 text-gray-400 w-4 h-4" }), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      value: productSearch,
      onChange: (e) => setProductSearch(e.target.value),
      placeholder: "Search by name, SKU, or brand...",
      className: `w-full pl-10 pr-4 py-2.5 border rounded-xl focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] focus:border-transparent text-sm ${isDark ? "bg-[#2A2724] border-gray-600 text-white placeholder-
gray-500" : "bg-white border-gray-300 text-gray-900"}`,
      autoFocus: true
    }
  )), /* @__PURE__ */ React.createElement(
    "select",
    {
      value: productCategoryFilter,
      onChange: (e) => setProductCategoryFilter(e.target.value),
      className: `px-4 py-2.5 border rounded-xl focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
focus:border-transparent text-sm min-w-[150px] ${isDark ? "bg-[#2A2724] border-gray-600 text-white" : "bg-
white border-gray-300 text-gray-900"}`
    },
    /* @__PURE__ */ React.createElement("option", { value: "All" }, "All Categories"),
    categories.map(
      (cat) => /* @__PURE__ */ React.createElement("option", { key: cat, value: cat }, cat)
    )
  )), /* @__PURE__ */ React.createElement("div", { className: "flex-1 overflow-y-auto px-6 py-4" },
productsLoading ? /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-center
py-16" }, /* @__PURE__ */ React.createElement("div", { className: "animate-spin rounded-full h-8 w-8 border-2
border-[#AD7F65] border-t-transparent" }), /* @__PURE__ */ React.createElement("span", { className: `ml-3
text-sm ${isDark ? "text-gray-400" : "text-gray-500"}` }, "Loading products...")) : filteredProducts.length
=== 0 ? /* @__PURE__ */ React.createElement("div", { className: "flex flex-col items-center justify-center
```

```
py-16" }, /* @__PURE__ */ React.createElement(FaBox, { className: `w-12 h-12 mb-3 ${isDark ? "text-gray-600" :
"text-gray-300"}` }), /* @__PURE__ */ React.createElement("p", { className: `text-sm ${isDark ? "text-gray-
400" : "text-gray-500"}` }, "No products found")) : /* @__PURE__ */ React.createElement("div", { className:
"grid grid-cols-1 gap-2" }, filteredProducts.map((product) => {
    const selected = isSelected(product._id);
    return /* @__PURE__ */ React.createElement(
      "button",
      {
        key: product._id,
        type: "button",
        onClick: () => toggleProductSelection(product),
        className: `w-full flex items-center gap-4 px-4 py-3 rounded-xl border transition-all text-left
${selected ? isDark ? "border-[#AD7F65] bg-[#352F2A] shadow-sm" : "border-[#AD7F65] bg-[#FDF7F1] shadow-sm" :
isDark ? "border-gray-700 hover:border-gray-500 hover:bg-[#352F2A]" : "border-gray-200 hover:border-gray-300
hover:bg-gray-50"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: `w-5 h-5 rounded flex items-center justify-
center flex-shrink-0 border-2 transition-colors ${selected ? "bg-[#AD7F65] border-[#AD7F65]" : isDark ?
"border-gray-500" : "border-gray-300"}` }, selected && /* @__PURE__ */ React.createElement(FaCheck,
{ className: "w-3 h-3 text-white" })),
      /* @__PURE__ */ React.createElement("div", { className: `w-10 h-10 rounded-lg flex-shrink-0 overflow-
hidden ${isDark ? "bg-[#1E1B18]" : "bg-gray-100"}` }, product.itemImage ? /* @__PURE__ */
React.createElement("img", { src: product.itemImage, alt: "", className: "w-full h-full object-cover" }) : /*
@__PURE__ */ React.createElement("div", { className: "w-full h-full flex items-center justify-center" }, /*
@__PURE__ */ React.createElement(FaBox, { className: `w-4 h-4 ${isDark ? "text-gray-600" : "text-gray-
400"}` }))),
      /* @__PURE__ */ React.createElement("div", { className: "flex-1 min-w-0" }, /* @__PURE__ */
React.createElement("p", { className: `text-sm font-semibold truncate ${isDark ? "text-white" : "text-gray-
900"}` }, product.itemName), /* @__PURE__ */ React.createElement("p", { className: `text-xs truncate mt-0.5
${isDark ? "text-gray-400" : "text-gray-500"}` }, product.sku || "No SKU", product.brandName ? ` \xB7
${product.brandName}` : "")),
      /* @__PURE__ */ React.createElement("span", { className: `text-xs px-2.5 py-1 rounded-full flex-shrink-
0 ${isDark ? "bg-[#1E1B18] text-gray-400" : "bg-gray-100 text-gray-500"}` }, product.category ||
"Uncategorized")
    );
  }))), /* @__PURE__ */ React.createElement("div", { className: `px-6 py-4 border-t flex items-center
justify-between ${isDark ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */ React.createElement("p",
{ className: `text-sm ${isDark ? "text-gray-400" : "text-gray-500"}` }, filteredProducts.length, " product",
filteredProducts.length !== 1 ? "s" : "", " shown"), /* @__PURE__ */ React.createElement("div", { className:
"flex gap-3" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: onClose,
      className: `px-5 py-2.5 rounded-lg text-sm font-medium border transition-colors ${isDark ? "border-
gray-600 text-gray-300 hover:bg-[#352F2A]" : "border-gray-300 text-gray-600 hover:bg-gray-50"}`
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: confirmProductSelection,
      className: "px-5 py-2.5 rounded-lg text-sm font-bold text-white transition-all hover:shadow-lg",
      style: {
        background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)"
      }
    },
    "Confirm Selection (",
    tempSelectedProducts.length,
    ")"
  )))));
};
export default AddDiscountModal;
```

FILE: pos/frontend/src/components/owner/AddEmployeeModal.jsx
==========

```
import { useEffect, useRef, useState } from "react";
import { FaCamera, FaCheck, FaSpinner, FaTimes, FaUserPlus } from "react-icons/fa";
import cameraIcon from "../../assets/owner/camera.svg";
import circleIcon from "../../assets/owner/circle.svg";
const getDisplayDate = () => (/* @__PURE__ */ new Date()).toLocaleDateString("en-US", {
  month: "long",
  day: "numeric",
  year: "numeric"
```

```
});
const getTodayISO = () => (/* @__PURE__ */ new Date()).toISOString().split("T")[0];
const buildEmployeeName = (firstName, middleInitial, lastName) => {
  const parts = [firstName, middleInitial, lastName].map((p) => String(p ?? "").trim()).filter(Boolean);
  return parts.join(" ").trim();
};
const ROLE_ACCESS_RECOMMENDATIONS = {
  Cashier: {
    posTerminal: true,
    inventory: false,
    viewTransactions: true,
    generateReports: false
  },
  Manager: {
    posTerminal: true,
    inventory: true,
    viewTransactions: true,
    generateReports: true
  },
  "Sub-Cashier": {
    posTerminal: true,
    inventory: false,
    viewTransactions: false,
    generateReports: false
  },
  "Stock Manager": {
    posTerminal: false,
    inventory: true,
    viewTransactions: true,
    generateReports: false
  }
};
const AddEmployeeModal = ({ isOpen, onClose, onEmployeeAdded, onEmployeeCreated }) => {
  const [formData, setFormData] = useState({
    firstName: "",
    middleInitial: "",
    lastName: "",
    contactNo: "",
    email: "",
    role: "Cashier",
    dateCreated: getDisplayDate(),
    dateJoined: getTodayISO()
  });
  const [accessControl, setAccessControl] = useState({
    posTerminal: true,
    inventory: false,
    viewTransactions: true,
    generateReports: false
  });
  const [profilePreview, setProfilePreview] = useState("");
  const [loading, setLoading] = useState(false);
  const [emailLoading, setEmailLoading] = useState(false);
  const [error, setError] = useState("");
  const fileInputRef = useRef(null);
  const [verificationCode, setVerificationCode] = useState("");
  const [isCodeSent, setIsCodeSent] = useState(false);
  const [isEmailVerified, setIsEmailVerified] = useState(false);
  const [verificationTimer, setVerificationTimer] = useState(0);
  const [showCodeSentCheck, setShowCodeSentCheck] = useState(false);
  const generateRandomPin = () => {
    return Math.floor(1e5 + Math.random() * 9e5).toString();
  };
  useEffect(() => {
    if (!isOpen) {
      setFormData({
        firstName: "",
        middleInitial: "",
        lastName: "",
        contactNo: "",
        email: "",
        role: "Cashier",
        dateCreated: getDisplayDate(),
        dateJoined: getTodayISO()
      });
      setAccessControl({
        posTerminal: true,
        inventory: false,
```

```
        viewTransactions: true,
        generateReports: false
      });
      setProfilePreview("");
      setVerificationCode("");
      setIsCodeSent(false);
      setIsEmailVerified(false);
      setVerificationTimer(0);
      setError("");
    }
  }, [isOpen]);
  const handleImageSelect = () => {
    fileInputRef.current?.click();
  };
  const handleImageChange = (e) => {
    const file = e.target.files?.[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onloadend = () => {
      setProfilePreview(reader.result);
    };
    reader.readAsDataURL(file);
  };
  const handleInputChange = (e) => {
    const { name, value } = e.target;
    const normalizedValue = name === "middleInitial" ? value.replace(/[^a-zA-Z]/g, "").slice(0,
1).toUpperCase() : value;
    setFormData((prev) => ({
      ...prev,
      [name]: normalizedValue
    }));
    if (name === "role") {
      const recommendedPermissions = ROLE_ACCESS_RECOMMENDATIONS[value];
      if (recommendedPermissions) {
        setAccessControl((prev) => ({
          ...prev,
          ...recommendedPermissions
        }));
      }
    }
  };
  const handleAccessControlToggle = (key) => {
    setAccessControl((prev) => ({
      ...prev,
      [key]: !prev[key]
    }));
  };
  const handleSendCode = async () => {
    if (!formData.email) {
      setError("Please enter an email address first");
      return;
    }
    setEmailLoading(true);
    try {
      const response = await fetch("http://localhost:5000/api/verification/send-code", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ email: formData.email })
      });
      const data = await response.json();
      if (data.success) {
        setShowCodeSentCheck(true);
        setTimeout(() => {
          setIsCodeSent(true);
          setShowCodeSentCheck(false);
          setVerificationTimer(60);
        }, 1e3);
        setError("");
        const timer = setInterval(() => {
          setVerificationTimer((prev) => {
            if (prev <= 1) {
              clearInterval(timer);
              return 0;
            }
            return prev - 1;
          });
        }, 1e3);
```

```
      } else {
        setError(data.message || "Failed to send verification code");
      }
    } catch (err) {
      setError("Failed to send verification code");
    } finally {
      setEmailLoading(false);
    }
  };
  const handleVerifyCode = async () => {
    if (!verificationCode) {
      setError("Please enter the verification code");
      return;
    }
    setEmailLoading(true);
    try {
      const response = await fetch("http://localhost:5000/api/verification/verify-code", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          email: formData.email,
          code: verificationCode
        })
      });
      const data = await response.json();
      if (data.success) {
        setIsEmailVerified(true);
        setIsCodeSent(false);
        setError("");
      } else {
        setError(data.message || "Invalid verification code");
      }
    } catch (err) {
      setError("Verification failed");
    } finally {
      setEmailLoading(false);
    }
  };
  const handleAddEmployee = async () => {
    if (!formData.firstName || !formData.lastName || !formData.email || !formData.contactNo
|| !formData.dateJoined) {
      setError("Please fill in all required fields");
      return;
    }
    if (!isEmailVerified) {
      setError("Please verify the email address first");
      return;
    }
    setError("");
    setLoading(true);
    const tempPin = generateRandomPin();
    try {
      const response = await fetch("http://localhost:5000/api/employees", {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          name: buildEmployeeName(
            formData.firstName.trim(),
            formData.middleInitial,
            formData.lastName.trim()
          ),
          firstName: formData.firstName.trim(),
          middleInitial: formData.middleInitial,
          lastName: formData.lastName.trim(),
          contactNo: formData.contactNo,
          email: formData.email,
          role: formData.role,
          pin: tempPin,
          dateJoined: new Date(formData.dateCreated),
          dateJoinedActual: new Date(formData.dateJoined),
          profileImage: profilePreview,
          permissions: accessControl,
          requiresPinReset: true
        })
      });
```

```
      const data = await response.json();
      if (data.success) {
        const employeeName = buildEmployeeName(
          formData.firstName.trim(),
          formData.middleInitial,
          formData.lastName.trim()
        );
        setFormData({
          firstName: "",
          middleInitial: "",
          lastName: "",
          contactNo: "",
          email: "",
          role: "Cashier",
          dateCreated: getDisplayDate(),
          dateJoined: getTodayISO()
        });
        setAccessControl({
          posTerminal: true,
          inventory: false,
          viewTransactions: true,
          generateReports: false
        });
        setProfilePreview("");
        onClose();
        if (onEmployeeCreated) {
          onEmployeeCreated(employeeName, tempPin);
        }
        if (onEmployeeAdded) {
          onEmployeeAdded();
        }
      } else {
        setError(data.message || "Failed to add employee");
      }
    } catch (error2) {
      console.error("Error adding employee:", error2);
      setError("Failed to connect to server. Please make sure the backend is running.");
    } finally {
      setLoading(false);
    }
  };
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10002] p-4 backdrop-blur-sm bg-black/20" }, /* @__PURE__ */ React.createElement("div", { className:
"bg-white w-full max-w-xl relative shadow-2xl overflow-hidden animate-fadeIn", style: { borderRadius:
"24px" } }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between px-8
py-6 border-b border-gray-100" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center
gap-3" }, /* @__PURE__ */ React.createElement("div", { className: "w-10 h-10 rounded-xl bg-[#AD7F65] flex
items-center justify-center text-white" }, /* @__PURE__ */ React.createElement(FaUserPlus, { className: "w-5
h-5" })), /* @__PURE__ */ React.createElement("h2", { className: "text-lg font-bold text-gray-800" }, "Add
Employee Profile")), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onClose,
      className: "text-gray-400 hover:text-gray-600 transition-colors p-2 hover:bg-gray-100 rounded-full"
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
  )), /* @__PURE__ */ React.createElement("div", { className: "p-8 max-h-[80vh] overflow-y-auto custom-
scrollbar" }, /* @__PURE__ */ React.createElement("div", { className: "flex justify-center mb-8" }, /*
@__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: handleImageSelect,
      className: "w-40 h-40 rounded-full overflow-hidden bg-gray-100 flex items-center justify-center
relative focus:outline-none group hover:opacity-90 transition-opacity"
    },
    profilePreview ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement("img", { src: profilePreview, alt: "Preview", className: "w-full h-full object-cover" }),
/* @__PURE__ */ React.createElement("div", { className: "absolute bottom-4 right-4 w-14 h-14 rounded-full bg-
white/95 flex items-center justify-center shadow-md border border-gray-200" }, /* @__PURE__ */
React.createElement("img", { src: cameraIcon, alt: "Camera", className: "w-7 h-7 opacity-80" }))) : /*
@__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("img", { src:
circleIcon, alt: "Circle background", className: "w-full h-full object-cover" }), /* @__PURE__ */
React.createElement("div", { className: "absolute bottom-4 right-4 w-14 h-14 rounded-full bg-white/95 flex
items-center justify-center shadow-md border border-gray-200" }, /* @__PURE__ */ React.createElement("img",
{ src: cameraIcon, alt: "Camera", className: "w-7 h-7 opacity-80" }))),
```

```
    /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "file",
        accept: "image/*",
        ref: fileInputRef,
        onChange: handleImageChange,
        className: "hidden"
      }
    )
  )), /* @__PURE__ */ React.createElement("h4", { className: "text-base font-bold text-gray-800 mb-4" },
"Personal Details"), /* @__PURE__ */ React.createElement("div", { className: "space-y-4 mb-8" }, /* @__PURE__
*/ React.createElement("div", { className: "grid grid-cols-2 gap-4" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: "text-xs font-bold
uppercase tracking-wide text-gray-500 mb-1 block" }, "First Name ", /* @__PURE__ */
React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      name: "firstName",
      value: formData.firstName,
      onChange: handleInputChange,
      placeholder: "First Name",
      className: "w-full px-3 py-2 border border-gray-200 rounded-lg focus:outline-none focus:border-[#AD7F65]
text-sm"
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Middle Initial"), /*
@__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      name: "middleInitial",
      value: formData.middleInitial,
      onChange: handleInputChange,
      placeholder: "A",
      maxLength: 1,
      inputMode: "text",
      className: "w-full px-3 py-2 border border-gray-200 rounded-lg focus:outline-none focus:border-[#AD7F65]
text-sm"
    }
  )), /* @__PURE__ */ React.createElement("div", { className: "col-span-2" }, /* @__PURE__ */
React.createElement("label", { className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1
block" }, "Last Name ", /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /*
@__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      name: "lastName",
      value: formData.lastName,
      onChange: handleInputChange,
      placeholder: "Last Name",
      className: "w-full px-3 py-2 border border-gray-200 rounded-lg focus:outline-none focus:border-[#AD7F65]
text-sm"
    }
  ))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Email ", /* @__PURE__
*/ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */
React.createElement("div", { className: "grid grid-cols-2 gap-4" }, /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "email",
      name: "email",
      value: formData.email,
      onChange: (e) => {
        handleInputChange(e);
        if (isEmailVerified) setIsEmailVerified(false);
        if (isCodeSent) setIsCodeSent(false);
      },
      placeholder: "user@example.com",
      className: `w-full px-3 py-2 border rounded-lg focus:outline-none focus:border-[#AD7F65] text-sm
${isEmailVerified ? "border-green-500 bg-green-50" : "border-gray-200"}`,
      readOnly: isEmailVerified
    }
  ), !isEmailVerified && !isCodeSent && /* @__PURE__ */ React.createElement(
    "button",
    {
```

```
      type: "button",
      onClick: handleSendCode,
      disabled: emailLoading || !formData.email || verificationTimer > 0 || showCodeSentCheck,
      className: "px-6 py-2 bg-[#C2A68C] text-white rounded-lg text-sm disabled:opacity-50 flex items-center
justify-center gap-2 transition-all whitespace-nowrap font-medium"
    },
    emailLoading ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(FaSpinner, { className: "animate-spin" }), /* @__PURE__ */ React.createElement("span",
null, "Sending...")) : showCodeSentCheck ? /* @__PURE__ */ React.createElement(React.Fragment, null, /*
@__PURE__ */ React.createElement(FaCheck, { className: "animate-pulse" }), /* @__PURE__ */
React.createElement("span", null, "Sent!")) : verificationTimer > 0 ? `Resend (${verificationTimer}s)` :
"Send Code"
  ), isEmailVerified && /* @__PURE__ */ React.createElement("div", { className: "px-6 py-2 bg-green-100 text-
green-700 rounded-lg text-sm font-medium flex items-center justify-center gap-2" }, /* @__PURE__ */
React.createElement(FaCheck, null), " Verified")), isCodeSent && !isEmailVerified && /* @__PURE__ */
React.createElement("div", { className: "grid grid-cols-2 gap-4 mt-2" }, /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      value: verificationCode,
      onChange: (e) => setVerificationCode(e.target.value),
      placeholder: "Enter 6-digit code",
      maxLength: 6,
      className: "w-full px-3 py-2 border border-gray-200 rounded-lg focus:outline-none focus:border-[#AD7F65]
text-sm"
    }
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: handleVerifyCode,
      disabled: emailLoading || !verificationCode,
      className: "px-6 py-2 bg-green-600 text-white rounded-lg text-sm whitespace-nowrap disabled:opacity-50
font-medium"
    },
    "Verify"
  ))), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-4" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: "text-xs font-bold
uppercase tracking-wide text-gray-500 mb-1 block" }, "Contact number"), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      name: "contactNo",
      value: formData.contactNo,
      onChange: handleInputChange,
      placeholder: "09123456789",
      className: "w-full px-3 py-2 border border-gray-200 rounded-lg focus:outline-none focus:border-[#AD7F65]
text-sm"
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Date Joined ", /*
@__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */
React.createElement(
    "input",
    {
      type: "date",
      name: "dateJoined",
      value: formData.dateJoined,
      onChange: handleInputChange,
      className: "w-full px-3 py-2 border border-gray-200 rounded-lg focus:outline-none focus:border-[#AD7F65]
text-sm bg-white"
    }
  ))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Position ", /*
@__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */
React.createElement(
    "select",
    {
      name: "role",
      value: formData.role,
      onChange: handleInputChange,
      className: "w-full px-3 py-2 border border-gray-200 rounded-lg focus:outline-none focus:border-[#AD7F65]
text-sm bg-white"
    },
    /* @__PURE__ */ React.createElement("option", { value: "Cashier" }, "Cashier"),
    /* @__PURE__ */ React.createElement("option", { value: "Manager" }, "Manager"),
    /* @__PURE__ */ React.createElement("option", { value: "Sub-Cashier" }, "Sub-Cashier"),
```

```
    /* @__PURE__ */ React.createElement("option", { value: "Stock Manager" }, "Stock Manager")
  ))), /* @__PURE__ */ React.createElement("h4", { className: "text-base font-bold text-gray-800 mb-4" }, 
"Permissions"), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-4 mb-6" }, [
    { label: "POS Terminal", desc: "Access to Point of Sale Terminal", key: "posTerminal" },
    { label: "Inventory", desc: "Add, Edit, Delete Products", key: "inventory" },
    { label: "View Transactions", desc: "View Sales History", key: "viewTransactions" },
    { label: "Generate Reports", desc: "Create and Download Business Reports", key: "generateReports" }
  ].map(
    ({ label, desc, key }) => /* @__PURE__ */ React.createElement("div", { key, className: "border border-
gray-200 rounded-xl p-3 flex items-center justify-between hover:border-[#AD7F65] transition-colors bg-
white" }, /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", 
{ className: "font-bold text-gray-800 text-xs" }, label), /* @__PURE__ */ React.createElement("p", 
{ className: "text-[10px] text-gray-500 mt-0.5" }, desc)), /* @__PURE__ */ React.createElement("label", 
{ className: "relative inline-flex items-center cursor-pointer ml-2" }, /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "checkbox",
        className: "sr-only peer",
        checked: accessControl[key],
        onChange: () => handleAccessControlToggle(key)
      }
    ), /* @__PURE__ */ React.createElement("div", { className: "w-11 h-6 bg-gray-200 rounded-full peer peer-
checked:bg-[#AD7F65] peer-focus:ring-2 peer-focus:ring-[#AD7F65]/20 transition-all" }), /* @__PURE__ */ 
React.createElement("div", { className: "absolute left-[2px] top-[2px] w-5 h-5 bg-white rounded-full 
transition-transform peer-checked:translate-x-5 shadow-sm" })))
  )), error && /* @__PURE__ */ React.createElement("div", { className: "bg-red-100 text-red-700 px-4 py-3 
rounded-xl text-sm mb-6" }, error), /* @__PURE__ */ React.createElement("div", { className: "flex justify-end 
gap-3 pt-4 border-t border-gray-100" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onClose,
      disabled: loading,
      className: "px-6 py-2.5 rounded-lg bg-gray-200 font-bold text-gray-700 hover:bg-gray-300 transition-all 
disabled:opacity-50"
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: handleAddEmployee,
      disabled: loading || emailLoading || !isEmailVerified,
      className: "px-8 py-2.5 rounded-lg text-white font-bold transition-all shadow-md hover:shadow-lg 
disabled:opacity-70 disabled:cursor-not-allowed bg-[#10B981]"
    },
    loading ? "Adding..." : "Add"
  )))));
};
export default AddEmployeeModal;
```

FILE: pos/frontend/src/components/owner/ArchiveBrandPartnerModal.jsx
==========

```
import { useState } from "react";
import { FaArchive, FaUndo } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const ArchiveBrandPartnerModal = ({ isOpen, onClose, onSuccess, brandPartner }) => {
  const { theme } = useTheme();
  const [processing, setProcessing] = useState(false);
  const [error, setError] = useState("");
  if (!isOpen || !brandPartner) return null;
  const isArchived = brandPartner.status === "archived";
  const handleToggleStatus = async () => {
    setProcessing(true);
    setError("");
    try {
      const response = await fetch(`http://localhost:5000/api/brand-partners/${brandPartner._id}`, {
        method: "PUT",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          status: isArchived ? "active" : "archived"
        })
      });
      const data = await response.json();
      if (!response.ok || !data.success) {
```

```
        throw new Error(data.message || `Failed to ${isArchived ? "unarchive" : "archive"} brand partner.`);
      }
      onSuccess?.(data.data);
      onClose();
    } catch (err) {
      console.error(`Failed to ${isArchived ? "unarchive" : "archive"} brand partner:`, err);
      setError(err.message || `Failed to ${isArchived ? "unarchive" : "archive"} brand partner.`);
    } finally {
      setProcessing(false);
    }
  };
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-[10010] flex items-center
justify-center p-4 bg-black/60 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement("div", { className:
`rounded-xl w-full max-w-md relative shadow-2xl p-6 flex flex-col items-center text-center ${theme ===
"dark" ? "bg-[#1E1B18]" : "bg-white"}` }, /* @__PURE__ */ React.createElement("div", { className: `w-16 h-16
rounded-full flex items-center justify-center mb-4 ${isArchived ? "bg-green-100" : "bg-orange-100"}` },
isArchived ? /* @__PURE__ */ React.createElement(FaUndo, { className: "w-8 h-8 text-green-600" }) : /*
@__PURE__ */ React.createElement(FaArchive, { className: "w-8 h-8 text-orange-600" })), /* @__PURE__ */
React.createElement("h2", { className: `text-xl font-bold mb-2 ${theme === "dark" ? "text-white" : "text-
gray-900"}` }, isArchived ? "Unarchive Brand Partner?" : "Archive Brand Partner?"), /* @__PURE__ */
React.createElement("p", { className: `mb-6 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` },
isArchived ? /* @__PURE__ */ React.createElement("span", null, "Are you sure you want to restore ", /*
@__PURE__ */ React.createElement("strong", null, brandPartner.brandName), "? This will make the brand active
again.") : /* @__PURE__ */ React.createElement("span", null, "Are you sure you want to archive ", /*
@__PURE__ */ React.createElement("strong", null, brandPartner.brandName), "? This will hide the brand from
active lists.")), error && /* @__PURE__ */ React.createElement("div", { className: "w-full text-sm text-red-
600 bg-red-50 border border-red-100 rounded-lg px-3 py-2 mb-4" }, error), /* @__PURE__ */
React.createElement("div", { className: "flex gap-3 w-full" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onClose,
      className: `flex-1 py-2.5 rounded-lg font-medium transition-colors ${theme === "dark" ? "bg-gray-700
text-gray-300 hover:bg-gray-600" : "bg-gray-100 text-gray-700 hover:bg-gray-200"}`,
      disabled: processing
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: handleToggleStatus,
      disabled: processing,
      className: `flex-1 py-2.5 rounded-lg text-white font-medium transition-colors disabled:opacity-70
${isArchived ? "bg-green-600 hover:bg-green-700" : "bg-orange-500 hover:bg-orange-600"}`
    },
    processing ? isArchived ? "Unarchiving..." : "Archiving..." : isArchived ? "Unarchive" : "Archive"
  ))));
};
export default ArchiveBrandPartnerModal;
```

FILE: pos/frontend/src/components/owner/DeleteBrandPartnerModal.jsx
==========

```
import { useState } from "react";
import { FaExclamationTriangle } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const DeleteBrandPartnerModal = ({ isOpen, onClose, onSuccess, brandPartner }) => {
  const { theme } = useTheme();
  const [deleting, setDeleting] = useState(false);
  const [error, setError] = useState("");
  if (!isOpen || !brandPartner) return null;
  const handleDelete = async () => {
    setDeleting(true);
    setError("");
    try {
      const response = await fetch(`http://localhost:5000/api/brand-partners/${brandPartner._id}`, {
        method: "DELETE"
      });
      const data = await response.json();
      if (!response.ok || !data.success) {
        throw new Error(data.message || "Failed to delete brand partner.");
      }
      onSuccess?.(brandPartner._id);
      onClose();
    } catch (err) {
      console.error("Failed to delete brand partner:", err);
      setError(err.message || "Failed to delete brand partner.");
```

```
    } finally {
      setDeleting(false);
    }
  };
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-[10010] flex items-center
justify-center p-4 bg-black/60 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement("div", { className:
`rounded-xl w-full max-w-md relative shadow-2xl p-6 flex flex-col items-center text-center ${theme ===
"dark" ? "bg-[#1E1B18]" : "bg-white"}` }, /* @__PURE__ */ React.createElement("div", { className: "w-16 h-16
rounded-full bg-red-100 flex items-center justify-center mb-4" }, /* @__PURE__ */
React.createElement(FaExclamationTriangle, { className: "w-8 h-8 text-red-600" })), /* @__PURE__ */
React.createElement("h2", { className: `text-xl font-bold mb-2 ${theme === "dark" ? "text-white" : "text-
gray-900"}` }, "Delete Brand Partner?"), /* @__PURE__ */ React.createElement("p", { className: `mb-6 ${theme
=== "dark" ? "text-gray-400" : "text-gray-500"}` }, "Are you sure you want to delete ", /* @__PURE__ */
React.createElement("strong", null, brandPartner.brandName), "? This action cannot be undone and will remove
all associated products."), error && /* @__PURE__ */ React.createElement("div", { className: "w-full text-sm
text-red-600 bg-red-50 border border-red-100 rounded-lg px-3 py-2 mb-4" }, error), /* @__PURE__ */
React.createElement("div", { className: "flex gap-3 w-full" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onClose,
      className: `flex-1 py-2.5 rounded-lg font-medium transition-colors ${theme === "dark" ? "bg-gray-700
text-gray-300 hover:bg-gray-600" : "bg-gray-100 text-gray-700 hover:bg-gray-200"}`,
      disabled: deleting
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: handleDelete,
      disabled: deleting,
      className: "flex-1 py-2.5 rounded-lg text-white font-medium bg-red-600 hover:bg-red-700 transition-
colors disabled:opacity-70"
    },
    deleting ? "Deleting..." : "Delete"
  ))));
};
export default DeleteBrandPartnerModal;
```

FILE: pos/frontend/src/components/owner/DeleteEmployeeModal.jsx
==========

```
import { FaTimes, FaTrash } from "react-icons/fa";
const DeleteEmployeeModal = ({ isOpen, onClose, onConfirm, employee }) => {
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10003] p-4 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement("div", { className: "bg-white
rounded-2xl w-full max-w-md relative shadow-2xl" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onClose,
      className: "absolute top-4 right-4 text-gray-400 hover:text-gray-600 transition-colors z-10"
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
  ), /* @__PURE__ */ React.createElement("div", { className: "p-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-center mb-6" }, /* @__PURE__ */
React.createElement("div", { className: "w-20 h-20 rounded-full bg-red-100 flex items-center justify-
center" }, /* @__PURE__ */ React.createElement(FaTrash, { className: "w-10 h-10 text-red-600" }))), /*
@__PURE__ */ React.createElement("h2", { className: "text-xl font-bold text-gray-800 text-center mb-2" },
"Are you sure you want to delete Account?"), /* @__PURE__ */ React.createElement("p", { className: "text-sm
text-gray-500 text-center mb-8" }, "This action can not be undone"), /* @__PURE__ */
React.createElement("div", { className: "flex gap-4" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onConfirm,
      className: "flex-1 px-6 py-3 rounded-lg text-white font-medium hover:opacity-90 transition-all shadow-
md",
      style: { background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)" }
    },
    "Confirm"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onClose,
      className: "flex-1 px-6 py-3 rounded-lg text-gray-700 bg-gray-200 hover:bg-gray-300 transition-all
font-medium shadow-md"
    },
```

```
    "Cancel"
  )))));
};
export default DeleteEmployeeModal;
```

FILE: pos/frontend/src/components/owner/DisableAccountModal.jsx
==========

```
import React from "react";
const DisableAccountModal = ({ isOpen, onClose, onConfirm, employee, action }) => {
  if (!isOpen || !employee) return null;
  const isDisabling = action === "disable";
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 flex items-center justify-center z-[10000]  bg-opacity-50"
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "bg-white rounded-2xl w-full max-w-md shadow-2xl overflow-hidden",
        onClick: (e) => e.stopPropagation()
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "h-2",
          style: {
            background: isDisabling ? "linear-gradient(to right, #F59E0B, #EF4444)" : "linear-gradient(to
right, #10B981, #059669)"
          }
        }
      ),
      /* @__PURE__ */ React.createElement("div", { className: "p-6" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center justify-center mb-4" }, isDisabling ? /* @__PURE__
*/ React.createElement("div", { className: "w-16 h-16 rounded-full bg-orange-100 flex items-center justify-
center" }, /* @__PURE__ */ React.createElement("svg", { className: "w-8 h-8 text-orange-600", fill: "none",
stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap:
"round", strokeLinejoin: "round", strokeWidth: 2, d: "M18.364 18.364A9 9 0 005.636 5.636m12.728 12.728A9 9 0
015.636 5.636m12.728 12.728L5.636 5.636" }))) : /* @__PURE__ */ React.createElement("div", { className: "w-16
h-16 rounded-full bg-green-100 flex items-center justify-center" }, /* @__PURE__ */ React.createElement("svg",
{ className: "w-8 h-8 text-green-600", fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /*
@__PURE__ */ React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d:
"M9 12l2 2 4-4m6 2a9 9 0 11-18 0 9 9 0 0118 0z" }))), /* @__PURE__ */ React.createElement("h2", { className:
"text-xl font-bold text-center text-gray-800 mb-2" }, isDisabling ? "Disable Account" : "Enable Account"), /*
@__PURE__ */ React.createElement("p", { className: "text-center text-gray-600 mb-6" }, "Are you sure you want
to ", action, " ", /* @__PURE__ */ React.createElement("span", { className: "font-semibold" }, employee.name),
"'s account?"), isDisabling && /* @__PURE__ */ React.createElement("div", { className: "bg-orange-50 border
border-orange-200 rounded-lg p-3 mb-6" }, /* @__PURE__ */ React.createElement("p", { className: "text-sm
text-orange-800" }, /* @__PURE__ */ React.createElement("strong", null, "Note:"), " This employee will not be
able to log in until the account is enabled again."), /* @__PURE__ */ React.createElement("div", { className:
"flex gap-3" }, /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onClose,
          className: "flex-1 px-4 py-2.5 rounded-lg bg-gray-200 text-gray-700 font-medium hover:bg-gray-300
transition-all"
        },
        "Cancel"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onConfirm,
          className: `flex-1 px-4 py-2.5 rounded-lg text-white font-medium transition-all ${isDisabling ?
"bg-orange-600 hover:bg-orange-700" : "bg-green-600 hover:bg-green-700"}`
        },
        isDisabling ? "Disable" : "Enable"
      )))
    )
  );
};
export default DisableAccountModal;
```

FILE: pos/frontend/src/components/owner/EditBrandPartnerModal.jsx

```
================================================================================
import { useEffect, useRef, useState } from "react";
import { FaTimes, FaUserFriends } from "react-icons/fa";
import cameraIcon from "../../assets/owner/camera.svg";
import circleIcon from "../../assets/owner/circle.svg";
import { useTheme } from "../../context/ThemeContext";
const EditBrandPartnerModal = ({ isOpen, onClose, onSuccess, brandPartner }) => {
  const { theme } = useTheme();
  const [formData, setFormData] = useState({
    brandName: "",
    email: "",
    contactNumber: "",
    contactPerson: "",
    logo: ""
  });
  const [submitting, setSubmitting] = useState(false);
  const [error, setError] = useState("");
  const fileInputRef = useRef(null);
  useEffect(() => {
    if (isOpen && brandPartner) {
      setFormData({
        brandName: brandPartner.brandName || "",
        email: brandPartner.email || "",
        contactNumber: brandPartner.contactNumber || "",
        contactPerson: brandPartner.contactPerson || "",
        logo: brandPartner.logo || ""
      });
      setError("");
    }
  }, [isOpen, brandPartner]);
  if (!isOpen) return null;
  const handleInputChange = (e) => {
    const { name, value } = e.target;
    setFormData((prev) => ({
      ...prev,
      [name]: value
    }));
  };
  const handleImageChange = (e) => {
    const file = e.target.files?.[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onloadend = () => {
      setFormData((prev) => ({
        ...prev,
        logo: reader.result
      }));
    };
    reader.readAsDataURL(file);
  };
  const handleSubmit = async (e) => {
    e.preventDefault();
    if (!formData.brandName.trim()) {
      setError("Please complete all required fields.");
      return;
    }
    setSubmitting(true);
    setError("");
    try {
      const response = await fetch(`http://localhost:5000/api/brand-partners/${brandPartner._id}`, {
        method: "PUT",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          ...formData,
          brandName: formData.brandName.trim()
        })
      });
      const data = await response.json();
      if (!response.ok || !data.success) {
        throw new Error(data.message || "Failed to update brand partner.");
      }
      onSuccess?.(data.data);
      onClose();
    } catch (err) {
      console.error("Failed to update brand partner:", err);
```

```
      setError(err.message || "Failed to update brand partner.");
    } finally {
      setSubmitting(false);
    }
  };
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-[10005] flex items-center
justify-center p-4 bg-black/60 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement("div", { className:
`rounded-3xl w-full max-w-lg relative shadow-2xl overflow-hidden ${theme === "dark" ? "bg-[#1E1B18]" : "bg-
white"}` }, /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onClose,
      className: "absolute top-5 right-5 text-gray-400 hover:text-gray-600 transition-colors",
      "aria-label": "Close edit brand partner modal"
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
  ), /* @__PURE__ */ React.createElement("form", { onSubmit: handleSubmit, className: "p-8 pt-10 space-y-6" },
/* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement("div", { className: "w-10 h-10 rounded-xl flex items-center justify-center", style:
{ background: "#E1D5CB" } }, /* @__PURE__ */ React.createElement(FaUserFriends, { className: "text-
[#76462B]" })), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("h2",
{ className: `text-2xl font-bold ${theme === "dark" ? "text-white" : "text-gray-900"}` }, "Edit Brand
Partner"), /* @__PURE__ */ React.createElement("p", { className: `text-sm ${theme === "dark" ? "text-gray-
400" : "text-gray-500"}` }, "Update brand partner profile"))), /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `w-40 h-40 mx-auto rounded-full flex items-center justify-center cursor-pointer border
border-dashed overflow-hidden relative ${theme === "dark" ? "bg-gradient-to-b from-[#2A2724] to-[#1E1B18]
border-gray-600 hover:border-[#AD7F65]" : "bg-gradient-to-b from-gray-50 to-gray-200 border-gray-300
hover:border-[#AD7F65]"}`,
      onClick: () => fileInputRef.current?.click()
    },
    formData.logo ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement("img", { src: formData.logo, alt: "Brand logo preview", className: "w-full h-full object-
cover" }), /* @__PURE__ */ React.createElement("div", { className: "absolute bottom-4 right-4 w-14 h-14
rounded-full bg-white/95 flex items-center justify-center shadow-md border border-gray-200" }, /* @__PURE__
*/ React.createElement("img", { src: cameraIcon, alt: "Camera", className: "w-7 h-7 opacity-80" }))) : /*
@__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("img", { src:
circleIcon, alt: "Circle background", className: "w-full h-full object-cover" }), /* @__PURE__ */
React.createElement("div", { className: "absolute bottom-4 right-4 w-14 h-14 rounded-full bg-white/95 flex
items-center justify-center shadow-md border border-gray-200" }, /* @__PURE__ */ React.createElement("img",
{ src: cameraIcon, alt: "Camera", className: "w-7 h-7 opacity-80" })))
  ), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "file",
      ref: fileInputRef,
      accept: "image/*",
      onChange: handleImageChange,
      className: "hidden"
    }
  ), /* @__PURE__ */ React.createElement("div", { className: `p-5 ${theme === "dark" ? "text-white" : "text-
gray-900"}` }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-2 mb-4" }, /*
@__PURE__ */ React.createElement("span", { className: "text-gray font-semibold text-sm" }, "Brand Partner
Information")), /* @__PURE__ */ React.createElement("div", { className: "space-y-4" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: "text-xs font-bold
uppercase tracking-wide text-gray-500 mb-1 block" }, "Brand ", /* @__PURE__ */ React.createElement("span",
{ className: "text-red-500" }, "*")), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      name: "brandName",
      placeholder: "e.g., John Doe",
      value: formData.brandName,
      onChange: handleInputChange,
      required: true,
      className: `w-full px-3 py-2 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" : "bg-white
border-gray-200 text-gray-900"}`
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Email"), /* @__PURE__
*/ React.createElement(
    "input",
    {
      type: "email",
      name: "email",
```

```
      placeholder: "yourname12345@gmail.com",
      value: formData.email,
      onChange: handleInputChange,
      className: `w-full px-3 py-2 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" : "bg-white
border-gray-200 text-gray-900"}`
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Contact No."), /*
@__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      name: "contactNumber",
      placeholder: "+63",
      value: formData.contactNumber,
      onChange: handleInputChange,
      className: `w-full px-3 py-2 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" : "bg-white
border-gray-200 text-gray-900"}`
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Contact Person"), /*
@__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      name: "contactPerson",
      placeholder: "e.g., John Doe",
      value: formData.contactPerson,
      onChange: handleInputChange,
      className: `w-full px-3 py-2 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] ${theme === "dark" ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" : "bg-white
border-gray-200 text-gray-900"}`
    }
  )))), error && /* @__PURE__ */ React.createElement("div", { className: "text-sm text-red-600 bg-red-50
border border-red-100 rounded-lg px-3 py-2" }, error), /* @__PURE__ */ React.createElement("div", { className:
"flex justify-end gap-3" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: onClose,
      className: `px-5 py-2 rounded-[15px] text-sm font-medium transition-colors ${theme === "dark" ? "bg-
gray-700 text-gray-300 hover:bg-gray-600" : "bg-gray-100 text-gray-700 hover:bg-gray-200"}`
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "submit",
      disabled: submitting,
      className: "px-6 py-2 rounded-[15px] text-white text-sm font-semibold shadow-lg disabled:opacity-60",
      style: { background: "linear-gradient(135deg, #1DB954 0%, #0A8A3E 100%)" }
    },
    submitting ? "Updating..." : "Save Changes"
  )))));
};
export default EditBrandPartnerModal;
```

FILE: pos/frontend/src/components/owner/EditEmployeeModal.jsx
==========

```
import { useEffect, useRef, useState } from "react";
import { FaCamera, FaTimes, FaUserEdit } from "react-icons/fa";
import defaultAvatar from "../../assets/default.jpeg";
import { API_BASE_URL } from "../../config/api";
const buildEmployeeName = (firstName, middleInitial, lastName) => {
  const parts = [firstName, middleInitial, lastName].map((p) => String(p ?? "").trim()).filter(Boolean);
  return parts.join(" ").trim();
};
const EditEmployeeProfile = ({ isOpen, onClose, employee, onEmployeeUpdated }) => {
  const [formData, setFormData] = useState({
    firstName: "",
    middleInitial: "",
    lastName: "",
    contactNo: "",
```

```
    email: "",
    role: "",
    dateJoined: ""
  });
  const [status, setStatus] = useState("Active");
  const [permissions, setPermissions] = useState({
    posTerminal: false,
    inventory: false,
    viewTransactions: false,
    generateReports: false
  });
  const [profilePreview, setProfilePreview] = useState("");
  const [loading, setLoading] = useState(false);
  const [message, setMessage] = useState("");
  const [error, setError] = useState("");
  const fileInputRef = useRef(null);
  useEffect(() => {
    if (isOpen && employee) {
      setFormData({
        firstName: employee.firstName || "",
        middleInitial: employee.middleInitial || "",
        lastName: employee.lastName || "",
        contactNo: employee.contactNo || "",
        email: employee.email || "",
        role: employee.role || "Sales Clerk",
        dateJoined: employee.dateJoinedActual ? new
Date(employee.dateJoinedActual).toISOString().split("T")[0] : ""
      });
      setStatus(employee.status || "Active");
      setPermissions({
        posTerminal: employee.permissions?.posTerminal ?? false,
        inventory: employee.permissions?.inventory ?? false,
        viewTransactions: employee.permissions?.viewTransactions ?? false,
        generateReports: employee.permissions?.generateReports ?? false
      });
      setProfilePreview(employee._id ? `${API_BASE_URL}/api/employees/${employee._id}/image?v=${new
Date(employee.updatedAt || employee.lastUpdated || 0).getTime()}` : defaultAvatar);
      setMessage("");
      setError("");
    }
  }, [isOpen, employee]);
  const handleInputChange = (e) => {
    const { name, value } = e.target;
    const normalizedValue = name === "middleInitial" ? value.replace(/[^a-zA-Z]/g, "").slice(0,
1).toUpperCase() : value;
    setFormData((prev) => ({ ...prev, [name]: normalizedValue }));
  };
  const handlePermissionToggle = (key) => {
    setPermissions((prev) => ({ ...prev, [key]: !prev[key] }));
  };
  const handlePhotoClick = () => {
    fileInputRef.current?.click();
  };
  const handleImageChange = (e) => {
    const file = e.target.files?.[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onloadend = () => setProfilePreview(reader.result);
    reader.readAsDataURL(file);
  };
  const handleSave = async () => {
    if (!formData.firstName || !formData.lastName || !formData.email || !formData.contactNo) {
      setError("Name, email, and contact number are required.");
      return;
    }
    setLoading(true);
    setError("");
    setMessage("");
    try {
      const employeeName = buildEmployeeName(
        formData.firstName,
        formData.middleInitial,
        formData.lastName
      );
      const payload = {
        firstName: formData.firstName.trim(),
        middleInitial: formData.middleInitial,
```

```
        lastName: formData.lastName.trim(),
        name: employeeName,
        contactNo: formData.contactNo,
        email: formData.email,
        role: formData.role,
        status,
        permissions,
        dateJoinedActual: formData.dateJoined
      };
      if (profilePreview && profilePreview.startsWith("data:image")) {
        payload.profileImage = profilePreview;
      }
      const response = await fetch(`${API_BASE_URL}/api/employees/${employee._id}`, {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(payload)
      });
      const data = await response.json();
      if (!data.success) {
        throw new Error(data.message || "Failed to update employee.");
      }
      setMessage("Employee updated successfully.");
      onEmployeeUpdated?.();
      onClose();
    } catch (err) {
      setError(err.message || "Failed to save changes.");
    } finally {
      setLoading(false);
    }
  };
  if (!isOpen || !employee) return null;
  const employeeDisplayName = buildEmployeeName(formData.firstName, formData.middleInitial, formData.lastName)
|| "Employee Name";
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10002] p-4 backdrop-blur-sm bg-black/20" }, /* @__PURE__ */ React.createElement("div", { className:
"bg-white w-full max-w-2xl relative shadow-2xl overflow-hidden animate-fadeIn", style: { borderRadius:
"24px" } }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between px-8
py-6 border-b border-gray-100" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center
gap-3" }, /* @__PURE__ */ React.createElement("div", { className: "w-10 h-10 rounded-xl bg-[#AD7F65] flex
items-center justify-center text-white" }, /* @__PURE__ */ React.createElement(FaUserEdit, { className: "w-5
h-5" })), /* @__PURE__ */ React.createElement("h2", { className: "text-lg font-bold text-gray-800" }, "Edit
Employee Profile")), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onClose,
      className: "text-gray-400 hover:text-gray-600 transition-colors p-2 hover:bg-gray-100 rounded-full"
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
  )), /* @__PURE__ */ React.createElement("div", { className: "p-8 max-h-[80vh] overflow-y-auto custom-
scrollbar" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-start gap-8 mb-10" }, /*
@__PURE__ */ React.createElement("div", { className: "shrink-0 relative group cursor-pointer", onClick:
handlePhotoClick }, /* @__PURE__ */ React.createElement("div", { className: "w-32 h-32 rounded-full overflow-
hidden border-4 border-white shadow-lg relative" }, /* @__PURE__ */ React.createElement(
    "img",
    {
      src: profilePreview || defaultAvatar,
      alt: employeeDisplayName,
      className: "w-full h-full object-cover transition-opacity group-hover:opacity-75",
      onError: (e) => {
        e.currentTarget.onerror = null;
        setProfilePreview(defaultAvatar);
      }
    }
  ), /* @__PURE__ */ React.createElement("div", { className: "absolute inset-0 flex items-center justify-
center opacity-0 group-hover:opacity-100 transition-opacity bg-black/30" }, /* @__PURE__ */
React.createElement(FaCamera, { className: "text-white w-8 h-8" }))), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "file",
      accept: "image/*",
      ref: fileInputRef,
      onChange: handleImageChange,
      className: "hidden"
    }
  )), /* @__PURE__ */ React.createElement("div", { className: "flex-1 pt-2" }, /* @__PURE__ */
React.createElement("h3", { className: "text-xl font-bold text-gray-900 mb-1" }, employeeDisplayName), /*
@__PURE__ */ React.createElement("p", { className: "text-sm text-[#AD7F65] font-medium mb-4" },
```

```
formData.role), /* @__PURE__ */ React.createElement("div", { className: "space-y-4" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className: "text-xs font-
semibold text-gray-400 uppercase tracking-wider mb-2 block" }, "Permissions:"), /* @__PURE__ */
React.createElement("div", { className: "flex gap-2 flex-wrap" }, permissions.posTerminal && /* @__PURE__ */
React.createElement("span", { className: "px-3 py-1 rounded-full text-xs font-medium border border-[#E5E7EB]
text-gray-600 bg-gray-50" }, "POS Terminal"), permissions.inventory && /* @__PURE__ */
React.createElement("span", { className: "px-3 py-1 rounded-full text-xs font-medium border border-[#E5E7EB]
text-gray-600 bg-gray-50" }, "Inventory"), permissions.viewTransactions && /* @__PURE__ */
React.createElement("span", { className: "px-3 py-1 rounded-full text-xs font-medium border border-[#E5E7EB]
text-gray-600 bg-gray-50" }, "View Transactions"), permissions.generateReports && /* @__PURE__ */
React.createElement("span", { className: "px-3 py-1 rounded-full text-xs font-medium border border-[#E5E7EB]
text-gray-600 bg-gray-50" }, "Generate Reports"), !permissions.posTerminal && !permissions.inventory
&& !permissions.viewTransactions && !permissions.generateReports && /* @__PURE__ */
React.createElement("span", { className: "text-xs text-gray-400 italic" }, "No active permissions"))), /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className:
"text-xs font-semibold text-gray-400 uppercase tracking-wider mb-2 block" }, "Status:"), /* @__PURE__ */
React.createElement(
    "span",
    {
      className: `inline-block px-3 py-1 rounded-lg text-xs font-bold ${status === "Active" ? "bg-green-100
text-green-700" : "bg-red-100 text-red-700"}`
    },
    status
  )))))), /* @__PURE__ */ React.createElement("h4", { className: "text-base font-bold text-gray-800 mb-6" },
"Personal Details"), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-x-8 gap-y-
6 mb-10" }, /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "First Name ", /*
@__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */
React.createElement(
    "input",
    {
      type: "text",
      name: "firstName",
      value: formData.firstName,
      onChange: handleInputChange,
      className: "w-full text-base font-semibold text-gray-800 border-b border-gray-200 focus:border-[#AD7F65]
focus:outline-none py-1 placeholder-gray-300",
      placeholder: "First Name"
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Middle Initial"), /*
@__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      name: "middleInitial",
      value: formData.middleInitial,
      onChange: handleInputChange,
      placeholder: "A",
      maxLength: 1,
      inputMode: "text",
      className: "w-full text-lg font-bold text-gray-800 border-b border-gray-200 focus:border-[#AD7F65]
focus:outline-none py-1 placeholder-gray-300"
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Last Name ", /*
@__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */
React.createElement(
    "input",
    {
      type: "text",
      name: "lastName",
      value: formData.lastName,
      onChange: handleInputChange,
      className: "w-full text-base font-semibold text-gray-800 border-b border-gray-200 focus:border-[#AD7F65]
focus:outline-none py-1 placeholder-gray-300",
      placeholder: "Last Name"
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Contact number"), /*
@__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      name: "contactNo",
      value: formData.contactNo,
      onChange: handleInputChange,
```

```
      className: "w-full text-lg font-bold text-gray-800 border-b border-gray-200 focus:border-[#AD7F65]
focus:outline-none py-1 placeholder-gray-300",
      placeholder: "09123456789"
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Email ", /* @__PURE__
*/ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "email",
      name: "email",
      value: formData.email,
      onChange: handleInputChange,
      className: "w-full text-lg font-bold text-gray-800 border-b border-gray-200 focus:border-[#AD7F65]
focus:outline-none py-1 placeholder-gray-300",
      placeholder: "email@example.com"
    }
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1 block" }, "Date Joined ", /*
@__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /* @__PURE__ */
React.createElement(
    "input",
    {
      type: "date",
      name: "dateJoined",
      value: formData.dateJoined,
      onChange: handleInputChange,
      className: "w-full text-lg font-bold text-gray-800 border-b border-gray-200 focus:border-[#AD7F65]
focus:outline-none py-1"
    }
  )), /* @__PURE__ */ React.createElement("div", { className: "col-span-2" }, /* @__PURE__ */
React.createElement("label", { className: "text-xs font-bold uppercase tracking-wide text-gray-500 mb-1
block" }, "Position ", /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")), /*
@__PURE__ */ React.createElement("div", { className: "text-base font-semibold text-gray-800 flex items-center
gap-2" }, /* @__PURE__ */ React.createElement("span", null, "Employee - "), /* @__PURE__ */
React.createElement(
    "select",
    {
      name: "role",
      value: formData.role,
      onChange: handleInputChange,
      className: "bg-transparent border-b border-gray-200 focus:border-[#AD7F65] focus:outline-none py-1
font-bold text-gray-800 cursor-pointer"
    },
    ["Cashier", "Sales Clerk", "Manager", "Supervisor"].map(
      (role) => /* @__PURE__ */ React.createElement("option", { value: role, key: role }, role)
    )
  )))), /* @__PURE__ */ React.createElement("h4", { className: "text-base font-bold text-gray-800 mb-6" },
"Permissions"), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-1 md:grid-cols-2 gap-
4 mb-8" }, [
    { label: "POS Terminal", desc: "Access to Point of Sale Terminal", key: "posTerminal" },
    { label: "Inventory", desc: "Add, Edit, Delete Products", key: "inventory" },
    { label: "View Transactions", desc: "View Sales History", key: "viewTransactions" },
    { label: "Generate Reports", desc: "Create and Download Business Reports", key: "generateReports" }
  ].map(
    ({ label, desc, key }) => /* @__PURE__ */ React.createElement("div", { key, className: "border border-
gray-200 rounded-xl p-4 flex items-center justify-between hover:border-[#AD7F65] transition-colors bg-
white" }, /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p",
{ className: "font-bold text-gray-800 text-sm" }, label), /* @__PURE__ */ React.createElement("p",
{ className: "text-xs text-gray-500 mt-0.5" }, desc)), /* @__PURE__ */ React.createElement("label",
{ className: "relative inline-flex items-center cursor-pointer" }, /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "checkbox",
        className: "sr-only peer",
        checked: permissions[key],
        onChange: () => handlePermissionToggle(key)
      }
    ), /* @__PURE__ */ React.createElement("div", { className: "w-11 h-6 bg-gray-200 rounded-full peer peer-
checked:bg-[#AD7F65] peer-focus:ring-2 peer-focus:ring-[#AD7F65]/20 transition-all" }), /* @__PURE__ */
React.createElement("div", { className: "absolute left-[2px] top-[2px] w-5 h-5 bg-white rounded-full
transition-transform peer-checked:translate-x-5 shadow-sm" })))
  )), (error || message) && /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `px-4 py-3 rounded-xl text-sm mb-6 ${error ? "bg-red-100 text-red-700" : "bg-green-100 text-
green-700"}`
```

```
    },
    error || message
  ), /* @__PURE__ */ React.createElement("div", { className: "flex justify-end gap-3 pt-4 border-t border-
gray-100" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onClose,
      className: "px-6 py-2.5 rounded-lg bg-gray-200 font-bold text-gray-700 hover:bg-gray-300 transition-
all"
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: handleSave,
      disabled: loading,
      className: "px-8 py-2.5 rounded-lg text-white font-bold transition-all shadow-md hover:shadow-lg
disabled:opacity-70 disabled:cursor-not-allowed",
      style: {
        background: "#007AFF"
      }
    },
    loading ? "Updating..." : "Update"
  )))));
};
export default EditEmployeeProfile;
```

FILE: pos/frontend/src/components/owner/ResetPinConfirmModal.jsx
==========

```
import React, { useState } from "react";
const ResetPinConfirmModal = ({ isOpen, onClose, onConfirm, employeeName }) => {
  const [pin, setPin] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  if (!isOpen) return null;
  const handlePinChange = (e) => {
    const digitsOnly = e.target.value.replace(/\D/g, "").slice(0, 6);
    setPin(digitsOnly);
    if (error) setError("");
  };
  const handleReset = async () => {
    const trimmedPin = pin.trim();
    if (trimmedPin.length !== 6) {
      setError("Please enter your 6-digit PIN");
      return;
    }
    try {
      setLoading(true);
      setError("");
      const currentUser = JSON.parse(localStorage.getItem("currentUser") || "{}");
      if (!currentUser.email) {
        setError("Current user information not found. Please log in again.");
        setLoading(false);
        return;
      }
      const response = await fetch("http://localhost:5000/api/employees/verify-pin", {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          email: currentUser.email,
          pin: trimmedPin
        })
      });
      const data = await response.json();
      if (!data.success) {
        setError(data.message || "PIN verification failed");
        setLoading(false);
        return;
      }
      await onConfirm();
      setPin("");
      setLoading(false);
    } catch (err) {
```

```
      console.error("Error verifying PIN for reset:", err);
      setError("Failed to verify PIN. Please try again.");
      setLoading(false);
    }
  };
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 flex items-center backdrop-blur-sm  justify-center z-[10000]  bg-opacity-50",
      onClick: onClose
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "bg-white rounded-2xl w-full max-w-md shadow-2xl overflow-hidden",
        onClick: (e) => e.stopPropagation()
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "h-2",
          style: {
            background: "linear-gradient(to right, #3B82F6, #2563EB)"
          }
        }
      ),
      /* @__PURE__ */ React.createElement("div", { className: "p-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center justify-center mb-6" }, /* @__PURE__ */
React.createElement("div", { className: "w-16 h-16 rounded-full bg-blue-100 flex items-center justify-
center" }, /* @__PURE__ */ React.createElement("svg", { className: "w-8 h-8 text-blue-600", fill: "none",
stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap:
"round", strokeLinejoin: "round", strokeWidth: 2, d: "M12 15v2m-6 4h12a2 2 0 002-2v-6a2 2 0 00-2-2H6a2 2 0
00-2 2v6a2 2 0 002 2zm10-10V7a4 4 0 00-8 0v4h8z" }))))), /* @__PURE__ */ React.createElement("h2", { className:
"text-xl font-bold text-center text-gray-800 mb-2" }, "Reset PIN for ", employeeName, "?"), /* @__PURE__ */
React.createElement("p", { className: "text-center text-gray-600 mb-4" }, "This will generate a new temporary
PIN that will be emailed to the employee and required on their next login."), /* @__PURE__ */
React.createElement("div", { className: "bg-blue-50 border border-blue-200 rounded-lg p-4 mb-4" }, /*
@__PURE__ */ React.createElement("div", { className: "flex gap-3" }, /* @__PURE__ */
React.createElement("svg", { className: "w-5 h-5 text-blue-600 flex-shrink-0 mt-0.5", fill: "none", stroke:
"currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap: "round",
strokeLinejoin: "round", strokeWidth: 2, d: "M13 16h-1v-4h-1m1-4h.01M21 12a9 9 0 11-18 0 9 9 0 0118 0z" })),
/* @__PURE__ */ React.createElement("div", { className: "text-sm text-blue-800" }, /* @__PURE__ */
React.createElement("p", { className: "font-semibold mb-1" }, "What happens next:"), /* @__PURE__ */
React.createElement("ul", { className: "list-disc list-inside space-y-1" }, /* @__PURE__ */
React.createElement("li", null, "A new 6-digit temporary PIN will be generated"), /* @__PURE__ */
React.createElement("li", null, "The employee's current PIN will be replaced"), /* @__PURE__ */
React.createElement("li", null, "The temporary PIN will be emailed to the employee"), /* @__PURE__ */
React.createElement("li", null, "They must change it on their next login"))))), /* @__PURE__ */
React.createElement("div", { className: "mb-4" }, /* @__PURE__ */ React.createElement("label", { className:
"block text-sm font-medium text-gray-700 mb-1" }, "Enter your 6-digit PIN to confirm"), /* @__PURE__ */
React.createElement(
        "input",
        {
          type: "password",
          value: pin,
          onChange: handlePinChange,
          maxLength: 6,
          className: "w-full px-3 py-2 border border-gray-300 rounded-lg focus:outline-none focus:ring-2
focus:ring-blue-500 focus:border-transparent tracking-[0.5em] text-center text-lg",
          placeholder: "\u2022\u2022\u2022\u2022\u2022\u2022"
        }
      ), error && /* @__PURE__ */ React.createElement("p", { className: "mt-1 text-sm text-red-600" },
error)), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: onClose,
          className: "flex-1 px-4 py-2.5 rounded-lg bg-gray-200 text-gray-700 font-medium hover:bg-gray-300
transition-all",
          disabled: loading
        },
        "Cancel"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: handleReset,
          className: "flex-1 px-4 py-2.5 rounded-lg bg-blue-600 hover:bg-blue-700 text-white font-medium
```

```
transition-all disabled:opacity-60",
          disabled: loading
        },
        loading ? "Verifying..." : "Reset PIN"
      )))
    )
  );
};
export default ResetPinConfirmModal;
```

FILE: pos/frontend/src/components/owner/TemporaryPinModal.jsx
==========

```
import React from "react";
import { FaTimes, FaCopy } from "react-icons/fa";
const TemporaryPinModal = ({ isOpen, onClose, employeeName, temporaryPin }) => {
  if (!isOpen) return null;
  const handleCopyPin = () => {
    navigator.clipboard.writeText(temporaryPin);
    alert("PIN copied to clipboard!");
  };
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 flex items-center backdrop-blur-sm justify-center z-[10003] bg-opacity-50"
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "bg-white rounded-2xl w-full max-w-lg shadow-2xl overflow-hidden",
        onClick: (e) => e.stopPropagation()
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "h-2",
          style: {
            background: "linear-gradient(to right, #C2A68C, #AD7F65, #76462B)"
          }
        }
      ),
      /* @__PURE__ */ React.createElement("div", { className: "p-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-between items-start mb-6" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement("div", { className: "w-12 h-12 rounded-full bg-green-100 flex items-center justify-
center" }, /* @__PURE__ */ React.createElement("svg", { className: "w-6 h-6 text-green-600", fill: "none",
stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap:
"round", strokeLinejoin: "round", strokeWidth: 2, d: "M9 12l2 2 4-4m6 2a9 9 0 11-18 0 9 9 0 0118 0z" }))), /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("h2", { className: "text-xl
font-bold text-gray-800" }, "Temporary PIN Generated!"), /* @__PURE__ */ React.createElement("p", { className:
"text-sm text-gray-600" }, "Employee: ", /* @__PURE__ */ React.createElement("span", { className: "font-
semibold" }, employeeName)))), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onClose,
          className: "text-gray-400 hover:text-gray-600 transition"
        },
        /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
      )), /* @__PURE__ */ React.createElement("div", { className: "bg-gradient-to-br from-amber-50 to-orange-
50 border-2 border-amber-200 rounded-xl p-6 mb-6" }, /* @__PURE__ */ React.createElement("div", { className:
"flex items-center gap-2 mb-3" }, /* @__PURE__ */ React.createElement("svg", { className: "w-5 h-5 text-
amber-600", fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */
React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M12 15v2m-
6 4h12a2 2 0 002-2v-6a2 2 0 00-2-2H6a2 2 0 00-2 2v6a2 2 0 002 2zm10-10V7a4 4 0 00-8 0v4h8z" })), /* @__PURE__
*/ React.createElement("h3", { className: "text-lg font-bold text-amber-900" }, "Temporary PIN")), /*
@__PURE__ */ React.createElement("div", { className: "flex items-center justify-center gap-3 mb-4" },
temporaryPin.split("").map(
        (digit, index) => /* @__PURE__ */ React.createElement(
          "div",
          {
            key: index,
            className: "w-14 h-16 bg-white border-2 border-amber-300 rounded-lg flex items-center justify-
center text-3xl font-bold text-amber-900 shadow-md"
          },
          digit
        )
```

```
      )), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: handleCopyPin,
          className: "w-full flex items-center justify-center gap-2 px-4 py-2 bg-amber-600 hover:bg-amber-700
text-white rounded-lg transition-all font-medium"
        },
        /* @__PURE__ */ React.createElement(FaCopy, { className: "w-4 h-4" }),
        "Copy PIN to Clipboard"
      )), /* @__PURE__ */ React.createElement("div", { className: "bg-blue-50 border border-blue-200 rounded-
lg p-4 mb-6" }, /* @__PURE__ */ React.createElement("div", { className: "flex gap-3" }, /* @__PURE__ */
React.createElement("svg", { className: "w-5 h-5 text-blue-600 flex-shrink-0 mt-0.5", fill: "none", stroke:
"currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap: "round",
strokeLinejoin: "round", strokeWidth: 2, d: "M13 16h-1v-4h-1m1-4h.01M21 12a9 9 0 11-18 0 9 9 0 0118 0z" })),
/* @__PURE__ */ React.createElement("div", { className: "text-sm text-blue-800" }, /* @__PURE__ */
React.createElement("p", { className: "font-semibold mb-1" }, "Important:"), /* @__PURE__ */
React.createElement("ul", { className: "list-disc list-inside space-y-1" }, /* @__PURE__ */
React.createElement("li", null, "This is a temporary PIN for the employee's first login"), /* @__PURE__ */
React.createElement("li", null, "The employee will be required to change this PIN on first login"), /*
@__PURE__ */ React.createElement("li", null, "Please share this PIN securely with the employee"), /*
@__PURE__ */ React.createElement("li", null, "This PIN will not be shown again"))))), /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: onClose,
          className: "w-full px-6 py-3 rounded-lg text-white font-medium transition-all",
          style: {
            background: "linear-gradient(to right, #C2A68C, #AD7F65, #76462B)"
          }
        },
        "I've Saved the PIN"
      ))
    )
  );
};
export default TemporaryPinModal;
```

FILE: pos/frontend/src/components/owner/TopSellingModal.jsx
==========

```
import { FaTimes } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const TopSellingModal = ({ isOpen, onClose, products, timeframe }) => {
  const { theme } = useTheme();
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10002] p-4 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement("div", { className: `rounded-
2xl w-full max-w-2xl max-h-[90vh] overflow-hidden shadow-2xl flex flex-col ${theme === "dark" ? "bg-[#1E1B18]
text-white" : "bg-white text-gray-900"}` }, /* @__PURE__ */ React.createElement("div", { className: `px-6 py-
4 border-b flex items-center justify-between ${theme === "dark" ? "border-gray-700" : "border-gray-200"}` },
/* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("h2", { className:
`text-2xl font-bold ${theme === "dark" ? "text-white" : "text-black"}` }, "Top Selling Products"), /*
@__PURE__ */ React.createElement("p", { className: `text-sm mt-1 ${theme === "dark" ? "text-gray-400" :
"text-gray-500"}` }, "Top Performing items this ", timeframe.toLowerCase())), /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: onClose,
      className: "text-gray-400 hover:text-gray-600 transition-colors"
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
  )), /* @__PURE__ */ React.createElement("div", { className: "flex-1 overflow-y-auto px-6 py-6" },
products.length === 0 ? /* @__PURE__ */ React.createElement("div", { className: "flex justify-center items-
center py-12" }, /* @__PURE__ */ React.createElement("div", { className: "text-gray-500" }, "No top selling
products found.")) : /* @__PURE__ */ React.createElement("div", { className: "space-y-4" }, products.map(
    (product, index) => /* @__PURE__ */ React.createElement(
      "div",
      {
        key: product.productId,
        className: `flex items-center justify-between p-3 rounded-xl transition-colors ${theme === "dark" ?
"bg-[#2A2724] border border-gray-700" : "bg-gray-50 border border-gray-100"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-4" }, /* @__PURE__ */
React.createElement("div", { className: "w-8 h-8 rounded-full bg-[#AD7F65] text-white flex items-center
justify-center font-bold text-sm shadow-sm flex-shrink-0" }, index + 1), /* @__PURE__ */ React.createElement(
        "div",
```

```
        {
          className: `w-14 h-14 rounded-lg overflow-hidden border flex-shrink-0 ${theme === "dark" ? "bg-
gray-800 border-gray-700" : "bg-white border-gray-200"}`
        },
        product.itemImage ? /* @__PURE__ */ React.createElement(
          "img",
          {
            src: product.itemImage,
            alt: product.itemName,
            className: "w-full h-full object-cover"
          }
        ) : /* @__PURE__ */ React.createElement("div", { className: "w-full h-full flex items-center justify-
center text-[10px] text-gray-400" }, "img")
      ), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-base font-bold ${theme === "dark" ? "text-gray-200" : "text-gray-800"}`
        },
        product.itemName
      ), /* @__PURE__ */ React.createElement("p", { className: "text-sm font-medium text-[#AD7F65]" },
product.totalQuantitySold, " Sold"))),
      /* @__PURE__ */ React.createElement("div", { className: "text-right" }, /* @__PURE__ */
React.createElement("p", { className: "text-sm font-bold text-green-600" }, "PHP ",
product.totalRevenue.toLocaleString()))
    )
  )))));
};
export default TopSellingModal;
```

FILE: pos/frontend/src/components/owner/ViewBrandProductsModal.jsx
==========

```
import { useEffect, useState } from "react";
import { FaEye, FaTimes } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const ViewBrandProductsModal = ({ isOpen, onClose, brandPartner, onEdit, onDelete }) => {
  const { theme } = useTheme();
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(false);
  useEffect(() => {
    if (isOpen && brandPartner) {
      fetchBrandProducts();
    }
  }, [isOpen, brandPartner]);
  const fetchBrandProducts = async () => {
    if (!brandPartner?.brandName) return;
    try {
      setLoading(true);
      const response = await fetch("http://localhost:5000/api/products");
      const data = await response.json();
      if (data.success) {
        const brandProducts = data.data.filter(
          (product) => product.brandName === brandPartner.brandName || product.supplierName ===
brandPartner.brandName
        );
        setProducts(brandProducts);
      }
    } catch (error) {
      console.error("Error fetching brand products:", error);
    } finally {
      setLoading(false);
    }
  };
  if (!isOpen || !brandPartner) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10002] p-4 backdrop-blur-sm bg-black/30" }, /* @__PURE__ */ React.createElement("div", { className:
`rounded-2xl w-full max-w-5xl max-h-[90vh] overflow-hidden shadow-2xl flex flex-col ${theme === "dark" ? "bg-
[#1E1B18]" : "bg-white"}` }, /* @__PURE__ */ React.createElement("div", { className: "px-8 py-6 flex items-
center justify-between" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-
3" }, /* @__PURE__ */ React.createElement("div", { className: `w-10 h-10 rounded-lg flex items-center
justify-center ${theme === "dark" ? "bg-[#8B5E3C]" : "bg-[#AD7F65]"} text-white` }, /* @__PURE__ */
React.createElement(FaEye, { className: "w-5 h-5" })), /* @__PURE__ */ React.createElement("h2", { className:
`text-2xl font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}` }, "View Products")), /* @__PURE__
*/ React.createElement(
    "button",
    {
```

```
      onClick: onClose,
      className: "text-gray-400 hover:text-gray-600 transition-colors"
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
  )), /* @__PURE__ */ React.createElement("div", { className: "px-8 pb-6 flex items-start gap-6" }, /*
@__PURE__ */ React.createElement("div", { className: "w-32 h-32 shrink-0" }, /* @__PURE__ */
React.createElement("div", { className: "w-32 h-32 bg-gray-200 rounded-full flex items-center justify-center
overflow-hidden" }, brandPartner.logo ? /* @__PURE__ */ React.createElement(
    "img",
    {
      src: brandPartner.logo,
      alt: brandPartner.brandName,
      className: "w-full h-full object-cover"
    }
  ) : /* @__PURE__ */ React.createElement("div", { className: "text-gray-400 text-4xl font-bold" },
brandPartner.brandName.charAt(0)))), /* @__PURE__ */ React.createElement("div", { className: "flex-1 pt-2" },
/* @__PURE__ */ React.createElement("h3", { className: `text-3xl font-bold mb-3 ${theme === "dark" ? "text-
white" : "text-gray-800"}` }, brandPartner.brandName), /* @__PURE__ */ React.createElement("div", { className:
`space-y-1 text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-600"}` }, /* @__PURE__ */
React.createElement("div", null, "Email: ", brandPartner.email), /* @__PURE__ */ React.createElement("div",
null, "Contact Person: ", brandPartner.contactPerson), /* @__PURE__ */ React.createElement("div", null,
"Contact Number: ", brandPartner.contactNumber)))), /* @__PURE__ */ React.createElement("div", { className:
"flex-1 overflow-y-auto px-8 pb-8" }, loading ? /* @__PURE__ */ React.createElement("div", { className: "flex
justify-center items-center py-12" }, /* @__PURE__ */ React.createElement("div", { className: "text-gray-
500" }, "Loading products...")) : products.length === 0 ? /* @__PURE__ */ React.createElement("div",
{ className: "flex justify-center items-center py-12" }, /* @__PURE__ */ React.createElement("div",
{ className: "text-gray-500" }, "No products found for this brand.")) : /* @__PURE__ */
React.createElement("div", { className: "overflow-hidden rounded-xl border border-gray-100 shadow-sm" }, /*
@__PURE__ */ React.createElement("table", { className: "w-full border-collapse" }, /* @__PURE__ */
React.createElement("thead", null, /* @__PURE__ */ React.createElement("tr", { className: `${theme ===
"dark" ? "bg-[#3A3734]" : "bg-[#EADBC8]"}` }, /* @__PURE__ */ React.createElement("th", { className: "px-6
py-4 text-center text-sm font-semibold text-gray-800 w-24" }, "SKU"), /* @__PURE__ */
React.createElement("th", { className: "px-6 py-4 text-left text-sm font-semibold text-gray-800" }, "Item
Name"), /* @__PURE__ */ React.createElement("th", { className: "px-6 py-4 text-center text-sm font-semibold
text-gray-800" }, "Category"), /* @__PURE__ */ React.createElement("th", { className: "px-6 py-4 text-center
text-sm font-semibold text-gray-800" }, "Variant"), /* @__PURE__ */ React.createElement("th", { className:
"px-6 py-4 text-center text-sm font-semibold text-gray-800" }, "Item Price"), /* @__PURE__ */
React.createElement("th", { className: "px-6 py-4 text-center text-sm font-semibold text-gray-800 w-24" },
"Stock"))), /* @__PURE__ */ React.createElement("tbody", { className: `${theme === "dark" ? "bg-[#2A2724]
divide-[#3A3734]" : "bg-white divide-gray-100"} divide-y` }, products.map(
    (product) => /* @__PURE__ */ React.createElement("tr", { key: product._id, className: `${theme ===
"dark" ? "hover:bg-[#3A3734]/50" : "hover:bg-gray-50"}` }, /* @__PURE__ */ React.createElement("td",
{ className: "px-6 py-4" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center
justify-center gap-3" }, /* @__PURE__ */ React.createElement("div", { className: "w-10 h-10 rounded overflow-
hidden bg-gray-100 shrink-0" }, product.itemImage ? /* @__PURE__ */ React.createElement("img", { src:
product.itemImage, alt: "", className: "w-full h-full object-cover" }) : /* @__PURE__ */
React.createElement("div", { className: "w-full h-full flex items-center justify-center text-xs text-gray-
400" }, "Img")), /* @__PURE__ */ React.createElement("span", { className: `text-sm ${theme === "dark" ?
"text-gray-300" : "text-gray-600"}` }, product.sku || "234TYP"))), /* @__PURE__ */ React.createElement("td",
{ className: `px-6 py-4 text-sm font-medium ${theme === "dark" ? "text-gray-200" : "text-gray-800"}` },
product.itemName), /* @__PURE__ */ React.createElement("td", { className: `px-6 py-4 text-center text-sm
${theme === "dark" ? "text-gray-400" : "text-gray-600"}` }, product.category), /* @__PURE__ */
React.createElement("td", { className: `px-6 py-4 text-center text-sm ${theme === "dark" ? "text-gray-400" :
"text-gray-600"}` }, product.variant || "-"), /* @__PURE__ */ React.createElement("td", { className: `px-6
py-4 text-center text-sm font-medium ${theme === "dark" ? "text-gray-300" : "text-gray-800"}` }, "PHP ",
(product.itemPrice || 0).toFixed(2)), /* @__PURE__ */ React.createElement("td", { className: "px-6 py-4 text-
center" }, /* @__PURE__ */ React.createElement(
      "span",
      {
        className: `inline-flex items-center justify-center w-8 h-8 rounded-md text-sm font-bold
${(product.currentStock || 0) === 0 ? "bg-red-200 text-red-800" : (product.currentStock || 0) <= 10 ? "bg-
[#FCD8B6] text-[#C05621]" : "bg-green-100 text-green-800"}`
      },
      product.currentStock || 0
    )))
  )))))));
};
export default ViewBrandProductsModal;
```

FILE: pos/frontend/src/components/owner/ViewCategoryProductsModal.jsx
==========

```
import { useEffect, useState } from "react";
import { FaTimes } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const ViewCategoryProductsModal = ({ isOpen, onClose, categoryName }) => {
```

```
  const { theme } = useTheme();
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(false);
  useEffect(() => {
    if (isOpen && categoryName) {
      fetchCategoryProducts();
    }
  }, [isOpen, categoryName]);
  const fetchCategoryProducts = async () => {
    if (!categoryName) return;
    try {
      setLoading(true);
      const response = await fetch("http://localhost:5000/api/products");
      const data = await response.json();
      if (data.success) {
        const categoryProducts = data.data.filter(
          (product) => product.category === categoryName
        );
        setProducts(categoryProducts);
      }
    } catch (error) {
      console.error("Error fetching category products:", error);
    } finally {
      setLoading(false);
    }
  };
  if (!isOpen || !categoryName) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10002] p-4 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement("div", { className: `rounded-
2xl w-full max-w-5xl max-h-[90vh] overflow-hidden shadow-2xl flex flex-col ${theme === "dark" ? "bg-[#1E1B18]
text-white" : "bg-white text-gray-900"}` }, /* @__PURE__ */ React.createElement("div", { className: `px-6 py-
4 border-b flex items-center justify-between ${theme === "dark" ? "border-gray-700" : "border-gray-200"}` },
/* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("h2", { className:
`text-2xl font-bold ${theme === "dark" ? "text-white" : "text-black"}` }, "Products in ", categoryName), /*
@__PURE__ */ React.createElement("p", { className: `text-sm mt-1 ${theme === "dark" ? "text-gray-400" :
"text-gray-500"}` }, products.length, " product(s) found")), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onClose,
      className: "text-gray-400 hover:text-gray-600 transition-colors"
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
  )), /* @__PURE__ */ React.createElement("div", { className: "flex-1 overflow-y-auto px-6 py-6" }, loading ?
/* @__PURE__ */ React.createElement("div", { className: "flex justify-center items-center py-12" }, /*
@__PURE__ */ React.createElement("div", { className: "text-gray-500" }, "Loading products...")) :
products.length === 0 ? /* @__PURE__ */ React.createElement("div", { className: "flex justify-center items-
center py-12" }, /* @__PURE__ */ React.createElement("div", { className: "text-gray-500" }, "No products
found in this category.")) : /* @__PURE__ */ React.createElement("div", { className: "overflow-x-auto" }, /*
@__PURE__ */ React.createElement("table", { className: "w-full border-collapse" }, /* @__PURE__ */
React.createElement("thead", null, /* @__PURE__ */ React.createElement(
    "tr",
    {
      className: `text-sm font-bold uppercase tracking-wider ${theme === "dark" ? "bg-[#3A3734] text-gray-
200" : "bg-[#EAE0D5] text-[#4A403A]"}`
    },
    /* @__PURE__ */ React.createElement("th", { className: "px-6 py-4 text-center font-bold" }, "Image"),
    /* @__PURE__ */ React.createElement("th", { className: "px-6 py-4 text-center font-bold" }, "SKU"),
    /* @__PURE__ */ React.createElement("th", { className: "px-6 py-4 text-left font-bold" }, "Item Name"),
    /* @__PURE__ */ React.createElement("th", { className: "px-6 py-4 text-center font-bold" }, "Brand"),
    /* @__PURE__ */ React.createElement("th", { className: "px-6 py-4 text-center font-bold" }, "Variant"),
    /* @__PURE__ */ React.createElement("th", { className: "px-6 py-4 text-center font-bold" }, "Price"),
    /* @__PURE__ */ React.createElement("th", { className: "px-6 py-4 text-center font-bold" }, "Stock")
  )), /* @__PURE__ */ React.createElement("tbody", null, products.map(
    (product) => /* @__PURE__ */ React.createElement("tr", { key: product._id, className: `border-b
transition-colors ${theme === "dark" ? "border-gray-800 hover:bg-[#2A2724]" : "border-gray-100 hover:bg-gray-
50"}` }, /* @__PURE__ */ React.createElement("td", { className: "px-6 py-4 text-center" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-center" }, product.itemImage ? /* @__PURE__ */
React.createElement(
      "img",
      {
        src: product.itemImage,
        alt: product.itemName,
        className: "w-12 h-12 object-cover rounded-lg shadow-sm"
      }
    ) : /* @__PURE__ */ React.createElement("div", { className: "w-12 h-12 bg-gray-100 rounded-lg flex items-
center justify-center text-gray-400 text-xs font-medium" }, "No img"))), /* @__PURE__ */
React.createElement("td", { className: `px-6 py-4 text-center font-medium ${theme === "dark" ? "text-gray-
```

```
300" : "text-gray-600"}` }, product.sku), /* @__PURE__ */ React.createElement("td", { className: `px-6 py-4
text-left font-medium ${theme === "dark" ? "text-white" : "text-gray-900"}` }, product.itemName), /*
@__PURE__ */ React.createElement("td", { className: `px-6 py-4 text-center ${theme === "dark" ? "text-gray-
400" : "text-gray-600"}` }, product.brandName || "-"), /* @__PURE__ */ React.createElement("td", { className:
`px-6 py-4 text-center ${theme === "dark" ? "text-gray-400" : "text-gray-600"}` }, product.variant || "-"),
/* @__PURE__ */ React.createElement("td", { className: `px-6 py-4 text-center font-medium ${theme === "dark" ?
"text-gray-300" : "text-gray-900"}` }, "PHP ", (product.itemPrice || 0).toFixed(2)), /* @__PURE__ */
React.createElement("td", { className: "px-6 py-4 text-center" }, /* @__PURE__ */ React.createElement("div",
{ className: "flex justify-center" }, /* @__PURE__ */ React.createElement(
      "span",
      {
        className: `w-8 h-8 flex items-center justify-center rounded-lg text-sm font-bold shadow-sm
${product.currentStock === 0 ? "bg-[#FCA5A5] text-red-900" : product.currentStock <= (product.reorderNumber
|| 10) ? "bg-[#FED7AA] text-orange-900" : "bg-[#86EFAC] text-green-900"}`
      },
      product.currentStock || 0
    ))))
  )))))));
};
export default ViewCategoryProductsModal;
```

FILE: pos/frontend/src/components/owner/ViewEmployeeModal.jsx
==========

```
import {
  FaArchive,
  FaCalendar,
  FaEdit,
  FaEnvelope,
  FaPhone,
  FaTimes,
  FaUndo,
  FaUser
} from "react-icons/fa";
import defaultAvatar from "../../assets/default.jpeg";
const ViewEmployeeModal = ({
  isOpen,
  onClose,
  employee,
  onEdit,
  onResetPin,
  onToggleStatus
}) => {
  if (!isOpen || !employee) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10002] p-4 backdrop-blur-sm bg-black/20" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "bg-white w-full max-w-2xl relative shadow-2xl overflow-hidden animate-fadeIn",
      style: { borderRadius: "24px" }
    },
    /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between px-8 py-6
border-b border-gray-100" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-
3" }, /* @__PURE__ */ React.createElement("div", { className: "w-10 h-10 rounded-xl bg-[#AD7F65] flex items-
center justify-center text-white" }, /* @__PURE__ */ React.createElement(FaUser, { className: "w-5 h-5" })),
/* @__PURE__ */ React.createElement("h2", { className: "text-xl font-bold text-gray-800" }, "Employee
Profile")), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: "text-gray-400 hover:text-gray-600 transition-colors p-2 hover:bg-gray-100 rounded-full"
      },
      /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
    )),
    /* @__PURE__ */ React.createElement("div", { className: "p-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-start gap-8 mb-10" }, /* @__PURE__ */
React.createElement("div", { className: "shrink-0 relative" }, /* @__PURE__ */ React.createElement("div",
{ className: "w-32 h-32 rounded-full overflow-hidden border-4 border-white shadow-lg" }, /* @__PURE__ */
React.createElement(
      "img",
      {
        src: employee.image || defaultAvatar,
        alt: employee.name,
        className: "w-full h-full object-cover",
        onError: (e) => {
          e.currentTarget.onerror = null;
```

```
          e.currentTarget.src = defaultAvatar;
        }
      }
    })), /* @__PURE__ */ React.createElement("div", { className: "flex-1 pt-2" }, /* @__PURE__ */
React.createElement("h3", { className: "text-3xl font-bold text-gray-900 mb-1" }, employee.name), /*
@__PURE__ */ React.createElement("p", { className: "text-[#AD7F65] font-medium mb-4" }, employee.role), /*
@__PURE__ */ React.createElement("div", { className: "space-y-4" }, /* @__PURE__ */ React.createElement("div",
null, /* @__PURE__ */ React.createElement("label", { className: "text-xs font-semibold text-gray-400
uppercase tracking-wider mb-2 block" }, "Permissions:"), /* @__PURE__ */ React.createElement("div",
{ className: "flex gap-2 flex-wrap" }, employee.permissions?.posTerminal && /* @__PURE__ */
React.createElement("span", { className: "px-3 py-1 rounded-full text-xs font-medium border border-[#E5E7EB]
text-gray-600 bg-gray-50" }, "POS Terminal"), employee.permissions?.inventory && /* @__PURE__ */
React.createElement("span", { className: "px-3 py-1 rounded-full text-xs font-medium border border-[#E5E7EB]
text-gray-600 bg-gray-50" }, "Inventory"), employee.permissions?.viewTransactions && /* @__PURE__ */
React.createElement("span", { className: "px-3 py-1 rounded-full text-xs font-medium border border-[#E5E7EB]
text-gray-600 bg-gray-50" }, "View Transactions"), employee.permissions?.generateReports && /* @__PURE__ */
React.createElement("span", { className: "px-3 py-1 rounded-full text-xs font-medium border border-[#E5E7EB]
text-gray-600 bg-gray-50" }, "Generate Reports"), !employee.permissions?.posTerminal
&& !employee.permissions?.inventory && !employee.permissions?.viewTransactions
&& !employee.permissions?.generateReports && /* @__PURE__ */ React.createElement("span", { className: "text-
xs text-gray-400 italic" }, "No active permissions"))), /* @__PURE__ */ React.createElement("div", null, /*
@__PURE__ */ React.createElement("label", { className: "text-xs font-semibold text-gray-400 uppercase
tracking-wider mb-2 block" }, "Status:"), /* @__PURE__ */ React.createElement(
      "span",
      {
        className: `inline-block px-3 py-1 rounded-full text-xs font-bold ${employee.status === "Active" ?
"bg-green-100 text-green-700" : "bg-red-100 text-red-700"}`
      },
      employee.status
    )))))), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-x-12 gap-y-8 mb-
10" }, /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className:
"text-sm font-medium text-gray-400 mb-1" }, "Name"), /* @__PURE__ */ React.createElement("p", { className:
"text-lg font-bold text-gray-800" }, employee.name)), /* @__PURE__ */ React.createElement("div", null, /*
@__PURE__ */ React.createElement("p", { className: "text-sm font-medium text-gray-400 mb-1" }, "Contact
number"), /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-2 text-gray-800
font-bold text-lg" }, /* @__PURE__ */ React.createElement(FaPhone, { className: "w-4 h-4 text-gray-400" }),
employee.contactNumber || "N/A")), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("p", { className: "text-sm font-medium text-gray-400 mb-1" }, "Email"), /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-2 text-gray-800 font-bold text-lg truncate" },
/* @__PURE__ */ React.createElement(FaEnvelope, { className: "w-4 h-4 text-gray-400" }), employee.email ||
"N/A")), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p",
{ className: "text-sm font-medium text-gray-400 mb-1" }, "Date Joined"), /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-2 text-gray-800 font-bold text-lg" }, /*
@__PURE__ */ React.createElement(FaCalendar, { className: "w-4 h-4 text-gray-400" }), (() => {
      const dateValue = employee.dateJoinedActual || employee.dateJoined || employee.createdAt;
      return dateValue ? new Date(dateValue).toLocaleDateString("en-US", {
        month: "short",
        day: "2-digit",
        year: "numeric"
      }) : "N/A";
    })())), /* @__PURE__ */ React.createElement("div", { className: "col-span-2" }, /* @__PURE__ */
React.createElement("p", { className: "text-sm font-medium text-gray-400 mb-1" }, "Position"), /* @__PURE__
*/ React.createElement("p", { className: "text-lg font-bold text-gray-800" }, "Employee - ", employee.role))),
/* @__PURE__ */ React.createElement("div", { className: "flex justify-end items-center gap-3 pt-6 border-t
border-gray-100" }, employee.role !== "Owner" && /* @__PURE__ */ React.createElement(React.Fragment, null, /*
@__PURE__ */ React.createElement(
      "button",
      {
        onClick: onResetPin,
        className: "px-4 py-2 bg-blue-100 text-blue-600 rounded-lg font-bold text-sm hover:bg-blue-200
transition-colors"
      },
      "Reset PIN"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onEdit,
        className: "w-10 h-10 flex items-center justify-center bg-[#007AFF] text-white rounded-lg hover:bg-
blue-600 transition-colors shadow-sm",
        title: "Edit"
      },
      /* @__PURE__ */ React.createElement(FaEdit, { className: "w-4 h-4" })
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onToggleStatus,
        className: `w-10 h-10 flex items-center justify-center rounded-lg text-white hover:opacity-90
```

```
transition-colors shadow-sm ${employee.status === "Active" ? "bg-[#FFA500]" : "bg-[#10B981]"}`,
        title: employee.status === "Active" ? "Archive" : "Restore"
      },
      employee.status === "Active" ? /* @__PURE__ */ React.createElement(FaArchive, { className: "w-4 h-
4" }) : /* @__PURE__ */ React.createElement(FaUndo, { className: "w-4 h-4" })
    ))))
  ));
};
export default ViewEmployeeModal;
```

FILE: pos/frontend/src/components/shared/CategoryButtons.jsx
==========

```
import React from "react";
import { useTheme } from "../../context/ThemeContext";
export default function CategoryButtons({
  categories = [],
  selectedCategory,
  onSelect,
  size = "md"
}) {
  const { theme } = useTheme();
  return /* @__PURE__ */ React.createElement("div", { className: "flex gap-3 flex-wrap" },
categories.map((cat) => {
    const isActive = selectedCategory === cat.name;
    return /* @__PURE__ */ React.createElement(
      "button",
      {
        key: cat.name,
        onClick: () => onSelect(cat.name),
        className: `px-4 py-2 rounded-lg transition-all font-medium text-sm ${isActive ? "text-white shadow-
md" : theme === "dark" ? "bg-[#1E1B18] text-gray-300 hover:bg-[#2A2724] border border-gray-700" : "bg-white
text-gray-700 hover:bg-gray-50 border border-gray-200"}`,
        style: {
          background: isActive ? "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)" : void 0
        }
      },
      cat.name
    );
  }));
}
```

FILE: pos/frontend/src/components/shared/header.jsx
==========

```
import { useContext, useEffect, useRef, useState } from "react";
import { createPortal } from "react-dom";
import toast from "react-hot-toast";
import {
  FaBell,
  FaExclamationTriangle,
  FaSearch,
  FaTimes,
  FaTimesCircle
} from "react-icons/fa";
import { useNavigate } from "react-router-dom";
import filterIcon from "../../assets/filter.svg";
import { useAuth } from "../../context/AuthContext";
import { API_BASE_URL } from "../../config/api";
import { SidebarContext } from "../../context/SidebarContext";
import { useTheme } from "../../context/ThemeContext";
const Header = ({
  pageName,
  subtitle,
  showSearch = false,
  showFilter = false,
  searchValue = "",
  onSearchChange = null,
  userName = "Barbie",
  userRole = "Staff",
  profileBackground = "bg-white",
  showBorder = true,
  className = "",
  hidePageName = false,
  centerProfile = false,
```

```
  filterNextToSearch = false,
  profileMinWidth = "300px",
  profilePadding = "px-7",
  profileGap = "gap-4",
  sortOption = "newest",
  onSortChange = null,
  categories = [],
  selectedCategory = "All",
  onCategoryChange = null,
  showTimeframeFilter = false,
  timeframeValue = "Daily",
  onTimeframeChange = null,
  timeframeOptions = ["Daily", "Weekly", "Monthly"]
}) => {
  const { isExpanded: sidebarExpanded } = useContext(SidebarContext) || {
    isExpanded: false
  };
  const { currentUser } = useAuth();
  const { theme } = useTheme();
  const [internalSearch, setInternalSearch] = useState(searchValue);
  const [lowStockItems, setLowStockItems] = useState([]);
  const [showSortDropdown, setShowSortDropdown] = useState(false);
  const sortButtonRef = useRef(null);
  const [dismissedItems, setDismissedItems] = useState(() => {
    try {
      const saved = localStorage.getItem("dismissedLowStockAlerts");
      return saved ? JSON.parse(saved) : {};
    } catch {
      return {};
    }
  });
  const [showNotifications, setShowNotifications] = useState(false);
  const [dropdownPosition, setDropdownPosition] = useState({
    top: 0,
    right: 0
  });
  const notificationRef = useRef(null);
  const buttonRef = useRef(null);
  const navigate = useNavigate();
  const [isOnline, setIsOnline] = useState(navigator.onLine);
  useEffect(() => {
    const handleOnline = () => {
      setIsOnline(true);
      toast.success("You are back online");
    };
    const handleOffline = () => {
      setIsOnline(false);
      toast.error("You are offline. Some features may be unavailable.");
    };
    window.addEventListener("online", handleOnline);
    window.addEventListener("offline", handleOffline);
    return () => {
      window.removeEventListener("online", handleOnline);
      window.removeEventListener("offline", handleOffline);
    };
  }, []);
  const DISMISS_COOLDOWN_MS = 5 * 60 * 60 * 1e3;
  const isDismissExpired = (dismissTimestamp) => {
    if (!dismissTimestamp) return true;
    const now = Date.now();
    return now - dismissTimestamp >= DISMISS_COOLDOWN_MS;
  };
  useEffect(() => {
    if (showNotifications && buttonRef.current) {
      const rect = buttonRef.current.getBoundingClientRect();
      setDropdownPosition({
        top: rect.bottom + 8,
        right: window.innerWidth - rect.right
      });
    }
  }, [showNotifications]);
  useEffect(() => {
    const fetchLowStock = async () => {
      try {
        const response = await fetch(
          `${API_BASE_URL}/api/products/low-stock`
        );
```

```
      const data = await response.json();
      if (data.success) {
        const items = data.data || [];
        setLowStockItems(items);
        const currentLowStockIds = new Set(
          items.map((item) => item.alertKey || item._id)
        );
        setDismissedItems((prev) => {
          const updated = { ...prev };
          let changed = false;
          Object.keys(updated).forEach((id) => {
            if (!currentLowStockIds.has(id)) {
              delete updated[id];
              changed = true;
            } else if (isDismissExpired(updated[id])) {
              delete updated[id];
              changed = true;
            }
          });
          if (changed) {
            localStorage.setItem(
              "dismissedLowStockAlerts",
              JSON.stringify(updated)
            );
          }
          return changed ? updated : prev;
        });
      }
    } catch (error) {
      console.error("Error fetching low stock items:", error);
    }
  };
  fetchLowStock();
  const interval = setInterval(fetchLowStock, 3e4);
  return () => clearInterval(interval);
}, []);
const dismissNotification = (e, itemId) => {
  e.stopPropagation();
  setDismissedItems((prev) => {
    const updated = { ...prev, [itemId]: Date.now() };
    localStorage.setItem("dismissedLowStockAlerts", JSON.stringify(updated));
    return updated;
  });
};
const visibleLowStockItems = lowStockItems.filter((item) => {
  const dismissTimestamp = dismissedItems[item.alertKey || item._id];
  return !dismissTimestamp || isDismissExpired(dismissTimestamp);
});
const getAlertVisuals = (item) => {
  switch (item?.alertType) {
    case "out_of_stock":
      return {
        badgeBg: "bg-red-100",
        icon: /* @__PURE__ */ React.createElement(FaTimesCircle, { className: "text-red-500" }),
        valueClass: "text-red-600",
        label: "out of stock",
        value: item.currentStock
      };
    case "expired":
      return {
        badgeBg: "bg-red-100",
        icon: /* @__PURE__ */ React.createElement(FaTimesCircle, { className: "text-red-500" }),
        valueClass: "text-red-600",
        label: "expired",
        value: "0d"
      };
    case "expiring_soon":
      return {
        badgeBg: "bg-orange-100",
        icon: /* @__PURE__ */ React.createElement(FaExclamationTriangle, { className: "text-orange-500" }),
        valueClass: "text-orange-500",
        label: "expiring soon",
        value: item.daysUntilExpiration <= 0 ? "today" : `${item.daysUntilExpiration}d`
      };
    default:
      return {
        badgeBg: "bg-orange-100",
```

```
          icon: /* @__PURE__ */ React.createElement(FaExclamationTriangle, { className: "text-orange-500" }),
          valueClass: "text-orange-500",
          label: "low stock",
          value: item.currentStock
        };
    }
  };
  const getNotificationSubtext = (item) => {
    if (item.alertType === "expired" && item.expirationDate) {
      return `Expired: ${item.expirationDate}`;
    }
    if (item.alertType === "expiring_soon" && item.expirationDate) {
      return `Expires: ${item.expirationDate}`;
    }
    return `SKU: ${item.sku}`;
  };
  useEffect(() => {
    const handleClickOutside = (event) => {
      const isOutsideDropdown = notificationRef.current && !notificationRef.current.contains(event.target);
      const isOutsideButton = buttonRef.current && !buttonRef.current.contains(event.target);
      if (isOutsideDropdown && isOutsideButton) {
        setShowNotifications(false);
      }
      if (showSortDropdown && sortButtonRef.current && !sortButtonRef.current.contains(event.target)) {
        setShowSortDropdown(false);
      }
    };
    if (showNotifications || showSortDropdown) {
      document.addEventListener("mousedown", handleClickOutside);
    }
    return () => document.removeEventListener("mousedown", handleClickOutside);
  }, [showNotifications, showSortDropdown]);
  useEffect(() => {
    setInternalSearch(searchValue);
  }, [searchValue]);
  const handleSearchChange = (event) => {
    if (onSearchChange) {
      onSearchChange(event);
    } else {
      setInternalSearch(event.target.value);
    }
  };
  const resolvedSearchValue = onSearchChange ? searchValue : internalSearch;
  const fullNameFromUser = (currentUser?.name || `${currentUser?.firstName || ""} ${currentUser?.lastName ||
""}`)?.trim();
  const displayName = fullNameFromUser || userName;
  const displayRole = currentUser?.role || userRole;
  const userId = currentUser?._id || currentUser?.id;
  const fallbackAvatarUrl = `https://ui-avatars.com/api/?name=${encodeURIComponent(
    displayName || "User"
  )}&background=AD7F65&color=fff`;
  const profileImageSrc = currentUser?.image?.startsWith("data:image") ||
currentUser?.profileImage?.startsWith("data:image") ? currentUser.image || currentUser.profileImage : userId ?
`${API_BASE_URL}/api/employees/${userId}/image?v=${new Date(currentUser?.updatedAt ||
currentUser?.lastUpdated || 0).getTime()}` : fallbackAvatarUrl;
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `relative flex items-center justify-between z-[100] ${showBorder ? `mb-6 pb-4 border-b
${theme === "dark" ? "border-gray-700" : "border-gray-200"}` : ""} ${className}`
    },
    /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-4" }, showSearch ? /*
@__PURE__ */ React.createElement(React.Fragment, null, !hidePageName && /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement(
      "h1",
      {
        className: `text-3xl font-bold ${theme === "dark" ? "text-white" : "text-gray-900"}`
      },
      pageName
    ), subtitle && /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-sm mt-1 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
      },
      subtitle
    )), /* @__PURE__ */ React.createElement(
      "div",
```

```
      {
        className: `relative transition-all duration-300 ease-in-out ${sidebarExpanded ? "w-[480px]" : "w-
[700px]"}`
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "absolute left-1 top-1/2 transform -translate-y-1/2 w-10 h-9 flex items-center justify-
center text-white rounded-xl",
          style: {
            background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)"
          }
        },
        /* @__PURE__ */ React.createElement(FaSearch, { className: "text-sm" })
      ),
      /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "text",
          placeholder: "Search For...",
          value: resolvedSearchValue,
          onChange: handleSearchChange,
          className: `w-full h-11 pl-14 pr-4 border rounded-xl shadow-md focus:outline-none focus:ring-2
focus:ring-[#AD7F65] focus:border-transparent transition-colors ${theme === "dark" ? "bg-[#2A2724] border-
gray-600 text-white placeholder-gray-400" : "bg-white border-gray-300 text-gray-900"}`
        }
      )
    ), categories.length > 0 && onCategoryChange && /* @__PURE__ */ React.createElement("div", { className:
"relative ml-2" }, /* @__PURE__ */ React.createElement(
      "select",
      {
        value: selectedCategory,
        onChange: (e) => onCategoryChange(e.target.value),
        className: `h-11 pl-4 pr-8 border rounded-xl shadow-md focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] focus:border-transparent transition-colors appearance-none cursor-pointer text-sm font-medium
${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white" : "bg-white border-gray-300 text-gray-700"}`,
        style: { minWidth: "150px" }
      },
      categories.map(
        (cat) => /* @__PURE__ */ React.createElement("option", { key: cat.name, value: cat.name }, cat.name
=== "All" ? "By Category" : cat.name)
      )
    ), /* @__PURE__ */ React.createElement("div", { className: "absolute right-3 top-1/2 transform -
translate-y-1/2 pointer-events-none" }, /* @__PURE__ */ React.createElement(
      "svg",
      {
        className: `w-4 h-4 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`,
        fill: "none",
        stroke: "currentColor",
        viewBox: "0 0 24 24"
      },
      /* @__PURE__ */ React.createElement(
        "path",
        {
          strokeLinecap: "round",
          strokeLinejoin: "round",
          strokeWidth: 2,
          d: "M19 9l-7 7-7-7"
        }
      )
    ))), filterNextToSearch && showFilter && /* @__PURE__ */ React.createElement(
      "button",
      {
        className: `ml-2 w-12 h-10 rounded-2xl flex items-center justify-center border shadow-
[0_8px_16px_rgba(0,0,0,0.12)] hover:shadow-[0_12px_22px_rgba(0,0,0,0.16)] transition-colors ${theme ===
"dark" ? "bg-[#2A2724] border-gray-600" : "bg-white border-gray-100"}`
      },
      /* @__PURE__ */ React.createElement(
        "img",
        {
          src: filterIcon,
          alt: "Filter",
          className: "w-5 h-5 opacity-90"
        }
      )
    )) : /* @__PURE__ */ React.createElement(React.Fragment, null, !hidePageName && /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement(
```

```
      "h1",
      {
        className: `text-3xl font-bold ${theme === "dark" ? "text-white" : "text-gray-900"}`
      },
      pageName
    ), subtitle && /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-sm mt-1 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
      },
      subtitle
    )), showTimeframeFilter && /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "flex gap-2 ml-4",
        role: "tablist",
        "aria-label": "Dashboard timeframe filter"
      },
      timeframeOptions.map(
        (option) => /* @__PURE__ */ React.createElement(
          "button",
          {
            key: option,
            onClick: () => onTimeframeChange && onTimeframeChange(option),
            role: "tab",
            "aria-selected": timeframeValue === option,
            className: `px-4 py-2 rounded-lg text-sm font-medium transition-colors focus:outline-none
focus:ring-2 focus:ring-[#AD7F65] focus:ring-offset-2 ${timeframeValue === option ? "bg-[#AD7F65] text-
white" : theme === "dark" ? "bg-[#2A2724] text-gray-300 hover:bg-[#352F2A]" : "bg-gray-100 text-gray-700
hover:bg-gray-200"}`
          },
          option
        )
      )
    ))),
    centerProfile && /* @__PURE__ */ React.createElement("div", { className: "absolute left-1/2 -translate-x-
1/2" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-4" }, /* @__PURE__ */
React.createElement("div", { className: "relative" }, /* @__PURE__ */ React.createElement(
      "button",
      {
        ref: buttonRef,
        onClick: () => setShowNotifications(!showNotifications),
        className: `w-12 h-12 rounded-full flex items-center justify-center shadow-
[0_10px_20px_rgba(0,0,0,0.12)] hover:shadow-[0_14px_26px_rgba(0,0,0,0.16)] relative transition-all`,
        style: theme === "dark" ? { backgroundColor: "#2A2521" } : {}
      },
      /* @__PURE__ */ React.createElement(
        FaBell,
        {
          className: `text-2xl ${theme === "dark" ? "text-gray-300" : "text-gray-800"}`
        }
      ),
      visibleLowStockItems.length > 0 && /* @__PURE__ */ React.createElement("span", { className: "absolute -
top-1 -right-1 min-w-[20px] h-5 bg-red-500 text-white text-xs font-bold rounded-full flex items-center
justify-center px-1" }, visibleLowStockItems.length > 99 ? "99+" : visibleLowStockItems.length)
    ), showNotifications && createPortal(
      /* @__PURE__ */ React.createElement(
        "div",
        {
          ref: notificationRef,
          className: `fixed w-80 rounded-2xl shadow-2xl border overflow-hidden ${theme === "dark" ? "bg-
[#2A2724] border-gray-600" : "bg-white border-gray-100"}`,
          style: {
            top: dropdownPosition.top,
            right: dropdownPosition.right,
            zIndex: 99999
          }
        },
        /* @__PURE__ */ React.createElement("div", { className: `px-4 py-3 border-b flex items-center
justify-between ${theme === "dark" ? "border-gray-600 text-white" : "border-gray-100 text-gray-800"}` }, /*
@__PURE__ */ React.createElement("span", { className: "font-semibold" }, "Inventory Alerts"), /* @__PURE__ */
React.createElement(
          "button",
          {
            onClick: () => setShowNotifications(false),
            className: `rounded-full p-1 transition-colors ${theme === "dark" ? "hover:bg-gray-700 text-gray-
400 hover:text-white" : "hover:bg-gray-100 text-gray-500 hover:text-gray-700"}`
```

```
          },
          /* @__PURE__ */ React.createElement(FaTimes, { className: "text-sm" })
        )),
        /* @__PURE__ */ React.createElement("div", { className: "max-h-80 overflow-y-auto" },
visibleLowStockItems.length === 0 ? /* @__PURE__ */ React.createElement("div", { className: "px-4 py-8 text-
center text-gray-500" }, /* @__PURE__ */ React.createElement(FaBell, { className: "text-4xl mx-auto mb-2
text-gray-300" }), /* @__PURE__ */ React.createElement("p", null, "No stock alerts")) :
visibleLowStockItems.map((item) => {
          const visuals = getAlertVisuals(item);
          const alertKey = item.alertKey || item._id;
          return /* @__PURE__ */ React.createElement(
            "div",
            {
              key: alertKey,
              className: "px-4 py-3 hover:bg-gray-50 border-b border-gray-100 last:border-b-0 flex items-
center gap-3"
            },
            /* @__PURE__ */ React.createElement(
              "div",
              {
                className: "flex items-center gap-3 flex-1 cursor-pointer",
                onClick: () => {
                  setShowNotifications(false);
                  navigate("/inventory");
                }
              },
              /* @__PURE__ */ React.createElement(
                "div",
                {
                  className: `w-10 h-10 rounded-lg flex items-center justify-center flex-shrink-0
${visuals.badgeBg}`
                },
                visuals.icon
              ),
              /* @__PURE__ */ React.createElement("div", { className: "flex-1 min-w-0" }, /* @__PURE__ */
React.createElement("p", { className: "text-sm font-medium text-gray-800 truncate" }, item.itemName), /*
@__PURE__ */ React.createElement("p", { className: "text-xs text-gray-500" }, getNotificationSubtext(item))),
              /* @__PURE__ */ React.createElement("div", { className: "text-right flex-shrink-0" }, /*
@__PURE__ */ React.createElement(
                "p",
                {
                  className: `text-sm font-bold ${visuals.valueClass}`
                },
                visuals.value
              ), /* @__PURE__ */ React.createElement("p", { className: "text-xs text-gray-400" },
visuals.label))
            ),
            /* @__PURE__ */ React.createElement(
              "button",
              {
                onClick: (e) => dismissNotification(e, alertKey),
                className: "p-1.5 hover:bg-gray-200 rounded-full transition-colors flex-shrink-0",
                title: "Dismiss"
              },
              /* @__PURE__ */ React.createElement(FaTimes, { className: "text-gray-400 text-xs" })
            )
          );
        })),
        visibleLowStockItems.length > 0 && /* @__PURE__ */ React.createElement("div", { className: "px-4 py-2
bg-gray-50 border-t border-gray-100" }, /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: () => {
              setShowNotifications(false);
              navigate("/inventory");
            },
            className: "w-full text-center text-sm text-[#76462B] font-medium hover:underline"
          },
          "View All in Inventory"
        ))
      ),
      document.body
    )), /* @__PURE__ */ React.createElement(
      "div",
      {
        onClick: () => navigate("/settings"),
        className: `flex items-center cursor-pointer ${profileGap} ${profilePadding} py-2.5 rounded-full
```

```
border shadow-[0_6px_14px_rgba(0,0,0,0.15)]`,
        style: {
          minWidth: profileMinWidth,
          ...theme === "dark" ? {
            background: "rgba(42, 37, 33, 0.9)",
            backdropFilter: "blur(6px)",
            border: "1px solid #3A332E"
          } : {}
        }
      },
      /* @__PURE__ */ React.createElement("div", { className: "relative" }), /* @__PURE__ */
React.createElement(
        "img",
        {
          src: profileImageSrc,
          alt: "User",
          onError: (e) => {
            e.target.onerror = null;
            e.target.src = fallbackAvatarUrl;
          },
          className: `w-10 h-10 rounded-full ring-2 shadow-sm object-cover ${theme === "dark" ? "ring-gray-
600" : "ring-white"}`
        }
      ), /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `absolute bottom-0 right-0 w-3 h-3 rounded-full border-2 ${theme === "dark" ? "border-
[#2A2521]" : "border-white"} ${isOnline ? "bg-green-500" : "bg-red-500"}`,
          title: isOnline ? "Online" : "Offline"
        }
      )),
      /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `text-sm font-semibold ${theme === "dark" ? "text-white" : "text-gray-900"}`
        },
        displayName
      ), /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        displayRole
      ))
    ))),
    /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-5" }, !centerProfile &&
showFilter && !filterNextToSearch && /* @__PURE__ */ React.createElement("div", { className: "relative", ref:
sortButtonRef }, /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => setShowSortDropdown(!showSortDropdown),
        className: `w-12 h-12 rounded-2xl flex items-center justify-center border shadow-
[0_10px_20px_rgba(0,0,0,0.12)] hover:shadow-[0_14px_26px_rgba(0,0,0,0.16)] ${sortOption !== "newest" ?
"border-[#AD7F65] bg-[#AD7F65]/10" : theme === "dark" ? "bg-[#2A2724] border-gray-600" : "bg-white border-
gray-100"}`
      },
      /* @__PURE__ */ React.createElement(
        "img",
        {
          src: filterIcon,
          alt: "Filter",
          className: `w-5 h-5 ${theme === "dark" ? "opacity-90 invert" : "opacity-90"}`
        }
      )
    ), showSortDropdown && /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `absolute top-full right-0 mt-2 w-48 rounded-xl shadow-2xl border overflow-hidden z-[9999]
${theme === "dark" ? "bg-[#1E1B18] border-gray-600" : "bg-white border-gray-200"}`
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `px-3 py-2 border-b ${theme === "dark" ? "bg-[#2A2724] border-gray-600" : "bg-gray-50
border-gray-200"}`
        },
        /* @__PURE__ */ React.createElement(
```

```
          "span",
          {
            className: `text-xs font-semibold ${theme === "dark" ? "text-gray-300" : "text-gray-600"}`
          },
          "Sort By"
        )
      ),
      /* @__PURE__ */ React.createElement("div", { className: "py-1" }, /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => {
            onSortChange && onSortChange("a-z");
            setShowSortDropdown(false);
          },
          className: `w-full px-4 py-2 text-left text-sm flex items-center justify-between ${sortOption ===
"a-z" ? "text-[#AD7F65] font-medium bg-[#AD7F65]/5" : theme === "dark" ? "text-gray-300 hover:bg-[#2A2724]" :
"text-gray-700 hover:bg-gray-50"}`
        },
        "Name (A-Z)",
        sortOption === "a-z" && /* @__PURE__ */ React.createElement("span", { className: "text-[#AD7F65]" },
"\u2713")
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => {
            onSortChange && onSortChange("z-a");
            setShowSortDropdown(false);
          },
          className: `w-full px-4 py-2 text-left text-sm flex items-center justify-between ${sortOption ===
"z-a" ? "text-[#AD7F65] font-medium bg-[#AD7F65]/5" : theme === "dark" ? "text-gray-300 hover:bg-[#2A2724]" :
"text-gray-700 hover:bg-gray-50"}`
        },
        "Name (Z-A)",
        sortOption === "z-a" && /* @__PURE__ */ React.createElement("span", { className: "text-[#AD7F65]" },
"\u2713")
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => {
            onSortChange && onSortChange("newest");
            setShowSortDropdown(false);
          },
          className: `w-full px-4 py-2 text-left text-sm flex items-center justify-between ${sortOption ===
"newest" ? "text-[#AD7F65] font-medium bg-[#AD7F65]/5" : theme === "dark" ? "text-gray-300 hover:bg-
[#2A2724]" : "text-gray-700 hover:bg-gray-50"}`
        },
        "Newest First",
        sortOption === "newest" && /* @__PURE__ */ React.createElement("span", { className: "text-
[#AD7F65]" }, "\u2713")
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => {
            onSortChange && onSortChange("oldest");
            setShowSortDropdown(false);
          },
          className: `w-full px-4 py-2 text-left text-sm flex items-center justify-between ${sortOption ===
"oldest" ? "text-[#AD7F65] font-medium bg-[#AD7F65]/5" : theme === "dark" ? "text-gray-300 hover:bg-
[#2A2724]" : "text-gray-700 hover:bg-gray-50"}`
        },
        "Oldest First",
        sortOption === "oldest" && /* @__PURE__ */ React.createElement("span", { className: "text-
[#AD7F65]" }, "\u2713")
      ))
    )), !centerProfile && /* @__PURE__ */ React.createElement("div", { className: "relative", ref:
notificationRef }, /* @__PURE__ */ React.createElement(
      "button",
      {
        ref: buttonRef,
        onClick: () => setShowNotifications(!showNotifications),
        className: "w-12 h-12 rounded-full flex items-center justify-center shadow-
[0_10px_20px_rgba(0,0,0,0.12)] hover:shadow-[0_14px_26px_rgba(0,0,0,0.16)] relative transition-all",
        style: theme === "dark" ? { backgroundColor: "#2A2521" } : { backgroundColor: "white" }
      },
      /* @__PURE__ */ React.createElement(
        FaBell,
        {
```

```
          className: `text-2xl ${theme === "dark" ? "text-gray-300" : "text-gray-800"}`
        }
      ),
      visibleLowStockItems.length > 0 && /* @__PURE__ */ React.createElement("span", { className: "absolute -
top-1 -right-1 min-w-[20px] h-5 bg-red-500 text-white text-xs font-bold rounded-full flex items-center
justify-center px-1" }, visibleLowStockItems.length > 99 ? "99+" : visibleLowStockItems.length)
    ), showNotifications && createPortal(
      /* @__PURE__ */ React.createElement(
        "div",
        {
          ref: notificationRef,
          className: "fixed w-80 bg-white rounded-2xl shadow-2xl border border-gray-100 overflow-hidden",
          style: {
            top: dropdownPosition.top,
            right: dropdownPosition.right,
            zIndex: 99999
          }
        },
        /* @__PURE__ */ React.createElement("div", { className: `px-4 py-3 border-b flex items-center
justify-between ${theme === "dark" ? "border-gray-600 text-white" : "border-gray-100 text-gray-800"}` }, /*
@__PURE__ */ React.createElement("span", { className: "font-semibold" }, "Inventory Alerts"), /* @__PURE__ */
React.createElement(
          "button",
          {
            onClick: () => setShowNotifications(false),
            className: `rounded-full p-1 transition-colors ${theme === "dark" ? "hover:bg-gray-700 text-gray-
400 hover:text-white" : "hover:bg-gray-100 text-gray-500 hover:text-gray-700"}`
          },
          /* @__PURE__ */ React.createElement(FaTimes, { className: "text-sm" })
        )),
        /* @__PURE__ */ React.createElement("div", { className: "max-h-80 overflow-y-auto" },
visibleLowStockItems.length === 0 ? /* @__PURE__ */ React.createElement("div", { className: "px-4 py-8 text-
center text-gray-500" }, /* @__PURE__ */ React.createElement(FaBell, { className: "text-4xl mx-auto mb-2
text-gray-300" }), /* @__PURE__ */ React.createElement("p", null, "No stock alerts")) :
visibleLowStockItems.map((item) => {
          const visuals = getAlertVisuals(item);
          const alertKey = item.alertKey || item._id;
          return /* @__PURE__ */ React.createElement(
            "div",
            {
              key: alertKey,
              className: "px-4 py-3 hover:bg-gray-50 border-b border-gray-100 last:border-b-0 flex items-
center gap-3"
            },
            /* @__PURE__ */ React.createElement(
              "div",
              {
                className: "flex items-center gap-3 flex-1 cursor-pointer",
                onClick: () => {
                  setShowNotifications(false);
                  navigate("/inventory");
                }
              },
              /* @__PURE__ */ React.createElement(
                "div",
                {
                  className: `w-10 h-10 rounded-lg flex items-center justify-center flex-shrink-0
${visuals.badgeBg}`
                },
                visuals.icon
              ),
              /* @__PURE__ */ React.createElement("div", { className: "flex-1 min-w-0" }, /* @__PURE__ */
React.createElement("p", { className: "text-sm font-medium text-gray-800 truncate" }, item.itemName), /*
@__PURE__ */ React.createElement("p", { className: "text-xs text-gray-500" }, getNotificationSubtext(item))),
              /* @__PURE__ */ React.createElement("div", { className: "text-right flex-shrink-0" }, /*
@__PURE__ */ React.createElement(
                "p",
                {
                  className: `text-sm font-bold ${visuals.valueClass}`
                },
                visuals.value
              ), /* @__PURE__ */ React.createElement("p", { className: "text-xs text-gray-400" },
visuals.label))
            ),
            /* @__PURE__ */ React.createElement(
              "button",
              {
```

```
                onClick: (e) => dismissNotification(e, alertKey),
                className: "p-1.5 hover:bg-gray-200 rounded-full transition-colors flex-shrink-0",
                title: "Dismiss"
              },
              /* @__PURE__ */ React.createElement(FaTimes, { className: "text-gray-400 text-xs" })
            )
          );
        })),
        visibleLowStockItems.length > 0 && /* @__PURE__ */ React.createElement("div", { className: "px-4 py-2
bg-gray-50 border-t border-gray-100" }, /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: () => {
              setShowNotifications(false);
              navigate("/inventory");
            },
            className: "w-full text-center text-sm text-[#76462B] font-medium hover:underline"
          },
          "View All in Inventory"
        ))
      ),
      document.body
    )), !centerProfile && /* @__PURE__ */ React.createElement(
      "div",
      {
        onClick: () => navigate("/settings"),
        className: `flex items-center cursor-pointer ${profileGap} ${profilePadding} py-2.5 rounded-full
shadow-[0_6px_14px_rgba(0,0,0,0.15)] ${theme === "dark" ? "border" : ""}`,
        style: {
          minWidth: profileMinWidth,
          ...theme === "dark" ? {
            background: "rgba(42, 37, 33, 0.9)",
            backdropFilter: "blur(6px)",
            border: "1px solid #3A332E"
          } : {}
        }
      },
      /* @__PURE__ */ React.createElement("div", { className: "relative" }, /* @__PURE__ */
React.createElement(
        "img",
        {
          src: profileImageSrc,
          alt: "User",
          onError: (e) => {
            e.target.onerror = null;
            e.target.src = fallbackAvatarUrl;
          },
          className: `w-10 h-10 rounded-full ring-2 shadow-sm object-cover ${theme === "dark" ? "ring-gray-
600" : "ring-white"}`
        }
      ), /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `absolute bottom-0 right-0 w-3 h-3 rounded-full border-2 ${theme === "dark" ? "border-
white" : "border-white"} ${isOnline ? "bg-green-500" : "bg-red-500"}`,
          title: isOnline ? "Online" : "Offline"
        }
      )),
      /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `text-sm font-semibold ${theme === "dark" ? "text-white" : "text-gray-900"}`
        },
        displayName
      ), /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        displayRole
      ))
    ))
  );
};
export default Header;
```

FILE: pos/frontend/src/components/shared/LogoutConfirmationModal.jsx
==========

```
import React from "react";
import { FaTimes } from "react-icons/fa";
const LogoutConfirmationModal = ({ isOpen, onClose, onConfirm }) => {
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex items-center justify-
center z-[10002] p-4 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "bg-white rounded-2xl w-full max-w-md relative shadow-2xl",
      style: { boxShadow: "0 25px 50px -12px rgba(0, 0, 0, 0.5)" }
    },
    /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: "absolute top-4 right-4 text-gray-400 hover:text-gray-600 transition-colors z-10"
      },
      /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
    ),
    /* @__PURE__ */ React.createElement("div", { className: "p-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-center mb-6" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "w-20 h-20 rounded-full flex items-center justify-center relative",
        style: {
          backgroundColor: "#E8D5C4",
          boxShadow: "0 4px 12px rgba(118, 70, 43, 0.2)"
        }
      },
      /* @__PURE__ */ React.createElement(
        "span",
        {
          className: "text-4xl font-bold text-[#76462B]",
          style: {
            animation: "questionPulse 0.6s ease-out"
          }
        },
        "?"
      ),
      /* @__PURE__ */ React.createElement("style", null, `
                @keyframes questionPulse {
                  0% {
                    opacity: 0;
                    transform: scale(0) rotate(-10deg);
                  }
                  50% {
                    opacity: 1;
                    transform: scale(1.2) rotate(5deg);
                  }
                  100% {
                    opacity: 1;
                    transform: scale(1) rotate(0deg);
                  }
                }
              `)
    )), /* @__PURE__ */ React.createElement("h3", { className: "text-xl font-bold text-gray-800 text-center
mb-3" }, "Logout Account"), /* @__PURE__ */ React.createElement("p", { className: "text-sm text-gray-500
text-center mb-8" }, "Are you sure you want to logout?"), /* @__PURE__ */ React.createElement("div",
{ className: "flex gap-4" }, /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onConfirm,
        className: "flex-1 py-3 px-6 rounded-lg font-bold text-white transition-all shadow-md hover:shadow-
lg",
        style: {
          background: "linear-gradient(135deg, #D4A59A 0%, #AD7F65 50%, #76462B 100%)"
        }
      },
      "Confirm"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: "flex-1 py-3 px-6 rounded-lg font-bold text-gray-700 bg-gray-200 hover:bg-gray-300
```

```
transition-all shadow-sm"
      },
      "Cancel"
    )))
  ));
};
export default LogoutConfirmationModal;
```

FILE: pos/frontend/src/components/shared/PageTitle.jsx
==========

```
import { useEffect } from "react";
import { useLocation } from "react-router-dom";
const PageTitle = () => {
  const location = useLocation();
  useEffect(() => {
    const path = location.pathname;
    let title = "CYSPOS";
    switch (path) {
      case "/":
        title = "Login - CYSPOS";
        break;
      case "/pin":
        title = "Enter PIN - CYSPOS";
        break;
      case "/staff":
        title = "Select Staff - CYSPOS";
        break;
      case "/set-pin":
        title = "Set PIN - CYSPOS";
        break;
      case "/dashboard":
        title = "Dashboard - CYSPOS";
        break;
      case "/reports":
        title = "Reports - CYSPOS";
        break;
      case "/manage-employees":
        title = "Manage Employees - CYSPOS";
        break;
      case "/inventory":
        title = "Inventory - CYSPOS";
        break;
      case "/stock-movement":
        title = "Stock Movement - CYSPOS";
        break;
      case "/terminal":
        title = "Terminal - CYSPOS";
        break;
      case "/transactions":
        title = "Transactions - CYSPOS";
        break;
      case "/settings":
        title = "Settings - CYSPOS";
        break;
      case "/discount-management":
        title = "Discounts - CYSPOS";
        break;
      case "/brand-partners":
        title = "Brand Partners - CYSPOS";
        break;
      case "/categories":
        title = "Categories - CYSPOS";
        break;
      default:
        if (path.startsWith("/inventory")) {
          title = "Inventory - CYSPOS";
        } else {
          title = "CYSPOS";
        }
        break;
    }
    document.title = title;
  }, [location]);
  return null;
};
```

```
export default PageTitle;
```

FILE: pos/frontend/src/components/shared/ProtectedRoute.jsx
==========

```
import { Navigate, useLocation } from "react-router-dom";
import { useAuth } from "../../context/AuthContext";
const ProtectedRoute = ({
  children,
  requiredPermission,
  ownerOnly = false
}) => {
  const { currentUser, hasPermission, isOwner } = useAuth();
  const location = useLocation();
  if (!currentUser) {
    return /* @__PURE__ */ React.createElement(Navigate, { to: "/", replace: true });
  }
  if (currentUser.requiresPinReset && location.pathname !== "/set-pin") {
    return /* @__PURE__ */ React.createElement(Navigate, { to: "/set-pin", replace: true });
  }
  if (isOwner()) {
    return children;
  }
  if (ownerOnly) {
    if (hasPermission("posTerminal")) {
      return /* @__PURE__ */ React.createElement(Navigate, { to: "/terminal", replace: true });
    } else if (hasPermission("inventory")) {
      return /* @__PURE__ */ React.createElement(Navigate, { to: "/inventory", replace: true });
    } else if (hasPermission("viewTransactions")) {
      return /* @__PURE__ */ React.createElement(Navigate, { to: "/transactions", replace: true });
    } else if (hasPermission("generateReports")) {
      return /* @__PURE__ */ React.createElement(Navigate, { to: "/reports", replace: true });
    } else {
      return /* @__PURE__ */ React.createElement(Navigate, { to: "/settings", replace: true });
    }
  }
  if (requiredPermission) {
    if (!hasPermission(requiredPermission)) {
      if (hasPermission("posTerminal")) {
        return /* @__PURE__ */ React.createElement(Navigate, { to: "/terminal", replace: true });
      } else if (hasPermission("inventory")) {
        return /* @__PURE__ */ React.createElement(Navigate, { to: "/inventory", replace: true });
      } else if (hasPermission("viewTransactions")) {
        return /* @__PURE__ */ React.createElement(Navigate, { to: "/transactions", replace: true });
      } else if (hasPermission("generateReports")) {
        return /* @__PURE__ */ React.createElement(Navigate, { to: "/reports", replace: true });
      } else {
        return /* @__PURE__ */ React.createElement(Navigate, { to: "/settings", replace: true });
      }
    }
  }
  return children;
};
export default ProtectedRoute;
```

FILE: pos/frontend/src/components/shared/Sidebar.jsx
==========

```
import { memo, useEffect, useRef, useState } from "react";
import { createPortal } from "react-dom";
import { Link, useLocation } from "react-router-dom";
import inventoryIcon from "../../assets/icons/invenory.svg";
import logoutIcon from "../../assets/icons/Logout.svg";
import settingsIcon from "../../assets/icons/Settings.svg";
import terminalIcon from "../../assets/icons/terminal.svg";
import transactionIcon from "../../assets/icons/transaction.svg";
import dashboardIcon from "../../assets/owner/dashboard.svg";
import manageIcon from "../../assets/owner/manage.svg";
import reportsIcon from "../../assets/owner/reports.svg";
import { useAuth } from "../../context/AuthContext";
import { useTheme } from "../../context/ThemeContext";
import LogoutConfirmationModal from "./LogoutConfirmationModal";
import logo from "../../assets/logo.png";
const SidebarTooltip = ({ label, targetRef, show, theme }) => {
  const [position, setPosition] = useState({ top: 0, left: 0 });
```

```
  useEffect(() => {
    if (show && targetRef.current) {
      const rect = targetRef.current.getBoundingClientRect();
      setPosition({
        top: rect.top + rect.height / 2,
        left: rect.right + 12
      });
    }
  }, [show, targetRef]);
  if (!show) return null;
  return createPortal(
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `fixed px-3 py-2 rounded-lg text-sm font-medium shadow-lg whitespace-nowrap z-[9999]
transform -translate-y-1/2 pointer-events-none ${theme === "dark" ? "bg-[#2A2724] text-gray-200 border
border-[#4A4037]" : "bg-white text-gray-700"}`,
        style: {
          top: position.top,
          left: position.left,
          boxShadow: "0 4px 12px rgba(0, 0, 0, 0.15)"
        }
      },
      label
    ),
    document.body
  );
};
const Sidebar = ({ isExpanded, setIsExpanded }) => {
  const location = useLocation();
  const { theme } = useTheme();
  const [showLogoutModal, setShowLogoutModal] = useState(false);
  const [inventoryExpanded, setInventoryExpanded] = useState(false);
  const [posExpanded, setPosExpanded] = useState(false);
  const [transactionsExpanded, setTransactionsExpanded] = useState(false);
  const [hoveredItem, setHoveredItem] = useState(null);
  const itemRefs = useRef({});
  const { logout, isOwner, hasPermission } = useAuth();
  const allMenuItems = [
    {
      name: "Dashboard",
      icon: dashboardIcon,
      path: "/dashboard",
      gradient: "linear-gradient(135deg, #C2A68C 0%, #AD7F65 50%, #76462B 100%)",
      ownerOnly: true,
      requiredPermission: null
    },
    {
      name: "POS",
      icon: terminalIcon,
      path: "/terminal",
      gradient: "linear-gradient(135deg, #C2A68C 0%, #AD7F65 50%, #76462B 100%)",
      ownerOnly: false,
      requiredPermission: "posTerminal"
    },
    {
      name: "Inventory",
      icon: inventoryIcon,
      path: "/inventory",
      gradient: "linear-gradient(135deg, #C2A68C 0%, #AD7F65 50%, #76462B 100%)",
      ownerOnly: false,
      requiredPermission: "inventory"
    },
    {
      name: "Transactions",
      icon: transactionIcon,
      path: "/transactions",
      gradient: "linear-gradient(135deg, #C2A68C 0%, #AD7F65 50%, #76462B 100%)",
      ownerOnly: false,
      requiredPermission: "viewTransactions"
    },
    {
      name: "Reports / Analytics",
      icon: reportsIcon,
      path: "/reports",
      gradient: "linear-gradient(135deg, #C2A68C 0%, #AD7F65 50%, #76462B 100%)",
      ownerOnly: false,
```

```
    requiredPermission: "generateReports"
  },
  {
    name: "Manage Employees",
    icon: manageIcon,
    path: "/manage-employees",
    gradient: "linear-gradient(135deg, #C2A68C 0%, #AD7F65 50%, #76462B 100%)",
    ownerOnly: true,
    requiredPermission: null
  },
  {
    name: "Settings",
    icon: settingsIcon,
    path: "/settings",
    gradient: "linear-gradient(135deg, #C2A68C 0%, #AD7F65 50%, #76462B 100%)",
    ownerOnly: false,
    requiredPermission: null
  }
];
const menuItems = allMenuItems.filter((item) => {
  if (isOwner()) {
    return true;
  }
  if (item.ownerOnly) {
    return false;
  }
  if (!item.requiredPermission) {
    return true;
  }
  return hasPermission(item.requiredPermission);
});
const handleLogout = () => {
  setShowLogoutModal(true);
};
const confirmLogout = () => {
  console.log("Logging out...");
  setShowLogoutModal(false);
  logout();
};
const isActive = (path) => {
  return location.pathname === path;
};
const isInventoryActive = () => {
  const inventoryPaths = [
    "/inventory",
    "/stock-movement",
    "/brand-partners",
    "/categories"
  ];
  return inventoryPaths.some((path) => location.pathname === path);
};
const isPosActive = () => {
  const posPaths = ["/terminal", "/discount-management"];
  return posPaths.some((path) => location.pathname === path);
};
const isTransactionsActive = () => {
  const transPaths = ["/transactions", "/cash-remittance"];
  return transPaths.some((path) => location.pathname === path);
};
const inventorySubItems = [
  { name: "Products", path: "/inventory" },
  { name: "Logs", path: "/stock-movement" },
  { name: "Brand Partners", path: "/brand-partners" },
  { name: "Categories", path: "/categories" }
];
const posSubItems = [
  { name: "Terminal", path: "/terminal" },
  { name: "Discount Management", path: "/discount-management" }
];
const transactionsSubItems = [
  { name: "All Transactions", path: "/transactions" },
  { name: "Cash Remittance", path: "/cash-remittance" }
];
useEffect(() => {
  const inventoryPaths = [
    "/inventory",
    "/stock-movement",
```

```
      "/brand-partners",
      "/categories"
    ];
    if (inventoryPaths.some((path) => location.pathname === path)) {
      setInventoryExpanded(true);
    }
  }, [location.pathname]);
  useEffect(() => {
    const posPaths = ["/terminal", "/discount-management"];
    if (posPaths.some((path) => location.pathname === path)) {
      setPosExpanded(true);
    }
  }, [location.pathname]);
  useEffect(() => {
    const transPaths = ["/transactions", "/cash-remittance"];
    if (transPaths.some((path) => location.pathname === path)) {
      setTransactionsExpanded(true);
    }
  }, [location.pathname]);
  return /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `fixed left-0 top-0 h-screen transition-all duration-300 ease-in-out z-50 flex flex-col
cursor-pointer ${isExpanded ? "w-70" : "w-20"} ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}`,
      style: {
        borderRadius: "0 30px 30px 0",
        boxShadow: "0 4px 24px rgba(0, 0, 0, 0.2)"
      },
      onClick: () => {
        setIsExpanded(!isExpanded);
      }
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `relative flex items-center px-4 py-4 overflow-hidden ${isExpanded ? "" : "justify-
center"}`,
        style: { minHeight: "120px" },
        onClick: (e) => {
          e.stopPropagation();
        }
      },
      /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: (e) => {
            e.stopPropagation();
            setIsExpanded(!isExpanded);
          },
          className: `p-2 rounded-lg transition-all duration-300 shrink-0 z-10 ${theme === "dark" ?
"hover:bg-[#352F2A]" : "hover:bg-gray-100"}`
        },
        /* @__PURE__ */ React.createElement("div", { className: "flex flex-col justify-center items-center w-
6 h-6 shrink-0" }, /* @__PURE__ */ React.createElement(
          "span",
          {
            className: `block h-0.5 w-5 rounded transition-all duration-300 ${theme === "dark" ? "bg-gray-
400" : "bg-gray-600"}`
          }
        ), /* @__PURE__ */ React.createElement(
          "span",
          {
            className: `block h-0.5 w-5 rounded transition-all duration-300 my-1 ${theme === "dark" ? "bg-
gray-400" : "bg-gray-600"}`
          }
        ), /* @__PURE__ */ React.createElement(
          "span",
          {
            className: `block h-0.5 w-5 rounded transition-all duration-300 ${theme === "dark" ? "bg-gray-
400" : "bg-gray-600"}`
          }
        ))
      ),
      isExpanded && /* @__PURE__ */ React.createElement("div", { className: "flex-1 flex items-center
justify-center pointer-events-none" }, /* @__PURE__ */ React.createElement(
        "img",
        {
```

```
          src: logo,
          alt: "Create Your Style Logo",
          className: "h-20 w-auto object-contain transition-all duration-300",
          style: {
            filter: "invert(53%) sepia(23%) saturate(828%) hue-rotate(343deg) brightness(92%) contrast(91%)"
          }
        }
      ))
    ),
    /* @__PURE__ */ React.createElement(
      "nav",
      {
        className: `flex-1 pb-8 px-2 ${isExpanded ? "pt-15 overflow-y-auto overflow-x-hidden" : "pt-15
overflow-visible"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "space-y-3" }, menuItems.map((item) => {
        if (item.name === "POS") {
          const posActive = isPosActive();
          const hasPosPermission = isOwner() || hasPermission("posTerminal");
          if (!hasPosPermission) return null;
          return /* @__PURE__ */ React.createElement("div", { key: item.path, className: "space-y-1" },
isExpanded ? /* @__PURE__ */ React.createElement(
            "button",
            {
              onClick: (e) => {
                e.stopPropagation();
                setPosExpanded(!posExpanded);
              },
              className: `w-full flex items-center justify-between rounded-2xl transition-all duration-300
group relative overflow-hidden py-3.5 ${posActive ? "shadow-lg" : theme === "dark" ? "hover:bg-[#352F2A]" :
"hover:bg-gray-50"}`,
              style: posActive ? {
                background: item.gradient,
                boxShadow: "0 4px 12px rgba(118, 70, 43, 0.25)"
              } : {}
            },
            /* @__PURE__ */ React.createElement("div", { className: "flex items-center flex-1" }, /*
@__PURE__ */ React.createElement("div", { className: "shrink-0 w-7 h-7 flex items-center justify-center ml-
4" }, /* @__PURE__ */ React.createElement(
              "img",
              {
                src: item.icon,
                alt: item.name,
                className: `w-6 h-6 transition-all duration-300 ${posActive ? "brightness-0 invert" : theme
=== "dark" ? "brightness-0 invert opacity-90 group-hover:opacity-100" : "opacity-80 group-hover:opacity-
100"}`
              }
            )), isExpanded && /* @__PURE__ */ React.createElement(
              "span",
              {
                className: `font-medium transition-all duration-300 whitespace-nowrap ml-4 ${posActive ?
"text-white" : theme === "dark" ? "text-gray-200 group-hover:text-[#C2A68C]" : "text-gray-800 group-
hover:text-[#76462B]"}`,
                style: {
                  fontSize: "16px"
                }
              },
              item.name
            )),
            isExpanded && /* @__PURE__ */ React.createElement(
              "svg",
              {
                className: `w-5 h-5 mr-4 transition-transform duration-300 ${posExpanded ? "rotate-180" : ""}
${posActive ? "text-white" : theme === "dark" ? "text-gray-400" : "text-gray-600"}`,
                fill: "none",
                stroke: "currentColor",
                viewBox: "0 0 24 24"
              },
              /* @__PURE__ */ React.createElement(
                "path",
                {
                  strokeLinecap: "round",
                  strokeLinejoin: "round",
                  strokeWidth: 2,
                  d: "M19 9l-7 7-7-7"
                }
              )
```

```
            ),
            posActive && /* @__PURE__ */ React.createElement(
              "div",
              {
                className: "absolute inset-0 rounded-2xl",
                style: {
                  background: "linear-gradient(180deg, rgba(255,255,255,0.15) 0%, rgba(255,255,255,0) 50%,
rgba(0,0,0,0.08) 100%)",
                  pointerEvents: "none"
                }
              }
            )
          ) : /* @__PURE__ */ React.createElement(
            Link,
            {
              to: "/terminal",
              ref: (el) => itemRefs.current["pos-terminal"] = el,
              onClick: (e) => {
                e.stopPropagation();
              },
              onMouseEnter: () => !isExpanded && setHoveredItem("pos-terminal"),
              onMouseLeave: () => setHoveredItem(null),
              className: `w-full flex items-center justify-between rounded-2xl transition-all duration-300
group relative overflow-hidden py-3.5 ${posActive ? "shadow-lg" : theme === "dark" ? "hover:bg-[#352F2A]" :
"hover:bg-gray-50"}`,
              style: posActive ? {
                background: item.gradient,
                boxShadow: "0 4px 12px rgba(118, 70, 43, 0.25)"
              } : {}
            },
            /* @__PURE__ */ React.createElement("div", { className: "flex items-center flex-1" }), /*
@__PURE__ */ React.createElement("div", { className: "shrink-0 w-7 h-7 flex items-center justify-center ml-4
relative" }), /* @__PURE__ */ React.createElement(
              "img",
              {
                src: item.icon,
                alt: item.name,
                className: `w-6 h-6 transition-all duration-300 ${posActive ? "brightness-0 invert" : theme
=== "dark" ? "brightness-0 invert opacity-90 group-hover:opacity-100" : "opacity-80 group-hover:opacity-
100"}`
              }
            ))),
            /* @__PURE__ */ React.createElement(
              SidebarTooltip,
              {
                label: item.name,
                targetRef: {
                  current: itemRefs.current["pos-terminal"]
                },
                show: hoveredItem === "pos-terminal" && !isExpanded,
                theme
              }
            ),
            posActive && /* @__PURE__ */ React.createElement(
              "div",
              {
                className: "absolute inset-0 rounded-2xl",
                style: {
                  background: "linear-gradient(180deg, rgba(255,255,255,0.15) 0%, rgba(255,255,255,0) 50%,
rgba(0,0,0,0.08) 100%)",
                  pointerEvents: "none"
                }
              }
            )
          ), isExpanded && posExpanded && /* @__PURE__ */ React.createElement("div", { className: "ml-4
space-y-1" }, posSubItems.map((subItem) => {
            const subActive = isActive(subItem.path);
            return /* @__PURE__ */ React.createElement(
              Link,
              {
                key: subItem.path,
                to: subItem.path,
                onClick: (e) => {
                  e.stopPropagation();
                },
                className: `w-full flex items-center rounded-lg transition-all duration-300 group relative
overflow-hidden py-2.5 ${subActive ? theme === "dark" ? "bg-[#352F2A]" : "bg-[#F5E6D3]" : theme === "dark" ?
```

```
"hover:bg-[#352F2A]" : "hover:bg-gray-50"}`
              },
              subActive && /* @__PURE__ */ React.createElement("div", { className: "absolute left-0 top-0
bottom-0 w-1 bg-[#8B7355] rounded-r" }),
              /* @__PURE__ */ React.createElement(
                "span",
                {
                  className: `font-medium transition-all duration-300 whitespace-nowrap ml-6 ${subActive ?
theme === "dark" ? "text-[#C2A68C] font-semibold" : "text-[#76462B] font-semibold" : theme === "dark" ?
"text-gray-300 group-hover:text-[#C2A68C]" : "text-gray-700 group-hover:text-[#76462B]"}`,
                  style: {
                    fontSize: "15px"
                  }
                },
                subItem.name
              )
            );
          }))));
        }
        if (item.name === "Inventory") {
          const inventoryActive = isInventoryActive();
          const hasInventoryPermission = isOwner() || hasPermission("inventory");
          if (!hasInventoryPermission) return null;
          return /* @__PURE__ */ React.createElement("div", { key: item.path, className: "space-y-1" },
isExpanded ? /* @__PURE__ */ React.createElement(
            "button",
            {
              onClick: (e) => {
                e.stopPropagation();
                setInventoryExpanded(!inventoryExpanded);
              },
              className: `w-full flex items-center justify-between rounded-2xl transition-all duration-300
group relative overflow-hidden py-3.5 ${inventoryActive ? "shadow-lg" : theme === "dark" ? "hover:bg-
[#352F2A]" : "hover:bg-gray-50"}`,
              style: inventoryActive ? {
                background: item.gradient,
                boxShadow: "0 4px 12px rgba(118, 70, 43, 0.25)"
              } : {}
            },
            /* @__PURE__ */ React.createElement("div", { className: "flex items-center flex-1" }, /*
@__PURE__ */ React.createElement("div", { className: "shrink-0 w-7 h-7 flex items-center justify-center ml-
4" }, /* @__PURE__ */ React.createElement(
              "img",
              {
                src: item.icon,
                alt: item.name,
                className: `w-6 h-6 transition-all duration-300 ${inventoryActive ? "brightness-0 invert" :
theme === "dark" ? "brightness-0 invert opacity-90 group-hover:opacity-100" : "opacity-80 group-
hover:opacity-100"}`
              }
            )), isExpanded && /* @__PURE__ */ React.createElement(
              "span",
              {
                className: `font-medium transition-all duration-300 whitespace-nowrap ml-4 ${inventoryActive ?
"text-white" : theme === "dark" ? "text-gray-200 group-hover:text-[#C2A68C]" : "text-gray-800 group-
hover:text-[#76462B]"}`,
                style: {
                  fontSize: "16px"
                }
              },
              item.name
            ),
            isExpanded && /* @__PURE__ */ React.createElement(
              "svg",
              {
                className: `w-5 h-5 mr-4 transition-transform duration-300 ${inventoryExpanded ? "rotate-
180" : ""} ${inventoryActive ? "text-white" : theme === "dark" ? "text-gray-400" : "text-gray-600"}`,
                fill: "none",
                stroke: "currentColor",
                viewBox: "0 0 24 24"
              },
              /* @__PURE__ */ React.createElement(
                "path",
                {
                  strokeLinecap: "round",
                  strokeLinejoin: "round",
                  strokeWidth: 2,
```

```
                d: "M19 9l-7 7-7-7"
              }
            )
          ),
          inventoryActive && /* @__PURE__ */ React.createElement(
            "div",
            {
              className: "absolute inset-0 rounded-2xl",
              style: {
                background: "linear-gradient(180deg, rgba(255,255,255,0.15) 0%, rgba(255,255,255,0) 50%,
rgba(0,0,0,0.08) 100%)",
                pointerEvents: "none"
              }
            }
          )
        ) : /* @__PURE__ */ React.createElement(
          Link,
          {
            to: "/inventory",
            ref: (el) => itemRefs.current["inventory"] = el,
            onClick: (e) => {
              e.stopPropagation();
            },
            onMouseEnter: () => !isExpanded && setHoveredItem("inventory"),
            onMouseLeave: () => setHoveredItem(null),
            className: `w-full flex items-center justify-between rounded-2xl transition-all duration-300
group relative overflow-hidden py-3.5 ${inventoryActive ? "shadow-lg" : theme === "dark" ? "hover:bg-
[#352F2A]" : "hover:bg-gray-50"}`,
            style: inventoryActive ? {
              background: item.gradient,
              boxShadow: "0 4px 12px rgba(118, 70, 43, 0.25)"
            } : {}
          },
          /* @__PURE__ */ React.createElement("div", { className: "flex items-center flex-1" }, /*
@__PURE__ */ React.createElement("div", { className: "shrink-0 w-7 h-7 flex items-center justify-center ml-4
relative" }, /* @__PURE__ */ React.createElement(
            "img",
            {
              src: item.icon,
              alt: item.name,
              className: `w-6 h-6 transition-all duration-300 ${inventoryActive ? "brightness-0 invert" :
theme === "dark" ? "brightness-0 invert opacity-90 group-hover:opacity-100" : "opacity-80 group-
hover:opacity-100"}`
            }
          ))),
          /* @__PURE__ */ React.createElement(
            SidebarTooltip,
            {
              label: item.name,
              targetRef: { current: itemRefs.current["inventory"] },
              show: hoveredItem === "inventory" && !isExpanded,
              theme
            }
          ),
          inventoryActive && /* @__PURE__ */ React.createElement(
            "div",
            {
              className: "absolute inset-0 rounded-2xl",
              style: {
                background: "linear-gradient(180deg, rgba(255,255,255,0.15) 0%, rgba(255,255,255,0) 50%,
rgba(0,0,0,0.08) 100%)",
                pointerEvents: "none"
              }
            }
          )
        ), isExpanded && inventoryExpanded && /* @__PURE__ */ React.createElement("div", { className: "ml-4
space-y-1" }, inventorySubItems.map((subItem) => {
          const subActive = isActive(subItem.path);
          return /* @__PURE__ */ React.createElement(
            Link,
            {
              key: subItem.path,
              to: subItem.path,
              onClick: (e) => {
                e.stopPropagation();
              },
              className: `w-full flex items-center rounded-lg transition-all duration-300 group relative
```

```
overflow-hidden py-2.5 ${subActive ? theme === "dark" ? "bg-[#352F2A]" : "bg-[#F5E6D3]" : theme === "dark" ?
"hover:bg-[#352F2A]" : "hover:bg-gray-50"}`
              },
              subActive && /* @__PURE__ */ React.createElement("div", { className: "absolute left-0 top-0
bottom-0 w-1 bg-[#8B7355] rounded-r" }),
              /* @__PURE__ */ React.createElement(
                "span",
                {
                  className: `font-medium transition-all duration-300 whitespace-nowrap ml-6 ${subActive ?
theme === "dark" ? "text-[#C2A68C] font-semibold" : "text-[#76462B] font-semibold" : theme === "dark" ?
"text-gray-300 group-hover:text-[#C2A68C]" : "text-gray-700 group-hover:text-[#76462B]"}`,
                  style: {
                    fontSize: "15px"
                  }
                },
                subItem.name
              )
            );
          }))));
        }
        if (item.name === "Transactions") {
          const transActive = isTransactionsActive();
          const hasTransPermission = isOwner() || hasPermission("viewTransactions");
          if (!hasTransPermission) return null;
          return /* @__PURE__ */ React.createElement("div", { key: item.path, className: "space-y-1" },
isExpanded ? /* @__PURE__ */ React.createElement(
            "button",
            {
              onClick: (e) => {
                e.stopPropagation();
                setTransactionsExpanded(!transactionsExpanded);
              },
              className: `w-full flex items-center justify-between rounded-2xl transition-all duration-300
group relative overflow-hidden py-3.5 ${transActive ? "shadow-lg" : theme === "dark" ? "hover:bg-[#352F2A]" :
"hover:bg-gray-50"}`,
              style: transActive ? {
                background: item.gradient,
                boxShadow: "0 4px 12px rgba(118, 70, 43, 0.25)"
              } : {}
            },
            /* @__PURE__ */ React.createElement("div", { className: "flex items-center flex-1" }, /*
@__PURE__ */ React.createElement("div", { className: "shrink-0 w-7 h-7 flex items-center justify-center ml-
4" }, /* @__PURE__ */ React.createElement(
              "img",
              {
                src: item.icon,
                alt: item.name,
                className: `w-6 h-6 transition-all duration-300 ${transActive ? "brightness-0 invert" : theme
=== "dark" ? "brightness-0 invert opacity-90 group-hover:opacity-100" : "opacity-80 group-hover:opacity-
100"}`
              }
            )), isExpanded && /* @__PURE__ */ React.createElement(
              "span",
              {
                className: `font-medium transition-all duration-300 whitespace-nowrap ml-4 ${transActive ?
"text-white" : theme === "dark" ? "text-gray-200 group-hover:text-[#C2A68C]" : "text-gray-800 group-
hover:text-[#76462B]"}`,
                style: {
                  fontSize: "16px"
                }
              },
              item.name
            ),
            isExpanded && /* @__PURE__ */ React.createElement(
              "svg",
              {
                className: `w-5 h-5 mr-4 transition-transform duration-300 ${transactionsExpanded ? "rotate-
180" : ""} ${transActive ? "text-white" : theme === "dark" ? "text-gray-400" : "text-gray-600"}`,
                fill: "none",
                stroke: "currentColor",
                viewBox: "0 0 24 24"
              },
              /* @__PURE__ */ React.createElement(
                "path",
                {
                  strokeLinecap: "round",
                  strokeLinejoin: "round",
```

```
                  strokeWidth: 2,
                  d: "M19 9l-7 7-7-7"
                }
              )
            ),
            transActive && /* @__PURE__ */ React.createElement(
              "div",
              {
                className: "absolute inset-0 rounded-2xl",
                style: {
                  background: "linear-gradient(180deg, rgba(255,255,255,0.15) 0%, rgba(255,255,255,0) 50%,
rgba(0,0,0,0.08) 100%)",
                  pointerEvents: "none"
                }
              }
            )
          ) : /* @__PURE__ */ React.createElement(
            Link,
            {
              to: "/transactions",
              ref: (el) => itemRefs.current["transactions"] = el,
              onClick: (e) => {
                e.stopPropagation();
              },
              onMouseEnter: () => !isExpanded && setHoveredItem("transactions"),
              onMouseLeave: () => setHoveredItem(null),
              className: `w-full flex items-center justify-between rounded-2xl transition-all duration-300
group relative overflow-hidden py-3.5 ${transActive ? "shadow-lg" : theme === "dark" ? "hover:bg-[#352F2A]" :
"hover:bg-gray-50"}`,
              style: transActive ? {
                background: item.gradient,
                boxShadow: "0 4px 12px rgba(118, 70, 43, 0.25)"
              } : {}
            },
            /* @__PURE__ */ React.createElement("div", { className: "flex items-center flex-1" }), /*
@__PURE__ */ React.createElement("div", { className: "shrink-0 w-7 h-7 flex items-center justify-center ml-4
relative" }), /* @__PURE__ */ React.createElement(
              "img",
              {
                src: item.icon,
                alt: item.name,
                className: `w-6 h-6 transition-all duration-300 ${transActive ? "brightness-0 invert" : theme
=== "dark" ? "brightness-0 invert opacity-90 group-hover:opacity-100" : "opacity-80 group-hover:opacity-
100"}`
              }
            ))),
            /* @__PURE__ */ React.createElement(
              SidebarTooltip,
              {
                label: item.name,
                targetRef: { current: itemRefs.current["transactions"] },
                show: hoveredItem === "transactions" && !isExpanded,
                theme
              }
            ),
            transActive && /* @__PURE__ */ React.createElement(
              "div",
              {
                className: "absolute inset-0 rounded-2xl",
                style: {
                  background: "linear-gradient(180deg, rgba(255,255,255,0.15) 0%, rgba(255,255,255,0) 50%,
rgba(0,0,0,0.08) 100%)",
                  pointerEvents: "none"
                }
              }
            )
          ), isExpanded && transactionsExpanded && /* @__PURE__ */ React.createElement("div", { className:
"ml-4 space-y-1" }, transactionsSubItems.map((subItem) => {
            const subActive = isActive(subItem.path);
            return /* @__PURE__ */ React.createElement(
              Link,
              {
                key: subItem.path,
                to: subItem.path,
                onClick: (e) => {
                  e.stopPropagation();
                },
```

```
                className: `w-full flex items-center rounded-lg transition-all duration-300 group relative
overflow-hidden py-2.5 ${subActive ? theme === "dark" ? "bg-[#352F2A]" : "bg-[#F5E6D3]" : theme === "dark" ?
"hover:bg-[#352F2A]" : "hover:bg-gray-50"}`
              },
              subActive && /* @__PURE__ */ React.createElement("div", { className: "absolute left-0 top-0
bottom-0 w-1 bg-[#8B7355] rounded-r" }),
              /* @__PURE__ */ React.createElement(
                "span",
                {
                  className: `font-medium transition-all duration-300 whitespace-nowrap ml-6 ${subActive ?
theme === "dark" ? "text-[#C2A68C] font-semibold" : "text-[#76462B] font-semibold" : theme === "dark" ?
"text-gray-300 group-hover:text-[#C2A68C]" : "text-gray-700 group-hover:text-[#76462B]"}`,
                  style: {
                    fontSize: "15px"
                  }
                },
                subItem.name
              )
            );
          }))));
        }
        const active = isActive(item.path);
        return /* @__PURE__ */ React.createElement(
          Link,
          {
            key: item.path,
            to: item.path,
            ref: (el) => itemRefs.current[item.path] = el,
            onClick: (e) => {
              e.stopPropagation();
            },
            onMouseEnter: () => !isExpanded && setHoveredItem(item.path),
            onMouseLeave: () => setHoveredItem(null),
            className: `w-full flex items-center rounded-2xl transition-all duration-300 group relative
overflow-hidden py-3.5 ${active ? "shadow-lg" : theme === "dark" ? "hover:bg-[#352F2A]" : "hover:bg-gray-
50"}`,
            style: active ? {
              background: item.gradient,
              boxShadow: "0 4px 12px rgba(118, 70, 43, 0.25)"
            } : {}
          },
          /* @__PURE__ */ React.createElement("div", { className: "shrink-0 w-7 h-7 flex items-center
justify-center ml-4 relative" }, /* @__PURE__ */ React.createElement(
            "img",
            {
              src: item.icon,
              alt: item.name,
              className: `w-6 h-6 transition-all duration-300 ${active ? "brightness-0 invert" : theme ===
"dark" ? "brightness-0 invert opacity-90 group-hover:opacity-100" : "opacity-80 group-hover:opacity-100"}`
            }
          )),
          /* @__PURE__ */ React.createElement(
            SidebarTooltip,
            {
              label: item.name,
              targetRef: { current: itemRefs.current[item.path] },
              show: hoveredItem === item.path && !isExpanded,
              theme
            }
          ),
          isExpanded && /* @__PURE__ */ React.createElement(
            "span",
            {
              className: `font-medium transition-all duration-300 whitespace-nowrap ml-4 ${active ? "text-
white" : theme === "dark" ? "text-gray-200 group-hover:text-[#C2A68C]" : "text-gray-800 group-hover:text-
[#76462B]"}`,
              style: {
                fontSize: "16px"
              }
            },
            item.name
          ),
          active && /* @__PURE__ */ React.createElement(
            "div",
            {
              className: "absolute inset-0 rounded-2xl",
              style: {
```

```
                background: "linear-gradient(180deg, rgba(255,255,255,0.15) 0%, rgba(255,255,255,0) 50%,
rgba(0,0,0,0.08) 100%)",
                pointerEvents: "none"
              }
            }
          )
        );
      }))
    ),
    /* @__PURE__ */ React.createElement("div", { className: "py-6 px-2" }, /* @__PURE__ */
React.createElement(
      "button",
      {
        ref: (el) => itemRefs.current["logout"] = el,
        onClick: (e) => {
          e.stopPropagation();
          handleLogout();
        },
        onMouseEnter: () => !isExpanded && setHoveredItem("logout"),
        onMouseLeave: () => setHoveredItem(null),
        className: `w-full flex items-center rounded-2xl transition-all duration-300 group py-3.5 ${theme ===
"dark" ? "hover:bg-[#352F2A]" : "hover:bg-gray-50"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "shrink-0 w-7 h-7 flex items-center justify-
center ml-4 relative" }, /* @__PURE__ */ React.createElement(
        "img",
        {
          src: logoutIcon,
          alt: "Log Out",
          className: `w-6 h-6 transition-all duration-300 ${theme === "dark" ? "brightness-0 invert opacity-
90 group-hover:opacity-100" : "opacity-80 group-hover:opacity-100"}`
        }
      )),
      /* @__PURE__ */ React.createElement(
        SidebarTooltip,
        {
          label: "Log Out",
          targetRef: { current: itemRefs.current["logout"] },
          show: hoveredItem === "logout" && !isExpanded,
          theme
        }
      ),
      isExpanded && /* @__PURE__ */ React.createElement(
        "span",
        {
          className: `font-medium transition-all duration-300 whitespace-nowrap ml-4 ${theme === "dark" ?
"text-gray-200 group-hover:text-[#C2A68C]" : "text-gray-800 group-hover:text-[#76462B]"}`,
          style: {
            fontSize: "16px"
          }
        },
        "Log Out"
      )
    ))
  ), /* @__PURE__ */ React.createElement(
    LogoutConfirmationModal,
    {
      isOpen: showLogoutModal,
      onClose: () => setShowLogoutModal(false),
      onConfirm: confirmLogout
    }
  ));
};
export default memo(Sidebar);
```

FILE: pos/frontend/src/components/terminal/CashPaymentModal.jsx
==========

```
import { useEffect, useState } from "react";
import { FaTimes } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
import { sendReceiptToPrinter } from "../../utils/printBridge";
import CheckoutConfirmationModal from "./CheckoutConfirmationModal";
import PrintingModal from "./PrintingModal";
import ReceiptModal from "./ReceiptModal";
import SuccessModal from "./SuccessModal";
```

```
const CashPaymentModal = ({
  isOpen,
  onClose,
  totalAmount,
  subtotalAmount = 0,
  discountAmount = 0,
  selectedDiscounts = [],
  onProceed,
  onTransactionDone,
  cartItems = [],
  cashierName = ""
}) => {
  const { theme } = useTheme();
  const [amountReceived, setAmountReceived] = useState("");
  const [change, setChange] = useState(0);
  const [showConfirmation, setShowConfirmation] = useState(false);
  const [showSuccess, setShowSuccess] = useState(false);
  const [showReceipt, setShowReceipt] = useState(false);
  const [receiptData, setReceiptData] = useState(null);
  const [isAutoPrinting, setIsAutoPrinting] = useState(false);
  const [printError, setPrintError] = useState(null);
  const [amountError, setAmountError] = useState("");
  useEffect(() => {
    if (amountReceived && !isNaN(parseFloat(amountReceived))) {
      const received = parseFloat(amountReceived);
      const changeAmount = received - totalAmount;
      setChange(changeAmount >= 0 ? changeAmount : 0);
      setAmountError("");
    } else {
      setChange(0);
      if (amountReceived) {
        setAmountError("Please enter a valid amount.");
      } else {
        setAmountError("");
      }
    }
  }, [amountReceived, totalAmount]);
  useEffect(() => {
    if (!isOpen) {
      setAmountReceived("");
      setChange(0);
      setShowConfirmation(false);
      setShowSuccess(false);
      setShowReceipt(false);
      setReceiptData(null);
      setIsAutoPrinting(false);
      setPrintError(null);
      setAmountError("");
    }
  }, [isOpen]);
  if (!isOpen) return null;
  const generateReceiptNumber = () => {
    return Math.floor(1e5 + Math.random() * 9e5).toString();
  };
  const handleProceed = () => {
    const received = parseFloat(amountReceived);
    if (!amountReceived || isNaN(received) || received < totalAmount) {
      if (!amountReceived || isNaN(received)) {
        setAmountError("Please enter a valid amount.");
      } else if (received < totalAmount) {
        setAmountError("Amount must be equal to or greater than the total.");
      }
      return;
    }
    setAmountError("");
    setShowConfirmation(true);
  };
  const handleConfirmCheckout = async () => {
    const received = parseFloat(amountReceived);
    const changeAmount = Math.max(0, received - totalAmount);
    setShowConfirmation(false);
    setShowSuccess(true);
    try {
      const generatedReceiptNo = generateReceiptNumber();
      const savedTransaction = await onProceed(
        received,
        changeAmount,
```

```
        generatedReceiptNo
      );
      const actualReceiptNo = savedTransaction?.receiptNo || generatedReceiptNo;
      const receipt = {
        receiptNo: actualReceiptNo,
        items: cartItems.map((item) => ({
          name: item.itemName || item.name || "Item",
          qty: item.quantity || 1,
          price: item.itemPrice || item.price || 0,
          total: (item.itemPrice || item.price || 0) * (item.quantity || 1),
          selectedSize: item.selectedSize || item.size || "",
          size: item.selectedSize || item.size || "",
          selectedVariation: item.selectedVariation || item.variant || "",
          variant: item.selectedVariation || item.variant || ""
        })),
        paymentMethod: "CASH",
        cashierName: cashierName || "Staff",
        subtotal: subtotalAmount || totalAmount + discountAmount,
        discount: discountAmount,
        discounts: selectedDiscounts.map((d) => ({
          title: d.title,
          value: d.discountValue
        })),
        total: totalAmount,
        cash: received,
        change: changeAmount,
        date: (/* @__PURE__ */ new Date()).toLocaleDateString(),
        time: (/* @__PURE__ */ new Date()).toLocaleTimeString()
      };
      setReceiptData(receipt);
      setShowSuccess(false);
      requestAnimationFrame(() => {
        attemptAutoPrint(receipt);
      });
    } catch (error) {
      setShowSuccess(false);
      setShowReceipt(false);
      console.error("Error saving transaction:", error);
      alert("Failed to save transaction. Please try again.");
    }
  };
  const handleNewTransaction = () => {
    setShowReceipt(false);
    if (onTransactionDone) onTransactionDone();
  };
  const attemptAutoPrint = async (receipt) => {
    setIsAutoPrinting(true);
    setPrintError(null);
    try {
      await sendReceiptToPrinter(receipt);
      setIsAutoPrinting(false);
      handleNewTransaction();
    } catch (error) {
      setIsAutoPrinting(false);
      setPrintError(error.message || "Unable to reach the printer.");
      setShowReceipt(true);
    }
  };
  const handleAmountChange = (e) => {
    const value = e.target.value;
    if (value === "" || /^\d*\.?\d*$/.test(value)) {
      setAmountReceived(value);
    }
  };
  return /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("div",
{ className: "fixed inset-0 bg-opacity-50 backdrop-blur-sm flex items-center justify-center z-50 font-poppins
p-4" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-2xl w-full max-w-md relative shadow-2xl ${theme === "dark" ? "bg-[#1E1B18]" : "bg-
white"}`
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `px-6 py-4 border-b relative ${theme === "dark" ? "border-gray-700" : "border-gray-200"}`
      },
```

```
      /* @__PURE__ */ React.createElement(
        "h2",
        {
          className: `text-xl font-semibold ${theme === "dark" ? "text-white" : "text-gray-900"}`
        },
        "Cash Payment"
      ),
      /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onClose,
          className: `absolute top-4 right-4 transition-colors ${theme === "dark" ? "text-gray-400
hover:text-gray-200" : "text-gray-400 hover:text-gray-600"}`
        },
        /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
      )
    ),
    /* @__PURE__ */ React.createElement("div", { className: "p-6" }, /* @__PURE__ */
React.createElement("div", { className: "mb-6" }, /* @__PURE__ */ React.createElement("div", { className:
"mb-2" }, /* @__PURE__ */ React.createElement(
      "label",
      {
        className: `text-sm font-medium ${theme === "dark" ? "text-gray-300" : "text-gray-700"}`
      },
      "Total:"
    )), /* @__PURE__ */ React.createElement("div", { className: "text-4xl font-bold text-orange-500" },
"\u20B1", totalAmount.toFixed(2))), /* @__PURE__ */ React.createElement("div", { className: "mb-4" }, /*
@__PURE__ */ React.createElement(
      "label",
      {
        className: `block text-sm font-medium mb-2 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}`
      },
      "Amount Received:"
    ), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "text",
        value: amountReceived,
        onChange: handleAmountChange,
        placeholder: "0.00",
        className: `w-full px-4 py-3 text-lg border rounded-lg focus:outline-none focus:ring-2 focus:ring-
orange-500 focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white
placeholder-gray-500" : "border-gray-300 bg-white text-gray-900"}`,
        autoFocus: true
      }
    ), amountError && /* @__PURE__ */ React.createElement("p", { className: "mt-2 text-sm text-red-600" },
amountError)), /* @__PURE__ */ React.createElement("div", { className: "mb-6" }, /* @__PURE__ */
React.createElement(
      "label",
      {
        className: `block text-xs font-medium mb-2 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
      },
      "Quick Select:"
    ), /* @__PURE__ */ React.createElement("div", { className: "flex flex-wrap gap-2" }, /* @__PURE__ */
React.createElement(
      "button",
      {
        onClick: () => setAmountReceived(totalAmount.toString()),
        className: `px-3 py-2 text-sm font-medium rounded-lg border transition-colors ${theme === "dark" ?
"border-orange-600 bg-orange-900/30 text-orange-400 hover:bg-orange-900/50" : "border-orange-300 bg-orange-50
text-orange-600 hover:bg-orange-100"}`
      },
      "Exact \u20B1",
      totalAmount.toFixed(0)
    ), [20, 50, 100, 200, 500, 1e3, 2e3, 5e3].filter((amount) => amount >= totalAmount).slice(0, 4).map(
      (amount) => /* @__PURE__ */ React.createElement(
        "button",
        {
          key: amount,
          onClick: () => setAmountReceived(amount.toString()),
          className: `px-3 py-2 text-sm font-medium rounded-lg border transition-colors ${theme === "dark" ?
"border-gray-600 bg-[#2A2724] text-gray-300 hover:bg-[#322f2c]" : "border-gray-300 bg-gray-50 text-gray-700
hover:bg-gray-100"}`
        },
        "\u20B1",
        amount.toLocaleString()
      )
```

```
    ))), /* @__PURE__ */ React.createElement("div", { className: "mb-8" }, /* @__PURE__ */
React.createElement(
      "label",
      {
        className: `block text-sm font-medium mb-2 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}`
      },
      "Change:"
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-lg p-4 text-center ${theme === "dark" ? "bg-green-900/30" : "bg-green-100"}`
      },
      /* @__PURE__ */ React.createElement(
        "span",
        {
          className: `text-2xl font-bold ${theme === "dark" ? "text-green-400" : "text-green-700"}`
        },
        "PHP ",
        change.toFixed(2)
      )
    )), /* @__PURE__ */ React.createElement("div", { className: "flex gap-4" }, /* @__PURE__ */
React.createElement(
      "button",
      {
        onClick: onClose,
        className: `flex-1 py-3 px-6 rounded-lg font-semibold transition-all ${theme === "dark" ? "bg-
[#2A2724] text-gray-300 hover:bg-[#322f2c]" : "text-gray-700 bg-gray-200 hover:bg-gray-300"}`
      },
      "Cancel"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: handleProceed,
        className: "flex-1 py-3 px-6 rounded-lg font-semibold text-white hover:opacity-90 transition-all",
        style: {
          background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)"
        }
      },
      "Confirm Payment"
    )))
  )), /* @__PURE__ */ React.createElement(
    CheckoutConfirmationModal,
    {
      isOpen: showConfirmation,
      onClose: () => setShowConfirmation(false),
      onConfirm: handleConfirmCheckout
    }
  ), /* @__PURE__ */ React.createElement(
    SuccessModal,
    {
      isOpen: showSuccess,
      onClose: () => {
        setShowSuccess(false);
      }
    }
  ), /* @__PURE__ */ React.createElement(
    ReceiptModal,
    {
      isOpen: showReceipt,
      onClose: () => setShowReceipt(false),
      receiptData,
      onNewTransaction: handleNewTransaction,
      initialPrintError: printError,
      onPrintSuccess: handleNewTransaction,
      disableAutoPrint: true
    }
  ), /* @__PURE__ */ React.createElement(PrintingModal, { isOpen: isAutoPrinting }));
};
export default CashPaymentModal;
```

FILE: pos/frontend/src/components/terminal/CheckoutConfirmationModal.jsx
==========

```
import React from "react";
import { FaTimes } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
```

```
const CheckoutConfirmationModal = ({ isOpen, onClose, onConfirm }) => {
  const { theme } = useTheme();
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 bg-opacity-50 backdrop-blur-sm flex items-center justify-center z-50 font-
poppins p-4"
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-2xl w-full max-w-md relative shadow-2xl ${theme === "dark" ? "bg-[#1E1B18]" :
"bg-white"}`,
        style: { boxShadow: "0 25px 50px -12px rgba(0, 0, 0, 0.5)" }
      },
      /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onClose,
          className: `absolute top-4 right-4 transition-colors z-10 ${theme === "dark" ? "text-gray-400
hover:text-gray-200" : "text-gray-400 hover:text-gray-600"}`
        },
        /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
      ),
      /* @__PURE__ */ React.createElement("div", { className: "p-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-center mb-6" }, /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "w-20 h-20 rounded-full flex items-center justify-center relative",
          style: {
            backgroundColor: "#E8D5C4",
            boxShadow: "0 4px 12px rgba(118, 70, 43, 0.2)"
          }
        },
        /* @__PURE__ */ React.createElement(
          "span",
          {
            className: "text-4xl font-bold text-[#76462B]",
            style: {
              animation: "questionPulse 0.6s ease-out"
            }
          },
          "?"
        ),
        /* @__PURE__ */ React.createElement("style", null, `
                @keyframes questionPulse {
                  0% {
                    opacity: 0;
                    transform: scale(0) rotate(-10deg);
                  }
                  50% {
                    opacity: 1;
                    transform: scale(1.2) rotate(5deg);
                  }
                  100% {
                    opacity: 1;
                    transform: scale(1) rotate(0deg);
                  }
                }
              `)
      )), /* @__PURE__ */ React.createElement("h3", { className: `text-xl font-bold text-center mb-3 ${theme
=== "dark" ? "text-white" : "text-gray-800"}` }, "Are you sure you want to proceed?"), /* @__PURE__ */
React.createElement("p", { className: `text-sm text-center mb-8 ${theme === "dark" ? "text-gray-400" : "text-
gray-500"}` }, "This action cannot be undone."), /* @__PURE__ */ React.createElement("div", { className:
"flex gap-4" }, /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onConfirm,
          className: "flex-1 py-3 px-6 rounded-lg font-bold text-white transition-all shadow-md hover:shadow-
lg",
          style: {
            background: "linear-gradient(135deg, #D4A59A 0%, #AD7F65 50%, #76462B 100%)"
          }
        },
        "Confirm"
      ), /* @__PURE__ */ React.createElement(
```

```
        "button",
        {
          onClick: onClose,
          className: `flex-1 py-3 px-6 rounded-lg font-bold transition-all shadow-sm ${theme === "dark" ?
"bg-[#2A2724] text-gray-300 hover:bg-[#322f2c]" : "text-gray-700 bg-gray-200 hover:bg-gray-300"}`
        },
        "Cancel"
      )))
    )
  );
};
export default CheckoutConfirmationModal;
```

FILE: pos/frontend/src/components/terminal/DiscountModal.jsx
==========

```
import { useEffect, useState } from "react";
import {
  FaBox,
  FaCalendar,
  FaSearch,
  FaTag,
  FaTimes,
  FaUsers
} from "react-icons/fa";
import filterIcon from "../../assets/filter.svg";
import { useTheme } from "../../context/ThemeContext";
const icon20Percent = new URL("../../assets/owner/20.png", import.meta.url).href;
const icon50Percent = new URL("../../assets/owner/50.png", import.meta.url).href;
const iconSenior = new URL("../../assets/owner/Senior&ani.png", import.meta.url).href;
const DiscountModal = ({
  isOpen,
  onClose,
  onSelectDiscount,
  cart = [],
  products = [],
  selectedDiscounts = []
}) => {
  const { theme } = useTheme();
  const [searchQuery, setSearchQuery] = useState("");
  const [discounts, setDiscounts] = useState([]);
  const [loading, setLoading] = useState(false);
  const [showDateFilter, setShowDateFilter] = useState(false);
  const [dateFilter, setDateFilter] = useState("all");
  const getDiscountIcon = (discount) => {
    const title = discount.title?.toLowerCase() || "";
    const discountValue = discount.discountValue || "";
    if (title.includes("senior")) {
      return {
        icon: iconSenior,
        iconColor: "linear-gradient(135deg, #9B59B6 0%, #E91E63 100%)"
      };
    }
    const match = discountValue.match(/(\d+)/);
    if (match) {
      const value = parseInt(match[1]);
      if (value >= 50) {
        return {
          icon: icon50Percent,
          iconColor: "linear-gradient(135deg, #6B7280 0%, #4B5563 100%)"
        };
      }
    }
    return {
      icon: icon20Percent,
      iconColor: "linear-gradient(135deg, #FF6B6B 0%, #FF8E53 100%)"
    };
  };
  useEffect(() => {
    if (isOpen) {
      fetchDiscounts();
    }
  }, [isOpen]);
  const fetchDiscounts = async () => {
    try {
      setLoading(true);
```

```
      const response = await fetch("http://localhost:5000/api/discounts");
      const data = await response.json();
      if (data.success) {
        const formattedDiscounts = data.data.filter((discount) => discount.status === "active").map((discount)
=> {
          const iconData = getDiscountIcon(discount);
          return {
            ...discount,
            ...iconData
          };
        });
        setDiscounts(formattedDiscounts);
      }
    } catch (error) {
      console.error("Error fetching discounts:", error);
    } finally {
      setLoading(false);
    }
  };
  const discountAppliesToCart = (discount) => {
    const appliesToType = discount.appliesToType || discount.appliesTo;
    if (!cart || cart.length === 0) {
      return appliesToType === "all";
    }
    if (appliesToType === "all") {
      return true;
    }
    if (appliesToType === "category" && discount.category) {
      const hasMatchingItem = cart.some((item) => {
        let itemCategory = item.category;
        if (!itemCategory) {
          const productId = item._id || item.productId || item.id;
          const product = products.find((p) => {
            const pId = p._id || p.id;
            return pId && productId && pId.toString() === productId.toString();
          });
          itemCategory = product?.category;
        }
        return itemCategory === discount.category;
      });
      return hasMatchingItem;
    }
    if (appliesToType === "products" && discount.productIds && discount.productIds.length > 0) {
      const hasMatchingItem = cart.some((item) => {
        const itemId = item._id || item.productId || item.id;
        return discount.productIds.some((pid) => {
          const pidStr = pid.toString ? pid.toString() : pid;
          const itemIdStr = itemId.toString ? itemId.toString() : itemId;
          return pidStr === itemIdStr;
        });
      });
      return hasMatchingItem;
    }
    return false;
  };
  const filteredDiscounts = discounts.filter((discount) => {
    const matchesSearch = discount.title.toLowerCase().includes(searchQuery.toLowerCase()) ||
discount.discountValue.toLowerCase().includes(searchQuery.toLowerCase());
    if (!matchesSearch) return false;
    const alreadySelected = selectedDiscounts.some(
      (d) => d._id === discount._id
    );
    if (alreadySelected) return false;
    if (dateFilter !== "all") {
      const now = /* @__PURE__ */ new Date();
      const today = new Date(now.getFullYear(), now.getMonth(), now.getDate());
      if (dateFilter === "permanent") {
        if (discount.validFrom !== "Permanent") return false;
      } else {
        if (discount.validFrom === "Permanent") {
        } else {
          const from = new Date(discount.validFrom);
          const to = discount.validTo ? new Date(discount.validTo) : null;
          if (dateFilter === "today") {
            const endOfToday = new Date(today);
            endOfToday.setDate(endOfToday.getDate() + 1);
            if (from > endOfToday || to && to < today) return false;
```

```
          } else if (dateFilter === "this_week") {
            const startOfWeek = new Date(today);
            startOfWeek.setDate(startOfWeek.getDate() - startOfWeek.getDay());
            const endOfWeek = new Date(startOfWeek);
            endOfWeek.setDate(endOfWeek.getDate() + 7);
            if (from > endOfWeek || to && to < startOfWeek) return false;
          } else if (dateFilter === "this_month") {
            const startOfMonth = new Date(now.getFullYear(), now.getMonth(), 1);
            const endOfMonth = new Date(
              now.getFullYear(),
              now.getMonth() + 1,
              0
            );
            if (from > endOfMonth || to && to < startOfMonth) return false;
          }
        }
      }
    }
    return discountAppliesToCart(discount);
  });
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 bg-opacity-50 backdrop-blur-
sm flex items-center justify-center z-50 font-poppins p-4" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-2xl w-full max-w-md h-[95vh] flex flex-col shadow-2xl ${theme === "dark" ? "bg-
[#1E1B18]" : "bg-white"}`
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `px-6 py-4 border-b flex items-center justify-between ${theme === "dark" ? "border-gray-
700" : "border-gray-200"}`
      },
      /* @__PURE__ */ React.createElement(
        "h2",
        {
          className: `text-xl font-bold ${theme === "dark" ? "text-white" : "text-gray-900"}`
        },
        "Discounts"
      ),
      /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onClose,
          className: `transition-colors ${theme === "dark" ? "text-gray-400 hover:text-gray-200" : "text-
gray-400 hover:text-gray-600"}`
        },
        /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
      )
    ),
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `px-6 py-4 border-b ${theme === "dark" ? "border-gray-700" : "border-gray-200"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-4" }, /* @__PURE__ */
React.createElement("div", { className: "relative flex-1" }, /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "absolute left-1 top-1/2 transform -translate-y-1/2 w-10 h-9 flex items-center justify-
center text-white rounded-lg",
          style: {
            background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)"
          }
        },
        /* @__PURE__ */ React.createElement(FaSearch, { className: "text-sm" })
      ), /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "text",
          placeholder: "Search For...",
          value: searchQuery,
          onChange: (e) => setSearchQuery(e.target.value),
          className: `w-full h-10 pl-14 pr-4 border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white placeholder-
gray-500" : "border-gray-300 bg-white text-gray-900"}`
```

```
        }
      )), /* @__PURE__ */ React.createElement("div", { className: "relative" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: () => setShowDateFilter(!showDateFilter),
          className: `w-10 h-10 rounded-lg flex items-center justify-center border shadow-sm hover:shadow-md
transition-shadow ${dateFilter !== "all" ? "border-[#AD7F65] bg-[#AD7F65]/10" : theme === "dark" ? "bg-
[#2A2724] border-gray-600" : "bg-white border-gray-200"}`
        },
        /* @__PURE__ */ React.createElement(
          "img",
          {
            src: filterIcon,
            alt: "Filter",
            className: `w-5 h-5 ${theme === "dark" ? "opacity-90 invert" : "opacity-90"}`
          }
        )
      ), showDateFilter && /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `absolute top-full right-0 mt-2 w-44 rounded-xl shadow-2xl border overflow-hidden z-50
${theme === "dark" ? "bg-[#1E1B18] border-gray-600" : "bg-white border-gray-200"}`
        },
        /* @__PURE__ */ React.createElement(
          "div",
          {
            className: `px-3 py-2 border-b ${theme === "dark" ? "bg-[#2A2724] border-gray-600" : "bg-gray-50
border-gray-200"}`
          },
          /* @__PURE__ */ React.createElement(
            "span",
            {
              className: `text-xs font-semibold ${theme === "dark" ? "text-gray-300" : "text-gray-600"}`
            },
            "Filter by Date"
          )
        ),
        /* @__PURE__ */ React.createElement("div", { className: "py-1" }, [
          { value: "all", label: "All" },
          { value: "today", label: "Valid Today" },
          { value: "this_week", label: "This Week" },
          { value: "this_month", label: "This Month" },
          { value: "permanent", label: "Permanent" }
        ].map(
          (opt) => /* @__PURE__ */ React.createElement(
            "button",
            {
              key: opt.value,
              onClick: () => {
                setDateFilter(opt.value);
                setShowDateFilter(false);
              },
              className: `w-full px-4 py-2 text-left text-sm flex items-center justify-between ${dateFilter
=== opt.value ? "text-[#AD7F65] font-medium bg-[#AD7F65]/5" : theme === "dark" ? "text-gray-300 hover:bg-
[#2A2724]" : "text-gray-700 hover:bg-gray-50"}`
            },
            opt.label,
            dateFilter === opt.value && /* @__PURE__ */ React.createElement("span", { className: "text-
[#AD7F65]" }, "\u2713")
          )
        ))
      )))
    ),
    /* @__PURE__ */ React.createElement("div", { className: "flex-1 overflow-auto px-6 py-4" }, loading ? /*
@__PURE__ */ React.createElement("div", { className: "flex justify-center items-center py-12" }, /* @__PURE__
*/ React.createElement(
      "div",
      {
        className: theme === "dark" ? "text-gray-400" : "text-gray-500"
      },
      "Loading discounts..."
    )) : filteredDiscounts.length === 0 ? /* @__PURE__ */ React.createElement("div", { className: "flex flex-
col justify-center items-center py-12" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `text-center ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
```

```
      },
      cart.length === 0 ? "Add items to your cart to see applicable discounts" : searchQuery ? "No discounts
match your search" : "No discounts available for items in your cart"
    )) : /* @__PURE__ */ React.createElement("div", { className: "space-y-4" }, filteredDiscounts.map(
      (discount) => /* @__PURE__ */ React.createElement(
        "div",
        {
          key: discount._id,
          onClick: () => {
            onSelectDiscount(discount);
            onClose();
          },
          className: `rounded-xl overflow-hidden border shadow-md flex items-stretch cursor-pointer
hover:shadow-lg transition-shadow min-h-[85px] ${theme === "dark" ? "border-gray-700 bg-[#2A2724]" : "border-
gray-200 bg-white"}`
        },
        /* @__PURE__ */ React.createElement(
          "div",
          {
            className: "w-16 flex items-center justify-center shrink-0 rounded-l-xl",
            style: { background: discount.iconColor }
          },
          /* @__PURE__ */ React.createElement(
            "img",
            {
              src: discount.icon,
              alt: discount.title,
              className: "w-12 h-12 object-contain"
            }
          )
        ),
        /* @__PURE__ */ React.createElement(
          "div",
          {
            className: `flex-1 p-2 flex flex-col justify-between ${theme === "dark" ? "bg-[#2A2724]" : "bg-
white"}`
          },
          /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("div",
{ className: "flex items-center justify-between mb-1" }, /* @__PURE__ */ React.createElement(
            "h3",
            {
              className: "text-base font-bold",
              style: { color: "#76462B" }
            },
            discount.title
          ), /* @__PURE__ */ React.createElement("span", { className: "px-1.5 py-0.5 rounded-full text-[10px]
font-medium bg-green-100 text-green-700" }, discount.status === "active" ? "Active" : "Inactive")), /*
@__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-2 mb-1" }, /* @__PURE__ */
React.createElement("div", { className: "space-y-0.5" }, /* @__PURE__ */ React.createElement("div",
{ className: "flex items-center gap-1 text-[10px]" }, /* @__PURE__ */ React.createElement(FaTag, { className:
"text-yellow-500 text-xs" }), /* @__PURE__ */ React.createElement(
            "span",
            {
              className: theme === "dark" ? "text-gray-300" : "text-gray-600"
            },
            "Discount Value:",
            " ",
            /* @__PURE__ */ React.createElement(
              "span",
              {
                className: "font-bold",
                style: { color: "#AD7F65" }
              },
              discount.discountValue
            )
          )), /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-1 text-[10px]" },
/* @__PURE__ */ React.createElement(FaCalendar, { className: "text-purple-500 text-xs" }), /* @__PURE__ */
React.createElement(
            "span",
            {
              className: theme === "dark" ? "text-gray-300" : "text-gray-600"
            },
            "Valid only from:",
            " ",
            /* @__PURE__ */ React.createElement(
              "span",
              {
```

```
                className: "font-bold",
                style: { color: "#AD7F65" }
              },
              discount.validFrom === "Permanent" ? "Permanent" : `${discount.validFrom} to
${discount.validTo}`
            )
          ))), /* @__PURE__ */ React.createElement("div", { className: "space-y-0.5" }), /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-1 text-[10px]" }), /* @__PURE__ */
React.createElement(FaBox, { className: "text-blue-400 text-xs" }), /* @__PURE__ */ React.createElement(
            "span",
            {
              className: theme === "dark" ? "text-gray-300" : "text-gray-600"
            },
            "Applies to:",
            " ",
            /* @__PURE__ */ React.createElement(
              "span",
              {
                className: "font-bold",
                style: { color: "#AD7F65" }
              },
              discount.appliesTo
            )
          )), discount.usage && /* @__PURE__ */ React.createElement("div", { className: "flex items-center
gap-1 text-[10px]" }), /* @__PURE__ */ React.createElement(FaUsers, { className: "text-green-500 text-xs" }),
/* @__PURE__ */ React.createElement(
            "span",
            {
              className: theme === "dark" ? "text-gray-300" : "text-gray-600"
            },
            "Used:",
            " ",
            /* @__PURE__ */ React.createElement(
              "span",
              {
                className: "font-bold",
                style: { color: "#AD7F65" }
              },
              discount.usage.used,
              "/",
              discount.usage.total
            ),
            " ",
            "times"
          )))),
          discount.description && /* @__PURE__ */ React.createElement(
            "div",
            {
              className: "text-[10px] italic",
              style: { color: "#AD7F65" }
            },
            discount.description
          )
        )
      )
    )))
  ));
};
export default DiscountModal;
```

FILE: pos/frontend/src/components/terminal/DuplicateItemModal.jsx
==========

```
import React from "react";
import { FaExclamationTriangle, FaTimes } from "react-icons/fa";
import { MdCategory } from "react-icons/md";
import { useTheme } from "../../context/ThemeContext";
const VARIANT_ONLY_SIZE_KEY = "__VARIANT_ONLY__";
const DuplicateItemModal = ({ isOpen, onClose, onConfirm, item, existingQuantity }) => {
  const { theme } = useTheme();
  if (!isOpen || !item) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 bg-black/50 flex items-center
justify-center z-[10000]" }), /* @__PURE__ */ React.createElement("div", { className: `rounded-2xl w-full max-
w-md mx-4 overflow-hidden shadow-2xl ${theme === "dark" ? "bg-[#1E1B18]" : "bg-white"}` }), /* @__PURE__ */
React.createElement("div", { className: `px-6 py-4 border-b flex items-center justify-between ${theme ===
"dark" ? "border-gray-700" : "border-gray-100"}` }), /* @__PURE__ */ React.createElement("div", { className:
```

```
"flex items-center gap-3" }, /* @__PURE__ */ React.createElement("div", { className: "w-10 h-10 rounded-full
bg-amber-100 flex items-center justify-center" }, /* @__PURE__ */ React.createElement(FaExclamationTriangle,
{ className: "text-amber-500 text-lg" })), /* @__PURE__ */ React.createElement("h2", { className: `text-lg
font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}` }, "Item Already in Cart")), /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: onClose,
      className: `p-2 rounded-full transition-colors ${theme === "dark" ? "hover:bg-gray-700" : "hover:bg-
gray-100"}`
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "text-gray-400" })
  )), /* @__PURE__ */ React.createElement("div", { className: "px-6 py-5" }, /* @__PURE__ */
React.createElement("div", { className: `flex items-center gap-4 p-4 rounded-xl mb-4 ${theme === "dark" ?
"bg-[#2A2724]" : "bg-gray-50"}` }, /* @__PURE__ */ React.createElement("div", { className: "w-16 h-16 bg-
gray-200 rounded-lg overflow-hidden shrink-0" }, item.itemImage && item.itemImage.trim() !== "" ? /*
@__PURE__ */ React.createElement(
    "img",
    {
      src: item.itemImage,
      alt: item.itemName,
      className: "w-full h-full object-cover"
    }
  ) : /* @__PURE__ */ React.createElement("div", { className: "w-full h-full flex items-center justify-
center" }, /* @__PURE__ */ React.createElement(MdCategory, { className: "text-2xl text-gray-400" }))), /*
@__PURE__ */ React.createElement("div", { className: "flex-1" }, /* @__PURE__ */ React.createElement("h3",
{ className: `font-semibold ${theme === "dark" ? "text-white" : "text-gray-800"}` }, item.itemName), /*
@__PURE__ */ React.createElement("p", { className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-
gray-500"}` }, item.selectedSize && item.selectedSize !== VARIANT_ONLY_SIZE_KEY && `Size:
${item.selectedSize}`, item.variant && ` \u2022 ${item.variant}`), /* @__PURE__ */ React.createElement("p",
{ className: "text-sm font-medium text-[#AD7F65] mt-1" }, "PHP ", (item.itemPrice || 0).toFixed(2)))), /*
@__PURE__ */ React.createElement("div", { className: "text-center mb-2" }, /* @__PURE__ */
React.createElement("p", { className: theme === "dark" ? "text-gray-300" : "text-gray-600" }, "This item is
already in your cart with a quantity of", " ", /* @__PURE__ */ React.createElement("span", { className:
"font-bold text-[#AD7F65]" }, existingQuantity), "."), /* @__PURE__ */ React.createElement("p", { className:
`mt-2 ${theme === "dark" ? "text-gray-300" : "text-gray-600"}` }, "Would you like to add more?"))), /*
@__PURE__ */ React.createElement("div", { className: `px-6 py-4 flex gap-3 ${theme === "dark" ? "bg-
[#2A2724]" : "bg-gray-50"}` }, /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onClose,
      className: `flex-1 py-3 px-4 border rounded-xl font-medium transition-colors ${theme === "dark" ?
"border-gray-600 text-gray-300 hover:bg-gray-700" : "border-gray-300 text-gray-700 hover:bg-gray-100"}`
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onConfirm,
      className: "flex-1 py-3 px-4 text-white rounded-xl font-medium hover:opacity-90 transition-all shadow-
md",
      style: { background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)" }
    },
    "Yes, Add More"
  ))));
};
export default DuplicateItemModal;
```

FILE: pos/frontend/src/components/terminal/OrderSummary.jsx
==========

```
import { memo, useCallback, useEffect, useMemo, useRef, useState } from "react";
import { FaMinus, FaPlus, FaTag, FaTimes } from "react-icons/fa";
import { HiDocumentRemove } from "react-icons/hi";
import { MdCategory } from "react-icons/md";
import cashIcon from "../../assets/cash.svg";
import qrIcon from "../../assets/qr.png";
import { useAuth } from "../../context/AuthContext";
import { useTheme } from "../../context/ThemeContext";
import RemoveItemPinModal from "./RemoveItemPinModal";
import VoidTransactionModal from "./VoidTransactionModal";
import { generateDynamicSku } from "../../utils/skuUtils";
const VARIANT_ONLY_SIZE_KEY = "__VARIANT_ONLY__";
const OrderSummary = memo(({
  cart,
  removeFromCart,
```

```
  removeFromCartDirect,
  updateQuantity,
  discountAmount,
  setDiscountAmount,
  selectedDiscounts = [],
  onRemoveDiscount,
  subtotal = 0,
  discount = 0,
  total = 0,
  handleCheckout,
  onCashPayment,
  onQRPayment,
  onOpenDiscountModal,
  onSelectDiscount,
  products = []
}) => {
  const { theme } = useTheme();
  const { currentUser } = useAuth();
  const userId = currentUser?._id || currentUser?.id || currentUser?.email || "guest";
  const [selectedPaymentMethod, setSelectedPaymentMethod] = useState(null);
  const [discountCode, setDiscountCode] = useState("");
  const [isRemoveModalOpen, setIsRemoveModalOpen] = useState(false);
  const [itemToRemove, setItemToRemove] = useState(null);
  const [applyingDiscount, setApplyingDiscount] = useState(false);
  const [isVoidModalOpen, setIsVoidModalOpen] = useState(false);
  const [itemsToVoid, setItemsToVoid] = useState([]);
  const [isBulkVoid, setIsBulkVoid] = useState(false);
  const isProcessingVoidRef = useRef(false);
  const [pendingQuantities, setPendingQuantities] = useState({});
  const [availableDiscounts, setAvailableDiscounts] = useState([]);
  const discountFetchedRef = useRef(false);
  useEffect(() => {
    if (discountFetchedRef.current) return;
    discountFetchedRef.current = true;
    const fetchDiscounts = async () => {
      try {
        const response = await fetch("http://localhost:5000/api/discounts");
        const data = await response.json();
        if (data.success && Array.isArray(data.data)) {
          setAvailableDiscounts(data.data.filter((d) => d.status === "active"));
        }
      } catch (error) {
        console.error("Error fetching discounts for auto-apply:", error);
      }
    };
    fetchDiscounts();
  }, []);
  useEffect(() => {
    if (!discountCode || !discountCode.trim() || applyingDiscount) return;
    const codeToFind = discountCode.trim().toLowerCase();
    const matchingDiscount = availableDiscounts.find(
      (discount2) => (discount2.discountCode || "").trim().toLowerCase() === codeToFind
    );
    if (matchingDiscount) {
      const isAlreadyApplied = selectedDiscounts.some((d) => d._id === matchingDiscount._id);
      if (!isAlreadyApplied) {
        if (onSelectDiscount) {
          onSelectDiscount(matchingDiscount);
          setDiscountCode("");
        }
      }
    }
  }, [discountCode, availableDiscounts, selectedDiscounts, applyingDiscount, onSelectDiscount]);
  const handleProceed = () => {
    if (selectedPaymentMethod === "cash" && onCashPayment) {
      onCashPayment();
    } else if (selectedPaymentMethod === "qr" && onQRPayment) {
      onQRPayment();
    } else {
      handleCheckout();
    }
  };
  const getItemKey = (item) => {
    return `${item._id || item.productId}-${item.selectedSize || item.size || ""}-${item.selectedVariation ||
""}`;
  };
  const getDisplayQuantity = (item) => {
```

```
    const key = getItemKey(item);
    return pendingQuantities[key] !== void 0 ? pendingQuantities[key] : item.quantity;
  };
  const hasPendingChange = (item) => {
    const key = getItemKey(item);
    return pendingQuantities[key] !== void 0 && pendingQuantities[key] !== item.quantity;
  };
  const handleMinusClick = (item) => {
    const key = getItemKey(item);
    const currentPending = pendingQuantities[key] !== void 0 ? pendingQuantities[key] : item.quantity;
    if (currentPending > 1) {
      setPendingQuantities((prev) => ({
        ...prev,
        [key]: currentPending - 1
      }));
    }
  };
  const handlePlusClickPending = (item) => {
    const key = getItemKey(item);
    const currentPending = pendingQuantities[key] !== void 0 ? pendingQuantities[key] : item.quantity;
    let maxQty = item.currentStock;
    if (item.selectedSize && item.sizes && item.sizes[item.selectedSize] !== void 0) {
      const sizeData = item.sizes[item.selectedSize];
      if (item.selectedVariation && typeof sizeData === "object" && sizeData !== null && sizeData.variants &&
sizeData.variants[item.selectedVariation] !== void 0) {
        const variantData = sizeData.variants[item.selectedVariation];
        maxQty = typeof variantData === "object" && variantData !== null ? variantData.quantity || 0 : typeof
variantData === "number" ? variantData : 0;
      } else {
        maxQty = typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0 ?
sizeData.quantity : typeof sizeData === "number" ? sizeData : 0;
      }
    }
    if (maxQty === void 0 || maxQty === null || currentPending < maxQty) {
      setPendingQuantities((prev) => ({
        ...prev,
        [key]: currentPending + 1
      }));
    }
  };
  const handleCancelPending = (item) => {
    const key = getItemKey(item);
    setPendingQuantities((prev) => {
      const newState = { ...prev };
      delete newState[key];
      return newState;
    });
  };
  const handleConfirmPending = (item) => {
    const key = getItemKey(item);
    const pendingQty = pendingQuantities[key];
    if (pendingQty < item.quantity) {
      const voidQuantity = item.quantity - pendingQty;
      const voidTotalAmount = item.itemPrice * voidQuantity;
      setItemToRemove({
        ...item,
        voidQuantity,
        newQuantity: pendingQty,
        itemPrice: voidTotalAmount,
        originalUnitPrice: item.itemPrice
      });
      setIsRemoveModalOpen(true);
    } else if (pendingQty > item.quantity) {
      updateQuantity(item, pendingQty);
      handleCancelPending(item);
    }
  };
  const productMap = useMemo(() => {
    const map = /* @__PURE__ */ new Map();
    products.forEach((p) => {
      const pId = String(p._id || p.id);
      if (pId) {
        map.set(pId, p);
      }
    });
    return map;
  }, [products]);
```

```
  const itemDiscountPercentMap = useMemo(() => {
    const map = /* @__PURE__ */ new Map();
    if (!selectedDiscounts || selectedDiscounts.length === 0) return map;
    cart.forEach((item) => {
      const key = getItemKey(item);
      let totalPercent = 0;
      let itemCategory = item.category;
      let itemSubCategory = item.subCategory;
      if ((!itemCategory || !itemSubCategory) && products.length > 0) {
        const productId = String(item._id || item.productId || item.id);
        const product = productMap.get(productId);
        itemCategory = itemCategory || product?.category;
        itemSubCategory = itemSubCategory || product?.subCategory;
      }
      selectedDiscounts.forEach((discount2) => {
        const appliesToType = discount2.appliesToType || discount2.appliesTo;
        let applies = false;
        if (appliesToType === "all") {
          applies = true;
        } else if (appliesToType === "category" && discount2.category) {
          const catMatch = String(itemCategory || "").trim().toLowerCase() === String(discount2.category ||
"").trim().toLowerCase();
          if (catMatch) {
            if (!discount2.subCategory) {
              applies = true;
            } else {
              const subMatch = String(itemSubCategory || "").trim().toLowerCase() ===
String(discount2.subCategory || "").trim().toLowerCase();
              if (subMatch) {
                applies = true;
              }
            }
          }
        } else if (appliesToType === "products" && discount2.productIds && discount2.productIds.length > 0) {
          const itemId = item._id || item.productId || item.id;
          const isMatch = discount2.productIds.some((pid) => {
            const pidStr = pid.toString ? pid.toString() : pid;
            const itemIdStr = itemId.toString ? itemId.toString() : itemId;
            return pidStr === itemIdStr;
          });
          if (isMatch) applies = true;
        }
        if (applies) {
          const discountValueStr = discount2.discountValue || "";
          if (typeof discountValueStr === "string" && discountValueStr.includes("%")) {
            const percentage = parseFloat(discountValueStr.replace("% OFF", "").replace(/\s/g, ""));
            if (!isNaN(percentage)) {
              totalPercent += percentage;
            }
          }
        }
      });
      map.set(key, totalPercent);
    });
    return map;
  }, [cart, selectedDiscounts, products]);
  const getItemTotalDiscountPercent = useCallback((item) => {
    return itemDiscountPercentMap.get(getItemKey(item)) || 0;
  }, [itemDiscountPercentMap]);
  const handleRemoveConfirm = async (reason, approverInfo = {}) => {
    if (!itemToRemove) {
      setIsRemoveModalOpen(false);
      setItemToRemove(null);
      return;
    }
    try {
      const voidQuantity = itemToRemove.voidQuantity || 1;
      const newQuantity = itemToRemove.newQuantity !== void 0 ? itemToRemove.newQuantity :
itemToRemove.quantity - 1;
      const voidAmount = itemToRemove.itemPrice * voidQuantity;
      const voidedItem = {
        productId: itemToRemove.productId || itemToRemove._id || itemToRemove.id,
        itemName: itemToRemove.itemName,
        sku: itemToRemove.sku,
        variant: itemToRemove.variant || itemToRemove.selectedVariation,
        selectedSize: itemToRemove.selectedSize || itemToRemove.size,
        quantity: voidQuantity,
```

```
        price: itemToRemove.itemPrice,
        itemImage: itemToRemove.itemImage || "",
        voidReason: reason
      };
      setIsRemoveModalOpen(false);
      setItemToRemove(null);
      if (newQuantity <= 0) {
        removeFromCart(itemToRemove);
      } else {
        updateQuantity(itemToRemove, newQuantity);
      }
      const key = getItemKey(itemToRemove);
      setPendingQuantities((prev) => {
        const newState = { ...prev };
        delete newState[key];
        return newState;
      });
      (async () => {
        try {
          const response = await fetch("http://localhost:5000/api/transactions", {
            method: "POST",
            headers: {
              "Content-Type": "application/json"
            },
            body: JSON.stringify({
              userId,
              items: [voidedItem],
              paymentMethod: "void",
              totalAmount: voidAmount,
              performedById: currentUser?._id || currentUser?.id || "",
              performedByName: currentUser?.name || `${currentUser?.firstName || ""} ${currentUser?.lastName
|| ""}`.trim() || "System",
              status: "Voided",
              voidReason: reason
            })
          });
          const data = await response.json();
          if (!data.success) {
            console.error("Failed to record void transaction:", data.message);
          }
          const voidLogResponse = await fetch("http://localhost:5000/api/void-logs", {
            method: "POST",
            headers: {
              "Content-Type": "application/json"
            },
            body: JSON.stringify({
              items: [voidedItem],
              totalAmount: voidAmount,
              voidReason: reason,
              voidedById: currentUser?._id || currentUser?.id || "",
              voidedByName: currentUser?.name || `${currentUser?.firstName || ""} ${currentUser?.lastName ||
""}`.trim() || "System",
              approvedBy: approverInfo.approvedBy || null,
              approvedByRole: approverInfo.approvedByRole || null,
              source: "cart",
              userId
            })
          });
          const voidLogData = await voidLogResponse.json();
          if (!voidLogData.success) {
            console.error("Failed to create void log:", voidLogData.message);
          }
        } catch (bgError) {
          console.error("Error in background void network requests:", bgError);
        }
      })();
    } catch (error) {
      console.error("Error preparing void transaction data:", error);
      const newQuantity = itemToRemove.newQuantity !== void 0 ? itemToRemove.newQuantity :
itemToRemove.quantity - 1;
      if (newQuantity <= 0) {
        removeFromCart(itemToRemove);
      } else {
        updateQuantity(itemToRemove, newQuantity);
      }
      setIsRemoveModalOpen(false);
      setItemToRemove(null);
```

```
  }
};
const handleRemoveModalClose = () => {
  setIsRemoveModalOpen(false);
  setItemToRemove(null);
  setItemsToVoid([]);
  setIsBulkVoid(false);
};
const handleVoidButtonClick = () => {
  if (cart.length > 0) {
    setIsVoidModalOpen(true);
  }
};
const handleVoidModalClose = () => {
  setIsVoidModalOpen(false);
};
const handleVoidItemsConfirm = (selectedItems) => {
  if (selectedItems.length > 0) {
    setItemsToVoid(selectedItems);
    setIsBulkVoid(true);
    setIsVoidModalOpen(false);
    const totalAmount = selectedItems.reduce((sum, item) => sum + item.itemPrice * item.quantity, 0);
    setItemToRemove({
      itemPrice: totalAmount,
      quantity: 1,
      itemName: `${selectedItems.length} item${selectedItems.length > 1 ? "s" : ""}`
    });
    setIsRemoveModalOpen(true);
  }
};
const handleBulkVoidConfirm = async (reason, approverInfo = {}) => {
  if (isProcessingVoidRef.current) {
    console.log("[OrderSummary] Already processing void, skipping");
    return;
  }
  if (!itemsToVoid || itemsToVoid.length === 0) {
    setIsRemoveModalOpen(false);
    setItemsToVoid([]);
    setIsBulkVoid(false);
    return;
  }
  isProcessingVoidRef.current = true;
  const itemsToRemove = [...itemsToVoid];
  setIsRemoveModalOpen(false);
  setItemToRemove(null);
  setItemsToVoid([]);
  setIsBulkVoid(false);
  await new Promise((resolve) => setTimeout(resolve, 100));
  const totalAmount = itemsToRemove.reduce((sum, item) => sum + item.itemPrice * item.quantity, 0);
  const voidedItems = itemsToRemove.map((item) => ({
    productId: item.productId || item._id || item.id,
    itemName: item.itemName,
    sku: item.sku,
    variant: item.variant || item.selectedVariation,
    selectedSize: item.selectedSize || item.size,
    quantity: item.quantity,
    price: item.itemPrice,
    itemImage: item.itemImage || "",
    voidReason: reason
  }));
  for (const item of itemsToRemove) {
    if (removeFromCartDirect) {
      removeFromCartDirect(item);
    } else {
      removeFromCart(item);
    }
  }
  isProcessingVoidRef.current = false;
  (async () => {
    try {
      const response = await fetch("http://localhost:5000/api/transactions", {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify({
          userId,
```

```
            items: voidedItems,
            paymentMethod: "void",
            totalAmount,
            performedById: currentUser?._id || currentUser?.id || "",
            performedByName: currentUser?.name || `${currentUser?.firstName || ""} ${currentUser?.lastName ||
""}`.trim() || "System",
            status: "Voided",
            voidReason: reason
          })
        });
        const data = await response.json();
        if (!data.success) {
          console.error("Failed to record void transaction:", data.message);
        }
        const voidLogResponse = await fetch("http://localhost:5000/api/void-logs", {
          method: "POST",
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify({
            items: voidedItems,
            totalAmount,
            voidReason: reason,
            voidedById: currentUser?._id || currentUser?.id || "",
            voidedByName: currentUser?.name || `${currentUser?.firstName || ""} ${currentUser?.lastName ||
""}`.trim() || "System",
            approvedBy: approverInfo.approvedBy || null,
            approvedByRole: approverInfo.approvedByRole || null,
            source: "cart",
            userId
          })
        });
        const voidLogData = await voidLogResponse.json();
        if (!voidLogData.success) {
          console.error("Failed to create void log:", voidLogData.message);
        }
      } catch (error) {
        console.error("Error recording bulk void network requests:", error);
      }
    })();
  };
  const applyDiscountCode = async () => {
    if (!discountCode || !discountCode.trim()) {
      alert("Please enter a discount code");
      return;
    }
    try {
      setApplyingDiscount(true);
      const response = await fetch("http://localhost:5000/api/discounts");
      const data = await response.json();
      if (!data.success || !Array.isArray(data.data)) {
        alert("Failed to fetch discounts. Please try again.");
        return;
      }
      const codeToFind = discountCode.trim().toLowerCase();
      const matchingDiscount = data.data.find((discount2) => {
        const discountCodeStr = (discount2.discountCode || "").trim().toLowerCase();
        return discountCodeStr === codeToFind && discount2.status === "active";
      });
      if (!matchingDiscount) {
        alert("Discount code not found or inactive. Please check the code and try again.");
        setDiscountCode("");
        return;
      }
      if (onSelectDiscount) {
        onSelectDiscount(matchingDiscount);
        setDiscountCode("");
      } else {
        alert("Unable to apply discount. Please try selecting from the discount list.");
      }
    } catch (error) {
      console.error("Error applying discount code:", error);
      alert("An error occurred while applying the discount code. Please try again.");
    } finally {
      setApplyingDiscount(false);
    }
  };
```

```
  return /* @__PURE__ */ React.createElement("div", { className: `h-full rounded-[22px] border flex flex-col
${theme === "dark" ? "bg-[#1E1B18] border-gray-700" : "bg-white border-gray-200"}`, style: { boxShadow: "5px
0 15px rgba(0,0,0,0.08), 0 7px 17px rgba(0,0,0,0.05)" } }, /* @__PURE__ */ React.createElement("div",
{ className: "px-6 py-5 flex items-center justify-between" }, /* @__PURE__ */ React.createElement("div",
{ className: "w-8" }), " ", /* @__PURE__ */ React.createElement("h2", { className: `text-xl font-bold text-
center ${theme === "dark" ? "text-white" : "text-gray-900"}` }, "Order Summary"), /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: handleVoidButtonClick,
      disabled: cart.length === 0,
      className: `w-8 h-8 flex items-center justify-center transition-all ${cart.length === 0 ? "text-gray-
300 cursor-not-allowed" : "text-red-500 hover:text-red-700"}`,
      title: "Void Transaction"
    },
    /* @__PURE__ */ React.createElement(HiDocumentRemove, { className: "w-6 h-6" })
  )), /* @__PURE__ */ React.createElement("div", { className: "flex-1 overflow-auto px-6 pb-6" }, cart.length
=== 0 ? /* @__PURE__ */ React.createElement("div", { className: "text-center text-gray-400 py-10" }, /*
@__PURE__ */ React.createElement("p", null, "No items in cart")) : /* @__PURE__ */ React.createElement("div",
{ className: "space-y-4" }, cart.map((item, index) => {
    const displayQty = getDisplayQuantity(item);
    const hasPending = hasPendingChange(item);
    const isReducing = hasPending && displayQty < item.quantity;
    const rawSize = item.selectedSize || item.size || "";
    const normalizedSize = rawSize === VARIANT_ONLY_SIZE_KEY ? "" : rawSize;
    return /* @__PURE__ */ React.createElement("div", { key: item._id || item.productId || `cart-item-
${index}`, className: `relative rounded-xl border shadow-[0_3px_8px_rgba(0,0,0,0.04)] p-3 ${hasPending ?
theme === "dark" ? "border-orange-700 bg-orange-900/20" : "border-orange-300 bg-orange-50" : theme ===
"dark" ? "bg-[#2A2724] border-gray-700" : "bg-white border-gray-200"}` }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement("div", { className: `w-16 h-16 rounded-lg shrink-0 overflow-hidden relative ${theme ===
"dark" ? "bg-[#1E1B18]" : "bg-gray-100"}` }, (() => {
      const discountPercent = getItemTotalDiscountPercent(item);
      if (discountPercent > 0) {
        return /* @__PURE__ */ React.createElement("div", { className: "absolute top-0 left-0 bg-red-500
text-white text-[10px] font-bold px-1.5 py-0.5 rounded-br-lg z-10" }, "-", discountPercent, "%");
      }
      return null;
    })(), item.itemImage && item.itemImage.trim() !== "" ? /* @__PURE__ */ React.createElement(
      "img",
      {
        src: item.itemImage,
        alt: item.itemName,
        className: "w-full h-full object-cover"
      }
    ) : /* @__PURE__ */ React.createElement("div", { className: "w-full h-full flex items-center justify-
center" }, /* @__PURE__ */ React.createElement(MdCategory, { className: "text-2xl text-gray-400" }))), /*
@__PURE__ */ React.createElement("div", { className: "flex-1" }, /* @__PURE__ */ React.createElement("h3",
{ className: `font-medium text-sm mb-1 pr-4 ${theme === "dark" ? "text-white" : "text-gray-900"}` },
item.itemName ? item.itemName.replace(/\s*\([^)]*\)\s*$/, "").trim() : "Item"), /* @__PURE__ */
React.createElement("p", { className: `text-xs mb-2 ${theme === "dark" ? "text-gray-400" : "text-gray-
500"}` }, "SKU: ", generateDynamicSku(item.sku, item.selectedVariation, normalizedSize), /* @__PURE__ */
React.createElement("br", null), "Size: ", normalizedSize || "\u2014", /* @__PURE__ */
React.createElement("br", null), "Color: ", item.selectedVariation || "N/A"), /* @__PURE__ */
React.createElement("p", { className: "text-sm font-bold text-[#AD7F65]" }, "PHP ", ((item.itemPrice || 0) *
displayQty).toFixed(2)), displayQty > 1 && /* @__PURE__ */ React.createElement("p", { className: `text-xs
${theme === "dark" ? "text-gray-500" : "text-gray-400"}` }, "\u20B1", (item.itemPrice || 0).toFixed(2), "
\xD7 ", displayQty), hasPending && isReducing && /* @__PURE__ */ React.createElement("p", { className: "text-
xs text-orange-600 mt-1" }, "Voiding ", item.quantity - displayQty, " item(s)")), /* @__PURE__ */
React.createElement("div", { className: "flex flex-col items-end justify-end gap-2" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-2" }, /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => handleMinusClick(item),
        disabled: displayQty <= 1,
        className: `w-6 h-6 flex items-center justify-center rounded-full shadow-sm transition-all
${displayQty <= 1 ? "bg-gray-300 text-white cursor-not-allowed" : "bg-[#AD7F65] text-white hover:bg-
[#8B5F45]"}`
      },
      /* @__PURE__ */ React.createElement(FaMinus, { className: "text-[10px]" })
    ), /* @__PURE__ */ React.createElement("span", { className: `font-semibold min-w-[18px] text-center text-
sm ${hasPending ? "text-orange-600" : theme === "dark" ? "text-white" : "text-gray-800"}` }, displayQty), /*
@__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => handlePlusClickPending(item),
        className: `w-6 h-6 flex items-center justify-center rounded-full shadow-sm transition-all ${(() => {
```

```
          let maxQty = item.currentStock;
          if (item.selectedSize && item.sizes && item.sizes[item.selectedSize] !== void 0) {
            const sizeData = item.sizes[item.selectedSize];
            if (item.selectedVariation && typeof sizeData === "object" && sizeData !== null &&
sizeData.variants && sizeData.variants[item.selectedVariation] !== void 0) {
              const variantData = sizeData.variants[item.selectedVariation];
              maxQty = typeof variantData === "object" && variantData !== null ? variantData.quantity || 0 :
typeof variantData === "number" ? variantData : 0;
            } else {
              maxQty = typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0 ?
sizeData.quantity : typeof sizeData === "number" ? sizeData : 0;
            }
          }
          return maxQty !== void 0 && maxQty !== null && displayQty >= maxQty;
        })() ? "bg-gray-300 text-white cursor-not-allowed" : "bg-[#AD7F65] text-white hover:bg-[#8B5F45]"}`,
        disabled: (() => {
          let maxQty = item.currentStock;
          if (item.selectedSize && item.sizes && item.sizes[item.selectedSize] !== void 0) {
            const sizeData = item.sizes[item.selectedSize];
            if (item.selectedVariation && typeof sizeData === "object" && sizeData !== null &&
sizeData.variants && sizeData.variants[item.selectedVariation] !== void 0) {
              const variantData = sizeData.variants[item.selectedVariation];
              maxQty = typeof variantData === "object" && variantData !== null ? variantData.quantity || 0 :
typeof variantData === "number" ? variantData : 0;
            } else {
              maxQty = typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0 ?
sizeData.quantity : typeof sizeData === "number" ? sizeData : 0;
            }
          }
          return maxQty !== void 0 && maxQty !== null && displayQty >= maxQty;
        })()
      },
      /* @__PURE__ */ React.createElement(FaPlus, { className: "text-[10px]" })
    )), hasPending && /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-1 mt-
1" }, /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => handleCancelPending(item),
        className: "px-2 py-1 text-xs bg-gray-200 text-gray-700 rounded hover:bg-gray-300 transition-all"
      },
      "Cancel"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => handleConfirmPending(item),
        className: "px-2 py-1 text-xs bg-[#AD7F65] text-white rounded hover:bg-[#8B5F45] transition-all"
      },
      "Confirm"
    )))));
  })})), /* @__PURE__ */ React.createElement("div", { className: `px-6 py-6 ${theme === "dark" ? "bg-
[#1E1B18]" : "bg-white"}` }, /* @__PURE__ */ React.createElement("div", { className: "mb-6" }, /* @__PURE__
*/ React.createElement("label", { className: "block text-xs font-semibold text-[#8B7355] mb-2" }, "Discount"),
selectedDiscounts.length > 0 && /* @__PURE__ */ React.createElement("div", { className: "space-y-2 mb-3" },
selectedDiscounts.map(
    (discount2) => /* @__PURE__ */ React.createElement("div", { key: discount2._id, className: `flex items-
center gap-2 p-2 rounded-lg border ${theme === "dark" ? "bg-[#2A2724] border-gray-600" : "bg-gray-50 border-
[#d6c1b5]"}` }, /* @__PURE__ */ React.createElement("div", { className: "flex-1" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-2" }, /* @__PURE__ */
React.createElement(FaTag, { className: "text-[#AD7F65] text-sm" }), /* @__PURE__ */
React.createElement("span", { className: `text-sm font-medium ${theme === "dark" ? "text-white" : "text-gray-
800"}` }, discount2.title)), /* @__PURE__ */ React.createElement("div", { className: `text-xs mt-0.5 ${theme
=== "dark" ? "text-gray-400" : "text-gray-600"}` }, discount2.discountValue)), /* @__PURE__ */
React.createElement(
      "button",
      {
        onClick: () => onRemoveDiscount(discount2._id),
        className: "p-1.5 text-red-500 hover:text-red-700 hover:bg-red-50 rounded transition-all",
        title: "Remove discount"
      },
      /* @__PURE__ */ React.createElement(FaTimes, { className: "w-4 h-4" })
    ))
  )), /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-2" }, /* @__PURE__ */
React.createElement(
    "input",
    {
      type: "text",
      placeholder: "Enter discount code",
```

```
      value: discountCode,
      onChange: (e) => setDiscountCode(e.target.value),
      onKeyPress: (e) => {
        if (e.key === "Enter" && !applyingDiscount) {
          applyDiscountCode();
        }
      },
      className: `flex-1 px-4 py-2 rounded-lg border text-sm focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white placeholder-
gray-500" : "bg-white border-[#d6c1b5] text-gray-900"}`
    }
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: onOpenDiscountModal,
      className: `p-2 rounded-lg transition-all ${theme === "dark" ? "bg-[#2A2724] text-gray-300 hover:bg-
[#322f2c]" : "bg-gray-100 text-gray-600 hover:bg-gray-200"}`,
      title: "Browse discounts"
    },
    /* @__PURE__ */ React.createElement(FaTag, { className: "w-4 h-4" })
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: applyDiscountCode,
      disabled: applyingDiscount || !discountCode || !discountCode.trim(),
      className: "px-4 py-2 text-white rounded-lg font-medium hover:opacity-90 transition-all shadow-md
disabled:opacity-50 disabled:cursor-not-allowed",
      style: { background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)" }
    },
    applyingDiscount ? "Applying..." : "Apply"
  ))), /* @__PURE__ */ React.createElement("div", { className: "space-y-2 mb-6 text-xs" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-between" }, /* @__PURE__ */ React.createElement("span",
{ className: `${theme === "dark" ? "text-gray-400" : "text-gray-600"}` }, "Subtotal"), /* @__PURE__ */
React.createElement("span", { className: `font-medium ${theme === "dark" ? "text-gray-200" : "text-gray-
800"}` }, "PHP ", subtotal.toFixed(2))), /* @__PURE__ */ React.createElement("div", { className: "flex
justify-between" }, /* @__PURE__ */ React.createElement("span", { className: `${theme === "dark" ? "text-
gray-400" : "text-gray-600"}` }, "Discount"), /* @__PURE__ */ React.createElement("span", { className: `font-
medium ${theme === "dark" ? "text-gray-200" : "text-gray-800"}` }, "PHP ", discount.toFixed(2))), /*
@__PURE__ */ React.createElement("div", { className: `border-t my-3 ${theme === "dark" ? "border-gray-700" :
"border-gray-300"}` }), /* @__PURE__ */ React.createElement("div", { className: "flex justify-between" }, /*
@__PURE__ */ React.createElement("span", { className: `text-sm font-bold ${theme === "dark" ? "text-white" :
"text-black"}` }, "Total"), /* @__PURE__ */ React.createElement("span", { className: "text-sm font-bold",
style: { color: "rgba(255, 133, 88, 1)" } }, "PHP ", total.toFixed(2)))), /* @__PURE__ */
React.createElement("div", { className: "mb-6" }, /* @__PURE__ */ React.createElement("div", { className:
"flex gap-2 justify-center" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setSelectedPaymentMethod("cash"),
      className: `w-24 flex flex-col items-center justify-center py-2 rounded-lg border-2 transition-all
${selectedPaymentMethod === "cash" ? "border-[#AD7F65] bg-[#f5f0ed]" : theme === "dark" ? "border-gray-600
bg-[#2A2724] hover:border-gray-500" : "border-gray-300 bg-white hover:border-gray-400"}`
    },
    /* @__PURE__ */ React.createElement("img", { src: cashIcon, alt: "Cash", className: "w-7 h-7 mb-1" }),
    /* @__PURE__ */ React.createElement("span", { className: `text-xs font-medium ${selectedPaymentMethod ===
"cash" ? "text-gray-900" : theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Cash")
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setSelectedPaymentMethod("qr"),
      className: `w-24 flex flex-col items-center justify-center py-0 rounded-lg border-2 transition-all
${selectedPaymentMethod === "qr" ? "border-[#AD7F65] bg-[#f5f0ed]" : theme === "dark" ? "border-gray-600 bg-
[#2A2724] hover:border-gray-500" : "border-gray-300 bg-white hover:border-gray-400"}`
    },
    /* @__PURE__ */ React.createElement("img", { src: qrIcon, alt: "QR Code", className: "w-12 h-12 mb-4 " }),
    /* @__PURE__ */ React.createElement("span", { className: `absolute text-xs font-medium translate-y-4
${selectedPaymentMethod === "qr" ? "text-gray-900" : theme === "dark" ? "text-gray-300" : "text-gray-700"}` },
"Gcash")
  ))), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: handleProceed,
      disabled: cart.length === 0 || !selectedPaymentMethod,
      className: "w-full py-3 text-white rounded-lg font-bold hover:opacity-90 disabled:opacity-50
disabled:cursor-not-allowed transition-all shadow-lg",
      style: { background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)" }
    },
    "Proceed"
```

```
  )), /* @__PURE__ */ React.createElement(
    VoidTransactionModal,
    {
      isOpen: isVoidModalOpen,
      onClose: handleVoidModalClose,
      onConfirm: handleVoidItemsConfirm,
      cartItems: cart
    }
  ), /* @__PURE__ */ React.createElement(
    RemoveItemPinModal,
    {
      isOpen: isRemoveModalOpen,
      onClose: handleRemoveModalClose,
      onConfirm: isBulkVoid ? handleBulkVoidConfirm : handleRemoveConfirm,
      item: itemToRemove
    }
  ));
});
export default OrderSummary;
```

FILE: pos/frontend/src/components/terminal/PrintCompleteModal.jsx
==========

```
import React from "react";
import { FaCheckCircle, FaQuestionCircle, FaExclamationTriangle } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const PrintCompleteModal = ({ isOpen, onConfirm, onRetry, error }) => {
  const { theme } = useTheme();
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 bg-opacity-50 backdrop-blur-sm flex items-center justify-center z-50 font-
poppins p-4"
    },
    /* @__PURE__ */ React.createElement("div", { className: `rounded-2xl p-6 shadow-2xl max-w-sm w-full
${theme === "dark" ? "bg-[#1E1B18]" : "bg-white"}` }, /* @__PURE__ */ React.createElement("div", { className:
"flex flex-col items-center" }, error ? /* @__PURE__ */ React.createElement(React.Fragment, null, /*
@__PURE__ */ React.createElement(
      FaExclamationTriangle,
      {
        className: "text-6xl text-red-500 mb-4",
        style: {
          animation: "pulseWarning 1s ease-in-out infinite"
        }
      }
    ), /* @__PURE__ */ React.createElement("h3", { className: `text-xl font-bold mb-2 ${theme === "dark" ?
"text-white" : "text-gray-800"}` }, "Print Failed"), /* @__PURE__ */ React.createElement("p", { className:
`text-center mb-3 ${theme === "dark" ? "text-gray-300" : "text-gray-600"}` }, error), /* @__PURE__ */
React.createElement("p", { className: `text-center text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-
500"}` }, "The transaction was saved. You can retry printing or continue with the next sale."), /* @__PURE__
*/ React.createElement("style", null, `
                @keyframes pulseWarning {
                  0% { transform: scale(1); opacity: 1; }
                  50% { transform: scale(1.1); opacity: 0.8; }
                  100% { transform: scale(1); opacity: 1; }
                }
              `)) : /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(FaQuestionCircle, { className: "text-6xl text-amber-500 mb-4" }), /* @__PURE__ */
React.createElement("h3", { className: `text-xl font-bold mb-2 ${theme === "dark" ? "text-white" : "text-
gray-800"}` }, "Print Complete?"), /* @__PURE__ */ React.createElement("p", { className: `text-center mb-6
${theme === "dark" ? "text-gray-300" : "text-gray-600"}` }, "Did the receipt print successfully?"), /*
@__PURE__ */ React.createElement("div", { className: "flex gap-3 w-full" }, /* @__PURE__ */
React.createElement(
      "button",
      {
        onClick: onConfirm,
        className: "flex-1 py-2 px-4 rounded-lg font-semibold text-white bg-green-500 hover:bg-green-600
transition-all flex items-center justify-center gap-2"
      },
      /* @__PURE__ */ React.createElement(FaCheckCircle, null),
      "Yes"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onRetry,
```

```
        className: "flex-1 py-2 px-4 rounded-lg font-semibold text-white bg-blue-500 hover:bg-blue-600
transition-all"
      },
      "Print Again"
    )))))
  );
};
export default PrintCompleteModal;
```

FILE: pos/frontend/src/components/terminal/PrintingModal.jsx
==========

```
import React from "react";
import { useTheme } from "../../context/ThemeContext";
const PrintingModal = ({ isOpen }) => {
  const { theme } = useTheme();
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 bg-opacity-50 backdrop-blur-sm flex items-center justify-center z-50 font-
poppins p-4"
    },
    /* @__PURE__ */ React.createElement("div", { className: `rounded-2xl p-8 shadow-2xl ${theme === "dark" ?
"bg-[#1E1B18]" : "bg-white"}` }, /* @__PURE__ */ React.createElement("div", { className: "flex flex-col
items-center" }, /* @__PURE__ */ React.createElement("div", { className: "relative mb-6" }, /* @__PURE__ */
React.createElement(
      "svg",
      {
        className: `w-24 h-24 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}`,
        fill: "none",
        stroke: "currentColor",
        viewBox: "0 0 24 24"
      },
      /* @__PURE__ */ React.createElement(
        "path",
        {
          strokeLinecap: "round",
          strokeLinejoin: "round",
          strokeWidth: "1.5",
          d: "M17 17h2a2 2 0 002-2v-4a2 2 0 00-2-2H5a2 2 0 00-2 2v4a2 2 0 002 2h2m2 4h6a2 2 0 002-2v-4a2 2 0
00-2-2H9a2 2 0 00-2 2v4a2 2 0 002 2zm8-12V5a2 2 0 00-2-2H9a2 2 0 00-2 2v4h10z"
        }
      )
    ), /* @__PURE__ */ React.createElement("div", { className: "absolute top-4 left-1/2 transform -translate-
x-1/2" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `w-12 h-16 border-2 rounded ${theme === "dark" ? "bg-gray-700 border-gray-500" : "bg-white
border-gray-400"}`,
        style: {
          animation: "paperPrint 2s ease-in-out infinite"
        }
      },
      /* @__PURE__ */ React.createElement("div", { className: `h-2 mt-2 mx-2 rounded ${theme === "dark" ?
"bg-gray-500" : "bg-gray-300"}` }),
      /* @__PURE__ */ React.createElement("div", { className: `h-1 mt-1 mx-2 rounded ${theme === "dark" ?
"bg-gray-500" : "bg-gray-300"}` }),
      /* @__PURE__ */ React.createElement("div", { className: `h-1 mt-1 mx-2 rounded ${theme === "dark" ?
"bg-gray-500" : "bg-gray-300"}` })
    )), /* @__PURE__ */ React.createElement("h3", { className: `text-xl font-semibold mb-2 ${theme ===
"dark" ? "text-white" : "text-gray-800"}` }, "Printing", /* @__PURE__ */ React.createElement("span",
{ className: "inline-flex ml-1" }, /* @__PURE__ */ React.createElement(
      "span",
      {
        className: "animate-bounce inline-block",
        style: { animationDelay: "0ms" }
      },
      "."
    ), /* @__PURE__ */ React.createElement(
      "span",
      {
        className: "animate-bounce inline-block",
        style: { animationDelay: "150ms" }
      },
      "."
```

```
    ), /* @__PURE__ */ React.createElement(
      "span",
      {
        className: "animate-bounce inline-block",
        style: { animationDelay: "300ms" }
      },
      "."
    ))), /* @__PURE__ */ React.createElement("div", { className: `w-48 h-2 rounded-full overflow-hidden
${theme === "dark" ? "bg-gray-700" : "bg-gray-200"}` }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "h-full bg-blue-500 rounded-full",
        style: {
          animation: "progressBar 3s ease-in-out infinite"
        }
      }
    )), /* @__PURE__ */ React.createElement("p", { className: `text-sm mt-3 ${theme === "dark" ? "text-gray-
400" : "text-gray-600"}` }, "Please wait while we print your receipt")), /* @__PURE__ */
React.createElement("style", null, `
          @keyframes paperPrint {
            0%, 100% {
              transform: translateY(0) translateX(-50%);
              opacity: 0;
            }
            20% {
              opacity: 1;
            }
            50% {
              transform: translateY(40px) translateX(-50%);
              opacity: 1;
            }
            80% {
              transform: translateY(40px) translateX(-50%);
              opacity: 0;
            }
          }

          @keyframes progressBar {
            0% {
              width: 0%;
            }
            100% {
              width: 100%;
            }
          }
        `))
  );
};
export default PrintingModal;
```

FILE: pos/frontend/src/components/terminal/ProductCard.jsx
==========

```
import { memo, useMemo } from "react";
import { MdCategory } from "react-icons/md";
import { useTheme } from "../../context/ThemeContext";
const ProductCard = memo(function ProductCard2({ product, onToggleExpand }) {
  const { theme } = useTheme();
  const getSizeQuantity = (sizeData) => {
    if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
      return sizeData.quantity;
    }
    return typeof sizeData === "number" ? sizeData : 0;
  };
  const getAllPricesFromSizeData = (sizeData) => {
    const prices = [];
    if (typeof sizeData !== "object" || sizeData === null) {
      return prices;
    }
    if (sizeData.price !== void 0 && sizeData.price > 0) {
      prices.push(sizeData.price);
    }
    if (sizeData.variantPrices && typeof sizeData.variantPrices === "object") {
      Object.values(sizeData.variantPrices).forEach((price) => {
        if (price > 0) prices.push(price);
      });
```

```
    }
    if (sizeData.variants && typeof sizeData.variants === "object") {
      Object.values(sizeData.variants).forEach((variantData) => {
        if (typeof variantData === "object" && variantData !== null && variantData.price > 0) {
          prices.push(variantData.price);
        }
      });
    }
    return prices;
  };
  const getSizePrice = (sizeData) => {
    if (typeof sizeData === "object" && sizeData !== null && sizeData.price !== void 0) {
      return sizeData.price;
    }
    return null;
  };
  const priceRange = useMemo(() => {
    if (product.sizes && typeof product.sizes === "object") {
      const prices = [];
      Object.values(product.sizes).forEach((sizeData) => {
        const sizePrices = getAllPricesFromSizeData(sizeData);
        prices.push(...sizePrices);
        const simplePrice = getSizePrice(sizeData);
        if (simplePrice !== null && simplePrice > 0 && !prices.includes(simplePrice)) {
          prices.push(simplePrice);
        }
      });
      if (prices.length > 0) {
        const minPrice = Math.min(...prices);
        const maxPrice = Math.max(...prices);
        if (minPrice !== maxPrice) {
          return { min: minPrice, max: maxPrice, isRange: true };
        }
        return { min: minPrice, max: maxPrice, isRange: false };
      }
    }
    return {
      min: product.itemPrice || 0,
      max: product.itemPrice || 0,
      isRange: false
    };
  }, [product.sizes, product.itemPrice]);
  const displayStock = useMemo(() => {
    if (typeof product.currentStock === "number") {
      return product.currentStock;
    }
    if (product.sizes && typeof product.sizes === "object") {
      return Object.values(product.sizes).reduce(
        (sum, sizeData) => sum + getSizeQuantity(sizeData),
        0
      );
    }
    return product.currentStock || 0;
  }, [product.currentStock, product.sizes]);
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `mb-4 rounded-2xl transition-all duration-300 ease-out overflow-hidden border-4 shadow
hover:shadow-xl hover:border-[#09A046] cursor-pointer ${theme === "dark" ? "bg-[#1E1B18] border-gray-700" :
"bg-white border-gray-200"}`,
      onClick: () => onToggleExpand(product)
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `aspect-square flex items-center justify-center overflow-hidden ${theme === "dark" ? "bg-
[#2A2724]" : "bg-gray-100"}`
      },
      product.itemImage && product.itemImage.trim() !== "" ? /* @__PURE__ */ React.createElement(
        "img",
        {
          src: product.itemImage,
          alt: product.itemName,
          loading: "lazy",
          decoding: "async",
          className: "w-full h-full object-cover"
        }
```

```
      ) : /* @__PURE__ */ React.createElement(
        MdCategory,
        {
          className: `text-4xl ${theme === "dark" ? "text-gray-600" : "text-gray-400"}`
        }
      )
    ),
    /* @__PURE__ */ React.createElement("div", { className: "p-3" }, /* @__PURE__ */ React.createElement(
      "h3",
      {
        className: `font-medium text-sm mb-1 truncate ${theme === "dark" ? "text-white" : "text-gray-900"}`
      },
      product.itemName
    ), /* @__PURE__ */ React.createElement("div", { className: "flex justify-between items-start gap-1 min-h-
[36px]" }, /* @__PURE__ */ React.createElement(
      "span",
      {
        className: `leading-tight ${priceRange.isRange ? "text-[11px]" : "text-sm"} ${theme === "dark" ?
"text-gray-400" : "text-gray-500"}`
      },
      priceRange.isRange ? `\u20B1${priceRange.min.toFixed(0)} - \u20B1${priceRange.max.toFixed(0)}` : `PHP
${priceRange.min.toFixed(2)}`
    ), /* @__PURE__ */ React.createElement(
      "span",
      {
        className: `text-xs whitespace-nowrap flex-shrink-0 ${displayStock === 0 ? "text-red-400" : theme ===
"dark" ? "text-gray-500" : "text-gray-500"}`
      },
      displayStock,
      " left"
    )), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: (e) => {
          e.stopPropagation();
          onToggleExpand(product);
        },
        className: "w-full mt-2 py-2 text-xs text-[#FFFFFF] rounded-lg border hover:opacity-90 transition-all
duration-300 transform hover:scale-105 active:scale-95 shadow-sm hover:shadow-md",
        style: {
          background: "linear-gradient(135deg, #10B981 0%, #059669 100%)",
          borderColor: "linear-gradient(135deg, #10B981 0%, #059669 100%)"
        }
      },
      "Add to Cart"
    ))
  );
});
export default ProductCard;
```

FILE: pos/frontend/src/components/terminal/ProductDetailsModal.jsx
==========

```
import { useEffect, useMemo, useState } from "react";
import { FaMinus, FaPlus, FaTimes } from "react-icons/fa";
import { MdCategory, MdShoppingBag } from "react-icons/md";
import { useTheme } from "../../context/ThemeContext";
import { generateDynamicSku } from "../../utils/skuUtils";
const ProductDetailsModal = ({
  isOpen,
  onClose,
  product,
  productQuantity,
  onDecrement,
  onIncrement,
  onAdd,
  onSetQuantity,
  selectedSize,
  onSelectSize,
  selectedVariant,
  onSelectVariant
}) => {
  const { theme } = useTheme();
  const VARIANT_ONLY_SIZE_KEY = "__VARIANT_ONLY__";
  if (!isOpen || !product) return null;
  const getSizeQuantity = (sizeData) => {
```

```
    if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
      return sizeData.quantity;
    }
    return typeof sizeData === "number" ? sizeData : 0;
  };
  const getSizePrice = (sizeData) => {
    if (typeof sizeData === "object" && sizeData !== null && sizeData.price !== void 0) {
      return sizeData.price;
    }
    return null;
  };
  const getSizeVariants = (sizeData) => {
    if (typeof sizeData === "object" && sizeData !== null && sizeData.variants && typeof sizeData.variants
=== "object") {
      return sizeData.variants;
    }
    return null;
  };
  const getSizeVariantPrices = (sizeData) => {
    if (typeof sizeData === "object" && sizeData !== null && sizeData.variantPrices && typeof
sizeData.variantPrices === "object") {
      return sizeData.variantPrices;
    }
    return null;
  };
  const getVariantQty = (variantData) => {
    if (typeof variantData === "number") return variantData;
    if (typeof variantData === "object" && variantData !== null) {
      return variantData.quantity || 0;
    }
    return 0;
  };
  const getSimpleVariants = () => {
    if (product.variant && typeof product.variant === "string") {
      const variants = product.variant.split(",").map((v) => v.trim()).filter((v) => v.length > 0);
      if (variants.length > 1) {
        return variants;
      }
    }
    return [];
  };
  const hasSizeVariants = () => {
    if (product.sizes && typeof product.sizes === "object") {
      return Object.values(product.sizes).some((sizeData) => {
        const variants = getSizeVariants(sizeData);
        return variants && typeof variants === "object" && Object.keys(variants).length > 0;
      });
    }
    return false;
  };
  const hasSimpleVariants = () => {
    if (hasSizeVariants()) return false;
    return getSimpleVariants().length > 0;
  };
  const hasVariants = () => {
    return hasSizeVariants() || hasSimpleVariants();
  };
  const getAllVariants = () => {
    const variantSet = /* @__PURE__ */ new Set();
    if (product.sizes && typeof product.sizes === "object") {
      Object.values(product.sizes).forEach((sizeData) => {
        const variants = getSizeVariants(sizeData);
        if (variants) {
          Object.keys(variants).forEach((variant) => {
            variantSet.add(variant);
          });
        }
      });
    }
    if (variantSet.size > 0) {
      return Array.from(variantSet);
    }
    const simpleVariants = getSimpleVariants();
    if (simpleVariants.length > 0) {
      simpleVariants.forEach((v) => variantSet.add(v));
    }
    return Array.from(variantSet);
```

```
  };
  const getAvailableSizesForVariant = (variant) => {
    if (!product.sizes || typeof product.sizes !== "object") return [];
    if (hasSimpleVariants()) {
      return Object.keys(product.sizes);
    }
    return Object.entries(product.sizes).filter(([size, sizeData]) => {
      const variants = getSizeVariants(sizeData);
      return variants && variants[variant] !== void 0;
    }).map(([size]) => size);
  };
  const getVariantQuantityInSize = (size, variant) => {
    if (hasSimpleVariants()) {
      if (!product.sizes || !product.sizes[size]) return 0;
      const sizeQty = getSizeQuantity(product.sizes[size]);
      const variantCount = getSimpleVariants().length;
      return sizeQty;
    }
    if (!product.sizes || !product.sizes[size]) return 0;
    const variants = getSizeVariants(product.sizes[size]);
    if (variants && variants[variant] !== void 0) {
      return getVariantQty(variants[variant]);
    }
    return 0;
  };
  const getVariantPriceInSize = (size, variant) => {
    if (!product.sizes || !product.sizes[size]) return null;
    const sizeData = product.sizes[size];
    if (!sizeData || typeof sizeData !== "object") return null;
    if (sizeData.variants && typeof sizeData.variants === "object") {
      const v = sizeData.variants?.[variant];
      if (v && typeof v === "object" && v.price !== void 0) return v.price;
    }
    const variantPrices = getSizeVariantPrices(sizeData);
    if (variantPrices && variantPrices[variant] !== void 0) return variantPrices[variant];
    return getSizePrice(sizeData);
  };
  const toFinitePrice = (v) => {
    const n = typeof v === "number" ? v : parseFloat(v);
    return Number.isFinite(n) ? n : null;
  };
  const collectVariantPrices = (variantData) => {
    const out = [];
    if (!variantData || typeof variantData !== "object") return out;
    if (Array.isArray(variantData.batches)) {
      variantData.batches.forEach((b) => {
        const n = toFinitePrice(b?.price);
        if (n !== null) out.push(n);
      });
    }
    const top = toFinitePrice(variantData.price);
    if (top !== null) out.push(top);
    return out;
  };
  const getDisplayPrice = () => {
    if (effectiveSelectedSize && selectedVariant && hasVariants()) {
      const variantPrice = getVariantPriceInSize(effectiveSelectedSize, selectedVariant);
      if (variantPrice !== null) {
        return variantPrice;
      }
    }
    if (effectiveSelectedSize && product.sizes && typeof product.sizes === "object" &&
product.sizes[effectiveSelectedSize]) {
      const sizeData = product.sizes[effectiveSelectedSize];
      const sizePrice = getSizePrice(sizeData);
      if (sizePrice !== null) {
        return sizePrice;
      }
    }
    return product.itemPrice || 0;
  };
  const getAllSellingPrices = () => {
    if (!product?.sizes || typeof product.sizes !== "object") return [];
    const prices = [];
    const keys = Object.keys(product.sizes);
    keys.forEach((sizeKey) => {
      const sizeData = product.sizes[sizeKey];
```

```
      if (!sizeData || typeof sizeData !== "object") return;
      if (sizeData.variantPrices && typeof sizeData.variantPrices === "object") {
        Object.values(sizeData.variantPrices).forEach((p) => {
          const n = parseFloat(p);
          if (Number.isFinite(n)) prices.push(n);
        });
        return;
      }
      const variants = getSizeVariants(sizeData);
      if (variants && sizeData.price !== void 0) {
        Object.keys(variants).forEach(() => {
          const n = parseFloat(sizeData.price);
          if (Number.isFinite(n)) prices.push(n);
        });
      }
    });
    return prices;
  };
  const getSellingPricesForVariant = (variant) => {
    if (!product?.sizes || typeof product.sizes !== "object") return [];
    const prices = [];
    const keys = Object.keys(product.sizes);
    keys.forEach((sizeKey) => {
      const sizeData = product.sizes[sizeKey];
      if (!sizeData || typeof sizeData !== "object") return;
      if (sizeData.variantPrices && typeof sizeData.variantPrices === "object" &&
sizeData.variantPrices[variant] !== void 0) {
        const n = parseFloat(sizeData.variantPrices[variant]);
        if (Number.isFinite(n)) prices.push(n);
        return;
      }
      const variants = getSizeVariants(sizeData);
      if (variants && variants[variant] !== void 0 && sizeData.price !== void 0) {
        const n = parseFloat(sizeData.price);
        if (Number.isFinite(n)) prices.push(n);
      }
    });
    return prices;
  };
  const getAllSellingPricesStrict = () => {
    if (!product?.sizes || typeof product.sizes !== "object") return [];
    const prices = [];
    Object.keys(product.sizes).forEach((sizeKey) => {
      const sizeData = product.sizes[sizeKey];
      if (!sizeData || typeof sizeData !== "object") return;
      if (sizeData.variants && typeof sizeData.variants === "object") {
        Object.values(sizeData.variants).forEach((v) => {
          prices.push(...collectVariantPrices(v));
        });
        return;
      }
      if (sizeData.variantPrices && typeof sizeData.variantPrices === "object") {
        Object.values(sizeData.variantPrices).forEach((p) => {
          const n = toFinitePrice(p);
          if (n !== null) prices.push(n);
        });
        return;
      }
      const shared = toFinitePrice(sizeData.price);
      if (shared !== null) prices.push(shared);
    });
    return Array.from(new Set(prices));
  };
  const getSellingPricesForVariantStrict = (variant) => {
    if (!product?.sizes || typeof product.sizes !== "object") return [];
    const prices = [];
    Object.keys(product.sizes).forEach((sizeKey) => {
      const sizeData = product.sizes[sizeKey];
      if (!sizeData || typeof sizeData !== "object") return;
      if (sizeData.variants && typeof sizeData.variants === "object" && sizeData.variants?.[variant]) {
        prices.push(...collectVariantPrices(sizeData.variants[variant]));
        return;
      }
      const vp = sizeData.variantPrices;
      if (vp && typeof vp === "object" && vp[variant] !== void 0) {
        const n = toFinitePrice(vp[variant]);
        if (n !== null) prices.push(n);
```

```
      return;
    }
  });
  return Array.from(new Set(prices));
};
const formatPriceRangeText = (vals) => {
  const nums = (vals || []).map((v) => parseFloat(v)).filter((n) => Number.isFinite(n));
  if (nums.length === 0) return `PHP 0.00`;
  const min = Math.min(...nums);
  const max = Math.max(...nums);
  if (min === max) return `PHP ${min.toFixed(2)}`;
  return `PHP ${min.toFixed(2)} - ${max.toFixed(2)}`;
};
const toQty = (v) => {
  const n = typeof v === "number" ? v : parseInt(v);
  return Number.isFinite(n) ? n : 0;
};
const getBatchPriceInfoForUI = () => {
  if (!productHasVariants) return null;
  if (!selectedVariant || !effectiveSelectedSize) return null;
  if (!product?.sizes || typeof product.sizes !== "object") return null;
  const sizeData = product.sizes[effectiveSelectedSize];
  if (!sizeData || typeof sizeData !== "object") return null;
  const variantData = sizeData?.variants?.[selectedVariant];
  if (variantData && typeof variantData === "object") {
    const batches = Array.isArray(variantData.batches) ? variantData.batches : [];
    if (batches.length > 0) {
      if (batches[0] && toQty(batches[0]?.qty) > 0) {
        const price2 = toFinitePrice(batches[0]?.price);
        if (price2 !== null) return { price: price2, batchSlotIndex: 0 };
      }
      const firstInStockIdx = batches.findIndex((b) => toQty(b?.qty) > 0);
      const idx = firstInStockIdx >= 0 ? firstInStockIdx : 0;
      const price = toFinitePrice(batches[idx]?.price);
      if (price !== null) return { price, batchSlotIndex: idx };
    }
    const top = toFinitePrice(variantData.price);
    if (top !== null) return { price: top, batchSlotIndex: null };
  }
  if (Array.isArray(sizeData.batches)) {
    const batches = sizeData.batches;
    if (batches.length > 0) {
      if (batches[0] && toQty(batches[0]?.qty) > 0) {
        const price2 = toFinitePrice(batches[0]?.price);
        if (price2 !== null) return { price: price2, batchSlotIndex: 0 };
      }
      const firstInStockIdx = batches.findIndex((b) => toQty(b?.qty) > 0);
      const idx = firstInStockIdx >= 0 ? firstInStockIdx : 0;
      const price = toFinitePrice(batches[idx]?.price);
      if (price !== null) return { price, batchSlotIndex: idx };
    }
  }
  return null;
};
const getSellingPriceText = () => {
  if (!productHasVariants) return `PHP ${Number(product.itemPrice || 0).toFixed(2)}`;
  const batchInfo = getBatchPriceInfoForUI();
  if (batchInfo?.price !== null && batchInfo?.price !== void 0) {
    return `PHP ${Number(batchInfo.price).toFixed(2)}`;
  }
  if (!selectedVariant) {
    const strictAll = getAllSellingPricesStrict();
    if (strictAll.length > 0) return formatPriceRangeText(strictAll);
    return formatPriceRangeText(getAllSellingPrices());
  }
  const strictForVariant = getSellingPricesForVariantStrict(selectedVariant);
  if (strictForVariant.length > 0) {
    const nums = strictForVariant.filter((v) => Number.isFinite(v));
    if (nums.length > 0) {
      const min = Math.min(...nums);
      const max = Math.max(...nums);
      if (min === max) return `PHP ${min.toFixed(2)}`;
      return formatPriceRangeText(nums);
    }
  }
  if (effectiveSelectedSize) {
    const exact = getVariantPriceInSize(effectiveSelectedSize, selectedVariant);
```

```
      if (exact !== null && exact !== void 0) {
        const n = parseFloat(exact);
        if (Number.isFinite(n)) return `PHP ${n.toFixed(2)}`;
      }
    }
    return formatPriceRangeText(getSellingPricesForVariant(selectedVariant));
  };
  const getDisplaySku = () => {
    if (!hasVariants()) return product.sku || "N/A";
    return generateDynamicSku(product.sku, selectedVariant, effectiveSelectedSize);
  };
  const getTotalStock = () => {
    if (product.sizes && typeof product.sizes === "object") {
      return Object.values(product.sizes).reduce(
        (sum, sizeData) => sum + getSizeQuantity(sizeData),
        0
      );
    }
    return product.currentStock || 0;
  };
  const productHasVariants = hasVariants();
  const allVariants = productHasVariants ? getAllVariants() : [];
  const productImages = Array.isArray(product?.productImages) && product.productImages.length > 0 ?
product.productImages : product?.itemImage ? [product.itemImage] : [];
  const [productImgIdx, setProductImgIdx] = useState(0);
  useEffect(() => {
    setProductImgIdx(0);
  }, [product?._id]);
  const sizeKeys = product?.sizes && typeof product.sizes === "object" ? Object.keys(product.sizes) : [];
  const availableSizesForSelectedVariant = selectedVariant ? getAvailableSizesForVariant(selectedVariant) :
[];
  const hasRealV2ForSelectedVariant = availableSizesForSelectedVariant.some(
    (s) => s && s !== VARIANT_ONLY_SIZE_KEY
  );
  const effectiveSelectedSize = selectedSize || (!hasRealV2ForSelectedVariant && selectedVariant ?
VARIANT_ONLY_SIZE_KEY : "");
  useEffect(() => {
    if (!selectedVariant) return;
    if (!onSelectSize) return;
    if (hasRealV2ForSelectedVariant) return;
    if (selectedSize === VARIANT_ONLY_SIZE_KEY) return;
    if (!sizeKeys.includes(VARIANT_ONLY_SIZE_KEY)) return;
    onSelectSize(VARIANT_ONLY_SIZE_KEY);
  }, [selectedVariant, hasRealV2ForSelectedVariant, selectedSize, onSelectSize, sizeKeys]);
  const availableSizes = productHasVariants ? selectedVariant ? hasRealV2ForSelectedVariant ?
availableSizesForSelectedVariant.filter((s) => s !== VARIANT_ONLY_SIZE_KEY) : [] : [] : product.sizes &&
typeof product.sizes === "object" ? Object.keys(product.sizes) : [];
  const getAvailableStock = () => {
    if (product.sizes && typeof product.sizes === "object" && effectiveSelectedSize) {
      if (hasSizeVariants() && selectedVariant) {
        return getVariantQuantityInSize(effectiveSelectedSize, selectedVariant);
      }
      return getSizeQuantity(product.sizes[effectiveSelectedSize]);
    }
    return product.currentStock || 0;
  };
  const isAddButtonDisabled = () => {
    if (productHasVariants && !selectedVariant) return true;
    if (product.sizes && typeof product.sizes === "object" && Object.keys(product.sizes).length > 0) {
      if (!effectiveSelectedSize) return true;
      let sizeStock;
      if (hasSizeVariants() && selectedVariant) {
        sizeStock = getVariantQuantityInSize(effectiveSelectedSize, selectedVariant);
      } else {
        sizeStock = getSizeQuantity(product.sizes[effectiveSelectedSize]);
      }
      if (sizeStock <= 0 || productQuantity > sizeStock) return true;
    } else {
      const totalStock = product.currentStock || 0;
      if (totalStock <= 0 || productQuantity > totalStock) return true;
    }
    return false;
  };
  const isIncrementDisabled = () => {
    const availableStock = getAvailableStock();
    return productQuantity >= availableStock;
  };
```

```
  const isDecrementDisabled = () => {
    return productQuantity <= 1;
  };
  const handleAdd = () => {
    onAdd();
  };
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 flex items-center justify-center z-[9999] backdrop-blur-sm bg-black/30",
      onClick: onClose
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-2xl w-full max-w-5xl shadow-2xl overflow-hidden mx-4 ${theme === "dark" ? "bg-
[#1E1B18]" : "bg-white"}`,
        onClick: (e) => e.stopPropagation()
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `px-6 py-4 flex items-center gap-3 border-b ${theme === "dark" ? "border-gray-700" :
"border-gray-100"}`
        },
        /* @__PURE__ */ React.createElement("div", { className: "w-10 h-10 rounded-lg bg-[#AD7F65] flex
items-center justify-center" }, /* @__PURE__ */ React.createElement(MdShoppingBag, { className: "text-white
text-xl" })),
        /* @__PURE__ */ React.createElement(
          "h2",
          {
            className: `text-xl font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}`
          },
          "Product Details"
        ),
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: onClose,
            className: `ml-auto transition ${theme === "dark" ? "text-gray-400 hover:text-gray-200" : "text-
gray-400 hover:text-gray-600"}`
          },
          /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
        )
      ),
      /* @__PURE__ */ React.createElement("div", { className: "p-6 overflow-hidden", style: { maxHeight:
"calc(100vh - 240px)" } }, /* @__PURE__ */ React.createElement("div", { className: "flex gap-6 h-full" }, /*
@__PURE__ */ React.createElement(
        "div",
        {
          className: `relative w-[420px] h-[420px] rounded-xl overflow-hidden flex-shrink-0 ${theme ===
"dark" ? "bg-[#2A2724]" : "bg-gray-100"}`
        },
        productImages.length > 0 && productImages[productImgIdx] ? /* @__PURE__ */
React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(
          "img",
          {
            src: productImages[productImgIdx],
            alt: product.itemName,
            className: "w-full h-full object-cover"
          }
        ), productImages.length > 1 && /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__
*/ React.createElement(
          "button",
          {
            type: "button",
            onClick: () => setProductImgIdx((i) => (i - 1 + productImages.length) % productImages.length),
            className: "absolute left-2 top-1/2 -translate-y-1/2 w-8 h-8 rounded-full bg-black/40 text-white
flex items-center justify-center"
          },
          "\u2039"
        ), /* @__PURE__ */ React.createElement(
          "button",
          {
            type: "button",
            onClick: () => setProductImgIdx((i) => (i + 1) % productImages.length),
            className: "absolute right-2 top-1/2 -translate-y-1/2 w-8 h-8 rounded-full bg-black/40 text-white
```

```
flex items-center justify-center"
          },
          "\u203A"
        ), /* @__PURE__ */ React.createElement("div", { className: "absolute bottom-2 left-0 right-0 flex
justify-center gap-1" }, productImages.map((_, idx) => /* @__PURE__ */ React.createElement(
          "button",
          {
            key: idx,
            type: "button",
            onClick: () => setProductImgIdx(idx),
            className: `w-2 h-2 rounded-full ${idx === productImgIdx ? "bg-white" : "bg-white/40"}`
          }
        )))) : /* @__PURE__ */ React.createElement("div", { className: "w-full h-full flex items-center
justify-center" }, /* @__PURE__ */ React.createElement(MdCategory, { className: "text-6xl text-gray-400" }))
      ), /* @__PURE__ */ React.createElement("div", { className: "flex-1 overflow-y-auto pr-2 min-h-0" }, /*
@__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm mb-1 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Product Name"
      ), /* @__PURE__ */ React.createElement(
        "h3",
        {
          className: `text-2xl font-bold mb-4 ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        product.itemName
      ), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-4 mb-4" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Category"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `font-medium flex items-center gap-1 ${theme === "dark" ? "text-gray-200" : "text-gray-
800"}`
        },
        /* @__PURE__ */ React.createElement(MdShoppingBag, { className: "text-gray-500" }),
        product.category || "N/A"
      )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Subcategory"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `font-medium ${theme === "dark" ? "text-gray-200" : "text-gray-800"}`
        },
        product.subCategory || "N/A"
      )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Price"
      ), /* @__PURE__ */ React.createElement("p", { className: "font-bold text-[#09A046] text-lg" },
getSellingPriceText())), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement(
        "p",
        {
          className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "SKU/Item Code"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `font-medium ${theme === "dark" ? "text-gray-200" : "text-gray-800"}`
        },
        getDisplaySku()
      )), !productHasVariants && /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement(
        "p",
```

```
        {
          className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Variant"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `font-medium ${theme === "dark" ? "text-gray-200" : "text-gray-800"}`
        },
        product.variant || "N/A"
      ))), productHasVariants && allVariants.length > 0 && /* @__PURE__ */ React.createElement("div",
{ className: "mb-4" }, /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm font-semibold mb-2 ${theme === "dark" ? "text-gray-300" : "text-gray-600"}`
        },
        "Select Variant ",
        /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")
      ), !selectedVariant && /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs mb-2 ${theme === "dark" ? "text-yellow-400" : "text-yellow-600"}`
        },
        "Please select a variant first"
      ), /* @__PURE__ */ React.createElement("div", { className: "flex flex-wrap gap-2" }, allVariants.map(
        (variant) => /* @__PURE__ */ React.createElement(
          "button",
          {
            key: variant,
            onClick: () => onSelectVariant(variant),
            className: `px-4 py-2 rounded-lg text-sm font-medium transition-all border-2 bg-transparent
${selectedVariant === variant ? "border-[#09A046] text-[#09A046] border-opacity-100" : theme === "dark" ?
"text-gray-300 border-gray-600 hover:border-[#09A046]" : "text-gray-700 border-gray-200 hover:border-
[#09A046]"}`
          },
          variant
        )
      ))), productHasVariants ? selectedVariant && availableSizes.length > 0 ? /* @__PURE__ */
React.createElement("div", { className: "mb-4" }, /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm font-semibold mb-2 ${theme === "dark" ? "text-gray-300" : "text-gray-600"}`
        },
        "Select Size ",
        /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")
      ), !selectedSize && /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs mb-2 ${theme === "dark" ? "text-yellow-400" : "text-yellow-600"}`
        },
        "Please select a size to continue"
      ), /* @__PURE__ */ React.createElement("div", { className: "flex flex-wrap gap-2" },
availableSizes.map((size) => {
        const variantStock = getVariantQuantityInSize(size, selectedVariant);
        const isOutOfStock = variantStock <= 0;
        return /* @__PURE__ */ React.createElement(
          "button",
          {
            key: size,
            onClick: () => !isOutOfStock && onSelectSize(size),
            disabled: isOutOfStock,
            className: `flex flex-col items-center px-4 py-2 rounded-lg text-xs font-medium transition-all
border-2 ${selectedSize === size ? "bg-transparent text-[#09A046] border-[#09A046]" : isOutOfStock ? theme
=== "dark" ? "bg-transparent text-gray-500 cursor-not-allowed border-gray-700" : "bg-transparent text-gray-
300 cursor-not-allowed border-gray-200" : theme === "dark" ? "bg-transparent text-gray-300 border-[#4A4037]
hover:border-[#09A046]" : "bg-transparent text-gray-700 border-gray-200 hover:border-[#09A046]"}`
          },
          /* @__PURE__ */ React.createElement("span", { className: "font-bold text-sm" }, size),
          /* @__PURE__ */ React.createElement(
            "span",
            {
              className: `text-[10px] mt-0.5 ${selectedSize === size ? "text-[#09A046]/80" : isOutOfStock ?
theme === "dark" ? "text-gray-600" : "text-gray-300" : theme === "dark" ? "text-gray-400" : "text-gray-500"}`
            },
            isOutOfStock ? "Out" : `${variantStock} pcs`
          )
        );
```

```
      })))) : !selectedVariant ? /* @__PURE__ */ React.createElement("div", { className: "mb-4" }, /*
@__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Size"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm mt-1 ${theme === "dark" ? "text-gray-500" : "text-gray-400"}`
        },
        "Select a variant first"
      )) : null : availableSizes.length > 0 ? /* @__PURE__ */ React.createElement("div", { className: "mb-
4" }, /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm font-semibold mb-2 ${theme === "dark" ? "text-gray-300" : "text-gray-600"}`
        },
        "Select Size ",
        /* @__PURE__ */ React.createElement("span", { className: "text-red-500" }, "*")
      ), !selectedSize && /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs mb-2 ${theme === "dark" ? "text-yellow-400" : "text-yellow-600"}`
        },
        "Please select a size to continue"
      ), /* @__PURE__ */ React.createElement("div", { className: "flex flex-wrap gap-2" },
availableSizes.map((size) => {
        const sizeStock = getSizeQuantity(product.sizes[size]);
        const isOutOfStock = sizeStock <= 0;
        return /* @__PURE__ */ React.createElement(
          "button",
          {
            key: size,
            onClick: () => !isOutOfStock && onSelectSize(size),
            disabled: isOutOfStock,
            className: `flex flex-col items-center px-4 py-2 rounded-lg text-xs font-medium transition-all
border-2 ${selectedSize === size ? "bg-transparent text-[#09A046] border-[#09A046]" : isOutOfStock ? theme
=== "dark" ? "bg-transparent text-gray-500 cursor-not-allowed border-gray-700" : "bg-transparent text-gray-
300 cursor-not-allowed border-gray-200" : theme === "dark" ? "bg-transparent text-gray-300 border-[#4A4037]
hover:border-[#09A046]" : "bg-transparent text-gray-700 border-gray-200 hover:border-[#09A046]"}`
          },
          /* @__PURE__ */ React.createElement("span", { className: "font-bold text-sm" }, size),
          /* @__PURE__ */ React.createElement(
            "span",
            {
              className: `text-[10px] mt-0.5 ${selectedSize === size ? "text-[#09A046]/80" : isOutOfStock ?
theme === "dark" ? "text-gray-600" : "text-gray-300" : theme === "dark" ? "text-gray-400" : "text-gray-500"}`
            },
            isOutOfStock ? "Out" : `${sizeStock} pcs`
          )
        );
      })))) : /* @__PURE__ */ React.createElement("div", { className: "mb-4" }, /* @__PURE__ */
React.createElement(
        "p",
        {
          className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Size"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `font-medium ${theme === "dark" ? "text-gray-200" : "text-gray-800"}`
        },
        product.size || "N/A"
      )), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-4 mb-4" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Quantity"
      ), /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `flex items-center gap-2 mt-1 ${productHasVariants && (!selectedVariant
```

```
|| !effectiveSelectedSize) || !productHasVariants && availableSizes.length > 0 && !effectiveSelectedSize ?
"opacity-40 pointer-events-none" : ""}`
        },
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: onDecrement,
            disabled: isDecrementDisabled() || productHasVariants && (!selectedVariant
|| !effectiveSelectedSize) || !productHasVariants && availableSizes.length > 0 && !effectiveSelectedSize,
            className: `w-8 h-8 flex items-center justify-center rounded-full transition-all
${isDecrementDisabled() || productHasVariants && (!selectedVariant || !effectiveSelectedSize)
|| !productHasVariants && availableSizes.length > 0 && !effectiveSelectedSize ? theme === "dark" ? "bg-gray-
700 text-gray-500 cursor-not-allowed" : "bg-gray-200 text-gray-400 cursor-not-allowed" : "bg-transparent
text-[#09A046] border-2 border-[#09A046] hover:bg-transparent"}`
          },
          /* @__PURE__ */ React.createElement(FaMinus, { className: "text-xs text-[#09A046]" })
        ),
        /* @__PURE__ */ React.createElement(
          "input",
          {
            type: "number",
            inputMode: "numeric",
            min: 1,
            value: productQuantity,
            onChange: (e) => {
              const n = parseInt(e.target.value, 10);
              if (Number.isFinite(n) && n > 0) onSetQuantity?.(n);
            },
            className: `w-10 text-center font-semibold px-2 py-1 rounded-lg border ${theme === "dark" ? "bg-
[#2A2724] border-gray-700 text-white" : "bg-white border-gray-200 text-gray-800"}`
          }
        ),
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: onIncrement,
            disabled: isIncrementDisabled() || productHasVariants && (!selectedVariant
|| !effectiveSelectedSize) || !productHasVariants && availableSizes.length > 0 && !effectiveSelectedSize,
            className: `w-8 h-8 flex items-center justify-center rounded-full transition-all
${isIncrementDisabled() || productHasVariants && (!selectedVariant || !effectiveSelectedSize)
|| !productHasVariants && availableSizes.length > 0 && !effectiveSelectedSize ? theme === "dark" ? "bg-gray-
700 text-gray-500 cursor-not-allowed" : "bg-gray-200 text-gray-400 cursor-not-allowed" : "bg-transparent
text-[#09A046] border-2 border-[#09A046] hover:bg-transparent"}`
          },
          /* @__PURE__ */ React.createElement(FaPlus, { className: "text-xs text-[#09A046]" })
        )
      )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Stock"
      ), productHasVariants ? selectedVariant && effectiveSelectedSize ? /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `font-medium mt-1 ${theme === "dark" ? "text-gray-200" : "text-gray-800"}`
        },
        getVariantQuantityInSize(effectiveSelectedSize, selectedVariant),
        " pcs"
      ) : !selectedVariant ? /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm mt-1 ${theme === "dark" ? "text-gray-500" : "text-gray-400"}`
        },
        "Select a variant"
      ) : /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm mt-1 ${theme === "dark" ? "text-gray-500" : "text-gray-400"}`
        },
        "Select a size"
      ) : availableSizes.length > 0 ? effectiveSelectedSize ? /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `font-medium mt-1 ${theme === "dark" ? "text-gray-200" : "text-gray-800"}`
        },
        getSizeQuantity(product.sizes[effectiveSelectedSize]),
```

```
        " pcs"
      ) : /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm mt-1 ${theme === "dark" ? "text-gray-500" : "text-gray-400"}`
        },
        "Select a size"
      ) : /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `font-medium mt-1 ${theme === "dark" ? "text-gray-200" : "text-gray-800"}`
        },
        getTotalStock()
      ))), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: handleAdd,
          disabled: isAddButtonDisabled(),
          className: `w-full py-3 rounded-lg font-semibold text-white transition-all ${isAddButtonDisabled() ?
"bg-gray-300 cursor-not-allowed" : ""}`,
          style: !isAddButtonDisabled() ? { background: "linear-gradient(135deg, #10B981 0%, #059669
100%)" } : {}
        },
        "Add to Cart"
      ))))
    )
  );
};
export default ProductDetailsModal;
```

FILE: pos/frontend/src/components/terminal/QRCodePaymentModal.jsx
==========

```
import { QRCodeSVG } from "qrcode.react";
import { useCallback, useEffect, useRef, useState } from "react";
import {
  FaCheckCircle,
  FaExclamationTriangle,
  FaRedo,
  FaSpinner,
  FaTimes,
  FaTimesCircle
} from "react-icons/fa";
import gcashHeader from "../../assets/gcashHeader.png";
import { API_BASE_URL as API_BASE, WS_BASE_URL as WS_BASE } from "../../config/api";
import { useTheme } from "../../context/ThemeContext";
import { sendReceiptToPrinter } from "../../utils/printBridge";
import PrintingModal from "./PrintingModal";
import ReceiptModal from "./ReceiptModal";
import SuccessModal from "./SuccessModal";
const POLL_INTERVAL = 3e3;
const STATUS = {
  IDLE: "IDLE",
  CREATING: "CREATING",
  PENDING: "PENDING",
  PAID: "PAID",
  FAILED: "FAILED",
  EXPIRED: "EXPIRED",
  CANCELLED: "CANCELLED"
};
const QRCodePaymentModal = ({
  isOpen,
  onClose,
  totalAmount,
  subtotalAmount = 0,
  discountAmount = 0,
  selectedDiscounts = [],
  cartItems = [],
  userId,
  performedById,
  performedByName,
  onTransactionComplete
}) => {
  const { theme } = useTheme();
  const [paymentStatus, setPaymentStatus] = useState(STATUS.IDLE);
  const [merchantOrderId, setMerchantOrderId] = useState(null);
```

```
const [checkoutUrl, setCheckoutUrl] = useState(null);
const [receiptNo, setReceiptNo] = useState(null);
const [gcashReference, setGcashReference] = useState(null);
const [expiresAt, setExpiresAt] = useState(null);
const [timeRemaining, setTimeRemaining] = useState(null);
const [errorMessage, setErrorMessage] = useState("");
const [showSuccess, setShowSuccess] = useState(false);
const [showReceipt, setShowReceipt] = useState(false);
const [receiptData, setReceiptData] = useState(null);
const [isAutoPrinting, setIsAutoPrinting] = useState(false);
const [printError, setPrintError] = useState(null);
const wsRef = useRef(null);
const pollIntervalRef = useRef(null);
const countdownRef = useRef(null);
const isMountedRef = useRef(true);
const cartSnapshotRef = useRef([]);
const paymentAmountsRef = useRef({ subtotal: 0, discount: 0, total: 0 });
useEffect(() => {
  isMountedRef.current = true;
  return () => {
    isMountedRef.current = false;
    cleanup();
  };
}, []);
useEffect(() => {
  if (!isOpen) {
    cleanup();
    resetState();
  }
}, [isOpen]);
const cleanup = useCallback(() => {
  if (wsRef.current) {
    wsRef.current.close();
    wsRef.current = null;
  }
  if (pollIntervalRef.current) {
    clearInterval(pollIntervalRef.current);
    pollIntervalRef.current = null;
  }
  if (countdownRef.current) {
    clearInterval(countdownRef.current);
    countdownRef.current = null;
  }
}, []);
const resetState = useCallback(() => {
  setPaymentStatus(STATUS.IDLE);
  setMerchantOrderId(null);
  setCheckoutUrl(null);
  setReceiptNo(null);
  setGcashReference(null);
  setExpiresAt(null);
  setTimeRemaining(null);
  setErrorMessage("");
  setShowSuccess(false);
  setShowReceipt(false);
  setReceiptData(null);
  setIsAutoPrinting(false);
  setPrintError(null);
  cartSnapshotRef.current = [];
  paymentAmountsRef.current = { subtotal: 0, discount: 0, total: 0 };
}, []);
const initiatePayment = useCallback(
  async () => {
    if (paymentStatus !== STATUS.IDLE) return;
    setPaymentStatus(STATUS.CREATING);
    setErrorMessage("");
    cartSnapshotRef.current = cartItems.map((item) => ({
      name: item.itemName || item.name || "Item",
      itemName: item.itemName || item.name || "Item",
      qty: item.quantity || 1,
      quantity: item.quantity || 1,
      price: item.itemPrice || item.price || 0,
      itemPrice: item.itemPrice || item.price || 0,
      total: (item.itemPrice || item.price || 0) * (item.quantity || 1),
      selectedSize: item.selectedSize || item.size || "",
      size: item.selectedSize || item.size || "",
      selectedVariation: item.selectedVariation || item.variant || "",
```

```
      variant: item.selectedVariation || item.variant || ""
    }));
    paymentAmountsRef.current = {
      subtotal: subtotalAmount || totalAmount + discountAmount,
      discount: discountAmount,
      total: totalAmount
    };
    try {
      const response = await fetch(`${API_BASE}/api/payments/gcash/create`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          items: cartItems.map((item) => ({
            productId: item.productId || item._id,
            itemName: item.itemName,
            sku: item.sku,
            variant: item.variant,
            selectedSize: item.selectedSize,
            quantity: item.quantity || 1,
            price: item.itemPrice || item.price || 0,
            itemImage: item.itemImage || ""
          })),
          totalAmount,
          subtotal: subtotalAmount || totalAmount + discountAmount,
          discount: discountAmount,
          userId: userId || performedById || "system",
          performedById: performedById || "",
          performedByName: performedByName || "",
          appliedDiscountIds: selectedDiscounts.map((d) => d._id)
        })
      });
      const data = await response.json();
      if (!response.ok || !data.success) {
        throw new Error(data.message || "Failed to create payment");
      }
      if (!isMountedRef.current) return;
      const {
        merchantOrderId: orderId,
        receiptNo: receipt,
        checkoutUrl: url,
        expiresAt: expiry
      } = data.data;
      setMerchantOrderId(orderId);
      setCheckoutUrl(url);
      setReceiptNo(receipt);
      setExpiresAt(expiry);
      setPaymentStatus(STATUS.PENDING);
      startCountdown(new Date(expiry));
      connectWebSocket(orderId);
      startPolling(orderId);
    } catch (error) {
      if (!isMountedRef.current) return;
      console.error("[GCash] Payment creation error:", error);
      setPaymentStatus(STATUS.FAILED);
      setErrorMessage(
        error.message || "Failed to create payment. Please try again."
      );
    }
  },
  [
    paymentStatus,
    cartItems,
    totalAmount,
    subtotalAmount,
    discountAmount,
    userId,
    performedById,
    performedByName,
    selectedDiscounts
  ]
);
useEffect(() => {
  if (isOpen && paymentStatus === STATUS.IDLE && cartItems.length > 0 && totalAmount > 0) {
    const timer = setTimeout(() => initiatePayment(), 300);
    return () => clearTimeout(timer);
  }
}, [isOpen, paymentStatus, cartItems.length, totalAmount]);
```

```
const connectWebSocket = useCallback((orderId) => {
  if (wsRef.current) {
    wsRef.current.close();
  }
  try {
    const ws = new WebSocket(
      `${WS_BASE}/ws/payments?orderId=${encodeURIComponent(orderId)}`
    );
    wsRef.current = ws;
    ws.onopen = () => {
      console.log("[WS] Connected for order:", orderId);
    };
    ws.onmessage = (event) => {
      try {
        const msg = JSON.parse(event.data);
        console.log("[WS] Message received:", msg);
        if (msg.type === "PAYMENT_UPDATE") {
          handlePaymentStatusChange(msg);
        }
      } catch (err) {
        console.error("[WS] Error parsing message:", err);
      }
    };
    ws.onclose = () => {
      console.log("[WS] Connection closed for order:", orderId);
    };
    ws.onerror = (error) => {
      console.warn("[WS] WebSocket error, falling back to polling:", error);
    };
  } catch (error) {
    console.warn("[WS] Failed to connect, using polling only:", error);
  }
}, []);
const startPolling = useCallback((orderId) => {
  if (pollIntervalRef.current) {
    clearInterval(pollIntervalRef.current);
  }
  pollIntervalRef.current = setInterval(async () => {
    try {
      const response = await fetch(
        `${API_BASE}/api/payments/gcash/status/${encodeURIComponent(orderId)}`
      );
      const data = await response.json();
      if (!isMountedRef.current) return;
      if (data.success && data.data) {
        const {
          status,
          paidAt,
          gcashReference: ref,
          receiptNo: receipt
        } = data.data;
        if (status === "PAID") {
          handlePaymentStatusChange({
            status: "PAID",
            paidAt,
            gcashReference: ref,
            receiptNo: receipt
          });
        } else if (status === "EXPIRED") {
          handlePaymentStatusChange({ status: "EXPIRED" });
        } else if (status === "FAILED") {
          handlePaymentStatusChange({ status: "FAILED" });
        }
      }
    } catch (error) {
      console.warn("[Poll] Error checking status:", error);
    }
  }, POLL_INTERVAL);
}, []);
const handlePaymentStatusChange = useCallback(
  (data) => {
    if (!isMountedRef.current) return;
    const { status, gcashReference: ref, receiptNo: receipt } = data;
    switch (status) {
      case "PAID":
        cleanup();
        setPaymentStatus(STATUS.PAID);
```

```
          if (ref) setGcashReference(ref);
          if (receipt) setReceiptNo(receipt);
          handlePaymentSuccess(ref, receipt);
          break;
        case "EXPIRED":
          cleanup();
          setPaymentStatus(STATUS.EXPIRED);
          setErrorMessage("Payment has expired. Please try again.");
          break;
        case "FAILED":
          cleanup();
          setPaymentStatus(STATUS.FAILED);
          setErrorMessage("Payment failed. Please try again.");
          break;
        default:
          break;
      }
    },
    [cleanup]
  );
  const handlePaymentSuccess = useCallback(
    (ref, receipt) => {
      setShowSuccess(true);
      const snapshotItems = cartSnapshotRef.current;
      const snapshotAmounts = paymentAmountsRef.current;
      const receiptInfo = {
        receiptNo: receipt || receiptNo || "------",
        items: snapshotItems.length > 0 ? snapshotItems : cartItems.map((item) => ({
          name: item.itemName || item.name || "Item",
          qty: item.quantity || 1,
          price: item.itemPrice || item.price || 0,
          total: (item.itemPrice || item.price || 0) * (item.quantity || 1),
          selectedSize: item.selectedSize || item.size || "",
          size: item.selectedSize || item.size || "",
          selectedVariation: item.selectedVariation || item.variant || "",
          variant: item.selectedVariation || item.variant || ""
        })),
        paymentMethod: "GCASH",
        subtotal: snapshotAmounts.subtotal || subtotalAmount || totalAmount + discountAmount,
        discount: snapshotAmounts.discount || discountAmount,
        discounts: selectedDiscounts.map((d) => ({
          title: d.title,
          value: d.discountValue
        })),
        total: snapshotAmounts.total || totalAmount,
        cash: snapshotAmounts.total || totalAmount,
        change: 0,
        referenceNo: ref || gcashReference || "",
        cashierName: performedByName || "Staff",
        date: (/* @__PURE__ */ new Date()).toLocaleDateString("en-US", { month: "short", day: "numeric", year:
"numeric" }),
        time: (/* @__PURE__ */ new Date()).toLocaleTimeString("en-US", { hour: "numeric", minute: "2-digit",
hour12: true })
      };
      setReceiptData(receiptInfo);
      if (onTransactionComplete) {
        onTransactionComplete({
          merchantOrderId,
          receiptNo: receipt || receiptNo,
          gcashReference: ref || gcashReference,
          totalAmount: snapshotAmounts.total || totalAmount
        });
      }
      setTimeout(() => {
        if (!isMountedRef.current) return;
        setShowSuccess(false);
        attemptAutoPrint(receiptInfo);
      }, 1500);
    },
    [
      cartItems,
      totalAmount,
      subtotalAmount,
      discountAmount,
      selectedDiscounts,
      performedByName,
      receiptNo,
```

```
      gcashReference,
      merchantOrderId,
      onTransactionComplete
    ]
  );
  const startCountdown = useCallback((expiryDate) => {
    if (countdownRef.current) {
      clearInterval(countdownRef.current);
    }
    const updateCountdown = () => {
      const now = /* @__PURE__ */ new Date();
      const diff = expiryDate - now;
      if (diff <= 0) {
        setTimeRemaining("0:00");
        clearInterval(countdownRef.current);
        return;
      }
      const minutes = Math.floor(diff / 6e4);
      const seconds = Math.floor(diff % 6e4 / 1e3);
      setTimeRemaining(`${minutes}:${seconds.toString().padStart(2, "0")}`);
    };
    updateCountdown();
    countdownRef.current = setInterval(updateCountdown, 1e3);
  }, []);
  const handleCancel = useCallback(async () => {
    cleanup();
    if (merchantOrderId && paymentStatus === STATUS.PENDING) {
      try {
        await fetch(
          `${API_BASE}/api/payments/gcash/cancel/${encodeURIComponent(merchantOrderId)}`,
          {
            method: "POST"
          }
        );
      } catch (error) {
        console.warn("[GCash] Error cancelling payment:", error);
      }
    }
    setPaymentStatus(STATUS.CANCELLED);
    onClose();
  }, [merchantOrderId, paymentStatus, cleanup, onClose]);
  const handleRetry = useCallback(() => {
    cleanup();
    resetState();
  }, [cleanup, resetState]);
  const attemptAutoPrint = async (receipt) => {
    setIsAutoPrinting(true);
    setPrintError(null);
    try {
      await sendReceiptToPrinter(receipt);
      setIsAutoPrinting(false);
      onClose();
    } catch (error) {
      setIsAutoPrinting(false);
      setPrintError(error.message || "Unable to reach the printer.");
      setShowReceipt(true);
    }
  };
  const handleNewTransaction = () => {
    setShowReceipt(false);
    onClose();
  };
  if (!isOpen) return null;
  const isDark = theme === "dark";
  return /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("div",
{ className: "fixed inset-0 bg-black/50 backdrop-blur-sm flex items-center justify-center z-50 font-poppins
p-4" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-2xl w-full max-w-md relative shadow-2xl ${isDark ? "bg-[#1E1B18]" : "bg-white"}`
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `px-6 py-4 border-b relative ${isDark ? "border-gray-700" : "border-gray-200"}`
      },
      /* @__PURE__ */ React.createElement(
```

```
        "button",
        {
          onClick: handleCancel,
          className: `absolute top-4 right-4 transition-colors ${isDark ? "text-gray-400 hover:text-gray-
200" : "text-gray-400 hover:text-gray-600"}`
        },
        /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
      ),
      /* @__PURE__ */ React.createElement(
        "h2",
        {
          className: `text-xl font-semibold text-center ${isDark ? "text-white" : "text-gray-900"}`
        },
        "GCash Payment"
      )
    ),
    /* @__PURE__ */ React.createElement("div", { className: "p-6" }, paymentStatus === STATUS.CREATING && /*
@__PURE__ */ React.createElement("div", { className: "flex flex-col items-center py-12" }, /* @__PURE__ */
React.createElement(FaSpinner, { className: "w-12 h-12 text-blue-500 animate-spin mb-4" }), /* @__PURE__ */
React.createElement(
      "p",
      {
        className: `text-lg font-medium ${isDark ? "text-gray-200" : "text-gray-800"}`
      },
      "Creating payment..."
    ), /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-sm mt-2 ${isDark ? "text-gray-400" : "text-gray-500"}`
      },
      "Connecting to GCash gateway"
    )), paymentStatus === STATUS.PENDING && checkoutUrl && /* @__PURE__ */ React.createElement(React.Fragment,
null, /* @__PURE__ */ React.createElement("div", { className: "mb-4 text-center" }, /* @__PURE__ */
React.createElement(
      "img",
      {
        src: gcashHeader,
        alt: "GCash",
        className: "mx-auto h-14 object-contain"
      }
    )), /* @__PURE__ */ React.createElement("div", { className: "flex justify-center mb-4" }, /* @__PURE__ */
React.createElement("div", { className: "bg-white p-4 border-2 border-blue-200 rounded-xl shadow-sm" }, /*
@__PURE__ */ React.createElement(
      QRCodeSVG,
      {
        value: checkoutUrl,
        size: 200,
        level: "H",
        includeMargin: true,
        imageSettings: {
          src: gcashHeader,
          height: 24,
          width: 60,
          excavate: true
        }
      }
    ))), /* @__PURE__ */ React.createElement("div", { className: "text-center mb-4" }, /* @__PURE__ */
React.createElement(
      "p",
      {
        className: `text-sm ${isDark ? "text-gray-400" : "text-gray-500"}`
      },
      "Amount to Pay"
    ), /* @__PURE__ */ React.createElement("p", { className: "text-3xl font-bold text-orange-500" }, "\u20B1",
totalAmount.toFixed(2))), timeRemaining && /* @__PURE__ */ React.createElement("div", { className: "text-
center mb-4" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `inline-flex items-center gap-2 px-4 py-2 rounded-full ${isDark ? "bg-yellow-900/30 text-
yellow-400" : "bg-yellow-50 text-yellow-700"}`
      },
      /* @__PURE__ */ React.createElement("span", { className: "text-sm font-medium" }, "\u23F1 Expires in ",
timeRemaining)
    )), /* @__PURE__ */ React.createElement("div", { className: "text-center mb-6" }, /* @__PURE__ */
React.createElement(
      "div",
      {
```

```
        className: `inline-flex items-center gap-2 px-4 py-2 rounded-full ${isDark ? "bg-blue-900/30" : "bg-
blue-50"}`
      },
      /* @__PURE__ */ React.createElement(FaSpinner, { className: "w-4 h-4 text-blue-500 animate-spin" }),
      /* @__PURE__ */ React.createElement(
        "span",
        {
          className: `text-sm font-medium ${isDark ? "text-blue-400" : "text-blue-700"}`
        },
        "Waiting for payment..."
      )
    )), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-lg p-3 mb-6 ${isDark ? "bg-[#2A2724]" : "bg-gray-50"}`
      },
      /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs font-medium mb-2 ${isDark ? "text-gray-300" : "text-gray-700"}`
        },
        "How to pay:"
      ),
      /* @__PURE__ */ React.createElement(
        "ol",
        {
          className: `text-xs space-y-1 ${isDark ? "text-gray-400" : "text-gray-500"}`
        },
        /* @__PURE__ */ React.createElement("li", null, "1. Open GCash app on your phone"),
        /* @__PURE__ */ React.createElement("li", null, '2. Tap "Scan QR" and scan the code above'),
        /* @__PURE__ */ React.createElement("li", null, "3. Confirm the amount and complete payment"),
        /* @__PURE__ */ React.createElement("li", null, "4. Payment will be confirmed automatically")
      )
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: handleCancel,
        className: `w-full py-3 px-6 rounded-lg font-semibold transition-all ${isDark ? "bg-[#2A2724] text-
gray-300 hover:bg-[#322f2c]" : "text-gray-700 bg-gray-200 hover:bg-gray-300"}`
      },
      "Cancel Payment"
    )), paymentStatus === STATUS.PAID && !showSuccess && !showReceipt && !isAutoPrinting && /* @__PURE__ */
React.createElement("div", { className: "flex flex-col items-center py-8" }, /* @__PURE__ */
React.createElement("div", { className: "w-20 h-20 rounded-full bg-green-100 flex items-center justify-center
mb-4" }, /* @__PURE__ */ React.createElement(FaCheckCircle, { className: "w-12 h-12 text-green-500" })), /*
@__PURE__ */ React.createElement(
      "p",
      {
        className: `text-xl font-bold mb-2 ${isDark ? "text-white" : "text-gray-900"}`
      },
      "Payment Confirmed!"
    ), /* @__PURE__ */ React.createElement("p", { className: "text-3xl font-bold text-green-500 mb-2" },
"\u20B1", totalAmount.toFixed(2)), gcashReference && /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-sm ${isDark ? "text-gray-400" : "text-gray-500"}`
      },
      "Ref: ",
      gcashReference
    )), paymentStatus === STATUS.FAILED && /* @__PURE__ */ React.createElement("div", { className: "flex
flex-col items-center py-8" }, /* @__PURE__ */ React.createElement("div", { className: "w-20 h-20 rounded-
full bg-red-100 flex items-center justify-center mb-4" }, /* @__PURE__ */ React.createElement(FaTimesCircle,
{ className: "w-12 h-12 text-red-500" })), /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-xl font-bold mb-2 ${isDark ? "text-white" : "text-gray-900"}`
      },
      "Payment Failed"
    ), /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-sm text-center mb-6 ${isDark ? "text-gray-400" : "text-gray-500"}`
      },
      errorMessage || "Something went wrong. Please try again."
    ), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3 w-full" }, /* @__PURE__ */
React.createElement(
      "button",
```

```
      {
        onClick: handleCancel,
        className: `flex-1 py-3 px-6 rounded-lg font-semibold transition-all ${isDark ? "bg-[#2A2724] text-
gray-300 hover:bg-[#322f2c]" : "text-gray-700 bg-gray-200 hover:bg-gray-300"}`
      },
      "Cancel"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: handleRetry,
        className: "flex-1 py-3 px-6 rounded-lg font-semibold text-white bg-[#8B7355] hover:bg-[#6d5a43]
transition-all flex items-center justify-center gap-2"
      },
      /* @__PURE__ */ React.createElement(FaRedo, { className: "w-4 h-4" }),
      " Retry"
    ))), paymentStatus === STATUS.EXPIRED && /* @__PURE__ */ React.createElement("div", { className: "flex
flex-col items-center py-8" }, /* @__PURE__ */ React.createElement("div", { className: "w-20 h-20 rounded-
full bg-yellow-100 flex items-center justify-center mb-4" }, /* @__PURE__ */
React.createElement(FaExclamationTriangle, { className: "w-12 h-12 text-yellow-500" })), /* @__PURE__ */
React.createElement(
      "p",
      {
        className: `text-xl font-bold mb-2 ${isDark ? "text-white" : "text-gray-900"}`
      },
      "Payment Expired"
    ), /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-sm text-center mb-6 ${isDark ? "text-gray-400" : "text-gray-500"}`
      },
      "The QR code has expired. Please generate a new one."
    ), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3 w-full" }, /* @__PURE__ */
React.createElement(
      "button",
      {
        onClick: handleCancel,
        className: `flex-1 py-3 px-6 rounded-lg font-semibold transition-all ${isDark ? "bg-[#2A2724] text-
gray-300 hover:bg-[#322f2c]" : "text-gray-700 bg-gray-200 hover:bg-gray-300"}`
      },
      "Cancel"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: handleRetry,
        className: "flex-1 py-3 px-6 rounded-lg font-semibold text-white bg-[#8B7355] hover:bg-[#6d5a43]
transition-all flex items-center justify-center gap-2"
      },
      /* @__PURE__ */ React.createElement(FaRedo, { className: "w-4 h-4" }),
      " New QR"
    ))), paymentStatus === STATUS.IDLE && /* @__PURE__ */ React.createElement("div", { className: "flex flex-
col items-center py-12" }, /* @__PURE__ */ React.createElement(
      "img",
      {
        src: gcashHeader,
        alt: "GCash",
        className: "mx-auto h-16 object-contain mb-6"
      }
    ), /* @__PURE__ */ React.createElement("p", { className: "text-3xl font-bold text-orange-500 mb-6" },
"\u20B1", totalAmount.toFixed(2)), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: initiatePayment,
        className: "w-full py-3 px-6 rounded-lg font-semibold text-white bg-[#007DFE] hover:bg-[#0066CC]
transition-all"
      },
      "Generate QR Code"
    )))
  )), /* @__PURE__ */ React.createElement(
    SuccessModal,
    {
      isOpen: showSuccess,
      onClose: () => setShowSuccess(false)
    }
  ), /* @__PURE__ */ React.createElement(
    ReceiptModal,
    {
      isOpen: showReceipt,
```

```
      onClose: () => setShowReceipt(false),
      receiptData,
      onNewTransaction: handleNewTransaction,
      initialPrintError: printError,
      onPrintSuccess: handleNewTransaction,
      disableAutoPrint: true
    }
  ), /* @__PURE__ */ React.createElement(PrintingModal, { isOpen: isAutoPrinting }));
};
export default QRCodePaymentModal;
```

FILE: pos/frontend/src/components/terminal/ReceiptModal.jsx
==========

```
import React, { useState, useEffect, useCallback } from "react";
import { FaPrint } from "react-icons/fa";
import { MdRefresh } from "react-icons/md";
import PrintingModal from "./PrintingModal";
import PrintCompleteModal from "./PrintCompleteModal";
import "../../styles/print.css";
import { sendReceiptToPrinter } from "../../utils/printBridge";
import { formatReceiptVariantSizeLine } from "../../utils/transactionDisplay";
import { useTheme } from "../../context/ThemeContext";
const ReceiptModal = ({
  isOpen,
  onClose,
  receiptData,
  onNewTransaction,
  initialPrintError = null,
  onPrintSuccess = null,
  disableAutoPrint = false
}) => {
  const { theme } = useTheme();
  const [isPrinting, setIsPrinting] = useState(false);
  const [showPrintComplete, setShowPrintComplete] = useState(false);
  const [printAttempts, setPrintAttempts] = useState(0);
  const [printError, setPrintError] = useState(null);
  const receipt = receiptData || {
    receiptNo: "000000",
    items: [],
    reference: "",
    paymentMethod: "",
    subtotal: 0,
    discount: 0,
    total: 0,
    cash: 0,
    change: 0,
    date: (/* @__PURE__ */ new Date()).toLocaleDateString(),
    time: (/* @__PURE__ */ new Date()).toLocaleTimeString()
  };
  const handlePrint = useCallback(async () => {
    setIsPrinting(true);
    setPrintAttempts((prev) => prev + 1);
    setPrintError(null);
    try {
      await sendReceiptToPrinter(receipt);
      if (onPrintSuccess) {
        onPrintSuccess();
      }
    } catch (error) {
      setPrintError(error.message || "Unable to open print dialog. Please allow pop-ups.");
      setShowPrintComplete(true);
    } finally {
      setIsPrinting(false);
    }
  }, [receipt]);
  useEffect(() => {
    if (!isOpen) {
      setIsPrinting(false);
      setShowPrintComplete(false);
      setPrintAttempts(0);
      setPrintError(null);
      return;
    }
    setPrintError(initialPrintError);
    if (disableAutoPrint) return;
```

```
    const autoPrintTimer = setTimeout(() => {
      handlePrint();
    }, 100);
    return () => clearTimeout(autoPrintTimer);
  }, [isOpen, disableAutoPrint, handlePrint]);
  useEffect(() => {
    if (printError && showPrintComplete) {
      const timer = setTimeout(() => {
        setShowPrintComplete(false);
      }, 2e3);
      return () => clearTimeout(timer);
    }
  }, [printError, showPrintComplete]);
  if (!isOpen) return null;
  const handlePrintSuccess = () => {
    setShowPrintComplete(false);
    setPrintError(null);
    if (onPrintSuccess) {
      onPrintSuccess();
    }
  };
  const handlePrintRetry = () => {
    setShowPrintComplete(false);
    setPrintError(null);
    handlePrint();
  };
  return /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "receipt-print-content",
      id: "receipt-to-print",
      style: {
        position: "fixed",
        left: "-9999px",
        top: "0",
        width: "58mm",
        background: "white",
        padding: "10px",
        fontFamily: "Arial, sans-serif"
      }
    },
    /* @__PURE__ */ React.createElement("div", { style: { textAlign: "center", marginBottom: "10px" } }, /*
@__PURE__ */ React.createElement("h2", { style: { margin: "5px 0", fontSize: "18px", fontWeight: "bold",
color: "#1a365d" } }, "Create Your Style"), /* @__PURE__ */ React.createElement("p", { style: { fontSize:
"10px", margin: "2px 0", color: "#4a5568" } }, "Pasonanca, Zamboanga City")),
    /* @__PURE__ */ React.createElement("div", { style: { borderBottom: "1px dashed #000", marginBottom:
"10px" } }),
    /* @__PURE__ */ React.createElement("div", { style: { textAlign: "center", marginBottom: "10px" } }, /*
@__PURE__ */ React.createElement("p", { style: { fontSize: "10px", margin: "2px 0", color: "#718096",
textTransform: "uppercase", letterSpacing: "1px" } }, "Receipt"), /* @__PURE__ */ React.createElement("p",
{ style: { fontSize: "18px", fontWeight: "bold", margin: "2px 0", color: "#2d3748" } }, "#",
receipt.receiptNo)),
    /* @__PURE__ */ React.createElement("div", { style: { fontSize: "11px", marginBottom: "10px" } }, /*
@__PURE__ */ React.createElement("div", { style: { display: "flex", justifyContent: "space-between", margin:
"4px 0" } }, /* @__PURE__ */ React.createElement("span", { style: { color: "#4a5568" } }, "Date:"), /*
@__PURE__ */ React.createElement("span", { style: { color: "#1a202c" } }, receipt.date, ", ", receipt.time)),
/* @__PURE__ */ React.createElement("div", { style: { display: "flex", justifyContent: "space-between",
margin: "4px 0" } }, /* @__PURE__ */ React.createElement("span", { style: { color: "#4a5568" } }, "Cashier:"),
/* @__PURE__ */ React.createElement("span", { style: { color: "#1a202c" } }, receipt.cashierName || "Staff")),
/* @__PURE__ */ React.createElement("div", { style: { display: "flex", justifyContent: "space-between",
margin: "4px 0" } }, /* @__PURE__ */ React.createElement("span", { style: { color: "#4a5568" } }, "Payment:"),
/* @__PURE__ */ React.createElement("span", { style: { color: "#1a202c" } }, receipt.paymentMethod ||
"Cash"))),
    /* @__PURE__ */ React.createElement("div", { style: { borderBottom: "1px dashed #000", marginBottom:
"10px" } }),
    /* @__PURE__ */ React.createElement("div", { style: { marginBottom: "10px" } }, receipt.items &&
receipt.items.map(
      (item, index) => /* @__PURE__ */ React.createElement("div", { key: index, style: { marginBottom:
"8px" } }, /* @__PURE__ */ React.createElement("p", { style: { fontSize: "11px", fontWeight: "600", margin:
"0", color: "#1a202c" } }, item.name), /* @__PURE__ */ React.createElement("p", { style: { fontSize: "10px",
margin: "2px 0", color: "#718096" } }, item.qty, " x PHP ", item.price.toFixed(2)))
    )),
    /* @__PURE__ */ React.createElement("div", { style: { borderBottom: "1px dashed #000", marginBottom:
"10px" } }),
    /* @__PURE__ */ React.createElement("div", { style: { fontSize: "11px" } }, /* @__PURE__ */
React.createElement("div", { style: { display: "flex", justifyContent: "space-between", margin: "4px 0" } },
/* @__PURE__ */ React.createElement("span", { style: { color: "#4a5568" } }, "Subtotal:"), /* @__PURE__ */
```

```
React.createElement("span", { style: { color: "#1a202c" } }, "PHP ", receipt.subtotal.toFixed(2))), /*
@__PURE__ */ React.createElement("div", { style: { display: "flex", justifyContent: "space-between", margin:
"4px 0" } }, /* @__PURE__ */ React.createElement("span", { style: { color: "#4a5568" } }, "Discount:"), /*
@__PURE__ */ React.createElement("span", { style: { color: "#1a202c" } }, "PHP ",
receipt.discount.toFixed(2))), /* @__PURE__ */ React.createElement("div", { style: { display: "flex",
justifyContent: "space-between", margin: "8px 0", paddingTop: "8px" } }, /* @__PURE__ */
React.createElement("span", { style: { fontWeight: "bold", color: "#1a365d", fontSize: "13px" } }, "Total:"),
/* @__PURE__ */ React.createElement("span", { style: { fontWeight: "bold", color: "#1a365d", fontSize:
"13px" } }, "PHP ", receipt.total.toFixed(2))), /* @__PURE__ */ React.createElement("div", { style: { display:
"flex", justifyContent: "space-between", margin: "4px 0" } }, /* @__PURE__ */ React.createElement("span",
{ style: { color: "#4a5568" } }, "Amount Received:"), /* @__PURE__ */ React.createElement("span", { style:
{ color: "#1a202c" } }, "PHP ", receipt.cash.toFixed(2))), /* @__PURE__ */ React.createElement("div", { style:
{ display: "flex", justifyContent: "space-between", margin: "4px 0" } }, /* @__PURE__ */
React.createElement("span", { style: { color: "#4a5568" } }, "Change:"), /* @__PURE__ */
React.createElement("span", { style: { color: "#1a202c" } }, "PHP ", receipt.change.toFixed(2)))),
    /* @__PURE__ */ React.createElement("div", { style: { borderBottom: "1px dashed #000", marginBottom:
"10px", marginTop: "10px" } }),
    /* @__PURE__ */ React.createElement("div", { style: { textAlign: "center", paddingTop: "5px" } }, /*
@__PURE__ */ React.createElement("p", { style: { fontSize: "11px", color: "#4a5568", margin: "2px 0" } },
"Thank you for your purchase!"), /* @__PURE__ */ React.createElement("p", { style: { fontSize: "10px", color:
"#a0aec0", margin: "2px 0" } }, "This is not an official receipt"))
  ), /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 bg-opacity-50 backdrop-blur-sm flex items-center justify-center z-50 font-
poppins p-4 no-print"
    },
    /* @__PURE__ */ React.createElement("div", { className: `rounded-2xl w-full max-w-md relative shadow-2xl
max-h-[90vh] overflow-y-auto ${theme === "dark" ? "bg-[#1E1B18]" : "bg-white"}` }, /* @__PURE__ */
React.createElement("div", { className: "p-6" }, /* @__PURE__ */ React.createElement("div", { className:
"text-center mb-4" }, /* @__PURE__ */ React.createElement("h2", { className: `text-xl font-bold ${theme ===
"dark" ? "text-white" : "text-[#1a365d]"}` }, "Create Your Style"), /* @__PURE__ */ React.createElement("p",
{ className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Pasonanca, Zamboanga
City")), /* @__PURE__ */ React.createElement("div", { className: `border-t border-b border-dashed py-4 mb-4
text-center ${theme === "dark" ? "border-gray-600" : "border-gray-300"}` }, /* @__PURE__ */
React.createElement("p", { className: `text-xs uppercase tracking-wider mb-1 ${theme === "dark" ? "text-gray-
500" : "text-gray-500"}` }, "Receipt"), /* @__PURE__ */ React.createElement("p", { className: `text-2xl font-
bold ${theme === "dark" ? "text-white" : "text-[#2d3748]"}` }, "#", receipt.receiptNo)), /* @__PURE__ */
React.createElement("div", { className: "mb-4 space-y-2" }, /* @__PURE__ */ React.createElement("div",
{ className: "flex justify-between text-sm" }, /* @__PURE__ */ React.createElement("span", { className: theme
=== "dark" ? "text-gray-400" : "text-gray-500" }, "Date:"), /* @__PURE__ */ React.createElement("span",
{ className: theme === "dark" ? "text-gray-200" : "text-gray-800" }, receipt.date, ", ", receipt.time)), /*
@__PURE__ */ React.createElement("div", { className: "flex justify-between text-sm" }, /* @__PURE__ */
React.createElement("span", { className: theme === "dark" ? "text-gray-400" : "text-gray-500" }, "Cashier:"),
/* @__PURE__ */ React.createElement("span", { className: theme === "dark" ? "text-gray-200" : "text-gray-
800" }, receipt.cashierName || "Staff")), /* @__PURE__ */ React.createElement("div", { className: "flex
justify-between text-sm" }, /* @__PURE__ */ React.createElement("span", { className: theme === "dark" ?
"text-gray-400" : "text-gray-500" }, "Payment:"), /* @__PURE__ */ React.createElement("span", { className:
theme === "dark" ? "text-gray-200" : "text-gray-800" }, receipt.paymentMethod || "Cash"))), /* @__PURE__ */
React.createElement("div", { className: `border-t pt-4 mb-4 ${theme === "dark" ? "border-gray-700" : "border-
gray-200"}` }, receipt.items && receipt.items.length > 0 ? receipt.items.map((item, index) => {
      const variantSizeLine = formatReceiptVariantSizeLine(item);
      return /* @__PURE__ */ React.createElement("div", { key: index, className: "mb-3" }, /* @__PURE__ */
React.createElement("p", { className: `text-sm font-semibold ${theme === "dark" ? "text-gray-200" : "text-
gray-800"}` }, item.name), variantSizeLine && /* @__PURE__ */ React.createElement("p", { className: `text-xs
${theme === "dark" ? "text-gray-500" : "text-gray-500"}` }, variantSizeLine), /* @__PURE__ */
React.createElement("p", { className: `text-sm ${theme === "dark" ? "text-gray-500" : "text-gray-500"}` },
item.qty, " x PHP ", item.price.toFixed(2)));
    }) : /* @__PURE__ */ React.createElement("p", { className: "text-center text-gray-500 py-4" }, "No items
in this transaction")), /* @__PURE__ */ React.createElement("div", { className: `border-t pt-4 space-y-2
${theme === "dark" ? "border-gray-700" : "border-gray-200"}` }, /* @__PURE__ */ React.createElement("div",
{ className: "flex justify-between text-sm" }, /* @__PURE__ */ React.createElement("span", { className: theme
=== "dark" ? "text-gray-400" : "text-gray-500" }, "Subtotal:"), /* @__PURE__ */ React.createElement("span",
{ className: theme === "dark" ? "text-gray-200" : "text-gray-800" }, "PHP ", receipt.subtotal.toFixed(2))),
/* @__PURE__ */ React.createElement("div", { className: "flex justify-between text-sm" }, /* @__PURE__ */
React.createElement("span", { className: theme === "dark" ? "text-gray-400" : "text-gray-500" }, "Discount:"),
/* @__PURE__ */ React.createElement("span", { className: theme === "dark" ? "text-gray-200" : "text-gray-
800" }, "PHP ", receipt.discount.toFixed(2))), /* @__PURE__ */ React.createElement("div", { className: `flex
justify-between pt-3 mt-2 border-t ${theme === "dark" ? "border-gray-700" : "border-gray-200"}` }, /*
@__PURE__ */ React.createElement("span", { className: `font-bold ${theme === "dark" ? "text-white" : "text-
[#1a365d]"}` }, "Total:"), /* @__PURE__ */ React.createElement("span", { className: `font-bold text-lg
${theme === "dark" ? "text-white" : "text-[#1a365d]"}` }, "PHP ", receipt.total.toFixed(2))), /* @__PURE__ */
React.createElement("div", { className: "flex justify-between text-sm" }, /* @__PURE__ */
React.createElement("span", { className: theme === "dark" ? "text-gray-400" : "text-gray-500" }, "Amount
Received:"), /* @__PURE__ */ React.createElement("span", { className: theme === "dark" ? "text-gray-200" :
"text-gray-800" }, "PHP ", receipt.cash.toFixed(2))), /* @__PURE__ */ React.createElement("div", { className:
"flex justify-between text-sm" }, /* @__PURE__ */ React.createElement("span", { className: theme === "dark" ?
```

```
"text-gray-400" : "text-gray-500" }, "Change:"), /* @__PURE__ */ React.createElement("span", { className:
theme === "dark" ? "text-gray-200" : "text-gray-800" }, "PHP ", receipt.change.toFixed(2)))), /* @__PURE__ */
React.createElement("div", { className: `text-center py-6 mt-4 border-t border-dashed ${theme === "dark" ?
"border-gray-600" : "border-gray-300"}` }, /* @__PURE__ */ React.createElement("p", { className: `text-sm
${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Thank you for your purchase!"), /* @__PURE__ */
React.createElement("p", { className: `text-xs mt-1 ${theme === "dark" ? "text-gray-500" : "text-gray-
400"}` }, "This is not an official receipt")), /* @__PURE__ */ React.createElement("div", { className:
"space-y-3" }, /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: handlePrint,
        className: "w-full py-3 px-6 rounded-lg font-semibold text-white bg-blue-500 hover:bg-blue-600
transition-all flex items-center justify-center gap-2"
      },
      /* @__PURE__ */ React.createElement(FaPrint, null),
      printError ? "Print Again" : "Print Receipt"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onNewTransaction,
        className: "w-full py-3 px-6 rounded-lg font-semibold text-white bg-green-500 hover:bg-green-600
transition-all flex items-center justify-center gap-2"
      },
      /* @__PURE__ */ React.createElement(MdRefresh, null),
      "New Transaction"
    )))),
    /* @__PURE__ */ React.createElement(PrintingModal, { isOpen: isPrinting }),
    /* @__PURE__ */ React.createElement(
      PrintCompleteModal,
      {
        isOpen: showPrintComplete,
        onConfirm: handlePrintSuccess,
        onRetry: handlePrintRetry,
        error: printError
      }
    )
  ));
};
export default ReceiptModal;
```

FILE: pos/frontend/src/components/terminal/RemoveItemPinModal.jsx
==========

```
import { useEffect, useRef, useState } from "react";
import { FaChevronDown, FaEye, FaEyeSlash, FaTimes } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const voidReasons = [
  "Customer cancellation",
  "Wrong transaction",
  "System error",
  "Payment issue",
  "Other"
];
const RemoveItemPinModal = ({
  isOpen,
  onClose,
  onConfirm,
  item,
  requireReason = true,
  allowedRoles = null,
  title = "Void Transaction",
  subtitle = "This action requires manager authorization.",
  confirmText = "Confirm Void",
  pinLabel = "Manager PIN",
  reasonLabel = "Reason for Void",
  reasonOptions = voidReasons,
  hideAmountCard = false,
  amountLabel = "Amount to Void:"
}) => {
  const { theme } = useTheme();
  const [pin, setPin] = useState("");
  const [showPin, setShowPin] = useState(false);
  const [reason, setReason] = useState("");
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const [isReasonDropdownOpen, setIsReasonDropdownOpen] = useState(false);
```

```
const reasonDropdownRef = useRef(null);
const isConfirmingRef = useRef(false);
useEffect(() => {
  if (isOpen) {
    setPin("");
    setShowPin(false);
    setReason("");
    setError("");
    setIsReasonDropdownOpen(false);
    isConfirmingRef.current = false;
  }
}, [isOpen]);
useEffect(() => {
  const handleClickOutside = (event) => {
    if (reasonDropdownRef.current && !reasonDropdownRef.current.contains(event.target)) {
      setIsReasonDropdownOpen(false);
    }
  };
  if (isReasonDropdownOpen) {
    document.addEventListener("mousedown", handleClickOutside);
  }
  return () => {
    document.removeEventListener("mousedown", handleClickOutside);
  };
}, [isReasonDropdownOpen]);
const handlePinChange = (event) => {
  const digitsOnly = event.target.value.replace(/\D/g, "").slice(0, 6);
  setPin(digitsOnly);
  if (error) {
    setError("");
  }
};
const handleReasonSelect = (selectedReason) => {
  setReason(selectedReason);
  setIsReasonDropdownOpen(false);
  setError("");
};
const handleConfirm = async (e) => {
  if (e) {
    e.preventDefault();
    e.stopPropagation();
  }
  if (loading || isConfirmingRef.current) {
    return;
  }
  console.log("[RemoveItemPinModal] handleConfirm called, reason:", reason, "pin length:", pin.length);
  if (requireReason && !reason) {
    setError("Please select a reason for void");
    return;
  }
  if (pin.length !== 6) {
    setError("Please enter a 6-digit PIN");
    return;
  }
  setError("");
  isConfirmingRef.current = true;
  setLoading(true);
  try {
    const currentUser = JSON.parse(localStorage.getItem("currentUser") || "{}");
    if (!currentUser.email) {
      setError("User information not found. Please log in again.");
      setLoading(false);
      isConfirmingRef.current = false;
      return;
    }
    const trimmedPin = pin.trim();
    if (trimmedPin.length !== 6 || !/^\d{6}$/.test(trimmedPin)) {
      setError("PIN must be exactly 6 digits");
      setLoading(false);
      isConfirmingRef.current = false;
      return;
    }
    const voidReason = reason;
    const response = await fetch("http://localhost:5000/api/employees/verify-pin", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
```

```
        },
        body: JSON.stringify({
          email: currentUser.email,
          pin: trimmedPin
        })
      });
      const data = await response.json();
      if (data.success) {
        if (Array.isArray(allowedRoles) && allowedRoles.length > 0 && !allowedRoles.includes(data.data?.role))
{
          setError(
            `Invalid PIN. Only ${allowedRoles.join(" or ")} can perform this action.`
          );
          setLoading(false);
          isConfirmingRef.current = false;
          return;
        }
        console.log("[RemoveItemPinModal] PIN verified successfully, calling onConfirm with reason:",
voidReason);
        const approverInfo = {
          approvedBy: data.data?.name || data.data?.firstName || "Unknown",
          approvedById: data.data?._id || data.data?.id || "",
          approvedByRole: data.data?.role || null
        };
        if (onConfirm && (!requireReason || voidReason)) {
          try {
            setLoading(false);
            if (requireReason) {
              await onConfirm(voidReason, approverInfo);
            } else {
              await onConfirm(approverInfo);
            }
            console.log("[RemoveItemPinModal] onConfirm called successfully with approver:", approverInfo);
            setLoading(false);
          } catch (error2) {
            console.error("[RemoveItemPinModal] Error calling onConfirm:", error2);
            setError(error2?.message || "Failed to complete authorization. Please try again.");
            setLoading(false);
            isConfirmingRef.current = false;
            return;
          }
        } else {
          console.error("[RemoveItemPinModal] Missing required input or onConfirm:", { voidReason,
onConfirm: !!onConfirm, requireReason });
          setError("Failed to continue. Missing required input or callback.");
          setLoading(false);
          isConfirmingRef.current = false;
          return;
        }
      } else {
        setError(data.message || "Invalid PIN. Please try again.");
        setLoading(false);
        isConfirmingRef.current = false;
      }
    } catch (error2) {
      console.error("Error verifying PIN:", error2);
      setError("Failed to connect to server. Please try again.");
      setLoading(false);
      isConfirmingRef.current = false;
    }
  };
  if (!isOpen) return null;
  const itemTotal = item ? (item.itemPrice || 0).toFixed(2) : "0.00";
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 flex items-center justify-center z-[10000] backdrop-blur-sm bg-opacity-50",
      onClick: onClose
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-2xl w-full max-w-md shadow-2xl overflow-hidden ${theme === "dark" ? "bg-
[#1E1B18]" : "bg-white"}`,
        onClick: (e) => e.stopPropagation()
      },
      /* @__PURE__ */ React.createElement("div", { className: "p-8" }, /* @__PURE__ */
```

```
React.createElement("div", { className: "flex justify-between items-start mb-6" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement("div", { className: "w-10 h-10 rounded-full bg-red-600 flex items-center justify-
center" }, /* @__PURE__ */ React.createElement("span", { className: "text-white text-xl font-bold" }, "!")),
/* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("h2", { className:
"text-xl font-bold text-red-600" }, title), /* @__PURE__ */ React.createElement("p", { className: `text-sm
mt-1 ${theme === "dark" ? "text-gray-400" : "text-gray-600"}` }, subtitle))), /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: onClose,
          className: `transition ${theme === "dark" ? "text-gray-400 hover:text-gray-200" : "text-gray-400
hover:text-gray-600"}`
        },
        /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
      )), !hideAmountCard && /* @__PURE__ */ React.createElement("div", { className: "mb-6" }, /* @__PURE__
*/ React.createElement("label", { className: `block text-sm font-semibold mb-2 ${theme === "dark" ? "text-
gray-300" : "text-gray-700"}` }, amountLabel), /* @__PURE__ */ React.createElement("div", { className:
`rounded-lg p-4 ${theme === "dark" ? "bg-red-900/30" : "bg-red-50"}` }, /* @__PURE__ */
React.createElement("span", { className: "text-2xl font-bold text-red-600" }, "PHP ", itemTotal))),
requireReason && /* @__PURE__ */ React.createElement("div", { className: "mb-6" }, /* @__PURE__ */
React.createElement("label", { className: `block text-sm font-semibold mb-2 ${theme === "dark" ? "text-gray-
300" : "text-gray-700"}` }, reasonLabel, " ", /* @__PURE__ */ React.createElement("span", { className: "text-
red-500" }, "*")), /* @__PURE__ */ React.createElement("div", { className: "relative", ref:
reasonDropdownRef }, /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: () => setIsReasonDropdownOpen(!isReasonDropdownOpen),
          className: `w-full px-4 py-3 border rounded-lg text-left flex items-center justify-between
focus:outline-none focus:ring-2 focus:ring-red-200 focus:border-red-400 transition-all ${theme === "dark" ?
"bg-[#2A2724] border-gray-600" : "bg-white border-gray-300"}`
        },
        /* @__PURE__ */ React.createElement("span", { className: reason ? theme === "dark" ? "text-gray-200" :
"text-gray-700" : "text-gray-400" }, reason || "Select a reason..."),
        /* @__PURE__ */ React.createElement(
          FaChevronDown,
          {
            className: `text-gray-500 transition-transform ${isReasonDropdownOpen ? "rotate-180" : ""}`
          }
        )
      ), isReasonDropdownOpen && /* @__PURE__ */ React.createElement("div", { className: `absolute z-10 w-
full mt-2 border rounded-lg shadow-lg overflow-hidden ${theme === "dark" ? "bg-[#2A2724] border-gray-600" :
"bg-white border-gray-200"}` }, reasonOptions.map(
        (voidReason) => /* @__PURE__ */ React.createElement(
          "button",
          {
            key: voidReason,
            type: "button",
            onClick: () => handleReasonSelect(voidReason),
            className: `w-full px-4 py-2 text-left text-sm transition-colors ${reason === voidReason ? theme
=== "dark" ? "bg-red-900/30 text-red-400 font-semibold" : "bg-red-50 text-red-600 font-semibold" : theme ===
"dark" ? "text-gray-300 hover:bg-gray-700" : "text-gray-700 hover:bg-gray-50"}`
          },
          voidReason
        )
      )))), /* @__PURE__ */ React.createElement("div", { className: "mb-6" }, /* @__PURE__ */
React.createElement("label", { className: `block text-sm font-semibold mb-2 ${theme === "dark" ? "text-gray-
300" : "text-gray-700"}` }, pinLabel, " ", /* @__PURE__ */ React.createElement("span", { className: "text-
red-500" }, "*")), /* @__PURE__ */ React.createElement("div", { className: "relative" }, /* @__PURE__ */
React.createElement(
        "input",
        {
          type: showPin ? "text" : "password",
          inputMode: "numeric",
          id: "void-transaction-pin",
          name: "void-transaction-pin",
          value: pin,
          onChange: handlePinChange,
          onBlur: (e) => {
            const digitsOnly = e.target.value.replace(/\D/g, "").slice(0, 6);
            if (digitsOnly !== pin) {
              setPin(digitsOnly);
            }
          },
          className: `w-full px-4 py-3 pr-12 border rounded-lg focus:outline-none focus:ring-2 focus:ring-
red-200 focus:border-red-400 transition-all ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white
```

```
placeholder-gray-500" : "bg-white border-gray-300 text-gray-900"}`,
          placeholder: "Enter 6-digit PIN",
          maxLength: 6,
          autoFocus: true,
          autoComplete: "new-password"
        }
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: () => setShowPin((v) => !v),
          className: `absolute right-3 top-1/2 -translate-y-1/2 transition ${theme === "dark" ? "text-gray-
400 hover:text-gray-200" : "text-gray-500 hover:text-gray-700"}`,
          "aria-label": showPin ? "Hide PIN" : "Show PIN"
        },
        showPin ? /* @__PURE__ */ React.createElement(FaEyeSlash, { className: "w-4 h-4" }) : /* @__PURE__ */
React.createElement(FaEye, { className: "w-4 h-4" })
      ))), error && /* @__PURE__ */ React.createElement("div", { className: `mb-4 p-3 border rounded-lg text-
sm text-center ${theme === "dark" ? "bg-red-900/30 border-red-800 text-red-400" : "bg-red-100 border-red-300
text-red-700"}` }, error), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3" }, /*
@__PURE__ */ React.createElement(
        "button",
        {
          onClick: onClose,
          className: `flex-1 px-4 py-3 rounded-lg font-medium transition-all ${theme === "dark" ? "bg-
[#2A2724] text-gray-300 hover:bg-[#322f2c]" : "bg-gray-200 text-gray-700 hover:bg-gray-300"}`,
          disabled: loading
        },
        "Cancel"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: (e) => {
            e.preventDefault();
            e.stopPropagation();
            handleConfirm();
          },
          disabled: loading || pin.length !== 6 || requireReason && !reason,
          className: "flex-1 px-4 py-3 rounded-lg bg-red-600 hover:bg-red-700 text-white font-medium
transition-all disabled:opacity-50 disabled:cursor-not-allowed"
        },
        loading ? "Verifying..." : confirmText
      )))
    )
  );
};
export default RemoveItemPinModal;
```

FILE: pos/frontend/src/components/terminal/SuccessModal.jsx
==========

```
import React from "react";
import { FaTimes } from "react-icons/fa";
import { useTheme } from "../../context/ThemeContext";
const SuccessModal = ({ isOpen, onClose }) => {
  const { theme } = useTheme();
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 bg-opacity-50 backdrop-blur-sm flex items-center justify-center z-50 font-
poppins p-4"
    },
    /* @__PURE__ */ React.createElement("div", { className: `rounded-2xl w-full max-w-sm relative shadow-2xl
${theme === "dark" ? "bg-[#1E1B18]" : "bg-white"}` }, /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: `absolute top-4 right-4 transition-colors z-10 ${theme === "dark" ? "text-gray-400
hover:text-gray-200" : "text-gray-400 hover:text-gray-600"}`
      },
      /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
    ), /* @__PURE__ */ React.createElement("div", { className: "p-8 text-center" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-center mb-6" }, /* @__PURE__ */ React.createElement(
      "div",
```

```
      {
        className: "w-24 h-24 rounded-full flex items-center justify-center",
        style: {
          backgroundColor: "#B8E7D3"
        }
      },
      /* @__PURE__ */ React.createElement(
        "svg",
        {
          className: "w-12 h-12 text-green-600",
          fill: "none",
          stroke: "currentColor",
          strokeWidth: "3",
          viewBox: "0 0 24 24",
          style: {
            animation: "checkmark 0.6s ease-out"
          }
        },
        /* @__PURE__ */ React.createElement(
          "path",
          {
            strokeLinecap: "round",
            strokeLinejoin: "round",
            d: "M5 1314 4L19 7",
            style: {
              strokeDasharray: "22",
              strokeDashoffset: "22",
              animation: "checkmark-draw 0.4s ease-out 0.2s forwards"
            }
          }
        )
      ),
      /* @__PURE__ */ React.createElement("style", null, `
                @keyframes checkmark {
                  0% {
                    transform: scale(0) rotate(-45deg);
                    opacity: 0;
                  }
                  50% {
                    transform: scale(1.2) rotate(10deg);
                    opacity: 1;
                  }
                  100% {
                    transform: scale(1) rotate(0deg);
                    opacity: 1;
                  }
                }
                @keyframes checkmark-draw {
                  to {
                    stroke-dashoffset: 0;
                  }
                }
              `)
    )), /* @__PURE__ */ React.createElement("h3", { className: `text-2xl font-bold mb-3 ${theme === "dark" ?
"text-white" : "text-gray-800"}` }, "Success!"), /* @__PURE__ */ React.createElement("p", { className: `mb-6
${theme === "dark" ? "text-gray-400" : "text-gray-600"}` }, "The transaction was successful."), /* @__PURE__
*/ React.createElement(
      "button",
      {
        onClick: onClose,
        className: "px-8 py-3 rounded-lg font-semibold text-white transition-all",
        style: {
          backgroundColor: "#8B7355"
        },
        onMouseEnter: (e) => e.target.style.backgroundColor = "#6d5a43",
        onMouseLeave: (e) => e.target.style.backgroundColor = "#8B7355"
      },
      "OK"
    )))
  );
};
export default SuccessModal;
```

FILE: pos/frontend/src/components/terminal/VoidTransactionModal.jsx
==========

```
import React, { useState, useEffect } from "react";
import { FaTimes } from "react-icons/fa";
import { MdCategory } from "react-icons/md";
import { HiDocumentRemove } from "react-icons/hi";
import { useTheme } from "../../context/ThemeContext";
const VARIANT_ONLY_SIZE_KEY = "__VARIANT_ONLY__";
const VoidTransactionModal = ({ isOpen, onClose, onConfirm, cartItems }) => {
  const { theme } = useTheme();
  const [selectedItems, setSelectedItems] = useState([]);
  const getItemUniqueKey = (item) => {
    const productId = item._id || item.productId || "";
    const size = item.selectedSize || item.size || "";
    const variant = item.selectedVariation || item.variant || "";
    return `${productId}-${size}-${variant}`;
  };
  useEffect(() => {
    if (isOpen) {
      setSelectedItems([]);
    }
  }, [isOpen]);
  const handleItemToggle = (item) => {
    const itemKey = getItemUniqueKey(item);
    setSelectedItems((prev) => {
      const isSelected = prev.some((i) => getItemUniqueKey(i) === itemKey);
      if (isSelected) {
        return prev.filter((i) => getItemUniqueKey(i) !== itemKey);
      } else {
        return [...prev, item];
      }
    });
  };
  const isItemSelected = (item) => {
    const itemKey = getItemUniqueKey(item);
    return selectedItems.some((i) => getItemUniqueKey(i) === itemKey);
  };
  const calculateSelectedTotal = () => {
    return selectedItems.reduce((total, item) => total + item.itemPrice * item.quantity, 0);
  };
  const handleVoidItems = () => {
    if (selectedItems.length > 0) {
      onConfirm(selectedItems);
    }
  };
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 flex items-center justify-center z-[10001] backdrop-blur-sm bg-black/30",
      onClick: onClose
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-2xl w-full max-w-md shadow-2xl overflow-hidden max-h-[90vh] flex flex-col ${theme
=== "dark" ? "bg-[#1E1B18]" : "bg-white"}`,
        onClick: (e) => e.stopPropagation()
      },
      /* @__PURE__ */ React.createElement("div", { className: "p-6 pb-4" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-between items-center" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */ React.createElement("h2",
{ className: `text-xl font-bold ${theme === "dark" ? "text-white" : "text-gray-900"}` }, "Void Transaction")),
/* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onClose,
          className: `transition ${theme === "dark" ? "text-gray-400 hover:text-gray-200" : "text-gray-400
hover:text-gray-600"}`
        },
        /* @__PURE__ */ React.createElement(HiDocumentRemove, { className: "w-6 h-6 text-red-500" })
      ))),
      /* @__PURE__ */ React.createElement("div", { className: "flex-1 overflow-auto px-6 pb-4" },
cartItems.length === 0 ? /* @__PURE__ */ React.createElement("div", { className: "text-center text-gray-400
py-10" }, /* @__PURE__ */ React.createElement("p", null, "No items in cart")) : /* @__PURE__ */
React.createElement("div", { className: "space-y-3" }, cartItems.map(
        (item, index) => /* @__PURE__ */ React.createElement(
          "div",
          {
```

```
            key: `${getItemUniqueKey(item)}-${index}`,
            onClick: () => handleItemToggle(item),
            className: `relative rounded-xl border p-4 cursor-pointer transition-all ${isItemSelected(item) ?
theme === "dark" ? "border-red-500 bg-red-900/20" : "border-red-400 bg-red-50/30" : theme === "dark" ?
"border-gray-700 bg-[#2A2724] hover:border-gray-600" : "border-gray-200 bg-white hover:border-gray-300"}`
          },
          /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__
*/ React.createElement("div", { className: `w-16 h-16 rounded-lg shrink-0 overflow-hidden ${theme === "dark" ?
"bg-gray-700" : "bg-gray-100"}` }, item.itemImage && item.itemImage.trim() !== "" ? /* @__PURE__ */
React.createElement(
            "img",
            {
              src: item.itemImage,
              alt: item.itemName,
              className: "w-full h-full object-cover"
            }
          ) : /* @__PURE__ */ React.createElement("div", { className: "w-full h-full flex items-center
justify-center" }, /* @__PURE__ */ React.createElement(MdCategory, { className: "text-2xl text-gray-400" }))),
/* @__PURE__ */ React.createElement("div", { className: "flex-1" }, /* @__PURE__ */ React.createElement("h3",
{ className: `font-semibold text-sm ${theme === "dark" ? "text-white" : "text-gray-900"}` }, item.itemName ?
item.itemName.replace(/\s*\([^)]*\)\s*$/, "").trim() : "Item"), /* @__PURE__ */ React.createElement("p",
{ className: `text-xs mt-1 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "SKU: ", item.sku), /*
@__PURE__ */ React.createElement("p", { className: `text-xs ${theme === "dark" ? "text-gray-400" : "text-
gray-500"}` }, "Size: ", (item.selectedSize || item.size) === VARIANT_ONLY_SIZE_KEY ? "\u2014" :
item.selectedSize || item.size || "N/A"), /* @__PURE__ */ React.createElement("p", { className: `text-xs
${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Color: ", item.selectedVariation || "N/A"), /*
@__PURE__ */ React.createElement("p", { className: "text-sm font-bold text-red-500 mt-1" }, "PHP ",
(item.itemPrice * item.quantity).toFixed(2))), /* @__PURE__ */ React.createElement("div", { className: "flex
items-center justify-center" }, /* @__PURE__ */ React.createElement(
            "div",
            {
              className: `w-6 h-6 rounded border-2 flex items-center justify-center transition-all
${isItemSelected(item) ? "bg-red-500 border-red-500" : theme === "dark" ? "border-gray-500 bg-[#2A2724]" :
"border-gray-300 bg-white"}`
            },
            isItemSelected(item) && /* @__PURE__ */ React.createElement("svg", { className: "w-4 h-4 text-
white", fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */
React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 3, d: "M5 13l4
4L19 7" }))
          )))
        )
      ))),
      /* @__PURE__ */ React.createElement("div", { className: `p-6 pt-4 border-t ${theme === "dark" ?
"border-gray-700" : "border-gray-100"}` }, /* @__PURE__ */ React.createElement("div", { className: "flex gap-
3" }, /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onClose,
          className: `flex-1 px-4 py-3 rounded-lg font-medium transition-all ${theme === "dark" ? "bg-
[#2A2724] text-gray-300 hover:bg-[#322f2c]" : "bg-gray-200 text-gray-700 hover:bg-gray-300"}`
        },
        "Cancel"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: handleVoidItems,
          disabled: selectedItems.length === 0,
          className: "flex-1 px-4 py-3 rounded-lg bg-red-600 hover:bg-red-700 text-white font-medium
transition-all disabled:opacity-50 disabled:cursor-not-allowed"
        },
        "Void Items"
      )))
    )
  );
};
export default VoidTransactionModal;
```

FILE: pos/frontend/src/components/transaction/CashTurnOverSlipModal.jsx
==========

```
import React, { useRef } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { FaPrint, FaCheck } from "react-icons/fa";
const DENOMINATIONS = [
  { key: "p1000", label: "\u20B11,000" },
  { key: "p500", label: "\u20B1500" },
```

```
  { key: "p200", label: "\u20B1200" },
  { key: "p100", label: "\u20B1100" },
  { key: "p50", label: "\u20B150" },
  { key: "p20", label: "\u20B120" },
  { key: "p10", label: "\u20B110" },
  { key: "p5", label: "\u20B15" },
  { key: "p1", label: "\u20B11" },
  { key: "c25", label: "25\xA2" },
  { key: "c10", label: "10\xA2" },
  { key: "c5", label: "5\xA2" }
];
const formatCurrency = (val) => {
  const abs = Math.abs(val).toLocaleString("en-PH", { minimumFractionDigits: 2, maximumFractionDigits: 2 });
  return val < 0 ? `-\u20B1${abs}` : `\u20B1${abs}`;
};
const formatAbs = (val) => `\u20B1${Math.abs(val).toLocaleString("en-PH", { minimumFractionDigits: 2,
maximumFractionDigits: 2 })}`;
const CashTurnOverSlipModal = ({
  isOpen,
  onClose,
  summary,
  denominations,
  totalCashOnHand,
  openingFloat,
  cashToRemit,
  variance,
  employeeName,
  receivedBy,
  slipNo = "CTS-2PQ5T",
  // In a real app, this should come from the backend response
  storeName = "Santos General Merchandise"
  // Can come from merchant settings later
}) => {
  const printRef = useRef(null);
  const handlePrint = () => {
    const printContent = printRef.current.innerHTML;
    const printWindow = window.open("", "", "width=800,height=900");
    printWindow.document.write(`
            <html>
                <head>
                    <title>Print Cash Turn-Over Slip</title>
                    <style>
                        body { font-family: sans-serif; padding: 20px; }
                        /* Add more styles to match the React components for printing */
                        .print-container { max-width: 600px; margin: 0 auto; }
                    </style>
                </head>
                <body>
                    <div class="print-container">
                        ${printContent}
                    </div>
                    <script>
                        window.onload = function() { window.print(); window.close(); }
                    <\/script>
                </body>
            </html>
        `);
    printWindow.document.close();
  };
  if (!isOpen) return null;
  const varianceLabel = variance === 0 ? "BALANCED" : variance > 0 ? "OVER" : "SHORT";
  const varianceColorClass = variance === 0 ? "text-green-600 bg-green-50" : variance > 0 ? "text-blue-600
bg-blue-50" : "text-red-500 bg-red-50";
  const nonZeroDenominations = DENOMINATIONS.map((d) => ({
    ...d,
    qty: denominations[d.key] || 0,
    amount: (denominations[d.key] || 0) * (d.key.startsWith("p") ? parseInt(d.key.slice(1)) :
parseInt(d.key.slice(1)) / 100)
  })).filter((d) => d.qty > 0);
  return /* @__PURE__ */ React.createElement(AnimatePresence, null, isOpen && /* @__PURE__ */
React.createElement("div", { className: "fixed inset-0 flex flex-col items-center justify-center z-[200] p-4
sm:p-6 pointer-events-none" }, /* @__PURE__ */ React.createElement(
    motion.div,
    {
      initial: { opacity: 0 },
      animate: { opacity: 1 },
      exit: { opacity: 0 },
```

```
      className: "absolute inset-0 bg-black/50 backdrop-blur-sm pointer-events-auto cursor-pointer",
      onClick: onClose
    }
  ), /* @__PURE__ */ React.createElement(
    motion.div,
    {
      initial: { opacity: 0, scale: 0.95, y: 20 },
      animate: { opacity: 1, scale: 1, y: 0 },
      exit: { opacity: 0, scale: 0.95, y: 20 },
      transition: { type: "spring", damping: 25, stiffness: 300 },
      className: "bg-white rounded-xl shadow-2xl w-full max-w-md flex flex-col max-h-full overflow-hidden
relative z-10 pointer-events-auto",
      onClick: (e) => e.stopPropagation()
    },
    /* @__PURE__ */ React.createElement("div", { className: "overflow-y-auto flex-1 p-0", ref: printRef }, /*
@__PURE__ */ React.createElement("div", { className: "bg-[#1A3A5C] text-white p-6 text-center" }, /*
@__PURE__ */ React.createElement("p", { className: "text-[10px] font-semibold tracking-widest text-blue-200
uppercase mb-2" }, "Official Document"), /* @__PURE__ */ React.createElement("h2", { className: "text-xl
font-serif font-bold mb-1" }, "Cash Turn-Over Slip"), /* @__PURE__ */ React.createElement("p", { className:
"text-xs text-blue-100 font-medium" }, storeName)), /* @__PURE__ */ React.createElement("div", { className:
"p-6 bg-[#FaFaFa]" }, /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-y-4 mb-
6" }, /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className:
"text-[10px] text-gray-400 font-bold uppercase tracking-wider mb-0.5" }, "Slip No."), /* @__PURE__ */
React.createElement("p", { className: "text-sm font-semibold text-gray-800" }, slipNo)), /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-[10px] text-
gray-400 font-bold uppercase tracking-wider mb-0.5" }, "Date"), /* @__PURE__ */ React.createElement("p",
{ className: "text-sm font-semibold text-gray-800" }, new Date(summary.shiftDate ||
Date.now()).toLocaleDateString("en-US", { month: "long", day: "numeric", year: "numeric" }))), /* @__PURE__
*/ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-[10px] text-
gray-400 font-bold uppercase tracking-wider mb-0.5" }, "Time"), /* @__PURE__ */ React.createElement("p",
{ className: "text-sm font-semibold text-gray-800" }, (/* @__PURE__ */ new Date()).toLocaleTimeString("en-US",
{ hour: "2-digit", minute: "2-digit" }))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("p", { className: "text-[10px] text-gray-400 font-bold uppercase tracking-wider mb-0.5" },
"Cashier"), /* @__PURE__ */ React.createElement("p", { className: "text-sm font-semibold text-gray-800" },
employeeName)), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p",
{ className: "text-[10px] text-gray-400 font-bold uppercase tracking-wider mb-0.5" }, "Float"), /* @__PURE__
*/ React.createElement("p", { className: "text-sm font-semibold text-gray-800" },
formatCurrency(openingFloat))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("p", { className: "text-[10px] text-gray-400 font-bold uppercase tracking-wider mb-0.5" },
"Type"), /* @__PURE__ */ React.createElement("p", { className: "text-sm font-semibold text-gray-800" },
"Regular"))), /* @__PURE__ */ React.createElement("div", { className: "border-t border-dashed border-gray-300
mb-6" }), /* @__PURE__ */ React.createElement("div", { className: "bg-blue-50 border border-blue-100 rounded-
lg p-4 mb-4" }, /* @__PURE__ */ React.createElement("p", { className: "text-[10px] font-bold text-blue-800
uppercase tracking-widest mb-3" }, "Z-Reading (Machine Reading)"), /* @__PURE__ */ React.createElement("div",
{ className: "flex justify-between items-center mb-1 text-sm text-gray-700" }, /* @__PURE__ */
React.createElement("span", null, "Gross Sales"), /* @__PURE__ */ React.createElement("span", { className:
"font-semibold text-gray-800" }, formatCurrency(summary.grossSales))), /* @__PURE__ */
React.createElement("div", { className: "flex justify-between items-center mb-3 text-sm text-gray-600" }, /*
@__PURE__ */ React.createElement("span", null, "Less: Returns"), /* @__PURE__ */ React.createElement("span",
{ className: "font-semibold text-red-500" }, "(", formatAbs(summary.returns), ")")), /* @__PURE__ */
React.createElement("div", { className: "border-t border-blue-200 my-3" }), /* @__PURE__ */
React.createElement("div", { className: "flex justify-between items-center text-sm font-bold text-blue-900" },
/* @__PURE__ */ React.createElement("span", null, "Net Z-Reading"), /* @__PURE__ */
React.createElement("span", null, formatCurrency(summary.netSales)))), /* @__PURE__ */
React.createElement("div", { className: "bg-white border border-gray-200 rounded-lg p-4 mb-4 shadow-sm" }, /*
@__PURE__ */ React.createElement("p", { className: "text-[10px] font-bold text-gray-500 uppercase tracking-
widest mb-3" }, "Cash Count Breakdown"), nonZeroDenominations.length > 0 ? /* @__PURE__ */
React.createElement("div", { className: "flex flex-wrap gap-x-4 gap-y-2 text-xs" },
nonZeroDenominations.map((d) => /* @__PURE__ */ React.createElement("div", { key: d.key, className: "flex
gap-2 w-[calc(50%-8px)]" }, /* @__PURE__ */ React.createElement("span", { className: "text-gray-400 w-12" },
d.label, "\xD7", d.qty), /* @__PURE__ */ React.createElement("span", { className: "font-bold text-gray-700" },
formatCurrency(d.amount))))) : totalCashOnHand > 0 ? /* @__PURE__ */ React.createElement("p", { className:
"text-xs text-gray-600" }, "Quick total: ", /* @__PURE__ */ React.createElement("span", { className: "font-
semibold" }, formatCurrency(totalCashOnHand)), " ", "(no per-denomination breakdown).") : /* @__PURE__ */
React.createElement("p", { className: "text-xs text-gray-400 italic" }, "No cash counted.")), /* @__PURE__ */
React.createElement("div", { className: "bg-white border border-gray-200 rounded-lg p-4 mb-6 shadow-sm" }, /*
@__PURE__ */ React.createElement("p", { className: "text-[10px] font-bold text-gray-500 uppercase tracking-
widest mb-3" }, "Remittance Computation"), /* @__PURE__ */ React.createElement("div", { className: "flex
justify-between items-center mb-1 text-sm text-gray-700" }, /* @__PURE__ */ React.createElement("span", null,
"Total Cash in Drawer"), /* @__PURE__ */ React.createElement("span", { className: "font-semibold text-gray-
800" }, formatCurrency(totalCashOnHand))), /* @__PURE__ */ React.createElement("div", { className: "flex
justify-between items-center mb-3 text-sm text-gray-600" }, /* @__PURE__ */ React.createElement("span", null,
"Less: Opening Float"), /* @__PURE__ */ React.createElement("span", { className: "font-semibold text-red-
500" }, "(", formatAbs(openingFloat), ")")), /* @__PURE__ */ React.createElement("div", { className: "border-
t-2 border-gray-800 my-3" }), /* @__PURE__ */ React.createElement("div", { className: "flex justify-between
items-center font-bold text-gray-900 mb-4" }, /* @__PURE__ */ React.createElement("span", { className:
"uppercase tracking-wide text-sm" }, "Cash to Remit"), /* @__PURE__ */ React.createElement("span",
```

```
{ className: "text-lg" }, formatCurrency(cashToRemit))), /* @__PURE__ */ React.createElement("div",
{ className: `flex justify-between items-center p-3 rounded-md text-sm font-bold ${varianceColorClass}` }, /*
@__PURE__ */ React.createElement("span", { className: "uppercase tracking-widest text-[10px]" }, "Variance"),
/* @__PURE__ */ React.createElement("span", null, variance < 0 ? "-" : variance > 0 ? "+" : "",
formatAbs(variance), " \u2014 ", varianceLabel))), /* @__PURE__ */ React.createElement("div", { className:
"border-t border-dashed border-gray-300 mb-8" }), /* @__PURE__ */ React.createElement("div", { className:
"flex justify-between gap-6 mb-6" }, /* @__PURE__ */ React.createElement("div", { className: "flex-1 text-
center" }, /* @__PURE__ */ React.createElement("p", { className: "font-bold text-gray-800 text-sm mb-1 line-
clamp-1" }, employeeName), /* @__PURE__ */ React.createElement("div", { className: "border-t border-gray-400
mb-1 w-full" }), /* @__PURE__ */ React.createElement("p", { className: "text-[8px] text-gray-400 uppercase
tracking-widest" }, "Prepared & Counted By")), /* @__PURE__ */ React.createElement("div", { className: "flex-
1 text-center" }, /* @__PURE__ */ React.createElement("p", { className: "font-bold text-gray-800 text-sm mb-1
line-clamp-1" }, receivedBy || "Pending"), /* @__PURE__ */ React.createElement("div", { className: "border-t
border-gray-400 mb-1 w-full" }), /* @__PURE__ */ React.createElement("p", { className: "text-[8px] text-gray-
400 uppercase tracking-widest" }, "Received & Verified By"))), /* @__PURE__ */ React.createElement("p",
{ className: "text-center text-[10px] text-gray-400" }, "Official Cash Turn-Over Slip \u2014 Keep for your
records"))),
    /* @__PURE__ */ React.createElement("div", { className: "p-4 bg-gray-50 border-t border-gray-200 flex
gap-3" }, /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: handlePrint,
        className: "flex items-center justify-center gap-2 px-5 py-3 border border-gray-300 text-gray-700
font-bold rounded-lg hover:bg-white transition-colors cursor-pointer"
      },
      /* @__PURE__ */ React.createElement(FaPrint, { className: "text-gray-500" }),
      "Print"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: "flex-1 flex items-center justify-center gap-2 px-5 py-3 bg-[#1A3A5C] text-white font-bold
rounded-lg hover:bg-[#11263d] transition-colors shadow-md cursor-pointer"
      },
      /* @__PURE__ */ React.createElement(FaCheck, null),
      "Done"
    ))
  )));
};
export default CashTurnOverSlipModal;
```

FILE: pos/frontend/src/components/transaction/PrintReceiptModal.jsx
==========

```
import { useState, useRef } from "react";
import { FaTimes, FaPrint } from "react-icons/fa";
import { sendReceiptToPrinter } from "../../utils/printBridge";
import {
  lineSubtotalFromItems,
  resolveTransactionDiscount,
  formatReceiptVariantSizeLine
} from "../../utils/transactionDisplay";
const formatCurrency = (value = 0) => new Intl.NumberFormat("en-PH", {
  style: "currency",
  currency: "PHP"
}).format(value);
const formatDateTime = (dateString) => {
  if (!dateString) return "-";
  return new Date(dateString).toLocaleString("en-US", {
    year: "numeric",
    month: "short",
    day: "numeric",
    hour: "2-digit",
    minute: "2-digit",
    hour12: true
  });
};
const formatTime = (dateString) => {
  if (!dateString) return "12:00PM";
  return new Date(dateString).toLocaleString("en-US", {
    hour: "2-digit",
    minute: "2-digit",
    hour12: true
  }).replace(" ", "");
};
const PrintReceiptModal = ({ isOpen, onClose, transaction }) => {
```

```
  const printRef = useRef(null);
  const [isPrinting, setIsPrinting] = useState(false);
  const [printError, setPrintError] = useState(null);
  if (!isOpen || !transaction) return null;
  const lineSub = lineSubtotalFromItems(transaction) || transaction.totalAmount || 0;
  const hasReturnActivity = (transaction.returnTransactions?.length || 0) > 0 || transaction.status ===
"Returned" || transaction.status === "Partially Returned";
  const discountAmount = resolveTransactionDiscount(transaction, lineSub, {
    skipInference: hasReturnActivity
  });
  const subtotal = lineSub;
  const handlePrint = async () => {
    setIsPrinting(true);
    setPrintError(null);
    try {
      const receipt = {
        receiptNo: transaction.receiptNo || "000000",
        storeName: "Create Your Style",
        location: "Pasonanca, Zamboanga City",
        contactNumber: "+631112224444",
        time: formatTime(transaction.checkedOutAt || transaction.createdAt),
        referenceNo: transaction.referenceNo || transaction._id?.substring(0, 12) || "-",
        items: transaction.items?.map((item) => ({
          name: item.itemName || item.name || "Item",
          itemName: item.itemName || item.name || "Item",
          qty: item.quantity || 1,
          quantity: item.quantity || 1,
          price: item.price || item.itemPrice || 0,
          itemPrice: item.price || item.itemPrice || 0,
          selectedSize: item.selectedSize || item.size || "",
          size: item.selectedSize || item.size || "",
          selectedVariation: item.selectedVariation || item.variant || "",
          variant: item.selectedVariation || item.variant || ""
        })) || [],
        subtotal,
        discount: discountAmount,
        total: transaction.totalAmount || 0,
        cash: transaction.amountReceived || 0,
        change: transaction.changeGiven || 0,
        paymentMethod: (transaction.paymentMethod || "cash").toUpperCase(),
        cashier: transaction.performedByName || "N/A"
      };
      await sendReceiptToPrinter(receipt);
      setIsPrinting(false);
      onClose();
    } catch (error) {
      console.error("Print error:", error);
      setIsPrinting(false);
      setPrintError(error.message || "Failed to print receipt. Please allow pop-ups for this site.");
    }
  };
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 flex items-center justify-center z-[10000] backdrop-blur-sm bg-black/50",
      onClick: onClose
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "bg-white rounded-2xl w-full max-w-md shadow-2xl overflow-hidden flex flex-col",
        onClick: (e) => e.stopPropagation()
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "h-2",
          style: {
            background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)"
          }
        }
      ),
      /* @__PURE__ */ React.createElement("div", { className: "p-8 overflow-y-auto flex-1" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-between items-start mb-6" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("h2", { className: "text-2xl font-bold
text-gray-800 mb-1" }, "Print Receipt")), /* @__PURE__ */ React.createElement(
        "button",
```

```
        {
          onClick: onClose,
          className: "text-gray-400 hover:text-gray-600 transition"
        },
        /* @__PURE__ */ React.createElement(FaTimes, { className: "w-6 h-6" })
      )), printError && /* @__PURE__ */ React.createElement("div", { className: "mb-4 p-3 bg-amber-50 border
border-amber-200 rounded-lg text-sm text-amber-700" }, printError, /* @__PURE__ */ React.createElement("p",
{ className: "text-xs mt-1" }, "Please allow pop-ups and try again.")), /* @__PURE__ */
React.createElement("div", { ref: printRef, className: "bg-white p-6 border-2 border-dashed border-gray-300
rounded-lg font-mono text-sm" }, /* @__PURE__ */ React.createElement("div", { className: "text-center mb-4" },
/* @__PURE__ */ React.createElement("h3", { className: "text-lg font-bold text-gray-800" }, "Create Your
Style"), /* @__PURE__ */ React.createElement("p", { className: "text-xs text-gray-600" }, "Pasonanca,
Zamboanga City")), /* @__PURE__ */ React.createElement("div", { className: "border-t border-b border-gray-300
py-3 my-3" }, /* @__PURE__ */ React.createElement("div", { className: "text-center" }, /* @__PURE__ */
React.createElement("p", { className: "text-xs text-gray-500 uppercase tracking-wider" }, "Receipt"), /*
@__PURE__ */ React.createElement("p", { className: "text-base font-bold text-gray-800 mt-1" }, "#",
transaction.receiptNo || "000000"))), /* @__PURE__ */ React.createElement("div", { className: "space-y-1 mb-4
text-xs" }, /* @__PURE__ */ React.createElement("div", { className: "flex justify-between" }, /* @__PURE__ */
React.createElement("span", { className: "text-gray-600" }, "Date:"), /* @__PURE__ */
React.createElement("span", { className: "text-gray-800" }, formatDateTime(transaction.checkedOutAt ||
transaction.createdAt))), /* @__PURE__ */ React.createElement("div", { className: "flex justify-between" },
/* @__PURE__ */ React.createElement("span", { className: "text-gray-600" }, "Cashier:"), /* @__PURE__ */
React.createElement("span", { className: "text-gray-800" }, transaction.performedByName || "N/A")), /*
@__PURE__ */ React.createElement("div", { className: "flex justify-between" }, /* @__PURE__ */
React.createElement("span", { className: "text-gray-600" }, "Payment:"), /* @__PURE__ */
React.createElement("span", { className: "text-gray-800 capitalize" }, transaction.paymentMethod || "N/A"))),
/* @__PURE__ */ React.createElement("div", { className: "border-t border-gray-300 py-3 my-3" }, /* @__PURE__
*/ React.createElement("div", { className: "space-y-2" }, transaction.items && transaction.items.length > 0 ?
transaction.items.map((item, idx) => {
        const variantSizeLine = formatReceiptVariantSizeLine(item);
        return /* @__PURE__ */ React.createElement("div", { key: idx, className: "text-xs" }, /* @__PURE__ */
React.createElement("p", { className: "font-medium text-gray-800" }, item.itemName), variantSizeLine && /*
@__PURE__ */ React.createElement("p", { className: "text-gray-500" }, variantSizeLine), /* @__PURE__ */
React.createElement("p", { className: "text-gray-500" }, item.quantity, " x ", formatCurrency(item.price ||
item.itemPrice)));
      }) : /* @__PURE__ */ React.createElement("p", { className: "text-center text-gray-500" }, "No items"))),
/* @__PURE__ */ React.createElement("div", { className: "border-t border-gray-300 pt-3 space-y-1 text-xs" },
/* @__PURE__ */ React.createElement("div", { className: "flex justify-between" }, /* @__PURE__ */
React.createElement("span", { className: "text-gray-600" }, "Subtotal:"), /* @__PURE__ */
React.createElement("span", { class="text-gray-800" }, formatCurrency(subtotal))), /* @__PURE__ */
React.createElement("div", { className: "flex justify-between" }, /* @__PURE__ */ React.createElement("span",
{ className: "text-gray-600" }, "Discount:"), /* @__PURE__ */ React.createElement("span", { className: "text-
gray-800" }, formatCurrency(discountAmount))), /* @__PURE__ */ React.createElement("div", { className: "flex
justify-between font-bold text-sm pt-1 border-t border-gray-200" }, /* @__PURE__ */
React.createElement("span", null, "Total:"), /* @__PURE__ */ React.createElement("span", null,
formatCurrency(transaction.totalAmount))), transaction.amountReceived > 0 && /* @__PURE__ */
React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("div", { className: "flex
justify-between" }, /* @__PURE__ */ React.createElement("span", { class="text-gray-600" }, "Amount
Received:"), /* @__PURE__ */ React.createElement("span", { class="text-gray-800" },
formatCurrency(transaction.amountReceived))), /* @__PURE__ */ React.createElement("div", { class="flex
justify-between" }, /* @__PURE__ */ React.createElement("span", { class="text-gray-600" }, "Change:"),
/* @__PURE__ */ React.createElement("span", { class="text-gray-800" },
formatCurrency(transaction.changeGiven))))), /* @__PURE__ */ React.createElement("div", { class="border-
t border-gray-300 mt-4 pt-4 text-center" }, /* @__PURE__ */ React.createElement("p", { class="text-xs
text-gray-500" }, "Thank you for your purchase!"), /* @__PURE__ */ React.createElement("p", { class:
"text-xs text-gray-400 mt-1" }, "This is not an official receipt")))),
      /* @__PURE__ */ React.createElement("div", { class="border-t border-gray-200 p-6 bg-gray-50 flex
gap-3" }, /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onClose,
          disabled: isPrinting,
          class="flex-1 px-6 py-3 rounded-xl bg-gray-200 hover:bg-gray-300 text-gray-700 font-semibold
transition-all disabled:opacity-50"
        },
        "Cancel"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: handlePrint,
          disabled: isPrinting,
          class="flex-1 px-6 py-3 rounded-xl bg-[#AD7F65] hover:bg-[#76462B] text-white font-semibold
transition-all flex items-center justify-center gap-2 disabled:opacity-50"
        },
        isPrinting ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement("div", { class="w-4 h-4 border-2 border-white border-t-transparent rounded-full
animate-spin" }), "Printing...") : /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
```

```
React.createElement(FaPrint, { className: "w-4 h-4" }), "Print")
      ))
    )
  );
};
export default PrintReceiptModal;
```

FILE: pos/frontend/src/components/transaction/RemittanceModal.jsx
==========

```
import React, { useState, useEffect, useCallback } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { FaClipboardList, FaCoins, FaFileAlt, FaTimes, FaCheck, FaChevronDown } from "react-icons/fa";
import { API_ENDPOINTS } from "../../config/api";
import CashTurnOverSlipModal from "./CashTurnOverSlipModal";
const DENOMINATIONS = [
  { key: "p1000", label: "\u20B11000", value: 1e3, bg: "#DBEAFE", text: "#1E40AF" },
  { key: "p500", label: "\u20B1500", value: 500, bg: "#FEF9C3", text: "#CA8A04" },
  { key: "p200", label: "\u20B1200", value: 200, bg: "#DCFCE7", text: "#166534" },
  { key: "p100", label: "\u20B1100", value: 100, bg: "#EDE9FE", text: "#6B21A8" },
  { key: "p50", label: "\u20B150", value: 50, bg: "#FCE7F3", text: "#BE185D" },
  { key: "p20", label: "\u20B120", value: 20, bg: "#FFEDD5", text: "#C2410C" },
  { key: "p10", label: "\u20B110", value: 10, bg: "#FAFAF9", text: "#171717" },
  { key: "p5", label: "\u20B15", value: 5, bg: "#FAFAF9", text: "#171717" },
  { key: "p1", label: "\u20B11", value: 1, bg: "#FAFAF9", text: "#171717" },
  { key: "c25", label: "25\xA2", value: 0.25, bg: "#F3F4F6", text: "#111827" },
  { key: "c10", label: "10\xA2", value: 0.1, bg: "#F3F4F6", text: "#111827" },
  { key: "c5", label: "5\xA2", value: 0.05, bg: "#F3F4F6", text: "#111827" }
];
const formatCurrency = (val) => {
  const abs = Math.abs(val).toLocaleString("en-PH", { minimumFractionDigits: 2, maximumFractionDigits: 2 });
  return val < 0 ? `-\u20B1${abs}` : `\u20B1${abs}`;
};
const formatAbs = (val) => `\u20B1${Math.abs(val).toLocaleString("en-PH", { minimumFractionDigits: 2,
maximumFractionDigits: 2 })}`;
const StepIndicator = ({ currentStep }) => {
  const steps = [
    { num: 1, label: "Summary" },
    { num: 2, label: "Cash Drawer" },
    { num: 3, label: "Remittance Report" }
  ];
  return /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-center gap-0 mb-
6" }, steps.map((step, i) => /* @__PURE__ */ React.createElement(React.Fragment, { key: step.num }, /*
@__PURE__ */ React.createElement("div", { className: "flex flex-col items-center" }, /* @__PURE__ */
React.createElement(
    "div",
    {
      className: `w-9 h-9 rounded-full flex items-center justify-center text-sm font-bold transition-all
${currentStep >= step.num ? "bg-[#22C55E] text-white shadow-md" : "bg-gray-200 text-gray-400"}`
    },
    currentStep > step.num ? /* @__PURE__ */ React.createElement(FaCheck, { className: "text-xs" }) :
step.num
  ), /* @__PURE__ */ React.createElement(
    "span",
    {
      className: `text-[10px] mt-1 font-medium ${currentStep >= step.num ? "text-[#22C55E]" : "text-gray-
400"}`
    },
    step.label
  )), i < steps.length - 1 && /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `w-16 h-0.5 mb-4 mx-1 rounded ${currentStep > step.num ? "bg-[#22C55E]" : "bg-gray-200"}`
    }
  ))));
};
const RemittanceModal = ({ isOpen, onClose, employeeId, employeeName }) => {
  const [step, setStep] = useState(1);
  const [loading, setLoading] = useState(false);
  const [submitting, setSubmitting] = useState(false);
  const [summary, setSummary] = useState({
    shiftDate: /* @__PURE__ */ new Date(),
    grossSales: 0,
    returns: 0,
    netSales: 0,
    noOfSales: 0,
```

```
    alreadyRemittedToday: false,
    remittedAmountToday: 0,
    remittedAtToday: null
  });
  const [denominations, setDenominations] = useState(
    DENOMINATIONS.reduce((acc, d) => ({ ...acc, [d.key]: 0 }), {})
  );
  const [remarks, setRemarks] = useState("");
  const [receivedBy, setReceivedBy] = useState("");
  const [submitSuccess, setSubmitSuccess] = useState(false);
  const [showSlip, setShowSlip] = useState(false);
  const [openingFloat, setOpeningFloat] = useState(2e3);
  const [quickTotalOpen, setQuickTotalOpen] = useState(true);
  const [manualTotalInput, setManualTotalInput] = useState("");
  const totalFromDenominations = React.useMemo(
    () => DENOMINATIONS.reduce(
      (sum, d) => sum + (denominations[d.key] || 0) * d.value,
      0
    ),
    [denominations]
  );
  const parsedManualTotal = React.useMemo(() => {
    const t = manualTotalInput.trim().replace(/,/g, "");
    if (t === "") return null;
    const n = parseFloat(t);
    return Number.isFinite(n) && n >= 0 ? n : null;
  }, [manualTotalInput]);
  const useManualTotal = parsedManualTotal !== null;
  const totalCashOnHand = useManualTotal ? parsedManualTotal : totalFromDenominations;
  const cashToRemit = totalCashOnHand - openingFloat;
  const variance = cashToRemit - summary.netSales;
  const fetchSummary = useCallback(async () => {
    if (!employeeId) return;
    setLoading(true);
    try {
      const res = await fetch(
        `${API_ENDPOINTS.remittanceSummary}?employeeId=${employeeId}`
      );
      const data = await res.json();
      if (data.success) {
        setSummary(data.data);
        setOpeningFloat(Number(data.data?.openingFloatTotal) || 0);
      }
    } catch (err) {
      console.error("Error fetching remittance summary:", err);
    } finally {
      setLoading(false);
    }
  }, [employeeId]);
  useEffect(() => {
    if (isOpen) {
      setStep(1);
      setDenominations(
        DENOMINATIONS.reduce((acc, d) => ({ ...acc, [d.key]: 0 }), {})
      );
      setRemarks("");
      setReceivedBy("");
      setSubmitSuccess(false);
      setShowSlip(false);
      setQuickTotalOpen(true);
      setManualTotalInput("");
      fetchSummary();
    }
  }, [isOpen, fetchSummary]);
  const handleDenominationChange = (key, value) => {
    const num = parseInt(value) || 0;
    setDenominations((prev) => ({ ...prev, [key]: Math.max(0, num) }));
  };
  const handleSubmit = async () => {
    if (summary.alreadyRemittedToday) {
      alert(`You already remitted today.
Amount remitted: ${formatCurrency(summary.remittedAmountToday || 0)}`);
      return;
    }
    setSubmitting(true);
    try {
      const res = await fetch(API_ENDPOINTS.remittances, {
```

```
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          employeeId,
          employeeName,
          shiftDate: summary.shiftDate,
          grossSales: summary.grossSales,
          returns: summary.returns,
          netSales: summary.netSales,
          noOfSales: summary.noOfSales,
          denominations,
          totalCashOnHand,
          openingFloat,
          cashToRemit,
          expectedCash: summary.netSales,
          variance,
          remarks,
          receivedBy
        })
      });
      const data = await res.json();
      if (data.success) {
        setSubmitSuccess(true);
        setTimeout(() => {
          setShowSlip(true);
        }, 1e3);
      } else {
        if (data.code === "ALREADY_REMITTED_TODAY") {
          const amount = data?.data?.amountRemitted;
          const amountText = typeof amount === "number" ? formatCurrency(amount) : null;
          alert(
            amountText ? `You already remitted today.
Amount remitted: ${amountText}` : data.message || "You already remitted today."
          );
          return;
        }
        alert("Failed to submit remittance: " + (data.message || "Unknown error"));
      }
    } catch (err) {
      console.error("Error submitting remittance:", err);
      alert("Failed to submit remittance. Please try again.");
    } finally {
      setSubmitting(false);
    }
  };
  if (!isOpen) return null;
  const varianceLabel = variance === 0 ? "BALANCED" : variance > 0 ? "OVER" : "SHORT";
  const varianceColor = variance === 0 ? "text-green-600" : variance > 0 ? "text-blue-600" : "text-red-600";
  return /* @__PURE__ */ React.createElement(AnimatePresence, null, isOpen && showSlip && /* @__PURE__ */
React.createElement(
    CashTurnOverSlipModal,
    {
      isOpen: true,
      onClose,
      summary,
      denominations,
      totalCashOnHand,
      openingFloat,
      cashToRemit,
      variance,
      employeeName,
      receivedBy
    }
  ), isOpen && !showSlip && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex
items-center justify-center z-[200] p-4 pointer-events-none" }, /* @__PURE__ */ React.createElement(
    motion.div,
    {
      initial: { opacity: 0 },
      animate: { opacity: 1 },
      exit: { opacity: 0 },
      className: "absolute inset-0 bg-black/50 backdrop-blur-sm pointer-events-auto",
      onClick: onClose
    }
  ), /* @__PURE__ */ React.createElement(
    motion.div,
    {
      initial: { opacity: 0, scale: 0.95, y: 20 },
```

```
      animate: { opacity: 1, scale: 1, y: 0 },
      exit: { opacity: 0, scale: 0.95, y: 20 },
      transition: { type: "spring", damping: 25, stiffness: 300 },
      className: "bg-white rounded-2xl shadow-2xl w-full max-w-lg max-h-[90vh] overflow-y-auto relative z-10
pointer-events-auto",
      onClick: (e) => e.stopPropagation()
    },
    /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between p-5 border-b
border-gray-100" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-2" }, step
=== 1 && /* @__PURE__ */ React.createElement(FaClipboardList, { className: "text-[#22C55E] text-lg" }), step
=== 2 && /* @__PURE__ */ React.createElement(FaCoins, { className: "text-[#22C55E] text-lg" }), step === 3 &&
/* @__PURE__ */ React.createElement(FaFileAlt, { className: "text-[#22C55E] text-lg" }), /* @__PURE__ */
React.createElement("h2", { className: "text-lg font-bold text-gray-800" }, step === 1 && "Cash Remittance",
step === 2 && "Count Your Cash Drawer", step === 3 && "Cash Remittance")), /* @__PURE__ */
React.createElement(
      "button",
      {
        onClick: onClose,
        className: "text-gray-400 hover:text-gray-600 transition-colors cursor-pointer"
      },
      /* @__PURE__ */ React.createElement(FaTimes, null)
    )),
    step === 2 && /* @__PURE__ */ React.createElement("p", { className: "text-xs text-gray-500 px-6 pt-2" },
"Count all cash in your drawer (including the float). Use", " ", /* @__PURE__ */ React.createElement("span",
{ className: "font-semibold text-gray-700" }, "Quick entry"), " below to type one total,", " ", /* @__PURE__
*/ React.createElement("span", { className: "font-semibold text-gray-700" }, "or"), " enter quantity per
denomination."),
    /* @__PURE__ */ React.createElement("div", { className: "p-6" }, /* @__PURE__ */
React.createElement(StepIndicator, { currentStep: step }), submitSuccess && /* @__PURE__ */
React.createElement(
      motion.div,
      {
        initial: { opacity: 0, scale: 0.8 },
        animate: { opacity: 1, scale: 1 },
        className: "flex flex-col items-center justify-center py-12"
      },
      /* @__PURE__ */ React.createElement("div", { className: "w-16 h-16 rounded-full bg-green-100 flex
items-center justify-center mb-4" }, /* @__PURE__ */ React.createElement(FaCheck, { className: "text-green-
600 text-2xl" })),
      /* @__PURE__ */ React.createElement("h3", { className: "text-lg font-bold text-gray-800 mb-1" },
"Remittance Submitted!"),
      /* @__PURE__ */ React.createElement("p", { className: "text-sm text-gray-500" }, "Your remittance has
been recorded successfully.")
    ), step === 1 && !submitSuccess && /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("h3", { className: "text-sm font-bold text-gray-800 mb-3" }, "Review Your Sales"),
loading ? /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-center py-8" },
/* @__PURE__ */ React.createElement("div", { className: "animate-spin rounded-full h-8 w-8 border-b-2 border-
[#22C55E]" })) : /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement("div", { className: "bg-gray-50 rounded-xl p-4 mb-4" }, /* @__PURE__ */
React.createElement("p", { className: "text-xs text-gray-500 mb-3 font-medium" }, "Summary \u2014 ", new
Date(summary.shiftDate).toLocaleDateString("en-US", {
      month: "long",
      day: "numeric",
      year: "numeric"
    })), /* @__PURE__ */ React.createElement("div", { className: "space-y-2" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-between text-sm" }, /* @__PURE__ */
React.createElement("span", { className: "text-gray-600" }, "Gross Sales"), /* @__PURE__ */
React.createElement("span", { className: "font-semibold text-gray-800" },
formatCurrency(summary.grossSales))), /* @__PURE__ */ React.createElement("div", { className: "flex justify-
between text-sm" }, /* @__PURE__ */ React.createElement("span", { className: "text-gray-600" }, "Less:
(Returns/Refunds)"), /* @__PURE__ */ React.createElement("span", { className: "font-semibold text-red-500" },
"(", formatAbs(summary.returns), ")")), /* @__PURE__ */ React.createElement("div", { className: "border-t
border-gray-200 my-2" }), /* @__PURE__ */ React.createElement("div", { className: "flex justify-between text-
sm" }, /* @__PURE__ */ React.createElement("span", { className: "text-gray-700 font-semibold" }, "Net Sales"),
/* @__PURE__ */ React.createElement("span", { className: "font-bold text-gray-800" },
formatCurrency(summary.netSales))), /* @__PURE__ */ React.createElement("div", { className: "flex justify-
between text-sm" }, /* @__PURE__ */ React.createElement("span", { className: "text-gray-600" }, "No. of
Sales"), /* @__PURE__ */ React.createElement("span", { className: "font-semibold text-gray-800" },
summary.noOfSales)))), summary.alreadyRemittedToday ? /* @__PURE__ */ React.createElement("div", { className:
"bg-black/60 backdrop-blur-sm border border-white/20 rounded-xl p-4 mb-3" }, /* @__PURE__ */
React.createElement("p", { className: "text-sm font-semibold text-red-300" }, "You already remitted today."),
/* @__PURE__ */ React.createElement("p", { className: "text-xs text-gray-200 mt-1" }, "Exact amount
remitted:"), /* @__PURE__ */ React.createElement("p", { className: "text-lg font-extrabold text-white mt-1" },
formatCurrency(summary.remittedAmountToday || 0)), summary.remittedAtToday && /* @__PURE__ */
React.createElement("p", { className: "text-[11px] text-gray-300 mt-2" }, "Submitted at ", new
Date(summary.remittedAtToday).toLocaleTimeString("en-US", { hour: "2-digit", minute: "2-digit" }))) : /*
@__PURE__ */ React.createElement("div", { className: "bg-blue-50 border border-blue-100 rounded-xl p-3 mb-
```

```
6" }, /* @__PURE__ */ React.createElement("p", { className: "text-xs text-blue-700" }, "\u{1F4A1} This is
your machine reading. You will now count the physical cash in your drawer and remit the difference after
deducting your opening float of", " ", formatCurrency(openingFloat), ".")), !summary.alreadyRemittedToday &&
/* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => setStep(2),
        className: "w-full bg-[#22C55E] hover:bg-[#16A34A] text-white font-semibold py-3 rounded-xl
transition-colors cursor-pointer"
      },
      "Proceed"
    ))), step === 2 && !submitSuccess && /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("div", { className: "mb-4 border border-gray-200 rounded-xl overflow-hidden" }, /*
@__PURE__ */ React.createElement(
      "button",
      {
        type: "button",
        onClick: () => setQuickTotalOpen((o) => !o),
        className: "w-full flex items-center justify-between gap-2 px-3 py-2.5 text-left text-sm font-
semibold text-gray-700 bg-gray-50 hover:bg-gray-100 transition-colors"
      },
      /* @__PURE__ */ React.createElement("span", { className: "flex items-center gap-2 flex-wrap" }, "Quick
entry \u2014 total cash counted", /* @__PURE__ */ React.createElement("span", { className: "text-[10px] font-
bold uppercase tracking-wide text-[#16A34A] bg-green-100 px-1.5 py-0.5 rounded" }, "Optional")),
      /* @__PURE__ */ React.createElement(
        FaChevronDown,
        {
          className: `text-gray-400 shrink-0 transition-transform ${quickTotalOpen ? "rotate-180" : ""}`
        }
      )
    ), quickTotalOpen && /* @__PURE__ */ React.createElement("div", { className: "px-3 pb-3 pt-1 border-t
border-gray-100 bg-white" }, /* @__PURE__ */ React.createElement("p", { className: "text-[11px] text-gray-500
mb-2" }, "Enter the full amount you counted. Leave empty to use denomination quantities below."), /*
@__PURE__ */ React.createElement("div", { className: "flex gap-2 items-center" }, /* @__PURE__ */
React.createElement("span", { className: "text-sm text-gray-600 shrink-0" }, "\u20B1"), /* @__PURE__ */
React.createElement(
      "input",
      {
        type: "text",
        inputMode: "decimal",
        value: manualTotalInput,
        onChange: (e) => setManualTotalInput(e.target.value),
        placeholder: "e.g. 6520.50",
        className: "flex-1 border border-gray-200 rounded-lg py-2 px-3 text-sm focus:outline-none
focus:border-[#22C55E] focus:ring-1 focus:ring-[#22C55E]/30"
      }
    ), manualTotalInput.trim() !== "" && /* @__PURE__ */ React.createElement(
      "button",
      {
        type: "button",
        onClick: () => setManualTotalInput(""),
        className: "text-xs font-medium text-[#22C55E] hover:underline shrink-0"
      },
      "Clear"
    )), useManualTotal && /* @__PURE__ */ React.createElement("p", { className: "text-[11px] text-amber-700
mt-2 bg-amber-50 rounded-lg px-2 py-1.5" }, "Using this total. Denomination fields are disabled \u2014 clear
quick entry to count by bill/coin.")), !quickTotalOpen && /* @__PURE__ */ React.createElement("p",
{ className: "text-[11px] text-[#16A34A] mb-2 -mt-1 px-0.5" }, "Tip: open ", /* @__PURE__ */
React.createElement("span", { className: "font-semibold" }, "Quick entry"), " above to type the total instead
of counting each bill."), /* @__PURE__ */ React.createElement("p", { className: "text-[11px] font-semibold
text-gray-500 mb-2 mt-1" }, useManualTotal ? "Denomination breakdown (disabled while using quick entry)" :
"Or count by denomination"), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-4 gap-2
mb-4" }, DENOMINATIONS.map((d) => /* @__PURE__ */ React.createElement("div", { key: d.key, className: "flex
flex-col items-center" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "w-full text-center text-xs font-bold py-1.5 rounded-lg mb-1 border border-black/5",
        style: { backgroundColor: d.bg, color: d.text }
      },
      d.label
    ), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "number",
        min: "0",
        disabled: useManualTotal,
        value: denominations[d.key] || "",
```

```
        onChange: (e) => handleDenominationChange(d.key, e.target.value),
        placeholder: "0",
        className: `w-full text-center border border-gray-200 rounded-lg py-1.5 text-sm focus:outline-none
focus:border-[#22C55E] focus:ring-1 focus:ring-[#22C55E]/30 ${useManualTotal ? "bg-gray-100 text-gray-500
cursor-not-allowed" : ""}`
      }
    ), /* @__PURE__ */ React.createElement("span", { className: "text-[10px] text-gray-400 mt-0.5" },
formatCurrency((denominations[d.key] || 0) * d.value))))), /* @__PURE__ */ React.createElement("div",
{ className: "bg-green-50 border border-green-100 rounded-xl p-3 mb-3" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-between items-center gap-2" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("span", { className: "text-sm font-
semibold text-green-700" }, "Total Counted"), useManualTotal && /* @__PURE__ */ React.createElement("span",
{ className: "ml-2 text-[10px] font-medium text-green-600 bg-green-100 px-1.5 py-0.5 rounded" }, "Quick
entry")), /* @__PURE__ */ React.createElement("span", { className: "text-lg font-bold text-green-700" },
formatCurrency(totalCashOnHand)))), /* @__PURE__ */ React.createElement("div", { className: "space-y-1.5 mb-
3" }, /* @__PURE__ */ React.createElement("div", { className: "flex justify-between text-sm" }, /* @__PURE__
*/ React.createElement("span", { className: "text-gray-600" }, "Total Cash on Hand"), /* @__PURE__ */
React.createElement("span", { className: "font-semibold" }, formatCurrency(totalCashOnHand))), /* @__PURE__
*/ React.createElement("div", { className: "flex justify-between text-sm" }, /* @__PURE__ */
React.createElement("span", { className: "text-gray-600" }, "Less: Opening Float"), /* @__PURE__ */
React.createElement("span", { className: "font-semibold text-red-500" }, "(", formatAbs(openingFloat), ")")),
/* @__PURE__ */ React.createElement("div", { className: "border-t border-gray-200 my-1" }), /* @__PURE__ */
React.createElement("div", { className: "flex justify-between text-sm" }, /* @__PURE__ */
React.createElement("span", { className: "text-gray-700 font-semibold" }, "Cash to Remit"), /* @__PURE__ */
React.createElement("span", { className: "text-xl font-bold text-gray-800" }, formatCurrency(cashToRemit))),
/* @__PURE__ */ React.createElement("div", { className: "flex justify-between text-sm" }, /* @__PURE__ */
React.createElement("span", { className: "text-gray-600" }, "Variance"), /* @__PURE__ */
React.createElement("span", { className: `font-semibold ${varianceColor}` }, variance > 0 ? "+" : "",
formatCurrency(variance), " \u2014 ", varianceLabel, " ", variance === 0 && "\u2713"))), /* @__PURE__ */
React.createElement("div", { className: "flex gap-3 mt-4" }, /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => setStep(1),
        className: "flex-1 border border-gray-200 text-gray-700 font-semibold py-3 rounded-xl hover:bg-gray-
50 transition-colors cursor-pointer"
      },
      "Back"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => setStep(3),
        className: "flex-1 bg-[#22C55E] hover:bg-[#16A34A] text-white font-semibold py-3 rounded-xl
transition-colors cursor-pointer"
      },
      "Review & Confirm"
    ))), step === 3 && !submitSuccess && /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("h3", { className: "text-sm font-bold text-gray-800 mb-3" }, "Confirm and Submit"), /*
@__PURE__ */ React.createElement("div", { className: "bg-gray-50 rounded-xl p-4 mb-4" }, /* @__PURE__ */
React.createElement("p", { className: "text-xs text-gray-500 font-medium mb-3" }, "Remittance Summary"), /*
@__PURE__ */ React.createElement("div", { className: "space-y-2" }, /* @__PURE__ */ React.createElement("div",
{ className: "flex justify-between text-sm" }, /* @__PURE__ */ React.createElement("span", { className:
"text-gray-600" }, "Net Sales"), /* @__PURE__ */ React.createElement("span", { className: "font-semibold" },
formatCurrency(summary.netSales))), /* @__PURE__ */ React.createElement("div", { className: "flex justify-
between text-sm" }, /* @__PURE__ */ React.createElement("span", { className: "text-gray-600" }, "Total Cash
on Hand"), /* @__PURE__ */ React.createElement("span", { className: "font-semibold" },
formatCurrency(totalCashOnHand))), /* @__PURE__ */ React.createElement("div", { className: "flex justify-
between text-sm" }, /* @__PURE__ */ React.createElement("span", { className: "text-gray-600" }, "Less Opening
Float"), /* @__PURE__ */ React.createElement("span", { className: "font-semibold text-red-500" }, "(",
formatAbs(openingFloat), ")")), /* @__PURE__ */ React.createElement("div", { className: "border-t border-
gray-200 my-2" }), /* @__PURE__ */ React.createElement("div", { className: "flex justify-between items-
center" }, /* @__PURE__ */ React.createElement("span", { className: "text-sm font-bold text-gray-700" },
"Cash to Remit"), /* @__PURE__ */ React.createElement("span", { className: "text-xl font-bold text-gray-
800" }, formatCurrency(cashToRemit))), /* @__PURE__ */ React.createElement("div", { className: "flex justify-
between text-sm" }, /* @__PURE__ */ React.createElement("span", { className: "text-gray-600" }, "Variance"),
/* @__PURE__ */ React.createElement("span", { className: `font-semibold ${varianceColor}` }, variance > 0 ?
"+" : "", formatCurrency(variance), " \u2014 ", varianceLabel, " ", variance === 0 && "\u2713")))), /*
@__PURE__ */ React.createElement("div", { className: "mb-3" }, /* @__PURE__ */ React.createElement("label",
{ className: "block text-sm font-semibold text-gray-700 mb-1" }, "Remarks / Explanation for Variance"), /*
@__PURE__ */ React.createElement(
      "textarea",
      {
        value: remarks,
        onChange: (e) => setRemarks(e.target.value),
        placeholder: "eg. Reason for shortage or overage...",
        rows: 3,
        className: "w-full border border-gray-200 rounded-xl p-3 text-sm focus:outline-none focus:border-
[#22C55E] focus:ring-1 focus:ring-[#22C55E]/30 resize-none"
```

```
      }
    )), /* @__PURE__ */ React.createElement("div", { className: "mb-6" }, /* @__PURE__ */
React.createElement("label", { className: "block text-sm font-semibold text-gray-700 mb-1" }, "Received by
(Owner / Manager)"), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "text",
        value: receivedBy,
        onChange: (e) => setReceivedBy(e.target.value),
        placeholder: "Full Name",
        className: "w-full border border-gray-200 rounded-xl p-3 text-sm focus:outline-none focus:border-
[#22C55E] focus:ring-1 focus:ring-[#22C55E]/30"
      }
    )), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3" }, /* @__PURE__ */
React.createElement(
      "button",
      {
        onClick: () => setStep(2),
        className: "flex-1 border border-gray-200 text-gray-700 font-semibold py-3 rounded-xl hover:bg-gray-
50 transition-colors cursor-pointer"
      },
      "Back"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: handleSubmit,
        disabled: submitting,
        className: "flex-1 bg-[#22C55E] hover:bg-[#16A34A] text-white font-semibold py-3 rounded-xl
transition-colors disabled:opacity-50 cursor-pointer"
      },
      submitting ? "Submitting..." : "Confirm"
    ))))
  )));
};
export default RemittanceModal;
```

FILE: pos/frontend/src/components/transaction/ReturnItemsModal.jsx
==========

```
import React, { useState, useEffect, useRef } from "react";
import { FaTimes, FaExclamationTriangle, FaMinus, FaPlus, FaLock, FaEye, FaEyeSlash } from "react-icons/fa";
import {
  cleanItemNameForDisplay,
  formatItemVariantSizeLabel
} from "../../utils/transactionDisplay";
const formatDate = (dateString) => {
  if (!dateString) return "N/A";
  const date = new Date(dateString);
  const options = { year: "numeric", month: "long", day: "numeric" };
  const timeOptions = { hour: "2-digit", minute: "2-digit", hour12: true };
  return `${date.toLocaleDateString("en-US", options)} at ${date.toLocaleTimeString("en-US", timeOptions)}`;
};
const returnReasons = [
  { value: "", label: "e.g Damaged, Expired, Sold, etc" },
  { value: "Damaged", label: "Damaged" },
  { value: "Defective", label: "Defective" },
  { value: "Wrong Item", label: "Wrong Item" },
  { value: "Wrong Size", label: "Wrong Size" },
  { value: "Customer Changed Mind", label: "Customer Changed Mind" },
  { value: "Expired", label: "Expired" },
  { value: "Other", label: "Other" }
];
const ReturnItemsModal = ({ isOpen, onClose, transaction, onConfirm }) => {
  const [selectedItems, setSelectedItems] = useState({});
  const [returnQuantities, setReturnQuantities] = useState({});
  const [globalReason, setGlobalReason] = useState("");
  const [otherReason, setOtherReason] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const [showPinModal, setShowPinModal] = useState(false);
  const [pin, setPin] = useState(["", "", "", "", "", ""]);
  const [showPin, setShowPin] = useState(false);
  const [pinError, setPinError] = useState("");
  const [verifyingPin, setVerifyingPin] = useState(false);
  const [approverName, setApproverName] = useState("");
  const pinInputRefs = useRef([]);
```

```
useEffect(() => {
  if (isOpen && transaction) {
    const initialSelected = {};
    const initialQuantities = {};
    if (transaction.items) {
      transaction.items.forEach((item, idx) => {
        initialSelected[idx] = false;
        initialQuantities[idx] = 1;
      });
    }
    setSelectedItems(initialSelected);
    setReturnQuantities(initialQuantities);
    setGlobalReason("");
    setOtherReason("");
    setError("");
    setShowPinModal(false);
    setPin(["", "", "", "", "", ""]);
    setShowPin(false);
    setPinError("");
    setApproverName("");
  }
}, [isOpen, transaction]);
const handlePinChange = (index, value) => {
  if (value.length > 1) return;
  if (value && !/^\d$/.test(value)) return;
  const newPin = [...pin];
  newPin[index] = value;
  setPin(newPin);
  setPinError("");
  if (value && index < 5) {
    pinInputRefs.current[index + 1]?.focus();
  }
};
const handlePinKeyDown = (index, e) => {
  if (e.key === "Backspace" && !pin[index] && index > 0) {
    pinInputRefs.current[index - 1]?.focus();
  }
};
const verifyPin = async () => {
  const enteredPin = pin.join("");
  if (enteredPin.length !== 6) {
    setPinError("Please enter a 6-digit PIN");
    return;
  }
  setVerifyingPin(true);
  setPinError("");
  try {
    const response = await fetch("http://localhost:5000/api/employees/verify-pin", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ pin: enteredPin })
    });
    const result = await response.json();
    if (result.success && result.data) {
      const employee = result.data;
      const role = employee.role?.toLowerCase();
      if (role === "owner" || role === "manager") {
        setApproverName(`${employee.firstName} ${employee.lastName}`);
        setShowPinModal(false);
        await processReturn();
      } else {
        setPinError("Only Owner or Manager can approve returns");
        setPin(["", "", "", "", "", ""]);
        pinInputRefs.current[0]?.focus();
      }
    } else {
      setPinError(result.message || "Invalid PIN. Please try again.");
      setPin(["", "", "", "", "", ""]);
      pinInputRefs.current[0]?.focus();
    }
  } catch (err) {
    console.error("Error verifying PIN:", err);
    setPinError("Error verifying PIN. Please try again.");
  } finally {
    setVerifyingPin(false);
  }
};
```

```
const processReturn = async () => {
  setLoading(true);
  try {
    const finalReason = globalReason === "Other" ? `Other: ${otherReason}` : globalReason;
    const itemsToReturn = transaction.items.map((item, idx) => {
      if (selectedItems[idx]) {
        return {
          productId: item.productId || item._id,
          itemName: item.itemName,
          sku: item.sku,
          variant: item.variant || item.selectedVariation || "",
          selectedSize: item.selectedSize || item.size || "",
          quantity: (returnQuantities[idx] === "" ? 1 : returnQuantities[idx]) || 1,
          price: item.price || item.itemPrice,
          reason: finalReason,
          originalIndex: idx
        };
      }
      return null;
    }).filter(Boolean);
    await onConfirm(itemsToReturn, transaction);
    onClose();
  } catch (err) {
    console.error("Error processing return:", err);
    setError("Failed to process return. Please try again.");
  } finally {
    setLoading(false);
  }
};
const handleItemToggle = (index) => {
  setSelectedItems((prev) => ({
    ...prev,
    [index]: !prev[index]
  }));
};
const handleQuantityChange = (index, delta) => {
  const item = transaction.items[index];
  const maxQty = item.quantity || 1;
  setReturnQuantities((prev) => {
    const current = prev[index] === "" ? 1 : prev[index] || 1;
    const newQty = Math.max(1, Math.min(Number(current) + delta, maxQty));
    return { ...prev, [index]: newQty };
  });
};
const clampReturnQty = (index, maxQty, raw) => {
  if (raw === "") {
    setReturnQuantities((prev) => ({ ...prev, [index]: "" }));
    return;
  }
  const n = parseInt(raw, 10);
  if (Number.isNaN(n)) return;
  setReturnQuantities((prev) => ({
    ...prev,
    [index]: Math.max(1, Math.min(n, maxQty))
  }));
};
const blurReturnQty = (index, maxQty) => {
  setReturnQuantities((prev) => {
    const v = prev[index];
    if (v === "" || v === void 0 || v === null) {
      return { ...prev, [index]: 1 };
    }
    const n = Math.max(1, Math.min(Number(v) || 1, maxQty));
    return { ...prev, [index]: n };
  });
};
const handleSubmit = async () => {
  const selectedCount = Object.values(selectedItems).filter(Boolean).length;
  if (selectedCount === 0) {
    setError("Please select at least one item to return");
    return;
  }
  if (!globalReason) {
    setError("Please select a reason for return");
    return;
  }
  if (globalReason === "Other" && !otherReason.trim()) {
```

```
      setError("Please specify the reason for return");
      return;
    }
    setError("");
    setShowPinModal(true);
    setPin(["", "", "", "", "", ""]);
    setPinError("");
    setTimeout(() => {
      pinInputRefs.current[0]?.focus();
    }, 100);
  };
  if (!isOpen || !transaction) return null;
  const transactionId = transaction.transactionNumber ? `TRX-
${String(transaction.transactionNumber).padStart(3, "0")}` : transaction.referenceNo ||
transaction._id?.substring(0, 8);
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "fixed inset-0 flex items-center justify-center z-[10000] bg-black/50",
      onClick: onClose
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "bg-white rounded-2xl w-full max-w-3xl max-h-[90vh] shadow-2xl overflow-hidden flex flex-
col relative",
        onClick: (e) => e.stopPropagation()
      },
      (loading || verifyingPin) && /* @__PURE__ */ React.createElement("div", { className: "absolute inset-0
z-50 bg-black/40 backdrop-blur-[1px] flex items-center justify-center" }, /* @__PURE__ */
React.createElement("div", { className: "bg-white rounded-2xl px-8 py-6 shadow-2xl flex flex-col items-
center" }, /* @__PURE__ */ React.createElement("div", { className: "inline-block animate-spin rounded-full h-
10 w-10 border-b-2 border-orange-500 mb-3" }), /* @__PURE__ */ React.createElement("p", { className: "text-
gray-800 font-semibold" }, "Processing return..."), /* @__PURE__ */ React.createElement("p", { className:
"text-gray-500 text-sm mt-1" }, "Please wait. This will close automatically."))),
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "px-6 py-5 flex justify-between items-center",
          style: {
            background: "linear-gradient(135deg, #FB923C 0%, #F97316 50%, #EA580C 100%)"
          }
        },
        /* @__PURE__ */ React.createElement("h2", { className: "text-2xl font-bold text-white" }, "Process
Return Request"),
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: onClose,
            className: "text-white hover:text-gray-200 transition"
          },
          /* @__PURE__ */ React.createElement(FaTimes, { className: "w-6 h-6" })
        )
      ),
      /* @__PURE__ */ React.createElement("div", { className: "p-6 overflow-y-auto flex-1" }, /* @__PURE__ */
React.createElement("div", { className: "mb-6" }, /* @__PURE__ */ React.createElement("h3", { className:
"text-2xl font-bold text-gray-800 mb-1" }, "Transaction Id: ", transactionId), /* @__PURE__ */
React.createElement("p", { className: "text-sm text-gray-500" }, "Date ", formatDate(transaction.checkedOutAt
|| transaction.createdAt)), /* @__PURE__ */ React.createElement("p", { className: "text-sm text-gray-500" },
"Performed By: ", transaction.performedByName || "N/A"), /* @__PURE__ */ React.createElement("div",
{ className: "flex items-center gap-2 mt-1" }, /* @__PURE__ */ React.createElement("span", { className:
"text-sm text-gray-500" }, "Payment: ", transaction.paymentMethod?.charAt(0).toUpperCase() +
transaction.paymentMethod?.slice(1) || "N/A"), /* @__PURE__ */ React.createElement("span", { className: "px-2
py-0.5 text-xs font-medium rounded-full bg-green-100 text-green-700" }, "\u25CF ", transaction.status ||
"Completed"))), error && /* @__PURE__ */ React.createElement("div", { className: "mb-4 p-3 bg-red-50 border
border-red-200 rounded-lg text-sm text-red-700 flex items-center gap-2" }, /* @__PURE__ */
React.createElement(FaExclamationTriangle, null), error), /* @__PURE__ */ React.createElement("div",
{ className: "mb-6" }, /* @__PURE__ */ React.createElement("p", { className: "text-sm font-medium text-gray-
700 mb-3" }, "Select items to return:"), /* @__PURE__ */ React.createElement("div", { className: "grid grid-
cols-12 gap-2 px-3 py-2 bg-gray-50 rounded-t-lg border-b border-gray-200" }, /* @__PURE__ */
React.createElement("div", { className: "col-span-4 text-sm font-semibold text-gray-600" }, "Item"), /*
@__PURE__ */ React.createElement("div", { className: "col-span-2 text-sm font-semibold text-gray-600 text-
center" }, "Quantity"), /* @__PURE__ */ React.createElement("div", { className: "col-span-2 text-sm font-
semibold text-gray-600 text-center" }, "Price"), /* @__PURE__ */ React.createElement("div", { className:
"col-span-2 text-sm font-semibold text-gray-600 text-center" }, "Total"), /* @__PURE__ */
React.createElement("div", { className: "col-span-2 text-sm font-semibold text-gray-600 text-center" },
"Return Qty")), /* @__PURE__ */ React.createElement("div", { className: "border border-t-0 border-gray-200
```

```
rounded-b-lg divide-y divide-gray-100" }, transaction.items && transaction.items.length > 0 ?
transaction.items.map((item, idx) => {
        const isFullyReturned = item.returnStatus === "Returned";
        const isPartiallyReturned = item.returnStatus === "Partially Returned";
        const canReturn = !isFullyReturned && item.quantity > 0;
        const availableQty = item.quantity;
        const variantLabel = formatItemVariantSizeLabel(item);
        return /* @__PURE__ */ React.createElement(
          "div",
          {
            key: idx,
            className: `grid grid-cols-12 gap-2 px-3 py-3 items-center transition-all ${isFullyReturned ?
"bg-gray-100 opacity-60" : isPartiallyReturned && !selectedItems[idx] ? "bg-amber-50" : selectedItems[idx] ?
"bg-orange-50" : "bg-white"}`
          },
          /* @__PURE__ */ React.createElement("div", { className: "col-span-4 flex items-center gap-2" }, /*
@__PURE__ */ React.createElement(
            "input",
            {
              type: "checkbox",
              checked: selectedItems[idx] || false,
              onChange: () => handleItemToggle(idx),
              disabled: !canReturn,
              className: "w-4 h-4 text-orange-600 border-gray-300 rounded focus:ring-orange-500
disabled:opacity-50 disabled:cursor-not-allowed"
            }
          ), /* @__PURE__ */ React.createElement("span", { className: `text-sm truncate ${isFullyReturned ?
"text-gray-400 line-through" : "text-gray-800"}` }, cleanItemNameForDisplay(item), variantLabel ? /*
@__PURE__ */ React.createElement("span", { className: "text-gray-500 text-xs ml-1" }, "(", variantLabel,
")") : null, isPartiallyReturned && /* @__PURE__ */ React.createElement("span", { className: "text-amber-500
text-xs ml-1" }, "(", item.returnedQuantity, " already returned)"))),
          /* @__PURE__ */ React.createElement("div", { className: `col-span-2 text-sm text-center
${isFullyReturned ? "text-gray-400" : "text-gray-600"}` }, "x", availableQty),
          /* @__PURE__ */ React.createElement("div", { className: `col-span-2 text-sm text-center
${isFullyReturned ? "text-gray-400" : "text-gray-600"}` }, "\u20B1", (item.price || item.itemPrice ||
0).toLocaleString()),
          /* @__PURE__ */ React.createElement("div", { className: `col-span-2 text-sm text-center
${isFullyReturned ? "text-gray-400" : "text-gray-600"}` }, "\u20B1", ((item.price || item.itemPrice || 0) *
item.quantity).toLocaleString()),
          /* @__PURE__ */ React.createElement("div", { className: "col-span-2 flex items-center justify-
center gap-1" }, isFullyReturned ? /* @__PURE__ */ React.createElement("span", { className: "px-2 py-0.5
text-xs font-medium rounded-full bg-orange-100 text-orange-700" }, "Returned") : selectedItems[idx] ? /*
@__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(
            "button",
            {
              type: "button",
              onClick: () => handleQuantityChange(idx, -1),
              disabled: (returnQuantities[idx] === "" ? 1 : returnQuantities[idx] || 1) <= 1,
              className: `w-6 h-6 flex items-center justify-center rounded-full transition-all
${(returnQuantities[idx] === "" ? 1 : returnQuantities[idx] || 1) <= 1 ? "bg-gray-200 text-gray-400 cursor-
not-allowed" : "bg-[#AD7F65] text-white hover:bg-[#8B5F45]"}`
            },
            /* @__PURE__ */ React.createElement(FaMinus, { className: "text-[8px]" })
          ), /* @__PURE__ */ React.createElement(
            "input",
            {
              type: "number",
              inputMode: "numeric",
              min: 1,
              max: availableQty,
              disabled: !canReturn,
              value: returnQuantities[idx] === "" ? "" : returnQuantities[idx] ?? 1,
              onChange: (e) => clampReturnQty(idx, availableQty, e.target.value),
              onBlur: () => blurReturnQty(idx, availableQty),
              className: "w-12 min-w-[2.75rem] max-w-[4rem] text-center text-sm font-medium border border-
gray-200 rounded-md px-1 py-0.5 [appearance:textfield] [&::-webkit-outer-spin-button]:appearance-none [&::-
webkit-inner-spin-button]:appearance-none"
            }
          ), /* @__PURE__ */ React.createElement(
            "button",
            {
              type: "button",
              onClick: () => handleQuantityChange(idx, 1),
              disabled: (returnQuantities[idx] === "" ? 1 : returnQuantities[idx] || 1) >= availableQty,
              className: `w-6 h-6 flex items-center justify-center rounded-full transition-all
${(returnQuantities[idx] === "" ? 1 : returnQuantities[idx] || 1) >= availableQty ? "bg-gray-200 text-gray-
400 cursor-not-allowed" : "bg-[#AD7F65] text-white hover:bg-[#8B5F45]"}`
```

```
            },
            /* @__PURE__ */ React.createElement(FaPlus, { className: "text-[8px]" })
          )) : /* @__PURE__ */ React.createElement("span", { className: "text-sm text-gray-400" }, "-"))
        );
      }) : /* @__PURE__ */ React.createElement("div", { className: "px-3 py-4 text-center text-gray-500" },
"No items found"))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("label", { className: "block text-sm font-medium text-gray-700 mb-2" }, "Reason for
Return:"), /* @__PURE__ */ React.createElement("div", { className: "relative" }, /* @__PURE__ */
React.createElement(
        "select",
        {
          value: globalReason,
          onChange: (e) => {
            setGlobalReason(e.target.value);
            if (e.target.value !== "Other") {
              setOtherReason("");
            }
          },
          className: "w-full px-4 py-3 border border-gray-300 rounded-lg focus:outline-none focus:ring-2
focus:ring-orange-500 focus:border-transparent appearance-none bg-white text-gray-600"
        },
        returnReasons.map(
          (reason) => /* @__PURE__ */ React.createElement("option", { key: reason.value, value: reason.value,
disabled: reason.value === "" }, reason.label)
        )
      ), /* @__PURE__ */ React.createElement("div", { className: "absolute inset-y-0 right-0 flex items-
center pr-3 pointer-events-none" }, /* @__PURE__ */ React.createElement("svg", { className: "w-5 h-5 text-
gray-400", fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */
React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M19 9l-7
7-7-7" }))))), globalReason === "Other" && /* @__PURE__ */ React.createElement("div", { className: "mt-3" },
/* @__PURE__ */ React.createElement("label", { className: "block text-sm font-medium text-gray-700 mb-2" },
"Please specify:"), /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "text",
          value: otherReason,
          onChange: (e) => setOtherReason(e.target.value),
          placeholder: "Enter the reason for return...",
          className: "w-full px-4 py-3 border border-gray-300 rounded-lg focus:outline-none focus:ring-2
focus:ring-orange-500 focus:border-transparent bg-white text-gray-600"
        }
      )))),
      /* @__PURE__ */ React.createElement("div", { className: "border-t border-gray-200 p-6 bg-white flex
justify-end gap-3" }, /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: handleSubmit,
          className: "px-8 py-3 rounded-lg bg-orange-500 hover:bg-orange-600 text-white font-semibold
transition-all disabled:opacity-50 disabled:cursor-not-allowed",
          disabled: loading
        },
        loading ? "Processing..." : "Confirm Return"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onClose,
          className: "px-8 py-3 rounded-lg bg-gray-200 hover:bg-gray-300 text-gray-700 font-semibold
transition-all",
          disabled: loading
        },
        "Cancel"
      ))
    ),
    showPinModal && /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "fixed inset-0 flex items-center justify-center z-[10001] bg-black/60",
        onClick: () => setShowPinModal(false)
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "bg-white rounded-2xl w-full max-w-md p-6 shadow-2xl",
          onClick: (e) => e.stopPropagation()
        },
        /* @__PURE__ */ React.createElement("div", { className: "flex justify-between items-center mb-6" },
/* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
```

```
React.createElement("div", { className: "w-12 h-12 rounded-full bg-orange-100 flex items-center justify-
center" }, /* @__PURE__ */ React.createElement(FaLock, { className: "w-5 h-5 text-orange-600" })), /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("h3", { className: "text-xl
font-bold text-gray-800" }, "Authorization Required"), /* @__PURE__ */ React.createElement("p", { className:
"text-sm text-gray-500" }, "Enter Owner/Manager PIN to approve return"))), /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-2" }, /* @__PURE__ */ React.createElement(
          "button",
          {
            type: "button",
            onClick: () => setShowPin((v) => !v),
            className: "text-gray-400 hover:text-gray-600 transition",
            "aria-label": showPin ? "Hide PIN" : "Show PIN"
          },
          showPin ? /* @__PURE__ */ React.createElement(FaEyeSlash, { className: "w-5 h-5" }) : /* @__PURE__
*/ React.createElement(FaEye, { className: "w-5 h-5" })
        ), /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: () => setShowPinModal(false),
            className: "text-gray-400 hover:text-gray-600"
          },
          /* @__PURE__ */ React.createElement(FaTimes, { className: "w-5 h-5" })
        ))),
        /* @__PURE__ */ React.createElement("div", { className: "flex justify-center gap-2 mb-6" }, pin.map(
          (digit, index) => /* @__PURE__ */ React.createElement(
            "input",
            {
              key: index,
              ref: (el) => pinInputRefs.current[index] = el,
              type: showPin ? "text" : "password",
              inputMode: "numeric",
              maxLength: 1,
              value: digit,
              onChange: (e) => handlePinChange(index, e.target.value),
              onKeyDown: (e) => handlePinKeyDown(index, e),
              className: "w-12 h-14 text-center text-2xl font-bold border-2 border-gray-300 rounded-lg
focus:outline-none focus:border-orange-500 focus:ring-2 focus:ring-orange-200"
            }
          )
        )),
        pinError && /* @__PURE__ */ React.createElement("div", { className: "mb-4 p-3 bg-red-50 border
border-red-200 rounded-lg text-sm text-red-700 text-center" }, pinError),
        /* @__PURE__ */ React.createElement("div", { className: "flex gap-3" }, /* @__PURE__ */
React.createElement(
          "button",
          {
            onClick: () => setShowPinModal(false),
            className: "flex-1 px-4 py-3 rounded-lg bg-gray-200 hover:bg-gray-300 text-gray-700 font-semibold
transition-all"
          },
          "Cancel"
        ), /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: verifyPin,
            disabled: verifyingPin || pin.some((d) => !d),
            className: "flex-1 px-4 py-3 rounded-lg bg-orange-500 hover:bg-orange-600 text-white font-
semibold transition-all disabled:opacity-50 disabled:cursor-not-allowed"
          },
          verifyingPin ? "Verifying..." : "Verify & Process"
        ))
      )
    )
  );
};
export default ReturnItemsModal;
```

FILE: pos/frontend/src/components/transaction/ViewTransactionModal.jsx
==========

```
import { FaPrint, FaTimes } from "react-icons/fa";
import {
  cleanItemNameForDisplay,
  formatItemVariantSizeLabel,
  lineSubtotalFromItems,
  resolveTransactionDiscount
```

```
} from "../../utils/transactionDisplay";
const formatDate = (dateString) => {
  if (!dateString) return "N/A";
  const date = new Date(dateString);
  const options = { year: "numeric", month: "long", day: "numeric" };
  const timeOptions = { hour: "2-digit", minute: "2-digit", hour12: true };
  return `${date.toLocaleDateString("en-US", options)} at ${date.toLocaleTimeString("en-US", timeOptions)}`;
};
const formatShortDate = (dateString) => {
  if (!dateString) return "";
  const date = new Date(dateString);
  return date.toLocaleString("en-US", {
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
    hour: "2-digit",
    minute: "2-digit",
    hour12: true
  }).replace(",", "");
};
const ViewTransactionModal = ({ isOpen, onClose, transaction, onReturnItems, onPrintReceipt }) => {
  if (!isOpen || !transaction) return null;
  const transactionId = transaction.transactionNumber ? `TRX-
${String(transaction.transactionNumber).padStart(3, "0")}` : transaction.referenceNo ||
transaction._id?.substring(0, 8);
  const returnedItemsMap = {};
  if (transaction.returnTransactions && transaction.returnTransactions.length > 0) {
    transaction.returnTransactions.forEach((returnTrx) => {
      if (returnTrx.items) {
        returnTrx.items.forEach((item) => {
          const key = `${item.productId || item._id}-${item.selectedSize || ""}`;
          if (!returnedItemsMap[key]) {
            returnedItemsMap[key] = {
              quantity: 0,
              reason: item.returnReason || item.reason || "N/A",
              date: returnTrx.checkedOutAt || returnTrx.createdAt,
              restocked: true
            };
          }
          returnedItemsMap[key].quantity += item.quantity || 1;
        });
      }
    });
  }
  const currentSubtotal = lineSubtotalFromItems(transaction) || transaction.totalAmount || 0;
  const totalReturned = transaction.returnTransactions?.reduce((sum, returnTrx) => {
    return sum + (returnTrx.totalAmount || 0);
  }, 0) || 0;
  const canReturn = transaction.status === "Completed" || transaction.status === "Partially Returned";
  const hasReturns = (transaction.returnTransactions?.length || 0) > 0;
  const latestReturnTransaction = hasReturns ? [...transaction.returnTransactions || []].sort((a, b) => {
    const aTime = new Date(a?.checkedOutAt || a?.createdAt || 0).getTime();
    const bTime = new Date(b?.checkedOutAt || b?.createdAt || 0).getTime();
    return bTime - aTime;
  })[0] : null;
  const returnedByName = latestReturnTransaction?.performedByName || latestReturnTransaction?.returnedByName
|| latestReturnTransaction?.cashierName || "";
  const subtotal = hasReturns ? currentSubtotal + Math.abs(totalReturned) : currentSubtotal;
  const discountAmount = resolveTransactionDiscount(transaction, subtotal, {
    skipInference: hasReturns
  });
  const adjustedTotal = subtotal - discountAmount - Math.abs(totalReturned);
  const amountPaid = transaction.amountReceived || 0;
  const change = transaction.changeGiven || 0;
  const getItemReturnInfo = (item) => {
    if (item.returnStatus === "Returned") {
      return {
        quantity: item.quantity,
        reason: item.returnReason || "Returned",
        date: null,
        restocked: true
      };
    }
    const key = `${item.productId || item._id}-${item.selectedSize || ""}`;
    return returnedItemsMap[key] || null;
  };
  return /* @__PURE__ */ React.createElement(
```

```
    "div",
    {
      className: "fixed inset-0 flex items-center justify-center z-[10000] bg-black/50",
      onClick: onClose
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "bg-white rounded-2xl w-full max-w-xl max-h-[90vh] shadow-2xl overflow-hidden flex flex-
col",
        onClick: (e) => e.stopPropagation()
      },
      /* @__PURE__ */ React.createElement("div", { className: "p-6 overflow-y-auto flex-1" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-between items-start mb-6" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("h2", { className: "text-2xl font-bold
text-[#5D4037] mb-1" }, "Transaction Id: ", transactionId), /* @__PURE__ */ React.createElement("p",
{ className: "text-sm text-gray-500" }, "Date ", formatDate(transaction.checkedOutAt ||
transaction.createdAt)), /* @__PURE__ */ React.createElement("p", { className: "text-sm text-gray-500" },
"Performed By: ", transaction.performedByName || "N/A"), hasReturns && returnedByName && /* @__PURE__ */
React.createElement("p", { className: "text-sm text-gray-500" }, "Returned By: ", returnedByName), /*
@__PURE__ */ React.createElement("div", { className: "flex items-center gap-2 mt-1" }, /* @__PURE__ */
React.createElement("span", { className: "text-sm text-gray-500" }, "Payment: ",
transaction.paymentMethod?.charAt(0).toUpperCase() + transaction.paymentMethod?.slice(1) || "N/A"), /*
@__PURE__ */ React.createElement("span", { className: `px-2 py-0.5 text-xs font-medium rounded-full
${transaction.status === "Completed" ? "bg-green-100 text-green-700" : transaction.status === "Returned" ?
"bg-orange-100 text-orange-700" : transaction.status === "Partially Returned" ? "bg-amber-100 text-amber-
700" : "bg-red-100 text-red-600"}` }, "\u25CF ", transaction.status || "Completed"))), /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: onClose,
          className: "text-gray-400 hover:text-gray-600 transition"
        },
        /* @__PURE__ */ React.createElement(FaTimes, { className: "w-6 h-6" })
      )), /* @__PURE__ */ React.createElement("div", { className: "mb-6" }, /* @__PURE__ */
React.createElement("div", { className: "grid grid-cols-12 gap-2 px-4 py-3 bg-gray-50 rounded-t-lg border-b
border-gray-200" }, /* @__PURE__ */ React.createElement("div", { className: `${hasReturns ? "col-span-3" :
"col-span-4"} text-sm font-semibold text-gray-600` }, "Item"), /* @__PURE__ */ React.createElement("div",
{ className: "col-span-2 text-sm font-semibold text-gray-600 text-center" }, "Quantity"), /* @__PURE__ */
React.createElement("div", { className: "col-span-2 text-sm font-semibold text-gray-600 text-center" },
"Price"), hasReturns && /* @__PURE__ */ React.createElement("div", { className: "col-span-2 text-sm font-
semibold text-gray-600 text-center" }, "Returned Amount"), /* @__PURE__ */ React.createElement("div",
{ className: `${hasReturns ? "col-span-1" : "col-span-2"} text-sm font-semibold text-gray-600 text-center` },
"Total"), /* @__PURE__ */ React.createElement("div", { className: "col-span-2 text-sm font-semibold text-
gray-600 text-center" }, "Status")), /* @__PURE__ */ React.createElement("div", { className: "border border-
t-0 border-gray-200 rounded-b-lg" }, transaction.items && transaction.items.length > 0 ?
transaction.items.map((item, idx) => {
        const returnInfo = getItemReturnInfo(item);
        const isFullyReturned = item.returnStatus === "Returned";
        const isPartiallyReturned = item.returnStatus === "Partially Returned";
        const hasReturnInfo = returnInfo !== null || isFullyReturned || isPartiallyReturned;
        const displayQty = item.quantity;
        const returnedQty = item.returnedQuantity || returnInfo?.quantity || 0;
        const variantLabel = formatItemVariantSizeLabel(item);
        const unitPrice = item.price || item.itemPrice || 0;
        const lineTotal = unitPrice * displayQty;
        const returnedLineAmount = returnedQty > 0 ? unitPrice * returnedQty : 0;
        return /* @__PURE__ */ React.createElement("div", { key: idx }, /* @__PURE__ */ React.createElement(
          "div",
          {
            className: `grid grid-cols-12 gap-2 px-4 py-3 items-center ${isFullyReturned ? "bg-orange-50
border-l-4 border-l-orange-400" : isPartiallyReturned ? "bg-amber-50 border-l-4 border-l-amber-400" : "bg-
white border-b border-gray-100 last:border-b-0"}`
          },
          /* @__PURE__ */ React.createElement("div", { className: `${hasReturns ? "col-span-3" : "col-span-4"}
text-sm ${isFullyReturned ? "text-gray-400 line-through" : "text-gray-800"}` }, cleanItemNameForDisplay(item),
variantLabel ? /* @__PURE__ */ React.createElement("span", { className: "text-gray-500 text-xs ml-1" }, "(",
variantLabel, ")") : null),
          /* @__PURE__ */ React.createElement("div", { className: "col-span-2 text-sm text-gray-600 text-
center" }, isPartiallyReturned ? /* @__PURE__ */ React.createElement("span", null, "x", displayQty, " ", /*
@__PURE__ */ React.createElement("span", { className: "text-orange-500 text-xs" }, "(", returnedQty, "
returned)")) : /* @__PURE__ */ React.createElement("span", null, "x", displayQty)),
          /* @__PURE__ */ React.createElement("div", { className: "col-span-2 text-sm text-gray-600 text-
center" }, "\u20B1", unitPrice.toLocaleString()),
          hasReturns && /* @__PURE__ */ React.createElement("div", { className: "col-span-2 text-sm text-
center" }, returnedQty > 0 ? /* @__PURE__ */ React.createElement("span", { className: "font-semibold text-
orange-500" }, "-\u20B1", returnedLineAmount.toLocaleString()) : /* @__PURE__ */ React.createElement("span",
```

```
{ className: "text-gray-400" }, "-")),
          /* @__PURE__ */ React.createElement("div", { className: `${hasReturns ? "col-span-1" : "col-span-2"}
text-sm text-gray-600 text-center` }, "\u20B1", lineTotal.toLocaleString()),
          /* @__PURE__ */ React.createElement("div", { className: "col-span-2 text-center" },
isFullyReturned ? /* @__PURE__ */ React.createElement("span", { className: "px-2 py-0.5 text-xs font-medium
rounded-full bg-orange-100 text-orange-700" }, "\u25CF Returned") : isPartiallyReturned ? /* @__PURE__ */
React.createElement("span", { className: "px-2 py-0.5 text-xs font-medium rounded-full bg-amber-100 text-
amber-700" }, "\u25CF Partial") : /* @__PURE__ */ React.createElement("span", { className: "text-sm text-
gray-400" }, "-"))
        ), (isFullyReturned || isPartiallyReturned) && /* @__PURE__ */ React.createElement("div", { className:
`px-4 py-2 border-b border-gray-100 ${isFullyReturned ? "bg-orange-50 border-l-4 border-l-orange-400" : "bg-
amber-50 border-l-4 border-l-amber-400"}` }, /* @__PURE__ */ React.createElement("p", { className: `text-xs
italic ${isFullyReturned ? "text-orange-600" : "text-amber-600"}` }, "Reason: ", item.returnReason ||
returnInfo?.reason || "N/A", returnedQty > 0 && ` \u2022 ${returnedQty} item(s) returned`, returnInfo?.date ?
` \u2022 ${formatShortDate(returnInfo.date)}` : "")));
      }) : /* @__PURE__ */ React.createElement("div", { className: "px-4 py-4 text-center text-gray-500" },
"No items found"))), /* @__PURE__ */ React.createElement("div", { className: "flex justify-end mb-6" }, /*
@__PURE__ */ React.createElement("div", { className: "w-72 bg-white border border-gray-200 rounded-xl p-4
shadow-sm" }, /* @__PURE__ */ React.createElement("div", { className: "space-y-2" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-between text-sm text-gray-600" }, /* @__PURE__ */
React.createElement("span", null, "Subtotal:"), /* @__PURE__ */ React.createElement("span", null, "\u20B1",
subtotal.toLocaleString(void 0, { minimumFractionDigits: 2, maximumFractionDigits: 2 }))), /* @__PURE__ */
React.createElement("div", { className: "flex justify-between text-sm text-gray-600" }, /* @__PURE__ */
React.createElement("span", null, "Discount:"), /* @__PURE__ */ React.createElement("span", null, "\u20B1",
discountAmount.toLocaleString(void 0, { minimumFractionDigits: 2, maximumFractionDigits: 2 }))), hasReturns
&& /* @__PURE__ */ React.createElement("div", { className: "flex justify-between text-sm text-orange-500" },
/* @__PURE__ */ React.createElement("span", null, "Returned Amount:"), /* @__PURE__ */
React.createElement("span", null, "-\u20B1", Math.abs(totalReturned).toLocaleString(void 0,
{ minimumFractionDigits: 2, maximumFractionDigits: 2 }))), /* @__PURE__ */ React.createElement("div",
{ className: "flex justify-between text-base font-bold pt-2 border-t border-gray-200" }, /* @__PURE__ */
React.createElement("span", { className: "text-orange-500" }, hasReturns ? "Adjusted Total:" : "Total:"), /*
@__PURE__ */ React.createElement("span", { className: "text-orange-500" }, "\u20B1",
adjustedTotal.toLocaleString(void 0, { minimumFractionDigits: 2, maximumFractionDigits: 2 }))), !hasReturns
&& /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement("div",
{ className: "flex justify-between text-sm text-gray-600 pt-2" }, /* @__PURE__ */ React.createElement("span",
null, "Amount Paid:"), /* @__PURE__ */ React.createElement("span", null, "\u20B1",
amountPaid.toLocaleString(void 0, { minimumFractionDigits: 2, maximumFractionDigits: 2 }))), /* @__PURE__ */
React.createElement("div", { className: "flex justify-between text-sm text-gray-600" }, /* @__PURE__ */
React.createElement("span", null, "Change"), /* @__PURE__ */ React.createElement("span", null, "\u20B1",
change.toLocaleString(void 0, { minimumFractionDigits: 2, maximumFractionDigits: 2 }))))))),
      /* @__PURE__ */ React.createElement("div", { className: "border-t border-gray-200 p-6 bg-white flex
justify-center gap-3" }, canReturn && onReturnItems && /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => onReturnItems(transaction),
          className: "px-6 py-3 rounded-lg bg-orange-500 hover:bg-orange-600 text-white font-semibold
transition-all flex items-center gap-2"
        },
        "Return Items"
      ), onPrintReceipt && /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => onPrintReceipt(transaction),
          className: "px-6 py-3 rounded-lg bg-blue-600 hover:bg-blue-700 text-white font-semibold transition-
all flex items-center gap-2"
        },
        /* @__PURE__ */ React.createElement(FaPrint, { className: "w-4 h-4" }),
        "Print Receipt"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onClose,
          className: "px-6 py-3 rounded-lg bg-gray-200 hover:bg-gray-300 text-gray-700 font-semibold
transition-all"
        },
        "Cancel"
      ))
    )
  );
};
export default ViewTransactionModal;
```

FILE: pos/frontend/src/config/api.js
==========

```
export const API_BASE_URL = import.meta.env.VITE_API_URL || "http://localhost:5000";
```

```
export const WS_BASE_URL = API_BASE_URL.replace(/^http/, "ws");
export const resolveApiUrl = (url) => {
  if (typeof url !== "string") return url;
  if (url.includes("http://localhost:5000")) {
    return url.replace("http://localhost:5000", API_BASE_URL);
  }
  if (url.includes("ws://localhost:5000")) {
    return url.replace("ws://localhost:5000", WS_BASE_URL);
  }
  return url;
};
export const apiUrl = (path) => {
  const normalizedPath = path.startsWith("/") ? path : `/${path}`;
  return `${API_BASE_URL}${normalizedPath}`;
};
export const API_ENDPOINTS = {
  products: `${API_BASE_URL}/api/products`,
  productById: (id) => `${API_BASE_URL}/api/products/${id}`,
  productsLowStock: `${API_BASE_URL}/api/products/low-stock`,
  productsUpdateStock: `${API_BASE_URL}/api/products/update-stock`,
  categories: `${API_BASE_URL}/api/categories`,
  categoryById: (id) => `${API_BASE_URL}/api/categories/${id}`,
  categoryArchive: (id) => `${API_BASE_URL}/api/categories/${id}/archive`,
  brandPartners: `${API_BASE_URL}/api/brand-partners`,
  brandPartnerById: (id) => `${API_BASE_URL}/api/brand-partners/${id}`,
  employees: `${API_BASE_URL}/api/employees`,
  employeeById: (id) => `${API_BASE_URL}/api/employees/${id}`,
  employeePin: (id) => `${API_BASE_URL}/api/employees/${id}/pin`,
  employeeSendTempPin: (id) => `${API_BASE_URL}/api/employees/${id}/send-temporary-pin`,
  employeeVerifyPin: `${API_BASE_URL}/api/employees/verify-pin`,
  employeeLogout: `${API_BASE_URL}/api/employees/logout`,
  employeesOnline: `${API_BASE_URL}/api/employees/online`,
  employeeHeartbeat: `${API_BASE_URL}/api/employees/heartbeat`,
  employeeSearch: (firstName) => `${API_BASE_URL}/api/employees/search/${encodeURIComponent(firstName)}`,
  transactions: `${API_BASE_URL}/api/transactions`,
  transactionById: (id) => `${API_BASE_URL}/api/transactions/${id}`,
  transactionsDashboardStats: `${API_BASE_URL}/api/transactions/dashboard/stats`,
  transactionsSalesOverTime: `${API_BASE_URL}/api/transactions/sales-over-time`,
  transactionsSalesByCategory: `${API_BASE_URL}/api/transactions/sales-by-category`,
  transactionsTopSelling: `${API_BASE_URL}/api/transactions/top-selling`,
  cart: `${API_BASE_URL}/api/cart`,
  cartById: (id) => `${API_BASE_URL}/api/cart/${encodeURIComponent(id)}`,
  discounts: `${API_BASE_URL}/api/discounts`,
  discountById: (id) => `${API_BASE_URL}/api/discounts/${id}`,
  archive: `${API_BASE_URL}/api/archive`,
  archiveAll: `${API_BASE_URL}/api/archive/all`,
  stockMovements: `${API_BASE_URL}/api/stock-movements`,
  stockMovementsBulk: `${API_BASE_URL}/api/stock-movements/bulk`,
  stockMovementsStatsToday: `${API_BASE_URL}/api/stock-movements/stats/today`,
  voidLogs: `${API_BASE_URL}/api/void-logs`,
  verificationSendCode: `${API_BASE_URL}/api/verification/send-code`,
  verificationVerifyCode: `${API_BASE_URL}/api/verification/verify-code`,
  reportsInventoryAnalytics: `${API_BASE_URL}/api/reports/inventory-analytics`,
  syncAll: `${API_BASE_URL}/api/sync/all`,
  gcashSettings: `${API_BASE_URL}/api/gcash`,
  merchantSettings: `${API_BASE_URL}/api/merchant-settings`,
  dataManagement: `${API_BASE_URL}/api/data-management`,
  remittances: `${API_BASE_URL}/api/remittances`,
  remittanceSummary: `${API_BASE_URL}/api/remittances/summary`,
  remittanceKpiStats: `${API_BASE_URL}/api/remittances/kpi-stats`,
  globalSettings: `${API_BASE_URL}/api/global-settings`
};
export const initializeApiInterceptor = () => {
  const originalFetch = window.fetch;
  window.fetch = function(input, init) {
    let url = input;
    if (input instanceof Request) {
      const resolvedUrl = resolveApiUrl(input.url);
      if (resolvedUrl !== input.url) {
        url = new Request(resolvedUrl, input);
      }
    } else if (typeof input === "string") {
      url = resolveApiUrl(input);
    }
    return originalFetch.call(this, url, init);
  };
  const OriginalWebSocket = window.WebSocket;
```

```
  window.WebSocket = function(url, protocols) {
    const resolvedUrl = resolveApiUrl(url);
    return new OriginalWebSocket(resolvedUrl, protocols);
  };
  Object.keys(OriginalWebSocket).forEach((key) => {
    window.WebSocket[key] = OriginalWebSocket[key];
  });
  window.WebSocket.prototype = OriginalWebSocket.prototype;
  console.log("[API] Interceptor initialized. API_BASE_URL:", API_BASE_URL);
};
export default API_BASE_URL;
```

FILE: pos/frontend/src/context/AuthContext.jsx
==========

```
import { createContext, useContext, useState, useEffect } from "react";
import { API_ENDPOINTS } from "../config/api";
const AuthContext = createContext();
export const AuthProvider = ({ children }) => {
  const [currentUser, setCurrentUser] = useState(() => {
    const savedUser = localStorage.getItem("currentUser");
    return savedUser ? JSON.parse(savedUser) : null;
  });
  useEffect(() => {
    let heartbeatInterval;
    const sendHeartbeat = async () => {
      if (currentUser?._id && currentUser?.role !== "Owner") {
        try {
          await fetch(API_ENDPOINTS.employeeHeartbeat, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ employeeId: currentUser._id })
          });
        } catch (err) {
        }
      }
    };
    const handleBeforeUnload = () => {
      if (currentUser?._id) {
        navigator.sendBeacon(
          API_ENDPOINTS.employeeLogout,
          new Blob([JSON.stringify({ employeeId: currentUser._id })], { type: "application/json" })
        );
      }
    };
    if (currentUser) {
      sendHeartbeat();
      heartbeatInterval = setInterval(sendHeartbeat, 3e4);
      window.addEventListener("beforeunload", handleBeforeUnload);
    }
    return () => {
      if (heartbeatInterval) clearInterval(heartbeatInterval);
      window.removeEventListener("beforeunload", handleBeforeUnload);
    };
  }, [currentUser]);
  const login = (user) => {
    setCurrentUser(user);
    localStorage.setItem("currentUser", JSON.stringify(user));
  };
  const logout = async () => {
    if (currentUser?._id) {
      try {
        await fetch(API_ENDPOINTS.employeeLogout, {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ employeeId: currentUser._id })
        });
      } catch (err) {
        console.error("Error notifying logout:", err);
      }
    }
    setCurrentUser(null);
    localStorage.removeItem("currentUser");
  };
  const isOwner = () => {
    return currentUser?.role === "Owner" || currentUser?.name === "owner" || currentUser?.id === 3;
```

```
  };
  const hasPermission = (permission) => {
    if (!currentUser) return false;
    if (isOwner()) return true;
    if (currentUser.permissions && currentUser.permissions[permission] !== void 0) {
      return currentUser.permissions[permission];
    }
    return false;
  };
  return /* @__PURE__ */ React.createElement(AuthContext.Provider, { value: { currentUser, login, logout,
isOwner, hasPermission } }, children);
};
export const useAuth = () => {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error("useAuth must be used within an AuthProvider");
  }
  return context;
};
export default AuthContext;
```

FILE: pos/frontend/src/context/DataCacheContext.jsx
==========

```
import { createContext, useContext, useState, useCallback } from "react";
const DataCacheContext = createContext();
export const DataCacheProvider = ({ children }) => {
  const [cache, setCache] = useState({
    products: null,
    transactions: null,
    stockMovements: null,
    employees: null,
    stats: null
  });
  const [cacheTimestamps, setCacheTimestamps] = useState({
    products: null,
    transactions: null,
    stockMovements: null,
    employees: null,
    stats: null
  });
  const CACHE_DURATION = 5 * 60 * 1e3;
  const isCacheValid = useCallback((key) => {
    const timestamp = cacheTimestamps[key];
    if (!timestamp) return false;
    return Date.now() - timestamp < CACHE_DURATION;
  }, [cacheTimestamps]);
  const getCachedData = useCallback((key) => {
    if (isCacheValid(key)) {
      return cache[key];
    }
    return null;
  }, [cache, isCacheValid]);
  const setCachedData = useCallback((key, data) => {
    setCache((prev) => ({
      ...prev,
      [key]: data
    }));
    setCacheTimestamps((prev) => ({
      ...prev,
      [key]: Date.now()
    }));
  }, []);
  const invalidateCache = useCallback((key) => {
    setCache((prev) => ({
      ...prev,
      [key]: null
    }));
    setCacheTimestamps((prev) => ({
      ...prev,
      [key]: null
    }));
  }, []);
  const clearAllCache = useCallback(() => {
    setCache({
      products: null,
```

```
      transactions: null,
      stockMovements: null,
      employees: null,
      stats: null
    });
    setCacheTimestamps({
      products: null,
      transactions: null,
      stockMovements: null,
      employees: null,
      stats: null
    });
  }, []);
  return /* @__PURE__ */ React.createElement(
    DataCacheContext.Provider,
    {
      value: {
        cache,
        getCachedData,
        setCachedData,
        invalidateCache,
        clearAllCache,
        isCacheValid
      }
    },
    children
  );
};
export const useDataCache = () => {
  const context = useContext(DataCacheContext);
  if (!context) {
    throw new Error("useDataCache must be used within a DataCacheProvider");
  }
  return context;
};
```

FILE: pos/frontend/src/context/SidebarContext.jsx
==========

```
import { createContext } from "react";
export const SidebarContext = createContext({ isExpanded: false });
```

FILE: pos/frontend/src/context/ThemeContext.jsx
==========

```
import { createContext, useContext, useState, useEffect } from "react";
const ThemeContext = createContext();
export const ThemeProvider = ({ children }) => {
  const [theme, setTheme] = useState(() => {
    const saved = localStorage.getItem("theme");
    return saved || "dark";
  });
  useEffect(() => {
    localStorage.setItem("theme", theme);
  }, [theme]);
  return /* @__PURE__ */ React.createElement(ThemeContext.Provider, { value: { theme, setTheme } }, children);
};
export const useTheme = () => {
  const context = useContext(ThemeContext);
  if (!context) {
    throw new Error("useTheme must be used within a ThemeProvider");
  }
  return context;
};
```

FILE: pos/frontend/src/main.jsx
==========

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App.jsx";
import { initializeApiInterceptor } from "./config/api";
initializeApiInterceptor();
```

```
createRoot(document.getElementById("root")).render(
  /* @__PURE__ */ React.createElement(StrictMode, null, /* @__PURE__ */ React.createElement(App, null))
);
```

FILE: pos/frontend/src/pages/Archives.jsx
==========
(empty)

FILE: pos/frontend/src/pages/CashRemittance.jsx
==========
```
import React, { useState, useEffect, useMemo, forwardRef } from "react";
import { startOfDay, endOfDay, subDays, startOfWeek, endOfWeek, startOfMonth, endOfMonth } from "date-fns";
import DatePicker from "react-datepicker";
import "react-datepicker/dist/react-datepicker.css";
import { useAuth } from "../context/AuthContext";
import { API_ENDPOINTS } from "../config/api";
import Header from "../components/shared/header";
import DeleteConfirmationModal from "../components/inventory/DeleteConfirmationModal";
import {
  FaMoneyBillWave,
  FaSearch,
  FaFileInvoiceDollar,
  FaCheckCircle,
  FaExclamationTriangle,
  FaChartLine,
  FaHandHoldingUsd,
  FaBalanceScale,
  FaClock,
  FaTimes,
  FaPrint,
  FaPlus,
  FaEdit,
  FaTrash,
  FaCalendarAlt,
  FaUser
} from "react-icons/fa";
const DENOMINATIONS = [
  { key: "p1000", label: "\u20B11,000", value: 1e3 },
  { key: "p500", label: "\u20B1500", value: 500 },
  { key: "p200", label: "\u20B1200", value: 200 },
  { key: "p100", label: "\u20B1100", value: 100 },
  { key: "p50", label: "\u20B150", value: 50 },
  { key: "p20", label: "\u20B120", value: 20 },
  { key: "p10", label: "\u20B110", value: 10 },
  { key: "p5", label: "\u20B15", value: 5 },
  { key: "p1", label: "\u20B11", value: 1 },
  { key: "c25", label: "25\xA2", value: 0.25 },
  { key: "c10", label: "10\xA2", value: 0.1 },
  { key: "c5", label: "5\xA2", value: 0.05 }
];
const formatCurrency = (val) => {
  const abs = Math.abs(val || 0).toLocaleString("en-PH", { minimumFractionDigits: 2, maximumFractionDigits:
2 });
  return val < 0 ? `-\u20B1${abs}` : `\u20B1${abs}`;
};
const formatAbs = (val) => `\u20B1${Math.abs(val || 0).toLocaleString("en-PH", { minimumFractionDigits: 2,
maximumFractionDigits: 2 })}`;
const formatFloatTimestamp = (value) => {
  if (!value) return "";
  const date = new Date(value);
  if (Number.isNaN(date.getTime())) return "";
  return date.toLocaleString("en-US", {
    month: "short",
    day: "numeric",
    year: "numeric",
    hour: "2-digit",
    minute: "2-digit"
  });
};
const OPENING_FLOAT_ALLOWED_ROLES = /* @__PURE__ */ new Set([
  "Cashier",
  "Subcashier",
  "Sub-Cashier",
  "Manager",
```

```
  "Owner"
]);
const isSameLocalDay = (left, right = /* @__PURE__ */ new Date()) => {
  const a = new Date(left);
  const b = new Date(right);
  if (Number.isNaN(a.getTime()) || Number.isNaN(b.getTime())) return false;
  return a.getFullYear() === b.getFullYear() && a.getMonth() === b.getMonth() && a.getDate() === b.getDate();
};
const formatEmployeeWithRole = (name, role) => {
  const safeName = String(name || "").trim() || "Unknown";
  const safeRole = String(role || "").trim();
  return safeRole ? `${safeName} (${safeRole})` : safeName;
};
const weekOpts = { weekStartsOn: 1 };
const getPresetBounds = (preset) => {
  const now = /* @__PURE__ */ new Date();
  switch (preset) {
    case "today": {
      const s = startOfDay(now);
      return { start: s, end: s };
    }
    case "yesterday": {
      const y = subDays(startOfDay(now), 1);
      return { start: y, end: y };
    }
    case "week": {
      return {
        start: startOfWeek(now, weekOpts),
        end: startOfDay(endOfWeek(now, weekOpts))
      };
    }
    case "month": {
      return {
        start: startOfMonth(now),
        end: startOfDay(endOfMonth(now))
      };
    }
    default:
      return null;
  }
};
const remitDay = (r) => startOfDay(new Date(r.shiftDate || r.createdAt));
const isWithinDayBounds = (r, start, end) => {
  const d = remitDay(r);
  return d >= start && d <= end;
};
const KpiCard = ({
  icon: Icon,
  label,
  value,
  barGradient = "linear-gradient(180deg, #2563EB 0%, #3B82F6 100%)",
  iconBg = "bg-blue-50",
  iconColor = "text-blue-500",
  textColor = "text-blue-500",
  valueColor = "text-gray-900"
}) => /* @__PURE__ */ React.createElement("div", { className: "relative bg-white rounded-2xl shadow-sm border
border-gray-100 p-5 min-w-0 min-h-[88px] overflow-hidden" }, /* @__PURE__ */ React.createElement(
  "div",
  {
    className: "absolute left-0 top-0 bottom-0 w-2",
    style: { backgroundImage: barGradient }
  }
), /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between gap-4" }, /*
@__PURE__ */ React.createElement("div", { className: "min-w-0" }, /* @__PURE__ */ React.createElement("p",
{ className: `text-2xl lg:text-3xl font-black truncate tracking-tight ${valueColor}` }, value), /* @__PURE__
*/ React.createElement("p", { className: `text-xs lg:text-sm font-bold truncate ${textColor}` }, label)), /*
@__PURE__ */ React.createElement("div", { className: `w-11 h-11 rounded-xl ${iconBg} flex items-center
justify-center flex-shrink-0` }, /* @__PURE__ */ React.createElement(Icon, { className: `text-xl
${iconColor}` }))));
const ReceiptContent = ({ remit }) => {
  if (!remit) {
    return /* @__PURE__ */ React.createElement("div", { className: "p-10 flex flex-col items-center justify-
center text-center min-h-[400px]" }, /* @__PURE__ */ React.createElement("div", { className: "w-16 h-16
rounded-full bg-gray-50 flex items-center justify-center mb-4 border border-gray-100" }, /* @__PURE__ */
React.createElement(FaFileInvoiceDollar, { className: "text-gray-300 text-2xl" })), /* @__PURE__ */
React.createElement("h4", { className: "text-sm font-bold text-gray-400 mb-1" }, "No Slip Selected"), /*
@__PURE__ */ React.createElement("p", { className: "text-xs text-gray-400" }, "Click a remittance row to view
```

```
its receipt here."));
  }
  const denoms = remit.denominations || {};
  const denomEntries = DENOMINATIONS.map((d) => ({ ...d, qty: denoms[d.key] || 0 })).filter((d) => d.qty > 0);
  return /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("div",
{ className: "bg-gradient-to-br from-[#1A2744] to-[#2A3F5F] px-4 py-3 text-center" }, /* @__PURE__ */
React.createElement("p", { className: "text-[8px] text-gray-400 uppercase tracking-[3px] font-mono" },
"Official Document"), /* @__PURE__ */ React.createElement("h4", { className: "text-sm font-extrabold text-
white mt-0.5 font-serif italic" }, "Cash Turn-Over Slip"), /* @__PURE__ */ React.createElement("p",
{ className: "text-[9px] text-gray-400 mt-0.5" }, "Santos General Merchandise")), /* @__PURE__ */
React.createElement("div", { className: "bg-gray-50 px-3 py-2.5 border-b border-gray-200" }, /* @__PURE__ */
React.createElement("div", { className: "grid grid-cols-2 gap-y-1.5 gap-x-3" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-[8px] text-gray-
400 uppercase tracking-wider font-mono font-bold" }, "Slip No."), /* @__PURE__ */ React.createElement("p",
{ className: "text-xs font-bold text-gray-800 font-mono" }, "CTS-", remit._id.slice(-5).toUpperCase())), /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-
[8px] text-gray-400 uppercase tracking-wider font-mono font-bold" }, "Date"), /* @__PURE__ */
React.createElement("p", { className: "text-xs font-bold text-gray-800" }, new
Date(remit.shiftDate).toLocaleDateString("en-US", { month: "short", day: "numeric", year: "numeric" }))), /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-
[8px] text-gray-400 uppercase tracking-wider font-mono font-bold" }, "Time"), /* @__PURE__ */
React.createElement("p", { className: "text-xs font-bold text-gray-800" }, new
Date(remit.createdAt).toLocaleTimeString("en-US", { hour: "2-digit", minute: "2-digit" }))), /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-[8px] text-gray-
400 uppercase tracking-wider font-mono font-bold" }, "Cashier"), /* @__PURE__ */ React.createElement("p",
{ className: "text-xs font-bold text-gray-800" }, remit.employeeName)), /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-[8px] text-gray-
400 uppercase tracking-wider font-mono font-bold" }, "Float"), /* @__PURE__ */ React.createElement("p",
{ className: "text-xs font-bold text-gray-800" }, formatCurrency(remit.openingFloat || 0))), /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-[8px] text-gray-
400 uppercase tracking-wider font-mono font-bold" }, "Type"), /* @__PURE__ */ React.createElement("p",
{ className: "text-xs font-bold text-gray-800" }, "Regular"))), /* @__PURE__ */ React.createElement("div",
{ className: "px-3 py-2 space-y-2" }, /* @__PURE__ */ React.createElement("div", { className: "border-l-3
border-blue-500 bg-blue-50/40 rounded-r-lg px-2.5 py-2" }, /* @__PURE__ */ React.createElement("p",
{ className: "text-[8px] text-gray-500 uppercase tracking-wider font-mono font-bold mb-1" }, "Z-Reading
(Machine Reading)"), /* @__PURE__ */ React.createElement("div", { className: "space-y-0.5" }, /* @__PURE__ */
React.createElement("div", { className: "flex justify-between text-[11px]" }, /* @__PURE__ */
React.createElement("span", { className: "text-gray-600" }, "Gross Sales"), /* @__PURE__ */
React.createElement("span", { className: "font-bold text-gray-800" }, formatCurrency(remit.grossSales))), /*
@__PURE__ */ React.createElement("div", { className: "flex justify-between text-[11px]" }, /* @__PURE__ */
React.createElement("span", { className: "text-gray-600" }, "Less: Returns"), /* @__PURE__ */
React.createElement("span", { className: "font-bold text-red-500" }, "(", formatAbs(remit.returns), ")")), /*
@__PURE__ */ React.createElement("div", { className: "border-t border-dashed border-gray-300 pt-1 flex
justify-between text-xs" }, /* @__PURE__ */ React.createElement("span", { className: "font-bold text-gray-
700" }, "Net Z-Reading"), /* @__PURE__ */ React.createElement("span", { className: "font-extrabold text-gray-
900" }, formatCurrency(remit.netSales))))), denomEntries.length > 0 && /* @__PURE__ */
React.createElement("div", { className: "border-l-3 border-blue-500 bg-blue-50/40 rounded-r-lg px-2.5 py-2" },
/* @__PURE__ */ React.createElement("p", { className: "text-[8px] text-gray-500 uppercase tracking-wider
font-mono font-bold mb-1" }, "Cash Count Breakdown"), /* @__PURE__ */ React.createElement("div", { className:
"flex flex-wrap gap-x-3 gap-y-0.5" }, denomEntries.map((d) => /* @__PURE__ */ React.createElement("span",
{ key: d.key, className: "text-[10px] text-gray-700" }, /* @__PURE__ */ React.createElement("span",
{ className: "font-mono text-gray-500" }, d.label, "\xD7", d.qty), " ", /* @__PURE__ */
React.createElement("span", { className: "font-bold" }, formatCurrency(d.qty * d.value)))))), /* @__PURE__ */
React.createElement("div", { className: "border-l-3 border-blue-500 bg-blue-50/40 rounded-r-lg px-2.5 py-2" },
/* @__PURE__ */ React.createElement("p", { className: "text-[8px] text-gray-500 uppercase tracking-wider
font-mono font-bold mb-1" }, "Remittance Computation"), /* @__PURE__ */ React.createElement("div",
{ className: "space-y-0.5" }, /* @__PURE__ */ React.createElement("div", { className: "flex justify-between
text-[11px]" }, /* @__PURE__ */ React.createElement("span", { className: "text-gray-600" }, "Total Cash in
Drawer"), /* @__PURE__ */ React.createElement("span", { className: "font-bold text-gray-800" },
formatCurrency(remit.totalCashOnHand))), /* @__PURE__ */ React.createElement("div", { className: "flex
justify-between text-[11px]" }, /* @__PURE__ */ React.createElement("span", { className: "text-gray-600" },
"Less: Opening Float"), /* @__PURE__ */ React.createElement("span", { className: "font-bold text-gray-800" },
"(", formatAbs(remit.openingFloat || 0), ")")), /* @__PURE__ */ React.createElement("div", { className:
"border-t border-dashed border-gray-300 pt-1 flex justify-between items-center" }, /* @__PURE__ */
React.createElement("span", { className: "text-xs font-extrabold text-gray-900 tracking-wide" }, "CASH TO
REMIT"), /* @__PURE__ */ React.createElement("span", { className: "text-sm font-extrabold text-gray-900" },
formatCurrency(remit.cashToRemit))))), /* @__PURE__ */ React.createElement("div", { className: `rounded-lg
px-2.5 py-2 flex items-center justify-between ${remit.variance === 0 ? "bg-green-50 border border-green-200" :
remit.variance > 0 ? "bg-blue-50 border border-blue-200" : "bg-red-50 border border-red-200"}` }, /*
@__PURE__ */ React.createElement("span", { className: `text-xs font-extrabold uppercase tracking-wider
${remit.variance === 0 ? "text-green-600" : remit.variance > 0 ? "text-blue-600" : "text-red-600"}` },
"Variance"), /* @__PURE__ */ React.createElement("span", { className: `text-xs font-extrabold
${remit.variance === 0 ? "text-green-600" : remit.variance > 0 ? "text-blue-600" : "text-red-600"}` },
formatCurrency(remit.variance), " \u2014 ", remit.variance === 0 ? "BALANCED" : remit.variance > 0 ? "OVER" :
"SHORT"))));
};
const CalendarTrigger = forwardRef(({ onClick, className }, ref) => /* @__PURE__ */ React.createElement(
```

```
  "button",
  {
    ref,
    type: "button",
    onClick,
    className,
    "aria-label": "Pick date range"
  },
  /* @__PURE__ */ React.createElement(FaCalendarAlt, { className: "text-gray-500 text-sm" })
));
CalendarTrigger.displayName = "CalendarTrigger";
const CashRemittance = () => {
  const { isOwner, hasPermission } = useAuth();
  const [remittances, setRemittances] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const [searchTerm, setSearchTerm] = useState("");
  const [statusFilter, setStatusFilter] = useState("ALL");
  const [salesSort, setSalesSort] = useState("");
  const [datePreset, setDatePreset] = useState("today");
  const [dateRange, setDateRange] = useState([null, null]);
  const [pickerOpen, setPickerOpen] = useState(false);
  const [selectedRemittance, setSelectedRemittance] = useState(null);
  const [userFilter, setUserFilter] = useState("");
  const [staffList, setStaffList] = useState([]);
  const [globalFloat, setGlobalFloat] = useState(2e3);
  const [openingFloatEntries, setOpeningFloatEntries] = useState([]);
  const [showFloatModal, setShowFloatModal] = useState(false);
  const [floatInput, setFloatInput] = useState("");
  const [selectedFloatEmployeeId, setSelectedFloatEmployeeId] = useState("");
  const [editingFloatEntryId, setEditingFloatEntryId] = useState("");
  const [savingFloat, setSavingFloat] = useState(false);
  const [entryPendingRemoval, setEntryPendingRemoval] = useState(null);
  const fetchGlobalFloat = async () => {
    try {
      const res = await fetch(API_ENDPOINTS.globalSettings);
      const data = await res.json();
      if (data.success && data.data) {
        setGlobalFloat(data.data.openingFloat || 2e3);
        setOpeningFloatEntries(Array.isArray(data.data.openingFloats) ? data.data.openingFloats : []);
      }
    } catch (err) {
      console.error("Error fetching global settings:", err);
    }
  };
  const handleSaveFloat = async () => {
    const val = parseFloat(floatInput);
    const selectedEmployee = staffList.find((employee) => String(employee._id) === selectedFloatEmployeeId);
    const selectedEmployeeName = selectedEmployee?.name || `${selectedEmployee?.firstName || ""}
${selectedEmployee?.lastName || ""}`.trim();
    const selectedEmployeeRole = String(selectedEmployee?.role || "").trim();
    if (isNaN(val) || val < 0 || !selectedFloatEmployeeId || !selectedEmployeeName) return;
    setSavingFloat(true);
    try {
      const res = await fetch(API_ENDPOINTS.globalSettings, {
        method: "PUT",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          ...editingFloatEntryId ? {
            updateOpeningFloat: {
              entryId: editingFloatEntryId,
              employeeName: selectedEmployeeName,
              employeeRole: selectedEmployeeRole,
              amount: val
            }
          } : {
            addOpeningFloat: {
              employeeId: selectedFloatEmployeeId,
              employeeName: selectedEmployeeName,
              employeeRole: selectedEmployeeRole,
              amount: val
            }
          }
        })
      });
      const data = await res.json();
      if (data.success) {
```

```
      setGlobalFloat(data.data.openingFloat || 2e3);
      setOpeningFloatEntries(Array.isArray(data.data.openingFloats) ? data.data.openingFloats : []);
      setFloatInput("");
      setEditingFloatEntryId("");
    }
  } catch (err) {
    console.error("Error saving float:", err);
  } finally {
    setSavingFloat(false);
  }
};
const handleRemoveFloatEntry = async () => {
  const entryId = String(entryPendingRemoval?._id || "").trim();
  if (!entryId) return;
  setSavingFloat(true);
  try {
    const res = await fetch(API_ENDPOINTS.globalSettings, {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        removeOpeningFloat: { entryId }
      })
    });
    const data = await res.json();
    if (data.success) {
      setGlobalFloat(data.data.openingFloat || 2e3);
      setOpeningFloatEntries(Array.isArray(data.data.openingFloats) ? data.data.openingFloats : []);
      if (editingFloatEntryId === entryId) {
        setEditingFloatEntryId("");
        setFloatInput("");
      }
      setEntryPendingRemoval(null);
    } else {
      alert(data.message || "Failed to remove opening float");
    }
  } catch (err) {
    console.error("Error removing opening float:", err);
    alert("Failed to remove opening float. Please try again.");
  } finally {
    setSavingFloat(false);
  }
};
const fetchRemittances = async () => {
  setLoading(true);
  setError(null);
  try {
    const res = await fetch(API_ENDPOINTS.remittances);
    const data = await res.json();
    if (data.success) {
      setRemittances(data.data);
    } else {
      setError(data.message || "Failed to load remittances.");
    }
  } catch (err) {
    console.error("Error fetching remittances:", err);
    setError("Unable to connect to the server.");
  } finally {
    setLoading(false);
  }
};
useEffect(() => {
  fetchRemittances();
  fetchGlobalFloat();
  (async () => {
    try {
      const res = await fetch(API_ENDPOINTS.employees);
      const data = await res.json();
      if (data.success && Array.isArray(data.data)) {
        setStaffList(
          data.data.filter(
            (e) => e.status === "Active" && OPENING_FLOAT_ALLOWED_ROLES.has(String(e.role || "").trim())
          )
        );
      }
    } catch (err) {
      console.error("Error fetching employees:", err);
    }
```

```
    })();
  }, []);
  useEffect(() => {
    if (!selectedFloatEmployeeId && staffList.length > 0) {
      setSelectedFloatEmployeeId(String(staffList[0]._id));
    }
  }, [selectedFloatEmployeeId, staffList]);
  const [startDate, endDate] = dateRange;
  const dateFilteredRemittances = useMemo(() => {
    if (datePreset === "all") return remittances;
    if (datePreset === "custom") {
      if (!startDate) return remittances;
      const end = endDate || startDate;
      const lo = startOfDay(startDate) <= startOfDay(end) ? startOfDay(startDate) : startOfDay(end);
      const hi = startOfDay(startDate) <= startOfDay(end) ? startOfDay(end) : startOfDay(startDate);
      return remittances.filter((remit) => isWithinDayBounds(remit, lo, hi));
    }
    const bounds = getPresetBounds(datePreset);
    if (!bounds) return remittances;
    return remittances.filter((remit) => isWithinDayBounds(remit, bounds.start, bounds.end));
  }, [remittances, datePreset, startDate, endDate]);
  const userDropdownOptions = useMemo(() => {
    const map = /* @__PURE__ */ new Map();
    staffList.forEach((e) => {
      if (e?._id) {
        const label = (e.name || `${e.firstName || ""} ${e.lastName || ""}`).trim() || "Unknown";
        map.set(String(e._id), label);
      }
    });
    remittances.forEach((r) => {
      const id = r.employeeId ? String(r.employeeId) : "";
      if (id && r.employeeName && !map.has(id)) map.set(id, r.employeeName);
    });
    return Array.from(map.entries()).map(([id, name]) => ({ id, name })).sort((a, b) =>
a.name.localeCompare(b.name));
  }, [staffList, remittances]);
  const userFilterLabel = useMemo(() => {
    if (!userFilter) return "";
    return userDropdownOptions.find((o) => o.id === userFilter)?.name || "";
  }, [userFilter, userDropdownOptions]);
  const openingFloatGroups = useMemo(() => {
    const groups = /* @__PURE__ */ new Map();
    openingFloatEntries.filter((entry) => isSameLocalDay(entry?.businessDate ||
entry?.createdAt)).forEach((entry) => {
      const employeeId = String(entry?.employeeId || "");
      if (!employeeId) return;
      const existing = groups.get(employeeId) || {
        employeeId,
        employeeName: entry.employeeName || "Unknown",
        employeeRole: entry.employeeRole || "",
        total: 0,
        entries: []
      };
      existing.employeeName = entry.employeeName || existing.employeeName;
      existing.employeeRole = entry.employeeRole || existing.employeeRole;
      existing.total += Number(entry.amount) || 0;
      existing.entries.push(entry);
      groups.set(employeeId, existing);
    });
    return Array.from(groups.values()).map((group) => ({
      ...group,
      entries: [...group.entries].sort(
        (a, b) => new Date(b.createdAt || 0).getTime() - new Date(a.createdAt || 0).getTime()
      )
    })).sort((a, b) => a.employeeName.localeCompare(b.employeeName));
  }, [openingFloatEntries]);
  const todaysOpeningFloatEntries = useMemo(
    () => openingFloatEntries.filter((entry) => isSameLocalDay(entry?.businessDate || entry?.createdAt)),
    [openingFloatEntries]
  );
  const totalAssignedOpeningFloats = useMemo(
    () => todaysOpeningFloatEntries.reduce((sum, entry) => sum + (Number(entry?.amount) || 0), 0),
    [todaysOpeningFloatEntries]
  );
  const baseFiltered = useMemo(() => {
    const q = searchTerm.trim().toLowerCase();
    return dateFilteredRemittances.filter((remit) => {
```

```
      const matchesSearch = !q || remit.employeeName?.toLowerCase().includes(q) || remit._id.slice(-
6).toLowerCase().includes(q);
      const matchesUser = !userFilter || String(remit.employeeId || "") === userFilter || !remit.employeeId
&& userFilterLabel && remit.employeeName === userFilterLabel;
      const variance = remit.variance || 0;
      let matchesStatus = true;
      if (statusFilter === "BALANCED") matchesStatus = variance === 0;
      if (statusFilter === "OVER") matchesStatus = variance > 0;
      if (statusFilter === "SHORT") matchesStatus = variance < 0;
      return matchesSearch && matchesUser && matchesStatus;
    });
  }, [dateFilteredRemittances, searchTerm, statusFilter, userFilter, userFilterLabel]);
  const displayRows = useMemo(() => {
    if (salesSort === "highest") {
      return [...baseFiltered].sort((a, b) => (b.netSales || 0) - (a.netSales || 0));
    }
    if (salesSort === "lowest") {
      return [...baseFiltered].sort((a, b) => (a.netSales || 0) - (b.netSales || 0));
    }
    return baseFiltered;
  }, [baseFiltered, salesSort]);
  const kpiPeriodLabel = useMemo(() => {
    if (datePreset === "all") return "All dates";
    if (datePreset === "custom" && !startDate) return "Select a range in the calendar";
    if (datePreset === "custom" && startDate && endDate) {
      const lo = startOfDay(startDate) <= startOfDay(endDate) ? startDate : endDate;
      const hi = startOfDay(startDate) <= startOfDay(endDate) ? endDate : startDate;
      const s = lo.toLocaleDateString("en-US", { month: "short", day: "numeric", year: "numeric" });
      const e = hi.toLocaleDateString("en-US", { month: "short", day: "numeric", year: "numeric" });
      return startOfDay(lo).getTime() === startOfDay(hi).getTime() ? s : `${s} \u2013 ${e}`;
    }
    if (datePreset === "custom" && startDate) {
      return startDate.toLocaleDateString("en-US", { month: "short", day: "numeric", year: "numeric" });
    }
    if (datePreset === "today") return "Today";
    if (datePreset === "yesterday") return "Yesterday";
    if (datePreset === "week") return "This week";
    if (datePreset === "month") return "This month";
    return "All dates";
  }, [datePreset, startDate, endDate]);
  const kpiQueryKey = useMemo(() => {
    if (datePreset === "all") return { all: true };
    if (datePreset === "custom" && !startDate) return { all: true };
    if (datePreset === "custom") {
      const end = endDate || startDate;
      const first = startOfDay(startDate) <= startOfDay(end) ? startDate : end;
      const last = startOfDay(startDate) <= startOfDay(end) ? endDate || startDate : startDate;
      return {
        startMs: startOfDay(first).getTime(),
        endMs: endOfDay(last).getTime()
      };
    }
    const b = getPresetBounds(datePreset);
    if (!b) return { all: true };
    return {
      startMs: startOfDay(b.start).getTime(),
      endMs: endOfDay(b.end).getTime()
    };
  }, [datePreset, startDate, endDate]);
  const [kpiStats, setKpiStats] = useState({
    posNetSales: 0,
    totalRemitted: 0,
    totalVariance: 0,
    unremittedCash: 0
  });
  const [kpiLoading, setKpiLoading] = useState(false);
  useEffect(() => {
    let cancelled = false;
    (async () => {
      setKpiLoading(true);
      try {
        const params = new URLSearchParams();
        if (!kpiQueryKey.all) {
          params.set("startMs", String(kpiQueryKey.startMs));
          params.set("endMs", String(kpiQueryKey.endMs));
        }
        if (userFilter) params.set("employeeId", userFilter);
```

```
        const qs = params.toString();
        const res = await fetch(
          `${API_ENDPOINTS.remittanceKpiStats}${qs ? `?${qs}` : ""}`
        );
        const data = await res.json();
        if (cancelled) return;
        if (data.success && data.data) {
          setKpiStats({
            posNetSales: data.data.posNetSales ?? 0,
            totalRemitted: data.data.totalRemitted ?? 0,
            totalVariance: data.data.totalVariance ?? 0,
            unremittedCash: data.data.unremittedCash ?? 0
          });
        } else {
          setKpiStats({
            posNetSales: 0,
            totalRemitted: 0,
            totalVariance: 0,
            unremittedCash: 0
          });
        }
      } catch (err) {
        console.error("Error loading remittance KPI stats:", err);
        if (!cancelled) {
          setKpiStats({
            posNetSales: 0,
            totalRemitted: 0,
            totalVariance: 0,
            unremittedCash: 0
          });
        }
      } finally {
        if (!cancelled) setKpiLoading(false);
      }
    })();
    return () => {
      cancelled = true;
    };
  }, [kpiQueryKey, userFilter]);
  useEffect(() => {
    if (displayRows.length === 0) {
      setSelectedRemittance(null);
      return;
    }
    const id = selectedRemittance?._id;
    const still = id && displayRows.some((r) => r._id === id);
    if (!still) setSelectedRemittance(displayRows[0]);
  }, [displayRows, selectedRemittance?._id]);
  const handleDatePresetChange = (e) => {
    const v = e.target.value;
    setDatePreset(v);
    if (v === "all") {
      setDateRange([null, null]);
    } else if (v === "custom") {
      setPickerOpen(true);
    } else {
      const b = getPresetBounds(v);
      if (b) setDateRange([b.start, b.end]);
    }
  };
  const handlePrint = () => {
    if (!selectedRemittance) return;
    const printContent = document.getElementById("receipt-print-area");
    if (!printContent) return;
    const win = window.open("", "_blank", "width=400,height=700");
    win.document.write(`
            <html><head><title>Cash Turn-Over Slip</title>
            <style>
                body { font-family: 'Courier New', monospace; padding: 20px; font-size: 12px; }
                .header { text-align: center; border-bottom: 2px dashed #000; padding-bottom: 10px; margin-
bottom: 10px; }
                .row { display: flex; justify-content: space-between; padding: 2px 0; }
                .section { border-top: 1px dashed #999; margin-top: 8px; padding-top: 8px; }
                .bold { font-weight: bold; }
                .big { font-size: 16px; }
            </style></head><body>${printContent.innerHTML}</body></html>
        `);
```

```
    win.document.close();
    win.print();
  };
  if (!isOwner() && !hasPermission("viewTransactions")) {
    return /* @__PURE__ */ React.createElement("div", { className: "flex h-[80vh] items-center justify-
center" }, /* @__PURE__ */ React.createElement("div", { className: "text-center p-8 bg-red-50 rounded-2xl
max-w-sm border border-red-100" }, /* @__PURE__ */ React.createElement(FaFileInvoiceDollar, { className: "mx-
auto text-4xl text-red-500 mb-4" }), /* @__PURE__ */ React.createElement("h2", { className: "text-xl font-
bold text-red-700 mb-2" }, "Access Denied"), /* @__PURE__ */ React.createElement("p", { className: "text-red-
500 text-sm" }, "You do not have permission to view cash remittances")));
  }
  return /* @__PURE__ */ React.createElement("div", { className: "p-8 min-h-screen animate-fade-in pb-24",
style: { fontFamily: "'Poppins', sans-serif" } }, /* @__PURE__ */ React.createElement(Header, { pageName:
"Cash Remittance", showBorder: false, profileBackground: "" }), /* @__PURE__ */ React.createElement("div",
{ className: "flex gap-6 items-start mb-6 mt-4" }, /* @__PURE__ */ React.createElement("div", { className:
"flex-1 min-w-0" }, /* @__PURE__ */ React.createElement("p", { className: "text-xs font-semibold text-gray-
400 mb-2" }, "Totals \u2014 ", kpiPeriodLabel, userFilterLabel ? ` \xB7 ${userFilterLabel}` : ""), /*
@__PURE__ */ React.createElement("div", { className: `grid grid-cols-4 gap-3 min-w-0 ${kpiLoading ? "opacity-
70" : ""}` }, /* @__PURE__ */ React.createElement(
    KpiCard,
    {
      icon: FaChartLine,
      label: "Total Net Sales",
      value: kpiLoading ? "\u2026" : formatCurrency(kpiStats.posNetSales),
      barGradient: "linear-gradient(180deg, #2563EB 0%, #3B82F6 100%)",
      iconBg: "bg-blue-50",
      iconColor: "text-blue-500",
      textColor: "text-blue-500",
      valueColor: "text-blue-600"
    }
  ), /* @__PURE__ */ React.createElement(
    KpiCard,
    {
      icon: FaHandHoldingUsd,
      label: "Total Remitted",
      value: kpiLoading ? "\u2026" : formatCurrency(kpiStats.totalRemitted),
      barGradient: "linear-gradient(180deg, #0EA5A4 0%, #22C55E 100%)",
      iconBg: "bg-green-50",
      iconColor: "text-green-500",
      textColor: "text-green-500",
      valueColor: "text-green-600"
    }
  ), /* @__PURE__ */ React.createElement(
    KpiCard,
    {
      icon: FaBalanceScale,
      label: "Total Variance",
      value: kpiLoading ? "\u2026" : `${kpiStats.totalVariance > 0 ? "+" :
""}${formatCurrency(kpiStats.totalVariance)}`,
      barGradient: "linear-gradient(180deg, #D97706 0%, #F59E0B 100%)",
      iconBg: "bg-amber-50",
      iconColor: "text-amber-500",
      textColor: "text-amber-500",
      valueColor: "text-amber-600"
    }
  ), /* @__PURE__ */ React.createElement(
    KpiCard,
    {
      icon: FaClock,
      label: "Outstanding (Unremitted)",
      value: kpiLoading ? "\u2026" : formatCurrency(kpiStats.unremittedCash),
      barGradient: "linear-gradient(180deg, #B91C1C 0%, #EF4444 100%)",
      iconBg: "bg-red-50",
      iconColor: "text-red-500",
      textColor: "text-red-500",
      valueColor: "text-red-600"
    }
  )), datePreset === "all" && /* @__PURE__ */ React.createElement("p", { className: "text-[10px] text-gray-
400 mt-2 leading-snug" }, "All dates: totals include every sale and every slip in the system.")), /*
@__PURE__ */ React.createElement("div", { className: "w-[380px] flex-shrink-0" }, /* @__PURE__ */
React.createElement("div", { className: "bg-white rounded-2xl shadow-sm border border-gray-100 p-5 h-full" },
/* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between" }, /* @__PURE__
*/ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-3xl font-
black text-gray-800" }, formatCurrency(totalAssignedOpeningFloats || globalFloat)), /* @__PURE__ */
React.createElement("p", { className: "text-sm font-semibold text-gray-400 mt-1" }, "Today's Assigned Opening
Floats")), isOwner() && /* @__PURE__ */ React.createElement(
    "button",
```

```
    {
      onClick: () => {
        setFloatInput("");
        setSelectedFloatEmployeeId((prev) => prev || String(staffList[0]?._id || ""));
        setEditingFloatEntryId("");
        setShowFloatModal(true);
      },
      className: "flex items-center gap-2 bg-green-500 hover:bg-green-600 text-white px-4 py-2.5 rounded-xl
font-bold text-sm transition-colors cursor-pointer shadow-sm"
    },
    /* @__PURE__ */ React.createElement(FaPlus, { className: "text-xs" }),
    " New"
  )))))), showFloatModal && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 bg-black/40
z-[200] flex items-center justify-center", onClick: () => setShowFloatModal(false) }, /* @__PURE__ */
React.createElement("div", { className: "bg-white rounded-2xl shadow-xl p-6 w-full max-w-4xl", onClick: (e)
=> e.stopPropagation() }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-
between mb-4" }, /* @__PURE__ */ React.createElement("h3", { className: "text-lg font-bold text-gray-800" },
"Set Opening Float"), /* @__PURE__ */ React.createElement("button", { onClick: () => setShowFloatModal(false),
className: "p-1.5 rounded-lg hover:bg-gray-100 text-gray-400 cursor-pointer" }, /* @__PURE__ */
React.createElement(FaTimes, null))), /* @__PURE__ */ React.createElement("p", { className: "text-xs text-
gray-500 mb-4" }, "Assign opening float amounts to eligible staff. Multiple entries for the same cashier are
combined, and only today's entries are used in today's remittance."), /* @__PURE__ */
React.createElement("div", { className: "grid grid-cols-1 lg:grid-cols-[360px_minmax(0,1fr)] gap-6" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className:
"block text-xs font-bold text-gray-500 uppercase tracking-wide mb-2" }, "Cashier"), /* @__PURE__ */
React.createElement(
    "select",
    {
      className: "w-full px-4 py-3 border border-gray-200 rounded-xl text-sm font-semibold text-gray-800
focus:outline-none focus:ring-2 focus:ring-green-500/20 focus:border-green-500 mb-3",
      value: selectedFloatEmployeeId,
      onChange: (e) => setSelectedFloatEmployeeId(e.target.value),
      disabled: Boolean(editingFloatEntryId)
    },
    /* @__PURE__ */ React.createElement("option", { value: "" }, "Select employee"),
    staffList.map((employee) => {
      const label = employee.name || `${employee.firstName || ""} ${employee.lastName || ""}`.trim() ||
"Unknown";
      return /* @__PURE__ */ React.createElement("option", { key: employee._id, value: employee._id },
formatEmployeeWithRole(label, employee.role));
    })
  ), editingFloatEntryId && /* @__PURE__ */ React.createElement("p", { className: "text-[11px] text-amber-700
bg-amber-50 rounded-lg px-3 py-2 mb-3" }, "Editing updates the saved float amount for this employee entry."),
/* @__PURE__ */ React.createElement("label", { className: "block text-xs font-bold text-gray-500 uppercase
tracking-wide mb-2" }, "Float Amount"), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "number",
      className: "w-full px-4 py-3 border border-gray-200 rounded-xl text-lg font-bold text-gray-800
focus:outline-none focus:ring-2 focus:ring-green-500/20 focus:border-green-500 mb-4",
      placeholder: "e.g. 2000",
      value: floatInput,
      onChange: (e) => setFloatInput(e.target.value),
      autoFocus: true
    }
  ), editingFloatEntryId && /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: () => {
        setEditingFloatEntryId("");
        setFloatInput("");
        setSelectedFloatEmployeeId(String(staffList[0]?._id || ""));
      },
      className: "text-xs font-semibold text-gray-500 hover:text-gray-700 mb-4"
    },
    "Cancel editing"
  ), /* @__PURE__ */ React.createElement("div", { className: "rounded-xl border border-green-100 bg-green-50
px-4 py-3 mb-4" }, /* @__PURE__ */ React.createElement("p", { className: "text-[11px] font-bold uppercase
tracking-wide text-green-600" }, "Total assigned floats"), /* @__PURE__ */ React.createElement("p",
{ className: "text-xl font-black text-green-700 mt-1" }, formatCurrency(totalAssignedOpeningFloats)))), /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("div", { className: "flex
items-center justify-between mb-2" }, /* @__PURE__ */ React.createElement("h4", { className: "text-sm font-
bold text-gray-800" }, "Today's Floats Per Employee"), /* @__PURE__ */ React.createElement("span",
{ className: "text-xs text-gray-400" }, todaysOpeningFloatEntries.length, " entr",
todaysOpeningFloatEntries.length === 1 ? "y" : "ies")), /* @__PURE__ */ React.createElement("div",
{ className: "max-h-[320px] overflow-y-auto rounded-xl border border-gray-100 bg-gray-50 p-3 space-y-3" },
```

```
openingFloatGroups.length === 0 ? /* @__PURE__ */ React.createElement("div", { className: "text-center py-
8" }, /* @__PURE__ */ React.createElement("p", { className: "text-sm font-semibold text-gray-500" }, "No
opening floats assigned for today."), /* @__PURE__ */ React.createElement("p", { className: "text-xs text-
gray-400 mt-1" }, "Tomorrow will start fresh automatically because only today's entries are counted.")) :
openingFloatGroups.map((group) => /* @__PURE__ */ React.createElement("div", { key: group.employeeId,
className: "bg-white border border-gray-100 rounded-xl p-3 shadow-sm" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center justify-between gap-3 mb-2" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-sm font-bold
text-gray-800" }, formatEmployeeWithRole(group.employeeName, group.employeeRole)), /* @__PURE__ */
React.createElement("p", { className: "text-[11px] text-gray-400" }, group.entries.length, " float entr",
group.entries.length === 1 ? "y" : "ies"))), /* @__PURE__ */ React.createElement("div", { className: "space-
y-1.5" }, group.entries.map((entry) => /* @__PURE__ */ React.createElement("div", { key: entry._id ||
`${group.employeeId}-${entry.createdAt}`, className: "flex items-center justify-between gap-2 text-xs text-
gray-600 bg-gray-50 rounded-lg px-2.5 py-2" }, /* @__PURE__ */ React.createElement("div", { className: "min-
w-0" }, /* @__PURE__ */ React.createElement("div", null, formatFloatTimestamp(entry.createdAt) || "Saved
float"), entry.employeeRole && /* @__PURE__ */ React.createElement("div", { className: "text-[10px] text-
gray-400 mt-0.5" }, entry.employeeRole)), /* @__PURE__ */ React.createElement("div", { className: "flex
items-center gap-2 shrink-0" }, /* @__PURE__ */ React.createElement("span", { className: "font-bold text-
gray-800" }, formatCurrency(entry.amount)), /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: () => {
        setEditingFloatEntryId(String(entry._id || ""));
        setSelectedFloatEmployeeId(String(entry.employeeId || ""));
        setFloatInput(String(entry.amount || ""));
      },
      className: "inline-flex items-center gap-1 rounded-lg border border-gray-200 bg-white px-2 py-1 text-
[11px] font-semibold text-gray-600 hover:bg-gray-100",
      title: "Edit float"
    },
    /* @__PURE__ */ React.createElement(FaEdit, { className: "text-[10px]" }),
    "Edit"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: () => setEntryPendingRemoval(entry),
      className: "inline-flex items-center gap-1 rounded-lg border border-red-200 bg-white px-2 py-1 text-
[11px] font-semibold text-red-600 hover:bg-red-50",
      title: "Remove float"
    },
    /* @__PURE__ */ React.createElement(FaTrash, { className: "text-[10px]" }),
    "Remove"
  )))))))))), /* @__PURE__ */ React.createElement("div", { className: "flex gap-2 mt-5" }, /* @__PURE__ */
React.createElement("button", { onClick: () => setShowFloatModal(false), className: "flex-1 py-2.5 rounded-xl
border border-gray-200 text-sm font-bold text-gray-600 hover:bg-gray-50 cursor-pointer" }, "Cancel"), /*
@__PURE__ */ React.createElement(
    "button",
    {
      onClick: handleSaveFloat,
      disabled: savingFloat || !floatInput || !selectedFloatEmployeeId,
      className: `flex-1 py-2.5 rounded-xl text-white text-sm font-bold disabled:opacity-50 cursor-pointer
${editingFloatEntryId ? "bg-blue-600 hover:bg-blue-700" : "bg-green-500 hover:bg-green-600"}`
    },
    savingFloat ? "Saving..." : editingFloatEntryId ? "Update" : "Save"
  )))), /* @__PURE__ */ React.createElement("div", { className: "flex gap-6 items-start" }, /* @__PURE__ */
React.createElement("div", { className: "flex-1 min-w-0 space-y-4" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement("div", { className: "relative flex-1" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "absolute left-1 top-1/2 transform -translate-y-1/2 w-10 h-9 flex items-center justify-
center text-white rounded-xl",
      style: { background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)" }
    },
    /* @__PURE__ */ React.createElement(FaSearch, { className: "text-sm" })
  ), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      placeholder: "Search For...",
      className: "w-full h-11 pl-14 pr-4 border border-gray-200 bg-white text-gray-900 rounded-xl shadow-sm
focus:outline-none focus:ring-2 focus:ring-[#AD7F65] focus:border-transparent transition-colors text-sm
placeholder-gray-400",
      value: searchTerm,
      onChange: (e) => setSearchTerm(e.target.value)
```

```
    }
  )), /* @__PURE__ */ React.createElement(
    "select",
    {
      className: "px-3 py-2 bg-gray-50 border border-gray-200 rounded-xl text-xs font-semibold text-gray-600
focus:outline-none focus:ring-1 focus:ring-gray-300 cursor-pointer appearance-none pr-7 bg-
[url('data:image/svg+xml;charset=US-
ASCII,%3Csvg%20xmlns%3D%22http%3A%2F%2Fwww.w3.org%2F2000%2Fsvg%22%20width%3D%2212%22%20height%3D%2212%22%20vi
ewBox%3D%220%200%2012%2012%22%3E%3Cpath%20fill%3D%22%23666%22%20d%3D%22M6%208L1%203h10z%22%2F%3E%3C%2Fsvg%3E')
] bg-no-repeat bg-[right_8px_center]",
      value: salesSort,
      onChange: (e) => setSalesSort(e.target.value)
    },
    /* @__PURE__ */ React.createElement("option", { value: "" }, "By Sales"),
    /* @__PURE__ */ React.createElement("option", { value: "highest" }, "Highest First"),
    /* @__PURE__ */ React.createElement("option", { value: "lowest" }, "Lowest First")
  ), /* @__PURE__ */ React.createElement(
    "select",
    {
      className: "px-3 py-2 bg-gray-50 border border-gray-200 rounded-xl text-xs font-semibold text-gray-600
focus:outline-none focus:ring-1 focus:ring-gray-300 cursor-pointer appearance-none pr-7 bg-
[url('data:image/svg+xml;charset=US-
ASCII,%3Csvg%20xmlns%3D%22http%3A%2F%2Fwww.w3.org%2F2000%2Fsvg%22%20width%3D%2212%22%20height%3D%2212%22%20vi
ewBox%3D%220%200%2012%2012%22%3E%3Cpath%20fill%3D%22%23666%22%20d%3D%22M6%208L1%203h10z%22%2F%3E%3C%2Fsvg%3E')
] bg-no-repeat bg-[right_8px_center]",
      value: userFilter,
      onChange: (e) => setUserFilter(e.target.value)
    },
    /* @__PURE__ */ React.createElement("option", { value: "" }, "All users"),
    userDropdownOptions.map(({ id, name }) => /* @__PURE__ */ React.createElement("option", { key: id, value:
id }, name))
  ), /* @__PURE__ */ React.createElement(
    "select",
    {
      className: "px-3 py-2 bg-gray-50 border border-gray-200 rounded-xl text-xs font-semibold text-gray-600
focus:outline-none focus:ring-1 focus:ring-gray-300 cursor-pointer appearance-none pr-7 bg-
[url('data:image/svg+xml;charset=US-
ASCII,%3Csvg%20xmlns%3D%22http%3A%2F%2Fwww.w3.org%2F2000%2Fsvg%22%20width%3D%2212%22%20height%3D%2212%22%20vi
ewBox%3D%220%200%2012%2012%22%3E%3Cpath%20fill%3D%22%23666%22%20d%3D%22M6%208L1%203h10z%22%2F%3E%3C%2Fsvg%3E')
] bg-no-repeat bg-[right_8px_center]",
      value: statusFilter,
      onChange: (e) => setStatusFilter(e.target.value)
    },
    /* @__PURE__ */ React.createElement("option", { value: "ALL" }, "By Status"),
    /* @__PURE__ */ React.createElement("option", { value: "BALANCED" }, "Balanced"),
    /* @__PURE__ */ React.createElement("option", { value: "OVER" }, "Over"),
    /* @__PURE__ */ React.createElement("option", { value: "SHORT" }, "Short")
  ), /* @__PURE__ */ React.createElement(
    "select",
    {
      className: "px-3 py-2 bg-gray-50 border border-gray-200 rounded-xl text-xs font-semibold text-gray-600
focus:outline-none focus:ring-1 focus:ring-gray-300 cursor-pointer appearance-none pr-7 bg-
[url('data:image/svg+xml;charset=US-
ASCII,%3Csvg%20xmlns%3D%22http%3A%2F%2Fwww.w3.org%2F2000%2Fsvg%22%20width%3D%2212%22%20height%3D%2212%22%20vi
ewBox%3D%220%200%2012%2012%22%3E%3Cpath%20fill%3D%22%23666%22%20d%3D%22M6%208L1%203h10z%22%2F%3E%3C%2Fsvg%3E')
] bg-no-repeat bg-[right_8px_center]",
      value: datePreset,
      onChange: handleDatePresetChange
    },
    /* @__PURE__ */ React.createElement("option", { value: "all" }, "All dates"),
    /* @__PURE__ */ React.createElement("option", { value: "today" }, "Today"),
    /* @__PURE__ */ React.createElement("option", { value: "yesterday" }, "Yesterday"),
    /* @__PURE__ */ React.createElement("option", { value: "week" }, "This Week"),
    /* @__PURE__ */ React.createElement("option", { value: "month" }, "This Month"),
    /* @__PURE__ */ React.createElement("option", { value: "custom" }, "Custom range\u2026")
  ), /* @__PURE__ */ React.createElement("div", { className: "relative flex-shrink-0 z-[100]" }, /* @__PURE__
*/ React.createElement(
    DatePicker,
    {
      selectsRange: true,
      selected: startDate,
      onChange: (update) => {
        setDateRange(update);
        setDatePreset("custom");
        if (update?.[0] && update?.[1]) setPickerOpen(false);
      },
      startDate,
```

```
      endDate,
      open: pickerOpen,
      onInputClick: () => setPickerOpen(true),
      onClickOutside: () => setPickerOpen(false),
      dateFormat: "MMM d, yyyy",
      popperPlacement: "bottom-end",
      popperClassName: "z-[100]",
      customInput: /* @__PURE__ */ React.createElement(CalendarTrigger, { className: `w-10 h-10 rounded-xl
border flex items-center justify-center flex-shrink-0 transition-colors cursor-pointer ${datePreset ===
"custom" && startDate && endDate ? "border-[#AD7F65] bg-amber-50/50" : "border-gray-200 bg-gray-50 hover:bg-
gray-100"}` })
    }
  })), /* @__PURE__ */ React.createElement("div", { className: "bg-white rounded-2xl shadow-sm border border-
gray-100 overflow-hidden" }, /* @__PURE__ */ React.createElement("div", { className: "overflow-x-auto" }, /*
@__PURE__ */ React.createElement("table", { className: "w-full text-left border-collapse" }, /* @__PURE__ */
React.createElement("thead", null, /* @__PURE__ */ React.createElement("tr", { className: "bg-gray-50 border-
b border-gray-100" }, /* @__PURE__ */ React.createElement("th", { className: "py-3 px-4 text-xs font-bold
text-gray-400 uppercase tracking-wider" }, "Date & Time"), /* @__PURE__ */ React.createElement("th",
{ className: "py-3 px-4 text-xs font-bold text-gray-400 uppercase tracking-wider" }, "Cashier"), /* @__PURE__
*/ React.createElement("th", { className: "py-3 px-4 text-xs font-bold text-gray-400 uppercase tracking-wider
text-right" }, "Net Sales"), /* @__PURE__ */ React.createElement("th", { className: "py-3 px-4 text-xs font-
bold text-gray-400 uppercase tracking-wider text-right" }, "Opening Float"), /* @__PURE__ */
React.createElement("th", { className: "py-3 px-4 text-xs font-bold text-gray-400 uppercase tracking-wider
text-right" }, "Cash to Remit"), /* @__PURE__ */ React.createElement("th", { className: "py-3 px-4 text-xs
font-bold text-gray-400 uppercase tracking-wider text-right" }, "Variance"), /* @__PURE__ */
React.createElement("th", { className: "py-3 px-4 text-xs font-bold text-gray-400 uppercase tracking-wider
text-center" }, "Status"))), /* @__PURE__ */ React.createElement("tbody", { className: "divide-y divide-gray-
50" }, loading ? /* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */ React.createElement("td",
{ colSpan: "7", className: "py-12 text-center" }, /* @__PURE__ */ React.createElement("div", { className:
"inline-block animate-spin rounded-full h-8 w-8 border-b-2 border-[#22C55E]" }), /* @__PURE__ */
React.createElement("p", { className: "mt-2 text-sm text-gray-500" }, "Loading remittances..."))) : error ?
/* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */ React.createElement("td", { colSpan: "7",
className: "py-12 text-center" }, /* @__PURE__ */ React.createElement("div", { className: "inline-flex items-
center justify-center w-12 h-12 rounded-full bg-red-100 mb-3" }, /* @__PURE__ */
React.createElement(FaExclamationTriangle, { className: "text-red-500 text-xl" })), /* @__PURE__ */
React.createElement("p", { className: "text-sm font-medium text-gray-800" }, "Error loading data"), /*
@__PURE__ */ React.createElement("p", { className: "text-xs text-red-500 mt-1" }, error))) :
displayRows.length === 0 ? /* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */
React.createElement("td", { colSpan: "7", className: "py-16 text-center" }, /* @__PURE__ */
React.createElement("div", { className: "inline-flex items-center justify-center w-16 h-16 rounded-full bg-
gray-50 mb-4 border border-gray-100" }, /* @__PURE__ */ React.createElement(FaFileInvoiceDollar, { className:
"text-gray-300 text-2xl" })), /* @__PURE__ */ React.createElement("h3", { className: "text-sm font-bold text-
gray-800 mb-1" }, "No Remittances Found"), /* @__PURE__ */ React.createElement("p", { className: "text-xs
text-gray-500" }, searchTerm || statusFilter !== "ALL" || datePreset !== "all" || userFilter ? "Try adjusting
your filters." : "No cash remittances have been submitted yet."))) : displayRows.map((remit) => /* @__PURE__
*/ React.createElement(
    "tr",
    {
      key: remit._id,
      onClick: () => setSelectedRemittance(remit),
      className: `transition-colors cursor-pointer ${selectedRemittance?._id === remit._id ? "bg-green-50/60
border-l-[3px] border-l-[#22C55E]" : "hover:bg-gray-50/50"}`
    },
    /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4" }, /* @__PURE__ */
React.createElement("p", { className: "text-sm font-semibold text-gray-800" }, new
Date(remit.shiftDate).toLocaleDateString("en-US", { month: "short", day: "numeric", year: "numeric" })), /*
@__PURE__ */ React.createElement("p", { className: "text-xs text-gray-400 mt-0.5" }, new
Date(remit.createdAt).toLocaleTimeString("en-US", { hour: "2-digit", minute: "2-digit" }))),
    /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4" }, /* @__PURE__ */
React.createElement("p", { className: "text-sm font-semibold text-gray-800" }, remit.employeeName), /*
@__PURE__ */ React.createElement("p", { className: "text-[10px] text-gray-400 font-mono mt-0.5" }, "#",
remit._id.slice(-6).toUpperCase())),
    /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4 text-right" }, /* @__PURE__ */
React.createElement("span", { className: "text-sm font-medium text-gray-600" },
formatCurrency(remit.netSales))),
    /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4 text-right" }, /* @__PURE__ */
React.createElement("span", { className: "text-sm font-medium text-gray-600" },
formatCurrency(remit.openingFloat || 0))),
    /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4 text-right" }, /* @__PURE__ */
React.createElement("span", { className: "text-sm font-bold text-gray-800" },
formatCurrency(remit.cashToRemit))),
    /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4 text-right" }, /* @__PURE__ */
React.createElement("span", { className: `text-sm font-bold ${remit.variance === 0 ? "text-green-600" :
remit.variance > 0 ? "text-blue-600" : "text-red-600"}` }, remit.variance > 0 ? "+" : "",
formatCurrency(remit.variance))),
    /* @__PURE__ */ React.createElement("td", { className: "py-3 px-4 text-center" }, /* @__PURE__ */
React.createElement("div", { className: `inline-flex items-center gap-1.5 px-2.5 py-1 rounded-full text-[10px]
```

```
font-bold uppercase tracking-wider ${remit.variance === 0 ? "bg-green-100 text-green-700" : remit.variance >
0 ? "bg-blue-100 text-blue-700" : "bg-red-100 text-red-700"}` }, remit.variance === 0 ? /* @__PURE__ */
React.createElement(FaCheckCircle, null) : /* @__PURE__ */ React.createElement(FaExclamationTriangle, null),
remit.variance === 0 ? "Balanced" : remit.variance > 0 ? "Over" : "Short"))
  )), /* @__PURE__ */ React.createElement(
    DeleteConfirmationModal,
    {
      isOpen: Boolean(entryPendingRemoval),
      onClose: () => setEntryPendingRemoval(null),
      onConfirm: handleRemoveFloatEntry,
      itemName: "opening float entry",
      title: "Remove this opening float entry?",
      description: "This can't be undone.",
      confirmLabel: "OK"
    }
  )))))), /* @__PURE__ */ React.createElement("div", { className: "text-center" }, /* @__PURE__ */
React.createElement("p", { className: "text-xs text-gray-400 flex items-center justify-center gap-2" }, /*
@__PURE__ */ React.createElement("span", { className: "w-2 h-2 rounded-full bg-green-500 inline-block" }),
"Remittances are automatically recorded here when cashiers submit their end-of-shift reports."))), /*
@__PURE__ */ React.createElement("div", { className: "w-[380px] flex-shrink-0" }, /* @__PURE__ */
React.createElement("div", { className: "bg-white rounded-2xl shadow-sm border border-gray-100 overflow-
hidden sticky top-6" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-
between p-4 border-b border-gray-100" }, /* @__PURE__ */ React.createElement("h3", { className: "text-sm
font-bold text-gray-800" }, "Cash Turn-Over Slip"), /* @__PURE__ */ React.createElement("div", { className:
"flex items-center gap-1" }, selectedRemittance && /* @__PURE__ */ React.createElement("button", { onClick:
handlePrint, className: "p-2 rounded-lg hover:bg-gray-100 text-gray-400 hover:text-gray-600 transition-colors
cursor-pointer", title: "Print" }, /* @__PURE__ */ React.createElement(FaPrint, { className: "text-sm" }))))),
/* @__PURE__ */ React.createElement("div", { id: "receipt-print-area" }, /* @__PURE__ */
React.createElement(ReceiptContent, { remit: selectedRemittance }))))));
};
export default CashRemittance;
```

FILE: pos/frontend/src/pages/Inventory.jsx
==========

```
import { memo, useCallback, useEffect, useMemo, useRef, useState } from "react";
import { FaPlus, FaSearch } from "react-icons/fa";
import Header from "../components/shared/header";
import { API_BASE_URL } from "../config/api";
import { useDataCache } from "../context/DataCacheContext";
import { useTheme } from "../context/ThemeContext";
import accessoriesIcon from "../assets/inventory-icons/accesories.svg";
import allIcon from "../assets/inventory-icons/ALL.svg";
import bottomsIcon from "../assets/inventory-icons/Bottoms.svg";
import dressesIcon from "../assets/inventory-icons/dresses.svg";
import headWearIcon from "../assets/inventory-icons/head wear.svg";
import makeupIcon from "../assets/inventory-icons/make up.svg";
import shoesIcon from "../assets/inventory-icons/shoe.svg";
import topIcon from "../assets/inventory-icons/Top.svg";
import AddProductModal from "../components/inventory/AddProductModal";
import ConfirmAddProductModal from "../components/inventory/ConfirmAddProductModal";
import DeleteConfirmationModal from "../components/inventory/DeleteConfirmationModal";
import ArchiveConfirmationModal from "../components/inventory/ArchiveConfirmationModal";
import EditConfirmationModal from "../components/inventory/EditConfirmationModal";
import Pagination from "../components/inventory/Pagination";
import ProductTable from "../components/inventory/ProductTable";
import StockInModal from "../components/inventory/StockInModal";
import StockOutModal from "../components/inventory/StockOutModal";
import SuccessModal from "../components/inventory/SuccessModal";
import ViewProductModal from "../components/inventory/ViewProductModal";
const sumAllVariantQuantities = (variantQuantities) => {
  if (!variantQuantities || typeof variantQuantities !== "object") return 0;
  return Object.values(variantQuantities).reduce((sum, perSize) => {
    if (!perSize || typeof perSize !== "object") return sum;
    return sum + Object.values(perSize).reduce((s, q) => s + (parseInt(q, 10) || 0), 0);
  }, 0);
};
const hasAnyOpeningStockInForm = (np) => {
  if (!np) return false;
  const hasSizeQuantities = np.selectedSizes?.length > 0 && Object.values(np.sizeQuantities || {}).some((qty)
=> parseInt(qty, 10) > 0);
  const hasStock = parseInt(np.currentStock, 10) > 0;
  const hasVariantQuantities = np.variantQuantities && Object.keys(np.variantQuantities).length > 0 &&
Object.values(np.variantQuantities).some(
    (sizeVariants) => sizeVariants && typeof sizeVariants === "object" &&
Object.values(sizeVariants).some((qty) => parseInt(qty, 10) > 0)
```

```
  );
  return hasSizeQuantities || hasVariantQuantities || hasStock;
};
const preferredExportFieldOrder = [
  "_id",
  "sku",
  "itemName",
  "category",
  "subCategory",
  "brandName",
  "variant",
  "itemPrice",
  "costPrice",
  "currentStock",
  "unitOfMeasure",
  "reorderNumber",
  "displayInTerminal",
  "terminalStatus",
  "selectedSizes",
  "sizeQuantities",
  "sizePrices",
  "sizes",
  "differentPricesPerSize",
  "foodSubtype",
  "supplierName",
  "supplierContact",
  "itemImage",
  "dateAdded",
  "lastUpdated",
  "createdAt",
  "updatedAt"
];
const fieldToHeaderMap = {
  _id: "ID",
  sku: "SKU",
  itemName: "Item Name",
  category: "Category",
  subCategory: "SubCategory",
  brandName: "Brand",
  variant: "Variant",
  itemPrice: "Item Price",
  costPrice: "Cost Price",
  currentStock: "Current Stock",
  unitOfMeasure: "Unit of Measure",
  reorderNumber: "Reorder Level",
  displayInTerminal: "Display In Terminal",
  terminalStatus: "Terminal Status",
  selectedSizes: "Selected Sizes",
  sizeQuantities: "Size Quantities",
  sizePrices: "Size Prices",
  sizes: "Sizes",
  differentPricesPerSize: "Different Prices Per Size",
  foodSubtype: "Food Subtype",
  supplierName: "Supplier Name",
  supplierContact: "Supplier Contact",
  itemImage: "Image URL",
  dateAdded: "Date Added",
  lastUpdated: "Last Updated",
  createdAt: "Created At",
  updatedAt: "Updated At",
  stockStatus: "Stock Status"
};
const ADD_PRODUCT_STORAGE_KEY = "addProductFormDraft";
const Inventory = () => {
  const { theme } = useTheme();
  const { cache, getCachedData, setCachedData, isCacheValid, invalidateCache } = useDataCache();
  const [products, setProducts] = useState(
    () => getCachedData("products") || []
  );
  const [selectedCategory, setSelectedCategory] = useState("All");
  const [searchQuery, setSearchQuery] = useState("");
  const [showAddModal, setShowAddModal] = useState(false);
  const [loading, setLoading] = useState(false);
  const [editingProduct, setEditingProduct] = useState(null);
  const [showStockModal, setShowStockModal] = useState(false);
  const [stockModalType, setStockModalType] = useState("in");
  const [stockAmount, setStockAmount] = useState("");
```

```
const [viewingProduct, setViewingProduct] = useState(null);
const [showViewModal, setShowViewModal] = useState(false);
const [currentPage, setCurrentPage] = useState(1);
const [showConfirmModal, setShowConfirmModal] = useState(false);
const [showSuccessModal, setShowSuccessModal] = useState(false);
const [successMessage, setSuccessMessage] = useState("");
const [showDeleteModal, setShowDeleteModal] = useState(false);
const [productToDelete, setProductToDelete] = useState(null);
const [showArchiveModal, setShowArchiveModal] = useState(false);
const [productToArchive, setProductToArchive] = useState(null);
const [showEditModal, setShowEditModal] = useState(false);
const [productToEdit, setProductToEdit] = useState(null);
const [filterBrand, setFilterBrand] = useState("All");
const [filterStatus, setFilterStatus] = useState("All");
const [sortBy, setSortBy] = useState("sku-new");
const [showImportResultModal, setShowImportResultModal] = useState(false);
const [importResult, setImportResult] = useState({
  successCount: 0,
  errorCount: 0,
  errors: []
});
const itemsPerPage = 6;
const [selectedProductIds, setSelectedProductIds] = useState([]);
const [isExportSelectionMode, setIsExportSelectionMode] = useState(false);
const defaultProductState = {
  sku: "",
  itemName: "",
  category: "",
  subCategory: "",
  unitOfMeasure: "pcs",
  brandName: "Default",
  variant: "",
  size: "",
  itemPrice: "",
  costPrice: "",
  currentStock: "",
  reorderNumber: "",
  supplierName: "",
  supplierContact: "",
  itemImage: "",
  selectedSizes: [],
  sizeQuantities: {},
  sizePrices: {},
  sizeCostPrices: {},
  differentPricesPerSize: false,
  foodSubtype: "",
  displayInTerminal: true,
  expirationDate: "",
  expirationThresholdDays: "",
  dateReceived: ""
};
const [newProduct, setNewProduct] = useState(() => {
  try {
    const savedDraft = localStorage.getItem(ADD_PRODUCT_STORAGE_KEY);
    if (savedDraft) {
      const parsed = JSON.parse(savedDraft);
      return { ...defaultProductState, ...parsed };
    }
  } catch (error) {
    console.error("Error loading product draft from localStorage:", error);
  }
  return defaultProductState;
});
const [selectedMainCategory, setSelectedMainCategory] = useState("All");
const [selectedSubCategory, setSelectedSubCategory] = useState("");
const defaultCategories = [
  { name: "All", icon: allIcon }
];
const categoryStructureOptions = [
  "Tops",
  "Bottoms",
  "Dresses",
  "Outerwear",
  "Beverages",
  "Snacks",
  "Meals",
  "Desserts",
```

```
    "Ingredients",
    "Other",
    "Face",
    "Eyes",
    "Lips",
    "Nails",
    "SkinCare",
    "Others",
    "Jewelry",
    "Bags",
    "Head Wear",
    "Glasses/Sunglasses",
    "Sneakers",
    "Boots",
    "Sandals",
    "Daily Essentials",
    "Personal Care",
    "Home Essentials"
  ];
  const categoryStructure = {
    "Apparel - Men": ["Tops", "Bottoms", "Outerwear"],
    "Apparel - Women": ["Tops", "Bottoms", "Dresses", "Outerwear"],
    "Apparel - Kids": ["Tops", "Bottoms", "Dresses", "Outerwear"],
    "Apparel - Unisex": ["Tops", "Bottoms", "Dresses", "Outerwear"],
    "Foods": ["Beverages", "Snacks", "Meals", "Desserts", "Ingredients", "Other"],
    "Makeup": ["Face", "Eyes", "Lips", "Nails", "SkinCare", "Others"],
    "Accessories": ["Jewelry", "Bags", "Head Wear", "Glasses/Sunglasses", "Others"],
    "Shoes": ["Sneakers", "Boots", "Sandals", "Others"],
    "Essentials": ["Daily Essentials", "Personal Care", "Home Essentials", "Others"]
  };
  const getParentCategoryForLegacy = (name) => {
    for (const [parent, subs] of Object.entries(categoryStructure)) {
      if (subs.includes(name)) return parent;
    }
    return null;
  };
  const [mainCategories, setMainCategories] = useState(defaultCategories);
  const [subCategories, setSubCategories] = useState([]);
  const [categories, setCategories] = useState([{ name: "All", icon: allIcon }]);
  const [brandPartners, setBrandPartners] = useState([]);
  useEffect(() => {
    if (!editingProduct && showAddModal) {
      try {
        const hasData = newProduct.itemName || newProduct.variant || newProduct.itemPrice ||
newProduct.costPrice || newProduct.currentStock || newProduct.itemImage || newProduct.selectedSizes &&
newProduct.selectedSizes.length > 0;
        if (hasData) {
          localStorage.setItem(
            ADD_PRODUCT_STORAGE_KEY,
            JSON.stringify(newProduct)
          );
        }
      } catch (error) {
        console.error("Error saving product draft to localStorage:", error);
      }
    }
  }, [newProduct, editingProduct, showAddModal]);
  const addProductHasOpeningStock = useMemo(
    () => hasAnyOpeningStockInForm(newProduct),
    [newProduct]
  );
  const categoryIconMap = {
    Tops: topIcon,
    Bottoms: bottomsIcon,
    Dresses: dressesIcon,
    Makeup: makeupIcon,
    Accessories: accessoriesIcon,
    Essentials: accessoriesIcon,
    Shoes: shoesIcon,
    "Head Wear": headWearIcon
  };
  const fetchCategories = async () => {
    try {
      const response = await fetch(`${API_BASE_URL}/api/categories`);
      const data = await response.json();
      if (data.success && Array.isArray(data.data)) {
        const activeDbCats = data.data.filter((cat) => cat.status === "active");
```

```
        const dbMainCats = activeDbCats.filter((c) => c.type !== "subcategory"
&& !categoryStructureOptions.includes(c.name)).map((cat) => ({ name: cat.name, icon: categoryIconMap[cat.name]
|| allIcon }));
        const rawSubCats = activeDbCats.filter((c) => c.type === "subcategory" ||
categoryStructureOptions.includes(c.name));
        const dbSubCats = [];
        for (const [parent, subs] of Object.entries(categoryStructure)) {
          subs.forEach((subName) => {
            const dbSub = rawSubCats.find((s) => s.name === subName);
            if (dbSub) {
              dbSubCats.push({ name: dbSub.name, parentCategory: parent });
            }
          });
        }
        rawSubCats.forEach((sub) => {
          if (!categoryStructureOptions.includes(sub.name)) {
            dbSubCats.push({ name: sub.name, parentCategory: sub.parentCategory });
          }
        });
        const mergedMainCategories = [...defaultCategories];
        const defaultNames = new Set(defaultCategories.map((c) => c.name));
        dbMainCats.forEach((cat) => {
          if (!defaultNames.has(cat.name)) {
            mergedMainCategories.push(cat);
          }
        });
        setMainCategories(mergedMainCategories);
        setSubCategories(dbSubCats);
        const activeCategories = data.data.filter((cat) => cat.status === "active" && cat.name !==
"Others").map((cat) => ({
          name: cat.name,
          icon: categoryIconMap[cat.name] || allIcon,
          parentCategory: cat.parentCategory || null
        }));
        const withEssentials = activeCategories.some((cat) => cat.name === "Essentials") ? activeCategories :
[...activeCategories, { name: "Essentials", icon: categoryIconMap.Essentials, parentCategory: null }];
        setCategories([{ name: "All", icon: allIcon, parentCategory: null }, ...withEssentials]);
      }
    } catch (error) {
      console.error("Error fetching categories:", error);
      setMainCategories(defaultCategories);
      setSubCategories([]);
      setCategories(
        [
          { name: "All", icon: allIcon, parentCategory: null },
          { name: "Tops", icon: topIcon, parentCategory: null },
          { name: "Bottoms", icon: bottomsIcon, parentCategory: null },
          { name: "Dresses", icon: dressesIcon, parentCategory: null },
          { name: "Makeup", icon: makeupIcon, parentCategory: null },
          { name: "Accessories", icon: accessoriesIcon, parentCategory: null },
          { name: "Essentials", icon: accessoriesIcon, parentCategory: null },
          { name: "Shoes", icon: shoesIcon, parentCategory: null },
          { name: "Head Wear", icon: headWearIcon, parentCategory: null }
        ]
      );
    }
  };
  const fetchBrandPartners = async () => {
    try {
      const response = await fetch(`${API_BASE_URL}/api/brand-partners`);
      const data = await response.json();
      if (data.success) {
        setBrandPartners(data.data || []);
      }
    } catch (error) {
      console.error("Error fetching brand partners:", error);
    }
  };
  const categoryCodeMap = {
    Tops: "TOP",
    Bottoms: "BOT",
    Dresses: "DRS",
    Makeup: "MUA",
    Accessories: "MUA",
    Essentials: "ESS",
    Shoes: "SHO",
    "Head Wear": "HDW",
```

```
  Foods: "FOD"
};
const getNextIncrementalCode = () => {
  const numericCodes = products.map((p) => {
    const match = p.sku?.match(/-(\d{5})-?/);
    return match ? parseInt(match[1]) : 0;
  }).filter((num) => num > 0);
  const maxCode = numericCodes.length > 0 ? Math.max(...numericCodes) : 0;
  return String(maxCode + 1).padStart(5, "0");
};
const getColorCode = (variant) => {
  if (!variant || variant.trim() === "") {
    return "XXX";
  }
  const cleaned = variant.replace(/\s+/g, "").toUpperCase();
  return cleaned.substring(0, 3).padEnd(3, "X");
};
const generateSKU = (category, subCategory, variant) => {
  const base = subCategory || category;
  const categoryCode = categoryCodeMap[base] || "OTH";
  const chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
  let randomCode = "";
  for (let i = 0; i < 6; i++) {
    randomCode += chars.charAt(Math.floor(Math.random() * chars.length));
  }
  if (!variant || variant.trim() === "") {
    return `${categoryCode}-${randomCode}`;
  }
  const colorCode = getColorCode(variant);
  return `${categoryCode}-${randomCode}-${colorCode}`;
};
useEffect(() => {
  const cachedProducts = getCachedData("products");
  if (!cachedProducts || !isCacheValid("products")) {
    fetchProducts();
  } else {
    setProducts(cachedProducts);
  }
  fetchCategories();
  fetchBrandPartners();
}, [getCachedData, isCacheValid]);
const getSkuNumber = (sku = "") => {
  const matches = sku.match(/\d+/g);
  if (!matches || matches.length === 0) return -Infinity;
  return parseInt(matches[matches.length - 1], 10);
};
const filteredProducts = useMemo(
  () => {
    let filtered = products;
    if (selectedMainCategory !== "All") {
      const matchMain = selectedMainCategory.toLowerCase().trim();
      if (selectedSubCategory && selectedSubCategory !== "All") {
        const matchSub = selectedSubCategory.toLowerCase().trim();
        filtered = filtered.filter((product) => {
          const pCat = (product.category || "").toLowerCase().trim();
          const pSubCat = (product.subCategory || "").toLowerCase().trim();
          return pCat === matchMain && pSubCat === matchSub;
        });
      } else {
        filtered = filtered.filter((product) => {
          const pCat = (product.category || "").toLowerCase().trim();
          return pCat === matchMain;
        });
      }
    }
    if (filterBrand !== "All") {
      filtered = filtered.filter(
        (product) => product.brandName === filterBrand
      );
    }
    if (filterStatus !== "All") {
      filtered = filtered.filter((product) => {
        const reorderLevel = product.reorderNumber || 10;
        if (filterStatus === "In Stock") {
          return product.currentStock > reorderLevel;
        } else if (filterStatus === "Low Stock") {
          return product.currentStock > 0 && product.currentStock <= reorderLevel;
```

```
          } else if (filterStatus === "Out of Stock") {
            return product.currentStock === 0;
          }
          return true;
        });
      }
      if (searchQuery) {
        filtered = filtered.filter(
          (product) => product.itemName.toLowerCase().includes(searchQuery.toLowerCase()) ||
product.sku.toLowerCase().includes(searchQuery.toLowerCase()) || product.category &&
product.category.toLowerCase().includes(searchQuery.toLowerCase()) || product.subCategory &&
product.subCategory.toLowerCase().includes(searchQuery.toLowerCase()) || product.brandName &&
product.brandName.toLowerCase().includes(searchQuery.toLowerCase())
        );
      }
      filtered = [...filtered].sort((a, b) => {
        switch (sortBy) {
          case "sku-new":
            return getSkuNumber(b.sku) - getSkuNumber(a.sku);
          case "sku-old":
            return getSkuNumber(a.sku) - getSkuNumber(b.sku);
          case "name":
            return a.itemName.localeCompare(b.itemName);
          case "price-low":
            return a.itemPrice - b.itemPrice;
          case "price-high":
            return b.itemPrice - a.itemPrice;
          case "stock-low":
            return a.currentStock - b.currentStock;
          case "stock-high":
            return b.currentStock - a.currentStock;
          case "date-new":
            return new Date(b.dateAdded) - new Date(a.dateAdded);
          case "date-old":
            return new Date(a.dateAdded) - new Date(b.dateAdded);
          default:
            return 0;
        }
      });
      const uniqueProducts = filtered.filter(
        (product, index, self) => index === self.findIndex((p) => p._id === product._id)
      );
      return uniqueProducts;
    },
    [
      products,
      selectedMainCategory,
      selectedSubCategory,
      filterBrand,
      filterStatus,
      searchQuery,
      sortBy
    ]
  );
  const stockStats = useMemo(() => {
    const totalItems = products.length;
    const outOfStockItems = products.filter((p) => p.currentStock === 0).length;
    const lowStockItems = products.filter((p) => {
      const reorderLevel = p.reorderNumber || 10;
      return p.currentStock > 0 && p.currentStock <= reorderLevel;
    }).length;
    const inStockItems = products.filter((p) => {
      const reorderLevel = p.reorderNumber || 10;
      return p.currentStock > reorderLevel;
    }).length;
    return { totalItems, lowStockItems, outOfStockItems, inStockItems };
  }, [products]);
  const uniqueBrands = useMemo(() => {
    return [
      ...new Set(products.map((p) => p.brandName).filter(Boolean))
    ].sort();
  }, [products]);
  const paginatedProducts = useMemo(() => {
    return filteredProducts.slice(
      (currentPage - 1) * itemsPerPage,
      currentPage * itemsPerPage
    );
```

```
  }, [filteredProducts, currentPage, itemsPerPage]);
  const paginatedProductIds = useMemo(
    () => paginatedProducts.map((product) => product._id),
    [paginatedProducts]
  );
  const allVisibleSelected = paginatedProductIds.length > 0 && paginatedProductIds.every((id) =>
selectedProductIds.includes(id));
  const someVisibleSelected = paginatedProductIds.some(
    (id) => selectedProductIds.includes(id)
  );
  useEffect(() => {
    setCurrentPage(1);
  }, [selectedMainCategory, selectedSubCategory, filterBrand, filterStatus, searchQuery, sortBy]);
  useEffect(() => {
    const totalPages = Math.max(
      1,
      Math.ceil(filteredProducts.length / itemsPerPage)
    );
    if (currentPage > totalPages) setCurrentPage(totalPages);
  }, [filteredProducts.length, itemsPerPage, currentPage]);
  useEffect(() => {
    setSelectedProductIds(
      (prev) => prev.filter(
        (id) => filteredProducts.some((product) => product._id === id)
      )
    );
  }, [filteredProducts]);
  const fetchProducts = async (opts) => {
    const silent = typeof opts === "boolean" ? opts : opts?.silent;
    try {
      if (!silent) setLoading(true);
      const response = await fetch(`${API_BASE_URL}/api/products`, {
        cache: "no-store"
      });
      const data = await response.json();
      if (data.success) {
        setProducts(data.data);
        setCachedData("products", data.data);
      }
    } catch (error) {
      console.error("Error fetching products:", error);
      alert(
        "Failed to fetch products. Make sure the backend server is running."
      );
    } finally {
      if (!silent) setLoading(false);
    }
  };
  const fetchProductsRef = useRef(fetchProducts);
  fetchProductsRef.current = fetchProducts;
  useEffect(() => {
    const syncProducts = () => {
      invalidateCache("products");
      fetchProductsRef.current({ silent: true });
    };
    window.addEventListener("pos-invalidate-products", syncProducts);
    const onStorage = (e) => {
      if (e.key === "pos-products-bump" && e.newValue != null) {
        syncProducts();
      }
    };
    window.addEventListener("storage", onStorage);
    return () => {
      window.removeEventListener("pos-invalidate-products", syncProducts);
      window.removeEventListener("storage", onStorage);
    };
  }, [invalidateCache]);
  const handleInputChange = (e) => {
    const { name, value, type, checked } = e.target;
    const inputValue = type === "checkbox" ? checked : value;
    if (name === "displayInTerminal" && type === "checkbox") {
      setNewProduct((prev) => ({
        ...prev,
        displayInTerminal: checked
      }));
      return;
    }
```

```
    setNewProduct((prev) => {
      const updatedProduct = {
        ...prev,
        [name]: inputValue
      };
      if (!editingProduct && (name === "category" || name === "subCategory" || name === "variant")) {
        const category = name === "category" ? inputValue : prev.category;
        const subCategory = name === "subCategory" ? inputValue : prev.subCategory;
        const variant = name === "variant" ? inputValue : prev.variant;
        updatedProduct.sku = generateSKU(category, subCategory, variant);
      }
      return updatedProduct;
    });
  };
  const handleSizeToggle = (size) => {
    setNewProduct((prev) => {
      const isSelected = prev.selectedSizes.includes(size);
      const newSelectedSizes = isSelected ? prev.selectedSizes.filter((s) => s !== size) :
[...prev.selectedSizes, size];
      const newSizeQuantities = { ...prev.sizeQuantities };
      const newSizePrices = { ...prev.sizePrices };
      if (isSelected) {
        delete newSizeQuantities[size];
        delete newSizePrices[size];
      } else {
        newSizeQuantities[size] = "";
        if (prev.differentPricesPerSize) {
          newSizePrices[size] = prev.itemPrice || "";
        }
      }
      return {
        ...prev,
        selectedSizes: newSelectedSizes,
        sizeQuantities: newSizeQuantities,
        sizePrices: newSizePrices
      };
    });
  };
  const handleSizeQuantityChange = (size, quantity) => {
    setNewProduct((prev) => ({
      ...prev,
      sizeQuantities: {
        ...prev.sizeQuantities,
        [size]: parseInt(quantity) || 0
      }
    }));
  };
  const handleSizePriceChange = (size, price) => {
    setNewProduct((prev) => ({
      ...prev,
      sizePrices: {
        ...prev.sizePrices,
        [size]: price
      }
    }));
  };
  const resetProductForm = (clearStorage = true) => {
    const defaultCategory = "Tops";
    const defaultSubCategory = "";
    const defaultVariant = "";
    setNewProduct({
      sku: generateSKU(defaultCategory, defaultSubCategory, defaultVariant),
      itemName: "",
      category: "",
      subCategory: "",
      unitOfMeasure: "pcs",
      brandName: "Default",
      variant: defaultVariant,
      size: "",
      itemPrice: "",
      costPrice: "",
      currentStock: "",
      reorderNumber: "",
      supplierName: "",
      supplierContact: "",
      itemImage: "",
      selectedSizes: [],
```

```
      sizeQuantities: {},
      sizePrices: {},
      differentPricesPerSize: false,
      foodSubtype: "",
      displayInTerminal: true,
      expirationDate: "",
      expirationThresholdDays: "",
      dateReceived: ""
    });
    setEditingProduct(null);
    if (clearStorage) {
      try {
        localStorage.removeItem(ADD_PRODUCT_STORAGE_KEY);
      } catch (error) {
        console.error("Error clearing product draft from localStorage:", error);
      }
    }
  };
  const handleAddProduct = async (e) => {
    e.preventDefault();
    if (!editingProduct) {
      const hasVariantQuantities = newProduct.variantQuantities &&
Object.keys(newProduct.variantQuantities).length > 0 && Object.values(newProduct.variantQuantities).some(
        (sizeVariants) => sizeVariants && typeof sizeVariants === "object" &&
Object.values(sizeVariants).some((qty) => parseInt(qty) > 0)
      );
      const hasAnyVariantPricing = newProduct.selectedSizes?.some(
        (size) => newProduct.differentPricesPerVariant?.[size]
      );
      const hasVariantPrices = newProduct.variantPrices && Object.keys(newProduct.variantPrices).length > 0
&& Object.values(newProduct.variantPrices).some(
        (sizeVariants) => sizeVariants && Object.values(sizeVariants).some((price) => parseFloat(price) > 0)
      );
      const hasVariantQuantitiesForConfirm = hasVariantQuantities;
      if (hasAnyVariantPricing || hasVariantPrices || hasVariantQuantitiesForConfirm) {
        setShowConfirmModal(true);
        return;
      }
      setShowConfirmModal(true);
    } else {
      setProductToEdit(editingProduct);
      setShowEditModal(true);
    }
  };
  const confirmAddProduct = async () => {
    setShowConfirmModal(false);
    const variantQtySum = sumAllVariantQuantities(newProduct.variantQuantities);
    const sizeQtySum = Object.values(newProduct.sizeQuantities || {}).reduce(
      (sum, qty) => sum + (parseInt(qty, 10) || 0),
      0
    );
    const totalStock = variantQtySum > 0 ? variantQtySum : (newProduct.selectedSizes || []).length > 0 ?
sizeQtySum : parseInt(newProduct.currentStock, 10) || 0;
    try {
      setLoading(true);
      const currentUser = JSON.parse(localStorage.getItem("currentUser") || "{}");
      const performedByName = currentUser.name || `${currentUser.firstName || ""} ${currentUser.lastName ||
""}`.trim() || "System";
      const performedById = currentUser._id || currentUser.id || "";
      const url = editingProduct ? `${API_BASE_URL}/api/products/${editingProduct._id}` :
`${API_BASE_URL}/api/products`;
      const method = editingProduct ? "PUT" : "POST";
      let defaultItemPrice = parseFloat(newProduct.itemPrice) || 0;
      if (!defaultItemPrice && newProduct.differentPricesPerSize && newProduct.selectedSizes?.length > 0) {
        const firstSizePrice = newProduct.sizePrices?.[newProduct.selectedSizes[0]];
        if (firstSizePrice) {
          defaultItemPrice = parseFloat(firstSizePrice) || 0;
        }
      }
      if (!defaultItemPrice && newProduct.variantPrices) {
        const firstSizeWithVariantPrices = Object.keys(newProduct.variantPrices)[0];
        if (firstSizeWithVariantPrices) {
          const firstVariantPrice = Object.values(newProduct.variantPrices[firstSizeWithVariantPrices])[0];
          if (firstVariantPrice) {
            defaultItemPrice = parseFloat(firstVariantPrice) || 0;
          }
        }
```

```
      }
      const payload = {
        ...newProduct,
        handledBy: performedByName,
        handledById: performedById,
        performedByName,
        performedById,
        itemPrice: defaultItemPrice,
        costPrice: parseFloat(newProduct.costPrice) || 0,
        reorderNumber: parseInt(newProduct.reorderNumber) || 0,
        expirationDate: newProduct.expirationDate || null,
        expirationThresholdDays: newProduct.expirationDate ? Math.max(0,
parseInt(newProduct.expirationThresholdDays) || 30) : 0,
        displayInTerminal: newProduct.displayInTerminal !== false
      };
      if (!editingProduct) {
        payload.currentStock = totalStock;
        const selected = newProduct.selectedSizes || [];
        const sizesList = selected.length > 0 ? selected : Object.keys(newProduct.variantQuantities ||
{}).length > 0 ? Object.keys(newProduct.variantQuantities) : [];
        if (sizesList.length > 0) {
          const hasVariantQuantities = newProduct.variantQuantities &&
Object.keys(newProduct.variantQuantities).length > 0;
          const hasVariantPrices = newProduct.variantPrices && Object.keys(newProduct.variantPrices).length >
0;
          if (newProduct.differentPricesPerSize && Object.keys(newProduct.sizePrices).length > 0) {
            const sizesWithPrices = {};
            sizesList.forEach((size) => {
              const sizePrice = parseFloat(newProduct.sizePrices[size]);
              let variantValue = "";
              if (newProduct.differentVariantsPerSize) {
                if (newProduct.multipleVariantsPerSize?.[size]) {
                  variantValue = newProduct.sizeMultiVariants?.[size] || [];
                } else {
                  variantValue = newProduct.sizeVariants?.[size] || "";
                }
              } else {
                variantValue = newProduct.variant || "";
              }
              if (hasVariantQuantities && newProduct.variantQuantities[size]) {
                sizesWithPrices[size] = {
                  quantity: Object.values(newProduct.variantQuantities[size]).reduce((sum, q) => sum +
(parseInt(q) || 0), 0),
                  price: sizePrice || defaultItemPrice || 0,
                  variant: variantValue,
                  variants: newProduct.variantQuantities[size]
                };
                if (hasVariantPrices && newProduct.variantPrices[size]) {
                  sizesWithPrices[size].variantPrices = newProduct.variantPrices[size];
                }
              } else {
                sizesWithPrices[size] = {
                  quantity: newProduct.sizeQuantities[size] || 0,
                  price: sizePrice || defaultItemPrice || 0,
                  variant: variantValue
                };
              }
            });
            payload.sizes = sizesWithPrices;
          } else {
            const sizesObject = {};
            const hasVariantCostPrices = newProduct.variantCostPrices &&
Object.keys(newProduct.variantCostPrices).length > 0;
            sizesList.forEach((size) => {
              let variantValue = "";
              if (newProduct.differentVariantsPerSize) {
                if (newProduct.multipleVariantsPerSize?.[size]) {
                  variantValue = newProduct.sizeMultiVariants?.[size] || [];
                } else {
                  variantValue = newProduct.sizeVariants?.[size] || "";
                }
              } else {
                variantValue = newProduct.variant || "";
              }
              const sizePrice = parseFloat(newProduct.sizePrices?.[size]) || 0;
              const sizeCostPrice = parseFloat(newProduct.sizeCostPrices?.[size]) || 0;
              if (hasVariantQuantities && newProduct.variantQuantities[size]) {
```

```
                sizesObject[size] = {
                  quantity: Object.values(newProduct.variantQuantities[size]).reduce((sum, q) => sum +
(parseInt(q) || 0), 0),
                  variant: variantValue,
                  variants: newProduct.variantQuantities[size]
                };
                if (hasVariantPrices && newProduct.variantPrices[size]) {
                  sizesObject[size].variantPrices = newProduct.variantPrices[size];
                  if (hasVariantCostPrices && newProduct.variantCostPrices[size]) {
                    sizesObject[size].variantCostPrices = newProduct.variantCostPrices[size];
                  }
                } else {
                  if (sizePrice > 0) {
                    sizesObject[size].price = sizePrice;
                  }
                  if (sizeCostPrice > 0) {
                    sizesObject[size].costPrice = sizeCostPrice;
                  }
                }
              } else {
                sizesObject[size] = {
                  quantity: newProduct.sizeQuantities[size] || 0,
                  variant: variantValue
                };
                if (sizePrice > 0) {
                  sizesObject[size].price = sizePrice;
                }
                if (sizeCostPrice > 0) {
                  sizesObject[size].costPrice = sizeCostPrice;
                }
              }
            });
            payload.sizes = sizesObject;
          }
        } else {
          payload.sizes = null;
        }
      }
      const response = await fetch(url, {
        method,
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify(payload)
      });
      if (!response.ok) {
        if (response.status === 413) {
          alert(
            "Image file is too large. Please use a smaller image or compress it before uploading."
          );
          return;
        }
        const errorText = await response.text();
        let errorMessage = `Failed to ${editingProduct ? "update" : "add"} product`;
        try {
          const errorData = JSON.parse(errorText);
          errorMessage = errorData.message || errorMessage;
        } catch (e) {
          errorMessage = `Server error (${response.status}): ${errorText.substring(0, 100)}`;
        }
        alert(errorMessage);
        return;
      }
      const data = await response.json();
      if (data.success) {
        setShowAddModal(false);
        resetProductForm();
        if (data.data) {
          setProducts((prev) => {
            const next = [data.data, ...prev || []];
            setCachedData("products", next);
            return next;
          });
          fetchProducts({ silent: true });
        } else {
          invalidateCache("products");
          fetchProducts();
```

```
        }
        setSuccessMessage("The item was added successfully!");
        setShowSuccessModal(true);
      } else {
        alert(
          data.message || `Failed to ${editingProduct ? "update" : "add"} product`
        );
      }
    } catch (error) {
      console.error("Error saving product:", error);
      alert(
        "Failed to save product. Please check your connection and try again."
      );
    } finally {
      setLoading(false);
    }
  };
  useEffect(() => {
    if (Array.isArray(cache?.products) && cache.products.length) {
      setProducts(cache.products);
    }
  }, [cache?.products]);
  const handleEditClick = (product) => {
    setEditingProduct(product);
    const existingSizes = product.sizes ? Object.keys(product.sizes) : [];
    let existingSizeQuantities = {};
    let existingSizePrices = {};
    let hasDifferentPrices = false;
    if (product.sizes) {
      existingSizes.forEach((size) => {
        if (typeof product.sizes[size] === "object" && product.sizes[size] !== null) {
          existingSizeQuantities[size] = product.sizes[size].quantity || 0;
          if (product.sizes[size].price !== void 0) {
            existingSizePrices[size] = product.sizes[size].price;
            hasDifferentPrices = true;
          }
        } else {
          existingSizeQuantities[size] = product.sizes[size] || 0;
        }
      });
    }
    setNewProduct({
      sku: product.sku || "",
      itemName: product.itemName || "",
      category: product.category || "",
      subCategory: product.subCategory || "",
      unitOfMeasure: product.unitOfMeasure || "pcs",
      brandName: product.brandName || "Default",
      variant: product.variant || "",
      size: product.size || "",
      itemPrice: product.itemPrice || "",
      costPrice: product.costPrice || "",
      currentStock: product.currentStock || "",
      reorderNumber: product.reorderNumber || "",
      supplierName: product.supplierName || "",
      supplierContact: product.supplierContact || "",
      itemImage: product.itemImage || "",
      selectedSizes: existingSizes,
      sizeQuantities: existingSizeQuantities,
      sizePrices: existingSizePrices,
      differentPricesPerSize: hasDifferentPrices,
      foodSubtype: product.foodSubtype || "",
      expirationDate: product.expirationDate ? new Date(product.expirationDate).toISOString().slice(0, 10) :
"",
      expirationThresholdDays: product.expirationThresholdDays !== void 0 &&
product.expirationThresholdDays !== null ? String(product.expirationThresholdDays) : "",
      displayInTerminal: product.displayInTerminal !== void 0 ? product.displayInTerminal : true
    });
    setShowAddModal(true);
  };
  const confirmEditProduct = async () => {
    setShowEditModal(false);
    if (!editingProduct) return;
    try {
      setLoading(true);
      const url = `${API_BASE_URL}/api/products/${editingProduct._id}`;
      const displayInTerminalValue = newProduct.displayInTerminal === false ? false : true;
```

```
      const payload = {
        ...newProduct,
        itemPrice: parseFloat(newProduct.itemPrice) || 0,
        costPrice: parseFloat(newProduct.costPrice) || 0,
        reorderNumber: parseInt(newProduct.reorderNumber) || 0,
        expirationDate: newProduct.expirationDate || null,
        expirationThresholdDays: newProduct.expirationDate ? Math.max(0,
parseInt(newProduct.expirationThresholdDays) || 30) : 0,
        displayInTerminal: displayInTerminalValue
      };
      console.log(
        "[confirmEditProduct] newProduct.displayInTerminal:",
        newProduct.displayInTerminal
      );
      console.log(
        "[confirmEditProduct] Sending displayInTerminal:",
        payload.displayInTerminal
      );
      delete payload.currentStock;
      delete payload.selectedSizes;
      delete payload.sizeQuantities;
      delete payload.sizePrices;
      delete payload.differentPricesPerSize;
      if (payload.editableSizePrices && Object.keys(payload.editableSizePrices).length > 0) {
        const updatedSizes = { ...editingProduct.sizes };
        Object.entries(payload.editableSizePrices).forEach(([size, sizeData]) => {
          if (updatedSizes[size]) {
            if (sizeData.hasVariants && sizeData.variants) {
              const currentSizeData = updatedSizes[size];
              const currentVariants = currentSizeData.variants || {};
              const currentVariantPrices = currentSizeData.variantPrices || {};
              const currentVariantCostPrices = currentSizeData.variantCostPrices || {};
              Object.entries(sizeData.variants).forEach(([variant, variantData]) => {
                const newPrice = parseFloat(variantData.price) || 0;
                const newCostPrice = parseFloat(variantData.costPrice) || 0;
                if (currentVariants[variant] !== void 0) {
                  if (typeof currentVariants[variant] === "number") {
                    currentVariantPrices[variant] = newPrice;
                    currentVariantCostPrices[variant] = newCostPrice;
                  } else if (typeof currentVariants[variant] === "object") {
                    currentVariants[variant] = {
                      ...currentVariants[variant],
                      price: newPrice,
                      costPrice: newCostPrice
                    };
                  }
                }
              });
              updatedSizes[size] = {
                ...currentSizeData,
                variants: currentVariants,
                variantPrices: Object.keys(currentVariantPrices).length > 0 ? currentVariantPrices :
currentSizeData.variantPrices,
                variantCostPrices: Object.keys(currentVariantCostPrices).length > 0 ?
currentVariantCostPrices : currentSizeData.variantCostPrices
              };
            } else {
              updatedSizes[size] = {
                ...updatedSizes[size],
                price: parseFloat(sizeData.price) || 0,
                costPrice: parseFloat(sizeData.costPrice) || 0
              };
            }
          }
        });
        payload.sizes = updatedSizes;
        delete payload.editableSizePrices;
      } else {
        delete payload.sizes;
      }
      const response = await fetch(url, {
        method: "PUT",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify(payload)
      });
```

```
      if (!response.ok) {
        if (response.status === 413) {
          alert(
            "Image file is too large. Please use a smaller image or compress it before uploading."
          );
          return;
        }
        const errorText = await response.text();
        let errorMessage = "Failed to update product";
        try {
          const errorData = JSON.parse(errorText);
          errorMessage = errorData.message || errorData.error || errorMessage;
          if (errorData.error && errorData.error !== errorData.message) {
            errorMessage += `: ${errorData.error}`;
          }
        } catch (e) {
          errorMessage = `Server error (${response.status}): ${errorText.substring(0, 100)}`;
        }
        console.error("Product update error:", errorMessage);
        alert(errorMessage);
        return;
      }
      const data = await response.json();
      if (data.success) {
        setShowAddModal(false);
        resetProductForm();
        invalidateCache("products");
        fetchProducts();
        setSuccessMessage("The item was edited successfully!");
        setShowSuccessModal(true);
        setProductToEdit(null);
      } else {
        alert(data.message || "Failed to update product");
      }
    } catch (error) {
      console.error("Error updating product:", error);
      alert(
        "Failed to update product. Please check your connection and try again."
      );
    } finally {
      setLoading(false);
    }
  };
  const handleViewProduct = useCallback(async (product) => {
    setViewingProduct(product);
    setShowViewModal(true);
    try {
      const response = await fetch(
        `${API_BASE_URL}/api/products/${product._id}`
      );
      const data = await response.json();
      if (data.success && data.data) {
        setViewingProduct(data.data);
      }
    } catch (error) {
      console.error("Error fetching product details:", error);
    }
  }, []);
  const handleDeleteClick = (product) => {
    setProductToDelete(product._id);
    setShowDeleteModal(true);
  };
  const handleArchiveClick = (product) => {
    setProductToArchive(product._id);
    setShowArchiveModal(true);
  };
  const confirmArchiveProduct = async () => {
    if (!productToArchive) return;
    const archiveId = String(productToArchive);
    setShowArchiveModal(false);
    try {
      const currentUser = JSON.parse(
        localStorage.getItem("currentUser") || "{}"
      );
      const archivedByName = currentUser.name || `${currentUser.firstName || ""} ${currentUser.lastName ||
""}`.trim() || "Owner";
      const archivedById = currentUser._id || currentUser.id || "";
```

```
      const response = await fetch(
        `${API_BASE_URL}/api/products/${archiveId}/archive`,
        {
          method: "PATCH",
          cache: "no-store",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ archivedByName, archivedById })
        }
      );
      const data = await response.json();
      if (data.success) {
        setProducts(
          (prev) => prev.filter((p) => String(p._id || p.id) !== archiveId)
        );
        setShowSuccessModal(true);
        setSuccessMessage("The item was archived successfully!");
        invalidateCache("products");
        fetchProducts({ silent: true });
      } else {
        alert(data.message || "Failed to archive product");
      }
    } catch (error) {
      console.error("Error archiving product:", error);
      alert("Failed to archive product. Please try again.");
    } finally {
      setProductToArchive(null);
    }
  };
  const confirmDeleteProduct = async () => {
    if (!productToDelete) return;
    setShowDeleteModal(false);
    try {
      setLoading(true);
      const response = await fetch(
        `${API_BASE_URL}/api/products/${productToDelete}`,
        {
          method: "DELETE"
        }
      );
      const data = await response.json();
      if (data.success) {
        setShowSuccessModal(true);
        setSuccessMessage("The item was deleted successfully!");
        invalidateCache("products");
        fetchProducts();
      } else {
        alert(data.message || "Failed to delete product");
      }
    } catch (error) {
      console.error("Error deleting product:", error);
      alert("Failed to delete product. Please try again.");
    } finally {
      setLoading(false);
      setProductToDelete(null);
    }
  };
  const handleStockUpdate = async (product, type) => {
    setEditingProduct(product);
    setStockModalType(type);
    setStockAmount("");
    setShowStockModal(true);
  };
  const handleStockSubmit = async (e) => {
    e.preventDefault();
    if (!stockAmount || parseInt(stockAmount) <= 0) {
      alert("Please enter a valid quantity");
      return;
    }
    try {
      setLoading(true);
      const amount = parseInt(stockAmount);
      const newStock = stockModalType === "in" ? editingProduct.currentStock + amount : Math.max(0,
editingProduct.currentStock - amount);
      const response = await fetch(
        `${API_BASE_URL}/api/products/${editingProduct._id}`,
        {
          method: "PUT",
```

```
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify({
            currentStock: newStock
          })
        }
      );
      const data = await response.json();
      if (data.success) {
        alert(
          `Stock ${stockModalType === "in" ? "added" : "removed"} successfully!`
        );
        setShowStockModal(false);
        setEditingProduct(null);
        setStockAmount("");
        invalidateCache("products");
        fetchProducts();
      } else {
        alert(data.message || "Failed to update stock");
      }
    } catch (error) {
      console.error("Error updating stock:", error);
      alert("Failed to update stock. Please try again.");
    } finally {
      setLoading(false);
    }
  };
  const handleStockInConfirm = async (stockData) => {
    if (!editingProduct) return;
    try {
      setLoading(true);
      const currentUser = JSON.parse(
        localStorage.getItem("currentUser") || "{}"
      );
      const handledBy = currentUser.name || `${currentUser.firstName || ""} ${currentUser.lastName ||
""}`.trim() || "System";
      const handledById = currentUser._id || currentUser.id || "";
      if (stockData.noSizes) {
        const response = await fetch(`${API_BASE_URL}/api/products/${editingProduct._id}/stock-in`, {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            ...stockData,
            handledBy,
            handledById
          })
        });
        const data = await response.json();
        if (!response.ok || !data.success) {
          alert(data.message || "Failed to update stock");
          return;
        }
      } else {
        const response = await fetch(`${API_BASE_URL}/api/products/${editingProduct._id}/stock-in`, {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            ...stockData,
            handledBy,
            handledById
          })
        });
        const data = await response.json();
        if (!response.ok || !data.success) {
          alert(data.message || "Failed to update stock");
          return;
        }
      }
      setShowStockModal(false);
      setEditingProduct(null);
      setStockAmount("");
      setSuccessMessage("Stock added successfully!");
      setShowSuccessModal(true);
      invalidateCache("products");
      fetchProducts();
    } catch (error) {
```

```
      console.error("Error updating stock:", error);
      alert("Failed to update stock. Please try again.");
    } finally {
      setLoading(false);
    }
  };
  const handleStockOutConfirm = async (stockData) => {
    if (!editingProduct) return;
    try {
      setLoading(true);
      const currentUser = JSON.parse(
        localStorage.getItem("currentUser") || "{}"
      );
      const handledBy = currentUser.name || `${currentUser.firstName || ""} ${currentUser.lastName ||
""}`.trim() || "System";
      const handledById = currentUser._id || currentUser.id || "";
      const movementType = stockData.reason === "Damaged" || stockData.reason === "Lost" || stockData.reason
=== "Expired" ? "Pull-Out" : "Stock-Out";
      let totalQuantityRemoved = 0;
      if (stockData.noSizes) {
        totalQuantityRemoved = stockData.quantity || 0;
        const response = await fetch(`${API_BASE_URL}/api/products/${editingProduct._id}/stock-out`, {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            ...stockData,
            handledBy,
            handledById
          })
        });
        const data = await response.json();
        if (!response.ok || !data.success) {
          alert(data.message || "Failed to update stock");
          return;
        }
      } else {
        if (stockData.hasVariants && stockData.variantQuantities) {
          stockData.selectedSizes?.forEach((size) => {
            const variantQtys = stockData.variantQuantities[size] || {};
            totalQuantityRemoved += Object.values(variantQtys).reduce((sum, q) => sum + (parseInt(q) || 0),
0);
          });
        } else {
          stockData.selectedSizes?.forEach((size) => {
            totalQuantityRemoved += parseInt(stockData.sizes?.[size]) || 0;
          });
        }
        const response = await fetch(`${API_BASE_URL}/api/products/${editingProduct._id}/stock-out`, {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            ...stockData,
            handledBy,
            handledById
          })
        });
        const data = await response.json();
        if (!response.ok || !data.success) {
          alert(data.message || "Failed to update stock");
          return;
        }
      }
      const archiveReasons = ["Damaged", "Defective", "Expired"];
      if (archiveReasons.includes(stockData.reason)) {
        const sizesString = stockData.selectedSizes ? stockData.selectedSizes.join(", ") : "";
        try {
          await fetch(`${API_BASE_URL}/api/archive`, {
            method: "POST",
            headers: {
              "Content-Type": "application/json"
            },
            body: JSON.stringify({
              productId: editingProduct._id,
              itemName: editingProduct.itemName,
              sku: editingProduct.sku,
              variant: editingProduct.variant || "",
              selectedSize: sizesString,
```

```
            category: editingProduct.category,
            brandName: editingProduct.brandName || "",
            itemPrice: editingProduct.itemPrice || 0,
            costPrice: editingProduct.costPrice || 0,
            quantity: totalQuantityRemoved,
            itemImage: editingProduct.itemImage || "",
            reason: stockData.reason === "Defective" ? "Defective" : stockData.reason,
            archivedBy: handledBy,
            archivedById: handledById,
            source: "stock-out",
            notes: `Stock out - ${stockData.reason}. Sizes: ${sizesString}`
          })
        });
      } catch (archiveError) {
        console.error("Error archiving item:", archiveError);
      }
    }
    setShowStockModal(false);
    setEditingProduct(null);
    setStockAmount("");
    setSuccessMessage("Stock removed successfully!");
    setShowSuccessModal(true);
    invalidateCache("products");
    fetchProducts();
  } catch (error) {
    console.error("Error updating stock:", error);
    alert("Failed to update stock. Please try again.");
  } finally {
    setLoading(false);
  }
};
const formatDate = (date) => {
  if (!date) return "-";
  return new Date(date).toLocaleDateString("en-US", {
    year: "numeric",
    month: "short",
    day: "numeric"
  });
};
const getStockStatus = (currentStock, reorderNumber = 10) => {
  if (currentStock === 0) {
    return { label: "Out of Stock", color: "bg-red-100 text-red-700" };
  } else if (currentStock <= reorderNumber) {
    return { label: "Low Stock", color: "bg-yellow-100 text-yellow-700" };
  } else {
    return { label: "In Stock", color: "bg-green-100 text-green-700" };
  }
};
const formatCsvValue = (value) => {
  if (value === null || value === void 0) return "";
  if (value instanceof Date) return value.toISOString();
  if (typeof value === "object") {
    try {
      return JSON.stringify(value);
    } catch (error) {
      return "";
    }
  }
  return String(value);
};
const handleToggleProductSelection = (productId) => {
  setSelectedProductIds(
    (prev) => prev.includes(productId) ? prev.filter((id) => id !== productId) : [...prev, productId]
  );
};
const handleToggleSelectAllVisible = () => {
  setSelectedProductIds((prev) => {
    if (allVisibleSelected) {
      return prev.filter((id) => !paginatedProductIds.includes(id));
    }
    const merged = new Set(prev);
    paginatedProductIds.forEach((id) => merged.add(id));
    return Array.from(merged);
  });
};
const handleExportButtonClick = () => {
  handleExportToCSV();
```

```
  };
  const handleCancelExportSelection = () => {
    setIsExportSelectionMode(false);
    setSelectedProductIds([]);
  };
  const handleExportToCSV = () => {
    try {
      const productsToExport = selectedProductIds.length > 0 ? filteredProducts.filter(
        (product) => selectedProductIds.includes(product._id)
      ) : filteredProducts;
      if (productsToExport.length === 0) {
        alert("No products to export.");
        return;
      }
      const headers = [
        "sku",
        "itemName",
        "category",
        "subCategory",
        "brandName",
        "variantSummary",
        "batchCount",
        "itemPrice",
        "costPrice",
        "currentStock",
        "unitOfMeasure",
        "reorderNumber",
        "stockStatus",
        "displayInTerminal",
        "dateAdded"
      ];
      const buildVariantSummary = (product) => {
        const sizes = product?.sizes;
        if (!sizes || typeof sizes !== "object") {
          return product?.variant || "N/A";
        }
        const sizeKeys = Object.keys(sizes);
        if (sizeKeys.length === 0) return product?.variant || "N/A";
        const variantSet = /* @__PURE__ */ new Set();
        sizeKeys.forEach((sizeKey) => {
          const sizeData = sizes[sizeKey];
          if (sizeData && typeof sizeData === "object" && sizeData.variants && typeof sizeData.variants ===
"object") {
            Object.keys(sizeData.variants).forEach((v) => {
              if (v) variantSet.add(v);
            });
          }
        });
        const sizeCount = sizeKeys.length;
        const variantCount = variantSet.size;
        if (variantCount > 0) return `${variantCount} variants x ${sizeCount} sizes`;
        return `${sizeCount} sizes`;
      };
      const countBatchRows = (product) => {
        const sizes = product?.sizes;
        if (!sizes || typeof sizes !== "object") return 0;
        let count = 0;
        Object.values(sizes).forEach((sizeData) => {
          if (!sizeData || typeof sizeData !== "object") return;
          if (Array.isArray(sizeData.batches)) {
            count += sizeData.batches.filter((b) => !!b && !b.batchSlotPadding).length;
          }
          if (sizeData.variants && typeof sizeData.variants === "object") {
            Object.values(sizeData.variants).forEach((vData) => {
              if (vData && typeof vData === "object" && Array.isArray(vData.batches)) {
                count += vData.batches.filter((b) => !!b && !b.batchSlotPadding).length;
              }
            });
          }
        });
        return count;
      };
      const rows = productsToExport.map((product) => {
        const status = getStockStatus(
          product.currentStock,
          product.reorderNumber
        );
```

```
      const variantSummary = buildVariantSummary(product);
      const batchCount = countBatchRows(product);
      return headers.map((field) => {
        let value = "";
        switch (field) {
          case "stockStatus":
            value = status.label;
            break;
          case "variantSummary":
            value = variantSummary;
            break;
          case "batchCount":
            value = batchCount;
            break;
          case "displayInTerminal":
            value = product.displayInTerminal !== false ? "Yes" : "No";
            break;
          case "dateAdded":
            value = formatDate(product.dateAdded);
            break;
          default:
            value = product[field];
            break;
        }
        return formatCsvValue(value);
      });
    });
    const humanReadableHeaders = headers.map(
      (field) => fieldToHeaderMap[field] || field
    );
    const csvContent = [
      humanReadableHeaders.join(","),
      ...rows.map(
        (row) => row.map((cell) => {
          const cellStr = String(cell ?? "");
          if (cellStr.includes(",") || cellStr.includes('"') || cellStr.includes("\n")) {
            return `"${cellStr.replace(/"/g, '""')}"`;
          }
          return cellStr;
        }).join(",")
      )
    ].join("\n");
    const blob = new Blob([csvContent], { type: "text/csv;charset=utf-8;" });
    const link = document.createElement("a");
    const url = URL.createObjectURL(blob);
    link.setAttribute("href", url);
    link.setAttribute(
      "download",
      `inventory_export_${(/* @__PURE__ */ new Date()).toISOString().split("T")[0]}.csv`
    );
    link.style.visibility = "hidden";
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
    alert(`${productsToExport.length} products exported successfully!`);
    setIsExportSelectionMode(false);
    setSelectedProductIds([]);
  } catch (error) {
    console.error("Error exporting inventory:", error);
    alert("Failed to export inventory. Please try again.");
  }
};
const handleImportFromCSV = async (event) => {
  const file = event.target.files[0];
  if (!file) return;
  try {
    setLoading(true);
    const text = await file.text();
    const lines = text.split("\n").filter((line) => line.trim());
    if (lines.length < 2) {
      alert("CSV file is empty or invalid");
      return;
    }
    const headers = lines[0].split(",").map((h) => h.trim().replace(/^"|"$/g, ""));
    const rows = lines.slice(1);
    let successCount = 0;
    let errorCount = 0;
```

```
const errors = [];
for (let i = 0; i < rows.length; i++) {
  try {
    const values = [];
    let currentValue = "";
    let insideQuotes = false;
    for (let char of rows[i]) {
      if (char === '"') {
        insideQuotes = !insideQuotes;
      } else if (char === "," && !insideQuotes) {
        values.push(currentValue.trim().replace(/^"|"$/g, ""));
        currentValue = "";
      } else {
        currentValue += char;
      }
    }
    values.push(currentValue.trim().replace(/^"|"$/g, ""));
    const sizesValue = values[headers.indexOf("Sizes")] || "";
    let parsedSizes = null;
    if (sizesValue && sizesValue.trim() !== "") {
      try {
        parsedSizes = JSON.parse(sizesValue);
      } catch (e) {
        console.warn("Failed to parse sizes JSON:", e);
      }
    }
    const skuFromCsv = values[headers.indexOf("SKU")]?.trim();
    const product = {
      itemName: values[headers.indexOf("Item Name")] || "",
      category: values[headers.indexOf("Category")] || "Foods",
      brandName: values[headers.indexOf("Brand")] || "Default",
      variant: values[headers.indexOf("Variant")] || "",
      itemPrice: parseFloat(values[headers.indexOf("Item Price")]) || 0,
      costPrice: parseFloat(values[headers.indexOf("Cost Price")]) || 0,
      currentStock: parseInt(values[headers.indexOf("Current Stock")]) || 0,
      reorderNumber: parseInt(values[headers.indexOf("Reorder Level")]) || 10,
      supplierName: values[headers.indexOf("Supplier Name")] || "",
      supplierContact: values[headers.indexOf("Supplier Contact")] || "",
      itemImage: values[headers.indexOf("Image URL")] || "",
      sizes: parsedSizes
    };
    if (skuFromCsv) {
      product.sku = skuFromCsv;
    }
    if (!product.itemName || !product.itemPrice) {
      errors.push(
        `Row ${i + 2}: Missing required fields (Item Name or Price)`
      );
      errorCount++;
      continue;
    }
    const response = await fetch(`${API_BASE_URL}/api/products`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(product)
    });
    const data = await response.json();
    if (data.success) {
      successCount++;
    } else {
      errors.push(`Row ${i + 2}: ${data.message}`);
      errorCount++;
    }
  } catch (rowError) {
    errors.push(`Row ${i + 2}: ${rowError.message}`);
    errorCount++;
  }
}
setImportResult({ successCount, errorCount, errors });
setShowImportResultModal(true);
if (successCount > 0) {
  invalidateCache("products");
  fetchProducts();
}
event.target.value = "";
```

```
    } catch (error) {
      console.error("Error importing CSV:", error);
      alert(
        "Failed to import CSV file. Please check the file format and try again."
      );
      event.target.value = "";
    } finally {
      setLoading(false);
    }
  };
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `p-8 min-h-screen ${theme === "dark" ? "bg-[#1E1B18]" : "bg-[#F9FAFB]"}`
    },
    /* @__PURE__ */ React.createElement(Header, { pageName: "Product & Stocks", showBorder: false }),
    /* @__PURE__ */ React.createElement("div", { className: "mb-6 mt-8 flex flex-col md:flex-row justify-
between items-start gap-4" }, /* @__PURE__ */ React.createElement("div", { className: "flex gap-4 flex-
wrap" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-2xl shadow-md flex items-center justify-between px-5 py-4 relative overflow-
hidden ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}`,
        style: { minWidth: "200px" }
      },
      /* @__PURE__ */ React.createElement("div", { className: "absolute left-0 top-0 bottom-0 w-2 bg-blue-
500" }),
      /* @__PURE__ */ React.createElement("div", { className: "ml-2" }, /* @__PURE__ */
React.createElement("div", { className: "text-3xl font-bold text-blue-500" },
stockStats.totalItems.toLocaleString()), /* @__PURE__ */ React.createElement("div", { className: "text-xs
text-blue-400 mt-0.5" }, "Total Items")),
      /* @__PURE__ */ React.createElement("div", { className: "w-16 h-16 bg-blue-100 rounded-full flex items-
center justify-center" }, /* @__PURE__ */ React.createElement(
        "svg",
        {
          className: "w-8 h-8 text-blue-400",
          fill: "currentColor",
          viewBox: "0 0 20 20"
        },
        /* @__PURE__ */ React.createElement("path", { d: "M3 1a1 1 0 000 2h1.22l.305 1.222a.997.997 0
00.01.042l1.358 5.43-.893.892C3.74 11.846 4.632 14 6.414 14H15a1 1 0 000-2H6.414l1-1H14a1 1 0 00.894-.553l3-
6A1 1 0 0017 3H6.28l-.31-1.243A1 1 0 005 1H3zM16 16.5a1.5 1.5 0 11-3 0 1.5 1.5 0 013 0zM6.5 18a1.5 1.5 0 100-
3 1.5 1.5 0 000 3z" })
      ))
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-2xl shadow-md flex items-center justify-between px-5 py-4 relative overflow-
hidden ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}`,
        style: { minWidth: "200px" }
      },
      /* @__PURE__ */ React.createElement("div", { className: "absolute left-0 top-0 bottom-0 w-2 bg-green-
500" }),
      /* @__PURE__ */ React.createElement("div", { className: "ml-2" }, /* @__PURE__ */
React.createElement("div", { className: "text-3xl font-bold text-green-500" },
stockStats.inStockItems.toLocaleString()), /* @__PURE__ */ React.createElement("div", { className: "text-xs
text-green-400 mt-0.5" }, "In Stock")),
      /* @__PURE__ */ React.createElement("div", { className: "w-16 h-16 bg-green-100 rounded-full flex
items-center justify-center" }, /* @__PURE__ */ React.createElement(
        "svg",
        {
          className: "w-8 h-8 text-green-400",
          fill: "currentColor",
          viewBox: "0 0 20 20"
        },
        /* @__PURE__ */ React.createElement("path", { d: "M8 16.5a1.5 1.5 0 11-3 0 1.5 1.5 0 013 0zM15
16.5a1.5 1.5 0 11-3 0 1.5 1.5 0 013 0z" }),
        /* @__PURE__ */ React.createElement("path", { d: "M3 4a1 1 0 00-1 1v10a1 1 0 001 1h1.05a2.5 2.5 0
014.9 0H10a1 1 0 001-1V5a1 1 0 00-1-1H3zM14 7a1 1 0 00-1 1v6.05A2.5 2.5 0 0115.95 16H17a1 1 0 001-1v-5a1 1 0
00-.293-.707l-2-2A1 1 0 0015 7h-1z" })
      ))
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-2xl shadow-md flex items-center justify-between px-5 py-4 relative overflow-
hidden ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}`,
        style: { minWidth: "200px" }
```

```
      },
      /* @__PURE__ */ React.createElement("div", { className: "absolute left-0 top-0 bottom-0 w-2 bg-orange-
500" }),
      /* @__PURE__ */ React.createElement("div", { className: "ml-2" }, /* @__PURE__ */
React.createElement("div", { className: "text-3xl font-bold text-orange-500" }, stockStats.lowStockItems), /*
@__PURE__ */ React.createElement("div", { className: "text-xs text-orange-400 mt-0.5" }, "Low Stock")),
      /* @__PURE__ */ React.createElement("div", { className: "w-16 h-16 bg-orange-100 rounded-full flex
items-center justify-center" }, /* @__PURE__ */ React.createElement(
        "svg",
        {
          className: "w-8 h-8 text-orange-400",
          fill: "currentColor",
          viewBox: "0 0 20 20"
        },
        /* @__PURE__ */ React.createElement(
          "path",
          {
            fillRule: "evenodd",
            d: "M18 10a8 8 0 11-16 0 8 8 0 0116 0zm-7 4a1 1 0 11-2 0 1 1 0 012 0zm-1-9a1 1 0 00-1 1v4a1 1 0
102 0V6a1 1 0 00-1-1z",
            clipRule: "evenodd"
          }
        )
      ))
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-2xl shadow-md flex items-center justify-between px-5 py-4 relative overflow-
hidden ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}`,
        style: { minWidth: "200px" }
      },
      /* @__PURE__ */ React.createElement("div", { className: "absolute left-0 top-0 bottom-0 w-2 bg-red-
500" }),
      /* @__PURE__ */ React.createElement("div", { className: "ml-2" }, /* @__PURE__ */
React.createElement("div", { className: "text-3xl font-bold text-red-500" }, stockStats.outOfStockItems), /*
@__PURE__ */ React.createElement("div", { className: "text-xs text-red-400 mt-0.5" }, "Out-of-Stock")),
      /* @__PURE__ */ React.createElement("div", { className: "w-16 h-16 bg-red-100 rounded-full flex items-
center justify-center" }, /* @__PURE__ */ React.createElement(
        "svg",
        {
          className: "w-8 h-8 text-red-400",
          fill: "currentColor",
          viewBox: "0 0 20 20"
        },
        /* @__PURE__ */ React.createElement(
          "path",
          {
            fillRule: "evenodd",
            d: "M10 18a8 8 0 100-16 8 8 0 000 16zM8.707 7.293a1 1 0 00-1.414 1.414L8.586 10l-1.293 1.293a1 1
0 101.414 1.414L10 11.414l1.293 1.293a1 1 0 001.414-1.414L11.414 10l1.293-1.293a1 1 0 00-1.414-1.414L10 8.586
8.707 7.293z",
            clipRule: "evenodd"
          }
        )
      ))
    )), /* @__PURE__ */ React.createElement("div", { className: "flex gap-4" }, /* @__PURE__ */
React.createElement(
      "button",
      {
        onClick: handleExportButtonClick,
        className: `rounded-2xl shadow-md flex flex-col items-center justify-center px-5 py-4 transition-
colors ${isExportSelectionMode ? "border border-[#AD7F65] bg-[#AD7F65]/5" : theme === "dark" ? "bg-[#2A2724]
hover:bg-[#352F2A]" : "bg-white hover:bg-gray-50"}`,
        style: { minWidth: "100px" }
      },
      /* @__PURE__ */ React.createElement(
        "svg",
        {
          className: `w-8 h-8 mb-1 ${theme === "dark" ? "text-gray-400" : "text-gray-700"}`,
          fill: "none",
          stroke: "currentColor",
          viewBox: "0 0 24 24"
        },
        /* @__PURE__ */ React.createElement(
          "path",
          {
            strokeLinecap: "round",
```

```
            strokeLinejoin: "round",
            strokeWidth: 2,
            d: "M12 10v6m0 0l-3-3m3 3l3-3m2 8H7a2 2 0 01-2-2V5a2 2 0 012-2h5.586a1 1 0 01.707.293l5.414
5.414a1 1 0 01.293.707V19a2 2 0 01-2 2z"
          }
        )
      ),
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `text-xs font-medium ${theme === "dark" ? "text-gray-400" : "text-gray-700"}`
        },
        isExportSelectionMode ? "Export Selected" : "Export"
      )
    ), isExportSelectionMode && /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: handleCancelExportSelection,
        className: `rounded-2xl shadow-md px-4 py-2 text-xs font-medium border transition-colors ${theme ===
"dark" ? "bg-[#2A2724] border-gray-600 text-gray-400 hover:bg-[#352F2A]" : "bg-white border-gray-200 text-
gray-600 hover:bg-gray-50"}`
      },
      "Cancel"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => document.getElementById("csv-file-input").click(),
        className: `rounded-2xl shadow-md flex flex-col items-center justify-center px-5 py-4 transition-
colors ${theme === "dark" ? "bg-[#2A2724] hover:bg-[#352F2A]" : "bg-white hover:bg-gray-50"}`,
        style: { minWidth: "100px" }
      },
      /* @__PURE__ */ React.createElement(
        "svg",
        {
          className: `w-8 h-8 mb-1 ${theme === "dark" ? "text-gray-400" : "text-gray-700"}`,
          fill: "none",
          stroke: "currentColor",
          viewBox: "0 0 24 24"
        },
        /* @__PURE__ */ React.createElement(
          "path",
          {
            strokeLinecap: "round",
            strokeLinejoin: "round",
            strokeWidth: 2,
            d: "M7 16a4 4 0 01-.88-7.903A5 5 0 1115.9 6L16 6a5 5 0 011 9.9M15 13l-3-3m0 0l-3 3m3-3v12"
          }
        )
      ),
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `text-xs font-medium ${theme === "dark" ? "text-gray-400" : "text-gray-700"}`
        },
        "Import"
      )
    ), /* @__PURE__ */ React.createElement(
      "input",
      {
        id: "csv-file-input",
        type: "file",
        accept: ".csv",
        onChange: handleImportFromCSV,
        style: { display: "none" }
      }
    ))),
    /* @__PURE__ */ React.createElement("div", { className: "flex justify-between items-center mb-4" }, /*
@__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement("div", { className: "relative", style: { width: "300px" } }, /* @__PURE__ */
React.createElement("div", { className: "absolute left-3 top-1/2 transform -translate-y-1/2" }, /* @__PURE__
*/ React.createElement(FaSearch, { className: "text-gray-400" })), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "text",
        placeholder: "Search For...",
        value: searchQuery,
        onChange: (e) => setSearchQuery(e.target.value),
```

```
        className: `w-full h-10 pl-10 pr-4 py-2 border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white placeholder-
gray-500" : "bg-white border-gray-300"}`
      }
    )), /* @__PURE__ */ React.createElement("div", { className: "flex gap-2" }, /* @__PURE__ */
React.createElement(
      "select",
      {
        value: selectedMainCategory,
        onChange: (e) => {
          setSelectedMainCategory(e.target.value);
          setSelectedSubCategory("");
        },
        className: `h-10 px-4 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65] ${theme
=== "dark" ? "bg-[#2A2724] border-gray-600 text-white" : "bg-white border-gray-300"}`
      },
      mainCategories.map((cat) => /* @__PURE__ */ React.createElement("option", { key: cat.name, value:
cat.name }, cat.name === "All" ? "By Category" : cat.name))
    ), /* @__PURE__ */ React.createElement(
      "select",
      {
        value: selectedSubCategory,
        onChange: (e) => setSelectedSubCategory(e.target.value),
        disabled: selectedMainCategory === "All",
        className: `h-10 px-4 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
${selectedMainCategory === "All" ? "opacity-50 cursor-not-allowed text-gray-400" : ""} ${theme === "dark" ?
"bg-[#2A2724] border-gray-600 text-white" : "bg-white border-gray-300"}`
      },
      /* @__PURE__ */ React.createElement("option", { value: "" }, selectedMainCategory === "All" ? "Select
Main First" : "All Subcategories"),
      subCategories.filter((sub) => sub.parentCategory === selectedMainCategory).map((sub) => /* @__PURE__ */
React.createElement("option", { key: `sub-${sub.name}`, value: sub.name }, sub.name))
    )), /* @__PURE__ */ React.createElement(
      "select",
      {
        value: filterBrand,
        onChange: (e) => setFilterBrand(e.target.value),
        className: `h-10 px-4 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65] ${theme
=== "dark" ? "bg-[#2A2724] border-gray-600 text-white" : "bg-white border-gray-300"}`
      },
      /* @__PURE__ */ React.createElement("option", { value: "All" }, "By Brand"),
      uniqueBrands.map(
        (brand) => /* @__PURE__ */ React.createElement("option", { key: brand, value: brand }, brand)
      )
    ), /* @__PURE__ */ React.createElement(
      "select",
      {
        value: filterStatus,
        onChange: (e) => setFilterStatus(e.target.value),
        className: `h-10 px-4 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65] ${theme
=== "dark" ? "bg-[#2A2724] border-gray-600 text-white" : "bg-white border-gray-300"}`
      },
      /* @__PURE__ */ React.createElement("option", { value: "All" }, "By Status"),
      /* @__PURE__ */ React.createElement("option", { value: "In Stock" }, "In Stock"),
      /* @__PURE__ */ React.createElement("option", { value: "Low Stock" }, "Low Stock"),
      /* @__PURE__ */ React.createElement("option", { value: "Out of Stock" }, "Out of Stock")
    ), /* @__PURE__ */ React.createElement(
      "select",
      {
        value: sortBy,
        onChange: (e) => setSortBy(e.target.value),
        className: `h-10 px-4 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65] ${theme
=== "dark" ? "bg-[#2A2724] border-gray-600 text-white" : "bg-white border-gray-300"}`
      },
      /* @__PURE__ */ React.createElement("option", { value: "sku-new" }, "SKU: Newest First"),
      /* @__PURE__ */ React.createElement("option", { value: "sku-old" }, "SKU: Oldest First"),
      /* @__PURE__ */ React.createElement("option", { value: "name" }, "Sort By Name"),
      /* @__PURE__ */ React.createElement("option", { value: "price-low" }, "Price: Low to High"),
      /* @__PURE__ */ React.createElement("option", { value: "price-high" }, "Price: High to Low"),
      /* @__PURE__ */ React.createElement("option", { value: "stock-low" }, "Stock: Low to High"),
      /* @__PURE__ */ React.createElement("option", { value: "stock-high" }, "Stock: High to Low"),
      /* @__PURE__ */ React.createElement("option", { value: "date-new" }, "Date: Newest First"),
      /* @__PURE__ */ React.createElement("option", { value: "date-old" }, "Date: Oldest First")
    )), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3" }, /* @__PURE__ */
React.createElement(
      "button",
      {
```

```
        onClick: () => {
          resetProductForm();
          setShowAddModal(true);
        },
        className: "px-6 py-2 text-white rounded-lg hover:opacity-90 flex items-center gap-2 font-medium
transition-all",
        style: {
          background: "linear-gradient(135deg, #10B981 0%, #059669 100%)"
        }
      },
      /* @__PURE__ */ React.createElement(FaPlus, null),
      " Add New Item"
    ))),
    /* @__PURE__ */ React.createElement(
      ProductTable,
      {
        loading,
        filteredProducts: paginatedProducts,
        handleEditProduct: handleEditClick,
        handleDeleteProduct: handleDeleteClick,
        handleArchiveProduct: handleArchiveClick,
        handleViewProduct,
        handleStockUpdate,
        selectedProductIds,
        onToggleSelect: handleToggleProductSelection,
        onToggleSelectAll: handleToggleSelectAllVisible,
        showSelection: isExportSelectionMode,
        allVisibleSelected,
        someVisibleSelected
      }
    ),
    filteredProducts.length >= itemsPerPage && /* @__PURE__ */ React.createElement(
      Pagination,
      {
        currentPage,
        totalPages: Math.ceil(filteredProducts.length / itemsPerPage),
        onPageChange: setCurrentPage
      }
    ),
    /* @__PURE__ */ React.createElement(
      AddProductModal,
      {
        showAddModal,
        setShowAddModal,
        editingProduct,
        setEditingProduct,
        newProduct,
        setNewProduct,
        handleAddProduct,
        handleInputChange,
        handleSizeToggle,
        handleSizeQuantityChange,
        handleSizePriceChange,
        resetProductForm,
        loading,
        categories,
        brandPartners,
        onCategoryAdd: fetchCategories,
        onBrandAdd: fetchBrandPartners
      }
    ),
    /* @__PURE__ */ React.createElement(
      ConfirmAddProductModal,
      {
        isOpen: showConfirmModal,
        onClose: () => setShowConfirmModal(false),
        onConfirm: confirmAddProduct,
        productName: newProduct.itemName,
        noStockWarning: !addProductHasOpeningStock,
        onCategoryAdd: fetchCategories,
        onBrandAdd: fetchBrandPartners
      }
    ),
    /* @__PURE__ */ React.createElement(
      SuccessModal,
      {
        isOpen: showSuccessModal,
```

```
    onClose: () => setShowSuccessModal(false),
    message: successMessage || "The item was added successfully!"
  }
),
/* @__PURE__ */ React.createElement(
  DeleteConfirmationModal,
  {
    isOpen: showDeleteModal,
    onClose: () => {
      setShowDeleteModal(false);
      setProductToDelete(null);
    },
    onConfirm: confirmDeleteProduct,
    itemName: productToDelete ? filteredProducts.find((p) => p._id === productToDelete)?.itemName : ""
  }
),
/* @__PURE__ */ React.createElement(
  ArchiveConfirmationModal,
  {
    isOpen: showArchiveModal,
    onClose: () => {
      setShowArchiveModal(false);
      setProductToArchive(null);
    },
    onConfirm: confirmArchiveProduct,
    itemName: productToArchive ? filteredProducts.find((p) => p._id === productToArchive)?.itemName : ""
  }
),
/* @__PURE__ */ React.createElement(
  EditConfirmationModal,
  {
    isOpen: showEditModal,
    onClose: () => {
      setShowEditModal(false);
      setProductToEdit(null);
    },
    onConfirm: confirmEditProduct,
    itemName: productToEdit?.itemName || editingProduct?.itemName || ""
  }
),
/* @__PURE__ */ React.createElement(
  ViewProductModal,
  {
    showViewModal,
    setShowViewModal,
    viewingProduct,
    formatDate
  }
),
showStockModal && stockModalType === "in" && /* @__PURE__ */ React.createElement(
  StockInModal,
  {
    isOpen: showStockModal,
    onClose: () => {
      setShowStockModal(false);
      setEditingProduct(null);
      setStockAmount("");
    },
    product: editingProduct,
    onConfirm: handleStockInConfirm,
    loading,
    brandPartners
  }
),
showStockModal && stockModalType === "out" && /* @__PURE__ */ React.createElement(
  StockOutModal,
  {
    isOpen: showStockModal,
    onClose: () => {
      setShowStockModal(false);
      setEditingProduct(null);
      setStockAmount("");
    },
    product: editingProduct,
    onConfirm: handleStockOutConfirm,
    loading
  }
```

```
    ),
    showImportResultModal && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex
items-center justify-center z-9999 p-4 bg-black/50 backdrop-blur-sm" }, /* @__PURE__ */
React.createElement("div", { className: "bg-[#2D2D2D] rounded-2xl w-full max-w-2xl p-8 text-white shadow-
2xl" }, /* @__PURE__ */ React.createElement("h2", { className: "text-2xl font-bold mb-4" }, "localhost:5173
says"), /* @__PURE__ */ React.createElement("div", { className: "space-y-4 mb-6" }, /* @__PURE__ */
React.createElement("p", { className: "text-lg" }, "Import completed!"), /* @__PURE__ */
React.createElement("div", { className: "space-y-2" }, /* @__PURE__ */ React.createElement("p", null,
"Successful: ", importResult.successCount), /* @__PURE__ */ React.createElement("p", null, "Failed: ",
importResult.errorCount)), importResult.errors.length > 0 && /* @__PURE__ */ React.createElement("div",
{ className: "mt-4" }, /* @__PURE__ */ React.createElement("p", { className: "mb-2" }, "Showing first 5
errors:"), /* @__PURE__ */ React.createElement("div", { className: "bg-black/30 rounded-lg p-4 max-h-60
overflow-y-auto" }, importResult.errors.slice(0, 5).map(
      (error, idx) => /* @__PURE__ */ React.createElement("p", { key: idx, className: "text-sm mb-1" }, error)
    )))), /* @__PURE__ */ React.createElement("div", { className: "flex justify-end" }, /* @__PURE__ */
React.createElement(
      "button",
      {
        onClick: () => setShowImportResultModal(false),
        className: "px-8 py-2 bg-green-500 hover:bg-green-600 text-white rounded-full font-medium transition-
colors"
      },
      "OK"
    ))))
  );
};
export default memo(Inventory);
```

FILE: pos/frontend/src/pages/logs.jsx
==========

```
import { memo, useEffect, useMemo, useRef, useState } from "react";
import { FaChevronLeft, FaChevronRight, FaSearch } from "react-icons/fa";
import PullOutIcon from "../assets/logs/pull_out.svg";
import StockInIcon from "../assets/logs/stock-in.svg";
import StockOutIcon from "../assets/logs/stock-out.svg";
import ViewVoidLogModal from "../components/logs/ViewVoidLogModal";
import Header from "../components/shared/header";
import { API_BASE_URL } from "../config/api";
import { useAuth } from "../context/AuthContext";
import { useDataCache } from "../context/DataCacheContext";
import { useTheme } from "../context/ThemeContext";
const Logs = () => {
  const { theme } = useTheme();
  const { currentUser } = useAuth();
  const { getCachedData, setCachedData, isCacheValid, invalidateCache } = useDataCache();
  const [activeTab, setActiveTab] = useState("stock-movement");
  const [movements, setMovements] = useState([]);
  const [voidLogs, setVoidLogs] = useState([]);
  const [loading, setLoading] = useState(false);
  const [showVoidLogModal, setShowVoidLogModal] = useState(false);
  const [selectedVoidLog, setSelectedVoidLog] = useState(null);
  const [stats, setStats] = useState(() => getCachedData("stats") || {
    stockIns: 0,
    stockOuts: 0,
    pullOuts: 0
  });
  const [searchQuery, setSearchQuery] = useState("");
  const [filterCategory, setFilterCategory] = useState("All");
  const [filterType, setFilterType] = useState("All");
  const [filterBrand, setFilterBrand] = useState("All");
  const [filterDate, setFilterDate] = useState("All");
  const [filterReason, setFilterReason] = useState("All");
  const [sortBy, setSortBy] = useState("date-desc");
  const [currentPage, setCurrentPage] = useState(1);
  const [voidCurrentPage, setVoidCurrentPage] = useState(1);
  const rowsPerPage = 8;
  const [totalPages, setTotalPages] = useState(1);
  const [voidTotalPages, setVoidTotalPages] = useState(1);
  const [selectedMovements, setSelectedMovements] = useState([]);
  const [isMovementExportMode, setIsMovementExportMode] = useState(false);
  const categories = ["All", "Tops", "Bottoms", "Dresses", "Makeup", "Accessories", "Shoes", "Head Wear",
"Foods"];
  const types = ["All", "Stock-In", "Stock-Out", "Pull-Out"];
  const reasons = ["All", "Restock", "Sold", "Returned Item", "Return", "Exchange", "Damaged", "Lost",
"Expired", "Adjustment", "Initial Stock", "Other"];
```

```
  const dateOptions = ["All", "Today", "This Week", "This Month"];
  const sortOptions = [
    { value: "date-desc", label: "Date: Newest First" },
    { value: "date-asc", label: "Date: Oldest First" },
    { value: "name-asc", label: "Name: A-Z" },
    { value: "name-desc", label: "Name: Z-A" },
    { value: "sku-asc", label: "SKU: A-Z" },
    { value: "sku-desc", label: "SKU: Z-A" }
  ];
  const selectAllMovementsRef = useRef(null);
  useEffect(() => {
    const cachedStats = getCachedData("stats");
    if (!cachedStats || !isCacheValid("stats")) {
      fetchStats();
    }
  }, []);
  useEffect(() => {
    if (activeTab === "stock-movement") {
      fetchMovements();
    }
  }, [activeTab, currentPage, searchQuery, filterCategory, filterType, filterBrand, filterDate, filterReason,
sortBy]);
  useEffect(() => {
    setCurrentPage(1);
  }, [searchQuery, filterCategory, filterType, filterBrand, filterDate, filterReason, sortBy]);
  useEffect(() => {
    if (activeTab === "void-logs") {
      fetchVoidLogs();
    }
  }, [activeTab, voidCurrentPage]);
  useEffect(() => {
    if (activeTab !== "stock-movement") {
      setIsMovementExportMode(false);
      setSelectedMovements([]);
    }
  }, [activeTab]);
  const fetchStats = async () => {
    try {
      const response = await fetch(`${API_BASE_URL}/api/stock-movements/stats/today`);
      const data = await response.json();
      if (data.success) {
        const mappedStats = {
          stockIns: data.data.stockIn || 0,
          stockOuts: data.data.stockOut || 0,
          pullOuts: data.data.pullOut || 0
        };
        setStats(mappedStats);
        setCachedData("stats", mappedStats);
      }
    } catch (error) {
      console.error("Error fetching stats:", error);
    }
  };
  const fetchMovements = async () => {
    try {
      setLoading(true);
      const params = new URLSearchParams({
        search: searchQuery,
        category: filterCategory,
        type: filterType,
        brand: filterBrand,
        date: filterDate,
        reason: filterReason,
        sortBy,
        page: currentPage,
        limit: rowsPerPage
      });
      const response = await fetch(`${API_BASE_URL}/api/stock-movements?${params}`);
      const data = await response.json();
      if (data.success) {
        setMovements(data.data || []);
        if (data.pagination) setTotalPages(data.pagination.pages || 1);
      }
    } catch (error) {
      console.error("Error fetching movements:", error);
      setMovements([]);
    } finally {
```

```
      setLoading(false);
    }
  };
  const fetchVoidLogs = async () => {
    try {
      setLoading(true);
      const params = new URLSearchParams({
        page: voidCurrentPage,
        limit: rowsPerPage,
        sortBy: "voidedAt",
        sortOrder: "desc"
      });
      const response = await fetch(`${API_BASE_URL}/api/void-logs?${params}`);
      const data = await response.json();
      if (data.success && Array.isArray(data.data)) {
        setVoidLogs(data.data);
        if (data.pagination) setVoidTotalPages(data.pagination.pages || 1);
      } else if (Array.isArray(data)) {
        setVoidLogs(data);
      } else {
        setVoidLogs([]);
      }
    } catch (error) {
      console.error("Error fetching void logs:", error);
      setVoidLogs([]);
    } finally {
      setLoading(false);
    }
  };
  const formatDateTime = (dateString) => {
    if (!dateString) return "-";
    const date = new Date(dateString);
    return date.toLocaleString("en-US", {
      year: "numeric",
      month: "2-digit",
      day: "2-digit",
      hour: "2-digit",
      minute: "2-digit",
      hour12: true
    });
  };
  const getTypeBadge = (type) => {
    const styles = {
      "Stock-In": "bg-green-100 text-green-700",
      "Stock-Out": "bg-red-100 text-red-700",
      "Pull-Out": "bg-orange-100 text-orange-700"
    };
    const dotStyles = {
      "Stock-In": "bg-green-600",
      "Stock-Out": "bg-red-600",
      "Pull-Out": "bg-orange-600"
    };
    return /* @__PURE__ */ React.createElement("span", { className: `inline-flex items-center gap-1.5 px-3
py-1 rounded-full text-xs font-medium ${styles[type] || "bg-gray-100 text-gray-700"}` }, /* @__PURE__ */
React.createElement("span", { className: `w-2 h-2 rounded-full ${dotStyles[type] || "bg-gray-500"}` }), type);
  };
  const getQuantityColor = (type, quantity) => {
    if (type === "Stock-In") return "text-green-600";
    if (type === "Stock-Out") return "text-red-600";
    if (type === "Pull-Out") return "text-orange-600";
    return "text-gray-600";
  };
  const handleToggleMovementSelection = (movement) => {
    if (!movement || !movement._id) return;
    setSelectedMovements(
      (prev) => prev.some((m) => m._id === movement._id) ? prev.filter((m) => m._id !== movement._id) :
[...prev, movement]
    );
  };
  const handleToggleSelectAllMovements = () => {
    setSelectedMovements((prev) => {
      if (allVisibleMovementsSelected) {
        return prev.filter((m) => !movements.some((vm) => vm._id === m._id));
      }
      const existingIds = new Set(prev.map((m) => m._id));
      const toAdd = movements.filter((m) => !existingIds.has(m._id));
      return [...prev, ...toAdd];
```

```
    });
  };
  const handleMovementExportButtonClick = () => {
    if (!isMovementExportMode) {
      setIsMovementExportMode(true);
      setSelectedMovements([]);
      return;
    }
    handleExport();
  };
  const handleCancelMovementSelection = () => {
    setIsMovementExportMode(false);
    setSelectedMovements([]);
  };
  const handleExport = () => {
    const movementsToExport = selectedMovements.length > 0 ? selectedMovements : [];
    if (movementsToExport.length === 0) {
      alert("Please select at least one stock movement to export.");
      return;
    }
    const headers = [
      "ID",
      "SKU",
      "Item Name",
      "Category",
      "Brand",
      "Type",
      "Quantity",
      "Size Breakdown",
      "Stock Before",
      "Stock After",
      "Reason",
      "Handled By",
      "Handled By ID",
      "Notes",
      "Item Image",
      "Date & Time",
      "Created At",
      "Updated At"
    ];
    const csvData = movementsToExport.map(
      (m) => [
        m._id || "",
        m.sku || "",
        m.itemName || "",
        m.category || "",
        m.brandName || "",
        m.type || "",
        m.quantity || 0,
        m.sizeQuantities ? JSON.stringify(m.sizeQuantities) : "",
        m.stockBefore || 0,
        m.stockAfter || 0,
        m.reason || "",
        m.handledBy || "",
        m.handledById || "",
        m.notes || "",
        m.itemImage || "",
        formatDateTime(m.createdAt),
        m.createdAt || "",
        m.updatedAt || ""
      ]
    );
    const csvContent = [
      headers.join(","),
      ...csvData.map((row) => row.map((cell) => `"${String(cell).replace(/"/g, '""')}"`).join(","))
    ].join("\n");
    const blob = new Blob([csvContent], { type: "text/csv;charset=utf-8;" });
    const link = document.createElement("a");
    const url = URL.createObjectURL(blob);
    link.setAttribute("href", url);
    link.setAttribute("download", `stock_movements_${(/* @__PURE__ */ new
Date()).toISOString().split("T")[0]}.csv`);
    link.style.visibility = "hidden";
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
    setIsMovementExportMode(false);
```

```
    setSelectedMovements([]);
  };
  const handleImport = async (event) => {
    const file = event.target.files?.[0];
    if (!file) return;
    const reader = new FileReader();
    reader.onload = async (e) => {
      try {
        const text = e.target?.result;
        const lines = text.split("\n").filter((line) => line.trim());
        if (lines.length < 2) {
          alert("CSV file is empty or invalid");
          return;
        }
        const dataRows = lines.slice(1);
        const importedMovements = [];
        for (const row of dataRows) {
          const values = row.match(/(".*?"|[^",\s]+)(?=\s*,|\s*$)/g)?.map((v) => v.replace(/^"|"$/g,
"").replace(/""/g, '"')) || [];
          if (values.length >= 11) {
            importedMovements.push({
              sku: values[1],
              itemName: values[2],
              category: values[3],
              brandName: values[4],
              type: values[5],
              quantity: parseInt(values[6]) || 0,
              sizeQuantities: values[7] ? JSON.parse(values[7]) : null,
              stockBefore: parseInt(values[8]) || 0,
              stockAfter: parseInt(values[9]) || 0,
              reason: values[10],
              handledBy: values[11] || currentUser?.name || "System",
              handledById: values[12] || currentUser?._id || "",
              notes: values[13] || "Imported from CSV"
            });
          }
        }
        if (importedMovements.length === 0) {
          alert("No valid data found in CSV file");
          return;
        }
        const response = await fetch(`${API_BASE_URL}/api/stock-movements/bulk`, {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ movements: importedMovements })
        });
        const result = await response.json();
        if (result.success) {
          alert(`Successfully imported ${importedMovements.length} stock movements`);
          fetchMovements();
          fetchStats();
        } else {
          alert(`Import failed: ${result.message || "Unknown error"}`);
        }
      } catch (error) {
        console.error("Import error:", error);
        alert("Failed to import CSV file. Please check the file format.");
      }
    };
    reader.readAsText(file);
    event.target.value = "";
  };
  const handleView = (item) => {
    if (activeTab === "void-logs") {
      setSelectedVoidLog(item);
      setShowVoidLogModal(true);
    } else {
      console.log("View movement:", item);
    }
  };
  const uniqueBrands = useMemo(() => {
    const brands = new Set(movements.map((m) => m.brandName).filter(Boolean));
    return ["All", ...Array.from(brands).sort()];
  }, [movements]);
  const allVisibleMovementsSelected = movements.length > 0 && movements.every((m) =>
selectedMovements.some((sm) => sm._id === m._id));
  const someVisibleMovementsSelected = movements.some(
```

```
    (m) => selectedMovements.some((sm) => sm._id === m._id)
  );
  const movementTableColumnCount = isMovementExportMode ? 10 : 9;
  useEffect(() => {
    if (selectAllMovementsRef.current) {
      selectAllMovementsRef.current.indeterminate = isMovementExportMode && !allVisibleMovementsSelected &&
someVisibleMovementsSelected;
    }
  }, [isMovementExportMode, allVisibleMovementsSelected, someVisibleMovementsSelected]);
  return /* @__PURE__ */ React.createElement("div", { className: `p-8 min-h-screen ${theme === "dark" ? "bg-
[#1E1B18]" : "bg-gray-50"}` }, /* @__PURE__ */ React.createElement(Header, { pageName: activeTab === "stock-
movement" ? "Inventory Logs" : "Void Logs", showBorder: false }), /* @__PURE__ */ React.createElement("div",
{ className: "flex gap-3 mb-6" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setActiveTab("stock-movement"),
      className: `px-6 py-3 font-bold rounded-xl transition-all shadow-md ${activeTab === "stock-movement" ?
`text-[#AD7F65] border-b-4 border-[#AD7F65] ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}` : `${theme ===
"dark" ? "bg-[#2A2724] text-gray-300 border border-gray-700" : "bg-white text-gray-800 border border-gray-
200"}`}`
    },
    "Inventory Logs"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setActiveTab("void-logs"),
      className: `px-6 py-3 font-bold rounded-xl transition-all shadow-md ${activeTab === "void-logs" ?
`text-[#AD7F65] border-b-4 border-[#AD7F65] ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}` : `${theme ===
"dark" ? "bg-[#2A2724] text-gray-300 border border-gray-700" : "bg-white text-gray-800 border border-gray-
200"}`}`
    },
    "Void Logs"
  )), activeTab === "stock-movement" && /* @__PURE__ */ React.createElement(React.Fragment, null, /*
@__PURE__ */ React.createElement("div", { className: "mb-6" }, /* @__PURE__ */ React.createElement("div",
{ className: "flex justify-between items-center gap-4 flex-wrap" }, /* @__PURE__ */ React.createElement("div",
{ className: "flex gap-4 flex-wrap" }, /* @__PURE__ */ React.createElement("div", { className: `rounded-2xl
shadow-md flex items-center justify-between px-5 py-4 relative overflow-hidden ${theme === "dark" ? "bg-
[#2A2724]" : "bg-white"}`, style: { minWidth: "200px" } }, /* @__PURE__ */ React.createElement("div",
{ className: "absolute left-0 top-0 bottom-0 w-2 bg-green-500" }), /* @__PURE__ */ React.createElement("div",
{ className: "ml-2" }, /* @__PURE__ */ React.createElement("div", { className: "text-3xl font-bold text-
green-500" }, stats.stockIns), /* @__PURE__ */ React.createElement("div", { className: "text-xs text-green-
400 mt-0.5" }, "Stock-ins Today")), /* @__PURE__ */ React.createElement("div", { className: "w-20 h-20
rounded-full flex items-center justify-center" }, /* @__PURE__ */ React.createElement("img", { src:
StockInIcon, alt: "Stock In", className: "w-16 h-16" }))), /* @__PURE__ */ React.createElement("div",
{ className: `rounded-2xl shadow-md flex items-center justify-between px-5 py-4 relative overflow-hidden
${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}`, style: { minWidth: "200px" } }, /* @__PURE__ */
React.createElement("div", { className: "absolute left-0 top-0 bottom-0 w-2 bg-red-500" }), /* @__PURE__ */
React.createElement("div", { className: "ml-2" }, /* @__PURE__ */ React.createElement("div", { className:
"text-3xl font-bold text-red-500" }, stats.stockOuts), /* @__PURE__ */ React.createElement("div", { className:
"text-xs text-red-400 mt-0.5" }, "Stock-outs Today")), /* @__PURE__ */ React.createElement("div", { className:
"w-20 h-20  rounded-full flex items-center justify-center" }, /* @__PURE__ */ React.createElement("img",
{ src: StockOutIcon, alt: "Stock Out", className: "w-16 h-16" }))), /* @__PURE__ */ React.createElement("div",
{ className: `rounded-2xl shadow-md flex items-center justify-between px-5 py-4 relative overflow-hidden
${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}`, style: { minWidth: "200px" } }, /* @__PURE__ */
React.createElement("div", { className: "absolute left-0 top-0 bottom-0 w-2 bg-orange-500" }), /* @__PURE__
*/ React.createElement("div", { className: "ml-2" }, /* @__PURE__ */ React.createElement("div", { className:
"text-3xl font-bold text-orange-500" }, stats.pullOuts), /* @__PURE__ */ React.createElement("div",
{ className: "text-xs text-orange-400 mt-0.5" }, "Pull-Outs Today")), /* @__PURE__ */
React.createElement("div", { className: "w-20 h-20 rounded-full flex items-center justify-center" }, /*
@__PURE__ */ React.createElement("img", { src: PullOutIcon, alt: "Pull Out", className: "w-16 h-16" }))))), /*
@__PURE__ */ React.createElement("div", { className: "flex gap-4 items-center" }, /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: handleMovementExportButtonClick,
      className: `rounded-2xl shadow-md flex flex-col items-center justify-center px-5 py-4 transition-colors
${theme === "dark" ? "bg-[#2A2724] text-gray-300 hover:bg-[#3A3734]" : "bg-white text-gray-700 hover:bg-gray-
50"} ${isMovementExportMode ? "border border-[#AD7F65] bg-[#AD7F65]/5" : ""}`,
      style: { minWidth: "100px" }
    },
    /* @__PURE__ */ React.createElement("svg", { className: "w-8 h-8 text-gray-700 mb-1", fill: "none",
stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap:
"round", strokeLinejoin: "round", strokeWidth: 2, d: "M12 10v6m0 01-3-3m3 3l3-3m2 8H7a2 2 0 01-2-2V5a2 2 0
012-2h5.586a1 1 0 01.707.293l5.414 5.414a1 1 0 01.293.707V19a2 2 0 01-2 2z" })),
    /* @__PURE__ */ React.createElement("div", { className: `text-xs font-medium ${theme === "dark" ? "text-
gray-400" : "text-gray-700"}` }, isMovementExportMode ? "Export Selected" : "Export")
  ), isMovementExportMode && /* @__PURE__ */ React.createElement(
```

```
    "button",
    {
      onClick: handleCancelMovementSelection,
      className: `rounded-2xl shadow-md px-4 py-2 text-xs font-medium border transition-colors ${theme ===
"dark" ? "bg-[#2A2724] border-gray-700 text-gray-300 hover:bg-[#3A3734]" : "bg-white border-gray-200 text-
gray-600 hover:bg-gray-50"}`
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "label",
    {
      className: `rounded-2xl shadow-md flex flex-col items-center justify-center px-5 py-4 transition-colors
cursor-pointer ${theme === "dark" ? "bg-[#2A2724] hover:bg-[#3A3734] text-gray-300" : "bg-white hover:bg-
gray-50"}`,
      style: { minWidth: "100px" }
    },
    /* @__PURE__ */ React.createElement("svg", { className: "w-8 h-8 text-gray-700 mb-1", fill: "none",
stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap:
"round", strokeLinejoin: "round", strokeWidth: 2, d: "M7 16a4 4 0 01-.88-7.903A5 5 0 1115.9 6L16 6a5 5 0 011
9.9M15 13l-3-3m0 0l-3 3m3-3v12" })),
    /* @__PURE__ */ React.createElement("div", { className: `text-xs font-medium ${theme === "dark" ? "text-
gray-400" : "text-gray-700"}` }, "Import"),
    /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "file",
        accept: ".csv",
        onChange: handleImport,
        className: "hidden"
      }
    )
  )))), /* @__PURE__ */ React.createElement("div", { className: "mb-6" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-3 flex-wrap" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center shadow-md overflow-hidden rounded-[10px]", style:
{ width: "400px" } }, /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      className: "px-4 py-3 flex items-center justify-center h-10",
      style: {
        background: "linear-gradient(135deg, rgba(173, 127, 101, 1) 0%,  rgba(118, 70, 43, 1) 100%)"
      }
    },
    /* @__PURE__ */ React.createElement(FaSearch, { className: "text-white text-sm" })
  ), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      placeholder: "Search for product, SKU, Employees...",
      value: searchQuery,
      onChange: (e) => setSearchQuery(e.target.value),
      className: `flex-1 px-4 py-3 h-10 focus:outline-none placeholder-gray-400 border-0 ${theme === "dark" ?
"bg-[#2A2724] text-white" : "bg-white text-gray-700"}`
    }
  )), /* @__PURE__ */ React.createElement(
    "select",
    {
      value: filterCategory,
      onChange: (e) => setFilterCategory(e.target.value),
      className: `h-10 px-3 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65] text-sm
${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-300"}`,
      style: { minWidth: "140px", maxWidth: "160px" }
    },
    categories.map(
      (cat) => /* @__PURE__ */ React.createElement("option", { key: cat, value: cat }, "By Category ",
cat !== "All" ? `(${cat})` : "")
    )
  ), /* @__PURE__ */ React.createElement(
    "select",
    {
      value: filterType,
      onChange: (e) => setFilterType(e.target.value),
      className: `h-10 px-3 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65] text-sm
${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-300"}`,
      style: { minWidth: "130px", maxWidth: "150px" }
    },
    types.map(
```

```
      (type) => /* @__PURE__ */ React.createElement("option", { key: type, value: type }, "By Type ",
type !== "All" ? `(${type})` : "")
    )
  ), /* @__PURE__ */ React.createElement(
    "select",
    {
      value: filterBrand,
      onChange: (e) => setFilterBrand(e.target.value),
      className: `h-10 px-3 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65] text-sm
${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-300"}`,
      style: { minWidth: "130px", maxWidth: "150px" }
    },
    uniqueBrands.map(
      (brand) => /* @__PURE__ */ React.createElement("option", { key: brand, value: brand }, "By Brand ",
brand !== "All" ? `(${brand})` : "")
    )
  ), /* @__PURE__ */ React.createElement(
    "select",
    {
      value: filterDate,
      onChange: (e) => setFilterDate(e.target.value),
      className: `h-10 px-3 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65] text-sm
${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-300"}`,
      style: { minWidth: "130px", maxWidth: "150px" }
    },
    dateOptions.map(
      (date) => /* @__PURE__ */ React.createElement("option", { key: date, value: date }, "By Date ",
date !== "All" ? `(${date})` : "")
    )
  ), /* @__PURE__ */ React.createElement(
    "select",
    {
      value: sortBy,
      onChange: (e) => setSortBy(e.target.value),
      className: `h-10 px-3 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65] text-sm
${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-300"}`,
      style: { minWidth: "160px", maxWidth: "180px" }
    },
    sortOptions.map(
      (opt) => /* @__PURE__ */ React.createElement("option", { key: opt.value, value: opt.value }, "Sort By ",
opt.label)
    )
  ), /* @__PURE__ */ React.createElement(
    "select",
    {
      value: filterReason,
      onChange: (e) => setFilterReason(e.target.value),
      className: `h-10 px-3 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65] text-sm
${theme === "dark" ? "bg-[#2A2724] border-gray-700 text-white" : "bg-white border-gray-300"}`,
      style: { minWidth: "140px", maxWidth: "160px" }
    },
    reasons.map(
      (reason) => /* @__PURE__ */ React.createElement("option", { key: reason, value: reason }, "By Reason ",
reason !== "All" ? `(${reason})` : "")
    )
  ))), /* @__PURE__ */ React.createElement("div", { className: `rounded-2xl shadow-md overflow-hidden ${theme
=== "dark" ? "bg-[#2A2724]" : "bg-white"}` }, /* @__PURE__ */ React.createElement("div", { className:
"overflow-x-auto" }, /* @__PURE__ */ React.createElement("table", { className: "w-full" }, /* @__PURE__ */
React.createElement("thead", { className: `${theme === "dark" ? "bg-[#1E1B18]" : "bg-gray-50"}` }, /*
@__PURE__ */ React.createElement("tr", null, isMovementExportMode && /* @__PURE__ */ React.createElement("th",
{ className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider ${theme === "dark" ? "text-
gray-400" : "text-gray-500"}` }, /* @__PURE__ */ React.createElement("label", { className: "flex items-center
gap-2 cursor-pointer select-none" }, /* @__PURE__ */ React.createElement(
    "input",
    {
      ref: selectAllMovementsRef,
      type: "checkbox",
      className: "w-4 h-4 text-[#AD7F65] border-[#AD7F65] rounded focus:ring-[#AD7F65]",
      onChange: handleToggleSelectAllMovements,
      checked: isMovementExportMode ? allVisibleMovementsSelected : false
    }
  ), /* @__PURE__ */ React.createElement("span", null, "All"))), /* @__PURE__ */ React.createElement("th",
{ className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider cursor-pointer ${theme ===
"dark" ? "text-gray-400 hover:bg-[#3A3734]" : "text-gray-500 hover:bg-gray-100"}` }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-1" }, "Date & Time", /* @__PURE__ */
React.createElement("svg", { className: "w-4 h-4", fill: "none", stroke: "currentColor", viewBox: "0 0 24
24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round",
```

```
strokeWidth: 2, d: "M7 16V4m0 0L3 8m4-4l4 4m6 0v12m0 0l4-4m-4 4l-4-4" }))))), /* @__PURE__ */
React.createElement("th", { className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider
${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "SKU"), /* @__PURE__ */ React.createElement("th",
{ className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider ${theme === "dark" ? "text-
gray-400" : "text-gray-500"}` }, "Product name"), /* @__PURE__ */ React.createElement("th", { className: `px-
4 py-3 text-left text-xs font-medium uppercase tracking-wider ${theme === "dark" ? "text-gray-400" : "text-
gray-500"}` }, "Type"), /* @__PURE__ */ React.createElement("th", { className: `px-4 py-3 text-left text-xs
font-medium uppercase tracking-wider ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Quantity"),
/* @__PURE__ */ React.createElement("th", { className: `px-4 py-3 text-left text-xs font-medium uppercase
tracking-wider ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Before"), /* @__PURE__ */
React.createElement("th", { className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider
${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "After"), /* @__PURE__ */
React.createElement("th", { className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider
${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Reason"), /* @__PURE__ */
React.createElement("th", { className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider
${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Handled by"))), /* @__PURE__ */
React.createElement("tbody", { className: `divide-y ${theme === "dark" ? "bg-[#2A2724] divide-gray-700" :
"bg-white divide-gray-200"}` }, loading && movements.length === 0 ? /* @__PURE__ */ React.createElement("tr",
null, /* @__PURE__ */ React.createElement("td", { colSpan: movementTableColumnCount, className: "px-4 py-8
text-center text-gray-500" }, /* @__PURE__ */ React.createElement("div", { className: "flex flex-col items-
center justify-center" }, /* @__PURE__ */ React.createElement("div", { className: "inline-block animate-spin
rounded-full h-8 w-8 border-b-2 border-[#8B7355] mb-2" }), /* @__PURE__ */ React.createElement("span", null,
"Loading..."))))) : movements.length === 0 ? /* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */
React.createElement("td", { colSpan: movementTableColumnCount, className: "px-4 py-8 text-center text-gray-
500" }, "No stock movements found")) : movements.map(
    (movement) => /* @__PURE__ */ React.createElement("tr", { key: movement._id, className: `${theme ===
"dark" ? "hover:bg-[#3A3734] hover:bg-opacity-50" : "hover:bg-gray-50"}` }, isMovementExportMode && /*
@__PURE__ */ React.createElement("td", { className: "px-4 py-3", onClick: (e) => e.stopPropagation() }, /*
@__PURE__ */ React.createElement(
      "input",
      {
        type: "checkbox",
        className: "w-4 h-4 text-[#AD7F65] border-[#AD7F65] rounded focus:ring-[#AD7F65]",
        checked: selectedMovements.some((sm) => sm._id === movement._id),
        onChange: () => handleToggleMovementSelection(movement),
        disabled: !movement._id
      }
    )), /* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 whitespace-nowrap text-sm ${theme
=== "dark" ? "text-gray-200" : "text-gray-900"}` }, formatDateTime(movement.createdAt)), /* @__PURE__ */
React.createElement("td", { className: `px-4 py-3 text-sm font-mono max-w-[140px] truncate ${theme ===
"dark" ? "text-gray-200" : "text-gray-900"}`, title: movement.sku || "" }, movement.sku || "\u2014"), /*
@__PURE__ */ React.createElement("td", { className: `px-4 py-3 text-sm max-w-[220px] ${theme === "dark" ?
"text-gray-200" : "text-gray-900"}` }, /* @__PURE__ */ React.createElement("span", { className: "line-clamp-
2", title: movement.itemName || "" }, movement.itemName || "\u2014")), /* @__PURE__ */
React.createElement("td", { className: "px-4 py-3 whitespace-nowrap" }, getTypeBadge(movement.type)), /*
@__PURE__ */ React.createElement("td", { className: `px-4 py-3 whitespace-nowrap text-sm font-medium
${getQuantityColor(movement.type, movement.quantity)}` }, movement.type === "Stock-In" ? "+" : "-",
movement.quantity), /* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 whitespace-nowrap text-
sm ${theme === "dark" ? "text-gray-200" : "text-gray-900"}` }, movement.stockBefore), /* @__PURE__ */
React.createElement("td", { className: `px-4 py-3 whitespace-nowrap text-sm ${theme === "dark" ? "text-gray-
200" : "text-gray-900"}` }, movement.stockAfter), /* @__PURE__ */ React.createElement("td", { className: `px-
4 py-3 whitespace-nowrap text-sm ${theme === "dark" ? "text-gray-200" : "text-gray-900"}` }, movement.reason),
/* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 whitespace-nowrap text-sm ${theme ===
"dark" ? "text-gray-200" : "text-gray-900"}` }, movement.handledBy))
  )))), movements.length > 0 && /* @__PURE__ */ React.createElement("div", { className: "flex items-center
justify-between mt-5" }, /* @__PURE__ */ React.createElement("div", { className: `text-xs ${theme === "dark" ?
"text-gray-400" : "text-gray-500"}` }, "Page ", currentPage, " of ", totalPages), totalPages > 1 && /*
@__PURE__ */ React.createElement("div", { className: `flex items-center gap-2 rounded-full border px-3 py-1
shadow-inner ${theme === "dark" ? "bg-[#2A2724] border-gray-700" : "bg-white border-gray-200"}` }, /*
@__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setCurrentPage((prev) => Math.max(1, prev - 1)),
      disabled: currentPage === 1,
      className: `p-2 rounded-full ${currentPage === 1 ? "text-gray-300" : `${theme === "dark" ? "hover:bg-
[#3A3734] text-gray-400" : "hover:bg-gray-50 text-gray-600"}`}`
    },
    /* @__PURE__ */ React.createElement(FaChevronLeft, null)
  ), Array.from({ length: totalPages }).slice(0, 5).map((_, idx) => {
    const pageNumber = idx + 1;
    return /* @__PURE__ */ React.createElement(
      "button",
      {
        key: pageNumber,
        onClick: () => setCurrentPage(pageNumber),
        className: `w-8 h-8 rounded-full text-sm font-semibold ${currentPage === pageNumber ? "bg-[#AD7F65]
text-white shadow-md" : `${theme === "dark" ? "text-gray-400 hover:bg-[#3A3734]" : "text-gray-600 hover:bg-
```

```
gray-50"}`}`
      },
      pageNumber
    );
  }), totalPages > 5 && /* @__PURE__ */ React.createElement("span", { className: "text-gray-400 px-2" },
"..."), totalPages > 5 && /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setCurrentPage(totalPages),
      className: `w-8 h-8 rounded-full text-sm font-semibold ${currentPage === totalPages ? "bg-[#AD7F65]
text-white shadow-md" : "text-gray-600 hover:bg-gray-50"}`
    },
    totalPages
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setCurrentPage((prev) => Math.min(totalPages, prev + 1)),
      disabled: currentPage === totalPages,
      className: `p-2 rounded-full ${currentPage === totalPages ? "text-gray-300" : `${theme === "dark" ?
"hover:bg-[#3A3734] text-gray-400" : "hover:bg-gray-50 text-gray-600"}`}`
    },
    /* @__PURE__ */ React.createElement(FaChevronRight, null)
  )))), activeTab === "void-logs" && /* @__PURE__ */ React.createElement("div", { className: `rounded-2xl
shadow-md overflow-hidden ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}` }, /* @__PURE__ */
React.createElement("div", { className: "overflow-x-auto" }, /* @__PURE__ */ React.createElement("table",
{ className: "w-full" }, /* @__PURE__ */ React.createElement("thead", { className: `${theme === "dark" ? "bg-
[#1E1B18]" : "bg-gray-50"}` }, /* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */
React.createElement("th", { className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider
${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Void Number"), /* @__PURE__ */
React.createElement("th", { className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider
${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Void ID"), /* @__PURE__ */
React.createElement("th", { className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider
${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Date & Time"), /* @__PURE__ */
React.createElement("th", { className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider
${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Handled By"), /* @__PURE__ */
React.createElement("th", { className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider
${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Approved By"), /* @__PURE__ */
React.createElement("th", { className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider
${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Total"))), /* @__PURE__ */
React.createElement("tbody", { className: `divide-y ${theme === "dark" ? "bg-[#2A2724] divide-gray-700" :
"bg-white divide-gray-200"}` }, loading && voidLogs.length === 0 ? /* @__PURE__ */ React.createElement("tr",
null, /* @__PURE__ */ React.createElement("td", { colSpan: "6", className: "px-4 py-8 text-center text-gray-
500" }, /* @__PURE__ */ React.createElement("div", { className: "flex flex-col items-center justify-center" },
/* @__PURE__ */ React.createElement("div", { className: "inline-block animate-spin rounded-full h-8 w-8
border-b-2 border-[#8B7355] mb-2" }), /* @__PURE__ */ React.createElement("span", null, "Loading...")))) :
voidLogs.length === 0 ? /* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */
React.createElement("td", { colSpan: "6", className: "px-4 py-8 text-center text-gray-500" }, "No void logs
found")) : voidLogs.map((log, index) => {
    const voidNumber = voidTotalPages === 1 ? voidLogs.length - index : voidTotalPages * rowsPerPage -
(voidCurrentPage - 1) * rowsPerPage - index;
    const getApprovedByDisplay = () => {
      if (log.approvedBy && log.approvedByRole) {
        return `${log.approvedBy} (${log.approvedByRole})`;
      }
      if (log.approvedBy) {
        return log.approvedBy;
      }
      if (log.approvedByRole) {
        return log.approvedByRole;
      }
      return "N/A";
    };
    return /* @__PURE__ */ React.createElement(
      "tr",
      {
        key: log._id,
        className: `cursor-pointer ${theme === "dark" ? "hover:bg-[#3A3734] hover:bg-opacity-50" : "hover:bg-
gray-50"}`,
        onClick: () => handleView(log)
      },
      /* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 whitespace-nowrap text-sm font-bold
${theme === "dark" ? "text-gray-200" : "text-gray-900"}` }, "#", voidNumber),
      /* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 whitespace-nowrap text-sm font-mono
${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, log.voidId || "N/A"),
      /* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 whitespace-nowrap text-sm ${theme ===
"dark" ? "text-gray-200" : "text-gray-900"}` }, formatDateTime(log.voidedAt || log.checkedOutAt ||
log.createdAt)),
```

```
      /* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 whitespace-nowrap text-sm ${theme ===
"dark" ? "text-gray-200" : "text-gray-900"}` }, log.voidedByName || log.performedByName || "N/A"),
      /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 whitespace-nowrap" }, /* @__PURE__ */
React.createElement("span", { className: `inline-flex items-center px-2.5 py-1 rounded-full text-xs font-
medium ${log.approvedByRole === "Owner" ? "bg-purple-100 text-purple-700 border border-purple-200" :
log.approvedByRole === "Manager" ? "bg-blue-100 text-blue-700 border border-blue-200" : "bg-gray-100 text-
gray-600 border border-gray-200"}` }, getApprovedByDisplay())),
      /* @__PURE__ */ React.createElement("td", { className: `px-4 py-3 whitespace-nowrap text-sm font-medium
${theme === "dark" ? "text-gray-200" : "text-gray-900"}` }, "\u20B1", parseFloat(log.totalAmount ||
0).toFixed(2))
    );
  })))), voidLogs.length > 0 && /* @__PURE__ */ React.createElement("div", { className: "flex items-center
justify-between mt-5" }, /* @__PURE__ */ React.createElement("div", { className: `text-xs ${theme === "dark" ?
"text-gray-400" : "text-gray-500"}` }, "Page ", voidCurrentPage, " of ", voidTotalPages), voidTotalPages > 1
&& /* @__PURE__ */ React.createElement("div", { className: `flex items-center gap-2 rounded-full border px-3
py-1 shadow-inner ${theme === "dark" ? "bg-[#2A2724] border-gray-700" : "bg-white border-gray-200"}` }, /*
@__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setVoidCurrentPage((prev) => Math.max(1, prev - 1)),
      disabled: voidCurrentPage === 1,
      className: `p-2 rounded-full ${voidCurrentPage === 1 ? "text-gray-300" : `${theme === "dark" ?
"hover:bg-[#3A3734] text-gray-400" : "hover:bg-gray-50 text-gray-600"}`}`
    },
    /* @__PURE__ */ React.createElement(FaChevronLeft, null)
  ), Array.from({ length: voidTotalPages }).slice(0, 5).map((_, idx) => {
    const pageNumber = idx + 1;
    return /* @__PURE__ */ React.createElement(
      "button",
      {
        key: pageNumber,
        onClick: () => setVoidCurrentPage(pageNumber),
        className: `w-8 h-8 rounded-full text-sm font-semibold ${voidCurrentPage === pageNumber ? "bg-
[#AD7F65] text-white shadow-md" : `${theme === "dark" ? "text-gray-400 hover:bg-[#3A3734]" : "text-gray-600
hover:bg-gray-50"}`}`
      },
      pageNumber
    );
  }), voidTotalPages > 5 && /* @__PURE__ */ React.createElement("span", { className: "text-gray-400 px-2" },
"..."), voidTotalPages > 5 && /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setVoidCurrentPage(voidTotalPages),
      className: `w-8 h-8 rounded-full text-sm font-semibold ${voidCurrentPage === voidTotalPages ? "bg-
[#AD7F65] text-white shadow-md" : "text-gray-600 hover:bg-gray-50"}`
    },
    voidTotalPages
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setVoidCurrentPage((prev) => Math.min(voidTotalPages, prev + 1)),
      disabled: voidCurrentPage === voidTotalPages,
      className: `p-2 rounded-full ${voidCurrentPage === voidTotalPages ? "text-gray-300" : `${theme ===
"dark" ? "hover:bg-[#3A3734] text-gray-400" : "hover:bg-gray-50 text-gray-600"}`}`
    },
    /* @__PURE__ */ React.createElement(FaChevronRight, null)
  )))), /* @__PURE__ */ React.createElement(
    ViewVoidLogModal,
    {
      isOpen: showVoidLogModal,
      onClose: () => {
        setShowVoidLogModal(false);
        setSelectedVoidLog(null);
      },
      voidLog: selectedVoidLog
    }
  ));
};
export default memo(Logs);
```

FILE: pos/frontend/src/pages/ManageData.jsx
==========

```
import jsPDF from "jspdf";
import {
  memo,
```

```
  useCallback,
  useContext,
  useEffect,
  useRef,
  useState
} from "react";
import {
  FaArrowLeft,
  FaDatabase,
  FaDownload,
  FaExclamationTriangle,
  FaFileExcel,
  FaFilePdf,
  FaLock,
  FaSpinner,
  FaTimes,
  FaTimesCircle,
  FaTrash
} from "react-icons/fa";
import { useNavigate } from "react-router-dom";
import SuccessModal from "../components/inventory/SuccessModal";
import Header from "../components/shared/header";
import { API_BASE_URL as API_BASE } from "../config/api";
import { useAuth } from "../context/AuthContext";
import { SidebarContext } from "../context/SidebarContext";
import { useTheme } from "../context/ThemeContext";
import {
  hasBusinessExportSchema,
  recordsForBusinessExport
} from "../utils/manageDataExport";
const PinModal = memo(({ isOpen, onClose, onVerified, theme }) => {
  const [pin, setPin] = useState(["", "", "", "", "", ""]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const inputRefs = useRef([]);
  const isDark = theme === "dark";
  useEffect(() => {
    if (isOpen) {
      setPin(["", "", "", "", "", ""]);
      setError("");
      setTimeout(() => inputRefs.current[0]?.focus(), 100);
    }
  }, [isOpen]);
  const handleInput = (value, index) => {
    if (value && !/^\d$/.test(value)) return;
    const updated = [...pin];
    updated[index] = value;
    setPin(updated);
    if (value && index < 5) {
      inputRefs.current[index + 1]?.focus();
    }
  };
  const handleKeyDown = (e, index) => {
    if (e.key === "Backspace" && !pin[index] && index > 0) {
      inputRefs.current[index - 1]?.focus();
    }
  };
  const handleSubmit = async () => {
    const pinValue = pin.join("");
    if (pinValue.length !== 6) {
      setError("Please enter a complete 6-digit PIN");
      return;
    }
    setLoading(true);
    setError("");
    try {
      const response = await fetch(`${API_BASE}/api/employees/verify-pin`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ pin: pinValue })
      });
      const data = await response.json();
      if (data.success) {
        const role = data.data?.role;
        if (role === "Owner" || role === "Manager") {
          onVerified(pinValue);
        } else {
```

```
          setError(
            "Access denied. Only Owner or Manager can access this feature."
          );
          setPin(["", "", "", "", "", ""]);
          setTimeout(() => inputRefs.current[0]?.focus(), 100);
        }
      } else {
        setError("Invalid PIN. Please try again.");
        setPin(["", "", "", "", "", ""]);
        setTimeout(() => inputRefs.current[0]?.focus(), 100);
      }
    } catch {
      setError("Unable to verify PIN. Please try again.");
    } finally {
      setLoading(false);
    }
  };
  if (!isOpen) return null;
  return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-70 flex items-center
justify-center p-4 backdrop-blur-sm bg-black/30" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `p-8 rounded-3xl shadow-2xl w-full max-w-sm ${isDark ? "bg-[#2A2724]" : "bg-white"}`
    },
    /* @__PURE__ */ React.createElement("div", { className: "flex justify-between items-center mb-6" }, /*
@__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement("div", { className: "w-10 h-10 rounded-xl bg-[#AD7F65] flex items-center justify-center
text-white" }, /* @__PURE__ */ React.createElement(FaLock, { className: "w-4 h-4" })), /* @__PURE__ */
React.createElement(
      "h3",
      {
        className: `text-lg font-bold ${isDark ? "text-white" : "text-gray-900"}`
      },
      "Enter PIN"
    )), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: onClose,
        className: `p-2 rounded-lg transition-colors ${isDark ? "hover:bg-gray-700 text-gray-400" :
"hover:bg-gray-100 text-gray-500"}`
      },
      /* @__PURE__ */ React.createElement(FaTimes, { className: "w-4 h-4" })
    )),
    /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-sm mb-6 text-center ${isDark ? "text-gray-400" : "text-gray-500"}`
      },
      "Owner or Manager PIN required to manage data"
    ),
    /* @__PURE__ */ React.createElement("div", { className: "flex gap-2 justify-center mb-6" }, [0, 1, 2, 3,
4, 5].map(
      (i) => /* @__PURE__ */ React.createElement(
        "input",
        {
          key: i,
          ref: (el) => inputRefs.current[i] = el,
          type: "password",
          maxLength: 1,
          value: pin[i],
          onChange: (e) => handleInput(e.target.value, i),
          onKeyDown: (e) => handleKeyDown(e, i),
          disabled: loading,
          className: `w-12 h-12 text-center text-lg font-bold rounded-xl border-2 shadow-sm focus:border-
[#AD7F65] focus:shadow-md transition-all outline-none disabled:opacity-50 ${isDark ? "bg-[#1E1B18] border-
gray-600 text-white focus:bg-[#352F2A]" : "bg-gray-50 border-gray-200 text-gray-900 focus:bg-white"}`
        }
      )
    )),
    error && /* @__PURE__ */ React.createElement("div", { className: "mb-4 p-3 rounded-xl bg-red-50 border
border-red-200 text-red-700 text-sm text-center flex items-center justify-center gap-2" }, /* @__PURE__ */
React.createElement(FaTimesCircle, { className: "w-3.5 h-3.5 shrink-0" }), error),
    /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: handleSubmit,
        disabled: loading || pin.join("").length !== 6,
```

```
        className: "w-full py-3 rounded-xl font-bold text-white bg-[#AD7F65] hover:bg-[#8e654e] transition-
all disabled:opacity-50 flex items-center justify-center gap-2"
      },
      loading ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(FaSpinner, { className: "w-4 h-4 animate-spin" }), " Verifying...") : "Verify"
    )
  ));
});
const DataSelectionModal = memo(
  ({
    isOpen,
    onClose,
    onConfirm,
    title,
    description,
    confirmLabel,
    confirmColor,
    collections,
    loading,
    theme,
    icon: Icon,
    showFormatPicker
  }) => {
    const [selected, setSelected] = useState({});
    const [exportFormat, setExportFormat] = useState("csv");
    const isDark = theme === "dark";
    useEffect(() => {
      if (isOpen) {
        const initial = {};
        collections.forEach((c) => initial[c.key] = false);
        setSelected(initial);
        setExportFormat("csv");
      }
    }, [isOpen, collections]);
    const toggleAll = () => {
      const allSelected = Object.values(selected).every(Boolean);
      const updated = {};
      Object.keys(selected).forEach((key) => updated[key] = !allSelected);
      setSelected(updated);
    };
    const toggleOne = (key) => {
      setSelected((prev) => ({ ...prev, [key]: !prev[key] }));
    };
    const selectedKeys = Object.entries(selected).filter(([, v]) => v).map(([k]) => k);
    const anySelected = selectedKeys.length > 0;
    if (!isOpen) return null;
    return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-70 flex items-center
justify-center p-4 backdrop-blur-sm bg-black/30" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-3xl shadow-2xl w-full max-w-lg overflow-hidden ${isDark ? "bg-[#2A2724]" : "bg-
white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "px-8 pt-8 pb-4" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center justify-between mb-2" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `w-10 h-10 rounded-xl flex items-center justify-center text-white ${confirmColor ===
"red" ? "bg-red-500" : "bg-[#AD7F65]"}`
        },
        /* @__PURE__ */ React.createElement(Icon, { className: "w-5 h-5" })
      ), /* @__PURE__ */ React.createElement(
        "h3",
        {
          className: `text-lg font-bold ${isDark ? "text-white" : "text-gray-900"}`
        },
        title
      )), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onClose,
          className: `p-2 rounded-lg transition-colors ${isDark ? "hover:bg-gray-700 text-gray-400" :
"hover:bg-gray-100 text-gray-500"}`
        },
        /* @__PURE__ */ React.createElement(FaTimes, { className: "w-4 h-4" })
      )), /* @__PURE__ */ React.createElement(
```

```
        "p",
        {
          className: `text-sm ${isDark ? "text-gray-400" : "text-gray-500"}`
        },
        description
      )),
      /* @__PURE__ */ React.createElement("div", { className: "px-8 pb-2" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: toggleAll,
          className: `text-sm font-semibold transition-colors ${isDark ? "text-[#C2A68C] hover:text-
[#dcc4ad]" : "text-[#AD7F65] hover:text-[#8e654e]"}`
        },
        Object.values(selected).every(Boolean) ? "Deselect All" : "Select All"
      )),
      /* @__PURE__ */ React.createElement("div", { className: "px-8 pb-4 max-h-80 overflow-y-auto" }, /*
@__PURE__ */ React.createElement("div", { className: "space-y-2" }, collections.map(
        (col) => /* @__PURE__ */ React.createElement(
          "label",
          {
            key: col.key,
            className: `flex items-center gap-4 p-3.5 rounded-xl cursor-pointer transition-all border
${selected[col.key] ? isDark ? "bg-[#352F2A] border-[#AD7F65]/50" : "bg-[#FDF8F5] border-[#AD7F65]/30" :
isDark ? "bg-[#1E1B18] border-transparent hover:border-gray-600" : "bg-gray-50 border-transparent
hover:border-gray-200"}`
          },
          /* @__PURE__ */ React.createElement(
            "input",
            {
              type: "checkbox",
              checked: selected[col.key] || false,
              onChange: () => toggleOne(col.key),
              className: "w-5 h-5 rounded-md accent-[#AD7F65] cursor-pointer"
            }
          ),
          /* @__PURE__ */ React.createElement("div", { className: "flex-1" }, /* @__PURE__ */
React.createElement(
            "span",
            {
              className: `text-sm font-semibold ${isDark ? "text-white" : "text-gray-800"}`
            },
            col.name
          )),
          /* @__PURE__ */ React.createElement(
            "span",
            {
              className: `text-xs font-medium px-2.5 py-1 rounded-full ${isDark ? "bg-[#1E1B18] text-gray-
400" : "bg-gray-100 text-gray-500"}`
            },
            col.count.toLocaleString(),
            " records"
          )
        )
      ))),
      showFormatPicker && /* @__PURE__ */ React.createElement("div", { className: "px-8 pb-4" }, /* @__PURE__
*/ React.createElement(
        "label",
        {
          className: `block text-sm font-semibold mb-3 ${isDark ? "text-gray-300" : "text-gray-700"}`
        },
        "Export Format"
      ), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: () => setExportFormat("csv"),
          className: `flex-1 flex items-center justify-center gap-2 py-3 rounded-xl text-sm font-bold
transition-all border-2 ${exportFormat === "csv" ? "border-green-500 bg-green-50 text-green-700 shadow-md
shadow-green-100" : isDark ? "border-gray-600 bg-[#1E1B18] text-gray-400 hover:border-gray-500" : "border-
gray-200 bg-gray-50 text-gray-500 hover:border-gray-300"}`
        },
        /* @__PURE__ */ React.createElement(FaFileExcel, { className: "w-4 h-4" }),
        "CSV (.csv)"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
```

```
          onClick: () => setExportFormat("pdf"),
          className: `flex-1 flex items-center justify-center gap-2 py-3 rounded-xl text-sm font-bold
transition-all border-2 ${exportFormat === "pdf" ? "border-red-500 bg-red-50 text-red-700 shadow-md shadow-
red-100" : isDark ? "border-gray-600 bg-[#1E1B18] text-gray-400 hover:border-gray-500" : "border-gray-200 bg-
gray-50 text-gray-500 hover:border-gray-300"}`
        },
        /* @__PURE__ */ React.createElement(FaFilePdf, { className: "w-4 h-4" }),
        "PDF (.pdf)"
      ))),
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `px-8 py-5 flex justify-end gap-3 ${isDark ? "bg-[#1E1B18]" : "bg-gray-50"}`
        },
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: onClose,
            disabled: loading,
            className: `px-5 py-2.5 rounded-xl font-bold text-sm transition-colors border ${isDark ? "border-
gray-600 text-gray-300 hover:bg-gray-700" : "border-gray-300 text-gray-600 hover:bg-gray-100"}`
          },
          "Cancel"
        ),
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: () => onConfirm(selectedKeys, exportFormat),
            disabled: !anySelected || loading,
            className: `px-6 py-2.5 rounded-xl font-bold text-sm text-white transition-all disabled:opacity-
40 flex items-center gap-2 shadow-md ${confirmColor === "red" ? "bg-red-600 hover:bg-red-700" : "bg-[#AD7F65]
hover:bg-[#8e654e]"}`
          },
          loading ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(FaSpinner, { className: "w-3.5 h-3.5 animate-spin" }), " ", "Processing...") : /*
@__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(Icon, { className:
"w-3.5 h-3.5" }), " ", confirmLabel, " (", selectedKeys.length, ")")
        )
      )
    ));
  }
);
const ClearConfirmModal = memo(
  ({ isOpen, onClose, onConfirm, selectedNames, loading, theme }) => {
    const [confirmText, setConfirmText] = useState("");
    const isDark = theme === "dark";
    useEffect(() => {
      if (isOpen) setConfirmText("");
    }, [isOpen]);
    if (!isOpen) return null;
    return /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-80 flex items-center
justify-center p-4 backdrop-blur-sm bg-black/30" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `p-8 rounded-3xl shadow-2xl w-full max-w-md ${isDark ? "bg-[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3 mb-4" }, /* @__PURE__
*/ React.createElement("div", { className: "w-12 h-12 rounded-xl bg-red-500 flex items-center justify-center
text-white" }, /* @__PURE__ */ React.createElement(FaExclamationTriangle, { className: "w-5 h-5" })), /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "h3",
        {
          className: `text-lg font-bold ${isDark ? "text-white" : "text-gray-900"}`
        },
        "Confirm Deletion"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs ${isDark ? "text-gray-400" : "text-gray-500"}`
        },
        "This action cannot be undone"
      ))),
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `rounded-xl p-4 mb-4 ${isDark ? "bg-red-900/20 border border-red-800/50" : "bg-red-50
border border-red-200"}`
```

```
        },
        /* @__PURE__ */ React.createElement(
          "p",
          {
            className: `text-sm mb-2 font-medium ${isDark ? "text-red-300" : "text-red-800"}`
          },
          "You are about to permanently delete:"
        ),
        /* @__PURE__ */ React.createElement("ul", { className: "space-y-1" }, selectedNames.map(
          (name, i) => /* @__PURE__ */ React.createElement(
            "li",
            {
              key: i,
              className: `text-sm flex items-center gap-2 ${isDark ? "text-red-400" : "text-red-700"}`
            },
            /* @__PURE__ */ React.createElement("span", { className: "w-1.5 h-1.5 rounded-full bg-red-400" }),
            name
          )
        ))
      ),
      /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm mb-3 ${isDark ? "text-gray-400" : "text-gray-600"}`
        },
        "Type ",
        /* @__PURE__ */ React.createElement("span", { className: "font-bold text-red-600" }, "DELETE"),
        " to confirm:"
      ),
      /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "text",
          value: confirmText,
          onChange: (e) => setConfirmText(e.target.value),
          placeholder: "Type DELETE",
          className: `w-full px-4 py-3 rounded-xl border-2 text-sm font-semibold mb-4 focus:outline-none
transition-all ${isDark ? "bg-[#1E1B18] border-gray-600 text-white focus:border-red-500" : "bg-gray-50
border-gray-200 text-gray-900 focus:border-red-500"}`
        }
      ),
      /* @__PURE__ */ React.createElement("div", { className: "flex gap-3" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: onClose,
          disabled: loading,
          className: `flex-1 py-3 rounded-xl font-bold text-sm transition-colors border ${isDark ? "border-
gray-600 text-gray-300 hover:bg-gray-700" : "border-gray-300 text-gray-600 hover:bg-gray-100"}`
        },
        "Cancel"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: onConfirm,
          disabled: confirmText !== "DELETE" || loading,
          className: "flex-1 py-3 rounded-xl font-bold text-sm text-white bg-red-600 hover:bg-red-700
transition-all disabled:opacity-40 flex items-center justify-center gap-2"
        },
        loading ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(FaSpinner, { className: "w-3.5 h-3.5 animate-spin" }), " Deleting...") : /* @__PURE__ */
React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(FaTrash, { className: "w-3.5 h-
3.5" }), " Delete Permanently")
      ))
    ));
  }
);
const ManageDataWrapper = () => {
  const { theme } = useTheme();
  const navigate = useNavigate();
  const [verifiedPin, setVerifiedPin] = useState(null);
  const [showPinModal, setShowPinModal] = useState(true);
  const handlePinVerified = (pin) => {
    setVerifiedPin(pin);
    setShowPinModal(false);
  };
  const handlePinClose = () => {
```

```
    setShowPinModal(false);
    navigate("/settings");
  };
  if (!verifiedPin) {
    return /* @__PURE__ */ React.createElement(
      PinModal,
      {
        isOpen: showPinModal,
        onClose: handlePinClose,
        onVerified: handlePinVerified,
        theme
      }
    );
  }
  return /* @__PURE__ */ React.createElement(ManageDataInner, { verifiedPin });
};
const ManageDataInner = ({ verifiedPin }) => {
  const { isExpanded } = useContext(SidebarContext);
  const { theme } = useTheme();
  const { currentUser } = useAuth();
  const navigate = useNavigate();
  const isDark = theme === "dark";
  const [collections, setCollections] = useState([]);
  const [collectionsLoading, setCollectionsLoading] = useState(true);
  const [showClearModal, setShowClearModal] = useState(false);
  const [showClearConfirm, setShowClearConfirm] = useState(false);
  const [showExportModal, setShowExportModal] = useState(false);
  const [clearLoading, setClearLoading] = useState(false);
  const [exportLoading, setExportLoading] = useState(false);
  const [pendingClearKeys, setPendingClearKeys] = useState([]);
  const [showSuccessModal, setShowSuccessModal] = useState(false);
  const [successMessage, setSuccessMessage] = useState("");
  const [error, setError] = useState("");
  const fetchCollections = useCallback(async () => {
    setCollectionsLoading(true);
    try {
      const res = await fetch(`${API_BASE}/api/data-management/collections`);
      const data = await res.json();
      if (data.success) {
        setCollections(data.data);
      }
    } catch (err) {
      console.error("Error fetching collections:", err);
    } finally {
      setCollectionsLoading(false);
    }
  }, []);
  useEffect(() => {
    fetchCollections();
  }, [fetchCollections]);
  const handleClearSelect = (selectedKeys) => {
    setPendingClearKeys(selectedKeys);
    setShowClearModal(false);
    setShowClearConfirm(true);
  };
  const handleClearConfirm = async () => {
    setClearLoading(true);
    setError("");
    try {
      const res = await fetch(`${API_BASE}/api/data-management/clear`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          collections: pendingClearKeys,
          pin: verifiedPin
        })
      });
      const data = await res.json();
      if (data.success) {
        setSuccessMessage(data.message);
        setShowSuccessModal(true);
        setShowClearConfirm(false);
        fetchCollections();
      } else {
        setError(data.message || "Failed to clear data");
      }
    } catch {
```

```
      setError("Network error. Please try again.");
    } finally {
      setClearLoading(false);
    }
  };
  const handleExport = async (selectedKeys, format) => {
    setExportLoading(true);
    setError("");
    try {
      const res = await fetch(`${API_BASE}/api/data-management/export`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ collections: selectedKeys })
      });
      if (!res.ok) {
        const errorText = await res.text();
        console.error("Export API error:", res.status, errorText);
        setError(`Export failed: Server returned ${res.status}`);
        setExportLoading(false);
        return;
      }
      let data;
      try {
        data = await res.json();
      } catch (parseError) {
        console.error("JSON parse error:", parseError);
        setError("Export failed: Invalid response from server");
        setExportLoading(false);
        return;
      }
      if (!data.success) {
        setError(data.message || "Export failed");
        setExportLoading(false);
        return;
      }
      const timestamp = (/* @__PURE__ */ new Date()).toISOString().split("T")[0];
      try {
        if (format === "csv") {
          exportToCsv(data, timestamp);
        } else {
          exportToPdf(data, timestamp);
        }
      } catch (formatError) {
        console.error("Export format error:", formatError);
      }
      exportJsonBackup(data, timestamp);
      setShowExportModal(false);
      setSuccessMessage(
        `Data exported successfully as ${format.toUpperCase()}! A JSON backup file was also saved.`
      );
      setShowSuccessModal(true);
    } catch (err) {
      console.error("Export error:", err);
      setError(`Network error during export: ${err.message || "Please check your connection and try
again."}`);
    } finally {
      setExportLoading(false);
    }
  };
  const flattenObject = (obj, prefix = "") => {
    const result = {};
    for (const [key, value] of Object.entries(obj)) {
      const newKey = prefix ? `${prefix}.${key}` : key;
      if (key === "_id" || key === "__v") {
        result[newKey] = String(value);
      } else if (Array.isArray(value)) {
        if (value.length > 0 && typeof value[0] === "object") {
          result[newKey] = value.map((item) => {
            if (typeof item === "object") {
              return Object.entries(item).filter(([k]) => k !== "_id").map(([k, v]) => `${k}: ${v}`).join(",
");
            }
            return String(item);
          }).join(" | ");
        } else {
          result[newKey] = value.join(", ");
        }
```

```
      } else if (value && typeof value === "object" && !(value instanceof Date)) {
        Object.assign(result, flattenObject(value, newKey));
      } else {
        result[newKey] = value;
      }
    }
    return result;
  };
  const formatHeaderName = (header) => {
    return header.replace(/([A-Z])/g, " $1").replace(/[._]/g, " ").replace(/^\s/, "").split(" ").map((w) =>
w.charAt(0).toUpperCase() + w.slice(1)).join(" ");
  };
  const csvEscape = (value) => {
    const s = String(value ?? "");
    if (/[",\n\r]/.test(s)) return `"${s.replace(/"/g, '""')}"`;
    return s;
  };
  const buildExportSections = (data) => {
    const exportedCounts = {};
    const sections = [];
    Object.entries(data.data || {}).forEach(([key, records]) => {
      if (!records || records.length === 0) return;
      const businessRows = recordsForBusinessExport(key, records);
      let flatRecords;
      if (hasBusinessExportSchema(key)) {
        if (businessRows.length === 0) {
          exportedCounts[key] = 0;
          return;
        }
        flatRecords = businessRows;
      } else {
        flatRecords = records.map((record) => flattenObject(record));
      }
      exportedCounts[key] = flatRecords.length;
      const name = collections.find((c) => c.key === key)?.name || key;
      sections.push({ key, name, flatRecords });
    });
    return { sections, exportedCounts };
  };
  const exportToCsv = (data, timestamp) => {
    const { sections, exportedCounts } = buildExportSections(data);
    const lines = [];
    lines.push("POS Data Export");
    lines.push(`Exported: ${(/* @__PURE__ */ new Date()).toLocaleString()}`);
    lines.push("");
    sections.forEach(({ name, flatRecords }) => {
      lines.push(`=== ${name} (${flatRecords.length} records) ===`);
      if (flatRecords.length === 0) {
        lines.push("");
        return;
      }
      const headers = Object.keys(flatRecords[0] || {});
      lines.push(headers.map(csvEscape).join(","));
      flatRecords.forEach((row) => {
        lines.push(headers.map((h) => csvEscape(row[h])).join(","));
      });
      lines.push("");
    });
    lines.push("=== Export Summary ===");
    lines.push(["Collection", "Records Exported"].map(csvEscape).join(","));
    const totalExported = data.summary.reduce(
      (sum, s) => sum + (exportedCounts[s.key] ?? s.count),
      0
    );
    data.summary.forEach((s) => {
      const n = exportedCounts[s.key] ?? s.count;
      lines.push([csvEscape(s.name), csvEscape(String(n))].join(","));
    });
    lines.push([csvEscape("TOTAL"), csvEscape(String(totalExported))].join(","));
    const BOM = "\uFEFF";
    const blob = new Blob([BOM + lines.join("\r\n")], {
      type: "text/csv;charset=utf-8;"
    });
    const link = document.createElement("a");
    link.href = URL.createObjectURL(blob);
    link.download = `POS_Data_Export_${timestamp}.csv`;
    document.body.appendChild(link);
```

```
  link.click();
  document.body.removeChild(link);
  URL.revokeObjectURL(link.href);
};
const exportToPdf = (data, timestamp) => {
  const { sections, exportedCounts } = buildExportSections(data);
  const doc = new jsPDF({
    orientation: "landscape",
    unit: "mm",
    format: "a4"
  });
  let isFirstPage = true;
  sections.forEach(({ name: collectionName, flatRecords }) => {
    if (!isFirstPage) doc.addPage();
    isFirstPage = false;
    const headers = Object.keys(flatRecords[0] || {});
    const nCols = Math.max(headers.length, 1);
    const colWidth = (297 - 28) / nCols;
    const startY = 28;
    doc.setFontSize(16);
    doc.setFont("helvetica", "bold");
    doc.setTextColor(60, 60, 60);
    doc.text(`${collectionName} (${flatRecords.length} records)`, 14, 15);
    doc.setFontSize(8);
    doc.setFont("helvetica", "normal");
    doc.text(`Exported: ${(/* @__PURE__ */ new Date()).toLocaleString()}`, 14, 21);
    doc.setFontSize(7);
    doc.setTextColor(120, 120, 120);
    doc.text(
      "Same columns and rows as CSV export; long values may wrap.",
      14,
      25
    );
    doc.setFillColor(173, 127, 101);
    doc.rect(14, startY, 297 - 28, 8, "F");
    doc.setFontSize(6);
    doc.setFont("helvetica", "bold");
    doc.setTextColor(255, 255, 255);
    headers.forEach((h, i) => {
      doc.text(formatHeaderName(h), 15 + i * colWidth, startY + 5.5, {
        maxWidth: colWidth - 2
      });
    });
    doc.setFont("helvetica", "normal");
    doc.setTextColor(60, 60, 60);
    const maxRows = Math.min(flatRecords.length, 35);
    flatRecords.slice(0, maxRows).forEach((record, rowIdx) => {
      const y = startY + 8 + rowIdx * 5;
      if (y > 195) return;
      if (rowIdx % 2 === 0) {
        doc.setFillColor(245, 245, 245);
        doc.rect(14, y, 297 - 28, 5, "F");
      }
      doc.setFontSize(5);
      headers.forEach((h, colIdx) => {
        const value = String(record[h] ?? "");
        doc.text(value, 15 + colIdx * colWidth, y + 3.5, {
          maxWidth: colWidth - 2
        });
      });
    });
    if (flatRecords.length > maxRows) {
      doc.setFontSize(7);
      doc.setTextColor(150, 150, 150);
      doc.text(
        `... and ${flatRecords.length - maxRows} more rows (see CSV for full data)`,
        14,
        200
      );
    }
  });
  if (sections.length > 0) {
    doc.addPage();
  }
  doc.setFontSize(18);
  doc.setFont("helvetica", "bold");
  doc.setTextColor(60, 60, 60);
```

```
    doc.text("Export Summary", 14, 20);
    doc.setFontSize(9);
    doc.setFont("helvetica", "normal");
    doc.text(`Generated: ${(/* @__PURE__ */ new Date()).toLocaleString()}`, 14, 28);
    let summaryY = 38;
    const pdfTotal = data.summary.reduce(
      (sum, s) => sum + (exportedCounts[s.key] ?? s.count),
      0
    );
    data.summary.forEach((s) => {
      doc.setFontSize(10);
      doc.setFont("helvetica", "bold");
      doc.text(s.name, 14, summaryY);
      doc.setFont("helvetica", "normal");
      const n = exportedCounts[s.key] ?? s.count;
      doc.text(`${n.toLocaleString()} records`, 120, summaryY);
      summaryY += 7;
    });
    doc.setFont("helvetica", "bold");
    doc.setFontSize(11);
    summaryY += 5;
    doc.text("Total Records:", 14, summaryY);
    doc.text(pdfTotal.toLocaleString(), 120, summaryY);
    doc.save(`POS_Data_Export_${timestamp}.pdf`);
  };
  const exportJsonBackup = (data, timestamp) => {
    const backup = {
      ...data,
      _meta: {
        exportedAt: (/* @__PURE__ */ new Date()).toISOString(),
        exportedBy: currentUser?.name || "Unknown",
        version: "1.0",
        type: "pos-system-backup"
      }
    };
    const blob = new Blob([JSON.stringify(backup, null, 2)], {
      type: "application/json"
    });
    const link = document.createElement("a");
    link.href = URL.createObjectURL(blob);
    link.download = `POS_Backup_${timestamp}.json`;
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
  };
  const pendingClearNames = pendingClearKeys.map((key) => collections.find((c) => c.key === key)?.name ||
key).filter(Boolean);
  const totalRecords = collections.reduce((sum, c) => sum + c.count, 0);
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `p-8 min-h-screen ${isDark ? "bg-[#1E1B18]" : "bg-gray-50"}`
    },
    /* @__PURE__ */ React.createElement(Header, { pageName: "Manage Data", showBorder: false }),
    /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => navigate("/settings"),
        className: `flex items-center gap-2 mb-6 text-sm font-semibold transition-colors ${isDark ? "text-
gray-400 hover:text-white" : "text-gray-500 hover:text-gray-800"}`
      },
      /* @__PURE__ */ React.createElement(FaArrowLeft, { className: "w-3.5 h-3.5" }),
      "Back to Settings"
    ),
    collectionsLoading ? /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-
center min-h-[50vh]" }, /* @__PURE__ */ React.createElement(FaSpinner, { className: "w-8 h-8 text-[#AD7F65]
animate-spin" })) : /* @__PURE__ */ React.createElement("div", { className: "max-w-4xl mx-auto space-y-6" },
/* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-3xl shadow-lg p-8 ${isDark ? "bg-[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3 mb-6" }, /* @__PURE__
*/ React.createElement("div", { className: "w-10 h-10 rounded-xl bg-[#AD7F65] flex items-center justify-
center text-white" }, /* @__PURE__ */ React.createElement(FaDatabase, { className: "w-5 h-5" })), /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "h3",
```

```
        {
          className: `text-lg font-bold ${isDark ? "text-white" : "text-gray-800"}`
        },
        "Data Overview"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm ${isDark ? "text-gray-400" : "text-gray-500"}`
        },
        totalRecords.toLocaleString(),
        " total records across",
        " ",
        collections.length,
        " collections"
      ))),
      /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 md:grid-cols-3 gap-3" },
collections.map(
        (col) => /* @__PURE__ */ React.createElement(
          "div",
          {
            key: col.key,
            className: `p-4 rounded-2xl border transition-all ${isDark ? "bg-[#1E1B18] border-gray-700" :
"bg-gray-50 border-gray-100"}`
          },
          /* @__PURE__ */ React.createElement(
            "p",
            {
              className: `text-sm font-semibold mb-1 ${isDark ? "text-white" : "text-gray-800"}`
            },
            col.name
          ),
          /* @__PURE__ */ React.createElement(
            "p",
            {
              className: `text-2xl font-bold ${isDark ? "text-[#C2A68C]" : "text-[#AD7F65]"}`
            },
            col.count.toLocaleString()
          )
        )
      ))
    ), error && /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-xl p-4 flex items-center gap-3 ${isDark ? "bg-red-900/20 border border-red-800
text-red-400" : "bg-red-50 border border-red-200 text-red-700"}`
      },
      /* @__PURE__ */ React.createElement(FaTimesCircle, { className: "w-4 h-4 shrink-0" }),
      /* @__PURE__ */ React.createElement("span", { className: "text-sm font-medium" }, error),
      /* @__PURE__ */ React.createElement("button", { onClick: () => setError(""), className: "ml-auto" }, /*
@__PURE__ */ React.createElement(FaTimes, { className: "w-3.5 h-3.5" }))
    ), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-1 md:grid-cols-2 gap-4" }, /*
@__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-3xl shadow-lg p-6 flex flex-col ${isDark ? "bg-[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3 mb-3" }, /* @__PURE__
*/ React.createElement("div", { className: "w-10 h-10 rounded-xl bg-red-500 flex items-center justify-center
text-white" }, /* @__PURE__ */ React.createElement(FaTrash, { className: "w-4 h-4" })), /* @__PURE__ */
React.createElement(
        "h3",
        {
          className: `text-base font-bold ${isDark ? "text-white" : "text-gray-800"}`
        },
        "Clear Data"
      )),
      /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm mb-6 flex-1 ${isDark ? "text-gray-400" : "text-gray-500"}`
        },
        "Permanently delete selected data from both cloud and local storage. Choose which data types to
clear."
      ),
      /* @__PURE__ */ React.createElement(
        "button",
        {
```

```
          onClick: () => setShowClearModal(true),
          className: "w-full py-3 rounded-xl font-bold text-sm text-white bg-red-600 hover:bg-red-700
transition-all flex items-center justify-center gap-2"
        },
        /* @__PURE__ */ React.createElement(FaTrash, { className: "w-3.5 h-3.5" }),
        "Clear Data"
      )
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-3xl shadow-lg p-6 flex flex-col ${isDark ? "bg-[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3 mb-3" }, /* @__PURE__
*/ React.createElement("div", { className: "w-10 h-10 rounded-xl bg-[#AD7F65] flex items-center justify-
center text-white" }, /* @__PURE__ */ React.createElement(FaDownload, { className: "w-4 h-4" })), /*
@__PURE__ */ React.createElement(
        "h3",
        {
          className: `text-base font-bold ${isDark ? "text-white" : "text-gray-800"}`
        },
        "Export Data"
      )),
      /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm mb-6 flex-1 ${isDark ? "text-gray-400" : "text-gray-500"}`
        },
        "Export all your data as PDF or CSV. Select which collections to export. A JSON backup is always
included."
      ),
      /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => setShowExportModal(true),
          className: "w-full py-3 rounded-xl font-bold text-sm text-white bg-[#AD7F65] hover:bg-[#8e654e]
transition-all flex items-center justify-center gap-2"
        },
        /* @__PURE__ */ React.createElement(FaDownload, { className: "w-3.5 h-3.5" }),
        "Export Data"
      )
    ))),
    /* @__PURE__ */ React.createElement(
      DataSelectionModal,
      {
        isOpen: showClearModal,
        onClose: () => setShowClearModal(false),
        onConfirm: handleClearSelect,
        title: "Clear Data",
        description: "Select which data you want to permanently delete. This will remove data from both cloud
and local databases.",
        confirmLabel: "Continue",
        confirmColor: "red",
        collections,
        loading: false,
        theme,
        icon: FaTrash,
        showFormatPicker: false
      }
    ),
    /* @__PURE__ */ React.createElement(
      ClearConfirmModal,
      {
        isOpen: showClearConfirm,
        onClose: () => setShowClearConfirm(false),
        onConfirm: handleClearConfirm,
        selectedNames: pendingClearNames,
        loading: clearLoading,
        theme
      }
    ),
    /* @__PURE__ */ React.createElement(
      DataSelectionModal,
      {
        isOpen: showExportModal,
        onClose: () => setShowExportModal(false),
        onConfirm: handleExport,
        title: "Export Data",
```

```
        description: "Select which data to export and choose your preferred format.",
        confirmLabel: "Export",
        confirmColor: "brown",
        collections,
        loading: exportLoading,
        theme,
        icon: FaDownload,
        showFormatPicker: true
      }
    ),
    /* @__PURE__ */ React.createElement(
      SuccessModal,
      {
        isOpen: showSuccessModal,
        onClose: () => {
          setShowSuccessModal(false);
          setError("");
        },
        message: successMessage
      }
    )
  );
};
export default memo(ManageDataWrapper);
```

FILE: pos/frontend/src/pages/owner/BrandPartners.jsx
==========

```
import { memo, useEffect, useState } from "react";
import { FaArchive, FaEdit, FaPlus, FaSearch, FaUndo } from "react-icons/fa";
import { useNavigate } from "react-router-dom";
import AddBrandPartnerModal from "../../components/owner/AddBrandPartnerModal";
import ArchiveBrandPartnerModal from "../../components/owner/ArchiveBrandPartnerModal";
import EditBrandPartnerModal from "../../components/owner/EditBrandPartnerModal";
import ViewBrandProductsModal from "../../components/owner/ViewBrandProductsModal";
import Header from "../../components/shared/header";
import { useAuth } from "../../context/AuthContext";
import { useTheme } from "../../context/ThemeContext";
const BrandPartners = () => {
  const { isOwner } = useAuth();
  const { theme } = useTheme();
  const navigate = useNavigate();
  const [searchQuery, setSearchQuery] = useState("");
  const [filterStatus, setFilterStatus] = useState("all");
  const [selectedBrand, setSelectedBrand] = useState(null);
  const [showViewModal, setShowViewModal] = useState(false);
  const [showAddModal, setShowAddModal] = useState(false);
  const [showEditModal, setShowEditModal] = useState(false);
  const [showArchiveModal, setShowArchiveModal] = useState(false);
  const [brandPartners, setBrandPartners] = useState([]);
  const [loading, setLoading] = useState(true);
  const [fetchError, setFetchError] = useState("");
  const fetchBrandPartners = async () => {
    try {
      setLoading(true);
      const response = await fetch("http://localhost:5000/api/brand-partners");
      const data = await response.json();
      if (!response.ok || !data.success) {
        throw new Error(data.message || "Failed to load brand partners.");
      }
      setBrandPartners(data.data || []);
      setFetchError("");
    } catch (error) {
      console.error("Error fetching brand partners:", error);
      setFetchError(error.message || "Failed to load brand partners.");
    } finally {
      setLoading(false);
    }
  };
  useEffect(() => {
    fetchBrandPartners();
  }, []);
  const filteredBrandPartners = brandPartners.filter((brand) => {
    const matchesSearch = brand.brandName?.toLowerCase().includes(searchQuery.toLowerCase()) ||
brand.contactPerson?.toLowerCase().includes(searchQuery.toLowerCase());
    if (filterStatus === "all") return matchesSearch;
```

```
    return matchesSearch && brand.status === filterStatus;
  });
  return /* @__PURE__ */ React.createElement("div", { className: `p-8 min-h-screen ${theme === "dark" ? "bg-
[#1E1B18]" : "bg-gray-50"}` }, /* @__PURE__ */ React.createElement(Header, { pageName: "Brand Partners",
showBorder: false }), /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-
between mb-6 mt-10" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-4" },
/* @__PURE__ */ React.createElement("div", { className: "relative", style: { width: "450px" } }, /* @__PURE__
*/ React.createElement("div", { className: "absolute left-1 top-1/2 transform -translate-y-1/2 w-12 h-9 flex
items-center justify-center text-white rounded-lg", style: { background: "linear-gradient(135deg, #AD7F65 0%,
#76462B 100%)" } }, /* @__PURE__ */ React.createElement(FaSearch, { className: "text-sm" })), /* @__PURE__ */
React.createElement(
    "input",
    {
      type: "text",
      placeholder: "Search For...",
      value: searchQuery,
      onChange: (e) => setSearchQuery(e.target.value),
      className: `w-full h-10 pl-16 pr-4 py-3 border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-[#4A4037] text-white
placeholder-gray-500" : "bg-white border-gray-300 text-gray-900"}`
    }
  )), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3 ml-4" }, /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: () => setFilterStatus("all"),
      className: `px-6 py-2.5 text-sm font-bold rounded-xl transition-all shadow-sm border ${filterStatus ===
"all" ? "bg-white text-[#AD7F65] border-gray-100 border-b-[4px] border-b-[#AD7F65]" : "bg-white text-gray-800
border-gray-200 border-b-[4px] border-b-gray-200 hover:bg-gray-50"}`
    },
    "All"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setFilterStatus("active"),
      className: `px-6 py-2.5 text-sm font-bold rounded-xl transition-all shadow-sm border ${filterStatus ===
"active" ? "bg-white text-[#AD7F65] border-gray-100 border-b-[4px] border-b-[#AD7F65]" : "bg-white text-gray-
800 border-gray-200 border-b-[4px] border-b-gray-200 hover:bg-gray-50"}`
    },
    "Active"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setFilterStatus("archived"),
      className: `px-6 py-2.5 text-sm font-bold rounded-xl transition-all shadow-sm border ${filterStatus ===
"archived" ? "bg-white text-[#AD7F65] border-gray-100 border-b-[4px] border-b-[#AD7F65]" : "bg-white text-
gray-800 border-gray-200 border-b-[4px] border-b-gray-200 hover:bg-gray-50"}`
    },
    "Archived"
  ))), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setShowAddModal(true),
      className: "flex items-center gap-2 px-6 py-3 text-white rounded-lg font-medium shadow-md hover:shadow-
lg transition-all",
      style: { background: "linear-gradient(135deg, #10B981 0%, #059669 100%)" }
    },
    /* @__PURE__ */ React.createElement(FaPlus, { className: "w-4 h-4" }),
    "Add Brand Partner"
  )), fetchError && /* @__PURE__ */ React.createElement("div", { className: "mb-4 px-4 py-3 rounded-lg bg-
red-50 text-red-700 border border-red-100 text-sm" }, fetchError), loading && /* @__PURE__ */
React.createElement("div", { className: "mb-6 px-4 py-3 rounded-lg bg-white shadow text-gray-500" }, "Loading
brand partners..."), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-3 gap-
6" }, !loading && filteredBrandPartners.length === 0 && /* @__PURE__ */ React.createElement("div",
{ className: "col-span-3" }, /* @__PURE__ */ React.createElement("div", { className: `rounded-2xl border
border-dashed p-10 text-center ${theme === "dark" ? "bg-[#2A2724] border-[#4A4037] text-gray-400" : "bg-white
border-gray-200 text-gray-500"}` }, "No brand partners found. Add your first brand partner to get started.")),
filteredBrandPartners.map(
    (brand) => /* @__PURE__ */ React.createElement(
      "div",
      {
        key: brand._id || brand.id,
        className: `rounded-xl shadow-md p-4 flex flex-col justify-between relative ${theme === "dark" ? "bg-
[#2A2724]" : "bg-white"} ${brand.status === "archived" ? "grayscale opacity-70" : ""}`,
        style: { minHeight: "auto" }
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex items-start gap-3" }, /* @__PURE__ */
```

```
React.createElement("div", { className: "w-16 h-16 shrink-0 rounded-full overflow-hidden bg-gray-200" },
brand.logo ? /* @__PURE__ */ React.createElement(
        "img",
        {
          src: brand.logo,
          alt: brand.brandName,
          className: "w-full h-full object-cover"
        }
      ) : /* @__PURE__ */ React.createElement("div", { className: "w-full h-full flex items-center justify-
center bg-gradient-to-br from-[#AD7F65] to-[#76462B] text-white text-xl font-bold" },
brand.brandName.charAt(0))), /* @__PURE__ */ React.createElement("div", { className: "flex-1 pt-1" }, /*
@__PURE__ */ React.createElement("h3", { className: `text-lg font-bold mb-1 ${theme === "dark" ? "text-
white" : "text-gray-900"}` }, brand.brandName), /* @__PURE__ */ React.createElement("div", { className:
`space-y-0.5 text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-600"}` }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement("span", { className: "opacity-70" },
"Email:"), " ", brand.email), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("span", { className: "opacity-70" }, "Contact Person:"), " ", brand.contactPerson || "-"),
/* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("span", { className:
"opacity-70" }, "Contact Number:"), " ", brand.contactNumber || "-")))),
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-2 mt-4 justify-end" },
/* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => {
            setSelectedBrand(brand);
            setShowViewModal(true);
          },
          className: `px-3 py-1.5 rounded-lg text-xs font-medium transition-colors ${theme === "dark" ? "bg-
gray-700 text-gray-200 hover:bg-gray-600" : "bg-gray-200 text-gray-800 hover:bg-gray-300"}`
        },
        "View Products"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => {
            setSelectedBrand(brand);
            setShowEditModal(true);
          },
          className: "w-8 h-8 flex items-center justify-center rounded-lg bg-[#007AFF] text-white hover:bg-
blue-600 transition-colors shadow-sm"
        },
        /* @__PURE__ */ React.createElement(FaEdit, { className: "w-3.5 h-3.5" })
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => {
            setSelectedBrand(brand);
            setShowArchiveModal(true);
          },
          className: `w-8 h-8 flex items-center justify-center rounded-lg text-white transition-colors
shadow-sm ${brand.status === "archived" ? "bg-[#10B981] hover:bg-green-600" : "bg-[#FFA500] hover:bg-orange-
500"}`,
          title: brand.status === "archived" ? "Unarchive" : "Archive"
        },
        brand.status === "archived" ? /* @__PURE__ */ React.createElement(FaUndo, { className: "w-3.5 h-
3.5" }) : /* @__PURE__ */ React.createElement(FaArchive, { className: "w-3.5 h-3.5" })
      ))
    )
  )), /* @__PURE__ */ React.createElement(
    ViewBrandProductsModal,
    {
      isOpen: showViewModal,
      onClose: () => {
        setShowViewModal(false);
        setSelectedBrand(null);
      },
      brandPartner: selectedBrand,
      onEdit: (brand) => {
        setSelectedBrand(brand);
        setShowViewModal(false);
        setShowEditModal(true);
      },
      onDelete: (brand) => {
        setSelectedBrand(brand);
        setShowViewModal(false);
        setShowArchiveModal(true);
      }
```

```
    }
  ), /* @__PURE__ */ React.createElement(
    AddBrandPartnerModal,
    {
      isOpen: showAddModal,
      onClose: () => setShowAddModal(false),
      onSuccess: (newBrand) => {
        setBrandPartners((prev) => [newBrand, ...prev]);
      }
    }
  ), /* @__PURE__ */ React.createElement(
    EditBrandPartnerModal,
    {
      isOpen: showEditModal,
      brandPartner: selectedBrand,
      onClose: () => {
        setShowEditModal(false);
        setSelectedBrand(null);
      },
      onSuccess: (updatedBrand) => {
        setBrandPartners(
          (prev) => prev.map((b) => b._id === updatedBrand._id ? updatedBrand : b)
        );
        setSelectedBrand(updatedBrand);
        setShowViewModal(true);
      }
    }
  ), /* @__PURE__ */ React.createElement(
    ArchiveBrandPartnerModal,
    {
      isOpen: showArchiveModal,
      brandPartner: selectedBrand,
      onClose: () => {
        setShowArchiveModal(false);
        setSelectedBrand(null);
      },
      onSuccess: (updatedBrand) => {
        setBrandPartners(
          (prev) => prev.map((b) => b._id === updatedBrand._id ? updatedBrand : b)
        );
      }
    }
  ));
};
export default memo(BrandPartners);
```

FILE: pos/frontend/src/pages/owner/Categories.jsx
==========

```
import React, { memo, useEffect, useMemo, useState } from "react";
import { FaChevronDown, FaChevronRight, FaEdit, FaPlus, FaSearch, FaTimes, FaTrash, FaUndo } from "react-
icons/fa";
import ViewCategoryProductsModal from "../../components/owner/ViewCategoryProductsModal";
import Header from "../../components/shared/header";
import { API_BASE_URL } from "../../config/api";
import { useAuth } from "../../context/AuthContext";
import { useTheme } from "../../context/ThemeContext";
const Categories = () => {
  const { currentUser } = useAuth();
  const { theme } = useTheme();
  const [searchQuery, setSearchQuery] = useState("");
  const [filterStatus, setFilterStatus] = useState("All");
  const [showAddMainModal, setShowAddMainModal] = useState(false);
  const [showAddSubModal, setShowAddSubModal] = useState(false);
  const [showEditModal, setShowEditModal] = useState(false);
  const [showViewProductsModal, setShowViewProductsModal] = useState(false);
  const [selectedCategory, setSelectedCategory] = useState(null);
  const [editingCategory, setEditingCategory] = useState(null);
  const [categories, setCategories] = useState([]);
  const [detailedSubcategoryCounts, setDetailedSubcategoryCounts] = useState({});
  const [loading, setLoading] = useState(true);
  const [categoryName, setCategoryName] = useState("");
  const [isActive, setIsActive] = useState(true);
  const [showOnPos, setShowOnPos] = useState(true);
  const [selectedParentCategory, setSelectedParentCategory] = useState("");
  const [error, setError] = useState("");
```

```
const [showArchiveCategoryModal, setShowArchiveCategoryModal] = useState(false);
const [categoryToArchive, setCategoryToArchive] = useState(null);
const [collapsedCategories, setCollapsedCategories] = useState({});
const categoryStructure = {
  "Apparel - Men": ["Tops", "Bottoms", "Outerwear"],
  "Apparel - Women": ["Tops", "Bottoms", "Dresses", "Outerwear"],
  "Apparel - Kids": ["Tops", "Bottoms", "Dresses", "Outerwear"],
  "Apparel - Unisex": ["Tops", "Bottoms", "Dresses", "Outerwear"],
  "Foods": ["Beverages", "Snacks", "Meals", "Desserts", "Ingredients", "Other"],
  "Makeup": ["Face", "Eyes", "Lips", "Nails", "SkinCare", "Others"],
  "Accessories": ["Jewelry", "Bags", "Head Wear", "Glasses/Sunglasses", "Others"],
  "Shoes": ["Sneakers", "Boots", "Sandals", "Others"],
  "Essentials": ["Daily Essentials", "Personal Care", "Home Essentials", "Others"]
};
const builtInCategories = [
  ...Object.keys(categoryStructure),
  ...new Set(Object.values(categoryStructure).flat())
];
useEffect(() => {
  fetchCategories().then((cats) => {
    initializeBuiltInCategories(Array.isArray(cats) ? cats : []);
  });
}, []);
const initializeBuiltInCategories = async (existingData) => {
  try {
    const existingCats = existingData.filter((c) => c.type !== "subcategory").map((c) => c.name);
    const existingSubs = existingData.filter((c) => c.type === "subcategory");
    const othersCategories = existingData.filter((cat) => cat.name === "Others");
    let needsRefresh = false;
    if (othersCategories.length > 0) {
      const archivePromises = othersCategories.map(
        (cat) => fetch(`${API_BASE_URL}/api/categories/${cat._id}/archive`, {
          method: "PATCH"
        }).catch((error2) => null)
      );
      await Promise.all(archivePromises);
      needsRefresh = true;
    }
    const knownSubNames = new Set(Object.values(categoryStructure).flat());
    const mistyped = existingData.filter(
      (c) => c.type !== "subcategory" && knownSubNames.has(c.name)
    );
    if (mistyped.length > 0) {
      const fixPromises = mistyped.map((cat) => {
        const parentName = Object.entries(categoryStructure).find(
          ([, subs]) => subs.includes(cat.name)
        )?.[0] || null;
        if (!parentName) return null;
        return fetch(`${API_BASE_URL}/api/categories/${cat._id}`, {
          method: "PUT",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ type: "subcategory", parentCategory: parentName })
        }).catch(() => null);
      }).filter(Boolean);
      await Promise.all(fixPromises);
      needsRefresh = true;
    }
    const createPromises = [];
    Object.keys(categoryStructure).forEach((parentName) => {
      if (!existingCats.includes(parentName)) {
        createPromises.push(
          fetch(`${API_BASE_URL}/api/categories`, {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ name: parentName, type: "category", status: "active" })
          }).catch(() => null)
        );
      }
    });
    Object.entries(categoryStructure).forEach(([parentName, subList]) => {
      subList.forEach((subName) => {
        const subExists = existingSubs.some((s) => s.name === subName && s.parentCategory === parentName);
        if (!subExists) {
          createPromises.push(
            fetch(`${API_BASE_URL}/api/categories`, {
              method: "POST",
              headers: { "Content-Type": "application/json" },
```

```
                body: JSON.stringify({ name: subName, type: "subcategory", parentCategory: parentName, status:
"active" })
              }).catch(() => null)
            );
          }
        });
      });
      if (createPromises.length > 0) {
        await Promise.all(createPromises);
        needsRefresh = true;
      }
      if (needsRefresh) {
        const response = await fetch(`${API_BASE_URL}/api/categories`);
        const data = await response.json();
        if (data.success && Array.isArray(data.data)) {
          setCategories(data.data);
        }
      }
    } catch (error2) {
      console.error("Error initializing categories background sync:", error2);
    }
  };
  const fetchCategories = async () => {
    try {
      setLoading(true);
      const response = await fetch(`${API_BASE_URL}/api/categories`);
      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }
      const data = await response.json();
      if (data.success && Array.isArray(data.data)) {
        setCategories(data.data);
        if (data.detailedSubcategoryCounts) {
          setDetailedSubcategoryCounts(data.detailedSubcategoryCounts);
        }
        return data.data;
      } else {
        console.warn("Invalid response format:", data);
        setCategories([]);
        return [];
      }
    } catch (error2) {
      console.error("Error fetching categories:", error2);
      setCategories([]);
      return [];
    } finally {
      setLoading(false);
    }
  };
  const handleAddCategory = async (type) => {
    if (!categoryName.trim()) {
      setError("Category name is required");
      return;
    }
    if (type === "subcategory" && !selectedParentCategory) {
      setError("Parent category is required");
      return;
    }
    try {
      setError("");
      const payload = {
        name: categoryName.trim(),
        status: isActive ? "active" : "inactive",
        showOnPos,
        type
      };
      if (type === "subcategory") {
        payload.parentCategory = selectedParentCategory;
      }
      const response = await fetch(`${API_BASE_URL}/api/categories`, {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify(payload)
      });
      const data = await response.json();
```

```
    if (data.success) {
      setCategoryName("");
      setShowAddMainModal(false);
      setShowAddSubModal(false);
      setIsActive(true);
      setShowOnPos(true);
      setSelectedParentCategory("");
      fetchCategories();
    } else {
      setError(data.message || "Failed to create category");
    }
  } catch (error2) {
    console.error("Error creating category:", error2);
    setError("Error creating category");
  }
};
const handleEditCategory = async () => {
  if (!categoryName.trim()) {
    setError("Category name is required");
    return;
  }
  try {
    setError("");
    const response = await fetch(`${API_BASE_URL}/api/categories/${editingCategory._id}`, {
      method: "PUT",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        name: categoryName.trim()
      })
    });
    const data = await response.json();
    if (data.success) {
      setCategoryName("");
      setEditingCategory(null);
      setShowEditModal(false);
      fetchCategories();
    } else {
      setError(data.message || "Failed to update category");
    }
  } catch (error2) {
    console.error("Error updating category:", error2);
    setError("Error updating category");
  }
};
const handleToggleStatus = async (id) => {
  try {
    const response = await fetch(`${API_BASE_URL}/api/categories/${id}/archive`, {
      method: "PATCH"
    });
    const data = await response.json();
    if (data.success) {
      fetchCategories();
    } else {
      alert("Failed to update category status");
    }
  } catch (error2) {
    console.error("Error updating category status:", error2);
    alert("Error updating category status");
  }
};
const handleDelete = (category) => {
  setCategoryToArchive(category);
  setShowArchiveCategoryModal(true);
};
const handleConfirmArchive = async () => {
  if (!categoryToArchive) return;
  try {
    const response = await fetch(`${API_BASE_URL}/api/categories/${categoryToArchive._id}/archive`, {
      method: "PATCH"
    });
    const data = await response.json();
    if (data.success) {
      setShowArchiveCategoryModal(false);
      setCategoryToArchive(null);
      fetchCategories();
```

```
      } else {
        alert(data.message || "Failed to archive category");
      }
    } catch (error2) {
      console.error("Error archiving category:", error2);
      alert("Error archiving category");
    }
  };
  const handleEdit = (category) => {
    setEditingCategory(category);
    setCategoryName(category.name);
    setError("");
    setShowEditModal(true);
  };
  const handleViewProducts = (category) => {
    setSelectedCategory(category);
    setShowViewProductsModal(true);
  };
  const knownSubcategoryNames = useMemo(() => {
    const names = /* @__PURE__ */ new Set();
    Object.values(categoryStructure).forEach((subs) => subs.forEach((s) => names.add(s)));
    return names;
  }, []);
  const groupedCategories = useMemo(() => {
    const mainCats = categories.filter(
      (cat) => cat.type !== "subcategory" && cat.name !== "Others" && !knownSubcategoryNames.has(cat.name)
    );
    const subCats = categories.filter(
      (cat) => cat.type === "subcategory" || knownSubcategoryNames.has(cat.name)
    );
    const results = [];
    mainCats.forEach((main) => {
      const matchMainSearch = main.name.toLowerCase().includes(searchQuery.toLowerCase());
      const matchMainFilter = filterStatus === "All" || filterStatus === "Active" && main.status === "active"
|| filterStatus === "Archived" && main.status === "inactive";
      const builtInNames = categoryStructure[main.name] || [];
      const builtInSubs = builtInNames.map((name) => {
        const sub = subCats.find((s) => s.name === name);
        if (sub) {
          const splitCount = detailedSubcategoryCounts[main.name] &&
detailedSubcategoryCounts[main.name][name] ? detailedSubcategoryCounts[main.name][name] : 0;
          return { ...sub, parentCategory: main.name, productCount: splitCount };
        }
        return null;
      }).filter(Boolean);
      const customSubs = subCats.filter(
        (sub) => sub.parentCategory === main.name && !builtInNames.includes(sub.name)
      );
      const relatedSubs = [...builtInSubs, ...customSubs];
      const filteredSubs = relatedSubs.filter((sub) => {
        const matchSearch = sub.name.toLowerCase().includes(searchQuery.toLowerCase());
        const matchFilter = filterStatus === "All" || filterStatus === "Active" && sub.status === "active" ||
filterStatus === "Archived" && sub.status === "inactive";
        return matchSearch && matchFilter;
      });
      if (matchMainSearch && matchMainFilter || filteredSubs.length > 0) {
        results.push({
          ...main,
          subcategories: filteredSubs
        });
      }
    });
    return results;
  }, [categories, searchQuery, filterStatus]);
  useEffect(() => {
    setCollapsedCategories((prev) => {
      const next = { ...prev };
      groupedCategories.forEach((main) => {
        if (next[main._id] === void 0) {
          next[main._id] = true;
        }
      });
      return next;
    });
  }, [groupedCategories]);
  return /* @__PURE__ */ React.createElement("div", { className: `p-8 min-h-screen ${theme === "dark" ? "bg-
[#1E1B18]" : "bg-[#F9FAFB]"}` }, /* @__PURE__ */ React.createElement(
```

```
    Header,
    {
      pageName: "Categories",
      profileBackground: theme === "dark" ? "bg-[#2A2724]" : "bg-gray-100",
      showBorder: false,
      userName: currentUser?.name || "Owner",
      userRole: "Owner"
    }
  ), /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-4 mb-6 justify-between
mt-5" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-30" }, /* @__PURE__
*/ React.createElement("div", { className: "relative", style: { maxWidth: "400px" } }, /* @__PURE__ */
React.createElement("div", { className: "absolute left-1 top-1/2 transform -translate-y-1/2 w-10 h-9 flex
items-center justify-center text-white rounded-xl", style: { background: "linear-gradient(135deg, #AD7F65 0%,
#76462B 100%)" } }, /* @__PURE__ */ React.createElement(FaSearch, { className: "text-sm" })), /* @__PURE__ */
React.createElement(
    "input",
    {
      type: "text",
      placeholder: "Search For...",
      value: searchQuery,
      onChange: (e) => setSearchQuery(e.target.value),
      className: `w-[500px] h-11 pl-14 pr-4 border rounded-xl focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] focus:border-transparent transition-colors ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-
white placeholder-gray-400" : "bg-white border-gray-300 text-gray-900 placeholder-gray-500"}`
    }
  )), /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: () => setFilterStatus("All"),
      className: `px-6 py-2.5 text-sm font-bold rounded-xl transition-all shadow-sm border ${filterStatus ===
"All" ? theme === "dark" ? "bg-[#2A2724] text-[#AD7F65] border-[#4A4037] border-b-[4px] border-b-[#AD7F65]" :
"bg-white text-[#AD7F65] border-gray-100 border-b-[4px] border-b-[#AD7F65]" : theme === "dark" ? "bg-[#2A2724]
text-gray-300 border-[#4A4037] border-b-[4px] border-b-[#4A4037] hover:bg-[#3A3734]" : "bg-white text-gray-
800 border-gray-200 border-b-[4px] border-b-gray-200 hover:bg-gray-50"}`
    },
    "All"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setFilterStatus("Active"),
      className: `px-6 py-2.5 text-sm font-bold rounded-xl transition-all shadow-sm border ${filterStatus ===
"Active" ? theme === "dark" ? "bg-[#2A2724] text-[#AD7F65] border-[#4A4037] border-b-[4px] border-b-
[#AD7F65]" : "bg-white text-[#AD7F65] border-gray-100 border-b-[4px] border-b-[#AD7F65]" : theme === "dark" ?
"bg-[#2A2724] text-gray-300 border-[#4A4037] border-b-[4px] border-b-[#4A4037] hover:bg-[#3A3734]" : "bg-
white text-gray-800 border-gray-200 border-b-[4px] border-b-gray-200 hover:bg-gray-50"}`
    },
    "Active"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setFilterStatus("Archived"),
      className: `px-6 py-2.5 text-sm font-bold rounded-xl transition-all shadow-sm border ${filterStatus ===
"Archived" ? theme === "dark" ? "bg-[#2A2724] text-[#AD7F65] border-[#4A4037] border-b-[4px] border-b-
[#AD7F65]" : "bg-white text-[#AD7F65] border-gray-100 border-b-[4px] border-b-[#AD7F65]" : theme === "dark" ?
"bg-[#2A2724] text-gray-300 border-[#4A4037] border-b-[4px] border-b-[#4A4037] hover:bg-[#3A3734]" : "bg-
white text-gray-800 border-gray-200 border-b-[4px] border-b-gray-200 hover:bg-gray-50"}`
    },
    "Archived"
  ))), /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: () => {
        setCategoryName("");
        setError("");
        setSelectedParentCategory("");
        setIsActive(true);
        setShowOnPos(true);
        setShowAddSubModal(true);
      },
      className: `flex items-center gap-2 px-6 py-3 rounded-lg font-medium shadow-sm transition-all border
${theme === "dark" ? "bg-[#2A2724] text-white border-gray-600 hover:bg-[#3A3734]" : "bg-white text-gray-800
border-gray-300 hover:bg-gray-50"}`
    },
    /* @__PURE__ */ React.createElement(FaPlus, { className: "w-3.5 h-3.5" }),
    "Add subcategory"
```

```
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => {
        setCategoryName("");
        setError("");
        setIsActive(true);
        setShowOnPos(true);
        setShowAddMainModal(true);
      },
      className: "flex items-center gap-2 px-6 py-3 text-white rounded-lg font-medium shadow-md hover:shadow-
lg transition-all hover:opacity-90",
      style: { background: "linear-gradient(135deg, #10B981 0%, #059669 100%)" }
    },
    /* @__PURE__ */ React.createElement(FaPlus, { className: "w-4 h-4" }),
    "Add main category"
  ))), "      ", loading ? /* @__PURE__ */ React.createElement("div", { className: "flex justify-center
items-center py-12" }, /* @__PURE__ */ React.createElement("div", { className: "text-gray-500" }, "Loading
categories...")) : groupedCategories.length === 0 ? /* @__PURE__ */ React.createElement("div", { className:
"flex justify-center items-center py-12" }, /* @__PURE__ */ React.createElement("div", { className: "text-
gray-500" }, "No categories found. Create your first category!")) : /* @__PURE__ */ React.createElement("div",
{ className: `overflow-x-auto rounded-xl border ${theme === "dark" ? "border-[#4A4037]" : "border-gray-200"}
bg-white dark:bg-[#1E1B18]` }, /* @__PURE__ */ React.createElement("table", { className: "w-full text-left
border-collapse" }, /* @__PURE__ */ React.createElement("thead", null, /* @__PURE__ */
React.createElement("tr", { className: `text-xs uppercase tracking-wider ${theme === "dark" ? "bg-[#2A2724]
text-gray-400" : "bg-gray-50 text-gray-500"} border-b ${theme === "dark" ? "border-[#4A4037]" : "border-gray-
200"}` }, /* @__PURE__ */ React.createElement("th", { className: "px-6 py-4 font-semibold" }, "Name"), /*
@__PURE__ */ React.createElement("th", { className: "px-6 py-4 font-semibold" }, "Type"), /* @__PURE__ */
React.createElement("th", { className: "px-6 py-4 font-semibold" }, "Products"), /* @__PURE__ */
React.createElement("th", { className: "px-6 py-4 font-semibold" }, "Status"), /* @__PURE__ */
React.createElement("th", { className: "px-6 py-4 font-semibold" }, "POS"), /* @__PURE__ */
React.createElement("th", { className: "px-6 py-4 font-semibold text-right" }, "Actions"))), /* @__PURE__ */
React.createElement("tbody", { className: `divide-y ${theme === "dark" ? "divide-[#4A4037]" : "divide-gray-
100"}` }, groupedCategories.map((mainGroup) => /* @__PURE__ */ React.createElement(React.Fragment, { key:
mainGroup._id }, /* @__PURE__ */ React.createElement("tr", { className: `hover:bg-opacity-50 transition-
colors ${theme === "dark" ? "hover:bg-[#3A3734] bg-[#2A2724]/30" : "hover:bg-gray-50 bg-white"}` }, /*
@__PURE__ */ React.createElement("td", { className: "px-6 py-4" }, /* @__PURE__ */ React.createElement("div",
{ className: "flex items-center gap-2" }, mainGroup.subcategories && mainGroup.subcategories.length > 0 && /*
@__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setCollapsedCategories((prev) => ({ ...prev, [mainGroup._id]: !prev[mainGroup._id] })),
      className: `p-1 rounded transition-colors ${theme === "dark" ? "hover:bg-[#3A3734] text-gray-400" :
"hover:bg-gray-200 text-gray-500"}`
    },
    collapsedCategories[mainGroup._id] ? /* @__PURE__ */ React.createElement(FaChevronRight, { className: "w-
3 h-3" }) : /* @__PURE__ */ React.createElement(FaChevronDown, { className: "w-3 h-3" })
  ), /* @__PURE__ */ React.createElement("span", { className: `font-bold ${theme === "dark" ? "text-white" :
"text-gray-900"} text-base` }, mainGroup.name), mainGroup.subcategories && mainGroup.subcategories.length > 0
&& /* @__PURE__ */ React.createElement("span", { className: `text-xs ${theme === "dark" ? "text-gray-500" :
"text-gray-400"}` }, "(", mainGroup.subcategories.length, ")"))), /* @__PURE__ */ React.createElement("td",
{ className: "px-6 py-4" }, /* @__PURE__ */ React.createElement("span", { className: "inline-flex items-
center px-2.5 py-1 rounded-full text-xs font-semibold bg-blue-50 text-blue-600 border border-blue-100" },
"Main category")), /* @__PURE__ */ React.createElement("td", { className: "px-6 py-4" }, /* @__PURE__ */
React.createElement("span", { className: `text-sm ${theme === "dark" ? "text-gray-300" : "text-gray-600"}` },
mainGroup.productCount || 0)), /* @__PURE__ */ React.createElement("td", { className: "px-6 py-4" }, /*
@__PURE__ */ React.createElement("div", { className: "flex items-center gap-2" }, /* @__PURE__ */
React.createElement("div", { className: `w-2 h-2 rounded-full ${mainGroup.status === "active" ? "bg-
[#10B981]" : "bg-gray-400"}` }), /* @__PURE__ */ React.createElement("span", { className: `text-sm
${mainGroup.status === "active" ? "text-[#10B981]" : "text-gray-500"}` }, mainGroup.status === "active" ?
"Active" : "Inactive"))), /* @__PURE__ */ React.createElement("td", { className: "px-6 py-4" }, /* @__PURE__
*/ React.createElement("span", { className: `text-sm ${mainGroup.showOnPos !== false ? "text-[#10B981]" :
"text-gray-400"}` }, mainGroup.showOnPos !== false ? "Yes" : "No")), /* @__PURE__ */ React.createElement("td",
{ className: "px-6 py-4 text-right" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-
center justify-end gap-1" }, !builtInCategories.includes(mainGroup.name) && /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: () => handleEdit(mainGroup),
      className: `p-2 rounded-lg transition-colors group ${theme === "dark" ? "hover:bg-[#352F2A]" :
"hover:bg-blue-50"}`,
      title: "Edit"
    },
    /* @__PURE__ */ React.createElement("svg", { className: "w-5 h-5 text-blue-500 group-hover:text-blue-600",
fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path",
{ strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M11 5H6a2 2 0 00-2 2v11a2 2 0 002
2h11a2 2 0 002-2v-5m-1.414-9.414a2 2 0 112.828 2.828L11.828 15H9v-2.828l8.586-8.586z" }))
```

```
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => {
        setCategoryName("");
        setError("");
        setSelectedParentCategory(mainGroup.name);
        setIsActive(true);
        setShowOnPos(true);
        setShowAddSubModal(true);
      },
      className: `px-4 py-1.5 text-xs font-medium border rounded-md transition-colors ${theme === "dark" ?
"border-gray-600 text-gray-300 hover:bg-gray-700" : "border-gray-200 text-gray-700 hover:bg-gray-50"}`
    },
    "+ Sub"
  ), mainGroup.status === "active" ? /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => handleDelete(mainGroup),
      className: `p-2 rounded-lg transition-colors group ${theme === "dark" ? "hover:bg-[#352F2A]" :
"hover:bg-orange-50"}`,
      title: "Archive"
    },
    /* @__PURE__ */ React.createElement("svg", { className: "w-5 h-5 text-orange-400 group-hover:text-orange-
500", fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */
React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M5 8h14M5
8a2 2 0 110-4h14a2 2 0 110 4M5 8v10a2 2 0 002 2h10a2 2 0 002-2V8m-9 4h4" }))
  ) : /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => handleToggleStatus(mainGroup._id),
      className: `p-2 rounded-lg transition-colors group ${theme === "dark" ? "hover:bg-[#352F2A]" :
"hover:bg-green-50"}`,
      title: "Activate"
    },
    /* @__PURE__ */ React.createElement("svg", { className: "w-5 h-5 text-green-500 group-hover:text-green-
600", fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */
React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M9 12l2 2
4-4m6 2a9 9 0 11-18 0 9 9 0 0118 0z" }))
  )))), !collapsedCategories[mainGroup._id] && mainGroup.subcategories && mainGroup.subcategories.map((subCat)
=> /* @__PURE__ */ React.createElement("tr", { key: subCat._id, className: `hover:bg-opacity-50 transition-
colors ${theme === "dark" ? "hover:bg-[#3A3734] bg-[#1E1B18]" : "hover:bg-gray-50 bg-white"}` }, /* @__PURE__
*/ React.createElement("td", { className: "px-6 py-4 pl-12" }, /* @__PURE__ */ React.createElement("div",
{ className: "flex items-center gap-2" }, /* @__PURE__ */ React.createElement("span", { className: "text-
gray-300 dark:text-[#4A4037] text-lg leading-none transform -translate-y-[2px]" }, "\u2514"), /* @__PURE__ */
React.createElement("span", { className: `text-sm ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` },
subCat.name))), /* @__PURE__ */ React.createElement("td", { className: "px-6 py-4" }, /* @__PURE__ */
React.createElement("span", { className: "inline-flex items-center px-2.5 py-1 rounded-full text-xs font-
semibold bg-green-50 text-green-600 border border-green-100" }, "Subcategory")), /* @__PURE__ */
React.createElement("td", { className: "px-6 py-4" }, /* @__PURE__ */ React.createElement("span", { className:
`text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, subCat.productCount || 0)), /* @__PURE__
*/ React.createElement("td", { className: "px-6 py-4" }, /* @__PURE__ */ React.createElement("div",
{ className: "flex items-center gap-2" }, /* @__PURE__ */ React.createElement("div", { className: `w-2 h-2
rounded-full ${subCat.status === "active" ? "bg-[#10B981]" : "bg-gray-400"}` }), /* @__PURE__ */
React.createElement("span", { className: `text-sm ${subCat.status === "active" ? "text-[#10B981]" : "text-
gray-500"}` }, subCat.status === "active" ? "Active" : "Inactive"))), /* @__PURE__ */
React.createElement("td", { className: "px-6 py-4" }, /* @__PURE__ */ React.createElement("span", { className:
`text-sm ${subCat.showOnPos !== false ? "text-[#10B981]" : "text-gray-400"}` }, subCat.showOnPos !== false ?
"Yes" : "No")), /* @__PURE__ */ React.createElement("td", { className: "px-6 py-4 text-right" }, /* @__PURE__
*/ React.createElement("div", { className: "flex items-center justify-end gap-
1" }, !builtInCategories.includes(subCat.name) && /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => handleEdit(subCat),
      className: `p-2 rounded-lg transition-colors group ${theme === "dark" ? "hover:bg-[#352F2A]" :
"hover:bg-blue-50"}`,
      title: "Edit"
    },
    /* @__PURE__ */ React.createElement("svg", { className: "w-5 h-5 text-blue-500 group-hover:text-blue-600",
fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path",
{ strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M11 5H6a2 2 0 00-2 2v11a2 2 0 002
2h11a2 2 0 002-2v-5m-1.414-9.414a2 2 0 112.828 2.828L11.828 15H9v-2.828l8.586-8.586z" }))
  ), subCat.status === "active" ? /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => handleDelete(subCat),
      className: `p-2 rounded-lg transition-colors group ${theme === "dark" ? "hover:bg-[#352F2A]" :
```

```
"hover:bg-orange-50"}`,
      title: "Archive"
    },
    /* @__PURE__ */ React.createElement("svg", { className: "w-5 h-5 text-orange-400 group-hover:text-orange-
500", fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */
React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M5 8h14M5
8a2 2 0 110-4h14a2 2 0 110 4M5 8v10a2 2 0 002 2h10a2 2 0 002-2V8m-9 4h4" }))
  ) : /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => handleToggleStatus(subCat._id),
      className: `p-2 rounded-lg transition-colors group ${theme === "dark" ? "hover:bg-[#352F2A]" :
"hover:bg-green-50"}`,
      title: "Activate"
    },
    /* @__PURE__ */ React.createElement("svg", { className: "w-5 h-5 text-green-500 group-hover:text-green-
600", fill: "none", stroke: "currentColor", viewBox: "0 0 24 24" }, /* @__PURE__ */
React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round", strokeWidth: 2, d: "M9 12l2 2
4-4m6 2a9 9 0 11-18 0 9 9 0 0118 0z" }))
  )))))))))), showAddMainModal && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0
backdrop-blur-sm bg-black/30 flex items-center justify-center z-50 p-4" }, /* @__PURE__ */
React.createElement("div", { className: `rounded-2xl p-8 w-full max-w-lg ${theme === "dark" ? "bg-[#1E1B18]
text-white" : "bg-white text-gray-900"} relative shadow-2xl transition-all transform scale-100` }, /*
@__PURE__ */ React.createElement("div", { className: "flex items-center justify-between mb-8" }, /* @__PURE__
*/ React.createElement("div", { className: "flex items-center gap-4" }, /* @__PURE__ */
React.createElement("div", { className: "w-12 h-12 rounded-xl flex items-center justify-center text-white
text-xl shadow-lg bg-[#AD7F65]" }, /* @__PURE__ */ React.createElement(FaPlus, null)), /* @__PURE__ */
React.createElement("h2", { className: "text-2xl font-bold" }, "Add Main Category")), /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: () => {
        setShowAddMainModal(false);
        setCategoryName("");
        setError("");
      },
      className: `p-2 rounded-full hover:bg-opacity-10 transition-colors ${theme === "dark" ? "hover:bg-white
text-gray-400" : "hover:bg-black text-gray-400"}`
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "text-xl" })
  )), "            ", /* @__PURE__ */ React.createElement("div", { className: "mb-6" }, /* @__PURE__ */
React.createElement("label", { className: `block text-sm font-semibold mb-2 ml-1 ${theme === "dark" ? "text-
gray-300" : "text-gray-700"}` }, "Category Name"), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      value: categoryName,
      onChange: (e) => {
        setCategoryName(e.target.value);
        setError("");
      },
      className: `w-full px-4 py-3 border rounded-xl focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
transition-all
                    ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white placeholder-gray-500" :
"bg-white border-gray-200 text-gray-900 placeholder-gray-400 shadow-sm"}`,
      placeholder: "eg. Casual Wear",
      autoFocus: true
    }
  ), error && /* @__PURE__ */ React.createElement("p", { className: "text-red-500 text-sm mt-2 ml-1 flex
items-center gap-1" }, /* @__PURE__ */ React.createElement(FaTimes, { className: "text-xs" }), " ", error)),
/* @__PURE__ */ React.createElement("div", { className: "flex flex-col gap-4 mb-8" }, /* @__PURE__ */
React.createElement("div", { className: `flex items-center justify-between py-3 border-b ${theme === "dark" ?
"border-[#4A4037]" : "border-gray-100"}` }, /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("h4", { className: `font-semibold ${theme === "dark" ? "text-white" : "text-gray-900"}` },
"Active"), /* @__PURE__ */ React.createElement("p", { className: `text-xs mt-1 ${theme === "dark" ? "text-
gray-400" : "text-gray-500"}` }, "Inactive categories are hidden from POS and reports.")), /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: () => setIsActive(!isActive),
      className: `relative inline-flex h-6 w-11 items-center rounded-full transition-colors focus:outline-
none focus:ring-2 focus:ring-[#AD7F65] focus:ring-offset-2 ${isActive ? "bg-[#AD7F65]" : "bg-gray-300"}`
    },
    /* @__PURE__ */ React.createElement("span", { className: `inline-block h-4 w-4 transform rounded-full bg-
white transition-transform ${isActive ? "translate-x-6" : "translate-x-1"}` })
  )), /* @__PURE__ */ React.createElement("div", { className: `flex items-center justify-between py-3 border-
b ${theme === "dark" ? "border-[#4A4037]" : "border-gray-100"}` }, /* @__PURE__ */ React.createElement("div",
```

```
null, /* @__PURE__ */ React.createElement("h4", { className: `font-semibold ${theme === "dark" ? "text-
white" : "text-gray-900"}` }, "Show on POS screen"), /* @__PURE__ */ React.createElement("p", { className:
`text-xs mt-1 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Display as a quick-filter button
on the cashier screen.")), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setShowOnPos(!showOnPos),
      className: `relative inline-flex h-6 w-11 items-center rounded-full transition-colors focus:outline-
none focus:ring-2 focus:ring-[#AD7F65] focus:ring-offset-2 ${showOnPos ? "bg-[#AD7F65]" : "bg-gray-300"}`
    },
    /* @__PURE__ */ React.createElement("span", { className: `inline-block h-4 w-4 transform rounded-full bg-
white transition-transform ${showOnPos ? "translate-x-6" : "translate-x-1"}` })
  ))), /* @__PURE__ */ React.createElement("div", { className: "flex gap-4 justify-center" }, /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: () => {
        setShowAddMainModal(false);
        setCategoryName("");
        setError("");
      },
      className: `px-8 py-3 rounded-xl font-bold transition-all transform active:scale-95 ${theme === "dark" ?
"bg-[#2A2724] text-gray-300 hover:bg-[#3A3734]" : "bg-gray-200 text-gray-700 hover:bg-gray-300"}`
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => handleAddCategory("category"),
      className: "px-8 py-3 text-white rounded-xl font-bold shadow-lg hover:shadow-xl transition-all
transform active:scale-95 bg-[#09A046] hover:bg-green-600"
    },
    "Add main category"
  )))), showAddSubModal && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 backdrop-
blur-sm bg-black/30 flex items-center justify-center z-50 p-4" }, /* @__PURE__ */ React.createElement("div",
{ className: `rounded-2xl p-8 w-full max-w-lg ${theme === "dark" ? "bg-[#1E1B18] text-white" : "bg-white
text-gray-900"} relative shadow-2xl transition-all transform scale-100` }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center justify-between mb-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-4" }, /* @__PURE__ */
React.createElement("div", { className: "w-12 h-12 rounded-xl flex items-center justify-center text-white-700
text-xl shadow-md border bg-[#AD7F65]" }, /* @__PURE__ */ React.createElement(FaPlus, null)), /* @__PURE__ */
React.createElement("h2", { className: "text-2xl font-bold" }, "Add Subcategory")), /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: () => {
        setShowAddSubModal(false);
        setCategoryName("");
        setSelectedParentCategory("");
        setError("");
      },
      className: `p-2 rounded-full hover:bg-opacity-10 transition-colors ${theme === "dark" ? "hover:bg-white
text-gray-400" : "hover:bg-black text-gray-400"}`
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "text-xl" })
  )), /* @__PURE__ */ React.createElement("div", { className: "mb-4" }, /* @__PURE__ */
React.createElement("label", { className: `block text-sm font-semibold mb-2 ml-1 ${theme === "dark" ? "text-
gray-300" : "text-gray-700"}` }, "Parent category"), /* @__PURE__ */ React.createElement(
    "select",
    {
      value: selectedParentCategory,
      onChange: (e) => {
        setSelectedParentCategory(e.target.value);
        setError("");
      },
      className: `w-full px-4 py-3 border rounded-xl focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
transition-all
                    ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white" : "bg-white border-gray-
200 text-gray-900 shadow-sm"}`
    },
    /* @__PURE__ */ React.createElement("option", { value: "", disabled: true }, "Select parent category..."),
    categories.filter((c) => c.type !== "subcategory").map((c) => /* @__PURE__ */
React.createElement("option", { key: c._id, value: c.name }, c.name))
  ), /* @__PURE__ */ React.createElement("p", { className: `text-xs mt-2 ml-1 ${theme === "dark" ? "text-
gray-500" : "text-gray-400"}` }, "This will be a subcategory.")), /* @__PURE__ */ React.createElement("div",
{ className: "mb-6" }, /* @__PURE__ */ React.createElement("label", { className: `block text-sm font-semibold
mb-2 ml-1 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Subcategory Name"), /* @__PURE__ */
```

```
React.createElement(
    "input",
    {
      type: "text",
      value: categoryName,
      onChange: (e) => {
        setCategoryName(e.target.value);
        setError("");
      },
      className: `w-full px-4 py-3 border rounded-xl focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
transition-all
                    ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white placeholder-gray-500" :
"bg-white border-gray-200 text-gray-900 placeholder-gray-400 shadow-sm"}`,
      placeholder: "eg. Casual Wear"
    }
  ), error && /* @__PURE__ */ React.createElement("p", { className: "text-red-500 text-sm mt-2 ml-1 flex
items-center gap-1" }, /* @__PURE__ */ React.createElement(FaTimes, { className: "text-xs" }), " ", error)),
/* @__PURE__ */ React.createElement("div", { className: "flex flex-col gap-4 mb-8" }, /* @__PURE__ */
React.createElement("div", { className: `flex items-center justify-between py-3 border-b ${theme === "dark" ?
"border-[#4A4037]" : "border-gray-100"}` }, /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("h4", { className: `font-semibold ${theme === "dark" ? "text-white" : "text-gray-900"}` },
"Active"), /* @__PURE__ */ React.createElement("p", { className: `text-xs mt-1 ${theme === "dark" ? "text-
gray-400" : "text-gray-500"}` }, "Inactive categories are hidden from POS and reports.")), /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: () => setIsActive(!isActive),
      className: `relative inline-flex h-6 w-11 items-center rounded-full transition-colors focus:outline-
none focus:ring-2 focus:ring-[#AD7F65] focus:ring-offset-2 ${isActive ? "bg-[#AD7F65]" : "bg-gray-300"}`
    },
    /* @__PURE__ */ React.createElement("span", { className: `inline-block h-4 w-4 transform rounded-full bg-
white transition-transform ${isActive ? "translate-x-6" : "translate-x-1"}` })
  )), /* @__PURE__ */ React.createElement("div", { className: `flex items-center justify-between py-3 border-
b ${theme === "dark" ? "border-[#4A4037]" : "border-gray-100"}` }, /* @__PURE__ */ React.createElement("div",
null, /* @__PURE__ */ React.createElement("h4", { className: `font-semibold ${theme === "dark" ? "text-
white" : "text-gray-900"}` }, "Show on POS screen"), /* @__PURE__ */ React.createElement("p", { className:
`text-xs mt-1 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` }, "Display as a quick-filter button
on the cashier screen.")), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setShowOnPos(!showOnPos),
      className: `relative inline-flex h-6 w-11 items-center rounded-full transition-colors focus:outline-
none focus:ring-2 focus:ring-[#AD7F65] focus:ring-offset-2 ${showOnPos ? "bg-[#AD7F65]" : "bg-gray-300"}`
    },
    /* @__PURE__ */ React.createElement("span", { className: `inline-block h-4 w-4 transform rounded-full bg-
white transition-transform ${showOnPos ? "translate-x-6" : "translate-x-1"}` })
  ))), /* @__PURE__ */ React.createElement("div", { className: "flex gap-4 justify-center" }, /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: () => {
        setShowAddSubModal(false);
        setCategoryName("");
        setSelectedParentCategory("");
        setError("");
      },
      className: `px-8 py-3 rounded-xl font-bold transition-all transform active:scale-95 ${theme === "dark" ?
"bg-[#2A2724] text-gray-300 hover:bg-[#3A3734]" : "bg-gray-200 text-gray-700 hover:bg-gray-300"}`
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => handleAddCategory("subcategory"),
      className: "px-8 py-3 text-white rounded-xl font-bold shadow-lg hover:shadow-xl transition-all
transform active:scale-95 bg-[#09A046] hover:bg-green-600"
    },
    "Add subcategory"
  )))), showEditModal && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 backdrop-
blur-sm bg-black/30 flex items-center justify-center z-50 p-4" }, /* @__PURE__ */ React.createElement("div",
{ className: `rounded-2xl p-8 w-full max-w-lg ${theme === "dark" ? "bg-[#1E1B18] text-white" : "bg-white
text-gray-900"} relative shadow-2xl transition-all transform scale-100` }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center justify-between mb-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-4" }, /* @__PURE__ */
React.createElement("div", { className: "w-12 h-12 rounded-xl flex items-center justify-center text-white
text-xl shadow-lg", style: { background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)" } }, /*
@__PURE__ */ React.createElement(FaEdit, null)), /* @__PURE__ */ React.createElement("h2", { className:
```

```
"text-2xl font-bold" }, "Edit Category")), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => {
        setShowEditModal(false);
        setEditingCategory(null);
        setCategoryName("");
        setError("");
      },
      className: `p-2 rounded-full hover:bg-opacity-10 transition-colors ${theme === "dark" ? "hover:bg-white
text-gray-400" : "hover:bg-black text-gray-400"}`
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "text-xl" })
  )), /* @__PURE__ */ React.createElement("div", { className: "mb-8" }, /* @__PURE__ */
React.createElement("label", { className: `block text-sm font-semibold mb-2 ml-1 ${theme === "dark" ? "text-
gray-300" : "text-gray-700"}` }, "Category Name"), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      value: categoryName,
      onChange: (e) => {
        setCategoryName(e.target.value);
        setError("");
      },
      className: `w-full px-4 py-3 border rounded-xl focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
transition-all
                  ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white placeholder-gray-500" : "bg-
white border-gray-200 text-gray-900 placeholder-gray-400 shadow-sm"}`,
      placeholder: "eg. Casual Wear",
      autoFocus: true
    }
  ), error && /* @__PURE__ */ React.createElement("p", { className: "text-red-500 text-sm mt-2 ml-1 flex
items-center gap-1" }, /* @__PURE__ */ React.createElement(FaTimes, { className: "text-xs" }), " ", error)),
/* @__PURE__ */ React.createElement("div", { className: "flex gap-4 justify-center" }, /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: () => {
        setShowEditModal(false);
        setEditingCategory(null);
        setCategoryName("");
        setError("");
      },
      className: `px-8 py-3 rounded-xl font-bold transition-all transform active:scale-95 ${theme === "dark" ?
"bg-[#2A2724] text-gray-300 hover:bg-[#3A3734]" : "bg-gray-200 text-gray-700 hover:bg-gray-300"}`
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: handleEditCategory,
      className: "px-8 py-3 text-white rounded-xl font-bold shadow-lg hover:shadow-xl transition-all
transform active:scale-95 hover:-translate-y-0.5",
      style: { background: "linear-gradient(135deg, #10B981 0%, #059669 100%)" }
    },
    "Save Changes"
  )))), /* @__PURE__ */ React.createElement(
    ViewCategoryProductsModal,
    {
      isOpen: showViewProductsModal,
      onClose: () => {
        setShowViewProductsModal(false);
        setSelectedCategory(null);
      },
      categoryName: selectedCategory?.name
    }
  ), showArchiveCategoryModal && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0
backdrop-blur-sm bg-black/30 flex items-center justify-center z-50 p-4" }, /* @__PURE__ */
React.createElement("div", { className: `rounded-2xl p-8 w-full max-w-md ${theme === "dark" ? "bg-[#1E1B18]
text-white" : "bg-white text-gray-900"} relative shadow-2xl transition-all transform scale-100` }, /*
@__PURE__ */ React.createElement("div", { className: "flex items-center justify-between mb-6" }, /* @__PURE__
*/ React.createElement("div", { className: "flex items-center gap-4" }, /* @__PURE__ */
React.createElement("div", { className: "w-12 h-12 rounded-xl flex items-center justify-center text-white
text-xl shadow-lg", style: { background: "linear-gradient(135deg, #EF4444 0%, #B91C1C 100%)" } }, /*
@__PURE__ */ React.createElement(FaTrash, null)), /* @__PURE__ */ React.createElement("h2", { className:
"text-2xl font-bold" }, "Archive Category")), /* @__PURE__ */ React.createElement(
    "button",
```

```
    {
      onClick: () => {
        setShowArchiveCategoryModal(false);
        setCategoryToArchive(null);
      },
      className: `p-2 rounded-full hover:bg-opacity-10 transition-colors ${theme === "dark" ? "hover:bg-white
text-gray-400" : "hover:bg-black text-gray-400"}`
    },
    /* @__PURE__ */ React.createElement(FaTimes, { className: "text-xl" })
  )), /* @__PURE__ */ React.createElement("div", { className: "mb-8" }, /* @__PURE__ */
React.createElement("p", { className: `text-base leading-relaxed ${theme === "dark" ? "text-gray-300" :
"text-gray-600"}` }, "Are you sure you want to archive the category ", /* @__PURE__ */
React.createElement("span", { className: "font-bold" }, '"', categoryToArchive?.name, '"'), "? It will be
hidden from POS/Terminal and Inventory filters.")), /* @__PURE__ */ React.createElement("div", { className:
"flex gap-4 justify-center" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => {
        setShowArchiveCategoryModal(false);
        setCategoryToArchive(null);
      },
      className: `px-6 py-3 rounded-xl font-bold transition-all transform active:scale-95 ${theme === "dark" ?
"bg-[#2A2724] text-gray-300 hover:bg-[#3A3734]" : "bg-gray-200 text-gray-700 hover:bg-gray-300"}`
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: handleConfirmArchive,
      className: "px-6 py-3 text-white rounded-xl font-bold shadow-lg hover:shadow-xl transition-all
transform active:scale-95 hover:-translate-y-0.5",
      style: { background: "linear-gradient(135deg, #EF4444 0%, #B91C1C 100%)" }
    },
    "Yes, Archive"
  )))));
};
export default memo(Categories);
```

FILE: pos/frontend/src/pages/owner/Dashboard.jsx
==========

```
import { format } from "date-fns";
import { useEffect, useRef, useState } from "react";
import DatePicker from "react-datepicker";
import "react-datepicker/dist/react-datepicker.css";
import {
  FaCalendarAlt,
  FaChartLine,
  FaExclamationTriangle,
  FaHandshake,
  FaShoppingBag
} from "react-icons/fa";
import { useNavigate } from "react-router-dom";
import {
  Area,
  Bar,
  CartesianGrid,
  Cell,
  ComposedChart,
  Legend,
  Line,
  Pie,
  PieChart,
  ResponsiveContainer,
  Tooltip,
  XAxis,
  YAxis
} from "recharts";
import TopSellingModal from "../../components/owner/TopSellingModal";
import Header from "../../components/shared/header";
import { useAuth } from "../../context/AuthContext";
import { useTheme } from "../../context/ThemeContext";
import { API_ENDPOINTS, API_BASE_URL } from "../../config/api";
const formatCompactPeso = (n) => {
  const abs = Math.abs(Number(n) || 0);
  if (abs === 0) return "\u20B10";
```

```
  const trimTwoDec = (s) => s.endsWith(".00") ? s.slice(0, -3) : s;
  if (abs >= 1e6) {
    const v = abs / 1e6;
    return `\u20B1${trimTwoDec(v.toFixed(2))}M`;
  }
  if (abs >= 1e3) {
    const v = abs / 1e3;
    return `\u20B1${trimTwoDec(v.toFixed(2))}k`;
  }
  return `\u20B1${abs.toLocaleString("en-US", { maximumFractionDigits: 0 })}`;
};
const growthVsLastPeriod = (current, previous, rate) => {
  const cur = Number(current) || 0;
  const prev = Number(previous) || 0;
  if (cur === 0 && prev === 0) {
    return { text: "\u2014 vs last period", tone: "neutral" };
  }
  const r = Number(rate) || 0;
  const sign = r > 0 ? "+" : "";
  const tone = r > 0 ? "up" : r < 0 ? "down" : "neutral";
  return { text: `${sign}${r}% vs last period`, tone };
};
const growthToneClass = (tone, dark) => {
  if (tone === "up") return "text-green-600 dark:text-green-400";
  if (tone === "down") return "text-red-600 dark:text-red-400";
  return dark ? "text-gray-400" : "text-gray-500";
};
const Dashboard = () => {
  const { currentUser } = useAuth();
  const { theme } = useTheme();
  const navigate = useNavigate();
  const userName = currentUser?.name || "Erika";
  const [timeframe, setTimeframe] = useState("Daily");
  const [dateRange, setDateRange] = useState([null, null]);
  const [startDate, endDate] = dateRange;
  const [showDatePicker, setShowDatePicker] = useState(false);
  const [metrics, setMetrics] = useState({
    totalSalesToday: 0,
    totalTransactions: 0,
    profit: 0,
    lowStockItems: 0,
    growthRate: 0,
    totalSalesPrevious: 0,
    totalTransactionsPrevious: 0,
    transactionGrowthRate: 0
  });
  const [salesOverTimeData, setSalesOverTimeData] = useState([]);
  const [salesByCategoryData, setSalesByCategoryData] = useState([]);
  const [topSellingProducts, setTopSellingProducts] = useState([]);
  const [lowStockProducts, setLowStockProducts] = useState([]);
  const [activeEmployees, setActiveEmployees] = useState([]);
  const [loading, setLoading] = useState(true);
  const [salesFilter, setSalesFilter] = useState("Both");
  const [topSellingSort, setTopSellingSort] = useState("most");
  const [lowStockFilter, setLowStockFilter] = useState("All");
  const [showSalesFilter, setShowSalesFilter] = useState(false);
  const [showTopSellingFilter, setShowTopSellingFilter] = useState(false);
  const [showLowStockFilter, setShowLowStockFilter] = useState(false);
  const [showTopSellingModal, setShowTopSellingModal] = useState(false);
  const COLORS = [
    "#8884d8",
    "#82ca9d",
    "#ffc658",
    "#ff8042",
    "#0088FE",
    "#00C49F"
  ];
  const PIE_COLORS = ["#F4A6C1", "#A7C7E7", "#FAD02E", "#98FB98", "#FFB7B2"];
  useEffect(() => {
    if (timeframe === "Custom" && (!startDate || !endDate)) return;
    const fetchCoreData = async () => {
      setLoading(true);
      try {
        await Promise.all(
          [
            fetchDashboardStats(),
            fetchSalesOverTime(),
```

```
          fetchSalesByCategory(),
          fetchActiveEmployees()
        ]
      );
    } catch (error) {
      console.error("Error fetching core dashboard data:", error);
    } finally {
      setLoading(false);
    }
  };
  fetchCoreData();
}, [timeframe, startDate, endDate]);
useEffect(() => {
  if (timeframe === "Custom" && (!startDate || !endDate)) return;
  fetchTopSellingProducts();
}, [timeframe, topSellingSort, startDate, endDate]);
useEffect(() => {
  fetchLowStockProducts();
}, [lowStockFilter]);
const refreshAllSilentRef = useRef(null);
refreshAllSilentRef.current = async () => {
  if (timeframe === "Custom" && (!startDate || !endDate)) return;
  try {
    await Promise.all([
      fetchDashboardStats(),
      fetchSalesOverTime(),
      fetchSalesByCategory(),
      fetchTopSellingProducts(),
      fetchLowStockProducts(),
      fetchActiveEmployees()
    ]);
  } catch (e) {
    console.error("Dashboard silent refresh:", e);
  }
};
useEffect(() => {
  if (timeframe === "Custom" && (!startDate || !endDate)) return;
  const intervalMs = 2e3;
  refreshAllSilentRef.current?.();
  const id = setInterval(() => {
    refreshAllSilentRef.current?.();
  }, intervalMs);
  const onVisibility = () => {
    if (document.visibilityState === "visible") {
      refreshAllSilentRef.current?.();
    }
  };
  const onWindowFocus = () => {
    refreshAllSilentRef.current?.();
  };
  const onOnline = () => {
    refreshAllSilentRef.current?.();
  };
  document.addEventListener("visibilitychange", onVisibility);
  window.addEventListener("focus", onWindowFocus);
  window.addEventListener("online", onOnline);
  return () => {
    clearInterval(id);
    document.removeEventListener("visibilitychange", onVisibility);
    window.removeEventListener("focus", onWindowFocus);
    window.removeEventListener("online", onOnline);
  };
}, [timeframe, startDate, endDate, topSellingSort, lowStockFilter]);
const getQueryParams = () => {
  let params = `?timeframe=${timeframe}`;
  if (timeframe === "Custom" && startDate && endDate) {
    params += `&startDate=${startDate.toISOString()}&endDate=${endDate.toISOString()}`;
  }
  return params;
};
const fetchDashboardStats = async () => {
  try {
    const timeMap = {
      Day: "daily",
      Week: "weekly",
      Month: "monthly",
      Year: "yearly",
```

```
        Custom: "custom"
      };
      const apiTimeframe = timeMap[timeframe] || "daily";
      let url = `${API_ENDPOINTS.transactionsDashboardStats}?timeframe=${apiTimeframe}`;
      if (timeframe === "Custom" && startDate && endDate) {
        url += `&startDate=${startDate.toISOString()}&endDate=${endDate.toISOString()}`;
      }
      const response = await fetch(url);
      const data = await response.json();
      if (data.success) {
        setMetrics(data.data);
      }
    } catch (error) {
      console.error("Error fetching stats:", error);
    }
  };
  const fetchSalesOverTime = async () => {
    try {
      const timeMap = {
        Day: "daily",
        Week: "weekly",
        Month: "monthly",
        Year: "yearly",
        Custom: "custom"
      };
      const apiTimeframe = timeMap[timeframe] || "daily";
      let url = `${API_ENDPOINTS.transactionsSalesOverTime}?timeframe=${apiTimeframe}`;
      if (timeframe === "Custom" && startDate && endDate) {
        url += `&startDate=${startDate.toISOString()}&endDate=${endDate.toISOString()}`;
      }
      const response = await fetch(url);
      const data = await response.json();
      if (data.success) {
        const mappedData = data.data.map((item) => ({
          ...item,
          revenue: item.revenue ?? item.totalSales ?? 0
        }));
        setSalesOverTimeData(mappedData);
      }
    } catch (error) {
      console.error("Error fetching sales over time:", error);
    }
  };
  const fetchSalesByCategory = async () => {
    try {
      let url = API_ENDPOINTS.transactionsSalesByCategory;
      if (timeframe === "Custom" && startDate && endDate) {
        url += `?startDate=${startDate.toISOString()}&endDate=${endDate.toISOString()}`;
      }
      const response = await fetch(url);
      const data = await response.json();
      if (data.success) {
        setSalesByCategoryData(data.data);
      }
    } catch (error) {
      console.error("Error fetching sales by category:", error);
    }
  };
  const fetchTopSellingProducts = async () => {
    try {
      const timeMap = {
        Day: "daily",
        Week: "weekly",
        Month: "monthly",
        Year: "yearly",
        Custom: "custom"
      };
      const apiTimeframe = timeMap[timeframe] || "daily";
      let url =
`${API_ENDPOINTS.transactionsTopSelling}?sort=${topSellingSort}&limit=10&period=${apiTimeframe}`;
      if (timeframe === "Custom" && startDate && endDate) {
        url += `&startDate=${startDate.toISOString()}&endDate=${endDate.toISOString()}`;
      }
      const response = await fetch(url);
      const data = await response.json();
      if (data.success) {
        setTopSellingProducts(data.data);
```

```
      }
    } catch (error) {
      console.error("Error fetching top selling:", error);
    }
  };
  const fetchLowStockProducts = async () => {
    try {
      const response = await fetch(API_ENDPOINTS.productsLowStock);
      const data = await response.json();
      if (data.success && Array.isArray(data.data)) {
        const mappedProducts = data.data.map((p) => ({
          ...p,
          calculatedStock: p.currentStock || 0
        }));
        const lowStock = mappedProducts.filter((p) => {
          const isOutOfStock = p.alertType === "out_of_stock" || p.calculatedStock === 0;
          const isLowStock = p.alertType === "low_stock" && p.calculatedStock > 0;
          if (lowStockFilter === "Out of Stock") return isOutOfStock;
          if (lowStockFilter === "Low Stock") return isLowStock;
          return true;
        }).slice(0, 5);
        setLowStockProducts(lowStock);
      }
    } catch (error) {
      console.error("Error fetching low stock:", error);
    }
  };
  const fetchActiveEmployees = async () => {
    try {
      const response = await fetch(`${API_ENDPOINTS.employeesOnline}`);
      const data = await response.json();
      if (data.success) {
        const timestamp = (/* @__PURE__ */ new Date()).getTime();
        const onlineEmployees = data.data.map((emp) => ({
          ...emp,
          image: `${API_BASE_URL}/api/employees/${emp._id}/image?t=${timestamp}`
        }));
        setActiveEmployees(onlineEmployees);
      }
    } catch (error) {
      console.error("Error fetching online employees:", error);
    }
  };
  const formatCurrency = (val) => `\u20B1${val.toLocaleString()}`;
  const salesGrowth = growthVsLastPeriod(
    metrics.totalSalesToday,
    metrics.totalSalesPrevious,
    metrics.growthRate
  );
  const txGrowth = growthVsLastPeriod(
    metrics.totalTransactions,
    metrics.totalTransactionsPrevious,
    metrics.transactionGrowthRate
  );
  const grossProfitMargin = (metrics.totalSalesToday ?? 0) > 0 ? (metrics.profit ?? 0) /
(metrics.totalSalesToday ?? 0) * 100 : 0;
  const grossProfitMarginTone = grossProfitMargin >= 0 ? "up" : "down";
  const isDark = theme === "dark";
  const renderDateRange = () => {
    if (timeframe === "Custom" && startDate && endDate) {
      return `${format(startDate, "MMM d, yyyy")} - ${format(endDate, "MMM d, yyyy")}`;
    }
    const now = /* @__PURE__ */ new Date();
    const options = { day: "numeric", month: "short", year: "numeric" };
    if (timeframe === "Daily" || timeframe === "Day") {
      return now.toLocaleDateString("en-US", options);
    } else if (timeframe === "Weekly" || timeframe === "Week") {
      const start = new Date(now);
      start.setDate(now.getDate() - 7);
      return `${start.toLocaleDateString("en-US", { day: "numeric", month: "short" })} -
${now.toLocaleDateString("en-US", options)}`;
    }
    return `${now.toLocaleDateString("en-US", { month: "short", year: "numeric" })}`;
  };
  return /* @__PURE__ */ React.createElement(
    "div",
    {
```

```
      className: `min-h-screen pb-10 ${theme === "dark" ? "bg-[#1E1B18]" : "bg-gray-50"}`
    },
    /* @__PURE__ */ React.createElement(
      Header,
      {
        pageName: `Welcome, ${userName}!`,
        subtitle: `Here's your overview for ${timeframe === "Custom" ? "custom range" :
timeframe.toLowerCase()}`,
        showTimeframeFilter: false,
        showBorder: false,
        userName,
        userRole: "Owner",
        className: "px-6 pt-8 pb-4 [&_h1]:text-4xl"
      }
    ),
    /* @__PURE__ */ React.createElement("div", { className: "px-6 pb-4 w-full space-y-6" }, /* @__PURE__ */
React.createElement("div", { className: "flex flex-col md:flex-row justify-between items-start md:items-
center gap-4 mt-2" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center" }, /*
@__PURE__ */ React.createElement("style", null, `
              @keyframes downloadPulse {
                0%, 100% { box-shadow: 0 0 0 0 rgba(173, 127, 101, 0.4); }
                50% { box-shadow: 0 0 0 10px rgba(173, 127, 101, 0); }
              }
              @keyframes downloadShimmer {
                0% { background-position: -200% center; }
                100% { background-position: 200% center; }
              }
              .download-app-btn {
                animation: downloadPulse 2s ease-in-out infinite;
                background-image: linear-gradient(90deg, #AD7F65 0%, #c9a48e 50%, #AD7F65 100%);
                background-size: 200% auto;
              }
              .download-app-btn:hover {
                animation: downloadShimmer 1.5s linear infinite;
                transform: scale(1.05);
              }
            `), /* @__PURE__ */ React.createElement(
      "a",
      {
        href: "https://expo.dev/accounts/consolve_studio/projects/cysmob/builds/d5631649-4cbe-4be1-9fb1-
c582d760147c",
        target: "_blank",
        rel: "noopener noreferrer",
        className: "download-app-btn px-5 py-2.5 text-sm font-bold rounded-lg shadow-md transition-all flex
items-center gap-2 text-white"
      },
      "\u{1F4F1} Click here to download the app"
    )), /* @__PURE__ */ React.createElement("div", { className: "flex flex-col sm:flex-row gap-4 items-center
w-full md:w-auto" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-lg shadow-sm p-1 flex space-x-1 ${theme === "dark" ? "bg-[#2A2724]" : "bg-
white"}`
      },
      ["Day", "Week", "Month", "Year"].map(
        (t) => /* @__PURE__ */ React.createElement(
          "button",
          {
            key: t,
            onClick: () => {
              setTimeframe(t);
              setDateRange([null, null]);
            },
            className: `px-4 py-1.5 text-sm font-medium rounded-md transition-colors cursor-pointer
${timeframe === t ? "bg-[#AD7F65] text-white shadow-sm" : theme === "dark" ? "text-gray-400 hover:bg-
[#3A3734]" : "text-gray-500 hover:bg-gray-100"}`
          },
          t
        )
      ),
      /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => {
            if (timeframe !== "Custom") {
              setShowDatePicker(true);
            }
```

```
          },
          className: `px-4 py-1.5 text-sm font-medium rounded-md transition-colors cursor-pointer ${timeframe
=== "Custom" ? "bg-[#AD7F65] text-white shadow-sm" : theme === "dark" ? "text-gray-400 hover:bg-[#3A3734]
hidden" : "text-gray-500 hover:bg-gray-100 hidden"}`
        },
        "Custom"
      )
    ), /* @__PURE__ */ React.createElement("div", { className: "relative z-50" }, /* @__PURE__ */
React.createElement(
      DatePicker,
      {
        selected: startDate,
        onChange: (update) => {
          setDateRange(update);
          if (update[0]) {
            setTimeframe("Custom");
          }
        },
        startDate,
        endDate,
        selectsRange: true,
        popperClassName: "z-50",
        customInput: /* @__PURE__ */ React.createElement(
          "button",
          {
            className: `flex items-center gap-2 px-3 py-1.5 rounded-lg shadow-sm border transition-colors
cursor-pointer ${theme === "dark" ? "bg-[#2A2724] border-gray-600 hover:bg-[#3A3734]" : "bg-white border-
gray-100 hover:bg-gray-50"}`
          },
          /* @__PURE__ */ React.createElement(
            FaCalendarAlt,
            {
              className: `${timeframe === "Custom" ? "text-[#AD7F65]" : theme === "dark" ? "text-gray-400" :
"text-gray-400"}`
            }
          ),
          /* @__PURE__ */ React.createElement(
            "span",
            {
              className: `text-sm font-medium ${timeframe === "Custom" ? "text-[#AD7F65]" : theme === "dark" ?
"text-gray-300" : "text-gray-600"}`
            },
            renderDateRange()
          )
        )
      }
    )))), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-1 lg:grid-cols-3 gap-6
items-stretch" }, /* @__PURE__ */ React.createElement("div", { className: "lg:col-span-2 flex flex-col gap-6
h-full" }, /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-1 sm:grid-cols-2 lg:grid-
cols-4 gap-4 sm:auto-rows-fr" }, /* @__PURE__ */ React.createElement("div", { className: `rounded-xl border
shadow-sm relative overflow-hidden hover:shadow-md transition-shadow min-h-[192px] ${isDark ? "bg-[#2A2724]
border-[#4A4037]" : "bg-white border-gray-100"}` }, /* @__PURE__ */ React.createElement("div", { className:
"absolute top-0 left-0 w-3 h-full rounded-l-xl", style: { backgroundImage: "linear-gradient(180deg, #93C5FD
0%, #2563EB 100%)" } }), /* @__PURE__ */ React.createElement("div", { className: "relative flex flex-col
items-center text-center pt-6 pb-5 px-4 pl-6 min-h-[192px]" }, /* @__PURE__ */ React.createElement("div",
{ className: `p-2.5 rounded-full mb-3 ${isDark ? "bg-blue-500/15" : "bg-blue-50"}` }, /* @__PURE__ */
React.createElement(FaShoppingBag, { className: "text-blue-500 text-base" })), /* @__PURE__ */
React.createElement("p", { className: `text-3xl sm:text-5xl font-black tabular-nums leading-none tracking-
tight ${isDark ? "text-white" : "text-gray-900"}` }, formatCompactPeso(metrics.totalSalesToday)), /*
@__PURE__ */ React.createElement("p", { className: `text-sm font-extrabold mt-2.5 ${isDark ? "text-blue-400" :
"text-blue-600"}` }, "Total Sales"), /* @__PURE__ */ React.createElement("p", { className: `text-[10px] font-
semibold mt-auto ${growthToneClass(salesGrowth.tone, isDark)}` }, salesGrowth.text))), /* @__PURE__ */
React.createElement("div", { className: `rounded-xl border shadow-sm relative overflow-hidden hover:shadow-md
transition-shadow min-h-[192px] ${isDark ? "bg-[#2A2724] border-[#4A4037]" : "bg-white border-gray-100"}` },
/* @__PURE__ */ React.createElement("div", { className: "absolute top-0 left-0 w-3 h-full rounded-l-xl",
style: { backgroundImage: "linear-gradient(180deg, #C4B5FD 0%, #7C3AED 100%)" } }), /* @__PURE__ */
React.createElement("div", { className: "relative flex flex-col items-center text-center pt-6 pb-5 px-4 pl-6
min-h-[192px]" }, /* @__PURE__ */ React.createElement("div", { className: `p-2.5 rounded-full mb-3 ${isDark ?
"bg-purple-500/15" : "bg-purple-50"}` }, /* @__PURE__ */ React.createElement(FaHandshake, { className: "text-
purple-500 text-base" })), /* @__PURE__ */ React.createElement("p", { className: `text-3xl sm:text-5xl font-
black tabular-nums leading-none tracking-tight ${isDark ? "text-white" : "text-gray-900"}` },
(metrics.totalTransactions ?? 0).toLocaleString("en-US")), /* @__PURE__ */ React.createElement("p",
{ className: `text-sm font-extrabold mt-2.5 ${isDark ? "text-purple-400" : "text-purple-700"}` }, "Total
Transactions"), /* @__PURE__ */ React.createElement("p", { className: `text-[10px] font-semibold mt-auto
${growthToneClass(txGrowth.tone, isDark)}` }, txGrowth.text))), /* @__PURE__ */ React.createElement("div",
{ className: `rounded-xl border shadow-sm relative overflow-hidden hover:shadow-md transition-shadow min-h-
[192px] ${isDark ? "bg-[#2A2724] border-[#4A4037]" : "bg-white border-gray-100"}` }, /* @__PURE__ */
```

```
React.createElement("div", { className: "absolute top-0 left-0 w-3 h-full rounded-l-xl", style:
{ backgroundImage: "linear-gradient(180deg, #86EFAC 0%, #16A34A 100%)" } }), /* @__PURE__ */
React.createElement("div", { className: "relative flex flex-col items-center text-center pt-6 pb-5 px-4 pl-6
min-h-[192px]" }, /* @__PURE__ */ React.createElement("div", { className: `p-2.5 rounded-full mb-3 ${isDark ?
"bg-green-500/15" : "bg-green-50"}` }, /* @__PURE__ */ React.createElement(FaChartLine, { className: "text-
green-500 text-base" })), /* @__PURE__ */ React.createElement("p", { className: `flex items-baseline justify-
center gap-0.5 font-black leading-none ${isDark ? "text-white" : "text-gray-900"}` }, /* @__PURE__ */
React.createElement("span", { className: "text-3xl sm:text-5xl tabular-nums tracking-tight" },
grossProfitMargin.toFixed(1)), /* @__PURE__ */ React.createElement("span", { className: `text-2xl font-bold
${isDark ? "text-gray-400" : "text-gray-600"}` }, "%")), /* @__PURE__ */ React.createElement("p", { className:
`text-sm font-extrabold mt-2.5 ${isDark ? "text-green-400" : "text-green-700"}` }, "Gross Profit Margin"), /*
@__PURE__ */ React.createElement("p", { className: `text-[10px] font-semibold mt-auto
${growthToneClass(grossProfitMarginTone, isDark)}` }, "Revenue - COGS / Revenue"))), /* @__PURE__ */
React.createElement("div", { className: `rounded-xl border shadow-sm relative overflow-hidden hover:shadow-md
transition-shadow min-h-[192px] ${isDark ? "bg-[#2A2724] border-[#4A4037]" : "bg-white border-gray-100"}` },
/* @__PURE__ */ React.createElement("div", { className: "absolute top-0 left-0 w-3 h-full rounded-l-xl",
style: { backgroundImage: "linear-gradient(180deg, #FCA5A5 0%, #DC2626 100%)" } }), /* @__PURE__ */
React.createElement("div", { className: "relative flex flex-col items-center text-center pt-6 pb-5 px-4 pl-6
min-h-[192px]" }, /* @__PURE__ */ React.createElement("div", { className: `p-2.5 rounded-full mb-3 ${isDark ?
"bg-red-500/15" : "bg-red-50"}` }, /* @__PURE__ */ React.createElement(FaExclamationTriangle, { className:
"text-red-500 text-base" })), /* @__PURE__ */ React.createElement("p", { className: `text-3xl sm:text-5xl
font-black tabular-nums leading-none tracking-tight ${isDark ? "text-white" : "text-gray-900"}` },
(metrics.lowStockItems ?? 0).toLocaleString("en-US")), /* @__PURE__ */ React.createElement("p", { className:
`text-sm font-extrabold mt-2.5 ${isDark ? "text-red-400" : "text-red-600"}` }, "Low Stock Items"), /*
@__PURE__ */ React.createElement("p", { className: `text-[10px] font-semibold mt-auto ${isDark ? "text-gray-
400" : "text-gray-500"}` }, "Below reorder level")))), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `p-6 rounded-xl shadow-sm border flex-1 flex flex-col ${theme === "dark" ? "bg-[#2A2724]
border-[#4A4037]" : "bg-white border-gray-100"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex justify-between items-center mb-6" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "h2",
        {
          className: `text-lg font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Sales Over Time and Growth"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Total sales and growth from all sales during a specific period."
      )), /* @__PURE__ */ React.createElement("div", { className: "flex gap-2" }, /* @__PURE__ */
React.createElement("div", { className: "relative" }, /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => setShowSalesFilter(!showSalesFilter),
          className: "text-xs bg-white border border-gray-200 px-3 py-1 rounded-full text-gray-600 flex
items-center gap-1 cursor-pointer"
        },
        "Filter by: ",
        salesFilter,
        " ",
        /* @__PURE__ */ React.createElement("span", { className: "text-[10px]" }, "\u25BC")
      ), showSalesFilter && /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `absolute right-0 mt-1 w-24 border rounded-lg shadow-lg z-10 py-1 ${theme === "dark" ?
"bg-[#1E1B18] border-gray-700" : "bg-white border-gray-100"}`
        },
        ["Sales", "Growth", "Both"].map(
          (option) => /* @__PURE__ */ React.createElement(
            "button",
            {
              key: option,
              onClick: () => {
                setSalesFilter(option);
                setShowSalesFilter(false);
              },
              className: `w-full text-left px-3 py-1.5 text-xs hover:bg-opacity-10 cursor-pointer ${theme ===
"dark" ? "text-gray-300 hover:bg-gray-500" : "text-gray-700 hover:bg-gray-50"}`
            },
            option
          )
        )
```

```
      )))),
      /* @__PURE__ */ React.createElement("div", { className: "h-[300px] min-h-[300px] w-full" },
salesOverTimeData && salesOverTimeData.length > 0 ? /* @__PURE__ */ React.createElement(ResponsiveContainer,
{ width: "100%", height: "100%" }, /* @__PURE__ */ React.createElement(ComposedChart, { data:
salesOverTimeData }, /* @__PURE__ */ React.createElement(
        CartesianGrid,
        {
          strokeDasharray: "3 3",
          vertical: false,
          stroke: "#f0f0f0"
        }
      ), /* @__PURE__ */ React.createElement(
        XAxis,
        {
          dataKey: "period",
          axisLine: false,
          tickLine: false,
          tick: { fontSize: 11, fill: "#6b7280" },
          dy: 10
        }
      ), /* @__PURE__ */ React.createElement(
        YAxis,
        {
          yAxisId: "left",
          orientation: "left",
          axisLine: false,
          tickLine: false,
          tick: { fontSize: 11, fill: "#6b7280" },
          domain: [0, "auto"],
          tickFormatter: (val) => `\u20B1${val >= 1e3 ? (val / 1e3).toFixed(0) + "k" : val}`
        }
      ), /* @__PURE__ */ React.createElement(
        YAxis,
        {
          yAxisId: "right",
          orientation: "right",
          axisLine: false,
          tickLine: false,
          tick: { fontSize: 11, fill: "#6b7280" },
          tickFormatter: (val) => `${val}%`
        }
      ), /* @__PURE__ */ React.createElement(
        Tooltip,
        {
          contentStyle: {
            borderRadius: "12px",
            border: "none",
            boxShadow: "0 10px 15px -3px rgb(0 0 0 / 0.1)"
          },
          formatter: (value, name) => {
            if (name === "Revenue")
              return [`\u20B1${value.toLocaleString()}`, name];
            if (name === "Growth") return [`${value}%`, name];
            return [value, name];
          }
        }
      ), /* @__PURE__ */ React.createElement(
        Legend,
        {
          iconType: "circle",
          wrapperStyle: { paddingTop: "20px" }
        }
      ), salesFilter === "Sales" && /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(
        Bar,
        {
          yAxisId: "left",
          dataKey: "revenue",
          barSize: 30,
          fill: "#8884d8",
          radius: [4, 4, 0, 0],
          name: "Revenue"
        }
      ), /* @__PURE__ */ React.createElement(
        Line,
        {
          yAxisId: "left",
```

```
          type: "monotone",
          dataKey: "target",
          stroke: "#FB923C",
          strokeDasharray: "5 5",
          name: "Target",
          dot: false,
          strokeWidth: 2
        }
      )), salesFilter === "Growth" && /* @__PURE__ */ React.createElement(
        Area,
        {
          yAxisId: "right",
          type: "monotone",
          dataKey: "growth",
          fill: "#D7D7D7",
          stroke: "#D7D7D7",
          name: "Growth",
          strokeWidth: 3,
          fillOpacity: 0.6
        }
      ), salesFilter === "Both" && /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(
        Bar,
        {
          yAxisId: "left",
          dataKey: "revenue",
          barSize: 30,
          fill: "#8884d8",
          radius: [4, 4, 0, 0],
          name: "Revenue"
        }
      ), /* @__PURE__ */ React.createElement(
        Line,
        {
          yAxisId: "left",
          type: "monotone",
          dataKey: "target",
          stroke: "#FB923C",
          strokeDasharray: "5 5",
          name: "Target",
          dot: false,
          strokeWidth: 2
        }
      ), /* @__PURE__ */ React.createElement(
        Area,
        {
          yAxisId: "right",
          type: "monotone",
          dataKey: "growth",
          fill: "#D7D7D7",
          stroke: "#D7D7D7",
          name: "Growth",
          strokeWidth: 3,
          fillOpacity: 0.6
        }
      )))) : /* @__PURE__ */ React.createElement("div", { className: "h-full w-full flex flex-col items-
center justify-center text-gray-400" }, /* @__PURE__ */ React.createElement(FaChartLine, { className: "text-
4xl mb-3 opacity-20" }), /* @__PURE__ */ React.createElement("p", { className: "text-sm" }, "No sales data
available for this period")))
    )), /* @__PURE__ */ React.createElement("div", { className: "flex flex-col gap-6" }, /* @__PURE__ */
React.createElement(
      "div",
      {
        className: `p-6 rounded-xl shadow-sm border ${theme === "dark" ? "bg-[#2A2724] border-[#4A4037]" :
"bg-white border-gray-100"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "mb-4" }, /* @__PURE__ */ React.createElement(
        "h2",
        {
          className: `text-lg font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Active Employees"
      )),
      /* @__PURE__ */ React.createElement("div", { className: "space-y-4" }, activeEmployees.length > 0 ?
activeEmployees.map((employee, index) => /* @__PURE__ */ React.createElement("div", { key: employee._id,
className: "flex items-center justify-between border-b border-gray-100 pb-3 last:border-0 last:pb-0" }, /*
@__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
```

```
React.createElement("div", { className: "relative" }, /* @__PURE__ */ React.createElement("div", { className:
"w-10 h-10 rounded-full overflow-hidden bg-purple-600 text-white flex items-center justify-center font-bold
text-sm" }, employee.image ? /* @__PURE__ */ React.createElement("img", { src: employee.image, alt:
employee.name, className: "w-full h-full object-cover", onError: (e) => {
        e.target.style.display = "none";
      } }) : employee.name.substring(0, 2).toUpperCase()), /* @__PURE__ */ React.createElement("div",
{ className: "absolute bottom-0 right-0 w-3 h-3 bg-green-500 rounded-full border-2 border-white" })), /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: `text-sm
font-bold ${theme === "dark" ? "text-gray-200" : "text-gray-800"}` }, employee.name), /* @__PURE__ */
React.createElement("p", { className: "text-[10px] text-gray-500" }, employee.role))), /* @__PURE__ */
React.createElement("div", { className: "text-right" }, /* @__PURE__ */ React.createElement("p", { className:
"text-[10px] text-gray-400 whitespace-nowrap" }, employee.lastLogin ? `Online since ${new
Date(employee.lastLogin).toLocaleTimeString("en-US", { hour: "numeric", minute: "2-digit", hour12: true })}` :
"Online")))) : /* @__PURE__ */ React.createElement("div", { className: "text-center text-sm text-gray-400 py-
4" }, "No active employees"), /* @__PURE__ */ React.createElement("div", { className: "pt-2 text-right mt-
auto" }, /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => navigate("/owner/manage-employees"),
          className: "text-xs text-gray-500 hover:text-[#AD7F65] flex items-center justify-end gap-1 ml-auto
cursor-pointer"
        },
        "More ",
        ">"
      )))
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `p-6 rounded-xl shadow-sm border flex-grow ${theme === "dark" ? "bg-[#2A2724] border-
[#4A4037]" : "bg-white border-gray-100"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "mb-4" }, /* @__PURE__ */ React.createElement(
        "h2",
        {
          className: `text-lg font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Sales By Category"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Revenue distribution across product categories"
      )),
      /* @__PURE__ */ React.createElement("div", { className: "h-[300px] w-full flex flex-col items-center
justify-center" }, salesByCategoryData && salesByCategoryData.length > 0 ? /* @__PURE__ */
React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(ResponsiveContainer, { width:
"100%", height: 220 }, /* @__PURE__ */ React.createElement(PieChart, null, /* @__PURE__ */
React.createElement(
        Pie,
        {
          data: salesByCategoryData,
          cx: "50%",
          cy: "50%",
          innerRadius: 60,
          outerRadius: 80,
          paddingAngle: 5,
          dataKey: "value"
        },
        salesByCategoryData.map(
          (entry, index) => /* @__PURE__ */ React.createElement(
            Cell,
            {
              key: `cell-${index}`,
              fill: PIE_COLORS[index % PIE_COLORS.length]
            }
          )
        )
      ), /* @__PURE__ */ React.createElement(Tooltip, null))), /* @__PURE__ */ React.createElement("div",
{ className: "grid grid-cols-2 gap-x-8 gap-y-2 mt-2 w-full px-4" }, salesByCategoryData.slice(0, 4).map(
        (item, index) => /* @__PURE__ */ React.createElement("div", { key: index, className: "flex items-
center gap-2" }, /* @__PURE__ */ React.createElement(
          "div",
          {
            className: "w-3 h-3 rounded-full",
            style: {
              backgroundColor: PIE_COLORS[index % PIE_COLORS.length]
```

```
            }
          }
        ), /* @__PURE__ */ React.createElement("div", { className: "flex flex-col" }, /* @__PURE__ */
React.createElement(
          "span",
          {
            className: `text-xs font-bold ${theme === "dark" ? "text-gray-300" : "text-gray-700"}`
          },
          item.name
        ), /* @__PURE__ */ React.createElement(
          "span",
          {
            className: `text-[10px] ${theme === "dark" ? "text-gray-500" : "text-gray-500"}`
          },
          "\u20B1",
          item.value.toLocaleString()
        )))
      ))) : /* @__PURE__ */ React.createElement("div", { className: "h-full w-full flex flex-col items-center
justify-center text-gray-400" }, /* @__PURE__ */ React.createElement(FaShoppingBag, { className: "text-4xl
mb-3 opacity-20" }), /* @__PURE__ */ React.createElement("p", { className: "text-sm" }, "No category data
available")))
    ))), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-1 lg:grid-cols-[1fr_2fr]
gap-6" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `p-6 rounded-xl shadow-sm border ${theme === "dark" ? "bg-[#2A2724] border-[#4A4037]" :
"bg-white border-gray-100"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex justify-between items-center mb-6" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "h2",
        {
          className: `text-lg font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Top Selling Products"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Top Performing items this ",
        timeframe.toLowerCase()
      )), /* @__PURE__ */ React.createElement("div", { className: "relative" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: () => setShowTopSellingFilter(!showTopSellingFilter),
          className: "text-xs text-gray-500 border rounded-md px-2 py-1 flex items-center gap-1 cursor-
pointer"
        },
        topSellingSort === "most" ? "Best Selling" : "Least Selling",
        " ",
        /* @__PURE__ */ React.createElement("span", { className: "text-[10px]" }, "\u25BC")
      ), showTopSellingFilter && /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `absolute right-0 mt-1 w-32 border rounded-lg shadow-lg z-10 py-1 ${theme === "dark" ?
"bg-[#1E1B18] border-gray-700" : "bg-white border-gray-100"}`
        },
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: () => {
              setTopSellingSort("most");
              setShowTopSellingFilter(false);
            },
            className: `w-full text-left px-3 py-1.5 text-xs hover:bg-opacity-10 cursor-pointer ${theme ===
"dark" ? "text-gray-300 hover:bg-gray-500" : "text-gray-700 hover:bg-gray-50"}`
          },
          "Best Selling"
        ),
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: () => {
              setTopSellingSort("least");
              setShowTopSellingFilter(false);
```

```
            },
            className: `w-full text-left px-3 py-1.5 text-xs hover:bg-opacity-10 cursor-pointer ${theme ===
"dark" ? "text-gray-300 hover:bg-gray-500" : "text-gray-700 hover:bg-gray-50"}`
          },
          "Least Selling"
        )
      )),
      /* @__PURE__ */ React.createElement("div", { className: "space-y-4" }, topSellingProducts.slice(0,
3).map(
        (product, index) => /* @__PURE__ */ React.createElement(
          "div",
          {
            key: product.productId,
            className: `flex items-center justify-between group cursor-pointer p-2 rounded-lg transition-
colors ${theme === "dark" ? "hover:bg-[#3A3734]" : "hover:bg-gray-50"}`
          },
          /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-4" }, /* @__PURE__
*/ React.createElement("div", { className: "w-8 h-8 rounded-full bg-[#AD7F65] text-white flex items-center
justify-center font-bold text-sm shadow-sm" }, index + 1), /* @__PURE__ */ React.createElement(
            "div",
            {
              className: `w-12 h-12 rounded-lg overflow-hidden border ${theme === "dark" ? "bg-gray-800
border-gray-700" : "bg-gray-100 border-gray-200"}`
            },
            product.itemImage ? /* @__PURE__ */ React.createElement(
              "img",
              {
                src: product.itemImage,
                alt: product.itemName,
                className: "w-full h-full object-cover"
              }
            ) : /* @__PURE__ */ React.createElement("div", { className: "w-full h-full flex items-center
justify-center text-[10px] text-gray-400" }, "img")
          ), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
            "p",
            {
              className: `text-sm font-bold ${theme === "dark" ? "text-gray-200" : "text-gray-800"}`
            },
            product.itemName
          ), /* @__PURE__ */ React.createElement("p", { className: "text-xs text-[#AD7F65]" },
product.totalQuantitySold, " Sold"))),
          /* @__PURE__ */ React.createElement("div", { className: "text-right" }, /* @__PURE__ */
React.createElement("p", { className: "text-sm font-bold text-green-600" }, "PHP ",
product.totalRevenue.toLocaleString()))
        )
      ), /* @__PURE__ */ React.createElement("div", { className: "pt-2 text-right" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: () => setShowTopSellingModal(true),
          className: "text-xs text-gray-500 hover:text-[#AD7F65] flex items-center justify-end gap-1 ml-auto
cursor-pointer"
        },
        "More ",
        ">"
      )))
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `p-6 rounded-xl shadow-sm border ${theme === "dark" ? "bg-[#2A2724] border-[#4A4037]" :
"bg-white border-gray-100"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex justify-between items-center mb-6" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "h2",
        {
          className: `text-lg font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Low & Out-of-Stock Items"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Items requiring immediate attention"
      )), /* @__PURE__ */ React.createElement("div", { className: "relative" }, /* @__PURE__ */
React.createElement(
```

```
        "button",
        {
          onClick: () => setShowLowStockFilter(!showLowStockFilter),
          className: "text-xs text-gray-500 border rounded-md px-2 py-1 flex items-center gap-1 cursor-
pointer"
        },
        lowStockFilter === "All" ? "Filter by" : lowStockFilter,
        " ",
        /* @__PURE__ */ React.createElement("span", { className: "text-[10px]" }, "\u25BC")
      ), showLowStockFilter && /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `absolute right-0 mt-1 w-32 border rounded-lg shadow-lg z-10 py-1 ${theme === "dark" ?
"bg-[#1E1B18] border-gray-700" : "bg-white border-gray-100"}`
        },
        ["All", "Out of Stock", "Low Stock"].map(
          (opt) => /* @__PURE__ */ React.createElement(
            "button",
            {
              key: opt,
              onClick: () => {
                setLowStockFilter(opt);
                setShowLowStockFilter(false);
                fetchLowStockProducts();
              },
              className: `w-full text-left px-3 py-1.5 text-xs hover:bg-opacity-10 cursor-pointer ${theme ===
"dark" ? "text-gray-300 hover:bg-gray-500" : "text-gray-700 hover:bg-gray-50"}`
            },
            opt
          )
        )
      )),
      /* @__PURE__ */ React.createElement("div", { className: "overflow-x-auto" }, /* @__PURE__ */
React.createElement("table", { className: "w-full text-sm text-left text-gray-500" }, /* @__PURE__ */
React.createElement(
        "thead",
        {
          className: `text-xs uppercase ${theme === "dark" ? "bg-[#1E1B18] text-gray-400" : "bg-gray-50 text-
gray-700"}`
        },
        /* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */ React.createElement("th", { className:
"px-4 py-3 rounded-l-lg" }, "Item Name"), /* @__PURE__ */ React.createElement("th", { className: "px-4 py-
3" }, "Category"), /* @__PURE__ */ React.createElement("th", { className: "px-4 py-3 text-center" }, "Stock"),
/* @__PURE__ */ React.createElement("th", { className: "px-4 py-3 text-center rounded-r-lg" }, "Status"))
      ), /* @__PURE__ */ React.createElement("tbody", null, lowStockProducts.map(
        (product) => /* @__PURE__ */ React.createElement(
          "tr",
          {
            key: product._id,
            className: `transition-colors ${theme === "dark" ? "bg-[#2A2724] hover:bg-[#3A3734]" : "bg-white
hover:bg-gray-50"}`
          },
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `px-4 py-3 font-medium flex items-center gap-3 ${theme === "dark" ? "text-gray-200" :
"text-gray-900"}`
            },
            /* @__PURE__ */ React.createElement(
              "div",
              {
                className: `w-10 h-10 rounded-md overflow-hidden border items-center justify-center flex
${theme === "dark" ? "bg-gray-800 border-gray-700" : "bg-gray-100 border-gray-200"}`
              },
              product.itemImage ? /* @__PURE__ */ React.createElement(
                "img",
                {
                  src: product.itemImage,
                  alt: "item",
                  className: "w-full h-full object-cover"
                }
              ) : /* @__PURE__ */ React.createElement("span", { className: "text-[10px] text-gray-400" },
"IMG")
            ),
            /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("div",
{ className: "text-sm font-bold" }, product.itemName), /* @__PURE__ */ React.createElement("div", { className:
"text-[10px] text-gray-400" }, product.sku || "No SKU"))
```

```
          ),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3" }, product.category),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 text-center font-bold text-
orange-600" }, product.calculatedStock),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 text-center" }, /* @__PURE__ */
React.createElement(
            "span",
            {
              className: `px-2 py-1 rounded-full text-[10px] font-bold ${product.calculatedStock === 0 ? "bg-
red-100 text-red-600" : "bg-orange-100 text-orange-600"}`
            },
            product.calculatedStock === 0 ? "Out of Stock" : "Low Stock"
          ))
        )
      ), lowStockProducts.length === 0 && /* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */
React.createElement(
        "td",
        {
          colSpan: "4",
          className: "px-4 py-8 text-center text-gray-400 italic"
        },
        "No low stock items"
      )))), /* @__PURE__ */ React.createElement("div", { className: "pt-4 text-right" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: () => navigate("/inventory"),
          className: "text-xs text-gray-500 hover:text-[#AD7F65] flex items-center justify-end gap-1 ml-auto
cursor-pointer"
        },
        "More ",
        ">"
      )))
    ))),
    /* @__PURE__ */ React.createElement(
      TopSellingModal,
      {
        isOpen: showTopSellingModal,
        onClose: () => setShowTopSellingModal(false),
        products: topSellingProducts,
        timeframe
      }
    )
  );
};
export default Dashboard;
```

FILE: pos/frontend/src/pages/owner/DiscountManagement.jsx
==========

```
import { useEffect, useMemo, useState } from "react";
import { FaBox, FaCalendar, FaEdit, FaPlus, FaSearch, FaTag, FaTrash, FaUsers } from "react-icons/fa";
import { useNavigate } from "react-router-dom";
import AddDiscountModal from "../../components/owner/AddDiscountModal";
import Header from "../../components/shared/header";
import { useAuth } from "../../context/AuthContext";
import { useTheme } from "../../context/ThemeContext";
import { API_ENDPOINTS } from "../../config/api";
const icon20Percent = new URL("../../assets/owner/20.png", import.meta.url).href;
const icon50Percent = new URL("../../assets/owner/50.png", import.meta.url).href;
const iconSenior = new URL("../../assets/owner/Senior&ani.png", import.meta.url).href;
const DiscountManagement = () => {
  const { currentUser, isOwner } = useAuth();
  const { theme } = useTheme();
  const navigate = useNavigate();
  const [searchQuery, setSearchQuery] = useState("");
  const [showAddModal, setShowAddModal] = useState(false);
  const [editingDiscount, setEditingDiscount] = useState(null);
  const [discounts, setDiscounts] = useState([]);
  const [loading, setLoading] = useState(true);
  const [filterType, setFilterType] = useState("all");
  const [filterCategory, setFilterCategory] = useState("all");
  const [showDeleteModal, setShowDeleteModal] = useState(false);
  const [discountToDelete, setDiscountToDelete] = useState(null);
  const getDiscountIcon = (discount) => {
    const title = discount.title?.toLowerCase() || "";
```

```
  const discountValue = discount.discountValue || "";
  if (title.includes("senior")) {
    return {
      icon: iconSenior,
      iconColor: "linear-gradient(135deg, #9B59B6 0%, #E91E63 100%)",
      titleColor: "#AD7F65"
    };
  }
  const match = discountValue.match(/(\d+)/);
  if (match) {
    const value = parseInt(match[1]);
    if (value >= 50) {
      return {
        icon: icon50Percent,
        iconColor: "linear-gradient(135deg, #6B7280 0%, #4B5563 100%)",
        titleColor: "#6B7280"
      };
    }
  }
  return {
    icon: icon20Percent,
    iconColor: "linear-gradient(135deg, #FF6B6B 0%, #FF8E53 100%)",
    titleColor: "#FF8E53"
  };
};
useEffect(() => {
  fetchDiscounts();
}, []);
const fetchDiscounts = async () => {
  try {
    setLoading(true);
    const response = await fetch(API_ENDPOINTS.discounts);
    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }
    const data = await response.json();
    if (data.success && Array.isArray(data.data)) {
      const formattedDiscounts = data.data.map((discount) => {
        const iconData = getDiscountIcon(discount);
        return {
          ...discount,
          ...iconData
        };
      });
      setDiscounts(formattedDiscounts);
    } else {
      console.warn("Invalid response format:", data);
      setDiscounts([]);
    }
  } catch (error) {
    console.error("Error fetching discounts:", error);
    setDiscounts([]);
  } finally {
    setLoading(false);
  }
};
const handleToggleStatus = async (id) => {
  try {
    const discount = discounts.find((d) => d._id === id);
    if (!discount) return;
    const newStatus = discount.status === "active" ? "inactive" : "active";
    const response = await fetch(API_ENDPOINTS.discountById(id), {
      method: "PUT",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({ status: newStatus })
    });
    const data = await response.json();
    if (data.success) {
      fetchDiscounts();
    } else {
      alert("Failed to update discount status");
    }
  } catch (error) {
    console.error("Error updating discount status:", error);
    alert("Error updating discount status");
```

```
  }
};
const handleDeleteClick = (discount) => {
  setDiscountToDelete(discount);
  setShowDeleteModal(true);
};
const confirmDelete = async () => {
  if (!discountToDelete) return;
  try {
    const response = await fetch(API_ENDPOINTS.discountById(discountToDelete._id), {
      method: "DELETE"
    });
    const data = await response.json();
    if (data.success) {
      setShowDeleteModal(false);
      setDiscountToDelete(null);
      fetchDiscounts();
    } else {
      alert("Failed to delete discount");
    }
  } catch (error) {
    console.error("Error deleting discount:", error);
    alert("Error deleting discount");
  }
};
const handleEditClick = async (discount) => {
  if (discount.appliesTo === "products" && discount.productIds && discount.productIds.length > 0) {
    try {
      const response = await fetch(API_ENDPOINTS.products);
      const data = await response.json();
      if (data.success && Array.isArray(data.data)) {
        const selectedProducts = data.data.filter(
          (p) => discount.productIds.includes(p._id)
        ).map((p) => ({
          _id: p._id,
          itemName: p.itemName,
          sku: p.sku,
          itemImage: p.itemImage,
          category: p.category,
          brandName: p.brandName
        }));
        setEditingDiscount({ ...discount, selectedProducts });
      } else {
        setEditingDiscount(discount);
      }
    } catch (error) {
      console.error("Error fetching products for edit:", error);
      setEditingDiscount(discount);
    }
  } else {
    setEditingDiscount(discount);
  }
  setShowAddModal(true);
};
const handleUpdateDiscount = async (id, formData) => {
  try {
    const discountData = {
      title: formData.discountName.toUpperCase(),
      discountCode: formData.discountCode || "",
      discountType: formData.discountType,
      discountValue: parseFloat(formData.discountValue),
      appliesTo: formData.appliesTo,
      category: formData.appliesTo === "category" ? formData.category : null,
      subCategory: formData.appliesTo === "category" && formData.subCategory ? formData.subCategory : null,
      productIds: formData.appliesTo === "products" ? formData.selectedProducts.map((p) => p._id) : [],
      validFrom: formData.noExpiration ? null : formData.validFrom,
      validTo: formData.noExpiration ? null : formData.validUntil,
      noExpiration: formData.noExpiration,
      minPurchaseAmount: formData.minPurchaseAmount ? parseFloat(formData.minPurchaseAmount) : 0,
      maxPurchaseAmount: formData.maxPurchaseAmount ? parseFloat(formData.maxPurchaseAmount) : null,
      usageLimit: formData.usageLimit && formData.usageLimit !== "0" ? parseInt(formData.usageLimit) : null,
      description: formData.description || ""
    };
    const response = await fetch(API_ENDPOINTS.discountById(id), {
      method: "PUT",
      headers: {
        "Content-Type": "application/json"
```

```
        },
        body: JSON.stringify(discountData)
      });
      const data = await response.json();
      if (data.success) {
        fetchDiscounts();
      } else {
        alert("Failed to update discount: " + (data.message || "Unknown error"));
      }
    } catch (error) {
      console.error("Error updating discount:", error);
      alert("Error updating discount");
    }
  };
  const handleAddDiscount = async (formData) => {
    try {
      const discountData = {
        title: formData.discountName.toUpperCase(),
        discountCode: formData.discountCode || "",
        discountType: formData.discountType,
        discountValue: parseFloat(formData.discountValue),
        appliesTo: formData.appliesTo,
        category: formData.appliesTo === "category" ? formData.category : null,
        subCategory: formData.appliesTo === "category" && formData.subCategory ? formData.subCategory : null,
        productIds: formData.appliesTo === "products" ? formData.selectedProducts.map((p) => p._id) : [],
        validFrom: formData.noExpiration ? null : formData.validFrom,
        validTo: formData.noExpiration ? null : formData.validUntil,
        noExpiration: formData.noExpiration,
        minPurchaseAmount: formData.minPurchaseAmount ? parseFloat(formData.minPurchaseAmount) : 0,
        maxPurchaseAmount: formData.maxPurchaseAmount ? parseFloat(formData.maxPurchaseAmount) : null,
        usageLimit: formData.usageLimit && formData.usageLimit !== "0" ? parseInt(formData.usageLimit) : null,
        description: formData.description || "",
        status: "active"
      };
      const response = await fetch(API_ENDPOINTS.discounts, {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify(discountData)
      });
      const data = await response.json();
      if (data.success) {
        fetchDiscounts();
      } else {
        alert("Failed to create discount: " + (data.message || "Unknown error"));
      }
    } catch (error) {
      console.error("Error creating discount:", error);
      alert("Error creating discount");
    }
  };
  const filteredDiscounts = discounts.filter((discount) => {
    const matchesSearch = discount.title.toLowerCase().includes(searchQuery.toLowerCase());
    const matchesType = filterType === "all" || discount.discountType === filterType;
    let matchesCategory = true;
    if (filterCategory !== "all") {
      if (filterCategory === "no-category") {
        matchesCategory = !discount.category || discount.category === null;
      } else {
        matchesCategory = String(discount.category || "").trim().toLowerCase() ===
String(filterCategory).trim().toLowerCase();
      }
    }
    return matchesSearch && matchesType && matchesCategory;
  });
  const categoryFilterOptions = useMemo(() => {
    const set = /* @__PURE__ */ new Set();
    discounts.forEach((d) => {
      if (d?.category) set.add(String(d.category).trim());
    });
    return Array.from(set).filter(Boolean).sort((a, b) => a.localeCompare(b));
  }, [discounts]);
  return /* @__PURE__ */ React.createElement("div", { className: `p-8 min-h-screen ${theme === "dark" ? "bg-
[#1E1B18]" : "bg-gray-50"}` }, /* @__PURE__ */ React.createElement(
    Header,
    {
```

```
      pageName: "Discount Management",
      profileBackground: theme === "dark" ? "bg-[#2A2724]" : "bg-gray-100",
      showBorder: false,
      userName: currentUser?.name || "Owner",
      userRole: "Owner"
    }
  ), /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-4 mb-6 justify-between
mt-8" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3 flex-1" }, /*
@__PURE__ */ React.createElement("div", { className: "relative flex-shrink-0", style: { width: "400px" } },
/* @__PURE__ */ React.createElement("div", { className: "absolute left-1 top-1/2 transform -translate-y-1/2
w-10 h-9 flex items-center justify-center text-white rounded-xl", style: { background: "linear-
gradient(135deg, #AD7F65 0%, #76462B 100%)" } }, /* @__PURE__ */ React.createElement(FaSearch, { className:
"text-sm" })), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      placeholder: "Search For...",
      value: searchQuery,
      onChange: (e) => setSearchQuery(e.target.value),
      className: `w-full h-11 pl-14 pr-4 border rounded-xl focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white placeholder-
gray-400" : "bg-white border-gray-300 text-gray-900"}`
    }
  )), /* @__PURE__ */ React.createElement(
    "select",
    {
      value: filterType,
      onChange: (e) => setFilterType(e.target.value),
      className: `px-4 py-2.5 rounded-lg border font-medium text-sm focus:outline-none focus:ring-2
focus:ring-[#AD7F65] focus:border-transparent flex-shrink-0 ${theme === "dark" ? "bg-[#2A2724] border-gray-
600 text-white" : "bg-white border-gray-300 text-gray-700"}`
    },
    /* @__PURE__ */ React.createElement("option", { value: "all" }, "All Types"),
    /* @__PURE__ */ React.createElement("option", { value: "percentage" }, "Percentage"),
    /* @__PURE__ */ React.createElement("option", { value: "fixed" }, "Fixed Amount")
  ), /* @__PURE__ */ React.createElement(
    "select",
    {
      value: filterCategory,
      onChange: (e) => setFilterCategory(e.target.value),
      className: `px-4 py-2.5 rounded-lg border font-medium text-sm focus:outline-none focus:ring-2
focus:ring-[#AD7F65] focus:border-transparent flex-shrink-0 ${theme === "dark" ? "bg-[#2A2724] border-gray-
600 text-white" : "bg-white border-gray-300 text-gray-700"}`
    },
    /* @__PURE__ */ React.createElement("option", { value: "all" }, "All Categories"),
    /* @__PURE__ */ React.createElement("option", { value: "no-category" }, "All Products"),
    categoryFilterOptions.map((category) => /* @__PURE__ */ React.createElement("option", { key: category,
value: category }, category))
  )), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => {
        setEditingDiscount(null);
        setShowAddModal(true);
      },
      className: "flex items-center gap-2 px-6 py-3 text-white rounded-lg font-medium shadow-md hover:shadow-
lg transition-all",
      style: { background: "linear-gradient(135deg, #10B981 0%, #059669 100%)" }
    },
    /* @__PURE__ */ React.createElement(FaPlus, { className: "w-4 h-4" }),
    "Add New Discount"
  )), loading ? /* @__PURE__ */ React.createElement("div", { className: "flex justify-center items-center py-
12" }, /* @__PURE__ */ React.createElement("div", { className: "text-gray-500" }, "Loading discounts...")) :
filteredDiscounts.length === 0 ? /* @__PURE__ */ React.createElement("div", { className: "flex justify-center
items-center py-12" }, /* @__PURE__ */ React.createElement("div", { className: "text-gray-500" }, "No
discounts found. Create your first discount!")) : /* @__PURE__ */ React.createElement("div", { className:
"grid grid-cols-2 gap-6" }, filteredDiscounts.map(
    (discount) => /* @__PURE__ */ React.createElement(
      "div",
      {
        key: discount._id,
        className: `rounded-xl overflow-hidden border shadow-lg flex ${theme === "dark" ? "bg-[#2A2724]
border-[#4A4037]" : "bg-white border-blue-200"}`
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
```

```
          className: "w-15 flex items-center justify-center shrink-0",
          style: { background: discount.iconColor }
        },
        /* @__PURE__ */ React.createElement(
          "img",
          {
            src: discount.icon,
            alt: discount.title,
            className: "w-full h-full object-contain p-2"
          }
        )
      ),
      /* @__PURE__ */ React.createElement("div", { className: `flex-1 p-4 relative ${theme === "dark" ? "bg-
[#2A2724]" : "bg-white"}` }, /* @__PURE__ */ React.createElement("div", { className: "absolute top-3 right-3
flex items-center gap-2" }, /* @__PURE__ */ React.createElement(
        "button",
        {
          className: "w-7 h-7 flex items-center justify-center bg-amber-100 hover:bg-amber-200 text-amber-700
rounded transition-colors",
          onClick: () => handleEditClick(discount)
        },
        /* @__PURE__ */ React.createElement(FaEdit, { className: "w-3 h-3" })
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          className: "w-7 h-7 flex items-center justify-center bg-red-100 hover:bg-red-200 text-red-700
rounded transition-colors",
          onClick: () => handleDeleteClick(discount)
        },
        /* @__PURE__ */ React.createElement(FaTrash, { className: "w-3 h-3" })
      ), /* @__PURE__ */ React.createElement("label", { className: "relative inline-flex items-center cursor-
pointer ml-1" }, /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "checkbox",
          checked: discount.status === "active",
          onChange: () => handleToggleStatus(discount._id),
          className: "sr-only peer"
        }
      ), /* @__PURE__ */ React.createElement("div", { className: `w-11 h-6 rounded-full peer peer-
checked:after:translate-x-full peer-checked:after:border-white after:content-['] after:absolute after:top-
[2px] after:left-[2px] after:bg-white after:border after:rounded-full after:h-5 after:w-5 after:transition-
all ${discount.status === "active" ? "bg-[#AD7F65] after:border-[#AD7F65]" : "bg-gray-200 after:border-gray-
300"}` }))), /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-2 mb-2 pr-32" },
/* @__PURE__ */ React.createElement(
        "h3",
        {
          className: "text-xl font-bold",
          style: { color: discount.status === "inactive" ? "#6B7280" : theme === "dark" ? "#C2A68C" :
"#AD7F65" }
        },
        discount.title
      ), /* @__PURE__ */ React.createElement(
        "span",
        {
          className: `px-2 py-0.5 rounded-full text-xs font-medium ${discount.status === "active" ? "bg-
green-500 text-white" : "bg-gray-300 text-gray-600"}`
        },
        discount.status === "active" ? "Active" : "Inactive"
      )), /* @__PURE__ */ React.createElement("div", { className: "flex gap-4" }, /* @__PURE__ */
React.createElement("div", { className: "flex-1 space-y-1" }, /* @__PURE__ */ React.createElement("div",
{ className: `flex items-center gap-1.5 text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-600"}` },
/* @__PURE__ */ React.createElement(FaTag, { className: "text-yellow-500 text-sm" }), /* @__PURE__ */
React.createElement("span", null, "Discount Value: ", /* @__PURE__ */ React.createElement("span", { className:
"font-bold" }, discount.discountValue))), /* @__PURE__ */ React.createElement("div", { className: `flex
items-center gap-1.5 text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-600"}` }, /* @__PURE__ */
React.createElement(FaCalendar, { className: "text-purple-500 text-sm" }), /* @__PURE__ */
React.createElement("span", null, "Valid only from: ", discount.validFrom === "Permanent" ? "Permanent" :
`${discount.validFrom} to ${discount.validTo}`)), discount.description && /* @__PURE__ */
React.createElement("div", { className: `text-xs italic ${theme === "dark" ? "text-gray-500" : "text-gray-
600"}` }, discount.description)), /* @__PURE__ */ React.createElement("div", { className: "flex-1 space-y-1
text-right" }, /* @__PURE__ */ React.createElement("div", { className: `flex items-center justify-end gap-1.5
text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-600"}` }, /* @__PURE__ */
React.createElement("span", null, "Applies to: ", /* @__PURE__ */ React.createElement("span", { className:
"font-bold" }, discount.appliesTo)), /* @__PURE__ */ React.createElement(FaBox, { className: "text-blue-400
text-sm" })), discount.usage && /* @__PURE__ */ React.createElement("div", { className: `flex items-center
justify-end gap-1.5 text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-600"}` }, /* @__PURE__ */
```

```
React.createElement("span", null, "Used: ", /* @__PURE__ */ React.createElement("span", { className: "font-
bold" }, discount.usage.used, "/", discount.usage.total), " times"), /* @__PURE__ */
React.createElement(FaUsers, { className: "text-green-500 text-sm" }))))))
    )
  )), /* @__PURE__ */ React.createElement(
    AddDiscountModal,
    {
      isOpen: showAddModal,
      onClose: () => {
        setShowAddModal(false);
        setEditingDiscount(null);
      },
      onAdd: handleAddDiscount,
      onEdit: handleUpdateDiscount,
      discountToEdit: editingDiscount
    }
  ), showDeleteModal && discountToDelete && /* @__PURE__ */ React.createElement("div", { className: "fixed
inset-0 z-[10000] flex items-center justify-center p-4 bg-black/60 backdrop-blur-sm" }, /* @__PURE__ */
React.createElement("div", { className: `rounded-2xl w-full max-w-md relative shadow-2xl overflow-hidden
${theme === "dark" ? "bg-[#1E1B18]" : "bg-white"}` }, /* @__PURE__ */ React.createElement("div", { className:
"p-6" }, /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3 mb-4" }, /*
@__PURE__ */ React.createElement("div", { className: "w-12 h-12 rounded-full bg-red-100 flex items-center
justify-center" }, /* @__PURE__ */ React.createElement(FaTrash, { className: "w-5 h-5 text-red-600" })), /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("h2", { className: `text-xl
font-bold ${theme === "dark" ? "text-white" : "text-gray-900"}` }, "Delete Discount"), /* @__PURE__ */
React.createElement("p", { className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}` },
"This action cannot be undone"))), /* @__PURE__ */ React.createElement("div", { className: `p-4 rounded-lg
mb-6 ${theme === "dark" ? "bg-[#2A2724]" : "bg-gray-50"}` }, /* @__PURE__ */ React.createElement("p",
{ className: `text-sm mb-2 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}` }, "Are you sure you want
to delete this discount?"), /* @__PURE__ */ React.createElement("div", { className: `mt-3 p-3 rounded-lg
border ${theme === "dark" ? "bg-[#1E1B18] border-gray-700" : "bg-white border-gray-200"}` }, /* @__PURE__ */
React.createElement("p", { className: "font-semibold text-[#AD7F65]" }, discountToDelete.title), /* @__PURE__
*/ React.createElement("p", { className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-600"}` },
discountToDelete.discountType === "percentage" ? `${discountToDelete.discountValue}% off` :
`\u20B1${discountToDelete.discountValue} off`))), /* @__PURE__ */ React.createElement("div", { className:
"flex gap-3 justify-end" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => {
        setShowDeleteModal(false);
        setDiscountToDelete(null);
      },
      className: `px-6 py-2.5 rounded-lg font-medium transition-colors ${theme === "dark" ? "bg-[#2A2724]
text-gray-300 hover:bg-[#3A3734]" : "bg-gray-100 text-gray-700 hover:bg-gray-200"}`
    },
    "Cancel"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: confirmDelete,
      className: "px-6 py-2.5 rounded-lg bg-red-600 text-white font-medium hover:bg-red-700 transition-colors
shadow-lg"
    },
    "Delete Discount"
  ))))));
};
export default DiscountManagement;
```

FILE: pos/frontend/src/pages/owner/GCashSettings.jsx
==========

```
import { memo, useCallback, useEffect, useState } from "react";
import {
  FaCheckCircle,
  FaCog,
  FaEye,
  FaEyeSlash,
  FaKey,
  FaLink,
  FaShieldAlt,
  FaSpinner,
  FaTimesCircle,
  FaTrash
} from "react-icons/fa";
import Header from "../../components/shared/header";
import { API_BASE_URL as API_BASE } from "../../config/api";
```

```
import { useAuth } from "../../context/AuthContext";
import { useTheme } from "../../context/ThemeContext";
const GCashSettings = () => {
  const { theme } = useTheme();
  const { currentUser } = useAuth();
  const isDark = theme === "dark";
  const [appId, setAppId] = useState("");
  const [privateKey, setPrivateKey] = useState("");
  const [publicKey, setPublicKey] = useState("");
  const [environment, setEnvironment] = useState("sandbox");
  const [merchantName, setMerchantName] = useState("POS System");
  const [paymentExpiryMinutes, setPaymentExpiryMinutes] = useState(15);
  const [webhookUrl, setWebhookUrl] = useState("");
  const [isConfigured, setIsConfigured] = useState(false);
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);
  const [testing, setTesting] = useState(false);
  const [deleting, setDeleting] = useState(false);
  const [showPrivateKey, setShowPrivateKey] = useState(false);
  const [message, setMessage] = useState({ type: "", text: "" });
  const [lastUpdated, setLastUpdated] = useState(null);
  const fetchSettings = useCallback(async () => {
    try {
      setLoading(true);
      const response = await fetch(`${API_BASE}/api/merchant-settings`);
      const data = await response.json();
      if (data.success && data.data) {
        const settings = data.data;
        setAppId(settings.appId || "");
        setPublicKey(settings.publicKey || "");
        setEnvironment(settings.environment || "sandbox");
        setMerchantName(settings.merchantName || "POS System");
        setPaymentExpiryMinutes(settings.paymentExpiryMinutes || 15);
        setWebhookUrl(settings.webhookUrl || "");
        setIsConfigured(true);
        setLastUpdated(settings.updatedAt || settings.createdAt);
        setPrivateKey("");
      } else {
        setIsConfigured(false);
      }
    } catch (error) {
      console.error("Error fetching settings:", error);
      setMessage({
        type: "error",
        text: "Failed to load settings. Is the backend running?"
      });
    } finally {
      setLoading(false);
    }
  }, []);
  useEffect(() => {
    fetchSettings();
  }, [fetchSettings]);
  const handleSave = async (e) => {
    e.preventDefault();
    setMessage({ type: "", text: "" });
    if (!appId.trim() || !publicKey.trim()) {
      setMessage({
        type: "error",
        text: "App ID and Public Key are required."
      });
      return;
    }
    if (!isConfigured && !privateKey.trim()) {
      setMessage({
        type: "error",
        text: "Private Key is required for initial setup."
      });
      return;
    }
    if (isConfigured && !privateKey.trim()) {
      setMessage({
        type: "error",
        text: "Please re-enter your Private Key to save changes."
      });
      return;
    }
```

```
  try {
    setSaving(true);
    const response = await fetch(`${API_BASE}/api/merchant-settings`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        appId: appId.trim(),
        privateKey: privateKey.trim(),
        publicKey: publicKey.trim(),
        environment,
        merchantName: merchantName.trim(),
        paymentExpiryMinutes,
        configuredBy: currentUser?._id || currentUser?.id || "",
        configuredByName: currentUser?.name || ""
      })
    });
    const data = await response.json();
    if (data.success) {
      setMessage({
        type: "success",
        text: "Payment gateway configured successfully!"
      });
      setPrivateKey("");
      setIsConfigured(true);
      if (data.data?.webhookUrl) setWebhookUrl(data.data.webhookUrl);
      setLastUpdated((/* @__PURE__ */ new Date()).toISOString());
    } else {
      setMessage({
        type: "error",
        text: data.message || "Failed to save settings."
      });
    }
  } catch (error) {
    console.error("Error saving settings:", error);
    setMessage({ type: "error", text: "Network error. Please try again." });
  } finally {
    setSaving(false);
  }
};
const handleTestConnection = async () => {
  setMessage({ type: "", text: "" });
  if (!isConfigured) {
    setMessage({
      type: "error",
      text: "Please save your credentials first before testing."
    });
    return;
  }
  try {
    setTesting(true);
    const response = await fetch(`${API_BASE}/api/merchant-settings/test`, {
      method: "POST"
    });
    const data = await response.json();
    if (data.success) {
      setMessage({ type: "success", text: `\u2705 ${data.message}` });
    } else {
      setMessage({
        type: "error",
        text: data.message || "Connection test failed."
      });
    }
  } catch (error) {
    setMessage({ type: "error", text: "Unable to test connection." });
  } finally {
    setTesting(false);
  }
};
const handleDelete = async () => {
  if (!window.confirm(
    "Are you sure you want to remove the payment gateway configuration? GCash payments will be disabled."
  )) {
    return;
  }
  try {
    setDeleting(true);
    const response = await fetch(`${API_BASE}/api/merchant-settings`, {
```

```
        method: "DELETE"
      });
      const data = await response.json();
      if (data.success) {
        setMessage({
          type: "success",
          text: "Configuration removed successfully."
        });
        setAppId("");
        setPrivateKey("");
        setPublicKey("");
        setEnvironment("sandbox");
        setMerchantName("POS System");
        setWebhookUrl("");
        setIsConfigured(false);
        setLastUpdated(null);
      } else {
        setMessage({
          type: "error",
          text: data.message || "Failed to remove configuration."
        });
      }
    } catch (error) {
      setMessage({ type: "error", text: "Network error." });
    } finally {
      setDeleting(false);
    }
  };
  if (loading) {
    return /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-center min-h-
[60vh]" }, /* @__PURE__ */ React.createElement(FaSpinner, { className: "w-8 h-8 text-[#8B7355] animate-
spin" }));
  }
  return /* @__PURE__ */ React.createElement("div", { className: "font-poppins" }, /* @__PURE__ */
React.createElement(Header, { pageName: "GCash Payment Settings", showBorder: false }), /* @__PURE__ */
React.createElement("div", { className: "max-w-2xl mx-auto mt-6" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-xl p-4 mb-6 flex items-center gap-3 ${isConfigured ? isDark ? "bg-green-900/20
border border-green-800" : "bg-green-50 border border-green-200" : isDark ? "bg-yellow-900/20 border border-
yellow-800" : "bg-yellow-50 border border-yellow-200"}`
    },
    isConfigured ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(FaCheckCircle, { className: "w-5 h-5 text-green-500 flex-shrink-0" }), /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `font-medium ${isDark ? "text-green-400" : "text-green-700"}`
      },
      "GCash Payment Gateway Connected"
    ), /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-xs ${isDark ? "text-green-500/70" : "text-green-600/70"}`
      },
      "Environment: ",
      environment.toUpperCase(),
      lastUpdated && ` \u2022 Updated: ${new Date(lastUpdated).toLocaleDateString()}`
    ))) : /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(FaCog,
{ className: "w-5 h-5 text-yellow-500 flex-shrink-0" }), /* @__PURE__ */ React.createElement("div", null, /*
@__PURE__ */ React.createElement(
      "p",
      {
        className: `font-medium ${isDark ? "text-yellow-400" : "text-yellow-700"}`
      },
      "Payment Gateway Not Configured"
    ), /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-xs ${isDark ? "text-yellow-500/70" : "text-yellow-600/70"}`
      },
      "Enter your GCash for Business / PayMongo credentials below"
    )))
  ), /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `rounded-xl p-4 mb-6 flex items-start gap-3 ${isDark ? "bg-[#2A2724]" : "bg-gray-50"}`
```

```
    },
    /* @__PURE__ */ React.createElement(
      FaShieldAlt,
      {
        className: `w-4 h-4 mt-0.5 flex-shrink-0 ${isDark ? "text-blue-400" : "text-blue-500"}`
      }
    ),
    /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-xs font-medium ${isDark ? "text-gray-300" : "text-gray-700"}`
      },
      "Security"
    ), /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-xs ${isDark ? "text-gray-400" : "text-gray-500"}`
      },
      "Your private key is encrypted with AES-256-GCM before storage and is never exposed to the frontend
after saving. All payment communications use HTTPS. Webhook signatures are verified using HMAC-SHA256."
    ))
  ), /* @__PURE__ */ React.createElement("form", { onSubmit: handleSave }, /* @__PURE__ */
React.createElement(
    "div",
    {
      className: `rounded-xl border p-6 space-y-5 ${isDark ? "bg-[#1E1B18] border-gray-700" : "bg-white
border-gray-200"}`
    },
    /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
      "label",
      {
        className: `block text-sm font-medium mb-1.5 ${isDark ? "text-gray-300" : "text-gray-700"}`
      },
      "App ID / API Key ID"
    ), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "text",
        value: appId,
        onChange: (e) => setAppId(e.target.value),
        placeholder: "[REDACTED]",
        className: `w-full px-4 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#8B7355] ${isDark ? "bg-[#2A2724] border-gray-600 text-white placeholder-gray-500" : "bg-white border-gray-
300 text-gray-900"}`,
        required: true
      }
    ), /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-xs mt-1 ${isDark ? "text-gray-500" : "text-gray-400"}`
      },
      "Your PayMongo public API key"
    )),
    /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
      "label",
      {
        className: `block text-sm font-medium mb-1.5 ${isDark ? "text-gray-300" : "text-gray-700"}`
      },
      /* @__PURE__ */ React.createElement(FaKey, { className: "inline w-3 h-3 mr-1" }),
      "Secret Key (Private Key)"
    ), /* @__PURE__ */ React.createElement("div", { className: "relative" }, /* @__PURE__ */
React.createElement(
      "input",
      {
        type: showPrivateKey ? "text" : "password",
        value: privateKey,
        onChange: (e) => setPrivateKey(e.target.value),
        placeholder: isConfigured ?
"\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022 (re-enter to
update)" : "[REDACTED]",
        className: `w-full px-4 py-2.5 pr-10 text-sm border rounded-lg focus:outline-none focus:ring-2
focus:ring-[#8B7355] ${isDark ? "bg-[#2A2724] border-gray-600 text-white placeholder-gray-500" : "bg-white
border-gray-300 text-gray-900"}`,
        required: !isConfigured
      }
    ), /* @__PURE__ */ React.createElement(
      "button",
```

```
      {
        type: "button",
        onClick: () => setShowPrivateKey(!showPrivateKey),
        className: `absolute right-3 top-1/2 -translate-y-1/2 ${isDark ? "text-gray-400" : "text-gray-500"}`
      },
      showPrivateKey ? /* @__PURE__ */ React.createElement(FaEyeSlash, { className: "w-4 h-4" }) : /*
@__PURE__ */ React.createElement(FaEye, { className: "w-4 h-4" })
    )), /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-xs mt-1 ${isDark ? "text-gray-500" : "text-gray-400"}`
      },
      "Your PayMongo secret key \u2014 encrypted before storage, never exposed after save"
    )),
    /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
      "label",
      {
        className: `block text-sm font-medium mb-1.5 ${isDark ? "text-gray-300" : "text-gray-700"}`
      },
      "Public Key"
    ), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "text",
        value: publicKey,
        onChange: (e) => setPublicKey(e.target.value),
        placeholder: "[REDACTED]",
        className: `w-full px-4 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#8B7355] ${isDark ? "bg-[#2A2724] border-gray-600 text-white placeholder-gray-500" : "bg-white border-gray-
300 text-gray-900"}`,
        required: true
      }
    )),
    /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
      "label",
      {
        className: `block text-sm font-medium mb-1.5 ${isDark ? "text-gray-300" : "text-gray-700"}`
      },
      "Environment"
    ), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3" }, /* @__PURE__ */
React.createElement(
      "button",
      {
        type: "button",
        onClick: () => setEnvironment("sandbox"),
        className: `flex-1 py-2.5 rounded-lg text-sm font-medium border-2 transition-all ${environment ===
"sandbox" ? "border-yellow-500 bg-yellow-50 text-yellow-700" : isDark ? "border-gray-600 bg-[#2A2724] text-
gray-400 hover:border-gray-500" : "border-gray-200 bg-gray-50 text-gray-500 hover:border-gray-300"}`
      },
      "\u{1F9EA} Sandbox"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        type: "button",
        onClick: () => setEnvironment("production"),
        className: `flex-1 py-2.5 rounded-lg text-sm font-medium border-2 transition-all ${environment ===
"production" ? "border-green-500 bg-green-50 text-green-700" : isDark ? "border-gray-600 bg-[#2A2724] text-
gray-400 hover:border-gray-500" : "border-gray-200 bg-gray-50 text-gray-500 hover:border-gray-300"}`
      },
      "\u{1F680} Production"
    ))),
    /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
      "label",
      {
        className: `block text-sm font-medium mb-1.5 ${isDark ? "text-gray-300" : "text-gray-700"}`
      },
      "Merchant Display Name"
    ), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "text",
        value: merchantName,
        onChange: (e) => setMerchantName(e.target.value),
        placeholder: "Your Store Name",
        className: `w-full px-4 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#8B7355] ${isDark ? "bg-[#2A2724] border-gray-600 text-white placeholder-gray-500" : "bg-white border-gray-
300 text-gray-900"}`
```

```
      }
    )),
    /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
      "label",
      {
        className: `block text-sm font-medium mb-1.5 ${isDark ? "text-gray-300" : "text-gray-700"}`
      },
      "Payment Expiry (minutes)"
    ), /* @__PURE__ */ React.createElement(
      "input",
      {
        type: "number",
        value: paymentExpiryMinutes,
        onChange: (e) => setPaymentExpiryMinutes(
          Math.min(60, Math.max(5, parseInt(e.target.value) || 15))
        ),
        min: "5",
        max: "60",
        className: `w-24 px-4 py-2.5 text-sm border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#8B7355] ${isDark ? "bg-[#2A2724] border-gray-600 text-white" : "bg-white border-gray-300 text-gray-900"}`
      }
    ), /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-xs mt-1 ${isDark ? "text-gray-500" : "text-gray-400"}`
      },
      "QR codes expire after this many minutes (5\u201360)"
    )),
    webhookUrl && /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
      "label",
      {
        className: `block text-sm font-medium mb-1.5 ${isDark ? "text-gray-300" : "text-gray-700"}`
      },
      /* @__PURE__ */ React.createElement(FaLink, { className: "inline w-3 h-3 mr-1" }),
      "Webhook URL (auto-generated)"
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `w-full px-4 py-2.5 border rounded-lg font-mono text-xs break-all ${isDark ? "bg-[#2A2724]
border-gray-600 text-gray-400" : "bg-gray-100 border-gray-200 text-gray-600"}`
      },
      webhookUrl
    ), /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-xs mt-1 ${isDark ? "text-gray-500" : "text-gray-400"}`
      },
      "Register this URL in your PayMongo dashboard under Webhooks \u2192 Source Chargeable"
    ))
  ), message.text && /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `mt-4 rounded-lg p-3 flex items-center gap-2 ${message.type === "success" ? isDark ? "bg-
green-900/20 text-green-400" : "bg-green-50 text-green-700" : isDark ? "bg-red-900/20 text-red-400" : "bg-
red-50 text-red-700"}`
    },
    message.type === "success" ? /* @__PURE__ */ React.createElement(FaCheckCircle, { className: "w-4 h-4
flex-shrink-0" }) : /* @__PURE__ */ React.createElement(FaTimesCircle, { className: "w-4 h-4 flex-shrink-
0" }),
    /* @__PURE__ */ React.createElement("span", { className: "text-sm" }, message.text)
  ), /* @__PURE__ */ React.createElement("div", { className: "mt-6 flex gap-3" }, /* @__PURE__ */
React.createElement(
    "button",
    {
      type: "submit",
      disabled: saving,
      className: "flex-1 py-3 px-6 rounded-lg font-semibold text-white bg-[#8B7355] hover:bg-[#6d5a43]
transition-all disabled:opacity-50 flex items-center justify-center gap-2"
    },
    saving ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(FaSpinner, { className: "w-4 h-4 animate-spin" }), "Saving...") : isConfigured ? "Update
Configuration" : "Save Configuration"
  ), isConfigured && /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(
    "button",
    {
      type: "button",
```

```
      onClick: handleTestConnection,
      disabled: testing,
      className: `py-3 px-6 rounded-lg font-semibold transition-all flex items-center gap-2 ${isDark ? "bg-
[#2A2724] text-gray-300 hover:bg-[#322f2c]" : "bg-gray-100 text-gray-700 hover:bg-gray-200"}
disabled:opacity-50`
    },
    testing ? /* @__PURE__ */ React.createElement(FaSpinner, { className: "w-4 h-4 animate-spin" }) : "Test"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: handleDelete,
      disabled: deleting,
      className: "py-3 px-4 rounded-lg font-semibold text-red-500 hover:bg-red-50 transition-all
disabled:opacity-50"
    },
    /* @__PURE__ */ React.createElement(FaTrash, { className: "w-4 h-4" })
  )))), /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `mt-8 rounded-xl border p-6 ${isDark ? "bg-[#1E1B18] border-gray-700" : "bg-white border-
gray-200"}`
    },
    /* @__PURE__ */ React.createElement(
      "h3",
      {
        className: `text-sm font-semibold mb-3 ${isDark ? "text-gray-200" : "text-gray-800"}`
      },
      "\u{1F4CB} Setup Guide"
    ),
    /* @__PURE__ */ React.createElement(
      "ol",
      {
        className: `text-xs space-y-2 ${isDark ? "text-gray-400" : "text-gray-500"}`
      },
      /* @__PURE__ */ React.createElement("li", null, /* @__PURE__ */ React.createElement("strong", null,
"1."), " Sign up for", " ", /* @__PURE__ */ React.createElement(
        "a",
        {
          href: "https://dashboard.paymongo.com",
          target: "_blank",
          rel: "noopener noreferrer",
          className: "text-blue-500 underline"
        },
        "PayMongo"
      ), " ", "(GCash for Business payment processor)"),
      /* @__PURE__ */ React.createElement("li", null, /* @__PURE__ */ React.createElement("strong", null,
"2."), " Get your API keys from the PayMongo Dashboard \u2192 Developers \u2192 API Keys"),
      /* @__PURE__ */ React.createElement("li", null, /* @__PURE__ */ React.createElement("strong", null,
"3."), " Enter your Secret Key ([REDACTED]) and Public Key ([REDACTED]) above"),
      /* @__PURE__ */ React.createElement("li", null, /* @__PURE__ */ React.createElement("strong", null,
"4."), " Start with ", /* @__PURE__ */ React.createElement("strong", null, "Sandbox"), " ", "environment for
testing"),
      /* @__PURE__ */ React.createElement("li", null, /* @__PURE__ */ React.createElement("strong", null,
"5."), ' Click "Save Configuration" then "Test" to verify'),
      /* @__PURE__ */ React.createElement("li", null, /* @__PURE__ */ React.createElement("strong", null,
"6."), " Register the Webhook URL above in PayMongo Dashboard \u2192 Webhooks \u2192 Event: ", /* @__PURE__
*/ React.createElement("code", null, "source.chargeable")),
      /* @__PURE__ */ React.createElement("li", null, /* @__PURE__ */ React.createElement("strong", null,
"7."), " Set ", /* @__PURE__ */ React.createElement("code", null, "PAYMONGO_WEBHOOK_SECRET"), " env var with
the webhook signing key from PayMongo"),
      /* @__PURE__ */ React.createElement("li", null, /* @__PURE__ */ React.createElement("strong", null,
"8."), " Switch to ", /* @__PURE__ */ React.createElement("strong", null, "Production"), " when ready to
accept real payments")
    )
  )));
};
export default memo(GCashSettings);
```

FILE: pos/frontend/src/pages/owner/ManageEmployees.jsx
==========

```
import { memo, useEffect, useState } from "react";
import {
  FaEdit,
  FaEllipsisV,
```

```
  FaPlus,
  FaSearch,
  FaTrash,
  FaUndo
} from "react-icons/fa";
import defaultAvatar from "../../assets/default.jpeg";
import Pagination from "../../components/inventory/Pagination";
import SuccessModal from "../../components/inventory/SuccessModal";
import AddEmployeeModal from "../../components/owner/AddEmployeeModal";
import DeleteEmployeeModal from "../../components/owner/DeleteEmployeeModal";
import DisableAccountModal from "../../components/owner/DisableAccountModal";
import EditEmployeeModal from "../../components/owner/EditEmployeeModal";
import ResetPinConfirmModal from "../../components/owner/ResetPinConfirmModal";
import TemporaryPinModal from "../../components/owner/TemporaryPinModal";
import ViewEmployeeModal from "../../components/owner/ViewEmployeeModal";
import Header from "../../components/shared/header";
import { useTheme } from "../../context/ThemeContext";
import { useAuth } from "../../context/AuthContext";
import { API_BASE_URL } from "../../config/api";
const ManageEmployees = () => {
  const { theme } = useTheme();
  const { currentUser } = useAuth();
  const [searchQuery, setSearchQuery] = useState("");
  const [currentPage, setCurrentPage] = useState(1);
  const [filterStatus, setFilterStatus] = useState("All");
  const [viewingEmployee, setViewingEmployee] = useState(null);
  const [showViewModal, setShowViewModal] = useState(false);
  const [editingEmployee, setEditingEmployee] = useState(null);
  const [showEditModal, setShowEditModal] = useState(false);
  const [deletingEmployee, setDeletingEmployee] = useState(null);
  const [showDeleteModal, setShowDeleteModal] = useState(false);
  const [showAddModal, setShowAddModal] = useState(false);
  const [togglingEmployee, setTogglingEmployee] = useState(null);
  const [showDisableModal, setShowDisableModal] = useState(false);
  const [toggleAction, setToggleAction] = useState("disable");
  const [showSuccessModal, setShowSuccessModal] = useState(false);
  const [successMessage, setSuccessMessage] = useState("");
  const [showResetPinModal, setShowResetPinModal] = useState(false);
  const [resettingEmployee, setResettingEmployee] = useState(null);
  const [showTempPinModal, setShowTempPinModal] = useState(false);
  const [newEmployeeName, setNewEmployeeName] = useState("");
  const [newEmployeePin, setNewEmployeePin] = useState("");
  const [openDropdown, setOpenDropdown] = useState(null);
  useEffect(() => {
    const handleClickOutside = (event) => {
      if (openDropdown && !event.target.closest(".dropdown-container")) {
        setOpenDropdown(null);
      }
    };
    document.addEventListener("mousedown", handleClickOutside);
    return () => {
      document.removeEventListener("mousedown", handleClickOutside);
    };
  }, [openDropdown]);
  const [employees, setEmployees] = useState([]);
  const [loading, setLoading] = useState(true);
  const [onlineEmployeeIds, setOnlineEmployeeIds] = useState(/* @__PURE__ */ new Set());
  const itemsPerPage = 12;
  const fetchOnlineEmployees = async () => {
    try {
      const response = await fetch(`${API_BASE_URL}/api/employees/online`);
      const data = await response.json();
      if (data?.success && Array.isArray(data?.data)) {
        setOnlineEmployeeIds(new Set(data.data.map((emp) => emp._id)));
        return;
      }
      if (Array.isArray(data)) {
        setOnlineEmployeeIds(new Set(data.map((emp) => emp._id)));
      }
    } catch (err) {
      console.error("Error fetching online employees:", err);
      setOnlineEmployeeIds(/* @__PURE__ */ new Set());
    }
  };
  const fetchEmployees = async (showLoading = true) => {
    try {
      if (showLoading) setLoading(true);
```

```
      const response = await fetch(`${API_BASE_URL}/api/employees`);
      const data = await response.json();
      if (data.success) {
        const timestamp = (/* @__PURE__ */ new Date()).getTime();
        const employeesWithImages = data.data.map((emp) => ({
          ...emp,
          image: `${API_BASE_URL}/api/employees/${emp._id}/image?t=${timestamp}`,
          id: emp._id,
          contactNumber: emp.contactNo || emp.contactNumber || ""
        }));
        setEmployees(employeesWithImages);
      }
    } catch (error) {
      console.error("Error fetching employees:", error);
      alert(
        "Failed to fetch employees. Make sure the backend server is running."
      );
    } finally {
      if (showLoading) setLoading(false);
    }
  };
  useEffect(() => {
    fetchEmployees();
  }, []);
  useEffect(() => {
    fetchOnlineEmployees();
    const t = setInterval(fetchOnlineEmployees, 3e4);
    return () => clearInterval(t);
  }, []);
  const filteredEmployees = employees.filter((employee) => {
    const matchesSearch = employee.name.toLowerCase().includes(searchQuery.toLowerCase()) || employee.role &&
employee.role.toLowerCase().includes(searchQuery.toLowerCase());
    let matchesStatus = true;
    if (filterStatus === "Active") matchesStatus = employee.status === "Active";
    if (filterStatus === "Archived")
      matchesStatus = employee.status === "Inactive";
    return matchesSearch && matchesStatus;
  });
  const totalPages = Math.ceil(filteredEmployees.length / itemsPerPage);
  const paginatedEmployees = filteredEmployees.slice(
    (currentPage - 1) * itemsPerPage,
    currentPage * itemsPerPage
  );
  const handleViewEmployee = (employee) => {
    setViewingEmployee(employee);
    setShowViewModal(true);
  };
  const handleEditEmployee = (employee) => {
    setEditingEmployee(employee);
    setShowEditModal(true);
  };
  const generateRandomPin = () => {
    return Math.floor(1e5 + Math.random() * 9e5).toString();
  };
  const handleResetPin = (employee) => {
    setResettingEmployee(employee);
    setShowResetPinModal(true);
  };
  const confirmResetPin = async () => {
    if (!resettingEmployee) return;
    setShowResetPinModal(false);
    try {
      const response = await fetch(
        `${API_BASE_URL}/api/employees/${resettingEmployee._id || resettingEmployee.id}/send-temporary-pin`,
        {
          method: "POST",
          headers: { "Content-Type": "application/json" }
        }
      );
      const data = await response.json();
      if (data.success) {
        setResettingEmployee(null);
        if (data.emailSent) {
          setSuccessMessage(
            `A new temporary PIN has been emailed to ${resettingEmployee.email}.`
          );
        } else {
```

```
          setSuccessMessage(
            `Temporary PIN generated: ${data.tempPin}. Email could not be sent - please share this PIN
manually.`
          );
        }
        setShowSuccessModal(true);
        fetchEmployees(false);
      } else {
        alert(data.message || "Failed to reset PIN");
      }
    } catch (error) {
      console.error("Error resetting PIN:", error);
      alert("Failed to reset PIN. Please try again.");
    }
  };
  const handleToggleStatus = (employee) => {
    setTogglingEmployee(employee);
    const action = employee.status === "Active" ? "disable" : "enable";
    setToggleAction(action);
    setShowDisableModal(true);
  };
  const confirmToggleStatus = async () => {
    if (!togglingEmployee) return;
    const newStatus = togglingEmployee.status === "Active" ? "Inactive" : "Active";
    const employeeName = togglingEmployee.name;
    try {
      const response = await fetch(
        `${API_BASE_URL}/api/employees/${togglingEmployee._id || togglingEmployee.id}`,
        {
          method: "PUT",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({ status: newStatus })
        }
      );
      const data = await response.json();
      if (data.success) {
        setShowDisableModal(false);
        setTogglingEmployee(null);
        const actionPastTense = toggleAction === "disable" ? "disabled" : "enabled";
        setSuccessMessage(
          `${employeeName}'s account has been ${actionPastTense} successfully!`
        );
        setShowSuccessModal(true);
        fetchEmployees(false);
      } else {
        alert(data.message || `Failed to ${toggleAction} account`);
      }
    } catch (error) {
      console.error(`Error ${toggleAction}ing account:`, error);
      alert(`Failed to ${toggleAction} account. Please try again.`);
    }
  };
  const handleDeleteEmployee = (employee) => {
    setDeletingEmployee(employee);
    setShowDeleteModal(true);
  };
  const confirmDeleteEmployee = async () => {
    if (!deletingEmployee) return;
    try {
      const response = await fetch(
        `${API_BASE_URL}/api/employees/${deletingEmployee._id || deletingEmployee.id}`,
        {
          method: "DELETE"
        }
      );
      const data = await response.json();
      if (data.success) {
        fetchEmployees(false);
        setShowDeleteModal(false);
        setDeletingEmployee(null);
      } else {
        alert(data.message || "Failed to delete employee");
      }
    } catch (error) {
      console.error("Error deleting employee:", error);
      alert("Failed to delete employee. Please try again.");
    }
```

```
  };
  return /* @__PURE__ */ React.createElement("div", { className: "p-8 min-h-screen" }, /* @__PURE__ */
React.createElement(
    Header,
    {
      pageName: "Manage Employees",
      profileBackground: "bg-gray-100",
      showBorder: false
    }
  ), /* @__PURE__ */ React.createElement("div", { className: "mt-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center justify-between mb-8" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-4" }, /* @__PURE__ */
React.createElement("div", { className: "relative", style: { width: "450px" } }, /* @__PURE__ */
React.createElement(
    "div",
    {
      className: "absolute left-1 top-1/2 transform -translate-y-1/2 w-12 h-9 flex items-center justify-
center text-white rounded-lg",
      style: {
        background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)"
      }
    },
    /* @__PURE__ */ React.createElement(FaSearch, { className: "text-sm" })
  ), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      placeholder: "Search For...",
      value: searchQuery,
      onChange: (e) => setSearchQuery(e.target.value),
      className: `w-full h-10 pl-16 pr-4 py-3 border rounded-lg focus:outline-none focus:ring-2 focus:ring-
[#AD7F65] focus:border-transparent ${theme === "dark" ? "bg-[#2A2724] border-[#4A4037] text-white
placeholder-gray-500" : "bg-white border-gray-300 text-gray-900"}`
    }
  )), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3 ml-4" }, /* @__PURE__ */
React.createElement(
    "button",
    {
      onClick: () => setFilterStatus("All"),
      className: `px-6 py-2.5 text-sm font-bold rounded-xl transition-all shadow-sm border ${filterStatus ===
"All" ? "bg-white text-[#AD7F65] border-gray-100 border-b-[4px] border-b-[#AD7F65]" : "bg-white text-gray-800
border-gray-200 border-b-[4px] border-b-gray-200 hover:bg-gray-50"}`
    },
    "All"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setFilterStatus("Active"),
      className: `px-6 py-2.5 text-sm font-bold rounded-xl transition-all shadow-sm border ${filterStatus ===
"Active" ? "bg-white text-[#AD7F65] border-gray-100 border-b-[4px] border-b-[#AD7F65]" : "bg-white text-gray-
800 border-gray-200 border-b-[4px] border-b-gray-200 hover:bg-gray-50"}`
    },
    "Active"
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setFilterStatus("Archived"),
      className: `px-6 py-2.5 text-sm font-bold rounded-xl transition-all shadow-sm border ${filterStatus ===
"Archived" ? "bg-white text-[#AD7F65] border-gray-100 border-b-[4px] border-b-[#AD7F65]" : "bg-white text-
gray-800 border-gray-200 border-b-[4px] border-b-gray-200 hover:bg-gray-50"}`
    },
    "Archived"
  ))), /* @__PURE__ */ React.createElement(
    "button",
    {
      onClick: () => setShowAddModal(true),
      className: "flex items-center gap-2 px-6 py-3 text-white rounded-lg font-medium shadow-md hover:shadow-
lg transition-all",
      style: {
        background: "linear-gradient(135deg, #10B981 0%, #059669 100%)"
      }
    },
    /* @__PURE__ */ React.createElement(FaPlus, { className: "w-4 h-4" }),
    "Add Employee"
  )), loading ? /* @__PURE__ */ React.createElement("div", { className: "flex justify-center items-center py-
12" }, /* @__PURE__ */ React.createElement(
    "div",
```

```
    {
      className: theme === "dark" ? "text-gray-400" : "text-gray-500"
    },
    "Loading employees..."
  )) : employees.length === 0 ? /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `flex flex-col items-center justify-center py-20 rounded-2xl shadow-inner border border-
dashed ${theme === "dark" ? "bg-[#2A2724] border-gray-600" : "bg-white border-gray-300"}`
    },
    /* @__PURE__ */ React.createElement(
      "p",
      {
        className: `text-2xl font-semibold mb-3 ${theme === "dark" ? "text-gray-200" : "text-gray-700"}`
      },
      "No accounts yet"
    ),
    /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => setShowAddModal(true),
        className: "px-6 py-3 text-white rounded-lg font-medium hover:opacity-90 transition-all shadow-md",
        style: {
          background: "linear-gradient(135deg, #10B981 0%, #059669 100%)"
        }
      },
      "+ Add Your First Employee"
    )
  ) : /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3
xl:grid-cols-4 gap-6 mb-4" }, paginatedEmployees.map(
    (employee) => /* @__PURE__ */ React.createElement(
      "div",
      {
        key: employee.id,
        className: `rounded-xl shadow-md p-4 flex gap-4 items-center relative transition-all ${theme ===
"dark" ? "bg-[#2A2724]" : "bg-white"} ${employee.status !== "Active" ? "opacity-50 grayscale" : ""}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "relative shrink-0" }, /* @__PURE__ */
React.createElement("div", { className: "w-20 h-20 rounded-full overflow-hidden bg-gray-200" }, /* @__PURE__
*/ React.createElement(
        "img",
        {
          src: employee.image,
          alt: employee.name,
          onError: (e) => {
            e.target.onerror = null;
            e.target.src = defaultAvatar;
          },
          className: "w-full h-full object-cover"
        }
      )), /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `absolute bottom-1 right-1 w-4 h-4 rounded-full border-2 border-white
${employee.status !== "Active" ? "bg-gray-400" : employee.role === "Owner" ? employee.isOnline ||
currentUser?.role === "Owner" && (employee.id === currentUser?._id || employee._id === currentUser?._id) ?
"bg-green-500" : "bg-red-500" : onlineEmployeeIds.has(employee.id) ? "bg-green-500" : "bg-red-500"}`
        }
      )),
      employee.role !== "Owner" && /* @__PURE__ */ React.createElement("div", { className: "absolute top-2
right-2 dropdown-container" }, /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: (e) => {
            e.stopPropagation();
            setOpenDropdown(
              openDropdown === employee.id ? null : employee.id
            );
          },
          className: `w-8 h-8 flex items-center justify-center rounded-lg transition-colors shadow-sm ${theme
=== "dark" ? "bg-[#3A3734] text-gray-300 hover:bg-[#4A4744]" : "bg-gray-100 text-gray-600 hover:bg-gray-
200"}`,
          title: "More Options"
        },
        /* @__PURE__ */ React.createElement(FaEllipsisV, { className: "w-3.5 h-3.5" })
      ), openDropdown === employee.id && /* @__PURE__ */ React.createElement(
        "div",
```

```
        {
          className: `absolute right-0 top-full mt-1 w-32 rounded-lg shadow-lg border z-20 ${theme ===
"dark" ? "bg-[#2A2724] border-gray-600" : "bg-white border-gray-200"}`
        },
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: (e) => {
              e.stopPropagation();
              handleResetPin(employee);
              setOpenDropdown(null);
            },
            className: `w-full text-left px-4 py-2 text-sm hover:bg-opacity-10 transition-colors ${theme ===
"dark" ? "text-gray-300 hover:bg-white" : "text-gray-700 hover:bg-black"}`
          },
          "Reset PIN"
        )
      )),
      /* @__PURE__ */ React.createElement("div", { className: "flex-1 min-w-0 flex flex-col justify-between
h-full gap-2" }, /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "h3",
        {
          className: `font-bold text-lg leading-tight truncate ${theme === "dark" ? "text-white" : "text-
gray-900"}`
        },
        employee.name
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        employee.role
      ), employee.contactNumber && /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs mt-1 flex items-center gap-1 ${theme === "dark" ? "text-gray-500" : "text-gray-
400"}`
        },
        "\u{1F4DE} ",
        employee.contactNumber
      ), (employee.dateJoinedActual || employee.dateJoined || employee.createdAt) && /* @__PURE__ */
React.createElement(
        "p",
        {
          className: `text-xs flex items-center gap-1 ${theme === "dark" ? "text-gray-500" : "text-gray-
400"}`
        },
        "\u{1F4C5}",
        " ",
        new Date(
          employee.dateJoinedActual || employee.dateJoined || employee.createdAt
        ).toLocaleDateString()
      )), employee.role !== "Owner" && /* @__PURE__ */ React.createElement("div", { className: "flex items-
center gap-2 mt-1 dropdown-container relative" }, /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => handleViewEmployee(employee),
          className: `px-3 py-1.5 rounded-lg text-xs font-bold transition-colors shadow-sm flex-1 text-center
bg-gray-200 text-gray-700 hover:bg-gray-300`
        },
        "View"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => handleEditEmployee(employee),
          className: "w-8 h-8 flex items-center justify-center rounded-lg bg-[#007AFF] text-white hover:bg-
blue-600 transition-colors shadow-sm",
          title: "Edit Details"
        },
        /* @__PURE__ */ React.createElement(FaEdit, { className: "w-3.5 h-3.5" })
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => handleToggleStatus(employee),
          className: `w-8 h-8 flex items-center justify-center rounded-lg text-white hover:opacity-90
transition-colors shadow-sm ${employee.status === "Active" ? "bg-[#FFA500]" : "bg-[#10B981]"}`,
          title: employee.status === "Active" ? "Disable Account" : "Enable Account"
```

```
        },
        employee.status === "Active" ? /* @__PURE__ */ React.createElement(FaTrash, { className: "w-3.5 h-
3.5" }) : /* @__PURE__ */ React.createElement(FaUndo, { className: "w-3.5 h-3.5" })
      )), employee.role === "Owner" && /* @__PURE__ */ React.createElement("div", { className: "mt-2" }, /*
@__PURE__ */ React.createElement("span", { className: "px-3 py-1 bg-purple-100 text-purple-700 rounded-full
text-xs font-bold" }, "Owner Access")))
    )
  )), totalPages > 1 && /* @__PURE__ */ React.createElement(
    Pagination,
    {
      currentPage,
      totalPages,
      onPageChange: setCurrentPage
    }
  )), /* @__PURE__ */ React.createElement(
    ViewEmployeeModal,
    {
      isOpen: showViewModal,
      onClose: () => {
        setShowViewModal(false);
        setViewingEmployee(null);
      },
      employee: viewingEmployee,
      onEdit: () => {
        setShowViewModal(false);
        handleEditEmployee(viewingEmployee);
      },
      onResetPin: () => {
        setShowViewModal(false);
        handleResetPin(viewingEmployee);
      },
      onToggleStatus: () => {
        setShowViewModal(false);
        handleToggleStatus(viewingEmployee);
      }
    }
  ), /* @__PURE__ */ React.createElement(
    EditEmployeeModal,
    {
      isOpen: showEditModal,
      onClose: () => {
        setShowEditModal(false);
        setEditingEmployee(null);
      },
      employee: editingEmployee,
      onEmployeeUpdated: () => fetchEmployees(false)
    }
  ), /* @__PURE__ */ React.createElement(
    DeleteEmployeeModal,
    {
      isOpen: showDeleteModal,
      onClose: () => {
        setShowDeleteModal(false);
        setDeletingEmployee(null);
      },
      onConfirm: confirmDeleteEmployee,
      employee: deletingEmployee
    }
  ), /* @__PURE__ */ React.createElement(
    AddEmployeeModal,
    {
      isOpen: showAddModal,
      onClose: () => {
        setShowAddModal(false);
      },
      onEmployeeAdded: () => fetchEmployees(false),
      onEmployeeCreated: (name, pin) => {
        setNewEmployeeName(name);
        setNewEmployeePin(pin);
        setShowTempPinModal(true);
      }
    }
  ), /* @__PURE__ */ React.createElement(
    DisableAccountModal,
    {
      isOpen: showDisableModal,
      onClose: () => {
```

```
        setShowDisableModal(false);
        setTogglingEmployee(null);
      },
      onConfirm: confirmToggleStatus,
      employee: togglingEmployee,
      action: toggleAction
    }
  ), /* @__PURE__ */ React.createElement(
    SuccessModal,
    {
      isOpen: showSuccessModal,
      onClose: () => setShowSuccessModal(false),
      message: successMessage
    }
  ), /* @__PURE__ */ React.createElement(
    ResetPinConfirmModal,
    {
      isOpen: showResetPinModal,
      onClose: () => {
        setShowResetPinModal(false);
        setResettingEmployee(null);
      },
      onConfirm: confirmResetPin,
      employeeName: resettingEmployee?.name || ""
    }
  ), /* @__PURE__ */ React.createElement(
    TemporaryPinModal,
    {
      isOpen: showTempPinModal,
      onClose: () => {
        setShowTempPinModal(false);
        setNewEmployeeName("");
        setNewEmployeePin("");
      },
      employeeName: newEmployeeName,
      temporaryPin: newEmployeePin
    }
  ));
};
export default memo(ManageEmployees);
```

FILE: pos/frontend/src/pages/owner/Reports.jsx
==========

```
import { format } from "date-fns";
import { useEffect, useState } from "react";
import DatePicker from "react-datepicker";
import "react-datepicker/dist/react-datepicker.css";
import {
  FaArrowDown,
  FaArrowUp,
  FaCalendarAlt,
  FaClipboardList,
  FaCubes,
  FaHandHoldingUsd,
  FaMoneyBillWave,
  FaShoppingBag,
  FaTruck
} from "react-icons/fa";
import {
  Bar,
  BarChart,
  CartesianGrid,
  Legend,
  ResponsiveContainer,
  Tooltip,
  XAxis,
  YAxis
} from "recharts";
import { utils, writeFile } from "xlsx";
import exportIcon from "../../assets/inventory-icons/export.svg";
import Header from "../../components/shared/header";
import { useTheme } from "../../context/ThemeContext";
import { API_ENDPOINTS } from "../../config/api";
const Reports = () => {
  const { theme } = useTheme();
```

```
const [activeTab, setActiveTab] = useState("inventory");
const [timePeriod, setTimePeriod] = useState("Day");
const [dateRange, setDateRange] = useState([null, null]);
const [startDate, endDate] = dateRange;
const [showDatePicker, setShowDatePicker] = useState(false);
const [inventoryMetrics, setInventoryMetrics] = useState({
  inventoryValue: 0,
  costOfGoodsSold: 0,
  grossProfitMargin: 0,
  totalItems: 0,
  inStock: 0,
  lowStock: 0,
  outOfStock: 0
});
const [lowStockItems, setLowStockItems] = useState([]);
const [stockMovements, setStockMovements] = useState([]);
const [lowStockFilter, setLowStockFilter] = useState("all");
const [stockInData, setStockInData] = useState([]);
const [damagedData, setDamagedData] = useState([]);
const [brandPartnersStats, setBrandPartnersStats] = useState([]);
const [inventoryAnalytics, setInventoryAnalytics] = useState({
  kpis: {
    totalSales: 0,
    totalTransactions: 0,
    totalUnitsSold: 0,
    cogs: 0,
    totalProfit: 0,
    profitMargin: 0,
    inventoryValue: 0,
    costValue: 0,
    totalStockUnits: 0
  },
  stats: {
    totalItems: 0,
    inStockCount: 0,
    lowStockCount: 0,
    outOfStockCount: 0,
    stockInCount: 0,
    stockOutCount: 0,
    pullOutCount: 0
  },
  inventoryChartData: [],
  profitChartData: [],
  damagedExpired: {
    items: [],
    summary: { damaged: 0, expired: 0, lost: 0, total: 0 }
  }
});
const [inventoryLoading, setInventoryLoading] = useState(false);
useEffect(() => {
  fetchInventoryMetrics();
  fetchInventoryAnalytics(timePeriod);
}, []);
useEffect(() => {
  if (timePeriod === "Custom" && (!startDate || !endDate)) {
    return;
  }
  fetchInventoryAnalytics(timePeriod);
}, [timePeriod, startDate, endDate]);
const formatCurrency = (amount) => {
  return `\u20B1${amount.toLocaleString("en-US", {
    minimumFractionDigits: 2,
    maximumFractionDigits: 2
  })}`;
};
const fetchInventoryMetrics = async () => {
  try {
    const response = await fetch(API_ENDPOINTS.products);
    const data = await response.json();
    if (data.success && Array.isArray(data.data)) {
      const products = data.data;
      const totalItems = products.length;
      const inStock = products.filter(
        (p) => (p.currentStock || 0) > (p.reorderNumber || 10)
      ).length;
      const lowStock = products.filter(
        (p) => (p.currentStock || 0) > 0 && (p.currentStock || 0) <= (p.reorderNumber || 10)
```

```
        ).length;
        const outOfStock = products.filter(
          (p) => (p.currentStock || 0) === 0
        ).length;
        const inventoryValue = products.reduce(
          (sum, p) => sum + (p.itemPrice || 0) * (p.currentStock || 0),
          0
        );
        const costOfGoodsSold = products.reduce(
          (sum, p) => sum + (p.costPrice || (p.itemPrice || 0) * 0.6) * (p.currentStock || 0),
          0
        );
        setInventoryMetrics({
          inventoryValue,
          costOfGoodsSold,
          grossProfitMargin: inventoryValue > 0 ? ((inventoryValue - costOfGoodsSold) / inventoryValue *
100).toFixed(0) : 0,
          totalItems,
          inStock,
          lowStock,
          outOfStock
        });
        const lowStockProducts = products.filter((p) => (p.currentStock || 0) <= (p.reorderNumber ||
10)).slice(0, 5).map((p) => ({
          name: p.itemName || p.name || "Unknown",
          stock: p.currentStock || 0,
          reorderLevel: p.reorderNumber || 10,
          status: (p.currentStock || 0) === 0 ? "Out of Stock" : "Low Stock"
        }));
        setLowStockItems(lowStockProducts);
      }
      const movementsResponse = await fetch(
        `${API_ENDPOINTS.stockMovements}?limit=50`
      );
      const movementsData = await movementsResponse.json();
      if (movementsData.success && Array.isArray(movementsData.data)) {
        setStockMovements(movementsData.data.slice(0, 5));
        const movements = movementsData.data;
        const stockInByWeek = {};
        const damagedByWeek = {};
        movements.forEach((m) => {
          const date = new Date(m.createdAt);
          const weekNum = Math.ceil(date.getDate() / 7);
          const weekKey = `Week ${weekNum}`;
          if (m.type === "Stock-In" || m.reason === "Restock") {
            stockInByWeek[weekKey] = (stockInByWeek[weekKey] || 0) + Math.abs(m.quantity);
          }
          if (m.type === "Pull-Out" || m.reason === "Damaged") {
            damagedByWeek[weekKey] = (damagedByWeek[weekKey] || 0) + Math.abs(m.quantity);
          }
        });
        setStockInData(
          Object.entries(stockInByWeek).map(([name, value]) => ({ name, value })).slice(0, 4)
        );
        setDamagedData(
          Object.entries(damagedByWeek).map(([name, value]) => ({ name, value })).slice(0, 4)
        );
      }
      const brandStats = {};
      if (data.success && Array.isArray(data.data)) {
        data.data.forEach((product) => {
          const brand = product.brandName || "Unknown";
          if (!brandStats[brand]) {
            brandStats[brand] = { name: brand, skuCount: 0, sales: 0 };
          }
          brandStats[brand].skuCount += 1;
          brandStats[brand].sales += (product.itemPrice || 0) * (product.totalSold || 0);
        });
        setBrandPartnersStats(Object.values(brandStats).slice(0, 4));
      }
    } catch (error) {
      console.error("Error fetching inventory metrics:", error);
    }
  };
  const fetchInventoryAnalytics = async (period = "Day", retryCount = 0) => {
    try {
      setInventoryLoading(true);
```

```
    const timeMap = {
      Day: "daily",
      Week: "weekly",
      Month: "monthly",
      Year: "yearly",
      Custom: "custom"
    };
    const apiTimeframe = timeMap[period] || "daily";
    let url = `${API_ENDPOINTS.reportsInventoryAnalytics}?timeframe=${apiTimeframe}`;
    if (period === "Custom" && startDate && endDate) {
      url += `&startDate=${startDate.toISOString()}&endDate=${endDate.toISOString()}`;
    }
    const controller = new AbortController();
    const timeoutMs = retryCount === 0 ? 25e3 : 3e4;
    const timeoutId = setTimeout(() => controller.abort(), timeoutMs);
    const response = await fetch(url, { signal: controller.signal });
    clearTimeout(timeoutId);
    const data = await response.json();
    if (data.success) {
      setInventoryAnalytics(data.data);
    }
  } catch (error) {
    console.error("Error fetching inventory analytics:", error);
    if (retryCount < 1 && (error.name === "AbortError" || error.message?.includes("fetch"))) {
      console.log("Retrying inventory analytics fetch...");
      return fetchInventoryAnalytics(period, retryCount + 1);
    }
  } finally {
    setInventoryLoading(false);
  }
};
const handleExport = () => {
  const wb = utils.book_new();
  const invData = [
    ["Metric", "Value"],
    ["Total Sales", inventoryAnalytics.kpis.totalSales],
    ["COGS", inventoryAnalytics.kpis.cogs],
    ["Total Profit", inventoryAnalytics.kpis.totalProfit],
    ["Inventory Value", inventoryAnalytics.kpis.inventoryValue],
    ["Total Items", inventoryAnalytics.stats.totalItems],
    ["Stock-Ins", inventoryAnalytics.stats.stockInCount],
    ["Stock-Outs", inventoryAnalytics.stats.stockOutCount],
    ["Pull-Outs", inventoryAnalytics.stats.pullOutCount]
  ];
  const invWs = utils.aoa_to_sheet(invData);
  utils.book_append_sheet(wb, invWs, "Inventory Analytics");
  if (inventoryAnalytics.profitChartData.length > 0) {
    const profitWs = utils.json_to_sheet(
      inventoryAnalytics.profitChartData.map((item) => ({
        Period: item.period,
        Revenue: item.revenue,
        COGS: item.cogs,
        Profit: item.profit
      }))
    );
    utils.book_append_sheet(wb, profitWs, "Profit Analysis");
  }
  if (inventoryAnalytics.inventoryChartData.length > 0) {
    const moveChartWs = utils.json_to_sheet(
      inventoryAnalytics.inventoryChartData.map((item) => ({
        Period: item.period,
        "Stock-In": item.stockIn,
        "Stock-Out": item.stockOut,
        "Pull-Out": item.pullOut
      }))
    );
    utils.book_append_sheet(wb, moveChartWs, "Inventory Movements");
  }
  if (inventoryAnalytics.damagedExpired.items.length > 0) {
    const damagedWs = utils.json_to_sheet(
      inventoryAnalytics.damagedExpired.items.map((item) => ({
        "Item Name": item.itemName,
        SKU: item.sku,
        Category: item.category,
        Type: item.type,
        Quantity: item.quantity,
        Date: new Date(item.date).toLocaleDateString(),
```

```
          "Handled By": item.handledBy,
          Notes: item.notes
        }))
      );
      utils.book_append_sheet(wb, damagedWs, "Damaged & Expired");
    }
    const dateStr = (/* @__PURE__ */ new Date()).toISOString().split("T")[0];
    writeFile(wb, `Reports_Export_${dateStr}.xlsx`);
  };
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `p-8 min-h-screen ${theme === "dark" ? "bg-[#1E1B18]" : "bg-gray-50"}`
    },
    /* @__PURE__ */ React.createElement(Header, { pageName: "Analytics", profileBackground: "", showBorder:
false }),
    /* @__PURE__ */ React.createElement("div", { className: "flex items-start gap-3 mb-6 mt-6 w-full" }, /*
@__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__ */
React.createElement(
      "div",
      {
        className: `rounded-lg shadow-sm p-1 flex space-x-1 ${theme === "dark" ? "bg-[#2A2724]" : "bg-
white"}`
      },
      ["Day", "Week", "Month", "Year"].map(
        (t) => /* @__PURE__ */ React.createElement(
          "button",
          {
            key: t,
            onClick: () => {
              setTimePeriod(t);
              setDateRange([null, null]);
            },
            className: `px-4 py-1.5 text-sm font-medium rounded-md transition-colors ${timePeriod === t ?
"bg-[#AD7F65] text-white shadow-sm" : theme === "dark" ? "text-gray-400 hover:bg-[#3A3734]" : "text-gray-500
hover:bg-gray-100"}`
          },
          t
        )
      ),
      /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => {
            if (timePeriod !== "Custom") {
              setShowDatePicker(true);
            }
          },
          className: `px-4 py-1.5 text-sm font-medium rounded-md transition-colors ${timePeriod === "Custom" ?
"bg-[#AD7F65] text-white shadow-sm" : theme === "dark" ? "text-gray-400 hover:bg-[#3A3734] hidden" : "text-
gray-500 hover:bg-gray-100 hidden"}`
        },
        "Custom"
      )
    ), /* @__PURE__ */ React.createElement("div", { className: "relative z-50" }, /* @__PURE__ */
React.createElement(
      DatePicker,
      {
        selected: startDate,
        onChange: (update) => {
          setDateRange(update);
          if (update[0]) {
            setTimePeriod("Custom");
          }
        },
        startDate,
        endDate,
        selectsRange: true,
        popperClassName: "z-50",
        customInput: /* @__PURE__ */ React.createElement(
          "button",
          {
            className: `flex items-center gap-2 px-3 py-1.5 rounded-lg shadow-sm border transition-colors
${theme === "dark" ? "bg-[#2A2724] border-gray-600 hover:bg-[#3A3734]" : "bg-white border-gray-100 hover:bg-
gray-50"}`
          },
          /* @__PURE__ */ React.createElement(
```

```
            FaCalendarAlt,
            {
              className: `${timePeriod === "Custom" ? "text-[#AD7F65]" : theme === "dark" ? "text-gray-400" :
"text-gray-400"}`
            }
          ),
          /* @__PURE__ */ React.createElement(
            "span",
            {
              className: `text-sm font-medium ${timePeriod === "Custom" ? "text-[#AD7F65]" : theme ===
"dark" ? "text-gray-300" : "text-gray-600"}`
            },
            (() => {
              if (timePeriod === "Custom" && startDate && endDate) {
                return `${format(startDate, "MMM d, yyyy")} - ${format(endDate, "MMM d, yyyy")}`;
              }
              const now = /* @__PURE__ */ new Date();
              const options = {
                day: "numeric",
                month: "short",
                year: "numeric"
              };
              if (timePeriod === "Day") {
                return now.toLocaleDateString("en-US", options);
              } else if (timePeriod === "Week") {
                const start = new Date(now);
                start.setDate(now.getDate() - 7);
                return `${start.toLocaleDateString("en-US", { day: "numeric", month: "short" })} -
${now.toLocaleDateString("en-US", options)}`;
              } else if (timePeriod === "Month") {
                return `${now.toLocaleDateString("en-US", { month: "short", year: "numeric" })}`;
              } else if (timePeriod === "Year") {
                return `${now.getFullYear()}`;
              }
              return now.toLocaleDateString("en-US", options);
            })()
          )
        )
      }
    ))), /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3 ml-auto" }, /*
@__PURE__ */ React.createElement(
      "button",
      {
        onClick: handleExport,
        className: `w-10 h-10 rounded-lg flex items-center justify-center border shadow-sm hover:shadow-md
transition-shadow ${theme === "dark" ? "bg-[#2A2724] border-gray-700 hover:bg-[#3A3734]" : "bg-white border-
gray-200"}`,
        title: "Export to Excel"
      },
      /* @__PURE__ */ React.createElement(
        "img",
        {
          src: exportIcon,
          alt: "Export",
          className: "w-5 h-5 object-contain"
        }
      )
    ))),
    /* @__PURE__ */ React.createElement("div", { className: "mt-6" }, activeTab === "inventory" && /*
@__PURE__ */ React.createElement("div", { className: "space-y-6" }, /* @__PURE__ */ React.createElement("div",
{ className: "grid grid-cols-4 gap-4" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-xl shadow-md px-5 py-4 relative overflow-hidden ${theme === "dark" ? "bg-
[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "absolute left-0 top-0 bottom-0 w-2 rounded-l-xl",
          style: {
            background: "linear-gradient(to bottom, #60A5FA, #3B82F6, #1D4ED8)"
          }
        }
      ),
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between pl-2" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
```

```
        "p",
        {
          className: `text-xs font-medium mb-1 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Total Sales"
      ), /* @__PURE__ */ React.createElement("p", { className: "text-2xl font-bold text-blue-500" },
inventoryLoading ? "..." : formatCurrency(inventoryAnalytics.kpis.totalSales)), /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-3 mt-1.5" }, /* @__PURE__ */
React.createElement(
        "span",
        {
          className: `text-[11px] ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        /* @__PURE__ */ React.createElement("span", { className: "font-semibold text-blue-400" },
inventoryAnalytics.kpis.totalTransactions),
        " ",
        "transactions"
      ), /* @__PURE__ */ React.createElement(
        "span",
        {
          className: `text-[11px] ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        /* @__PURE__ */ React.createElement("span", { className: "font-semibold text-blue-400" },
inventoryAnalytics.kpis.totalUnitsSold),
        " ",
        "units sold"
      ))), /* @__PURE__ */ React.createElement("div", { className: "bg-blue-100 rounded-full p-3" }, /*
@__PURE__ */ React.createElement(FaShoppingBag, { className: "text-blue-500 w-5 h-5" })))
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-xl shadow-md px-5 py-4 relative overflow-hidden ${theme === "dark" ? "bg-
[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "absolute left-0 top-0 bottom-0 w-2 rounded-l-xl",
          style: {
            background: "linear-gradient(to bottom, #F87171, #EF4444, #DC2626)"
          }
        }
      ),
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between pl-2" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs font-medium mb-1 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Cost of Goods Sold"
      ), /* @__PURE__ */ React.createElement("p", { className: "text-2xl font-bold text-red-500" },
inventoryLoading ? "..." : formatCurrency(inventoryAnalytics.kpis.cogs)), /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-3 mt-1.5" }, /* @__PURE__ */
React.createElement(
        "span",
        {
          className: `text-[11px] ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        /* @__PURE__ */ React.createElement("span", { className: "font-semibold text-red-400" },
inventoryAnalytics.kpis.totalUnitsSold),
        " ",
        "units"
      ), /* @__PURE__ */ React.createElement(
        "span",
        {
          className: `text-[11px] ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Avg:",
        " ",
        /* @__PURE__ */ React.createElement("span", { className: "font-semibold text-red-400" },
inventoryAnalytics.kpis.totalUnitsSold > 0 ? formatCurrency(
          inventoryAnalytics.kpis.cogs / inventoryAnalytics.kpis.totalUnitsSold
        ) : "\u20B10.00"),
        "/unit"
      ))), /* @__PURE__ */ React.createElement("div", { className: "bg-red-100 rounded-full p-3" }, /*
@__PURE__ */ React.createElement(FaHandHoldingUsd, { className: "text-red-500 w-5 h-5" })))
    ), /* @__PURE__ */ React.createElement(
```

```
      "div",
      {
        className: `rounded-xl shadow-md px-5 py-4 relative overflow-hidden ${theme === "dark" ? "bg-
[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "absolute left-0 top-0 bottom-0 w-2 rounded-l-xl",
          style: {
            background: "linear-gradient(to bottom, #34D399, #10B981, #059669)"
          }
        }
      ),
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between pl-2" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs font-medium mb-1 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Total Profit"
      ), /* @__PURE__ */ React.createElement("p", { className: "text-2xl font-bold text-green-500" },
inventoryLoading ? "..." : formatCurrency(inventoryAnalytics.kpis.totalProfit)), /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-3 mt-1.5" }, /* @__PURE__ */
React.createElement(
        "span",
        {
          className: `text-[11px] px-1.5 py-0.5 rounded ${inventoryAnalytics.kpis.profitMargin >= 0 ? "bg-
green-100 text-green-600" : "bg-red-100 text-red-600"}`
        },
        inventoryAnalytics.kpis.profitMargin >= 0 ? "\u25B2" : "\u25BC",
        " ",
        Math.abs(inventoryAnalytics.kpis.profitMargin).toFixed(
          1
        ),
        "% margin"
      ), /* @__PURE__ */ React.createElement(
        "span",
        {
          className: `text-[11px] ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Revenue \u2212 COGS"
      ))), /* @__PURE__ */ React.createElement("div", { className: "bg-green-100 rounded-full p-3" }, /*
@__PURE__ */ React.createElement(FaMoneyBillWave, { className: "text-green-500 w-5 h-5" })))
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-xl shadow-md px-5 py-4 relative overflow-hidden ${theme === "dark" ? "bg-
[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "absolute left-0 top-0 bottom-0 w-2 rounded-l-xl",
          style: {
            background: "linear-gradient(to bottom, #A78BFA, #8B5CF6, #7C3AED)"
          }
        }
      ),
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between pl-2" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs font-medium mb-1 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Inventory Value"
      ), /* @__PURE__ */ React.createElement("p", { className: "text-2xl font-bold text-purple-500" },
inventoryLoading ? "..." : formatCurrency(
        inventoryAnalytics.kpis.inventoryValue
      )), /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3 mt-1.5" }, /*
@__PURE__ */ React.createElement(
        "span",
        {
          className: `text-[11px] ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        /* @__PURE__ */ React.createElement("span", { className: "font-semibold text-purple-400" },
inventoryAnalytics.kpis.totalStockUnits.toLocaleString()),
```

```
        " ",
        "units in stock"
      ), /* @__PURE__ */ React.createElement(
        "span",
        {
          className: `text-[11px] ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Cost:",
        " ",
        /* @__PURE__ */ React.createElement("span", { className: "font-semibold text-purple-400" },
formatCurrency(inventoryAnalytics.kpis.costValue))
      ))), /* @__PURE__ */ React.createElement("div", { className: "bg-purple-100 rounded-full p-3" }, /*
@__PURE__ */ React.createElement(FaCubes, { className: "text-purple-500 w-5 h-5" })))
    )), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-4 gap-4" }, /* @__PURE__ */
React.createElement("div", { className: "flex flex-col gap-3" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-xl shadow-sm relative overflow-hidden p-4 ${theme === "dark" ? "bg-[#2A2724]" :
"bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "absolute left-0 top-0 bottom-0 w-1.5 bg-blue-
500" }),
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between pl-2" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-3xl
font-bold text-blue-500" }, inventoryAnalytics.stats.totalItems), /* @__PURE__ */ React.createElement("p",
{ className: "text-xs text-blue-500 font-medium mt-0.5" }, "Total Items")), /* @__PURE__ */
React.createElement("div", { className: "bg-blue-50 rounded-full p-3" }, /* @__PURE__ */
React.createElement(FaClipboardList, { className: "text-blue-400 w-5 h-5" })))
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-xl shadow-sm relative overflow-hidden p-4 ${theme === "dark" ? "bg-[#2A2724]" :
"bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "absolute left-0 top-0 bottom-0 w-1.5 bg-green-
500" }),
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between pl-2" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-3xl
font-bold text-green-500" }, inventoryAnalytics.stats.stockInCount), /* @__PURE__ */ React.createElement("p",
{ className: "text-xs text-green-500 font-medium mt-0.5" }, "Stock-ins")), /* @__PURE__ */
React.createElement("div", { className: "bg-green-50 rounded-full p-3" }, /* @__PURE__ */
React.createElement(FaArrowDown, { className: "text-green-400 w-5 h-5" })))
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-xl shadow-sm relative overflow-hidden p-4 ${theme === "dark" ? "bg-[#2A2724]" :
"bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "absolute left-0 top-0 bottom-0 w-1.5 bg-amber-
500" }),
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between pl-2" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-3xl
font-bold text-amber-500" }, inventoryAnalytics.stats.stockOutCount), /* @__PURE__ */ React.createElement("p",
{ className: "text-xs text-amber-500 font-medium mt-0.5" }, "Stock-outs")), /* @__PURE__ */
React.createElement("div", { className: "bg-amber-50 rounded-full p-3" }, /* @__PURE__ */
React.createElement(FaArrowUp, { className: "text-amber-400 w-5 h-5" })))
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-xl shadow-sm relative overflow-hidden p-4 ${theme === "dark" ? "bg-[#2A2724]" :
"bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "absolute left-0 top-0 bottom-0 w-1.5 bg-red-
500" }),
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between pl-2" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className: "text-3xl
font-bold text-red-500" }, inventoryAnalytics.stats.pullOutCount), /* @__PURE__ */ React.createElement("p",
{ className: "text-xs text-red-500 font-medium mt-0.5" }, "Pull-Outs")), /* @__PURE__ */
React.createElement("div", { className: "bg-red-50 rounded-full p-3" }, /* @__PURE__ */
React.createElement(FaTruck, { className: "text-red-400 w-5 h-5" })))
    )), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `col-span-3 rounded-2xl shadow-md p-5 flex flex-col ${theme === "dark" ? "bg-[#2A2724]" :
"bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "mb-3" }, /* @__PURE__ */ React.createElement(
```

```
        "h3",
        {
          className: `text-base font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Inventory Analysis"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-[11px] ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Stock-In vs Stock-Out vs Pull-Out movement comparison"
      )),
      /* @__PURE__ */ React.createElement("div", { className: "flex-1 min-h-0" },
inventoryAnalytics.inventoryChartData.length === 0 ? /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `w-full h-full flex items-center justify-center rounded-lg ${theme === "dark" ? "bg-
[#1E1B18]" : "bg-gray-50"}`
        },
        /* @__PURE__ */ React.createElement("p", { className: "text-gray-400 text-sm" }, "No movement data
available")
      ) : /* @__PURE__ */ React.createElement(ResponsiveContainer, { width: "100%", height: "100%" }, /*
@__PURE__ */ React.createElement(
        BarChart,
        {
          data: inventoryAnalytics.inventoryChartData,
          barCategoryGap: "20%"
        },
        /* @__PURE__ */ React.createElement(
          CartesianGrid,
          {
            strokeDasharray: "3 3",
            vertical: false,
            stroke: theme === "dark" ? "#374151" : "#f0f0f0"
          }
        ),
        /* @__PURE__ */ React.createElement(
          XAxis,
          {
            dataKey: "period",
            axisLine: false,
            tickLine: false,
            tick: {
              fontSize: 11,
              fill: theme === "dark" ? "#9CA3AF" : "#6b7280"
            }
          }
        ),
        /* @__PURE__ */ React.createElement(
          YAxis,
          {
            axisLine: false,
            tickLine: false,
            tick: {
              fontSize: 11,
              fill: theme === "dark" ? "#9CA3AF" : "#6b7280"
            }
          }
        ),
        /* @__PURE__ */ React.createElement(
          Tooltip,
          {
            contentStyle: {
              backgroundColor: theme === "dark" ? "#1F2937" : "white",
              border: theme === "dark" ? "1px solid #374151" : "none",
              borderRadius: "12px",
              boxShadow: "0 4px 12px rgb(0 0 0 / 0.1)",
              color: theme === "dark" ? "#F3F4F6" : "#000"
            }
          }
        ),
        /* @__PURE__ */ React.createElement(
          Legend,
          {
            iconType: "circle",
            wrapperStyle: {
              paddingTop: "10px",
```

```
              fontSize: "11px"
            }
          }
        ),
        /* @__PURE__ */ React.createElement(
          Bar,
          {
            dataKey: "stockIn",
            name: "Stock-In",
            fill: "#10B981",
            radius: [4, 4, 0, 0],
            barSize: 18
          }
        ),
        /* @__PURE__ */ React.createElement(
          Bar,
          {
            dataKey: "stockOut",
            name: "Stock-Out",
            fill: "#F59E0B",
            radius: [4, 4, 0, 0],
            barSize: 18
          }
        ),
        /* @__PURE__ */ React.createElement(
          Bar,
          {
            dataKey: "pullOut",
            name: "Pull-Out",
            fill: "#EF4444",
            radius: [4, 4, 0, 0],
            barSize: 18
          }
        )
      )))
    )), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-4 gap-4" }, /* @__PURE__ */
React.createElement(
      "div",
      {
        className: `col-start-2 col-span-3 rounded-2xl shadow-md p-5 ${theme === "dark" ? "bg-[#2A2724]" :
"bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "mb-3" }, /* @__PURE__ */ React.createElement(
        "h3",
        {
          className: `text-base font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Profit Analysis"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-[11px] ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Revenue, Cost of Goods Sold, and Profit breakdown over time"
      )),
      /* @__PURE__ */ React.createElement("div", { className: "h-[260px]" },
inventoryAnalytics.profitChartData.length === 0 ? /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `w-full h-full flex items-center justify-center rounded-lg ${theme === "dark" ? "bg-
[#1E1B18]" : "bg-gray-50"}`
        },
        /* @__PURE__ */ React.createElement("p", { className: "text-gray-400 text-sm" }, "No profit data
available")
      ) : /* @__PURE__ */ React.createElement(ResponsiveContainer, { width: "100%", height: "100%" }, /*
@__PURE__ */ React.createElement(
        BarChart,
        {
          data: inventoryAnalytics.profitChartData,
          barCategoryGap: "15%"
        },
        /* @__PURE__ */ React.createElement(
          CartesianGrid,
          {
            strokeDasharray: "3 3",
            vertical: false,
            stroke: theme === "dark" ? "#374151" : "#f0f0f0"
```

```
          }
        ),
        /* @__PURE__ */ React.createElement(
          XAxis,
          {
            dataKey: "period",
            axisLine: false,
            tickLine: false,
            tick: {
              fontSize: 11,
              fill: theme === "dark" ? "#9CA3AF" : "#6b7280"
            }
          }
        ),
        /* @__PURE__ */ React.createElement(
          YAxis,
          {
            axisLine: false,
            tickLine: false,
            tick: {
              fontSize: 11,
              fill: theme === "dark" ? "#9CA3AF" : "#6b7280"
            },
            tickFormatter: (val) => `\u20B1${val >= 1e3 ? (val / 1e3).toFixed(0) + "k" : val}`
          }
        ),
        /* @__PURE__ */ React.createElement(
          Tooltip,
          {
            contentStyle: {
              backgroundColor: theme === "dark" ? "#1F2937" : "white",
              border: theme === "dark" ? "1px solid #374151" : "none",
              borderRadius: "12px",
              boxShadow: "0 4px 12px rgb(0 0 0 / 0.1)",
              color: theme === "dark" ? "#F3F4F6" : "#000"
            },
            formatter: (value) => [
              `\u20B1${value.toLocaleString("en-US", { minimumFractionDigits: 2, maximumFractionDigits:
2 })}`,
              ""
            ]
          }
        ),
        /* @__PURE__ */ React.createElement(
          Legend,
          {
            iconType: "circle",
            wrapperStyle: {
              paddingTop: "10px",
              fontSize: "11px"
            }
          }
        ),
        /* @__PURE__ */ React.createElement(
          Bar,
          {
            dataKey: "revenue",
            name: "Revenue",
            fill: "#3B82F6",
            radius: [4, 4, 0, 0],
            barSize: 24
          }
        ),
        /* @__PURE__ */ React.createElement(
          Bar,
          {
            dataKey: "cogs",
            name: "COGS",
            fill: "#EF4444",
            radius: [4, 4, 0, 0],
            barSize: 24
          }
        ),
        /* @__PURE__ */ React.createElement(
          Bar,
          {
            dataKey: "profit",
```

```
            name: "Profit",
            fill: "#10B981",
            radius: [4, 4, 0, 0],
            barSize: 24
          }
        )
      )))
    )), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-4 gap-4" }, /* @__PURE__ */
React.createElement(
      "div",
      {
        className: `col-start-2 col-span-3 rounded-2xl shadow-md p-5 ${theme === "dark" ? "bg-[#2A2724]" :
"bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between mb-4" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "h3",
        {
          className: `text-base font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Damaged & Expired Stock"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-[11px] ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Items pulled out due to damage, expiration, or loss"
      )), /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__
*/ React.createElement(
        "div",
        {
          className: `flex items-center gap-1.5 px-3 py-1.5 rounded-lg text-xs font-medium ${theme ===
"dark" ? "bg-[#1E1B18]" : "bg-red-50"}`
        },
        /* @__PURE__ */ React.createElement("span", { className: "w-2 h-2 rounded-full bg-red-500" }),
        /* @__PURE__ */ React.createElement(
          "span",
          {
            className: `${theme === "dark" ? "text-gray-300" : "text-red-700"}`
          },
          "Damaged:",
          " ",
          inventoryAnalytics.damagedExpired.summary.damaged
        )
      ), /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `flex items-center gap-1.5 px-3 py-1.5 rounded-lg text-xs font-medium ${theme ===
"dark" ? "bg-[#1E1B18]" : "bg-amber-50"}`
        },
        /* @__PURE__ */ React.createElement("span", { className: "w-2 h-2 rounded-full bg-amber-500" }),
        /* @__PURE__ */ React.createElement(
          "span",
          {
            className: `${theme === "dark" ? "text-gray-300" : "text-amber-700"}`
          },
          "Expired:",
          " ",
          inventoryAnalytics.damagedExpired.summary.expired
        )
      ), /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `flex items-center gap-1.5 px-3 py-1.5 rounded-lg text-xs font-medium ${theme ===
"dark" ? "bg-[#1E1B18]" : "bg-gray-50"}`
        },
        /* @__PURE__ */ React.createElement("span", { className: "w-2 h-2 rounded-full bg-gray-500" }),
        /* @__PURE__ */ React.createElement(
          "span",
          {
            className: `${theme === "dark" ? "text-gray-300" : "text-gray-700"}`
          },
          "Lost: ",
          inventoryAnalytics.damagedExpired.summary.lost
        )
      ))),
      /* @__PURE__ */ React.createElement("div", { className: "overflow-x-auto" }, /* @__PURE__ */
```

```
React.createElement("table", { className: "w-full text-xs" }, /* @__PURE__ */ React.createElement("thead",
null, /* @__PURE__ */ React.createElement(
        "tr",
        {
          className: `border-b ${theme === "dark" ? "text-gray-400 border-gray-700" : "text-gray-500 border-
gray-200"}`
        },
        /* @__PURE__ */ React.createElement("th", { className: "text-left py-2.5 font-medium" }, "Item Name"),
        /* @__PURE__ */ React.createElement("th", { className: "text-left py-2.5 font-medium" }, "SKU"),
        /* @__PURE__ */ React.createElement("th", { className: "text-left py-2.5 font-medium" }, "Category"),
        /* @__PURE__ */ React.createElement("th", { className: "text-center py-2.5 font-medium" }, "Type"),
        /* @__PURE__ */ React.createElement("th", { className: "text-center py-2.5 font-medium" }, "Qty"),
        /* @__PURE__ */ React.createElement("th", { className: "text-center py-2.5 font-medium" }, "Date"),
        /* @__PURE__ */ React.createElement("th", { className: "text-left py-2.5 font-medium" }, "Handled
By"),
        /* @__PURE__ */ React.createElement("th", { className: "text-left py-2.5 font-medium" }, "Notes")
      )), /* @__PURE__ */ React.createElement("tbody", null, inventoryAnalytics.damagedExpired.items.length
=== 0 ? /* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */ React.createElement(
        "td",
        {
          colSpan: "8",
          className: "text-center py-8 text-gray-400"
        },
        "No damaged, expired, or lost items for this period"
      )) : inventoryAnalytics.damagedExpired.items.map(
        (item, index) => /* @__PURE__ */ React.createElement(
          "tr",
          {
            key: index,
            className: `${theme === "dark" ? "border-gray-700/50 hover:bg-[#3A3734]" : "border-gray-50
hover:bg-gray-50"} ${index < inventoryAnalytics.damagedExpired.items.length - 1 ? "border-b" : ""}`
          },
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `py-2.5 font-medium ${theme === "dark" ? "text-gray-200" : "text-gray-700"}`
            },
            item.itemName
          ),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `py-2.5 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
            },
            item.sku
          ),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `py-2.5 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
            },
            item.category
          ),
          /* @__PURE__ */ React.createElement("td", { className: "py-2.5 text-center" }, /* @__PURE__ */
React.createElement(
            "span",
            {
              className: `px-2 py-1 rounded-full text-[10px] font-medium ${item.type === "Damaged" ? "bg-red-
100 text-red-600" : item.type === "Expired" ? "bg-amber-100 text-amber-600" : "bg-gray-100 text-gray-600"}`
            },
            item.type
          )),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `py-2.5 text-center font-medium ${theme === "dark" ? "text-gray-200" : "text-gray-
700"}`
            },
            item.quantity
          ),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `py-2.5 text-center ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
            },
            new Date(item.date).toLocaleDateString()
          ),
```

```
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `py-2.5 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
            },
            item.handledBy
          ),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `py-2.5 ${theme === "dark" ? "text-gray-400" : "text-gray-500"} max-w-[150px]
truncate`
            },
            item.notes || "-"
          )
        )
      ))))
    ))))
  );
};
export default Reports;
```

FILE: pos/frontend/src/pages/owner/test.jsx
==========

```
import { Text, View } from "react-native";
import React, { Component } from "react";
export default class test extends Component {
  render() {
    return /* @__PURE__ */ React.createElement(View, null, /* @__PURE__ */ React.createElement(Text, null,
"test"));
  }
}
```

FILE: pos/frontend/src/pages/OwnerOnboarding.jsx
==========

```
import { memo, useRef, useState } from "react";
import { FaCheck, FaSpinner } from "react-icons/fa";
import { useNavigate } from "react-router-dom";
import bgImage from "../assets/bg.png";
import logo from "../assets/logo.png";
import { API_BASE_URL } from "../config/api";
import { useAuth } from "../context/AuthContext";
import { validatePinSecurity } from "../utils/pinValidation";
const PIN_LENGTH = 6;
const EMPTY_PIN = Array(PIN_LENGTH).fill("");
const OwnerOnboarding = ({ onSetupComplete }) => {
  const { login } = useAuth();
  const navigate = useNavigate();
  const [formValues, setFormValues] = useState(() => {
    const saved = localStorage.getItem("ownerOnboardingForm");
    if (saved) {
      try {
        return JSON.parse(saved);
      } catch {
        return {
          firstName: "",
          lastName: "",
          email: "",
          contactNo: ""
        };
      }
    }
    return {
      firstName: "",
      lastName: "",
      email: "",
      contactNo: ""
    };
  });
  const [pinDigits, setPinDigits] = useState(() => {
    const saved = localStorage.getItem("ownerOnboardingPin");
    if (saved) {
      try {
```

```
      return JSON.parse(saved);
    } catch {
      return EMPTY_PIN;
    }
  }
  return EMPTY_PIN;
});
const [confirmPinDigits, setConfirmPinDigits] = useState(() => {
  const saved = localStorage.getItem("ownerOnboardingConfirmPin");
  if (saved) {
    try {
      return JSON.parse(saved);
    } catch {
      return EMPTY_PIN;
    }
  }
  return EMPTY_PIN;
});
const [loading, setLoading] = useState(false);
const [error, setError] = useState("");
const [fieldErrors, setFieldErrors] = useState({
  firstName: "",
  lastName: "",
  email: "",
  contactNo: "",
  pin: "",
  confirmPin: ""
});
const [touched, setTouched] = useState({
  firstName: false,
  lastName: false,
  email: false,
  contactNo: false,
  pin: false,
  confirmPin: false,
  verificationCode: false
});
const [verificationCode, setVerificationCode] = useState("");
const [isCodeSent, setIsCodeSent] = useState(false);
const [isEmailVerified, setIsEmailVerified] = useState(false);
const [verificationTimer, setVerificationTimer] = useState(0);
const [showCodeSentCheck, setShowCodeSentCheck] = useState(false);
const [showPinModal, setShowPinModal] = useState(false);
const pinRefs = useRef([]);
const confirmPinRefs = useRef([]);
const handleInputChange = (field, value) => {
  setFormValues((prev) => {
    const updated = {
      ...prev,
      [field]: value
    };
    localStorage.setItem("ownerOnboardingForm", JSON.stringify(updated));
    return updated;
  });
  if (touched[field]) {
    validateField(field, value);
  }
};
const handleBlur = (field) => {
  setTouched((prev) => ({ ...prev, [field]: true }));
  if (field === "contactNo") {
    validateField(field, formValues[field]);
  } else if (field === "email") {
    validateField(field, formValues[field]);
    if (isEmailVerified) {
      setIsEmailVerified(false);
      setIsCodeSent(false);
      setVerificationCode("");
    }
  } else if (field === "firstName" || field === "lastName") {
    validateField(field, formValues[field]);
  }
};
const validateField = (field, value) => {
  let errorMsg = "";
  switch (field) {
    case "firstName":
```

```
        if (!value.trim()) {
          errorMsg = "First name is required";
        }
        break;
      case "lastName":
        if (!value.trim()) {
          errorMsg = "Last name is required";
        }
        break;
      case "email":
        if (!value.trim()) {
          errorMsg = "Email is required";
        } else if (!value.includes("@")) {
          errorMsg = "Please enter a valid email address";
        }
        break;
      case "contactNo":
        if (!value.trim()) {
          errorMsg = "Contact number is required";
        } else if (!validatePhilippineNumber(value)) {
          errorMsg = "Invalid format. Use 09XXXXXXXXX or +639XXXXXXXXX";
        }
        break;
      case "pin":
        const pin = typeof value === "string" ? value : pinDigits.join("");
        if (pin.length > 0 && pin.length < PIN_LENGTH) {
          errorMsg = "PIN must be exactly 6 digits";
        } else if (pin.length === PIN_LENGTH) {
          const pinValidation = validatePinSecurity(pin);
          if (!pinValidation.isValid) {
            errorMsg = pinValidation.error;
          } else {
            errorMsg = "";
          }
        }
        break;
      case "confirmPin":
        const confirmPin = typeof value === "string" ? value : confirmPinDigits.join("");
        const mainPin = pinDigits.join("");
        if (confirmPin.length > 0 && confirmPin.length < PIN_LENGTH) {
          errorMsg = "PIN must be exactly 6 digits";
        } else if (confirmPin.length === PIN_LENGTH && mainPin.length === PIN_LENGTH && confirmPin !==
mainPin) {
          errorMsg = "PINs do not match";
        }
        if (confirmPin.length === PIN_LENGTH && mainPin.length === PIN_LENGTH && confirmPin === mainPin) {
          errorMsg = "";
        }
        break;
      default:
        break;
    }
    setFieldErrors((prev) => ({ ...prev, [field]: errorMsg }));
  };
  const handleDigitChange = (type, index, value) => {
    if (!/^\d?$/.test(value)) {
      return;
    }
    const setter = type === "pin" ? setPinDigits : setConfirmPinDigits;
    const refs = type === "pin" ? pinRefs : confirmPinRefs;
    const field = type === "pin" ? "pin" : "confirmPin";
    const storageKey = type === "pin" ? "ownerOnboardingPin" : "ownerOnboardingConfirmPin";
    setter((prev) => {
      const updated = [...prev];
      updated[index] = value;
      localStorage.setItem(storageKey, JSON.stringify(updated));
      if (field === "pin") {
        const pinString = updated.join("");
        if (pinString.length === PIN_LENGTH) {
          const pinValidation = validatePinSecurity(pinString);
          if (!pinValidation.isValid) {
            setFieldErrors((prev2) => ({ ...prev2, pin: pinValidation.error }));
          } else {
            setFieldErrors((prev2) => ({ ...prev2, pin: "" }));
          }
        }
      }
```

```
      if (touched[field]) {
        setTimeout(() => {
          validateField(field, updated.join(""));
        }, 0);
      }
      return updated;
    });
    if (value && index < PIN_LENGTH - 1) {
      refs.current[index + 1]?.focus();
    }
    if (!touched[field]) {
      setTouched((prev) => ({ ...prev, [field]: true }));
    }
  };
  const handleKeyDown = (type, index, event) => {
    if (event.key !== "Backspace") {
      return;
    }
    const values = type === "pin" ? pinDigits : confirmPinDigits;
    const refs = type === "pin" ? pinRefs : confirmPinRefs;
    if (!values[index] && index > 0) {
      refs.current[index - 1]?.focus();
    }
  };
  const handlePaste = (type, event) => {
    event.preventDefault();
    const digits = event.clipboardData.getData("Text").replace(/\D/g, "").slice(0, PIN_LENGTH).split("");
    if (!digits.length) {
      return;
    }
    while (digits.length < PIN_LENGTH) {
      digits.push("");
    }
    const setter = type === "pin" ? setPinDigits : setConfirmPinDigits;
    const refs = type === "pin" ? pinRefs : confirmPinRefs;
    setter(digits);
    let lastIndex = 0;
    digits.forEach((digit, idx) => {
      if (digit !== "") {
        lastIndex = idx;
      }
    });
    refs.current[lastIndex]?.focus();
  };
  const validatePhilippineNumber = (number) => {
    const cleaned = number.replace(/[\s\-()]/g, "");
    const pattern09 = /^09\d{9}$/;
    const patternPlus63 = /^\+639\d{9}$/;
    const pattern63 = /^639\d{9}$/;
    return pattern09.test(cleaned) || patternPlus63.test(cleaned) || pattern63.test(cleaned);
  };
  const validateForm = () => {
    setTouched({
      firstName: true,
      lastName: true,
      email: true,
      contactNo: true,
      pin: true,
      confirmPin: true
    });
    validateField("firstName", formValues.firstName);
    validateField("lastName", formValues.lastName);
    validateField("email", formValues.email);
    validateField("contactNo", formValues.contactNo);
    validateField("pin", pinDigits.join(""));
    validateField("confirmPin", confirmPinDigits.join(""));
    const hasErrors = !formValues.firstName.trim() || !formValues.lastName.trim() || !formValues.email.trim()
|| !formValues.email.includes("@") || !formValues.contactNo.trim()
|| !validatePhilippineNumber(formValues.contactNo) || pinDigits.join("").length !== PIN_LENGTH ||
confirmPinDigits.join("").length !== PIN_LENGTH || pinDigits.join("") !== confirmPinDigits.join("");
    pinDigits.join("") !== confirmPinDigits.join("") || !isEmailVerified;
    return !hasErrors;
  };
  const handleSendCode = async () => {
    if (!formValues.email || fieldErrors.email) {
      setTouched((prev) => ({ ...prev, email: true }));
      return;
```

```
    }
    setLoading(true);
    try {
      const response = await fetch(`${API_BASE_URL}/api/verification/send-code`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ email: formValues.email })
      });
      const data = await response.json();
      if (data.success) {
        setShowCodeSentCheck(true);
        setTimeout(() => {
          setIsCodeSent(true);
          setShowCodeSentCheck(false);
          setVerificationTimer(60);
        }, 1e3);
        const timer = setInterval(() => {
          setVerificationTimer((prev) => {
            if (prev <= 1) {
              clearInterval(timer);
              return 0;
            }
            return prev - 1;
          });
        }, 1e3);
      } else {
        setError(data.message || "Failed to send verification code");
      }
    } catch (err) {
      setError("Failed to send verification code");
    } finally {
      setLoading(false);
    }
  };
  const handleVerifyCode = async () => {
    if (!verificationCode) {
      setFieldErrors((prev) => ({ ...prev, verificationCode: "Please enter the code" }));
      return;
    }
    setLoading(true);
    try {
      const response = await fetch(`${API_BASE_URL}/api/verification/verify-code`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          email: formValues.email,
          code: verificationCode
        })
      });
      const data = await response.json();
      if (data.success) {
        setIsEmailVerified(true);
        setFieldErrors((prev) => ({ ...prev, verificationCode: "" }));
        setShowPinModal(true);
      } else {
        setFieldErrors((prev) => ({ ...prev, verificationCode: data.message || "Invalid code" }));
      }
    } catch (err) {
      setFieldErrors((prev) => ({ ...prev, verificationCode: "Verification failed" }));
    } finally {
      setLoading(false);
    }
  };
  const handleSubmit = async (event) => {
    event.preventDefault();
    if (!validateForm()) return;
    const pin = pinDigits.join("");
    const pinValidation = validatePinSecurity(pin);
    if (!pinValidation.isValid) {
      setFieldErrors((prev) => ({ ...prev, pin: pinValidation.error }));
      setTouched((prev) => ({ ...prev, pin: true }));
      setError(pinValidation.error);
      return;
    }
    setLoading(true);
    const payload = {
      name: `${formValues.firstName.trim()} ${formValues.lastName.trim()}`.trim(),
```

```
      firstName: formValues.firstName.trim(),
      lastName: formValues.lastName.trim(),
      contactNo: formValues.contactNo.trim(),
      email: formValues.email.trim(),
      role: "Owner",
      pin: pinDigits.join(""),
      permissions: {
        posTerminal: true,
        inventory: true,
        viewTransactions: true,
        generateReports: true
      },
      requiresPinReset: false,
      status: "Active",
      dateJoinedActual: (/* @__PURE__ */ new Date()).toISOString()
    };
    try {
      const response = await fetch(`${API_BASE_URL}/api/employees`, {
        method: "POST",
        headers: {
          "Content-Type": "application/json"
        },
        body: JSON.stringify(payload)
      });
      const data = await response.json();
      if (!data.success) {
        throw new Error(data.message || "Failed to create owner account.");
      }
      const ownerProfile = {
        ...data.data,
        id: data.data._id,
        permissions: payload.permissions,
        role: "Owner"
      };
      login(ownerProfile);
      localStorage.removeItem("ownerOnboardingForm");
      localStorage.removeItem("ownerOnboardingPin");
      localStorage.removeItem("ownerOnboardingConfirmPin");
      onSetupComplete?.();
      navigate("/dashboard");
    } catch (submissionError) {
      console.error("Error creating owner account:", submissionError);
      setError(submissionError.message || "Unable to create account. Please try again.");
    } finally {
      setLoading(false);
    }
  };
  const renderPinRow = (label, digits, type, refs) => {
    const field = type === "pin" ? "pin" : "confirmPin";
    const hasError = fieldErrors[field];
    return /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "block text-base font-medium text-gray-700 mb-4" }, label), /* @__PURE__ */
React.createElement("div", { className: "flex gap-3 justify-center" }, digits.map(
      (digit, index) => /* @__PURE__ */ React.createElement(
        "input",
        {
          key: `${label}-${index}`,
          ref: (node) => {
            refs.current[index] = node;
          },
          type: "password",
          inputMode: "numeric",
          maxLength: 1,
          value: digit,
          onChange: (event) => handleDigitChange(type, index, event.target.value),
          onKeyDown: (event) => handleKeyDown(type, index, event),
          onPaste: (event) => handlePaste(type, event),
          className: `w-16 h-16 rounded-xl border-2 ${hasError ? "border-red-500" : "border-gray-300"} bg-
white text-center text-2xl font-semibold text-[#2D2D2D] focus:border-[#8B7355] focus:outline-none`,
          "aria-label": `${label} digit ${index + 1}`
        }
      )
    )), hasError && /* @__PURE__ */ React.createElement("p", { className: "text-red-500 text-xs mt-2 text-
center" }, hasError));
  };
  return /* @__PURE__ */ React.createElement("div", { className: "flex w-screen h-screen overflow-hidden
flex-col lg:flex-row bg-white" }, /* @__PURE__ */ React.createElement("div", { className: "flex-1 bg-white
```

```
flex items-center justify-center px-6 py-10 lg:px-16" }, /* @__PURE__ */ React.createElement("div",
{ className: "w-full max-w-[600px]" }, /* @__PURE__ */ React.createElement("div", { className: "text-center
mb-10" }, /* @__PURE__ */ React.createElement("h1", { className: "text-5xl lg:text-6xl font-bold mb-4" }, /*
@__PURE__ */ React.createElement("span", { className: "text-[#2D2D2D]" }, "Welcome to "), /* @__PURE__ */
React.createElement("span", { className: "text-[#76462B]" }, "CYSPOS!")), /* @__PURE__ */
React.createElement("div", { className: "w-full max-w-md h-1 bg-[#C2A68C] mx-auto" })), /* @__PURE__ */
React.createElement("form", { className: "space-y-5", onSubmit: handleSubmit }, /* @__PURE__ */
React.createElement("div", { className: "grid grid-cols-2 gap-4" }, /* @__PURE__ */ React.createElement("div",
null, /* @__PURE__ */ React.createElement("label", { className: "block text-sm font-medium text-gray-600 mb-
2" }, "First Name"), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      className: `w-full px-4 py-3 rounded-xl border ${fieldErrors.firstName ? "border-red-500" : "border-
gray-200"} focus:outline-none focus:border-[#8B7355] bg-white`,
      placeholder: "John",
      value: formValues.firstName,
      onChange: (event) => handleInputChange("firstName", event.target.value),
      onBlur: () => handleBlur("firstName")
    }
  ), fieldErrors.firstName && /* @__PURE__ */ React.createElement("p", { className: "text-red-500 text-xs mt-
1" }, fieldErrors.firstName)), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("label", { className: "block text-sm font-medium text-gray-600 mb-2" }, "Last Name"), /*
@__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      className: `w-full px-4 py-3 rounded-xl border ${fieldErrors.lastName ? "border-red-500" : "border-
gray-200"} focus:outline-none focus:border-[#8B7355] bg-white`,
      placeholder: "Doe",
      value: formValues.lastName,
      onChange: (event) => handleInputChange("lastName", event.target.value),
      onBlur: () => handleBlur("lastName")
    }
  ), fieldErrors.lastName && /* @__PURE__ */ React.createElement("p", { className: "text-red-500 text-xs mt-
1" }, fieldErrors.lastName))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("label", { className: "block text-sm font-medium text-gray-600 mb-2" }, "Contact Number"),
/* @__PURE__ */ React.createElement(
    "input",
    {
      type: "tel",
      className: `w-full px-4 py-3 rounded-xl border ${fieldErrors.contactNo ? "border-red-500" : "border-
gray-200"} focus:outline-none focus:border-[#8B7355] bg-white`,
      placeholder: "+63 900 000 0000",
      value: formValues.contactNo,
      onChange: (event) => handleInputChange("contactNo", event.target.value),
      onBlur: () => handleBlur("contactNo")
    }
  ), fieldErrors.contactNo && /* @__PURE__ */ React.createElement("p", { className: "text-red-500 text-xs mt-
1" }, fieldErrors.contactNo)), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("label", { className: "block text-sm font-medium text-gray-600 mb-2" }, "Email"), /*
@__PURE__ */ React.createElement("div", { className: "flex gap-0" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center justify-center bg-[#8B7355] rounded-l-xl px-4" },
/* @__PURE__ */ React.createElement("svg", { className: "w-5 h-5 text-white", fill: "currentColor", viewBox:
"0 0 20 20" }, /* @__PURE__ */ React.createElement("path", { fillRule: "evenodd", d: "M10 9a3 3 0 100-6 3 3 0
000 6zm-7 9a7 7 0 1114 0H3z", clipRule: "evenodd" }))), /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "email",
      className: `flex-1 px-4 py-3 rounded-r-xl border-l-0 border ${fieldErrors.email ? "border-red-500" :
"border-gray-200"} focus:outline-none focus:border-[#8B7355] bg-white`,
      placeholder: "Enter your Email...",
      value: formValues.email,
      onChange: (event) => handleInputChange("email", event.target.value),
      onBlur: () => handleBlur("email")
    }
  )), fieldErrors.email && /* @__PURE__ */ React.createElement("p", { className: "text-red-500 text-xs mt-
1" }, fieldErrors.email)), !isEmailVerified && /* @__PURE__ */ React.createElement("div", { className:
"space-y-3" }, !isCodeSent ? /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: handleSendCode,
      disabled: loading || !formValues.email || !!fieldErrors.email || verificationTimer > 0 ||
showCodeSentCheck,
      className: "w-full py-3 rounded-xl border border-[#8B7355] text-[#8B7355] font-medium hover:bg-[#8B7355]
hover:text-white transition-colors disabled:opacity-50 disabled:cursor-not-allowed flex items-center justify-
```

```
center gap-2"
    },
    loading ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(FaSpinner, { className: "animate-spin" }), /* @__PURE__ */ React.createElement("span",
null, "Sending...")) : showCodeSentCheck ? /* @__PURE__ */ React.createElement(React.Fragment, null, /*
@__PURE__ */ React.createElement(FaCheck, { className: "animate-pulse" }), /* @__PURE__ */
React.createElement("span", null, "Code Sent!")) : verificationTimer > 0 ? `Resend in ${verificationTimer}s` :
"Send Verification Code"
  ) : /* @__PURE__ */ React.createElement("div", { className: "flex gap-2" }, /* @__PURE__ */
React.createElement("div", { className: "flex-1" }, /* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      className: `w-full px-4 py-3 rounded-xl border ${fieldErrors.verificationCode ? "border-red-500" :
"border-gray-200"} focus:outline-none focus:border-[#8B7355] bg-white`,
      placeholder: "Enter 6-digit code",
      value: verificationCode,
      onChange: (e) => setVerificationCode(e.target.value),
      maxLength: 6
    }
  ), fieldErrors.verificationCode && /* @__PURE__ */ React.createElement("p", { className: "text-red-500
text-xs mt-1" }, fieldErrors.verificationCode)), /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: handleVerifyCode,
      disabled: loading,
      className: "px-6 py-3 rounded-xl bg-[#8B7355] text-white font-medium hover:bg-[#6d5a43] transition-
colors disabled:opacity-50"
    },
    "Verify"
  ))), isEmailVerified && /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-2
text-green-600 bg-green-50 p-3 rounded-xl border border-green-200" }, /* @__PURE__ */
React.createElement("svg", { className: "w-5 h-5", fill: "none", stroke: "currentColor", viewBox: "0 0 24
24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap: "round", strokeLinejoin: "round",
strokeWidth: "2", d: "M5 1314 4L19 7" })), /* @__PURE__ */ React.createElement("span", { className: "font-
medium" }, "Email Verified Successfully")), isEmailVerified && !showPinModal && /* @__PURE__ */
React.createElement(
    "button",
    {
      type: "button",
      onClick: () => setShowPinModal(true),
      className: "w-full bg-[#8B7355] text-white rounded-2xl px-8 py-4 text-lg font-semibold transition-all
duration-300 hover:bg-[#6d5a43] hover:shadow-lg mt-4"
    },
    "Continue to PIN Setup"
  ), showPinModal && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-50 flex items-
center justify-center bg-black/50 backdrop-blur-sm p-4" }, /* @__PURE__ */ React.createElement("div",
{ className: "bg-white rounded-3xl p-8 w-full max-w-lg shadow-2xl relative animate-in fade-in zoom-in
duration-300" }, /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "button",
      onClick: () => setShowPinModal(false),
      className: "absolute top-4 right-4 text-gray-400 hover:text-gray-600"
    },
    /* @__PURE__ */ React.createElement("svg", { className: "w-6 h-6", fill: "none", stroke: "currentColor",
viewBox: "0 0 24 24" }, /* @__PURE__ */ React.createElement("path", { strokeLinecap: "round", strokeLinejoin:
"round", strokeWidth: "2", d: "M6 18L18 6M6 6l12 12" }))
  ), /* @__PURE__ */ React.createElement("h2", { className: "text-2xl font-bold text-[#2D2D2D] mb-6 text-
center" }, "Secure your Account"), /* @__PURE__ */ React.createElement("div", { className: "space-y-8" }, /*
@__PURE__ */ React.createElement("div", { className: "text-center" }, renderPinRow("Create PIN Code",
pinDigits, "pin", pinRefs)), /* @__PURE__ */ React.createElement("div", { className: "text-center" },
renderPinRow("Confirm PIN Code", confirmPinDigits, "confirm", confirmPinRefs)), /* @__PURE__ */
React.createElement("div", { className: "flex flex-col gap-3 items-center" }, /* @__PURE__ */
React.createElement(
    "button",
    {
      type: "submit",
      disabled: loading,
      className: "w-full bg-[#8B7355] text-white rounded-2xl px-8 py-4 text-lg font-semibold transition-all
duration-300 hover:bg-[#6d5a43] hover:shadow-lg disabled:opacity-50 disabled:cursor-not-allowed"
    },
    loading ? "Creating Account..." : "Complete Setup"
  )))))))), /* @__PURE__ */ React.createElement("div", { className: "flex-1 relative flex items-center
justify-center p-8 bg-white min-h-[40vh] lg:min-h-full" }, /* @__PURE__ */ React.createElement(
    "div",
```

```
    {
      className: "absolute inset-8 rounded-[20px] bg-cover bg-center",
      style: {
        backgroundImage: `linear-gradient(rgba(139, 115, 85, 0.7), rgba(139, 115, 85, 0.7)), url(${bgImage})`
      }
    }
  ), /* @__PURE__ */ React.createElement("div", { className: "relative z-10 text-center p-12 flex items-
center justify-center" }, /* @__PURE__ */ React.createElement(
    "img",
    {
      src: logo,
      alt: "Create Your Style",
      className: "max-w-[80%] lg:max-w-[70%] h-auto object-contain drop-shadow-[2px_2px_8px_rgba(0,0,0,0.3)]"
    }
  ))));
};
export default memo(OwnerOnboarding);
```

FILE: pos/frontend/src/pages/PinEntry.jsx
==========

```
import { memo, useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import bgImage from "../assets/bg.png";
import defaultAvatar from "../assets/default.jpeg";
import logo from "../assets/logo.png";
import { API_BASE_URL } from "../config/api";
import { useAuth } from "../context/AuthContext";
const PinEntry = () => {
  const navigate = useNavigate();
  const [pin, setPin] = useState("");
  const [showKeypad, setShowKeypad] = useState(false);
  const [error, setError] = useState("");
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();
  const handleNumberClick = (number) => {
    if (pin.length < 6) {
      setPin(pin + number);
    }
  };
  const handleBackspace = () => {
    setPin(pin.slice(0, -1));
  };
  const redirectAfterLogin = (employeeData) => {
    if (employeeData.role === "Owner" || employeeData.name === "owner" || employeeData.id === 3) {
      navigate("/dashboard");
    } else if (employeeData.permissions?.posTerminal) {
      navigate("/terminal");
    } else if (employeeData.permissions?.inventory) {
      navigate("/inventory");
    } else if (employeeData.permissions?.viewTransactions) {
      navigate("/transactions");
    } else if (employeeData.permissions?.generateReports) {
      navigate("/reports");
    } else {
      navigate("/settings");
    }
  };
  const rememberPendingTempPin = (employeeData, pinValue) => {
    if (typeof window === "undefined") return;
    try {
      sessionStorage.setItem(
        "pendingTempPin",
        JSON.stringify({
          employeeId: employeeData._id,
          pin: pinValue
        })
      );
    } catch (storageError) {
      console.warn("Unable to cache temporary PIN", storageError);
    }
  };
  const clearPendingTempPin = () => {
    if (typeof window === "undefined") return;
    try {
      sessionStorage.removeItem("pendingTempPin");
```

```
    } catch (storageError) {
      console.warn("Unable to clear cached temporary PIN", storageError);
    }
  };
  const handleLogin = async () => {
    if (pin.length !== 6) {
      setError("Please enter a 6-digit PIN");
      return;
    }
    setError("");
    setLoading(true);
    try {
      const response = await fetch(
        `${API_BASE_URL}/api/employees/verify-pin`,
        {
          method: "POST",
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify({
            pin
          })
        }
      );
      const data = await response.json();
      if (data.success) {
        const employeeData = {
          ...data.data,
          id: data.data._id,
          image: data.data.profileImage || defaultAvatar
        };
        login(employeeData);
        if (employeeData.requiresPinReset) {
          rememberPendingTempPin(employeeData, pin);
          navigate("/set-pin", {
            state: { employee: employeeData, tempPin: pin }
          });
        } else {
          clearPendingTempPin();
          redirectAfterLogin(employeeData);
        }
        setPin("");
      } else {
        setError(data.message || "Invalid PIN. Please try again.");
        setPin("");
      }
    } catch (error2) {
      console.error("Error verifying PIN:", error2);
      setError(
        "Failed to connect to server. Please make sure the backend is running."
      );
      setPin("");
    } finally {
      setLoading(false);
    }
  };
  useEffect(() => {
    const handleKeyPress = (event) => {
      if (event.key >= "0" && event.key <= "9") {
        handleNumberClick(event.key);
      } else if (event.key === "Backspace" || event.key === "Delete") {
        handleBackspace();
      } else if (event.key === "Enter") {
        handleLogin();
      }
    };
    window.addEventListener("keydown", handleKeyPress);
    return () => {
      window.removeEventListener("keydown", handleKeyPress);
    };
  }, [pin]);
  return /* @__PURE__ */ React.createElement("div", { className: "flex w-screen h-screen overflow-hidden
flex-col lg:flex-row" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "flex-1 relative flex items-center justify-center p-8 min-h-[40vh] lg:min-h-full",
      style: { backgroundColor: "#FFFFFF" }
```

```
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "absolute inset-8 lg:inset-8 rounded-[20px] bg-cover bg-center",
        style: {
          backgroundImage: `linear-gradient(rgba(139, 115, 85, 0.7), rgba(139, 115, 85, 0.7)),
url(${bgImage})`
        }
      }
    ),
    /* @__PURE__ */ React.createElement("div", { className: "relative z-10 text-center p-12 flex items-center
justify-center" }, /* @__PURE__ */ React.createElement(
      "img",
      {
        src: logo,
        alt: "Create Your Style",
        className: "max-w-[80%] lg:max-w-[70%] h-auto object-contain drop-shadow-
[2px_2px_8px_rgba(0,0,0,0.3)]"
      }
    ))
  ), /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "flex-1 flex items-center justify-center p-8 min-h-[60vh] lg:min-h-full",
      style: { backgroundColor: "#FFFFFF" }
    },
    /* @__PURE__ */ React.createElement("div", { className: "w-full max-w-[600px]" }, /* @__PURE__ */
React.createElement("div", { className: "text-center mb-12" }, /* @__PURE__ */ React.createElement("h2",
{ className: "text-5xl font-bold text-[#76462B] mb-4 tracking-[8px]" }, "CYSPOS"), /* @__PURE__ */
React.createElement("div", { className: "w-full h-[6px]", style: { background: "linear-gradient(135deg,
#AD7F65 0%, #76462B 100%)" } })), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `flex items-start gap-12 transition-all duration-500 ${showKeypad ? "justify-center" :
"justify-center"}`
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `flex flex-col items-center transition-all duration-500 ${showKeypad ? "mt-10" : "mt-
0"}`
        },
        /* @__PURE__ */ React.createElement("div", { className: "w-[120px] h-[120px] rounded-full overflow-
hidden mb-4 bg-gray-200 flex items-center justify-center" }, /* @__PURE__ */ React.createElement(
          "img",
          {
            src: defaultAvatar,
            alt: "Employee",
            className: "w-full h-full object-cover"
          }
        )),
        /* @__PURE__ */ React.createElement(
          "p",
          {
            className: "text-xl font-semibold text-gray-800 mb-1",
            style: { fontFamily: "'Poppins', sans-serif" }
          },
          "Enter Your PIN"
        ),
        /* @__PURE__ */ React.createElement(
          "p",
          {
            className: "text-sm text-gray-600 mb-2",
            style: { fontFamily: "'Poppins', sans-serif" }
          },
          "Please enter your 6-digit PIN to continue"
        ),
        error && /* @__PURE__ */ React.createElement(
          "p",
          {
            className: "text-xs text-red-600 mb-4 text-center max-w-xs",
            style: { fontFamily: "'Poppins', sans-serif" }
          },
          error
        ),
        /* @__PURE__ */ React.createElement(
```

```
          "p",
          {
            className: "text-sm text-gray-600 mb-6",
            style: { fontFamily: "'Poppins', sans-serif" }
          },
          "PIN"
        ),
        /* @__PURE__ */ React.createElement("div", { className: "flex justify-center gap-4 mb-6" }, [0, 1, 2,
3, 4, 5].map(
          (index) => /* @__PURE__ */ React.createElement(
            "div",
            {
              key: index,
              className: "w-12 h-12 rounded-xl bg-white border-2 border-gray-300 flex items-center justify-
center shadow-sm"
            },
            index < pin.length && /* @__PURE__ */ React.createElement("div", { className: "w-3 h-3 rounded-
full bg-gray-800" })
          )
        )),
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: () => setShowKeypad(!showKeypad),
            className: "text-[#8B7355] text-sm font-semibold cursor-pointer hover:text-[#6d5a43] transition-
colors duration-200 mb-4",
            style: { fontFamily: "'Poppins', sans-serif" }
          },
          showKeypad ? "Hide Keypad" : "Show Keypad"
        )
      ),
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `flex flex-col items-center transition-all duration-500 overflow-hidden ${showKeypad ?
"opacity-100 max-w-[280px]" : "opacity-0 max-w-0"}`
        },
        /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-3 gap-4 mb-8" }, [7, 8, 9, 4,
5, 6, 1, 2, 3].map(
          (number) => /* @__PURE__ */ React.createElement(
            "button",
            {
              key: number,
              onClick: () => handleNumberClick(number.toString()),
              className: "w-16 h-16 rounded-full bg-white text-gray-900 text-2xl font-semibold cursor-pointer
transition-all duration-200 shadow-md hover:shadow-lg hover:bg-gray-50 active:scale-95",
              style: { fontFamily: "'Poppins', sans-serif" }
            },
            number
          )
        ), /* @__PURE__ */ React.createElement("div", { className: "w-16 h-16" }), /* @__PURE__ */
React.createElement(
          "button",
          {
            onClick: () => handleNumberClick("0"),
            className: "w-16 h-16 rounded-full bg-white text-gray-900 text-2xl font-semibold cursor-pointer
transition-all duration-200 shadow-md hover:shadow-lg hover:bg-gray-50 active:scale-95",
            style: { fontFamily: "'Poppins', sans-serif" }
          },
          "0"
        ), /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: handleBackspace,
            className: "w-16 h-16 rounded-full bg-white text-gray-900 cursor-pointer transition-all duration-
200 shadow-md hover:shadow-lg hover:bg-gray-50 active:scale-95 flex items-center justify-center"
          },
          /* @__PURE__ */ React.createElement(
            "svg",
            {
              xmlns: "http://www.w3.org/2000/svg",
              className: "h-6 w-6",
              fill: "none",
              viewBox: "0 0 24 24",
              stroke: "currentColor",
              strokeWidth: 2
            },
```

```
            /* @__PURE__ */ React.createElement(
              "path",
              {
                strokeLinecap: "round",
                strokeLinejoin: "round",
                d: "M6 18L18 6M6 6l12 12"
              }
            )
          )
        )),
        showKeypad && /* @__PURE__ */ React.createElement(
          "button",
          {
            className: "text-white border-none rounded-full px-20 py-3 text-lg font-semibold cursor-pointer
transition-all duration-300 uppercase shadow-md hover:shadow-lg active:translate-y-0.5 disabled:opacity-50
disabled:cursor-not-allowed",
            onClick: handleLogin,
            disabled: loading || pin.length !== 6,
            style: { fontFamily: "'Poppins', sans-serif", background: "linear-gradient(135deg, #AD7F65 0%,
#76462B 100%)" }
          },
          loading ? "VERIFYING..." : "LOGIN"
        )
      )
    ), !showKeypad && /* @__PURE__ */ React.createElement("div", { className: "text-center mt-8" }, /*
@__PURE__ */ React.createElement(
      "button",
      {
        className: "text-white border-none rounded-[10px] px-12 py-3 text-lg font-semibold cursor-pointer
transition-all duration-300 uppercase shadow-md hover:shadow-lg active:translate-y-0.5 disabled:opacity-50
disabled:cursor-not-allowed -translate-x-5",
        onClick: handleLogin,
        disabled: loading || pin.length !== 6,
        style: { fontFamily: "sans-serif", background: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)" }
      },
      loading ? "VERIFYING..." : "LOGIN"
    )))
  ));
};
export default memo(PinEntry);
```

FILE: pos/frontend/src/pages/SetNewPin.jsx
==========

```
import { memo, useEffect, useRef, useState } from "react";
import { useLocation, useNavigate } from "react-router-dom";
import bgImage from "../assets/bg.png";
import defaultAvatar from "../assets/default.jpeg";
import logo from "../assets/logo.png";
import { API_BASE_URL } from "../config/api";
import { useAuth } from "../context/AuthContext";
import { validatePinSecurity } from "../utils/pinValidation";
const PIN_LENGTH = 6;
const EMPTY_PIN = Array(PIN_LENGTH).fill("");
const SetNewPin = () => {
  const location = useLocation();
  const navigate = useNavigate();
  const { currentUser, login } = useAuth();
  const employee = location.state?.employee || currentUser;
  const [pendingTempData, setPendingTempData] = useState(() => {
    if (location.state?.tempPin && location.state?.employee?._id) {
      const payload = {
        pin: location.state.tempPin,
        employeeId: location.state.employee._id
      };
      if (typeof window !== "undefined") {
        try {
          sessionStorage.setItem("pendingTempPin", JSON.stringify(payload));
        } catch (storageError) {
          console.warn("Unable to persist temporary PIN", storageError);
        }
      }
      return payload;
    }
    if (typeof window !== "undefined") {
      const cached = sessionStorage.getItem("pendingTempPin");
```

```
    if (cached) {
      try {
        return JSON.parse(cached);
      } catch {
        sessionStorage.removeItem("pendingTempPin");
      }
    }
  }
  return null;
});
const pendingTempPin = pendingTempData?.pin || "";
const pendingEmployeeId = pendingTempData?.employeeId || null;
const [newPinDigits, setNewPinDigits] = useState(EMPTY_PIN);
const [confirmPinDigits, setConfirmPinDigits] = useState(EMPTY_PIN);
const [loading, setLoading] = useState(false);
const [error, setError] = useState("");
const [pinUpdated, setPinUpdated] = useState(false);
const newPinRefs = useRef([]);
const confirmPinRefs = useRef([]);
useEffect(() => {
  if (!employee) {
    navigate("/");
  } else if (!employee.requiresPinReset) {
    navigate("/dashboard");
  }
}, [employee, navigate]);
useEffect(() => {
  if (location.state?.tempPin && location.state?.employee?._id) {
    const payload = {
      pin: location.state.tempPin,
      employeeId: location.state.employee._id
    };
    setPendingTempData(payload);
    if (typeof window !== "undefined") {
      try {
        sessionStorage.setItem("pendingTempPin", JSON.stringify(payload));
      } catch (storageError) {
        console.warn("Unable to persist temporary PIN", storageError);
      }
    }
  }
}, [location.state]);
useEffect(() => {
  newPinRefs.current[0]?.focus();
}, []);
if (!employee) {
  return null;
}
const redirectAfterLogin = (updatedEmployee) => {
  if (updatedEmployee.role === "Owner") {
    navigate("/dashboard");
  } else if (updatedEmployee.permissions?.posTerminal) {
    navigate("/terminal");
  } else if (updatedEmployee.permissions?.inventory) {
    navigate("/inventory");
  } else if (updatedEmployee.permissions?.viewTransactions) {
    navigate("/transactions");
  } else if (updatedEmployee.permissions?.generateReports) {
    navigate("/reports");
  } else {
    navigate("/settings");
  }
};
const handleDigitChange = (type, index, value) => {
  if (!/^\d?$/.test(value)) {
    return;
  }
  const setter = type === "new" ? setNewPinDigits : setConfirmPinDigits;
  const refs = type === "new" ? newPinRefs : confirmPinRefs;
  setter((prev) => {
    const updated = [...prev];
    updated[index] = value;
    if (type === "new") {
      const pinString = updated.join("");
      if (pinString.length === PIN_LENGTH) {
        const pinValidation = validatePinSecurity(pinString);
        if (!pinValidation.isValid) {
```

```
          setError(pinValidation.error);
        } else {
          setError("");
        }
      } else if (updated.join("").length < PIN_LENGTH) {
        setError("");
      }
    }
    return updated;
  });
  if (value && index < PIN_LENGTH - 1) {
    refs.current[index + 1]?.focus();
  }
};
const handleKeyDown = (type, index, event) => {
  if (event.key !== "Backspace") {
    return;
  }
  const values = type === "new" ? newPinDigits : confirmPinDigits;
  const refs = type === "new" ? newPinRefs : confirmPinRefs;
  if (!values[index] && index > 0) {
    refs.current[index - 1]?.focus();
  }
};
const handlePaste = (type, event) => {
  event.preventDefault();
  const pasted = event.clipboardData.getData("Text").replace(/\D/g, "").slice(0, PIN_LENGTH);
  if (!pasted) {
    return;
  }
  const setter = type === "new" ? setNewPinDigits : setConfirmPinDigits;
  const refs = type === "new" ? newPinRefs : confirmPinRefs;
  const digits = pasted.split("");
  while (digits.length < PIN_LENGTH) {
    digits.push("");
  }
  setter(digits);
  const lastFilledIndex = Math.min(PIN_LENGTH - 1, pasted.length - 1);
  refs.current[lastFilledIndex]?.focus();
};
const handleSubmit = async (e) => {
  e.preventDefault();
  const newPin = newPinDigits.join("");
  const confirmPin = confirmPinDigits.join("");
  let employeeId = null;
  if (employee && typeof employee === "object") {
    employeeId = employee._id || employee.id;
  }
  if (!employeeId && pendingEmployeeId) {
    employeeId = pendingEmployeeId;
  }
  if (!employeeId) {
    setError("Session expired. Please log in again.");
    navigate("/");
    return;
  }
  if (!pendingTempPin || pendingTempPin.length !== PIN_LENGTH) {
    setError("Session expired. Please log in with your temporary PIN again.");
    return;
  }
  if (pendingEmployeeId && employeeId && pendingEmployeeId !== employeeId) {
    setError("Selected account mismatch. Please log in again.");
    return;
  }
  if (newPin.length !== PIN_LENGTH || confirmPin.length !== PIN_LENGTH) {
    setError("New PIN must have exactly 6 digits.");
    return;
  }
  if (newPin !== confirmPin) {
    setError("New PIN and confirmation do not match.");
    return;
  }
  if (!/^\d{6}$/.test(newPin)) {
    setError("PIN should contain numbers only.");
    return;
  }
  const pinValidation = validatePinSecurity(newPin);
```

```
    if (!pinValidation.isValid) {
      setError(pinValidation.error);
      return;
    }
    setError("");
    setLoading(true);
    try {
      const response = await fetch(
        `${API_BASE_URL}/api/employees/${employeeId}/pin`,
        {
          method: "PUT",
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify({
            currentPin: pendingTempPin,
            newPin,
            requiresPinReset: false
          })
        }
      );
      const data = await response.json();
      if (data.success && data.data) {
        const updatedEmployee = {
          ...data.data,
          id: data.data._id || data.data.id,
          image: data.data.profileImage || employee?.image || defaultAvatar
        };
        login(updatedEmployee);
        setPinUpdated(true);
        setNewPinDigits(EMPTY_PIN);
        setConfirmPinDigits(EMPTY_PIN);
        if (typeof window !== "undefined") {
          sessionStorage.removeItem("pendingTempPin");
        }
        redirectAfterLogin(updatedEmployee);
      } else {
        setError(data.message || "Failed to update PIN. Please try again.");
      }
    } catch (err) {
      console.error("Error updating PIN:", err);
      setError("Unable to connect to the server. Please verify your network.");
    } finally {
      setLoading(false);
    }
  };
  const renderPinInputs = (label, digits, type, refs) => /* @__PURE__ */ React.createElement("div", null, /*
@__PURE__ */ React.createElement("p", { className: "text-sm text-gray-500 mb-3" }, label), /* @__PURE__ */
React.createElement("div", { className: "flex gap-3 justify-center" }, digits.map(
    (digit, index) => /* @__PURE__ */ React.createElement(
      "input",
      {
        key: `${label}-${index}`,
        ref: (node) => {
          refs.current[index] = node;
        },
        value: digit,
        onChange: (event) => handleDigitChange(type, index, event.target.value),
        onKeyDown: (event) => handleKeyDown(type, index, event),
        onPaste: (event) => handlePaste(type, event),
        type: "password",
        inputMode: "numeric",
        maxLength: 1,
        className: "w-12 h-12 rounded-xl border border-gray-200 bg-gray-50 text-center text-xl font-semibold
text-[#2F1E12] shadow-[inset_0_2px_4px_rgba(0,0,0,0.08)] focus:border-[#B37A5C] focus:bg-white focus:outline-
none",
        "aria-label": `${label} digit ${index + 1}`
      }
    )
  )));
  return /* @__PURE__ */ React.createElement("div", { className: "relative w-screen h-screen overflow-
hidden" }, /* @__PURE__ */ React.createElement("div", { className: "flex w-full h-full overflow-hidden flex-
col lg:flex-row" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "flex-1 relative flex items-center justify-center p-8 min-h-[40vh] lg:min-h-full",
      style: { backgroundColor: "#FFFFFF" }
```

```
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "absolute inset-8 lg:inset-8 rounded-[20px] bg-cover bg-center",
        style: {
          backgroundImage: `linear-gradient(rgba(139, 115, 85, 0.7), rgba(139, 115, 85, 0.7)),
url(${bgImage})`
        }
      }
    ),
    /* @__PURE__ */ React.createElement("div", { className: "relative z-10 text-center p-12 flex items-center
justify-center" }), /* @__PURE__ */ React.createElement(
      "img",
      {
        src: logo,
        alt: "Create Your Style",
        className: "max-w-[80%] lg:max-w-[70%] h-auto object-contain drop-shadow-
[2px_2px_8px_rgba(0,0,0,0.3)]"
      }
    ))
  ), /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "flex-1 flex items-center justify-center p-8 min-h-[60vh] lg:min-h-full",
      style: { backgroundColor: "#FFFFFF" }
    },
    /* @__PURE__ */ React.createElement("div", { className: "w-full max-w-[600px]" }, /* @__PURE__ */
React.createElement("div", { className: "text-center mb-12" }, /* @__PURE__ */ React.createElement("h2",
{ className: "text-5xl font-bold text-[#8B7355] mb-4 tracking-[8px]" }, "CYSPOS"), /* @__PURE__ */
React.createElement("div", { className: "w-full h-0 pb-6 border-b-[3px] border-[#8B7355]" })))
  )), /* @__PURE__ */ React.createElement("div", { className: "absolute inset-0 z-50 flex items-center
justify-center px-4 py-10 bg-black/20 backdrop-blur-sm" }, /* @__PURE__ */ React.createElement("div",
{ className: "relative w-full max-w-md rounded-4xl bg-white/95 shadow-[0px_25px_60px_rgba(0,0,0,0.25)] px-8
py-10" }, /* @__PURE__ */ React.createElement("div", { className: "absolute -top-5 left-1/2 -translate-x-
1/2" }, /* @__PURE__ */ React.createElement("div", { className: "px-6 py-1 rounded-full bg-white/90 shadow
text-xs font-semibold text-gray-500 uppercase tracking-[0.4em]" }, "Loading...")), /* @__PURE__ */
React.createElement(
    "button",
    {
      type: "button",
      className: "absolute top-4 right-4 text-gray-400 hover:text-gray-600 transition-colors",
      onClick: () => navigate(-1),
      "aria-label": "Close"
    },
    "\xD7"
  ), /* @__PURE__ */ React.createElement("div", { className: "flex flex-col items-center gap-6 text-center" },
/* @__PURE__ */ React.createElement("div", { className: "w-28 h-28 rounded-full bg-[#F4E6DD] flex items-
center justify-center" }), /* @__PURE__ */ React.createElement(
    "svg",
    {
      width: "72",
      height: "72",
      viewBox: "0 0 54 54",
      fill: "none",
      xmlns: "http://www.w3.org/2000/svg"
    },
    /* @__PURE__ */ React.createElement(
      "rect",
      {
        x: "3",
        y: "9",
        width: "48",
        height: "40",
        rx: "12",
        fill: "#C7A086"
      }
    ),
    /* @__PURE__ */ React.createElement(
      "path",
      {
        d: "M36 21H18C16.8954 21 16 21.8954 16 23V35C16 36.1046 16.8954 37 18 37H36C37.1046 37 38 36.1046 38
35V23C38 21.8954 37.1046 21 36 21Z",
        stroke: "#5E3B28",
        strokeWidth: "2",
        strokeLinecap: "round",
        strokeLinejoin: "round"
```

```
      }
    ),
    /* @__PURE__ */ React.createElement(
      "path",
      {
        d: "M32 21V17C32 14.2386 29.7614 12 27 12C24.2386 12 22 14.2386 22 17V21",
        stroke: "#5E3B28",
        strokeWidth: "2",
        strokeLinecap: "round",
        strokeLinejoin: "round"
      }
    )
  )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("h1", { className:
"text-2xl font-semibold text-[#1F140E]" }, "Set New PIN"), /* @__PURE__ */ React.createElement("p",
{ className: "text-sm text-gray-500 mt-1" }, "Please create your new PIN")), /* @__PURE__ */
React.createElement("div", { className: "flex flex-col items-center gap-1" }, /* @__PURE__ */
React.createElement("div", { className: "w-16 h-16 rounded-2xl overflow-hidden shadow" }, /* @__PURE__ */
React.createElement(
    "img",
    {
      src: employee.image || defaultAvatar,
      alt: employee.name,
      className: "w-full h-full object-cover"
    }
  )), /* @__PURE__ */ React.createElement("p", { className: "text-base font-semibold text-[#2F1D13]" },
employee.name), /* @__PURE__ */ React.createElement("p", { className: "text-xs text-gray-500" },
employee.email))), /* @__PURE__ */ React.createElement("form", { onSubmit: handleSubmit, className: "mt-8
space-y-7" }, error && /* @__PURE__ */ React.createElement("div", { className: "bg-red-50 border border-red-
200 text-red-600 px-4 py-3 rounded-2xl text-sm" }, error), renderPinInputs("New PIN", newPinDigits, "new",
newPinRefs), renderPinInputs(
    "Confirm PIN",
    confirmPinDigits,
    "confirm",
    confirmPinRefs
  ), /* @__PURE__ */ React.createElement(
    "button",
    {
      type: "submit",
      disabled: loading,
      className: "w-full py-4 rounded-[18px] text-white font-semibold disabled:opacity-60 disabled:cursor-
not-allowed",
      style: {
        background: "linear-gradient(135deg, #b37a5c 0%, #7a4b2e 100%)"
      }
    },
    loading ? "Saving..." : "Save"
  ), /* @__PURE__ */ React.createElement("p", { className: "text-center text-xs text-gray-500" }, pinUpdated ?
"PIN updated successfully." : "Temporary PIN verified. Please create a new one."))));
};
export default memo(SetNewPin);
```

FILE: pos/frontend/src/pages/Settings.jsx
==========

```
import {
  memo,
  useCallback,
  useContext,
  useEffect,
  useMemo,
  useRef,
  useState
} from "react";
import {
  FaBox,
  FaCamera,
  FaCheckCircle,
  FaCog,
  FaDatabase,
  FaDownload,
  FaEye,
  FaEyeSlash,
  FaInfoCircle,
  FaKey,
  FaLink,
  FaPalette,
```

```
  FaShieldAlt,
  FaSpinner,
  FaSync,
  FaTimesCircle,
  FaTrash,
  FaUndo,
  FaUser
} from "react-icons/fa";
import { useNavigate } from "react-router-dom";
import defaultAvatar from "../assets/default.jpeg";
import SuccessModal from "../components/inventory/SuccessModal";
import Header from "../components/shared/header";
import { API_BASE_URL as API_BASE } from "../config/api";
import { useAuth } from "../context/AuthContext";
import { useDataCache } from "../context/DataCacheContext";
import { SidebarContext } from "../context/SidebarContext";
import { useTheme } from "../context/ThemeContext";
const PRODUCTS_BUMP_KEY = "pos-products-bump";
const Settings = () => {
  const { isExpanded } = useContext(SidebarContext);
  const { isOwner, currentUser, login } = useAuth();
  const { theme, setTheme } = useTheme();
  const { invalidateCache } = useDataCache();
  const navigate = useNavigate();
  const [activeTab, setActiveTab] = useState("personal");
  const [currentPin, setCurrentPin] = useState(["", "", "", "", "", ""]);
  const [newPin, setNewPin] = useState(["", "", "", "", "", ""]);
  const [confirmPin, setConfirmPin] = useState(["", "", "", "", "", ""]);
  const [firstName, setFirstName] = useState("");
  const [middleInitial, setMiddleInitial] = useState("");
  const [lastName, setLastName] = useState("");
  const [email, setEmail] = useState("");
  const [contactNumber, setContactNumber] = useState("");
  const [role, setRole] = useState("");
  const [status, setStatus] = useState("");
  const [dateJoined, setDateJoined] = useState("");
  const [profileImage, setProfileImage] = useState("");
  const [permissions, setPermissions] = useState({
    posTerminal: false,
    viewTransactions: false
  });
  const [notificationsEnabled, setNotificationsEnabled] = useState(false);
  const [archives, setArchives] = useState([]);
  const [archivesLoading, setArchivesLoading] = useState(false);
  const [selectedArchiveIds, setSelectedArchiveIds] = useState([]);
  const [bulkArchivesLoading, setBulkArchivesLoading] = useState(false);
  const [showBulkRestoreModal, setShowBulkRestoreModal] = useState(false);
  const [showBulkClearModal, setShowBulkClearModal] = useState(false);
  const selectAllArchivesRef = useRef(null);
  const [profileLoading, setProfileLoading] = useState(false);
  const [pinLoading, setPinLoading] = useState(false);
  const [syncLoading, setSyncLoading] = useState(false);
  const [showSuccessModal, setShowSuccessModal] = useState(false);
  const [successMessage, setSuccessMessage] = useState("");
  const [error, setError] = useState("");
  const [showClearArchiveModal, setShowClearArchiveModal] = useState(false);
  const [clearArchivesLoading, setClearArchivesLoading] = useState(false);
  const [exportArchivesLoading, setExportArchivesLoading] = useState(false);
  const fileInputRef = useRef(null);
  const isDark = theme === "dark";
  const [appId, setAppId] = useState("");
  const [privateKey, setPrivateKey] = useState("");
  const [publicKey, setPublicKey] = useState("");
  const [gcashEnvironment, setGcashEnvironment] = useState("sandbox");
  const [merchantName, setMerchantName] = useState("POS System");
  const [paymentExpiryMinutes, setPaymentExpiryMinutes] = useState(15);
  const [webhookUrl, setWebhookUrl] = useState("");
  const [isConfigured, setIsConfigured] = useState(false);
  const [gcashLoading, setGcashLoading] = useState(false);
  const [gcashSaving, setGcashSaving] = useState(false);
  const [gcashTesting, setGcashTesting] = useState(false);
  const [gcashDeleting, setGcashDeleting] = useState(false);
  const [showPrivateKey, setShowPrivateKey] = useState(false);
  const [gcashMessage, setGcashMessage] = useState({ type: "", text: "" });
  const [lastUpdated, setLastUpdated] = useState(null);
  useEffect(() => {
    if (currentUser) {
```

```
      const fallbackFirst = currentUser.firstName || currentUser.name?.split(" ")[0] || "";
      const fallbackLast = currentUser.lastName || currentUser.name?.split(" ").slice(1).join(" ") || "";
      const fallbackMiddle = currentUser.middleInitial || (currentUser.name?.split(" ").length >= 3 ?
currentUser.name.split(" ")[1].replace(".", "") : "");
      setFirstName(fallbackFirst);
      setMiddleInitial(fallbackMiddle.replace(/[^a-zA-Z]/g, "").slice(0, 1).toUpperCase());
      setLastName(fallbackLast);
      setEmail(currentUser.email || "");
      setContactNumber(currentUser.contactNo || "");
      setRole(currentUser.role || "");
      setStatus(currentUser.status || "Active");
      const userId = currentUser._id || currentUser.id;
      setProfileImage(
        currentUser.image?.startsWith("data:image") || currentUser.profileImage?.startsWith("data:image") ?
currentUser.image || currentUser.profileImage : userId ? `${API_BASE}/api/employees/${userId}/image?v=${new
Date(currentUser.updatedAt || currentUser.lastUpdated || 0).getTime()}` : defaultAvatar
      );
      setPermissions(
        currentUser.permissions || {
          posTerminal: false,
          viewTransactions: false
        }
      );
      if (currentUser.dateJoinedActual || currentUser.dateJoined) {
        const date = new Date(
          currentUser.dateJoinedActual || currentUser.dateJoined
        );
        setDateJoined(
          date.toLocaleDateString("en-US", {
            year: "numeric",
            month: "short",
            day: "2-digit"
          })
        );
      }
    }
  }, [currentUser]);
  const fetchGcashSettings = useCallback(async () => {
    try {
      setGcashLoading(true);
      const response = await fetch(`${API_BASE}/api/merchant-settings`);
      const data = await response.json();
      if (data.success && data.data) {
        const settings = data.data;
        setAppId(settings.appId || "");
        setPublicKey(settings.publicKey || "");
        setGcashEnvironment(settings.environment || "sandbox");
        setMerchantName(settings.merchantName || "POS System");
        setPaymentExpiryMinutes(settings.paymentExpiryMinutes || 15);
        setWebhookUrl(settings.webhookUrl || "");
        setIsConfigured(true);
        setLastUpdated(settings.updatedAt || settings.createdAt);
        setPrivateKey("");
      } else {
        setIsConfigured(false);
      }
    } catch (error2) {
      console.error("Error fetching GCash settings:", error2);
      setGcashMessage({
        type: "error",
        text: "Failed to load settings. Is the backend running?"
      });
    } finally {
      setGcashLoading(false);
    }
  }, []);
  const archiveRowId = (a) => String(a._id || a.id);
  const restoreTargetIds = useMemo(
    () => selectedArchiveIds.length > 0 ? selectedArchiveIds : archives.map(archiveRowId),
    [selectedArchiveIds, archives]
  );
  const fetchArchives = useCallback(async () => {
    setArchivesLoading(true);
    try {
      const response = await fetch(`${API_BASE}/api/archive`, {
        cache: "no-store"
      });
```

```
      const data = await response.json();
      if (data.success) {
        setArchives(data.data || []);
      } else {
        setArchives([]);
      }
    } catch (error2) {
      console.error("Error fetching archives:", error2);
      setArchives([]);
    } finally {
      setArchivesLoading(false);
    }
  }, []);
  useEffect(() => {
    setSelectedArchiveIds(
      (prev) => prev.filter((id) => archives.some((a) => archiveRowId(a) === id))
    );
  }, [archives]);
  useEffect(() => {
    const el = selectAllArchivesRef.current;
    if (!el) return;
    const n = selectedArchiveIds.length;
    const t = archives.length;
    el.indeterminate = t > 0 && n > 0 && n < t;
  }, [selectedArchiveIds, archives.length]);
  const toggleArchiveRowSelect = (id) => {
    const s = String(id);
    setSelectedArchiveIds(
      (prev) => prev.includes(s) ? prev.filter((x) => x !== s) : [...prev, s]
    );
  };
  const toggleSelectAllArchives = () => {
    if (!archives.length || archivesLoading) return;
    if (selectedArchiveIds.length === archives.length) {
      setSelectedArchiveIds([]);
    } else {
      setSelectedArchiveIds(archives.map((a) => archiveRowId(a)));
    }
  };
  const handleBulkRestore = async () => {
    const ids = selectedArchiveIds.length > 0 ? [...selectedArchiveIds] : archives.map((a) =>
archiveRowId(a));
    if (ids.length === 0) return;
    setBulkArchivesLoading(true);
    setError("");
    try {
      const removed = [];
      const failed = [];
      for (const id of ids) {
        const res = await fetch(`${API_BASE}/api/archive/${id}/restore`, {
          method: "POST",
          cache: "no-store"
        });
        const data = await res.json().catch(() => ({}));
        if (data.success) {
          removed.push(id);
        } else {
          failed.push(data.message || "Restore failed");
        }
      }
      await fetchArchives();
      setSelectedArchiveIds((prev) => prev.filter((id) => !removed.includes(id)));
      setShowBulkRestoreModal(false);
      if (removed.length > 0) {
        invalidateCache("products");
        try {
          localStorage.setItem(PRODUCTS_BUMP_KEY, String(Date.now()));
        } catch (e) {
        }
        window.dispatchEvent(new CustomEvent("pos-invalidate-products"));
      }
      if (failed.length === 0) {
        setSuccessMessage(
          removed.length === 1 ? "Item restored to inventory." : `Restored ${removed.length} items.`
        );
      } else {
        setSuccessMessage(
```

```
          `Restored ${removed.length}. Failed: ${failed.slice(0, 3).join("; ")}${failed.length > 3 ?
"\u2026" : ""}`
        );
      }
      setShowSuccessModal(true);
    } catch (err) {
      console.error(err);
      setError("Could not complete restore. Try again.");
    } finally {
      setBulkArchivesLoading(false);
    }
  };
  const handleBulkClearArchives = async () => {
    if (selectedArchiveIds.length === 0) return;
    setBulkArchivesLoading(true);
    setError("");
    try {
      const ids = [...selectedArchiveIds];
      const removed = [];
      const failed = [];
      for (const id of ids) {
        const res = await fetch(`${API_BASE}/api/archive/${id}`, {
          method: "DELETE",
          cache: "no-store"
        });
        const data = await res.json().catch(() => ({}));
        if (data.success) {
          removed.push(id);
        } else {
          failed.push(data.message || "Delete failed");
        }
      }
      await fetchArchives();
      setSelectedArchiveIds((prev) => prev.filter((id) => !removed.includes(id)));
      setShowBulkClearModal(false);
      if (failed.length === 0) {
        setSuccessMessage(
          removed.length === 1 ? "Archive entry removed." : `Removed ${removed.length} archive entries.`
        );
      } else {
        setSuccessMessage(
          `Removed ${removed.length}. Failed: ${failed.slice(0, 3).join("; ")}${failed.length > 3 ? "\u2026" :
""}`
        );
      }
      setShowSuccessModal(true);
    } catch (err) {
      console.error(err);
      setError("Could not remove entries. Try again.");
    } finally {
      setBulkArchivesLoading(false);
    }
  };
  const handleGcashSave = async (e) => {
    e.preventDefault();
    setGcashMessage({ type: "", text: "" });
    if (!appId.trim() || !publicKey.trim()) {
      setGcashMessage({
        type: "error",
        text: "App ID and Public Key are required."
      });
      return;
    }
    if (!isConfigured && !privateKey.trim()) {
      setGcashMessage({
        type: "error",
        text: "Private Key is required for initial setup."
      });
      return;
    }
    if (isConfigured && !privateKey.trim()) {
      setGcashMessage({
        type: "error",
        text: "Please re-enter your Private Key to save changes."
      });
      return;
    }
```

```
  try {
    setGcashSaving(true);
    const response = await fetch(`${API_BASE}/api/merchant-settings`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        appId: appId.trim(),
        privateKey: privateKey.trim(),
        publicKey: publicKey.trim(),
        environment: gcashEnvironment,
        merchantName: merchantName.trim(),
        paymentExpiryMinutes,
        configuredBy: currentUser?._id || currentUser?.id || "",
        configuredByName: currentUser?.name || ""
      })
    });
    const data = await response.json();
    if (data.success) {
      setGcashMessage({
        type: "success",
        text: "Payment gateway configured successfully!"
      });
      setPrivateKey("");
      setIsConfigured(true);
      if (data.data?.webhookUrl) setWebhookUrl(data.data.webhookUrl);
      setLastUpdated((/* @__PURE__ */ new Date()).toISOString());
    } else {
      setGcashMessage({
        type: "error",
        text: data.message || "Failed to save settings."
      });
    }
  } catch (error2) {
    console.error("Error saving settings:", error2);
    setGcashMessage({
      type: "error",
      text: "Network error. Please try again."
    });
  } finally {
    setGcashSaving(false);
  }
};
const handleGcashTestConnection = async () => {
  setGcashMessage({ type: "", text: "" });
  if (!isConfigured) {
    setGcashMessage({
      type: "error",
      text: "Please save your credentials first before testing."
    });
    return;
  }
  try {
    setGcashTesting(true);
    const response = await fetch(`${API_BASE}/api/merchant-settings/test`, {
      method: "POST"
    });
    const data = await response.json();
    if (data.success) {
      setGcashMessage({ type: "success", text: `\u2705 ${data.message}` });
    } else {
      setGcashMessage({
        type: "error",
        text: data.message || "Connection test failed."
      });
    }
  } catch (error2) {
    setGcashMessage({ type: "error", text: "Unable to test connection." });
  } finally {
    setGcashTesting(false);
  }
};
const handleGcashDelete = async () => {
  if (!window.confirm(
    "Are you sure you want to remove the payment gateway configuration? GCash payments will be disabled."
  )) {
    return;
  }
```

```
  try {
    setGcashDeleting(true);
    const response = await fetch(`${API_BASE}/api/merchant-settings`, {
      method: "DELETE"
    });
    const data = await response.json();
    if (data.success) {
      setGcashMessage({
        type: "success",
        text: "Configuration removed successfully."
      });
      setAppId("");
      setPrivateKey("");
      setPublicKey("");
      setGcashEnvironment("sandbox");
      setMerchantName("POS System");
      setWebhookUrl("");
      setIsConfigured(false);
      setLastUpdated(null);
    } else {
      setGcashMessage({
        type: "error",
        text: data.message || "Failed to remove configuration."
      });
    }
  } catch (error2) {
    setGcashMessage({ type: "error", text: "Network error." });
  } finally {
    setGcashDeleting(false);
  }
};
useEffect(() => {
  if (activeTab !== "archives") return;
  fetchArchives();
  const intervalMs = 25e3;
  const id = setInterval(() => {
    fetchArchives();
  }, intervalMs);
  const onVisibility = () => {
    if (document.visibilityState === "visible") {
      fetchArchives();
    }
  };
  document.addEventListener("visibilitychange", onVisibility);
  return () => {
    clearInterval(id);
    document.removeEventListener("visibilitychange", onVisibility);
  };
}, [activeTab, fetchArchives]);
useEffect(() => {
  if (activeTab === "gcash") {
    fetchGcashSettings();
  }
}, [activeTab, fetchGcashSettings]);
const formatDateTime = (dateString) => {
  if (!dateString) return "-";
  const date = new Date(dateString);
  return date.toLocaleString("en-US", {
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
    hour: "2-digit",
    minute: "2-digit",
    hour12: true
  });
};
const handlePinInput = (value, index, type) => {
  if (value && !/^\d$/.test(value)) return;
  const updatedPin = [
    ...type === "current" ? currentPin : type === "new" ? newPin : confirmPin
  ];
  updatedPin[index] = value;
  if (type === "current") {
    setCurrentPin(updatedPin);
  } else if (type === "new") {
    setNewPin(updatedPin);
  } else {
```

```
      setConfirmPin(updatedPin);
    }
    if (value && index < 5) {
      const nextInput = document.getElementById(`${type}-pin-${index + 1}`);
      if (nextInput) nextInput.focus();
    }
  };
  const handleImageChange = (event) => {
    const file = event.target.files?.[0];
    if (!file) return;
    if (!file.type.startsWith("image/")) {
      setError("Please select a valid image file");
      return;
    }
    if (file.size > 2 * 1024 * 1024) {
      setError("Image must be smaller than 2MB");
      return;
    }
    const reader = new FileReader();
    reader.onloadend = () => {
      const newImageData = reader.result;
      setProfileImage(newImageData);
      setError("");
      handleAutoSaveProfileImage(newImageData);
    };
    reader.readAsDataURL(file);
  };
  const saveProfile = async (overrides = {}) => {
    if (!currentUser?._id && !currentUser?.id) {
      setError("User not found");
      return;
    }
    const mergedProfile = {
      firstName: overrides.firstName ?? firstName,
      middleInitial: overrides.middleInitial ?? middleInitial,
      lastName: overrides.lastName ?? lastName,
      email: overrides.email ?? email,
      contactNumber: overrides.contactNumber ?? contactNumber,
      profileImage: overrides.profileImage ?? profileImage,
      image: overrides.image ?? overrides.profileImage ?? profileImage
    };
    if (!mergedProfile.firstName?.trim() || !mergedProfile.lastName?.trim()) {
      setError("First name and last name are required");
      return;
    }
    if (!mergedProfile.email?.trim()) {
      setError("Email is required");
      return;
    }
    setProfileLoading(true);
    setError("");
    try {
      const response = await fetch(
        `${API_BASE}/api/employees/${currentUser._id || currentUser.id}`,
        {
          method: "PUT",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            firstName: mergedProfile.firstName.trim(),
            middleInitial: mergedProfile.middleInitial,
            lastName: mergedProfile.lastName.trim(),
            name: [mergedProfile.firstName, mergedProfile.middleInitial, mergedProfile.lastName].map((p) =>
String(p ?? "").trim()).filter(Boolean).join(" ").trim(),
            email: mergedProfile.email.trim().toLowerCase(),
            contactNo: mergedProfile.contactNumber.trim(),
            profileImage: mergedProfile.profileImage,
            image: mergedProfile.image
          })
        }
      );
      const data = await response.json();
      if (data.success) {
        setSuccessMessage("Profile updated successfully!");
        setShowSuccessModal(true);
        const apiUser = data.data || {};
        const finalProfileImage = apiUser.profileImage || apiUser.image || mergedProfile.profileImage;
        const updatedUser = {
```

```
          ...currentUser,
          ...apiUser,
          firstName: apiUser.firstName ?? mergedProfile.firstName.trim(),
          middleInitial: apiUser.middleInitial ?? mergedProfile.middleInitial,
          lastName: apiUser.lastName ?? mergedProfile.lastName.trim(),
          name: apiUser.name ?? [mergedProfile.firstName, mergedProfile.middleInitial,
mergedProfile.lastName].map((p) => String(p ?? "").trim()).filter(Boolean).join(" ").trim(),
          email: apiUser.email ?? mergedProfile.email.trim().toLowerCase(),
          contactNo: apiUser.contactNo ?? mergedProfile.contactNumber.trim(),
          profileImage: finalProfileImage,
          image: finalProfileImage
        };
        login(updatedUser);
      } else {
        setError(data.message || "Failed to update profile");
      }
    } catch (error2) {
      console.error("Error updating profile:", error2);
      setError("Failed to update profile. Please try again.");
    } finally {
      setProfileLoading(false);
    }
  };
  const handleAutoSaveProfileImage = async (imageData) => {
    await saveProfile({ profileImage: imageData, image: imageData });
  };
  const handleUpdatePin = async () => {
    const currentPinValue = currentPin.join("");
    const newPinValue = newPin.join("");
    const confirmPinValue = confirmPin.join("");
    if (currentPinValue.length !== 6) {
      setError("Please enter your current 6-digit PIN");
      return;
    }
    if (newPinValue.length !== 6) {
      setError("New PIN must be 6 digits");
      return;
    }
    if (newPinValue !== confirmPinValue) {
      setError("New PIN and Confirm PIN do not match!");
      return;
    }
    if (!currentUser?._id && !currentUser?.id) {
      setError("User not found");
      return;
    }
    setPinLoading(true);
    setError("");
    try {
      const response = await fetch(
        `${API_BASE}/api/employees/${currentUser._id || currentUser.id}/pin`,
        {
          method: "PUT",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            currentPin: currentPinValue,
            newPin: newPinValue,
            requiresPinReset: false
          })
        }
      );
      const data = await response.json();
      if (data.success) {
        setSuccessMessage("PIN updated successfully!");
        setShowSuccessModal(true);
        setCurrentPin(["", "", "", "", "", ""]);
        setNewPin(["", "", "", "", "", ""]);
        setConfirmPin(["", "", "", "", "", ""]);
        if (data.data) {
          login({ ...currentUser, ...data.data });
        }
      } else {
        setError(data.message || "Failed to update PIN");
      }
    } catch (error2) {
      console.error("Error updating PIN:", error2);
      setError("Failed to update PIN. Please try again.");
```

```
  } finally {
    setPinLoading(false);
  }
};
const handleSyncData = async () => {
  setSyncLoading(true);
  try {
    const response = await fetch(`${API_BASE}/api/sync/all`, {
      method: "POST",
      headers: { "Content-Type": "application/json" }
    });
    const data = await response.json();
    if (data.success) {
      setSuccessMessage(data.message || "Data synchronized successfully!");
      setShowSuccessModal(true);
    } else {
      setSuccessMessage("");
      alert(data.message || "Sync failed. Please try again.");
    }
  } catch (error2) {
    console.error("Sync error:", error2);
    alert(
      "Sync failed. Make sure you are connected to the internet and the server is running."
    );
  } finally {
    setSyncLoading(false);
  }
};
const computedName = useMemo(() => {
  return [firstName, middleInitial, lastName].filter(Boolean).join(" ") || currentUser?.name || "";
}, [firstName, middleInitial, lastName, currentUser?.name]);
const handleClearArchives = async () => {
  setClearArchivesLoading(true);
  setError("");
  try {
    const response = await fetch(`${API_BASE}/api/archive/all`, {
      method: "DELETE"
    });
    const data = await response.json();
    if (data.success) {
      setSuccessMessage(data.message || "All archives cleared successfully!");
      setShowSuccessModal(true);
      setArchives([]);
      setSelectedArchiveIds([]);
      setShowClearArchiveModal(false);
    } else {
      setError(data.message || "Failed to clear archives");
    }
  } catch (error2) {
    console.error("Error clearing archives:", error2);
    setError("Failed to clear archives. Please try again.");
  } finally {
    setClearArchivesLoading(false);
  }
};
const handleExportArchives = async () => {
  setExportArchivesLoading(true);
  try {
    const response = await fetch(
      `${API_BASE}/api/archive?limit=1000000000&sortBy=date-desc`
    );
    const data = await response.json();
    if (data.success && data.data) {
      const archivesToExport = data.data;
      if (archivesToExport.length === 0) {
        setError("No archives available to export.");
        return;
      }
      const headers = [
        "Archive Number",
        "SKU",
        "Item Name",
        "Category",
        "Quantity",
        "Price",
        "Reason",
        "Date & Time",
```

```
          "Archived By",
          "Notes"
        ];
        const csvContent = [
          headers.join(","),
          ...archivesToExport.map((archive, index) => {
            return [
              archivesToExport.length - index,
              `"${archive.sku || ""}"`,
              `"${archive.itemName || ""}"`,
              `"${archive.category || ""}"`,
              archive.quantity || 0,
              archive.itemPrice || 0,
              `"${archive.reason || ""} ${archive.returnReason || ""}"`,
              `"${formatDateTime(archive.archivedAt)}"`,
              `"${archive.archivedBy || ""}"`,
              `"${archive.notes || ""}"`
            ].join(",");
          })
        ].join("\n");
        const blob = new Blob([csvContent], {
          type: "text/csv;charset=utf-8;"
        });
        const link = document.createElement("a");
        const url = URL.createObjectURL(blob);
        link.setAttribute("href", url);
        link.setAttribute(
          "download",
          `archives_export_${(/* @__PURE__ */ new Date()).toISOString().split("T")[0]}.csv`
        );
        document.body.appendChild(link);
        link.click();
        document.body.removeChild(link);
      } else {
        setError("Failed to fetch archives for export.");
      }
    } catch (error2) {
      console.error("Error exporting archives:", error2);
      setError("Failed to export archives. Please try again.");
    } finally {
      setExportArchivesLoading(false);
    }
  };
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `p-8 min-h-screen ${theme === "dark" ? "bg-[#1E1B18]" : "bg-gray-50"}`
    },
    /* @__PURE__ */ React.createElement(Header, { pageName: "Settings", showBorder: false }),
    isOwner() && /* @__PURE__ */ React.createElement("div", { className: "flex gap-3 mb-6" }, /* @__PURE__ */
React.createElement(
      "button",
      {
        onClick: () => setActiveTab("personal"),
        className: `px-6 py-3 font-bold rounded-xl transition-all shadow-md ${activeTab === "personal" ?
`text-[#AD7F65] border-b-4 border-[#AD7F65] ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}` : `${theme ===
"dark" ? "bg-[#2A2724] text-gray-300 border border-gray-700" : "bg-white text-gray-800 border border-gray-
200"}`}`
      },
      "Account"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => setActiveTab("archives"),
        className: `px-6 py-3 font-bold rounded-xl transition-all shadow-md ${activeTab === "archives" ?
`text-[#AD7F65] border-b-4 border-[#AD7F65] ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}` : `${theme ===
"dark" ? "bg-[#2A2724] text-gray-300 border border-gray-700" : "bg-white text-gray-800 border border-gray-
200"}`}`
      },
      "Archives"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => setActiveTab("gcash"),
        className: `px-6 py-3 font-bold rounded-xl transition-all shadow-md ${activeTab === "gcash" ? `text-
[#AD7F65] border-b-4 border-[#AD7F65] ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}` : `${theme ===
"dark" ? "bg-[#2A2724] text-gray-300 border border-gray-700" : "bg-white text-gray-800 border border-gray-
```

```
200"}`}`
      },
      "GCash Configuration"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => navigate("/manage-data"),
        className: `px-6 py-3 font-bold rounded-xl transition-all shadow-md flex items-center gap-2 ${theme
=== "dark" ? "bg-[#2A2724] text-gray-300 border border-gray-700 hover:border-[#AD7F65]" : "bg-white text-
gray-800 border border-gray-200 hover:border-[#AD7F65]"}`
      },
      /* @__PURE__ */ React.createElement(FaDatabase, { className: "w-4 h-4" }),
      "Manage Data"
    )),
    activeTab === "gcash" ? gcashLoading ? /* @__PURE__ */ React.createElement("div", { className: "flex
items-center justify-center min-h-[60vh]" }, /* @__PURE__ */ React.createElement(FaSpinner, { className: "w-8
h-8 text-[#8B7355] animate-spin" })) : /* @__PURE__ */ React.createElement("div", { className: "max-w-2xl mx-
auto mt-6" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-3xl shadow-lg overflow-hidden ${isDark ? "bg-[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "px-8 pt-8 pb-4" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-4 mb-1" }, /* @__PURE__ */
React.createElement("div", { className: "w-12 h-12 rounded-2xl bg-gradient-to-br from-[#C2A68C] via-[#AD7F65]
to-[#76462B] flex items-center justify-center shadow-md" }, /* @__PURE__ */ React.createElement(FaCog,
{ className: "w-5 h-5 text-white" })), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement(
        "h3",
        {
          className: `text-xl font-bold ${isDark ? "text-white" : "text-gray-900"}`
        },
        "Payment Gateway"
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-sm ${isDark ? "text-gray-400" : "text-gray-500"}`
        },
        isConfigured ? /* @__PURE__ */ React.createElement(React.Fragment, null, "Connected \xB7", " ", /*
@__PURE__ */ React.createElement("span", { className: "text-green-500 font-medium" },
gcashEnvironment.toUpperCase()), lastUpdated && ` \xB7 Updated ${new
Date(lastUpdated).toLocaleDateString()}`) : "Enter your GCash for Business/PayMongo credentials below"
      )), isConfigured && /* @__PURE__ */ React.createElement("div", { className: "ml-auto" }, /* @__PURE__
*/ React.createElement("span", { className: "flex items-center gap-1.5 px-3 py-1 rounded-full bg-green-100
text-green-700 text-xs font-semibold" }, /* @__PURE__ */ React.createElement("span", { className: "w-2 h-2
rounded-full bg-green-500 animate-pulse" }), "Active")))),
      /* @__PURE__ */ React.createElement("div", { className: "px-8 pb-4" }, /* @__PURE__ */
React.createElement(
        "div",
        {
          className: `rounded-xl p-4 flex items-start gap-3 ${isDark ? "bg-amber-900/15 border border-amber-
800/40" : "bg-amber-50 border border-amber-100"}`
        },
        /* @__PURE__ */ React.createElement(
          FaInfoCircle,
          {
            className: `w-5 h-5 shrink-0 mt-0.5 ${isDark ? "text-amber-400" : "text-amber-600"}`
          }
        ),
        /* @__PURE__ */ React.createElement(
          "p",
          {
            className: `text-sm leading-relaxed ${isDark ? "text-amber-100/90" : "text-amber-900/90"}`
          },
          "If you want to configure GCash, contact the developer for this."
        )
      )),
      /* @__PURE__ */ React.createElement("div", { className: "px-8 pb-4" }, /* @__PURE__ */
React.createElement(
        "div",
        {
          className: `rounded-xl p-4 flex items-start gap-3 ${isDark ? "bg-blue-900/10 border border-blue-
900/30" : "bg-blue-50/70 border border-blue-100"}`
        },
        /* @__PURE__ */ React.createElement("div", { className: "w-8 h-8 rounded-lg bg-gradient-to-br from-
[#42A5F5] to-[#1565C0] flex items-center justify-center shrink-0 mt-0.5" }, /* @__PURE__ */
React.createElement(FaShieldAlt, { className: "w-3.5 h-3.5 text-white" })),
```

```
        /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
          "p",
          {
            className: `text-sm font-semibold mb-0.5 ${isDark ? "text-blue-300" : "text-blue-800"}`
          },
          "Security"
        ), /* @__PURE__ */ React.createElement(
          "p",
          {
            className: `text-xs leading-relaxed ${isDark ? "text-blue-400/70" : "text-blue-600/80"}`
          },
          "Your private key is encrypted with AES-256-GCM before storage and is never exposed to the front-
end after saving. All payment communications use HTTPS. Webhook signatures are verified using HMAC-SHA256."
        ))
      )),
      /* @__PURE__ */ React.createElement("form", { onSubmit: handleGcashSave }, /* @__PURE__ */
React.createElement("div", { className: "px-8 pb-8 space-y-5" }, /* @__PURE__ */ React.createElement("div",
null, /* @__PURE__ */ React.createElement(
        "label",
        {
          className: `block text-sm font-semibold mb-2 ${isDark ? "text-gray-300" : "text-gray-700"}`
        },
        "App ID / API Key ID"
      ), /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "text",
          value: appId,
          onChange: (e) => setAppId(e.target.value),
          placeholder: "[REDACTED]",
          className: `w-full px-4 py-3 text-sm border rounded-xl focus:outline-none focus:ring-2 focus:ring-
[#42A5F5] transition-all ${isDark ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" : "bg-
gray-50 border-gray-200 text-gray-900 placeholder-gray-400"}`,
          required: true
        }
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs mt-1.5 ${isDark ? "text-gray-500" : "text-gray-400"}`
        },
        "Your PayMongo public API key"
      )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "label",
        {
          className: `block text-sm font-semibold mb-2 ${isDark ? "text-gray-300" : "text-gray-700"}`
        },
        "Secret Key (Private Key)"
      ), /* @__PURE__ */ React.createElement("div", { className: "relative" }, /* @__PURE__ */
React.createElement(
        "input",
        {
          type: showPrivateKey ? "text" : "password",
          value: privateKey,
          onChange: (e) => setPrivateKey(e.target.value),
          placeholder: isConfigured ?
"\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022\u2022 (re-enter to
update)" : "[REDACTED]",
          className: `w-full px-4 py-3 pr-12 text-sm border rounded-xl focus:outline-none focus:ring-2
focus:ring-[#42A5F5] transition-all ${isDark ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-
500" : "bg-gray-50 border-gray-200 text-gray-900 placeholder-gray-400"}`,
          required: !isConfigured
        }
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: () => setShowPrivateKey(!showPrivateKey),
          className: `absolute right-4 top-1/2 -translate-y-1/2 p-1 rounded-lg transition-colors ${isDark ?
"text-gray-400 hover:text-gray-300 hover:bg-gray-700" : "text-gray-400 hover:text-gray-600 hover:bg-gray-
100"}`
        },
        showPrivateKey ? /* @__PURE__ */ React.createElement(FaEyeSlash, { className: "w-4 h-4" }) : /*
@__PURE__ */ React.createElement(FaEye, { className: "w-4 h-4" })
      )), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs mt-1.5 ${isDark ? "text-gray-500" : "text-gray-400"}`
```

```
        },
        "Your PayMongo secret key \u2014 encrypted before storage, never exposed after save"
      )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "label",
        {
          className: `block text-sm font-semibold mb-2 ${isDark ? "text-gray-300" : "text-gray-700"}`
        },
        "Public Key"
      ), /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "text",
          value: publicKey,
          onChange: (e) => setPublicKey(e.target.value),
          placeholder: "[REDACTED]",
          className: `w-full px-4 py-3 text-sm border rounded-xl focus:outline-none focus:ring-2 focus:ring-
[#42A5F5] transition-all ${isDark ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" : "bg-
gray-50 border-gray-200 text-gray-900 placeholder-gray-400"}`,
          required: true
        }
      )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "label",
        {
          className: `block text-sm font-semibold mb-2 ${isDark ? "text-gray-300" : "text-gray-700"}`
        },
        "Environment"
      ), /* @__PURE__ */ React.createElement("div", { className: "flex gap-3" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          type: "button",
          onClick: () => setGcashEnvironment("sandbox"),
          className: `flex-1 py-3 rounded-xl text-sm font-bold transition-all ${gcashEnvironment ===
"sandbox" ? "bg-gradient-to-r from-yellow-400 to-orange-400 text-white shadow-md shadow-yellow-200/40" :
isDark ? "bg-[#1E1B18] border border-gray-600 text-gray-400 hover:border-gray-500" : "bg-gray-100 border
border-gray-200 text-gray-500 hover:bg-gray-150"}`
        },
        "SandBox"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: () => setGcashEnvironment("production"),
          className: `flex-1 py-3 rounded-xl text-sm font-bold transition-all ${gcashEnvironment ===
"production" ? "bg-gradient-to-r from-green-400 to-emerald-500 text-white shadow-md shadow-green-200/40" :
isDark ? "bg-[#1E1B18] border border-gray-600 text-gray-400 hover:border-gray-500" : "bg-gray-100 border
border-gray-200 text-gray-500 hover:bg-gray-150"}`
        },
        "Production"
      ))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "label",
        {
          className: `block text-sm font-semibold mb-2 ${isDark ? "text-gray-300" : "text-gray-700"}`
        },
        "Merchant Display Name"
      ), /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "text",
          value: merchantName,
          onChange: (e) => setMerchantName(e.target.value),
          placeholder: "Your Store Name",
          className: `w-full px-4 py-3 text-sm border rounded-xl focus:outline-none focus:ring-2 focus:ring-
[#42A5F5] transition-all ${isDark ? "bg-[#1E1B18] border-gray-600 text-white placeholder-gray-500" : "bg-
gray-50 border-gray-200 text-gray-900 placeholder-gray-400"}`
        }
      )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("div",
{ className: "flex items-center gap-3 flex-wrap" }, /* @__PURE__ */ React.createElement(
        "label",
        {
          className: `text-sm font-semibold ${isDark ? "text-gray-300" : "text-gray-700"}`
        },
        "Payment Expiry (minutes)"
      ), /* @__PURE__ */ React.createElement(
        "span",
        {
          className: `text-xs ${isDark ? "text-gray-500" : "text-gray-400"}`
```

```
        },
        "QR codes expires after this many minutes (5\u201360)"
      )), /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "number",
          value: paymentExpiryMinutes,
          onChange: (e) => setPaymentExpiryMinutes(
            Math.min(
              60,
              Math.max(5, parseInt(e.target.value) || 15)
            )
          ),
          min: "5",
          max: "60",
          className: `w-28 mt-2 px-4 py-3 text-sm border rounded-xl focus:outline-none focus:ring-2
focus:ring-[#42A5F5] transition-all ${isDark ? "bg-[#1E1B18] border-gray-600 text-white" : "bg-gray-50
border-gray-200 text-gray-900"}`
        }
      )), webhookUrl && /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "label",
        {
          className: `block text-sm font-semibold mb-2 ${isDark ? "text-gray-300" : "text-gray-700"}`
        },
        /* @__PURE__ */ React.createElement(FaLink, { className: "inline w-3 h-3 mr-1.5" }),
        " Webhook URL (auto-generated)"
      ), /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `w-full px-4 py-3 border rounded-xl font-mono text-xs break-all ${isDark ? "bg-[#1E1B18]
border-gray-600 text-gray-400" : "bg-gray-50 border-gray-200 text-gray-600"}`
        },
        webhookUrl
      ), /* @__PURE__ */ React.createElement(
        "p",
        {
          className: `text-xs mt-1.5 ${isDark ? "text-gray-500" : "text-gray-400"}`
        },
        "Register this URL in your PayMongo dashboard under Webhooks \u2192 Source Chargeable"
      )), gcashMessage.text && /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `rounded-xl p-3.5 flex items-center gap-2.5 ${gcashMessage.type === "success" ? isDark ?
"bg-green-900/20 text-green-400 border border-green-800" : "bg-green-50 text-green-700 border border-green-
200" : isDark ? "bg-red-900/20 text-red-400 border border-red-800" : "bg-red-50 text-red-700 border border-
red-200"}`
        },
        gcashMessage.type === "success" ? /* @__PURE__ */ React.createElement(FaCheckCircle, { className: "w-
4 h-4 flex-shrink-0" }) : /* @__PURE__ */ React.createElement(FaTimesCircle, { className: "w-4 h-4 flex-
shrink-0" }),
        /* @__PURE__ */ React.createElement("span", { className: "text-sm font-medium" }, gcashMessage.text)
      ), /* @__PURE__ */ React.createElement("div", { className: "flex flex-col items-center gap-3 pt-2" },
/* @__PURE__ */ React.createElement(
        "button",
        {
          type: "submit",
          disabled: gcashSaving,
          className: "w-64 py-3.5 rounded-2xl font-bold text-white bg-gradient-to-r from-green-500 to-
emerald-600 hover:from-green-600 hover:to-emerald-700 transition-all disabled:opacity-50 flex items-center
justify-center gap-2 shadow-lg shadow-green-200/30 hover:shadow-green-300/40"
        },
        gcashSaving ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(FaSpinner, { className: "w-4 h-4 animate-spin" }), " ", "Saving...") : isConfigured ?
"Update Configuration" : "Save Configuration"
      ), isConfigured && /* @__PURE__ */ React.createElement("div", { className: "flex gap-3" }, /* @__PURE__
*/ React.createElement(
        "button",
        {
          type: "button",
          onClick: handleGcashTestConnection,
          disabled: gcashTesting,
          className: `py-2.5 px-6 rounded-xl font-semibold text-sm transition-all flex items-center gap-2
${isDark ? "bg-[#1E1B18] text-gray-300 hover:bg-[#322f2c] border border-gray-600" : "bg-gray-100 text-gray-
600 hover:bg-gray-200 border border-gray-200"} disabled:opacity-50`
        },
        gcashTesting ? /* @__PURE__ */ React.createElement(FaSpinner, { className: "w-3.5 h-3.5 animate-
spin" }) : "Test Connection"
```

```
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: handleGcashDelete,
          disabled: gcashDeleting,
          className: "py-2.5 px-5 rounded-xl font-semibold text-sm text-red-500 hover:bg-red-50 border
border-red-200 transition-all disabled:opacity-50 flex items-center gap-2"
        },
        /* @__PURE__ */ React.createElement(FaTrash, { className: "w-3.5 h-3.5" }),
        " Remove"
      )))))
    )) : activeTab === "archives" ? /* @__PURE__ */ React.createElement("div", { className: "space-y-4" }, /*
@__PURE__ */ React.createElement("div", { className: "flex flex-wrap justify-end items-center pr-2 gap-3" },
archives.length > 0 && /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(
      "button",
      {
        type: "button",
        onClick: () => {
          setError("");
          setShowBulkRestoreModal(true);
        },
        className: "px-4 py-2 rounded-lg text-sm font-bold text-white shadow transition-all hover:opacity-95
flex items-center gap-2 disabled:opacity-50 disabled:cursor-not-allowed",
        style: {
          background: "linear-gradient(135deg, #10B981 0%, #059669 100%)"
        },
        disabled: archivesLoading || bulkArchivesLoading || exportArchivesLoading || clearArchivesLoading
      },
      /* @__PURE__ */ React.createElement(FaUndo, { className: "w-4 h-4" }),
      selectedArchiveIds.length === 0 ? "Restore all" : "Restore selected"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        type: "button",
        onClick: () => {
          setError("");
          if (selectedArchiveIds.length > 0) {
            setShowBulkClearModal(true);
          } else {
            setShowClearArchiveModal(true);
          }
        },
        className: "bg-red-600/90 hover:bg-red-700 text-white px-4 py-2 rounded-lg text-sm font-bold shadow
transition-colors flex items-center gap-2 disabled:opacity-50",
        disabled: archivesLoading || bulkArchivesLoading || exportArchivesLoading || clearArchivesLoading
      },
      /* @__PURE__ */ React.createElement(FaTrash, { className: "w-4 h-4" }),
      selectedArchiveIds.length === 0 ? "Clear all data" : "Clear selected"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: handleExportArchives,
        className: "bg-[#AD7F65] hover:bg-[#8e654e] text-white px-4 py-2 rounded-lg text-sm font-bold shadow
transition-colors flex items-center gap-2",
        disabled: exportArchivesLoading || clearArchivesLoading || bulkArchivesLoading
      },
      /* @__PURE__ */ React.createElement(FaDownload, { className: "w-4 h-4" }),
      exportArchivesLoading ? "Exporting..." : "Export Data"
    ))), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-2xl shadow-lg overflow-hidden ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "overflow-x-auto" }), /* @__PURE__ */
React.createElement("table", { className: "w-full" }), /* @__PURE__ */ React.createElement(
        "thead",
        {
          className: theme === "dark" ? "bg-[#1E1B18]" : "bg-gray-50"
        },
        /* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */ React.createElement(
          "th",
          {
            className: `px-3 py-3 w-12 text-left text-xs font-medium uppercase tracking-wider ${theme ===
"dark" ? "text-gray-400" : "text-gray-500"}`
          },
```

```
          /* @__PURE__ */ React.createElement(
            "input",
            {
              ref: selectAllArchivesRef,
              type: "checkbox",
              className: "w-4 h-4 rounded border-gray-400 text-[#AD7F65] focus:ring-[#AD7F65]",
              checked: archives.length > 0 && selectedArchiveIds.length === archives.length,
              onChange: toggleSelectAllArchives,
              disabled: archivesLoading || bulkArchivesLoading || archives.length === 0,
              title: "Select all"
            }
          )
        ), /* @__PURE__ */ React.createElement(
          "th",
          {
            className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider ${theme === "dark" ?
"text-gray-400" : "text-gray-500"}`
          },
          "Archive Number"
        ), /* @__PURE__ */ React.createElement(
          "th",
          {
            className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider ${theme === "dark" ?
"text-gray-400" : "text-gray-500"}`
          },
          "Item Image"
        ), /* @__PURE__ */ React.createElement(
          "th",
          {
            className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider ${theme === "dark" ?
"text-gray-400" : "text-gray-500"}`
          },
          "SKU"
        ), /* @__PURE__ */ React.createElement(
          "th",
          {
            className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider ${theme === "dark" ?
"text-gray-400" : "text-gray-500"}`
          },
          "Item Name"
        ), /* @__PURE__ */ React.createElement(
          "th",
          {
            className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider ${theme === "dark" ?
"text-gray-400" : "text-gray-500"}`
          },
          "Category"
        ), /* @__PURE__ */ React.createElement(
          "th",
          {
            className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider ${theme === "dark" ?
"text-gray-400" : "text-gray-500"}`
          },
          "Quantity"
        ), /* @__PURE__ */ React.createElement(
          "th",
          {
            className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider ${theme === "dark" ?
"text-gray-400" : "text-gray-500"}`
          },
          "Price"
        ), /* @__PURE__ */ React.createElement(
          "th",
          {
            className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider ${theme === "dark" ?
"text-gray-400" : "text-gray-500"}`
          },
          "Reason"
        ), /* @__PURE__ */ React.createElement(
          "th",
          {
            className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider ${theme === "dark" ?
"text-gray-400" : "text-gray-500"}`
          },
          "Date & Time"
        ), /* @__PURE__ */ React.createElement(
          "th",
```

```
          {
            className: `px-4 py-3 text-left text-xs font-medium uppercase tracking-wider ${theme === "dark" ?
"text-gray-400" : "text-gray-500"}`
          },
          "Archived By"
        ))
      ), /* @__PURE__ */ React.createElement(
        "tbody",
        {
          className: `divide-y ${theme === "dark" ? "bg-[#2A2724] divide-gray-700" : "bg-white divide-gray-
200"}`
        },
        archivesLoading ? /* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */
React.createElement(
          "td",
          {
            colSpan: "11",
            className: "px-4 py-8 text-center text-gray-500"
          },
          /* @__PURE__ */ React.createElement("div", { className: "flex flex-col items-center justify-
center" }, /* @__PURE__ */ React.createElement("div", { className: "inline-block animate-spin rounded-full h-
8 w-8 border-b-2 border-[#8B7355] mb-2" }), /* @__PURE__ */ React.createElement("span", null, "Loading..."))
        )) : archives.length === 0 ? /* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */
React.createElement(
          "td",
          {
            colSpan: "11",
            className: "px-4 py-8 text-center text-gray-500"
          },
          /* @__PURE__ */ React.createElement("div", { className: "flex flex-col items-center justify-
center" }, /* @__PURE__ */ React.createElement("div", { className: "w-24 h-24 flex items-center justify-
center mb-4" }, /* @__PURE__ */ React.createElement(FaBox, { className: "w-full h-full text-gray-300" })), /*
@__PURE__ */ React.createElement("p", { className: "text-gray-400 text-lg" }, "No Archive yet"))
        )) : archives.map((archive, index) => {
          const archiveNumber = archives.length - index;
          return /* @__PURE__ */ React.createElement(
            "tr",
            {
              key: archive._id || archive.id,
              className: `hover:${theme === "dark" ? "bg-[#1E1B18]" : "bg-gray-50"}`
            },
            /* @__PURE__ */ React.createElement(
              "td",
              {
                className: "px-3 py-3 whitespace-nowrap",
                onClick: (e) => e.stopPropagation()
              },
              /* @__PURE__ */ React.createElement(
                "input",
                {
                  type: "checkbox",
                  className: "w-4 h-4 rounded border-gray-400 text-[#AD7F65] focus:ring-[#AD7F65]",
                  checked: selectedArchiveIds.includes(archiveRowId(archive)),
                  onChange: () => toggleArchiveRowSelect(archiveRowId(archive)),
                  disabled: archivesLoading || bulkArchivesLoading
                }
              )
            ),
            /* @__PURE__ */ React.createElement(
              "td",
              {
                className: `px-4 py-3 whitespace-nowrap text-sm font-bold ${theme === "dark" ? "text-white" :
"text-gray-900"}`
              },
              "#",
              archiveNumber
            ),
            /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 whitespace-nowrap" },
archive.itemImage ? /* @__PURE__ */ React.createElement(
              "img",
              {
                src: archive.itemImage,
                alt: archive.itemName,
                className: "w-12 h-12 object-cover rounded",
                onError: (e) => {
                  e.target.src = "https://via.placeholder.com/50";
                }
```

```
              }
            ) : /* @__PURE__ */ React.createElement("div", { className: "w-12 h-12 bg-gray-200 rounded flex
items-center justify-center text-gray-400" }, /* @__PURE__ */ React.createElement("span", { className: "text-
xs" }, "No Image"))),
            /* @__PURE__ */ React.createElement(
              "td",
              {
                className: `px-4 py-3 whitespace-nowrap text-sm font-mono ${theme === "dark" ? "text-gray-
300" : "text-gray-700"}`
              },
              archive.sku
            ),
            /* @__PURE__ */ React.createElement(
              "td",
              {
                className: `px-4 py-3 whitespace-nowrap text-sm ${theme === "dark" ? "text-white" : "text-
gray-900"}`
              },
              /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("div",
{ className: "font-medium" }, archive.itemName), archive.variant && /* @__PURE__ */ React.createElement("div",
{ className: "text-xs text-gray-500" }, "(", archive.variant, ")"), archive.selectedSize && /* @__PURE__ */
React.createElement("div", { className: "text-xs text-gray-500" }, "Size: ", archive.selectedSize),
archive.brandName && /* @__PURE__ */ React.createElement("div", { className: "text-xs text-gray-500" },
"Brand: ", archive.brandName))
            ),
            /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 whitespace-nowrap" }, /*
@__PURE__ */ React.createElement("span", { className: "px-2 py-1 bg-gray-100 text-gray-700 rounded text-xs
font-medium" }, archive.category)),
            /* @__PURE__ */ React.createElement(
              "td",
              {
                className: `px-4 py-3 whitespace-nowrap text-sm ${theme === "dark" ? "text-white" : "text-
gray-900"}`
              },
              archive.quantity
            ),
            /* @__PURE__ */ React.createElement(
              "td",
              {
                className: `px-4 py-3 whitespace-nowrap text-sm font-medium ${theme === "dark" ? "text-
white" : "text-gray-900"}`
              },
              "\u20B1",
              parseFloat(archive.itemPrice || 0).toFixed(2)
            ),
            /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 whitespace-nowrap" }, /*
@__PURE__ */ React.createElement("span", { className: "inline-flex items-center px-2 py-1 rounded text-xs
font-medium bg-red-100 text-red-700" }, archive.reason), archive.returnReason && /* @__PURE__ */
React.createElement("div", { className: "mt-1 text-xs text-gray-600" }, archive.returnReason), archive.notes
&& /* @__PURE__ */ React.createElement("div", { className: "mt-1 text-xs text-gray-500 italic" },
archive.notes)),
            /* @__PURE__ */ React.createElement(
              "td",
              {
                className: `px-4 py-3 whitespace-nowrap text-sm ${theme === "dark" ? "text-white" : "text-
gray-900"}`
              },
              formatDateTime(archive.archivedAt)
            ),
            /* @__PURE__ */ React.createElement(
              "td",
              {
                className: `px-4 py-3 whitespace-nowrap text-sm ${theme === "dark" ? "text-white" : "text-
gray-900"}`
              },
              archive.archivedBy || "N/A"
            )
          );
        })
      )))
    )) : /* @__PURE__ */ React.createElement("div", { className: "max-w-4xl mx-auto space-y-6" }, /*
@__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-3xl shadow-lg p-8 ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3 mb-6" }, /* @__PURE__
```

```
*/ React.createElement("div", { className: "w-10 h-10 rounded-xl bg-[#AD7F65] flex items-center justify-
center text-white" }, /* @__PURE__ */ React.createElement(FaUser, { className: "w-5 h-5" })), /* @__PURE__ */
React.createElement(
        "h3",
        {
          className: `text-lg font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Profile Information"
      )),
      /* @__PURE__ */ React.createElement("div", { className: "flex items-start gap-8 mb-8" }, /* @__PURE__
*/ React.createElement(
        "div",
        {
          className: "relative group cursor-pointer",
          onClick: () => fileInputRef.current?.click()
        },
        /* @__PURE__ */ React.createElement("div", { className: "w-32 h-32 rounded-full overflow-hidden
border-4 border-white shadow-lg relative" }, /* @__PURE__ */ React.createElement(
          "img",
          {
            src: profileImage || defaultAvatar,
            alt: computedName,
            onError: (e) => {
              e.target.onerror = null;
              e.target.src = defaultAvatar;
            },
            className: "w-full h-full object-cover transition-opacity group-hover:opacity-75"
          }
        ), /* @__PURE__ */ React.createElement("div", { className: "absolute inset-0 flex items-center
justify-center opacity-0 group-hover:opacity-100 transition-opacity bg-black/30" }, /* @__PURE__ */
React.createElement(FaCamera, { className: "text-white w-8 h-8" }))),
        /* @__PURE__ */ React.createElement(
          "input",
          {
            type: "file",
            accept: "image/*",
            ref: fileInputRef,
            onChange: handleImageChange,
            className: "hidden"
          }
        )
      ), /* @__PURE__ */ React.createElement("div", { className: "flex-1" }, /* @__PURE__ */
React.createElement(
        "h2",
        {
          className: `text-2xl font-bold mb-1 ${theme === "dark" ? "text-white" : "text-gray-900"}`
        },
        computedName
      ), /* @__PURE__ */ React.createElement("p", { className: "text-[#AD7F65] font-medium mb-3" }, role), /*
@__PURE__ */ React.createElement("div", { className: "mb-4" }, /* @__PURE__ */ React.createElement("label",
{ className: "text-xs font-semibold text-gray-400 uppercase tracking-wider mb-2 block" }, "Permissions:"), /*
@__PURE__ */ React.createElement("div", { className: "flex gap-2 flex-wrap" }, role?.toLowerCase() ===
"owner" ? /* @__PURE__ */ React.createElement("span", { className: "px-3 py-1 rounded-full text-xs font-
medium border border-gray-200 text-gray-600 bg-gray-50" }, "All") : /* @__PURE__ */
React.createElement(React.Fragment, null, permissions.posTerminal && /* @__PURE__ */
React.createElement("span", { className: "px-3 py-1 rounded-full text-xs font-medium border border-gray-200
text-gray-600 bg-gray-50" }, "POS Terminal"), permissions.viewTransactions && /* @__PURE__ */
React.createElement("span", { className: "px-3 py-1 rounded-full text-xs font-medium border border-gray-200
text-gray-600 bg-gray-50" }, "View Transactions"), !permissions.posTerminal && !permissions.viewTransactions
&& /* @__PURE__ */ React.createElement("span", { className: "text-xs text-gray-400 italic" }, "No active
permissions")))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement("label", { className: "text-xs font-semibold text-gray-400 uppercase tracking-wider mb-2
block" }, "Status:"), /* @__PURE__ */ React.createElement(
        "span",
        {
          className: `inline-block px-3 py-1 rounded-lg text-xs font-bold ${status === "Active" ? "bg-green-
100 text-green-700" : "bg-red-100 text-red-700"}`
        },
        status
      )))),
      /* @__PURE__ */ React.createElement(
        "h4",
        {
          className: `text-base font-bold mb-4 ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Personal Details"
      ),
```

```
      /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-x-8 gap-y-5" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label", { className:
"text-sm font-medium text-gray-500 mb-1 block" }, "First Name"), /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "text",
          value: firstName,
          onChange: (e) => setFirstName(e.target.value),
          className: `w-full text-base font-semibold border-b-2 border-gray-200 focus:border-[#AD7F65]
focus:outline-none py-1 bg-transparent ${theme === "dark" ? "text-white" : "text-gray-800"}`
        }
      )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-sm font-medium text-gray-500 mb-1 block" }, "Middle Initial"), /* @__PURE__ */
React.createElement(
        "input",
        {
          type: "text",
          value: middleInitial,
          onChange: (e) => setMiddleInitial(
            e.target.value.replace(/[^a-zA-Z]/g, "").slice(0, 1).toUpperCase()
          ),
          className: `w-full text-base font-semibold border-b-2 border-gray-200 focus:border-[#AD7F65]
focus:outline-none py-1 bg-transparent ${theme === "dark" ? "text-white" : "text-gray-800"}`
        }
      )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-sm font-medium text-gray-500 mb-1 block" }, "Last Name"), /* @__PURE__ */
React.createElement(
        "input",
        {
          type: "text",
          value: lastName,
          onChange: (e) => setLastName(e.target.value),
          className: `w-full text-base font-semibold border-b-2 border-gray-200 focus:border-[#AD7F65]
focus:outline-none py-1 bg-transparent ${theme === "dark" ? "text-white" : "text-gray-800"}`
        }
      )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-sm font-medium text-gray-500 mb-1 block" }, "Email"), /* @__PURE__ */ React.createElement(
        "input",
        {
          type: "email",
          value: email,
          onChange: (e) => setEmail(e.target.value),
          className: `w-full text-base font-semibold border-b-2 border-gray-200 focus:border-[#AD7F65]
focus:outline-none py-1 bg-transparent ${theme === "dark" ? "text-white" : "text-gray-800"}`
        }
      )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-sm font-medium text-gray-500 mb-1 block" }, "Contact number"), /* @__PURE__ */
React.createElement(
        "input",
        {
          type: "text",
          value: contactNumber,
          onChange: (e) => setContactNumber(e.target.value),
          className: `w-full text-base font-semibold border-b-2 border-gray-200 focus:border-[#AD7F65]
focus:outline-none py-1 bg-transparent ${theme === "dark" ? "text-white" : "text-gray-800"}`
        }
      )), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("label",
{ className: "text-sm font-medium text-gray-500 mb-1 block" }, "Date Joined"), /* @__PURE__ */
React.createElement(
        "p",
        {
          className: `text-base font-semibold py-1 ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        dateJoined || "N/A"
      )), /* @__PURE__ */ React.createElement("div", { className: "col-span-2" }, /* @__PURE__ */
React.createElement("label", { className: "text-sm font-medium text-gray-500 mb-1 block" }, "Position"), /*
@__PURE__ */ React.createElement(
        "p",
        {
          className: `text-base font-semibold py-1 ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Employee - ",
        role
      ))),
      error && /* @__PURE__ */ React.createElement("div", { className: "mt-4 rounded-xl p-3.5 flex items-
center gap-2.5 bg-red-50 text-red-700 border border-red-200" }, /* @__PURE__ */
React.createElement(FaTimesCircle, { className: "w-4 h-4 flex-shrink-0" }), /* @__PURE__ */
```

```
React.createElement("span", { className: "text-sm font-medium" }, error)),
      /* @__PURE__ */ React.createElement("div", { className: "mt-6 flex justify-end" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: () => saveProfile(),
          disabled: profileLoading,
          className: "px-8 py-3 rounded-xl font-bold text-white bg-[#1B89CD] hover:from-green-600 hover:to-
emerald-700 transition-all disabled:opacity-50 flex items-center justify-center gap-2 shadow-lg shadow-green-
200/30 hover:shadow-green-300/40"
        },
        profileLoading ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(FaSpinner, { className: "w-4 h-4 animate-spin" }), " Saving...") : "Save Changes"
      ))
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-3xl shadow-lg p-8 ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3 mb-6" }, /* @__PURE__
*/ React.createElement("div", { className: "w-10 h-10 rounded-xl bg-[#AD7F65] flex items-center justify-
center text-white" }, /* @__PURE__ */ React.createElement(FaPalette, { className: "w-5 h-5" })), /* @__PURE__
*/ React.createElement(
        "h3",
        {
          className: `text-lg font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Appearance"
      )),
      /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-2 gap-6" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: () => setTheme("dark"),
          className: `flex flex-col items-center justify-center p-4 rounded-2xl border-2 transition-all
${theme === "dark" ? "border-[#AD7F65] shadow-md" : "border-gray-200 hover:border-gray-300"}`
        },
        /* @__PURE__ */ React.createElement("div", { className: "w-full h-16 rounded-xl bg-[#1E1B18] shadow-
inner mb-2" }),
        /* @__PURE__ */ React.createElement(
          "span",
          {
            className: `text-sm font-medium ${theme === "dark" ? "text-white" : "text-gray-700"}`
          },
          "Dark Mode"
        )
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: () => setTheme("light"),
          className: `flex flex-col items-center justify-center p-4 rounded-2xl border-2 transition-all
${theme === "light" ? "border-[#AD7F65] shadow-md" : "border-gray-200 hover:border-gray-300"}`
        },
        /* @__PURE__ */ React.createElement("div", { className: "w-full h-16 rounded-xl bg-white shadow-inner
mb-2 border border-gray-100" }),
        /* @__PURE__ */ React.createElement(
          "span",
          {
            className: `text-sm font-medium ${theme === "dark" ? "text-white" : "text-gray-700"}`
          },
          "Light Mode"
        )
      ))
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-3xl shadow-lg p-8 ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between" }, /*
@__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("div", { className: "flex
items-center gap-3 mb-2" }, /* @__PURE__ */ React.createElement("div", { className: "w-10 h-10 rounded-xl bg-
[#AD7F65] flex items-center justify-center text-white" }, /* @__PURE__ */ React.createElement(FaSync,
{ className: "w-5 h-5" })), /* @__PURE__ */ React.createElement(
        "h3",
        {
          className: `text-lg font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
```

```
        "Data Synchronization"
      )), /* @__PURE__ */ React.createElement("p", { className: "text-sm text-gray-500 ml-13" }, "Keep your
data up to date across all devices. Manual sync ensures changes are saved.")), /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: handleSyncData,
          disabled: syncLoading,
          className: "px-6 py-2.5 rounded-xl text-white font-bold bg-[#10B981] hover:bg-[#059669] transition-
all shadow-md disabled:opacity-50 whitespace-nowrap"
        },
        syncLoading ? "Syncing..." : "Sync Now"
      ))
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-3xl shadow-lg p-8 ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3 mb-6" }, /* @__PURE__
*/ React.createElement("div", { className: "w-10 h-10 rounded-xl bg-[#AD7F65] flex items-center justify-
center text-white" }, /* @__PURE__ */ React.createElement(FaKey, { className: "w-5 h-5" })), /* @__PURE__ */
React.createElement(
        "h3",
        {
          className: `text-lg font-bold ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Change PIN"
      )),
      error && /* @__PURE__ */ React.createElement("div", { className: "mb-4 p-3 bg-red-100 border border-
red-300 rounded-xl text-sm text-red-700" }, error),
      /* @__PURE__ */ React.createElement("div", { className: "space-y-6 mb-6" }, /* @__PURE__ */
React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "label",
        {
          className: `block text-sm font-medium mb-2 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}`
        },
        "Current PIN"
      ), /* @__PURE__ */ React.createElement("div", { className: "flex gap-2" }, [0, 1, 2, 3, 4, 5].map(
        (i) => /* @__PURE__ */ React.createElement(
          "input",
          {
            key: i,
            id: `current-pin-${i}`,
            type: "password",
            maxLength: 1,
            value: currentPin[i],
            onChange: (e) => handlePinInput(e.target.value, i, "current"),
            disabled: pinLoading,
            className: `w-12 h-12 text-center text-lg font-bold rounded-xl border-2 shadow-sm focus:border-
[#AD7F65] focus:shadow-md transition-all outline-none disabled:opacity-50 ${theme === "dark" ? "bg-[#1E1B18]
border-gray-600 text-white focus:bg-[#352F2A]" : "bg-gray-50 border-gray-200 text-gray-900 focus:bg-white"}`
          }
        )
      ))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "label",
        {
          className: `block text-sm font-medium mb-2 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}`
        },
        "New PIN"
      ), /* @__PURE__ */ React.createElement("div", { className: "flex gap-2" }, [0, 1, 2, 3, 4, 5].map(
        (i) => /* @__PURE__ */ React.createElement(
          "input",
          {
            key: i,
            id: `new-pin-${i}`,
            type: "password",
            maxLength: 1,
            value: newPin[i],
            onChange: (e) => handlePinInput(e.target.value, i, "new"),
            disabled: pinLoading,
            className: `w-12 h-12 text-center text-lg font-bold rounded-xl border-2 shadow-sm focus:border-
[#AD7F65] focus:shadow-md transition-all outline-none disabled:opacity-50 ${theme === "dark" ? "bg-[#1E1B18]
border-gray-600 text-white focus:bg-[#352F2A]" : "bg-gray-50 border-gray-200 text-gray-900 focus:bg-white"}`
          }
        )
      ))), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "label",
```

```
        {
          className: `block text-sm font-medium mb-2 ${theme === "dark" ? "text-gray-300" : "text-gray-700"}`
        },
        "Confirm PIN"
      ), /* @__PURE__ */ React.createElement("div", { className: "flex gap-2" }, [0, 1, 2, 3, 4, 5].map(
        (i) => /* @__PURE__ */ React.createElement(
          "input",
          {
            key: i,
            id: `confirm-pin-${i}`,
            type: "password",
            maxLength: 1,
            value: confirmPin[i],
            onChange: (e) => handlePinInput(e.target.value, i, "confirm"),
            disabled: pinLoading,
            className: `w-12 h-12 text-center text-lg font-bold rounded-xl border-2 shadow-sm focus:border-
[#AD7F65] focus:shadow-md transition-all outline-none disabled:opacity-50 ${theme === "dark" ? "bg-[#1E1B18]
border-gray-600 text-white focus:bg-[#352F2A]" : "bg-gray-50 border-gray-200 text-gray-900 focus:bg-white"}`
          }
        )
      ))),
      /* @__PURE__ */ React.createElement("div", { className: "flex justify-end" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: handleUpdatePin,
          disabled: pinLoading,
          className: "px-8 py-2.5 rounded-xl text-white font-bold bg-[#1B89CD] hover:bg-[#2563EB] transition-
all shadow-md disabled:opacity-50"
        },
        pinLoading ? "Changing PIN..." : "Change PIN"
      ))
    )),
    showClearArchiveModal && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-[60]
flex items-center justify-center p-4 backdrop-blur-sm bg-black/20" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `p-6 rounded-2xl shadow-xl w-full max-w-sm ${theme === "dark" ? "bg-[#2A2724] text-white" :
"bg-white text-gray-900"}`
      },
      /* @__PURE__ */ React.createElement("h3", { className: "text-xl font-bold mb-4 text-center" }, "Clear
Archives"),
      /* @__PURE__ */ React.createElement("p", { className: "text-sm text-center mb-6" }, "Are you sure you
want to permanently delete all archive data? This action cannot be undone."),
      error && /* @__PURE__ */ React.createElement("p", { className: "text-red-500 text-sm text-center mb-
4" }, error),
      /* @__PURE__ */ React.createElement("div", { className: "flex gap-4" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          onClick: () => {
            setShowClearArchiveModal(false);
            setError("");
          },
          className: `flex-1 py-2 rounded-lg font-bold border transition-colors ${theme === "dark" ? "border-
gray-600 hover:bg-gray-700 text-white" : "border-gray-300 hover:bg-gray-100 text-gray-700"}`,
          disabled: clearArchivesLoading
        },
        "Cancel"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: handleClearArchives,
          className: "flex-1 py-2 rounded-lg font-bold bg-red-600 text-white hover:bg-red-700 transition-
colors flex justify-center items-center",
          disabled: clearArchivesLoading
        },
        clearArchivesLoading ? /* @__PURE__ */ React.createElement("div", { className: "w-5 h-5 border-2
border-white border-t-transparent rounded-full animate-spin" }) : "Delete All"
      ))
    )),
    showBulkRestoreModal && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-[61]
flex items-center justify-center p-4 backdrop-blur-sm bg-black/20" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `p-6 rounded-2xl shadow-xl w-full max-w-md ${theme === "dark" ? "bg-[#2A2724] text-white" :
"bg-white text-gray-900"}`
```

```
      },
      /* @__PURE__ */ React.createElement("h3", { className: "text-xl font-bold mb-3 text-center" }, "Restore
to inventory?"),
      /* @__PURE__ */ React.createElement("p", { className: "text-sm text-center mb-4 opacity-90" },
selectedArchiveIds.length, " item(s) will return to Product & Stocks and be removed from this archive list."),
      /* @__PURE__ */ React.createElement(
        "ul",
        {
          className: `text-xs max-h-28 overflow-y-auto mb-4 space-y-1 ${theme === "dark" ? "text-gray-300" :
"text-gray-600"}`
        },
        archives.filter((a) => selectedArchiveIds.includes(archiveRowId(a))).slice(0, 8).map(
          (a) => /* @__PURE__ */ React.createElement("li", { key: archiveRowId(a) }, /* @__PURE__ */
React.createElement("span", { className: "font-mono" }, a.sku), " \u2014 ", a.itemName)
        ),
        selectedArchiveIds.length > 8 && /* @__PURE__ */ React.createElement("li", { className: "italic" },
"+", selectedArchiveIds.length - 8, " more\u2026")
      ),
      error && /* @__PURE__ */ React.createElement("p", { className: "text-red-500 text-sm text-center mb-
3" }, error),
      /* @__PURE__ */ React.createElement("div", { className: "flex gap-4" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          type: "button",
          onClick: () => {
            setShowBulkRestoreModal(false);
            setError("");
          },
          className: `flex-1 py-2 rounded-lg font-bold border transition-colors ${theme === "dark" ? "border-
gray-600 hover:bg-gray-700 text-white" : "border-gray-300 hover:bg-gray-100 text-gray-700"}`,
          disabled: bulkArchivesLoading
        },
        "Cancel"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: handleBulkRestore,
          className: "flex-1 py-2 rounded-lg font-bold text-white transition-all hover:opacity-95 flex
justify-center items-center",
          style: {
            background: "linear-gradient(135deg, #10B981 0%, #059669 100%)"
          },
          disabled: bulkArchivesLoading
        },
        bulkArchivesLoading ? /* @__PURE__ */ React.createElement("div", { className: "w-5 h-5 border-2
border-white border-t-transparent rounded-full animate-spin" }) : "Restore"
      ))
    )),
    showBulkClearModal && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-[61] flex
items-center justify-center p-4 backdrop-blur-sm bg-black/20" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `p-6 rounded-2xl shadow-xl w-full max-w-md ${theme === "dark" ? "bg-[#2A2724] text-white" :
"bg-white text-gray-900"}`
      },
      /* @__PURE__ */ React.createElement("h3", { className: "text-xl font-bold mb-3 text-center" }, "Remove
archive entries?"),
      /* @__PURE__ */ React.createElement("p", { className: "text-sm text-center mb-4 opacity-90" },
"Permanently delete ", selectedArchiveIds.length, " row(s) from the archive history. This does not delete
live products."),
      error && /* @__PURE__ */ React.createElement("p", { className: "text-red-500 text-sm text-center mb-
3" }, error),
      /* @__PURE__ */ React.createElement("div", { className: "flex gap-4" }, /* @__PURE__ */
React.createElement(
        "button",
        {
          type: "button",
          onClick: () => {
            setShowBulkClearModal(false);
            setError("");
          },
          className: `flex-1 py-2 rounded-lg font-bold border transition-colors ${theme === "dark" ? "border-
gray-600 hover:bg-gray-700 text-white" : "border-gray-300 hover:bg-gray-100 text-gray-700"}`,
          disabled: bulkArchivesLoading
        },
```

```
        "Cancel"
      ), /* @__PURE__ */ React.createElement(
        "button",
        {
          type: "button",
          onClick: handleBulkClearArchives,
          className: "flex-1 py-2 rounded-lg font-bold bg-amber-800 text-white hover:bg-amber-900 transition-
colors flex justify-center items-center",
          disabled: bulkArchivesLoading
        },
        bulkArchivesLoading ? /* @__PURE__ */ React.createElement("div", { className: "w-5 h-5 border-2
border-white border-t-transparent rounded-full animate-spin" }) : "Remove"
      ))
    )),
    /* @__PURE__ */ React.createElement(
      SuccessModal,
      {
        isOpen: showSuccessModal,
        onClose: () => {
          setShowSuccessModal(false);
          setError("");
        },
        message: successMessage
      }
    )
  );
};
export default memo(Settings);
```

FILE: pos/frontend/src/pages/StaffSelection.jsx
==========

```
import { useState, memo } from "react";
import { useNavigate } from "react-router-dom";
import logo from "../assets/logo.png";
import bgImage from "../assets/bg.png";
import defaultAvatar from "../assets/default.jpeg";
import { FaUser } from "react-icons/fa";
import { API_BASE_URL } from "../config/api";
const StaffSelection = () => {
  const [firstName, setFirstName] = useState("");
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const navigate = useNavigate();
  const handleProceed = async () => {
    if (!firstName.trim()) {
      setError("Please enter your first name");
      return;
    }
    setError("");
    setLoading(true);
    try {
      const response = await fetch(`${API_BASE_URL}/api/employees/search/${encodeURIComponent(firstName)}`);
      const data = await response.json();
      if (data.success && data.data.length > 0) {
        const firstNameLower = firstName.toLowerCase().trim();
        const employee = data.data.find((emp) => {
          const empFirstName = emp.firstName?.toLowerCase().trim();
          const nameFirstWord = emp.name?.toLowerCase().trim().split(/\s+/)[0];
          return empFirstName === firstNameLower || nameFirstWord === firstNameLower;
        });
        if (!employee) {
          setError("Employee not found. Please check your first name. Enter your registered first name");
          setLoading(false);
          return;
        }
        if (employee.status !== "Active") {
          setError("Your account is inactive. Please contact administrator.");
          setLoading(false);
          return;
        }
        const staffInfo = {
          _id: employee._id,
          id: employee._id,
          name: employee.name,
          role: employee.role,
```

```
          image: employee.profileImage || defaultAvatar,
          permissions: employee.permissions
        };
        try {
          sessionStorage.setItem("selectedStaff", JSON.stringify(staffInfo));
        } catch (storageError) {
          console.warn("Unable to cache selected staff", storageError);
        }
        navigate("/pin", { state: { staff: staffInfo } });
      } else {
        const firstNameLower = firstName.toLowerCase().trim();
        if (firstNameLower === "owner" || firstNameLower.includes("owner")) {
          const ownerInfo = {
            id: 3,
            name: "owner",
            email: "owner",
            role: "Owner",
            image: defaultAvatar,
            permissions: {
              posTerminal: true,
              inventory: true,
              viewTransactions: true,
              generateReports: true
            }
          };
          try {
            sessionStorage.setItem("selectedStaff", JSON.stringify(ownerInfo));
          } catch (storageError) {
            console.warn("Unable to cache selected staff", storageError);
          }
          navigate("/pin", { state: { staff: ownerInfo } });
        } else {
          setError("Employee not found. Please check your first name.");
        }
      }
    } catch (error2) {
      console.error("Error searching for employee:", error2);
      setError("Failed to connect to server. Please make sure the backend is running.");
    } finally {
      setLoading(false);
    }
  };
  const handleKeyPress = (event) => {
    if (event.key === "Enter") {
      handleProceed();
    }
  };
  const handleReturn = () => {
    navigate(-1);
  };
  return /* @__PURE__ */ React.createElement("div", { className: "flex w-screen h-screen overflow-hidden
flex-col lg:flex-row" }, /* @__PURE__ */ React.createElement("div", { className: "flex-1 relative flex items-
center justify-center p-8 bg-white min-h-[40vh] lg:min-h-full" }, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: "absolute inset-8 lg:inset-8 rounded-[20px] bg-cover bg-center",
      style: {
        backgroundImage: `linear-gradient(rgba(139, 115, 85, 0.7), rgba(139, 115, 85, 0.7)), url(${bgImage})`
      }
    }
  ), /* @__PURE__ */ React.createElement("div", { className: "relative z-10 text-center p-12 flex items-
center justify-center" }, /* @__PURE__ */ React.createElement(
    "img",
    {
      src: logo,
      alt: "Create Your Style",
      className: "max-w-[80%] lg:max-w-[70%] h-auto object-contain drop-shadow-[2px_2px_8px_rgba(0,0,0,0.3)]"
    }
  ))), /* @__PURE__ */ React.createElement("div", { className: "flex-1 bg-white flex items-center justify-
center p-8 min-h-[60vh] lg:min-h-full" }, /* @__PURE__ */ React.createElement("div", { className: "w-full
max-w-[400px]" }, /* @__PURE__ */ React.createElement("div", { className: "text-center mb-8" }, /* @__PURE__
*/ React.createElement("h2", { className: "text-4xl font-bold text-[#333] mb-2" }, "Welcome to CYSPOS!"), /*
@__PURE__ */ React.createElement("div", { className: "w-[100px] h-1 bg-[#8B7355] mx-auto" })), /* @__PURE__
*/ React.createElement("div", { className: "mb-8 flex justify-center" }, /* @__PURE__ */
React.createElement("div", { className: "w-32 h-32 rounded-full bg-[#C8A882] flex items-center justify-
center" }, /* @__PURE__ */ React.createElement(FaUser, { className: "text-white text-6xl" }))), /* @__PURE__
*/ React.createElement("div", { className: "mb-8" }, /* @__PURE__ */ React.createElement("label", { className:
```

```
"block text-gray-700 text-sm mb-2" }, "Enter your first name"), /* @__PURE__ */ React.createElement("div",
{ className: "relative" }, /* @__PURE__ */ React.createElement("div", { className: "absolute left-3 top-1/2
transform -translate-y-1/2" }, /* @__PURE__ */ React.createElement(FaUser, { className: "text-gray-400" })),
/* @__PURE__ */ React.createElement(
    "input",
    {
      type: "text",
      value: firstName,
      onChange: (e) => {
        setFirstName(e.target.value);
        setError("");
      },
      onKeyPress: handleKeyPress,
      placeholder: "Username",
      className: "w-full pl-10 pr-4 py-3 border border-gray-300 rounded-lg focus:outline-none focus:border-
[#8B7355] text-gray-700",
      autoFocus: true
    }
  )), error && /* @__PURE__ */ React.createElement("div", { className: "mt-2 text-xs text-red-600" }, error),
/* @__PURE__ */ React.createElement("div", { className: "mt-2 text-xs text-gray-500" }, "Enter your
registered first name")), /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */
React.createElement(
    "button",
    {
      className: "w-full bg-[#8B7355] text-white border-none rounded-lg py-3 text-lg font-semibold cursor-
pointer transition-all duration-300 hover:bg-[#6d5a43] hover:shadow-[0_4px_12px_rgba(139,115,85,0.3)]
active:translate-y-0 disabled:opacity-50 disabled:cursor-not-allowed",
      onClick: handleProceed,
      disabled: loading
    },
    loading ? "Searching..." : "Proceed"
  )))));
};
export default memo(StaffSelection);
```

FILE: pos/frontend/src/pages/terminal.jsx
==========

```
import { memo, useCallback, useEffect, useMemo, useRef, useState } from "react";
import { flushSync } from "react-dom";
import Pagination from "../components/inventory/Pagination";
import Header from "../components/shared/header";
import CashPaymentModal from "../components/terminal/CashPaymentModal";
import CheckoutConfirmationModal from "../components/terminal/CheckoutConfirmationModal";
import DiscountModal from "../components/terminal/DiscountModal";
import DuplicateItemModal from "../components/terminal/DuplicateItemModal";
import OrderSummary from "../components/terminal/OrderSummary";
import ProductCard from "../components/terminal/ProductCard";
import ProductDetailsModal from "../components/terminal/ProductDetailsModal";
import QRCodePaymentModal from "../components/terminal/QRCodePaymentModal";
import RemoveItemPinModal from "../components/terminal/RemoveItemPinModal";
import { API_BASE_URL } from "../config/api";
import toast from "react-hot-toast";
import { useAuth } from "../context/AuthContext";
import { useDataCache } from "../context/DataCacheContext";
import { useTheme } from "../context/ThemeContext";
import accessoriesIcon from "../assets/inventory-icons/accesories.svg";
import allIcon from "../assets/inventory-icons/ALL.svg";
import bottomsIcon from "../assets/inventory-icons/Bottoms.svg";
import dressesIcon from "../assets/inventory-icons/dresses.svg";
import makeupIcon from "../assets/inventory-icons/make up.svg";
import shoesIcon from "../assets/inventory-icons/shoe.svg";
import topIcon from "../assets/inventory-icons/Top.svg";
const CART_ITEM_LIMIT = 100;
const OVERRIDE_ROLES = ["Manager", "Owner"];
const Terminal = () => {
  const { theme } = useTheme();
  const { currentUser } = useAuth();
  const { cache, getCachedData, setCachedData, isCacheValid, invalidateCache } = useDataCache();
  const userId = currentUser?._id || currentUser?.id || currentUser?.email || "guest";
  const cartId = userId;
  const [products, setProducts] = useState(
    () => getCachedData("products") || []
  );
  const [selectedMainCategory, setSelectedMainCategory] = useState("All");
  const [selectedSubCategory, setSelectedSubCategory] = useState("");
```

```
  const [searchQuery, setSearchQuery] = useState("");
  const [cart, setCart] = useState([]);
  const [discountAmount, setDiscountAmount] = useState("");
  const [selectedDiscounts, setSelectedDiscounts] = useState([]);
  const [loading, setLoading] = useState(false);
  const [expandedProductId, setExpandedProductId] = useState(null);
  const [productQuantities, setProductQuantities] = useState({});
  const [productSizes, setProductSizes] = useState({});
  const [productVariants, setProductVariants] = useState({});
  const [currentPage, setCurrentPage] = useState(1);
  const [showCheckoutModal, setShowCheckoutModal] = useState(false);
  const [showCashPaymentModal, setShowCashPaymentModal] = useState(false);
  const [showQRPaymentModal, setShowQRPaymentModal] = useState(false);
  const [showDiscountModal, setShowDiscountModal] = useState(false);
  const [cartReadyForSync, setCartReadyForSync] = useState(false);
  const [showRemoveItemModal, setShowRemoveItemModal] = useState(false);
  const [itemToRemove, setItemToRemove] = useState(null);
  const [showDuplicateModal, setShowDuplicateModal] = useState(false);
  const [pendingDuplicateItem, setPendingDuplicateItem] = useState(null);
  const [showProductModal, setShowProductModal] = useState(false);
  const [selectedProduct, setSelectedProduct] = useState(null);
  const [showItemLimitOverrideModal, setShowItemLimitOverrideModal] = useState(false);
  const [itemLimitOverrideApproved, setItemLimitOverrideApproved] = useState(false);
  const [sortOption, setSortOption] = useState("newest");
  const [isProcessingTransaction, setIsProcessingTransaction] = useState(false);
  const productsBeforeTxnRef = useRef(null);
  const itemsPerPage = 10;
  const toastBr = useMemo(
    () => ({
      success: (msg) => toast.success(msg),
      error: (msg) => toast.error(msg)
    }),
    []
  );
  const getCartItemCount = useCallback(
    (items = cart) => (Array.isArray(items) ? items : []).reduce(
      (sum, item) => sum + (parseInt(item?.quantity, 10) || 0),
      0
    ),
    [cart]
  );
  const requestItemLimitOverride = useCallback(() => {
    setShowItemLimitOverrideModal(true);
  }, []);
  const ensureCanAddItems = useCallback(
    (additionalItems, referenceCart = cart) => {
      const nextAdd = Math.max(0, parseInt(additionalItems, 10) || 0);
      if (!nextAdd) return true;
      if (itemLimitOverrideApproved) return true;
      const totalAfterAdd = getCartItemCount(referenceCart) + nextAdd;
      if (totalAfterAdd <= CART_ITEM_LIMIT) return true;
      toastBr.error(
        `Maximum ${CART_ITEM_LIMIT} items per transaction reached. Request manager override to continue.`
      );
      requestItemLimitOverride();
      return false;
    },
    [cart, getCartItemCount, itemLimitOverrideApproved, requestItemLimitOverride, toastBr]
  );
  const resolveItemSize = (item = {}) => {
    if (item.selectedSize) return item.selectedSize;
    if (item.sizes && typeof item.sizes === "object") {
      return Object.keys(item.sizes)[0] || "";
    }
    if (item.size) return item.size;
    return "";
  };
  const normalizeCartItem = (item = {}) => ({
    ...item,
    _id: item._id || item.productId || item.id,
    productId: item.productId || item._id || item.id,
    itemPrice: item.itemPrice || item.price || 0,
    quantity: item.quantity || 1,
    selectedSize: resolveItemSize(item),
    selectedVariation: item.selectedVariation || item.variant || "",
    sizes: item.sizes || productSizes[item._id || item.productId] || productSizes[item.productId || item._id]
|| {}
```

```
  });
  const defaultCategories = [
    { name: "All", icon: allIcon }
  ];
  const categoryStructureOptions = [
    "Tops",
    "Bottoms",
    "Dresses",
    "Outerwear",
    "Beverages",
    "Snacks",
    "Meals",
    "Desserts",
    "Ingredients",
    "Other",
    "Face",
    "Eyes",
    "Lips",
    "Nails",
    "SkinCare",
    "Others",
    "Jewelry",
    "Bags",
    "Head Wear",
    "Glasses/Sunglasses",
    "Sneakers",
    "Boots",
    "Sandals",
    "Daily Essentials",
    "Personal Care",
    "Home Essentials"
  ];
  const [mainCategories, setMainCategories] = useState(defaultCategories);
  const [subCategories, setSubCategories] = useState([]);
  const categoryIconMap = {
    Tops: topIcon,
    Bottoms: bottomsIcon,
    Dresses: dressesIcon,
    Makeup: makeupIcon,
    Essentials: accessoriesIcon,
    Shoes: shoesIcon
  };
  const categoryStructure = {
    "Apparel - Men": ["Tops", "Bottoms", "Outerwear"],
    "Apparel - Women": ["Tops", "Bottoms", "Dresses", "Outerwear"],
    "Apparel - Kids": ["Tops", "Bottoms", "Dresses", "Outerwear"],
    "Apparel - Unisex": ["Tops", "Bottoms", "Dresses", "Outerwear"],
    "Foods": ["Beverages", "Snacks", "Meals", "Desserts", "Ingredients", "Other"],
    "Makeup": ["Face", "Eyes", "Lips", "Nails", "SkinCare", "Others"],
    "Accessories": ["Jewelry", "Bags", "Head Wear", "Glasses/Sunglasses", "Others"],
    "Shoes": ["Sneakers", "Boots", "Sandals", "Others"],
    "Essentials": ["Daily Essentials", "Personal Care", "Home Essentials", "Others"]
  };
  const getParentCategoryForLegacy = (name) => {
    for (const [parent, subs] of Object.entries(categoryStructure)) {
      if (subs.includes(name)) return parent;
    }
    return null;
  };
  const fetchCategories = async () => {
    try {
      const response = await fetch(`${API_BASE_URL}/api/categories`);
      const data = await response.json();
      if (data.success && Array.isArray(data.data)) {
        const activeDbCats = data.data.filter((cat) => cat.status === "active");
        const dbMainCats = activeDbCats.filter((c) => c.type !== "subcategory"
&& !categoryStructureOptions.includes(c.name)).map((cat) => ({ name: cat.name, icon: categoryIconMap[cat.name]
|| allIcon }));
        const rawSubCats = activeDbCats.filter((c) => c.type === "subcategory" ||
categoryStructureOptions.includes(c.name));
        const dbSubCats = [];
        for (const [parent, subs] of Object.entries(categoryStructure)) {
          subs.forEach((subName) => {
            const dbSub = rawSubCats.find((s) => s.name === subName);
            if (dbSub) {
              dbSubCats.push({ name: dbSub.name, parentCategory: parent });
            }
```

```
        });
      }
      rawSubCats.forEach((sub) => {
        if (!categoryStructureOptions.includes(sub.name)) {
          dbSubCats.push({ name: sub.name, parentCategory: sub.parentCategory });
        }
      });
      const mergedMainCategories = [...defaultCategories];
      const defaultNames = new Set(defaultCategories.map((c) => c.name));
      dbMainCats.forEach((cat) => {
        if (!defaultNames.has(cat.name)) {
          mergedMainCategories.push(cat);
        }
      });
      setMainCategories(mergedMainCategories);
      setSubCategories(dbSubCats);
    }
  } catch (error) {
    console.error("Error fetching categories:", error);
    setMainCategories(defaultCategories);
    setSubCategories([]);
  }
};
useEffect(() => {
  const cachedProducts = getCachedData("products");
  if (!cachedProducts || !isCacheValid("products")) {
    fetchProducts();
  } else {
    setProducts(cachedProducts);
  }
  fetchCategories();
}, []);
useEffect(() => {
  if (Array.isArray(cache?.products) && cache.products.length) {
    setProducts(cache.products);
  }
}, [cache?.products]);
useEffect(() => {
  let isMounted = true;
  const loadCart = async () => {
    try {
      const response = await fetch(
        `${API_BASE_URL}/api/cart/${encodeURIComponent(cartId)}`
      );
      const data = await response.json();
      if (isMounted && data.success) {
        if (data.data?.items?.length) {
          setCart(data.data.items.map(normalizeCartItem));
        } else {
          try {
            const savedCart = localStorage.getItem(`pos_cart_${cartId}`);
            if (savedCart) {
              const parsedCart = JSON.parse(savedCart);
              if (Array.isArray(parsedCart) && parsedCart.length > 0) {
                setCart(
                  parsedCart.map(
                    (item) => normalizeCartItem({
                      ...item,
                      sizes: item.sizes || productSizes[item._id || item.productId] || {}
                    })
                  )
                );
              }
            }
          } catch (localError) {
            console.warn(
              "Unable to load saved cart from localStorage",
              localError
            );
          }
        }
      }
    } catch (error) {
      console.warn(
        "Unable to load cart from server, trying localStorage",
        error
      );
```

```
      try {
        const savedCart = localStorage.getItem(`pos_cart_${cartId}`);
        if (savedCart) {
          const parsedCart = JSON.parse(savedCart);
          if (Array.isArray(parsedCart)) {
            setCart(
              parsedCart.map(
                (item) => normalizeCartItem({
                  ...item,
                  sizes: item.sizes || productSizes[item._id || item.productId] || {}
                })
              )
            );
          }
        }
      } catch (localError) {
        console.warn(
          "Unable to load saved cart from localStorage",
          localError
        );
      }
    } finally {
      if (isMounted) {
        setCartReadyForSync(true);
      }
    }
  };
  loadCart();
  return () => {
    isMounted = false;
  };
}, [cartId]);
useEffect(() => {
  if (!cartReadyForSync) return;
  const saveCartToServer = async () => {
    try {
      await fetch(
        `${API_BASE_URL}/api/cart/${encodeURIComponent(cartId)}`,
        {
          method: "PUT",
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify({ items: cart })
        }
      );
    } catch (error) {
      console.warn("Unable to save cart to server", error);
    }
  };
  saveCartToServer();
}, [cart, cartId, cartReadyForSync]);
useEffect(() => {
  if (!cartReadyForSync) return;
  try {
    localStorage.setItem(`pos_cart_${cartId}`, JSON.stringify(cart));
  } catch (error) {
    console.warn("Unable to persist cart", error);
  }
}, [cart, cartId, cartReadyForSync]);
useEffect(() => {
  if (cart.length === 0 && itemLimitOverrideApproved) {
    setItemLimitOverrideApproved(false);
  }
}, [cart.length, itemLimitOverrideApproved]);
const filteredProducts = useMemo(() => {
  let filtered = products;
  filtered = filtered.filter(
    (product) => product.displayInTerminal !== false
  );
  filtered = filtered.filter((product) => {
    if (product.sizes && typeof product.sizes === "object" && Object.keys(product.sizes).length > 0) {
      const hasStock = Object.values(product.sizes).some((sizeData) => {
        if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
          return sizeData.quantity > 0;
        }
        return (typeof sizeData === "number" ? sizeData : 0) > 0;
```

```
        });
        return hasStock;
      }
      return (product.currentStock || 0) > 0;
    });
    if (selectedMainCategory !== "All") {
      const matchMain = selectedMainCategory.toLowerCase().trim();
      if (selectedSubCategory && selectedSubCategory !== "All") {
        const matchSub = selectedSubCategory.toLowerCase().trim();
        filtered = filtered.filter((product) => {
          const pCat = (product.category || "").toLowerCase().trim();
          const pSubCat = (product.subCategory || "").toLowerCase().trim();
          return pCat === matchMain && pSubCat === matchSub;
        });
      } else {
        filtered = filtered.filter((product) => {
          const pCat = (product.category || "").toLowerCase().trim();
          return pCat === matchMain;
        });
      }
    }
    if (searchQuery) {
      filtered = filtered.filter(
        (product) => product.itemName.toLowerCase().includes(searchQuery.toLowerCase()) ||
product.sku.toLowerCase().includes(searchQuery.toLowerCase()) ||
product.category.toLowerCase().includes(searchQuery.toLowerCase())
      );
    }
    filtered = [...filtered].sort((a, b) => {
      switch (sortOption) {
        case "a-z":
          return (a.itemName || "").localeCompare(b.itemName || "");
        case "z-a":
          return (b.itemName || "").localeCompare(a.itemName || "");
        case "oldest":
          return new Date(a.dateAdded || a.createdAt || 0) - new Date(b.dateAdded || b.createdAt || 0);
        case "newest":
        default:
          return new Date(b.dateAdded || b.createdAt || 0) - new Date(a.dateAdded || a.createdAt || 0);
      }
    });
    return filtered;
  }, [products, selectedMainCategory, selectedSubCategory, searchQuery, sortOption]);
  useEffect(() => {
    setCurrentPage(1);
  }, [selectedMainCategory, selectedSubCategory, searchQuery]);
  const fetchProducts = async (background = false) => {
    try {
      if (!background) setLoading(true);
      const minimalRes = await fetch(`${API_BASE_URL}/api/products?fields=minimal`);
      const minimalData = await minimalRes.json();
      if (minimalData.success) {
        setProducts(minimalData.data);
        if (!background) setLoading(false);
        try {
          const fullRes = await fetch(`${API_BASE_URL}/api/products`);
          const fullData = await fullRes.json();
          if (fullData.success) {
            setProducts(fullData.data);
            setCachedData("products", fullData.data);
          }
        } catch (imgErr) {
          console.warn("Background image fetch failed:", imgErr);
        }
      }
    } catch (error) {
      console.error("Error fetching products:", error);
    } finally {
      if (!background) setLoading(false);
    }
  };
  const productMap = useMemo(() => {
    const map = /* @__PURE__ */ new Map();
    products.forEach((p) => {
      const pId = String(p._id || p.id);
      if (pId) {
        map.set(pId, p);
```

```
    }
  });
  return map;
}, [products]);
const handleProductClick = useCallback(async (product) => {
  setSelectedProduct(product);
  setShowProductModal(true);
  if (!productQuantities[product._id]) {
    setProductQuantities({ ...productQuantities, [product._id]: 1 });
  }
  if (!productSizes[product._id]) {
    setProductSizes({ ...productSizes, [product._id]: "" });
  }
  if (!productVariants[product._id]) {
    setProductVariants({ ...productVariants, [product._id]: "" });
  }
  try {
    const response = await fetch(
      `${API_BASE_URL}/api/products/${product._id}`
    );
    const data = await response.json();
    if (data.success && data.data) {
      setSelectedProduct(data.data);
    }
  } catch (error) {
    console.error("Error fetching product details:", error);
  }
}, [productQuantities, productSizes, productVariants]);
const handleCloseProductModal = useCallback(() => {
  setShowProductModal(false);
  setSelectedProduct(null);
}, []);
const updateProductQuantity = (productId, delta) => {
  const product = products.find((p) => p._id === productId);
  if (!product) return;
  const currentQuantity = productQuantities[productId] || 1;
  const newQuantity = currentQuantity + delta;
  const selectedSize = productSizes[productId];
  const selectedVariant = productVariants[productId] || "";
  let availableStock = 0;
  const getVariantQty = (variantData) => {
    if (typeof variantData === "number") return variantData;
    if (typeof variantData === "object" && variantData !== null) {
      return variantData.quantity || 0;
    }
    return 0;
  };
  if (product.sizes && typeof product.sizes === "object" && selectedSize) {
    const sizeData = product.sizes[selectedSize];
    if (typeof sizeData === "object" && sizeData !== null && sizeData.variants && selectedVariant) {
      const variantData = sizeData.variants[selectedVariant];
      availableStock = getVariantQty(variantData);
    } else if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
      availableStock = sizeData.quantity;
    } else if (typeof sizeData === "number") {
      availableStock = sizeData;
    }
  } else {
    availableStock = product.currentStock || 0;
  }
  const clampedQuantity = Math.max(1, Math.min(newQuantity, availableStock));
  setProductQuantities({
    ...productQuantities,
    [productId]: clampedQuantity
  });
};
const setProductQuantityTo = (productId, nextQty) => {
  const product = products.find((p) => p._id === productId);
  if (!product) return;
  const selectedSize = productSizes[productId];
  const selectedVariant = productVariants[productId] || "";
  const getVariantQty = (variantData) => {
    if (typeof variantData === "number") return variantData;
    if (typeof variantData === "object" && variantData !== null) {
      return variantData.quantity || 0;
    }
    return 0;
```

```
    };
    let availableStock = 0;
    if (product.sizes && typeof product.sizes === "object" && selectedSize) {
      const sizeData = product.sizes[selectedSize];
      if (typeof sizeData === "object" && sizeData !== null && sizeData.variants && selectedVariant) {
        const variantData = sizeData.variants[selectedVariant];
        availableStock = getVariantQty(variantData);
      } else if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
        availableStock = sizeData.quantity;
      } else if (typeof sizeData === "number") {
        availableStock = sizeData;
      }
    } else {
      availableStock = product.currentStock || 0;
    }
    const clamped = Math.max(1, Math.min(parseInt(nextQty, 10) || 1, availableStock || product.currentStock
|| 1));
    setProductQuantities((prev) => ({ ...prev, [productId]: clamped }));
  };
  const handleVariantSelection = (productId, variant) => {
    setProductVariants({ ...productVariants, [productId]: variant });
    setProductSizes({ ...productSizes, [productId]: "" });
    setProductQuantities({ ...productQuantities, [productId]: 1 });
  };
  const handleSizeSelection = (productId, size) => {
    const product = products.find((p) => p._id === productId);
    if (!product) return;
    setProductSizes({ ...productSizes, [productId]: size });
    const selectedVariant = productVariants[productId] || "";
    const getVariantQty = (variantData) => {
      if (typeof variantData === "number") return variantData;
      if (typeof variantData === "object" && variantData !== null) {
        return variantData.quantity || 0;
      }
      return 0;
    };
    let availableStock = 0;
    if (product.sizes && typeof product.sizes === "object" && size) {
      const sizeData = product.sizes[size];
      if (typeof sizeData === "object" && sizeData !== null && sizeData.variants && selectedVariant) {
        const variantData = sizeData.variants[selectedVariant];
        availableStock = getVariantQty(variantData);
      } else if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
        availableStock = sizeData.quantity;
      } else if (typeof sizeData === "number") {
        availableStock = sizeData;
      }
    } else {
      availableStock = product.currentStock || 0;
    }
    const currentQuantity = productQuantities[productId] || 1;
    if (currentQuantity > availableStock && availableStock > 0) {
      setProductQuantities({
        ...productQuantities,
        [productId]: availableStock
      });
    } else if (availableStock === 0) {
      setProductQuantities({
        ...productQuantities,
        [productId]: 1
      });
    }
  };
  const addToCartFromExpanded = (product) => {
    const quantity = productQuantities[product._id] || 1;
    const size = productSizes[product._id] || "";
    const variant = productVariants[product._id] || "";
    const getVariantQty = (variantData) => {
      if (typeof variantData === "number") return variantData;
      if (typeof variantData === "object" && variantData !== null) {
        return variantData.quantity || 0;
      }
      return 0;
    };
    const hasVariantsPerSize = () => {
      if (product.sizes && typeof product.sizes === "object") {
        return Object.values(product.sizes).some((sizeData) => {
```

```
          return typeof sizeData === "object" && sizeData !== null && sizeData.variants && typeof
sizeData.variants === "object" && Object.keys(sizeData.variants).length > 0;
        });
      }
      return false;
    };
    const hasSimpleVariants = () => {
      if (product.variant && typeof product.variant === "string") {
        const variants = product.variant.split(",").map((v) => v.trim()).filter((v) => v.length > 0);
        return variants.length > 1;
      }
      return false;
    };
    const productHasSizeVariants = hasVariantsPerSize();
    const productHasSimpleVariants = hasSimpleVariants();
    const productHasAnyVariants = productHasSizeVariants || productHasSimpleVariants;
    if (productHasAnyVariants && !variant) {
      alert("Please select a variant before adding to cart");
      return;
    }
    if (product.sizes && typeof product.sizes === "object" && Object.keys(product.sizes).length > 0) {
      if (!size) {
        alert("Please select a size before adding to cart");
        return;
      }
    }
    let availableStock = 0;
    let itemPrice = product.itemPrice || 0;
    if (product.sizes && typeof product.sizes === "object" && size) {
      const sizeData = product.sizes[size];
      if (typeof sizeData === "object" && sizeData !== null) {
        if (sizeData.variants && variant && !productHasSimpleVariants) {
          const variantData = sizeData.variants[variant];
          availableStock = getVariantQty(variantData);
          if (typeof variantData === "object" && variantData !== null && variantData.price !== void 0) {
            itemPrice = variantData.price;
          } else if (sizeData.variantPrices && sizeData.variantPrices[variant] !== void 0) {
            itemPrice = sizeData.variantPrices[variant];
          } else if (sizeData.price !== void 0) {
            itemPrice = sizeData.price;
          }
        } else {
          availableStock = sizeData.quantity || 0;
          itemPrice = sizeData.price !== void 0 ? sizeData.price : product.itemPrice || 0;
        }
      } else {
        availableStock = typeof sizeData === "number" ? sizeData : 0;
      }
    } else {
      availableStock = product.currentStock || 0;
    }
    if (availableStock <= 0) {
      alert("This item is out of stock");
      return;
    }
    if (!ensureCanAddItems(quantity)) {
      return;
    }
    const existingItem = productHasAnyVariants ? cart.find(
      (item) => item._id === product._id && item.selectedSize === size && item.selectedVariation === variant
    ) : cart.find(
      (item) => item._id === product._id && item.selectedSize === size
    );
    const totalQuantityAfterAdd = existingItem ? existingItem.quantity + quantity : quantity;
    if (totalQuantityAfterAdd > availableStock) {
      alert(
        `Only ${availableStock} item(s) available in stock. You already have ${existingItem?.quantity || 0}
in cart.`
      );
      return;
    }
    const productToAdd = {
      ...product,
      productId: product._id,
      selectedSize: size,
      selectedVariation: variant,
      quantity,
```

```
      itemPrice
    };
    if (existingItem) {
      setPendingDuplicateItem({
        product: productToAdd,
        existingQuantity: existingItem.quantity
      });
      setShowDuplicateModal(true);
      return;
    }
    setCart([...cart, productToAdd]);
    setExpandedProductId(null);
    setShowProductModal(false);
    setSelectedProduct(null);
  };
  const handleConfirmDuplicateAdd = () => {
    if (!pendingDuplicateItem) return;
    const { product } = pendingDuplicateItem;
    if (!ensureCanAddItems(product?.quantity || 0)) {
      return;
    }
    setCart(
      cart.map(
        (item) => item._id === product._id && item.selectedSize === product.selectedSize &&
item.selectedVariation === product.selectedVariation ? { ...item, quantity: item.quantity +
product.quantity } : item
      )
    );
    setShowDuplicateModal(false);
    setPendingDuplicateItem(null);
    setExpandedProductId(null);
    setShowProductModal(false);
    setSelectedProduct(null);
  };
  const handleCancelDuplicateAdd = () => {
    setShowDuplicateModal(false);
    setPendingDuplicateItem(null);
  };
  const handleItemLimitOverrideApproved = async (approverInfo = {}) => {
    const role = approverInfo?.approvedByRole || "";
    if (!OVERRIDE_ROLES.includes(role)) {
      throw new Error("Only Manager or Owner PIN can approve this override.");
    }
    setItemLimitOverrideApproved(true);
    setShowItemLimitOverrideModal(false);
    toastBr.success(
      `Item limit override approved by ${approverInfo?.approvedBy || role}.`
    );
  };
  const addToCart = (product) => {
    if (!ensureCanAddItems(1)) {
      return;
    }
    const defaultSize = product.sizes && typeof product.sizes === "object" ? Object.keys(product.sizes)[0] ||
"" : product.size || "";
    let itemPrice = product.itemPrice || 0;
    if (product.sizes && typeof product.sizes === "object" && defaultSize) {
      const sizeData = product.sizes[defaultSize];
      if (typeof sizeData === "object" && sizeData !== null && sizeData.price !== void 0) {
        itemPrice = sizeData.price;
      }
    }
    const existingItem = cart.find(
      (item) => item._id === product._id && (item.selectedSize || "") === (defaultSize || "")
    );
    if (existingItem) {
      setCart(
        cart.map(
          (item) => item._id === product._id && (item.selectedSize || "") === (defaultSize || "") ? { ...item,
quantity: item.quantity + 1 } : item
        )
      );
    } else {
      setCart(
        [
          ...cart,
          {
```

```
            ...product,
            productId: product._id,
            selectedSize: defaultSize,
            quantity: 1,
            itemPrice
          }
        ]
      );
    }
  };
  const updateQuantity = (itemOrId, newQuantity) => {
    const item = typeof itemOrId === "object" ? itemOrId : cart.find((i) => i._id === itemOrId);
    const productId = typeof itemOrId === "object" ? itemOrId._id : itemOrId;
    const selectedSize = typeof itemOrId === "object" ? itemOrId.selectedSize || "" : "";
    const selectedVariation = typeof itemOrId === "object" ? itemOrId.selectedVariation || "" : "";
    if (newQuantity <= 0) {
      if (item) {
        handleRemoveItemClick(item);
      } else {
        removeFromCart(productId);
      }
    } else {
      const currentItemQuantity = parseInt(item?.quantity, 10) || 0;
      const requestedQuantity = parseInt(newQuantity, 10) || 0;
      const increaseBy = requestedQuantity - currentItemQuantity;
      if (increaseBy > 0 && !ensureCanAddItems(increaseBy)) {
        return;
      }
      setCart(
        cart.map(
          (cartItem) => cartItem._id === productId && (cartItem.selectedSize || "") === selectedSize &&
(cartItem.selectedVariation || "") === selectedVariation ? { ...cartItem, quantity: newQuantity } : cartItem
        )
      );
    }
  };
  const handleRemoveItemClick = (item) => {
    setItemToRemove(item);
    setShowRemoveItemModal(true);
  };
  const recordVoidedItem = async (item, voidReason) => {
    const res = await fetch(`${API_BASE_URL}/api/transactions`, {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify({
        userId,
        performedById: currentUser?._id || currentUser?.id,
        performedByName: currentUser?.name,
        items: [
          {
            productId: item.productId || item._id,
            itemName: item.itemName,
            sku: item.sku,
            variant: item.variant,
            selectedSize: resolveItemSize(item) || null,
            quantity: item.quantity || 1,
            price: item.itemPrice || 0,
            itemImage: item.itemImage || "",
            voidReason: voidReason || null
          }
        ],
        paymentMethod: "void",
        amountReceived: 0,
        changeGiven: 0,
        referenceNo: `VOID-${Date.now()}`,
        totalAmount: (item.itemPrice || 0) * (item.quantity || 1),
        status: "Voided",
        voidReason: voidReason || null
      })
    });
    const data = await res.json().catch(() => ({}));
    if (!res.ok || data.success === false) {
      throw new Error(
        data.message || data.error || `Void recording failed (${res.status})`
      );
```

```
  }
};
const confirmRemoveItem = async (voidReason) => {
  console.log(
    "[confirmRemoveItem] Called with reason:",
    voidReason,
    "itemToRemove:",
    itemToRemove
  );
  if (!itemToRemove) {
    console.warn("[confirmRemoveItem] No item to remove");
    setShowRemoveItemModal(false);
    setItemToRemove(null);
    return;
  }
  if (!voidReason) {
    console.warn("[confirmRemoveItem] No void reason provided");
    return;
  }
  const itemToVoid = JSON.parse(JSON.stringify(itemToRemove));
  try {
    let itemWasRemoved = false;
    let removedItemDetails = null;
    console.log("[confirmRemoveItem] Item to void details:", {
      _id: itemToVoid._id,
      productId: itemToVoid.productId,
      id: itemToVoid.id,
      selectedSize: itemToVoid.selectedSize,
      size: itemToVoid.size,
      sizes: itemToVoid.sizes,
      resolvedSize: resolveItemSize(itemToVoid),
      fullItem: itemToVoid
    });
    flushSync(() => {
      setCart((prevCart) => {
        console.log(
          "[confirmRemoveItem] Current cart before removal (length:",
          prevCart.length,
          "):",
          prevCart.map((item) => ({
            _id: item._id,
            productId: item.productId,
            id: item.id,
            selectedSize: item.selectedSize,
            size: item.size,
            resolvedSize: resolveItemSize(item),
            quantity: item.quantity
          }))
        );
        const initialLength = prevCart.length;
        const newCart = [];
        let foundMatch = false;
        for (const item of prevCart) {
          const itemId = String(
            item._id || item.productId || item.id || ""
          ).trim();
          const voidId = String(
            itemToVoid._id || itemToVoid.productId || itemToVoid.id || ""
          ).trim();
          const sameProduct = itemId !== "" && voidId !== "" && itemId === voidId;
          const itemSize = String(
            item.selectedSize || item.size || resolveItemSize(item) || ""
          ).toLowerCase().trim();
          const voidSize = String(
            itemToVoid.selectedSize || itemToVoid.size || resolveItemSize(itemToVoid) || ""
          ).toLowerCase().trim();
          const sameSize = itemSize === voidSize || itemSize === "" && voidSize === "";
          const shouldRemove = sameProduct && sameSize;
          if (shouldRemove) {
            console.log(
              "[confirmRemoveItem] \u2713 MATCH FOUND - Removing item:",
              {
                itemId,
                voidId,
                itemSelectedSize: item.selectedSize,
                voidSelectedSize: itemToVoid.selectedSize,
                itemSize,
```

```
            voidSize,
            sameProduct,
            sameSize
          }
        );
        foundMatch = true;
        removedItemDetails = { ...item };
        continue;
      }
      newCart.push(item);
    }
    const finalLength = newCart.length;
    itemWasRemoved = foundMatch && initialLength > finalLength;
    const itemStillInCart = newCart.some((item) => {
      const itemId = String(
        item._id || item.productId || item.id || ""
      ).trim();
      const voidId = String(
        itemToVoid._id || itemToVoid.productId || itemToVoid.id || ""
      ).trim();
      const sameProduct = itemId !== "" && voidId !== "" && itemId === voidId;
      const itemSize = String(
        item.selectedSize || item.size || resolveItemSize(item) || ""
      ).toLowerCase().trim();
      const voidSize = String(
        itemToVoid.selectedSize || itemToVoid.size || resolveItemSize(itemToVoid) || ""
      ).toLowerCase().trim();
      const sameSize = itemSize === voidSize || itemSize === "" && voidSize === "";
      return sameProduct && sameSize;
    });
    itemWasRemoved = foundMatch && initialLength > finalLength && !itemStillInCart;
    console.log("[confirmRemoveItem] Cart update result:", {
      foundMatch,
      oldLength: initialLength,
      newLength: finalLength,
      lengthDecreased: initialLength > finalLength,
      itemStillInCart,
      itemWasRemoved,
      removedItem: removedItemDetails,
      newCartItems: newCart.map((item) => ({
        _id: item._id,
        productId: item.productId,
        selectedSize: item.selectedSize,
        resolvedSize: resolveItemSize(item),
        quantity: item.quantity
      }))
    });
    return newCart;
  });
});
await new Promise((resolve) => setTimeout(resolve, 10));
const verifyCartUpdate = () => {
  return new Promise((resolve) => {
    setCart((currentCart) => {
      const itemStillExists = currentCart.some((item) => {
        const itemId = String(
          item._id || item.productId || item.id || ""
        ).trim();
        const voidId = String(
          itemToVoid._id || itemToVoid.productId || itemToVoid.id || ""
        ).trim();
        const sameProduct = itemId !== "" && voidId !== "" && itemId === voidId;
        const itemSize = String(
          item.selectedSize || item.size || resolveItemSize(item) || ""
        ).toLowerCase().trim();
        const voidSize = String(
          itemToVoid.selectedSize || itemToVoid.size || resolveItemSize(itemToVoid) || ""
        ).toLowerCase().trim();
        const sameSize = itemSize === voidSize || itemSize === "" && voidSize === "";
        return sameProduct && sameSize;
      });
      resolve(!itemStillExists);
      return currentCart;
    });
  });
};
const verifiedRemoved = await verifyCartUpdate();
```

```
    if (!verifiedRemoved) {
      itemWasRemoved = false;
      console.error(
        "[confirmRemoveItem] \u274C Verification failed: Item still exists in cart after update!"
      );
    }
    if (!itemWasRemoved || !verifiedRemoved) {
      console.error(
        "[confirmRemoveItem] \u274C Item was NOT removed from cart! Aborting void transaction."
      );
      console.error("[confirmRemoveItem] Debug info:", {
        itemToVoid: {
          _id: itemToVoid._id,
          productId: itemToVoid.productId,
          id: itemToVoid.id,
          selectedSize: itemToVoid.selectedSize,
          size: itemToVoid.size,
          resolvedSize: resolveItemSize(itemToVoid)
        },
        currentCart: cart.map((item) => ({
          _id: item._id,
          productId: item.productId,
          id: item.id,
          selectedSize: item.selectedSize,
          size: item.size,
          resolvedSize: resolveItemSize(item)
        }))
      });
      toastBr.error(
        "Could not remove that line from the cart. It may already be removed."
      );
      return;
    }
    if (itemWasRemoved !== true) {
      console.error(
        "[confirmRemoveItem] \u274C Safety check failed: itemWasRemoved is not true! Aborting."
      );
      toastBr.error("Could not void this item. Please try again.");
      return;
    }
    console.log(
      "[confirmRemoveItem] \u2705 Item removed successfully, recording void transaction..."
    );
    const voidedQty = itemToVoid.quantity || 1;
    setShowRemoveItemModal(false);
    setItemToRemove(null);
    toastBr.success(
      `Item removed from this sale. (${voidedQty} ${voidedQty === 1 ? "item" : "items"} voided)`
    );
    recordVoidedItem(itemToVoid, voidReason).then(() => {
      console.log(
        "[confirmRemoveItem] Void transaction recorded successfully"
      );
      console.log("[confirmRemoveItem] Modal closed and item cleared");
    }).catch((error) => {
      console.error(
        "[confirmRemoveItem] Error recording void transaction:",
        error
      );
      toastBr.error(
        error?.message || "Removed from cart, but the void could not be saved. Check your connection."
      );
    });
    return { voidedQty };
  } catch (error) {
    console.error("[confirmRemoveItem] Error:", error);
    toastBr.error("Failed to void item. Please try again.");
  }
};
const removeFromCart = (item) => {
  handleRemoveItemClick(item);
};
const removeFromCartDirect = (item) => {
  setCart((prevCart) => {
    const itemId = String(item._id || item.productId || item.id || "").trim();
    const itemSize = String(item.selectedSize || item.size || "").toLowerCase().trim();
    return prevCart.filter((cartItem) => {
```

```
        const cartItemId = String(
          cartItem._id || cartItem.productId || cartItem.id || ""
        ).trim();
        const cartItemSize = String(
          cartItem.selectedSize || cartItem.size || ""
        ).toLowerCase().trim();
        const sameProduct = cartItemId === itemId;
        const sameSize = cartItemSize === itemSize || cartItemSize === "" && itemSize === "";
        return !(sameProduct && sameSize);
      });
    });
  };
  const subtotal = useMemo(() => {
    return cart.reduce((sum, item) => sum + item.itemPrice * item.quantity, 0);
  }, [cart]);
  const normalizeCategory = useCallback((value) => String(value || "").trim().toLowerCase(), []);
  const itemMatchesDiscountCategory = useCallback((item, discountCategory, discountSubCategory) => {
    const targetCategory = normalizeCategory(discountCategory);
    if (!targetCategory) return false;
    const productId = String(item?._id || item?.productId || item?.id || "");
    const product = productMap.get(productId);
    const itemCategory = normalizeCategory(item?.category || product?.category);
    const itemSubCategory = normalizeCategory(item?.subCategory || product?.subCategory);
    if (itemCategory !== targetCategory) return false;
    const targetSubCategory = normalizeCategory(discountSubCategory);
    if (!targetSubCategory) return true;
    return itemSubCategory === targetSubCategory;
  }, [normalizeCategory, productMap]);
  const validateDiscountForCart = (discountItem, cartItems) => {
    if (!discountItem || !cartItems || cartItems.length === 0) {
      return { valid: false, message: "Cart is empty" };
    }
    const appliesToType = discountItem.appliesToType || discountItem.appliesTo;
    if (appliesToType === "all") {
      return { valid: true };
    }
    if (appliesToType === "category" && discountItem.category) {
      const hasMatchingItem = cartItems.some(
        (item) => itemMatchesDiscountCategory(item, discountItem.category, discountItem.subCategory)
      );
      if (!hasMatchingItem) {
        return {
          valid: false,
          message: `This discount only applies to items in the "${discountItem.category}" category.`
        };
      }
      return { valid: true };
    }
    if (appliesToType === "products" && discountItem.productIds && discountItem.productIds.length > 0) {
      const hasMatchingItem = cartItems.some((item) => {
        const itemId = item._id || item.productId || item.id;
        return discountItem.productIds.some((pid) => {
          const pidStr = pid.toString ? pid.toString() : pid;
          const itemIdStr = itemId.toString ? itemId.toString() : itemId;
          return pidStr === itemIdStr;
        });
      });
      if (!hasMatchingItem) {
        return {
          valid: false,
          message: "This discount only applies to specific products. Your cart does not contain any eligible
items."
        };
      }
      return { valid: true };
    }
    return { valid: false, message: "Discount configuration is invalid" };
  };
  useEffect(() => {
    if (selectedDiscounts.length > 0 && cart.length > 0) {
      const validDiscounts = selectedDiscounts.filter((discount2) => {
        const validation = validateDiscountForCart(discount2, cart);
        return validation.valid;
      });
      if (validDiscounts.length !== selectedDiscounts.length) {
        setSelectedDiscounts(validDiscounts);
        if (validDiscounts.length === 0) {
```

```
          setDiscountAmount("");
        }
      }
    }
  }, [cart, selectedDiscounts, products]);
  const discount = useMemo(() => {
    if (selectedDiscounts.length === 0) {
      return parseFloat(discountAmount) || 0;
    }
    let totalDiscount = 0;
    for (const selectedDiscount of selectedDiscounts) {
      const validation = validateDiscountForCart(selectedDiscount, cart);
      if (!validation.valid) {
        continue;
      }
      const discountValueStr = selectedDiscount.discountValue || "";
      try {
        let totalEligibleAmount = 0;
        const appliesToType = selectedDiscount.appliesToType || selectedDiscount.appliesTo;
        if (appliesToType === "all") {
          totalEligibleAmount = subtotal;
        } else if (appliesToType === "category" && selectedDiscount.category) {
          totalEligibleAmount = cart.reduce((sum, item) => {
            if (itemMatchesDiscountCategory(item, selectedDiscount.category, selectedDiscount.subCategory)) {
              return sum + item.itemPrice * item.quantity;
            }
            return sum;
          }, 0);
        } else if (appliesToType === "products" && selectedDiscount.productIds &&
selectedDiscount.productIds.length > 0) {
          totalEligibleAmount = cart.reduce((sum, item) => {
            const itemId = item._id || item.productId || item.id;
            const isEligible = selectedDiscount.productIds.some((pid) => {
              const pidStr = pid.toString ? pid.toString() : pid;
              const itemIdStr = itemId.toString ? itemId.toString() : itemId;
              return pidStr === itemIdStr;
            });
            if (isEligible) {
              return sum + item.itemPrice * item.quantity;
            }
            return sum;
          }, 0);
        }
        if (typeof discountValueStr === "string" && discountValueStr.includes("%")) {
          const percentage = parseFloat(
            discountValueStr.replace("% OFF", "").replace(/\s/g, "")
          );
          if (!isNaN(percentage)) {
            totalDiscount += totalEligibleAmount * percentage / 100;
          }
        } else if (typeof discountValueStr === "string" && (discountValueStr.includes("₱") ||
discountValueStr.includes("\u20B1"))) {
          const amount = parseFloat(discountValueStr.replace(/[P₱\sOFF]/g, ""));
          if (!isNaN(amount)) {
            totalDiscount += amount;
          }
        }
      } catch (error) {
        console.error("Error calculating discount:", error);
      }
    }
    return totalDiscount || parseFloat(discountAmount) || 0;
  }, [discountAmount, selectedDiscounts, subtotal, cart, products]);
  const total = useMemo(() => {
    return subtotal - discount;
  }, [subtotal, discount]);
  const paginatedProducts = useMemo(() => {
    return filteredProducts.slice(
      (currentPage - 1) * itemsPerPage,
      currentPage * itemsPerPage
    );
  }, [filteredProducts, currentPage, itemsPerPage]);
  const handleCheckout = () => {
    if (cart.length === 0) {
      alert("Cart is empty!");
      return;
    }
```

```
    setShowCheckoutModal(true);
  };
  const confirmCheckout = () => {
    setShowCheckoutModal(false);
    alert("Proceeding to checkout...");
  };
  const handleCashPayment = () => {
    setShowCashPaymentModal(true);
  };
  const mapCartItemsForStockUpdate = () => cart.map((item) => ({
    _id: item.productId || item._id,
    sku: item.sku || null,
    size: item.selectedSize || item.size || resolveItemSize(item) || null,
    variant: item.selectedVariation || item.variant || null,
    quantity: item.quantity || 1
  }));
  const mapCartItemsForTransaction = () => cart.map((item) => ({
    productId: item.productId || item._id,
    itemName: item.itemName,
    sku: item.sku,
    variant: item.selectedVariation || item.variant || "",
    selectedSize: resolveItemSize(item) || null,
    quantity: item.quantity || 1,
    price: item.itemPrice || 0,
    itemImage: item.itemImage || ""
  }));
  const applyStockOutOptimistically = useCallback((stockItems, { alsoUpdateSelectedProduct = true,
capturePrev = false } = {}) => {
    if (!Array.isArray(stockItems) || stockItems.length === 0) return;
    setProducts((prev) => {
      const prevList = Array.isArray(prev) ? prev : [];
      if (capturePrev) {
        productsBeforeTxnRef.current = prevList;
      }
      const next = prevList.map((p) => ({ ...p }));
      const findSizeKey = (sizesObj, size) => {
        if (!sizesObj || typeof sizesObj !== "object" || !size) return null;
        if (Object.prototype.hasOwnProperty.call(sizesObj, size)) return size;
        const target = String(size).trim().toLowerCase();
        return Object.keys(sizesObj).find(
          (k) => String(k).trim().toLowerCase() === target
        ) || null;
      };
      const getSizeQty = (sizeData) => {
        if (typeof sizeData === "number") return sizeData;
        if (typeof sizeData === "object" && sizeData !== null) {
          return Number(sizeData.quantity || 0);
        }
        return 0;
      };
      const setSizeQty = (sizeData, qty) => {
        const clamped = Math.max(0, Number(qty || 0));
        if (typeof sizeData === "number") return clamped;
        if (typeof sizeData === "object" && sizeData !== null) {
          return { ...sizeData, quantity: clamped };
        }
        return clamped;
      };
      const getVariantQty = (variantData) => {
        if (typeof variantData === "number") return variantData;
        if (typeof variantData === "object" && variantData !== null) {
          return Number(variantData.quantity || 0);
        }
        return 0;
      };
      const setVariantQty = (variantData, qty) => {
        const clamped = Math.max(0, Number(qty || 0));
        if (typeof variantData === "number") return clamped;
        if (typeof variantData === "object" && variantData !== null) {
          return { ...variantData, quantity: clamped };
        }
        return clamped;
      };
      const recomputeCurrentStock = (product) => {
        if (product?.sizes && typeof product.sizes === "object") {
          return Object.values(product.sizes).reduce(
            (sum, sd) => sum + getSizeQty(sd),
```

```
            0
          );
        }
        return Math.max(0, Number(product?.currentStock || 0));
      };
      for (const si of stockItems) {
        const id = String(si?._id || "");
        if (!id) continue;
        const idx = next.findIndex(
          (p) => String(p?._id || p?.id || "") === id
        );
        if (idx === -1) continue;
        const product = next[idx];
        const qty = Math.max(0, Number(si.quantity || 0));
        if (!qty) continue;
        if (product?.sizes && typeof product.sizes === "object" && si.size) {
          const sizeKey = findSizeKey(product.sizes, si.size);
          if (sizeKey) {
            const sizeData = product.sizes[sizeKey];
            if (si.variant && typeof sizeData === "object" && sizeData !== null && sizeData.variants &&
typeof sizeData.variants === "object") {
              const currentVar = sizeData.variants[si.variant];
              const nextVar = setVariantQty(
                currentVar,
                getVariantQty(currentVar) - qty
              );
              const nextVariants = { ...sizeData.variants, [si.variant]: nextVar };
              const nextSizeTotal = Object.values(nextVariants).reduce(
                (sum, vd) => sum + getVariantQty(vd),
                0
              );
              product.sizes[sizeKey] = {
                ...sizeData,
                variants: nextVariants,
                quantity: nextSizeTotal
              };
            } else {
              product.sizes[sizeKey] = setSizeQty(
                sizeData,
                getSizeQty(sizeData) - qty
              );
            }
          }
          product.currentStock = recomputeCurrentStock(product);
        } else {
          product.currentStock = Math.max(
            0,
            Number(product.currentStock || 0) - qty
          );
        }
      }
      setCachedData("products", next);
      if (alsoUpdateSelectedProduct) {
        setSelectedProduct((prevSel) => {
          if (!prevSel) return prevSel;
          const key = String(prevSel._id || prevSel.id || "");
          const updated = next.find((p) => String(p?._id || p?.id || "") === key);
          return updated ? { ...prevSel, ...updated } : prevSel;
        });
      }
      return next;
    });
  }, [setCachedData]);
  const finalizeTransaction = async (meta = {}) => {
    if (!cart.length) return null;
    if (isProcessingTransaction) {
      console.warn("Transaction is already processing, ignoring duplicate call.");
      return null;
    }
    setIsProcessingTransaction(true);
    const cartSnapshot = [...cart];
    const stockItems = mapCartItemsForStockUpdate();
    const transactionItems = mapCartItemsForTransaction();
    const currentTotal = total;
    const currentDiscountIds = selectedDiscounts.map((d) => d._id);
    try {
      applyStockOutOptimistically(stockItems, { capturePrev: true });
```

```
      const lineSubtotal = cartSnapshot.reduce(
        (s, item) => s + (item.itemPrice || 0) * (item.quantity || 1),
        0
      );
      const discountSaved = Math.max(
        0,
        Math.round((lineSubtotal - currentTotal) * 100) / 100
      );
      const transactionResponse = await fetch(
        `${API_BASE_URL}/api/transactions`,
        {
          method: "POST",
          headers: {
            "Content-Type": "application/json"
          },
          body: JSON.stringify({
            userId,
            items: transactionItems,
            paymentMethod: meta.paymentMethod || "cash",
            amountReceived: meta.amountReceived,
            changeGiven: meta.change,
            referenceNo: meta.referenceNo,
            receiptNo: meta.receiptNo,
            subtotal: lineSubtotal,
            discount: discountSaved,
            totalAmount: currentTotal,
            performedById: currentUser?._id || currentUser?.id,
            performedByName: currentUser?.name,
            status: "Completed",
            appliedDiscountIds: currentDiscountIds
          })
        }
      );
      if (!transactionResponse.ok) {
        const errorData = await transactionResponse.json().catch(() => ({}));
        throw new Error(
          errorData.message || errorData.error || `Transaction recording failed: ${transactionResponse.status}
${transactionResponse.statusText}`
        );
      }
      const transactionData = await transactionResponse.json();
      if (!transactionData.success) {
        throw new Error(
          transactionData.message || transactionData.error || "Failed to record transaction"
        );
      }
      setCart([]);
      setSelectedDiscounts([]);
      setDiscountAmount("");
      setItemLimitOverrideApproved(false);
      invalidateCache("transactions");
      try {
        const newTxId = transactionData.data?._id || transactionData.data?.id || null;
        const stockItemsWithLine = stockItems.map((row, idx) => ({
          ...row,
          lineIndex: idx
        }));
        const stockRes = await fetch(`${API_BASE_URL}/api/products/update-stock`, {
          method: "POST",
          headers: { "Content-Type": "application/json" },
          body: JSON.stringify({
            type: "Stock-Out",
            reason: "Sold",
            linkTransactionId: newTxId,
            items: stockItemsWithLine,
            performedByName: currentUser?.name || `${currentUser?.firstName || ""} ${currentUser?.lastName ||
""}`.trim() || "System",
            performedById: currentUser?._id || currentUser?.id || ""
          })
        });
        const stockData = await stockRes.json().catch(() => ({}));
        if (!stockRes.ok || !stockData.success) {
          console.error("Stock update failed:", stockData);
          throw new Error(stockData.message || "Stock update failed");
        }
        if (Array.isArray(stockData.data) && stockData.data.length > 0) {
          const updatedMap = new Map(
```

```
          stockData.data.filter((p) => p && (p._id || p.id)).map((p) => [String(p._id || p.id), p])
        );
        setProducts((prev) => {
          const prevList = Array.isArray(prev) ? prev : [];
          const next = prevList.map((p) => {
            const key = String(p?._id || p?.id || "");
            const updated = updatedMap.get(key);
            return updated ? { ...p, ...updated } : p;
          });
          setCachedData("products", next);
          return next;
        });
        setSelectedProduct((prev) => {
          if (!prev) return prev;
          const key = String(prev._id || prev.id || "");
          const updated = updatedMap.get(key);
          return updated ? { ...prev, ...updated } : prev;
        });
      }
    } catch (stockErr) {
      console.error("Stock update error:", stockErr);
      toastBr.error(
        `Sale saved, but inventory update failed: ${stockErr.message || "Unknown error"}`
      );
    }
    productsBeforeTxnRef.current = null;
    return transactionData.data;
  } catch (error) {
    console.error("Error finalizing transaction:", error);
    if (Array.isArray(productsBeforeTxnRef.current)) {
      const prevProducts = productsBeforeTxnRef.current;
      setProducts(prevProducts);
      setCachedData("products", prevProducts);
    }
    productsBeforeTxnRef.current = null;
    setCart(cartSnapshot);
    const errorMessage = error.message || "Unknown error occurred";
    toastBr.error(
      `Transaction failed: ${errorMessage}. Your cart was restored.`
    );
    throw error;
  } finally {
    setIsProcessingTransaction(false);
  }
};
const handleCashProceed = async (amountReceived, change, receiptNo) => {
  console.log("handleCashProceed received receiptNo:", receiptNo);
  setShowCashPaymentModal(false);
  return await finalizeTransaction({
    paymentMethod: "cash",
    amountReceived,
    change,
    receiptNo
  });
};
const handleQRPayment = () => {
  setShowQRPaymentModal(true);
};
const handleGCashTransactionComplete = (paymentData) => {
  console.log("[GCash] Transaction complete:", paymentData);
  const stockItems = mapCartItemsForStockUpdate();
  applyStockOutOptimistically(stockItems);
  setCart([]);
  setSelectedDiscounts([]);
  setDiscountAmount("");
  setItemLimitOverrideApproved(false);
  invalidateCache("transactions");
  toastBr.success("Transaction completed successfully.");
  fetchProducts(true).catch(
    (err) => console.warn("Background product refresh failed:", err)
  );
  setShowQRPaymentModal(false);
};
const handleOpenDiscountModal = useCallback(() => {
  setShowDiscountModal(true);
}, []);
const handleSelectDiscount = useCallback((discountItem) => {
```

```
    try {
      if (!discountItem) {
        console.error("No discount item provided");
        alert("Invalid discount selected. Please try again.");
        return;
      }
      const alreadySelected = selectedDiscounts.some(
        (d) => d._id === discountItem._id
      );
      if (alreadySelected) {
        alert("This discount is already applied.");
        return;
      }
      const validation = validateDiscountForCart(discountItem, cart);
      if (!validation.valid) {
        alert(validation.message);
        return;
      }
      setSelectedDiscounts((prev) => [...prev, discountItem]);
      setDiscountAmount("");
    } catch (error) {
      console.error("Error selecting discount:", error);
      alert("An error occurred while applying the discount. Please try again.");
    }
  }, [selectedDiscounts, cart, validateDiscountForCart]);
  const handleRemoveDiscount = useCallback((discountId) => {
    if (discountId) {
      setSelectedDiscounts((prev) => prev.filter((d) => d._id !== discountId));
    } else {
      setSelectedDiscounts([]);
    }
    setDiscountAmount("");
  }, []);
  return /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `relative flex flex-col h-screen ${theme === "dark" ? "bg-[#121212]" : "bg-[#F9FAFB]"}`
    },
    /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `absolute top-0 left-0 right-[420px] px-6 py-4 z-40 transition-colors duration-300 flex
flex-col gap-4 ${theme === "dark" ? "bg-[#121212]" : "bg-[#FFFFFF]"}`,
        style: { paddingRight: "24px" }
      },
      /* @__PURE__ */ React.createElement(
        Header,
        {
          pageName: "Terminal",
          showSearch: true,
          showFilter: true,
          searchValue: searchQuery,
          onSearchChange: (e) => setSearchQuery(e.target.value),
          profileBackground: "",
          showBorder: false,
          hidePageName: true,
          centerProfile: false,
          profileMinWidth: "220px",
          profilePadding: "px-4",
          profileGap: "gap-3",
          sortOption,
          onSortChange: setSortOption
        }
      ),
      /* @__PURE__ */ React.createElement("div", { className: "flex gap-2 w-full px-2", style: { transform:
"translateY(-5px)" } }, /* @__PURE__ */ React.createElement(
        "select",
        {
          value: selectedMainCategory,
          onChange: (e) => {
            setSelectedMainCategory(e.target.value);
            setSelectedSubCategory("");
          },
          className: `h-10 px-4 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-white" : "bg-white border-gray-300"}`
        },
        mainCategories.map((cat) => /* @__PURE__ */ React.createElement("option", { key: `main-${cat.name}`,
```

```
value: cat.name }, cat.name === "All" ? "By Category" : cat.name))
      ), /* @__PURE__ */ React.createElement(
        "select",
        {
          value: selectedSubCategory,
          onChange: (e) => setSelectedSubCategory(e.target.value),
          disabled: selectedMainCategory === "All",
          className: `h-10 px-4 border rounded-lg focus:outline-none focus:ring-2 focus:ring-[#AD7F65]
${selectedMainCategory === "All" ? "opacity-50 cursor-not-allowed text-gray-400" : ""} ${theme === "dark" ?
"bg-[#2A2724] border-gray-600 text-white" : "bg-white border-gray-300"}`
        },
        /* @__PURE__ */ React.createElement("option", { value: "" }, selectedMainCategory === "All" ? "Select
Main First" : "All Subcategories"),
        subCategories.filter((sub) => sub.parentCategory === selectedMainCategory).map((sub) => /* @__PURE__
*/ React.createElement("option", { key: `sub-${sub.name}`, value: sub.name }, sub.name))
      ))
    ),
    /* @__PURE__ */ React.createElement("div", { className: "flex flex-1 overflow-hidden" }, /* @__PURE__ */
React.createElement(
      "div",
      {
        className: "flex-1 overflow-auto p-6",
        style: {
          paddingTop: `200px`
        }
      },
      /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement(
        "h2",
        {
          className: `text-lg font-semibold mb-3 ${theme === "dark" ? "text-white" : "text-gray-800"}`
        },
        "Products"
      ), loading ? /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `text-center py-10 ${theme === "dark" ? "text-gray-300" : "text-gray-600"}`
        },
        "Loading..."
      ) : filteredProducts.length === 0 ? /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `text-center py-10 ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "No products found"
      ) : /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-5 gap-4 items-start" },
paginatedProducts.map(
        (product) => /* @__PURE__ */ React.createElement(
          ProductCard,
          {
            key: product._id,
            product,
            onToggleExpand: handleProductClick
          }
        )
      ))),
      filteredProducts.length >= itemsPerPage && /* @__PURE__ */ React.createElement(
        Pagination,
        {
          currentPage,
          totalPages: Math.ceil(filteredProducts.length / itemsPerPage),
          onPageChange: setCurrentPage
        }
      )
    ), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `w-[420px] border-l-0 p-4 relative ${theme === "dark" ? "bg-[#121212]" : "bg-gray-50"}`
      },
      /* @__PURE__ */ React.createElement(
        OrderSummary,
        {
          cart,
          products,
          removeFromCart,
          removeFromCartDirect,
          updateQuantity,
          discountAmount,
```

```
        setDiscountAmount,
        selectedDiscounts,
        onRemoveDiscount: handleRemoveDiscount,
        subtotal,
        discount,
        total,
        handleCheckout,
        onCashPayment: handleCashPayment,
        onQRPayment: handleQRPayment,
        onOpenDiscountModal: handleOpenDiscountModal,
        onSelectDiscount: handleSelectDiscount
      }
    )
  ))
), /* @__PURE__ */ React.createElement(
  CheckoutConfirmationModal,
  {
    isOpen: showCheckoutModal,
    onClose: () => setShowCheckoutModal(false),
    onConfirm: confirmCheckout
  }
), /* @__PURE__ */ React.createElement(
  CashPaymentModal,
  {
    isOpen: showCashPaymentModal,
    onClose: () => setShowCashPaymentModal(false),
    totalAmount: total,
    subtotalAmount: subtotal,
    discountAmount: discount,
    selectedDiscounts,
    onProceed: handleCashProceed,
    onTransactionDone: () => toastBr.success("Transaction completed successfully."),
    cartItems: cart,
    cashierName: currentUser?.name
  }
), /* @__PURE__ */ React.createElement(
  QRCodePaymentModal,
  {
    isOpen: showQRPaymentModal,
    onClose: () => setShowQRPaymentModal(false),
    totalAmount: total,
    subtotalAmount: subtotal,
    discountAmount: discount,
    selectedDiscounts,
    cartItems: cart,
    userId,
    performedById: currentUser?._id || currentUser?.id,
    performedByName: currentUser?.name,
    onTransactionComplete: handleGCashTransactionComplete
  }
), /* @__PURE__ */ React.createElement(
  DiscountModal,
  {
    isOpen: showDiscountModal,
    onClose: () => setShowDiscountModal(false),
    onSelectDiscount: handleSelectDiscount,
    cart,
    products,
    selectedDiscounts
  }
), /* @__PURE__ */ React.createElement(
  RemoveItemPinModal,
  {
    isOpen: showRemoveItemModal,
    onClose: () => {
      setShowRemoveItemModal(false);
      setItemToRemove(null);
    },
    onConfirm: confirmRemoveItem,
    item: itemToRemove
  }
), /* @__PURE__ */ React.createElement(
  DuplicateItemModal,
  {
    isOpen: showDuplicateModal,
    onClose: handleCancelDuplicateAdd,
    onConfirm: handleConfirmDuplicateAdd,
```

```
      item: pendingDuplicateItem?.product,
      existingQuantity: pendingDuplicateItem?.existingQuantity || 0
    }
  ), /* @__PURE__ */ React.createElement(
    RemoveItemPinModal,
    {
      isOpen: showItemLimitOverrideModal,
      onClose: () => setShowItemLimitOverrideModal(false),
      onConfirm: handleItemLimitOverrideApproved,
      requireReason: false,
      hideAmountCard: true,
      allowedRoles: OVERRIDE_ROLES,
      title: "Item Limit Override",
      subtitle: `This transaction has reached the ${CART_ITEM_LIMIT}-item limit. Manager/Owner PIN is
required to continue.`,
      confirmText: "Approve Override",
      pinLabel: "Manager/Owner PIN"
    }
  ), /* @__PURE__ */ React.createElement(
    ProductDetailsModal,
    {
      isOpen: showProductModal,
      onClose: handleCloseProductModal,
      product: selectedProduct,
      productQuantity: selectedProduct ? productQuantities[selectedProduct._id] || 1 : 1,
      onDecrement: () => selectedProduct && updateProductQuantity(selectedProduct._id, -1),
      onIncrement: () => selectedProduct && updateProductQuantity(selectedProduct._id, 1),
      onAdd: () => selectedProduct && addToCartFromExpanded(selectedProduct),
      onSetQuantity: (qty) => selectedProduct && setProductQuantityTo(selectedProduct._id, qty),
      selectedSize: selectedProduct ? productSizes[selectedProduct._id] : "",
      onSelectSize: (size) => selectedProduct && handleSizeSelection(selectedProduct._id, size),
      selectedVariant: selectedProduct ? productVariants[selectedProduct._id] : "",
      onSelectVariant: (variant) => selectedProduct && handleVariantSelection(selectedProduct._id, variant)
    }
  ), isProcessingTransaction && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 z-
[10000] flex flex-col justify-center items-center bg-black/60 backdrop-blur-sm" }, /* @__PURE__ */
React.createElement("div", { className: "w-16 h-16 border-4 border-[#AD7F65]/30 border-t-[#AD7F65] rounded-
full animate-spin shadow-lg" }), /* @__PURE__ */ React.createElement("p", { className: "mt-4 text-white text-
lg font-bold tracking-wide shadow-black drop-shadow-md" }, "Processing Transaction..."), /* @__PURE__ */
React.createElement("p", { className: "mt-1 text-gray-300 text-sm" }, "Please wait")));
};
export default memo(Terminal);
```

FILE: pos/frontend/src/pages/transaction.jsx
==========

```
import { AnimatePresence, motion } from "framer-motion";
import { endOfDay, isWithinInterval, startOfDay } from "date-fns";
import DatePicker from "react-datepicker";
import "react-datepicker/dist/react-datepicker.css";
import React, {
  forwardRef,
  memo,
  useCallback,
  useEffect,
  useMemo,
  useRef,
  useState
} from "react";
import {
  FaCalendarAlt,
  FaCheckCircle,
  FaChevronDown,
  FaChevronLeft,
  FaChevronRight,
  FaClipboardList,
  FaExclamationTriangle,
  FaEye,
  FaPrint,
  FaSearch,
  FaUndoAlt
} from "react-icons/fa";
import ShopBagSales from "../assets/ShopBagSales.svg";
import TransactionsTotalGreen from "../assets/TransactionsTotalGreen.svg";
import HandCashIcon from "../assets/hand-cash.svg";
import Header from "../components/shared/header";
```

```
import PrintReceiptModal from "../components/transaction/PrintReceiptModal";
import RemittanceModal from "../components/transaction/RemittanceModal";
import ReturnItemsModal from "../components/transaction/ReturnItemsModal";
import ViewTransactionModal from "../components/transaction/ViewTransactionModal";
import { API_BASE_URL, API_ENDPOINTS } from "../config/api";
import {
  lineSubtotalFromItems,
  resolveTransactionDiscount
} from "../utils/transactionDisplay";
import { useAuth } from "../context/AuthContext";
import { useDataCache } from "../context/DataCacheContext";
import { useTheme } from "../context/ThemeContext";
const STATUS_STYLES = {
  Completed: "bg-green-100 text-green-700 border border-green-200",
  Returned: "bg-orange-100 text-orange-700 border border-orange-200",
  "Partially Returned": "bg-amber-100 text-amber-700 border border-amber-200",
  Voided: "bg-red-100 text-red-600 border border-red-200"
};
const paymentOptions = ["All", "cash", "gcash"];
const statusOptions = ["All", "Completed", "Returned", "Partially Returned"];
const userOptions = ["All"];
const dateOptions = ["Today", "All", "Last 7 days", "Last 30 days", "Custom"];
const CalendarTrigger = forwardRef(({ onClick, className }, ref) => /* @__PURE__ */ React.createElement(
  "button",
  {
    ref,
    type: "button",
    onClick,
    className,
    "aria-label": "Pick date range"
  },
  /* @__PURE__ */ React.createElement(FaCalendarAlt, { className: "text-gray-500 text-sm" })
));
CalendarTrigger.displayName = "CalendarTrigger";
const getInitials = (name = "") => name.split(" ").filter(Boolean).map((n) => n[0]).slice(0,
2).join("").toUpperCase();
const formatCurrency = (value = 0) => new Intl.NumberFormat("en-PH", {
  style: "currency",
  currency: "PHP"
}).format(value);
const formatCurrencyCompact = (value = 0) => {
  const n = parseFloat(value) || 0;
  const abs = Math.abs(n).toLocaleString("en-PH", {
    minimumFractionDigits: 0,
    maximumFractionDigits: 0
  });
  return n < 0 ? `-\u20B1${abs}` : `\u20B1${abs}`;
};
const toNumber = (value, fallback = 0) => {
  const parsed = parseFloat(value);
  return Number.isFinite(parsed) ? parsed : fallback;
};
const originalLineSubtotalFromItems = (transaction) => {
  if (!transaction?.items?.length) return 0;
  return transaction.items.reduce((sum, item) => {
    return sum + (item.price || item.itemPrice || 0) * (item.quantity || 1);
  }, 0);
};
const sameTransactionId = (a, b) => String(a?._id ?? a ?? "") === String(b?._id ?? b ?? "");
const saleDate = (trx) => {
  if (trx.checkedOutAt != null && trx.checkedOutAt !== "") {
    return new Date(trx.checkedOutAt);
  }
  return new Date(trx.createdAt);
};
const generateTransactionNumber = (transaction) => {
  if (!transaction) return "---";
  if (transaction.transactionNumber) {
    return transaction.transactionNumber.toString();
  }
  return "---";
};
const statusIcon = {
  Completed: /* @__PURE__ */ React.createElement(FaCheckCircle, { className: "text-green-500" }),
  Returned: /* @__PURE__ */ React.createElement(FaUndoAlt, { className: "text-orange-500" }),
  "Partially Returned": /* @__PURE__ */ React.createElement(FaUndoAlt, { className: "text-amber-500" }),
  Voided: /* @__PURE__ */ React.createElement(FaExclamationTriangle, { className: "text-red-500" })
```

```
};
const normalizeDropdownOptions = (options) => {
  if (!options?.length) return [];
  if (typeof options[0] === "object" && options[0] !== null && "value" in options[0]) {
    return options.map((o) => ({
      value: String(o.value),
      label: o.label ?? String(o.value)
    }));
  }
  return options.map((o) => ({ value: o, label: o }));
};
const Dropdown = ({
  label,
  options,
  selected,
  onSelect,
  isOpen,
  setIsOpen,
  showAllAsOptionLabel = false
}) => {
  const dropdownRef = React.useRef(null);
  const { theme } = useTheme();
  const normalizedOptions = React.useMemo(
    () => normalizeDropdownOptions(options),
    [options]
  );
  const selectedLabel = normalizedOptions.find((o) => o.value === selected)?.label ?? selected;
  React.useEffect(() => {
    const handleClickOutside = (event) => {
      if (dropdownRef.current && !dropdownRef.current.contains(event.target)) {
        setIsOpen(false);
      }
    };
    if (isOpen) {
      document.addEventListener("mousedown", handleClickOutside);
    }
    return () => {
      document.removeEventListener("mousedown", handleClickOutside);
    };
  }, [isOpen, setIsOpen]);
  return /* @__PURE__ */ React.createElement("div", { className: "relative", ref: dropdownRef }, /* @__PURE__
*/ React.createElement(
    "button",
    {
      type: "button",
      onClick: (e) => {
        e.stopPropagation();
        setIsOpen((prev) => !prev);
      },
      className: `flex items-center gap-2 px-4 py-2 rounded-xl border transition-all ${isOpen ? "border-
[#AD7F65] shadow-lg " + (theme === "dark" ? "bg-[#2A2724] text-white" : "bg-white text-gray-700") : theme ===
"dark" ? "border-gray-600 bg-[#2A2724] text-gray-300 hover:border-[#AD7F65]" : "border-gray-200 bg-white
hover:border-[#AD7F65] text-gray-700"}`
    },
    /* @__PURE__ */ React.createElement("span", { className: "text-sm font-medium" }, selected === "All" ?
showAllAsOptionLabel ? "All" : label : selectedLabel),
    /* @__PURE__ */ React.createElement(
      FaChevronDown,
      {
        className: `text-xs text-gray-500 transition-transform ${isOpen ? "rotate-180" : ""}`
      }
    )
  ), /* @__PURE__ */ React.createElement(AnimatePresence, null, isOpen && /* @__PURE__ */
React.createElement(
    motion.ul,
    {
      initial: { opacity: 0, y: -8 },
      animate: { opacity: 1, y: 0 },
      exit: { opacity: 0, y: -8 },
      className: `absolute z-20 mt-2 w-44 rounded-xl border border-gray-100 shadow-2xl overflow-hidden
${theme === "dark" ? "bg-[#2A2724] border-gray-600" : "bg-white border-gray-100"}`,
      onClick: (e) => e.stopPropagation()
    },
    normalizedOptions.map(
      (option) => /* @__PURE__ */ React.createElement(
        "li",
        {
```

```
          key: option.value,
          onClick: () => {
            onSelect(option.value);
            setIsOpen(false);
          },
          className: `px-4 py-2 text-sm cursor-pointer transition-colors ${option.value === selected ? "bg-
[#F6EEE7] text-[#76462B] font-semibold" : theme === "dark" ? "text-gray-300 hover:bg-[#352F2A]" : "text-gray-
700 hover:bg-gray-50"}`
        },
        option.label
      )
    )
  )));
};
const Transaction = () => {
  const { theme } = useTheme();
  const { currentUser } = useAuth();
  const { setCachedData, invalidateCache } = useDataCache();
  const currentUserFilterId = String(currentUser?._id || currentUser?.id || "All");
  const [transactions, setTransactions] = useState([]);
  const [loading, setLoading] = useState(false);
  const [selectedTransaction, setSelectedTransaction] = useState(null);
  const [search, setSearch] = useState("");
  const [debouncedSearch, setDebouncedSearch] = useState("");
  const [dropdownOpen, setDropdownOpen] = useState({
    date: false,
    method: false,
    status: false,
    user: false,
    returnedBy: false
  });
  const [filters, setFilters] = useState({
    date: "Today",
    method: "All",
    status: "All",
    user: currentUserFilterId || "All",
    returnedBy: "All"
  });
  const [dateRange, setDateRange] = useState([null, null]);
  const [pickerOpen, setPickerOpen] = useState(false);
  const [startDate, endDate] = dateRange;
  const [currentPage, setCurrentPage] = useState(1);
  const [showViewModal, setShowViewModal] = useState(false);
  const [transactionToView, setTransactionToView] = useState(null);
  const [showPrintModal, setShowPrintModal] = useState(false);
  const [transactionToPrint, setTransactionToPrint] = useState(null);
  const [showReturnModal, setShowReturnModal] = useState(false);
  const [transactionToReturn, setTransactionToReturn] = useState(null);
  const [showReturnSuccessModal, setShowReturnSuccessModal] = useState(false);
  const rowsPerPage = 8;
  const [selectedTransactionIds, setSelectedTransactionIds] = useState([]);
  const [selectedReturnedLogIds, setSelectedReturnedLogIds] = useState([]);
  const [isExportSelectionMode, setIsExportSelectionMode] = useState(false);
  const [showRemittanceModal, setShowRemittanceModal] = useState(false);
  const [staffList, setStaffList] = useState([]);
  const [activeTab, setActiveTab] = useState("transactions");
  useEffect(() => {
    const timer = setTimeout(() => setDebouncedSearch(search), 350);
    return () => clearTimeout(timer);
  }, [search]);
  useEffect(() => {
    (async () => {
      try {
        const res = await fetch(API_ENDPOINTS.employees);
        const data = await res.json();
        if (data.success && Array.isArray(data.data)) {
          setStaffList(data.data.filter((e) => e.status === "Active"));
        }
      } catch (err) {
        console.error("Failed to load employees:", err);
      }
    })();
  }, []);
  const isInitialMount = useRef(true);
  const hasLoaded = useRef(false);
  const isInitialLoading = useRef(true);
  const setCachedDataRef = useRef(setCachedData);
```

```
const selectAllTransactionsRef = useRef(null);
const selectAllReturnedLogsRef = useRef(null);
useEffect(() => {
  setCachedDataRef.current = setCachedData;
}, [setCachedData]);
const fetchTransactions = useCallback(async () => {
  setLoading(true);
  try {
    const params = new URLSearchParams();
    if (debouncedSearch) params.append("search", debouncedSearch);
    if (filters.method !== "All")
      params.append("paymentMethod", filters.method);
    if (filters.status !== "All") params.append("status", filters.status);
    if (filters.user !== "All") params.append("userId", filters.user);
    params.append("limit", "3000");
    const qs = params.toString() ? `?${params.toString()}` : "";
    const response = await fetch(
      `${API_BASE_URL}/api/transactions${qs}`
    );
    const data = await response.json();
    if (data.success && Array.isArray(data.data)) {
      const allTransactions = data.data;
      const returnTransactions = allTransactions.filter(
        (t) => t.paymentMethod === "return" && t.originalTransactionId
      );
      const regularTransactions = allTransactions.filter(
        (t) => (t.paymentMethod !== "return" || !t.originalTransactionId) && t.status !== "Voided"
      );
      const returnTransactionsMap = /* @__PURE__ */ new Map();
      returnTransactions.forEach((returnTrx) => {
        const originalId = returnTrx.originalTransactionId?.toString();
        if (originalId) {
          if (!returnTransactionsMap.has(originalId)) {
            returnTransactionsMap.set(originalId, []);
          }
          returnTransactionsMap.get(originalId).push(returnTrx);
        }
      });
      const transactionsWithReturns = regularTransactions.map((trx) => ({
        ...trx,
        returnTransactions: returnTransactionsMap.get(trx._id?.toString()) || []
      }));
      transactionsWithReturns.sort((a, b) => {
        const dateA = new Date(
          a.checkedOutAt || a.createdAt || a.updatedAt || 0
        );
        const dateB = new Date(
          b.checkedOutAt || b.createdAt || b.updatedAt || 0
        );
        return dateB - dateA;
      });
      const payload = transactionsWithReturns.length ? transactionsWithReturns : [];
      setTransactions(payload);
      setCachedDataRef.current("transactions", payload);
      setSelectedTransaction((prev) => {
        if (!payload.length) {
          return null;
        }
        if (prev && payload.some((t) => sameTransactionId(t, prev))) {
          return payload.find((t) => sameTransactionId(t, prev));
        }
        return payload[0];
      });
    } else {
      setTransactions([]);
      setCachedDataRef.current("transactions", []);
    }
  } catch (error) {
    console.error("Failed to load transactions:", error);
    setTransactions([]);
    setCachedDataRef.current("transactions", []);
  } finally {
    setLoading(false);
  }
}, [debouncedSearch, filters.method, filters.status, filters.user]);
useEffect(() => {
  if (hasLoaded.current) return;
```

```
    hasLoaded.current = true;
    const loadInitialData = async () => {
      try {
        invalidateCache("transactions");
        await fetchTransactions();
        isInitialMount.current = false;
        isInitialLoading.current = false;
      } catch (error) {
        console.error("Error loading transactions:", error);
        try {
          await fetchTransactions();
        } catch (fetchError) {
          console.error("Failed to fetch transactions:", fetchError);
          setTransactions([]);
          setLoading(false);
        }
        isInitialMount.current = false;
        isInitialLoading.current = false;
      }
    };
    loadInitialData();
  }, []);
  useEffect(() => {
    if (currentUserFilterId && currentUserFilterId !== "All") {
      setFilters(
        (prev) => prev.user === "All" ? { ...prev, user: currentUserFilterId } : prev
      );
    }
  }, [currentUserFilterId]);
  useEffect(
    () => {
      if (isInitialMount.current) {
        return;
      }
      fetchTransactions();
    },
    [
      debouncedSearch,
      filters.method,
      filters.status,
      filters.user,
      fetchTransactions
    ]
  );
  useEffect(() => {
    if (isInitialMount.current) return;
    const refresh = () => {
      if (document.visibilityState === "visible") {
        fetchTransactions();
      }
    };
    const intervalId = window.setInterval(refresh, 1e4);
    window.addEventListener("focus", refresh);
    document.addEventListener("visibilitychange", refresh);
    return () => {
      window.clearInterval(intervalId);
      window.removeEventListener("focus", refresh);
      document.removeEventListener("visibilitychange", refresh);
    };
  }, [fetchTransactions]);
  const generateSampleTransactions = () => [];
  const allRegularTransactions = useMemo(() => {
    return transactions.filter(
      (trx) => !(trx.paymentMethod === "return" && trx.originalTransactionId)
    );
  }, [transactions]);
  const matchesTransactionFilters = useCallback(
    (trx) => {
      if (trx.paymentMethod === "return" && trx.originalTransactionId) {
        return false;
      }
      if (trx.status === "Voided") {
        return false;
      }
      const matchesSearch = !search || trx.receiptNo?.toLowerCase().includes(search.toLowerCase());
      const matchesMethod = filters.method === "All" || trx.paymentMethod?.toLowerCase() ===
filters.method.toLowerCase();
```

```
      const matchesStatus = filters.status === "All" || trx.status === filters.status;
      const selectedEmp = filters.user !== "All" ? staffList.find((e) => String(e._id) === filters.user) :
null;
      const selectedEmpName = selectedEmp ? (selectedEmp.name || `${selectedEmp.firstName || ""}
${selectedEmp.lastName || ""}`).trim() : "";
      const matchesUser = filters.user === "All" || String(trx.performedById || "") === filters.user ||
selectedEmpName && trx.performedByName === selectedEmpName;
      let matchesDate = true;
      if (filters.date === "Custom") {
        if (startDate) {
          const end = endDate || startDate;
          const lo = startOfDay(startDate) <= startOfDay(end) ? startOfDay(startDate) : startOfDay(end);
          const hi = startOfDay(startDate) <= startOfDay(end) ? startOfDay(end) : startOfDay(startDate);
          const trxDay = startOfDay(saleDate(trx));
          matchesDate = trxDay >= lo && trxDay <= hi;
        }
      } else if (filters.date !== "All") {
        const trxDate = saleDate(trx);
        const now = /* @__PURE__ */ new Date();
        const today = new Date(
          now.getFullYear(),
          now.getMonth(),
          now.getDate()
        );
        if (filters.date === "Today") {
          matchesDate = isWithinInterval(trxDate, {
            start: startOfDay(now),
            end: endOfDay(now)
          });
        } else if (filters.date === "Last 7 days") {
          const sevenDaysAgo = new Date(today);
          sevenDaysAgo.setDate(sevenDaysAgo.getDate() - 7);
          matchesDate = trxDate >= sevenDaysAgo;
        } else if (filters.date === "Last 30 days") {
          const thirtyDaysAgo = new Date(today);
          thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);
          matchesDate = trxDate >= thirtyDaysAgo;
        }
      }
      return matchesSearch && matchesMethod && matchesStatus && matchesUser && matchesDate;
    },
    [search, filters, startDate, endDate, staffList]
  );
  const kpiFilteredTransactions = useMemo(() => {
    const filtered = transactions.filter((trx) => matchesTransactionFilters(trx));
    return filtered.sort((a, b) => {
      const dateA = new Date(a.checkedOutAt || a.createdAt || a.updatedAt || 0);
      const dateB = new Date(b.checkedOutAt || b.createdAt || b.updatedAt || 0);
      return dateB - dateA;
    });
  }, [transactions, matchesTransactionFilters]);
  const filteredTransactions = useMemo(
    () => kpiFilteredTransactions.filter(
      (trx) => trx.status !== "Pending" && trx.status !== "Failed"
    ),
    [kpiFilteredTransactions]
  );
  const returnedLogs = useMemo(() => {
    const rows = allRegularTransactions.map((trx) => {
      const returnEntries = Array.isArray(trx.returnTransactions) ? trx.returnTransactions : [];
      const hasReturnActivity = returnEntries.length > 0 || trx.status === "Returned" || trx.status ===
"Partially Returned";
      if (!hasReturnActivity) {
        return null;
      }
      const sortedReturns = [...returnEntries].sort((a, b) => {
        const aDate = new Date(a.checkedOutAt || a.createdAt || 0);
        const bDate = new Date(b.checkedOutAt || b.createdAt || 0);
        return bDate - aDate;
      });
      const latestReturn = sortedReturns[0];
      const reasons = /* @__PURE__ */ new Set();
      returnEntries.forEach((entry) => {
        (entry.items || []).forEach((item) => {
          const reason = String(item?.returnReason || "").trim();
          if (reason) reasons.add(reason);
        });
```

```
      });
      if (!reasons.size) {
        (trx.items || []).forEach((item) => {
          const reason = String(item?.returnReason || "").trim();
          if (reason) reasons.add(reason);
        });
      }
      const returnedAmountRaw = returnEntries.reduce((sum, entry) => {
        const explicitTotal = toNumber(entry.totalAmount, NaN);
        if (Number.isFinite(explicitTotal) && explicitTotal !== 0) {
          return sum + explicitTotal;
        }
        const itemTotal = (entry.items || []).reduce((acc, item) => {
          const unit = toNumber(item.price || item.itemPrice);
          const qty = toNumber(item.quantity, 1);
          return acc + unit * qty;
        }, 0);
        return sum + itemTotal;
      }, 0);
      const returnedAmount = Math.abs(returnedAmountRaw);
      const remainingAmount = Math.max(0, toNumber(trx.totalAmount));
      const itemOriginalAmount = originalLineSubtotalFromItems(trx);
      const inferredOriginalAmount = itemOriginalAmount || remainingAmount + returnedAmount;
      const originalAmount = Math.max(inferredOriginalAmount, remainingAmount + returnedAmount);
      const discountedAmount = Math.max(
        0,
        originalAmount - (remainingAmount + returnedAmount)
      );
      return {
        _id: trx._id,
        receiptNo: trx.receiptNo,
        transactionId: trx.referenceNo || trx._id?.substring(0, 12) || "---",
        returnedAt: new Date(
          latestReturn?.checkedOutAt || latestReturn?.createdAt || trx.updatedAt || trx.checkedOutAt ||
trx.createdAt
        ),
        performedByName: trx.performedByName || "Staff",
        performedById: String(trx.performedById || ""),
        returnedByName: latestReturn?.performedByName || trx.performedByName || "Staff",
        returnedById: String(latestReturn?.performedById || trx.performedById || ""),
        reason: Array.from(reasons).join(", ") || "Returned item(s)",
        originalAmount,
        discountedAmount,
        totalAmount: remainingAmount,
        returnedAmount
      };
    }).filter(Boolean);
    rows.sort((a, b) => b.returnedAt - a.returnedAt);
    return rows;
  }, [allRegularTransactions]);
  const filteredReturnedLogs = useMemo(() => {
    return returnedLogs.filter((row) => {
      const receiptLabel = row.receiptNo ? `#${row.receiptNo}` : "";
      const matchesSearch = !search || receiptLabel.toLowerCase().includes(search.toLowerCase()) ||
String(row.transactionId).toLowerCase().includes(search.toLowerCase());
      const selectedEmp = filters.user !== "All" ? staffList.find((e) => String(e._id) === filters.user) :
null;
      const selectedEmpName = selectedEmp ? (selectedEmp.name || `${selectedEmp.firstName || ""}
${selectedEmp.lastName || ""}`).trim() : "";
      const matchesUser = filters.user === "All" || row.performedById === filters.user || selectedEmpName &&
row.performedByName === selectedEmpName;
      const selectedReturnedByEmp = filters.returnedBy !== "All" ? staffList.find((e) => String(e._id) ===
filters.returnedBy) : null;
      const selectedReturnedByName = selectedReturnedByEmp ? (selectedReturnedByEmp.name ||
`${selectedReturnedByEmp.firstName || ""} ${selectedReturnedByEmp.lastName || ""}`).trim() : "";
      const matchesReturnedBy = filters.returnedBy === "All" || row.returnedById === filters.returnedBy ||
selectedReturnedByName && row.returnedByName === selectedReturnedByName;
      let matchesDate = true;
      if (filters.date === "Custom") {
        if (startDate) {
          const end = endDate || startDate;
          const lo = startOfDay(startDate) <= startOfDay(end) ? startOfDay(startDate) : startOfDay(end);
          const hi = startOfDay(startDate) <= startOfDay(end) ? startOfDay(end) : startOfDay(startDate);
          const rowDay = startOfDay(row.returnedAt);
          matchesDate = rowDay >= lo && rowDay <= hi;
        }
      } else if (filters.date !== "All") {
```

```
        const rowDate = row.returnedAt;
        const now = /* @__PURE__ */ new Date();
        const today = new Date(
          now.getFullYear(),
          now.getMonth(),
          now.getDate()
        );
        if (filters.date === "Today") {
          matchesDate = isWithinInterval(rowDate, {
            start: startOfDay(now),
            end: endOfDay(now)
          });
        } else if (filters.date === "Last 7 days") {
          const sevenDaysAgo = new Date(today);
          sevenDaysAgo.setDate(sevenDaysAgo.getDate() - 7);
          matchesDate = rowDate >= sevenDaysAgo;
        } else if (filters.date === "Last 30 days") {
          const thirtyDaysAgo = new Date(today);
          thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);
          matchesDate = rowDate >= thirtyDaysAgo;
        }
      }
      return matchesSearch && matchesUser && matchesReturnedBy && matchesDate;
    });
  }, [returnedLogs, search, filters.user, filters.returnedBy, filters.date, startDate, endDate, staffList]);
  const paginatedTransactions = useMemo(() => {
    const startIndex = (currentPage - 1) * rowsPerPage;
    return filteredTransactions.slice(startIndex, startIndex + rowsPerPage);
  }, [filteredTransactions, currentPage]);
  const paginatedReturnedLogs = useMemo(() => {
    const startIndex = (currentPage - 1) * rowsPerPage;
    return filteredReturnedLogs.slice(startIndex, startIndex + rowsPerPage);
  }, [filteredReturnedLogs, currentPage]);
  const paginatedTransactionIds = useMemo(
    () => paginatedTransactions.map((trx) => trx._id).filter(Boolean),
    [paginatedTransactions]
  );
  const allVisibleTransactionsSelected = paginatedTransactionIds.length > 0 &&
paginatedTransactionIds.every((id) => selectedTransactionIds.includes(id));
  const someVisibleTransactionsSelected = paginatedTransactionIds.some(
    (id) => selectedTransactionIds.includes(id)
  );
  const paginatedReturnedLogIds = useMemo(
    () => paginatedReturnedLogs.map((row) => row._id).filter(Boolean),
    [paginatedReturnedLogs]
  );
  const allVisibleReturnedLogsSelected = paginatedReturnedLogIds.length > 0 &&
paginatedReturnedLogIds.every((id) => selectedReturnedLogIds.includes(id));
  const someVisibleReturnedLogsSelected = paginatedReturnedLogIds.some(
    (id) => selectedReturnedLogIds.includes(id)
  );
  const kpis = useMemo(() => {
    const list = kpiFilteredTransactions;
    const totalSales = list.reduce((sum, trx) => {
      if (String(trx.paymentMethod || "").toLowerCase() === "return") return sum;
      return sum + (parseFloat(trx.totalAmount) || 0);
    }, 0);
    const transactionTotal = list.filter((trx) => {
      if (/^voided$/i.test(String(trx.status || ""))) return false;
      if (String(trx.paymentMethod || "").toLowerCase() === "return") return false;
      return true;
    }).length;
    const returnedItems = list.reduce((sum, trx) => {
      const items = Array.isArray(trx.items) ? trx.items : [];
      const returnedCount = items.filter((item) => {
        const rs = item?.returnStatus;
        return rs === "Returned" || rs === "Partially Returned";
      }).length;
      return sum + returnedCount;
    }, 0);
    return { totalSales, transactionTotal, returnedItems };
  }, [kpiFilteredTransactions]);
  const sidebarReceiptTotals = useMemo(() => {
    const trx = selectedTransaction;
    if (!trx) return { lineSub: 0, discount: 0 };
    const lineSub = originalLineSubtotalFromItems(trx) || trx.totalAmount || 0;
    const hasReturnActivity = (trx.returnTransactions?.length || 0) > 0 || trx.status === "Returned" ||
```

```
trx.status === "Partially Returned";
    const discount = resolveTransactionDiscount(trx, lineSub, {
      skipInference: hasReturnActivity
    });
    return { lineSub, discount };
  }, [selectedTransaction]);
  useEffect(() => {
    setSelectedTransactionIds(
      (prev) => prev.filter((id) => filteredTransactions.some((trx) => trx._id === id))
    );
  }, [filteredTransactions]);
  useEffect(() => {
    setSelectedReturnedLogIds(
      (prev) => prev.filter((id) => filteredReturnedLogs.some((row) => row._id === id))
    );
  }, [filteredReturnedLogs]);
  useEffect(() => {
    setCurrentPage(1);
    setIsExportSelectionMode(false);
    setSelectedTransactionIds([]);
    setSelectedReturnedLogIds([]);
  }, [activeTab]);
  useEffect(
    () => {
      if (selectAllTransactionsRef.current) {
        selectAllTransactionsRef.current.indeterminate = isExportSelectionMode
&& !allVisibleTransactionsSelected && someVisibleTransactionsSelected;
      }
      if (selectAllReturnedLogsRef.current) {
        selectAllReturnedLogsRef.current.indeterminate = isExportSelectionMode
&& !allVisibleReturnedLogsSelected && someVisibleReturnedLogsSelected;
      }
    },
    [
      isExportSelectionMode,
      allVisibleTransactionsSelected,
      someVisibleTransactionsSelected,
      allVisibleReturnedLogsSelected,
      someVisibleReturnedLogsSelected
    ]
  );
  const userDropdownOptions = useMemo(() => {
    const fromStaff = staffList.map((e) => ({
      value: String(e._id),
      label: (e.name || `${e.firstName || ""} ${e.lastName || ""}`).trim() || "Unknown"
    })).sort((a, b) => a.label.localeCompare(b.label));
    const seen = new Set(fromStaff.map((r) => r.value));
    const fromTx = [];
    transactions.forEach((t) => {
      const id = t.performedById ? String(t.performedById) : "";
      if (id && t.performedByName && !seen.has(id)) {
        fromTx.push({ value: id, label: t.performedByName });
        seen.add(id);
      }
    });
    fromTx.sort((a, b) => a.label.localeCompare(b.label));
    return [{ value: "All", label: "All" }, ...fromStaff, ...fromTx];
  }, [staffList, transactions]);
  const returnedByDropdownOptions = useMemo(() => {
    const fromStaff = staffList.map((e) => ({
      value: String(e._id),
      label: (e.name || `${e.firstName || ""} ${e.lastName || ""}`).trim() || "Unknown"
    })).sort((a, b) => a.label.localeCompare(b.label));
    const seen = new Set(fromStaff.map((r) => r.value));
    const fromReturns = [];
    returnedLogs.forEach((row) => {
      const id = row.returnedById ? String(row.returnedById) : "";
      if (id && row.returnedByName && !seen.has(id)) {
        fromReturns.push({ value: id, label: row.returnedByName });
        seen.add(id);
      }
    });
    fromReturns.sort((a, b) => a.label.localeCompare(b.label));
    return [{ value: "All", label: "All" }, ...fromStaff, ...fromReturns];
  }, [staffList, returnedLogs]);
  const handleRowClick = (trx) => {
    setSelectedTransaction(trx);
```

```
  };
  const handleToggleTransactionSelection = (transactionId) => {
    if (!transactionId) return;
    setSelectedTransactionIds(
      (prev) => prev.includes(transactionId) ? prev.filter((id) => id !== transactionId) : [...prev,
transactionId]
    );
  };
  const handleToggleSelectAllTransactions = () => {
    setSelectedTransactionIds((prev) => {
      if (allVisibleTransactionsSelected) {
        return prev.filter((id) => !paginatedTransactionIds.includes(id));
      }
      const merged = new Set(prev);
      paginatedTransactionIds.forEach((id) => merged.add(id));
      return Array.from(merged);
    });
  };
  const handleToggleReturnedLogSelection = (logId) => {
    if (!logId) return;
    setSelectedReturnedLogIds(
      (prev) => prev.includes(logId) ? prev.filter((id) => id !== logId) : [...prev, logId]
    );
  };
  const handleToggleSelectAllReturnedLogs = () => {
    setSelectedReturnedLogIds((prev) => {
      if (allVisibleReturnedLogsSelected) {
        return prev.filter((id) => !paginatedReturnedLogIds.includes(id));
      }
      const merged = new Set(prev);
      paginatedReturnedLogIds.forEach((id) => merged.add(id));
      return Array.from(merged);
    });
  };
  const handleExportButtonClick = () => {
    if (!isExportSelectionMode) {
      setIsExportSelectionMode(true);
      if (activeTab === "returned") {
        setSelectedReturnedLogIds([]);
      } else {
        setSelectedTransactionIds([]);
      }
      return;
    }
    if (activeTab === "returned") {
      handleExportReturnedLogsToCSV();
      return;
    }
    handleExportToCSV();
  };
  const handleCancelExportSelection = () => {
    setIsExportSelectionMode(false);
    setSelectedTransactionIds([]);
    setSelectedReturnedLogIds([]);
  };
  const renderStatusPill = (status = "Completed") => /* @__PURE__ */ React.createElement(
    "span",
    {
      className: `inline-flex items-center gap-1 px-3 py-1 rounded-full text-xs font-semibold transition-
transform ${STATUS_STYLES[status]}`
    },
    statusIcon[status],
    status
  );
  const isTransactionReturnable = (transaction) => {
    if (!transaction || !transaction.checkedOutAt) return false;
    const transactionDate = new Date(transaction.checkedOutAt);
    const now = /* @__PURE__ */ new Date();
    const diffTime = Math.abs(now - transactionDate);
    const diffHours = diffTime / (1e3 * 60 * 60);
    return diffHours <= 48;
  };
  const handleViewClick = (transaction) => {
    setTransactionToView(transaction);
    setShowViewModal(true);
  };
  const handlePrintClick = (transaction) => {
```

```
    setTransactionToPrint(transaction);
    setShowPrintModal(true);
  };
  const handleExportToCSV = () => {
    try {
      const transactionsToExport = selectedTransactionIds.length > 0 ? filteredTransactions.filter(
        (trx) => selectedTransactionIds.includes(trx._id)
      ) : [];
      if (transactionsToExport.length === 0) {
        alert("Please select at least one transaction to export.");
        return;
      }
      const headers = [
        "Receipt No.",
        "Transaction ID",
        "Date",
        "Time",
        "User ID",
        "Performed By ID",
        "Performed By Name",
        "Payment Method",
        "Reference No.",
        "Total Amount",
        "Amount Received",
        "Change Given",
        "Status",
        "Item Count",
        "Items (Name)",
        "Items (SKU)",
        "Items (Variant)",
        "Items (Size)",
        "Items (Qty)",
        "Items (Price)",
        "Items (Subtotal)",
        "Voided By",
        "Voided By Name",
        "Voided At",
        "Void Reason",
        "Original Transaction ID",
        "Created At",
        "Updated At"
      ];
      const escapeCSV = (value) => {
        if (value === null || value === void 0) return "";
        const str = String(value);
        if (str.includes(",") || str.includes('"') || str.includes("\n")) {
          return `"${str.replace(/"/g, '""')}"`;
        }
        return str;
      };
      const csvRows = transactionsToExport.map((trx) => {
        const checkoutDate = trx.checkedOutAt ? new Date(trx.checkedOutAt) : new Date(trx.createdAt);
        const createdDate = trx.createdAt ? new Date(trx.createdAt) : null;
        const updatedDate = trx.updatedAt ? new Date(trx.updatedAt) : null;
        const voidedDate = trx.voidedAt ? new Date(trx.voidedAt) : null;
        const itemNames = trx.items?.map((item) => item.itemName || "").join("; ") || "";
        const itemSkus = trx.items?.map((item) => item.sku || "").join("; ") || "";
        const itemVariants = trx.items?.map((item) => item.variant || "").join("; ") || "";
        const itemSizes = trx.items?.map((item) => item.selectedSize || "").join("; ") || "";
        const itemQtys = trx.items?.map((item) => item.quantity || 0).join("; ") || "";
        const itemPrices = trx.items?.map((item) => item.price || 0).join("; ") || "";
        const itemSubtotals = trx.items?.map((item) => (item.quantity || 0) * (item.price || 0)).join("; ")
|| "";
        return [
          escapeCSV(trx.receiptNo || ""),
          escapeCSV(trx._id || ""),
          escapeCSV(checkoutDate.toLocaleDateString()),
          escapeCSV(checkoutDate.toLocaleTimeString()),
          escapeCSV(trx.userId || ""),
          escapeCSV(trx.performedById || ""),
          escapeCSV(trx.performedByName || ""),
          escapeCSV(trx.paymentMethod || ""),
          escapeCSV(trx.referenceNo || ""),
          escapeCSV(trx.totalAmount || 0),
          escapeCSV(trx.amountReceived || 0),
          escapeCSV(trx.changeGiven || 0),
          escapeCSV(trx.status || ""),
```

```
        escapeCSV(trx.items?.length || 0),
        escapeCSV(itemNames),
        escapeCSV(itemSkus),
        escapeCSV(itemVariants),
        escapeCSV(itemSizes),
        escapeCSV(itemQtys),
        escapeCSV(itemPrices),
        escapeCSV(itemSubtotals),
        escapeCSV(trx.voidedBy || ""),
        escapeCSV(trx.voidedByName || ""),
        escapeCSV(voidedDate ? voidedDate.toLocaleString() : ""),
        escapeCSV(trx.voidReason || ""),
        escapeCSV(trx.originalTransactionId || ""),
        escapeCSV(createdDate ? createdDate.toLocaleString() : ""),
        escapeCSV(updatedDate ? updatedDate.toLocaleString() : "")
      ].join(",");
    });
    const csvContent = [headers.join(","), ...csvRows].join("\n");
    const blob = new Blob([csvContent], { type: "text/csv;charset=utf-8;" });
    const url = URL.createObjectURL(blob);
    const link = document.createElement("a");
    link.setAttribute("href", url);
    link.setAttribute(
      "download",
      `transactions_export_${(/* @__PURE__ */ new Date()).toISOString().split("T")[0]}.csv`
    );
    link.style.visibility = "hidden";
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
    alert("Transactions exported successfully!");
    setIsExportSelectionMode(false);
    setSelectedTransactionIds([]);
  } catch (error) {
    console.error("Error exporting transactions:", error);
    alert("Failed to export transactions. Please try again.");
  }
};
const handleExportReturnedLogsToCSV = () => {
  try {
    const rowsToExport = selectedReturnedLogIds.length > 0 ? filteredReturnedLogs.filter(
      (row) => selectedReturnedLogIds.includes(row._id)
    ) : [];
    if (rowsToExport.length === 0) {
      alert("Please select at least one returned log to export.");
      return;
    }
    const headers = [
      "Receipt No.",
      "Transaction ID",
      "Returned Date",
      "Performed By",
      "Returned By",
      "Reason",
      "Original Amount",
      "Discounted Amount",
      "Returned Amount",
      "Final Total"
    ];
    const escapeCSV = (value) => {
      if (value === null || value === void 0) return "";
      const str = String(value);
      if (str.includes(",") || str.includes('"') || str.includes("\n")) {
        return `"${str.replace(/"/g, '""')}"`;
      }
      return str;
    };
    const csvRows = rowsToExport.map(
      (row) => [
        escapeCSV(row.receiptNo ? `#${row.receiptNo}` : ""),
        escapeCSV(row.transactionId || ""),
        escapeCSV(
          row.returnedAt ? row.returnedAt.toLocaleDateString() : ""
        ),
        escapeCSV(row.performedByName || ""),
        escapeCSV(row.returnedByName || ""),
        escapeCSV(row.reason || ""),
```

```
        escapeCSV(row.originalAmount || 0),
        escapeCSV(row.discountedAmount || 0),
        escapeCSV(-Math.abs(row.returnedAmount || 0)),
        escapeCSV(row.totalAmount || 0)
      ].join(",")
    );
    const csvContent = [headers.join(","), ...csvRows].join("\n");
    const blob = new Blob([csvContent], { type: "text/csv;charset=utf-8;" });
    const url = URL.createObjectURL(blob);
    const link = document.createElement("a");
    link.setAttribute("href", url);
    link.setAttribute(
      "download",
      `returned_logs_export_${(/* @__PURE__ */ new Date()).toISOString().split("T")[0]}.csv`
    );
    link.style.visibility = "hidden";
    document.body.appendChild(link);
    link.click();
    document.body.removeChild(link);
    alert("Returned logs exported successfully!");
    setIsExportSelectionMode(false);
    setSelectedReturnedLogIds([]);
  } catch (error) {
    console.error("Error exporting returned logs:", error);
    alert("Failed to export returned logs. Please try again.");
  }
};
const handleImportFromCSV = async (event) => {
  const file = event.target.files?.[0];
  if (!file) return;
  try {
    alert(
      "Transaction import is not supported. Transactions are created through the POS terminal."
    );
    event.target.value = "";
  } catch (error) {
    console.error("Error with import:", error);
    event.target.value = "";
  }
};
const handleReturnClick = (transaction) => {
  if (!isTransactionReturnable(transaction)) {
    alert("This transaction is more than 2 days old and cannot be returned.");
    return;
  }
  if (transaction.status === "Returned" || transaction.status === "Voided") {
    alert("This transaction has already been returned or voided.");
    return;
  }
  setTransactionToReturn(transaction);
  setShowReturnModal(true);
};
const handleReturnConfirm = async (itemsToReturn, transaction) => {
  try {
    setLoading(true);
    console.log("Processing return for items:", itemsToReturn);
    const returnedIndices = itemsToReturn.map((item) => item.originalIndex);
    const isArchiveReturnReason = (reason) => {
      const raw = String(reason || "").trim();
      const head = raw.split(":")[0].trim().toLowerCase();
      const archiveHeads = /* @__PURE__ */ new Set([
        "damaged",
        "defective",
        "expired"
      ]);
      if (archiveHeads.has(head)) return true;
      return false;
    };
    const damagedItems = itemsToReturn.filter(
      (item) => isArchiveReturnReason(item.reason)
    );
    const returnableItems = itemsToReturn.filter(
      (item) => !isArchiveReturnReason(item.reason)
    );
    console.log("Damaged items (to archive):", damagedItems.length);
    console.log("Returnable items (to stock):", returnableItems.length);
    const updatedItems = transaction.items.map((item, idx) => {
```

```
  if (returnedIndices.includes(idx)) {
    const returnedItem = itemsToReturn.find(
      (ri) => ri.originalIndex === idx
    );
    const returnedQty = returnedItem?.quantity || item.quantity;
    const originalQty = item.quantity;
    if (returnedQty >= originalQty) {
      return {
        ...item,
        returnStatus: "Returned",
        returnReason: returnedItem?.reason || "Returned",
        returnedQuantity: originalQty
      };
    } else {
      return {
        ...item,
        quantity: originalQty - returnedQty,
        returnStatus: "Partially Returned",
        returnReason: returnedItem?.reason || "Returned",
        returnedQuantity: returnedQty
      };
    }
  }
  return item;
});
const fullyReturnedCount = updatedItems.filter(
  (item) => item.returnStatus === "Returned"
).length;
const partiallyReturnedCount = updatedItems.filter(
  (item) => item.returnStatus === "Partially Returned"
).length;
const allItemsFullyReturned = fullyReturnedCount === updatedItems.length;
const hasAnyReturns = fullyReturnedCount > 0 || partiallyReturnedCount > 0;
let newStatus = "Completed";
if (allItemsFullyReturned) {
  newStatus = "Returned";
} else if (hasAnyReturns) {
  newStatus = "Partially Returned";
}
const newTotalAmount = updatedItems.filter((item) => item.returnStatus !== "Returned").reduce(
  (sum, item) => sum + item.quantity * (item.price || item.itemPrice || 0),
  0
);
const updatePayload = {
  status: newStatus,
  items: updatedItems,
  totalAmount: newTotalAmount
};
console.log("Updating original transaction FIRST:", updatePayload);
const updateResponse = await fetch(
  `${API_BASE_URL}/api/transactions/${transaction._id}`,
  {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(updatePayload)
  }
);
const updateData = await updateResponse.json();
console.log("Transaction update response:", updateData);
if (!updateResponse.ok || !updateData.success) {
  throw new Error(updateData.message || "Failed to update transaction");
}
const updatedOriginalTransaction = updateData.data || {
  ...transaction,
  ...updatePayload
};
setTransactions((prev) => {
  const next = prev.map(
    (trx) => sameTransactionId(trx, transaction) ? { ...trx, ...updatedOriginalTransaction } : trx
  );
  setCachedDataRef.current("transactions", next);
  return next;
});
setSelectedTransaction((prev) => {
  if (!prev || !sameTransactionId(prev, transaction)) {
    return prev;
  }
```

```
        return { ...prev, ...updatedOriginalTransaction };
      });
      for (const item of damagedItems) {
        let productDetails = null;
        try {
          const productResponse = await fetch(
            `${API_BASE_URL}/api/products/${item.productId}`
          );
          const productData = await productResponse.json();
          if (productData.success) {
            productDetails = productData.data;
          }
        } catch (error) {
          console.warn("Failed to fetch product details for archiving:", error);
        }
        const archivePayload = {
          productId: item.productId,
          itemName: item.itemName,
          sku: item.sku || "N/A",
          variant: item.variant || "",
          selectedSize: item.selectedSize || "",
          category: productDetails?.category || "Others",
          brandName: productDetails?.brandName || "",
          itemPrice: item.price || 0,
          costPrice: productDetails?.costPrice || 0,
          quantity: item.quantity,
          itemImage: productDetails?.itemImage || "",
          reason: item.reason,
          returnReason: item.reason,
          originalTransactionId: transaction._id,
          archivedBy: transaction.performedByName || "System",
          archivedById: transaction.performedById || "",
          notes: `Returned due to: ${item.reason}`
        };
        console.log("Archiving item:", archivePayload);
        const archiveResponse = await fetch(
          `${API_BASE_URL}/api/archive`,
          {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(archivePayload)
          }
        );
        const archiveData = await archiveResponse.json();
        console.log("Archive response:", archiveData);
        if (!archiveData.success) {
          console.error("Failed to archive item:", archiveData);
          continue;
        }
      }
      let stockUpdateFailed = false;
      if (returnableItems.length > 0) {
        const stockUpdatePayload = {
          items: returnableItems.map((item) => {
            const orig = transaction.items?.[item.originalIndex];
            const sizeRaw = item.selectedSize || item.size || orig?.selectedSize || orig?.size || "";
            const size = sizeRaw && String(sizeRaw).trim() ? String(sizeRaw).trim() : null;
            const variantRaw = item.variant || item.selectedVariation || orig?.variant ||
orig?.selectedVariation || "";
            const variant = variantRaw && String(variantRaw).trim() ? String(variantRaw).trim() : null;
            return {
              _id: item.productId,
              sku: item.sku || orig?.sku,
              size,
              selectedSize: size,
              variant,
              selectedVariation: variant,
              quantity: item.quantity,
              price: item.price ?? orig?.price ?? orig?.itemPrice,
              originalTransactionId: transaction._id,
              originalLineIndex: item.originalIndex,
              batchAllocations: orig?.batchAllocations
            };
          }),
          performedByName: transaction.performedByName || "System",
          performedById: transaction.performedById || "",
          reason: "Returned Item",
```

```
          type: "Stock-In"
        };
        console.log("Updating stock (Stock-In):", stockUpdatePayload);
        const stockResponse = await fetch(
          `${API_BASE_URL}/api/products/update-stock`,
          {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(stockUpdatePayload)
          }
        );
        let stockData = {};
        try {
          stockData = await stockResponse.json();
        } catch {
          stockData = {};
        }
        console.log("Stock update response:", stockData);
        if (!stockResponse.ok || !stockData.success) {
          stockUpdateFailed = true;
          console.error("Failed to update stock:", stockData);
          alert(
            stockData.message || "Inventory could not be updated. Ensure this sale recorded size and variant
correctly, then try again or adjust stock manually."
          );
        }
      }
      for (const item of itemsToReturn) {
        const returnTransaction = {
          userId: transaction.userId,
          items: [
            {
              productId: item.productId,
              itemName: item.itemName,
              sku: item.sku,
              variant: item.variant,
              selectedSize: item.selectedSize,
              quantity: item.quantity,
              price: item.price,
              returnReason: item.reason
            }
          ],
          paymentMethod: "return",
          amountReceived: 0,
          changeGiven: 0,
          referenceNo: `RET-${transaction.referenceNo || transaction._id?.substring(0, 12)}-${Date.now()}-
${itemsToReturn.indexOf(item)}`,
          receiptNo: null,
          totalAmount: item.quantity * item.price,
          performedById: transaction.performedById,
          performedByName: transaction.performedByName,
          status: "Returned",
          originalTransactionId: transaction._id,
          checkedOutAt: /* @__PURE__ */ new Date()
        };
        console.log("Creating return transaction:", returnTransaction);
        const returnTrxResponse = await fetch(
          `${API_BASE_URL}/api/transactions`,
          {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(returnTransaction)
          }
        );
        const returnTrxData = await returnTrxResponse.json();
        console.log("Return transaction response:", returnTrxData);
      }
      await fetchTransactions();
      if (!stockUpdateFailed) {
        setShowReturnSuccessModal(true);
      }
    } catch (error) {
      console.error("Error processing return:", error);
      alert("Failed to process return. Please try again.");
    } finally {
      setLoading(false);
    }
```

```
  };
  const isReturnedLogsTab = activeTab === "returned";
  const activeRowCount = isReturnedLogsTab ? filteredReturnedLogs.length : filteredTransactions.length;
  const totalPages = Math.ceil(activeRowCount / rowsPerPage) || 1;
  const transactionTableColumnCount = isExportSelectionMode ? 11 : 10;
  const returnedTableColumnCount = isExportSelectionMode ? 10 : 9;
  const showingStart = activeRowCount === 0 ? 0 : (currentPage - 1) * rowsPerPage + 1;
  const showingEnd = Math.min(currentPage * rowsPerPage, activeRowCount);
  if (isInitialLoading.current && transactions.length === 0) {
    return /* @__PURE__ */ React.createElement(
      "div",
      {
        className: "p-6 min-h-screen flex items-center justify-center",
        style: { background: "#FFFFFF" }
      },
      /* @__PURE__ */ React.createElement("div", { className: "text-center" }, /* @__PURE__ */
React.createElement("div", { className: "inline-block animate-spin rounded-full h-12 w-12 border-b-2 border-
[#8B7355] mb-4" }), /* @__PURE__ */ React.createElement("p", { className: "text-gray-600" }, "Loading
transactions..."))
    );
  }
  return /* @__PURE__ */ React.createElement(
    "div",
    {
      className: `p-6 h-screen overflow-hidden flex flex-col ${theme === "dark" ? "bg-[#1E1B18]" : "bg-
[#F9FAFB]"}`
    },
    /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(
      Header,
      {
        pageName: "Transactions",
        showBorder: false,
        profileBackground: ""
      }
    ), /* @__PURE__ */ React.createElement("div", { className: "mt-4 mb-4 flex items-center gap-3" }, /*
@__PURE__ */ React.createElement(
      "button",
      {
        type: "button",
        onClick: () => setActiveTab("transactions"),
        className: `px-5 py-2.5 rounded-xl text-base font-semibold border transition-all ${theme === "dark" ?
"bg-[#2A2724] border-gray-600 text-gray-200" : "bg-white border-gray-200 text-[#22314A]"} ${activeTab ===
"transactions" ? "shadow-[0_4px_0_0_rgba(173,127,101,0.75)] !text-[#AD7F65]" : "shadow-
[0_6px_14px_rgba(15,23,42,0.08)] hover:-translate-y-0.5"}`
      },
      "Transaction Logs"
    ), /* @__PURE__ */ React.createElement(
      "button",
      {
        type: "button",
        onClick: () => setActiveTab("returned"),
        className: `px-5 py-2.5 rounded-xl text-base font-semibold border transition-all ${theme === "dark" ?
"bg-[#2A2724] border-gray-600 text-gray-200" : "bg-white border-gray-200 text-[#22314A]"} ${activeTab ===
"returned" ? "shadow-[0_4px_0_0_rgba(173,127,101,0.75)] !text-[#AD7F65]" : "shadow-
[0_6px_14px_rgba(15,23,42,0.08)] hover:-translate-y-0.5"}`
      },
      "Returned Logs"
    )), /* @__PURE__ */ React.createElement("div", { className: "grid grid-cols-1 sm:grid-cols-2 lg:grid-
cols-4 gap-4 mb-6 mt-4 w-full" }, /* @__PURE__ */ React.createElement("div", { className: "contents" }, [
      {
        label: "Total Sales",
        value: formatCurrencyCompact(kpis.totalSales),
        barGradient: "linear-gradient(180deg, #2563EB 0%, #60A5FA 100%)",
        textColor: "#2563EB",
        iconBgClass: "bg-blue-100",
        icon: /* @__PURE__ */ React.createElement(
          "img",
          {
            src: ShopBagSales,
            alt: "Total Sales",
            className: "w-10 h-10"
          }
        )
      },
      {
        label: "Total Transactions",
        value: kpis.transactionTotal,
```

```
        barGradient: "linear-gradient(180deg, #22C55E 0%, #4ADE80 100%)",
        textColor: "#22C55E",
        iconBgClass: "bg-green-100",
        icon: /* @__PURE__ */ React.createElement(
          "img",
          {
            src: TransactionsTotalGreen,
            alt: "Transactions",
            className: "w-10 h-10"
          }
        )
      },
      {
        label: "Returned",
        value: kpis.returnedItems,
        barGradient: "linear-gradient(180deg, #D97706 0%, #F59E0B 100%)",
        textColor: "#F59E0B",
        iconBgClass: "bg-orange-100",
        icon: /* @__PURE__ */ React.createElement(FaUndoAlt, { className: "text-xl" })
      }
    ].map((card) => /* @__PURE__ */ React.createElement(
      motion.div,
      {
        key: card.label,
        whileHover: { y: -4 },
        whileTap: { scale: 0.98 },
        className: `rounded-2xl shadow-md flex items-center justify-between px-5 py-4 relative overflow-
hidden text-left w-full min-h-[92px] ${theme === "dark" ? "bg-[#2A2724]" : "bg-white"}`
      },
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: "absolute left-0 top-0 bottom-0 w-2",
          style: { backgroundImage: card.barGradient }
        }
      ),
      /* @__PURE__ */ React.createElement("div", { className: "ml-2" }, /* @__PURE__ */ React.createElement(
        motion.p,
        {
          key: card.value,
          initial: { opacity: 0, y: 6 },
          animate: { opacity: 1, y: 0 },
          className: "text-2xl lg:text-3xl font-extrabold",
          style: { color: card.textColor }
        },
        card.value
      ), /* @__PURE__ */ React.createElement("p", { className: "text-xs mt-0.5", style: { color:
card.textColor } }, card.label)),
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `w-14 h-14 rounded-full flex items-center justify-center ${card.iconBgClass}`,
          style: { color: card.textColor }
        },
        card.icon
      )
    )), /* @__PURE__ */ React.createElement(
      motion.div,
      {
        whileHover: { y: -4 },
        whileTap: { scale: 0.98 },
        className: `rounded-2xl shadow-md flex items-center justify-between px-5 py-4 relative overflow-
hidden cursor-pointer ${theme === "dark" ? "bg-[#1a2332]" : "bg-[#E8F0FE]"} w-full min-h-[92px]`,
        onClick: () => setShowRemittanceModal(true)
      },
      /* @__PURE__ */ React.createElement("div", null, /* @__PURE__ */ React.createElement("p", { className:
`text-base font-bold ${theme === "dark" ? "text-blue-300" : "text-[#1a3a5c]"}` }, "Ready to Remit?"), /*
@__PURE__ */ React.createElement("p", { className: `text-xs mt-0.5 ${theme === "dark" ? "text-blue-400/70" :
"text-[#5a7a9a]"}` }, "Submit you remittance by the end of the day."), /* @__PURE__ */ React.createElement(
        "button",
        {
          className: "mt-2 bg-[#1a3a5c] hover:bg-[#0f2a4a] text-white text-xs font-semibold px-4 py-2
rounded-lg flex items-center gap-2 transition-colors cursor-pointer",
          onClick: (e) => {
            e.stopPropagation();
            setShowRemittanceModal(true);
          }
```

```
        },
        /* @__PURE__ */ React.createElement(FaClipboardList, { className: "text-xs" }),
        "Start Remittance"
      )),
      /* @__PURE__ */ React.createElement("div", { className: `w-14 h-14 rounded-full flex items-center
justify-center ${theme === "dark" ? "bg-blue-900/40" : "bg-[#d0e0f0]"}` }, /* @__PURE__ */
React.createElement(
        "img",
        {
          src: HandCashIcon,
          alt: "Hand Cash",
          className: "w-8 h-8"
        }
      ))
    ))), /* @__PURE__ */ React.createElement("div", { className: "flex-1 min-h-0 overflow-hidden flex flex-
col lg:flex-row gap-6" }), /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `flex-1 min-h-0 flex flex-col rounded-2xl shadow-[0_12px_40px_rgba(0,0,0,0.06)] p-6 border
overflow-hidden ${theme === "dark" ? "bg-[#2A2724] border-[#4A4037]" : "bg-white border-white/80"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "flex flex-col xl:flex-row xl:items-center gap-
4 mb-4" }), /* @__PURE__ */ React.createElement("div", { className: "relative flex-1 min-w-0 w-full xl:max-w-
xs 2xl:max-w-sm" }), /* @__PURE__ */ React.createElement(FaSearch, { className: "absolute left-4 top-1/2 -
translate-y-1/2 text-[#AD7F65]" }), /* @__PURE__ */ React.createElement(
        "input",
        {
          value: search,
          onChange: (e) => {
            setSearch(e.target.value);
            setCurrentPage(1);
          },
          placeholder: "Search by receipt number...",
          className: `w-full border rounded-2xl h-12 pl-12 pr-4 shadow-inner focus:outline-none focus:border-
[#AD7F65] focus:ring focus:ring-[#AD7F65]/20 transition-all ${theme === "dark" ? "bg-[#1E1B18] border-gray-
600 text-white placeholder-gray-500" : "bg-white border-gray-200 text-gray-900"}`
        }
      )), /* @__PURE__ */ React.createElement("div", { className: "flex flex-wrap items-center gap-3" }, /*
@__PURE__ */ React.createElement(
        Dropdown,
        {
          label: "Date",
          options: dateOptions,
          selected: filters.date,
          showAllAsOptionLabel: true,
          onSelect: (value) => {
            setCurrentPage(1);
            if (value !== "Custom") {
              setDateRange([null, null]);
            }
            if (value === "Custom") {
              setPickerOpen(true);
            }
            setFilters((prev) => ({ ...prev, date: value }));
          },
          isOpen: dropdownOpen.date,
          setIsOpen: (value) => setDropdownOpen((prev) => ({ ...prev, date: value }))
        }
      ), /* @__PURE__ */ React.createElement("div", { className: "relative z-[100] flex-shrink-0" }, /*
@__PURE__ */ React.createElement(
        DatePicker,
        {
          selectsRange: true,
          selected: startDate,
          onChange: (update) => {
            setDateRange(update);
            setFilters((prev) => ({ ...prev, date: "Custom" }));
            setCurrentPage(1);
            if (update?.[0] && update?.[1]) setPickerOpen(false);
          },
          startDate,
          endDate,
          open: pickerOpen,
          onInputClick: () => setPickerOpen(true),
          onClickOutside: () => setPickerOpen(false),
          dateFormat: "MMM d, yyyy",
          popperPlacement: "bottom-start",
```

```
          popperClassName: "z-[100]",
          customInput: /* @__PURE__ */ React.createElement(
            CalendarTrigger,
            {
              className: `w-10 h-10 rounded-xl border flex items-center justify-center flex-shrink-0
transition-colors cursor-pointer ${filters.date === "Custom" && startDate && endDate ? "border-[#AD7F65] bg-
[#AD7F65]/10" : theme === "dark" ? "border-gray-600 bg-[#2A2724] hover:border-[#AD7F65]" : "border-gray-200
bg-white hover:border-[#AD7F65]"}`
            }
          )
        }
      )), /* @__PURE__ */ React.createElement(
        Dropdown,
        {
          label: "User",
          options: userDropdownOptions,
          selected: filters.user,
          onSelect: (value) => {
            setFilters((prev) => ({ ...prev, user: value }));
            setCurrentPage(1);
          },
          isOpen: dropdownOpen.user,
          setIsOpen: (value) => setDropdownOpen((prev) => ({ ...prev, user: value }))
        }
      ), isReturnedLogsTab && /* @__PURE__ */ React.createElement(
        Dropdown,
        {
          label: "Returned By",
          options: returnedByDropdownOptions,
          selected: filters.returnedBy,
          onSelect: (value) => {
            setFilters((prev) => ({ ...prev, returnedBy: value }));
            setCurrentPage(1);
          },
          isOpen: dropdownOpen.returnedBy,
          setIsOpen: (value) => setDropdownOpen((prev) => ({ ...prev, returnedBy: value }))
        }
      ), !isReturnedLogsTab && /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(
        Dropdown,
        {
          label: "Payment Method",
          options: paymentOptions,
          selected: filters.method,
          onSelect: (value) => setFilters((prev) => ({ ...prev, method: value })),
          isOpen: dropdownOpen.method,
          setIsOpen: (value) => setDropdownOpen((prev) => ({ ...prev, method: value }))
        }
      ), /* @__PURE__ */ React.createElement(
        Dropdown,
        {
          label: "Status",
          options: statusOptions,
          selected: filters.status,
          onSelect: (value) => setFilters((prev) => ({ ...prev, status: value })),
          isOpen: dropdownOpen.status,
          setIsOpen: (value) => setDropdownOpen((prev) => ({ ...prev, status: value }))
        }
      ))), /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-3" }, /* @__PURE__
*/ React.createElement(
        "button",
        {
          onClick: handleExportButtonClick,
          className: `rounded-xl shadow-md flex items-center justify-center gap-2 px-4 py-2.5 transition-
colors ${isExportSelectionMode ? "border border-[#AD7F65] bg-[#AD7F65]/5" : theme === "dark" ? "bg-[#2A2724]
hover:bg-[#352F2A]" : "bg-white hover:bg-gray-50"}`,
          style: { minWidth: "120px" }
        },
        /* @__PURE__ */ React.createElement(
          "svg",
          {
            className: `w-5 h-5 ${theme === "dark" ? "text-gray-400" : "text-gray-700"}`,
            fill: "none",
            stroke: "currentColor",
            viewBox: "0 0 24 24"
          },
          /* @__PURE__ */ React.createElement(
```

```
            "path",
            {
              strokeLinecap: "round",
              strokeLinejoin: "round",
              strokeWidth: 2,
              d: "M12 10v6m0 0l-3-3m3 3l3-3m2 8H7a2 2 0 01-2-2V5a2 2 0 012-2h5.586a1 1 0 01.707.293l5.414
5.414a1 1 0 01.293.707V19a2 2 0 01-2 2z"
            }
          )
        ),
        /* @__PURE__ */ React.createElement("span", { className: `text-xs font-medium ${theme === "dark" ?
"text-gray-400" : "text-gray-700"}` }, isExportSelectionMode ? "Export Selected" : "Export")
      ), isExportSelectionMode && /* @__PURE__ */ React.createElement(
        "button",
        {
          onClick: handleCancelExportSelection,
          className: `rounded-xl shadow-md px-3 py-2 text-xs font-medium border transition-colors ${theme ===
"dark" ? "bg-[#2A2724] border-gray-600 text-gray-400 hover:bg-[#352F2A]" : "bg-white border-gray-200 text-
gray-600 hover:bg-gray-50"}`
        },
        "Cancel"
      ))),
      /* @__PURE__ */ React.createElement("div", { className: "relative overflow-x-auto overflow-y-auto flex-
1 min-h-0" }, /* @__PURE__ */ React.createElement("table", { className: "w-full text-sm text-left" }, /*
@__PURE__ */ React.createElement("thead", { className: "sticky top-0" }, /* @__PURE__ */ React.createElement(
        "tr",
        {
          className: `${theme === "dark" ? "bg-[#352F2A] text-[#C2A68C]" : "bg-[#F6EEE7] text-[#4A3B2F]"}
text-xs uppercase tracking-wider`
        },
        isExportSelectionMode && /* @__PURE__ */ React.createElement("th", { className: "px-4 py-3 font-
semibold" }, /* @__PURE__ */ React.createElement(
          "label",
          {
            className: `flex items-center gap-2 ${theme === "dark" ? "text-[#C2A68C]" : "text-[#4A3B2F]"}`
          },
          /* @__PURE__ */ React.createElement(
            "input",
            {
              ref: isReturnedLogsTab ? selectAllReturnedLogsRef : selectAllTransactionsRef,
              type: "checkbox",
              className: "w-4 h-4 text-[#AD7F65] border-[#AD7F65] rounded focus:ring-[#AD7F65]",
              onChange: isReturnedLogsTab ? handleToggleSelectAllReturnedLogs :
handleToggleSelectAllTransactions,
              checked: isExportSelectionMode ? isReturnedLogsTab ? allVisibleReturnedLogsSelected :
allVisibleTransactionsSelected : false
            }
          ),
          /* @__PURE__ */ React.createElement("span", { className: "text-[11px] tracking-wide" }, "All")
        )),
        (isReturnedLogsTab ? [
          "Receipt No.",
          "Transaction ID",
          "Returned Date",
          "Performed By",
          "Returned By",
          "Reason",
          "Original Amount",
          "Discounted Amount",
          "Returned",
          "Final Total"
        ] : [
          "Receipt No.",
          "Transaction ID",
          "Date",
          "Performed By",
          "Payment Method",
          "Original Amount",
          "Discounted Amount",
          "Total",
          "Status",
          "Quick Action"
        ]).map(
          (col) => /* @__PURE__ */ React.createElement("th", { key: col, className: "px-4 py-3 font-
semibold" }, col)
        )
      )), /* @__PURE__ */ React.createElement("tbody", null, loading && activeRowCount === 0 && /* @__PURE__
```

```
*/ React.createElement("tr", null, /* @__PURE__ */ React.createElement(
        "td",
        {
          colSpan: isReturnedLogsTab ? returnedTableColumnCount : transactionTableColumnCount,
          className: "py-10 text-center text-gray-500"
        },
        "Loading transactions..."
      )), !loading && activeRowCount === 0 && /* @__PURE__ */ React.createElement("tr", null, /* @__PURE__ */
React.createElement(
        "td",
        {
          colSpan: isReturnedLogsTab ? returnedTableColumnCount : transactionTableColumnCount,
          className: "py-10 text-center text-gray-400 italic"
        },
        isReturnedLogsTab ? "No returned transactions found" : "No transactions found"
      )), isReturnedLogsTab ? paginatedReturnedLogs.map((row) => {
        const isActive = selectedTransaction?._id === row._id;
        return /* @__PURE__ */ React.createElement(
          "tr",
          {
            key: row._id,
            onClick: () => {
              const transactionRecord = transactions.find(
                (t) => sameTransactionId(t, row._id)
              );
              if (transactionRecord) {
                handleRowClick(transactionRecord);
              }
            },
            className: `cursor-pointer border-b transition-all ${theme === "dark" ? "border-gray-700" :
"border-gray-100"} ${isActive ? theme === "dark" ? "bg-[#352F2A]" : "bg-[#FDF7F1] shadow-inner" : theme ===
"dark" ? "hover:bg-[#2A2521] text-gray-300" : "hover:bg-[#F9F2EC]"}`
          },
          isExportSelectionMode && /* @__PURE__ */ React.createElement(
            "td",
            {
              className: "px-4 py-3",
              onClick: (e) => e.stopPropagation()
            },
            /* @__PURE__ */ React.createElement(
              "input",
              {
                type: "checkbox",
                className: "w-4 h-4 text-[#AD7F65] border-[#AD7F65] rounded focus:ring-[#AD7F65]",
                checked: selectedReturnedLogIds.includes(row._id),
                onChange: () => handleToggleReturnedLogSelection(row._id),
                disabled: !row._id
              }
            )
          ),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `px-4 py-3 font-semibold ${theme === "dark" ? "text-white" : "text-gray-800"}`
            },
            row.receiptNo ? `#${row.receiptNo}` : "---"
          ),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `px-4 py-3 font-semibold text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-
700"}`
            },
            row.transactionId
          ),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 text-gray-500" },
row.returnedAt.toLocaleDateString(void 0, {
            month: "short",
            day: "numeric",
            year: "numeric"
          })),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `px-4 py-3 align-middle ${theme === "dark" ? "text-gray-300" : "text-gray-700"}`
            },
            /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-2 min-w-[160px]" },
```

```
/* @__PURE__ */ React.createElement("span", { className: "w-8 h-8 rounded-full bg-[#F0E5DB] flex items-center
justify-center text-xs font-bold text-[#8B6B55] shrink-0" }, getInitials(row.performedByName || "Staff")), /*
@__PURE__ */ React.createElement("span", { className: "truncate" }, row.performedByName || "Staff"))
          ),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `px-4 py-3 align-middle ${theme === "dark" ? "text-gray-300" : "text-gray-700"}`
            },
            /* @__PURE__ */ React.createElement("div", { className: "flex items-center gap-2 min-w-[160px]" },
/* @__PURE__ */ React.createElement("span", { className: "w-8 h-8 rounded-full bg-[#F0E5DB] flex items-center
justify-center text-xs font-bold text-[#8B6B55] shrink-0" }, getInitials(row.returnedByName || "Staff")), /*
@__PURE__ */ React.createElement("span", { className: "truncate" }, row.returnedByName || "Staff"))
          ),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `px-4 py-3 ${theme === "dark" ? "text-gray-300" : "text-gray-600"}`
            },
            row.reason
          ),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 font-semibold" },
formatCurrency(row.originalAmount)),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 font-semibold" },
formatCurrency(row.discountedAmount)),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 font-semibold text-orange-600" },
formatCurrency(-Math.abs(row.returnedAmount))),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 font-semibold" },
formatCurrency(row.totalAmount))
        );
      }) : paginatedTransactions.map((trx) => {
        const isActive = selectedTransaction?._id === trx._id;
        const lineSub = originalLineSubtotalFromItems(trx) || trx.totalAmount || 0;
        const hasReturnActivity = (trx.returnTransactions?.length || 0) > 0 || trx.status === "Returned" ||
trx.status === "Partially Returned";
        const discountedAmount = resolveTransactionDiscount(trx, lineSub, {
          skipInference: hasReturnActivity
        });
        return /* @__PURE__ */ React.createElement(
          "tr",
          {
            key: trx._id,
            onClick: () => handleRowClick(trx),
            className: `cursor-pointer border-b transition-all ${theme === "dark" ? "border-gray-700" :
"border-gray-100"} ${isActive ? theme === "dark" ? "bg-[#352F2A]" : "bg-[#FDF7F1] shadow-inner" : theme ===
"dark" ? "hover:bg-[#2A2521] text-gray-300" : "hover:bg-[#F9F2EC]"}`
          },
          isExportSelectionMode && /* @__PURE__ */ React.createElement(
            "td",
            {
              className: "px-4 py-3",
              onClick: (e) => e.stopPropagation()
            },
            /* @__PURE__ */ React.createElement(
              "input",
              {
                type: "checkbox",
                className: "w-4 h-4 text-[#AD7F65] border-[#AD7F65] rounded focus:ring-[#AD7F65]",
                checked: selectedTransactionIds.includes(trx._id),
                onChange: () => handleToggleTransactionSelection(trx._id),
                disabled: !trx._id
              }
            )
          ),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `px-4 py-3 font-semibold ${theme === "dark" ? "text-white" : "text-gray-800"}`
            },
            trx.receiptNo ? `#${trx.receiptNo}` : "---"
          ),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `px-4 py-3 font-semibold text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-
700"}`
            },
```

```
            trx.referenceNo || trx._id?.substring(0, 12) || "---"
          ),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 text-gray-500" }, new
Date(trx.checkedOutAt).toLocaleDateString(
            void 0,
            {
              month: "short",
              day: "numeric",
              year: "numeric"
            }
          )),
          /* @__PURE__ */ React.createElement(
            "td",
            {
              className: `px-4 py-3 flex items-center gap-2 ${theme === "dark" ? "text-gray-300" : "text-
gray-700"}`
            },
            /* @__PURE__ */ React.createElement("span", { className: "w-8 h-8 rounded-full bg-[#F0E5DB] flex
items-center justify-center text-xs font-bold text-[#8B6B55]" }, getInitials(trx.performedByName || "Staff")),
            trx.performedByName || "Staff"
          ),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 capitalize" }, trx.paymentMethod),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 font-semibold" },
formatCurrency(lineSub)),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 font-semibold" },
formatCurrency(discountedAmount)),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3 font-semibold" },
formatCurrency(trx.totalAmount)),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3" }, renderStatusPill(trx.status)),
          /* @__PURE__ */ React.createElement("td", { className: "px-4 py-3" }, /* @__PURE__ */
React.createElement("div", { className: "flex items-center gap-2" }, /* @__PURE__ */ React.createElement(
            "button",
            {
              title: "View",
              onClick: (e) => {
                e.stopPropagation();
                handleViewClick(trx);
              },
              className: `w-9 h-9 border rounded-xl flex items-center justify-center shadow-sm hover:shadow-
md hover:-translate-y-0.5 whitespace-nowrap transition-all ${theme === "dark" ? "bg-[#2A2724] border-gray-600
text-gray-400 hover:border-green-500 hover:text-green-500" : "bg-white border-gray-200 text-gray-500
hover:border-green-500 hover:text-green-600"}`
            },
            /* @__PURE__ */ React.createElement(FaEye, null)
          ), /* @__PURE__ */ React.createElement(
            "button",
            {
              title: "Print",
              onClick: (e) => {
                e.stopPropagation();
                handlePrintClick(trx);
              },
              className: `w-9 h-9 border rounded-xl flex items-center justify-center shadow-sm hover:shadow-
md hover:-translate-y-0.5 transition-all ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-gray-400
hover:border-blue-500 hover:text-blue-500" : "bg-white border-gray-200 text-gray-500 hover:border-blue-500
hover:text-blue-600"}`
            },
            /* @__PURE__ */ React.createElement(FaPrint, null)
          ), isTransactionReturnable(trx) && trx.status !== "Returned" && trx.status !== "Voided" && /*
@__PURE__ */ React.createElement(
            "button",
            {
              title: "Return",
              onClick: (e) => {
                e.stopPropagation();
                handleReturnClick(trx);
              },
              className: `w-9 h-9 border rounded-xl flex items-center justify-center shadow-sm hover:shadow-
md hover:-translate-y-0.5 transition-all ${theme === "dark" ? "bg-[#2A2724] border-gray-600 text-gray-400
hover:border-orange-500 hover:text-orange-500" : "bg-white border-gray-200 text-gray-500 hover:border-orange-
500 hover:text-orange-600"}`
            },
            /* @__PURE__ */ React.createElement(FaUndoAlt, null)
          )))
        );
      })))),
      /* @__PURE__ */ React.createElement("div", { className: "flex items-center justify-between mt-5" }, /*
```

```
@__PURE__ */ React.createElement(
        "div",
        {
          className: `text-xs ${theme === "dark" ? "text-gray-400" : "text-gray-500"}`
        },
        "Showing ",
        showingStart,
        "-",
        showingEnd,
        " of ",
        activeRowCount
      ), /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `flex items-center gap-2 rounded-full border px-3 py-1 shadow-inner ${theme === "dark" ?
"bg-[#1E1B18] border-gray-600" : "bg-white border-gray-200"}`
        },
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: () => setCurrentPage((prev) => Math.max(1, prev - 1)),
            disabled: currentPage === 1,
            className: `p-2 rounded-full ${currentPage === 1 ? theme === "dark" ? "text-gray-600" : "text-
gray-300" : theme === "dark" ? "hover:bg-[#2A2724] text-gray-400" : "hover:bg-gray-50 text-gray-600"}`
          },
          /* @__PURE__ */ React.createElement(FaChevronLeft, null)
        ),
        Array.from({ length: totalPages }).slice(0, 5).map((_, idx) => {
          const pageNumber = idx + 1;
          return /* @__PURE__ */ React.createElement(
            "button",
            {
              key: pageNumber,
              onClick: () => setCurrentPage(pageNumber),
              className: `w-8 h-8 rounded-full text-sm font-semibold ${currentPage === pageNumber ? "bg-
[#AD7F65] text-white shadow-md" : "text-gray-600 hover:bg-gray-50"}`
            },
            pageNumber
          );
        }),
        /* @__PURE__ */ React.createElement("span", { className: "text-gray-400 px-2" }, "..."),
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: () => setCurrentPage(totalPages),
            className: `w-8 h-8 rounded-full text-sm font-semibold ${currentPage === totalPages ? "bg-
[#AD7F65] text-white shadow-md" : "text-gray-600 hover:bg-gray-50"}`
          },
          totalPages
        ),
        /* @__PURE__ */ React.createElement(
          "button",
          {
            onClick: () => setCurrentPage((prev) => Math.min(totalPages, prev + 1)),
            disabled: currentPage === totalPages,
            className: `p-2 rounded-full ${currentPage === totalPages ? theme === "dark" ? "text-gray-600" :
"text-gray-300" : theme === "dark" ? "hover:bg-[#2A2724] text-gray-400" : "hover:bg-gray-50 text-gray-600"}`
          },
          /* @__PURE__ */ React.createElement(FaChevronRight, null)
        )
      ))
    ), !isReturnedLogsTab && /* @__PURE__ */ React.createElement("div", { className: "w-full lg:w-[380px]
xl:w-[420px]" }, /* @__PURE__ */ React.createElement(
      "div",
      {
        className: `rounded-2xl border shadow-[0_20px_45px_rgba(0,0,0,0.08)] p-6 sticky top-8 ${theme ===
"dark" ? "bg-[#2A2724] border-[#4A4037]" : "bg-white border-white"}`
      },
      /* @__PURE__ */ React.createElement("div", { className: "mb-4" }, /* @__PURE__ */
React.createElement("p", { className: "text-sm text-gray-400" }, "Create Your Style"), /* @__PURE__ */
React.createElement("p", { className: "text-xs text-gray-400" }, "Pasonanca, Zamboanga City")),
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `font-mono text-xs space-y-2 ${theme === "dark" ? "text-gray-300" : ""}`
        },
        !isReturnedLogsTab && /* @__PURE__ */ React.createElement("div", { className: "flex justify-between
```

```
text-gray-500" }, /* @__PURE__ */ React.createElement("span", null, "Receipt No:"), /* @__PURE__ */
React.createElement("span", { className: "font-bold text-[#AD7F65]" }, selectedTransaction?.status ===
"Completed" && selectedTransaction?.receiptNo ? `#${selectedTransaction.receiptNo}` : "---")),
        /* @__PURE__ */ React.createElement("div", { className: "flex justify-between text-gray-500" }, /*
@__PURE__ */ React.createElement("span", null, "Date:"), /* @__PURE__ */ React.createElement("span", null,
selectedTransaction ? new Date(
          selectedTransaction.checkedOutAt
        ).toLocaleString() : "-")),
        /* @__PURE__ */ React.createElement("div", { className: "flex justify-between text-gray-500" }, /*
@__PURE__ */ React.createElement("span", null, "Cashier:"), /* @__PURE__ */ React.createElement("span", null,
selectedTransaction?.performedByName || "---"))
      ),
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `border-t border-b my-4 py-3 font-mono text-sm ${theme === "dark" ? "border-gray-700
text-gray-300" : "border-gray-200"}`
        },
        /* @__PURE__ */ React.createElement("div", { className: "flex justify-between font-semibold" }, /*
@__PURE__ */ React.createElement("span", null, "Item"), /* @__PURE__ */ React.createElement("span", null,
"Qty x Price")),
        /* @__PURE__ */ React.createElement(
          "div",
          {
            className: `mt-2 space-y-1 ${theme === "dark" ? "text-gray-400" : "text-gray-600"}`
          },
          selectedTransaction?.items?.map(
            (item, idx) => /* @__PURE__ */ React.createElement("div", { key: idx, className: "flex justify-
between" }, /* @__PURE__ */ React.createElement("span", null, item.itemName), /* @__PURE__ */
React.createElement("span", null, item.quantity, " x ", formatCurrency(item.price)))
          ) || /* @__PURE__ */ React.createElement("p", { className: "text-center text-gray-400" }, "No
items")
        )
      ),
      /* @__PURE__ */ React.createElement(
        "div",
        {
          className: `font-mono text-xs space-y-1 ${theme === "dark" ? "text-gray-400" : "text-gray-600"}`
        },
        /* @__PURE__ */ React.createElement("div", { className: "flex justify-between" }, /* @__PURE__ */
React.createElement("span", null, "Payment Method:"), /* @__PURE__ */ React.createElement("span", { className:
"uppercase" }, selectedTransaction?.paymentMethod)),
        /* @__PURE__ */ React.createElement("div", { className: "flex justify-between" }, /* @__PURE__ */
React.createElement("span", null, "Subtotal:"), /* @__PURE__ */ React.createElement("span", null,
formatCurrency(sidebarReceiptTotals.lineSub))),
        /* @__PURE__ */ React.createElement("div", { className: "flex justify-between" }, /* @__PURE__ */
React.createElement("span", null, "Discount:"), /* @__PURE__ */ React.createElement("span", null,
formatCurrency(sidebarReceiptTotals.discount))),
        /* @__PURE__ */ React.createElement(
          "div",
          {
            className: `flex justify-between font-semibold text-base pt-2 border-t ${theme === "dark" ?
"text-white border-gray-700" : "text-gray-800 border-gray-100"}`
          },
          /* @__PURE__ */ React.createElement("span", null, "Total"),
          /* @__PURE__ */ React.createElement("span", null, formatCurrency(selectedTransaction?.totalAmount
|| 0))
        )
      ),
      /* @__PURE__ */ React.createElement(
        "button",
        {
          className: "w-full mt-6 py-3 rounded-xl text-white font-semibold shadow-lg transition-all
hover:shadow-xl hover:brightness-105 active:scale-98",
          style: {
            backgroundImage: "linear-gradient(135deg, #AD7F65 0%, #76462B 100%)",
            boxShadow: "0 12px 20px rgba(118,70,43,0.25)"
          }
        },
        "Print Receipt"
      ),
      /* @__PURE__ */ React.createElement("p", { className: "text-center text-[11px] text-gray-400 mt-4
tracking-wide" }, "This is not an official receipt")
    ))), /* @__PURE__ */ React.createElement(
      ViewTransactionModal,
      {
        isOpen: showViewModal,
```

```
        onClose: () => {
          setShowViewModal(false);
          setTransactionToView(null);
        },
        transaction: transactionToView,
        onReturnItems: (trx) => {
          setShowViewModal(false);
          setTransactionToView(null);
          handleReturnClick(trx);
        },
        onPrintReceipt: (trx) => {
          setShowViewModal(false);
          setTransactionToView(null);
          handlePrintClick(trx);
        }
      }
    ), /* @__PURE__ */ React.createElement(
      PrintReceiptModal,
      {
        isOpen: showPrintModal,
        onClose: () => {
          setShowPrintModal(false);
          setTransactionToPrint(null);
        },
        transaction: transactionToPrint
      }
    ), /* @__PURE__ */ React.createElement(
      ReturnItemsModal,
      {
        isOpen: showReturnModal,
        onClose: () => {
          setShowReturnModal(false);
          setTransactionToReturn(null);
        },
        transaction: transactionToReturn,
        onConfirm: handleReturnConfirm
      }
    ), showReturnSuccessModal && /* @__PURE__ */ React.createElement("div", { className: "fixed inset-0 flex
items-center justify-center z-[10002] bg-black/50 backdrop-blur-sm" }, /* @__PURE__ */
React.createElement("div", { className: "bg-white rounded-2xl w-full max-w-md p-8 shadow-2xl text-center" },
/* @__PURE__ */ React.createElement("div", { className: "w-20 h-20 mx-auto mb-6 rounded-full bg-green-100
flex items-center justify-center" }, /* @__PURE__ */ React.createElement(FaCheckCircle, { className: "w-10 h-
10 text-green-500" })), /* @__PURE__ */ React.createElement("h3", { className: "text-2xl font-bold text-gray-
800 mb-2" }, "Return Processed!"), /* @__PURE__ */ React.createElement("p", { className: "text-gray-500 mb-
6" }, "The return has been processed successfully."), /* @__PURE__ */ React.createElement(
      "button",
      {
        onClick: () => {
          setShowReturnSuccessModal(false);
        },
        className: "px-8 py-3 rounded-lg text-white font-semibold transition-all hover:opacity-90",
        style: {
          background: "linear-gradient(135deg, #22C55E 0%, #16A34A 100%)"
        }
      },
      "OK"
    ))), /* @__PURE__ */ React.createElement(
      RemittanceModal,
      {
        isOpen: showRemittanceModal,
        onClose: () => setShowRemittanceModal(false),
        employeeId: currentUser?._id,
        employeeName: currentUser?.name || currentUser?.firstName || ""
      }
    ))
  );
};
export default memo(Transaction);
```

FILE: pos/frontend/src/utils/manageDataExport.js
==========

```
function fmtDateTime(value) {
  if (value == null || value === "") return "";
  const d = new Date(value);
  if (Number.isNaN(d.getTime())) return String(value);
```

```
  return d.toLocaleString("en-PH", {
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
    hour: "2-digit",
    minute: "2-digit",
    second: "2-digit",
    hour12: true
  });
}
function fmtDate(value) {
  if (value == null || value === "") return "";
  const d = new Date(value);
  if (Number.isNaN(d.getTime())) return String(value);
  return d.toLocaleDateString("en-PH", {
    year: "numeric",
    month: "short",
    day: "numeric"
  });
}
function shortRef(id) {
  if (id == null) return "";
  const s = typeof id === "object" && id?.toString ? id.toString() : String(id);
  return s.length > 10 ? s.slice(-8) : s;
}
function formatPhp(n) {
  const x = Number(n);
  if (!Number.isFinite(x)) return "0.00";
  return x.toLocaleString("en-PH", {
    minimumFractionDigits: 2,
    maximumFractionDigits: 2
  });
}
function formatTransactionItems(items) {
  if (!Array.isArray(items) || items.length === 0) return "";
  return items.map((i) => {
    const name = (i.itemName || "Item").trim().replace(/\s+/g, " ");
    const q = Math.max(0, Number(i.quantity ?? 1)) || 1;
    const unit = Number(i.price ?? i.itemPrice ?? 0) || 0;
    const lineTotal = Math.round(unit * q * 100) / 100;
    let line = `${q}x ${name}; unit PHP ${formatPhp(unit)}; line PHP ${formatPhp(lineTotal)}`;
    if (i.returnStatus === "Returned") line += "; returned";
    else if (i.returnStatus === "Partially Returned") {
      const rq = i.returnedQuantity != null ? String(i.returnedQuantity) : "";
      line += rq ? `; partial return (${rq})` : "; partial return";
    }
    return line;
  }).join(" | ");
}
function formatVoidItems(items) {
  if (!Array.isArray(items) || items.length === 0) return "";
  return items.map((i) => {
    const name = (i.itemName || "Item").trim().replace(/\s+/g, " ");
    const q = Math.max(0, Number(i.quantity ?? 1)) || 1;
    const unit = Number(i.price ?? 0) || 0;
    const lineTotal = Math.round(unit * q * 100) / 100;
    return `${q}x ${name}; unit PHP ${formatPhp(unit)}; line PHP ${formatPhp(lineTotal)}`;
  }).join(" | ");
}
function formatCartItems(items) {
  if (!Array.isArray(items) || items.length === 0) return "";
  return items.map((i) => {
    const name = (i.itemName || "Item").trim().replace(/\s+/g, " ");
    const q = Math.max(0, Number(i.quantity ?? 1)) || 1;
    return `${q}x ${name}`;
  }).join(" | ");
}
function isVoidSalesRow(r) {
  const st = String(r.status || "").toLowerCase();
  if (st === "voided") return true;
  if (String(r.paymentMethod || "").toLowerCase() === "void") return true;
  return false;
}
function hasVariantMatrix(sizes) {
  return sizes && typeof sizes === "object" && Object.keys(sizes).length > 0;
}
const TRANSFORMERS = {
```

```
transactions: (r) => {
  const ref = r.receiptNo || r.referenceNo || (r._id ? `TX-${shortRef(r._id)}` : "");
  const orig = r.originalTransactionId ? shortRef(r.originalTransactionId) : "";
  return {
    "Receipt / Ref": String(ref || ""),
    "Date & Time": fmtDateTime(r.checkedOutAt || r.createdAt),
    Cashier: String(r.performedByName || ""),
    Status: String(r.status || ""),
    "Payment Method": String(r.paymentMethod || ""),
    Items: formatTransactionItems(r.items),
    Subtotal: r.subtotal != null && r.subtotal !== "" ? Number(r.subtotal) : "",
    Discount: Number(r.discount ?? r.discountAmount ?? 0) || 0,
    "Total (PHP)": Number(r.totalAmount ?? 0) || 0,
    "Amount Received": Number(r.amountReceived ?? 0) || 0,
    "Change Given": Number(r.changeGiven ?? 0) || 0,
    "Original Sale Ref": orig ? `\u2026${orig}` : ""
  };
},
products: (r) => ({
  SKU: String(r.sku || ""),
  "Item Name": String(r.itemName || ""),
  Category: String(r.category || ""),
  Subcategory: String(r.subCategory || ""),
  Brand: String(r.brandName || ""),
  Variant: String(r.variant || ""),
  "Food Subtype": String(r.foodSubtype || ""),
  Size: String(r.size || ""),
  "Unit of Measure": String(r.unitOfMeasure || ""),
  "Selling Price": Number(r.itemPrice ?? 0) || 0,
  "Cost Price": Number(r.costPrice ?? 0) || 0,
  "Current Stock": Number(r.currentStock ?? 0) || 0,
  "Reorder Level": Number(r.reorderNumber ?? 0) || 0,
  Supplier: String(r.supplierName || ""),
  "Supplier Contact": String(r.supplierContact || ""),
  "Display in Terminal": r.displayInTerminal !== false ? "Yes" : "No",
  Archived: r.isArchived ? "Yes" : "No",
  "Expiration Date": r.expirationDate ? fmtDate(r.expirationDate) : "",
  "Has Variant Matrix": hasVariantMatrix(r.sizes) ? "Yes" : "No",
  "Date Added": r.dateAdded ? fmtDateTime(r.dateAdded) : ""
}),
stockMovements: (r) => ({
  "Date & Time": fmtDateTime(r.createdAt),
  SKU: String(r.sku || ""),
  "Item Name": String(r.itemName || ""),
  Category: String(r.category || ""),
  Brand: String(r.brandName || ""),
  Type: String(r.type || ""),
  Quantity: Number(r.quantity ?? 0) || 0,
  "Stock Before": Number(r.stockBefore ?? 0) || 0,
  "Stock After": Number(r.stockAfter ?? 0) || 0,
  Reason: String(r.reason || ""),
  "Handled By": String(r.handledBy || ""),
  Notes: String(r.notes || "")
}),
categories: (r) => ({
  Name: String(r.name || ""),
  Type: String(r.type || ""),
  "Parent Category": String(r.parentCategory || ""),
  Status: String(r.status || ""),
  "Show on POS": r.showOnPos !== false ? "Yes" : "No",
  "Date Created": r.dateCreated ? fmtDateTime(r.dateCreated) : ""
}),
discounts: (r) => ({
  Title: String(r.title || ""),
  Code: String(r.discountCode || ""),
  "Discount Type": String(r.discountType || ""),
  Value: Number(r.discountValue ?? 0) || 0,
  "Applies To": String(r.appliesTo || ""),
  Category: String(r.category || ""),
  Subcategory: String(r.subCategory || ""),
  "Valid From": r.validFrom ? fmtDate(r.validFrom) : "",
  "Valid To": r.validTo ? fmtDate(r.validTo) : "",
  "No Expiration": r.noExpiration ? "Yes" : "No",
  "Min Purchase": Number(r.minPurchaseAmount ?? 0) || 0,
  "Max Purchase": r.maxPurchaseAmount != null ? Number(r.maxPurchaseAmount) : "",
  "Usage Limit": r.usageLimit != null ? Number(r.usageLimit) : "",
  "Usage Count": Number(r.usageCount ?? 0) || 0,
```

```
    Status: String(r.status || ""),
    Description: String(r.description || "")
  }),
  brandPartners: (r) => ({
    "Brand Name": String(r.brandName || ""),
    Email: String(r.email || ""),
    "Contact Number": String(r.contactNumber || ""),
    "Contact Person": String(r.contactPerson || ""),
    Status: String(r.status || ""),
    "Created At": r.createdAt ? fmtDateTime(r.createdAt) : ""
  }),
  voidLogs: (r) => ({
    "Void ID": String(r.voidId || ""),
    "Voided At": fmtDateTime(r.voidedAt || r.createdAt),
    "Void Reason": String(r.voidReason || ""),
    "Voided By": String(r.voidedByName || r.voidedBy || ""),
    "Approved By": String(r.approvedBy || ""),
    "Approver Role": String(r.approvedByRole || ""),
    "Total Amount": Number(r.totalAmount ?? 0) || 0,
    Items: formatVoidItems(r.items),
    Source: String(r.source || ""),
    Notes: String(r.notes || "")
  }),
  archives: (r) => ({
    "Item Name": String(r.itemName || ""),
    SKU: String(r.sku || ""),
    Category: String(r.category || ""),
    Brand: String(r.brandName || ""),
    Variant: String(r.variant || ""),
    Size: String(r.selectedSize || ""),
    Quantity: Number(r.quantity ?? 0) || 0,
    "Item Price": Number(r.itemPrice ?? 0) || 0,
    "Cost Price": Number(r.costPrice ?? 0) || 0,
    Reason: String(r.reason || ""),
    "Return Reason": String(r.returnReason || ""),
    Source: String(r.source || ""),
    "Archived By": String(r.archivedBy || ""),
    "Archived At": r.archivedAt ? fmtDateTime(r.archivedAt) : "",
    Notes: String(r.notes || "")
  }),
  carts: (r) => ({
    "Staff / User Id": String(r.userId || ""),
    "Line Count": Array.isArray(r.items) ? r.items.length : 0,
    Items: formatCartItems(r.items),
    "Updated At": r.updatedAt ? fmtDateTime(r.updatedAt) : ""
  }),
  employees: (r) => ({
    Name: String(r.name || ""),
    Email: String(r.email || ""),
    Role: String(r.role || ""),
    "Contact No": String(r.contactNo || ""),
    Status: String(r.status || ""),
    "Date Joined": r.dateJoined ? fmtDate(r.dateJoined) : "",
    "POS Terminal": r.permissions?.posTerminal !== false ? "Yes" : "No",
    Inventory: r.permissions?.inventory ? "Yes" : "No",
    "View Transactions": r.permissions?.viewTransactions !== false ? "Yes" : "No",
    "Generate Reports": r.permissions?.generateReports ? "Yes" : "No",
    "Last Login": r.lastLogin ? fmtDateTime(r.lastLogin) : "",
    "Requires PIN Reset": r.requiresPinReset ? "Yes" : "No"
  }),
  merchantSettings: (r) => ({
    "Merchant Name": String(r.merchantName || ""),
    Active: r.isActive ? "Yes" : "No",
    Environment: String(r.environment || ""),
    "Payment Expiry (min)": Number(r.paymentExpiryMinutes ?? 0) || 0,
    "Webhook URL": String(r.webhookUrl || ""),
    "Configured By": String(r.configuredByName || r.configuredBy || ""),
    "App ID (masked)": r.appId && String(r.appId).length > 6 ? `\u2026${String(r.appId).slice(-4)}` :
String(r.appId || ""),
    "Public Key (masked)": r.publicKey && String(r.publicKey).length > 12 ? `${String(r.publicKey).slice(0,
8)}\u2026` : String(r.publicKey || ""),
    "Updated At": r.updatedAt ? fmtDateTime(r.updatedAt) : ""
  })
};
export function recordsForBusinessExport(collectionKey, records) {
  if (!Array.isArray(records) || records.length === 0) return [];
  const fn = TRANSFORMERS[collectionKey];
```

```
  if (!fn) return [];
  let list = records;
  if (collectionKey === "transactions") {
    list = records.filter((rec) => !isVoidSalesRow(rec));
  }
  if (list.length === 0) return [];
  return list.map((rec) => fn(rec));
}
export function hasBusinessExportSchema(collectionKey) {
  return Object.prototype.hasOwnProperty.call(TRANSFORMERS, collectionKey);
}
export { TRANSFORMERS };
```

FILE: pos/frontend/src/utils/pinValidation.js
==========

```
export const validatePinSecurity = (pin) => {
  if (!pin || pin.length !== 6) {
    return { isValid: false, error: "PIN must be exactly 6 digits" };
  }
  const digitCounts = {};
  for (const digit of pin) {
    digitCounts[digit] = (digitCounts[digit] || 0) + 1;
    if (digitCounts[digit] >= 3) {
      return {
        isValid: false,
        error: "No digit may appear 3 times or more (e.g., 121212, 111222)"
      };
    }
  }
  const isSequence = (str) => {
    const digits = str.split("").map(Number);
    let isAscending = true;
    for (let i = 1; i < digits.length; i++) {
      if (digits[i] !== digits[i - 1] + 1) {
        isAscending = false;
        break;
      }
    }
    let isDescending = true;
    for (let i = 1; i < digits.length; i++) {
      if (digits[i] !== digits[i - 1] - 1) {
        isDescending = false;
        break;
      }
    }
    return isAscending || isDescending;
  };
  if (isSequence(pin)) {
    return {
      isValid: false,
      error: "No sequences allowed (e.g., 123456, 234567, 987654)"
    };
  }
  const firstDigit = pin[0];
  const allSame = pin.split("").every((digit) => digit === firstDigit);
  if (allSame) {
    return {
      isValid: false,
      error: "No all-same PINs allowed (e.g., 000000, 111111)"
    };
  }
  return { isValid: true, error: "" };
};
```

FILE: pos/frontend/src/utils/printBridge.js
==========

```
import { formatReceiptVariantSizeLine } from "./transactionDisplay";
const MAX_WIDTH = Number(import.meta.env.VITE_RECEIPT_LINE_WIDTH || 32);
const padLine = (left, right = "") => {
  const cleanLeft = String(left ?? "").trim();
  const cleanRight = String(right ?? "").trim();
  const available = MAX_WIDTH - (cleanLeft.length + cleanRight.length);
  const spacer = available > 0 ? " ".repeat(available) : " ";
```

```
  return (cleanLeft + spacer + cleanRight).slice(0, MAX_WIDTH);
};
const centerText = (text) => {
  const padding = Math.floor((MAX_WIDTH - text.length) / 2);
  return " ".repeat(Math.max(0, padding)) + text;
};
const chunkText = (text) => {
  if (!text) return [""];
  const normalized = String(text);
  const chunks = [];
  for (let i = 0; i < normalized.length; i += MAX_WIDTH) {
    chunks.push(normalized.slice(i, i + MAX_WIDTH));
  }
  return chunks.length ? chunks : [""];
};
export const buildReceiptLines = (receipt) => {
  const lines = [];
  const storeName = receipt.storeName || "Create Your Style";
  const location = receipt.location || "Pasonanca, Zamboanga City";
  const receiptNo = receipt.receiptNo || "000000";
  const paymentMethod = receipt.paymentMethod || "Cash";
  const cashier = receipt.cashier || receipt.cashierName || receipt.performedByName || "Staff";
  const receiptDate = receipt.date || (/* @__PURE__ */ new Date()).toLocaleDateString("en-US", {
    month: "numeric",
    day: "numeric",
    year: "numeric"
  });
  const receiptTime = receipt.time || (/* @__PURE__ */ new Date()).toLocaleTimeString("en-US", {
    hour: "numeric",
    minute: "2-digit",
    hour12: true
  });
  lines.push(centerText(storeName));
  lines.push(centerText(location));
  lines.push("--------------------------------");
  lines.push(centerText("RECEIPT"));
  lines.push(centerText(`#${receiptNo}`));
  lines.push("");
  lines.push(padLine("Date:", `${receiptDate}, ${receiptTime}`));
  lines.push(padLine("Cashier:", cashier));
  lines.push(padLine("Payment:", paymentMethod));
  lines.push("--------------------------------");
  (receipt.items || []).forEach((item) => {
    let itemName = (item.name || item.itemName || "Item").toString();
    itemName = itemName.replace(/\s*\([^)]*\)\s*$/, "").trim();
    const qty = item.qty || item.quantity || 1;
    const price = item.price || item.itemPrice || 0;
    const variantSizeLine = formatReceiptVariantSizeLine(item);
    lines.push(itemName);
    if (variantSizeLine) {
      lines.push(variantSizeLine);
    }
    lines.push(`${qty} x PHP ${Number(price).toFixed(2)}`);
  });
  lines.push("--------------------------------");
  lines.push(padLine("Subtotal:", `PHP ${Number(receipt.subtotal || 0).toFixed(2)}`));
  lines.push(padLine("Discount:", `PHP ${Number(receipt.discount || 0).toFixed(2)}`));
  lines.push("");
  lines.push(padLine("Total:", `PHP ${Number(receipt.total || 0).toFixed(2)}`));
  if (receipt.cash !== void 0) {
    lines.push(padLine("Amount Received:", `PHP ${Number(receipt.cash).toFixed(2)}`));
  }
  if (receipt.change !== void 0) {
    lines.push(padLine("Change:", `PHP ${Number(receipt.change).toFixed(2)}`));
  }
  lines.push("--------------------------------");
  lines.push(centerText("Thank you for your purchase!"));
  lines.push(centerText("This is not an official receipt"));
  return lines;
};
const buildReceiptHTML = (receipt) => {
  const storeName = "Create Your Style";
  const location = receipt.location || "Pasonanca, Zamboanga City";
  const receiptNo = receipt.receiptNo || "000000";
  const paymentMethod = receipt.paymentMethod || "Cash";
  const subtotal = Number(receipt.subtotal || 0);
  const discount = Number(receipt.discount || 0);
```

```
const total = Number(receipt.total || 0);
const cash = receipt.cash !== void 0 ? Number(receipt.cash) : null;
const change = receipt.change !== void 0 ? Number(receipt.change) : null;
const cashier = receipt.cashier || receipt.cashierName || receipt.performedByName || "Staff";
const receiptDate = receipt.date || (/* @__PURE__ */ new Date()).toLocaleDateString("en-US", {
  month: "short",
  day: "numeric",
  year: "numeric"
});
const receiptTime = receipt.time || (/* @__PURE__ */ new Date()).toLocaleTimeString("en-US", {
  hour: "numeric",
  minute: "2-digit",
  hour12: true
});
const itemsHTML = (receipt.items || []).map((item) => {
  let itemName = (item.name || item.itemName || "Item").toString();
  itemName = itemName.replace(/\s*\([^)]*\)\s*$/, "").trim();
  const qty = item.qty || item.quantity || 1;
  const price = Number(item.price || item.itemPrice || 0);
  const variantSizeLine = formatReceiptVariantSizeLine(item);
  let sizeColorInfo = "";
  if (variantSizeLine) {
    sizeColorInfo = `<div style="font-size: 9px; color: #a0aec0;">${variantSizeLine}</div>`;
  }
  return `
    <div style="margin-bottom: 8px;">
      <div style="font-weight: 600; font-size: 11px; color: #1a202c;">${itemName}</div>
      ${sizeColorInfo}
      <div style="font-size: 10px; color: #718096;">${qty} x PHP ${price.toFixed(2)}</div>
    </div>
  `;
}).join("");
const dashedLine = '<div class="dashed-line">--------------------------------</div>';
return `
  <!DOCTYPE html>
  <html>
  <head>
    <meta charset="UTF-8">
    <title>Receipt - #${receiptNo}</title>
    <style>
      * {
        margin: 0;
        padding: 0;
        box-sizing: border-box;
      }
      @page {
        size: 58mm auto;
        margin: 0;
      }
      @media print {
        body {
          width: 58mm;
          margin: 0;
          padding: 8px;
          -webkit-print-color-adjust: exact !important;
          print-color-adjust: exact !important;
        }
        .no-print {
          display: none !important;
        }
        .dashed-line {
          display: block !important;
          text-align: center;
          font-family: monospace;
          font-size: 10px;
          color: #333 !important;
          letter-spacing: -1px;
          margin: 8px 0;
          -webkit-print-color-adjust: exact !important;
          print-color-adjust: exact !important;
        }
      }
      body {
        font-family: Arial, sans-serif;
        font-size: 11px;
        line-height: 1.4;
        width: 58mm;
```

```
          max-width: 58mm;
          padding: 10px;
          background: white;
          color: #1a202c;
        }
        .receipt-container {
          width: 100%;
        }
        .dashed-line {
          display: block;
          text-align: center;
          font-family: monospace;
          font-size: 10px;
          color: #333;
          letter-spacing: -1px;
          margin: 8px 0;
        }
      </style>
    </head>
    <body>
      <div class="receipt-container">
        <!-- Header -->
        <div style="text-align: center; margin-bottom: 10px;">
          <div style="font-size: 16px; font-weight: bold; color: #1a365d; margin-bottom:
4px;">${storeName}</div>
          <div style="font-size: 10px; color: #4a5568;">${location}</div>
        </div>

        ${dashedLine}

        <!-- Receipt Number -->
        <div style="text-align: center; margin: 10px 0;">
          <div style="font-size: 9px; color: #718096; text-transform: uppercase; letter-spacing:
1px;">Receipt</div>
          <div style="font-size: 16px; font-weight: bold; color: #2d3748;">#${receiptNo}</div>
        </div>

        <!-- Date, Cashier, Payment -->
        <div style="margin-bottom: 10px;">
          <div style="display: flex; justify-content: space-between; margin: 4px 0; font-size: 11px;">
            <span style="color: #4a5568;">Date:</span>
            <span style="color: #1a202c;">${receiptDate}, ${receiptTime}</span>
          </div>
          <div style="display: flex; justify-content: space-between; margin: 4px 0; font-size: 11px;">
            <span style="color: #4a5568;">Cashier:</span>
            <span style="color: #1a202c;">${cashier}</span>
          </div>
          <div style="display: flex; justify-content: space-between; margin: 4px 0; font-size: 11px;">
            <span style="color: #4a5568;">Payment:</span>
            <span style="color: #1a202c;">${paymentMethod}</span>
          </div>
        </div>

        ${dashedLine}

        <!-- Items -->
        <div style="padding: 10px 0;">
          ${itemsHTML || '<div style="text-align: center; color: #718096; padding: 8px;">No items</div>'}
        </div>

        ${dashedLine}

        <!-- Summary -->
        <div style="padding-top: 10px;">
          <div style="display: flex; justify-content: space-between; margin: 4px 0; font-size: 11px;">
            <span style="color: #4a5568;">Subtotal:</span>
            <span style="color: #1a202c;">PHP ${subtotal.toFixed(2)}</span>
          </div>
          <div style="display: flex; justify-content: space-between; margin: 4px 0; font-size: 11px;">
            <span style="color: #4a5568;">Discount:</span>
            <span style="color: #1a202c;">PHP ${discount.toFixed(2)}</span>
          </div>

          <!-- Total -->
          <div style="display: flex; justify-content: space-between; margin: 8px 0; padding-top: 8px;">
            <span style="font-weight: bold; color: #1a365d; font-size: 13px;">Total:</span>
            <span style="font-weight: bold; color: #1a365d; font-size: 13px;">PHP ${total.toFixed(2)}</span>
```

```
          </div>

          ${cash !== null ? `
          <div style="display: flex; justify-content: space-between; margin: 4px 0; font-size: 11px;">
            <span style="color: #4a5568;">Amount Received:</span>
            <span style="color: #1a202c;">PHP ${cash.toFixed(2)}</span>
          </div>
          ` : ""}

          ${change !== null ? `
          <div style="display: flex; justify-content: space-between; margin: 4px 0; font-size: 11px;">
            <span style="color: #4a5568;">Change:</span>
            <span style="color: #1a202c;">PHP ${change.toFixed(2)}</span>
          </div>
          ` : ""}
        </div>

        ${dashedLine}

        <!-- Footer -->
        <div style="text-align: center; margin-top: 10px;">
          <div style="font-size: 11px; color: #4a5568;">Thank you for your purchase!</div>
          <div style="font-size: 10px; color: #a0aec0; margin-top: 2px;">This is not an official
receipt</div>
        </div>

      </div>
    </body>
    </html>
  `;
};
export async function sendReceiptToPrinter(receipt) {
  if (!receipt) throw new Error("No receipt payload provided");
  const printData = {
    storeName: receipt.storeName || "Create Your Style",
    contactNumber: receipt.contactNumber || "+631112224444",
    location: receipt.location || "Pasonanca, Zamboanga City",
    receiptNo: receipt.receiptNo || "000000",
    referenceNo: receipt.referenceNo || receipt.reference || "-",
    date: receipt.date || (/* @__PURE__ */ new Date()).toLocaleDateString(),
    time: receipt.time || (/* @__PURE__ */ new Date()).toLocaleTimeString(),
    cashier: receipt.cashier || receipt.performedByName || "N/A",
    items: (receipt.items || []).map((item) => ({
      name: item.name || item.itemName || "Item",
      qty: item.qty || item.quantity || 1,
      price: item.price || item.itemPrice || 0,
      total: (item.price || item.itemPrice || 0) * (item.qty || item.quantity || 1),
      size: item.size || item.selectedSize || "",
      selectedSize: item.selectedSize || item.size || "",
      variant: item.selectedVariation || item.variant || "",
      selectedVariation: item.selectedVariation || item.variant || ""
    })),
    paymentMethod: receipt.paymentMethod || "CASH",
    subtotal: receipt.subtotal || 0,
    discount: receipt.discount || 0,
    discounts: receipt.discounts || [],
    total: receipt.total || 0,
    cash: receipt.cash,
    change: receipt.change
  };
  try {
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), 500);
    const response = await fetch("http://localhost:9100/print", {
      method: "POST",
      headers: {
        "Content-Type": "application/json"
      },
      body: JSON.stringify(printData),
      signal: controller.signal
    });
    clearTimeout(timeoutId);
    const result = await response.json();
    if (result.success) {
      return { success: true, message: "Receipt printed successfully" };
    } else {
      throw new Error(result.error || "Print failed");
```

```
    }
  } catch (error) {
    console.warn("Print server not available, falling back to window.print()", error);
    return new Promise((resolve, reject) => {
      try {
        const receiptHTML = buildReceiptHTML(receipt);
        let iframe = document.getElementById("receipt-print-iframe");
        if (!iframe) {
          iframe = document.createElement("iframe");
          iframe.id = "receipt-print-iframe";
          iframe.style.cssText = "position:fixed;top:-10000px;left:-
10000px;width:0;height:0;border:none;visibility:hidden;";
          document.body.appendChild(iframe);
        }
        const doc = iframe.contentDocument || iframe.contentWindow.document;
        doc.open();
        doc.write(receiptHTML);
        doc.close();
        iframe.onload = () => {
          try {
            iframe.contentWindow.focus();
            iframe.contentWindow.print();
            resolve({ success: true, message: "Print dialog opened (iframe fallback)" });
          } catch (printErr) {
            reject(printErr);
          }
        };
      } catch (fallbackError) {
        reject(fallbackError);
      }
    });
  }
}
```

FILE: pos/frontend/src/utils/skuUtils.js
==========

```
export const generateDynamicSku = (baseSku, variantName, sizeName) => {
  let sku = baseSku || "N/A";
  if (sku === "N/A" || !variantName && !sizeName) return sku;
  let suffix = "";
  if (variantName) {
    suffix += `-${variantName.substring(0, 2).toUpperCase()}`;
  }
  if (sizeName) {
    const sizeInitial = sizeName === "Free Size" ? "FS" : sizeName.substring(0, 2).toUpperCase();
    suffix += `-${sizeInitial}`;
  }
  return `${sku}${suffix}`;
};
```

FILE: pos/frontend/src/utils/transactionDisplay.js
==========

```
const VARIANT_ONLY_SIZE_KEY = "__VARIANT_ONLY__";
export function cleanItemNameForDisplay(item) {
  let name = String(item?.itemName || "").trim();
  name = name.replace(/\s*\([^)]*__VARIANT_ONLY__[^)]*\)\s*$/i, "").trim();
  return name || "Item";
}
export function formatItemVariantSizeLabel(item) {
  const size = item?.selectedSize || item?.size || "";
  const variant = String(item?.variant || item?.selectedVariation || "").trim();
  if (size === VARIANT_ONLY_SIZE_KEY) {
    return variant || "";
  }
  if (size && variant) return `${variant} / ${size}`;
  if (size) return size;
  return variant;
}
export function formatReceiptVariantSizeLine(item) {
  const rawSize = item?.selectedSize || item?.size || "";
  const size = rawSize === VARIANT_ONLY_SIZE_KEY ? "" : String(rawSize).trim();
  const variant = String(item?.variant || item?.selectedVariation || "").trim();
  const parts = [];
```

```
  if (variant) parts.push(variant);
  if (size) parts.push(size);
  return parts.join(" | ");
}
export function lineSubtotalFromItems(transaction) {
  if (!transaction?.items?.length) return 0;
  return transaction.items.reduce((sum, item) => {
    if (item.returnStatus === "Returned") return sum;
    return sum + (item.price || item.itemPrice || 0) * (item.quantity || 1);
  }, 0);
}
export function resolveTransactionDiscount(transaction, lineSubtotal, { skipInference = false } = {}) {
  const stored = Number(transaction?.discount ?? transaction?.discountAmount ?? 0) || 0;
  if (stored > 0) return stored;
  if (skipInference) return 0;
  const status = transaction?.status || "";
  if (status === "Returned" || status === "Partially Returned") return 0;
  const hasLinkedReturns = (transaction?.returnTransactions?.length || 0) > 0 ||
(transaction?.returnTransactionIds?.length || 0) > 0;
  if (hasLinkedReturns) return 0;
  const total = Number(transaction?.totalAmount);
  if (!Number.isFinite(total) || lineSubtotal <= total + 5e-3) return 0;
  return Math.round((lineSubtotal - total) * 100) / 100;
}
```

FILE: pos/frontend/vercel.json
==========

```
{
  "rewrites": [
    {
      "source": "/((?!assets/).*)",
      "destination": "/index.html"
    }
  ],
  "headers": [
    {
      "source": "/assets/(.*)",
      "headers": [
        {
          "key": "Cache-Control",
          "value": "public, max-age=31536000, immutable"
        }
      ]
    },
    {
      "source": "/(.*)",
      "headers": [
        {
          "key": "X-Content-Type-Options",
          "value": "nosniff"
        },
        {
          "key": "X-Frame-Options",
          "value": "DENY"
        },
        {
          "key": "X-XSS-Protection",
          "value": "1; mode=block"
        }
      ]
    }
  ]
}
```

FILE: pos/frontend/vite.config.js
==========

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";
import { VitePWA } from "vite-plugin-pwa";
export default defineConfig({
  plugins: [
    react(),
    tailwindcss(),
```

```
VitePWA({
  registerType: "autoUpdate",
  includeAssets: ["pwa-192x192.png", "pwa-512x512.png"],
  manifest: {
    name: "CYS POS - Create Your Style",
    short_name: "CYS POS",
    description: "Point of Sale System for Create Your Style",
    theme_color: "#8B7355",
    background_color: "#FFFFFF",
    display: "standalone",
    orientation: "any",
    start_url: "/",
    scope: "/",
    icons: [
      {
        src: "/pwa-192x192.png",
        sizes: "192x192",
        type: "image/png"
      },
      {
        src: "/pwa-512x512.png",
        sizes: "512x512",
        type: "image/png"
      },
      {
        src: "/pwa-512x512.png",
        sizes: "512x512",
        type: "image/png",
        purpose: "maskable"
      }
    ]
  },
  workbox: {
    // Precache all static assets (JS, CSS, HTML, images, fonts)
    globPatterns: ["**/*.{js,css,html,ico,png,svg,jpg,jpeg,woff,woff2}"],
    // Runtime caching for API calls and external resources
    runtimeCaching: [
      {
        // Cache API responses with NetworkFirst (always try network, fall back to cache)
        urlPattern: /\/api\/.*/i,
        handler: "NetworkFirst",
        options: {
          cacheName: "api-cache",
          expiration: {
            maxEntries: 200,
            maxAgeSeconds: 60 * 60
            // 1 hour
          },
          cacheableResponse: {
            statuses: [0, 200]
          }
        }
      },
      {
        // Cache Google Fonts stylesheets
        urlPattern: /^https:\/\/fonts\.googleapis\.com\/.*/i,
        handler: "StaleWhileRevalidate",
        options: {
          cacheName: "google-fonts-stylesheets",
          expiration: {
            maxEntries: 10,
            maxAgeSeconds: 60 * 60 * 24 * 365
            // 1 year
          }
        }
      },
      {
        // Cache Google Fonts webfont files
        urlPattern: /^https:\/\/fonts\.gstatic\.com\/.*/i,
        handler: "CacheFirst",
        options: {
          cacheName: "google-fonts-webfonts",
          expiration: {
            maxEntries: 30,
            maxAgeSeconds: 60 * 60 * 24 * 365
            // 1 year
          },
```

```
              cacheableResponse: {
                statuses: [0, 200]
              }
            }
          }
        ]
      }
    })
  ],
  resolve: {
    dedupe: ["react", "react-dom"]
  },
  optimizeDeps: {
    include: ["react", "react-dom", "recharts"]
  }
});
```

FILE: pos/mobile/__tests__/components/dashboard.test.js
==========

```
const getWelcomeName = (currentUser) => {
  if (!currentUser) return "Owner";
  return currentUser.firstName || currentUser.name || currentUser.email?.split("@")[0] || "Owner";
};
describe("Dashboard", () => {
  describe("Property 9: Dashboard Owner Name Display", () => {
    const ownerTestCases = [
      {
        user: { firstName: "John", lastName: "Doe", name: "John Doe", email: "john@test.com", role: "Owner" },
        expectedName: "John",
        description: "should display firstName when available"
      },
      {
        user: { name: "Jane Smith", email: "jane@test.com", role: "Owner" },
        expectedName: "Jane Smith",
        description: "should display name when firstName is not available"
      },
      {
        user: { email: "bob@company.com", role: "Owner" },
        expectedName: "bob",
        description: "should display email prefix when name fields are not available"
      },
      {
        user: { firstName: "", name: "", email: "test@example.com", role: "Owner" },
        expectedName: "test",
        description: "should fall back to email when name fields are empty strings"
      },
      {
        user: null,
        expectedName: "Owner",
        description: 'should display "Owner" when no user is logged in'
      },
      {
        user: void 0,
        expectedName: "Owner",
        description: 'should display "Owner" when user is undefined'
      }
    ];
    ownerTestCases.forEach(({ user, expectedName, description }) => {
      it(description, () => {
        const welcomeName = getWelcomeName(user);
        expect(welcomeName).toBe(expectedName);
      });
    });
    it("should always prefer firstName over other fields", () => {
      const testUsers = [
        { firstName: "Alice", name: "Alice Wonder", email: "alice@test.com" },
        { firstName: "Bob", name: "Robert Smith", email: "bob@test.com" },
        { firstName: "Charlie", name: "Charles Brown", email: "charlie@test.com" }
      ];
      testUsers.forEach((user) => {
        const welcomeName = getWelcomeName(user);
        expect(welcomeName).toBe(user.firstName);
      });
    });
    it("should never return empty or undefined welcome name", () => {
```

```
      const edgeCases = [
        null,
        void 0,
        {},
        { firstName: "", name: "", email: "" },
        { firstName: null, name: null, email: null }
      ];
      edgeCases.forEach((user) => {
        const welcomeName = getWelcomeName(user);
        expect(welcomeName).toBeTruthy();
        expect(welcomeName.length).toBeGreaterThan(0);
      });
    });
    it("should correctly extract username from email", () => {
      const emailTestCases = [
        { email: "john.doe@company.com", expected: "john.doe" },
        { email: "admin@test.org", expected: "admin" },
        { email: "user123@example.net", expected: "user123" }
      ];
      emailTestCases.forEach(({ email, expected }) => {
        const user = { email, role: "Owner" };
        const welcomeName = getWelcomeName(user);
        expect(welcomeName).toBe(expected);
      });
    });
  });
});



FILE: pos/mobile/__tests__/components/inventory.test.js
==============================================================================
const normalizeProduct = (product = {}) => {
  return {
    ...product,
    name: product.name || product.itemName || "No Name",
    brand: product.brand || product.brandName || "N/A",
    price: product.price ?? product.itemPrice ?? 0,
    stock: product.stock ?? product.currentStock ?? 0,
    category: product.category || "Uncategorized",
    sku: product.sku || "N/A",
    itemImage: product.itemImage || product.image || "",
    dateAdded: product.dateAdded || product.createdAt || product.updatedAt || null
  };
};
const isLowStock = (product) => {
  const stock = product.stock || product.quantity || 0;
  return stock > 0 && stock < 5;
};
const hasRequiredFields = (product) => {
  const normalized = normalizeProduct(product);
  return normalized.name !== void 0 && normalized.sku !== void 0 && normalized.brand !== void 0 &&
normalized.category !== void 0 && normalized.price !== void 0 && normalized.stock !== void 0;
};
describe("Inventory Display", () => {
  describe("Property 5: Product Display Completeness", () => {
    const productTestCases = [
      {
        product: {
          _id: "1",
          name: "Test Product",
          sku: "SKU001",
          brand: "Test Brand",
          category: "Electronics",
          price: 99.99,
          stock: 50
        },
        description: "complete product data"
      },
      {
        product: {
          _id: "2",
          itemName: "Legacy Product",
          brandName: "Old Brand",
          itemPrice: 49.99,
          currentStock: 25,
          category: "Clothing"
```

```
      },
      description: "legacy field names"
    },
    {
      product: {
        _id: "3",
        name: "Minimal Product"
      },
      description: "minimal product data"
    },
    {
      product: {},
      description: "empty product object"
    }
  ];
  productTestCases.forEach(({ product, description }) => {
    it(`should have all required fields for ${description}`, () => {
      expect(hasRequiredFields(product)).toBe(true);
    });
  });
  it("should normalize all products to have consistent field names", () => {
    const products = [
      { name: "Product A", sku: "A001", brand: "Brand A", category: "Cat A", price: 10, stock: 5 },
      { itemName: "Product B", brandName: "Brand B", itemPrice: 20, currentStock: 10 },
      { name: "Product C" }
    ];
    products.forEach((product) => {
      const normalized = normalizeProduct(product);
      expect(normalized).toHaveProperty("name");
      expect(normalized).toHaveProperty("sku");
      expect(normalized).toHaveProperty("brand");
      expect(normalized).toHaveProperty("category");
      expect(normalized).toHaveProperty("price");
      expect(normalized).toHaveProperty("stock");
    });
  });
  it("should provide default values for missing fields", () => {
    const emptyProduct = {};
    const normalized = normalizeProduct(emptyProduct);
    expect(normalized.name).toBe("No Name");
    expect(normalized.brand).toBe("N/A");
    expect(normalized.sku).toBe("N/A");
    expect(normalized.category).toBe("Uncategorized");
    expect(normalized.price).toBe(0);
    expect(normalized.stock).toBe(0);
  });
});
describe("Property 6: Low Stock Highlighting", () => {
  const lowStockTestCases = [
    { stock: 0, expected: false, description: "out of stock (0)" },
    { stock: 1, expected: true, description: "very low stock (1)" },
    { stock: 2, expected: true, description: "low stock (2)" },
    { stock: 3, expected: true, description: "low stock (3)" },
    { stock: 4, expected: true, description: "low stock (4)" },
    { stock: 5, expected: false, description: "adequate stock (5)" },
    { stock: 10, expected: false, description: "good stock (10)" },
    { stock: 100, expected: false, description: "high stock (100)" }
  ];
  lowStockTestCases.forEach(({ stock, expected, description }) => {
    it(`should ${expected ? "highlight" : "not highlight"} ${description}`, () => {
      const product = { stock };
      expect(isLowStock(product)).toBe(expected);
    });
  });
  it("should handle products with quantity field instead of stock", () => {
    const product = { quantity: 3 };
    expect(isLowStock(product)).toBe(true);
  });
  it("should handle products with no stock field", () => {
    const product = { name: "No Stock Field" };
    expect(isLowStock(product)).toBe(false);
  });
  it("should highlight all products with stock between 1 and 4", () => {
    for (let stock = 1; stock <= 4; stock++) {
      const product = { stock };
      expect(isLowStock(product)).toBe(true);
    }
```

```
    });
    it("should not highlight products with stock >= 5", () => {
      const stockLevels = [5, 6, 10, 50, 100, 1e3];
      stockLevels.forEach((stock) => {
        const product = { stock };
        expect(isLowStock(product)).toBe(false);
      });
    });
  });
});
```

FILE: pos/mobile/__tests__/config/api.test.js
==========

```
import { createApiConfig } from "../../config/api";
describe("API Configuration", () => {
  describe("Property 1: API URL Construction", () => {
    const testCases = [
      { ip: "192.168.1.1", port: 5e3 },
      { ip: "192.168.0.100", port: 5e3 },
      { ip: "10.0.0.1", port: 3e3 },
      { ip: "172.16.0.1", port: 8080 },
      { ip: "127.0.0.1", port: 5e3 },
      { ip: "192.168.1.6", port: 5e3 }
    ];
    testCases.forEach(({ ip, port }) => {
      it(`should construct correct URLs for IP ${ip} and port ${port}`, () => {
        const config = createApiConfig(ip, port);
        const expectedBaseUrl = `http://${ip}:${port}`;
        const expectedApiUrl = `${expectedBaseUrl}/api`;
        expect(config.BASE_URL).toBe(expectedBaseUrl);
        expect(config.API_URL).toBe(expectedApiUrl);
      });
    });
    it("should use default port 5000 when not specified", () => {
      const config = createApiConfig("192.168.1.1");
      expect(config.BASE_URL).toBe("http://192.168.1.1:5000");
      expect(config.API_URL).toBe("http://192.168.1.1:5000/api");
    });
    it("should always have API_URL equal to BASE_URL + /api", () => {
      const randomIps = ["192.168.1.1", "10.0.0.5", "172.16.0.10"];
      const randomPorts = [3e3, 5e3, 8080];
      randomIps.forEach((ip) => {
        randomPorts.forEach((port) => {
          const config = createApiConfig(ip, port);
          expect(config.API_URL).toBe(`${config.BASE_URL}/api`);
        });
      });
    });
  });
});
```

FILE: pos/mobile/__tests__/services/auth.test.js
==========

```
const createMockVerifyOwnerPin = (mockApiCall) => {
  return async (pin) => {
    if (!pin || typeof pin !== "string" || pin.length !== 6) {
      return {
        success: false,
        message: "Invalid PIN format. PIN must be 6 digits.",
        data: null
      };
    }
    try {
      const response = await mockApiCall("/employees/verify-pin", {
        method: "POST",
        body: JSON.stringify({ pin })
      });
      if (response.success && response.data) {
        if (response.data.role !== "Owner") {
          return {
            success: false,
            message: "Only owner account can access the mobile app.",
            data: null
```

```
          };
        }
        return {
          success: true,
          message: "Owner PIN verified successfully",
          data: response.data,
          requiresPinReset: response.requiresPinReset || false
        };
      }
      return response;
    } catch (error) {
      return {
        success: false,
        message: error.message || "Failed to verify PIN. Please check your connection.",
        data: null
      };
    }
  };
};
describe("Owner Authentication", () => {
  describe("Property 2: Owner Role Access Grant", () => {
    const ownerEmployees = [
      { _id: "1", name: "John Owner", email: "john@test.com", role: "Owner", firstName: "John" },
      { _id: "2", name: "Jane Owner", email: "jane@test.com", role: "Owner", firstName: "Jane" },
      { _id: "3", name: "Bob Owner", email: "bob@test.com", role: "Owner", firstName: "Bob" }
    ];
    ownerEmployees.forEach((owner) => {
      it(`should grant access for Owner: ${owner.name}`, async () => {
        const mockApiCall = jest.fn().mockResolvedValue({
          success: true,
          data: owner,
          requiresPinReset: false
        });
        const verifyOwnerPin = createMockVerifyOwnerPin(mockApiCall);
        const result = await verifyOwnerPin("123456");
        expect(result.success).toBe(true);
        expect(result.data).toEqual(owner);
        expect(result.data.role).toBe("Owner");
      });
    });
    it("should include requiresPinReset flag when present", async () => {
      const mockApiCall = jest.fn().mockResolvedValue({
        success: true,
        data: { _id: "1", name: "Owner", role: "Owner" },
        requiresPinReset: true
      });
      const verifyOwnerPin = createMockVerifyOwnerPin(mockApiCall);
      const result = await verifyOwnerPin("123456");
      expect(result.success).toBe(true);
      expect(result.requiresPinReset).toBe(true);
    });
  });
  describe("Property 3: Non-Owner Role Access Denial", () => {
    const nonOwnerRoles = ["Staff", "Manager", "Cashier", "Admin", "Supervisor"];
    nonOwnerRoles.forEach((role) => {
      it(`should deny access for role: ${role}`, async () => {
        const mockApiCall = jest.fn().mockResolvedValue({
          success: true,
          data: { _id: "1", name: "Test User", role }
        });
        const verifyOwnerPin = createMockVerifyOwnerPin(mockApiCall);
        const result = await verifyOwnerPin("123456");
        expect(result.success).toBe(false);
        expect(result.message).toBe("Only owner account can access the mobile app.");
        expect(result.data).toBeNull();
      });
    });
    it("should deny access for empty role", async () => {
      const mockApiCall = jest.fn().mockResolvedValue({
        success: true,
        data: { _id: "1", name: "Test User", role: "" }
      });
      const verifyOwnerPin = createMockVerifyOwnerPin(mockApiCall);
      const result = await verifyOwnerPin("123456");
      expect(result.success).toBe(false);
      expect(result.message).toBe("Only owner account can access the mobile app.");
    });
```

```
  });
  describe("Property 4: Invalid PIN Rejection", () => {
    const invalidPins = [
      { pin: "", reason: "empty string" },
      { pin: "12345", reason: "too short (5 digits)" },
      { pin: "1234567", reason: "too long (7 digits)" },
      { pin: "abcdef", reason: "non-numeric" },
      { pin: null, reason: "null value" },
      { pin: void 0, reason: "undefined value" }
    ];
    invalidPins.forEach(({ pin, reason }) => {
      it(`should reject invalid PIN: ${reason}`, async () => {
        const mockApiCall = jest.fn();
        const verifyOwnerPin = createMockVerifyOwnerPin(mockApiCall);
        const result = await verifyOwnerPin(pin);
        expect(result.success).toBe(false);
        expect(result.message).toContain("Invalid PIN format");
        expect(mockApiCall).not.toHaveBeenCalled();
      });
    });
    it("should reject PIN that does not match any employee", async () => {
      const mockApiCall = jest.fn().mockResolvedValue({
        success: false,
        message: "Invalid PIN"
      });
      const verifyOwnerPin = createMockVerifyOwnerPin(mockApiCall);
      const result = await verifyOwnerPin("999999");
      expect(result.success).toBe(false);
    });
    it("should handle API errors gracefully", async () => {
      const mockApiCall = jest.fn().mockRejectedValue(new Error("Network error"));
      const verifyOwnerPin = createMockVerifyOwnerPin(mockApiCall);
      const result = await verifyOwnerPin("123456");
      expect(result.success).toBe(false);
      expect(result.message).toContain("Network error");
    });
  });
});
```

FILE: pos/mobile/app.json
==========

```
{
  "expo": {
    "name": "Create Your Style",
    "slug": "cysmob",
    "version": "1.0.0",
    "orientation": "default",
    "icon": "./app/(tabs)/iconz/iconn.jpeg",
    "scheme": "cysmob",
    "splash": {
      "image": "./app/(tabs)/iconz/iconn.jpeg",
      "resizeMode": "contain",
      "backgroundColor": "#ffffff"
    },
    "userInterfaceStyle": "automatic",
    "newArchEnabled": true,
    "ios": {
      "supportsTablet": true,
      "bundleIdentifier": "com.cys.pos.mobile"
    },
    "android": {
      "package": "com.cys.pos.mobile",
      "adaptiveIcon": {
        "backgroundColor": "#ffffff",
        "foregroundImage": "./app/(tabs)/iconz/iconn.jpeg"
      },
      "edgeToEdgeEnabled": true,
      "predictiveBackGestureEnabled": false
    },
    "web": {
      "output": "static",
      "favicon": "./assets/images/favicon.png"
    },
    "plugins": [
      "expo-router",
```

```
      [
        "expo-splash-screen",
        {
          "image": "./app/(tabs)/iconz/iconn.jpeg",
          "imageWidth": 200,
          "resizeMode": "contain",
          "backgroundColor": "#ffffff",
          "dark": {
            "backgroundColor": "#000000"
          }
        }
      ],
      "expo-font",
      "expo-screen-orientation"
    ],
    "experiments": {
      "typedRoutes": true,
      "reactCompiler": true
    },
    "extra": {
      "eas": {
        "projectId": "33ea0bb5-e5ad-4ca8-ac2b-1ba1a113ef23"
      }
    }
  }
}
```

FILE: pos/mobile/app/_layout.tsx
==========

```
import { useColorScheme } from "@/hooks/use-color-scheme";
import {
  DarkTheme,
  DefaultTheme,
  ThemeProvider
} from "@react-navigation/native";
import { useFonts } from "expo-font";
import { Stack } from "expo-router";
import * as SplashScreen from "expo-splash-screen";
import { StatusBar } from "expo-status-bar";
import { useEffect } from "react";
import "react-native-reanimated";
export const unstable_settings = {
  anchor: "(tabs)"
};
SplashScreen.preventAutoHideAsync();
export default function RootLayout() {
  const colorScheme = useColorScheme();
  const [loaded, error] = useFonts({
    Allura: require("../components/shared/fontz/Allura-Regular.ttf"),
    Insta: require("../components/shared/fontz/Instagram Sans.ttf"),
    DancingS: require("../components/shared/fontz/DancingScript.ttf"),
    Poppins: require("../components/shared/fontz/Poppins-Bold.ttf")
  });
  useEffect(() => {
    if (loaded || error) {
      SplashScreen.hideAsync();
    }
  }, [loaded, error]);
  if (!loaded && !error) {
    return null;
  }
  return /* @__PURE__ */ React.createElement(ThemeProvider, { value: colorScheme === "dark" ? DarkTheme :
DefaultTheme }, /* @__PURE__ */ React.createElement(Stack, null, /* @__PURE__ */
React.createElement(Stack.Screen, { name: "pinlogin", options: { headerShown: false } }), /* @__PURE__ */
React.createElement(Stack.Screen, { name: "(tabs)", options: { headerShown: false } }), /* @__PURE__ */
React.createElement(
    Stack.Screen,
    {
      name: "Notification",
      options: { title: "Notifications", headerShown: false }
    }
  )), /* @__PURE__ */ React.createElement(StatusBar, { style: "auto" }));
}
```

FILE: pos/mobile/app/(tabs)/_layout.tsx
==========

```
import * as Notifications from "expo-notifications";
import { Tabs } from "expo-router";
import { useEffect } from "react";
import { Image, useWindowDimensions } from "react-native";
import { HapticTab } from "@/components/haptic-tab";
import { Colors } from "@/constants/theme";
import { DataProvider } from "@/context/DataContext";
import { useColorScheme } from "@/hooks/use-color-scheme";
import { setupNotificationChannel } from "@/services/notificationService";
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: true,
    shouldShowBanner: true,
    shouldShowList: true
  })
});
export default function TabLayout() {
  const colorScheme = useColorScheme();
  const { width, height } = useWindowDimensions();
  const isLandscape = width > height;
  useEffect(() => {
    setupNotificationChannel();
  }, []);
  return /* @__PURE__ */ React.createElement(DataProvider, null, /* @__PURE__ */ React.createElement(
    Tabs,
    {
      screenOptions: {
        tabBarActiveTintColor: Colors[colorScheme ?? "light"].tint,
        headerShown: false,
        tabBarButton: HapticTab,
        tabBarStyle: {
          height: 60,
          paddingTop: 8,
          paddingBottom: 8,
          borderTopWidth: 0,
          elevation: 10,
          shadowColor: "#000",
          shadowOffset: { width: 0, height: -2 },
          shadowOpacity: 0.1,
          shadowRadius: 4,
          backgroundColor: "#fff",
          display: isLandscape ? "none" : "flex"
        }
      }
    },
    /* @__PURE__ */ React.createElement(
      Tabs.Screen,
      {
        name: "index",
        options: {
          title: "",
          tabBarIcon: ({ focused }) => /* @__PURE__ */ React.createElement(
            Image,
            {
              source: focused ? require("../(tabs)/iconz/home-active.png") : require("../(tabs)/iconz/home-
inactive.png"),
              style: { width: 28, height: 28, marginTop: 8 },
              resizeMode: "contain"
            }
          )
        }
      }
    ),
    /* @__PURE__ */ React.createElement(
      Tabs.Screen,
      {
        name: "transaction",
        options: {
          title: "",
          tabBarIcon: ({ focused }) => /* @__PURE__ */ React.createElement(
            Image,
            {
              source: focused ? require("../(tabs)/iconz/transaction-active.png") :
```

```
require("../(tabs)/iconz/transaction-inactive.png"),
              style: { width: 28, height: 28, marginTop: 8 },
              resizeMode: "contain"
            }
          )
        }
      }
    ),
    /* @__PURE__ */ React.createElement(
      Tabs.Screen,
      {
        name: "analytics",
        options: {
          title: "",
          tabBarIcon: ({ focused }) => /* @__PURE__ */ React.createElement(
            Image,
            {
              source: focused ? require("../(tabs)/iconz/analytics-active.png") :
require("../(tabs)/iconz/analytics-inactive.png"),
              style: { width: 28, height: 28, marginTop: 8 },
              resizeMode: "contain"
            }
          )
        }
      }
    ),
    /* @__PURE__ */ React.createElement(
      Tabs.Screen,
      {
        name: "inventory",
        options: {
          title: "",
          tabBarIcon: ({ focused }) => /* @__PURE__ */ React.createElement(
            Image,
            {
              source: focused ? require("../(tabs)/iconz/basket-active.png") :
require("../(tabs)/iconz/basket-inactive.png"),
              style: { width: 28, height: 28, marginTop: 8 },
              resizeMode: "contain"
            }
          )
        }
      }
    ),
    /* @__PURE__ */ React.createElement(
      Tabs.Screen,
      {
        name: "profile",
        options: {
          title: "",
          tabBarIcon: ({ focused }) => /* @__PURE__ */ React.createElement(
            Image,
            {
              source: focused ? require("../(tabs)/iconz/profile-active.png") :
require("../(tabs)/iconz/profile-inactive.png"),
              style: { width: 28, height: 28, marginTop: 8 },
              resizeMode: "contain"
            }
          )
        }
      }
    )
  ));
}
```

FILE: pos/mobile/app/(tabs)/analytics.jsx
==========

```
import Header from "@/components/shared/header";
import { useData } from "@/context/DataContext";
import { useFocusEffect } from "@react-navigation/native";
import { Buffer } from "buffer";
import * as FileSystem from "expo-file-system";
import * as Print from "expo-print";
import * as Sharing from "expo-sharing";
import {
```

```
  useCallback,
  useMemo,
  useRef,
  useState
} from "react";
import {
  ActivityIndicator,
  Alert,
  Dimensions,
  FlatList,
  Image,
  LayoutAnimation,
  RefreshControl,
  ScrollView,
  StyleSheet,
  Text,
  TouchableOpacity,
  View
} from "react-native";
import { BarChart } from "react-native-chart-kit";
import Animated, {
  FadeIn,
  Layout
} from "react-native-reanimated";
import * as XLSX from "xlsx";
import {
  stockAPI,
  stockMovementAPI,
  transactionAPI
} from "../../services/api";
global.Buffer = Buffer;
export default function Analytics() {
  const {
    transactions: cachedTransactions,
    products: cachedProducts,
    fetchTransactions,
    fetchProducts,
    invalidateCache,
    inventoryStats,
    calculateDashboardStats
  } = useData();
  const [expanded, setExpanded] = useState(false);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const [chartType, setChartType] = useState("daily");
  const [initialLoad, setInitialLoad] = useState(true);
  const [statsLoading, setStatsLoading] = useState(false);
  const analyticsCalculated = useRef(false);
  const [dashboardStats, setDashboardStats] = useState({
    totalSalesToday: 0,
    totalTransactions: 0,
    averageTransactionValue: 0,
    salesGrowthRate: 0,
    previousPeriodSales: 0,
    previousPeriodTransactions: 0,
    profit: 0,
    cogs: 0
  });
  const [chartData, setChartData] = useState({
    daily: {
      labels: ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"],
      datasets: [{ data: [0] }]
    },
    monthly: {
      labels: ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
      datasets: [{ data: [0] }]
    },
    yearly: { labels: ["2023", "2024", "2025"], datasets: [{ data: [0] }] }
  });
  const [topProducts, setTopProducts] = useState([]);
  const [lowStockItems, setLowStockItems] = useState([]);
  const [stockInData, setStockInData] = useState([0]);
  const [stockOutData, setStockOutData] = useState([0]);
  const [stockChartLabels, setStockChartLabels] = useState([
    "Mon",
    "Tue",
    "Wed",
```

```
  "Thu",
  "Fri",
  "Sat",
  "Sun"
]);
const commonChartConfig = useMemo(() => ({
  backgroundColor: "#fff",
  backgroundGradientFrom: "#fff",
  backgroundGradientTo: "#fff",
  decimalPlaces: 0,
  labelColor: (opacity = 1) => `rgba(0, 0, 0, ${opacity})`,
  style: { borderRadius: 16 },
  propsForDots: { r: "4", strokeWidth: "2", stroke: "#fff" }
}), []);
const salesChartConfig = useMemo(() => ({
  ...commonChartConfig,
  color: (opacity = 1) => `rgba(173, 127, 101, ${opacity})`
}), [commonChartConfig]);
const stockInChartConfig = useMemo(() => ({
  ...commonChartConfig,
  color: (opacity = 1) => `rgba(16, 185, 129, 1)`,
  // Green #10B981 (Solid)
  barPercentage: 0.6,
  fillShadowGradient: "#10B981",
  fillShadowGradientFrom: "#10B981",
  fillShadowGradientTo: "#10B981",
  fillShadowGradientOpacity: 1,
  propsForBackgroundLines: {
    strokeWidth: 0.5,
    stroke: "#e0e0e0"
  },
  propsForLabels: { fontSize: 10 }
}), [commonChartConfig]);
const stockOutChartConfig = useMemo(() => ({
  ...commonChartConfig,
  color: (opacity = 1) => `rgba(239, 68, 68, 1)`,
  // Red #EF4444 (Solid)
  barPercentage: 0.6,
  fillShadowGradient: "#EF4444",
  fillShadowGradientFrom: "#EF4444",
  fillShadowGradientTo: "#EF4444",
  fillShadowGradientOpacity: 1,
  propsForBackgroundLines: {
    strokeWidth: 0.5,
    stroke: "#e0e0e0"
  },
  propsForLabels: { fontSize: 10 }
}), [commonChartConfig]);
const screenWidth = useMemo(() => Dimensions.get("window").width - 64, []);
const fetchAnalyticsData = async () => {
  try {
    setLoading(true);
    await Promise.all([
      fetchDashboardStats(chartType),
      // Use current chartType
      fetchSalesOverTime(),
      fetchLowStockItems(),
      fetchStockMovements(chartType),
      fetchTopProducts(),
      calculateDashboardStats(true)
      // Fetch inventory stats from DataContext
    ]);
  } catch (error) {
    console.warn("Failed to fetch analytics:", error?.message);
  } finally {
    setLoading(false);
    setRefreshing(false);
  }
};
const fetchTopProducts = async () => {
  try {
    const response = await transactionAPI.getTopSelling({
      limit: 5,
      period: chartType
    });
    if (response?.success && response.data) {
      const mappedProducts = response.data.map((product) => ({
```

```
        item: product.itemName || "Unknown",
        quantity: product.totalQuantitySold || 0,
        total: product.totalQuantitySold || 0,
        // Using quantity as total for now based on UI
        image: product.itemImage ? { uri: product.itemImage } : require("./iconz/profile3.png")
      }));
      setTopProducts(mappedProducts);
    }
  } catch (error) {
    console.warn("Failed to fetch top products:", error?.message);
  }
};
const fetchDashboardStats = async (timeframe = "daily", showLoader = true) => {
  try {
    if (showLoader) setLoading(true);
    const response = await transactionAPI.getDashboardStats(timeframe);
    if (response?.success && response.data) {
      const {
        totalSalesToday,
        totalTransactions,
        growthRate,
        lowStockItems: lowStockItems2,
        totalSalesPrevious,
        profit
      } = response.data;
      const avgValue = totalTransactions > 0 ? totalSalesToday / totalTransactions : 0;
      const cogs = (totalSalesToday || 0) - (profit || 0);
      setDashboardStats({
        totalSalesToday: totalSalesToday || 0,
        totalTransactions: totalTransactions || 0,
        averageTransactionValue: avgValue,
        salesGrowthRate: growthRate || 0,
        previousPeriodSales: totalSalesPrevious || 0,
        previousPeriodTransactions: 0,
        profit: profit || 0,
        cogs: cogs > 0 ? cogs : 0
      });
    }
  } catch (error) {
    console.warn("Failed to fetch dashboard stats:", error?.message);
  } finally {
    if (showLoader) setLoading(false);
  }
};
const [salesData, setSalesData] = useState({
  labels: ["Loading..."],
  datasets: [{ data: [0] }]
});
const preparedSalesData = useMemo(() => ({
  labels: salesData.labels.length > 0 ? salesData.labels : ["No Data"],
  datasets: [
    {
      data: salesData.datasets[0].data.length > 0 ? salesData.datasets[0].data : [0]
    }
  ]
}), [salesData]);
const fetchSalesOverTime = async () => {
  try {
    const response = await transactionAPI.getSalesOverTime(chartType);
    if (response && response.success && response.data) {
      const labels = response.data.map((item) => item.period);
      const data = response.data.map(
        (item) => item.revenue || item.totalSales || 0
      );
      if (labels.length === 0) {
        setSalesData({
          labels: ["No Data"],
          datasets: [{ data: [0] }]
        });
      } else {
        setSalesData({
          labels,
          datasets: [{ data }]
        });
      }
    }
  } catch (error) {
```

```
      console.warn("Failed to fetch sales over time:", error?.message);
    }
  };
  const fetchLowStockItems = async () => {
    try {
      const response = await stockAPI.getLowStock();
      if (response?.success && response.data) {
        const items = response.data.map((p, idx) => ({
          id: p._id || p.sku || idx,
          name: p.itemName || "Unknown Product",
          stocks: p.currentStock || 0,
          status: p.alertType === "out_of_stock" || (p.currentStock || 0) === 0 ? "Out of Stock" : "Low
Stock"
        }));
        setLowStockItems(items);
      }
    } catch (error) {
      console.warn("Failed to fetch low stock items:", error?.message);
    }
  };
  const fetchStockMovements = async (timeframe = "daily") => {
    try {
      const response = await stockMovementAPI.getStockStatsOverTime(timeframe);
      if (response?.success && response.data) {
        const { stockIn, pullOut, labels } = response.data;
        setStockInData(stockIn);
        setStockOutData(pullOut);
        setStockChartLabels(labels);
      }
    } catch (error) {
      console.warn("Failed to fetch stock movements:", error?.message);
    }
  };
  useFocusEffect(
    useCallback(() => {
      const loadStats = async () => {
        await fetchAnalyticsData();
        analyticsCalculated.current = true;
        setInitialLoad(false);
      };
      loadStats();
    }, [])
  );
  const statsCache = useRef({});
  useFocusEffect(
    useCallback(() => {
      const updateForFilter = async () => {
        if (statsCache.current[chartType]) {
          setDashboardStats(statsCache.current[chartType]);
          setStatsLoading(false);
        } else {
          setStatsLoading(true);
        }
        await Promise.all([
          fetchSalesOverTime(),
          fetchDashboardStats(chartType, false),
          fetchStockMovements(chartType),
          fetchTopProducts()
        ]);
        setDashboardStats((prev) => {
          statsCache.current[chartType] = prev;
          return prev;
        });
        setStatsLoading(false);
      };
      updateForFilter();
    }, [chartType])
  );
  const onRefresh = useCallback(async () => {
    setRefreshing(true);
    invalidateCache("all");
    statsCache.current = {};
    analyticsCalculated.current = false;
    await Promise.all([fetchTransactions(true), fetchProducts(true)]);
    await Promise.all([
      fetchDashboardStats(chartType),
      fetchSalesOverTime(),
```

```
      fetchLowStockItems(),
      fetchStockMovements(chartType),
      fetchTopProducts()
    ]);
    setRefreshing(false);
  }, [invalidateCache, fetchTransactions, fetchProducts, chartType]);
  const toggleExpand = () => {
    LayoutAnimation.configureNext(LayoutAnimation.Presets.easeInEaseOut);
    setExpanded(!expanded);
  };
  const visibleData = useMemo(() => expanded ? lowStockItems : lowStockItems.slice(0, 3), [expanded,
lowStockItems]);
  const formatCurrency = (value) => {
    return `\u20B1${(value || 0).toLocaleString("en-PH", { minimumFractionDigits: 0, maximumFractionDigits:
0 })}`;
  };
  const formatCurrencyShort = (v) => {
    const n = Number(v) || 0;
    if (n === 0) return "\u20B10";
    if (n >= 1e6) {
      const m = n / 1e6;
      return `\u20B1${m % 1 === 0 ? m.toFixed(0) : m.toFixed(1)}M`;
    }
    if (n >= 1e3) {
      const k = n / 1e3;
      return `\u20B1${k % 1 === 0 ? k.toFixed(0) : k.toFixed(1)}K`;
    }
    return `\u20B1${n.toLocaleString("en-PH", { maximumFractionDigits: 0 })}`;
  };
  const getPeriodLabel = () => {
    switch (chartType) {
      case "daily":
        return "today";
      case "weekly":
        return "this week";
      case "monthly":
        return "this month";
      case "yearly":
        return "this year";
      default:
        return "today";
    }
  };
  const TAB_LABELS = [
    { label: "Today", value: "daily" },
    { label: "Week", value: "weekly" },
    { label: "Month", value: "monthly" },
    { label: "Year", value: "yearly" }
  ];
  const formatPercentage = (value) => {
    const sign = value >= 0 ? "+" : "";
    return `${sign}${(value || 0).toFixed(1)}%`;
  };
  const getComparisonText = (current, previous) => {
    if (previous === 0 || !previous) return "";
    const change = (current - previous) / previous * 100;
    return `${change >= 0 ? "+" : ""}${change.toFixed(0)}% vs last period`;
  };
  const handleExportExcel = async () => {
    try {
      const sales = salesData.datasets[0].data.map((val, i) => ({
        Period: salesData.labels[i] || `Period ${i + 1}`,
        Sales: val
      }));
      const lowStock = lowStockItems.map((item) => ({
        Product: item.name,
        Stock: item.stocks,
        Status: item.status
      }));
      const wb = XLSX.utils.book_new();
      XLSX.utils.book_append_sheet(
        wb,
        XLSX.utils.json_to_sheet(sales),
        "Sales Data"
      );
      XLSX.utils.book_append_sheet(
        wb,
```

```
        XLSX.utils.json_to_sheet(lowStock),
        "Low Stock"
      );
      const wbout = XLSX.write(wb, { type: "base64", bookType: "xlsx" });
      const fileUri = FileSystem.cacheDirectory + "Analytics_Report.xlsx";
      await FileSystem.writeAsStringAsync(fileUri, wbout, {
        encoding: FileSystem.EncodingType.Base64
      });
      await Sharing.shareAsync(fileUri, {
        mimeType: "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet",
        dialogTitle: "Export Analytics Report",
        UTI: "com.microsoft.excel.xlsx"
      });
      Alert.alert("Success", "Analytics report exported!");
    } catch (error) {
      console.error("Export Error:", error);
      Alert.alert("Error", "Failed to export Excel: " + error.message);
    }
  };
  const handlePrint = async () => {
    try {
      const html = `
        <html>
          <body style="font-family: Arial; padding: 20px;">
            <h2 style="text-align:center;">Create Your Style Analytics Report</h2>

            <h3>Dashboard Summary</h3>
            <table border="1" style="width:100%;border-collapse:collapse;">
              <tr><td>Total Sales Today</td><td>${formatCurrency(dashboardStats.totalSalesToday)}</td></tr>
              <tr><td>Total Transactions</td><td>${dashboardStats.totalTransactions}</td></tr>
              <tr><td>Average
Transaction</td><td>${formatCurrency(dashboardStats.averageTransactionValue)}</td></tr>
              <tr><td>Growth Rate</td><td>${formatPercentage(dashboardStats.salesGrowthRate)}</td></tr>
            </table>

            <h3 style="margin-top:25px;">Sales Over Time (${chartType})</h3>
            <table border="1" style="width:100%;border-collapse:collapse;">
              <tr><th>Period</th><th>Sales</th></tr>
              ${salesData.labels.map(
        (label, i) => `<tr><td>${label}</td><td>${formatCurrency(salesData.datasets[0].data[i])}</td></tr>`
      ).join("")}
            </table>

            <h3 style="margin-top:25px;">Low Stock Items</h3>
            <table border="1" style="width:100%;border-collapse:collapse;">
              <tr><th>Product</th><th>Stock</th><th>Status</th></tr>
              ${lowStockItems.map(
        (item) => `<tr><td>${item.name}</td><td>${item.stocks}</td><td>${item.status}</td></tr>`
      ).join("")}
            </table>

            <p style="text-align:center;margin-top:30px;">Generated on ${(/* @__PURE__ */ new
Date()).toLocaleString()}</p>
          </body>
        </html>
      `;
      await Print.printAsync({ html });
    } catch (error) {
      console.error("Print Error:", error);
      Alert.alert("Error", "Failed to print report: " + error.message);
    }
  };
  if (loading && !refreshing) {
    return /* @__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */
React.createElement(Header, null), /* @__PURE__ */ React.createElement(View, { style: [styles.whitecard,
styles.loadingContainer] }, /* @__PURE__ */ React.createElement(ActivityIndicator, { size: "large", color:
"#AD7F65" }), /* @__PURE__ */ React.createElement(Text, { style: styles.loadingText }, "Loading
analytics...")));
  }
  return /* @__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */
React.createElement(Header, null), /* @__PURE__ */ React.createElement(View, { style: styles.whitecard }, /*
@__PURE__ */ React.createElement(
    ScrollView,
    {
      style: styles.main,
      refreshControl: /* @__PURE__ */ React.createElement(
        RefreshControl,
```

```
        {
          refreshing,
          onRefresh,
          colors: ["#AD7F65"]
        }
      )
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.filterRow }, /* @__PURE__ */
React.createElement(View, { style: styles.tabRow }, TAB_LABELS.map((t) => /* @__PURE__ */
React.createElement(
      TouchableOpacity,
      {
        key: t.value,
        style: [styles.filterTab, chartType === t.value && styles.filterTabActive],
        onPress: () => setChartType(t.value)
      },
      /* @__PURE__ */ React.createElement(Text, { style: [styles.filterTabText, chartType === t.value &&
styles.filterTabTextActive] }, t.label)
    ))), /* @__PURE__ */ React.createElement(View, { style: styles.filterActions }, /* @__PURE__ */
React.createElement(TouchableOpacity, { style: styles.filterIconBtn, onPress: handleExportExcel }, /*
@__PURE__ */ React.createElement(
      Image,
      {
        source: require("./iconz/print.png"),
        style: { width: 19, height: 19 }
      }
    )))),
    statsLoading ? /* @__PURE__ */ React.createElement(View, { style: styles.statsLoadingContainer }, /*
@__PURE__ */ React.createElement(View, { style: styles.statsRow }, /* @__PURE__ */ React.createElement(View,
{ style: [styles.statCard, styles.statCardSkeleton] }, /* @__PURE__ */ React.createElement(View, { style:
styles.skeletonValue }), /* @__PURE__ */ React.createElement(View, { style: styles.skeletonLabel }), /*
@__PURE__ */ React.createElement(View, { style: styles.skeletonDesc })), /* @__PURE__ */
React.createElement(View, { style: [styles.statCard, styles.statCardSkeleton] }, /* @__PURE__ */
React.createElement(View, { style: styles.skeletonValue }), /* @__PURE__ */ React.createElement(View, { style:
styles.skeletonLabel }), /* @__PURE__ */ React.createElement(View, { style: styles.skeletonDesc }))), /*
@__PURE__ */ React.createElement(View, { style: styles.statsRow }, /* @__PURE__ */ React.createElement(View,
{ style: [styles.statCard, styles.statCardSkeleton] }, /* @__PURE__ */ React.createElement(View, { style:
styles.skeletonValue }), /* @__PURE__ */ React.createElement(View, { style: styles.skeletonLabel }), /*
@__PURE__ */ React.createElement(View, { style: styles.skeletonDesc })), /* @__PURE__ */
React.createElement(View, { style: [styles.statCard, styles.statCardSkeleton] }, /* @__PURE__ */
React.createElement(View, { style: styles.skeletonValue }), /* @__PURE__ */ React.createElement(View, { style:
styles.skeletonLabel }), /* @__PURE__ */ React.createElement(View, { style: styles.skeletonDesc }))))) : /*
@__PURE__ */ React.createElement(Animated.View, { entering: FadeIn.duration(350), layout: Layout }, /*
@__PURE__ */ React.createElement(View, { style: styles.statsRow }, /* @__PURE__ */ React.createElement(View,
{ style: styles.statCard }, /* @__PURE__ */ React.createElement(Text, { style: styles.statValue },
formatCurrencyShort(dashboardStats.totalSalesToday)), /* @__PURE__ */ React.createElement(Text, { style:
[styles.statLabel, { color: "#2563EB" }] }, "Total Sales"), /* @__PURE__ */ React.createElement(Text, { style:
styles.statLabel1 }, "Total revenue ", getPeriodLabel()), /* @__PURE__ */ React.createElement(Text, { style:
styles.statLabel2 }, getComparisonText(
      dashboardStats.totalSalesToday,
      dashboardStats.previousPeriodSales
    ))), /* @__PURE__ */ React.createElement(View, { style: styles.statCard }, /* @__PURE__ */
React.createElement(Text, { style: styles.statValue }, formatCurrencyShort(dashboardStats.profit)), /*
@__PURE__ */ React.createElement(Text, { style: [styles.statLabel, { color: "#16a34a" }] }, "Total Profit"),
/* @__PURE__ */ React.createElement(Text, { style: styles.statLabel1 }, "Total earnings ",
getPeriodLabel())))), /* @__PURE__ */ React.createElement(View, { style: styles.statsRow }, /* @__PURE__ */
React.createElement(View, { style: styles.statCard }, /* @__PURE__ */ React.createElement(Text, { style:
styles.statValue }, formatCurrencyShort(dashboardStats.cogs)), /* @__PURE__ */ React.createElement(Text,
{ style: [styles.statLabel, { color: "#D97706" }] }, "Cost of Goods Sold"), /* @__PURE__ */
React.createElement(Text, { style: styles.statLabel1 }, "Cost of items sold ", getPeriodLabel())), /*
@__PURE__ */ React.createElement(View, { style: styles.statCard }, /* @__PURE__ */ React.createElement(
      Text,
      {
        style: styles.statValue
      },
      formatCurrencyShort(inventoryStats.inventoryValue || 0)
    ), /* @__PURE__ */ React.createElement(Text, { style: [styles.statLabel, { color: "#DC2626" }] },
"Inventory Value"), /* @__PURE__ */ React.createElement(Text, { style: styles.statLabel1 }, "Total value of
products in inventory")))),
    /* @__PURE__ */ React.createElement(View, { style: [styles.chartContainer, { marginBottom: 24 }] }, /*
@__PURE__ */ React.createElement(Text, { style: [styles.chartTitle, { marginBottom: 16 }] }, "Stock In"), /*
@__PURE__ */ React.createElement(
      BarChart,
      {
        data: {
          labels: stockChartLabels,
          datasets: [{ data: stockInData }]
```

```
        },
        width: screenWidth,
        height: 220,
        yAxisLabel: "",
        chartConfig: stockInChartConfig,
        style: { marginTop: 8, marginBottom: 8, paddingRight: 0 },
        fromZero: true,
        showBarTops: false,
        showValuesOnTopOfBars: true
      }
    )),
    /* @__PURE__ */ React.createElement(View, { style: [styles.chartContainer, { marginBottom: 24 }] }, /*
@__PURE__ */ React.createElement(Text, { style: [styles.chartTitle, { marginBottom: 16 }] }, "Pull Outs"), /*
@__PURE__ */ React.createElement(
      BarChart,
      {
        data: {
          labels: stockChartLabels,
          datasets: [{ data: stockOutData }]
        },
        width: screenWidth,
        height: 220,
        yAxisLabel: "",
        chartConfig: stockOutChartConfig,
        style: { marginTop: 8, marginBottom: 8, paddingRight: 0 },
        fromZero: true,
        showBarTops: false,
        showValuesOnTopOfBars: true
      }
    )),
    /* @__PURE__ */ React.createElement(View, { style: [styles.chartContainer, { height: 190 }] }, /*
@__PURE__ */ React.createElement(Text, { style: styles.chartTitle }, "Products Performance"),
topProducts.length === 0 ? /* @__PURE__ */ React.createElement(
      View,
      {
        style: {
          flex: 1,
          justifyContent: "center",
          alignItems: "center"
        }
      },
      /* @__PURE__ */ React.createElement(Text, { style: { color: "#888", fontSize: 14 } }, "No products sold
yet.")
    ) : /* @__PURE__ */ React.createElement(
      FlatList,
      {
        data: topProducts,
        keyExtractor: (item, index) => item.item + index,
        horizontal: true,
        showsHorizontalScrollIndicator: false,
        contentContainerStyle: { paddingHorizontal: 10 },
        renderItem: ({ item }) => /* @__PURE__ */ React.createElement(View, { style: styles.carouselCard },
/* @__PURE__ */ React.createElement(Image, { source: item.image, style: styles.carouselImage }), /* @__PURE__
*/ React.createElement(Text, { style: styles.carouselLabel }, item.item), /* @__PURE__ */
React.createElement(Text, { style: styles.carouselLabel }, "Sold: ", item.total))
      }
    )),
    /* @__PURE__ */ React.createElement(View, { style: styles.sectionContainer }, /* @__PURE__ */
React.createElement(View, { style: styles.sectionHeader }, /* @__PURE__ */ React.createElement(View, { style:
styles.sectionTitleContainer }, /* @__PURE__ */ React.createElement(Text, { style: styles.sectionTitle },
"Low & Out-of-Stock Items"), /* @__PURE__ */ React.createElement(Text, { style: styles.sectionSubtitle },
"Inventory items that need attention")), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        onPress: toggleExpand,
        style: styles.viewMoreButton,
        activeOpacity: 0.7
      },
      /* @__PURE__ */ React.createElement(Text, { style: styles.viewMoreText }, expanded ? "Show Less" :
"View All")
    )), /* @__PURE__ */ React.createElement(View, { style: styles.tableContainer }, /* @__PURE__ */
React.createElement(View, { style: styles.tableHeader }, /* @__PURE__ */ React.createElement(
      Text,
      {
        style: [styles.headerText, { flex: 3, textAlign: "left" }]
      },
      "Product"
```

```
    ), /* @__PURE__ */ React.createElement(
      Text,
      {
        style: [styles.headerText, { flex: 1, textAlign: "center" }]
      },
      "Stocks"
    ), /* @__PURE__ */ React.createElement(
      Text,
      {
        style: [styles.headerText, { flex: 1.5, textAlign: "right" }]
      },
      "Status"
    )), /* @__PURE__ */ React.createElement(View, { style: styles.tableBody }, visibleData.length === 0 ? /*
@__PURE__ */ React.createElement(View, { style: styles.emptyStateContainer }, /* @__PURE__ */
React.createElement(Text, { style: styles.emptyStateText }, "\u2713 All items are well stocked"), /*
@__PURE__ */ React.createElement(Text, { style: styles.emptyStateSubtext }, "No items need attention at this
time")) : visibleData.map((item, index) => /* @__PURE__ */ React.createElement(
      View,
      {
        key: item.id.toString(),
        style: [
          styles.tableRow,
          index % 2 === 0 && styles.evenRow
        ]
      },
      /* @__PURE__ */ React.createElement(
        Text,
        {
          style: [
            styles.rowText,
            { flex: 3, textAlign: "left", fontWeight: "500" }
          ],
          numberOfLines: 1
        },
        item.name
      ),
      /* @__PURE__ */ React.createElement(
        Text,
        {
          style: [
            styles.rowText,
            { flex: 1, textAlign: "center" }
          ]
        },
        item.stocks
      ),
      /* @__PURE__ */ React.createElement(View, { style: [styles.statusCell, { flex: 1.5 }] }, /* @__PURE__
*/ React.createElement(
        Text,
        {
          style: [
            styles.statusText,
            item.status === "Out of Stock" ? styles.statusTextCritical : styles.statusTextLow
          ]
        },
        item.status
      ))
    )))))
  )));
}
const styles = StyleSheet.create({
  loadingContainer: {
    justifyContent: "center",
    alignItems: "center"
  },
  loadingText: {
    marginTop: 12,
    fontSize: 14,
    color: "#666"
  },
  sectionContainer: {
    backgroundColor: "#fff",
    borderRadius: 12,
    marginTop: 16,
    marginBottom: 30,
    padding: 16,
    width: "100%",
```

```
  alignSelf: "center",
  shadowColor: "#000",
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.1,
  shadowRadius: 4,
  elevation: 3
},
sectionHeader: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  marginBottom: 16
},
sectionTitleContainer: {
  flex: 1
},
sectionTitle: {
  fontSize: 18,
  fontWeight: "700",
  color: "#333",
  marginBottom: 4
},
sectionSubtitle: {
  fontSize: 13,
  color: "#888"
},
viewMoreButton: {
  padding: 8,
  borderRadius: 6,
  backgroundColor: "#f5f5f5"
},
viewMoreText: {
  color: "#666",
  fontSize: 13,
  fontWeight: "500"
},
tableContainer: {
  borderRadius: 8,
  overflow: "hidden"
},
tableHeader: {
  flexDirection: "row",
  backgroundColor: "#f8f8f8",
  paddingVertical: 12,
  paddingHorizontal: 16,
  borderBottomWidth: 1,
  borderBottomColor: "#eee"
},
headerText: {
  fontSize: 12,
  fontWeight: "600",
  color: "#666",
  textTransform: "uppercase",
  letterSpacing: 0.5
},
tableBody: {
  backgroundColor: "#fff",
  width: "100%"
},
tableRow: {
  flexDirection: "row",
  paddingVertical: 14,
  paddingHorizontal: 16,
  alignItems: "center",
  borderBottomWidth: 1,
  borderBottomColor: "#f5f5f5",
  width: "100%"
},
evenRow: {
  backgroundColor: "#fcfcfc"
},
rowText: {
  fontSize: 14,
  color: "#444"
},
statusCell: {
  justifyContent: "center",
```

```
  alignItems: "flex-end",
  paddingRight: 4
},
statusText: {
  fontSize: 13,
  fontWeight: "500",
  textAlign: "right",
  width: "100%",
  paddingRight: 8
},
statusTextLow: {
  color: "#E6A23C"
},
statusTextCritical: {
  color: "#F56C6C"
},
container: {
  flex: 1,
  backgroundColor: "#53321c"
},
main: {
  width: "100%",
  paddingHorizontal: 16
},
whitecard: {
  flex: 1,
  backgroundColor: "#fff",
  marginTop: -10,
  borderTopLeftRadius: 15,
  borderTopRightRadius: 15,
  paddingTop: 2
},
filterRow: {
  flexDirection: "row",
  alignItems: "center",
  justifyContent: "space-between",
  marginBottom: 16,
  paddingHorizontal: 4,
  marginTop: 8
},
tabRow: {
  flexDirection: "row",
  backgroundColor: "#f3f4f6",
  borderRadius: 10,
  padding: 3,
  flex: 1,
  marginRight: 12
},
filterTab: {
  flex: 1,
  paddingVertical: 8,
  alignItems: "center",
  borderRadius: 8
},
filterTabActive: {
  backgroundColor: "#1f1f1f"
},
filterTabText: {
  fontSize: 13,
  fontWeight: "600",
  color: "#6b7280"
},
filterTabTextActive: {
  color: "#fff"
},
filterActions: {
  flexDirection: "row",
  gap: 8
},
filterIconBtn: {
  backgroundColor: "#f3f4f6",
  padding: 10,
  borderRadius: 10
},
statsLoadingContainer: {
  marginBottom: 4
},
```

```
statCardSkeleton: {
  justifyContent: "center",
  minHeight: 100
},
skeletonValue: {
  width: "60%",
  height: 22,
  borderRadius: 6,
  backgroundColor: "#e5e7eb",
  marginBottom: 8
},
skeletonLabel: {
  width: "50%",
  height: 12,
  borderRadius: 4,
  backgroundColor: "#e5e7eb",
  marginBottom: 6
},
skeletonDesc: {
  width: "80%",
  height: 8,
  borderRadius: 3,
  backgroundColor: "#f3f4f6"
},
statsRow: {
  flexDirection: "row",
  justifyContent: "space-between",
  marginBottom: 14
},
statCard: {
  flex: 1,
  backgroundColor: "#fff",
  padding: 16,
  borderRadius: 12,
  marginHorizontal: 6,
  borderWidth: 1,
  borderColor: "#e5e7eb",
  shadowColor: "#000",
  shadowOpacity: 0.04,
  shadowRadius: 4,
  shadowOffset: { width: 0, height: 2 },
  elevation: 2
},
statValue: {
  fontSize: 26,
  fontWeight: "800",
  color: "#1a1a1a",
  letterSpacing: -0.5,
  marginBottom: 4
},
statValuePositive: {
  color: "#09A046"
},
statValueNegative: {
  color: "#F56C6C"
},
statLabel: {
  fontSize: 13,
  fontWeight: "700",
  fontStyle: "italic",
  marginBottom: 6
},
statLabel1: {
  fontSize: 9,
  color: "#9ca3af",
  lineHeight: 12
},
statLabel2: {
  fontSize: 10,
  color: "#09A046",
  marginTop: 4,
  fontWeight: "500"
},
chartContainer: {
  backgroundColor: "#fff",
  borderRadius: 12,
  padding: 16,
```

```
  marginBottom: 16
},
chartHeader: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  marginBottom: 16,
  width: "100%",
  paddingHorizontal: 2
},
chartTitle: {
  fontSize: 18,
  fontWeight: "700",
  color: "#53321c"
},
chart: {
  borderRadius: 16
},
buttonContainer: {
  flexDirection: "row"
},
chartButton: {
  paddingVertical: 6,
  paddingHorizontal: 10,
  borderRadius: 8,
  marginLeft: 6,
  backgroundColor: "#e4d7ce"
},
activeButton: {
  backgroundColor: "#AD7F65"
},
buttonText: {
  fontSize: 13,
  color: "#53321c",
  fontWeight: "600"
},
activeButtonText: {
  color: "#fff"
},
carouselCard: {
  width: 110,
  height: 130,
  borderRadius: 8,
  backgroundColor: "#fafafa",
  marginRight: 10,
  justifyContent: "center",
  alignItems: "center",
  elevation: 1
},
carouselImage: {
  width: 85,
  height: 75,
  borderRadius: 6,
  resizeMode: "cover"
},
carouselLabel: {
  fontSize: 9,
  fontWeight: "500",
  marginTop: 6,
  textAlign: "center",
  color: "#333"
},
emptyStateContainer: {
  paddingVertical: 40,
  paddingHorizontal: 20,
  alignItems: "center",
  justifyContent: "center"
},
emptyStateText: {
  fontSize: 16,
  fontWeight: "600",
  color: "#09A046",
  marginBottom: 8,
  textAlign: "center"
},
emptyStateSubtext: {
  fontSize: 13,
```

```
    color: "#888",
    textAlign: "center"
  }
});



FILE: pos/mobile/app/(tabs)/index.jsx
==============================================================================
import Header from "@/components/shared/header";
import AsyncStorage from "@react-native-async-storage/async-storage";
import { useFocusEffect } from "@react-navigation/native";
import { useCallback, useEffect, useRef, useState } from "react";
import {
  ActivityIndicator,
  Animated,
  Dimensions,
  FlatList,
  Image,
  RefreshControl,
  ScrollView,
  StyleSheet,
  Text,
  TouchableOpacity,
  View
} from "react-native";
import { BarChart } from "react-native-chart-kit";
import Svg, { Circle, G } from "react-native-svg";
import { employeeAPI, transactionAPI } from "../../services/api";
const SCREEN_WIDTH = Dimensions.get("window").width;
const PIE_COLORS = ["#F4A6C1", "#A7C7E7", "#FAD02E", "#98FB98", "#FFB7B2", "#C3B1E1"];
const ONLINE_AVATAR_BACKGROUNDS = ["#EDE8E4", "#E8EDF2", "#E6EDE8", "#EEE6F0", "#EAF0E6"];
function initialsFromName(name) {
  if (!name || typeof name !== "string") return "?";
  const parts = name.trim().split(/\s+/).filter(Boolean);
  if (parts.length === 0) return "?";
  if (parts.length === 1) return parts[0].slice(0, 2).toUpperCase();
  return (parts[0][0] + parts[parts.length - 1][0]).toUpperCase();
}
function avatarBgForName(name) {
  let h = 0;
  const s = String(name || "");
  for (let i = 0; i < s.length; i++) h = (h + s.charCodeAt(i) * (i + 1)) % 997;
  return ONLINE_AVATAR_BACKGROUNDS[h % ONLINE_AVATAR_BACKGROUNDS.length];
}
function SimplePieChart({ data }) {
  if (!data || data.length === 0) return null;
  const total = data.reduce((sum, d) => sum + (d.value || 0), 0);
  if (total === 0) return null;
  const size = 160;
  const radius = size / 4;
  const strokeWidth = size / 2;
  const circumference = 2 * Math.PI * radius;
  const center = size / 2;
  let accumulated = 0;
  return /* @__PURE__ */ React.createElement(View, { style: { width: "100%", alignItems: "center",
marginVertical: 8 } }, /* @__PURE__ */ React.createElement(Svg, { width: size, height: size }, /* @__PURE__
*/ React.createElement(G, { rotation: "-90", origin: `${center}, ${center}` }, data.map((item, i) => {
    const pct = item.value / total;
    const dashLength = circumference * pct;
    const dashGap = circumference - dashLength;
    const offset = circumference * accumulated;
    accumulated += pct;
    return /* @__PURE__ */ React.createElement(
      Circle,
      {
        key: i,
        cx: center,
        cy: center,
        r: radius,
        stroke: PIE_COLORS[i % PIE_COLORS.length],
        strokeWidth,
        fill: "none",
        strokeDasharray: `${dashLength} ${dashGap}`,
        strokeDashoffset: -offset,
        strokeLinecap: "butt"
      }
```

```
    );
  })), /* @__PURE__ */ React.createElement(View, { style: { flexDirection: "row", flexWrap: "wrap",
marginTop: 16, gap: 6, justifyContent: "center" } }, data.map((item, i) => {
    const pct = (item.value / total * 100).toFixed(0);
    return /* @__PURE__ */ React.createElement(View, { key: i, style: { flexDirection: "row", alignItems:
"center", width: "46%", marginBottom: 6 } }, /* @__PURE__ */ React.createElement(
      View,
      {
        style: {
          width: 10,
          height: 10,
          borderRadius: 5,
          backgroundColor: PIE_COLORS[i % PIE_COLORS.length],
          marginRight: 6
        }
      }
    ), /* @__PURE__ */ React.createElement(View, { style: { flex: 1 } }, /* @__PURE__ */
React.createElement(Text, { style: { fontSize: 11, fontWeight: "700", color: "#333" }, numberOfLines: 1 },
item.name), /* @__PURE__ */ React.createElement(Text, { style: { fontSize: 10, color: "#888" } }, "\u20B1",
(item.value || 0).toLocaleString(), " \xB7 ", pct, "%")));
  }))));
}
export default function Dashboard() {
  const [timeframe, setTimeframe] = useState("daily");
  const [currentUser, setCurrentUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const [syncMessage, setSyncMessage] = useState("");
  const [syncModalVisible, setSyncModalVisible] = useState(false);
  const [salesFilter, setSalesFilter] = useState("Both");
  const [showSalesFilterMenu, setShowSalesFilterMenu] = useState(false);
  const [metrics, setMetrics] = useState({
    totalSalesToday: 0,
    totalTransactions: 0,
    growthRate: 0,
    lowStockItems: 0
  });
  const [salesOverTimeData, setSalesOverTimeData] = useState([]);
  const [salesByCategoryData, setSalesByCategoryData] = useState([]);
  const [topSellingProducts, setTopSellingProducts] = useState([]);
  const [teamOnline, setTeamOnline] = useState([]);
  const toastTranslate = useRef(new Animated.Value(-60)).current;
  const toastOpacity = useRef(new Animated.Value(0)).current;
  const timeframeApiMap = {
    daily: "daily",
    weekly: "weekly",
    monthly: "monthly",
    yearly: "yearly"
  };
  const fetchAll = async (tf = timeframe, showLoader = true) => {
    try {
      if (showLoader) setLoading(true);
      const apiTf = timeframeApiMap[tf] || "daily";
      const [statsRes, sotRes, catRes, topRes, onlineRes] = await Promise.allSettled([
        transactionAPI.getDashboardStats(apiTf),
        transactionAPI.getSalesOverTime(apiTf),
        transactionAPI.getSalesByCategory(),
        transactionAPI.getTopSelling({ sort: "most", limit: 10, period: apiTf }),
        employeeAPI.getOnline()
      ]);
      if (statsRes.status === "fulfilled" && statsRes.value?.success) {
        const d = statsRes.value.data;
        setMetrics({
          totalSalesToday: d.totalSalesToday || 0,
          totalTransactions: d.totalTransactions || 0,
          growthRate: d.growthRate || 0,
          lowStockItems: d.lowStockItems || 0
        });
      }
      if (sotRes.status === "fulfilled" && sotRes.value?.success) {
        const mapped = (sotRes.value.data || []).map((item) => ({
          ...item,
          revenue: item.revenue ?? item.totalSales ?? 0
        }));
        setSalesOverTimeData(mapped);
      }
      if (catRes.status === "fulfilled" && catRes.value?.success) {
```

```
        setSalesByCategoryData(catRes.value.data || []);
      }
      if (topRes.status === "fulfilled" && topRes.value?.success) {
        setTopSellingProducts(topRes.value.data || []);
      }
      if (onlineRes.status === "fulfilled" && onlineRes.value?.success) {
        setTeamOnline(onlineRes.value.data || []);
      } else {
        setTeamOnline([]);
      }
    } catch (err) {
      console.warn("Dashboard fetch error:", err?.message);
    } finally {
      if (showLoader) setLoading(false);
    }
  };
  const onRefresh = async () => {
    setRefreshing(true);
    setSyncMessage("Syncing...");
    setSyncModalVisible(true);
    try {
      await fetchAll(timeframe, false);
      setSyncMessage("Sync complete");
    } catch {
      setSyncMessage("Sync failed");
    } finally {
      setRefreshing(false);
      setTimeout(() => setSyncModalVisible(false), 1200);
    }
  };
  useEffect(() => {
    if (syncModalVisible) {
      Animated.parallel([
        Animated.timing(toastTranslate, { toValue: 0, duration: 280, useNativeDriver: true }),
        Animated.timing(toastOpacity, { toValue: 1, duration: 280, useNativeDriver: true })
      ]).start();
    } else {
      Animated.parallel([
        Animated.timing(toastTranslate, { toValue: -20, duration: 220, useNativeDriver: true }),
        Animated.timing(toastOpacity, { toValue: 0, duration: 220, useNativeDriver: true })
      ]).start();
    }
  }, [syncModalVisible]);
  useFocusEffect(
    useCallback(() => {
      const load = async () => {
        const stored = await AsyncStorage.getItem("currentEmployee");
        if (stored) setCurrentUser(JSON.parse(stored));
        await fetchAll(timeframe);
      };
      load();
    }, [timeframe])
  );
  const welcomeName = currentUser?.firstName || currentUser?.name || currentUser?.email?.split("@")[0] ||
"Owner";
  const formatCurrency = (v) => `\u20B1${(v || 0).toLocaleString("en-US", { minimumFractionDigits: 2,
maximumFractionDigits: 2 })}`;
  const formatCurrencyShort = (v) => {
    if (v >= 1e6) return `\u20B1${(v / 1e6).toFixed(1)}M`;
    if (v >= 1e3) return `\u20B1${(v / 1e3).toFixed(1)}K`;
    return `\u20B1${v}`;
  };
  const TAB_LABELS = [
    { label: "Today", value: "daily" },
    { label: "Week", value: "weekly" },
    { label: "Month", value: "monthly" },
    { label: "Year", value: "yearly" }
  ];
  const chartLabels = salesOverTimeData.slice(-7).map((d, i) => {
    if (timeframe === "weekly") return `W${i + 1}`;
    const p = d.period || "";
    const parts = p.split(" ");
    return parts[parts.length - 1] || p;
  });
  const chartRevenue = salesOverTimeData.slice(-7).map((d) => d.revenue || 0);
  const hasChartData = chartRevenue.length > 0 && chartRevenue.some((v) => v > 0);
  const barData = {
```

```
    labels: chartLabels.length > 0 ? chartLabels : ["No Data"],
    datasets: [{ data: chartRevenue.length > 0 ? chartRevenue : [0] }]
  };
  const chartConfig = {
    backgroundColor: "#fff",
    backgroundGradientFrom: "#fff",
    backgroundGradientTo: "#fff",
    decimalPlaces: 0,
    color: (opacity = 1) => `rgba(136, 132, 216, ${opacity})`,
    labelColor: () => "#6b7280",
    style: { borderRadius: 12 },
    propsForDots: { r: "3", strokeWidth: "2", stroke: "#fff" },
    barPercentage: 0.6,
    fillShadowGradient: "#8884d8",
    fillShadowGradientOpacity: 1
  };
  const KPI_CARDS = [
    {
      label: "Total Sales",
      desc: "Total revenue from all transactions",
      value: formatCurrencyShort(metrics.totalSalesToday),
      labelColor: "#2563EB"
    },
    {
      label: "Total Transactions",
      desc: "Number of sales made today",
      value: metrics.totalTransactions,
      labelColor: "#7C3AED"
    },
    {
      label: "Average Growth Rate",
      desc: "Sales performance change (%).",
      value: `${metrics.growthRate ? Number(metrics.growthRate).toFixed(1) : "0"}%`,
      labelColor: "#16a34a"
    },
    {
      label: "Low Stock Items",
      desc: "Number of products below stock threshold",
      value: metrics.lowStockItems,
      labelColor: "#DC2626"
    }
  ];
  return /* @__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */
React.createElement(Header, null), /* @__PURE__ */ React.createElement(
    Animated.View,
    {
      pointerEvents: syncModalVisible ? "auto" : "none",
      style: [
        styles.toastContainer,
        { transform: [{ translateY: toastTranslate }], opacity: toastOpacity }
      ]
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.toastInner }, /* @__PURE__ */
React.createElement(Text, { style: styles.toastText }, syncMessage), /* @__PURE__ */
React.createElement(TouchableOpacity, { onPress: () => setSyncModalVisible(false), style: styles.toastClose },
/* @__PURE__ */ React.createElement(Text, { style: styles.toastCloseText }, "\u2715")))
  ), /* @__PURE__ */ React.createElement(View, { style: styles.whitecard }, /* @__PURE__ */
React.createElement(
    ScrollView,
    {
      style: styles.scroll,
      showsVerticalScrollIndicator: false,
      refreshControl: /* @__PURE__ */ React.createElement(RefreshControl, { refreshing, onRefresh, colors:
["#AD7F65"], tintColor: "#AD7F65" })
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.welcomeCard }, /* @__PURE__ */
React.createElement(
      Image,
      {
        source: currentUser?.profileImage || currentUser?.image ? { uri: currentUser.profileImage ||
currentUser.image } : require("../(tabs)/iconz/default.jpeg"),
        style: styles.profileImage
      }
    ), /* @__PURE__ */ React.createElement(View, { style: { flex: 1 } }, /* @__PURE__ */
React.createElement(Text, { style: styles.welcomeText }, "Hi, ", welcomeName, "!"), /* @__PURE__ */
React.createElement(Text, { style: styles.welcomeSub }, "Here's your overview for today"))),
    /* @__PURE__ */ React.createElement(View, { style: styles.tabRow }, TAB_LABELS.map((t) => /* @__PURE__ */
```

```
React.createElement(
      TouchableOpacity,
      {
        key: t.value,
        style: [styles.tab, timeframe === t.value && styles.tabActive],
        onPress: () => setTimeframe(t.value)
      },
      /* @__PURE__ */ React.createElement(Text, { style: [styles.tabText, timeframe === t.value &&
styles.tabTextActive] }, t.label)
    ))),
    /* @__PURE__ */ React.createElement(View, { style: styles.onlineSection }, /* @__PURE__ */
React.createElement(View, { style: styles.onlineHeader }, /* @__PURE__ */ React.createElement(Text, { style:
styles.onlineTitle }, "Online now"), teamOnline.length > 0 ? /* @__PURE__ */ React.createElement(View,
{ style: styles.onlineCountPill }, /* @__PURE__ */ React.createElement(View, { style: styles.onlineDot }), /*
@__PURE__ */ React.createElement(Text, { style: styles.onlineCountText }, teamOnline.length)) : null),
teamOnline.length === 0 ? /* @__PURE__ */ React.createElement(Text, { style: styles.onlineEmpty }, "No active
employees online right now.") : /* @__PURE__ */ React.createElement(
      ScrollView,
      {
        horizontal: true,
        showsHorizontalScrollIndicator: false,
        contentContainerStyle: styles.onlineScroll
      },
      teamOnline.map((member) => {
        const displayName = member.name || member.firstName || "Member";
        const shortName = displayName.split(/\s+/)[0];
        const ini = initialsFromName(displayName);
        const bg = avatarBgForName(displayName);
        return /* @__PURE__ */ React.createElement(View, { key: String(member._id || member.id), style:
styles.onlinePerson }, /* @__PURE__ */ React.createElement(View, { style: styles.onlineAvatarWrap }, /*
@__PURE__ */ React.createElement(View, { style: [styles.onlineAvatar, { backgroundColor: bg }] }, /*
@__PURE__ */ React.createElement(Text, { style: styles.onlineAvatarText }, ini)), /* @__PURE__ */
React.createElement(View, { style: styles.onlineStatusDot })), /* @__PURE__ */ React.createElement(Text,
{ style: styles.onlineName, numberOfLines: 1 }, shortName));
      })
    )),
    loading ? /* @__PURE__ */ React.createElement(View, { style: styles.loadingBox }, /* @__PURE__ */
React.createElement(ActivityIndicator, { size: "large", color: "#AD7F65" }), /* @__PURE__ */
React.createElement(Text, { style: styles.loadingText }, "Loading...")) : /* @__PURE__ */
React.createElement(View, { style: styles.kpiGrid }, /* @__PURE__ */ React.createElement(View, { style:
styles.kpiRow }, KPI_CARDS.slice(0, 2).map((card, idx) => /* @__PURE__ */ React.createElement(View, { key:
idx, style: styles.kpiCard }, /* @__PURE__ */ React.createElement(Text, { style: styles.kpiValue,
numberOfLines: 1, adjustsFontSizeToFit: true }, card.value), /* @__PURE__ */ React.createElement(Text,
{ style: [styles.kpiLabel, { color: card.labelColor }] }, card.label), /* @__PURE__ */
React.createElement(Text, { style: styles.kpiDesc, numberOfLines: 2 }, card.desc)))), /* @__PURE__ */
React.createElement(View, { style: styles.kpiRow }, KPI_CARDS.slice(2, 4).map((card, idx) => /* @__PURE__ */
React.createElement(View, { key: idx, style: styles.kpiCard }, /* @__PURE__ */ React.createElement(Text,
{ style: styles.kpiValue, numberOfLines: 1, adjustsFontSizeToFit: true }, card.value), /* @__PURE__ */
React.createElement(Text, { style: [styles.kpiLabel, { color: card.labelColor }] }, card.label), /* @__PURE__
*/ React.createElement(Text, { style: styles.kpiDesc, numberOfLines: 2 }, card.desc))))),
    /* @__PURE__ */ React.createElement(View, { style: styles.sectionHeader }, /* @__PURE__ */
React.createElement(Text, { style: styles.sectionTitle }, "Top Selling Products"), /* @__PURE__ */
React.createElement(View, { style: styles.sortBadge }, /* @__PURE__ */ React.createElement(Text, { style:
styles.sortBadgeText }, "Sort By \u25BE"))),
    /* @__PURE__ */ React.createElement(
      FlatList,
      {
        horizontal: true,
        data: topSellingProducts.slice(0, 6),
        keyExtractor: (item, i) => item.productId || String(i),
        showsHorizontalScrollIndicator: false,
        contentContainerStyle: { paddingBottom: 8, paddingTop: 4, paddingLeft: 2 },
        ListEmptyComponent: /* @__PURE__ */ React.createElement(Text, { style: { color: "#aaa", fontSize: 13,
paddingVertical: 16 } }, "No data available"),
        renderItem: ({ item, index }) => /* @__PURE__ */ React.createElement(View, { style:
styles.topProductCard }, /* @__PURE__ */ React.createElement(View, { style: styles.rankBadge }, /* @__PURE__
*/ React.createElement(Text, { style: styles.rankText }, index + 1)), /* @__PURE__ */
React.createElement(View, { style: styles.productImageWrap }, item.itemImage ? /* @__PURE__ */
React.createElement(Image, { source: { uri: item.itemImage }, style: styles.productImage }) : /* @__PURE__ */
React.createElement(View, { style: [styles.productImage, { backgroundColor: "#eee", justifyContent: "center",
alignItems: "center" }] }, /* @__PURE__ */ React.createElement(Text, { style: { fontSize: 9, color:
"#999" } }, "IMG"))), /* @__PURE__ */ React.createElement(Text, { style: styles.productName, numberOfLines:
2 }, item.itemName), /* @__PURE__ */ React.createElement(Text, { style: styles.productSold },
item.totalQuantitySold, " sold"))
      }
    ),
    /* @__PURE__ */ React.createElement(View, { style: [styles.sectionHeader, { marginTop: 8 }] }, /*
```

```
@__PURE__ */ React.createElement(View, null, /* @__PURE__ */ React.createElement(Text, { style:
styles.sectionTitle }, "Sales Over Time"), /* @__PURE__ */ React.createElement(Text, { style:
styles.sectionSub }, "Total sales from all transactions during a specific period."))),
    /* @__PURE__ */ React.createElement(View, { style: styles.chartCard }, hasChartData ? /* @__PURE__ */
React.createElement(
      BarChart,
      {
        data: barData,
        width: SCREEN_WIDTH - 56,
        height: 180,
        chartConfig,
        style: { borderRadius: 10, marginLeft: -8 },
        withInnerLines: false,
        withVerticalLines: false,
        showBarTops: false,
        fromZero: true,
        yAxisLabel: "",
        yAxisSuffix: "",
        formatYLabel: (v) => formatCurrencyShort(Number(v))
      }
    ) : /* @__PURE__ */ React.createElement(View, { style: styles.emptyChart }, /* @__PURE__ */
React.createElement(Text, { style: styles.emptyChartText }, "No sales data for this period"))),
    /* @__PURE__ */ React.createElement(View, { style: [styles.sectionHeader, { marginTop: 8 }] }, /*
@__PURE__ */ React.createElement(View, null, /* @__PURE__ */ React.createElement(Text, { style:
styles.sectionTitle }, "Sales By Category"), /* @__PURE__ */ React.createElement(Text, { style:
styles.sectionSub }, "Revenue distribution across product categories"))),
    /* @__PURE__ */ React.createElement(View, { style: styles.chartCard }, salesByCategoryData.length > 0 ?
/* @__PURE__ */ React.createElement(SimplePieChart, { data: salesByCategoryData }) : /* @__PURE__ */
React.createElement(View, { style: styles.emptyChart }, /* @__PURE__ */ React.createElement(Text, { style:
styles.emptyChartText }, "No category data available"))),
    /* @__PURE__ */ React.createElement(View, { style: { height: 32 } })
  )));
}
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#53321c" },
  whitecard: {
    flex: 1,
    backgroundColor: "#fff",
    marginTop: -10,
    borderTopLeftRadius: 15,
    borderTopRightRadius: 15,
    paddingTop: 4
  },
  scroll: { paddingHorizontal: 16 },
  // Toast
  toastContainer: {
    position: "absolute",
    top: 50,
    left: 16,
    right: 16,
    zIndex: 1e3,
    borderRadius: 14
  },
  toastInner: {
    flexDirection: "row",
    alignItems: "center",
    backgroundColor: "#53321c",
    paddingVertical: 14,
    paddingHorizontal: 16,
    borderRadius: 14
  },
  toastText: { color: "#fff", fontSize: 14, fontWeight: "500", flex: 1, marginLeft: 8 },
  toastClose: { padding: 4 },
  toastCloseText: { color: "rgba(255,255,255,0.8)", fontSize: 16, fontWeight: "600" },
  // Welcome
  welcomeCard: {
    flexDirection: "row",
    alignItems: "center",
    paddingTop: 16,
    paddingBottom: 12
  },
  profileImage: {
    width: 56,
    height: 56,
    borderRadius: 28,
    marginRight: 14,
    borderWidth: 2,
```

```
  borderColor: "#AD7F65"
},
welcomeText: { fontSize: 20, fontWeight: "800", color: "#1a1a1a" },
welcomeSub: { fontSize: 13, color: "#666", marginTop: 2 },
// Tabs
tabRow: {
  flexDirection: "row",
  backgroundColor: "#f3f4f6",
  borderRadius: 10,
  padding: 3,
  marginBottom: 16
},
tab: {
  flex: 1,
  paddingVertical: 7,
  alignItems: "center",
  borderRadius: 8
},
tabActive: { backgroundColor: "#1f1f1f" },
tabText: { fontSize: 13, fontWeight: "600", color: "#6b7280" },
tabTextActive: { color: "#fff" },
// Online team
onlineSection: {
  marginBottom: 14,
  paddingVertical: 12,
  paddingHorizontal: 12,
  backgroundColor: "#fafafa",
  borderRadius: 12,
  borderWidth: 1,
  borderColor: "#f0f0f0"
},
onlineHeader: {
  flexDirection: "row",
  alignItems: "center",
  justifyContent: "space-between",
  marginBottom: 10
},
onlineTitle: { fontSize: 13, fontWeight: "700", color: "#1a1a1a" },
onlineCountPill: {
  flexDirection: "row",
  alignItems: "center",
  backgroundColor: "#fff",
  paddingHorizontal: 8,
  paddingVertical: 4,
  borderRadius: 20,
  borderWidth: 1,
  borderColor: "#e5e7eb"
},
onlineDot: {
  width: 6,
  height: 6,
  borderRadius: 3,
  backgroundColor: "#22c55e",
  marginRight: 6
},
onlineCountText: { fontSize: 12, fontWeight: "700", color: "#374151" },
onlineEmpty: { fontSize: 12, color: "#9ca3af", lineHeight: 17 },
onlineScroll: { flexDirection: "row", alignItems: "flex-start", paddingRight: 4 },
onlinePerson: { alignItems: "center", marginRight: 14, maxWidth: 72 },
onlineAvatarWrap: { position: "relative", marginBottom: 4 },
onlineAvatar: {
  width: 44,
  height: 44,
  borderRadius: 22,
  alignItems: "center",
  justifyContent: "center",
  borderWidth: 2,
  borderColor: "#fff"
},
onlineAvatarText: { fontSize: 14, fontWeight: "800", color: "#53321c" },
onlineStatusDot: {
  position: "absolute",
  right: -1,
  bottom: -1,
  width: 11,
  height: 11,
  borderRadius: 6,
```

```
  backgroundColor: "#22c55e",
  borderWidth: 2,
  borderColor: "#fafafa"
},
onlineName: { fontSize: 11, fontWeight: "600", color: "#6b7280", textAlign: "center", width: 72 },
// Loading
loadingBox: { padding: 40, alignItems: "center", justifyContent: "center" },
loadingText: { marginTop: 12, color: "#666", fontSize: 14 },
// KPI Grid
kpiGrid: {
  marginBottom: 20
},
kpiRow: {
  flexDirection: "row",
  justifyContent: "space-between",
  marginBottom: 10
},
kpiCard: {
  width: (SCREEN_WIDTH - 52) / 2,
  backgroundColor: "#fff",
  borderRadius: 12,
  padding: 16,
  flexDirection: "column",
  borderWidth: 1,
  borderColor: "#e5e7eb",
  shadowColor: "#000",
  shadowOpacity: 0.04,
  shadowRadius: 4,
  shadowOffset: { width: 0, height: 2 },
  elevation: 2
},
kpiValue: { fontSize: 26, fontWeight: "800", color: "#1a1a1a", marginBottom: 4, letterSpacing: -0.5 },
kpiLabel: { fontSize: 13, fontWeight: "700", marginBottom: 6 },
kpiDesc: { fontSize: 9, color: "#9ca3af", lineHeight: 12 },
// Section headers
sectionHeader: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "flex-start",
  marginBottom: 10
},
sectionTitle: { fontSize: 15, fontWeight: "800", color: "#1a1a1a" },
sectionSub: { fontSize: 11, color: "#9ca3af", marginTop: 2, maxWidth: "90%" },
sortBadge: {
  borderWidth: 1,
  borderColor: "#e5e7eb",
  borderRadius: 8,
  paddingHorizontal: 8,
  paddingVertical: 4
},
sortBadgeText: { fontSize: 12, color: "#6b7280" },
// Top selling
topProductCard: {
  width: 100,
  marginRight: 10,
  backgroundColor: "#f9fafb",
  borderRadius: 12,
  padding: 8,
  alignItems: "center",
  borderWidth: 1,
  borderColor: "#f0f0f0"
},
rankBadge: {
  position: "absolute",
  top: 6,
  left: 6,
  backgroundColor: "#AD7F65",
  borderRadius: 10,
  width: 20,
  height: 20,
  alignItems: "center",
  justifyContent: "center",
  zIndex: 1
},
rankText: { color: "#fff", fontSize: 10, fontWeight: "800" },
productImageWrap: {
  width: 76,
```

```
    height: 76,
    borderRadius: 10,
    overflow: "hidden",
    marginBottom: 6,
    backgroundColor: "#e5e7eb"
  },
  productImage: { width: "100%", height: "100%", resizeMode: "cover" },
  productName: { fontSize: 10, fontWeight: "700", color: "#1a1a1a", textAlign: "center", marginBottom: 2 },
  productSold: { fontSize: 10, color: "#AD7F65", fontWeight: "600" },
  // Chart
  chartCard: {
    backgroundColor: "#fff",
    borderRadius: 14,
    padding: 16,
    marginBottom: 16,
    borderWidth: 1,
    borderColor: "#f0f0f0",
    shadowColor: "#000",
    shadowOpacity: 0.04,
    shadowRadius: 6,
    shadowOffset: { width: 0, height: 2 },
    elevation: 2
  },
  emptyChart: {
    height: 120,
    alignItems: "center",
    justifyContent: "center"
  },
  emptyChartText: { color: "#9ca3af", fontSize: 13 },
  // Filter dropdown
  filterBtn: {
    borderWidth: 1,
    borderColor: "#e5e7eb",
    borderRadius: 20,
    paddingHorizontal: 10,
    paddingVertical: 5
  },
  filterBtnText: { fontSize: 11, color: "#6b7280" },
  filterMenu: {
    position: "absolute",
    right: 0,
    top: 32,
    backgroundColor: "#fff",
    borderRadius: 10,
    borderWidth: 1,
    borderColor: "#e5e7eb",
    shadowColor: "#000",
    shadowOpacity: 0.12,
    shadowRadius: 8,
    elevation: 5,
    zIndex: 100,
    minWidth: 90
  },
  filterMenuItem: { paddingHorizontal: 14, paddingVertical: 10 },
  filterMenuText: { fontSize: 13, color: "#374151" },
  // Chart section divider (used in Both mode)
  chartDivider: {
    flexDirection: "row",
    alignItems: "center",
    marginTop: 8,
    marginBottom: 4
  },
  chartDividerText: {
    fontSize: 12,
    fontWeight: "700",
    color: "#6b7280"
  }
});


FILE: pos/mobile/app/(tabs)/inventory.jsx
================================================================================
import AddItem from "@/app/AddItem";
import StockInModal from "@/components/inventory/StockInModal";
import StockOutModal from "@/components/inventory/StockOutModal";
import Header from "@/components/shared/header";
```

```
import { useData } from "@/context/DataContext";
import { Ionicons } from "@expo/vector-icons";
import AsyncStorage from "@react-native-async-storage/async-storage";
import { useFocusEffect } from "@react-navigation/native";
import { useRouter } from "expo-router";
import * as ScreenOrientation from "expo-screen-orientation";
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import {
  ActivityIndicator,
  Alert,
  AppState,
  FlatList,
  Platform,
  RefreshControl,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View
} from "react-native";
import { SafeAreaView } from "react-native-safe-area-context";
const InventoryTable = ({
  items = [],
  onBack,
  onEditItem,
  onRefresh,
  loading,
  onEndReached,
  loadingMore
}) => {
  const [menuVisible, setMenuVisible] = useState(null);
  const [searchQuery, setSearchQuery] = useState("");
  const [showStockIn, setShowStockIn] = useState(false);
  const [showStockOut, setShowStockOut] = useState(false);
  const [stockItem, setStockItem] = useState(null);
  const [stockLoading, setStockLoading] = useState(false);
  useEffect(() => {
    let mounted = true;
    (async () => {
      try {
        if (ScreenOrientation && typeof ScreenOrientation.lockAsync === "function") {
          await ScreenOrientation.lockAsync(
            ScreenOrientation.OrientationLock.LANDSCAPE
          );
        }
      } catch (e) {
        console.warn("Failed to lock orientation:", e?.message || e);
      }
    })();
    return () => {
      mounted = false;
      (async () => {
        try {
          if (ScreenOrientation && typeof ScreenOrientation.lockAsync === "function") {
            await ScreenOrientation.lockAsync(
              ScreenOrientation.OrientationLock.PORTRAIT
            );
          }
        } catch (e) {
          console.warn("Failed to unlock orientation:", e?.message || e);
        }
      })();
    };
  }, []);
  useEffect(() => {
    if (Platform.OS === "web") return;
    const modalOpen = showStockIn || showStockOut;
    (async () => {
      try {
        if (ScreenOrientation && typeof ScreenOrientation.lockAsync === "function") {
          await ScreenOrientation.lockAsync(
            modalOpen ? ScreenOrientation.OrientationLock.PORTRAIT :
ScreenOrientation.OrientationLock.LANDSCAPE
          );
        }
      } catch (e) {
```

```
        console.warn("Stock modal orientation:", e?.message || e);
      }
    })();
  }, [showStockIn, showStockOut]);
  const filteredItems = useMemo(() => {
    const q = searchQuery.trim().toLowerCase();
    if (!q) return items;
    return items.filter((row) => {
      const sku = String(row.sku ?? "").toLowerCase();
      const name = String(row.name ?? row.itemName ?? "").toLowerCase();
      const brand = String(row.brand ?? row.brandName ?? "").toLowerCase();
      const category = String(row.category ?? "").toLowerCase();
      return sku.includes(q) || name.includes(q) || brand.includes(q) || category.includes(q);
    });
  }, [items, searchQuery]);
  const menuItems = [
    {
      value: "edit",
      icon: "pencil",
      iconColor: "#4a90e2",
      bgColor: "rgba(74, 144, 226, 0.1)"
    },
    {
      value: "stockIn",
      icon: "add-circle",
      iconColor: "#2ecc71",
      bgColor: "rgba(46, 204, 113, 0.1)"
    },
    {
      value: "stockOut",
      icon: "remove-circle",
      iconColor: "#e67e22",
      bgColor: "rgba(230, 126, 34, 0.1)"
    },
    {
      value: "archive",
      icon: "archive",
      iconColor: "#9b59b6",
      bgColor: "rgba(155, 89, 182, 0.1)"
    }
  ];
  const toggleMenu = (itemId) => {
    setMenuVisible(menuVisible === itemId ? null : itemId);
  };
  const handleMenuAction = (action, itemId) => {
    setMenuVisible(null);
    handleAction(action, itemId);
  };
  const renderItem = ({ item = {} }) => {
    const itemId = item._id || item.id || "";
    const sku = item.sku || "N/A";
    const name = item.name || "No Name";
    const brand = item.brand || "N/A";
    const category = item.category || "N/A";
    const price = item.price || 0;
    const stock = item.stock || item.quantity || 0;
    const dateAdded = item.dateAdded || item.createdAt ? new Date(item.dateAdded ||
item.createdAt).toLocaleDateString() : "N/A";
    const isMenuVisible = menuVisible === itemId;
    return /* @__PURE__ */ React.createElement(View, { style: styles.tableRow }, /* @__PURE__ */
React.createElement(Text, { style: [styles.tableCell, { flex: 0.7 }] }, sku), /* @__PURE__ */
React.createElement(Text, { style: [styles.tableCell, { flex: 1.3 }] }, name), /* @__PURE__ */
React.createElement(Text, { style: [styles.tableCell, { flex: 0.9 }] }, brand), /* @__PURE__ */
React.createElement(Text, { style: [styles.tableCell, { flex: 0.9 }] }, category), /* @__PURE__ */
React.createElement(Text, { style: [styles.tableCell, { flex: 0.7 }] }, "\u20B1", typeof price === "number" ?
price.toFixed(2) : "0.00"), /* @__PURE__ */ React.createElement(
      Text,
      {
        style: [
          styles.tableCell,
          { flex: 0.4, color: (stock || 0) < 4 ? "#E74C3C" : "#000" }
        ]
      },
      stock
    ), /* @__PURE__ */ React.createElement(Text, { style: [styles.tableCell, { flex: 0.7 }] }, dateAdded), /*
@__PURE__ */ React.createElement(View, { style: [styles.tableCell, { flex: 0.8 }] }, /* @__PURE__ */
React.createElement(View, { style: styles.actionButtonContainer }, /* @__PURE__ */ React.createElement(
```

```
      TouchableOpacity,
      {
        onPress: () => toggleMenu(itemId),
        style: styles.menuButton
      },
      /* @__PURE__ */ React.createElement(Ionicons, { name: "ellipsis-vertical", size: 20, color: "#666" })
    ), isMenuVisible && /* @__PURE__ */ React.createElement(View, { style: [styles.menuContainer, { right: 10,
top: 30 }] }, menuItems.map((menuItem) => /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        key: menuItem.value,
        style: [
          styles.menuItem,
          { backgroundColor: menuItem.bgColor }
        ],
        onPress: () => handleMenuAction(menuItem.value, itemId)
      },
      /* @__PURE__ */ React.createElement(
        Ionicons,
        {
          name: menuItem.icon,
          size: 20,
          color: menuItem.iconColor,
          style: styles.menuIcon
        }
      )
    ))))));
  };
  const handleAction = async (action, itemId) => {
    const item = items.find((item2) => (item2._id || item2.id) === itemId);
    if (!item) return;
    if (action === "edit") {
      onEditItem(item);
      return;
    }
    if (action === "stockIn") {
      setStockItem(item);
      setShowStockIn(true);
      return;
    }
    if (action === "stockOut") {
      setStockItem(item);
      setShowStockOut(true);
      return;
    }
    if (action === "archive") {
      Alert.alert(
        "Archive Item",
        "Are you sure you want to archive this item?",
        [
          {
            text: "Cancel",
            style: "cancel"
          },
          {
            text: "Archive",
            style: "destructive",
            onPress: async () => {
              try {
                const { archiveAPI } = require("../../services/api");
                await archiveAPI.archive({
                  productId: item._id || item.id,
                  productName: item.name,
                  reason: "Archived from mobile app"
                });
                Alert.alert("Success", "Item archived successfully");
                onRefresh();
              } catch (error) {
                Alert.alert(
                  "Error",
                  "Failed to archive item: " + error.message
                );
              }
            }
          }
        ],
        { cancelable: true }
```

```
      );
    }
  };
  const handleStockInConfirm = async (stockData) => {
    if (!stockItem) return;
    try {
      setStockLoading(true);
      const { productAPI } = require("../../services/api");
      const storedEmployeeStr = await AsyncStorage.getItem("currentEmployee");
      let handledBy = "Mobile User";
      let handledById = "mobile_user_id";
      if (storedEmployeeStr) {
        try {
          const parsedUser = JSON.parse(storedEmployeeStr);
          handledBy = parsedUser.name || `${parsedUser.firstName || ""} ${parsedUser.lastName || ""}`.trim()
|| "Mobile User";
          handledById = parsedUser._id || parsedUser.id || "mobile_user_id";
        } catch (e) {
          console.error("Failed to parse current employee", e);
        }
      }
      const id = stockItem._id || stockItem.id;
      await productAPI.stockIn(id, {
        ...stockData,
        handledBy,
        handledById
      });
      Alert.alert("Success", "Stock added successfully");
      setShowStockIn(false);
      setStockItem(null);
      onRefresh(true);
    } catch (error) {
      console.error("Error adding stock:", error);
      Alert.alert("Error", "Failed to update stock: " + error.message);
    } finally {
      setStockLoading(false);
    }
  };
  const handleStockOutConfirm = async (stockData) => {
    if (!stockItem) return;
    try {
      setStockLoading(true);
      const { productAPI, archiveAPI } = require("../../services/api");
      let totalQuantityRemoved = 0;
      if (stockData.noSizes) {
        totalQuantityRemoved = stockData.quantity || 0;
      } else if (stockData.hasVariants && stockData.variantQuantities) {
        stockData.selectedSizes?.forEach((size) => {
          const variantQtys = stockData.variantQuantities[size] || {};
          totalQuantityRemoved += Object.values(variantQtys).reduce(
            (sum, q) => sum + (parseInt(q, 10) || 0),
            0
          );
        });
      } else {
        stockData.selectedSizes?.forEach((size) => {
          totalQuantityRemoved += parseInt(stockData.sizes?.[size], 10) || 0;
        });
      }
      const storedEmployeeStr = await AsyncStorage.getItem("currentEmployee");
      let handledBy = "Mobile User";
      let handledById = "mobile_user_id";
      if (storedEmployeeStr) {
        try {
          const parsedUser = JSON.parse(storedEmployeeStr);
          handledBy = parsedUser.name || `${parsedUser.firstName || ""} ${parsedUser.lastName || ""}`.trim()
|| "Mobile User";
          handledById = parsedUser._id || parsedUser.id || "mobile_user_id";
        } catch (e) {
          console.error("Failed to parse current employee", e);
        }
      }
      const id = stockItem._id || stockItem.id;
      await productAPI.stockOut(id, {
        ...stockData,
        handledBy,
        handledById
```

```
      });
      const archiveReasons = ["Damaged", "Defective", "Expired"];
      if (archiveReasons.includes(stockData.reason)) {
        const sizesString = stockData.selectedSizes ? stockData.selectedSizes.join(", ") : "";
        const itemName = stockItem.itemName || stockItem.name || "";
        const brandName = stockItem.brandName || stockItem.brand || "";
        try {
          await archiveAPI.archive({
            productId: id,
            itemName,
            sku: stockItem.sku,
            variant: stockItem.variant || "",
            selectedSize: sizesString,
            category: stockItem.category,
            brandName,
            itemPrice: stockItem.itemPrice ?? stockItem.price ?? 0,
            costPrice: stockItem.costPrice ?? 0,
            quantity: totalQuantityRemoved,
            itemImage: stockItem.itemImage || "",
            reason: stockData.reason === "Defective" ? "Defective" : stockData.reason,
            archivedBy: handledBy,
            archivedById: handledById,
            source: "stock-out",
            notes: `Stock out - ${stockData.reason}. Sizes: ${sizesString}`
          });
        } catch (archiveError) {
          console.error("Error archiving item:", archiveError);
        }
      }
      Alert.alert("Success", "Stock removed successfully");
      setShowStockOut(false);
      setStockItem(null);
      onRefresh(true);
    } catch (error) {
      console.error("Error removing stock:", error);
      Alert.alert("Error", "Failed to update stock: " + error.message);
    } finally {
      setStockLoading(false);
    }
  };
  return /* @__PURE__ */ React.createElement(SafeAreaView, { style: styles.tableContainer }, /* @__PURE__ */
React.createElement(View, { style: styles.tableHeader }, /* @__PURE__ */ React.createElement(View, { style:
styles.headerLeft }), /* @__PURE__ */ React.createElement(Text, { style: styles.tableTitle }, "Inventory
Table"), /* @__PURE__ */ React.createElement(TouchableOpacity, { onPress: onBack, style: styles.closeButton },
/* @__PURE__ */ React.createElement(Ionicons, { name: "close", size: 24, color: "#fff" }))), /* @__PURE__ */
React.createElement(View, { style: styles.tableSearchWrap }, /* @__PURE__ */ React.createElement(
    Ionicons,
    {
      name: "search",
      size: 20,
      color: "#6b7280",
      style: styles.tableSearchIcon
    }
  ), /* @__PURE__ */ React.createElement(
    TextInput,
    {
      style: styles.tableSearchInput,
      placeholder: "Search by SKU, name, brand, or category\u2026",
      placeholderTextColor: "#9ca3af",
      value: searchQuery,
      onChangeText: setSearchQuery,
      autoCorrect: false,
      autoCapitalize: "none",
      clearButtonMode: "while-editing"
    }
  ), searchQuery.length > 0 ? /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      onPress: () => setSearchQuery(""),
      style: styles.tableSearchClear,
      hitSlop: { top: 8, bottom: 8, left: 8, right: 8 }
    },
    /* @__PURE__ */ React.createElement(Ionicons, { name: "close-circle", size: 20, color: "#9ca3af" })
  ) : null), /* @__PURE__ */ React.createElement(View, { style: styles.tableHeaderRow }, /* @__PURE__ */
React.createElement(Text, { style: [styles.tableHeaderCell, { flex: 0.7 }] }, "SKU"), /* @__PURE__ */
React.createElement(Text, { style: [styles.tableHeaderCell, { flex: 1.3 }] }, "Item Name"), /* @__PURE__ */
React.createElement(Text, { style: [styles.tableHeaderCell, { flex: 0.9 }] }, "Brand"), /* @__PURE__ */
```

```
React.createElement(Text, { style: [styles.tableHeaderCell, { flex: 0.9 }] }, "Category"), /* @__PURE__ */
React.createElement(Text, { style: [styles.tableHeaderCell, { flex: 0.7 }] }, "Price"), /* @__PURE__ */
React.createElement(Text, { style: [styles.tableHeaderCell, { flex: 0.4 }] }, "Stock"), /* @__PURE__ */
React.createElement(Text, { style: [styles.tableHeaderCell, { flex: 0.7 }] }, "Date Added"), /* @__PURE__ */
React.createElement(
    Text,
    {
      style: [styles.tableHeaderCell, { flex: 0.8, textAlign: "center" }]
    },
    "Actions"
  )), loading ? /* @__PURE__ */ React.createElement(View, { style: styles.loadingContainer }, /* @__PURE__ */
React.createElement(ActivityIndicator, { size: "large", color: "#8B4513" }), /* @__PURE__ */
React.createElement(Text, { style: styles.loadingText }, "Loading products...")) : /* @__PURE__ */
React.createElement(
    FlatList,
    {
      data: filteredItems,
      renderItem,
      keyExtractor: (item) => (item?._id || item?.id || Math.random()).toString(),
      contentContainerStyle: { paddingBottom: 20 },
      onEndReached,
      onEndReachedThreshold: 0.5,
      ListEmptyComponent: !loading && items.length > 0 && filteredItems.length === 0 ? /* @__PURE__ */
React.createElement(View, { style: styles.tableSearchEmpty }, /* @__PURE__ */ React.createElement(Text,
{ style: styles.tableSearchEmptyText }, 'No products match "', searchQuery.trim(), '"')) : null,
      ListFooterComponent: loadingMore ? /* @__PURE__ */ React.createElement(View, { style: { padding: 10 } },
/* @__PURE__ */ React.createElement(ActivityIndicator, { size: "small", color: "#8B4513" })) : null,
      refreshControl: /* @__PURE__ */ React.createElement(
        RefreshControl,
        {
          refreshing: loading,
          onRefresh,
          colors: ["#8B4513"],
          tintColor: "#8B4513"
        }
      )
    }
  ), /* @__PURE__ */ React.createElement(
    StockInModal,
    {
      visible: showStockIn,
      onClose: () => {
        setShowStockIn(false);
        setStockItem(null);
      },
      product: stockItem,
      onConfirm: handleStockInConfirm,
      loading: stockLoading
    }
  ), /* @__PURE__ */ React.createElement(
    StockOutModal,
    {
      visible: showStockOut,
      onClose: () => {
        setShowStockOut(false);
        setStockItem(null);
      },
      product: stockItem,
      onConfirm: handleStockOutConfirm,
      loading: stockLoading
    }
  ));
};
const STALE_TIME = 3e4;
const MIN_FETCH_INTERVAL = 5e3;
export default function Inventory() {
  const router = useRouter();
  const {
    products: cachedProducts,
    productsLoading,
    fetchProducts: fetchCachedProducts,
    invalidateCache,
    inventoryStats,
    // Use stats from context
    calculateDashboardStats
    // To refresh stats
  } = useData();
```

```
const [showTableView, setShowTableView] = useState(false);
const [showAddItem, setShowAddItem] = useState(false);
const [refreshing, setRefreshing] = useState(false);
const [initialLoad, setInitialLoad] = useState(true);
const [page, setPage] = useState(1);
const [hasMore, setHasMore] = useState(true);
const [loadingMore, setLoadingMore] = useState(false);
const lastFetchTime = useRef(0);
const isFetching = useRef(false);
const appState = useRef(AppState.currentState);
useEffect(() => {
  return () => {
    if (Platform.OS !== "web") {
      try {
        if (ScreenOrientation && typeof ScreenOrientation.lockAsync === "function") {
          ScreenOrientation.lockAsync(ScreenOrientation.OrientationLock.PORTRAIT);
        }
      } catch (e) {
        console.warn("Failed to reset orientation:", e?.message || e);
      }
    }
  };
}, []);
useEffect(() => {
  const subscription = AppState.addEventListener("change", (nextAppState) => {
    if (appState.current.match(/inactive|background/) && nextAppState === "active") {
      const timeSinceLastFetch = Date.now() - lastFetchTime.current;
      if (timeSinceLastFetch > STALE_TIME) {
        fetchProductsOptimized(true);
      }
    }
    appState.current = nextAppState;
  });
  return () => subscription?.remove();
}, []);
const normalizeProduct = useCallback((product) => {
  const p = product || {};
  return {
    ...p,
    name: p.name || p.itemName || "No Name",
    brand: p.brand || p.brandName || "N/A",
    price: p.price ?? p.itemPrice ?? 0,
    stock: p.stock ?? p.currentStock ?? 0,
    category: p.category || "Uncategorized",
    sku: p.sku || "N/A",
    itemImage: p.itemImage || p.image || "",
    dateAdded: p.dateAdded || p.createdAt || p.updatedAt || null,
    costPrice: p.costPrice ?? 0
  };
}, []);
const products = useMemo(() => {
  return (Array.isArray(cachedProducts) ? cachedProducts : []).filter(Boolean).map(normalizeProduct);
}, [cachedProducts, normalizeProduct]);
const formatDateAdded = useCallback((date) => {
  if (!date) return "Unknown";
  const now = /* @__PURE__ */ new Date();
  const itemDate = new Date(date);
  const diffTime = Math.abs(now - itemDate);
  const diffDays = Math.floor(diffTime / (1e3 * 60 * 60 * 24));
  if (diffDays === 0) return "Today";
  if (diffDays === 1) return "Yesterday";
  if (diffDays < 7) return `${diffDays} days ago`;
  return itemDate.toLocaleDateString();
}, []);
const recentlyAddedItems = useMemo(() => {
  return [...products].sort((a, b) => {
    const dateA = new Date(a.createdAt || a.dateAdded || 0);
    const dateB = new Date(b.createdAt || b.dateAdded || 0);
    return dateB - dateA;
  }).slice(0, 5).map((p) => ({
    id: p._id || p.id,
    name: p.name,
    category: p.category,
    stock: p.stock || 0,
    dateAdded: formatDateAdded(p.createdAt || p.dateAdded)
  }));
}, [products, formatDateAdded]);
```

```
const loadingProducts = productsLoading && initialLoad;
const fetchProductsOptimized = useCallback(
  async (forceRefresh = false, loadMore = false) => {
    if (isFetching.current) {
      return;
    }
    const requestedPage = loadMore ? page + 1 : 1;
    try {
      isFetching.current = true;
      if (loadMore) setLoadingMore(true);
      if (forceRefresh) {
        invalidateCache("products");
        calculateDashboardStats(true);
      } else if (!loadMore) {
        calculateDashboardStats(false);
      }
      const response = await fetchCachedProducts(forceRefresh, {
        page: requestedPage,
        limit: 20
      });
      if (response && response.data) {
        if (loadMore) {
          setPage((prev) => prev + 1);
        } else {
          setPage(1);
        }
        if (response.totalPages) {
          setHasMore(requestedPage < response.totalPages);
        } else {
          setHasMore(response.data.length === 20);
        }
      }
      lastFetchTime.current = Date.now();
    } catch (error) {
      console.error("Error fetching products:", error);
    } finally {
      isFetching.current = false;
      setLoadingMore(false);
    }
  },
  [invalidateCache, fetchCachedProducts, calculateDashboardStats, page]
);
useFocusEffect(
  useCallback(() => {
    const loadData = async () => {
      const timeSinceLastFetch = Date.now() - lastFetchTime.current;
      const shouldFetch = cachedProducts.length === 0 || timeSinceLastFetch > STALE_TIME;
      if (shouldFetch) {
        await fetchProductsOptimized(false);
      }
      setInitialLoad(false);
    };
    loadData();
  }, [fetchProductsOptimized, cachedProducts.length])
);
const fetchProducts = useCallback(
  async (forceRefresh = false) => {
    await fetchProductsOptimized(forceRefresh);
  },
  [fetchProductsOptimized]
);
const [editingItem, setEditingItem] = useState(null);
const handleEditItem = (item) => {
  setEditingItem(item);
  setShowAddItem(true);
};
const handleAddItemBack = () => {
  setShowAddItem(false);
  setEditingItem(null);
};
const handleRefresh = useCallback(async () => {
  try {
    setRefreshing(true);
    lastFetchTime.current = 0;
    await fetchProducts(true);
  } finally {
    setRefreshing(false);
```

```
    }
  }, [fetchProducts]);
  const handleLoadMore = useCallback(() => {
    if (!isFetching.current && hasMore) {
      fetchProductsOptimized(false, true);
    }
  }, [fetchProductsOptimized, hasMore]);
  if (showAddItem) {
    return /* @__PURE__ */ React.createElement(View, { style: { flex: 1 } }, /* @__PURE__ */
React.createElement(
      AddItem,
      {
        onBack: handleAddItemBack,
        item: editingItem,
        isEditing: !!editingItem
      }
    ));
  }
  if (showTableView) {
    return /* @__PURE__ */ React.createElement(
      InventoryTable,
      {
        items: products,
        loading: loadingProducts,
        onBack: () => setShowTableView(false),
        onEditItem: handleEditItem,
        onRefresh: fetchProducts,
        onEndReached: handleLoadMore,
        loadingMore
      }
    );
  }
  return /* @__PURE__ */ React.createElement(View, { style: { flex: 1 } }, /* @__PURE__ */
React.createElement(View, { style: styles.container }, /* @__PURE__ */ React.createElement(Header, null), /*
@__PURE__ */ React.createElement(View, { style: styles.whitecard }, /* @__PURE__ */ React.createElement(
    ScrollView,
    {
      style: styles.main,
      contentContainerStyle: styles.scrollContent,
      showsVerticalScrollIndicator: false,
      refreshControl: /* @__PURE__ */ React.createElement(
        RefreshControl,
        {
          refreshing,
          onRefresh: handleRefresh,
          colors: ["#8B4513"],
          tintColor: "#8B4513"
        }
      )
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.actionButtonsContainer }, /* @__PURE__ */
React.createElement(
      TouchableOpacity,
      {
        style: [
          styles.actionButton,
          { backgroundColor: "#16a34a", flex: 1 }
        ],
        onPress: () => setShowAddItem(true)
      },
      /* @__PURE__ */ React.createElement(
        Ionicons,
        {
          name: "add",
          size: 20,
          color: "#fff",
          style: styles.buttonIcon
        }
      ),
      /* @__PURE__ */ React.createElement(Text, { style: styles.actionButtonText }, "Add Item")
    )),
    /* @__PURE__ */ React.createElement(View, { style: styles.statsRow }, /* @__PURE__ */
React.createElement(View, { style: [styles.statCard, styles.statCardDark] }, /* @__PURE__ */
React.createElement(Text, { style: [styles.statValue, { color: "#fff" }] }, inventoryStats.totalItems), /*
@__PURE__ */ React.createElement(Text, { style: [styles.statLabel, { color: "#d1d5db" }] }, "Total Items")),
/* @__PURE__ */ React.createElement(View, { style: styles.statCard }, /* @__PURE__ */
React.createElement(Text, { style: [styles.statValue, { color: "#16a34a" }] }, inventoryStats.inStock), /*
```

```
@__PURE__ */ React.createElement(Text, { style: styles.statLabel }, "In-Stock"))),
    /* @__PURE__ */ React.createElement(View, { style: styles.statsRow }, /* @__PURE__ */
React.createElement(View, { style: styles.statCard }, /* @__PURE__ */ React.createElement(Text, { style:
[styles.statValue, { color: "#D97706" }] }, inventoryStats.lowStock), /* @__PURE__ */
React.createElement(Text, { style: styles.statLabel }, "Low Stock")), /* @__PURE__ */
React.createElement(View, { style: styles.statCard }, /* @__PURE__ */ React.createElement(Text, { style:
[styles.statValue, { color: "#DC2626" }] }, inventoryStats.outOfStock), /* @__PURE__ */
React.createElement(Text, { style: styles.statLabel }, "Out of Stock"))),
    /* @__PURE__ */ React.createElement(View, { style: styles.recentlyAddedContainer }, /* @__PURE__ */
React.createElement(View, { style: styles.headerRow }, /* @__PURE__ */ React.createElement(View, null, /*
@__PURE__ */ React.createElement(Text, { style: styles.coltitle }, "Recently Added Items"), /* @__PURE__ */
React.createElement(Text, { style: styles.colsubtitle }, "Your newly added products"))),
recentlyAddedItems.map((item) => /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        key: (item.id || item._id || Math.random()).toString(),
        style: styles.recentItemCard
      },
      /* @__PURE__ */ React.createElement(View, { style: styles.recentItemContent }, /* @__PURE__ */
React.createElement(Text, { style: styles.recentItemName }, item.name), /* @__PURE__ */
React.createElement(View, { style: styles.infoRow }, /* @__PURE__ */ React.createElement(Text, { style:
styles.recentItemCategory }, item.category), /* @__PURE__ */ React.createElement(Text, { style:
styles.separator }, "\u2022"), /* @__PURE__ */ React.createElement(
        Text,
        {
          style: [
            styles.stockText,
            { color: item.stock <= 10 ? "#DC2626" : "#0369A1" }
          ]
        },
        item.stock,
        " in stock"
      ))),
      /* @__PURE__ */ React.createElement(Text, { style: styles.recentItemDate }, item.dateAdded)
    ))),
    /* @__PURE__ */ React.createElement(
      View,
      {
        style: [
          styles.actionButtonsContainer,
          { justifyContent: "center", marginTop: 5, marginBottom: 10 }
        ]
      },
      /* @__PURE__ */ React.createElement(
        TouchableOpacity,
        {
          style: [
            styles.actionButton,
            {
              backgroundColor: "#1f1f1f",
              width: "90%",
              maxWidth: "100%",
              paddingVertical: 15
            }
          ],
          onPress: () => setShowTableView(true)
        },
        /* @__PURE__ */ React.createElement(
          Ionicons,
          {
            name: "grid",
            size: 20,
            color: "#ffffffff",
            style: styles.buttonIcon
          }
        ),
        /* @__PURE__ */ React.createElement(Text, { style: styles.actionButtonText }, "View Inventory")
      )
    ),
    /* @__PURE__ */ React.createElement(View, { style: styles.allProductsContainer }, /* @__PURE__ */
React.createElement(Text, { style: styles.coltitle }, "All Products"), /* @__PURE__ */
React.createElement(Text, { style: styles.colsubtitle }, "Showing ", products.length, " items from local
database"), products.length === 0 ? /* @__PURE__ */ React.createElement(View, { style: styles.emptyState },
/* @__PURE__ */ React.createElement(Text, { style: styles.emptyText }, "No products found.")) :
products.map((product) => /* @__PURE__ */ React.createElement(
      View,
      {
```

```
        key: (product._id || product.id || Math.random()).toString(),
        style: styles.productListItem
      },
      /* @__PURE__ */ React.createElement(View, null, /* @__PURE__ */ React.createElement(Text, { style:
styles.productName }, product.name || "Unnamed Product"), /* @__PURE__ */ React.createElement(Text, { style:
styles.productMeta }, product.category || "Uncategorized", " \u2022 SKU:", " ", product.sku || "N/A")),
      /* @__PURE__ */ React.createElement(View, { style: styles.productRight }, /* @__PURE__ */
React.createElement(Text, { style: styles.productPrice }, "\u20B1", Number(product.price || 0).toFixed(2)),
/* @__PURE__ */ React.createElement(
        Text,
        {
          style: [
            styles.productStock,
            (product.stock || 0) <= 10 ? styles.lowStock : null
          ]
        },
        product.stock || 0,
        " in stock"
      ))
    )))
  ))));
}
const styles = StyleSheet.create({
  actionButtonsContainer: {
    width: "100%",
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    marginBottom: 20,
    paddingHorizontal: 10,
    gap: 10
  },
  actionButton: {
    flexDirection: "row",
    justifyContent: "center",
    alignItems: "center",
    paddingVertical: 12,
    borderRadius: 8,
    flex: 1
  },
  buttonIcon: {
    marginRight: 8
  },
  actionButtonText: {
    color: "#fff",
    fontWeight: "600",
    fontSize: 16
  },
  tableContainer: {
    flex: 1,
    backgroundColor: "#fff",
    zIndex: 1
  },
  tableHeader: {
    flexDirection: "row",
    backgroundColor: "#8B4513",
    padding: 15,
    alignItems: "center",
    justifyContent: "space-between"
  },
  headerLeft: {
    flexDirection: "row",
    alignItems: "center",
    flex: 1
  },
  backButton: {
    flexDirection: "row",
    alignItems: "center",
    padding: 5,
    backgroundColor: "rgba(255,255,255,0.2)",
    borderRadius: 20,
    paddingHorizontal: 12,
    paddingVertical: 6
  },
  closeButton: {
    backgroundColor: "#000000ff",
    borderRadius: 20,
```

```
    width: 36,
    height: 36,
    alignItems: "center",
    justifyContent: "center",
    marginLeft: 10,
    // Reduced from 20 to 10 to move it left
    borderWidth: 2,
    borderColor: "#fff",
    elevation: 5,
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.3,
    shadowRadius: 3
  },
  tableTitle: {
    color: "#fff",
    fontSize: 18,
    fontWeight: "bold"
  },
  tableSearchWrap: {
    flexDirection: "row",
    alignItems: "center",
    backgroundColor: "#f9fafb",
    borderBottomWidth: 1,
    borderBottomColor: "#e5e7eb",
    paddingHorizontal: 12,
    paddingVertical: 10
  },
  tableSearchIcon: {
    marginRight: 8
  },
  tableSearchInput: {
    flex: 1,
    fontSize: 15,
    color: "#111827",
    paddingVertical: Platform.OS === "ios" ? 10 : 8,
    paddingHorizontal: 4,
    minHeight: 40
  },
  tableSearchClear: {
    marginLeft: 4,
    padding: 4
  },
  tableSearchEmpty: {
    padding: 24,
    alignItems: "center"
  },
  tableSearchEmptyText: {
    fontSize: 14,
    color: "#6b7280",
    textAlign: "center"
  },
  tableHeaderRow: {
    flexDirection: "row",
    backgroundColor: "#F5F0E5",
    paddingVertical: 14,
    borderBottomWidth: 1,
    borderTopWidth: 1,
    borderColor: "#E0D9C8"
  },
  tableHeaderCell: {
    fontWeight: "600",
    textAlign: "left",
    paddingHorizontal: 12,
    color: "#333333"
  },
  tableRow: {
    flexDirection: "row",
    borderBottomWidth: 1,
    borderBottomColor: "#E0E0E0",
    paddingVertical: 8,
    alignItems: "center"
  },
  actionButtonContainer: {
    flex: 1,
    alignItems: "center",
    justifyContent: "center"
```

```
},
menuButton: {
  padding: 10,
  borderRadius: 20,
  width: 42,
  height: 42,
  justifyContent: "center",
  alignItems: "center",
  backgroundColor: "rgba(0,0,0,0.03)"
},
menuContainer: {
  position: "absolute",
  right: 0,
  backgroundColor: "#fff",
  borderRadius: 12,
  padding: 8,
  shadowColor: "#000",
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.15,
  shadowRadius: 6,
  elevation: 8,
  zIndex: 1e3,
  flexDirection: "row",
  gap: 8
},
menuItem: {
  width: 40,
  height: 40,
  borderRadius: 8,
  justifyContent: "center",
  alignItems: "center"
},
menuIcon: {
  fontSize: 20
},
tableCell: {
  padding: 10,
  textAlign: "left",
  justifyContent: "center",
  borderRightWidth: 1,
  borderRightColor: "#eee",
  zIndex: 1
},
tableList: {
  flex: 1
},
actionButtonsContainer: {
  flexDirection: "row",
  justifyContent: "space-between",
  paddingHorizontal: 16,
  marginTop: 10,
  marginBottom: 10,
  gap: 10
},
actionButton: {
  flex: 1,
  flexDirection: "row",
  alignItems: "center",
  justifyContent: "center",
  paddingVertical: 12,
  borderRadius: 8,
  marginHorizontal: 5,
  elevation: 2
},
buttonIcon: {
  marginRight: 8
},
actionButtonText: {
  color: "#fff",
  fontWeight: "600",
  fontSize: 14
},
container: {
  flex: 1,
  backgroundColor: "#53321c"
},
headerBackground: {
```

```
  height: 120,
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  paddingHorizontal: 20,
  paddingTop: 40
},
headerTitle: {
  fontSize: 20,
  fontWeight: "bold",
  color: "#fff"
},
button: {
  padding: 10
},
buttonImage: {
  width: 20,
  height: 20,
  resizeMode: "contain"
},
main: {
  flex: 1,
  width: "100%"
},
scrollContent: {
  paddingBottom: 20
},
card: {
  borderRadius: 12,
  backgroundColor: "#f7f2ef",
  marginTop: 10,
  marginHorizontal: 16,
  marginBottom: 12,
  padding: 16,
  elevation: 2,
  borderWidth: 1,
  borderColor: "#f0f0f0"
},
card2: {
  height: 100,
  borderRadius: 12,
  marginTop: 10,
  marginBottom: 10,
  marginRight: 20,
  marginLeft: 25,
  padding: 25,
  backgroundColor: "#fff",
  justifyContent: "center",
  alignItems: "center",
  shadowColor: "#000",
  shadowOpacity: 0.1,
  shadowOffset: { width: 0, height: 4 },
  shadowRadius: 6,
  elevation: 6
},
mainValue: {
  fontSize: 32,
  fontWeight: "800",
  color: "#AD7F65"
},
whitecard: {
  flex: 1,
  backgroundColor: "#fff",
  marginTop: -10,
  borderTopLeftRadius: 15,
  borderTopRightRadius: 15,
  paddingTop: 2
},
title: {
  fontSize: 13,
  color: "#999",
  fontWeight: "500",
  marginTop: 2,
  marginBottom: 12
},
row: {
```

```
  flexDirection: "row",
  justifyContent: "space-between",
  width: "100%",
  marginTop: 10
},
column: {
  alignItems: "flex-start",
  flex: 1
},
subValue: {
  fontSize: 16,
  fontWeight: "700",
  color: "#333",
  marginBottom: 2
},
subLabel: {
  fontSize: 11,
  color: "#999",
  marginTop: 0,
  textAlign: "left"
},
statsRow: {
  flexDirection: "row",
  justifyContent: "space-between",
  marginBottom: 10,
  paddingHorizontal: 16,
  gap: 10
},
statCard: {
  flex: 1,
  backgroundColor: "#fff",
  padding: 16,
  borderRadius: 14,
  marginHorizontal: 4,
  borderWidth: 1,
  borderColor: "#e5e7eb",
  shadowColor: "#000",
  shadowOpacity: 0.04,
  shadowRadius: 4,
  shadowOffset: { width: 0, height: 2 },
  elevation: 2
},
statCardDark: {
  backgroundColor: "#1f1f1f",
  borderColor: "#1f1f1f"
},
statValue: {
  fontSize: 28,
  fontWeight: "800",
  color: "#1a1a1a",
  letterSpacing: -0.5
},
statLabel: {
  fontSize: 13,
  marginTop: 4,
  fontWeight: "600",
  color: "#6b7280"
},
colcontainer: {
  backgroundColor: "#fafafa",
  borderRadius: 8,
  marginHorizontal: 16,
  marginTop: 10,
  padding: 14,
  elevation: 1,
  marginBottom: 15
},
headerRow: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  marginBottom: 12
},
coltitle: {
  fontSize: 13,
  fontWeight: "600",
  color: "#333"
```

```
},
colsubtitle: {
  color: "#999",
  fontSize: 11,
  marginTop: 2
},
viewMore: { color: "#AD7F65", fontWeight: "600", fontSize: 12 },
recentlyAddedContainer: {
  backgroundColor: "#fafafa",
  borderRadius: 8,
  marginHorizontal: 16,
  marginTop: 10,
  padding: 14,
  elevation: 1,
  marginBottom: 15
},
recentItemCard: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  paddingVertical: 12,
  paddingHorizontal: 10,
  borderBottomWidth: 1,
  borderBottomColor: "#e0e0e0"
},
recentItemContent: {
  flex: 1
},
infoRow: {
  flexDirection: "row",
  alignItems: "center",
  marginTop: 2,
  flexWrap: "wrap"
},
recentItemCategory: {
  fontSize: 12,
  color: "#6B7280"
},
separator: {
  marginHorizontal: 6,
  color: "#D1D5DB"
},
stockText: {
  fontSize: 12,
  fontWeight: "500"
},
recentItemName: {
  fontSize: 14,
  fontWeight: "600",
  color: "#333",
  marginBottom: 4
},
recentItemDate: {
  fontSize: 10,
  color: "#AD7F65",
  fontWeight: "500",
  marginLeft: 8
},
tableHeader: {
  flexDirection: "row",
  borderBottomWidth: 1,
  borderBottomColor: "#e0e0e0",
  paddingBottom: 8,
  marginBottom: 8
},
headerText: { fontSize: 11, color: "#999", fontWeight: "600" },
rowtext: { fontSize: 12, color: "#333", paddingVertical: 6 },
statusBadge: {
  backgroundColor: "#FFE8CC",
  borderRadius: 12,
  paddingHorizontal: 8,
  paddingVertical: 2,
  alignItems: "center"
},
statusText: { color: "#AD7F65", fontWeight: "600", fontSize: 10 },
addItemButton: {
  backgroundColor: "#8B4513",
```

```
    padding: 15,
    borderRadius: 8,
    margin: 16,
    alignItems: "center",
    flexDirection: "row",
    justifyContent: "center"
  },
  addItemText: {
    color: "#fff",
    fontSize: 16,
    fontWeight: "600",
    marginLeft: 8
  },
  loadingContainer: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    padding: 32
  },
  loadingText: {
    marginTop: 12,
    color: "#666",
    fontSize: 14
  },
  allProductsContainer: {
    backgroundColor: "#fafafa",
    borderRadius: 8,
    marginHorizontal: 16,
    marginTop: 10,
    padding: 14,
    elevation: 1,
    marginBottom: 25
  },
  productListItem: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    paddingVertical: 12,
    borderBottomWidth: 1,
    borderBottomColor: "#e0e0e0"
  },
  productName: {
    fontSize: 14,
    fontWeight: "600",
    color: "#333"
  },
  productMeta: {
    fontSize: 12,
    color: "#6B7280",
    marginTop: 2
  },
  productRight: {
    alignItems: "flex-end"
  },
  productPrice: {
    fontSize: 14,
    fontWeight: "600",
    color: "#5D4037"
  },
  productStock: {
    fontSize: 12,
    color: "#0369A1",
    marginTop: 2
  },
  lowStock: {
    color: "#DC2626",
    fontWeight: "600"
  }
});
```

FILE: pos/mobile/app/(tabs)/profile.jsx
==========

```
import Header from "@/components/shared/header";
import AsyncStorage from "@react-native-async-storage/async-storage";
import { useFocusEffect } from "@react-navigation/native";
```

```
import * as LocalAuthentication from "expo-local-authentication";
import { useRouter } from "expo-router";
import React, { useEffect, useRef, useState } from "react";
import {
  Alert,
  Animated,
  Image,
  Modal,
  ScrollView,
  StyleSheet,
  Switch,
  Text,
  TouchableOpacity,
  View
} from "react-native";
import { employeeAPI } from "../../services/api";
const AccountSettings = () => {
  const router = useRouter();
  const [logoutModalVisible, setLogoutModalVisible] = useState(false);
  const [syncModalVisible, setSyncModalVisible] = useState(false);
  const [syncMessage, setSyncMessage] = useState("");
  const [toastIcon, setToastIcon] = useState("sync");
  const toastTranslate = useRef(new Animated.Value(-60)).current;
  const toastOpacity = useRef(new Animated.Value(0)).current;
  const [biometricEnabled, setBiometricEnabled] = useState(false);
  const [biometricAvailable, setBiometricAvailable] = useState(false);
  const [profile, setProfile] = useState({
    name: "",
    role: "",
    status: "",
    contactNo: "",
    email: "",
    ownerOf: "Owner of Create Your Style",
    image: null
  });
  const loadProfile = async () => {
    try {
      const stored = await AsyncStorage.getItem("currentEmployee");
      if (!stored) return;
      const parsed = JSON.parse(stored);
      let latest = parsed;
      if (parsed._id || parsed.id) {
        try {
          const res = await employeeAPI.getById(parsed._id || parsed.id);
          if (res?.success && res.data) {
            latest = res.data;
            await AsyncStorage.setItem("currentEmployee", JSON.stringify(latest));
          }
        } catch {
        }
      }
      setProfile({
        name: latest.name || `${latest.firstName || ""} ${latest.lastName || ""}`.trim(),
        role: latest.role || "",
        status: latest.status || "Active",
        contactNo: latest.contactNo || "",
        email: latest.email || "",
        ownerOf: "Owner of Create Your Style",
        image: latest.profileImage || latest.image || null,
        id: latest._id || latest.id
      });
    } catch (error) {
      console.warn("Failed to load profile:", error?.message);
    }
  };
  useFocusEffect(
    React.useCallback(() => {
      loadProfile();
      loadBiometricState();
      return () => {
      };
    }, [])
  );
  const loadBiometricState = async () => {
    try {
      if (!LocalAuthentication?.hasHardwareAsync) return;
      const hasHardware = await LocalAuthentication.hasHardwareAsync();
```

```
      const isEnrolled = await LocalAuthentication.isEnrolledAsync();
      setBiometricAvailable(hasHardware && isEnrolled);
      const enabled = await AsyncStorage.getItem("@biometric_enabled");
      setBiometricEnabled(enabled === "true");
    } catch (e) {
      setBiometricAvailable(false);
    }
  };
  const toggleBiometric = async () => {
    try {
      if (!biometricEnabled) {
        const result = await LocalAuthentication.authenticateAsync({
          promptMessage: "Verify your identity to enable biometric login",
          cancelLabel: "Cancel",
          disableDeviceFallback: false
        });
        if (result.success) {
          await AsyncStorage.setItem("@biometric_enabled", "true");
          await AsyncStorage.setItem("@biometric_asked", "true");
          setBiometricEnabled(true);
        } else if (result.error !== "user_cancel") {
          Alert.alert("Verification Failed", "Could not verify your identity. Please try again.");
        }
      } else {
        await AsyncStorage.setItem("@biometric_enabled", "false");
        setBiometricEnabled(false);
      }
    } catch (error) {
      console.error("Biometric toggle error:", error);
      Alert.alert("Error", "Biometric authentication is not available on this device.");
    }
  };
  useEffect(() => {
    if (syncModalVisible) {
      Animated.parallel([
        Animated.timing(toastTranslate, {
          toValue: 0,
          duration: 280,
          useNativeDriver: true
        }),
        Animated.timing(toastOpacity, {
          toValue: 1,
          duration: 280,
          useNativeDriver: true
        })
      ]).start();
    } else {
      Animated.parallel([
        Animated.timing(toastTranslate, {
          toValue: -60,
          duration: 220,
          useNativeDriver: true
        }),
        Animated.timing(toastOpacity, {
          toValue: 0,
          duration: 220,
          useNativeDriver: true
        })
      ]).start();
    }
  }, [syncModalVisible, toastTranslate, toastOpacity]);
  const handleLogout = () => {
    console.log("User logged out");
    setLogoutModalVisible(false);
    router.replace("/pinlogin");
  };
  return /* @__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */
React.createElement(Header, null), /* @__PURE__ */ React.createElement(
    Animated.View,
    {
      style: [
        styles.toastContainer,
        {
          transform: [
            {
              translateY: toastTranslate
            },
```

```
            {
              scale: toastOpacity.interpolate({
                inputRange: [0, 1],
                outputRange: [0.95, 1]
              })
            }
          ],
          opacity: toastOpacity,
          shadowColor: "#000",
          shadowOffset: { width: 0, height: 2 },
          shadowOpacity: 0.15,
          shadowRadius: 8,
          elevation: 5
        }
      ]
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.toastInner }, /* @__PURE__ */
React.createElement(View, { style: styles.toastContent }, /* @__PURE__ */ React.createElement(Text, { style:
styles.toastText }, syncMessage)), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        onPress: () => setSyncModalVisible(false),
        style: styles.toastCloseButton
      },
      /* @__PURE__ */ React.createElement(Text, { style: styles.toastCloseText }, "\u2715")
    ))
  ), /* @__PURE__ */ React.createElement(View, { style: styles.whitecard }, /* @__PURE__ */
React.createElement(
    ScrollView,
    {
      style: styles.scrollView,
      contentContainerStyle: styles.scrollViewContent
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.profileCard }, /* @__PURE__ */
React.createElement(View, { style: styles.flexprofile }, profile.image ? /* @__PURE__ */ React.createElement(
      Image,
      {
        source: { uri: profile.image },
        style: styles.profileImageLarge
      }
    ) : /* @__PURE__ */ React.createElement(
      Image,
      {
        source: require("../(tabs)/iconz/default.jpeg"),
        style: styles.profileImageLarge
      }
    ), /* @__PURE__ */ React.createElement(View, { style: styles.profileInfo }, /* @__PURE__ */
React.createElement(Text, { style: styles.profileName }, profile.name), /* @__PURE__ */
React.createElement(Text, { style: styles.profileJob }, profile.role), /* @__PURE__ */
React.createElement(View, { style: styles.activeBadge }, /* @__PURE__ */ React.createElement(Text, { style:
styles.activeText }, "Active")))), /* @__PURE__ */ React.createElement(View, null, /* @__PURE__ */
React.createElement(Text, { style: styles.profileDetails }, profile.contactNo), /* @__PURE__ */
React.createElement(Text, { style: styles.profileDetails }, profile.email), /* @__PURE__ */
React.createElement(Text, { style: styles.profileDetails }, profile.ownerOf))),
    /* @__PURE__ */ React.createElement(View, { style: styles.settingsList }, /* @__PURE__ */
React.createElement(
      TouchableOpacity,
      {
        style: styles.settingItem,
        onPress: () => router.push("../Edit Profile")
      },
      /* @__PURE__ */ React.createElement(
        Image,
        {
          source: require("../(tabs)/iconz/edit.png"),
          style: styles.settingIcon
        }
      ),
      /* @__PURE__ */ React.createElement(Text, { style: styles.settingText }, "Edit Profile")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: styles.settingItem,
        onPress: () => router.push("../Update Pin")
      },
      /* @__PURE__ */ React.createElement(
        Image,
```

```
        {
          source: require("../(tabs)/iconz/pin.png"),
          style: styles.settingIcon
        }
      ),
      /* @__PURE__ */ React.createElement(Text, { style: styles.settingText }, "Update Pincode")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: styles.settingItem,
        onPress: () => router.push("../RolesAndPermission")
      },
      /* @__PURE__ */ React.createElement(
        Image,
        {
          source: require("../(tabs)/iconz/roles.png"),
          style: styles.settingIcon
        }
      ),
      /* @__PURE__ */ React.createElement(Text, { style: styles.settingText }, "Roles and Permission")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: styles.settingItem,
        onPress: () => router.push("../Preferences")
      },
      /* @__PURE__ */ React.createElement(
        Image,
        {
          source: require("../(tabs)/iconz/preferences.png"),
          style: styles.settingIcon
        }
      ),
      /* @__PURE__ */ React.createElement(Text, { style: styles.settingText }, "Push Notification")
    ), biometricAvailable && /* @__PURE__ */ React.createElement(View, { style: [styles.settingItem,
{ justifyContent: "space-between" }] }, /* @__PURE__ */ React.createElement(View, { style: { flexDirection:
"row", alignItems: "center" } }, /* @__PURE__ */ React.createElement(Text, { style: { fontSize: 20,
marginRight: 15, width: 20, textAlign: "center" } }, "\u{1F512}"), /* @__PURE__ */ React.createElement(Text,
{ style: styles.settingText }, "Biometric Login")), /* @__PURE__ */ React.createElement(
      Switch,
      {
        trackColor: { false: "#ccc", true: "#16a34a" },
        thumbColor: "#fff",
        value: biometricEnabled,
        onValueChange: toggleBiometric
      }
    )), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: styles.settingItem,
        onPress: () => router.push("../SyncLog")
      },
      /* @__PURE__ */ React.createElement(
        Image,
        {
          source: require("../(tabs)/iconz/sync.png"),
          style: styles.settingIcon
        }
      ),
      /* @__PURE__ */ React.createElement(Text, { style: styles.settingText }, "Sync Log")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: styles.settingItem,
        onPress: () => router.push("../VoidLog")
      },
      /* @__PURE__ */ React.createElement(
        Image,
        {
          source: require("../(tabs)/iconz/void.png"),
          style: styles.settingIcon
        }
      ),
      /* @__PURE__ */ React.createElement(Text, { style: styles.settingText }, "Void Log")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
```

```
        style: styles.settingItem,
        onPress: () => router.push("../Archive")
      },
      /* @__PURE__ */ React.createElement(
        Image,
        {
          source: require("../(tabs)/iconz/archive.png"),
          style: styles.settingIcon
        }
      ),
      /* @__PURE__ */ React.createElement(Text, { style: styles.settingText }, "Archive")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: styles.settingItem,
        onPress: () => router.push("../About")
      },
      /* @__PURE__ */ React.createElement(
        Image,
        {
          source: require("../(tabs)/iconz/about.png"),
          style: styles.settingIcon
        }
      ),
      /* @__PURE__ */ React.createElement(Text, { style: styles.settingText }, "About")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: styles.settingItem,
        onPress: () => setLogoutModalVisible(true)
      },
      /* @__PURE__ */ React.createElement(
        Image,
        {
          source: require("../(tabs)/iconz/logout.png"),
          style: styles.settingIcon
        }
      ),
      /* @__PURE__ */ React.createElement(Text, { style: styles.settingText }, "Logout")
    ))
  )), /* @__PURE__ */ React.createElement(
    Modal,
    {
      transparent: true,
      visible: logoutModalVisible,
      animationType: "fade",
      onRequestClose: () => setLogoutModalVisible(false)
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.modalOverlay }, /* @__PURE__ */
React.createElement(View, { style: styles.modalContent }, /* @__PURE__ */ React.createElement(View, { style:
styles.modalHeader }, /* @__PURE__ */ React.createElement(Text, { style: styles.modalTitle }, "Logout")), /*
@__PURE__ */ React.createElement(View, { style: styles.modalBody }, /* @__PURE__ */ React.createElement(
      Image,
      {
        source: require("../(tabs)/iconz/logout.png"),
        style: styles.modalIcon,
        resizeMode: "contain"
      }
    ), /* @__PURE__ */ React.createElement(Text, { style: styles.modalMessage }, "Are you sure you want to
log out of your account?")), /* @__PURE__ */ React.createElement(View, { style: styles.modalFooter }, /*
@__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [styles.modalButton, styles.cancelButton],
        onPress: () => setLogoutModalVisible(false),
        activeOpacity: 0.7
      },
      /* @__PURE__ */ React.createElement(Text, { style: styles.cancelButtonText }, "Cancel")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [styles.modalButton, styles.logoutButton],
        onPress: handleLogout,
        activeOpacity: 0.7
      },
      /* @__PURE__ */ React.createElement(Text, { style: styles.logoutButtonText }, "Logout")
    ))))
```

```
  ));
};
export default AccountSettings;
const styles = StyleSheet.create({
  // Modal Styles
  modalOverlay: {
    flex: 1,
    backgroundColor: "rgba(0, 0, 0, 0.5)",
    justifyContent: "center",
    alignItems: "center",
    padding: 20
  },
  modalContent: {
    width: "100%",
    backgroundColor: "white",
    borderRadius: 16,
    overflow: "hidden",
    maxWidth: 400
  },
  modalHeader: {
    padding: 20,
    borderBottomWidth: 1,
    borderBottomColor: "#f0f0f0",
    alignItems: "center"
  },
  modalTitle: {
    fontSize: 20,
    fontWeight: "bold",
    color: "#333"
  },
  modalBody: {
    padding: 25,
    alignItems: "center"
  },
  modalIcon: {
    width: 60,
    height: 60,
    marginBottom: 20,
    tintColor: "#AD7F65"
  },
  modalMessage: {
    fontSize: 16,
    textAlign: "center",
    color: "#555",
    lineHeight: 24
  },
  modalFooter: {
    flexDirection: "row",
    justifyContent: "space-between",
    padding: 15,
    borderTopWidth: 1,
    borderTopColor: "#f0f0f0"
  },
  modalButton: {
    flex: 1,
    paddingVertical: 12,
    borderRadius: 8,
    alignItems: "center",
    justifyContent: "center",
    marginHorizontal: 5
  },
  cancelButton: {
    backgroundColor: "#f5f5f5"
  },
  logoutButton: {
    backgroundColor: "#AD7F65"
  },
  cancelButtonText: {
    color: "#555",
    fontSize: 16,
    fontWeight: "600"
  },
  logoutButtonText: {
    color: "white",
    fontSize: 16,
    fontWeight: "600"
  },
```

```
container: {
  flex: 1,
  backgroundColor: "#f6f6f6"
},
whitecard: {
  flex: 1,
  backgroundColor: "#fff",
  marginTop: -10,
  borderTopLeftRadius: 15,
  borderTopRightRadius: 15,
  paddingTop: 2,
  pointerEvents: "box-none"
},
scrollView: {
  flex: 1
},
scrollViewContent: {
  paddingBottom: 30
},
profileCard: {
  backgroundColor: "#fff",
  padding: 20,
  borderRadius: 16,
  width: "92%",
  maxWidth: 400,
  alignSelf: "center",
  marginTop: 15,
  elevation: 5
},
profileInfo: {
  alignItems: "flex-start"
},
flexprofile: {
  flexDirection: "row",
  alignItems: "center",
  gap: 10,
  marginBottom: 10
},
profileImageLarge: {
  width: 85,
  height: 85,
  borderRadius: 42,
  marginRight: 12
},
profileName: {
  fontSize: 18,
  fontWeight: "bold",
  color: "#2b2b2b"
},
profileJob: {
  fontSize: 14,
  color: "#d17b59",
  marginBottom: 5
},
activeBadge: {
  backgroundColor: "#c8f7c5",
  paddingHorizontal: 10,
  paddingVertical: 2,
  borderRadius: 10,
  marginBottom: 10,
  alignSelf: "flex-start"
},
activeText: {
  color: "#2b8a3e",
  fontSize: 12
},
profileDetails: {
  fontSize: 13,
  color: "#555",
  marginTop: 2
},
settingsList: {
  marginTop: 20,
  width: "88%",
  alignSelf: "center"
},
settingItem: {
```

```
  flexDirection: "row",
  alignItems: "center",
  paddingVertical: 16,
  paddingHorizontal: 2,
  borderBottomWidth: 1,
  borderBottomColor: "#f0f0f0"
},
settingIcon: {
  width: 20,
  height: 20,
  marginRight: 15,
  tintColor: "#333"
},
settingText: {
  fontSize: 16,
  color: "#3b3b3b"
},
/* MODALS */
modalContainer: {
  flex: 1,
  backgroundColor: "rgba(0,0,0,0.45)",
  justifyContent: "center",
  alignItems: "center"
},
modalBox: {
  backgroundColor: "#fff",
  width: "80%",
  padding: 20,
  borderRadius: 14,
  alignItems: "center"
},
modalTitle: {
  fontSize: 18,
  fontWeight: "700",
  textAlign: "center"
},
modalClose: {
  marginTop: 20
},
modalCloseText: {
  fontSize: 16,
  fontWeight: "600",
  color: "#d37d4c"
},
syncButton: {
  marginTop: 12,
  alignSelf: "center",
  backgroundColor: "#AD7F65",
  paddingVertical: 10,
  paddingHorizontal: 18,
  borderRadius: 10,
  marginBottom: 10
},
syncButtonText: {
  color: "#fff",
  fontSize: 14,
  fontWeight: "700"
},
// Modern Toast
toastContainer: {
  position: "absolute",
  top: 50,
  left: 16,
  right: 16,
  zIndex: 1e3,
  borderRadius: 14,
  overflow: "hidden",
  backgroundColor: "white"
},
toastInner: {
  flexDirection: "row",
  alignItems: "center",
  backgroundColor: "#53321c",
  paddingVertical: 14,
  paddingHorizontal: 16,
  borderRadius: 14
},
```

```
  toastContent: {
    flex: 1,
    flexDirection: "row",
    alignItems: "center"
  },
  toastText: {
    color: "#fff",
    fontSize: 14,
    fontWeight: "500",
    lineHeight: 20,
    marginLeft: 10
  },
  toastCloseButton: {
    marginLeft: 12,
    padding: 4,
    borderRadius: 12,
    width: 24,
    height: 24,
    alignItems: "center",
    justifyContent: "center"
  },
  toastCloseText: {
    color: "rgba(255, 255, 255, 0.8)",
    fontSize: 16,
    lineHeight: 16,
    fontWeight: "600"
  }
});
```

FILE: pos/mobile/app/(tabs)/transaction.jsx
==========

```
import Header from "@/components/shared/header";
import { useData } from "@/context/DataContext";
import { Ionicons } from "@expo/vector-icons";
import { useFocusEffect } from "@react-navigation/native";
import { FlashList } from "@shopify/flash-list";
import * as Print from "expo-print";
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import {
  ActivityIndicator,
  Modal,
  RefreshControl,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View
} from "react-native";
import { SafeAreaView } from "react-native-safe-area-context";
const formatDateTime = (date = /* @__PURE__ */ new Date()) => {
  const options = {
    year: "numeric",
    month: "long",
    day: "numeric",
    hour: "2-digit",
    minute: "2-digit",
    hour12: true
  };
  return date.toLocaleString("en-US", options);
};
export default function Transaction() {
  const {
    transactions: cachedTransactions,
    transactionsLoading,
    fetchTransactions: fetchCachedTransactions,
    invalidateCache
  } = useData();
  const [search, setSearch] = useState("");
  const [selectedTransaction, setSelectedTransaction] = useState(null);
  const [modalVisible, setModalVisible] = useState(false);
  const [currentDateTime, setCurrentDateTime] = useState(formatDateTime());
  const [refreshing, setRefreshing] = useState(false);
  const [timeFilter, setTimeFilter] = useState("All");
  const TIME_FILTERS = ["All", "Today", "Yesterday", "This Week", "This Month"];
```

```
  const [page, setPage] = useState(1);
  const [hasMore, setHasMore] = useState(true);
  const [loadingMore, setLoadingMore] = useState(false);
  const isFetching = useRef(false);
  const lastFetchTime = useRef(0);
  const filteredTransactions = useMemo(() => {
    const searchLower = search?.toLowerCase() || "";
    const now = /* @__PURE__ */ new Date();
    const startOfToday = new Date(now.getFullYear(), now.getMonth(), now.getDate());
    const startOfYesterday = new Date(startOfToday);
    startOfYesterday.setDate(startOfYesterday.getDate() - 1);
    const startOfWeek = new Date(startOfToday);
    startOfWeek.setDate(startOfWeek.getDate() - startOfWeek.getDay());
    const startOfMonth = new Date(now.getFullYear(), now.getMonth(), 1);
    return cachedTransactions.filter((t) => {
      if (t.status === "Voided" || t.paymentMethod === "return" && t.originalTransactionId) {
        return false;
      }
      if (timeFilter !== "All") {
        const txDate = new Date(t.checkedOutAt || t.createdAt || t.date || 0);
        if (timeFilter === "Today" && txDate < startOfToday) return false;
        if (timeFilter === "Yesterday" && (txDate < startOfYesterday || txDate >= startOfToday)) return false;
        if (timeFilter === "This Week" && txDate < startOfWeek) return false;
        if (timeFilter === "This Month" && txDate < startOfMonth) return false;
      }
      if (!search) return true;
      const receiptNo = (t.receiptNo || t._id || "").toLowerCase();
      const date = (t.checkedOutAt || t.createdAt || t.date || "").toLowerCase();
      const status = (t.status || "").toLowerCase();
      const cashier = (t.performedByName || t.userName || t.user?.name || "").toLowerCase();
      return receiptNo.includes(searchLower) || date.includes(searchLower) || status.includes(searchLower) ||
cashier.includes(searchLower);
    }).sort((a, b) => {
      const dateA = new Date(a.checkedOutAt || a.createdAt || a.date || 0).getTime();
      const dateB = new Date(b.checkedOutAt || b.createdAt || b.date || 0).getTime();
      return dateB - dateA;
    });
  }, [cachedTransactions, search, timeFilter]);
  useEffect(() => {
    const timer = setInterval(() => {
      setCurrentDateTime(formatDateTime());
    }, 6e4);
    return () => clearInterval(timer);
  }, []);
  const loadTransactions = useCallback(async (forceRefresh = false, loadMore = false) => {
    if (isFetching.current) return;
    const requestedPage = loadMore ? page + 1 : 1;
    try {
      isFetching.current = true;
      if (loadMore) setLoadingMore(true);
      const params = {
        page: requestedPage,
        limit: 20
      };
      if (forceRefresh) {
        invalidateCache("transactions");
      }
      const response = await fetchCachedTransactions(forceRefresh, params);
      if (response && (response.success || response.data)) {
        const data = response.data || response;
        if (response.totalPages) {
          setHasMore(requestedPage < response.totalPages);
        } else {
          setHasMore(data.length === 20);
        }
        if (loadMore) {
          setPage((p) => p + 1);
        } else {
          setPage(1);
        }
      }
      lastFetchTime.current = Date.now();
    } catch (error) {
      console.error("Error loading transactions:", error);
    } finally {
      isFetching.current = false;
      setLoadingMore(false);
```

```
      setRefreshing(false);
    }
  }, [fetchCachedTransactions, invalidateCache, page]);
  useFocusEffect(
    useCallback(() => {
      if (cachedTransactions.length === 0) {
        loadTransactions(false, false);
      }
    }, [loadTransactions, cachedTransactions.length])
  );
  const onRefresh = () => {
    setRefreshing(true);
    loadTransactions(true, false);
  };
  const onEndReached = () => {
    if (!loadingMore && hasMore && !search) {
      loadTransactions(false, true);
    }
  };
  const getCashierName = (transaction) => {
    return transaction.performedByName || transaction.userName || transaction.user?.name || "Unknown";
  };
  const handleView = (item) => {
    setSelectedTransaction(item);
    setModalVisible(true);
  };
  const calculateTotals = (transaction) => {
    const items = transaction.items || [];
    const subtotal = items.reduce((sum, item) => {
      const qty = item.quantity || item.qty || 0;
      const price = item.price || item.unitPrice || 0;
      return sum + qty * price;
    }, 0);
    const discount = transaction.discountAmount || transaction.discount || 0;
    const total = transaction.totalAmount || transaction.total || subtotal - discount;
    return { subtotal, discount, total };
  };
  const handlePrint = async () => {
    if (!selectedTransaction) return;
    const items = selectedTransaction.items || [];
    const totals = calculateTotals(selectedTransaction);
    let htmlContent = `
      <div style="width:220px; font-family: monospace; padding:10px;">
        <h2 style="text-align:center; margin:0; font-size: 18px;">CREATE YOUR STYLE</h2>
        <p style="text-align:center; margin:2px 0; font-size: 10px;">KM 7 Pasonanca, Zamboanga City</p>
        <p style="text-align:center; margin:0;">Cashier: ${getCashierName(selectedTransaction)}</p>
        <p style="text-align:center; margin:0;">Date & Time: ${new Date(
      selectedTransaction.checkedOutAt || selectedTransaction.createdAt || selectedTransaction.date
    ).toLocaleString()}</p>
        <p style="text-align:center; margin:0;">Receipt No: ${selectedTransaction.receiptNo ||
selectedTransaction._id}</p>
        <p style="text-align:center; margin:0;">Status: ${selectedTransaction.status}</p>
        <hr style="border:1px dashed #000;"/>
        <table style="width:100%; border-collapse: collapse;">
          <thead>
            <tr>
              <th style="text-align:left;">Item</th>
              <th style="text-align:center;">Qty</th>
              <th style="text-align:right;">Price</th>
            </tr>
          </thead>
          <tbody>
            ${items.map((item) => {
      const name = item.name || item.productName || "Unknown";
      const qty = item.quantity || item.qty || 0;
      const price = item.price || item.unitPrice || 0;
      return `<tr>
                    <td>${name}</td>
                    <td style="text-align:center;">${qty}</td>
                    <td style="text-align:right;">\u20B1${price.toFixed(2)}</td>
                  </tr>`;
    }).join("")}
          </tbody>
        </table>
        <hr style="border:1px dashed #000;"/>
        <p style="text-align:right; margin:0;">Subtotal: \u20B1${totals.subtotal.toFixed(
      2
```

```
    )}</p>
        <p style="text-align:right; margin:0;">Discount: \u20B1${totals.discount.toFixed(
      2
    )}</p>
        <p style="text-align:right; font-weight:bold; margin:0;">Total: \u20B1${totals.total.toFixed(
      2
    )}</p>
        <hr style="border:1px dashed #000;"/>
        <p style="text-align:center; font-style:italic; margin:0;">This is not an official receipt</p>
        <p style="text-align:center; margin-top:10px;">Thank you for your purchase!</p>
      </div>
    `;
    try {
      await Print.printAsync({ html: htmlContent });
    } catch (error) {
      alert("Print Error: " + error.message);
    }
  };
  const renderItem = ({ item }) => {
    const status = item.status || "Completed";
    const statusColor = status === "Completed" || status === "Paid" ? "#09A046" : status === "Returned" ?
"#FF8C00" : "#C80000";
    const totals = calculateTotals(item);
    const receiptNo = item.receiptNo || item._id || "N/A";
    const date = new Date(
      item.checkedOutAt || item.createdAt || item.date || Date.now()
    ).toLocaleString("en-US", {
      year: "numeric",
      month: "long",
      day: "numeric",
      hour: "2-digit",
      minute: "2-digit"
    });
    const items = item.items || [];
    return /* @__PURE__ */ React.createElement(TouchableOpacity, { style: styles.logItem, onPress: () =>
handleView(item) }, /* @__PURE__ */ React.createElement(View, { style: styles.logHeader }, /* @__PURE__ */
React.createElement(Text, { style: styles.transactionId }, receiptNo), /* @__PURE__ */
React.createElement(Text, { style: styles.date }, getCashierName(item))), /* @__PURE__ */
React.createElement(View, { style: styles.logDetails }, items.length > 0 && /* @__PURE__ */
React.createElement(Text, { style: styles.items }, items.map((d, idx) => {
      const name = d.itemName || d.name || d.productName || "Unknown";
      const qty = d.quantity || d.qty || 0;
      return `${qty}x ${name}`;
    }).join(", ")), /* @__PURE__ */ React.createElement(
      View,
      {
        style: [
          styles.statusBadge,
          { backgroundColor: `${statusColor}20` }
        ]
      },
      /* @__PURE__ */ React.createElement(Text, { style: [styles.statusText, { color: statusColor }] },
status)
    )), /* @__PURE__ */ React.createElement(View, { style: styles.logFooter }, /* @__PURE__ */
React.createElement(Text, { style: styles.total }, "\u20B1", totals.total.toFixed(2)), /* @__PURE__ */
React.createElement(Text, { style: styles.transactionDate }, date)));
  };
  return /* @__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */
React.createElement(Header, null), /* @__PURE__ */ React.createElement(View, { style: styles.whitecard }, /*
@__PURE__ */ React.createElement(SafeAreaView, { style: { flex: 1, width: "100%" } }, /* @__PURE__ */
React.createElement(View, { style: styles.searchContainer }, /* @__PURE__ */ React.createElement(
    Ionicons,
    {
      name: "search",
      size: 20,
      color: "#AD7F65",
      style: styles.searchIcon
    }
  ), /* @__PURE__ */ React.createElement(
    TextInput,
    {
      placeholder: "Search loaded transactions...",
      placeholderTextColor: "#B8A69B",
      style: styles.searchBox,
      value: search,
      onChangeText: setSearch,
      selectionColor: "#AD7F65"
```

```
    }
  ), search.length > 0 && /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      onPress: () => setSearch(""),
      style: styles.clearButton
    },
    /* @__PURE__ */ React.createElement(Ionicons, { name: "close-circle", size: 18, color: "#B8A69B" })
  )), /* @__PURE__ */ React.createElement(View, { style: styles.filterRow }, /* @__PURE__ */
React.createElement(ScrollView, { horizontal: true, showsHorizontalScrollIndicator: false,
contentContainerStyle: { paddingHorizontal: 15 } }, TIME_FILTERS.map((f) => /* @__PURE__ */
React.createElement(
    TouchableOpacity,
    {
      key: f,
      style: [styles.filterPill, timeFilter === f && styles.filterPillActive],
      onPress: () => setTimeFilter(f)
    },
    /* @__PURE__ */ React.createElement(Text, { style: [styles.filterPillText, timeFilter === f &&
styles.filterPillTextActive] }, f)
  )))), /* @__PURE__ */ React.createElement(View, { style: styles.listWrapper }, transactionsLoading &&
cachedTransactions.length === 0 ? /* @__PURE__ */ React.createElement(View, { style:
styles.loadingContainer }, /* @__PURE__ */ React.createElement(ActivityIndicator, { size: "large", color:
"#AD7F65" }), /* @__PURE__ */ React.createElement(Text, { style: styles.loadingText }, "Loading
transactions...")) : /* @__PURE__ */ React.createElement(
    FlashList,
    {
      data: filteredTransactions,
      renderItem,
      keyExtractor: (item) => item._id || item.id || Math.random().toString(),
      estimatedItemSize: 150,
      onEndReached,
      onEndReachedThreshold: 0.5,
      refreshControl: /* @__PURE__ */ React.createElement(
        RefreshControl,
        {
          refreshing,
          onRefresh,
          colors: ["#AD7F65"],
          tintColor: "#AD7F65"
        }
      ),
      ListFooterComponent: () => loadingMore ? /* @__PURE__ */ React.createElement(View, { style: { padding:
10 } }, /* @__PURE__ */ React.createElement(ActivityIndicator, { size: "small", color: "#AD7F65" })) : null,
      ListEmptyComponent: /* @__PURE__ */ React.createElement(View, { style: styles.emptyState }, /*
@__PURE__ */ React.createElement(Text, { style: styles.emptyText }, search ? "\u2014 No matching transactions
found \u2014" : "\u2014 No transactions yet \u2014")),
      contentContainerStyle: styles.listContent
    }
  )))), /* @__PURE__ */ React.createElement(Modal, { visible: modalVisible, animationType: "fade",
transparent: true }, /* @__PURE__ */ React.createElement(View, { style: styles.modalOverlay }, /* @__PURE__
*/ React.createElement(View, { style: styles.modalBox }, selectedTransaction && /* @__PURE__ */
React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(View, { style:
styles.buttonRow }, /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: styles.iconButton,
      onPress: () => setModalVisible(false)
    },
    /* @__PURE__ */ React.createElement(Ionicons, { name: "close-circle", size: 28, color: "#AD7F65" })
  )), /* @__PURE__ */ React.createElement(Text, { style: styles.receiptHeader }, "CREATE YOUR STYLE"), /*
@__PURE__ */ React.createElement(Text, { style: styles.locationText }, "KM 7 Pasonanca, Zamboanga City"), /*
@__PURE__ */ React.createElement(Text, { style: styles.receiptDate }, new Date(
    selectedTransaction.checkedOutAt || selectedTransaction.createdAt || selectedTransaction.date
  ).toLocaleString()), /* @__PURE__ */ React.createElement(Text, { style: styles.receiptStatus }, "Status: ",
selectedTransaction.status), /* @__PURE__ */ React.createElement(Text, { style: styles.cashierLabel },
"Receipt No:", " ", selectedTransaction.receiptNo || selectedTransaction._id), /* @__PURE__ */
React.createElement(Text, { style: styles.cashierLabel }, "Cashier: ", getCashierName(selectedTransaction)),
/* @__PURE__ */ React.createElement(ScrollView, { style: { marginVertical: 10, maxHeight: 250 } },
(selectedTransaction.items || []).map((item, i) => {
    const name = item.itemName || item.name || item.productName || "Unknown";
    const qty = item.quantity || item.qty || 0;
    const price = item.price || item.unitPrice || 0;
    return /* @__PURE__ */ React.createElement(View, { key: i, style: styles.receiptRow }, /* @__PURE__ */
React.createElement(Text, { style: styles.receiptItem }, name), /* @__PURE__ */ React.createElement(Text,
{ style: styles.receiptQty }, qty), /* @__PURE__ */ React.createElement(Text, { style: styles.receiptPrice },
"\u20B1", price.toFixed(2)));
```

```
  })), /* @__PURE__ */ React.createElement(View, { style: styles.receiptTotals }, (() => {
    const totals = calculateTotals(selectedTransaction);
    return /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(Text,
null, "Subtotal: \u20B1", totals.subtotal.toFixed(2)), /* @__PURE__ */ React.createElement(Text, null,
"Discount: \u20B1", totals.discount.toFixed(2)), /* @__PURE__ */ React.createElement(Text, { style:
{ fontWeight: "bold" } }, "Total: \u20B1", totals.total.toFixed(2)));
  })()), /* @__PURE__ */ React.createElement(View, { style: styles.receiptFooter }, /* @__PURE__ */
React.createElement(Text, { style: styles.receiptFooterText }, "This is not an official receipt")))))));
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#f8f9fa"
  },
  whitecard: {
    flex: 1,
    backgroundColor: "#fff",
    marginTop: -10,
    borderTopLeftRadius: 15,
    borderTopRightRadius: 15,
    paddingTop: 2,
    pointerEvents: "box-none"
  },
  searchContainer: {
    flexDirection: "row",
    alignItems: "center",
    backgroundColor: "#fff",
    borderRadius: 10,
    paddingHorizontal: 10,
    marginHorizontal: 15,
    marginBottom: 10,
    height: 40,
    elevation: 1,
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.1,
    shadowRadius: 2
  },
  searchIcon: {
    marginRight: 4,
    fontSize: 18
  },
  searchBox: {
    flex: 1,
    height: 35,
    color: "#333",
    fontSize: 13,
    paddingVertical: 0,
    marginLeft: 4
  },
  clearButton: {
    padding: 4
  },
  listWrapper: {
    flex: 1,
    padding: 10
  },
  listContent: {
    padding: 10,
    backgroundColor: "#f8f9fa",
    borderRadius: 10,
    paddingBottom: 20
    // Add padding at bottom
  },
  logItem: {
    backgroundColor: "#fff",
    borderRadius: 10,
    padding: 16,
    marginBottom: 12,
    elevation: 2,
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.05,
    shadowRadius: 4,
    borderWidth: 1,
    borderColor: "rgba(0,0,0,0.03)"
  },
```

```
logHeader: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  marginBottom: 10,
  paddingBottom: 8,
  borderBottomWidth: 1,
  borderBottomColor: "#f0f0f0"
},
transactionId: {
  fontSize: 16,
  fontWeight: "600",
  color: "#2c3e50"
},
date: {
  fontSize: 12,
  color: "#7f8c8d"
},
logDetails: {
  marginBottom: 10
},
items: {
  fontSize: 14,
  color: "#34495e",
  marginBottom: 8
},
statusBadge: {
  alignSelf: "flex-start",
  paddingHorizontal: 8,
  paddingVertical: 3,
  borderRadius: 12,
  marginTop: 5
},
statusText: {
  fontSize: 12,
  fontWeight: "600"
},
logFooter: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  paddingTop: 12,
  marginTop: 8,
  borderTopWidth: 1,
  borderTopColor: "#f0f0f0"
},
total: {
  fontSize: 16,
  fontWeight: "700",
  color: "#2c3e50",
  letterSpacing: 0.3
},
transactionDate: {
  fontSize: 12,
  color: "#666",
  textAlign: "right"
},
modalOverlay: {
  flex: 1,
  backgroundColor: "rgba(0,0,0,0.6)",
  justifyContent: "center",
  alignItems: "center"
},
modalBox: {
  backgroundColor: "#fff",
  width: "85%",
  borderRadius: 15,
  padding: 20,
  paddingTop: 50,
  elevation: 10,
  alignItems: "center"
},
buttonRow: {
  flexDirection: "column",
  position: "absolute",
  top: 10,
  right: 10,
```

```
  gap: 8,
  alignItems: "center"
},
iconButton: {
  flex: 1,
  alignItems: "center",
  justifyContent: "center",
  paddingVertical: 8
},
receiptHeader: {
  fontSize: 20,
  fontWeight: "bold",
  color: "#000000",
  textAlign: "center",
  marginBottom: 10
},
receiptDate: {
  fontSize: 12,
  color: "#555",
  marginBottom: 2
},
receiptStatus: {
  fontSize: 14,
  color: "#333",
  fontWeight: "bold",
  marginBottom: 5
},
cashierLabel: {
  fontSize: 12,
  color: "#333",
  marginBottom: 10
},
receiptRow: {
  flexDirection: "row",
  justifyContent: "space-between",
  marginVertical: 3,
  width: "100%"
},
receiptItem: {
  flex: 2,
  fontSize: 14
},
receiptQty: {
  width: 30,
  textAlign: "center",
  fontSize: 14
},
receiptPrice: {
  width: 60,
  textAlign: "right",
  fontSize: 14
},
receiptTotals: {
  borderTopWidth: 1,
  borderTopColor: "#ccc",
  width: "100%",
  marginTop: 10,
  paddingTop: 5,
  marginBottom: 10
},
receiptFooter: {
  marginTop: 10
},
receiptFooterText: {
  fontSize: 12,
  color: "#95a5a6",
  textAlign: "center",
  fontStyle: "italic"
},
printButton: {
  backgroundColor: "#AD7F65",
  paddingVertical: 15,
  paddingHorizontal: 60,
  borderRadius: 8,
  marginTop: 15,
  width: "95%",
  alignSelf: "center"
```

```
  },
  printButtonText: {
    color: "#fff",
    fontSize: 16,
    fontWeight: "600",
    textAlign: "center"
  },
  disabledButton: {
    backgroundColor: "#95a5a6",
    opacity: 0.7
  },
  emptyState: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    padding: 40
  },
  emptyText: {
    marginTop: 16,
    fontSize: 16,
    color: "#95a5a6",
    textAlign: "center"
  },
  loadingContainer: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    padding: 32
  },
  loadingText: {
    marginTop: 12,
    color: "#666",
    fontSize: 14
  },
  locationText: {
    textAlign: "center",
    fontSize: 12,
    color: "#555",
    marginBottom: 5
  },
  // Time filter pills
  filterRow: {
    marginBottom: 8
  },
  filterPill: {
    paddingHorizontal: 16,
    paddingVertical: 7,
    borderRadius: 20,
    backgroundColor: "#f3f4f6",
    marginRight: 8
  },
  filterPillActive: {
    backgroundColor: "#1f1f1f"
  },
  filterPillText: {
    fontSize: 13,
    fontWeight: "600",
    color: "#6b7280"
  },
  filterPillTextActive: {
    color: "#fff"
  }
});



FILE: pos/mobile/app/About.jsx
================================================================================
import { useNavigation } from "@react-navigation/native";
import { Image, StyleSheet, Text, View } from "react-native";
export default function About() {
  const navigation = useNavigation();
  return /* @__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */
React.createElement(Text, { style: styles.spacer }, "The Create Your Style Tracking App is a simple and
efficient system designed to help our thrift and clothing shop manage daily reports, inventory concerns, and
shop updates in one organized place. It streamlines how staff record issues, track stock movement, and
communicate important shop activities, making operations faster, clearer, and more reliable. With this app,"),
```

```
/* @__PURE__ */ React.createElement(Text, { style: styles.spacer2 }, "we aim to improve coordination, reduce
errors, and maintain smooth store management\u2014all while supporting the overall mission of delivering a
better and more organized shopping experience for our customers."), /* @__PURE__ */ React.createElement(View,
{ style: styles.logoContainer }, /* @__PURE__ */ React.createElement(
    Image,
    {
      source: require("./(tabs)/iconz/AAAAAA.png"),
      style: styles.logo,
      resizeMode: "contain"
    }
  ), /* @__PURE__ */ React.createElement(
    Image,
    {
      source: require("./(tabs)/iconz/profile1.png"),
      style: styles.logo,
      resizeMode: "contain"
    }
  )));
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#FAF6F2",
    paddingHorizontal: 20,
    paddingTop: 20
  },
  spacer: {
    fontSize: 16,
    fontWeight: "400",
    lineHeight: 20
  },
  spacer2: {
    marginTop: 10,
    fontSize: 16,
    fontWeight: "400",
    lineHeight: 20
  },
  logoContainer: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    marginTop: 30,
    marginLeft: 30,
    marginRight: 30
  },
  logo: {
    width: 150,
    height: 100
  }
});
```

FILE: pos/mobile/app/AddItem.jsx
==========

```
import { Ionicons } from "@expo/vector-icons";
import AsyncStorage from "@react-native-async-storage/async-storage";
import DateTimePicker from "@react-native-community/datetimepicker";
import * as ExpoFileSystem from "expo-file-system/legacy";
import * as ImageManipulator from "expo-image-manipulator";
import * as ImagePicker from "expo-image-picker";
import { useRouter } from "expo-router";
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import {
  ActivityIndicator,
  Alert,
  Animated,
  BackHandler,
  Image,
  Modal,
  Platform,
  Pressable,
  ScrollView,
  StyleSheet,
  Switch,
  Text,
  TextInput,
```

```
  TouchableOpacity,
  View
} from "react-native";
import AddBrandModal from "../components/AddBrandModal";
import AddCategoryModal from "../components/AddCategoryModal";
import FormSelect from "../components/FormSelect";
import {
  COMMON_COLORS,
  VARIANT_ONLY_KEY,
  UNIT_OPTIONS,
  buildCreateProductPayload,
  buildEditProductPayload,
  generateSKU,
  getCustomSubCategories,
  getSizeOptions,
  getSubcategories,
  inferParentSubFromProduct,
  needsConfirmModal,
  parentCategories
} from "../constants/addProductWebParity";
import { brandPartnerAPI, categoryAPI, productAPI } from "../services/api";
const ADD_PRODUCT_DRAFT_KEY = "addProductFormDraftMobile";
const PLACEHOLDER_PH = "#9ca3af";
const STEPS = [
  { id: 1, label: "Basic Info" },
  { id: 2, label: "Variants" },
  { id: 3, label: "Stock and Price" },
  { id: 4, label: "Batch" },
  { id: 5, label: "Review" }
];
const defaultForm = () => ({
  sku: "",
  itemName: "",
  category: "",
  subCategory: "",
  unitOfMeasure: "pcs",
  brandName: "Default",
  variant: "",
  size: "",
  itemPrice: "",
  costPrice: "",
  currentStock: "",
  reorderNumber: "",
  supplierName: "",
  supplierContact: "",
  itemImage: "",
  selectedSizes: [],
  sizeQuantities: {},
  sizePrices: {},
  sizeCostPrices: {},
  differentPricesPerSize: false,
  foodSubtype: "",
  displayInTerminal: true,
  expirationDate: "",
  /** Opening batch — aligns with web Add Product step 4 */
  dateReceived: "",
  batchNumber: ""
});
async function convertImageToBase64(uri) {
  try {
    if (!uri) return "";
    if (uri.startsWith("data:image")) return uri;
    const manipulatedResult = await ImageManipulator.manipulateAsync(
      uri,
      [{ resize: { width: 1024 } }],
      { compress: 0.6, format: ImageManipulator.SaveFormat.JPEG }
    );
    if (!ExpoFileSystem || typeof ExpoFileSystem.readAsStringAsync !== "function") {
      return uri;
    }
    const encodingOptions = ExpoFileSystem.EncodingType ? { encoding: ExpoFileSystem.EncodingType.Base64 } :
{ encoding: "base64" };
    const base64 = await ExpoFileSystem.readAsStringAsync(
      manipulatedResult.uri,
      encodingOptions
    );
    return `${base64}`;
```

```
  } catch (error) {
    console.error("Error converting image:", error);
    return uri || "";
  }
}
function buildEditableSizePricesFromProduct(item) {
  if (!item?.sizes || typeof item.sizes !== "object") return {};
  const sizePrices = {};
  Object.entries(item.sizes).forEach(([size, sizeData]) => {
    if (typeof sizeData === "object" && sizeData !== null) {
      if (sizeData.variants && typeof sizeData.variants === "object") {
        sizePrices[size] = {
          hasVariants: true,
          basePrice: sizeData.price || item.itemPrice || 0,
          baseCostPrice: sizeData.costPrice || item.costPrice || 0,
          variants: {}
        };
        Object.entries(sizeData.variants).forEach(([variant, variantData]) => {
          if (typeof variantData === "object" && variantData !== null) {
            sizePrices[size].variants[variant] = {
              price: variantData.price || sizeData.variantPrices?.[variant] || sizeData.price ||
item.itemPrice || 0,
              costPrice: variantData.costPrice || sizeData.variantCostPrices?.[variant] || sizeData.costPrice
|| item.costPrice || 0,
              quantity: variantData.quantity || 0
            };
          } else {
            sizePrices[size].variants[variant] = {
              price: sizeData.variantPrices?.[variant] || sizeData.price || item.itemPrice || 0,
              costPrice: sizeData.variantCostPrices?.[variant] || sizeData.costPrice || item.costPrice || 0,
              quantity: typeof variantData === "number" ? variantData : 0
            };
          }
        });
      } else {
        sizePrices[size] = {
          hasVariants: false,
          price: sizeData.price || item.itemPrice || 0,
          costPrice: sizeData.costPrice || item.costPrice || 0,
          quantity: sizeData.quantity || 0
        };
      }
    } else if (typeof sizeData === "number") {
      sizePrices[size] = {
        hasVariants: false,
        price: item.itemPrice || 0,
        costPrice: item.costPrice || 0,
        quantity: sizeData
      };
    }
  });
  return sizePrices;
}
export default function AddItem({ onBack, item, isEditing = false }) {
  const router = useRouter();
  const [form, setForm] = useState(defaultForm);
  const [currentStep, setCurrentStep] = useState(1);
  const [productImageUris, setProductImageUris] = useState([]);
  const [selectedVariants, setSelectedVariants] = useState([]);
  const [showVariantDropdown, setShowVariantDropdown] = useState(false);
  const [showSizeDropdown, setShowSizeDropdown] = useState(false);
  const [customSizes, setCustomSizes] = useState([]);
  const [customColorInput, setCustomColorInput] = useState("");
  const [customSizeValue, setCustomSizeValue] = useState("");
  const [optionGroup1Name, setOptionGroup1Name] = useState("Color");
  const [optionGroup2Name, setOptionGroup2Name] = useState("Size");
  const [variantQuantities, setVariantQuantities] = useState({});
  const [variantPrices, setVariantPrices] = useState({});
  const [variantCostPrices, setVariantCostPrices] = useState({});
  const [fillAllCost, setFillAllCost] = useState("");
  const [fillAllPrice, setFillAllPrice] = useState("");
  const [fillAllQty, setFillAllQty] = useState("");
  const [reviewImgIdx, setReviewImgIdx] = useState(0);
  const [apiCategoryNames, setApiCategoryNames] = useState([]);
  const [brandsList, setBrandsList] = useState([]);
  const [showAddCategoryModal, setShowAddCategoryModal] = useState(false);
  const [showAddBrandModal, setShowAddBrandModal] = useState(false);
```

```
  const [categoryModalForSub, setCategoryModalForSub] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [showSuccess, setShowSuccess] = useState(false);
  const [successMessage, setSuccessMessage] = useState("");
  const [showDiscardModal, setShowDiscardModal] = useState(false);
  const [showConfirmModal, setShowConfirmModal] = useState(false);
  const [editableSizePrices, setEditableSizePrices] = useState({});
  const [showExpiryPicker, setShowExpiryPicker] = useState(false);
  const [showDateReceivedPicker, setShowDateReceivedPicker] = useState(false);
  const toastTranslate = useRef(new Animated.Value(-60)).current;
  const toastOpacity = useRef(new Animated.Value(0)).current;
  const customSubCategories = useMemo(
    () => getCustomSubCategories(apiCategoryNames),
    [apiCategoryNames]
  );
  const subcategoryOptions = useMemo(
    () => getSubcategories(
      form.category,
      customSubCategories,
      form.subCategory,
      form.category
    ),
    [form.category, form.subCategory, customSubCategories]
  );
  const partnerNames = useMemo(() => {
    const names = Array.from(
      new Set(brandsList.map((b) => b.brandName).filter(Boolean))
    ).sort((a, b) => a.localeCompare(b));
    return names;
  }, [brandsList]);
  const legacyBrandSelected = form.brandName && form.brandName !== "Default"
&& !partnerNames.includes(form.brandName);
  const categorySelectItems = useMemo(
    () => [
      { label: "Select category", value: "" },
      ...parentCategories.map((c) => ({ label: c, value: c }))
    ],
    []
  );
  const subcategorySelectItems = useMemo(
    () => [
      { label: "Select subcategory", value: "" },
      ...subcategoryOptions.map((s) => ({ label: s, value: s })),
      { label: "+ Add Subcategory", value: "__add_new__" }
    ],
    [subcategoryOptions]
  );
  const brandSelectItems = useMemo(() => {
    const items = [
      { label: "Default", value: "Default" },
      ...partnerNames.map((n) => ({ label: n, value: n })),
      { label: "+ Add Brand", value: "__add_new__" }
    ];
    if (legacyBrandSelected) {
      items.push({
        label: `${form.brandName} (Inactive)`,
        value: form.brandName
      });
    }
    return items;
  }, [partnerNames, legacyBrandSelected, form.brandName]);
  const unitSelectItems = useMemo(
    () => [
      { label: "Select unit", value: "" },
      ...UNIT_OPTIONS.map((u) => ({ label: u.label, value: u.value }))
    ],
    []
  );
  const variantStr = useMemo(
    () => selectedVariants.filter(Boolean).join(", "),
    [selectedVariants]
  );
  const hasVariants = selectedVariants.length > 0 || form.selectedSizes?.length > 0;
  const sizeOptions = useMemo(
    () => getSizeOptions(form.category, form.subCategory, customSizes),
    [form.category, form.subCategory, customSizes]
  );
```

```
  const combos = useMemo(() => {
    const list = [];
    const variants = selectedVariants.length > 0 ? selectedVariants : [null];
    const sizes = form.selectedSizes?.length > 0 ? form.selectedSizes : [VARIANT_ONLY_KEY];
    variants.forEach((v) => {
      sizes.forEach((s) => {
        list.push({
          variant: v,
          size: s,
          key: `${v || ""}-${s || ""}`
        });
      });
    });
    return list;
  }, [selectedVariants, form.selectedSizes]);
  const hasAnyCombos = hasVariants && combos.length > 0 && (combos[0].variant != null || combos[0].size !=
null);
  const generatedBatchNumber = useMemo(() => {
    const now = /* @__PURE__ */ new Date();
    const y = now.getFullYear();
    const m = String(now.getMonth() + 1).padStart(2, "0");
    return `B${y}${m} \u2013 001`;
  }, []);
  const fetchMeta = useCallback(async () => {
    try {
      const [catRes, brandRes] = await Promise.all([
        categoryAPI.getAll(),
        brandPartnerAPI.getAll()
      ]);
      if (catRes.success && Array.isArray(catRes.data)) {
        setApiCategoryNames(
          catRes.data.filter((c) => c.status === "active" && c.name !== "Others").map((c) => c.name)
        );
      }
      if (brandRes.success && brandRes.data) {
        setBrandsList(brandRes.data);
      }
    } catch (e) {
      console.error("fetchMeta", e);
    }
  }, []);
  useEffect(() => {
    fetchMeta();
  }, [fetchMeta]);
  useEffect(() => {
    if (isEditing) return;
    setForm((f) => ({
      ...f,
      variant: variantStr,
      sku: generateSKU(f.category, f.subCategory, variantStr)
    }));
  }, [isEditing, form.category, form.subCategory, variantStr]);
  useEffect(() => {
    if (isEditing && item) {
      const { category, subCategory } = inferParentSubFromProduct(item);
      setForm({
        ...defaultForm(),
        sku: item.sku || "",
        itemName: item.itemName || item.name || "",
        category,
        subCategory,
        unitOfMeasure: item.unitOfMeasure || "pcs",
        brandName: item.brandName || item.brand || "Default",
        variant: item.variant || "",
        size: item.size || "",
        itemPrice: item.itemPrice !== void 0 ? String(item.itemPrice) : item.price != null ?
String(item.price) : "",
        costPrice: item.costPrice !== void 0 ? String(item.costPrice) : "",
        currentStock: item.currentStock !== void 0 ? String(item.currentStock) : item.stock != null ?
String(item.stock) : "",
        reorderNumber: item.reorderNumber !== void 0 ? String(item.reorderNumber) : "",
        supplierName: item.supplierName || "",
        supplierContact: item.supplierContact || "",
        itemImage: item.itemImage || item.image || "",
        foodSubtype: item.foodSubtype || "",
        displayInTerminal: item.displayInTerminal !== void 0 ? item.displayInTerminal : true,
        expirationDate: item.expirationDate ? new Date(item.expirationDate).toISOString().slice(0, 10) : "",
```

```
        batchNumber: item.batchNumber || "",
        selectedSizes: [],
        sizeQuantities: {},
        sizePrices: {},
        sizeCostPrices: {}
      });
      if (item.variant) {
        setSelectedVariants(
          item.variant.split(", ").map((v) => v.trim()).filter(Boolean)
        );
      } else {
        setSelectedVariants([]);
      }
      const imgs = Array.isArray(item.productImages) && item.productImages.length > 0 ?
item.productImages.filter(Boolean) : item.itemImage ? [item.itemImage] : [];
      setProductImageUris(imgs);
      setEditableSizePrices(buildEditableSizePricesFromProduct(item));
      return;
    }
    if (!isEditing) {
      (async () => {
        try {
          const raw = await AsyncStorage.getItem(ADD_PRODUCT_DRAFT_KEY);
          if (raw) {
            const parsed = JSON.parse(raw);
            if (parsed.form) setForm((f) => ({ ...f, ...parsed.form }));
            if (parsed.selectedVariants)
              setSelectedVariants(parsed.selectedVariants);
            if (parsed.variantQuantities)
              setVariantQuantities(parsed.variantQuantities);
            if (parsed.variantPrices) setVariantPrices(parsed.variantPrices);
            if (parsed.variantCostPrices)
              setVariantCostPrices(parsed.variantCostPrices);
            if (parsed.productImageUris)
              setProductImageUris(parsed.productImageUris);
            if (parsed.customSizes) setCustomSizes(parsed.customSizes);
            if (parsed.currentStep) setCurrentStep(parsed.currentStep);
          }
        } catch (e) {
          console.error("draft load", e);
        }
      })();
    }
  }, [isEditing, item]);
  useEffect(() => {
    if (isEditing || !form.itemName) return;
    const t = setTimeout(() => {
      AsyncStorage.setItem(
        ADD_PRODUCT_DRAFT_KEY,
        JSON.stringify({
          form,
          selectedVariants,
          variantQuantities,
          variantPrices,
          variantCostPrices,
          productImageUris,
          customSizes,
          currentStep
        })
      ).catch(() => {
      });
    }, 400);
    return () => clearTimeout(t);
  }, [
    form,
    selectedVariants,
    variantQuantities,
    variantPrices,
    variantCostPrices,
    productImageUris,
    customSizes,
    currentStep,
    isEditing
  ]);
  useEffect(() => {
    if (!showSuccess) {
      Animated.parallel([
```

```
        Animated.timing(toastTranslate, {
          toValue: -60,
          duration: 220,
          useNativeDriver: true
        }),
        Animated.timing(toastOpacity, {
          toValue: 0,
          duration: 220,
          useNativeDriver: true
        })
      ]).start();
      return;
    }
    Animated.parallel([
      Animated.timing(toastTranslate, {
        toValue: 0,
        duration: 280,
        useNativeDriver: true
      }),
      Animated.timing(toastOpacity, {
        toValue: 1,
        duration: 280,
        useNativeDriver: true
      })
    ]).start();
    const timer = setTimeout(() => setShowSuccess(false), 2e3);
    return () => clearTimeout(timer);
  }, [showSuccess, toastTranslate, toastOpacity]);
  const handleBack = () => {
    if (isEditing) {
      if (typeof onBack === "function") onBack();
      else router.back();
      return;
    }
    const dirty = !!form.itemName || !!form.category || selectedVariants.length > 0 ||
productImageUris.length > 0;
    if (dirty) setShowDiscardModal(true);
    else if (typeof onBack === "function") onBack();
    else router.back();
  };
  useEffect(() => {
    const sub = BackHandler.addEventListener("hardwareBackPress", () => {
      handleBack();
      return true;
    });
    return () => sub.remove();
  }, [form.itemName, form.category, selectedVariants.length, productImageUris.length, isEditing]);
  const handleSizeToggle = (size) => {
    setForm((prev) => {
      const isSelected = prev.selectedSizes.includes(size);
      const newSelectedSizes = isSelected ? prev.selectedSizes.filter((s) => s !== size) :
[...prev.selectedSizes, size];
      const newSizeQuantities = { ...prev.sizeQuantities };
      const newSizePrices = { ...prev.sizePrices };
      const newSizeCostPrices = { ...prev.sizeCostPrices };
      if (isSelected) {
        delete newSizeQuantities[size];
        delete newSizePrices[size];
        delete newSizeCostPrices[size];
      } else {
        newSizeQuantities[size] = 0;
        if (prev.differentPricesPerSize) {
          newSizePrices[size] = prev.itemPrice || "";
          newSizeCostPrices[size] = prev.costPrice || "";
        }
      }
      return {
        ...prev,
        selectedSizes: newSelectedSizes,
        sizeQuantities: newSizeQuantities,
        sizePrices: newSizePrices,
        sizeCostPrices: newSizeCostPrices
      };
    });
  };
  const setSizeQty = (size, qty) => {
    const n = parseInt(qty, 10) || 0;
```

```
  setForm((p) => ({
    ...p,
    sizeQuantities: { ...p.sizeQuantities, [size]: n }
  }));
};
const setSizePrice = (size, price) => {
  setForm((p) => ({
    ...p,
    sizePrices: { ...p.sizePrices, [size]: price }
  }));
};
const setSizeCost = (size, cost) => {
  setForm((p) => ({
    ...p,
    sizeCostPrices: { ...p.sizeCostPrices, [size]: cost }
  }));
};
const setVariantQty = (size, variant, qty) => {
  const n = parseInt(qty, 10) || 0;
  setVariantQuantities((prev) => ({
    ...prev,
    [size]: { ...prev[size] || {}, [variant]: n }
  }));
};
const setVPrice = (size, variant, price) => {
  const v = parseFloat(price) || 0;
  setVariantPrices((prev) => ({
    ...prev,
    [size]: { ...prev[size] || {}, [variant]: v }
  }));
};
const setVCost = (size, variant, price) => {
  const v = parseFloat(price) || 0;
  setVariantCostPrices((prev) => ({
    ...prev,
    [size]: { ...prev[size] || {}, [variant]: v }
  }));
};
const handleFillAll = () => {
  if (!hasAnyCombos) return;
  combos.forEach(({ variant: v, size: s }) => {
    if (v && s) {
      if (fillAllCost) setVCost(s, v, fillAllCost);
      if (fillAllPrice) setVPrice(s, v, fillAllPrice);
      if (fillAllQty) setVariantQty(s, v, fillAllQty);
    } else if (s && !v) {
      if (fillAllCost) setSizeCost(s, fillAllCost);
      if (fillAllPrice) setSizePrice(s, fillAllPrice);
      if (fillAllQty) setSizeQty(s, fillAllQty);
    } else if (v && !s) {
      if (fillAllCost)
        setForm((p) => ({ ...p, costPrice: fillAllCost }));
      if (fillAllPrice)
        setForm((p) => ({ ...p, itemPrice: fillAllPrice }));
      if (fillAllQty)
        setForm((p) => ({ ...p, currentStock: fillAllQty }));
    }
  });
};
const totalReviewStock = useMemo(() => {
  if (!hasAnyCombos) return parseInt(form.currentStock, 10) || 0;
  let t = 0;
  combos.forEach(({ variant: v, size: s }) => {
    if (v && s) {
      t += parseInt(variantQuantities[s]?.[v], 10) || 0;
    } else if (s) {
      t += parseInt(form.sizeQuantities[s], 10) || 0;
    } else {
      t += parseInt(form.currentStock, 10) || 0;
    }
  });
  return t;
}, [
  hasAnyCombos,
  combos,
  variantQuantities,
  form.sizeQuantities,
```

```
    form.currentStock
  ]);
  const isStepValid = (step) => {
    if (isEditing) return true;
    switch (step) {
      case 1:
        if (!form.itemName?.trim()) return false;
        if (!form.category) return false;
        if (!form.subCategory || form.subCategory === "__add_new__")
          return false;
        if (!form.brandName) return false;
        if (!form.unitOfMeasure) return false;
        return true;
      case 2:
        return selectedVariants.length > 0;
      case 3:
        if (hasVariants) return true;
        if (!form.itemPrice || parseFloat(form.itemPrice) <= 0) return false;
        if (!form.costPrice || parseFloat(form.costPrice) <= 0) return false;
        if (!form.currentStock || parseInt(form.currentStock, 10) <= 0)
          return false;
        return true;
      case 4:
        if (totalReviewStock > 0 && !form.dateReceived?.trim()) return false;
        return true;
      default:
        return true;
    }
  };
  const validatePreConfirm = (np) => {
    const hasSizeQuantities = np.selectedSizes?.length > 0 && Object.values(np.sizeQuantities ||
{}).some((qty) => parseInt(qty, 10) > 0);
    const hasStock = parseInt(np.currentStock, 10) > 0;
    const hasVariantQuantities = np.variantQuantities && Object.keys(np.variantQuantities).length > 0 &&
Object.values(np.variantQuantities).some(
      (sizeVariants) => sizeVariants && typeof sizeVariants === "object" &&
Object.values(sizeVariants).some((qty) => parseInt(qty, 10) > 0)
    );
    if (!hasSizeQuantities && !hasVariantQuantities && !hasStock) {
      Alert.alert(
        "Validation",
        "Please either select sizes with quantities or provide a stock value."
      );
      return false;
    }
    if (needsConfirmModal(np)) {
      return true;
    }
    if (np.differentPricesPerSize && np.selectedSizes?.length > 0) {
      const invalidSizes = np.selectedSizes.filter((size) => {
        const price = np.sizePrices?.[size];
        return !price || price === "" || parseFloat(price) <= 0;
      });
      if (invalidSizes.length > 0) {
        Alert.alert(
          "Validation",
          `Please enter prices for all selected sizes: ${invalidSizes.join(", ")}`
        );
        return false;
      }
    } else if (!np.differentPricesPerSize) {
      if (!np.itemPrice || parseFloat(np.itemPrice) <= 0) {
        Alert.alert("Validation", "Please enter a selling price.");
        return false;
      }
    }
    return true;
  };
  const mergeSubmitProduct = async () => {
    const images = [];
    for (const uri of productImageUris) {
      if (uri.startsWith("http") || uri.startsWith("data:")) {
        images.push(uri);
      } else {
        images.push(await convertImageToBase64(uri));
      }
    }
```

```
  const np = {
    ...form,
    variant: variantStr,
    foodSubtype: form.category === "Foods" ? form.subCategory || "" : "",
    variantQuantities,
    variantPrices,
    variantCostPrices,
    differentPricesPerVariant: {},
    productImages: images,
    itemImage: images[0] || "",
    batchNumber: generatedBatchNumber,
    optionGroup1Name: optionGroup1Name || "Color",
    ...form.selectedSizes?.length > 0 ? { optionGroup2Name: optionGroup2Name || "Size" } : {}
  };
  return np;
};
const runCreateOrUpdate = async () => {
  setIsLoading(true);
  try {
    const np = await mergeSubmitProduct();
    if (isEditing && (item?._id || item?.id)) {
      const editPayload = buildEditProductPayload(
        { ...np, editableSizePrices },
        item
      );
      delete editPayload.isForPOS;
      const res2 = await productAPI.update(item._id || item.id, editPayload);
      if (!res2.success) throw new Error(res2.message || "Update failed");
      setSuccessMessage("Item updated successfully!");
      setShowSuccess(true);
      setTimeout(() => {
        AsyncStorage.removeItem(ADD_PRODUCT_DRAFT_KEY).catch(() => {
        });
        if (typeof onBack === "function") onBack();
        else router.back();
      }, 2200);
      return;
    }
    const payload = buildCreateProductPayload(np, false);
    delete payload.isForPOS;
    const res = await productAPI.create(payload);
    if (!res.success) throw new Error(res.message || "Create failed");
    setSuccessMessage("Item added successfully!");
    setShowSuccess(true);
    await AsyncStorage.removeItem(ADD_PRODUCT_DRAFT_KEY);
    setTimeout(() => {
      if (typeof onBack === "function") onBack();
      else router.back();
    }, 2200);
  } catch (e) {
    console.error(e);
    Alert.alert("Error", e.message || "Failed to save");
  } finally {
    setIsLoading(false);
    setShowConfirmModal(false);
  }
};
const onPressSubmit = async () => {
  if (isLoading) return;
  const np = await mergeSubmitProduct();
  if (!validatePreConfirm(np)) return;
  if (needsConfirmModal(np)) {
    setShowConfirmModal(true);
    return;
  }
  await runCreateOrUpdate();
};
const pickImage = async () => {
  const { status } = await ImagePicker.requestMediaLibraryPermissionsAsync();
  if (status !== "granted") {
    Alert.alert("Permission", "Photo library access is required.");
    return;
  }
  const result = await ImagePicker.launchImageLibraryAsync({
    mediaTypes: ImagePicker.MediaTypeOptions.Images,
    allowsMultipleSelection: true,
    quality: 0.85
```

```
  });
  if (!result.canceled && result.assets?.length) {
    setProductImageUris((prev) => [
      ...prev,
      ...result.assets.map((a) => a.uri)
    ]);
  }
};
const removeImageAt = (index) => {
  setProductImageUris((prev) => prev.filter((_, i) => i !== index));
};
const toggleVariant = (color) => {
  setSelectedVariants(
    (prev) => prev.includes(color) ? prev.filter((x) => x !== color) : [...prev, color]
  );
};
const addCustomColor = () => {
  const t = customColorInput.trim();
  if (t && !selectedVariants.includes(t)) {
    setSelectedVariants((p) => [...p, t]);
    setCustomColorInput("");
  }
};
const renderEditMode = () => /* @__PURE__ */ React.createElement(
  ScrollView,
  {
    style: styles.scrollView,
    contentContainerStyle: styles.scrollContainer,
    keyboardShouldPersistTaps: "handled"
  },
  /* @__PURE__ */ React.createElement(Text, { style: styles.sectionTitle }, "Edit Product"),
  /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Product Name *"),
  /* @__PURE__ */ React.createElement(
    TextInput,
    {
      style: styles.input,
      placeholderTextColor: PLACEHOLDER_PH,
      placeholder: "Enter product name",
      value: form.itemName,
      onChangeText: (t) => setForm((f) => ({ ...f, itemName: t })),
      underlineColorAndroid: "transparent"
    }
  ),
  /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Category *"),
  /* @__PURE__ */ React.createElement(
    FormSelect,
    {
      items: categorySelectItems,
      selectedValue: form.category,
      onValueChange: (v) => setForm((f) => ({ ...f, category: v, subCategory: "" }))
    }
  ),
  /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Subcategory *"),
  /* @__PURE__ */ React.createElement(
    FormSelect,
    {
      items: subcategorySelectItems,
      selectedValue: form.subCategory,
      enabled: !!form.category,
      onValueChange: (v) => {
        if (v === "__add_new__") {
          setCategoryModalForSub(true);
          setShowAddCategoryModal(true);
          return;
        }
        setForm((f) => ({ ...f, subCategory: v }));
      }
    }
  ),
  /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Brand Partner *"),
  /* @__PURE__ */ React.createElement(
    FormSelect,
    {
      items: brandSelectItems,
      selectedValue: form.brandName,
      onValueChange: (v) => {
        if (v === "__add_new__") {
```

```
            setShowAddBrandModal(true);
            return;
          }
          setForm((f) => ({ ...f, brandName: v }));
        }
      }
    ),
    /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Unit of measure *"),
    /* @__PURE__ */ React.createElement(
      FormSelect,
      {
        items: unitSelectItems,
        selectedValue: form.unitOfMeasure,
        onValueChange: (v) => setForm((f) => ({ ...f, unitOfMeasure: v }))
      }
    ),
    /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Cost / Selling (simple product)"),
    /* @__PURE__ */ React.createElement(View, { style: styles.row2 }, /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: [styles.input, styles.flex1],
        keyboardType: "decimal-pad",
        placeholderTextColor: PLACEHOLDER_PH,
        placeholder: "Cost (\u20B1)",
        value: String(form.costPrice),
        onChangeText: (t) => setForm((f) => ({ ...f, costPrice: t })),
        underlineColorAndroid: "transparent"
      }
    ), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: [styles.input, styles.flex1],
        keyboardType: "decimal-pad",
        placeholderTextColor: PLACEHOLDER_PH,
        placeholder: "Selling price (\u20B1)",
        value: String(form.itemPrice),
        onChangeText: (t) => setForm((f) => ({ ...f, itemPrice: t })),
        underlineColorAndroid: "transparent"
      }
    )),
    /* @__PURE__ */ React.createElement(View, { style: styles.switchRow }, /* @__PURE__ */
React.createElement(Text, null, "Display in Terminal"), /* @__PURE__ */ React.createElement(
      Switch,
      {
        value: form.displayInTerminal !== false,
        onValueChange: (v) => setForm((f) => ({ ...f, displayInTerminal: v }))
      }
    )),
    Object.keys(editableSizePrices).length > 0 && /* @__PURE__ */ React.createElement(View, { style:
{ marginTop: 16 } }, /* @__PURE__ */ React.createElement(Text, { style: styles.sectionTitle }, "Size &
variant prices"), Object.entries(editableSizePrices).map(([size, sd]) => /* @__PURE__ */
React.createElement(View, { key: size, style: styles.card }, /* @__PURE__ */ React.createElement(Text,
{ style: styles.bold }, size), sd.hasVariants ? Object.entries(sd.variants || {}).map(([vr, vd]) => /*
@__PURE__ */ React.createElement(View, { key: vr, style: styles.row2 }, /* @__PURE__ */
React.createElement(Text, { style: styles.flex1 }, vr), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: [styles.input, styles.flex1],
        keyboardType: "decimal-pad",
        value: String(vd.costPrice ?? ""),
        onChangeText: (t) => setEditableSizePrices((p) => ({
          ...p,
          [size]: {
            ...p[size],
            variants: {
              ...p[size].variants,
              [vr]: { ...p[size].variants[vr], costPrice: t }
            }
          }
        }))
      }
    ), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: [styles.input, styles.flex1],
        keyboardType: "decimal-pad",
        value: String(vd.price ?? ""),
```

```
        onChangeText: (t) => setEditableSizePrices((p) => ({
          ...p,
          [size]: {
            ...p[size],
            variants: {
              ...p[size].variants,
              [vr]: { ...p[size].variants[vr], price: t }
            }
          }
        }))
      }
    ))) : /* @__PURE__ */ React.createElement(View, { style: styles.row2 }, /* @__PURE__ */
React.createElement(
      TextInput,
      {
        style: [styles.input, styles.flex1],
        keyboardType: "decimal-pad",
        placeholderTextColor: PLACEHOLDER_PH,
        placeholder: "Cost",
        value: String(sd.costPrice ?? ""),
        onChangeText: (t) => setEditableSizePrices((p) => ({
          ...p,
          [size]: { ...p[size], costPrice: t }
        })),
        underlineColorAndroid: "transparent"
      }
    ), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: [styles.input, styles.flex1],
        keyboardType: "decimal-pad",
        placeholderTextColor: PLACEHOLDER_PH,
        placeholder: "Price",
        value: String(sd.price ?? ""),
        onChangeText: (t) => setEditableSizePrices((p) => ({
          ...p,
          [size]: { ...p[size], price: t }
        })),
        underlineColorAndroid: "transparent"
      }
    )))),
    /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: styles.primaryBtn,
        onPress: onPressSubmit,
        disabled: isLoading
      },
      isLoading ? /* @__PURE__ */ React.createElement(ActivityIndicator, { color: "#fff" }) : /* @__PURE__ */
React.createElement(Text, { style: styles.primaryBtnText }, "Update Product")
    )
  );
  const renderStepContent = () => {
    if (currentStep === 1) {
      return /* @__PURE__ */ React.createElement(View, null, /* @__PURE__ */ React.createElement(Text,
{ style: styles.label }, "Product Name *"), /* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: styles.input,
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "Enter product name",
          value: form.itemName,
          onChangeText: (t) => setForm((f) => ({ ...f, itemName: t })),
          underlineColorAndroid: "transparent"
        }
      ), /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Category *"), /* @__PURE__ */
React.createElement(
        FormSelect,
        {
          items: categorySelectItems,
          selectedValue: form.category,
          onValueChange: (v) => setForm((f) => ({ ...f, category: v, subCategory: "" }))
        }
      ), /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Subcategory *"), /* @__PURE__ */
React.createElement(
        FormSelect,
        {
```

```
          items: subcategorySelectItems,
          selectedValue: form.subCategory,
          enabled: !!form.category,
          onValueChange: (v) => {
            if (v === "__add_new__") {
              setCategoryModalForSub(true);
              setShowAddCategoryModal(true);
              return;
            }
            setForm((f) => ({ ...f, subCategory: v }));
            setSelectedVariants([]);
            setVariantQuantities({});
            setVariantPrices({});
            setVariantCostPrices({});
          }
        }
      ), /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Brand Partner *"), /* @__PURE__
*/ React.createElement(
        FormSelect,
        {
          items: brandSelectItems,
          selectedValue: form.brandName,
          onValueChange: (v) => {
            if (v === "__add_new__") {
              setShowAddBrandModal(true);
              return;
            }
            setForm((f) => ({ ...f, brandName: v }));
          }
        }
      ), /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Unit of measure *"), /*
@__PURE__ */ React.createElement(
        FormSelect,
        {
          items: unitSelectItems,
          selectedValue: form.unitOfMeasure,
          onValueChange: (v) => setForm((f) => ({ ...f, unitOfMeasure: v }))
        }
      ), /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Product images"), /* @__PURE__
*/ React.createElement(ScrollView, { horizontal: true, showsHorizontalScrollIndicator: false }, /* @__PURE__
*/ React.createElement(TouchableOpacity, { style: styles.addImgBox, onPress: pickImage }, /* @__PURE__ */
React.createElement(Ionicons, { name: "add", size: 28, color: "#6b7280" })), productImageUris.map((uri, i) =>
/* @__PURE__ */ React.createElement(View, { key: i, style: styles.thumbWrap }, /* @__PURE__ */
React.createElement(Image, { source: { uri }, style: styles.thumb }), i === 0 && /* @__PURE__ */
React.createElement(View, { style: styles.mainBadge }, /* @__PURE__ */ React.createElement(Text, { style:
styles.mainBadgeText }, "Main")), /* @__PURE__ */ React.createElement(
        Pressable,
        {
          style: styles.thumbRemove,
          onPress: () => removeImageAt(i)
        },
        /* @__PURE__ */ React.createElement(Text, { style: { color: "#fff" } }, "\xD7")
      )))), productImageUris.length > 3 && /* @__PURE__ */ React.createElement(Text, { style: styles.warn },
"Adding more than 3 photos may slow down the system."));
    }
    if (currentStep === 2) {
      return /* @__PURE__ */ React.createElement(View, null, /* @__PURE__ */ React.createElement(Text,
{ style: styles.sectionTitle }, "Option group 1 \u2013 required"), /* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: styles.input,
          value: optionGroup1Name,
          onChangeText: setOptionGroup1Name,
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "e.g. Color",
          underlineColorAndroid: "transparent"
        }
      ), /* @__PURE__ */ React.createElement(
        TouchableOpacity,
        {
          style: styles.dropdown,
          onPress: () => setShowVariantDropdown(!showVariantDropdown)
        },
        /* @__PURE__ */ React.createElement(Text, null, selectedVariants.length === 0 ? `Select
${optionGroup1Name.toLowerCase()}\u2026` : `${selectedVariants.length} selected`),
        /* @__PURE__ */ React.createElement(
          Ionicons,
```

```
          {
            name: showVariantDropdown ? "chevron-up" : "chevron-down",
            size: 20
          }
        )
      ), showVariantDropdown && /* @__PURE__ */ React.createElement(View, { style: styles.dropdownList },
COMMON_COLORS.filter((c) => c !== "Custom").map((c) => /* @__PURE__ */ React.createElement(
        Pressable,
        {
          key: c,
          style: styles.ddItem,
          onPress: () => toggleVariant(c)
        },
        /* @__PURE__ */ React.createElement(Text, null, c),
        selectedVariants.includes(c) ? /* @__PURE__ */ React.createElement(Ionicons, { name: "checkmark",
size: 18, color: "#09A046" }) : null
      ))), /* @__PURE__ */ React.createElement(View, { style: styles.chips }, selectedVariants.map((v) => /*
@__PURE__ */ React.createElement(View, { key: v, style: styles.chip }, /* @__PURE__ */
React.createElement(Text, { style: styles.chipText }, v), /* @__PURE__ */ React.createElement(Pressable,
{ onPress: () => toggleVariant(v) }, /* @__PURE__ */ React.createElement(Text, { style: styles.chipX },
"\xD7")))), /* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: styles.chipInput,
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "Add",
          value: customColorInput,
          onChangeText: setCustomColorInput,
          onSubmitEditing: addCustomColor,
          underlineColorAndroid: "transparent"
        }
      )), /* @__PURE__ */ React.createElement(Text, { style: [styles.sectionTitle, { marginTop: 20 }] },
"Option group 2 (sizes)"), /* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: styles.input,
          value: optionGroup2Name,
          onChangeText: setOptionGroup2Name,
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "e.g. Size",
          underlineColorAndroid: "transparent"
        }
      ), /* @__PURE__ */ React.createElement(
        TouchableOpacity,
        {
          style: styles.dropdown,
          onPress: () => setShowSizeDropdown(!showSizeDropdown)
        },
        /* @__PURE__ */ React.createElement(Text, null, form.selectedSizes.length === 0 ? `Select
${optionGroup2Name.toLowerCase()}\u2026` : `${form.selectedSizes.length} selected`),
        /* @__PURE__ */ React.createElement(
          Ionicons,
          {
            name: showSizeDropdown ? "chevron-up" : "chevron-down",
            size: 20
          }
        )
      ), showSizeDropdown && /* @__PURE__ */ React.createElement(View, { style: styles.dropdownList },
sizeOptions.map((sz) => /* @__PURE__ */ React.createElement(
        Pressable,
        {
          key: sz,
          style: styles.ddItem,
          onPress: () => handleSizeToggle(sz)
        },
        /* @__PURE__ */ React.createElement(Text, null, sz),
        form.selectedSizes.includes(sz) ? /* @__PURE__ */ React.createElement(Ionicons, { name: "checkmark",
size: 18, color: "#09A046" }) : null
      ))), /* @__PURE__ */ React.createElement(View, { style: styles.chips }, form.selectedSizes.map((sz) =>
/* @__PURE__ */ React.createElement(View, { key: sz, style: styles.chip }, /* @__PURE__ */
React.createElement(Text, { style: styles.chipText }, sz), /* @__PURE__ */ React.createElement(Pressable,
{ onPress: () => handleSizeToggle(sz) }, /* @__PURE__ */ React.createElement(Text, { style: styles.chipX },
"\xD7")))), /* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: styles.chipInput,
          placeholderTextColor: PLACEHOLDER_PH,
```

```
          placeholder: "Add",
          value: customSizeValue,
          onChangeText: setCustomSizeValue,
          onSubmitEditing: () => {
            const t = customSizeValue.trim();
            if (t && !customSizes.includes(t)) {
              setCustomSizes((p) => [...p, t]);
              handleSizeToggle(t);
              setCustomSizeValue("");
            }
          },
          underlineColorAndroid: "transparent"
        }
      )));
    }
    if (currentStep === 3) {
      return /* @__PURE__ */ React.createElement(View, null, hasAnyCombos ? /* @__PURE__ */
React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(View, { style: styles.fillAll },
/* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: [styles.input, styles.fillField],
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "Cost (\u20B1)",
          keyboardType: "decimal-pad",
          value: fillAllCost,
          onChangeText: setFillAllCost,
          underlineColorAndroid: "transparent"
        }
      ), /* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: [styles.input, styles.fillField],
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "Sell (\u20B1)",
          keyboardType: "decimal-pad",
          value: fillAllPrice,
          onChangeText: setFillAllPrice,
          underlineColorAndroid: "transparent"
        }
      ), /* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: [styles.input, styles.fillField],
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "Quantity",
          keyboardType: "number-pad",
          value: fillAllQty,
          onChangeText: setFillAllQty,
          underlineColorAndroid: "transparent"
        }
      ), /* @__PURE__ */ React.createElement(
        TouchableOpacity,
        {
          style: styles.fillBtn,
          onPress: handleFillAll,
          disabled: !fillAllCost && !fillAllPrice && !fillAllQty
        },
        /* @__PURE__ */ React.createElement(Text, { style: styles.fillBtnText }, "Fill all")
      )), /* @__PURE__ */ React.createElement(Text, { style: styles.comboHeader }, "Variant \xD7 size"),
combos.map(({ variant: v, size: s, key }) => /* @__PURE__ */ React.createElement(View, { key, style:
styles.comboRow }, /* @__PURE__ */ React.createElement(Text, { style: styles.comboLabel, numberOfLines: 2 },
s && s !== VARIANT_ONLY_KEY ? `${s} ` : "", v ? `\xD7 ${v}` : "", !v && s === VARIANT_ONLY_KEY ? " (variant
only)" : ""), /* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: [styles.input, styles.comboInput],
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "Cost (\u20B1)",
          keyboardType: "decimal-pad",
          value: v && s ? String(variantCostPrices[s]?.[v] ?? "") : s && s !== VARIANT_ONLY_KEY ?
String(form.sizeCostPrices[s] ?? "") : String(form.costPrice ?? ""),
          onChangeText: (t) => {
            if (v && s) setVCost(s, v, t);
            else if (s && s !== VARIANT_ONLY_KEY)
              setSizeCost(s, t);
            else setForm((p) => ({ ...p, costPrice: t }));
```

```
          },
          underlineColorAndroid: "transparent"
        }
      ), /* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: [styles.input, styles.comboInput],
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "Sell (\u20B1)",
          keyboardType: "decimal-pad",
          value: v && s ? String(variantPrices[s]?.[v] ?? "") : s && s !== VARIANT_ONLY_KEY ?
String(form.sizePrices[s] ?? "") : String(form.itemPrice ?? ""),
          onChangeText: (t) => {
            if (v && s) setVPrice(s, v, t);
            else if (s && s !== VARIANT_ONLY_KEY)
              setSizePrice(s, t);
            else setForm((p) => ({ ...p, itemPrice: t }));
          },
          underlineColorAndroid: "transparent"
        }
      ), /* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: [styles.input, styles.comboInput],
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "Qty",
          keyboardType: "number-pad",
          value: v && s ? String(variantQuantities[s]?.[v] ?? "") : s && s !== VARIANT_ONLY_KEY ?
String(form.sizeQuantities[s] ?? "") : String(form.currentStock ?? ""),
          onChangeText: (t) => {
            if (v && s) setVariantQty(s, v, t);
            else if (s && s !== VARIANT_ONLY_KEY) setSizeQty(s, t);
            else setForm((p) => ({ ...p, currentStock: t }));
          },
          underlineColorAndroid: "transparent"
        }
      )))) : /* @__PURE__ */ React.createElement(View, null, /* @__PURE__ */ React.createElement(View,
{ style: styles.row2 }, /* @__PURE__ */ React.createElement(View, { style: styles.flex1 }, /* @__PURE__ */
React.createElement(Text, { style: styles.label }, "Cost \u20B1 *"), /* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: styles.input,
          keyboardType: "decimal-pad",
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "e.g. 100.00",
          value: String(form.costPrice),
          onChangeText: (t) => setForm((f) => ({ ...f, costPrice: t })),
          underlineColorAndroid: "transparent"
        }
      )), /* @__PURE__ */ React.createElement(View, { style: styles.flex1 }, /* @__PURE__ */
React.createElement(Text, { style: styles.label }, "Selling \u20B1 *"), /* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: styles.input,
          keyboardType: "decimal-pad",
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "e.g. 149.00",
          value: String(form.itemPrice),
          onChangeText: (t) => setForm((f) => ({ ...f, itemPrice: t })),
          underlineColorAndroid: "transparent"
        }
      ))), /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Quantity"), /* @__PURE__ */
React.createElement(
        TextInput,
        {
          style: styles.input,
          keyboardType: "number-pad",
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "Units in stock",
          value: String(form.currentStock),
          onChangeText: (t) => setForm((f) => ({ ...f, currentStock: t })),
          underlineColorAndroid: "transparent"
        }
      )), /* @__PURE__ */ React.createElement(View, { style: styles.switchRow }, /* @__PURE__ */
React.createElement(View, null, /* @__PURE__ */ React.createElement(Text, { style: styles.bold }, "Display in
Terminal"), /* @__PURE__ */ React.createElement(Text, { style: styles.hint }, "Show in POS")), /* @__PURE__
*/ React.createElement(
```

```
        Switch,
        {
          value: form.displayInTerminal !== false,
          onValueChange: (v) => setForm((f) => ({ ...f, displayInTerminal: v }))
        }
      )));
    }
    if (currentStep === 4) {
      const hasOpening = totalReviewStock > 0;
      return /* @__PURE__ */ React.createElement(View, null, /* @__PURE__ */ React.createElement(Text,
{ style: styles.label }, "Reorder level (per SKU)"), /* @__PURE__ */ React.createElement(Text, { style:
styles.hintMuted }, "System alerts when any SKU falls below this level."), /* @__PURE__ */
React.createElement(
        TextInput,
        {
          style: styles.input,
          keyboardType: "number-pad",
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "e.g. 23",
          value: String(form.reorderNumber),
          onChangeText: (t) => setForm((f) => ({ ...f, reorderNumber: t })),
          underlineColorAndroid: "transparent"
        }
      ), /* @__PURE__ */ React.createElement(Text, { style: [styles.sectionTitle, { marginTop: 16 }] },
"Opening batch"), /* @__PURE__ */ React.createElement(Text, { style: styles.hintMuted }, hasOpening ? "Batch
1 for this opening stock. Add more lots via Stock In." : "No batch until you receive stock \u2014 use Stock
In."), hasOpening ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(Text, { style: styles.batchBig }, generatedBatchNumber), /* @__PURE__ */
React.createElement(Text, { style: styles.label }, "Date received ", /* @__PURE__ */ React.createElement(Text,
{ style: styles.reqMark }, "*")), /* @__PURE__ */ React.createElement(
        Pressable,
        {
          style: styles.input,
          onPress: () => setShowDateReceivedPicker(true)
        },
        /* @__PURE__ */ React.createElement(
          Text,
          {
            style: form.dateReceived ? styles.datePressTxt : styles.datePressPlaceholder
          },
          form.dateReceived || "Select date"
        )
      ), showDateReceivedPicker && /* @__PURE__ */ React.createElement(
        DateTimePicker,
        {
          value: form.dateReceived ? /* @__PURE__ */ new Date(form.dateReceived + "T12:00:00") : /* @__PURE__
*/ new Date(),
          mode: "date",
          display: Platform.OS === "ios" ? "spinner" : "default",
          onChange: (event, d) => {
            if (Platform.OS === "android") {
              setShowDateReceivedPicker(false);
            }
            if (event?.type === "dismissed") {
              setShowDateReceivedPicker(false);
              return;
            }
            if (d) {
              setForm((f) => ({
                ...f,
                dateReceived: d.toISOString().slice(0, 10)
              }));
            }
          }
        }
      ), /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Expiration date (optional)"), /*
@__PURE__ */ React.createElement(
        Pressable,
        {
          style: styles.input,
          onPress: () => setShowExpiryPicker(true)
        },
        /* @__PURE__ */ React.createElement(
          Text,
          {
            style: form.expirationDate ? styles.datePressTxt : styles.datePressPlaceholder
          },
```

```
          form.expirationDate || "Select date (optional)"
        )
      ), showExpiryPicker && /* @__PURE__ */ React.createElement(
        DateTimePicker,
        {
          value: form.expirationDate ? /* @__PURE__ */ new Date(form.expirationDate + "T12:00:00") : /*
@__PURE__ */ new Date(),
          mode: "date",
          display: Platform.OS === "ios" ? "spinner" : "default",
          onChange: (event, d) => {
            if (Platform.OS === "android") {
              setShowExpiryPicker(false);
            }
            if (event?.type === "dismissed") {
              setShowExpiryPicker(false);
              return;
            }
            if (d) {
              setForm((f) => ({
                ...f,
                expirationDate: d.toISOString().slice(0, 10)
              }));
            }
          }
        }
      )) : null);
    }
    if (currentStep === 5) {
      const img = productImageUris[reviewImgIdx] || productImageUris[0] || form.itemImage;
      return /* @__PURE__ */ React.createElement(View, null, img ? /* @__PURE__ */ React.createElement(View,
{ style: styles.reviewImgBox }, /* @__PURE__ */ React.createElement(Image, { source: { uri: img }, style:
styles.reviewImg }), productImageUris.length > 1 && /* @__PURE__ */ React.createElement(View, { style:
styles.reviewNav }, /* @__PURE__ */ React.createElement(
        Pressable,
        {
          onPress: () => setReviewImgIdx(
            (i) => (i - 1 + productImageUris.length) % productImageUris.length
          )
        },
        /* @__PURE__ */ React.createElement(Text, { style: styles.reviewNavText }, "\u2039")
      ), /* @__PURE__ */ React.createElement(
        Pressable,
        {
          onPress: () => setReviewImgIdx(
            (i) => (i + 1) % productImageUris.length
          )
        },
        /* @__PURE__ */ React.createElement(Text, { style: styles.reviewNavText }, "\u203A")
      ))) : /* @__PURE__ */ React.createElement(Text, { style: styles.hint }, "No image"), /* @__PURE__ */
React.createElement(Text, { style: styles.reviewTitle }, form.itemName || "\u2014"), /* @__PURE__ */
React.createElement(Text, { style: styles.meta }, form.category, " \xB7 ", form.subCategory), /* @__PURE__ */
React.createElement(Text, { style: styles.meta }, "Total stock: ", totalReviewStock, " \xB7 SKUs: ",
hasAnyCombos ? combos.length : 1), /* @__PURE__ */ React.createElement(Text, { style: styles.meta }, "Batch:
", generatedBatchNumber));
    }
    return null;
  };
  return /* @__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */
React.createElement(
    AddCategoryModal,
    {
      visible: showAddCategoryModal,
      onClose: () => {
        setShowAddCategoryModal(false);
        setCategoryModalForSub(false);
      },
      onAdd: (name) => {
        fetchMeta();
        if (categoryModalForSub) {
          setForm((f) => ({ ...f, subCategory: name }));
        } else {
          setForm((f) => ({ ...f, category: name }));
        }
        setCategoryModalForSub(false);
      }
    }
  ), /* @__PURE__ */ React.createElement(
```

```
    AddBrandModal,
    {
      visible: showAddBrandModal,
      onClose: () => setShowAddBrandModal(false),
      onAdd: (name) => {
        setBrandsList((p) => [...p, { _id: String(Date.now()), brandName: name }]);
        setForm((f) => ({ ...f, brandName: name }));
      }
    }
  ), /* @__PURE__ */ React.createElement(Modal, { visible: showConfirmModal, transparent: true, animationType:
"fade" }, /* @__PURE__ */ React.createElement(View, { style: styles.modalOverlay }, /* @__PURE__ */
React.createElement(View, { style: styles.modalBox }, /* @__PURE__ */ React.createElement(Text, { style:
styles.modalTitle }, "Confirm add product"), /* @__PURE__ */ React.createElement(Text, { style:
styles.modalBody }, 'Add "', form.itemName, '" to inventory?'), /* @__PURE__ */ React.createElement(View,
{ style: styles.modalActions }, /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: styles.modalCancel,
      onPress: () => {
        setShowConfirmModal(false);
      }
    },
    /* @__PURE__ */ React.createElement(Text, null, "Cancel")
  ), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: styles.modalOk,
      onPress: runCreateOrUpdate
    },
    /* @__PURE__ */ React.createElement(Text, { style: { color: "#fff" } }, "Confirm")
  )))), /* @__PURE__ */ React.createElement(Modal, { visible: showDiscardModal, transparent: true,
animationType: "fade" }, /* @__PURE__ */ React.createElement(View, { style: styles.modalOverlay }, /*
@__PURE__ */ React.createElement(View, { style: styles.modalBox }, /* @__PURE__ */ React.createElement(Text,
{ style: styles.modalTitle }, "Discard draft?"), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: styles.modalOk,
      onPress: () => {
        AsyncStorage.removeItem(ADD_PRODUCT_DRAFT_KEY).catch(() => {
        });
        setShowDiscardModal(false);
        if (typeof onBack === "function") onBack();
        else router.back();
      }
    },
    /* @__PURE__ */ React.createElement(Text, { style: { color: "#fff" } }, "Discard")
  ), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: styles.modalCancel,
      onPress: () => setShowDiscardModal(false)
    },
    /* @__PURE__ */ React.createElement(Text, null, "Stay")
  )))), /* @__PURE__ */ React.createElement(
    Animated.View,
    {
      style: [
        styles.toast,
        {
          opacity: toastOpacity,
          transform: [{ translateY: toastTranslate }]
        }
      ]
    },
    /* @__PURE__ */ React.createElement(Text, null, successMessage)
  ), /* @__PURE__ */ React.createElement(View, { style: styles.header }, /* @__PURE__ */
React.createElement(TouchableOpacity, { onPress: handleBack }, /* @__PURE__ */ React.createElement(Ionicons,
{ name: "arrow-back", size: 24, color: "#333" })), /* @__PURE__ */ React.createElement(Text, { style:
styles.headerTitle }, isEditing ? "Edit Product" : "Add New Product")), isEditing ? renderEditMode() : /*
@__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */ React.createElement(
    ScrollView,
    {
      horizontal: true,
      showsHorizontalScrollIndicator: false,
      style: styles.stepper
    },
    STEPS.map((s) => /* @__PURE__ */ React.createElement(
```

```
      TouchableOpacity,
      {
        key: s.id,
        onPress: () => {
          if (s.id < currentStep) setCurrentStep(s.id);
        },
        style: styles.stepPill
      },
      /* @__PURE__ */ React.createElement(
        View,
        {
          style: [
            styles.stepDot,
            currentStep >= s.id && styles.stepDotOn
          ]
        },
        /* @__PURE__ */ React.createElement(Text, { style: styles.stepNum }, s.id)
      ),
      /* @__PURE__ */ React.createElement(Text, { style: styles.stepLbl }, s.label)
    ))
  ), /* @__PURE__ */ React.createElement(
    ScrollView,
    {
      style: styles.scrollView,
      contentContainerStyle: styles.scrollContainer,
      keyboardShouldPersistTaps: "handled"
    },
    renderStepContent()
  ), /* @__PURE__ */ React.createElement(View, { style: styles.footer }, /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: styles.secondaryBtn,
      onPress: () => currentStep === 1 ? handleBack() : setCurrentStep((x) => x - 1)
    },
    /* @__PURE__ */ React.createElement(Text, null, currentStep === 1 ? "Cancel" : "Back")
  ), currentStep < 5 ? /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: [
        styles.primaryBtn,
        !isStepValid(currentStep) && { opacity: 0.5 }
      ],
      disabled: !isStepValid(currentStep),
      onPress: () => setCurrentStep((x) => x + 1)
    },
    /* @__PURE__ */ React.createElement(Text, { style: styles.primaryBtnText }, "Continue")
  ) : /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: styles.primaryBtn,
      onPress: onPressSubmit,
      disabled: isLoading
    },
    isLoading ? /* @__PURE__ */ React.createElement(ActivityIndicator, { color: "#fff" }) : /* @__PURE__ */
React.createElement(Text, { style: styles.primaryBtnText }, "Add Product")
  ))));
}
const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: "#f5f5f5" },
  header: {
    flexDirection: "row",
    alignItems: "center",
    padding: 16,
    backgroundColor: "#fff",
    borderBottomWidth: 1,
    borderBottomColor: "#e5e7eb"
  },
  headerTitle: { marginLeft: 12, fontSize: 18, fontWeight: "600" },
  stepper: { maxHeight: 72, paddingVertical: 8, backgroundColor: "#fff" },
  stepPill: { alignItems: "center", marginHorizontal: 10 },
  stepDot: {
    width: 32,
    height: 32,
    borderRadius: 16,
    borderWidth: 2,
    borderColor: "#d1d5db",
    alignItems: "center",
```

```
  justifyContent: "center"
},
stepDotOn: { backgroundColor: "#09A046", borderColor: "#09A046" },
stepNum: { fontSize: 12, fontWeight: "700", color: "#374151" },
stepLbl: { fontSize: 10, marginTop: 4, maxWidth: 72, textAlign: "center" },
scrollView: { flex: 1 },
scrollContainer: { padding: 16, paddingBottom: 120 },
label: {
  fontSize: 13,
  fontWeight: "600",
  color: "#374151",
  marginBottom: 6,
  marginTop: 10
},
input: {
  borderWidth: 1,
  borderColor: "#d1d5db",
  borderRadius: 8,
  padding: 12,
  backgroundColor: "#fff",
  fontSize: 15,
  color: "#111827"
},
pickerWrap: {
  borderWidth: 1,
  borderColor: "#d1d5db",
  borderRadius: 8,
  overflow: "hidden",
  backgroundColor: "#fff"
},
sectionTitle: { fontSize: 16, fontWeight: "700", marginBottom: 8 },
row2: { flexDirection: "row", gap: 8, alignItems: "center" },
flex1: { flex: 1 },
switchRow: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  marginTop: 20
},
primaryBtn: {
  backgroundColor: "#09A046",
  padding: 14,
  borderRadius: 10,
  alignItems: "center",
  marginTop: 24
},
primaryBtnText: { color: "#fff", fontWeight: "700" },
secondaryBtn: {
  padding: 14,
  borderRadius: 10,
  borderWidth: 1,
  borderColor: "#d1d5db",
  alignItems: "center",
  flex: 1,
  marginRight: 8
},
footer: {
  flexDirection: "row",
  padding: 16,
  backgroundColor: "#fff",
  borderTopWidth: 1,
  borderTopColor: "#e5e7eb"
},
addImgBox: {
  width: 88,
  height: 88,
  borderWidth: 2,
  borderColor: "#d1d5db",
  borderStyle: "dashed",
  borderRadius: 12,
  alignItems: "center",
  justifyContent: "center",
  marginRight: 8
},
thumbWrap: {
  width: 88,
  height: 88,
```

```
  marginRight: 8,
  borderRadius: 12,
  overflow: "hidden"
},
thumb: { width: "100%", height: "100%" },
mainBadge: {
  position: "absolute",
  top: 4,
  left: 4,
  backgroundColor: "#BAE4CB",
  paddingHorizontal: 6,
  borderRadius: 8
},
mainBadgeText: { fontSize: 9, color: "#fff", fontWeight: "600" },
thumbRemove: {
  position: "absolute",
  top: 4,
  right: 4,
  backgroundColor: "rgba(0,0,0,0.55)",
  width: 22,
  height: 22,
  borderRadius: 11,
  alignItems: "center",
  justifyContent: "center"
},
warn: { color: "#b45309", fontSize: 12, marginTop: 8 },
dropdown: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  borderWidth: 1,
  borderColor: "#d1d5db",
  borderRadius: 8,
  padding: 12,
  backgroundColor: "#fff",
  marginTop: 8
},
dropdownList: {
  borderWidth: 1,
  borderColor: "#e5e7eb",
  borderRadius: 8,
  maxHeight: 200,
  backgroundColor: "#fff"
},
ddItem: {
  flexDirection: "row",
  justifyContent: "space-between",
  padding: 12,
  borderBottomWidth: 1,
  borderBottomColor: "#f3f4f6"
},
chips: { flexDirection: "row", flexWrap: "wrap", marginTop: 10, gap: 6 },
chip: {
  flexDirection: "row",
  alignItems: "center",
  backgroundColor: "rgba(9,160,70,0.12)",
  paddingHorizontal: 10,
  paddingVertical: 6,
  borderRadius: 16
},
chipText: { color: "#09A046", fontSize: 13 },
chipX: { marginLeft: 6, color: "#09A046", fontSize: 16 },
chipInput: {
  minWidth: 56,
  borderWidth: 2,
  borderColor: "#d1d5db",
  borderStyle: "dashed",
  borderRadius: 16,
  paddingHorizontal: 10,
  paddingVertical: 6,
  fontSize: 12,
  color: "#111827"
},
fillAll: { flexDirection: "row", flexWrap: "wrap", gap: 6, marginBottom: 12 },
fillField: { width: "30%", minWidth: 90, minHeight: 44 },
fillBtn: {
  backgroundColor: "#09A046",
```

```
  paddingHorizontal: 12,
  paddingVertical: 12,
  borderRadius: 8,
  justifyContent: "center"
},
fillBtnText: { color: "#fff", fontWeight: "600", fontSize: 12 },
comboHeader: { fontWeight: "700", marginBottom: 8 },
comboRow: {
  flexDirection: "row",
  flexWrap: "wrap",
  alignItems: "center",
  gap: 6,
  marginBottom: 10,
  borderBottomWidth: 1,
  borderBottomColor: "#eee",
  paddingBottom: 10
},
comboLabel: { width: "100%", fontSize: 13, marginBottom: 4 },
comboInput: { flex: 1, minWidth: 80, minHeight: 44 },
batchBig: { fontSize: 20, fontWeight: "800", marginVertical: 8 },
reviewImgBox: {
  height: 160,
  borderRadius: 12,
  overflow: "hidden",
  marginBottom: 12
},
reviewImg: { width: "100%", height: "100%" },
reviewNav: {
  position: "absolute",
  bottom: 8,
  flexDirection: "row",
  width: "100%",
  justifyContent: "space-between",
  paddingHorizontal: 12
},
reviewNavText: {
  color: "#fff",
  fontSize: 22,
  backgroundColor: "rgba(0,0,0,0.35)",
  paddingHorizontal: 10,
  borderRadius: 16
},
reviewTitle: { fontSize: 20, fontWeight: "800" },
meta: { color: "#6b7280", marginTop: 4 },
toast: {
  position: "absolute",
  top: 48,
  left: 16,
  right: 16,
  backgroundColor: "#fff",
  padding: 14,
  borderRadius: 10,
  zIndex: 50,
  elevation: 4
},
modalOverlay: {
  flex: 1,
  backgroundColor: "rgba(0,0,0,0.45)",
  justifyContent: "center",
  padding: 24
},
modalBox: {
  backgroundColor: "#fff",
  borderRadius: 14,
  padding: 20
},
modalTitle: { fontSize: 18, fontWeight: "700", marginBottom: 8 },
modalBody: { color: "#4b5563", marginBottom: 16 },
modalActions: { flexDirection: "row", justifyContent: "flex-end", gap: 12 },
modalCancel: { padding: 12 },
modalOk: {
  backgroundColor: "#09A046",
  paddingHorizontal: 20,
  paddingVertical: 12,
  borderRadius: 8
},
card: {
```

```
    backgroundColor: "#f9fafb",
    padding: 12,
    borderRadius: 10,
    marginBottom: 10,
    borderWidth: 1,
    borderColor: "#e5e7eb"
  },
  bold: { fontWeight: "700" },
  hint: { fontSize: 12, color: "#6b7280" },
  hintMuted: { fontSize: 11, color: "#6b7280", marginBottom: 8 },
  reqMark: { color: "#ef4444" },
  datePressTxt: { fontSize: 15, color: "#111827" },
  datePressPlaceholder: { fontSize: 15, color: "#9ca3af" }
});



FILE: pos/mobile/app/Archive.jsx
================================================================================
import { Ionicons } from "@expo/vector-icons";
import { useRouter } from "expo-router";
import { useCallback, useEffect, useState } from "react";
import {
  ActivityIndicator,
  FlatList,
  Image,
  Modal,
  RefreshControl,
  StyleSheet,
  Text,
  TouchableOpacity,
  TouchableWithoutFeedback,
  View
} from "react-native";
import { archiveAPI } from "../services/api";
const formatDateTime = (dateString) => {
  if (!dateString) return "-";
  const date = new Date(dateString);
  return date.toLocaleString("en-US", {
    year: "numeric",
    month: "2-digit",
    day: "2-digit",
    hour: "2-digit",
    minute: "2-digit",
    hour12: true
  });
};
const reasonColor = (reason) => {
  switch (reason) {
    case "Defective":
      return { bg: "#fef2f2", text: "#dc2626" };
    case "Damaged":
      return { bg: "#fef2f2", text: "#dc2626" };
    case "Expired":
      return { bg: "#fef2f2", text: "#dc2626" };
    default:
      return { bg: "#fef2f2", text: "#dc2626" };
  }
};
export default function Archive() {
  const router = useRouter();
  const [archives, setArchives] = useState([]);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const [showRestoreModal, setShowRestoreModal] = useState(false);
  const [itemToRestore, setItemToRestore] = useState(null);
  const [restoring, setRestoring] = useState(false);
  const fetchArchives = useCallback(async () => {
    try {
      const response = await archiveAPI.getAll();
      if (response.success && response.data) {
        setArchives(response.data);
      } else {
        setArchives([]);
      }
    } catch (error) {
      console.error("Error fetching archives:", error);
```

```
      setArchives([]);
    } finally {
      setLoading(false);
      setRefreshing(false);
    }
  }, []);
  useEffect(() => {
    fetchArchives();
  }, [fetchArchives]);
  const onRefresh = () => {
    setRefreshing(true);
    fetchArchives();
  };
  const handleRestore = (item) => {
    setItemToRestore(item);
    setShowRestoreModal(true);
  };
  const confirmRestore = async () => {
    if (!itemToRestore) return;
    setRestoring(true);
    try {
      const id = itemToRestore._id || itemToRestore.id;
      const response = await archiveAPI.restore(id);
      if (response.success) {
        await fetchArchives();
      }
    } catch (error) {
      console.error("Error restoring item:", error);
    } finally {
      setRestoring(false);
      setShowRestoreModal(false);
      setItemToRestore(null);
    }
  };
  const cancelRestore = () => {
    setShowRestoreModal(false);
    setItemToRestore(null);
  };
  const renderArchiveItem = ({ item, index }) => {
    const archiveNumber = archives.length - index;
    const colors = reasonColor(item.reason);
    return /* @__PURE__ */ React.createElement(View, { style: styles.card }, /* @__PURE__ */
React.createElement(View, { style: styles.cardHeader }, /* @__PURE__ */ React.createElement(Text, { style:
styles.archiveNumber }, "#", archiveNumber), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        onPress: () => handleRestore(item),
        style: styles.restoreButton
      },
      /* @__PURE__ */ React.createElement(Ionicons, { name: "refresh", size: 18, color: "#16a34a" })
    )), /* @__PURE__ */ React.createElement(View, { style: styles.cardBody }, /* @__PURE__ */
React.createElement(View, { style: styles.imageContainer }, item.itemImage ? /* @__PURE__ */
React.createElement(
      Image,
      {
        source: { uri: item.itemImage },
        style: styles.itemImage,
        resizeMode: "cover",
        onError: () => {
        }
      }
    ) : /* @__PURE__ */ React.createElement(View, { style: styles.noImage }, /* @__PURE__ */
React.createElement(Text, { style: styles.noImageText }, "No Image"))), /* @__PURE__ */
React.createElement(View, { style: styles.itemDetails }, /* @__PURE__ */ React.createElement(Text, { style:
styles.itemName, numberOfLines: 1 }, item.itemName || "Unknown"), item.variant ? /* @__PURE__ */
React.createElement(Text, { style: styles.itemSubtext }, "(", item.variant, ")") : null, item.selectedSize ?
/* @__PURE__ */ React.createElement(Text, { style: styles.itemSubtext }, "Size: ", item.selectedSize) : null,
item.brandName ? /* @__PURE__ */ React.createElement(Text, { style: styles.itemSubtext }, "Brand: ",
item.brandName) : null, /* @__PURE__ */ React.createElement(Text, { style: styles.skuText }, "SKU: ",
item.sku || "N/A"))), /* @__PURE__ */ React.createElement(View, { style: styles.detailsGrid }, /* @__PURE__
*/ React.createElement(View, { style: styles.detailItem }, /* @__PURE__ */ React.createElement(Text, { style:
styles.detailLabel }, "Category"), /* @__PURE__ */ React.createElement(View, { style: styles.categoryBadge },
/* @__PURE__ */ React.createElement(Text, { style: styles.categoryText }, item.category || "N/A"))), /*
@__PURE__ */ React.createElement(View, { style: styles.detailItem }, /* @__PURE__ */ React.createElement(Text,
{ style: styles.detailLabel }, "Quantity"), /* @__PURE__ */ React.createElement(Text, { style:
styles.detailValue }, item.quantity || 0)), /* @__PURE__ */ React.createElement(View, { style:
styles.detailItem }, /* @__PURE__ */ React.createElement(Text, { style: styles.detailLabel }, "Price"), /*
```

```
@__PURE__ */ React.createElement(Text, { style: styles.priceValue }, "\u20B1", parseFloat(item.itemPrice ||
0).toFixed(2)))), /* @__PURE__ */ React.createElement(View, { style: styles.reasonRow }, /* @__PURE__ */
React.createElement(Text, { style: styles.detailLabel }, "Reason"), /* @__PURE__ */ React.createElement(View,
{ style: [styles.reasonBadge, { backgroundColor: colors.bg }] }, /* @__PURE__ */ React.createElement(Text,
{ style: [styles.reasonText, { color: colors.text }] }, item.reason))), item.returnReason ? /* @__PURE__ */
React.createElement(Text, { style: styles.returnReasonText }, item.returnReason) : null, item.notes ? /*
@__PURE__ */ React.createElement(Text, { style: styles.notesText }, item.notes) : null, /* @__PURE__ */
React.createElement(View, { style: styles.cardFooter }, /* @__PURE__ */ React.createElement(View, { style:
styles.footerItem }, /* @__PURE__ */ React.createElement(Ionicons, { name: "calendar-outline", size: 14,
color: "#888" }), /* @__PURE__ */ React.createElement(Text, { style: styles.footerText },
formatDateTime(item.archivedAt))), /* @__PURE__ */ React.createElement(View, { style: styles.footerItem }, /*
@__PURE__ */ React.createElement(Ionicons, { name: "person-outline", size: 14, color: "#888" }), /* @__PURE__
*/ React.createElement(Text, { style: styles.footerText }, item.archivedBy || "N/A"))));
  };
  return /* @__PURE__ */ React.createElement(View, { style: styles.container }, loading ? /* @__PURE__ */
React.createElement(View, { style: styles.loadingContainer }, /* @__PURE__ */
React.createElement(ActivityIndicator, { size: "large", color: "#8B4513" }), /* @__PURE__ */
React.createElement(Text, { style: styles.loadingText }, "Loading archives...")) : /* @__PURE__ */
React.createElement(
    FlatList,
    {
      data: archives,
      renderItem: renderArchiveItem,
      keyExtractor: (item) => (item._id || item.id || Math.random()).toString(),
      contentContainerStyle: archives.length === 0 ? styles.emptyListContent : styles.listContent,
      refreshControl: /* @__PURE__ */ React.createElement(
        RefreshControl,
        {
          refreshing,
          onRefresh,
          colors: ["#8B4513"],
          tintColor: "#8B4513"
        }
      ),
      ListEmptyComponent: /* @__PURE__ */ React.createElement(View, { style: styles.emptyState }, /*
@__PURE__ */ React.createElement(Ionicons, { name: "archive-outline", size: 64, color: "#ccc" }), /*
@__PURE__ */ React.createElement(Text, { style: styles.emptyText }, "No Archive yet"), /* @__PURE__ */
React.createElement(Text, { style: styles.emptySubtext }, "Items removed from inventory will appear here"))
    }
  ), /* @__PURE__ */ React.createElement(
    Modal,
    {
      visible: showRestoreModal,
      transparent: true,
      animationType: "fade",
      onRequestClose: cancelRestore
    },
    /* @__PURE__ */ React.createElement(TouchableWithoutFeedback, { onPress: cancelRestore }, /* @__PURE__ */
React.createElement(View, { style: styles.modalOverlay }, /* @__PURE__ */
React.createElement(TouchableWithoutFeedback, null, /* @__PURE__ */ React.createElement(View, { style:
styles.modalContent }, /* @__PURE__ */ React.createElement(Text, { style: styles.modalTitle }, "Confirm
Restore"), /* @__PURE__ */ React.createElement(Text, { style: styles.modalText }, "Are you sure you want to
restore this item? It will be moved back to your inventory."), /* @__PURE__ */ React.createElement(View,
{ style: styles.modalButtons }, /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [styles.modalButton, styles.cancelButton],
        onPress: cancelRestore,
        disabled: restoring
      },
      /* @__PURE__ */ React.createElement(Text, { style: styles.cancelButtonText }, "Cancel")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [styles.modalButton, styles.confirmBtn],
        onPress: confirmRestore,
        disabled: restoring
      },
      restoring ? /* @__PURE__ */ React.createElement(ActivityIndicator, { size: "small", color: "#fff" }) :
/* @__PURE__ */ React.createElement(Text, { style: styles.confirmButtonText }, "Restore")
    ))))))
  ));
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#f5f5f5"
```

```
},
loadingContainer: {
  flex: 1,
  justifyContent: "center",
  alignItems: "center",
  padding: 32
},
loadingText: {
  marginTop: 12,
  color: "#666",
  fontSize: 14
},
listContent: {
  padding: 16,
  paddingBottom: 32
},
emptyListContent: {
  flex: 1
},
emptyState: {
  flex: 1,
  justifyContent: "center",
  alignItems: "center",
  padding: 32
},
emptyText: {
  marginTop: 16,
  fontSize: 18,
  color: "#999",
  fontWeight: "500"
},
emptySubtext: {
  marginTop: 8,
  fontSize: 14,
  color: "#aaa",
  textAlign: "center"
},
// Card styles
card: {
  backgroundColor: "#fff",
  borderRadius: 12,
  padding: 16,
  marginBottom: 12,
  elevation: 2,
  shadowColor: "#000",
  shadowOffset: { width: 0, height: 1 },
  shadowOpacity: 0.08,
  shadowRadius: 4
},
cardHeader: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  marginBottom: 12
},
archiveNumber: {
  fontSize: 16,
  fontWeight: "700",
  color: "#333"
},
restoreButton: {
  width: 36,
  height: 36,
  borderRadius: 18,
  backgroundColor: "#f0fdf4",
  justifyContent: "center",
  alignItems: "center"
},
cardBody: {
  flexDirection: "row",
  marginBottom: 12
},
imageContainer: {
  marginRight: 12
},
itemImage: {
  width: 56,
```

```
    height: 56,
    borderRadius: 8,
    backgroundColor: "#f0f0f0"
  },
  noImage: {
    width: 56,
    height: 56,
    borderRadius: 8,
    backgroundColor: "#f0f0f0",
    justifyContent: "center",
    alignItems: "center"
  },
  noImageText: {
    fontSize: 10,
    color: "#999"
  },
  itemDetails: {
    flex: 1,
    justifyContent: "center"
  },
  itemName: {
    fontSize: 15,
    fontWeight: "600",
    color: "#333",
    marginBottom: 2
  },
  itemSubtext: {
    fontSize: 12,
    color: "#888"
  },
  skuText: {
    fontSize: 12,
    color: "#666",
    fontFamily: "monospace",
    marginTop: 2
  },
  // Details grid
  detailsGrid: {
    flexDirection: "row",
    justifyContent: "space-between",
    backgroundColor: "#fafafa",
    borderRadius: 8,
    padding: 12,
    marginBottom: 10
  },
  detailItem: {
    alignItems: "center"
  },
  detailLabel: {
    fontSize: 11,
    color: "#888",
    fontWeight: "500",
    textTransform: "uppercase",
    marginBottom: 4
  },
  detailValue: {
    fontSize: 14,
    fontWeight: "600",
    color: "#333"
  },
  priceValue: {
    fontSize: 14,
    fontWeight: "600",
    color: "#333"
  },
  categoryBadge: {
    backgroundColor: "#f3f4f6",
    paddingHorizontal: 8,
    paddingVertical: 2,
    borderRadius: 4
  },
  categoryText: {
    fontSize: 12,
    color: "#555",
    fontWeight: "500"
  },
  // Reason
```

```
reasonRow: {
  flexDirection: "row",
  alignItems: "center",
  marginBottom: 4,
  gap: 8
},
reasonBadge: {
  paddingHorizontal: 8,
  paddingVertical: 3,
  borderRadius: 4
},
reasonText: {
  fontSize: 12,
  fontWeight: "600"
},
returnReasonText: {
  fontSize: 12,
  color: "#666",
  marginBottom: 2,
  marginLeft: 2
},
notesText: {
  fontSize: 12,
  color: "#888",
  fontStyle: "italic",
  marginBottom: 2,
  marginLeft: 2
},
// Footer
cardFooter: {
  flexDirection: "row",
  justifyContent: "space-between",
  borderTopWidth: 1,
  borderTopColor: "#f0f0f0",
  paddingTop: 10,
  marginTop: 8
},
footerItem: {
  flexDirection: "row",
  alignItems: "center",
  gap: 4
},
footerText: {
  fontSize: 12,
  color: "#888"
},
// Modal styles
modalOverlay: {
  flex: 1,
  backgroundColor: "rgba(0, 0, 0, 0.5)",
  justifyContent: "center",
  alignItems: "center",
  padding: 20
},
modalContent: {
  backgroundColor: "white",
  borderRadius: 12,
  padding: 24,
  width: "100%",
  maxWidth: 320
},
modalTitle: {
  fontSize: 18,
  fontWeight: "600",
  marginBottom: 12,
  color: "#333"
},
modalText: {
  fontSize: 15,
  color: "#555",
  marginBottom: 24,
  lineHeight: 22
},
modalButtons: {
  flexDirection: "row",
  justifyContent: "flex-end",
  marginTop: 8
```

```
  },
  modalButton: {
    paddingVertical: 10,
    paddingHorizontal: 20,
    borderRadius: 8,
    marginLeft: 12,
    minWidth: 100,
    alignItems: "center"
  },
  cancelButton: {
    backgroundColor: "#f0f0f0"
  },
  confirmBtn: {
    backgroundColor: "#8B4513"
  },
  cancelButtonText: {
    color: "#333",
    fontWeight: "500"
  },
  confirmButtonText: {
    color: "white",
    fontWeight: "500"
  }
});



FILE: pos/mobile/app/Edit Profile.jsx
================================================================================
import { Ionicons } from "@expo/vector-icons";
import AsyncStorage from "@react-native-async-storage/async-storage";
import { useNavigation } from "@react-navigation/native";
import * as ImagePicker from "expo-image-picker";
import { useEffect, useState } from "react";
import {
  ActivityIndicator,
  Alert,
  Image,
  KeyboardAvoidingView,
  Platform,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View
} from "react-native";
import { employeeAPI } from "../services/api";
export default function EditProfile() {
  const navigation = useNavigation();
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [contactNumber, setContactNumber] = useState("");
  const [bio, setBio] = useState("");
  const [profilePicture, setProfilePicture] = useState(null);
  const [employeeId, setEmployeeId] = useState(null);
  const [saving, setSaving] = useState(false);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    const loadProfile = async () => {
      try {
        const stored = await AsyncStorage.getItem("currentEmployee");
        if (!stored) {
          setLoading(false);
          return;
        }
        const parsed = JSON.parse(stored);
        let latest = parsed;
        const id = parsed._id || parsed.id;
        if (id) {
          try {
            const res = await employeeAPI.getById(id);
            if (res?.success && res.data) {
              latest = res.data;
              await AsyncStorage.setItem("currentEmployee", JSON.stringify(latest));
            }
          } catch (e) {
```

```
          console.warn("Failed to fetch fresh profile, using stored data", e);
        }
      }
      setEmployeeId(latest._id || latest.id || null);
      setName(
        latest.name || `${latest.firstName || ""} ${latest.lastName || ""}`.trim()
      );
      setEmail(latest.email || "");
      setContactNumber(latest.contactNo || "");
      setBio(latest.bio || "");
      setProfilePicture(latest.profileImage || latest.image || null);
    } catch (error) {
      console.warn("Failed to load profile for editing:", error?.message);
      Alert.alert("Error", "Failed to load profile data.");
    } finally {
      setLoading(false);
    }
  };
  loadProfile();
}, []);
const pickImageFromGallery = async () => {
  try {
    const { status } = await ImagePicker.requestMediaLibraryPermissionsAsync();
    if (status !== "granted") {
      Alert.alert("Permission required", "Gallery permission is required to select an image.");
      return;
    }
    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ImagePicker.MediaTypeOptions.Images,
      allowsEditing: true,
      // enable cropping
      aspect: [1, 1],
      // square crop
      quality: 0.8
    });
    if (!result.canceled && result.assets && result.assets.length > 0) {
      try {
        const asset = result.assets[0];
        const manipulated = await ImageManipulator.manipulateAsync(
          asset.uri,
          [{ resize: { width: 800 } }],
          // Profile pics can be smaller (800px)
          { compress: 0.6, format: ImageManipulator.SaveFormat.JPEG }
        );
        setProfilePicture(manipulated.uri);
      } catch (err) {
        console.log("Error compressing image", err);
        setProfilePicture(result.assets[0].uri);
      }
    }
  } catch (error) {
    console.log("Error picking image:", error);
    Alert.alert("Error", "Failed to pick image.");
  }
};
const takePhoto = async () => {
  try {
    const { status } = await ImagePicker.requestCameraPermissionsAsync();
    if (status !== "granted") {
      Alert.alert("Permission required", "Camera permission is required to take a photo.");
      return;
    }
    const result = await ImagePicker.launchCameraAsync({
      allowsEditing: true,
      // enable cropping
      aspect: [1, 1],
      // square crop
      quality: 0.8
    });
    if (!result.canceled && result.assets && result.assets.length > 0) {
      try {
        const asset = result.assets[0];
        const manipulated = await ImageManipulator.manipulateAsync(
          asset.uri,
          [{ resize: { width: 800 } }],
          { compress: 0.6, format: ImageManipulator.SaveFormat.JPEG }
        );
```

```
        setProfilePicture(manipulated.uri);
      } catch (err) {
        console.log("Error compressing photo", err);
        setProfilePicture(result.assets[0].uri);
      }
    }
  } catch (error) {
    console.log("Error taking photo:", error);
    Alert.alert("Error", "Failed to take photo.");
  }
};
const openImageOptions = () => {
  Alert.alert("Change Profile Picture", "Choose an option", [
    { text: "Take Photo", onPress: takePhoto },
    { text: "Choose from Gallery", onPress: pickImageFromGallery },
    { text: "Cancel", style: "cancel" }
  ]);
};
const handleSave = async () => {
  if (!employeeId) {
    Alert.alert("Error", "Cannot update profile. Employee ID is missing.");
    return;
  }
  if (!name.trim()) {
    Alert.alert("Error", "Name is required.");
    return;
  }
  try {
    setSaving(true);
    const parts = (name || "").trim().split(" ");
    const firstName = parts[0] || "";
    const lastName = parts.slice(1).join(" ") || "";
    const updatePayload = {
      name: name.trim(),
      firstName,
      lastName,
      email: email.trim(),
      contactNo: contactNumber.trim(),
      profileImage: profilePicture || "",
      bio: bio.trim()
      // If backend supports it
    };
    if (profilePicture && !profilePicture.startsWith("http") && !profilePicture.startsWith("data:image")) {
      try {
        const { readAsStringAsync, EncodingType } = require("expo-file-system/legacy");
        const base64 = await readAsStringAsync(profilePicture, { encoding: EncodingType.Base64 });
        updatePayload.profileImage = `${base64}`;
      } catch (err) {
        console.warn("Failed to convert profile image to base64", err);
      }
    }
    const res = await employeeAPI.update(employeeId, updatePayload);
    if (res?.success && res.data) {
      await AsyncStorage.setItem(
        "currentEmployee",
        JSON.stringify(res.data)
      );
      Alert.alert("Success", "Profile updated successfully.", [
        { text: "OK", onPress: () => navigation.goBack() }
      ]);
    } else {
      Alert.alert(
        "Error",
        res?.message || "Failed to update profile. Please try again."
      );
    }
  } catch (error) {
    console.log("Error saving profile:", error);
    Alert.alert(
      "Error",
      error?.message || "Failed to update profile. Please try again."
    );
  } finally {
    setSaving(false);
  }
};
if (loading) {
```

```
    return /* @__PURE__ */ React.createElement(View, { style: [styles.container, styles.centerContent] }, /*
@__PURE__ */ React.createElement(ActivityIndicator, { size: "large", color: "#AD7F65" }));
  }
  return /* @__PURE__ */ React.createElement(
    KeyboardAvoidingView,
    {
      behavior: Platform.OS === "ios" ? "padding" : "height",
      style: { flex: 1 }
    },
    /* @__PURE__ */ React.createElement(ScrollView, { contentContainerStyle: styles.scrollContainer }, /*
@__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: styles.backButton,
        onPress: () => navigation.goBack()
      },
      /* @__PURE__ */ React.createElement(Ionicons, { name: "arrow-back", size: 24, color: "#333" }),
      /* @__PURE__ */ React.createElement(Text, { style: styles.headerTitle }, "Edit Profile")
    ), /* @__PURE__ */ React.createElement(View, { style: styles.card }, /* @__PURE__ */
React.createElement(View, { style: styles.profilePictureContainer }, /* @__PURE__ */
React.createElement(TouchableOpacity, { onPress: openImageOptions, activeOpacity: 0.8 }, profilePicture ? /*
@__PURE__ */ React.createElement(
      Image,
      {
        source: { uri: profilePicture },
        style: styles.profilePicture
      }
    ) : /* @__PURE__ */ React.createElement(View, { style: styles.profilePicturePlaceholder }, /* @__PURE__
*/ React.createElement(Ionicons, { name: "camera", size: 30, color: "#FFF" })), /* @__PURE__ */
React.createElement(View, { style: styles.editIconBadge }, /* @__PURE__ */ React.createElement(Ionicons,
{ name: "pencil", size: 14, color: "#FFF" }))), /* @__PURE__ */ React.createElement(Text, { style:
styles.npplabelText }, "Change Profile Picture")), /* @__PURE__ */ React.createElement(Text, { style:
styles.label }, "Full Name"), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: styles.input,
        value: name,
        onChangeText: setName,
        placeholder: "Enter your full name",
        placeholderTextColor: "#999"
      }
    ), /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Email Address"), /* @__PURE__ */
React.createElement(
      TextInput,
      {
        style: styles.input,
        value: email,
        onChangeText: setEmail,
        placeholder: "Enter your email",
        placeholderTextColor: "#999",
        keyboardType: "email-address",
        autoCapitalize: "none"
      }
    ), /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Contact Number"), /* @__PURE__ */
React.createElement(
      TextInput,
      {
        style: styles.input,
        value: contactNumber,
        onChangeText: setContactNumber,
        placeholder: "Enter your contact number",
        placeholderTextColor: "#999",
        keyboardType: "phone-pad"
      }
    ), /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Bio"), /* @__PURE__ */
React.createElement(
      TextInput,
      {
        style: [styles.input, styles.bioInput],
        value: bio,
        onChangeText: setBio,
        placeholder: "Tell us about yourself",
        placeholderTextColor: "#999",
        multiline: true,
        numberOfLines: 4,
        textAlignVertical: "top"
      }
```

```
    ), /* @__PURE__ */ React.createElement(View, { style: styles.buttonContainer }, /* @__PURE__ */
React.createElement(TouchableOpacity, { style: styles.cancelButton, onPress: () => navigation.goBack() }, /*
@__PURE__ */ React.createElement(Text, { style: styles.cancelButtonText }, "Cancel")), /* @__PURE__ */
React.createElement(
      TouchableOpacity,
      {
        style: [styles.saveButton, saving && styles.disabledButton],
        onPress: handleSave,
        disabled: saving
      },
      saving ? /* @__PURE__ */ React.createElement(ActivityIndicator, { size: "small", color: "#FFF" }) : /*
@__PURE__ */ React.createElement(Text, { style: styles.saveButtonText }, "Save Changes")
    )))))
  );
}
const styles = StyleSheet.create({
  scrollContainer: {
    flexGrow: 1,
    backgroundColor: "#f6f6f6"
  },
  container: {
    flex: 1,
    padding: 20,
    paddingTop: 50
    // For status bar
  },
  centerContent: {
    justifyContent: "center",
    alignItems: "center"
  },
  backButton: {
    flexDirection: "row",
    alignItems: "center",
    marginBottom: 20
  },
  headerTitle: {
    fontSize: 20,
    fontWeight: "bold",
    color: "#333",
    marginLeft: 10
  },
  card: {
    backgroundColor: "#FFF",
    borderRadius: 20,
    padding: 20,
    width: "100%",
    shadowColor: "#000",
    shadowOpacity: 0.05,
    shadowRadius: 10,
    elevation: 3
  },
  profilePictureContainer: {
    alignItems: "center",
    marginBottom: 25
  },
  profilePicture: {
    width: 100,
    height: 100,
    borderRadius: 50,
    borderWidth: 3,
    borderColor: "#AD7F65"
  },
  profilePicturePlaceholder: {
    width: 100,
    height: 100,
    borderRadius: 50,
    backgroundColor: "#C2A68C",
    justifyContent: "center",
    alignItems: "center",
    borderWidth: 3,
    borderColor: "#AD7F65"
  },
  editIconBadge: {
    position: "absolute",
    bottom: 0,
    right: 0,
    backgroundColor: "#AD7F65",
```

```
    width: 30,
    height: 30,
    borderRadius: 15,
    justifyContent: "center",
    alignItems: "center",
    borderWidth: 2,
    borderColor: "#FFF"
  },
  npplabelText: {
    marginTop: 12,
    fontSize: 14,
    fontWeight: "600",
    color: "#AD7F65"
  },
  label: {
    fontSize: 14,
    fontWeight: "600",
    color: "#555",
    marginBottom: 8,
    marginTop: 16
  },
  input: {
    backgroundColor: "#F9F9F9",
    borderWidth: 1,
    borderColor: "#E0E0E0",
    borderRadius: 12,
    padding: 14,
    fontSize: 15,
    color: "#333"
  },
  bioInput: {
    height: 100,
    paddingTop: 14
  },
  buttonContainer: {
    flexDirection: "row",
    marginTop: 30,
    gap: 15
  },
  saveButton: {
    flex: 1,
    backgroundColor: "#AD7F65",
    paddingVertical: 16,
    borderRadius: 12,
    alignItems: "center",
    justifyContent: "center"
  },
  disabledButton: {
    opacity: 0.7
  },
  cancelButton: {
    flex: 1,
    backgroundColor: "#FFF",
    paddingVertical: 16,
    borderRadius: 12,
    alignItems: "center",
    justifyContent: "center",
    borderWidth: 1,
    borderColor: "#DDD"
  },
  saveButtonText: {
    color: "#FFF",
    fontSize: 16,
    fontWeight: "600"
  },
  cancelButtonText: {
    color: "#777",
    fontSize: 16,
    fontWeight: "600"
  }
});



FILE: pos/mobile/app/index.tsx
================================================================================
import { Redirect } from "expo-router";
```

```
export default function Index() {
  return /* @__PURE__ */ React.createElement(Redirect, { href: "/pinlogin" });
}
```

FILE: pos/mobile/app/ItemArchive.jsx
==========

```
import { Ionicons } from "@expo/vector-icons";
import { useRouter } from "expo-router";
import { useEffect, useState } from "react";
import { ActivityIndicator, Animated, FlatList, LayoutAnimation, Modal, Platform, RefreshControl, StyleSheet,
Text, TouchableOpacity, TouchableWithoutFeedback, UIManager, View } from "react-native";
import { archiveAPI } from "../services/api";
if (Platform.OS === "android" && UIManager.setLayoutAnimationEnabledExperimental) {
  UIManager.setLayoutAnimationEnabledExperimental(true);
}
const CollapsibleItem = ({ item, isExpanded, onPress, onRestore }) => {
  const [animation] = useState(new Animated.Value(0));
  useEffect(() => {
    Animated.timing(animation, {
      toValue: isExpanded ? 1 : 0,
      duration: 200,
      useNativeDriver: false
    }).start();
  }, [isExpanded, animation]);
  const height = animation.interpolate({
    inputRange: [0, 1],
    outputRange: [0, 100]
    // Adjust based on your content height
  });
  const handleRestoreClick = (e) => {
    e.stopPropagation();
    onRestore(item._id || item.id);
  };
  const name = item.itemName || item.name || "Unknown Item";
  const sku = item.sku || "N/A";
  const dateArchived = item.archivedAt || item.dateArchived || /* @__PURE__ */ new Date();
  const formattedDate = new Date(dateArchived).toLocaleDateString("en-US", {
    year: "numeric",
    month: "short",
    day: "numeric"
  });
  const brand = item.brandName || item.brand || "N/A";
  const category = item.category || "N/A";
  const price = item.itemPrice || item.price || 0;
  return /* @__PURE__ */ React.createElement(View, { style: styles.itemContainer }, /* @__PURE__ */
React.createElement(
    TouchableOpacity,
    {
      style: styles.itemHeader,
      onPress,
      activeOpacity: 0.7
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.itemInfo }, /* @__PURE__ */
React.createElement(View, { style: styles.itemHeaderRow }, /* @__PURE__ */ React.createElement(View, { style:
styles.nameAndRestoreContainer }, /* @__PURE__ */ React.createElement(Text, { style: styles.itemName,
numberOfLines: 1 }, name), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        onPress: handleRestoreClick,
        style: styles.restoreIcon,
        hitSlop: { top: 10, bottom: 10, left: 10, right: 10 }
      },
      /* @__PURE__ */ React.createElement(Ionicons, { name: "refresh", size: 20, color: "#3498db" })
    ))), /* @__PURE__ */ React.createElement(Text, { style: styles.itemSku }, "SKU: ", sku), /* @__PURE__ */
React.createElement(Text, { style: styles.itemDate }, "Archived: ", formattedDate)),
    /* @__PURE__ */ React.createElement(
      Ionicons,
      {
        name: isExpanded ? "chevron-up" : "chevron-down",
        size: 20,
        color: "#666"
      }
    )
  ), /* @__PURE__ */ React.createElement(Animated.View, { style: [styles.collapsibleContent, { height }] },
/* @__PURE__ */ React.createElement(View, { style: styles.detailsContainer }, /* @__PURE__ */
```

```
React.createElement(View, { style: styles.detailRow }, /* @__PURE__ */ React.createElement(Text, { style:
styles.detailLabel }, "Brand:"), /* @__PURE__ */ React.createElement(Text, { style: styles.detailValue },
brand)), /* @__PURE__ */ React.createElement(View, { style: styles.detailRow }, /* @__PURE__ */
React.createElement(Text, { style: styles.detailLabel }, "Category:"), /* @__PURE__ */
React.createElement(Text, { style: styles.detailValue }, category)), /* @__PURE__ */ React.createElement(View,
{ style: styles.detailRow }, /* @__PURE__ */ React.createElement(Text, { style: styles.detailLabel },
"Price:"), /* @__PURE__ */ React.createElement(Text, { style: [styles.detailValue, styles.price] }, "\u20B1",
price.toFixed(2))), /* @__PURE__ */ React.createElement(View, { style: styles.actionsContainer }, /*
@__PURE__ */ React.createElement(Text, { style: styles.restoreLabel }, "Restore this item?"), /* @__PURE__ */
React.createElement(
    TouchableOpacity,
    {
      style: styles.actionButton,
      onPress: handleRestoreClick,
      activeOpacity: 0.9
    },
    /* @__PURE__ */ React.createElement(Ionicons, { name: "refresh", size: 16, color: "#fff" }),
    /* @__PURE__ */ React.createElement(Text, { style: styles.actionButtonText }, "Restore Item")
  )))));
};
export default function ItemArchive() {
  const router = useRouter();
  const [sortBy, setSortBy] = useState("newest");
  const [isFilterVisible, setIsFilterVisible] = useState(false);
  const [expandedItems, setExpandedItems] = useState({});
  const [showRestoreModal, setShowRestoreModal] = useState(false);
  const [itemToRestore, setItemToRestore] = useState(null);
  const [archivedItems, setArchivedItems] = useState([]);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const fetchArchivedItems = async () => {
    try {
      const response = await archiveAPI.getAll();
      if (response.success && response.data) {
        setArchivedItems(response.data);
      }
    } catch (error) {
      console.error("Error fetching archived items:", error);
      setArchivedItems([]);
    } finally {
      setLoading(false);
      setRefreshing(false);
    }
  };
  useEffect(() => {
    fetchArchivedItems();
  }, []);
  const onRefresh = () => {
    setRefreshing(true);
    fetchArchivedItems();
  };
  const toggleItem = (itemId) => {
    LayoutAnimation.configureNext(LayoutAnimation.Presets.easeInEaseOut);
    setExpandedItems((prev) => ({
      ...prev,
      [itemId]: !prev[itemId]
    }));
  };
  const getSortedItems = () => {
    const items = [...archivedItems];
    switch (sortBy) {
      case "a-z":
        return items.sort((a, b) => {
          const nameA = a.itemName || a.name || "";
          const nameB = b.itemName || b.name || "";
          return nameA.localeCompare(nameB);
        });
      case "z-a":
        return items.sort((a, b) => {
          const nameA = a.itemName || a.name || "";
          const nameB = b.itemName || b.name || "";
          return nameB.localeCompare(nameA);
        });
      case "newest":
        return items.sort((a, b) => {
          const dateA = new Date(a.archivedAt || a.dateArchived);
          const dateB = new Date(b.archivedAt || b.dateArchived);
```

```
          return dateB - dateA;
        });
      case "oldest":
        return items.sort((a, b) => {
          const dateA = new Date(a.archivedAt || a.dateArchived);
          const dateB = new Date(b.archivedAt || b.dateArchived);
          return dateA - dateB;
        });
      default:
        return items;
    }
  };
  const renderFilterButton = (label, value) => /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      key: value,
      style: [
        styles.filterButton,
        sortBy === value && styles.activeFilter
      ],
      onPress: () => {
        setSortBy(value);
        setIsFilterVisible(false);
      }
    },
    /* @__PURE__ */ React.createElement(Text, { style: [
      styles.filterButtonText,
      sortBy === value && styles.activeFilterText
    ] }, label)
  );
  const renderItem = ({ item }) => /* @__PURE__ */ React.createElement(
    CollapsibleItem,
    {
      item,
      isExpanded: !!expandedItems[item._id || item.id],
      onPress: () => toggleItem(item._id || item.id),
      onRestore: (itemId) => handleRestore(itemId)
    }
  );
  const handleRestore = (itemId) => {
    setItemToRestore(itemId);
    setShowRestoreModal(true);
  };
  const confirmRestore = async () => {
    try {
      console.log("Restoring item:", itemToRestore);
      const response = await archiveAPI.restore(itemToRestore);
      if (response.success) {
        await fetchArchivedItems();
      }
    } catch (error) {
      console.error("Error restoring item:", error);
    } finally {
      setShowRestoreModal(false);
      setItemToRestore(null);
    }
  };
  const cancelRestore = () => {
    setShowRestoreModal(false);
    setItemToRestore(null);
  };
  return /* @__PURE__ */ React.createElement(View, { style: { flex: 1, backgroundColor: "#f5f5f5" } }, /*
@__PURE__ */ React.createElement(View, { style: styles.header }, /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: styles.filterToggle,
      onPress: () => setIsFilterVisible(!isFilterVisible)
    },
    /* @__PURE__ */ React.createElement(Ionicons, { name: "filter", size: 20, color: "#333" }),
    /* @__PURE__ */ React.createElement(Text, { style: styles.filterToggleText }, "Sort"),
    /* @__PURE__ */ React.createElement(
      Ionicons,
      {
        name: isFilterVisible ? "chevron-up" : "chevron-down",
        size: 16,
        color: "#666"
      }
```

```
    )
  )), isFilterVisible && /* @__PURE__ */ React.createElement(View, { style: styles.filterContainer }, /*
@__PURE__ */ React.createElement(Text, { style: styles.filterTitle }, "Sort By:"), /* @__PURE__ */
React.createElement(View, { style: styles.filterButtons }, renderFilterButton("A to Z", "a-z"),
renderFilterButton("Z to A", "z-a"), renderFilterButton("Newest First", "newest"), renderFilterButton("Oldest
First", "oldest"))), loading ? /* @__PURE__ */ React.createElement(View, { style: styles.loadingContainer },
/* @__PURE__ */ React.createElement(ActivityIndicator, { size: "large", color: "#8B4513" }), /* @__PURE__ */
React.createElement(Text, { style: styles.loadingText }, "Loading archive...")) : /* @__PURE__ */
React.createElement(
    FlatList,
    {
      data: getSortedItems(),
      renderItem,
      keyExtractor: (item) => (item._id || item.id).toString(),
      contentContainerStyle: archivedItems.length === 0 ? styles.emptyListContent : styles.listContent,
      style: styles.list,
      refreshControl: /* @__PURE__ */ React.createElement(
        RefreshControl,
        {
          refreshing,
          onRefresh,
          colors: ["#8B4513"],
          tintColor: "#8B4513"
        }
      ),
      ListEmptyComponent: /* @__PURE__ */ React.createElement(View, { style: styles.emptyState }, /*
@__PURE__ */ React.createElement(Ionicons, { name: "archive-outline", size: 64, color: "#ccc" }), /*
@__PURE__ */ React.createElement(Text, { style: styles.emptyText }, "No archived items"), /* @__PURE__ */
React.createElement(Text, { style: styles.emptySubtext }, "Items removed from inventory will appear here"))
    }
  ), /* @__PURE__ */ React.createElement(
    Modal,
    {
      visible: showRestoreModal,
      transparent: true,
      animationType: "fade",
      onRequestClose: cancelRestore
    },
    /* @__PURE__ */ React.createElement(TouchableWithoutFeedback, { onPress: cancelRestore }, /* @__PURE__ */
React.createElement(View, { style: styles.modalOverlay }, /* @__PURE__ */
React.createElement(TouchableWithoutFeedback, null, /* @__PURE__ */ React.createElement(View, { style:
styles.modalContent }, /* @__PURE__ */ React.createElement(Text, { style: styles.modalTitle }, "Confirm
Restore"), /* @__PURE__ */ React.createElement(Text, { style: styles.modalText }, "Are you sure you want to
restore this item? It will be moved back to your inventory."), /* @__PURE__ */ React.createElement(View,
{ style: styles.modalButtons }, /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [styles.modalButton, styles.cancelButton],
        onPress: cancelRestore
      },
      /* @__PURE__ */ React.createElement(Text, { style: styles.cancelButtonText }, "Cancel")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [styles.modalButton, styles.confirmButton],
        onPress: confirmRestore
      },
      /* @__PURE__ */ React.createElement(Text, { style: styles.confirmButtonText }, "Restore")
    ))))))
  ));
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#f5f5f5"
  },
  backButton: {
    padding: 8,
    marginLeft: 8,
    flexDirection: "row",
    alignItems: "center"
  },
  itemHeaderRow: {
    flex: 1,
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center"
```

```
},
nameAndRestoreContainer: {
  flexDirection: "row",
  alignItems: "center",
  flex: 1
},
restoreIcon: {
  marginLeft: 8,
  padding: 4
},
filterToggle: {
  flexDirection: "row",
  alignItems: "center",
  paddingVertical: 8,
  paddingHorizontal: 16,
  borderRadius: 20,
  borderWidth: 1,
  borderColor: "#ddd",
  backgroundColor: "#fff",
  marginLeft: "auto",
  marginTop: 12
},
filterToggleText: {
  marginLeft: 6,
  marginRight: 6,
  fontSize: 14,
  color: "#333",
  fontWeight: "500"
},
filterContainer: {
  padding: 16,
  backgroundColor: "#fff",
  borderBottomWidth: 1,
  borderBottomColor: "#f0f0f0"
},
filterTitle: {
  fontSize: 15,
  fontWeight: "600",
  marginBottom: 12,
  color: "#333"
},
filterButtons: {
  flexDirection: "row",
  flexWrap: "wrap",
  marginHorizontal: -4
},
filterButton: {
  paddingVertical: 8,
  paddingHorizontal: 16,
  margin: 4,
  borderRadius: 16,
  borderWidth: 1,
  borderColor: "#e0e0e0",
  backgroundColor: "#f8f8f8"
},
activeFilter: {
  backgroundColor: "#5D4037",
  borderColor: "#5D4037"
},
filterButtonText: {
  fontSize: 13,
  color: "#666"
},
activeFilterText: {
  color: "#fff",
  fontWeight: "500"
},
list: {
  flex: 1
},
listContent: {
  padding: 16
},
itemContainer: {
  backgroundColor: "#fff",
  borderRadius: 12,
  marginBottom: 12,
```

```
    overflow: "hidden",
    elevation: 2,
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.05,
    shadowRadius: 4
  },
  itemHeader: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    padding: 16
  },
  itemInfo: {
    flex: 1,
    marginRight: 12
  },
  itemName: {
    fontSize: 16,
    fontWeight: "600",
    color: "#333",
    marginBottom: 4
  },
  itemSku: {
    fontSize: 13,
    color: "#666",
    marginBottom: 2
  },
  itemDate: {
    fontSize: 12,
    color: "#888",
    marginTop: 2
  },
  collapsibleContent: {
    overflow: "hidden",
    backgroundColor: "#f9f9f9"
  },
  detailsContainer: {
    padding: 16,
    paddingTop: 0
  },
  detailRow: {
    flexDirection: "row",
    justifyContent: "space-between",
    marginBottom: 10
  },
  detailLabel: {
    fontSize: 14,
    color: "#666",
    fontWeight: "500"
  },
  detailValue: {
    fontSize: 14,
    color: "#333"
  },
  price: {
    fontWeight: "600",
    color: "#2ecc71"
  },
  actionsContainer: {
    marginTop: 12,
    paddingTop: 12,
    borderTopWidth: 1,
    borderTopColor: "#eee",
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center"
  },
  actionButton: {
    flexDirection: "row",
    alignItems: "center",
    paddingVertical: 10,
    paddingHorizontal: 16,
    borderRadius: 20,
    backgroundColor: "#2ecc71",
    elevation: 2,
    shadowColor: "#000",
```

```
  shadowOffset: { width: 0, height: 1 },
  shadowOpacity: 0.2,
  shadowRadius: 1.5
},
actionButtonText: {
  color: "#fff",
  marginLeft: 6,
  fontSize: 14,
  fontWeight: "600"
},
restoreLabel: {
  fontSize: 13,
  color: "#666",
  fontStyle: "italic"
},
// Modal styles
modalOverlay: {
  flex: 1,
  backgroundColor: "rgba(0, 0, 0, 0.5)",
  justifyContent: "center",
  alignItems: "center",
  padding: 20
},
modalContent: {
  backgroundColor: "white",
  borderRadius: 12,
  padding: 24,
  width: "100%",
  maxWidth: 320
},
modalTitle: {
  fontSize: 18,
  fontWeight: "600",
  marginBottom: 12,
  color: "#333"
},
modalText: {
  fontSize: 15,
  color: "#555",
  marginBottom: 24,
  lineHeight: 22
},
modalButtons: {
  flexDirection: "row",
  justifyContent: "flex-end",
  marginTop: 8
},
modalButton: {
  paddingVertical: 10,
  paddingHorizontal: 20,
  borderRadius: 8,
  marginLeft: 12,
  minWidth: 100,
  alignItems: "center"
},
cancelButton: {
  backgroundColor: "#f0f0f0"
},
confirmButton: {
  backgroundColor: "#8B4513"
},
cancelButtonText: {
  color: "#333",
  fontWeight: "500"
},
confirmButtonText: {
  color: "white",
  fontWeight: "500"
},
loadingContainer: {
  flex: 1,
  justifyContent: "center",
  alignItems: "center",
  padding: 32
},
loadingText: {
  marginTop: 12,
```

```
    color: "#666",
    fontSize: 14
  },
  emptyListContent: {
    flex: 1
  },
  emptySubtext: {
    marginTop: 8,
    fontSize: 14,
    color: "#aaa",
    textAlign: "center"
  }
});
```

FILE: pos/mobile/app/Notification.jsx
==========

```
import { Ionicons } from "@expo/vector-icons";
import { useNavigation, useFocusEffect } from "@react-navigation/native";
import { useState, useCallback } from "react";
import {
  ActivityIndicator,
  FlatList,
  RefreshControl,
  StyleSheet,
  Text,
  TouchableOpacity,
  View
} from "react-native";
import { stockAPI } from "../services/api";
export default function Notification() {
  const navigation = useNavigation();
  const [notifications, setNotifications] = useState([]);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const fetchNotifications = async () => {
    try {
      const response = await stockAPI.getLowStock();
      if (response && response.success && response.data) {
        const stockAlerts = response.data.map((item) => ({
          id: item._id,
          title: item.alertType === "out_of_stock" ? `${item.itemName} is OUT OF STOCK` : `${item.itemName}
is running low`,
          description: `Current stock: ${item.currentStock} ${item.reorderNumber ? `| Reorder level:
${item.reorderNumber}` : ""}`,
          time: item.alertType === "out_of_stock" ? "Urgent" : "Low Stock",
          isRead: false,
          type: item.alertType,
          itemData: item
        }));
        setNotifications(stockAlerts);
      } else {
        setNotifications([]);
      }
    } catch (error) {
      console.error("Error fetching notifications:", error);
      setNotifications([]);
    } finally {
      setLoading(false);
      setRefreshing(false);
    }
  };
  useFocusEffect(
    useCallback(() => {
      fetchNotifications();
    }, [])
  );
  const onRefresh = () => {
    setRefreshing(true);
    fetchNotifications();
  };
  const getIconColor = (type) => {
    return type === "out_of_stock" ? "#D45D3E" : "#E6A23C";
  };
  const getIconBg = (type) => {
    return type === "out_of_stock" ? "#FFE8E0" : "#FFF6E0";
```

```
  };
  const getIconName = (type) => {
    return type === "out_of_stock" ? "alert-circle" : "warning";
  };
  const renderEmptyState = () => /* @__PURE__ */ React.createElement(View, { style: styles.emptyState }, /*
@__PURE__ */ React.createElement(Ionicons, { name: "checkmark-circle-outline", size: 80, color: "#4CAF50" }),
/* @__PURE__ */ React.createElement(Text, { style: styles.emptyText }, "All Stocks Healthy"), /* @__PURE__ */
React.createElement(Text, { style: styles.emptySubtext }, "No low stock or out-of-stock items found."));
  return /* @__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */
React.createElement(View, { style: styles.headerContainer }, /* @__PURE__ */ React.createElement(View,
{ style: styles.headerContent }, /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: styles.backButton,
      onPress: () => navigation.goBack(),
      activeOpacity: 0.7
    },
    /* @__PURE__ */ React.createElement(Ionicons, { name: "arrow-back", size: 24, color: "#76462B" })
  ), /* @__PURE__ */ React.createElement(Text, { style: styles.headerTitle }, "Stock Alerts"), /* @__PURE__
*/ React.createElement(View, { style: { width: 40 } })), notifications.length > 0 && /* @__PURE__ */
React.createElement(View, { style: styles.unreadBadge }, /* @__PURE__ */ React.createElement(Text, { style:
styles.unreadText }, notifications.length, " ", notifications.length === 1 ? "alert" : "alerts"))), loading ?
/* @__PURE__ */ React.createElement(View, { style: styles.loadingContainer }, /* @__PURE__ */
React.createElement(ActivityIndicator, { size: "large", color: "#AD7F65" }), /* @__PURE__ */
React.createElement(Text, { style: styles.loadingText }, "Checking stock levels...")) : /* @__PURE__ */
React.createElement(
    FlatList,
    {
      data: notifications,
      keyExtractor: (item) => item.id,
      contentContainerStyle: notifications.length === 0 ? styles.emptyListContent : styles.listContent,
      showsVerticalScrollIndicator: false,
      ListEmptyComponent: renderEmptyState,
      refreshControl: /* @__PURE__ */ React.createElement(
        RefreshControl,
        {
          refreshing,
          onRefresh,
          colors: ["#AD7F65"],
          tintColor: "#AD7F65"
        }
      ),
      renderItem: ({ item }) => /* @__PURE__ */ React.createElement(
        TouchableOpacity,
        {
          style: [
            styles.card,
            item.type === "out_of_stock" && styles.outOfStockCard,
            item.type === "low_stock" && styles.lowStockCard
          ],
          activeOpacity: 0.9
        },
        /* @__PURE__ */ React.createElement(View, { style: [
          styles.iconContainer,
          { backgroundColor: getIconBg(item.type) }
        ] }, /* @__PURE__ */ React.createElement(
          Ionicons,
          {
            name: getIconName(item.type),
            size: 24,
            color: getIconColor(item.type)
          }
        )),
        /* @__PURE__ */ React.createElement(View, { style: styles.textContainer }, /* @__PURE__ */
React.createElement(View, { style: styles.titleRow }, /* @__PURE__ */ React.createElement(Text, { style: [
          styles.title,
          item.type === "out_of_stock" ? styles.outOfStockTitle : styles.lowStockTitle
        ] }, item.title)), /* @__PURE__ */ React.createElement(Text, { style: styles.description },
item.description), /* @__PURE__ */ React.createElement(Text, { style: styles.skuText }, "SKU: ",
item.itemData.sku))
      )
    }
  ));
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
```

```
  backgroundColor: "#F9F5F0"
},
headerContainer: {
  backgroundColor: "#FFFFFF",
  paddingTop: 50,
  paddingBottom: 15,
  borderBottomLeftRadius: 20,
  borderBottomRightRadius: 20,
  marginBottom: 10,
  shadowColor: "#000",
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.05,
  shadowRadius: 10,
  elevation: 5
},
headerContent: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  paddingHorizontal: 20
},
backButton: {
  padding: 8,
  borderRadius: 12,
  backgroundColor: "#F0E6DD"
},
headerTitle: {
  fontSize: 22,
  fontWeight: "700",
  color: "#3E3E3E",
  textAlign: "center",
  flex: 1,
  marginRight: 0
  // Adjusted simple alignment
},
unreadBadge: {
  backgroundColor: "#FFE8E0",
  alignSelf: "center",
  paddingVertical: 4,
  paddingHorizontal: 12,
  borderRadius: 12,
  marginTop: 10
},
unreadText: {
  color: "#D45D3E",
  fontSize: 12,
  fontWeight: "600"
},
listContent: {
  padding: 15,
  paddingTop: 10,
  paddingBottom: 30
},
emptyListContent: {
  padding: 15,
  paddingTop: 10,
  paddingBottom: 30,
  flex: 1,
  justifyContent: "center"
},
card: {
  flexDirection: "row",
  backgroundColor: "#FFFFFF",
  borderRadius: 12,
  padding: 16,
  marginBottom: 10,
  shadowColor: "#000",
  shadowOffset: { width: 0, height: 1 },
  shadowOpacity: 0.05,
  shadowRadius: 4,
  elevation: 2,
  borderWidth: 1,
  borderColor: "#F0F0F0",
  alignItems: "center"
},
outOfStockCard: {
  borderLeftWidth: 4,
```

```
    borderLeftColor: "#D45D3E"
  },
  lowStockCard: {
    borderLeftWidth: 4,
    borderLeftColor: "#E6A23C"
  },
  iconContainer: {
    width: 48,
    height: 48,
    borderRadius: 12,
    justifyContent: "center",
    alignItems: "center",
    marginRight: 16
  },
  textContainer: {
    flex: 1,
    justifyContent: "center"
  },
  titleRow: {
    flexDirection: "row",
    alignItems: "center",
    marginBottom: 4
  },
  title: {
    fontSize: 15,
    fontWeight: "600",
    color: "#2D2D2D",
    lineHeight: 20
  },
  outOfStockTitle: {
    color: "#D45D3E"
  },
  lowStockTitle: {
    color: "#D9822B"
    // Slightly darker orange for text readability
  },
  description: {
    fontSize: 13.5,
    color: "#5A5A5A",
    marginBottom: 4,
    lineHeight: 19
  },
  skuText: {
    fontSize: 12,
    color: "#8E8E8E",
    fontWeight: "500"
  },
  loadingContainer: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center"
  },
  loadingText: {
    marginTop: 12,
    color: "#666",
    fontSize: 14
  },
  emptyState: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    padding: 40
  },
  emptyText: {
    fontSize: 18,
    fontWeight: "600",
    color: "#3E3E3E",
    marginTop: 16,
    marginBottom: 8
  },
  emptySubtext: {
    fontSize: 14,
    color: "#8E8E8E",
    textAlign: "center"
  }
});
```

FILE: pos/mobile/app/pinlogin.tsx
==========

```
import AsyncStorage from "@react-native-async-storage/async-storage";
import * as LocalAuthentication from "expo-local-authentication";
import { useRouter } from "expo-router";
import { useEffect, useState } from "react";
import {
  ActivityIndicator,
  Image,
  Modal,
  StyleSheet,
  Text,
  TouchableOpacity,
  View
} from "react-native";
import config from "../config/api";
import { employeeAPI } from "../services/api";
export default function PinLogin() {
  const [pin, setPin] = useState("");
  const PIN_LENGTH = 6;
  const [showModal, setShowModal] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [errorMessage, setErrorMessage] = useState("");
  const [showBiometricModal, setShowBiometricModal] = useState(false);
  const [biometricAvailable, setBiometricAvailable] = useState(false);
  const [biometricEnabled, setBiometricEnabled] = useState(false);
  const router = useRouter();
  useEffect(() => {
    fetch(`${config.API_URL}/ping`).catch(() => {
    });
    checkBiometricOnMount();
  }, []);
  const checkBiometricOnMount = async () => {
    try {
      if (!LocalAuthentication?.hasHardwareAsync) return;
      const hasHardware = await LocalAuthentication.hasHardwareAsync();
      const isEnrolled = await LocalAuthentication.isEnrolledAsync();
      const hasBiometric = hasHardware && isEnrolled;
      setBiometricAvailable(hasBiometric);
      if (!hasBiometric) return;
      const enabled = await AsyncStorage.getItem("@biometric_enabled");
      if (enabled === "true") {
        setBiometricEnabled(true);
        attemptBiometricLogin();
      }
    } catch (error) {
      setBiometricAvailable(false);
    }
  };
  const attemptBiometricLogin = async () => {
    try {
      const result = await LocalAuthentication.authenticateAsync({
        promptMessage: "Verify to login",
        cancelLabel: "Use PIN",
        disableDeviceFallback: false
      });
      if (result.success) {
        const storedEmployee = await AsyncStorage.getItem("currentEmployee");
        if (storedEmployee) {
          router.replace("/(tabs)");
        } else {
          console.log("No stored employee data, falling back to PIN");
        }
      }
    } catch (error) {
      console.error("Biometric auth error:", error);
    }
  };
  const handleKeyPress = (value) => {
    if (value === "del") {
      setPin(pin.slice(0, -1));
    } else if (pin.length < PIN_LENGTH) {
      setPin(pin + value);
    }
  };
```

```
  const handleLogin = async () => {
    if (pin.length !== PIN_LENGTH) return;
    setIsLoading(true);
    setErrorMessage("");
    try {
      const response = await employeeAPI.verifyOwnerPin(pin);
      if (response.success && response.data) {
        await AsyncStorage.setItem("currentEmployee", JSON.stringify(response.data));
        if (biometricAvailable) {
          const alreadyAsked = await AsyncStorage.getItem("@biometric_asked");
          if (alreadyAsked !== "true") {
            setShowBiometricModal(true);
            setIsLoading(false);
            return;
          }
        }
        router.replace("/(tabs)");
      } else {
        setErrorMessage(response.message || "Invalid PIN. Only owner account can access the mobile app.");
        setShowModal(true);
        setPin("");
      }
    } catch (error) {
      console.error("Login error:", error);
      setErrorMessage(error.message || "Failed to verify PIN. Please check your connection.");
      setShowModal(true);
      setPin("");
    } finally {
      setIsLoading(false);
    }
  };
  const handleBiometricAllow = async () => {
    setShowBiometricModal(false);
    try {
      const result = await LocalAuthentication.authenticateAsync({
        promptMessage: "Verify your identity",
        cancelLabel: "Cancel",
        disableDeviceFallback: true
      });
      if (result.success) {
        await AsyncStorage.setItem("@biometric_enabled", "true");
        await AsyncStorage.setItem("@biometric_asked", "true");
        setBiometricEnabled(true);
      } else {
        await AsyncStorage.setItem("@biometric_asked", "true");
      }
    } catch (error) {
      console.error("Biometric setup error:", error);
      await AsyncStorage.setItem("@biometric_asked", "true");
    }
    router.replace("/(tabs)");
  };
  const handleBiometricSkip = async () => {
    setShowBiometricModal(false);
    await AsyncStorage.setItem("@biometric_asked", "true");
    router.replace("/(tabs)");
  };
  return /* @__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */
React.createElement(View, { style: styles.header }, /* @__PURE__ */ React.createElement(
    Image,
    {
      source: require("../app/(tabs)/iconz/pinbg.png"),
      style: styles.headerImage,
      resizeMode: "cover"
    }
  )), /* @__PURE__ */ React.createElement(View, { style: styles.card }, /* @__PURE__ */
React.createElement(Text, { style: styles.title }, "Enter your PIN"), /* @__PURE__ */
React.createElement(View, { style: styles.pinContainer }, Array.from({ length: PIN_LENGTH }, (_, i) =>
i).map((i) => /* @__PURE__ */ React.createElement(
    View,
    {
      key: i,
      style: [
        styles.pinBox,
        pin[i] ? styles.pinBoxFilled : null
      ]
    },
```

```
    /* @__PURE__ */ React.createElement(Text, { style: [styles.pinText, pin[i] ? styles.pinFilled : null] },
pin[i] ? "\u2022" : "")
  ))), /* @__PURE__ */ React.createElement(View, { style: styles.keypad }, [
    ["1", "2", "3"],
    ["4", "5", "6"],
    ["7", "8", "9"],
    [biometricEnabled ? "bio" : "empty", "0", "del"]
  ].map((row, rIdx) => /* @__PURE__ */ React.createElement(View, { key: rIdx, style: styles.keypadRow },
row.map((key) => key === "empty" ? /* @__PURE__ */ React.createElement(View, { key, style: [styles.key,
{ backgroundColor: "transparent", elevation: 0, shadowOpacity: 0 }] }) : key === "bio" ? /* @__PURE__ */
React.createElement(
    TouchableOpacity,
    {
      key,
      style: [styles.key, { backgroundColor: "#f0f7f0" }],
      onPress: attemptBiometricLogin
    },
    /* @__PURE__ */ React.createElement(Text, { style: { fontSize: 26 } }, "\u{1F512}")
  ) : /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      key,
      style: styles.key,
      onPress: () => handleKeyPress(key)
    },
    /* @__PURE__ */ React.createElement(Text, { style: styles.keyText }, key === "del" ? "\u232B" : key)
  )))), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: [
        styles.loginButton,
        (pin.length !== 6 || isLoading) && styles.loginButtonDisabled
      ],
      disabled: pin.length !== 6 || isLoading,
      onPress: handleLogin
    },
    isLoading ? /* @__PURE__ */ React.createElement(ActivityIndicator, { color: "#fff" }) : /* @__PURE__ */
React.createElement(Text, { style: styles.loginButtonText }, "LOGIN")
  )), /* @__PURE__ */ React.createElement(Modal, { visible: showModal, transparent: true, animationType:
"fade" }, /* @__PURE__ */ React.createElement(View, { style: styles.modalOverlay }, /* @__PURE__ */
React.createElement(View, { style: styles.modalBox }, /* @__PURE__ */ React.createElement(Text, { style:
styles.modalTitle }, "Incorrect PIN"), /* @__PURE__ */ React.createElement(Text, { style:
styles.modalMessage }, errorMessage || "The PIN you entered is incorrect. Please try again."), /* @__PURE__
*/ React.createElement(
    TouchableOpacity,
    {
      style: styles.modalButton,
      onPress: () => setShowModal(false)
    },
    /* @__PURE__ */ React.createElement(Text, { style: styles.modalButtonText }, "OK")
  )))), /* @__PURE__ */ React.createElement(Modal, { visible: showBiometricModal, transparent: true,
animationType: "fade" }, /* @__PURE__ */ React.createElement(View, { style: styles.modalOverlay }, /*
@__PURE__ */ React.createElement(View, { style: styles.modalBox }, /* @__PURE__ */ React.createElement(Text,
{ style: { fontSize: 48, marginBottom: 16 } }, "\u{1F510}"), /* @__PURE__ */ React.createElement(Text,
{ style: styles.modalTitle }, "Want to use phone biometrics?"), /* @__PURE__ */ React.createElement(Text,
{ style: styles.modalMessage }, "Use your fingerprint or face to login faster next time. You can always
change this in settings."), /* @__PURE__ */ React.createElement(View, { style: styles.biometricButtons }, /*
@__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: [styles.biometricBtn, styles.biometricBtnSkip],
      onPress: handleBiometricSkip
    },
    /* @__PURE__ */ React.createElement(Text, { style: styles.biometricBtnSkipText }, "Not Now")
  ), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: [styles.biometricBtn, styles.biometricBtnAllow],
      onPress: handleBiometricAllow
    },
    /* @__PURE__ */ React.createElement(Text, { style: styles.biometricBtnAllowText }, "Allow")
  ))))));
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#111"
```

```
},
header: {
  height: "30%",
  justifyContent: "flex-end",
  alignItems: "center"
},
headerImage: {
  width: "100%",
  height: "100%"
},
card: {
  flex: 1,
  backgroundColor: "#fff",
  marginTop: -30,
  borderTopLeftRadius: 15,
  borderTopRightRadius: 15,
  alignItems: "center",
  paddingTop: 40
},
title: {
  fontSize: 14,
  fontWeight: "bold",
  marginBottom: 20,
  color: "#000"
},
pinContainer: {
  flexDirection: "row",
  justifyContent: "center",
  marginBottom: 20
},
pinBox: {
  width: 42,
  height: 48,
  borderRadius: 6,
  marginHorizontal: 4,
  backgroundColor: "#fff",
  borderWidth: 1,
  borderColor: "#E0E0E0",
  justifyContent: "center",
  alignItems: "center"
},
pinBoxFilled: {
  borderColor: "#333"
},
pinFilled: {
  color: "#000"
},
pinText: {
  fontSize: 24,
  fontWeight: "bold",
  color: "#000",
  textAlign: "center"
},
keypad: {
  marginBottom: 12
},
keypadRow: {
  flexDirection: "row",
  justifyContent: "center",
  marginVertical: 8
},
key: {
  width: 65,
  height: 65,
  marginHorizontal: 10,
  backgroundColor: "#F5F5F5",
  borderRadius: 32.5,
  justifyContent: "center",
  alignItems: "center"
},
keyText: {
  fontSize: 22,
  fontWeight: "600",
  color: "#000"
},
loginButton: {
  width: "65%",
```

```
  backgroundColor: "rgb(38, 38, 38)",
  borderRadius: 8,
  paddingVertical: 14,
  alignItems: "center",
  alignSelf: "center",
  marginTop: 10
},
loginButtonDisabled: {
  opacity: 0.6
},
loginButtonText: {
  color: "#fff",
  fontSize: 14,
  fontWeight: "bold"
},
// Modal Styles
modalOverlay: {
  flex: 1,
  backgroundColor: "rgba(0,0,0,0.5)",
  justifyContent: "center",
  alignItems: "center"
},
modalBox: {
  backgroundColor: "#fff",
  width: "80%",
  borderRadius: 20,
  padding: 25,
  alignItems: "center"
},
modalTitle: {
  fontSize: 22,
  fontWeight: "bold",
  color: "#7C4A21",
  marginBottom: 10,
  textAlign: "center"
},
modalMessage: {
  textAlign: "center",
  fontSize: 16,
  color: "#555",
  marginBottom: 20
},
modalButton: {
  backgroundColor: "#8b5e39ff",
  borderRadius: 10,
  paddingVertical: 10,
  paddingHorizontal: 30
},
modalButtonText: {
  color: "#fff",
  fontSize: 16,
  fontWeight: "bold"
},
biometricButtons: {
  flexDirection: "row",
  width: "100%",
  justifyContent: "space-between",
  gap: 12
},
biometricBtn: {
  flex: 1,
  paddingVertical: 14,
  borderRadius: 12,
  alignItems: "center"
},
biometricBtnSkip: {
  backgroundColor: "#f3f4f6",
  borderWidth: 1,
  borderColor: "#e5e7eb"
},
biometricBtnSkipText: {
  color: "#6b7280",
  fontSize: 15,
  fontWeight: "600"
},
biometricBtnAllow: {
  backgroundColor: "#1f1f1f"
```

```
  },
  biometricBtnAllowText: {
    color: "#fff",
    fontSize: 15,
    fontWeight: "600"
  }
});



FILE: pos/mobile/app/Preferences.jsx
================================================================================
import { useEffect, useState } from "react";
import {
  Alert,
  StyleSheet,
  Switch,
  Text,
  View
} from "react-native";
import {
  isNotificationsEnabled,
  requestPermissions,
  setNotificationsEnabled,
  setupNotificationChannel
} from "../services/notificationService";
export default function SettingsScreen() {
  const [notifications, setNotifications] = useState(false);
  const [loading, setLoading] = useState(true);
  useEffect(() => {
    (async () => {
      const enabled = await isNotificationsEnabled();
      setNotifications(enabled);
      setLoading(false);
    })();
  }, []);
  const toggleNotifications = async () => {
    const newValue = !notifications;
    if (newValue) {
      await setupNotificationChannel();
      const granted = await requestPermissions();
      if (!granted) {
        Alert.alert(
          "Permission Required",
          "Please enable notifications in your device settings to receive stock alerts.",
          [{ text: "OK" }]
        );
        return;
      }
    }
    setNotifications(newValue);
    await setNotificationsEnabled(newValue);
  };
  if (loading) return null;
  return /* @__PURE__ */ React.createElement(
    View,
    {
      style: [
        styles.container,
        { backgroundColor: "#FAF6F2" }
      ]
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.switchContainer }, /* @__PURE__ */
React.createElement(View, null, /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Push
Notifications"), /* @__PURE__ */ React.createElement(Text, { style: styles.sublabel }, notifications ? "You
will receive stock alerts" : "Notifications are off")), /* @__PURE__ */ React.createElement(
      Switch,
      {
        trackColor: { false: "#ccc", true: "#16a34a" },
        thumbColor: "#fff",
        value: notifications,
        onValueChange: toggleNotifications
      }
    ))
  );
}
const styles = StyleSheet.create({
```

```
  container: {
    flex: 1,
    paddingHorizontal: 20,
    paddingTop: 20
  },
  switchContainer: {
    flexDirection: "row",
    alignItems: "center",
    justifyContent: "space-between",
    backgroundColor: "#fff",
    padding: 15,
    borderRadius: 15,
    shadowColor: "#000",
    shadowOpacity: 0.1,
    shadowRadius: 3,
    elevation: 2,
    marginBottom: 15
  },
  label: {
    fontSize: 16,
    fontWeight: "500",
    color: "#333"
  },
  sublabel: {
    fontSize: 12,
    color: "#999",
    marginTop: 3
  }
});



FILE: pos/mobile/app/RolesAndPermission.jsx
==============================================================================
import { Ionicons } from "@expo/vector-icons";
import DateTimePicker from "@react-native-community/datetimepicker";
import { Picker } from "@react-native-picker/picker";
import { useLocalSearchParams, useRouter } from "expo-router";
import { useEffect, useState, useCallback } from "react";
import { useFocusEffect } from "@react-navigation/native";
import {
  Alert,
  Modal,
  Platform,
  ScrollView,
  StyleSheet,
  Switch,
  Text,
  TextInput,
  TouchableOpacity,
  View,
  ActivityIndicator
} from "react-native";
import { employeeAPI } from "../services/api";
const RolesAndPermission = () => {
  const router = useRouter();
  const [employees, setEmployees] = useState([]);
  const [filteredEmployees, setFilteredEmployees] = useState([]);
  const [loading, setLoading] = useState(true);
  const [searchQuery, setSearchQuery] = useState("");
  const [editingEmployee, setEditingEmployee] = useState(null);
  const [isEditModalVisible, setIsEditModalVisible] = useState(false);
  const [showStatusModal, setShowStatusModal] = useState(false);
  const [selectedEmployee, setSelectedEmployee] = useState(null);
  const [showDatePicker, setShowDatePicker] = useState(false);
  const [saving, setSaving] = useState(false);
  const [formData, setFormData] = useState({
    name: "",
    email: "",
    position: "Cashier",
    // Maps to 'role'
    contactNumber: "",
    dateJoined: (/* @__PURE__ */ new Date()).toISOString().split("T")[0],
    isActive: true,
    // Maps to 'status'
    permissions: {
      pos: false,
```

```
      inventory: false,
      viewTransaction: false,
      generateReports: false
    }
  });
  const loadEmployees = async () => {
    try {
      setLoading(true);
      const res = await employeeAPI.getAll();
      if (res && res.success && res.data) {
        const staff = res.data.filter((emp) => emp.role.toLowerCase() !== "owner");
        setEmployees(staff);
        setFilteredEmployees(staff);
      }
    } catch (error) {
      console.error("Failed to load employees:", error);
      Alert.alert("Error", "Failed to load employees.");
    } finally {
      setLoading(false);
    }
  };
  useFocusEffect(
    useCallback(() => {
      loadEmployees();
      return () => {
      };
    }, [])
  );
  const handleEdit = (employee) => {
    setEditingEmployee(employee);
    setFormData({
      name: employee.name,
      email: employee.email,
      position: employee.role,
      // Mapping role -> position for UI consistency
      contactNumber: employee.contactNo,
      dateJoined: employee.dateJoined ? new Date(employee.dateJoined).toISOString().split("T")[0] : (/*
@__PURE__ */ new Date()).toISOString().split("T")[0],
      isActive: employee.status === "Active",
      permissions: {
        pos: false,
        inventory: false,
        viewTransaction: false,
        generateReports: false,
        ...employee.permissions || {}
      }
    });
    setIsEditModalVisible(true);
  };
  const handleSave = async () => {
    if (!formData.name || !formData.email || !formData.position) {
      Alert.alert("Error", "Please fill in all required fields.");
      return;
    }
    try {
      setSaving(true);
      const payload = {
        name: formData.name,
        email: formData.email,
        role: formData.position,
        // Map UI position to backend role
        contactNo: formData.contactNumber,
        dateJoined: new Date(formData.dateJoined),
        status: formData.isActive ? "Active" : "Inactive",
        permissions: formData.permissions
      };
      let res;
      if (editingEmployee) {
        res = await employeeAPI.update(editingEmployee._id || editingEmployee.id, payload);
      } else {
        const randomPin = Math.floor(1e5 + Math.random() * 9e5).toString();
        const newEmployeePayload = {
          ...payload,
          pin: randomPin,
          firstName: formData.name.split(" ")[0],
          // Simple split
          lastName: formData.name.split(" ").slice(1).join(" ") || "",
```

```
          requiresPinReset: true,
          sendPinEmail: true
          // Backend handles email sending
        };
        res = await employeeAPI.create(newEmployeePayload);
      }
      if (res && res.success) {
        Alert.alert(
          "Success",
          editingEmployee ? "Employee updated successfully." : "Employee added successfully. A temporary PIN
has been sent to their email.",
          [{ text: "OK", onPress: () => setIsEditModalVisible(false) }]
        );
        loadEmployees();
      } else {
        Alert.alert("Error", res?.message || "Failed to save employee.");
      }
    } catch (error) {
      console.error("Save error:", error);
      Alert.alert("Error", "An unexpected error occurred.");
    } finally {
      setSaving(false);
    }
  };
  const toggleEmployeeStatus = (employee) => {
    setSelectedEmployee(employee);
    setShowStatusModal(true);
  };
  const confirmStatusChange = async () => {
    if (selectedEmployee) {
      try {
        const newStatus = selectedEmployee.status === "Active" ? "Inactive" : "Active";
        const res = await employeeAPI.update(selectedEmployee._id || selectedEmployee.id, { status:
newStatus });
        if (res && res.success) {
          loadEmployees();
        } else {
          Alert.alert("Error", "Failed to update status.");
        }
      } catch (error) {
        console.error("Status update error:", error);
        Alert.alert("Error", "Failed to update status.");
      }
    }
    setShowStatusModal(false);
  };
  const archiveEmployee = (employee) => {
    Alert.alert(
      "Delete Employee",
      `Are you sure you want to delete ${employee.name}? This action cannot be undone.`,
      [
        {
          text: "Cancel",
          style: "cancel"
        },
        {
          text: "Delete",
          onPress: async () => {
            try {
              const res = await employeeAPI.delete(employee._id || employee.id);
              if (res && res.success) {
                loadEmployees();
              } else {
                Alert.alert("Error", res?.message || "Failed to delete employee.");
              }
            } catch (error) {
              console.error("Delete error:", error);
              Alert.alert("Error", "Failed to delete employee.");
            }
          },
          style: "destructive"
        }
      ]
    );
  };
  const handlePermissionToggle = (permission) => {
    setFormData({
```

```
      ...formData,
      permissions: {
        ...formData.permissions,
        [permission]: !formData.permissions[permission]
      }
    });
  };
  const handleInputChange = (field, value) => {
    setFormData({
      ...formData,
      [field]: value
    });
  };
  const handleSearch = (query) => {
    setSearchQuery(query);
    if (query === "") {
      setFilteredEmployees(employees);
    } else {
      const lowerQuery = query.toLowerCase();
      const filtered = employees.filter(
        (employee) => employee.name && employee.name.toLowerCase().includes(lowerQuery) || employee.email &&
employee.email.toLowerCase().includes(lowerQuery) || employee.role &&
employee.role.toLowerCase().includes(lowerQuery)
      );
      setFilteredEmployees(filtered);
    }
  };
  if (loading && !employees.length) {
    return /* @__PURE__ */ React.createElement(View, { style: [styles.container, { justifyContent: "center",
alignItems: "center" }] }, /* @__PURE__ */ React.createElement(ActivityIndicator, { size: "large", color:
"#AD7F65" }));
  }
  return /* @__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */
React.createElement(View, { style: styles.searchContainer }, /* @__PURE__ */ React.createElement(View,
{ style: styles.searchInputContainer }, /* @__PURE__ */ React.createElement(Ionicons, { name: "search", size:
20, color: "#666", style: styles.searchIcon }), /* @__PURE__ */ React.createElement(
    TextInput,
    {
      style: styles.searchInput,
      placeholder: "Search employees...",
      value: searchQuery,
      onChangeText: handleSearch,
      placeholderTextColor: "#999"
    }
  )), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      onPress: () => router.push("/Archive"),
      style: styles.addButton
    },
    /* @__PURE__ */ React.createElement(Ionicons, { name: "archive", size: 24, color: "white" })
  ), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: styles.addButton,
      onPress: () => {
        setEditingEmployee(null);
        setFormData({
          name: "",
          email: "",
          position: "Cashier",
          contactNumber: "",
          dateJoined: (/* @__PURE__ */ new Date()).toISOString().split("T")[0],
          isActive: true,
          permissions: {
            pos: false,
            inventory: false,
            viewTransaction: false,
            generateReports: false
          }
        });
        setIsEditModalVisible(true);
      }
    },
    /* @__PURE__ */ React.createElement(Ionicons, { name: "add", size: 24, color: "white" })
  )), /* @__PURE__ */ React.createElement(ScrollView, { style: styles.scrollView }, filteredEmployees.length
=== 0 ? /* @__PURE__ */ React.createElement(View, { style: { padding: 20, alignItems: "center" } }, /*
```

```
@__PURE__ */ React.createElement(Text, { style: { color: "#888" } }, "No employees found.")) :
filteredEmployees.map((employee) => /* @__PURE__ */ React.createElement(View, { key: employee._id ||
employee.id, style: [
    styles.employeeCard,
    employee.status !== "Active" && styles.inactiveEmployee
  ] }, /* @__PURE__ */ React.createElement(View, { style: styles.employeeHeader }, /* @__PURE__ */
React.createElement(View, { style: styles.employeeInfo }, /* @__PURE__ */ React.createElement(Text, { style:
styles.employeeName }, employee.name), /* @__PURE__ */ React.createElement(Text, { style:
styles.employeePosition }, employee.role), /* @__PURE__ */ React.createElement(View, { style:
[styles.statusBadge, employee.status === "Active" ? styles.activeBadge : styles.inactiveBadge] }, /*
@__PURE__ */ React.createElement(Text, { style: styles.statusText }, employee.status || "Inactive"))), /*
@__PURE__ */ React.createElement(View, { style: styles.employeeActions }, /* @__PURE__ */
React.createElement(
    TouchableOpacity,
    {
      style: styles.actionButton,
      onPress: () => handleEdit(employee)
    },
    /* @__PURE__ */ React.createElement(Ionicons, { name: "pencil", size: 20, color: "#4A90E2" })
  ), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      onPress: () => archiveEmployee(employee),
      style: styles.actionButton
    },
    /* @__PURE__ */ React.createElement(Ionicons, { name: "trash", size: 20, color: "#f44336" })
  ), /* @__PURE__ */ React.createElement(
    Switch,
    {
      value: employee.status === "Active",
      onValueChange: () => toggleEmployeeStatus(employee),
      trackColor: { false: "#D3D3D3", true: "#AD7F65" },
      thumbColor: "#FFFFFF"
    }
  ))), /* @__PURE__ */ React.createElement(View, { style: styles.employeeDetails }, /* @__PURE__ */
React.createElement(View, { style: styles.detailRow }, /* @__PURE__ */ React.createElement(Ionicons, { name:
"mail", size: 16, color: "#666" }), /* @__PURE__ */ React.createElement(Text, { style: styles.detailText },
employee.email)), /* @__PURE__ */ React.createElement(View, { style: styles.detailRow }, /* @__PURE__ */
React.createElement(Ionicons, { name: "call", size: 16, color: "#666" }), /* @__PURE__ */
React.createElement(Text, { style: styles.detailText }, employee.contactNo)), /* @__PURE__ */
React.createElement(View, { style: styles.detailRow }, /* @__PURE__ */ React.createElement(Ionicons, { name:
"calendar", size: 16, color: "#666" }), /* @__PURE__ */ React.createElement(Text, { style:
styles.detailText }, "Joined: ", employee.dateJoined ? new Date(employee.dateJoined).toLocaleDateString() :
"N/A"))), /* @__PURE__ */ React.createElement(View, { style: styles.permissionsContainer }, /* @__PURE__ */
React.createElement(Text, { style: styles.permissionsTitle }, "Permissions:"), /* @__PURE__ */
React.createElement(View, { style: styles.permissionsGrid }, employee.permissions &&
Object.entries(employee.permissions).map(([key, value]) => /* @__PURE__ */ React.createElement(View, { key,
style: styles.permissionItem }, /* @__PURE__ */ React.createElement(Text, { style: styles.permissionText },
key.split(/(?=[A-Z])/).join(" ").toUpperCase()), /* @__PURE__ */ React.createElement(View, { style: [
    styles.permissionBadge,
    value ? styles.permissionGranted : styles.permissionDenied
  ] }, /* @__PURE__ */ React.createElement(Text, { style: styles.permissionBadgeText }, value ? "\u2713" :
"\u2717")))), (!employee.permissions || Object.keys(employee.permissions).length === 0) && /* @__PURE__ */
React.createElement(Text, { style: { fontSize: 12, color: "#999", fontStyle: "italic" } }, "No specific
permissions set")))))), /* @__PURE__ */ React.createElement(
    Modal,
    {
      visible: isEditModalVisible,
      animationType: "slide",
      transparent: true,
      onRequestClose: () => !saving && setIsEditModalVisible(false)
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.modalOverlay }, /* @__PURE__ */
React.createElement(View, { style: styles.modalContent }, /* @__PURE__ */ React.createElement(View, { style:
styles.modalHeader }, /* @__PURE__ */ React.createElement(Text, { style: styles.modalTitle },
editingEmployee ? "Edit Employee" : "Add New Employee"), /* @__PURE__ */ React.createElement(TouchableOpacity,
{ onPress: () => !saving && setIsEditModalVisible(false), disabled: saving }, /* @__PURE__ */
React.createElement(Ionicons, { name: "close", size: 24, color: "#666" }))), /* @__PURE__ */
React.createElement(ScrollView, { style: styles.modalBody }, /* @__PURE__ */ React.createElement(View,
{ style: styles.formGroup }, /* @__PURE__ */ React.createElement(Text, { style: styles.label }, "Full Name"),
/* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: styles.input,
        value: formData.name,
        onChangeText: (text) => handleInputChange("name", text),
        placeholder: "Enter full name",
```

```
        editable: !saving
      }
    )), /* @__PURE__ */ React.createElement(View, { style: styles.formGroup }, /* @__PURE__ */
React.createElement(Text, { style: styles.label }, "Email"), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: styles.input,
        value: formData.email,
        onChangeText: (text) => handleInputChange("email", text),
        placeholder: "Enter email address",
        keyboardType: "email-address",
        autoCapitalize: "none",
        editable: !saving
      }
    ), !editingEmployee && /* @__PURE__ */ React.createElement(Text, { style: styles.hintText }, "A temporary
PIN will be sent to this email.")), /* @__PURE__ */ React.createElement(View, { style: styles.formGroup }, /*
@__PURE__ */ React.createElement(Text, { style: styles.label }, "Position (Role)"), /* @__PURE__ */
React.createElement(View, { style: styles.pickerContainer }, /* @__PURE__ */ React.createElement(
      Picker,
      {
        selectedValue: formData.position,
        onValueChange: (itemValue) => handleInputChange("position", itemValue),
        style: styles.picker,
        enabled: !saving
      },
      /* @__PURE__ */ React.createElement(Picker.Item, { label: "Cashier", value: "Cashier" }),
      /* @__PURE__ */ React.createElement(Picker.Item, { label: "Manager", value: "Manager" }),
      /* @__PURE__ */ React.createElement(Picker.Item, { label: "Admin", value: "Admin" })
    ))), /* @__PURE__ */ React.createElement(View, { style: styles.formGroup }, /* @__PURE__ */
React.createElement(Text, { style: styles.label }, "Contact Number"), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: styles.input,
        value: formData.contactNumber,
        onChangeText: (text) => handleInputChange("contactNumber", text),
        placeholder: "Enter contact number",
        keyboardType: "phone-pad",
        editable: !saving
      }
    )), /* @__PURE__ */ React.createElement(View, { style: styles.formGroup }, /* @__PURE__ */
React.createElement(Text, { style: styles.label }, "Date Joined"), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        onPress: () => !saving && setShowDatePicker(true),
        style: styles.dateInputContainer,
        disabled: saving
      },
      /* @__PURE__ */ React.createElement(Text, { style: styles.dateInputText }, formData.dateJoined ||
"Select date"),
      /* @__PURE__ */ React.createElement(Ionicons, { name: "calendar", size: 20, color: "#666" })
    ), showDatePicker && /* @__PURE__ */ React.createElement(
      DateTimePicker,
      {
        value: formData.dateJoined ? new Date(formData.dateJoined) : /* @__PURE__ */ new Date(),
        mode: "date",
        display: Platform.OS === "ios" ? "spinner" : "default",
        onChange: (event, selectedDate) => {
          setShowDatePicker(Platform.OS === "ios");
          if (selectedDate) {
            const formattedDate = selectedDate.toISOString().split("T")[0];
            handleInputChange("dateJoined", formattedDate);
          }
        },
        style: styles.datePicker
      }
    )), /* @__PURE__ */ React.createElement(View, { style: styles.formGroup }, /* @__PURE__ */
React.createElement(Text, { style: styles.label }, "Account Status"), /* @__PURE__ */
React.createElement(View, { style: styles.switchContainer }, /* @__PURE__ */ React.createElement(Text,
{ style: styles.switchLabel }, formData.isActive ? "Active" : "Inactive"), /* @__PURE__ */
React.createElement(
      Switch,
      {
        value: formData.isActive,
        onValueChange: (value) => handleInputChange("isActive", value),
        trackColor: { false: "#D3D3D3", true: "#AD7F65" },
        thumbColor: "#FFFFFF",
        disabled: saving
```

```
      }
    ))), /* @__PURE__ */ React.createElement(View, { style: styles.formGroup }, /* @__PURE__ */
React.createElement(Text, { style: styles.label }, "Permissions"), /* @__PURE__ */ React.createElement(View,
{ style: styles.permissionsList }, /* @__PURE__ */ React.createElement(View, { style:
styles.permissionToggle }, /* @__PURE__ */ React.createElement(Text, { style: styles.permissionLabel }, "POS
Access"), /* @__PURE__ */ React.createElement(
      Switch,
      {
        value: formData.permissions.pos,
        onValueChange: () => handlePermissionToggle("pos"),
        trackColor: { false: "#D3D3D3", true: "#AD7F65" },
        thumbColor: "#FFFFFF",
        disabled: saving
      }
    )), /* @__PURE__ */ React.createElement(View, { style: styles.permissionToggle }, /* @__PURE__ */
React.createElement(Text, { style: styles.permissionLabel }, "Inventory Management"), /* @__PURE__ */
React.createElement(
      Switch,
      {
        value: formData.permissions.inventory,
        onValueChange: () => handlePermissionToggle("inventory"),
        trackColor: { false: "#D3D3D3", true: "#AD7F65" },
        thumbColor: "#FFFFFF",
        disabled: saving
      }
    )), /* @__PURE__ */ React.createElement(View, { style: styles.permissionToggle }, /* @__PURE__ */
React.createElement(Text, { style: styles.permissionLabel }, "View Transactions"), /* @__PURE__ */
React.createElement(
      Switch,
      {
        value: formData.permissions.viewTransaction,
        onValueChange: () => handlePermissionToggle("viewTransaction"),
        trackColor: { false: "#D3D3D3", true: "#AD7F65" },
        thumbColor: "#FFFFFF",
        disabled: saving
      }
    )), /* @__PURE__ */ React.createElement(View, { style: styles.permissionToggle }, /* @__PURE__ */
React.createElement(Text, { style: styles.permissionLabel }, "Generate Reports"), /* @__PURE__ */
React.createElement(
      Switch,
      {
        value: formData.permissions.generateReports,
        onValueChange: () => handlePermissionToggle("generateReports"),
        trackColor: { false: "#D3D3D3", true: "#AD7F65" },
        thumbColor: "#FFFFFF",
        disabled: saving
      }
    )))), /* @__PURE__ */ React.createElement(View, { style: styles.modalFooter }, /* @__PURE__ */
React.createElement(
      TouchableOpacity,
      {
        style: [styles.button, styles.cancelButton],
        onPress: () => setIsEditModalVisible(false),
        disabled: saving
      },
      /* @__PURE__ */ React.createElement(Text, { style: styles.buttonText }, "Cancel")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [styles.button, styles.saveButton],
        onPress: handleSave,
        disabled: saving
      },
      saving ? /* @__PURE__ */ React.createElement(ActivityIndicator, { size: "small", color: "white" }) : /*
@__PURE__ */ React.createElement(Text, { style: [styles.buttonText, { color: "white" }] }, editingEmployee ?
"Save Changes" : "Add Employee")
    ))))
  ), /* @__PURE__ */ React.createElement(
    Modal,
    {
      visible: showStatusModal,
      transparent: true,
      animationType: "fade",
      onRequestClose: () => setShowStatusModal(false)
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.modalOverlay }, /* @__PURE__ */
React.createElement(View, { style: styles.confirmModalContent }, /* @__PURE__ */ React.createElement(Text,
```

```
{ style: styles.modalTitle }, selectedEmployee?.status === "Active" ? "Disable Employee" : "Enable Employee"),
/* @__PURE__ */ React.createElement(Text, { style: styles.confirmText }, "Are you sure you want to ",
selectedEmployee?.status === "Active" ? "disable" : "enable", " ", selectedEmployee?.name, "?"), /* @__PURE__
*/ React.createElement(View, { style: styles.confirmButtons }, /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [styles.confirmButton, styles.cancelButton],
        onPress: () => setShowStatusModal(false)
      },
      /* @__PURE__ */ React.createElement(Text, { style: styles.buttonText }, "Cancel")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [styles.confirmButton, styles.confirmActionButton],
        onPress: confirmStatusChange
      },
      /* @__PURE__ */ React.createElement(Text, { style: [styles.buttonText, { color: "white" }] },
selectedEmployee?.status === "Active" ? "Disable" : "Enable")
    ))))
  ));
};
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#f5f5f5",
    padding: 16
  },
  searchContainer: {
    flexDirection: "row",
    marginBottom: 16,
    alignItems: "center"
  },
  searchInputContainer: {
    flex: 1,
    flexDirection: "row",
    alignItems: "center",
    backgroundColor: "white",
    borderRadius: 8,
    paddingHorizontal: 12,
    marginRight: 12,
    elevation: 2,
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 1 },
    shadowOpacity: 0.1,
    shadowRadius: 2
  },
  searchInput: {
    flex: 1,
    height: 48,
    paddingLeft: 8,
    color: "#333"
  },
  searchIcon: {
    marginRight: 8
  },
  addButton: {
    width: 48,
    height: 48,
    backgroundColor: "#AD7F65",
    borderRadius: 24,
    alignItems: "center",
    justifyContent: "center",
    elevation: 2,
    marginLeft: 8
  },
  scrollView: {
    flex: 1
  },
  employeeCard: {
    backgroundColor: "white",
    borderRadius: 12,
    padding: 16,
    marginBottom: 16,
    marginHorizontal: 8,
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
```

```
  shadowRadius: 4,
  elevation: 3,
  width: "95%",
  alignSelf: "center"
},
inactiveEmployee: {
  opacity: 0.7,
  backgroundColor: "#F2F2F2"
},
employeeHeader: {
  flexDirection: "row",
  justifyContent: "space-between",
  marginBottom: 12,
  borderBottomWidth: 1,
  borderBottomColor: "#f0f0f0",
  paddingBottom: 12
},
employeeInfo: {
  flex: 1
},
employeeName: {
  fontSize: 16,
  fontWeight: "600",
  color: "#333",
  marginBottom: 4
},
employeePosition: {
  fontSize: 14,
  color: "#666",
  marginBottom: 6
},
statusBadge: {
  alignSelf: "flex-start",
  paddingHorizontal: 8,
  paddingVertical: 2,
  borderRadius: 10
},
activeBadge: {
  backgroundColor: "#E8F5E9",
  borderColor: "#4CAF50",
  borderWidth: 1
},
inactiveBadge: {
  backgroundColor: "#FFF3E0",
  // Changed to orange for inactive/pending
  borderColor: "#FF9800",
  borderWidth: 1
},
statusText: {
  fontSize: 12,
  fontWeight: "500"
},
employeeActions: {
  flexDirection: "row",
  alignItems: "center"
},
actionButton: {
  padding: 8,
  marginLeft: 8
},
employeeDetails: {
  marginBottom: 12
},
detailRow: {
  flexDirection: "row",
  alignItems: "center",
  marginBottom: 6
},
detailText: {
  marginLeft: 8,
  fontSize: 14,
  color: "#666"
},
permissionsContainer: {
  marginTop: 8
},
permissionsTitle: {
```

```
  fontSize: 14,
  fontWeight: "600",
  marginBottom: 8,
  color: "#333"
},
permissionsGrid: {
  flexDirection: "row",
  flexWrap: "wrap",
  marginTop: 4
},
permissionItem: {
  flexDirection: "row",
  alignItems: "center",
  marginRight: 16,
  marginBottom: 8
},
permissionText: {
  fontSize: 12,
  color: "#555",
  marginRight: 6
},
permissionBadge: {
  width: 20,
  height: 20,
  borderRadius: 10,
  justifyContent: "center",
  alignItems: "center"
},
permissionGranted: {
  backgroundColor: "#E8F5E9"
},
permissionDenied: {
  backgroundColor: "#FFEBEE"
},
permissionBadgeText: {
  fontSize: 10,
  fontWeight: "bold"
},
modalOverlay: {
  flex: 1,
  backgroundColor: "rgba(0, 0, 0, 0.5)",
  justifyContent: "center",
  alignItems: "center",
  padding: 20
},
modalContent: {
  backgroundColor: "white",
  borderRadius: 12,
  width: "100%",
  maxHeight: "90%"
},
confirmModalContent: {
  backgroundColor: "white",
  borderRadius: 10,
  padding: 20,
  width: "80%",
  alignItems: "center"
},
confirmText: {
  fontSize: 16,
  textAlign: "center",
  marginVertical: 20,
  color: "#333"
},
confirmButtons: {
  flexDirection: "row",
  justifyContent: "space-between",
  width: "100%",
  marginTop: 10
},
confirmButton: {
  padding: 12,
  borderRadius: 8,
  alignItems: "center",
  flex: 1,
  marginHorizontal: 5
},
```

```
cancelButton: {
  backgroundColor: "#f0f0f0"
},
confirmActionButton: {
  backgroundColor: "#AD7F65"
},
buttonText: {
  fontWeight: "600",
  fontSize: 16
},
modalHeader: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  padding: 16,
  borderBottomWidth: 1,
  borderBottomColor: "#f0f0f0"
},
modalTitle: {
  fontSize: 18,
  fontWeight: "600",
  color: "#333"
},
modalBody: {
  padding: 16
},
formGroup: {
  marginBottom: 16
},
label: {
  fontSize: 14,
  fontWeight: "500",
  marginBottom: 6,
  color: "#333"
},
input: {
  borderWidth: 1,
  borderColor: "#ddd",
  borderRadius: 8,
  padding: 12,
  fontSize: 16,
  color: "#333",
  backgroundColor: "#fafafa"
},
pickerContainer: {
  borderWidth: 1,
  borderColor: "#ddd",
  borderRadius: 8,
  backgroundColor: "#fafafa",
  overflow: "hidden"
},
picker: {
  height: 50,
  width: "100%"
},
dateInputContainer: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  borderWidth: 1,
  borderColor: "#ddd",
  borderRadius: 8,
  padding: 12,
  backgroundColor: "#fafafa"
},
dateInputText: {
  fontSize: 16,
  color: "#333"
},
switchContainer: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  borderWidth: 1,
  borderColor: "#ddd",
  borderRadius: 8,
  padding: 12,
```

```
    backgroundColor: "#fafafa"
  },
  switchLabel: {
    fontSize: 16,
    color: "#333"
  },
  permissionsList: {
    marginTop: 8
  },
  permissionToggle: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    paddingVertical: 12,
    borderBottomWidth: 1,
    borderBottomColor: "#f0f0f0"
  },
  permissionLabel: {
    fontSize: 15,
    color: "#444"
  },
  modalFooter: {
    flexDirection: "row",
    padding: 16,
    borderTopWidth: 1,
    borderTopColor: "#f0f0f0",
    gap: 12
  },
  button: {
    flex: 1,
    padding: 14,
    borderRadius: 8,
    alignItems: "center",
    justifyContent: "center"
  },
  saveButton: {
    backgroundColor: "#AD7F65"
  },
  hintText: {
    fontSize: 12,
    color: "#777",
    marginTop: 4,
    fontStyle: "italic"
  }
});
export default RolesAndPermission;
```

FILE: pos/mobile/app/SyncLog.jsx
==========

```
import { MaterialIcons } from "@expo/vector-icons";
import AsyncStorage from "@react-native-async-storage/async-storage";
import { useRouter } from "expo-router";
import { useEffect, useState } from "react";
import { FlatList, ScrollView, StyleSheet, Text, TouchableOpacity, View } from "react-native";
import { syncAPI } from "../services/api";
const SYNC_LOGS_KEY = "@sync_logs";
const SyncLog = () => {
  const [syncLogs, setSyncLogs] = useState([]);
  const [selectedLog, setSelectedLog] = useState(null);
  const [isSyncing, setIsSyncing] = useState(false);
  const router = useRouter();
  useEffect(() => {
    loadSyncLogs();
  }, []);
  const loadSyncLogs = async () => {
    try {
      let logsJson = await AsyncStorage.getItem(SYNC_LOGS_KEY);
      let logs = [];
      if (logsJson) {
        logs = JSON.parse(logsJson);
      }
      setSyncLogs(logs);
    } catch (error) {
      console.error("Failed to load sync logs", error);
    }
```

```
  };
  const formatDate = (timestamp) => {
    if (!timestamp) return "N/A";
    const date = new Date(timestamp);
    return date.toLocaleString();
  };
  const renderLogItem = ({ item }) => /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: styles.logItem,
      onPress: () => setSelectedLog(item)
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.logHeader }, /* @__PURE__ */
React.createElement(
      MaterialIcons,
      {
        name: item.status === "success" ? "check-circle" : "error",
        size: 24,
        color: item.status === "success" ? "#4CAF50" : "#F44336"
      }
    ), /* @__PURE__ */ React.createElement(Text, { style: styles.logDate }, formatDate(item.timestamp)), /*
@__PURE__ */ React.createElement(Text, { style: [styles.logStatus, { color: item.status === "success" ?
"#4CAF50" : "#F44336" }] }, item.status === "success" ? "Success" : "Failed")),
    /* @__PURE__ */ React.createElement(Text, { style: styles.logSummary }, item.itemsSynced?.length || 0, "
items synced"),
    item.message ? /* @__PURE__ */ React.createElement(Text, { style: styles.logMessage }, item.message) :
null
  );
  const renderLogDetails = () => /* @__PURE__ */ React.createElement(View, { style: styles.detailsContainer },
/* @__PURE__ */ React.createElement(View, { style: styles.detailsHeader }, /* @__PURE__ */
React.createElement(Text, { style: styles.detailsTitle }, "Sync Details"), /* @__PURE__ */
React.createElement(TouchableOpacity, { onPress: () => setSelectedLog(null) }, /* @__PURE__ */
React.createElement(MaterialIcons, { name: "close", size: 24, color: "#333" }))), /* @__PURE__ */
React.createElement(Text, { style: styles.detailText }, /* @__PURE__ */ React.createElement(Text, { style:
styles.detailLabel }, "Date: "), formatDate(selectedLog.timestamp)), /* @__PURE__ */ React.createElement(Text,
{ style: styles.detailText }, /* @__PURE__ */ React.createElement(Text, { style: styles.detailLabel },
"Status: "), /* @__PURE__ */ React.createElement(Text, { style: { color: selectedLog.status === "success" ?
"#4CAF50" : "#F44336" } }, selectedLog.status === "success" ? "Success" : "Failed")), selectedLog.message ?
/* @__PURE__ */ React.createElement(Text, { style: styles.detailText }, /* @__PURE__ */
React.createElement(Text, { style: styles.detailLabel }, "Message: "), selectedLog.message) : null, /*
@__PURE__ */ React.createElement(Text, { style: styles.detailLabel }, "Items Synced (",
selectedLog.itemsSynced?.length || 0, "):"), /* @__PURE__ */ React.createElement(ScrollView, { style:
styles.itemsList }, selectedLog.itemsSynced?.map((item, index) => /* @__PURE__ */ React.createElement(View,
{ key: index, style: styles.itemRow }, /* @__PURE__ */ React.createElement(Text, { style: styles.itemName },
item.name), /* @__PURE__ */ React.createElement(Text, { style: styles.itemStatus }, item.status ===
"success" ? "\u2713" : "\u2717")))));
  const handleSyncNow = async () => {
    setIsSyncing(true);
    try {
      const response = await syncAPI.syncAll();
      const success = response?.success !== false;
      const newLog = {
        timestamp: Date.now(),
        status: success ? "success" : "failed",
        message: response?.message || (success ? "Sync completed successfully" : "Sync failed"),
        hasChanges: typeof response?.hasChanges === "boolean" ? response.hasChanges : true,
        itemsSynced: [
          { name: "Products", status: "success" },
          { name: "Inventory", status: "success" },
          { name: "Transactions", status: "success" }
        ]
      };
      const updatedLogs = [newLog, ...syncLogs || []];
      await AsyncStorage.setItem(SYNC_LOGS_KEY, JSON.stringify(updatedLogs));
      setSyncLogs(updatedLogs);
    } catch (error) {
      console.error("Sync failed:", error);
      const failedLog = {
        timestamp: Date.now(),
        status: "failed",
        message: error?.message || "Sync failed. Please check your connection.",
        itemsSynced: []
      };
      const updatedLogs = [failedLog, ...syncLogs || []];
      await AsyncStorage.setItem(SYNC_LOGS_KEY, JSON.stringify(updatedLogs));
      setSyncLogs(updatedLogs);
    } finally {
```

```
      setIsSyncing(false);
    }
  };
  return /* @__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */
React.createElement(View, { style: styles.headerContainer }, /* @__PURE__ */ React.createElement(Text,
{ style: styles.header }, "Sync History"), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: [styles.syncButton, isSyncing && styles.syncButtonDisabled],
      onPress: handleSyncNow,
      disabled: isSyncing
    },
    /* @__PURE__ */ React.createElement(
      MaterialIcons,
      {
        name: isSyncing ? "refresh" : "sync",
        size: 20,
        color: "#fff",
        style: isSyncing ? styles.rotate : null
      }
    ),
    /* @__PURE__ */ React.createElement(Text, { style: styles.syncButtonText }, isSyncing ? "Syncing..." :
"Sync Now")
  )), selectedLog ? renderLogDetails() : /* @__PURE__ */ React.createElement(
    FlatList,
    {
      data: syncLogs.sort((a, b) => b.timestamp - a.timestamp),
      renderItem: renderLogItem,
      keyExtractor: (item, index) => index.toString(),
      ListEmptyComponent: /* @__PURE__ */ React.createElement(Text, { style: styles.emptyText }, "No sync
history found"),
      contentContainerStyle: styles.listContainer
    }
  ));
};
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#f5f5f5",
    padding: 16,
    paddingTop: 20
  },
  headerContainer: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    marginBottom: 20
  },
  header: {
    fontSize: 24,
    fontWeight: "bold",
    marginBottom: 20,
    color: "#333"
  },
  listContainer: {
    paddingBottom: 20
  },
  logItem: {
    backgroundColor: "white",
    borderRadius: 8,
    padding: 16,
    marginBottom: 12,
    elevation: 2
  },
  logHeader: {
    flexDirection: "row",
    alignItems: "center",
    marginBottom: 8
  },
  logDate: {
    flex: 1,
    marginLeft: 12,
    fontSize: 16,
    color: "#333"
  },
  logStatus: {
    fontWeight: "bold",
```

```
    fontSize: 14
  },
  logSummary: {
    color: "#666",
    fontSize: 14
  },
  logMessage: {
    marginTop: 4,
    color: "#555",
    fontSize: 13
  },
  detailsContainer: {
    backgroundColor: "white",
    borderRadius: 8,
    padding: 16,
    elevation: 2
  },
  detailsHeader: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    marginBottom: 16
  },
  detailsTitle: {
    fontSize: 20,
    fontWeight: "bold",
    color: "#333"
  },
  detailText: {
    marginBottom: 12,
    fontSize: 16,
    color: "#333"
  },
  detailLabel: {
    fontWeight: "bold",
    color: "#333"
  },
  itemsList: {
    maxHeight: 300,
    marginTop: 8
  },
  itemRow: {
    flexDirection: "row",
    justifyContent: "space-between",
    paddingVertical: 8,
    borderBottomWidth: 1,
    borderBottomColor: "#eee"
  },
  itemName: {
    flex: 1,
    color: "#333"
  },
  itemStatus: {
    color: "#4CAF50",
    fontWeight: "bold"
  },
  emptyText: {
    textAlign: "center",
    color: "#999",
    marginTop: 40,
    fontSize: 16
  },
  syncButton: {
    flexDirection: "row",
    alignItems: "center",
    backgroundColor: "#AD7F65",
    paddingVertical: 8,
    paddingHorizontal: 12,
    borderRadius: 20,
    elevation: 2
  },
  syncButtonDisabled: {
    backgroundColor: "#AD7F65"
  },
  syncButtonText: {
    color: "#fff",
    marginLeft: 6,
```

```
    fontWeight: "500"
  },
  rotate: {
    transform: [{ rotate: "360deg" }],
    animationKey: "spin",
    animationDuration: "1s",
    animationIterationCount: "infinite",
    animationTimingFunction: "linear"
  }
});
export default SyncLog;
```

FILE: pos/mobile/app/Update Pin.jsx
==========

```
import { Ionicons } from "@expo/vector-icons";
import { useNavigation } from "@react-navigation/native";
import { router } from "expo-router";
import { useState, useEffect } from "react";
import AsyncStorage from "@react-native-async-storage/async-storage";
import {
  Modal,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View,
  ActivityIndicator,
  Alert
} from "react-native";
import { employeeAPI } from "../services/api";
export default function UpdatePin() {
  const navigation = useNavigation();
  const [oldPin, setOldPin] = useState("");
  const [newPin, setNewPin] = useState("");
  const [rePin, setRePin] = useState("");
  const [showOld, setShowOld] = useState(false);
  const [showNew, setShowNew] = useState(false);
  const [showRe, setShowRe] = useState(false);
  const [message, setMessage] = useState("");
  const [messageColor, setMessageColor] = useState("red");
  const [successModal, setSuccessModal] = useState(false);
  const [loading, setLoading] = useState(false);
  const [currentUser, setCurrentUser] = useState(null);
  const PIN_LENGTH = 6;
  useEffect(() => {
    const loadUser = async () => {
      try {
        const stored = await AsyncStorage.getItem("currentEmployee");
        if (stored) {
          setCurrentUser(JSON.parse(stored));
        }
      } catch (error) {
        console.error("Failed to load user:", error);
      }
    };
    loadUser();
  }, []);
  const handleConfirm = async () => {
    setMessage("");
    if (!currentUser) {
      setMessage("User session not found. Please re-login.");
      setMessageColor("red");
      return;
    }
    if (oldPin.length !== PIN_LENGTH) {
      setMessage(`Old PIN must be ${PIN_LENGTH} digits.`);
      setMessageColor("red");
      return;
    }
    if (newPin.length !== PIN_LENGTH) {
      setMessage(`New PIN must be ${PIN_LENGTH} digits.`);
      setMessageColor("red");
      return;
    }
    if (newPin !== rePin) {
```

```
      setMessage("New PIN and Re-enter PIN do not match.");
      setMessageColor("red");
      return;
    }
    if (oldPin === newPin) {
      setMessage("New PIN cannot be the same as the old PIN.");
      setMessageColor("red");
      return;
    }
    try {
      setLoading(true);
      const employeeId = currentUser._id || currentUser.id;
      const verifyRes = await employeeAPI.verifyPin(oldPin, employeeId);
      if (!verifyRes || !verifyRes.success) {
        setMessage("Incorrect Old PIN.");
        setMessageColor("red");
        setLoading(false);
        return;
      }
      const updateRes = await employeeAPI.updatePin(employeeId, {
        pin: newPin,
        newPin
      });
      if (updateRes && updateRes.success) {
        setSuccessModal(true);
        setTimeout(() => {
          setSuccessModal(false);
          router.back();
        }, 1500);
      } else {
        setMessage(updateRes?.message || "Failed to update PIN.");
        setMessageColor("red");
      }
    } catch (error) {
      console.error("Update PIN error:", error);
      setMessage("An error occurred. Please try again.");
      setMessageColor("red");
    } finally {
      setLoading(false);
    }
  };
  const PinField = ({ value, setValue, show, setShow, label }) => /* @__PURE__ */ React.createElement(View,
{ style: styles.inputGroup }, /* @__PURE__ */ React.createElement(Text, { style: styles.label }, label), /*
@__PURE__ */ React.createElement(View, { style: styles.inputWrapper }, /* @__PURE__ */ React.createElement(
    TextInput,
    {
      value,
      onChangeText: (text) => {
        if (text.length <= PIN_LENGTH) setValue(text);
      },
      secureTextEntry: !show,
      keyboardType: "numeric",
      maxLength: PIN_LENGTH,
      style: styles.input,
      editable: !loading
    }
  ), /* @__PURE__ */ React.createElement(TouchableOpacity, { onPress: () => setShow(!show) }, /* @__PURE__ */
React.createElement(
    Ionicons,
    {
      name: show ? "eye" : "eye-off",
      size: 22,
      color: "#444",
      style: { paddingRight: 10 }
    }
  ))));
  return /* @__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */
React.createElement(View, { style: styles.header }, /* @__PURE__ */ React.createElement(TouchableOpacity,
{ onPress: () => navigation.goBack(), style: styles.backBtn }, /* @__PURE__ */ React.createElement(Ionicons,
{ name: "arrow-back", size: 24, color: "#333" })), /* @__PURE__ */ React.createElement(Text, { style:
styles.headerTitle }, "Update Pincode")), /* @__PURE__ */ React.createElement(View, { style: styles.content },
/* @__PURE__ */ React.createElement(
    PinField,
    {
      label: "Old PIN",
      value: oldPin,
      setValue: setOldPin,
```

```
      show: showOld,
      setShow: setShowOld
    }
  ), /* @__PURE__ */ React.createElement(
    PinField,
    {
      label: "New PIN",
      value: newPin,
      setValue: setNewPin,
      show: showNew,
      setShow: setShowNew
    }
  ), /* @__PURE__ */ React.createElement(
    PinField,
    {
      label: "Re-enter New PIN",
      value: rePin,
      setValue: setRePin,
      show: showRe,
      setShow: setShowRe
    }
  ), message !== "" && /* @__PURE__ */ React.createElement(Text, { style: [styles.message, { color:
messageColor }] }, message), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: [styles.confirmBtn, loading && styles.disabledBtn],
      onPress: handleConfirm,
      disabled: loading
    },
    loading ? /* @__PURE__ */ React.createElement(ActivityIndicator, { size: "small", color: "#FFF" }) : /*
@__PURE__ */ React.createElement(Text, { style: styles.confirmText }, "Confirm Change")
  ), /* @__PURE__ */ React.createElement(Text, { style: styles.instruction }, "Enter your current 6-digit PIN
to verify your identity, then create and confirm a new 6-digit PIN.")), /* @__PURE__ */
React.createElement(Modal, { transparent: true, visible: successModal, animationType: "fade" }, /* @__PURE__
*/ React.createElement(View, { style: styles.modalWrapper }, /* @__PURE__ */ React.createElement(View,
{ style: styles.modalContent }, /* @__PURE__ */ React.createElement(Ionicons, { name: "checkmark-circle",
size: 50, color: "green" }), /* @__PURE__ */ React.createElement(Text, { style: styles.successText }, "PIN
updated successfully!")))));
}
const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#fff"
  },
  header: {
    flexDirection: "row",
    alignItems: "center",
    padding: 20,
    paddingTop: 50,
    // Status bar
    borderBottomWidth: 1,
    borderBottomColor: "#F0F0F0"
  },
  backBtn: {
    padding: 5
  },
  headerTitle: {
    fontSize: 20,
    fontWeight: "bold",
    marginLeft: 15,
    color: "#333"
  },
  content: {
    padding: 25
  },
  inputGroup: {
    marginBottom: 20
  },
  label: {
    fontSize: 15,
    fontWeight: "600",
    marginBottom: 8,
    color: "#444"
  },
  inputWrapper: {
    flexDirection: "row",
    alignItems: "center",
```

```
    borderColor: "#ccc",
    borderWidth: 1,
    borderRadius: 10,
    paddingHorizontal: 5,
    backgroundColor: "#FAFAFA"
  },
  input: {
    flex: 1,
    padding: 12,
    fontSize: 18,
    letterSpacing: 3,
    color: "#333"
  },
  message: {
    fontSize: 14,
    textAlign: "center",
    marginBottom: 15,
    fontWeight: "500"
  },
  confirmBtn: {
    backgroundColor: "#AD7F65",
    padding: 15,
    borderRadius: 12,
    marginTop: 10,
    elevation: 2,
    shadowColor: "#000",
    shadowOpacity: 0.1,
    shadowRadius: 4,
    shadowOffset: { width: 0, height: 2 }
  },
  disabledBtn: {
    opacity: 0.7
  },
  confirmText: {
    textAlign: "center",
    color: "#fff",
    fontSize: 18,
    fontWeight: "600"
  },
  instruction: {
    marginTop: 20,
    fontSize: 13,
    color: "#888",
    textAlign: "center",
    lineHeight: 20,
    paddingHorizontal: 10
  },
  modalWrapper: {
    flex: 1,
    backgroundColor: "rgba(0,0,0,0.4)",
    justifyContent: "center",
    alignItems: "center"
  },
  modalContent: {
    backgroundColor: "#fff",
    padding: 30,
    borderRadius: 20,
    alignItems: "center",
    width: "80%",
    shadowColor: "#000",
    shadowOpacity: 0.25,
    shadowRadius: 10,
    elevation: 5
  },
  successText: {
    marginTop: 15,
    fontSize: 18,
    fontWeight: "600",
    color: "#333"
  }
});



FILE: pos/mobile/app/VoidLog.jsx
==============================================================================
import { Ionicons } from "@expo/vector-icons";
```

```
import { useRouter } from "expo-router";
import { useEffect, useState } from "react";
import {
  FlatList,
  Modal,
  SafeAreaView,
  ScrollView,
  StatusBar,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View,
  ActivityIndicator,
  RefreshControl
} from "react-native";
import { voidLogAPI } from "../services/api";
export default function VoidLog() {
  const [search, setSearch] = useState("");
  const [voidLogs, setVoidLogs] = useState([]);
  const [selectedVoid, setSelectedVoid] = useState(null);
  const [modalVisible, setModalVisible] = useState(false);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const router = useRouter();
  const handleVoidPress = (voidItem) => {
    setSelectedVoid(voidItem);
    setModalVisible(true);
  };
  const closeModal = () => {
    setModalVisible(false);
    setSelectedVoid(null);
  };
  const fetchVoidLogs = async () => {
    try {
      const response = await voidLogAPI.getAll();
      if (response.success && response.data) {
        setVoidLogs(response.data);
      }
    } catch (error) {
      console.error("Error fetching void logs:", error);
      setVoidLogs([]);
    } finally {
      setLoading(false);
      setRefreshing(false);
    }
  };
  useEffect(() => {
    fetchVoidLogs();
  }, []);
  const onRefresh = () => {
    setRefreshing(true);
    fetchVoidLogs();
  };
  const filteredLogs = voidLogs.filter((log) => {
    const transactionId = log.voidId || log.transactionId || log.transaction?.receiptNo || "";
    const reason = log.voidReason || "";
    const approvedBy = log.approvedBy || log.approvedByName || "";
    const searchLower = search.toLowerCase();
    return transactionId.toLowerCase().includes(searchLower) || reason.toLowerCase().includes(searchLower) ||
approvedBy.toLowerCase().includes(searchLower);
  });
  const renderItem = ({ item }) => {
    const transactionId = item.voidId || item.transactionId || item.transaction?.receiptNo || item._id ||
"N/A";
    const date = item.voidedAt || item.date || item.createdAt || item.timestamp || /* @__PURE__ */ new Date();
    const reason = item.voidReason || "No reason provided";
    const items = item.items || [];
    const total = item.totalAmount || item.total || 0;
    const approvedBy = item.approvedBy || item.voidedByName || "N/A";
    return /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: styles.logItem,
        onPress: () => handleVoidPress(item)
      },
      /* @__PURE__ */ React.createElement(View, { style: styles.logHeader }, /* @__PURE__ */
```

```
React.createElement(Text, { style: styles.transactionId }, transactionId), /* @__PURE__ */
React.createElement(Text, { style: styles.date }, new Date(date).toLocaleDateString("en-US", {
        year: "numeric",
        month: "short",
        day: "numeric",
        hour: "2-digit",
        minute: "2-digit"
      }))),
      /* @__PURE__ */ React.createElement(View, { style: styles.logDetails }, /* @__PURE__ */
React.createElement(Text, { style: styles.reason }, reason), items.length > 0 && /* @__PURE__ */
React.createElement(Text, { style: styles.items }, items.map((i, idx) => {
        const qty = i.quantity || i.qty || 1;
        const name = i.itemName || i.name || i.productName || "Unknown";
        return `${qty}x ${name}`;
      }).join(", "))),
      /* @__PURE__ */ React.createElement(View, { style: styles.logFooter }, /* @__PURE__ */
React.createElement(Text, { style: styles.total }, "\u20B1", parseFloat(total).toFixed(2)), /* @__PURE__ */
React.createElement(Text, { style: styles.approvedBy }, "Approved by: ", approvedBy))
    );
  };
  return /* @__PURE__ */ React.createElement(SafeAreaView, { style: styles.container }, /* @__PURE__ */
React.createElement(StatusBar, { barStyle: "dark-content", backgroundColor: "#fff" }), /* @__PURE__ */
React.createElement(View, { style: styles.searchContainer }, /* @__PURE__ */ React.createElement(Ionicons,
{ name: "search", size: 20, color: "#666", style: styles.searchIcon }), /* @__PURE__ */ React.createElement(
    TextInput,
    {
      style: styles.searchInput,
      placeholder: "Search void logs...",
      value: search,
      onChangeText: setSearch,
      placeholderTextColor: "#999"
    }
  ), search.length > 0 && /* @__PURE__ */ React.createElement(TouchableOpacity, { onPress: () =>
setSearch(""), style: styles.clearButton }, /* @__PURE__ */ React.createElement(Ionicons, { name: "close-
circle", size: 18, color: "#999" }))), loading ? /* @__PURE__ */ React.createElement(View, { style:
styles.loadingContainer }, /* @__PURE__ */ React.createElement(ActivityIndicator, { size: "large", color:
"#AD7F65" }), /* @__PURE__ */ React.createElement(Text, { style: styles.loadingText }, "Loading void
logs...")) : /* @__PURE__ */ React.createElement(
    FlatList,
    {
      data: filteredLogs,
      renderItem,
      keyExtractor: (item) => item._id || item.id || Math.random().toString(),
      contentContainerStyle: styles.listContent,
      refreshControl: /* @__PURE__ */ React.createElement(
        RefreshControl,
        {
          refreshing,
          onRefresh,
          colors: ["#AD7F65"],
          tintColor: "#AD7F65"
        }
      ),
      ListEmptyComponent: /* @__PURE__ */ React.createElement(View, { style: styles.emptyState }, /*
@__PURE__ */ React.createElement(Ionicons, { name: "receipt-outline", size: 48, color: "#ccc" }), /*
@__PURE__ */ React.createElement(Text, { style: styles.emptyText }, "No void logs found"))
    }
  ), /* @__PURE__ */ React.createElement(
    Modal,
    {
      animationType: "slide",
      transparent: true,
      visible: modalVisible,
      onRequestClose: closeModal
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.modalOverlay }, /* @__PURE__ */
React.createElement(View, { style: styles.modalContainer }, /* @__PURE__ */ React.createElement(View, { style:
styles.modalHeader }, /* @__PURE__ */ React.createElement(Text, { style: styles.modalTitle }, "Void Details"),
/* @__PURE__ */ React.createElement(TouchableOpacity, { onPress: closeModal, style: styles.closeButton }, /*
@__PURE__ */ React.createElement(Ionicons, { name: "close", size: 24, color: "#666" }))), /* @__PURE__ */
React.createElement(ScrollView, { style: styles.modalContent }, selectedVoid && (() => {
      const transactionId = selectedVoid.voidId || selectedVoid.transactionId ||
selectedVoid.transaction?.receiptNo || selectedVoid._id || "N/A";
      const date = selectedVoid.voidedAt || selectedVoid.date || selectedVoid.createdAt ||
selectedVoid.timestamp || /* @__PURE__ */ new Date();
      const reason = selectedVoid.voidReason || "No reason provided";
      const items = selectedVoid.items || [];
```

```
      const total = selectedVoid.totalAmount || selectedVoid.total || 0;
      const approvedBy = selectedVoid.approvedBy || selectedVoid.voidedByName || "N/A";
      const status = selectedVoid.status || "Voided";
      return /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(View, { style: styles.detailRow }, /* @__PURE__ */ React.createElement(Text, { style:
styles.detailLabel }, "Transaction ID:"), /* @__PURE__ */ React.createElement(Text, { style:
styles.detailValue }, transactionId)), /* @__PURE__ */ React.createElement(View, { style: styles.detailRow },
/* @__PURE__ */ React.createElement(Text, { style: styles.detailLabel }, "Date & Time:"), /* @__PURE__ */
React.createElement(Text, { style: styles.detailValue }, new Date(date).toLocaleString("en-US", {
        year: "numeric",
        month: "short",
        day: "numeric",
        hour: "2-digit",
        minute: "2-digit"
      })})), items.length > 0 && /* @__PURE__ */ React.createElement(View, { style: styles.section }, /*
@__PURE__ */ React.createElement(Text, { style: styles.sectionTitle }, "Items Voided"), items.map((item,
index) => {
        const qty = item.quantity || item.qty || 1;
        const name = item.itemName || item.name || item.productName || "Unknown";
        const price = item.price || 0;
        return /* @__PURE__ */ React.createElement(View, { key: index, style: styles.itemRow }, /* @__PURE__
*/ React.createElement(Text, { style: styles.itemName }, qty, "x ", name), /* @__PURE__ */
React.createElement(Text, { style: styles.itemPrice }, "\u20B1", (qty * price).toFixed(2)));
      })), /* @__PURE__ */ React.createElement(View, { style: styles.section }, /* @__PURE__ */
React.createElement(View, { style: styles.totalRow }, /* @__PURE__ */ React.createElement(Text, { style:
styles.totalLabel }, "Total Voided:"), /* @__PURE__ */ React.createElement(Text, { style:
styles.totalAmount }, "\u20B1", parseFloat(total).toFixed(2)))), /* @__PURE__ */ React.createElement(View,
{ style: styles.section }, /* @__PURE__ */ React.createElement(Text, { style: styles.sectionTitle }, "Reason
for Void"), /* @__PURE__ */ React.createElement(View, { style: styles.reasonBox }, /* @__PURE__ */
React.createElement(Text, { style: styles.reasonText }, reason))), /* @__PURE__ */ React.createElement(View,
{ style: styles.detailRow }, /* @__PURE__ */ React.createElement(Text, { style: styles.detailLabel },
"Approved By:"), /* @__PURE__ */ React.createElement(Text, { style: [styles.detailValue, { color:
"#2E7D32" }] }, approvedBy)), /* @__PURE__ */ React.createElement(View, { style: styles.detailRow }, /*
@__PURE__ */ React.createElement(Text, { style: styles.detailLabel }, "Status:"), /* @__PURE__ */
React.createElement(View, { style: [
        styles.statusBadge,
        { backgroundColor: status === "Completed" ? "#E8F5E9" : "#FFF3E0" }
      ] }, /* @__PURE__ */ React.createElement(Text, { style: [
        styles.statusText,
        { color: status === "Completed" ? "#2E7D32" : "#E65100" }
      ] }, status))));
    })()))
  ));
}
const styles = StyleSheet.create({
  // Modal Styles
  modalOverlay: {
    flex: 1,
    backgroundColor: "rgba(0, 0, 0, 0.5)",
    justifyContent: "center",
    padding: 20
  },
  modalContainer: {
    backgroundColor: "#fff",
    borderRadius: 12,
    maxHeight: "80%",
    overflow: "hidden"
  },
  modalHeader: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    padding: 16,
    borderBottomWidth: 1,
    borderBottomColor: "#eee"
  },
  modalTitle: {
    fontSize: 18,
    fontWeight: "bold",
    color: "#333"
  },
  closeButton: {
    padding: 4
  },
  modalContent: {
    padding: 16
  },
```

```
section: {
  marginBottom: 20
},
sectionTitle: {
  fontSize: 14,
  fontWeight: "600",
  color: "#666",
  marginBottom: 12,
  textTransform: "uppercase",
  letterSpacing: 0.5
},
detailRow: {
  flexDirection: "row",
  justifyContent: "space-between",
  marginBottom: 12
},
detailLabel: {
  fontSize: 14,
  color: "#666",
  flex: 1
},
detailValue: {
  fontSize: 14,
  color: "#333",
  fontWeight: "500",
  flex: 2,
  textAlign: "right"
},
itemRow: {
  flexDirection: "row",
  justifyContent: "space-between",
  paddingVertical: 8,
  borderBottomWidth: 1,
  borderBottomColor: "#f5f5f5"
},
itemName: {
  fontSize: 14,
  color: "#333"
},
itemPrice: {
  fontSize: 14,
  fontWeight: "500",
  color: "#333"
},
totalRow: {
  flexDirection: "row",
  justifyContent: "space-between",
  marginTop: 8,
  paddingTop: 8,
  borderTopWidth: 1,
  borderTopColor: "#eee"
},
totalLabel: {
  fontSize: 16,
  fontWeight: "600",
  color: "#333"
},
totalAmount: {
  fontSize: 16,
  fontWeight: "bold",
  color: "#D32F2F"
},
reasonBox: {
  backgroundColor: "#F5F5F5",
  borderRadius: 8,
  padding: 12,
  marginTop: 4
},
reasonText: {
  fontSize: 14,
  color: "#333",
  lineHeight: 20
},
statusBadge: {
  paddingHorizontal: 10,
  paddingVertical: 4,
  borderRadius: 12,
```

```
  alignSelf: "flex-end"
},
statusText: {
  fontSize: 12,
  fontWeight: "600"
},
// Existing styles
container: {
  flex: 1,
  backgroundColor: "#f5f5f5"
},
header: {
  flexDirection: "row",
  alignItems: "center",
  justifyContent: "space-between",
  padding: 16,
  backgroundColor: "#fff",
  borderBottomWidth: 1,
  borderBottomColor: "#eee"
},
backButton: {
  padding: 8
},
headerTitle: {
  fontSize: 18,
  fontWeight: "bold",
  color: "#333"
},
searchContainer: {
  flexDirection: "row",
  alignItems: "center",
  backgroundColor: "#fff",
  margin: 16,
  borderRadius: 10,
  paddingHorizontal: 12,
  shadowColor: "#000",
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.1,
  shadowRadius: 4,
  elevation: 2
},
searchIcon: {
  marginRight: 8
},
searchInput: {
  flex: 1,
  height: 44,
  color: "#333"
},
clearButton: {
  padding: 8
},
listContent: {
  padding: 16,
  paddingBottom: 32
},
logItem: {
  backgroundColor: "#fff",
  borderRadius: 12,
  padding: 16,
  marginBottom: 12,
  shadowColor: "#000",
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.05,
  shadowRadius: 4,
  elevation: 1,
  borderLeftWidth: 4,
  borderLeftColor: "#D32F2F"
},
logHeader: {
  flexDirection: "row",
  justifyContent: "space-between",
  marginBottom: 8
},
transactionId: {
  fontWeight: "bold",
  color: "#333",
```

```
    fontSize: 16
  },
  date: {
    color: "#666",
    fontSize: 12
  },
  logDetails: {
    marginBottom: 12
  },
  reason: {
    fontSize: 14,
    color: "#333",
    marginBottom: 4
  },
  items: {
    fontSize: 13,
    color: "#666",
    fontStyle: "italic"
  },
  logFooter: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center"
  },
  total: {
    fontWeight: "bold",
    color: "#D32F2F",
    fontSize: 16
  },
  approvedBy: {
    fontSize: 12,
    color: "#888"
  },
  emptyState: {
    flex: 1,
    alignItems: "center",
    justifyContent: "center",
    padding: 32
  },
  emptyText: {
    marginTop: 16,
    color: "#999",
    fontSize: 16,
    textAlign: "center"
  },
  loadingContainer: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    padding: 32
  },
  loadingText: {
    marginTop: 12,
    color: "#666",
    fontSize: 14
  }
});
```

FILE: pos/mobile/babel.config.js
==========

```
module.exports = function(api) {
  api.cache(true);
  return {
    presets: ["babel-preset-expo"],
    plugins: [
      "react-native-reanimated/plugin"
    ]
  };
};
```

FILE: pos/mobile/components/AddBrandModal.jsx
==========

```
import React, { useState } from "react";
```

```
import {
  Modal,
  View,
  Text,
  TextInput,
  TouchableOpacity,
  StyleSheet,
  ActivityIndicator,
  Alert
} from "react-native";
import { brandPartnerAPI } from "../services/api";
const AddBrandModal = ({ visible, onClose, onAdd }) => {
  const [brandName, setBrandName] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const handleSubmit = async () => {
    if (!brandName.trim()) {
      Alert.alert("Error", "Please enter a brand name");
      return;
    }
    setIsLoading(true);
    try {
      const response = await brandPartnerAPI.create({ brandName: brandName.trim() });
      if (response.success) {
        onAdd(brandName.trim());
        setBrandName("");
        onClose();
      } else {
        Alert.alert("Error", response.message || "Failed to add brand");
      }
    } catch (error) {
      console.error("Error adding brand:", error);
      Alert.alert("Error", "Failed to add brand. Please try again.");
    } finally {
      setIsLoading(false);
    }
  };
  const handleCancel = () => {
    setBrandName("");
    onClose();
  };
  return /* @__PURE__ */ React.createElement(
    Modal,
    {
      visible,
      transparent: true,
      animationType: "fade",
      onRequestClose: handleCancel
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.modalOverlay }, /* @__PURE__ */
React.createElement(View, { style: styles.modalContainer }, /* @__PURE__ */ React.createElement(Text, { style:
styles.modalTitle }, "Add New Brand"), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: styles.input,
        placeholder: "Brand Name",
        value: brandName,
        onChangeText: setBrandName,
        autoFocus: true
      }
    ), /* @__PURE__ */ React.createElement(View, { style: styles.modalButtons }, /* @__PURE__ */
React.createElement(
      TouchableOpacity,
      {
        style: [styles.modalButton, styles.cancelButton],
        onPress: handleCancel,
        disabled: isLoading
      },
      /* @__PURE__ */ React.createElement(Text, { style: styles.buttonText }, "Cancel")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [
          styles.modalButton,
          styles.confirmButton,
          isLoading && styles.disabledButton
        ],
        onPress: handleSubmit,
```

```
        disabled: isLoading
      },
      isLoading ? /* @__PURE__ */ React.createElement(ActivityIndicator, { color: "#fff", size: "small" }) :
/* @__PURE__ */ React.createElement(Text, { style: [styles.buttonText, { color: "white" }] }, "Add")
    ))))
  );
};
const styles = StyleSheet.create({
  modalOverlay: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    backgroundColor: "rgba(0, 0, 0, 0.5)",
    padding: 20
  },
  modalContainer: {
    width: "100%",
    backgroundColor: "white",
    borderRadius: 12,
    padding: 20,
    alignItems: "center",
    elevation: 5,
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.25,
    shadowRadius: 4
  },
  modalTitle: {
    fontSize: 18,
    fontWeight: "bold",
    marginBottom: 20,
    color: "#333"
  },
  input: {
    width: "100%",
    borderWidth: 1,
    borderColor: "#d1d5db",
    borderRadius: 8,
    padding: 12,
    fontSize: 16,
    marginBottom: 20,
    backgroundColor: "#f9fafb"
  },
  modalButtons: {
    flexDirection: "row",
    justifyContent: "space-between",
    width: "100%"
  },
  modalButton: {
    flex: 1,
    padding: 12,
    borderRadius: 8,
    alignItems: "center",
    marginHorizontal: 5
  },
  cancelButton: {
    backgroundColor: "#f0f0f0"
  },
  confirmButton: {
    backgroundColor: "#AD7F65"
  },
  disabledButton: {
    opacity: 0.7
  },
  buttonText: {
    fontSize: 16,
    fontWeight: "600",
    color: "#333"
  }
});
export default AddBrandModal;
```

FILE: pos/mobile/components/AddCategoryModal.jsx
==========

```
import React, { useState } from "react";
```

```
import {
  Modal,
  View,
  Text,
  TextInput,
  TouchableOpacity,
  StyleSheet,
  ActivityIndicator,
  Alert
} from "react-native";
import { categoryAPI } from "../services/api";
const AddCategoryModal = ({ visible, onClose, onAdd }) => {
  const [categoryName, setCategoryName] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const handleSubmit = async () => {
    if (!categoryName.trim()) {
      Alert.alert("Error", "Please enter a category name");
      return;
    }
    setIsLoading(true);
    try {
      const response = await categoryAPI.create({ name: categoryName.trim() });
      if (response.success) {
        onAdd(categoryName.trim());
        setCategoryName("");
        onClose();
      } else {
        Alert.alert("Error", response.message || "Failed to add category");
      }
    } catch (error) {
      console.error("Error adding category:", error);
      Alert.alert("Error", "Failed to add category. Please try again.");
    } finally {
      setIsLoading(false);
    }
  };
  const handleCancel = () => {
    setCategoryName("");
    onClose();
  };
  return /* @__PURE__ */ React.createElement(
    Modal,
    {
      visible,
      transparent: true,
      animationType: "fade",
      onRequestClose: handleCancel
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.modalOverlay }, /* @__PURE__ */
React.createElement(View, { style: styles.modalContainer }, /* @__PURE__ */ React.createElement(Text, { style:
styles.modalTitle }, "Add New Category"), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: styles.input,
        placeholder: "Category Name",
        value: categoryName,
        onChangeText: setCategoryName,
        autoFocus: true
      }
    ), /* @__PURE__ */ React.createElement(View, { style: styles.modalButtons }, /* @__PURE__ */
React.createElement(
      TouchableOpacity,
      {
        style: [styles.modalButton, styles.cancelButton],
        onPress: handleCancel,
        disabled: isLoading
      },
      /* @__PURE__ */ React.createElement(Text, { style: styles.buttonText }, "Cancel")
    ), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [
          styles.modalButton,
          styles.confirmButton,
          isLoading && styles.disabledButton
        ],
        onPress: handleSubmit,
```

```
        disabled: isLoading
      },
      isLoading ? /* @__PURE__ */ React.createElement(ActivityIndicator, { color: "#fff", size: "small" }) :
/* @__PURE__ */ React.createElement(Text, { style: [styles.buttonText, { color: "white" }] }, "Add")
    ))))
  );
};
const styles = StyleSheet.create({
  modalOverlay: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    backgroundColor: "rgba(0, 0, 0, 0.5)",
    padding: 20
  },
  modalContainer: {
    width: "100%",
    backgroundColor: "white",
    borderRadius: 12,
    padding: 20,
    alignItems: "center",
    elevation: 5,
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.25,
    shadowRadius: 4
  },
  modalTitle: {
    fontSize: 18,
    fontWeight: "bold",
    marginBottom: 20,
    color: "#333"
  },
  input: {
    width: "100%",
    borderWidth: 1,
    borderColor: "#d1d5db",
    borderRadius: 8,
    padding: 12,
    fontSize: 16,
    marginBottom: 20,
    backgroundColor: "#f9fafb"
  },
  modalButtons: {
    flexDirection: "row",
    justifyContent: "space-between",
    width: "100%"
  },
  modalButton: {
    flex: 1,
    padding: 12,
    borderRadius: 8,
    alignItems: "center",
    marginHorizontal: 5
  },
  cancelButton: {
    backgroundColor: "#f0f0f0"
  },
  confirmButton: {
    backgroundColor: "#AD7F65"
  },
  disabledButton: {
    opacity: 0.7
  },
  buttonText: {
    fontSize: 16,
    fontWeight: "600",
    color: "#333"
  }
});
export default AddCategoryModal;
```

FILE: pos/mobile/components/FormSelect.jsx
==========

```
import { Picker } from "@react-native-picker/picker";
```

```
import { Platform, StyleSheet, View } from "react-native";
export default function FormSelect({
  items,
  selectedValue,
  onValueChange,
  enabled = true,
  style
}) {
  return /* @__PURE__ */ React.createElement(View, { style: [styles.pickerWrap, style] }, /* @__PURE__ */
React.createElement(
    Picker,
    {
      selectedValue,
      onValueChange,
      enabled,
      style: styles.picker,
      itemStyle: Platform.OS === "ios" ? styles.pickerItemIOS : void 0,
      dropdownIconColor: "#6b7280",
      mode: Platform.OS === "android" ? "dropdown" : void 0
    },
    items.map(({ label, value }) => /* @__PURE__ */ React.createElement(
      Picker.Item,
      {
        key: value === "" ? "__placeholder__" : String(value),
        label,
        value,
        color: "#111827"
      }
    ))
  ));
}
const styles = StyleSheet.create({
  pickerWrap: {
    borderWidth: 1,
    borderColor: "#d1d5db",
    borderRadius: 8,
    overflow: "hidden",
    backgroundColor: "#fff"
  },
  picker: {
    color: "#111827",
    backgroundColor: "#ffffff",
    ...Platform.OS === "android" ? { minHeight: 48 } : {}
  },
  pickerItemIOS: {
    color: "#111827",
    fontSize: 15
  }
});
```

FILE: pos/mobile/components/FormSelect.web.jsx
==========

```
import { StyleSheet, View } from "react-native";
const WEB_SELECT_STYLE = {
  width: "100%",
  padding: 12,
  fontSize: 15,
  borderStyle: "none",
  borderWidth: 0,
  backgroundColor: "transparent",
  outlineStyle: "none",
  cursor: "pointer",
  fontFamily: "system-ui, -apple-system, sans-serif",
  color: "#111827"
};
export default function FormSelect({
  items,
  selectedValue,
  onValueChange,
  enabled = true,
  style
}) {
  return /* @__PURE__ */ React.createElement(View, { style: [styles.pickerWrap, style] }, /* @__PURE__ */
React.createElement(
    "select",
```

```
    {
      disabled: !enabled,
      value: selectedValue,
      onChange: (e) => onValueChange(e.target.value),
      style: WEB_SELECT_STYLE
    },
    items.map(({ label, value }) => /* @__PURE__ */ React.createElement(
      "option",
      {
        key: value === "" ? "__placeholder__" : String(value),
        value
      },
      label
    ))
  ));
}
const styles = StyleSheet.create({
  pickerWrap: {
    borderWidth: 1,
    borderColor: "#d1d5db",
    borderRadius: 8,
    overflow: "hidden",
    backgroundColor: "#fff"
  }
});
```

FILE: pos/mobile/components/haptic-tab.tsx
==========

```
import { PlatformPressable } from "@react-navigation/elements";
import * as Haptics from "expo-haptics";
import { useRef } from "react";
import { Animated } from "react-native";
export function HapticTab(props) {
  const scaleAnim = useRef(new Animated.Value(1)).current;
  const animatePress = (toValue) => {
    Animated.spring(scaleAnim, {
      toValue,
      useNativeDriver: true,
      mass: 0.8
    }).start();
  };
  const handlePressIn = (ev) => {
    animatePress(0.9);
    if (process.env.EXPO_OS === "ios") {
      Haptics.impactAsync(Haptics.ImpactFeedbackStyle.Light);
    }
    props.onPressIn?.(ev);
  };
  const handlePressOut = (ev) => {
    animatePress(1);
    props.onPressOut?.(ev);
  };
  return /* @__PURE__ */ React.createElement(Animated.View, { style: { transform: [{ scale: scaleAnim }] } },
/* @__PURE__ */ React.createElement(
    PlatformPressable,
    {
      ...props,
      onPressIn: handlePressIn,
      onPressOut: handlePressOut
    }
  ));
}
```

FILE: pos/mobile/components/inventory/nativePickerProps.js
==========

```
import { Platform } from "react-native";
export const PLACEHOLDER_PH = "#9ca3af";
export const PICKER_ITEM_COLOR = "#111827";
export const nativePickerProps = {
  style: {
    color: PICKER_ITEM_COLOR,
    backgroundColor: "#ffffff",
    ...Platform.OS === "android" ? { minHeight: 48 } : {}
```

```
  },
  itemStyle: Platform.OS === "ios" ? { color: PICKER_ITEM_COLOR, fontSize: 15 } : void 0,
  dropdownIconColor: "#6b7280",
  mode: Platform.OS === "android" ? "dropdown" : void 0
};
```

FILE: pos/mobile/components/inventory/StockInModal.jsx
==========

```
import DateTimePicker from "@react-native-community/datetimepicker";
import { Ionicons } from "@expo/vector-icons";
import { Picker } from "@react-native-picker/picker";
import { Fragment, useCallback, useEffect, useMemo, useState } from "react";
import {
  ActivityIndicator,
  Alert,
  Modal,
  Platform,
  Pressable,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View
} from "react-native";
import { brandPartnerAPI } from "../../services/api";
import {
  PLACEHOLDER_PH,
  PICKER_ITEM_COLOR,
  nativePickerProps
} from "./nativePickerProps";
const VARIANT_ONLY_SIZE_KEY = "__VARIANT_ONLY__";
const buildPickerCombos = (v1Tags, v2Tags, existingSet, addedSet) => {
  const out = [];
  if (v1Tags.length > 0 && v2Tags.length > 0) {
    v1Tags.forEach((v1) => {
      v2Tags.forEach((v2) => {
        const key = `${v2}|${v1}`;
        if (!existingSet.has(key) && !addedSet.has(key))
          out.push({ size: v2, variant: v1 });
      });
    });
  } else if (v1Tags.length > 0) {
    v1Tags.forEach((v1) => {
      const key = `${VARIANT_ONLY_SIZE_KEY}|${v1}`;
      if (!existingSet.has(key) && !addedSet.has(key))
        out.push({ size: VARIANT_ONLY_SIZE_KEY, variant: v1 });
    });
  }
  return out;
};
const COLOR_OPTIONS = [
  "White",
  "Black",
  "Red",
  "Blue",
  "Green",
  "Yellow",
  "Pink",
  "Purple",
  "Orange",
  "Brown",
  "Gray",
  "Beige",
  "Navy",
  "Maroon",
  "Cream",
  "Teal"
];
const V2_SIZE_OPTIONS = [
  "XS",
  "S",
  "M",
  "L",
```

```
  "XL",
  "XXL",
  "XXXL",
  "Free Size",
  "Small",
  "Medium",
  "Large"
];
const StockInModal = ({ visible, onClose, product, onConfirm, loading }) => {
  const [brandPartners, setBrandPartners] = useState([]);
  const [selectedSizes, setSelectedSizes] = useState([]);
  const [sizeQuantities, setSizeQuantities] = useState({});
  const [variantQuantities, setVariantQuantities] = useState({});
  const [quantity, setQuantity] = useState("");
  const [batchCode, setBatchCode] = useState("");
  const [batchExpirationDate, setBatchExpirationDate] = useState("");
  const [reason, setReason] = useState("Restock");
  const [otherReason, setOtherReason] = useState("");
  const [newVariantInputs, setNewVariantInputs] = useState({});
  const [addedVariants, setAddedVariants] = useState([]);
  const [newVariantPrices, setNewVariantPrices] = useState({});
  const [addedNewSizes, setAddedNewSizes] = useState([]);
  const [newSizePrices, setNewSizePrices] = useState({});
  const [diffPricesPerVariant, setDiffPricesPerVariant] = useState({});
  const [stockVariantPrices, setStockVariantPrices] = useState({});
  const [currentStep, setCurrentStep] = useState(1);
  const [checkedCombos, setCheckedCombos] = useState({});
  const [showAddNewSection, setShowAddNewSection] = useState(false);
  const [newV1Tags, setNewV1Tags] = useState([]);
  const [newV2Tags, setNewV2Tags] = useState([]);
  const [newV1Input, setNewV1Input] = useState("");
  const [newV2Input, setNewV2Input] = useState("");
  const [addedNewCombos, setAddedNewCombos] = useState([]);
  const [newComboData, setNewComboData] = useState({});
  const [fillAllCostSI, setFillAllCostSI] = useState("");
  const [fillAllSellSI, setFillAllSellSI] = useState("");
  const [fillAllQtySI, setFillAllQtySI] = useState("");
  const [stockInBrandPartner, setStockInBrandPartner] = useState("");
  const [dateReceived, setDateReceived] = useState("");
  const [stockInBatchChoice, setStockInBatchChoice] = useState("new");
  const [showDateReceivedPicker, setShowDateReceivedPicker] = useState(false);
  const [showExpiryPicker, setShowExpiryPicker] = useState(false);
  const [v1PickerKey, setV1PickerKey] = useState("");
  const [v2PickerKey, setV2PickerKey] = useState("");
  const reasons = ["Restock", "Returned Item", "Exchange", "Other"];
  const displayName = product?.itemName || product?.name || "";
  const hasSizes = product?.sizes && typeof product.sizes === "object" && Object.keys(product.sizes).length >
0;
  const getSizeQuantity = (sizeData) => {
    if (typeof sizeData === "object" && sizeData !== null && sizeData.quantity !== void 0) {
      return sizeData.quantity;
    }
    return typeof sizeData === "number" ? sizeData : 0;
  };
  const getSizeVariants = (size) => {
    if (hasSizes && product.sizes[size] && typeof product.sizes[size] === "object" &&
product.sizes[size].variants) {
      return product.sizes[size].variants;
    }
    return null;
  };
  const getAllVariants = () => {
    const variantSet = /* @__PURE__ */ new Set();
    if (hasSizes) {
      Object.values(product.sizes).forEach((sizeData) => {
        if (typeof sizeData === "object" && sizeData?.variants) {
          Object.keys(sizeData.variants).forEach((v) => variantSet.add(v));
        }
      });
    }
    addedVariants.forEach((v) => variantSet.add(v));
    return Array.from(variantSet);
  };
  const allVariants = getAllVariants();
  const hasVariants = allVariants.length > 0;
  const stockInBatchList = useMemo(() => {
    if (!hasSizes || !product) return [];
```

```
    const getVariantQty = (vData) => {
      if (typeof vData === "number") return parseInt(vData, 10) || 0;
      if (vData && typeof vData === "object") {
        const q = vData.qty ?? vData.quantity ?? 0;
        return parseInt(q, 10) || 0;
      }
      return 0;
    };
    if (hasVariants) {
      const maxBatchDepth2 = (() => {
        let max = 0;
        Object.values(product.sizes).forEach((sd) => {
          if (!sd || typeof sd !== "object" || !sd?.variants) return;
          Object.values(sd.variants).forEach((vData) => {
            const batches = typeof vData === "object" && Array.isArray(vData.batches) ? vData.batches : [];
            max = Math.max(max, batches.length);
          });
        });
        return max;
      })();
      if (maxBatchDepth2 <= 0) {
        const openingBatchCode = product?.batchNumber || product?.openingBatchNumber || "B1";
        let totalQty = 0;
        Object.entries(product.sizes).forEach(([_, sd]) => {
          if (!sd || typeof sd !== "object" || !sd?.variants) return;
          Object.entries(sd.variants).forEach(([__, vData]) => {
            totalQty += getVariantQty(vData);
          });
        });
        if (totalQty <= 0) return [];
        return [{ slotIndex: 0, code: openingBatchCode, totalQty }];
      }
      const slots2 = Array.from({ length: maxBatchDepth2 }, (_, slotIndex) => ({
        slotIndex,
        code: "",
        totalQty: 0
      }));
      Object.entries(product.sizes).forEach(([_, sd]) => {
        if (!sd || typeof sd !== "object" || !sd?.variants) return;
        Object.entries(sd.variants).forEach(([__, vData]) => {
          const batches = typeof vData === "object" && Array.isArray(vData.batches) ? vData.batches : [];
          const fallbackQty = (idx) => idx === 0 ? getVariantQty(vData) : 0;
          for (let slotIndex = 0; slotIndex < maxBatchDepth2; slotIndex++) {
            const b = batches[slotIndex];
            const qty = b ? parseInt(b.qty, 10) || 0 : fallbackQty(slotIndex);
            if (qty <= 0) continue;
            const slot = slots2[slotIndex];
            slot.totalQty += qty;
            if (!slot.code) {
              slot.code = b?.batchCode || (slotIndex === 0 ? product?.batchNumber ||
product?.openingBatchNumber || "" : "");
            }
          }
        });
      });
      return slots2.filter((s2) => s2.totalQty > 0);
    }
    const maxBatchDepth = Math.max(
      0,
      ...Object.values(product.sizes).map((sd) => {
        if (!sd || typeof sd !== "object") return 0;
        return Array.isArray(sd.batches) ? sd.batches.length : 0;
      })
    );
    if (maxBatchDepth <= 0) {
      const openingBatchCode = product?.batchNumber || product?.openingBatchNumber || "B1";
      let totalQty = 0;
      Object.values(product.sizes).forEach((sd) => {
        if (typeof sd === "object" && sd !== null)
          totalQty += getSizeQuantity(sd);
      });
      if (totalQty <= 0) return [];
      return [{ slotIndex: 0, code: openingBatchCode, totalQty }];
    }
    const slots = Array.from({ length: maxBatchDepth }, (_, slotIndex) => ({
      slotIndex,
      code: "",
```

```
      totalQty: 0
    }));
    Object.values(product.sizes).forEach((sd) => {
      if (!sd || typeof sd !== "object") return;
      const batches = Array.isArray(sd.batches) ? sd.batches : [];
      for (let slotIndex = 0; slotIndex < maxBatchDepth; slotIndex++) {
        const b = batches[slotIndex];
        const qty = b ? parseInt(b.qty, 10) || 0 : slotIndex === 0 ? getSizeQuantity(sd) : 0;
        if (qty <= 0) continue;
        const slot = slots[slotIndex];
        slot.totalQty += qty;
        if (!slot.code) {
          slot.code = b?.batchCode || (slotIndex === 0 ? product?.batchNumber || product?.openingBatchNumber
|| "" : "");
        }
      }
    });
    return slots.filter((s2) => s2.totalQty > 0);
  }, [product, hasSizes, hasVariants]);
  const existingSizes = hasSizes ? Object.keys(product.sizes) : [];
  const availableSizes = [...existingSizes, ...addedNewSizes];
  const allPossibleSizes = (() => {
    const category = product?.category || "";
    let sizes = [];
    if (["Tops", "Bottoms", "Dresses"].includes(category)) {
      sizes = ["XS", "S", "M", "L", "XL", "XXL", "XXXL", "Free Size"];
    } else if (category === "Shoes") {
      sizes = ["5", "6", "7", "8", "9", "10", "11", "12"];
    } else if (["Accessories", "Head Wear", "Makeup"].includes(category)) {
      sizes = ["Free Size"];
    } else if (category === "Foods") {
      const subtype = product?.foodSubtype || "";
      if (["Beverages", "Drinks"].includes(subtype)) {
        sizes = ["Small", "Medium", "Large", "Family Size", "Free Size"];
      } else {
        sizes = ["Small", "Medium", "Large", "Free Size"];
      }
    } else {
      sizes = [
        "XS",
        "S",
        "M",
        "L",
        "XL",
        "XXL",
        "XXXL",
        "Small",
        "Medium",
        "Large",
        "Free Size"
      ];
    }
    return sizes.filter(
      (s2) => !existingSizes.includes(s2) && !addedNewSizes.includes(s2)
    );
  })();
  const existingCombos = (() => {
    const combos = [];
    if (hasSizes) {
      existingSizes.forEach((size) => {
        const sizeVariants = getSizeVariants(size);
        if (sizeVariants) {
          Object.keys(sizeVariants).forEach((variant) => {
            const vData = sizeVariants[variant];
            const stock = typeof vData === "object" ? vData.quantity || 0 : typeof vData === "number" ?
vData : 0;
            combos.push({ size, variant, stock });
          });
        }
      });
    }
    return combos;
  })();
  const fetchPartners = useCallback(async () => {
    try {
      const res = await brandPartnerAPI.getAll();
      if (res.success && Array.isArray(res.data)) setBrandPartners(res.data);
```

```
    } catch (e) {
      console.error("brandPartnerAPI.getAll", e);
    }
  }, []);
  useEffect(() => {
    if (visible) fetchPartners();
  }, [visible, fetchPartners]);
  useEffect(() => {
    if (visible && product) {
      setSizeQuantities({});
      setVariantQuantities({});
      setQuantity("");
      const now = /* @__PURE__ */ new Date();
      const y = now.getFullYear();
      const m = String(now.getMonth() + 1).padStart(2, "0");
      const d = String(now.getDate()).padStart(2, "0");
      const h = String(now.getHours()).padStart(2, "0");
      const min = String(now.getMinutes()).padStart(2, "0");
      setBatchCode(`B${y}${m}${d}-${h}${min}`);
      setBatchExpirationDate("");
      setReason("Restock");
      setOtherReason("");
      setNewVariantInputs({});
      setAddedVariants([]);
      setNewVariantPrices({});
      setAddedNewSizes([]);
      setNewSizePrices({});
      setCurrentStep(1);
      setShowAddNewSection(false);
      setNewV1Tags([]);
      setNewV2Tags([]);
      setNewV1Input("");
      setNewV2Input("");
      setAddedNewCombos([]);
      setNewComboData({});
      setFillAllCostSI("");
      setFillAllSellSI("");
      setFillAllQtySI("");
      setStockInBrandPartner("");
      setDateReceived((/* @__PURE__ */ new Date()).toISOString().split("T")[0]);
      setStockInBatchChoice("new");
      const initChecked = {};
      const initPrices = {};
      const initDiff = {};
      if (product.sizes && typeof product.sizes === "object") {
        const sizes = Object.keys(product.sizes);
        setSelectedSizes(sizes);
        sizes.forEach((size) => {
          const sd = product.sizes[size];
          if (typeof sd === "object" && sd?.variants) {
            initDiff[size] = true;
            initPrices[size] = {};
            Object.entries(sd.variants).forEach(([variant, vData]) => {
              initChecked[`${size}|${variant}`] = true;
              const cost = typeof vData === "object" ? vData.costPrice || sd.variantCostPrices?.[variant] ||
sd.costPrice || product.costPrice || 0 : sd.variantCostPrices?.[variant] || sd.costPrice || product.costPrice
|| 0;
              const sell = typeof vData === "object" ? vData.price || sd.variantPrices?.[variant] || sd.price
|| product.itemPrice || 0 : sd.variantPrices?.[variant] || sd.price || product.itemPrice || 0;
              initPrices[size][variant] = { price: sell, costPrice: cost };
            });
          }
        });
      } else {
        setSelectedSizes([]);
      }
      setCheckedCombos(initChecked);
      setStockVariantPrices(initPrices);
      setDiffPricesPerVariant(initDiff);
    }
  }, [visible, product?._id, product?.id]);
  const previewCombosMemo = useMemo(() => {
    const existingSet = new Set(
      existingCombos.map((c) => `${c.size}|${c.variant}`)
    );
    const addedSet = new Set(
      addedNewCombos.map((c) => `${c.size}|${c.variant}`)
```

```
  );
  return buildPickerCombos(newV1Tags, newV2Tags, existingSet, addedSet);
}, [newV1Tags, newV2Tags, existingCombos, addedNewCombos]);
if (!visible || !product) return null;
const handleClose = () => {
  setSelectedSizes([]);
  setSizeQuantities({});
  setVariantQuantities({});
  setQuantity("");
  setBatchCode("");
  setBatchExpirationDate("");
  setStockInBrandPartner("");
  setDateReceived("");
  setStockInBatchChoice("new");
  setReason("Restock");
  setOtherReason("");
  setNewVariantInputs({});
  setAddedVariants([]);
  setNewVariantPrices({});
  setAddedNewSizes([]);
  setNewSizePrices({});
  setDiffPricesPerVariant({});
  setStockVariantPrices({});
  setCurrentStep(1);
  setCheckedCombos({});
  setShowAddNewSection(false);
  setNewV1Tags([]);
  setNewV2Tags([]);
  setNewV1Input("");
  setNewV2Input("");
  setAddedNewCombos([]);
  setNewComboData({});
  setFillAllCostSI("");
  setFillAllSellSI("");
  setFillAllQtySI("");
  onClose();
};
const handleSizeToggle = (size) => {
  const isSelected = selectedSizes.includes(size);
  if (isSelected) {
    setSelectedSizes((prev) => prev.filter((s2) => s2 !== size));
    setSizeQuantities((prev) => {
      const n = { ...prev };
      delete n[size];
      return n;
    });
    setVariantQuantities((prev) => {
      const n = { ...prev };
      delete n[size];
      return n;
    });
  } else {
    setSelectedSizes((prev) => [...prev, size]);
    setSizeQuantities((prev) => ({ ...prev, [size]: "" }));
    if (hasVariants) {
      setVariantQuantities((prev) => ({ ...prev, [size]: {} }));
    }
  }
};
const handleSizeQuantityChange = (size, qty) => {
  setSizeQuantities((prev) => ({ ...prev, [size]: qty }));
};
const handleVariantQuantityChange = (size, variant, qty) => {
  setVariantQuantities((prev) => ({
    ...prev,
    [size]: { ...prev[size] || {}, [variant]: qty }
  }));
};
const handleNewVariantInputChange = (size, value) => {
  setNewVariantInputs((prev) => ({ ...prev, [size]: value }));
};
const handleAddNewVariant = (size) => {
  const variantName = (newVariantInputs[size] || "").trim();
  if (!variantName) {
    Alert.alert("Error", "Please enter a variant name");
    return;
  }
```

```
  if (allVariants.includes(variantName)) {
    Alert.alert("Error", "This variant already exists");
    return;
  }
  setAddedVariants((prev) => [...prev, variantName]);
  setNewVariantPrices((prev) => ({
    ...prev,
    [variantName]: {
      price: product.itemPrice || 0,
      costPrice: product.costPrice || 0
    }
  }));
  setNewVariantInputs((prev) => ({ ...prev, [size]: "" }));
};
const handleStockVariantPriceChange = (size, variant, field, value) => {
  setStockVariantPrices((prev) => ({
    ...prev,
    [size]: {
      ...prev[size] || {},
      [variant]: {
        ...prev[size]?.[variant] || {
          price: product.itemPrice || 0,
          costPrice: product.costPrice || 0
        },
        [field]: value
      }
    }
  }));
};
const handleComboCheck = (size, variant) => {
  const key = `${size}|${variant}`;
  const wasChecked = checkedCombos[key] !== false;
  setCheckedCombos((prev) => ({ ...prev, [key]: !wasChecked }));
  if (wasChecked) {
    setVariantQuantities((prev) => {
      const u = { ...prev };
      if (u[size]) {
        const row = { ...u[size] };
        delete row[variant];
        u[size] = row;
      }
      return u;
    });
  }
};
const handleAddNewCombosFromPicker = () => {
  const existingSet = new Set(
    existingCombos.map((c) => `${c.size}|${c.variant}`)
  );
  const addedSet = new Set(
    addedNewCombos.map((c) => `${c.size}|${c.variant}`)
  );
  const previewCombos = buildPickerCombos(
    newV1Tags,
    newV2Tags,
    existingSet,
    addedSet
  );
  if (previewCombos.length === 0) return;
  setAddedNewCombos((prev) => [...prev, ...previewCombos]);
  const newVars = newV1Tags.filter((v) => !allVariants.includes(v));
  const newSzs = newV2Tags.filter(
    (s2) => !existingSizes.includes(s2) && !addedNewSizes.includes(s2)
  );
  if (newVars.length > 0)
    setAddedVariants((prev) => [.../* @__PURE__ */ new Set([...prev, ...newVars])]);
  if (newSzs.length > 0) {
    setAddedNewSizes((prev) => [.../* @__PURE__ */ new Set([...prev, ...newSzs])]);
    setSelectedSizes((prev) => [.../* @__PURE__ */ new Set([...prev, ...newSzs])]);
  }
  newV2Tags.filter((s2) => existingSizes.includes(s2) && !selectedSizes.includes(s2)).forEach((s2) => {
    setSelectedSizes((prev) => prev.includes(s2) ? prev : [...prev, s2]);
  });
  if (previewCombos.some((c) => c.size === VARIANT_ONLY_SIZE_KEY)) {
    setSelectedSizes(
      (prev) => prev.includes(VARIANT_ONLY_SIZE_KEY) ? prev : [...prev, VARIANT_ONLY_SIZE_KEY]
    );
```

```
    }
    const checkedUpdates = {};
    const priceUpdates = {};
    const qtyUpdates = {};
    previewCombos.forEach(({ size, variant }) => {
      const key = `${size}|${variant}`;
      const data = newComboData[key] || {};
      checkedUpdates[key] = true;
      if (!priceUpdates[size]) priceUpdates[size] = {};
      priceUpdates[size][variant] = {
        price: data.sell || product.itemPrice || 0,
        costPrice: data.cost || product.costPrice || 0
      };
      if (data.qty) {
        if (!qtyUpdates[size]) qtyUpdates[size] = {};
        qtyUpdates[size][variant] = data.qty;
      }
    });
    setCheckedCombos((prev) => ({ ...prev, ...checkedUpdates }));
    setStockVariantPrices((prev) => {
      const u = { ...prev };
      Object.entries(priceUpdates).forEach(([s2, vs]) => {
        u[s2] = { ...u[s2] || {}, ...vs };
      });
      return u;
    });
    if (Object.keys(qtyUpdates).length > 0) {
      setVariantQuantities((prev) => {
        const u = { ...prev };
        Object.entries(qtyUpdates).forEach(([s2, vs]) => {
          u[s2] = { ...u[s2] || {}, ...vs };
        });
        return u;
      });
    }
    setNewV1Tags([]);
    setNewV2Tags([]);
    setNewComboData({});
    setShowAddNewSection(false);
  };
  const handleFillAllSI = () => {
    const allCombosList = [
      ...existingCombos.map((c) => ({ size: c.size, variant: c.variant })),
      ...addedNewCombos
    ];
    const qFill = String(fillAllQtySI ?? "").trim();
    const cFill = String(fillAllCostSI ?? "").trim();
    const sFill = String(fillAllSellSI ?? "").trim();
    if (cFill !== "" || sFill !== "") {
      setStockVariantPrices((prev) => {
        const u = { ...prev };
        allCombosList.forEach(({ size, variant }) => {
          if (checkedCombos[`${size}|${variant}`] !== false) {
            if (!u[size]) u[size] = {};
            u[size][variant] = {
              ...u[size]?.[variant] || {},
              ...cFill !== "" ? { costPrice: cFill } : {},
              ...sFill !== "" ? { price: sFill } : {}
            };
          }
        });
        return u;
      });
    }
    if (qFill !== "") {
      setVariantQuantities((prev) => {
        const u = { ...prev };
        allCombosList.forEach(({ size, variant }) => {
          if (checkedCombos[`${size}|${variant}`] !== false) {
            if (!u[size]) u[size] = {};
            u[size][variant] = qFill;
          }
        });
        return u;
      });
    }
  };
```

```
  const isStepValid = (step) => {
    if (step === 1) {
      if (!hasSizes) return (parseInt(quantity, 10) || 0) > 0;
      if (hasVariants) {
        const allCombosList = [
          ...existingCombos.map((c) => ({ size: c.size, variant: c.variant })),
          ...addedNewCombos
        ];
        return allCombosList.some(({ size, variant }) => {
          return checkedCombos[`${size}|${variant}`] !== false &&
(parseInt(variantQuantities[size]?.[variant], 10) || 0) > 0;
        });
      }
      return selectedSizes.some((s2) => (parseInt(sizeQuantities[s2], 10) || 0) > 0);
    }
    if (step === 2) {
      if (!stockInBrandPartner) return false;
      if (!dateReceived) return false;
      if (reason === "Other" && !otherReason.trim()) return false;
      return true;
    }
    return true;
  };
  const handleSubmit = () => {
    if (currentStep !== 3) return;
    if (reason === "Other" && !otherReason.trim()) {
      Alert.alert("Error", "Please specify the reason");
      return;
    }
    const finalReason = reason === "Other" ? `Other: ${otherReason.trim()}` : reason;
    const stockInBatchPayload = !hasSizes ? {
      ...batchCode.trim() ? { batchCode: batchCode.trim() } : {},
      ...batchExpirationDate ? { expirationDate: batchExpirationDate } : {}
    } : stockInBatchChoice !== "new" ? {
      targetBatchSlotIndex: parseInt(stockInBatchChoice, 10),
      ...batchExpirationDate ? { expirationDate: batchExpirationDate } : {}
    } : {
      ...batchCode.trim() ? { batchCode: batchCode.trim() } : {},
      ...batchExpirationDate ? { expirationDate: batchExpirationDate } : {}
    };
    if (!hasSizes) {
      const qty = parseInt(quantity, 10) || 0;
      if (qty <= 0) {
        Alert.alert("Error", "Please enter a valid quantity");
        return;
      }
      onConfirm({
        quantity: qty,
        noSizes: true,
        reason: finalReason,
        ...stockInBatchPayload,
        ...stockInBrandPartner ? { brandPartner: stockInBrandPartner } : {},
        ...dateReceived ? { dateReceived } : {}
      });
      return;
    }
    if (selectedSizes.length === 0) {
      Alert.alert("Error", "Please select at least one size");
      return;
    }
    if (hasVariants) {
      const filteredVarQtys = {};
      const checkedSizesSet = /* @__PURE__ */ new Set();
      [
        ...existingCombos.map((c) => ({ size: c.size, variant: c.variant })),
        ...addedNewCombos
      ].forEach(({ size, variant }) => {
        const key = `${size}|${variant}`;
        const qty = parseInt(variantQuantities[size]?.[variant], 10) || 0;
        if (checkedCombos[key] !== false && qty > 0) {
          if (!filteredVarQtys[size]) filteredVarQtys[size] = {};
          filteredVarQtys[size][variant] = variantQuantities[size][variant];
          checkedSizesSet.add(size);
        }
      });
      if (Object.keys(filteredVarQtys).length === 0) {
        Alert.alert(
```

```
          "Error",
          "Please enter quantities for at least one checked variant"
        );
        return;
      }
      const filteredSizes = Array.from(checkedSizesSet);
      onConfirm({
        sizes: sizeQuantities,
        variantQuantities: filteredVarQtys,
        selectedSizes: filteredSizes,
        reason: finalReason,
        hasVariants: true,
        newVariantPrices: addedVariants.length > 0 ? newVariantPrices : null,
        newSizePrices: addedNewSizes.length > 0 ? newSizePrices : null,
        diffPricesPerVariant: Object.keys(diffPricesPerVariant).some(
          (k) => diffPricesPerVariant[k]
        ) ? diffPricesPerVariant : null,
        stockVariantPrices: Object.keys(stockVariantPrices).length > 0 ? stockVariantPrices : null,
        ...stockInBatchPayload,
        ...stockInBrandPartner ? { brandPartner: stockInBrandPartner } : {},
        ...dateReceived ? { dateReceived } : {}
      });
      return;
    }
    const hasValidQuantities = selectedSizes.some((size) => {
      const qty = sizeQuantities[size] || 0;
      return parseInt(qty, 10) > 0;
    });
    if (!hasValidQuantities) {
      Alert.alert(
        "Error",
        "Please enter quantities for at least one selected size"
      );
      return;
    }
    onConfirm({
      sizes: sizeQuantities,
      selectedSizes,
      reason: finalReason,
      newSizePrices: addedNewSizes.length > 0 ? newSizePrices : null,
      ...stockInBatchPayload,
      ...stockInBrandPartner ? { brandPartner: stockInBrandPartner } : {},
      ...dateReceived ? { dateReceived } : {}
    });
  };
  const brandNameOptions = [
    ...new Set(brandPartners.map((bp) => bp.brandName).filter(Boolean))
  ].sort();
  const renderStep1 = () => /* @__PURE__ */ React.createElement(View, { style: s.stepBlock }, /* @__PURE__ */
React.createElement(Text, { style: s.stepHeading }, "ITEMS TO STOCK IN"), /* @__PURE__ */
React.createElement(Text, { style: s.stepSub }, "Check variants that arrived; unchecked = skip"), hasSizes &&
hasVariants ? /* @__PURE__ */ React.createElement(React.Fragment, null, /* @__PURE__ */
React.createElement(Text, { style: s.muted }, "Existing variants"), /* @__PURE__ */ React.createElement(View,
{ style: s.tableBorder }, /* @__PURE__ */ React.createElement(View, { style: s.comboHeaderRow }, /* @__PURE__
*/ React.createElement(Text, { style: [s.comboH, { width: 22 }] }), /* @__PURE__ */ React.createElement(Text,
{ style: [s.comboH, { flex: 1 }] }, "V1"), /* @__PURE__ */ React.createElement(Text, { style: [s.comboH,
{ flex: 1 }] }, "V2"), /* @__PURE__ */ React.createElement(Text, { style: [s.comboH, { width: 36, textAlign:
"center" }] }, "Stk"), /* @__PURE__ */ React.createElement(Text, { style: [s.comboH, { width: 44, textAlign:
"center" }] }, "Qty"), /* @__PURE__ */ React.createElement(Text, { style: [s.comboH, { width: 52, textAlign:
"center" }] }, "Cost"), /* @__PURE__ */ React.createElement(Text, { style: [s.comboH, { width: 52, textAlign:
"center" }] }, "Sell")), /* @__PURE__ */ React.createElement(ScrollView, { style: { maxHeight: 220 },
nestedScrollEnabled: true }, existingCombos.map(({ size, variant, stock }) => {
    const key = `${size}|${variant}`;
    const checked = checkedCombos[key] !== false;
    return /* @__PURE__ */ React.createElement(View, { key, style: s.comboRow }, /* @__PURE__ */
React.createElement(
      Pressable,
      {
        style: { width: 22 },
        onPress: () => handleComboCheck(size, variant)
      },
      /* @__PURE__ */ React.createElement(
        View,
        {
          style: [
            s.cb,
            checked && s.cbOn
```

```
          ]
        },
        checked ? /* @__PURE__ */ React.createElement(Text, { style: s.cbTick }, "\u2713") : null
      )
    ), /* @__PURE__ */ React.createElement(Text, { style: [s.pillPink, { flex: 1 }], numberOfLines: 1 },
variant), /* @__PURE__ */ React.createElement(Text, { style: [s.pillGrn, { flex: 1 }], numberOfLines: 1 },
size === VARIANT_ONLY_SIZE_KEY ? "\u2014" : size), /* @__PURE__ */ React.createElement(Text, { style: [s.stk,
{ width: 36 }] }, stock), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: [s.tinyIn, { width: 44 }],
        editable: checked,
        keyboardType: "number-pad",
        value: String(variantQuantities[size]?.[variant] ?? ""),
        onChangeText: (t) => handleVariantQuantityChange(size, variant, t),
        placeholderTextColor: PLACEHOLDER_PH,
        placeholder: "Qty",
        underlineColorAndroid: "transparent"
      }
    ), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: [s.tinyIn, { width: 52 }],
        editable: checked,
        keyboardType: "decimal-pad",
        value: String(
          stockVariantPrices[size]?.[variant]?.costPrice ?? ""
        ),
        onChangeText: (t) => handleStockVariantPriceChange(
          size,
          variant,
          "costPrice",
          t
        ),
        placeholderTextColor: PLACEHOLDER_PH,
        placeholder: "Cost",
        underlineColorAndroid: "transparent"
      }
    ), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: [s.tinyIn, { width: 52 }],
        editable: checked,
        keyboardType: "decimal-pad",
        value: String(
          stockVariantPrices[size]?.[variant]?.price ?? ""
        ),
        onChangeText: (t) => handleStockVariantPriceChange(size, variant, "price", t),
        placeholderTextColor: PLACEHOLDER_PH,
        placeholder: "Sell",
        underlineColorAndroid: "transparent"
      }
    ));
  }), addedNewCombos.length > 0 && /* @__PURE__ */ React.createElement(Text, { style: s.newLbl }, "NEW"),
addedNewCombos.map(({ size, variant }) => {
    const key = `${size}|${variant}`;
    const checked = checkedCombos[key] !== false;
    return /* @__PURE__ */ React.createElement(View, { key: `n-${key}`, style: s.comboRow }, /* @__PURE__ */
React.createElement(
      Pressable,
      {
        style: { width: 22 },
        onPress: () => handleComboCheck(size, variant)
      },
      /* @__PURE__ */ React.createElement(View, { style: [s.cb, checked && s.cbOn] }, checked ? /* @__PURE__
*/ React.createElement(Text, { style: s.cbTick }, "\u2713") : null)
    ), /* @__PURE__ */ React.createElement(Text, { style: [s.pillPink, { flex: 1 }], numberOfLines: 1 },
variant), /* @__PURE__ */ React.createElement(Text, { style: [s.pillGrn, { flex: 1 }], numberOfLines: 1 },
size === VARIANT_ONLY_SIZE_KEY ? "\u2014" : size), /* @__PURE__ */ React.createElement(Text, { style: [s.stk,
{ width: 36, color: "#9ca3af" }] }, "\u2014"), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: [s.tinyIn, { width: 44 }],
        editable: checked,
        keyboardType: "number-pad",
        value: String(variantQuantities[size]?.[variant] ?? ""),
        onChangeText: (t) => handleVariantQuantityChange(size, variant, t),
```

```
        placeholderTextColor: PLACEHOLDER_PH,
        placeholder: "Qty",
        underlineColorAndroid: "transparent"
      }
    ), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: [s.tinyIn, { width: 52 }],
        editable: checked,
        keyboardType: "decimal-pad",
        value: String(
          stockVariantPrices[size]?.[variant]?.costPrice ?? ""
        ),
        onChangeText: (t) => handleStockVariantPriceChange(
          size,
          variant,
          "costPrice",
          t
        ),
        placeholderTextColor: PLACEHOLDER_PH,
        placeholder: "Cost",
        underlineColorAndroid: "transparent"
      }
    ), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: [s.tinyIn, { width: 52 }],
        editable: checked,
        keyboardType: "decimal-pad",
        value: String(
          stockVariantPrices[size]?.[variant]?.price ?? ""
        ),
        onChangeText: (t) => handleStockVariantPriceChange(size, variant, "price", t),
        placeholderTextColor: PLACEHOLDER_PH,
        placeholder: "Sell",
        underlineColorAndroid: "transparent"
      }
    ));
  })), /* @__PURE__ */ React.createElement(View, { style: s.fillRow }, /* @__PURE__ */
React.createElement(Text, { style: s.fillLbl }, "FILL"), /* @__PURE__ */ React.createElement(
    TextInput,
    {
      style: s.tinyIn,
      placeholderTextColor: PLACEHOLDER_PH,
      placeholder: "Qty",
      keyboardType: "number-pad",
      value: fillAllQtySI,
      onChangeText: setFillAllQtySI,
      underlineColorAndroid: "transparent"
    }
  ), /* @__PURE__ */ React.createElement(
    TextInput,
    {
      style: s.tinyIn,
      placeholderTextColor: PLACEHOLDER_PH,
      placeholder: "Cost",
      keyboardType: "decimal-pad",
      value: fillAllCostSI,
      onChangeText: setFillAllCostSI,
      underlineColorAndroid: "transparent"
    }
  ), /* @__PURE__ */ React.createElement(
    TextInput,
    {
      style: s.tinyIn,
      placeholderTextColor: PLACEHOLDER_PH,
      placeholder: "Sell",
      keyboardType: "decimal-pad",
      value: fillAllSellSI,
      onChangeText: setFillAllSellSI,
      underlineColorAndroid: "transparent"
    }
  ), /* @__PURE__ */ React.createElement(TouchableOpacity, { onPress: handleFillAllSI }, /* @__PURE__ */
React.createElement(Text, { style: s.fillBtn }, "Fill All")))), /* @__PURE__ */ React.createElement(View,
{ style: s.addNewBox }, /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
```

```
      style: s.addNewHead,
      onPress: () => setShowAddNewSection(!showAddNewSection)
    },
    /* @__PURE__ */ React.createElement(Text, { style: s.addNewTitle }, "+ add NEW Variants"),
    /* @__PURE__ */ React.createElement(Text, { style: s.addNewSub }, showAddNewSection ? "collapse" :
"expand")
  ), showAddNewSection && /* @__PURE__ */ React.createElement(View, { style: s.addNewBody }, /* @__PURE__ */
React.createElement(View, { style: s.twoCol }, /* @__PURE__ */ React.createElement(View, { style: s.vCard },
/* @__PURE__ */ React.createElement(Text, { style: s.vCardTitle }, "VARIANT 1 \u2013 Colors"), /* @__PURE__
*/ React.createElement(View, { style: s.pickerWrap }, /* @__PURE__ */ React.createElement(
    Picker,
    {
      ...nativePickerProps,
      selectedValue: v1PickerKey,
      onValueChange: (val) => {
        setV1PickerKey("");
        if (val && !newV1Tags.includes(val))
          setNewV1Tags((p) => [...p, val]);
      }
    },
    /* @__PURE__ */ React.createElement(
      Picker.Item,
      {
        label: "Select color...",
        value: "",
        color: PICKER_ITEM_COLOR
      }
    ),
    COLOR_OPTIONS.filter((c) => !newV1Tags.includes(c)).map(
      (c) => /* @__PURE__ */ React.createElement(
        Picker.Item,
        {
          key: c,
          label: c,
          value: c,
          color: PICKER_ITEM_COLOR
        }
      )
    )
  )), /* @__PURE__ */ React.createElement(View, { style: s.tagRow }, newV1Tags.map((tag) => /* @__PURE__ */
React.createElement(View, { key: tag, style: s.tagPink }, /* @__PURE__ */ React.createElement(Text, { style:
s.tagTxt }, tag), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      onPress: () => setNewV1Tags((p) => p.filter((t) => t !== tag))
    },
    /* @__PURE__ */ React.createElement(Text, { style: s.tagX }, "\xD7")
  ))), /* @__PURE__ */ React.createElement(
    TextInput,
    {
      style: s.tagInput,
      value: newV1Input,
      onChangeText: setNewV1Input,
      placeholderTextColor: PLACEHOLDER_PH,
      placeholder: "Add +",
      onSubmitEditing: () => {
        const v = newV1Input.trim();
        if (v && !newV1Tags.includes(v)) {
          setNewV1Tags((p) => [...p, v]);
          setNewV1Input("");
        }
      },
      underlineColorAndroid: "transparent"
    }
  ))), /* @__PURE__ */ React.createElement(View, { style: s.vCard }, /* @__PURE__ */ React.createElement(Text,
{ style: s.vCardTitle }, "VARIANT 2 \u2013 Size", " ", /* @__PURE__ */ React.createElement(Text, { style:
s.optional }, "(optional)")), /* @__PURE__ */ React.createElement(View, { style: s.pickerWrap }, /* @__PURE__
*/ React.createElement(
    Picker,
    {
      ...nativePickerProps,
      selectedValue: v2PickerKey,
      onValueChange: (val) => {
        setV2PickerKey("");
        if (val && !newV2Tags.includes(val))
          setNewV2Tags((p) => [...p, val]);
      }
```

```
    },
    /* @__PURE__ */ React.createElement(
      Picker.Item,
      {
        label: "Select size...",
        value: "",
        color: PICKER_ITEM_COLOR
      }
    ),
    V2_SIZE_OPTIONS.filter(
      (sz) => !newV2Tags.includes(sz)
    ).map((sz) => /* @__PURE__ */ React.createElement(
      Picker.Item,
      {
        key: sz,
        label: sz,
        value: sz,
        color: PICKER_ITEM_COLOR
      }
    ))
  )), /* @__PURE__ */ React.createElement(View, { style: s.tagRow }, newV2Tags.map((tag) => /* @__PURE__ */
React.createElement(View, { key: tag, style: s.tagGrn }, /* @__PURE__ */ React.createElement(Text, { style:
s.tagTxtGrn }, tag), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      onPress: () => setNewV2Tags((p) => p.filter((t) => t !== tag))
    },
    /* @__PURE__ */ React.createElement(Text, { style: s.tagXGrn }, "\xD7")
  ))), /* @__PURE__ */ React.createElement(
    TextInput,
    {
      style: s.tagInput,
      value: newV2Input,
      onChangeText: setNewV2Input,
      placeholderTextColor: PLACEHOLDER_PH,
      placeholder: "Add +",
      onSubmitEditing: () => {
        const v = newV2Input.trim();
        if (v && !newV2Tags.includes(v)) {
          setNewV2Tags((p) => [...p, v]);
          setNewV2Input("");
        }
      },
      underlineColorAndroid: "transparent"
    }
  )))), previewCombosMemo.length > 0 && /* @__PURE__ */ React.createElement(React.Fragment, null, /*
@__PURE__ */ React.createElement(Text, { style: s.previewCount }, previewCombosMemo.length, " COMBINATIONS
GENERATED"), /* @__PURE__ */ React.createElement(ScrollView, { style: { maxHeight: 160 }, nestedScrollEnabled:
true }, previewCombosMemo.map(({ size, variant }) => {
    const k = `${size}|${variant}`;
    return /* @__PURE__ */ React.createElement(View, { key: k, style: s.previewRow }, /* @__PURE__ */
React.createElement(View, { style: { flex: 1, flexDirection: "row", flexWrap: "wrap", gap: 4 } }, size !==
VARIANT_ONLY_SIZE_KEY && /* @__PURE__ */ React.createElement(Text, { style: s.pillGrnSm }, size), size !==
VARIANT_ONLY_SIZE_KEY && /* @__PURE__ */ React.createElement(Text, { style: s.mul }, "\xD7"), /* @__PURE__ */
React.createElement(Text, { style: s.pillPinkSm }, variant)), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: s.prevIn,
        keyboardType: "decimal-pad",
        placeholderTextColor: PLACEHOLDER_PH,
        placeholder: "Cost",
        value: newComboData[k]?.cost || "",
        onChangeText: (t) => setNewComboData((p) => ({
          ...p,
          [k]: { ...p[k] || {}, cost: t }
        })),
        underlineColorAndroid: "transparent"
      }
    ), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: s.prevIn,
        keyboardType: "decimal-pad",
        placeholderTextColor: PLACEHOLDER_PH,
        placeholder: "Sell",
        value: newComboData[k]?.sell || "",
        onChangeText: (t) => setNewComboData((p) => ({
```

```
          ...p,
          [k]: { ...p[k] || {}, sell: t }
        })),
        underlineColorAndroid: "transparent"
      }
    ), /* @__PURE__ */ React.createElement(
      TextInput,
      {
        style: s.prevIn,
        keyboardType: "number-pad",
        placeholderTextColor: PLACEHOLDER_PH,
        placeholder: "Qty",
        value: newComboData[k]?.qty || "",
        onChangeText: (t) => setNewComboData((p) => ({
          ...p,
          [k]: { ...p[k] || {}, qty: t }
        })),
        underlineColorAndroid: "transparent"
      }
    ));
  })), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: s.addComboBtn,
      onPress: handleAddNewCombosFromPicker
    },
    /* @__PURE__ */ React.createElement(Text, { style: s.addComboBtnTxt }, "Add")
  ))))) : hasSizes ? /* @__PURE__ */ React.createElement(View, null, /* @__PURE__ */ React.createElement(Text,
{ style: s.muted }, "Select sizes to add stock to"), /* @__PURE__ */ React.createElement(View, { style:
s.sizeChecks }, availableSizes.map((size) => {
    const currentQty = hasSizes && product.sizes[size] ? getSizeQuantity(product.sizes[size]) : 0;
    return /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        key: size,
        style: s.sizeCheckRow,
        onPress: () => handleSizeToggle(size)
      },
      /* @__PURE__ */ React.createElement(View, { style: [s.cb, selectedSizes.includes(size) && s.cbOn] },
selectedSizes.includes(size) ? /* @__PURE__ */ React.createElement(Text, { style: s.cbTick }, "\u2713") :
null),
      /* @__PURE__ */ React.createElement(Text, { style: s.sizeCheckTxt }, size, " ", /* @__PURE__ */
React.createElement(Text, { style: s.sizeQty }, "(", currentQty, ")"))
    );
  })), selectedSizes.length > 0 && /* @__PURE__ */ React.createElement(View, { style: s.qtyBox }, /*
@__PURE__ */ React.createElement(Text, { style: s.qtyBoxTitle }, "Quantity per Size:"),
selectedSizes.map((size) => /* @__PURE__ */ React.createElement(View, { key: size, style: s.qtyLine }, /*
@__PURE__ */ React.createElement(Text, { style: s.qtyLab }, size), /* @__PURE__ */ React.createElement(
    TextInput,
    {
      style: s.qtyIn,
      keyboardType: "number-pad",
      value: String(sizeQuantities[size] ?? ""),
      onChangeText: (t) => handleSizeQuantityChange(size, t),
      placeholderTextColor: PLACEHOLDER_PH,
      placeholder: "Qty",
      underlineColorAndroid: "transparent"
    }
  ))))) : /* @__PURE__ */ React.createElement(View, null, /* @__PURE__ */ React.createElement(Text, { style:
s.muted }, "Current Stock: ", product.currentStock || 0), /* @__PURE__ */ React.createElement(
    TextInput,
    {
      style: s.input,
      keyboardType: "number-pad",
      value: quantity,
      onChangeText: setQuantity,
      placeholderTextColor: PLACEHOLDER_PH,
      placeholder: "Enter quantity to add",
      underlineColorAndroid: "transparent"
    }
  )));
  const renderStep2 = () => /* @__PURE__ */ React.createElement(View, { style: s.stepBlock }, /* @__PURE__ */
React.createElement(Text, { style: s.stepHeading }, "ENTRY DETAILS"), /* @__PURE__ */
React.createElement(Text, { style: s.label }, "BRAND PARTNER/SUPPLIER ", /* @__PURE__ */
React.createElement(Text, { style: s.req }, "*")), /* @__PURE__ */ React.createElement(View, { style:
s.pickerWrap }, /* @__PURE__ */ React.createElement(
    Picker,
```

```
    {
      ...nativePickerProps,
      selectedValue: stockInBrandPartner,
      onValueChange: setStockInBrandPartner
    },
    /* @__PURE__ */ React.createElement(
      Picker.Item,
      {
        label: "Select Brand Partner",
        value: "",
        color: PICKER_ITEM_COLOR
      }
    ),
    brandNameOptions.map((name) => /* @__PURE__ */ React.createElement(
      Picker.Item,
      {
        key: name,
        label: name,
        value: name,
        color: PICKER_ITEM_COLOR
      }
    ))
  )), /* @__PURE__ */ React.createElement(Text, { style: s.label }, "DATE RECEIVED ", /* @__PURE__ */
React.createElement(Text, { style: s.req }, "*")), /* @__PURE__ */ React.createElement(
    Pressable,
    {
      style: s.datePress,
      onPress: () => setShowDateReceivedPicker(true)
    },
    /* @__PURE__ */ React.createElement(Text, { style: dateReceived ? s.datePressTxt :
s.datePressPlaceholder }, dateReceived || "Select date")
  ), showDateReceivedPicker && /* @__PURE__ */ React.createElement(
    DateTimePicker,
    {
      value: dateReceived ? /* @__PURE__ */ new Date(dateReceived + "T12:00:00") : /* @__PURE__ */ new Date(),
      mode: "date",
      display: Platform.OS === "ios" ? "spinner" : "default",
      onChange: (event, d) => {
        if (Platform.OS === "android") setShowDateReceivedPicker(false);
        if (event?.type === "dismissed") {
          setShowDateReceivedPicker(false);
          return;
        }
        if (d) {
          setDateReceived(d.toISOString().slice(0, 10));
        }
        if (Platform.OS === "ios") setShowDateReceivedPicker(false);
      }
    }
  ), /* @__PURE__ */ React.createElement(Text, { style: s.label }, "BATCH NUMBER"), hasSizes ? /* @__PURE__
*/ React.createElement(View, { style: s.pickerWrap }, /* @__PURE__ */ React.createElement(
    Picker,
    {
      ...nativePickerProps,
      selectedValue: stockInBatchChoice,
      onValueChange: (v) => {
        setStockInBatchChoice(v);
        if (v === "new") {
          const now = /* @__PURE__ */ new Date();
          const y = now.getFullYear();
          const mo = String(now.getMonth() + 1).padStart(2, "0");
          const d = String(now.getDate()).padStart(2, "0");
          const h = String(now.getHours()).padStart(2, "0");
          const min = String(now.getMinutes()).padStart(2, "0");
          setBatchCode(`B${y}${mo}${d}-${h}${min}`);
        } else {
          const slot = stockInBatchList.find(
            (b) => String(b.slotIndex) === v
          );
          setBatchCode(slot?.code || "");
        }
      }
    },
    /* @__PURE__ */ React.createElement(
      Picker.Item,
      {
        label: `New batch${stockInBatchChoice === "new" && batchCode ? ` (${batchCode})` : ""}`,
```

```
        value: "new",
        color: PICKER_ITEM_COLOR
      }
    ),
    stockInBatchList.map((b) => /* @__PURE__ */ React.createElement(
      Picker.Item,
      {
        key: b.slotIndex,
        label: `Batch ${b.slotIndex + 1}${b.code ? ` (${b.code})` : ""} \xB7 ${b.totalQty} pcs`,
        value: String(b.slotIndex),
        color: PICKER_ITEM_COLOR
      }
    ))
  )) : /* @__PURE__ */ React.createElement(Text, { style: s.batchBig }, batchCode || "\u2014"), /* @__PURE__
*/ React.createElement(Text, { style: s.label }, "EXPIRING DATE (IF APPLICABLE)"), /* @__PURE__ */
React.createElement(Pressable, { style: s.datePress, onPress: () => setShowExpiryPicker(true) }, /* @__PURE__
*/ React.createElement(
    Text,
    {
      style: batchExpirationDate ? s.datePressTxt : s.datePressPlaceholder
    },
    batchExpirationDate || "Select date (optional)"
  )), showExpiryPicker && /* @__PURE__ */ React.createElement(
    DateTimePicker,
    {
      value: batchExpirationDate ? /* @__PURE__ */ new Date(batchExpirationDate + "T12:00:00") : /* @__PURE__
*/ new Date(),
      mode: "date",
      display: Platform.OS === "ios" ? "spinner" : "default",
      onChange: (event, d) => {
        if (Platform.OS === "android") setShowExpiryPicker(false);
        if (event?.type === "dismissed") {
          setShowExpiryPicker(false);
          return;
        }
        if (d) {
          setBatchExpirationDate(d.toISOString().slice(0, 10));
        }
        if (Platform.OS === "ios") setShowExpiryPicker(false);
      }
    }
  ), /* @__PURE__ */ React.createElement(Text, { style: s.label }, "REASON"), /* @__PURE__ */
React.createElement(View, { style: s.pickerWrap }, /* @__PURE__ */ React.createElement(
    Picker,
    {
      ...nativePickerProps,
      selectedValue: reason,
      onValueChange: (v) => {
        setReason(v);
        if (v !== "Other") setOtherReason("");
      }
    },
    reasons.map((r) => /* @__PURE__ */ React.createElement(
      Picker.Item,
      {
        key: r,
        label: r,
        value: r,
        color: PICKER_ITEM_COLOR
      }
    ))
  )), reason === "Other" && /* @__PURE__ */ React.createElement(
    TextInput,
    {
      style: [s.input, { marginTop: 8 }],
      value: otherReason,
      onChangeText: setOtherReason,
      placeholderTextColor: PLACEHOLDER_PH,
      placeholder: "Please specify the reason",
      underlineColorAndroid: "transparent"
    }
  ));
  const renderStep3 = () => {
    const reviewRows = [];
    if (hasSizes && hasVariants) {
      const allCombos = [
        ...existingCombos.map((c) => ({ size: c.size, variant: c.variant })),
```

```
        ...addedNewCombos
      ];
      allCombos.forEach(({ size, variant }) => {
        const key = `${size}|${variant}`;
        if (checkedCombos[key] === false) return;
        const qty = parseInt(variantQuantities[size]?.[variant], 10) || 0;
        if (qty <= 0) return;
        const sizeData = product.sizes?.[size];
        const existingStock = typeof sizeData?.variants?.[variant] === "object" ?
sizeData.variants[variant].quantity || 0 : typeof sizeData?.variants?.[variant] === "number" ?
sizeData.variants[variant] : 0;
        const cost = stockVariantPrices[size]?.[variant]?.costPrice || 0;
        const sell = stockVariantPrices[size]?.[variant]?.price || 0;
        reviewRows.push({
          variant,
          size,
          stock: existingStock,
          qtyIn: qty,
          cost,
          sell
        });
      });
    } else if (hasSizes) {
      selectedSizes.forEach((size) => {
        const qty = parseInt(sizeQuantities[size], 10) || 0;
        if (qty <= 0) return;
        const sizeData = product.sizes?.[size];
        const existingStock = sizeData?.quantity || 0;
        reviewRows.push({
          variant: null,
          size,
          stock: existingStock,
          qtyIn: qty,
          cost: 0,
          sell: 0
        });
      });
    } else {
      const qty = parseInt(quantity, 10) || 0;
      if (qty > 0)
        reviewRows.push({
          variant: null,
          size: null,
          stock: product.currentStock || 0,
          qtyIn: qty,
          cost: 0,
          sell: 0
        });
    }
    return /* @__PURE__ */ React.createElement(View, { style: s.stepBlock }, /* @__PURE__ */
React.createElement(Text, { style: s.stepHeading }, "REVIEW BEFORE SAVING"), /* @__PURE__ */
React.createElement(Text, { style: s.stepSub }, "Double-check everything. Inventory updates immediately."),
/* @__PURE__ */ React.createElement(ScrollView, { style: { maxHeight: 280 }, nestedScrollEnabled: true },
reviewRows.length === 0 ? /* @__PURE__ */ React.createElement(Text, { style: s.emptyReview }, "No items to
review") : reviewRows.map((row, i) => /* @__PURE__ */ React.createElement(View, { key: i, style:
s.reviewRow }, hasVariants && /* @__PURE__ */ React.createElement(Text, { style: s.reviewMain }, row.variant,
row.size && row.size !== VARIANT_ONLY_SIZE_KEY ? ` \xD7 ${row.size}` : ""), !hasVariants && row.size && /*
@__PURE__ */ React.createElement(Text, { style: s.reviewMain }, row.size), !hasVariants && !row.size && /*
@__PURE__ */ React.createElement(Text, { style: s.reviewMain }, "Qty add"), /* @__PURE__ */
React.createElement(Text, { style: s.reviewMeta }, "Stock ", row.stock, " \u2192 +", row.qtyIn, (hasVariants
|| row.cost > 0 || row.sell > 0) && ` \xB7 \u20B1${Number(row.cost).toFixed(2)} /
\u20B1${Number(row.sell).toFixed(2)}`)))));
  };
  return /* @__PURE__ */ React.createElement(
    Modal,
    {
      visible,
      transparent: true,
      animationType: "slide",
      onRequestClose: handleClose,
      supportedOrientations: Platform.OS === "ios" ? ["portrait", "portrait-upside-down"] : void 0
    },
    /* @__PURE__ */ React.createElement(View, { style: s.overlay }, /* @__PURE__ */ React.createElement(View,
{ style: s.card }, /* @__PURE__ */ React.createElement(View, { style: s.header }, /* @__PURE__ */
React.createElement(View, { style: s.headerLeft }, /* @__PURE__ */ React.createElement(View, { style:
s.headerIconWrap }, /* @__PURE__ */ React.createElement(Ionicons, { name: "cube-outline", size: 18, color:
"#111" }), /* @__PURE__ */ React.createElement(View, { style: s.headerBadge }, /* @__PURE__ */
```

```
React.createElement(Text, { style: s.headerBadgeTxt }, "+"))), /* @__PURE__ */ React.createElement(Text,
{ style: s.title }, "Stock In")), /* @__PURE__ */ React.createElement(TouchableOpacity, { onPress:
handleClose, hitSlop: 12 }, /* @__PURE__ */ React.createElement(Text, { style: s.close }, "\xD7"))), /*
@__PURE__ */ React.createElement(View, { style: s.stepper }, [
      { id: 1, label: "Add Items" },
      { id: 2, label: "Details" },
      { id: 3, label: "Review" }
    ].map((st, idx) => /* @__PURE__ */ React.createElement(Fragment, { key: st.id }, /* @__PURE__ */
React.createElement(View, { style: s.stepColumn }, /* @__PURE__ */ React.createElement(
      View,
      {
        style: [
          s.stepCircle,
          currentStep >= st.id && s.stepCircleOn
        ]
      },
      /* @__PURE__ */ React.createElement(
        Text,
        {
          style: [
            s.stepNum,
            currentStep >= st.id && s.stepNumOn
          ]
        },
        currentStep > st.id ? "\u2713" : st.id
      )
    ), /* @__PURE__ */ React.createElement(
      Text,
      {
        style: [
          s.stepLab,
          currentStep >= st.id && s.stepLabOn
        ],
        numberOfLines: 2
      },
      st.label
    )), idx < 2 && /* @__PURE__ */ React.createElement(View, { style: s.stepLineWrap }, /* @__PURE__ */
React.createElement(
      View,
      {
        style: [
          s.stepLine,
          currentStep > st.id && s.stepLineOn
        ]
      }
    )))))), /* @__PURE__ */ React.createElement(
      ScrollView,
      {
        style: s.body,
        keyboardShouldPersistTaps: "handled",
        showsVerticalScrollIndicator: false
      },
      /* @__PURE__ */ React.createElement(Text, { style: s.productLine, numberOfLines: 2 }, displayName),
      currentStep === 1 && renderStep1(),
      currentStep === 2 && renderStep2(),
      currentStep === 3 && renderStep3()
    ), /* @__PURE__ */ React.createElement(View, { style: s.footer }, /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: s.btnCancel,
        onPress: currentStep === 1 ? handleClose : () => setCurrentStep((p) => p - 1)
      },
      /* @__PURE__ */ React.createElement(Text, { style: s.btnCancelTxt }, currentStep === 1 ? "Cancel" :
"\u2190 Back")
    ), currentStep < 3 ? /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [
          s.btnGo,
          !isStepValid(currentStep) && s.btnDisabled
        ],
        disabled: !isStepValid(currentStep),
        onPress: () => {
          if (isStepValid(currentStep))
            setCurrentStep((p) => p + 1);
        }
      },
```

```
      /* @__PURE__ */ React.createElement(Text, { style: s.btnGoTxt }, "Continue")
    ) : /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [s.btnGo, loading && s.btnDisabled],
        disabled: loading,
        onPress: handleSubmit
      },
      loading ? /* @__PURE__ */ React.createElement(ActivityIndicator, { color: "#fff", size: "small" }) : /*
@__PURE__ */ React.createElement(Text, { style: s.btnGoTxt }, "Stock In")
    ))))
  );
};
const s = StyleSheet.create({
  overlay: {
    flex: 1,
    backgroundColor: "rgba(0,0,0,0.45)",
    justifyContent: "center",
    padding: 10
  },
  card: {
    backgroundColor: "#fff",
    borderRadius: 16,
    maxHeight: "92%",
    overflow: "hidden"
  },
  header: {
    flexDirection: "row",
    alignItems: "center",
    justifyContent: "space-between",
    paddingHorizontal: 14,
    paddingVertical: 10,
    borderBottomWidth: 1,
    borderBottomColor: "#eee"
  },
  headerLeft: { flexDirection: "row", alignItems: "center", gap: 8 },
  headerIconWrap: {
    width: 26,
    height: 26,
    justifyContent: "center",
    alignItems: "center"
  },
  headerBadge: {
    position: "absolute",
    top: -2,
    right: -2,
    width: 14,
    height: 14,
    borderRadius: 7,
    backgroundColor: "#09A046",
    borderWidth: 2,
    borderColor: "#fff",
    alignItems: "center",
    justifyContent: "center"
  },
  headerBadgeTxt: {
    color: "#fff",
    fontSize: 8,
    fontWeight: "800",
    marginTop: -1
  },
  title: { fontSize: 17, fontWeight: "700" },
  close: { fontSize: 26, color: "#9ca3af", fontWeight: "300" },
  stepper: {
    flexDirection: "row",
    justifyContent: "center",
    alignItems: "flex-start",
    paddingVertical: 16,
    paddingHorizontal: 10,
    marginBottom: 4,
    borderBottomWidth: 1,
    borderBottomColor: "#f3f4f6"
  },
  stepColumn: {
    alignItems: "center",
    width: 80,
    paddingHorizontal: 4
```

```
},
stepCircle: {
  width: 32,
  height: 32,
  borderRadius: 16,
  borderWidth: 2,
  borderColor: "#d1d5db",
  alignItems: "center",
  justifyContent: "center"
},
stepCircleOn: {
  backgroundColor: "#09A046",
  borderColor: "#09A046"
},
stepNum: { fontSize: 12, fontWeight: "700", color: "#9ca3af" },
stepNumOn: { color: "#fff" },
stepLab: {
  fontSize: 10,
  lineHeight: 13,
  marginTop: 8,
  color: "#9ca3af",
  textAlign: "center",
  width: "100%"
},
stepLabOn: { color: "#09A046", fontWeight: "600" },
stepLineWrap: {
  width: 28,
  minHeight: 32,
  justifyContent: "center",
  alignItems: "center",
  paddingTop: 0
},
stepLine: {
  width: "100%",
  height: 3,
  borderRadius: 2,
  backgroundColor: "#e5e7eb"
},
stepLineOn: { backgroundColor: "#09A046" },
body: { paddingHorizontal: 16, paddingTop: 4, paddingBottom: 10, maxHeight: 440 },
productLine: {
  fontSize: 13,
  fontWeight: "600",
  color: "#374151",
  marginBottom: 8
},
stepBlock: { paddingBottom: 12 },
stepHeading: {
  fontSize: 12,
  fontWeight: "700",
  letterSpacing: 0.5,
  marginBottom: 4
},
stepSub: { fontSize: 11, color: "#9ca3af", marginBottom: 10 },
muted: { fontSize: 11, color: "#6b7280", marginBottom: 8 },
tableBorder: {
  borderWidth: 1,
  borderColor: "#e5e7eb",
  borderRadius: 12,
  marginBottom: 10,
  overflow: "hidden"
},
comboHeaderRow: {
  flexDirection: "row",
  alignItems: "center",
  paddingVertical: 6,
  paddingHorizontal: 6,
  backgroundColor: "#f9fafb",
  borderBottomWidth: 1,
  borderBottomColor: "#e5e7eb",
  gap: 4
},
comboH: { fontSize: 8, fontWeight: "700", color: "#6b7280" },
comboRow: {
  flexDirection: "row",
  alignItems: "center",
  paddingVertical: 6,
```

```
  paddingHorizontal: 6,
  borderBottomWidth: 1,
  borderBottomColor: "#f3f4f6",
  gap: 4
},
cb: {
  width: 18,
  height: 18,
  borderRadius: 4,
  borderWidth: 2,
  borderColor: "#d1d5db",
  alignItems: "center",
  justifyContent: "center"
},
cbOn: { backgroundColor: "#09A046", borderColor: "#09A046" },
cbTick: { color: "#fff", fontSize: 11, fontWeight: "800" },
pillPink: {
  fontSize: 10,
  fontWeight: "600",
  backgroundColor: "#fce7f3",
  color: "#9d174d",
  paddingHorizontal: 6,
  paddingVertical: 3,
  borderRadius: 999,
  overflow: "hidden"
},
pillGrn: {
  fontSize: 10,
  fontWeight: "600",
  backgroundColor: "#d1fae5",
  color: "#065f46",
  paddingHorizontal: 6,
  paddingVertical: 3,
  borderRadius: 999,
  overflow: "hidden"
},
stk: { fontSize: 11, fontWeight: "700", color: "#09A046", textAlign: "center" },
tinyIn: {
  borderWidth: 1,
  borderColor: "#d1d5db",
  borderRadius: 6,
  paddingVertical: 4,
  paddingHorizontal: 4,
  fontSize: 11,
  minHeight: 28,
  color: "#111827"
},
fillRow: {
  flexDirection: "row",
  flexWrap: "wrap",
  alignItems: "center",
  gap: 6,
  padding: 8,
  borderTopWidth: 1,
  borderTopColor: "#e5e7eb",
  backgroundColor: "#fafafa"
},
fillLbl: {
  fontSize: 9,
  fontWeight: "700",
  color: "#09A046",
  marginRight: 4
},
fillBtn: { fontSize: 11, fontWeight: "700", color: "#09A046" },
newLbl: {
  fontSize: 11,
  fontWeight: "700",
  color: "#09A046",
  paddingHorizontal: 8,
  paddingVertical: 4
},
addNewBox: {
  borderWidth: 2,
  borderStyle: "dashed",
  borderColor: "#6ee7b7",
  borderRadius: 12,
  marginTop: 6,
```

```
  overflow: "hidden"
},
addNewHead: {
  flexDirection: "row",
  justifyContent: "space-between",
  padding: 10
},
addNewTitle: { fontSize: 13, fontWeight: "700", color: "#09A046" },
addNewSub: { fontSize: 11, color: "#09A046" },
addNewBody: { paddingHorizontal: 10, paddingBottom: 10 },
twoCol: { gap: 10 },
vCard: {
  borderWidth: 1,
  borderColor: "#e5e7eb",
  borderRadius: 10,
  padding: 8,
  marginBottom: 8
},
vCardTitle: { fontSize: 11, fontWeight: "700", marginBottom: 6 },
optional: { fontWeight: "400", color: "#9ca3af" },
pickerWrap: {
  borderWidth: 1,
  borderColor: "#d1d5db",
  borderRadius: 10,
  overflow: "hidden",
  marginBottom: 6
},
tagRow: { flexDirection: "row", flexWrap: "wrap", gap: 6, alignItems: "center" },
tagPink: {
  flexDirection: "row",
  alignItems: "center",
  gap: 4,
  backgroundColor: "#fce7f3",
  paddingHorizontal: 8,
  paddingVertical: 3,
  borderRadius: 999
},
tagTxt: { fontSize: 11, color: "#9d174d", fontWeight: "600" },
tagX: { fontSize: 14, color: "#831843" },
tagGrn: {
  flexDirection: "row",
  alignItems: "center",
  gap: 4,
  backgroundColor: "#d1fae5",
  paddingHorizontal: 8,
  paddingVertical: 3,
  borderRadius: 999
},
tagTxtGrn: { fontSize: 11, color: "#065f46", fontWeight: "600" },
tagXGrn: { fontSize: 14, color: "#064e3b" },
tagInput: {
  minWidth: 48,
  fontSize: 11,
  borderBottomWidth: 1,
  borderColor: "#d1d5db",
  paddingVertical: 2,
  color: "#111827"
},
previewCount: {
  fontSize: 10,
  fontWeight: "600",
  color: "#6b7280",
  marginVertical: 6
},
previewRow: {
  flexDirection: "row",
  alignItems: "center",
  gap: 6,
  paddingVertical: 6,
  borderBottomWidth: 1,
  borderBottomColor: "#f3f4f6"
},
pillGrnSm: {
  fontSize: 10,
  fontWeight: "600",
  backgroundColor: "#d1fae5",
  color: "#065f46",
```

```
  paddingHorizontal: 6,
  paddingVertical: 2,
  borderRadius: 999
},
pillPinkSm: {
  fontSize: 10,
  fontWeight: "600",
  backgroundColor: "#fce7f3",
  color: "#9d174d",
  paddingHorizontal: 6,
  paddingVertical: 2,
  borderRadius: 999
},
mul: { fontSize: 10, color: "#9ca3af" },
prevIn: {
  width: 56,
  borderWidth: 1,
  borderColor: "#d1d5db",
  borderRadius: 6,
  padding: 4,
  fontSize: 10,
  color: "#111827"
},
addComboBtn: {
  backgroundColor: "#09A046",
  paddingVertical: 10,
  borderRadius: 10,
  alignItems: "center",
  marginTop: 8
},
addComboBtnTxt: { color: "#fff", fontWeight: "700", fontSize: 14 },
sizeChecks: { flexDirection: "row", flexWrap: "wrap", gap: 8 },
sizeCheckRow: { flexDirection: "row", alignItems: "center", gap: 8, width: "48%" },
sizeCheckTxt: { fontSize: 13, color: "#111" },
sizeQty: { fontSize: 11, color: "#6b7280" },
qtyBox: {
  marginTop: 12,
  padding: 10,
  backgroundColor: "#f9fafb",
  borderRadius: 10
},
qtyBoxTitle: { fontSize: 12, fontWeight: "600", marginBottom: 8 },
qtyLine: { flexDirection: "row", alignItems: "center", marginBottom: 8, gap: 10 },
qtyLab: { width: 40, fontSize: 13 },
qtyIn: {
  flex: 1,
  borderWidth: 1,
  borderColor: "#d1d5db",
  borderRadius: 8,
  padding: 8,
  fontSize: 14,
  color: "#111827"
},
input: {
  borderWidth: 1,
  borderColor: "#d1d5db",
  borderRadius: 10,
  padding: 10,
  fontSize: 14,
  color: "#111827"
},
label: {
  fontSize: 10,
  fontWeight: "700",
  color: "#374151",
  marginTop: 10,
  marginBottom: 6,
  letterSpacing: 0.4
},
req: { color: "#ef4444" },
datePress: {
  borderWidth: 1,
  borderColor: "#d1d5db",
  borderRadius: 10,
  padding: 12,
  marginBottom: 4
},
```

```
  datePressTxt: { fontSize: 14, color: "#111827" },
  datePressPlaceholder: { fontSize: 14, color: "#9ca3af" },
  batchBig: { fontSize: 16, fontWeight: "700", marginVertical: 6 },
  emptyReview: {
    textAlign: "center",
    color: "#9ca3af",
    fontStyle: "italic",
    padding: 20,
    fontSize: 12
  },
  reviewRow: {
    paddingVertical: 10,
    borderBottomWidth: 1,
    borderBottomColor: "#f3f4f6"
  },
  reviewMain: { fontSize: 13, fontWeight: "600", color: "#111" },
  reviewMeta: { fontSize: 11, color: "#6b7280", marginTop: 4 },
  footer: {
    flexDirection: "row",
    padding: 12,
    borderTopWidth: 1,
    borderTopColor: "#eee",
    gap: 10
  },
  btnCancel: {
    flex: 1,
    paddingVertical: 11,
    borderRadius: 12,
    borderWidth: 2,
    borderColor: "#d1d5db",
    alignItems: "center"
  },
  btnCancelTxt: { fontSize: 14, fontWeight: "600", color: "#4b5563" },
  btnGo: {
    flex: 1,
    paddingVertical: 11,
    borderRadius: 12,
    backgroundColor: "#09A046",
    alignItems: "center"
  },
  btnGoTxt: { fontSize: 14, fontWeight: "600", color: "#fff" },
  btnDisabled: { opacity: 0.45 }
});
export default StockInModal;
```

FILE: pos/mobile/components/inventory/StockOutModal.jsx
==========

```
import { Ionicons } from "@expo/vector-icons";
import { Picker } from "@react-native-picker/picker";
import { useEffect, useMemo, useState } from "react";
import {
  ActivityIndicator,
  Alert,
  Modal,
  Platform,
  ScrollView,
  StyleSheet,
  Text,
  TextInput,
  TouchableOpacity,
  View
} from "react-native";
import {
  PLACEHOLDER_PH,
  PICKER_ITEM_COLOR,
  nativePickerProps
} from "./nativePickerProps";
const VARIANT_ONLY_SIZE_KEY = "__VARIANT_ONLY__";
const StockOutModal = ({ visible, onClose, product, onConfirm, loading }) => {
  const [selectedBatch, setSelectedBatch] = useState("");
  const [selectedCombos, setSelectedCombos] = useState([]);
  const [comboQuantities, setComboQuantities] = useState({});
  const [quantity, setQuantity] = useState("");
  const [reason, setReason] = useState("");
  const [otherReason, setOtherReason] = useState("");
```

```
  const reasons = [
    "Sold",
    "Damaged",
    "Defective",
    "Returned Item",
    "Lost",
    "Expired",
    "Other"
  ];
  const displayName = product?.itemName || product?.name || "";
  const hasSizes = product?.sizes && typeof product.sizes === "object" && Object.keys(product.sizes).length >
0;
  const hasVariants = useMemo(() => {
    if (!hasSizes) return false;
    return Object.values(product.sizes).some(
      (sd) => typeof sd === "object" && sd?.variants && Object.keys(sd.variants).length > 0
    );
  }, [product, hasSizes]);
  const batchList = useMemo(() => {
    if (!hasSizes || !hasVariants) return [];
    const getVariantQty = (vData) => {
      if (typeof vData === "number") return parseInt(vData, 10) || 0;
      if (vData && typeof vData === "object") {
        const qty = vData.qty ?? vData.quantity ?? 0;
        return parseInt(qty, 10) || 0;
      }
      return 0;
    };
    const maxBatchDepth = ((() => {
      let max = 0;
      Object.entries(product.sizes).forEach(([_, sd]) => {
        if (!sd || typeof sd !== "object" || !sd?.variants) return;
        Object.values(sd.variants).forEach((vData) => {
          const batches = typeof vData === "object" && Array.isArray(vData.batches) ? vData.batches : [];
          max = Math.max(max, batches.length);
        });
      });
      return max;
    })();
    if (maxBatchDepth <= 0) {
      const openingBatchCode = product?.batchNumber || product?.openingBatchNumber || "B1";
      const combos = {};
      Object.entries(product.sizes).forEach(([size, sd]) => {
        if (!sd || typeof sd !== "object" || !sd?.variants) return;
        Object.entries(sd.variants).forEach(([variant, vData]) => {
          const qty = getVariantQty(vData);
          if (qty > 0) combos[`${size}|${variant}`] = { size, variant, qty };
        });
      });
      const comboArr = Object.values(combos);
      const totalQty = comboArr.reduce((sum, c) => sum + (c.qty || 0), 0);
      if (totalQty <= 0) return [];
      return [{ slotIndex: 0, code: openingBatchCode, totalQty, combos }];
    }
    const slots = Array.from({ length: maxBatchDepth }, (_, slotIndex) => ({
      slotIndex,
      code: "",
      totalQty: 0,
      combos: {}
    }));
    Object.entries(product.sizes).forEach(([size, sd]) => {
      if (!sd || typeof sd !== "object" || !sd?.variants) return;
      Object.entries(sd.variants).forEach(([variant, vData]) => {
        const batches = typeof vData === "object" && Array.isArray(vData.batches) ? vData.batches : [];
        const fallbackQty = (slotIndexForFallback) => slotIndexForFallback === 0 ? getVariantQty(vData) : 0;
        for (let slotIndex = 0; slotIndex < maxBatchDepth; slotIndex++) {
          const b = batches[slotIndex];
          const qty = b ? parseInt(b.qty, 10) || 0 : fallbackQty(slotIndex);
          if (qty <= 0) continue;
          const slot = slots[slotIndex];
          const key = `${size}|${variant}`;
          if (!slot.combos[key]) slot.combos[key] = { size, variant, qty: 0 };
          slot.combos[key].qty += qty;
          slot.totalQty += qty;
          if (!slot.code) {
            slot.code = b?.batchCode || (slotIndex === 0 ? product?.batchNumber ||
product?.openingBatchNumber || "B1" : "");
```

```
        }
      }
    });
  });
  return slots.filter((s) => s.totalQty > 0);
}, [product, hasSizes, hasVariants]);
const batchCombos = useMemo(() => {
  if (!selectedBatch) return [];
  const slot = batchList.find((b) => String(b.slotIndex) === selectedBatch);
  if (!slot) return [];
  return Object.values(slot.combos).filter((c) => c.qty > 0);
}, [selectedBatch, batchList]);
useEffect(() => {
  if (visible && product) {
    setSelectedBatch("");
    setSelectedCombos([]);
    setComboQuantities({});
    setQuantity("");
    setReason("");
    setOtherReason("");
  }
}, [visible, product?._id, product?.id]);
if (!visible || !product) return null;
const handleClose = () => {
  setSelectedBatch("");
  setSelectedCombos([]);
  setComboQuantities({});
  setQuantity("");
  setReason("");
  setOtherReason("");
  onClose();
};
const comboKey = (size, variant) => `${size}|${variant}`;
const toggleCombo = (size, variant) => {
  const key = comboKey(size, variant);
  const isSelected = selectedCombos.includes(key);
  if (isSelected) {
    setSelectedCombos((prev) => prev.filter((k) => k !== key));
    setComboQuantities((prev) => {
      const n = { ...prev };
      delete n[key];
      return n;
    });
  } else {
    setSelectedCombos((prev) => [...prev, key]);
  }
};
const handleQtyChange = (key, val) => {
  setComboQuantities((prev) => ({ ...prev, [key]: val }));
};
const isValid = () => {
  if (!reason) return false;
  if (reason === "Other" && !otherReason.trim()) return false;
  if (!hasSizes) return (parseInt(quantity, 10) || 0) > 0;
  if (hasVariants) {
    if (!selectedBatch) return false;
    return selectedCombos.some((key) => (comboQuantities[key] || 0) > 0);
  }
  return (parseInt(quantity, 10) || 0) > 0;
};
const handleSubmit = () => {
  const finalReason = reason === "Other" ? `Other: ${otherReason.trim()}` : reason;
  if (!hasSizes || !hasVariants) {
    const qty = parseInt(quantity, 10) || 0;
    if (qty > (product.currentStock || 0)) {
      Alert.alert(
        "Cannot remove stock",
        `Cannot remove more than available stock (${product.currentStock || 0})`
      );
      return;
    }
    onConfirm({ quantity: qty, noSizes: true, reason: finalReason });
    return;
  }
  const variantQuantities = {};
  const selectedSizesSet = /* @__PURE__ */ new Set();
  const overLimitItems = [];
```

```
    selectedCombos.forEach((key) => {
      const qty = parseInt(comboQuantities[key], 10) || 0;
      if (qty <= 0) return;
      const [size, variant] = key.split("|");
      const combo = batchCombos.find(
        (c) => c.size === size && c.variant === variant
      );
      if (combo && qty > combo.qty) {
        overLimitItems.push(
          `${variant}${size !== VARIANT_ONLY_SIZE_KEY ? ` \xD7 ${size}` : ""} (max: ${combo.qty})`
        );
      }
      if (!variantQuantities[size]) variantQuantities[size] = {};
      variantQuantities[size][variant] = qty;
      selectedSizesSet.add(size);
    });
    if (overLimitItems.length > 0) {
      Alert.alert(
        "Cannot remove stock",
        `Cannot remove more than available stock for:
${overLimitItems.join("\n")}`
      );
      return;
    }
    onConfirm({
      sizes: {},
      variantQuantities,
      selectedSizes: Array.from(selectedSizesSet),
      reason: finalReason,
      hasVariants: true
    });
  };
  return /* @__PURE__ */ React.createElement(
    Modal,
    {
      visible,
      transparent: true,
      animationType: "slide",
      onRequestClose: handleClose,
      supportedOrientations: Platform.OS === "ios" ? ["portrait", "portrait-upside-down"] : void 0
    },
    /* @__PURE__ */ React.createElement(View, { style: styles.overlay }, /* @__PURE__ */
React.createElement(View, { style: styles.card }, /* @__PURE__ */ React.createElement(View, { style:
styles.header }, /* @__PURE__ */ React.createElement(View, { style: styles.headerLeft }, /* @__PURE__ */
React.createElement(View, { style: styles.headerIconWrap }, /* @__PURE__ */ React.createElement(Ionicons,
{ name: "cube-outline", size: 18, color: "#111" }), /* @__PURE__ */ React.createElement(View, { style:
styles.headerBadge }, /* @__PURE__ */ React.createElement(Text, { style: styles.headerBadgeText },
"\u2212"))), /* @__PURE__ */ React.createElement(Text, { style: styles.title }, "Stock Out")), /* @__PURE__
*/ React.createElement(TouchableOpacity, { onPress: handleClose, hitSlop: 12 }, /* @__PURE__ */
React.createElement(Text, { style: styles.close }, "\xD7"))), /* @__PURE__ */ React.createElement(
      ScrollView,
      {
        style: styles.body,
        keyboardShouldPersistTaps: "handled",
        showsVerticalScrollIndicator: false
      },
      /* @__PURE__ */ React.createElement(View, { style: styles.row2 }, /* @__PURE__ */
React.createElement(View, { style: styles.col }, /* @__PURE__ */ React.createElement(Text, { style:
styles.label }, "PRODUCT NAME ", /* @__PURE__ */ React.createElement(Text, { style: styles.req }, "*")), /*
@__PURE__ */ React.createElement(Text, { style: styles.productName }, displayName)), hasVariants &&
batchList.length > 0 && /* @__PURE__ */ React.createElement(View, { style: styles.col }, /* @__PURE__ */
React.createElement(Text, { style: styles.label }, "BATCH NUMBER ", /* @__PURE__ */ React.createElement(Text,
{ style: styles.req }, "*")), /* @__PURE__ */ React.createElement(View, { style: styles.pickerWrap }, /*
@__PURE__ */ React.createElement(
        Picker,
        {
          ...nativePickerProps,
          selectedValue: selectedBatch,
          onValueChange: (v) => {
            setSelectedBatch(v);
            setSelectedCombos([]);
            setComboQuantities({});
          }
        },
        /* @__PURE__ */ React.createElement(
          Picker.Item,
          {
```

```
            label: "Select Batch",
            value: "",
            color: PICKER_ITEM_COLOR
          }
        ),
        batchList.map((b) => /* @__PURE__ */ React.createElement(
          Picker.Item,
          {
            key: b.slotIndex,
            label: `Batch ${b.slotIndex + 1}${b.code ? ` (${b.code})` : ""} (${b.totalQty})`,
            value: String(b.slotIndex),
            color: PICKER_ITEM_COLOR
          }
        ))
      )))),
      hasVariants && selectedBatch && batchCombos.length > 0 && /* @__PURE__ */ React.createElement(View,
{ style: styles.chipSection }, /* @__PURE__ */ React.createElement(Text, { style: styles.chipSectionTitle },
"SELECT VARIANTS TO REMOVE"), /* @__PURE__ */ React.createElement(View, { style: styles.chipRow },
batchCombos.map((c) => {
        const key = comboKey(c.size, c.variant);
        const isSel = selectedCombos.includes(key);
        const label = c.size !== VARIANT_ONLY_SIZE_KEY ? `${c.variant} x ${c.size}` : c.variant;
        return /* @__PURE__ */ React.createElement(
          TouchableOpacity,
          {
            key,
            style: [styles.chip, isSel && styles.chipSel],
            onPress: () => toggleCombo(c.size, c.variant)
          },
          /* @__PURE__ */ React.createElement(Text, { style: [styles.chipTxt, isSel && styles.chipTxtSel] },
label)
        );
      })),
      hasVariants && selectedCombos.length > 0 && /* @__PURE__ */ React.createElement(View, { style:
styles.tableWrap }, /* @__PURE__ */ React.createElement(View, { style: styles.tableHead }, /* @__PURE__ */
React.createElement(Text, { style: [styles.th, { flex: 1.4 }] }, "VARIANT"), /* @__PURE__ */
React.createElement(Text, { style: [styles.th, { flex: 0.5, textAlign: "center" }] }, "STOCK"), /* @__PURE__
*/ React.createElement(Text, { style: [styles.th, { flex: 0.55, textAlign: "center" }] }, "QTY OUT")), /*
@__PURE__ */ React.createElement(ScrollView, { style: styles.tableBody, nestedScrollEnabled: true },
selectedCombos.map((key) => {
        const combo = batchCombos.find(
          (c) => comboKey(c.size, c.variant) === key
        );
        if (!combo) return null;
        const [size, variant] = key.split("|");
        const maxQty = combo.qty;
        const val = comboQuantities[key] || "";
        return /* @__PURE__ */ React.createElement(View, { key, style: styles.tr }, /* @__PURE__ */
React.createElement(View, { style: [styles.td, { flex: 1.4, flexDirection: "row", flexWrap: "wrap",
alignItems: "center", gap: 4 }] }, size !== VARIANT_ONLY_SIZE_KEY && /* @__PURE__ */ React.createElement(Text,
{ style: styles.badgeSize }, size), size !== VARIANT_ONLY_SIZE_KEY && /* @__PURE__ */
React.createElement(Text, { style: styles.mul }, "\xD7"), /* @__PURE__ */ React.createElement(Text, { style:
styles.badgeVar }, variant)), /* @__PURE__ */ React.createElement(Text, { style: [styles.td, { flex: 0.5,
textAlign: "center" }] }, maxQty), /* @__PURE__ */ React.createElement(View, { style: [styles.td, { flex:
0.55 }] }, /* @__PURE__ */ React.createElement(
          TextInput,
          {
            style: styles.qtyIn,
            keyboardType: "number-pad",
            value: val,
            onChangeText: (t) => handleQtyChange(key, t),
            placeholderTextColor: PLACEHOLDER_PH,
            placeholder: "Qty out",
            underlineColorAndroid: "transparent"
          }
        )));
      })),
      !hasVariants && /* @__PURE__ */ React.createElement(View, { style: styles.block }, /* @__PURE__ */
React.createElement(Text, { style: styles.label }, "QUANTITY TO REMOVE"), /* @__PURE__ */
React.createElement(Text, { style: styles.hint }, "Current Stock: ", product.currentStock || 0), /* @__PURE__
*/ React.createElement(
        TextInput,
        {
          style: styles.input,
          keyboardType: "number-pad",
          value: quantity,
          onChangeText: setQuantity,
```

```
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "Enter quantity",
          underlineColorAndroid: "transparent"
        }
      )),
      /* @__PURE__ */ React.createElement(View, { style: styles.block }, /* @__PURE__ */
React.createElement(Text, { style: styles.label }, "REASON ", /* @__PURE__ */ React.createElement(Text,
{ style: styles.req }, "*")), /* @__PURE__ */ React.createElement(View, { style: styles.pickerWrap }, /*
@__PURE__ */ React.createElement(
        Picker,
        {
          ...nativePickerProps,
          selectedValue: reason,
          onValueChange: (v) => {
            setReason(v);
            if (v !== "Other") setOtherReason("");
          }
        },
        /* @__PURE__ */ React.createElement(
          Picker.Item,
          {
            label: "Select reason",
            value: "",
            color: PICKER_ITEM_COLOR
          }
        ),
        reasons.map((r) => /* @__PURE__ */ React.createElement(
          Picker.Item,
          {
            key: r,
            label: r,
            value: r,
            color: PICKER_ITEM_COLOR
          }
        ))
      )), reason === "Other" && /* @__PURE__ */ React.createElement(
        TextInput,
        {
          style: [styles.input, { marginTop: 8 }],
          value: otherReason,
          onChangeText: setOtherReason,
          placeholderTextColor: PLACEHOLDER_PH,
          placeholder: "Please specify the reason",
          underlineColorAndroid: "transparent"
        }
      ))
    ), /* @__PURE__ */ React.createElement(View, { style: styles.footer }, /* @__PURE__ */
React.createElement(TouchableOpacity, { style: styles.btnCancel, onPress: handleClose }, /* @__PURE__ */
React.createElement(Text, { style: styles.btnCancelTxt }, "Cancel")), /* @__PURE__ */ React.createElement(
      TouchableOpacity,
      {
        style: [styles.btnGo, (!isValid() || loading) && styles.btnDisabled],
        disabled: loading || !isValid(),
        onPress: handleSubmit
      },
      loading ? /* @__PURE__ */ React.createElement(ActivityIndicator, { color: "#fff", size: "small" }) : /*
@__PURE__ */ React.createElement(Text, { style: styles.btnGoTxt }, "Remove")
    ))))
  );
};
const styles = StyleSheet.create({
  overlay: {
    flex: 1,
    backgroundColor: "rgba(0,0,0,0.45)",
    justifyContent: "center",
    padding: 14
  },
  card: {
    backgroundColor: "#fff",
    borderRadius: 16,
    maxHeight: "88%",
    overflow: "hidden",
    elevation: 8
  },
  header: {
    flexDirection: "row",
    alignItems: "center",
```

```
  justifyContent: "space-between",
  paddingHorizontal: 16,
  paddingVertical: 12,
  borderBottomWidth: 1,
  borderBottomColor: "#eee"
},
headerLeft: { flexDirection: "row", alignItems: "center", gap: 10 },
headerIconWrap: {
  width: 28,
  height: 28,
  justifyContent: "center",
  alignItems: "center"
},
headerBadge: {
  position: "absolute",
  top: -2,
  right: -2,
  width: 14,
  height: 14,
  borderRadius: 7,
  backgroundColor: "#ef4444",
  borderWidth: 2,
  borderColor: "#fff",
  alignItems: "center",
  justifyContent: "center"
},
headerBadgeText: {
  color: "#fff",
  fontSize: 8,
  fontWeight: "800",
  marginTop: -1
},
title: { fontSize: 17, fontWeight: "700", color: "#111" },
close: { fontSize: 26, color: "#9ca3af", fontWeight: "300" },
body: { paddingHorizontal: 16, paddingTop: 12, maxHeight: 420 },
row2: { flexDirection: "row", gap: 12, marginBottom: 14 },
col: { flex: 1 },
label: {
  fontSize: 10,
  fontWeight: "700",
  letterSpacing: 0.6,
  color: "#374151",
  marginBottom: 6
},
req: { color: "#ef4444" },
productName: { fontSize: 15, fontWeight: "700", color: "#111" },
pickerWrap: {
  borderWidth: 1,
  borderColor: "#d1d5db",
  borderRadius: 10,
  overflow: "hidden",
  backgroundColor: "#fff"
},
chipSection: {
  borderWidth: 1,
  borderColor: "#e5e7eb",
  borderRadius: 12,
  padding: 12,
  backgroundColor: "#f9fafb",
  marginBottom: 12
},
chipSectionTitle: {
  fontSize: 10,
  fontWeight: "700",
  color: "#6b7280",
  marginBottom: 10,
  letterSpacing: 0.5
},
chipRow: { flexDirection: "row", flexWrap: "wrap", gap: 8 },
chip: {
  paddingHorizontal: 10,
  paddingVertical: 6,
  borderRadius: 999,
  borderWidth: 1,
  borderColor: "#d1d5db",
  backgroundColor: "#fff"
},
```

```
chipSel: {
  backgroundColor: "#fef2f2",
  borderColor: "#f87171"
},
chipTxt: { fontSize: 11, fontWeight: "600", color: "#4b5563" },
chipTxtSel: { color: "#b91c1c" },
tableWrap: {
  borderWidth: 1,
  borderColor: "#e5e7eb",
  borderRadius: 12,
  marginBottom: 12,
  overflow: "hidden"
},
tableHead: {
  flexDirection: "row",
  paddingVertical: 8,
  paddingHorizontal: 10,
  backgroundColor: "#f9fafb",
  borderBottomWidth: 1,
  borderBottomColor: "#e5e7eb"
},
th: {
  fontSize: 10,
  fontWeight: "700",
  color: "#6b7280",
  letterSpacing: 0.4
},
tableBody: { maxHeight: 200 },
tr: {
  flexDirection: "row",
  alignItems: "center",
  paddingVertical: 10,
  paddingHorizontal: 10,
  borderBottomWidth: 1,
  borderBottomColor: "#f3f4f6"
},
td: { fontSize: 12, color: "#374151" },
badgeSize: {
  fontSize: 10,
  fontWeight: "600",
  paddingHorizontal: 8,
  paddingVertical: 3,
  borderRadius: 999,
  borderWidth: 1,
  borderStyle: "dashed",
  borderColor: "#93c5fd",
  color: "#1d4ed8",
  overflow: "hidden"
},
mul: { fontSize: 9, color: "#9ca3af" },
badgeVar: {
  fontSize: 10,
  fontWeight: "600",
  paddingHorizontal: 8,
  paddingVertical: 3,
  borderRadius: 999,
  borderWidth: 1,
  borderStyle: "dashed",
  borderColor: "#f9a8d4",
  color: "#be185d",
  overflow: "hidden"
},
qtyIn: {
  borderWidth: 1,
  borderColor: "#d1d5db",
  borderRadius: 8,
  paddingVertical: 6,
  paddingHorizontal: 6,
  fontSize: 13,
  textAlign: "center",
  backgroundColor: "#fff",
  color: "#111827"
},
block: { marginBottom: 14 },
hint: { fontSize: 11, color: "#9ca3af", marginBottom: 6 },
input: {
  borderWidth: 1,
```

```
    borderColor: "#d1d5db",
    borderRadius: 10,
    paddingHorizontal: 12,
    paddingVertical: 10,
    fontSize: 14,
    backgroundColor: "#fff",
    color: "#111827"
  },
  footer: {
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    padding: 14,
    borderTopWidth: 1,
    borderTopColor: "#eee",
    gap: 12
  },
  btnCancel: {
    flex: 1,
    paddingVertical: 11,
    borderRadius: 12,
    borderWidth: 2,
    borderColor: "#d1d5db",
    alignItems: "center"
  },
  btnCancelTxt: { fontSize: 14, fontWeight: "600", color: "#4b5563" },
  btnGo: {
    flex: 1,
    paddingVertical: 11,
    borderRadius: 12,
    backgroundColor: "#dc2626",
    alignItems: "center"
  },
  btnDisabled: { opacity: 0.45 },
  btnGoTxt: { fontSize: 14, fontWeight: "600", color: "#fff" }
});
export default StockOutModal;
```

FILE: pos/mobile/components/shared/header.jsx
==========

```
import { Ionicons } from "@expo/vector-icons";
import { useFocusEffect } from "@react-navigation/native";
import { usePathname, useRouter } from "expo-router";
import { useCallback, useState } from "react";
import { StyleSheet, Text, TouchableOpacity, View } from "react-native";
import { stockAPI } from "../../services/api";
import { checkAndNotify } from "../../services/notificationService";
const Header = () => {
  const router = useRouter();
  const pathname = usePathname();
  const [notificationCount, setNotificationCount] = useState(0);
  const getPageTitle = (path) => {
    switch (path) {
      case "/":
      case "/index":
        return "Dashboard";
      case "/transaction":
        return "Transactions";
      case "/analytics":
        return "Analytics";
      case "/inventory":
        return "Inventory";
      case "/profile":
        return "Profile";
      case "/Notification":
        return "Notifications";
      default:
        return "Create your style";
    }
  };
  useFocusEffect(
    useCallback(() => {
      const fetchNotifications = async () => {
        try {
          const response = await stockAPI.getLowStock();
```

```
          if (response && response.success) {
            setNotificationCount(response.count || 0);
          }
          await checkAndNotify();
        } catch (error) {
          console.error("Error fetching notifications:", error);
        }
      };
      fetchNotifications();
    }, [])
  );
  return /* @__PURE__ */ React.createElement(View, { style: styles.container }, /* @__PURE__ */
React.createElement(View, { style: styles.headerBackground }, /* @__PURE__ */ React.createElement(Text,
{ style: styles.headerTitle }, getPageTitle(pathname)), /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: styles.notificationButton,
      onPress: () => router.push("/Notification")
    },
    /* @__PURE__ */ React.createElement(Ionicons, { name: "notifications-outline", size: 25, color:
"#1a1a1a" }),
    notificationCount > 0 && /* @__PURE__ */ React.createElement(View, { style: styles.notificationDot }, /*
@__PURE__ */ React.createElement(Text, { style: styles.notificationText }, notificationCount > 9 ? "9+" :
notificationCount))
  )));
};
const styles = StyleSheet.create({
  container: {
    zIndex: 10
  },
  headerBackground: {
    height: 100,
    backgroundColor: "#fff",
    flexDirection: "row",
    justifyContent: "space-between",
    alignItems: "center",
    paddingHorizontal: 20,
    paddingTop: 40
  },
  headerTitle: {
    marginTop: -1,
    fontSize: 22,
    color: "#1a1a1a",
    fontFamily: "Poppins"
  },
  notificationButton: {
    position: "relative",
    bottom: 3,
    padding: 8,
    zIndex: 10
  },
  notificationDot: {
    position: "absolute",
    top: 4,
    right: 4,
    minWidth: 16,
    height: 16,
    backgroundColor: "#FF3B30",
    borderRadius: 8,
    justifyContent: "center",
    alignItems: "center",
    paddingHorizontal: 3,
    borderWidth: 1.5,
    borderColor: "#AD7F65"
  },
  notificationText: {
    color: "white",
    fontSize: 10,
    fontWeight: "bold"
  }
});
export default Header;
```

FILE: pos/mobile/components/themed-text.tsx
==========

```
import { StyleSheet, Text } from "react-native";
import { useThemeColor } from "@/hooks/use-theme-color";
export function ThemedText({
  style,
  lightColor,
  darkColor,
  type = "default",
  ...rest
}) {
  const color = useThemeColor({ light: lightColor, dark: darkColor }, "text");
  return /* @__PURE__ */ React.createElement(
    Text,
    {
      style: [
        { color },
        type === "default" ? styles.default : void 0,
        type === "title" ? styles.title : void 0,
        type === "defaultSemiBold" ? styles.defaultSemiBold : void 0,
        type === "subtitle" ? styles.subtitle : void 0,
        type === "link" ? styles.link : void 0,
        style
      ],
      ...rest
    }
  );
}
const styles = StyleSheet.create({
  default: {
    fontSize: 16,
    lineHeight: 24
  },
  defaultSemiBold: {
    fontSize: 16,
    lineHeight: 24,
    fontWeight: "600"
  },
  title: {
    fontSize: 32,
    fontWeight: "bold",
    lineHeight: 32
  },
  subtitle: {
    fontSize: 20,
    fontWeight: "bold"
  },
  link: {
    lineHeight: 30,
    fontSize: 16,
    color: "#0a7ea4"
  }
});
```

FILE: pos/mobile/components/themed-view.tsx
==========

```
import { View } from "react-native";
import { useThemeColor } from "@/hooks/use-theme-color";
export function ThemedView({ style, lightColor, darkColor, ...otherProps }) {
  const backgroundColor = useThemeColor({ light: lightColor, dark: darkColor }, "background");
  return /* @__PURE__ */ React.createElement(View, { style: [{ backgroundColor }, style], ...otherProps });
}
```

FILE: pos/mobile/components/ui/collapsible.tsx
==========

```
import { useState } from "react";
import { StyleSheet, TouchableOpacity } from "react-native";
import { ThemedText } from "@/components/themed-text";
import { ThemedView } from "@/components/themed-view";
import { IconSymbol } from "@/components/ui/icon-symbol";
import { Colors } from "@/constants/theme";
import { useColorScheme } from "@/hooks/use-color-scheme";
export function Collapsible({ children, title }) {
  const [isOpen, setIsOpen] = useState(false);
  const theme = useColorScheme() ?? "light";
```

```
  return /* @__PURE__ */ React.createElement(ThemedView, null, /* @__PURE__ */ React.createElement(
    TouchableOpacity,
    {
      style: styles.heading,
      onPress: () => setIsOpen((value) => !value),
      activeOpacity: 0.8
    },
    /* @__PURE__ */ React.createElement(
      IconSymbol,
      {
        name: "chevron.right",
        size: 18,
        weight: "medium",
        color: theme === "light" ? Colors.light.icon : Colors.dark.icon,
        style: { transform: [{ rotate: isOpen ? "90deg" : "0deg" }] }
      }
    ),
    /* @__PURE__ */ React.createElement(ThemedText, { type: "defaultSemiBold" }, title)
  ), isOpen && /* @__PURE__ */ React.createElement(ThemedView, { style: styles.content }, children));
}
const styles = StyleSheet.create({
  heading: {
    flexDirection: "row",
    alignItems: "center",
    gap: 6
  },
  content: {
    marginTop: 6,
    marginLeft: 24
  }
});
```

FILE: pos/mobile/components/ui/icon-symbol.ios.tsx
==========

```
import { SymbolView } from "expo-symbols";
export function IconSymbol({
  name,
  size = 24,
  color,
  style,
  weight = "regular"
}) {
  return /* @__PURE__ */ React.createElement(
    SymbolView,
    {
      weight,
      tintColor: color,
      resizeMode: "scaleAspectFit",
      name,
      style: [
        {
          width: size,
          height: size
        },
        style
      ]
    }
  );
}
```

FILE: pos/mobile/components/ui/icon-symbol.tsx
==========

```
import MaterialIcons from "@expo/vector-icons/MaterialIcons";
const MAPPING = {
  "house.fill": "home",
  "paperplane.fill": "send",
  "chevron.left.forwardslash.chevron.right": "code",
  "chevron.right": "chevron-right"
};
export function IconSymbol({
  name,
  size = 24,
  color,
```

```
  style
}) {
  return /* @__PURE__ */ React.createElement(
    MaterialIcons,
    {
      color,
      size,
      name: MAPPING[name],
      style
    }
  );
}
```

FILE: pos/mobile/config/api.js
==========

```
const PRODUCTION_URL = "https://final-defense-w44w.onrender.com";
const LOCAL_IP = "192.168.56.1";
const LOCALHOST = "localhost";
const PORT = 5e3;
const TIMEOUT = 3e4;
const normalizeBaseUrl = (url) => (url || "").replace(/\/+$/, "");
export const createApiConfig = (host = LOCAL_IP, port = PORT, protocol = "http") => {
  const baseUrl = normalizeBaseUrl(`${protocol}://${host}:${port}`);
  return {
    BASE_URL: baseUrl,
    API_URL: `${baseUrl}/api`,
    IP: host,
    PORT: port,
    TIMEOUT
  };
};
const getDevBaseUrl = () => {
  const envBaseUrl = normalizeBaseUrl(process.env.EXPO_PUBLIC_API_URL);
  if (envBaseUrl) return envBaseUrl;
  if (typeof window !== "undefined") {
    const host = window.location?.hostname || LOCALHOST;
    return `http://${host}:${PORT}`;
  }
  return `http://${LOCAL_IP}:${PORT}`;
};
const API_BASE_URL = normalizeBaseUrl(__DEV__ ? getDevBaseUrl() : PRODUCTION_URL);
export const API_URL = `${API_BASE_URL}/api`;
export default {
  BASE_URL: API_BASE_URL,
  API_URL,
  IP: LOCAL_IP,
  PORT,
  TIMEOUT
};
```

FILE: pos/mobile/constants/addProductWebParity.js
==========

```
export const VARIANT_ONLY_KEY = "__VARIANT_ONLY__";
export const COMMON_COLORS = [
  "Black",
  "White",
  "Red",
  "Blue",
  "Navy Blue",
  "Green",
  "Yellow",
  "Orange",
  "Pink",
  "Purple",
  "Brown",
  "Gray",
  "Beige",
  "Cream",
  "Maroon",
  "Olive",
  "Teal",
  "Coral",
  "Lavender",
```

```
  "Mint",
  "Gold",
  "Silver",
  "Rose Gold",
  "Custom"
];
export const categoryStructure = {
  "Apparel - Men": ["Tops", "Bottoms", "Outerwear"],
  "Apparel - Women": ["Tops", "Bottoms", "Dresses", "Outerwear"],
  "Apparel - Kids": ["Tops", "Bottoms", "Dresses", "Outerwear"],
  "Apparel - Unisex": ["Tops", "Bottoms", "Dresses", "Outerwear"],
  Foods: ["Beverages", "Snacks", "Meals", "Desserts", "Ingredients", "Other"],
  Makeup: ["Face", "Eyes", "Lips", "Nails", "SkinCare", "Others"],
  Accessories: ["Jewelry", "Bags", "Head Wear", "Glasses/Sunglasses", "Others"],
  Shoes: ["Sneakers", "Boots", "Sandals", "Others"],
  Others: ["Others"]
};
export const parentCategories = Object.keys(categoryStructure);
const legacyParentCategories = [
  "Apparel",
  "Shoes",
  "Foods",
  "Accessories",
  "Makeup",
  "Head Wear"
];
export const UNIT_OPTIONS = [
  { value: "pcs", label: "Pieces (pcs)" },
  { value: "kg", label: "Kilograms (kg)" },
  { value: "g", label: "Grams (g)" },
  { value: "L", label: "Liters (L)" },
  { value: "ml", label: "Milliliters (ml)" },
  { value: "mg", label: "Milligrams (mg)" },
  { value: "packs", label: "Packs" },
  { value: "boxes", label: "Boxes" }
];
export const categoryCodeMap = {
  Tops: "TOP",
  Bottoms: "BOT",
  Dresses: "DRS",
  Makeup: "MUA",
  Accessories: "MUA",
  Shoes: "SHO",
  "Head Wear": "HDW",
  Foods: "FOD"
};
export const sumAllVariantQuantities = (variantQuantities) => {
  if (!variantQuantities || typeof variantQuantities !== "object") return 0;
  return Object.values(variantQuantities).reduce((sum, perSize) => {
    if (!perSize || typeof perSize !== "object") return sum;
    return sum + Object.values(perSize).reduce((s, q) => s + (parseInt(q, 10) || 0), 0);
  }, 0);
};
export function getCustomSubCategories(apiCategoryNames = []) {
  const allKnownDefaultSubs = new Set(Object.values(categoryStructure).flat());
  return apiCategoryNames.filter(
    (name) => name && name !== "All" && name !== "Others" && !parentCategories.includes(name)
&& !allKnownDefaultSubs.has(name) && !legacyParentCategories.includes(name)
  );
}
export function getSubcategories(parentCat, customSubs, currentSub, currentParent) {
  const defaultSubs = categoryStructure[parentCat] || [];
  const subs = [...defaultSubs, ...customSubs];
  if (currentSub && currentSub !== "__add_new__" && !subs.includes(currentSub) && currentParent === parentCat)
{
    subs.push(currentSub);
  }
  return [...new Set(subs)];
}
export function getSizeOptions(parentCategory, subCategory, customSizes = []) {
  const category = parentCategory;
  const sub = subCategory || "";
  let sizes = [];
  const parentHasSizes = categoryStructure[category] !== void 0;
  if (!parentHasSizes) {
    sizes = ["Free Size"];
  } else if (category === "Foods") {
```

```
    switch (sub) {
      case "Beverages":
        sizes = ["Small", "Medium", "Large", "Extra Large", "Free Size"];
        break;
      case "Snacks":
        sizes = [
          "Small Pack",
          "Medium Pack",
          "Large Pack",
          "Family Pack",
          "Free Size"
        ];
        break;
      case "Meals":
        sizes = ["Regular", "Large", "Family Size", "Free Size"];
        break;
      case "Desserts":
        sizes = ["Small", "Medium", "Large", "Free Size"];
        break;
      case "Ingredients":
        sizes = ["100g", "250g", "500g", "1kg", "Free Size"];
        break;
      default:
        sizes = ["Small", "Medium", "Large", "Free Size"];
    }
  } else if (["Tops", "Bottoms", "Dresses", "Outerwear"].includes(sub)) {
    sizes = ["XS", "S", "M", "L", "XL", "XXL", "Free Size"];
  } else if (category === "Shoes") {
    sizes = ["5", "6", "7", "8", "9", "10", "11", "12"];
  } else if (category === "Accessories" || category === "Makeup") {
    sizes = ["Free Size"];
  } else {
    sizes = ["Free Size"];
  }
  return [...sizes, ...customSizes];
}
export function getColorCode(variant) {
  if (!variant || String(variant).trim() === "") {
    return "XXX";
  }
  const cleaned = String(variant).replace(/\s+/g, "").toUpperCase();
  return cleaned.substring(0, 3).padEnd(3, "X");
}
export function generateSKU(category, subCategory, variant) {
  const base = subCategory || category;
  const categoryCode = categoryCodeMap[base] || "OTH";
  const chars = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
  let randomCode = "";
  for (let i = 0; i < 6; i++) {
    randomCode += chars.charAt(Math.floor(Math.random() * chars.length));
  }
  if (!variant || String(variant).trim() === "") {
    return `${categoryCode}-${randomCode}`;
  }
  const colorCode = getColorCode(variant);
  return `${categoryCode}-${randomCode}-${colorCode}`;
}
export function inferParentSubFromProduct(item) {
  const sub = item?.subCategory || "";
  const cat = item?.category || "";
  if (cat && sub) {
    return { category: cat, subCategory: sub };
  }
  if (!cat && !sub) {
    return { category: "", subCategory: "" };
  }
  const only = cat || sub;
  for (const parent of parentCategories) {
    const subs = categoryStructure[parent] || [];
    if (subs.includes(only)) {
      return { category: parent, subCategory: only };
    }
    if (parent === only && subs.length) {
      return { category: parent, subCategory: subs[0] };
    }
  }
  return { category: "Others", subCategory: only || "Others" };
```

```
}
export function buildCreateProductPayload(np, editingProduct = false) {
  const variantQtySum = sumAllVariantQuantities(np.variantQuantities);
  const sizeQtySum = Object.values(np.sizeQuantities || {}).reduce(
    (sum, qty) => sum + (parseInt(qty, 10) || 0),
    0
  );
  const totalStock = variantQtySum > 0 ? variantQtySum : (np.selectedSizes || []).length > 0 ? sizeQtySum :
parseInt(np.currentStock, 10) || 0;
  let defaultItemPrice = parseFloat(np.itemPrice) || 0;
  if (!defaultItemPrice && np.differentPricesPerSize && np.selectedSizes?.length > 0) {
    const firstSizePrice = np.sizePrices?.[np.selectedSizes[0]];
    if (firstSizePrice) {
      defaultItemPrice = parseFloat(firstSizePrice) || 0;
    }
  }
  if (!defaultItemPrice && np.variantPrices) {
    const firstSizeWithVariantPrices = Object.keys(np.variantPrices)[0];
    if (firstSizeWithVariantPrices) {
      const firstVariantPrice = Object.values(
        np.variantPrices[firstSizeWithVariantPrices]
      )[0];
      if (firstVariantPrice) {
        defaultItemPrice = parseFloat(firstVariantPrice) || 0;
      }
    }
  }
  const payload = {
    ...np,
    itemPrice: defaultItemPrice,
    costPrice: parseFloat(np.costPrice) || 0,
    reorderNumber: parseInt(np.reorderNumber, 10) || 0,
    expirationDate: np.expirationDate || null,
    displayInTerminal: np.displayInTerminal !== false
  };
  if (!editingProduct) {
    payload.currentStock = totalStock;
    const selected = np.selectedSizes || [];
    const sizesList = selected.length > 0 ? selected : Object.keys(np.variantQuantities || {}).length > 0 ?
Object.keys(np.variantQuantities) : [];
    if (sizesList.length > 0) {
      const hasVariantQuantities = np.variantQuantities && Object.keys(np.variantQuantities).length > 0;
      const hasVariantPrices = np.variantPrices && Object.keys(np.variantPrices).length > 0;
      if (np.differentPricesPerSize && Object.keys(np.sizePrices || {}).length > 0) {
        const sizesWithPrices = {};
        sizesList.forEach((size) => {
          const sizePrice = parseFloat(np.sizePrices[size]);
          let variantValue = "";
          if (np.differentVariantsPerSize) {
            if (np.multipleVariantsPerSize?.[size]) {
              variantValue = np.sizeMultiVariants?.[size] || [];
            } else {
              variantValue = np.sizeVariants?.[size] || "";
            }
          } else {
            variantValue = np.variant || "";
          }
          if (hasVariantQuantities && np.variantQuantities[size]) {
            sizesWithPrices[size] = {
              quantity: Object.values(np.variantQuantities[size]).reduce(
                (sum, q) => sum + (parseInt(q, 10) || 0),
                0
              ),
              price: sizePrice || defaultItemPrice || 0,
              variant: variantValue,
              variants: np.variantQuantities[size]
            };
            if (hasVariantPrices && np.variantPrices[size]) {
              sizesWithPrices[size].variantPrices = np.variantPrices[size];
            }
          } else {
            sizesWithPrices[size] = {
              quantity: np.sizeQuantities[size] || 0,
              price: sizePrice || defaultItemPrice || 0,
              variant: variantValue
            };
          }
```

```
        });
        payload.sizes = sizesWithPrices;
      } else {
        const sizesObject = {};
        const hasVariantCostPrices = np.variantCostPrices && Object.keys(np.variantCostPrices).length > 0;
        sizesList.forEach((size) => {
          let variantValue = "";
          if (np.differentVariantsPerSize) {
            if (np.multipleVariantsPerSize?.[size]) {
              variantValue = np.sizeMultiVariants?.[size] || [];
            } else {
              variantValue = np.sizeVariants?.[size] || "";
            }
          } else {
            variantValue = np.variant || "";
          }
          const sizePrice = parseFloat(np.sizePrices?.[size]) || 0;
          const sizeCostPrice = parseFloat(np.sizeCostPrices?.[size]) || 0;
          if (hasVariantQuantities && np.variantQuantities[size]) {
            sizesObject[size] = {
              quantity: Object.values(np.variantQuantities[size]).reduce(
                (sum, q) => sum + (parseInt(q, 10) || 0),
                0
              ),
              variant: variantValue,
              variants: np.variantQuantities[size]
            };
            if (hasVariantPrices && np.variantPrices[size]) {
              sizesObject[size].variantPrices = np.variantPrices[size];
              if (hasVariantCostPrices && np.variantCostPrices[size]) {
                sizesObject[size].variantCostPrices = np.variantCostPrices[size];
              }
            } else {
              if (sizePrice > 0) {
                sizesObject[size].price = sizePrice;
              }
              if (sizeCostPrice > 0) {
                sizesObject[size].costPrice = sizeCostPrice;
              }
            }
          } else {
            sizesObject[size] = {
              quantity: np.sizeQuantities[size] || 0,
              variant: variantValue
            };
            if (sizePrice > 0) {
              sizesObject[size].price = sizePrice;
            }
            if (sizeCostPrice > 0) {
              sizesObject[size].costPrice = sizeCostPrice;
            }
          }
        });
        payload.sizes = sizesObject;
      }
    } else {
      payload.sizes = null;
    }
  }
  return payload;
}
export function buildEditProductPayload(newProduct, editingProduct) {
  const displayInTerminalValue = newProduct.displayInTerminal === false ? false : true;
  const payload = {
    ...newProduct,
    itemPrice: parseFloat(newProduct.itemPrice) || 0,
    costPrice: parseFloat(newProduct.costPrice) || 0,
    reorderNumber: parseInt(newProduct.reorderNumber, 10) || 0,
    expirationDate: newProduct.expirationDate || null,
    displayInTerminal: displayInTerminalValue
  };
  delete payload.currentStock;
  delete payload.selectedSizes;
  delete payload.sizeQuantities;
  delete payload.sizePrices;
  delete payload.differentPricesPerSize;
  if (payload.editableSizePrices && Object.keys(payload.editableSizePrices).length > 0) {
```

```
    const updatedSizes = { ...editingProduct.sizes };
    Object.entries(payload.editableSizePrices).forEach(([size, sizeData]) => {
      if (updatedSizes[size]) {
        if (sizeData.hasVariants && sizeData.variants) {
          const currentSizeData = updatedSizes[size];
          const currentVariants = currentSizeData.variants || {};
          const currentVariantPrices = currentSizeData.variantPrices || {};
          const currentVariantCostPrices = currentSizeData.variantCostPrices || {};
          Object.entries(sizeData.variants).forEach(([variant, variantData]) => {
            const newPrice = parseFloat(variantData.price) || 0;
            const newCostPrice = parseFloat(variantData.costPrice) || 0;
            if (currentVariants[variant] !== void 0) {
              if (typeof currentVariants[variant] === "number") {
                currentVariantPrices[variant] = newPrice;
                currentVariantCostPrices[variant] = newCostPrice;
              } else if (typeof currentVariants[variant] === "object") {
                currentVariants[variant] = {
                  ...currentVariants[variant],
                  price: newPrice,
                  costPrice: newCostPrice
                };
              }
            }
          });
          updatedSizes[size] = {
            ...currentSizeData,
            variants: currentVariants,
            variantPrices: Object.keys(currentVariantPrices).length > 0 ? currentVariantPrices :
currentSizeData.variantPrices,
            variantCostPrices: Object.keys(currentVariantCostPrices).length > 0 ? currentVariantCostPrices :
currentSizeData.variantCostPrices
          };
        } else {
          updatedSizes[size] = {
            ...updatedSizes[size],
            price: parseFloat(sizeData.price) || 0,
            costPrice: parseFloat(sizeData.costPrice) || 0
          };
        }
      }
    });
    payload.sizes = updatedSizes;
    delete payload.editableSizePrices;
  } else {
    delete payload.sizes;
  }
  return payload;
}
export function needsConfirmModal(np) {
  const hasAnyVariantPricing = np.selectedSizes?.some(
    (size) => np.differentPricesPerVariant?.[size]
  );
  const hasVariantPrices = np.variantPrices && Object.keys(np.variantPrices).length > 0 &&
Object.values(np.variantPrices).some(
    (sizeVariants) => sizeVariants && Object.values(sizeVariants).some((price) => parseFloat(price) > 0)
  );
  const hasVariantQuantities = np.variantQuantities && Object.keys(np.variantQuantities).length > 0 &&
Object.values(np.variantQuantities).some(
    (sizeVariants) => sizeVariants && typeof sizeVariants === "object" &&
Object.values(sizeVariants).some((qty) => parseInt(qty, 10) > 0)
  );
  return !!(hasAnyVariantPricing || hasVariantPrices || hasVariantQuantities);
}
```

FILE: pos/mobile/constants/theme.ts
==========

```
import { Platform } from "react-native";
const tintColorLight = "#AD7F65";
const tintColorDark = "#fff";
export const Colors = {
  light: {
    text: "#11181C",
    background: "#fff",
    tint: tintColorLight,
    icon: "#ffffffff",
```

```
    tabIconDefault: "#ffffffff",
    tabIconSelected: tintColorLight
  },
  dark: {
    text: "#ECEDEE",
    background: "#151718",
    tint: tintColorDark,
    icon: "#9BA1A6",
    tabIconDefault: "#9BA1A6",
    tabIconSelected: tintColorDark
  }
};
export const Fonts = Platform.select({
  ios: {
    /** iOS `UIFontDescriptorSystemDesignDefault` */
    sans: "system-ui",
    /** iOS `UIFontDescriptorSystemDesignSerif` */
    serif: "ui-serif",
    /** iOS `UIFontDescriptorSystemDesignRounded` */
    rounded: "ui-rounded",
    /** iOS `UIFontDescriptorSystemDesignMonospaced` */
    mono: "ui-monospace"
  },
  default: {
    sans: "normal",
    serif: "serif",
    rounded: "normal",
    mono: "monospace"
  },
  web: {
    sans: "system-ui, -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, Helvetica, Arial, sans-serif",
    serif: "Georgia, 'Times New Roman', serif",
    rounded: "'SF Pro Rounded', 'Hiragino Maru Gothic ProN', Meiryo, 'MS PGothic', sans-serif",
    mono: "SFMono-Regular, Menlo, Monaco, Consolas, 'Liberation Mono', 'Courier New', monospace"
  }
});



FILE: pos/mobile/context/DataContext.js
================================================================================
import { createContext, useCallback, useContext, useMemo, useRef, useState } from 'react';
import { brandPartnerAPI, categoryAPI, productAPI, transactionAPI } from '../services/api';

const DataContext = createContext(null);

const CACHE_DURATION = 5 * 60 * 1000;

export function DataProvider({ children }) {

  const [products, setProducts] = useState([]);
  const [productsLoading, setProductsLoading] = useState(false);
  const productsLastFetch = useRef(null);

  const [transactions, setTransactions] = useState([]);
  const [transactionsLoading, setTransactionsLoading] = useState(false);
  const transactionsLastFetch = useRef(null);

  const [brands, setBrands] = useState([]);
  const [brandsLoading, setBrandsLoading] = useState(false);
  const brandsLastFetch = useRef(null);

  const [categories, setCategories] = useState([]);
  const [categoriesLoading, setCategoriesLoading] = useState(false);
  const categoriesLastFetch = useRef(null);

  const [dashboardStats, setDashboardStats] = useState({
    totalSalesToday: 0,
    totalTransactions: 0,
    averageTransactionValue: 0,
    lowStockItems: 0,
  });
  const [inventoryStats, setInventoryStats] = useState({
    totalItems: 0,
    inStock: 0,
    lowStock: 0,
    outOfStock: 0,
```

```
    inventoryValue: 0,
    totalCostValue: 0,
    grossMargin: 0,
  });
  const dashboardLastFetch = useRef(null);

  const isCacheValid = (lastFetch) => {
    if (!lastFetch) return false;
    return Date.now() - lastFetch < CACHE_DURATION;
  };


  const fetchProducts = useCallback(async (forceRefresh = false, params = {}) => {




    const isPagination = params.page && params.page > 1;

    try {
      setProductsLoading(true);
      const response = await productAPI.getAll(params);

      if (response.success && response.data) {
        if (isPagination) {
          setProducts(prev => {

            const newItems = response.data.filter(newItem =>
              !prev.some(existing => existing._id === newItem._id)
            );
            return [...prev, ...newItems];
          });
        } else {

          setProducts(response.data);
          productsLastFetch.current = Date.now();
        }
        return response; // Return full response for pagination metadata (totalPages etc)
      }
      return { data: products }; // Fallback
    } catch (error) {
      console.error('Error fetching products:', error);
      return { data: products };
    } finally {
      setProductsLoading(false);
    }
  }, [products]);

  const fetchTransactions = useCallback(async (forceRefresh = false, params = {}) => {
    const isPagination = params.page && params.page > 1;

    if (!forceRefresh && !isPagination && isCacheValid(transactionsLastFetch.current) &&
transactions.length > 0) {
      return { data: transactions };
    }

    try {
      setTransactionsLoading(true);

      const response = await transactionAPI.getAll(params);

      if (response.success && response.data) {
        if (isPagination) {
          setTransactions(prev => {

            const newItems = response.data.filter(newItem =>
              !prev.some(existing => existing._id === newItem._id)
            );
            return [...prev, ...newItems];
          });
        } else {
          setTransactions(response.data);
          transactionsLastFetch.current = Date.now();
        }
        return response;
      }
      return { data: transactions };
```

```
    } catch (error) {
      console.error('Error fetching transactions:', error);
      return { data: transactions };
    } finally {
      setTransactionsLoading(false);
    }
  }, [transactions]);

  const fetchBrands = useCallback(async (forceRefresh = false) => {
    if (!forceRefresh && isCacheValid(brandsLastFetch.current) && brands.length > 0) {
      return brands;
    }

    try {
      setBrandsLoading(true);
      const response = await brandPartnerAPI.getAll();
      if (response.success && response.data) {
        setBrands(response.data);
        brandsLastFetch.current = Date.now();
        return response.data;
      }
      return brands;
    } catch (error) {
      console.error('Error fetching brands:', error);
      return brands;
    } finally {
      setBrandsLoading(false);
    }
  }, [brands]);

  const fetchCategories = useCallback(async (forceRefresh = false) => {
    if (!forceRefresh && isCacheValid(categoriesLastFetch.current) && categories.length > 0) {
      return categories;
    }

    try {
      setCategoriesLoading(true);
      const response = await categoryAPI.getAll();
      if (response.success && response.data) {
        setCategories(response.data);
        categoriesLastFetch.current = Date.now();
        return response.data;
      }
      return categories;
    } catch (error) {
      console.error('Error fetching categories:', error);
      return categories;
    } finally {
      setCategoriesLoading(false);
    }
  }, [categories]);

  const calculateDashboardStats = useCallback(async (forceRefresh = false) => {
    if (!forceRefresh && isCacheValid(dashboardLastFetch.current)) {
      return dashboardStats;
    }

    try {

      const [txnStatsResponse, inventoryStatsResponse] = await Promise.all([
        transactionAPI.getDashboardStats(), // You might need to ensure this endpoint exists and returns
correct structure
        productAPI.getInventoryStats()
      ]);

      let newStats = {
        totalSalesToday: 0,
        totalTransactions: 0,
        averageTransactionValue: 0,
        lowStockItems: 0,
      };

      if (txnStatsResponse.success && txnStatsResponse.data) {
        newStats.totalSalesToday = txnStatsResponse.data.totalSales || 0;
        newStats.totalTransactions = txnStatsResponse.data.transactionCount || 0;
        newStats.averageTransactionValue = txnStatsResponse.data.averageValue || 0;
      }
```

```
    if (inventoryStatsResponse.success && inventoryStatsResponse.data) {
      newStats.lowStockItems = inventoryStatsResponse.data.lowStock || 0;
      setInventoryStats(inventoryStatsResponse.data); // Save full stats for Inventory screen
    }

    setDashboardStats(newStats);
    dashboardLastFetch.current = Date.now();
    return newStats;
  } catch (error) {
    console.error('Error calculating dashboard stats:', error);
    return dashboardStats;
  }
}, [dashboardStats]);

const invalidateCache = useCallback((type) => {
  switch (type) {
    case 'products':
      productsLastFetch.current = null;
      break;
    case 'transactions':
      transactionsLastFetch.current = null;
      break;
    case 'brands':
      brandsLastFetch.current = null;
      break;
    case 'categories':
      categoriesLastFetch.current = null;
      break;
    case 'dashboard':
      dashboardLastFetch.current = null;
      break;
    case 'all':
      productsLastFetch.current = null;
      transactionsLastFetch.current = null;
      brandsLastFetch.current = null;
      categoriesLastFetch.current = null;
      dashboardLastFetch.current = null;
      break;
  }
}, []);

const refreshAll = useCallback(async () => {
  invalidateCache('all');
  await Promise.all([
    fetchProducts(true),
    fetchTransactions(true),
    fetchBrands(true),
    fetchCategories(true),
  ]);
}, [invalidateCache, fetchProducts, fetchTransactions, fetchBrands, fetchCategories]);

const value = useMemo(() => ({

  products,
  productsLoading,
  fetchProducts,
  setProducts,

  transactions,
  transactionsLoading,
  fetchTransactions,
  setTransactions,

  brands,
  brandsLoading,
  fetchBrands,

  categories,
  categoriesLoading,
  fetchCategories,

  dashboardStats,
  inventoryStats,
  calculateDashboardStats,

  invalidateCache,
```

```
    refreshAll,
  }), [
    products, productsLoading, fetchProducts,
    transactions, transactionsLoading, fetchTransactions,
    brands, brandsLoading, fetchBrands,
    categories, categoriesLoading, fetchCategories,
    dashboardStats, inventoryStats, calculateDashboardStats,
    invalidateCache, refreshAll
  ]);

  return (
    <DataContext.Provider value={value}>
      {children}
    </DataContext.Provider>
  );
}

export function useData() {
  const context = useContext(DataContext);
  if (!context) {
    throw new Error('useData must be used within a DataProvider');
  }
  return context;
}

export default DataContext;
```

FILE: pos/mobile/eas.json
==========

```
{
  "cli": {
    "version": ">= 3.0.0",
    "appVersionSource": "remote"
  },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "distribution": "internal",
      "android": {
        "buildType": "apk"
      }
    },
    "production": {
      "autoIncrement": true
    }
  },
  "submit": {
    "production": {}
  }
}
```

FILE: pos/mobile/eslint.config.js
==========

```
const { defineConfig } = require("eslint/config");
const expoConfig = require("eslint-config-expo/flat");
module.exports = defineConfig([
  expoConfig,
  {
    ignores: ["dist/*"]
  }
]);
```

FILE: pos/mobile/expo-env.d.ts
==========

(empty)

FILE: pos/mobile/hooks/use-color-scheme.ts
==========

```
export { useColorScheme } from "react-native";
```

FILE: pos/mobile/hooks/use-color-scheme.web.ts
==========

```
import { useEffect, useState } from "react";
import { useColorScheme as useRNColorScheme } from "react-native";
export function useColorScheme() {
  const [hasHydrated, setHasHydrated] = useState(false);
  useEffect(() => {
    setHasHydrated(true);
  }, []);
  const colorScheme = useRNColorScheme();
  if (hasHydrated) {
    return colorScheme;
  }
  return "light";
}
```

FILE: pos/mobile/hooks/use-theme-color.ts
==========

```
import { Colors } from "@/constants/theme";
import { useColorScheme } from "@/hooks/use-color-scheme";
export function useThemeColor(props, colorName) {
  const theme = useColorScheme() ?? "light";
  const colorFromProps = props[theme];
  if (colorFromProps) {
    return colorFromProps;
  } else {
    return Colors[theme][colorName];
  }
}
```

FILE: pos/mobile/package.json
==========

```
{
  "name": "cysmob",
  "main": "expo-router/entry",
  "version": "1.0.0",
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web",
    "lint": "expo lint"
  },
  "dependencies": {
    "@expo/vector-icons": "^15.0.3",
    "@react-native-async-storage/async-storage": "^2.2.0",
    "@react-native-community/datetimepicker": "^8.4.4",
    "@react-native-community/netinfo": "^11.4.1",
    "@react-native-picker/picker": "2.11.1",
    "@react-navigation/bottom-tabs": "^7.8.7",
    "@react-navigation/elements": "^2.6.3",
    "@react-navigation/native": "^7.1.22",
    "@shopify/flash-list": "2.0.2",
    "buffer": "^6.0.3",
    "expo-constants": "~18.0.10",
    "expo-file-system": "^19.0.17",
    "expo-font": "~14.0.9",
    "expo-haptics": "~15.0.7",
    "expo-image": "~3.0.10",
    "expo-image-manipulator": "~14.0.8",
    "expo-image-picker": "~17.0.8",
    "expo-linear-gradient": "~15.0.7",
    "expo-linking": "~8.0.9",
    "expo-local-authentication": "~17.0.8",
    "expo-notifications": "~0.32.13",
    "expo-print": "~15.0.7",
    "expo-router": "~6.0.15",
    "expo-screen-orientation": "~9.0.7",
    "expo-sharing": "~14.0.7",
```

```
    "expo-splash-screen": "~31.0.11",
    "expo-status-bar": "~3.0.8",
    "expo-symbols": "~1.0.7",
    "expo-system-ui": "~6.0.8",
    "expo-web-browser": "~15.0.9",
    "react": "19.1.0",
    "react-dom": "19.1.0",
    "react-native": "0.81.5",
    "react-native-chart-kit": "^6.12.0",
    "react-native-gesture-handler": "~2.28.0",
    "react-native-reanimated": "^4.1.1",
    "react-native-safe-area-context": "~5.6.0",
    "react-native-screens": "~4.16.0",
    "react-native-svg": "15.12.1",
    "react-native-toast-message": "^2.3.3",
    "react-native-web": "~0.21.0",
    "react-native-worklets": "0.5.1",
    "xlsx": "^0.18.5"
  },
  "devDependencies": {
    "@types/react": "~19.1.0",
    "@types/react-native": "^0.72.8",
    "eslint": "^9.25.0",
    "eslint-config-expo": "~10.0.0",
    "expo": "^54.0.33",
    "typescript": "~5.9.2"
  },
  "private": true
}


FILE: pos/mobile/services/api.js
================================================================================
import config from "../config/api";
import NetworkService from "./NetworkService";
import OfflineService from "./OfflineService";
NetworkService.init();
NetworkService.subscribe(async (isConnected) => {
  if (isConnected) {
    console.log("Network restored. Syncing offline queue...");
    await syncOfflineQueue();
  }
});
const timeoutPromise = (ms) => {
  return new Promise((_, reject) => {
    setTimeout(() => reject(new Error(`Request timed out after ${ms}ms`)), ms);
  });
};
const syncOfflineQueue = async () => {
  const queue = await OfflineService.getQueue();
  if (queue.length === 0) return;
  console.log(`Syncing ${queue.length} offline actions...`);
  for (const action of queue) {
    try {
      console.log("Replaying action:", action);
      const { endpoint, options, id } = action;
      await apiCall(endpoint, options, true);
      await OfflineService.removeFromQueue(id);
    } catch (error) {
      console.error("Failed to sync action:", action, error);
    }
  }
};
const apiCall = async (endpoint, options = {}, forceOnline = false) => {
  const isConnected = NetworkService.getIsConnected();
  const method = options.method || "GET";
  if (!isConnected && !forceOnline) {
    console.log(`[OFFLINE] ${method} ${endpoint}`);
    if (method === "GET") {
      const cachedData = await OfflineService.getCache(endpoint);
      if (cachedData) {
        console.log("Serving from cache");
        return cachedData;
      } else {
        throw new Error("No internet connection and no cached data available.");
      }
    } else {
```

```
      const success = await OfflineService.queueAction({ endpoint, options });
      if (success) {
        console.log("Action queued for sync");
        return { success: true, message: "Action queued (Offline Mode)", data: {} };
      } else {
        throw new Error("Failed to queue offline action.");
      }
    }
  }
  const url = `${config.API_URL}${endpoint}`;
  const timeout = config.TIMEOUT || 3e4;
  const defaultOptions = {
    headers: {
      "Content-Type": "application/json",
      "Accept-Encoding": "gzip, deflate"
    }
  };
  const finalOptions = {
    ...defaultOptions,
    ...options,
    headers: {
      ...defaultOptions.headers,
      ...options.headers
    }
  };
  try {
    const response = await Promise.race([
      fetch(url, finalOptions),
      timeoutPromise(timeout)
    ]);
    const data = await response.json();
    if (!response.ok) {
      throw new Error(data.message || `HTTP error! status: ${response.status}`);
    }
    if (method === "GET") {
      OfflineService.saveCache(endpoint, data);
    }
    return data;
  } catch (error) {
    console.error(`API Error [${endpoint}]:`, error.message);
    if (method === "GET") {
      console.log("Request failed, attempting to serve from cache...");
      const cachedData = await OfflineService.getCache(endpoint);
      if (cachedData) {
        console.log("Serving from cache (fallback)");
        return cachedData;
      }
    }
    throw error;
  }
};
export const productAPI = {
  // Get all products (uses minimal fields to exclude heavy base64 images)
  getAll: async (params = {}) => {
    const queryParams = { fields: "minimal", ...params };
    const queryString = new URLSearchParams(queryParams).toString();
    const endpoint = `/products${queryString ? `?${queryString}` : ""}`;
    return apiCall(endpoint);
  },
  // Get product by ID
  getById: async (id) => {
    return apiCall(`/products/${id}`);
  },
  // Search products
  search: async (query) => {
    return apiCall(`/products/search/${encodeURIComponent(query)}`);
  },
  // Get products by category
  getByCategory: async (category) => {
    return apiCall(`/products/category/${encodeURIComponent(category)}`);
  },
  // Create product
  create: async (productData) => {
    return apiCall("/products", {
      method: "POST",
      body: JSON.stringify(productData)
    });
```

```
  },
  // Update product
  update: async (id, productData) => {
    return apiCall(`/products/${id}`, {
      method: "PUT",
      body: JSON.stringify(productData)
    });
  },
  // Stock in (server-side batches / FIFO — same as web Inventory)
  stockIn: async (id, payload) => {
    return apiCall(`/products/${id}/stock-in`, {
      method: "POST",
      body: JSON.stringify(payload)
    });
  },
  // Stock out (server consumes batches FIFO — same as web)
  stockOut: async (id, payload) => {
    return apiCall(`/products/${id}/stock-out`, {
      method: "POST",
      body: JSON.stringify(payload)
    });
  },
  // Delete product
  delete: async (id) => {
    return apiCall(`/products/${id}`, {
      method: "DELETE"
    });
  },
  // Update stock after transaction
  updateStock: async (items, performedByName, performedById) => {
    return apiCall("/products/update-stock", {
      method: "POST",
      body: JSON.stringify({
        items,
        performedByName,
        performedById
      })
    });
  },
  // Get inventory stats (value, margins, counts)
  getInventoryStats: async () => {
    return apiCall("/products/inventory-stats");
  }
};
export const transactionAPI = {
  // Get all transactions
  getAll: async (params = {}) => {
    const queryString = new URLSearchParams(params).toString();
    const endpoint = `/transactions${queryString ? `?${queryString}` : ""}`;
    return apiCall(endpoint);
  },
  // Get transaction by ID
  getById: async (id) => {
    return apiCall(`/transactions/${id}`);
  },
  // Create transaction
  create: async (transactionData) => {
    return apiCall("/transactions", {
      method: "POST",
      body: JSON.stringify(transactionData)
    });
  },
  // Update transaction
  update: async (id, transactionData) => {
    return apiCall(`/transactions/${id}`, {
      method: "PUT",
      body: JSON.stringify(transactionData)
    });
  },
  // Get dashboard stats
  getDashboardStats: async (timeframe = "daily") => {
    return apiCall(`/transactions/dashboard/stats?timeframe=${encodeURIComponent(timeframe)}`);
  },
  // Get sales over time (for charts)
  getSalesOverTime: async (timeframe = "daily") => {
    const tf = timeframe.toLowerCase();
    return apiCall(`/transactions/sales-over-time?timeframe=${encodeURIComponent(tf)}`);
```

```
  },
  // Get sales by category
  getSalesByCategory: async () => {
    return apiCall("/transactions/sales-by-category");
  },
  // Get top selling products
  getTopSelling: async (params = {}) => {
    const queryString = new URLSearchParams(params).toString();
    return apiCall(`/transactions/top-selling${queryString ? `?${queryString}` : ""}`);
  }
};
export const employeeAPI = {
  // Get all employees
  getAll: async () => {
    return apiCall("/employees");
  },
  /** Same source as web dashboard: staff with isOnline + recent heartbeat; Owner excluded server-side */
  getOnline: async () => {
    return apiCall("/employees/online");
  },
  // Get employee by ID
  getById: async (id) => {
    return apiCall(`/employees/${id}`);
  },
  // Search employees by first name
  search: async (query) => {
    return apiCall(`/employees/search/${encodeURIComponent(query)}`);
  },
  // Create employee
  create: async (employeeData) => {
    return apiCall("/employees", {
      method: "POST",
      body: JSON.stringify(employeeData)
    });
  },
  // Update employee
  update: async (id, employeeData) => {
    return apiCall(`/employees/${id}`, {
      method: "PUT",
      body: JSON.stringify(employeeData)
    });
  },
  // Delete employee
  delete: async (id) => {
    return apiCall(`/employees/${id}`, {
      method: "DELETE"
    });
  },
  // Verify PIN
  verifyPin: async (pin, employeeId) => {
    return apiCall("/employees/verify-pin", {
      method: "POST",
      body: JSON.stringify({ pin, employeeId })
    });
  },
  /**
   * Verify Owner PIN (for mobile app - only accepts owner accounts)
   * This function enforces owner-only access to the mobile application.
   *
   * @param {string} pin - The 6-digit PIN to verify
   * @returns {Promise<{success: boolean, message: string, data: object|null}>}
   */
  verifyOwnerPin: async (pin) => {
    if (!pin || typeof pin !== "string" || pin.length !== 6) {
      return {
        success: false,
        message: "Invalid PIN format. PIN must be 6 digits.",
        data: null
      };
    }
    try {
      const response = await apiCall("/employees/verify-pin", {
        method: "POST",
        body: JSON.stringify({ pin })
      });
      if (response.success && response.data) {
        if (response.data.role !== "Owner") {
```

```
          return {
            success: false,
            message: "Only owner account can access the mobile app.",
            data: null
          };
        }
        return {
          success: true,
          message: "Owner PIN verified successfully",
          data: response.data,
          requiresPinReset: response.requiresPinReset || false
        };
      }
      return response;
    } catch (error) {
      return {
        success: false,
        message: error.message || "Failed to verify PIN. Please check your connection.",
        data: null
      };
    }
  },
  // Update PIN
  updatePin: async (id, pinData) => {
    return apiCall(`/employees/${id}/pin`, {
      method: "PUT",
      body: JSON.stringify(pinData)
    });
  }
};
export const cartAPI = {
  // Get cart by user ID
  getByUserId: async (userId) => {
    return apiCall(`/cart/${encodeURIComponent(userId)}`);
  },
  // Save cart
  save: async (userId, items) => {
    return apiCall(`/cart/${encodeURIComponent(userId)}`, {
      method: "PUT",
      body: JSON.stringify({ items })
    });
  }
};
export const stockMovementAPI = {
  // Get all stock movements
  getAll: async (params = {}) => {
    const queryString = new URLSearchParams(params).toString();
    const endpoint = `/stock-movements${queryString ? `?${queryString}` : ""}`;
    return apiCall(endpoint);
  },
  // Create stock movement
  create: async (movementData) => {
    return apiCall("/stock-movements", {
      method: "POST",
      body: JSON.stringify(movementData)
    });
  },
  // Get stock stats over time
  getStockStatsOverTime: async (timeframe = "daily") => {
    return apiCall(`/stock-movements/stats/over-time?timeframe=${encodeURIComponent(timeframe)}`);
  }
};
export const archiveAPI = {
  // Get all archived items
  getAll: async (params = {}) => {
    const queryString = new URLSearchParams(params).toString();
    const endpoint = `/archive${queryString ? `?${queryString}` : ""}`;
    return apiCall(endpoint);
  },
  // Archive item
  archive: async (itemData) => {
    return apiCall("/archive", {
      method: "POST",
      body: JSON.stringify(itemData)
    });
  },
  // Restore archived item
```

```
  restore: async (id) => {
    return apiCall(`/archive/${id}/restore`, {
      method: "PUT"
    });
  }
};
export const voidLogAPI = {
  // Get all void logs
  getAll: async (params = {}) => {
    const queryString = new URLSearchParams(params).toString();
    const endpoint = `/void-logs${queryString ? `?${queryString}` : ""}`;
    return apiCall(endpoint);
  },
  // Create void log
  create: async (voidLogData) => {
    return apiCall("/void-logs", {
      method: "POST",
      body: JSON.stringify(voidLogData)
    });
  }
};
export const discountAPI = {
  // Get all discounts
  getAll: async () => {
    return apiCall("/discounts");
  },
  // Get discount by ID
  getById: async (id) => {
    return apiCall(`/discounts/${id}`);
  },
  // Create discount
  create: async (discountData) => {
    return apiCall("/discounts", {
      method: "POST",
      body: JSON.stringify(discountData)
    });
  },
  // Update discount
  update: async (id, discountData) => {
    return apiCall(`/discounts/${id}`, {
      method: "PUT",
      body: JSON.stringify(discountData)
    });
  },
  // Delete discount
  delete: async (id) => {
    return apiCall(`/discounts/${id}`, {
      method: "DELETE"
    });
  }
};
export const categoryAPI = {
  // Get all categories
  getAll: async () => {
    return apiCall("/categories");
  },
  // Get category by ID
  getById: async (id) => {
    return apiCall(`/categories/${id}`);
  },
  // Create category
  create: async (categoryData) => {
    return apiCall("/categories", {
      method: "POST",
      body: JSON.stringify(categoryData)
    });
  },
  // Update category
  update: async (id, categoryData) => {
    return apiCall(`/categories/${id}`, {
      method: "PUT",
      body: JSON.stringify(categoryData)
    });
  },
  // Delete category
  delete: async (id) => {
    return apiCall(`/categories/${id}`, {
```

```
      method: "DELETE"
    });
  }
};
export const syncAPI = {
  // Sync all data
  syncAll: async () => {
    return apiCall("/sync/all", {
      method: "POST"
    });
  }
};
export const brandPartnerAPI = {
  // Get all brand partners
  getAll: async () => {
    return apiCall("/brand-partners");
  },
  // Get brand partner by ID
  getById: async (id) => {
    return apiCall(`/brand-partners/${id}`);
  },
  // Create brand partner
  create: async (brandData) => {
    return apiCall("/brand-partners", {
      method: "POST",
      body: JSON.stringify(brandData)
    });
  }
};
export const stockAPI = {
  // Get low stock and out of stock items
  getLowStock: async () => {
    return apiCall("/products/low-stock");
  }
};
```

FILE: pos/mobile/services/NetworkService.js
==========

```
import NetInfo from "@react-native-community/netinfo";
let isConnected = true;
const listeners = [];
const init = () => {
  NetInfo.fetch().then((state) => {
    isConnected = state.isConnected;
  });
  const unsubscribe = NetInfo.addEventListener((state) => {
    const wasConnected = isConnected;
    isConnected = state.isConnected;
    if (wasConnected !== isConnected) {
      listeners.forEach((listener) => listener(isConnected));
    }
  });
  return unsubscribe;
};
const getIsConnected = () => {
  return isConnected;
};
const subscribe = (callback) => {
  listeners.push(callback);
  return () => {
    const index = listeners.indexOf(callback);
    if (index > -1) {
      listeners.splice(index, 1);
    }
  };
};
export default {
  init,
  getIsConnected,
  subscribe
};
```

FILE: pos/mobile/services/notificationService.js

```
================================================================================
import AsyncStorage from "@react-native-async-storage/async-storage";
import * as Notifications from "expo-notifications";
import { Platform } from "react-native";
import { stockAPI } from "./api";
const NOTIF_ENABLED_KEY = "@notifications_enabled";
const NOTIFIED_IDS_KEY = "@notified_alert_ids";
export async function setupNotificationChannel() {
  if (Platform.OS === "android") {
    await Notifications.setNotificationChannelAsync("stock-alerts", {
      name: "Stock Alerts",
      importance: Notifications.AndroidImportance.HIGH,
      vibrationPattern: [0, 250, 250, 250],
      sound: "default"
    });
  }
}
export async function requestPermissions() {
  const { status: existingStatus } = await Notifications.getPermissionsAsync();
  let finalStatus = existingStatus;
  if (existingStatus !== "granted") {
    const { status } = await Notifications.requestPermissionsAsync();
    finalStatus = status;
  }
  return finalStatus === "granted";
}
export async function isNotificationsEnabled() {
  try {
    const value = await AsyncStorage.getItem(NOTIF_ENABLED_KEY);
    return value === "true";
  } catch {
    return false;
  }
}
export async function setNotificationsEnabled(enabled) {
  try {
    await AsyncStorage.setItem(NOTIF_ENABLED_KEY, enabled ? "true" : "false");
    if (!enabled) {
      await Notifications.cancelAllScheduledNotificationsAsync();
    }
  } catch (e) {
    console.error("Error saving notification preference:", e);
  }
}
async function getNotifiedIds() {
  try {
    const json = await AsyncStorage.getItem(NOTIFIED_IDS_KEY);
    return json ? JSON.parse(json) : [];
  } catch {
    return [];
  }
}
async function addNotifiedIds(newIds) {
  try {
    const existing = await getNotifiedIds();
    const merged = [.../* @__PURE__ */ new Set([...existing, ...newIds])];
    const trimmed = merged.slice(-500);
    await AsyncStorage.setItem(NOTIFIED_IDS_KEY, JSON.stringify(trimmed));
  } catch (e) {
    console.error("Error saving notified IDs:", e);
  }
}
export async function clearNotifiedIds() {
  try {
    await AsyncStorage.removeItem(NOTIFIED_IDS_KEY);
  } catch (e) {
    console.error("Error clearing notified IDs:", e);
  }
}
export async function checkAndNotify() {
  try {
    const enabled = await isNotificationsEnabled();
    if (!enabled) return;
    const response = await stockAPI.getLowStock();
    if (!response?.success || !response.data || response.data.length === 0) return;
    const alreadyNotified = await getNotifiedIds();
    const newAlerts = response.data.filter(
```

```
      (item) => !alreadyNotified.includes(item._id)
    );
    if (newAlerts.length === 0) return;
    const newIds = [];
    for (const alert of newAlerts) {
      const isOutOfStock = alert.alertType === "out_of_stock";
      await Notifications.scheduleNotificationAsync({
        content: {
          title: isOutOfStock ? "\u26A0\uFE0F Out of Stock" : "\u{1F4E6} Low Stock Alert",
          body: isOutOfStock ? `${alert.itemName} is OUT OF STOCK! Restock immediately.` : `${alert.itemName}
is running low (${alert.currentStock} left)`,
          sound: "default",
          priority: Notifications.AndroidNotificationPriority.HIGH,
          ...Platform.OS === "android" && { channelId: "stock-alerts" }
        },
        trigger: null
        // Fire immediately
      });
      newIds.push(alert._id);
    }
    await addNotifiedIds(newIds);
  } catch (error) {
    console.error("Error in checkAndNotify:", error);
  }
}
```

FILE: pos/mobile/services/OfflineService.js
==========

```
import AsyncStorage from "@react-native-async-storage/async-storage";
const CACHE_PREFIX = "offline_cache_";
const QUEUE_KEY = "offline_action_queue";
const saveCache = async (key, data) => {
  try {
    await AsyncStorage.setItem(`${CACHE_PREFIX}${key}`, JSON.stringify(data));
  } catch (error) {
    console.error("Error saving to cache:", error);
  }
};
const getCache = async (key) => {
  try {
    const data = await AsyncStorage.getItem(`${CACHE_PREFIX}${key}`);
    return data ? JSON.parse(data) : null;
  } catch (error) {
    console.error("Error getting from cache:", error);
    return null;
  }
};
const queueAction = async (action) => {
  try {
    const queue = await getQueue();
    queue.push({
      ...action,
      timestamp: Date.now(),
      id: Math.random().toString(36).substr(2, 9)
    });
    await AsyncStorage.setItem(QUEUE_KEY, JSON.stringify(queue));
    return true;
  } catch (error) {
    console.error("Error queueing action:", error);
    return false;
  }
};
const getQueue = async () => {
  try {
    const queue = await AsyncStorage.getItem(QUEUE_KEY);
    return queue ? JSON.parse(queue) : [];
  } catch (error) {
    console.error("Error getting queue:", error);
    return [];
  }
};
const clearQueue = async () => {
  try {
    await AsyncStorage.removeItem(QUEUE_KEY);
  } catch (error) {
```

```
    console.error("Error clearing queue:", error);
  }
};
const removeFromQueue = async (id) => {
  try {
    const queue = await getQueue();
    const newQueue = queue.filter((item) => item.id !== id);
    await AsyncStorage.setItem(QUEUE_KEY, JSON.stringify(newQueue));
  } catch (error) {
    console.error("Error removing from queue:", error);
  }
};
export default {
  saveCache,
  getCache,
  queueAction,
  getQueue,
  clearQueue,
  removeFromQueue
};
```

FILE: pos/mobile/tsconfig.json
==========

```
{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true,
    "paths": {
      "@/*": [
        "./*"
      ]
    },
    "jsx": "react-native"
  },
  "include": [
    "**/*.ts",
    "**/*.tsx",
    ".expo/types/**/*.ts",
    "expo-env.d.ts"
  ]
}
```

FILE: pos/print-bridge/main.js
==========

```
const path = require("path");
const { app, BrowserWindow, dialog } = require("electron");
const { startPrintServer } = require("./printService");
const PRINT_SERVER_PORT = Number(process.env.PRINT_BRIDGE_PORT || 3e3);
let mainWindow;
let server;
function createWindow() {
  mainWindow = new BrowserWindow({
    width: 520,
    height: 460,
    resizable: false,
    autoHideMenuBar: true,
    webPreferences: {
      preload: path.join(__dirname, "preload.js"),
      nodeIntegration: false,
      contextIsolation: true
    }
  });
  mainWindow.loadFile(path.join(__dirname, "renderer.html"));
  mainWindow.on("closed", () => {
    mainWindow = null;
  });
}
async function bootstrap() {
  try {
    server = await startPrintServer({
      port: PRINT_SERVER_PORT,
      printerAddress: process.env.PRINTER_BT_ADDRESS,
      printerChannel: Number(process.env.PRINTER_BT_CHANNEL || 1)
    });
```

```
  } catch (error) {
    dialog.showErrorBox("Print Bridge Error", `Could not start print server: ${error.message}`);
    app.quit();
    return;
  }
  createWindow();
}
app.whenReady().then(bootstrap);
app.on("window-all-closed", () => {
  if (process.platform !== "darwin") {
    app.quit();
  }
});
app.on("before-quit", () => {
  if (server) {
    server.close();
  }
});
app.on("activate", () => {
  if (mainWindow === null) {
    createWindow();
  }
});
```

FILE: pos/print-bridge/package.json
==========

```
{
  "name": "thermal-print-bridge",
  "version": "1.0.0",
  "description": "Cross-platform Electron bridge that exposes a local HTTP API for Bluetooth ESC/POS receipt
printers.",
  "main": "main.js",
  "scripts": {
    "start": "electron .",
    "lint": "echo \"No lint step configured\""
  },
  "keywords": [
    "electron",
    "express",
    "escpos",
    "bluetooth",
    "thermal",
    "printer"
  ],
  "author": "",
  "license": "MIT",
  "dependencies": {
    "body-parser": "^1.20.3",
    "cors": "^2.8.5",
    "escpos": "^3.0.0-alpha.8",
    "escpos-bluetooth": "^0.1.1",
    "express": "^4.19.2"
  },
  "devDependencies": {
    "electron": "^32.0.1"
  },
  "optionalDependencies": {
    "bluetooth-serial-port": "^3.0.3"
  }
}
```

FILE: pos/print-bridge/preload.js
==========

```
const { contextBridge } = require("electron");
contextBridge.exposeInMainWorld("printBridge", {
  version: "1.0.0"
});
```

FILE: pos/print-bridge/printService.js
==========

```
const express = require("express");
const cors = require("cors");
```

```
const bodyParser = require("body-parser");
const escpos = require("escpos");
let BluetoothAdapter;
try {
  BluetoothAdapter = require("escpos-bluetooth");
  escpos.Bluetooth = BluetoothAdapter;
} catch (error) {
  console.warn("[printService] escpos-bluetooth not available:", error.message);
}
let bluetoothSerialPort;
try {
  bluetoothSerialPort = require("bluetooth-serial-port");
} catch (error) {
  console.warn("[printService] bluetooth-serial-port fallback not available:", error.message);
}
const DEFAULT_BT_ADDRESS = process.env.PRINTER_BT_ADDRESS || "00:00:00:00:00:00";
const DEFAULT_BT_CHANNEL = Number(process.env.PRINTER_BT_CHANNEL || 1);
const MAX_LINE_CHARS = Number(process.env.PRINTER_LINE_CHARS || 32);
const VARIANT_ONLY_SIZE_KEY = "__VARIANT_ONLY__";
const formatReceiptVariantSizeLine = (item) => {
  const rawSize = item?.selectedSize || item?.size || "";
  const size = rawSize === VARIANT_ONLY_SIZE_KEY ? "" : String(rawSize).trim();
  const variant = String(item?.variant || item?.selectedVariation || "").trim();
  const parts = [];
  if (variant) parts.push(variant);
  if (size) parts.push(size);
  return parts.join(" | ");
};
const formatCurrency = (value) => `PHP ${Number(value || 0).toFixed(2)}`;
const padLine = (left, right = "") => {
  const cleanLeft = String(left ?? "").trim();
  const cleanRight = String(right ?? "").trim();
  const available = MAX_LINE_CHARS - (cleanLeft.length + cleanRight.length);
  const spacer = available > 0 ? " ".repeat(available) : " ";
  return (cleanLeft + spacer + cleanRight).slice(0, MAX_LINE_CHARS);
};
const buildReceiptLines = (receipt) => {
  const lines = [];
  const storeName = receipt.storeName || "Create Your Style";
  const location = receipt.location || "Pasonanca, Zamboanga City";
  const receiptNo = receipt.receiptNo || "000000";
  const paymentMethod = receipt.paymentMethod || "Cash";
  const cashier = receipt.cashier || receipt.cashierName || receipt.performedByName || "Staff";
  const receiptDate = receipt.date || (/* @__PURE__ */ new Date()).toLocaleDateString("en-US", {
    month: "numeric",
    day: "numeric",
    year: "numeric"
  });
  const receiptTime = receipt.time || (/* @__PURE__ */ new Date()).toLocaleTimeString("en-US", {
    hour: "numeric",
    minute: "2-digit",
    hour12: true
  });
  const centerText = (text) => {
    const padding = Math.floor((MAX_LINE_CHARS - text.length) / 2);
    return " ".repeat(Math.max(0, padding)) + text;
  };
  lines.push(centerText(storeName));
  lines.push(centerText(location));
  lines.push("");
  lines.push(centerText("RECEIPT"));
  lines.push(centerText(`#${receiptNo}`));
  lines.push("");
  lines.push(padLine("Date:", `${receiptDate}, ${receiptTime}`));
  lines.push(padLine("Cashier:", cashier));
  lines.push(padLine("Payment:", paymentMethod));
  lines.push("");
  (receipt.items || []).forEach((item) => {
    let itemName = (item.name || item.itemName || "Item").toString();
    itemName = itemName.replace(/\s*\([^)]*\)\s*$/, "").trim();
    const qty = item.qty || item.quantity || 1;
    const price = item.price || item.itemPrice || 0;
    const variantSizeLine = formatReceiptVariantSizeLine(item);
    lines.push(itemName);
    if (variantSizeLine) {
      lines.push(variantSizeLine);
    }
```

```
    lines.push(`${qty} x P${Number(price).toFixed(2)}`);
  });
  lines.push("");
  lines.push(padLine("Subtotal:", `P${Number(receipt.subtotal || 0).toFixed(2)}`));
  lines.push(padLine("Discount:", `P${Number(receipt.discount || 0).toFixed(2)}`));
  lines.push("");
  lines.push(padLine("Total:", `P${Number(receipt.total || 0).toFixed(2)}`));
  if (receipt.cash !== void 0) {
    lines.push(padLine("Amount Received:", `P${Number(receipt.cash).toFixed(2)}`));
  }
  if (receipt.change !== void 0) {
    lines.push(padLine("Change:", `P${Number(receipt.change).toFixed(2)}`));
  }
  lines.push("");
  lines.push(centerText("Thank you for your purchase!"));
  lines.push(centerText("This is not an official receipt"));
  return lines;
};
const normalizeLines = (rawLines) => {
  const expandedLines = [];
  rawLines.forEach((line) => {
    const text = String(line || "");
    if (text.length <= MAX_LINE_CHARS) {
      expandedLines.push(text);
      return;
    }
    for (let i = 0; i < text.length; i += MAX_LINE_CHARS) {
      expandedLines.push(text.slice(i, i + MAX_LINE_CHARS));
    }
  });
  return expandedLines;
};
const printViaEscpos = (lines, { address, channel }) => new Promise((resolve, reject) => {
  if (!BluetoothAdapter) {
    return reject(new Error("escpos-bluetooth adapter missing"));
  }
  const device = new escpos.Bluetooth(address, channel);
  const printer = new escpos.Printer(device, { encoding: "GB18030" });
  device.open((error) => {
    if (error) {
      return reject(error);
    }
    try {
      lines.forEach((line) => printer.text(line));
      printer.cut().close();
      resolve();
    } catch (printError) {
      reject(printError);
    }
  });
});
const printViaBluetoothSerial = (lines, { address, channel }) => new Promise((resolve, reject) => {
  if (!bluetoothSerialPort) {
    return reject(new Error("bluetooth-serial-port module missing"));
  }
  const btSerial = new bluetoothSerialPort.BluetoothSerialPort();
  const payload = Buffer.from(`${lines.join("\n")}

`, "ascii");
  const connect = (targetChannel) => {
    btSerial.connect(
      address,
      targetChannel,
      () => {
        btSerial.write(payload, (err) => {
          if (err) {
            return reject(err);
          }
          btSerial.close();
          resolve();
        });
      },
      (err) => reject(err)
    );
  };
  if (channel) {
    connect(channel);
```

```
    return;
  }
  btSerial.findSerialPortChannel(
    address,
    (discoveredChannel) => connect(discoveredChannel),
    (err) => reject(err || new Error("Unable to locate printer channel"))
  );
});
async function handlePrint(lines, config) {
  try {
    await printViaEscpos(lines, config);
  } catch (error) {
    console.warn("[printService] escpos-bluetooth failed, trying fallback:", error.message);
    await printViaBluetoothSerial(lines, config);
  }
}
function startPrintServer({
  port = 3e3,
  printerAddress = DEFAULT_BT_ADDRESS,
  printerChannel = DEFAULT_BT_CHANNEL
} = {}) {
  const app = express();
  app.use(cors());
  app.use(bodyParser.json({ limit: "10kb" }));
  app.get("/health", (_req, res) => {
    res.json({ status: "ok" });
  });
  app.post("/print", async (req, res) => {
    const body = req.body || {};
    let lines;
    if (body.storeName || body.items || body.receiptNo) {
      lines = buildReceiptLines(body);
    } else if (Array.isArray(body.lines) && body.lines.length > 0) {
      lines = body.lines;
    } else {
      return res.status(400).json({ message: "Receipt object or `lines` array is required" });
    }
    try {
      await handlePrint(
        normalizeLines(lines),
        { address: printerAddress, channel: printerChannel }
      );
      res.json({ ok: true });
    } catch (error) {
      console.error("[printService] Failed to print:", error);
      res.status(500).json({ message: error.message });
    }
  });
  return new Promise((resolve, reject) => {
    const server = app.listen(port, () => {
      console.log(`[printService] Listening on http://localhost:${port}`);
      resolve(server);
    }).on("error", reject);
  });
}
module.exports = {
  startPrintServer
};
```

FILE: pos/print-server.ps1
==========

```
$port = 9100
$printerName = "USB Printing Support"  # Your thermal printer

$listener = New-Object System.Net.HttpListener
$listener.Prefixes.Add("http://localhost:$port/")
$listener.Start()

Write-Host "========================================" -ForegroundColor Cyan
Write-Host " POS Print Server Started" -ForegroundColor Green
Write-Host " Listening on: http://localhost:$port" -ForegroundColor Yellow
Write-Host " Printer: $printerName" -ForegroundColor Yellow
```

```
Write-Host " Press Ctrl+C to stop" -ForegroundColor Gray
Write-Host "========================================" -ForegroundColor Cyan

function Format-Currency($value) {
    return "PHP " + [math]::Round([double]$value, 2).ToString("0.00")
}

function Pad-Line($left, $right, $width = 32) {
    $leftStr = $left.ToString().Trim()
    $rightStr = $right.ToString().Trim()
    $available = $width - $leftStr.Length - $rightStr.Length
    if ($available -lt 1) { $available = 1 }
    $spacer = " " * $available
    return ($leftStr + $spacer + $rightStr).Substring(0, [Math]::Min($width, $leftStr.Length + $available +
$rightStr.Length))
}

function Build-ReceiptLines($receipt) {
    $MAX_WIDTH = 32
    $lines = @()

    $storeName = if ($receipt.storeName) { $receipt.storeName } else { "Create Your Style" }
    $contactNumber = if ($receipt.contactNumber) { $receipt.contactNumber } else { "+631112224444" }
    $location = if ($receipt.location) { $receipt.location } else { "Pasonanca, Zamboanga City" }
    $issueTime = if ($receipt.time) { $receipt.time } else { "12:00PM" }
    $referenceNo = if ($receipt.referenceNo) { $receipt.referenceNo } elseif ($receipt.reference)
{ $receipt.reference } else { "-" }

    $lines += $storeName
    $lines += Pad-Line $issueTime $contactNumber $MAX_WIDTH
    $lines += $location
    $lines += "-" * $MAX_WIDTH

    $lines += "Receipt No:"
    $receiptNo = if ($receipt.receiptNo) { $receipt.receiptNo } else { "000000" }
    $lines += "#$receiptNo"
    $lines += "-" * $MAX_WIDTH

    $lines += "Item".PadRight(20) + "Qty".PadLeft(3) + "Price".PadLeft(9)
    $lines += "-" * $MAX_WIDTH

    if ($receipt.items) {
        foreach ($item in $receipt.items) {
            $itemName = if ($item.name) { $item.name } elseif ($item.itemName) { $item.itemName } else
{ "Item" }
            $qty = if ($item.qty) { $item.qty } elseif ($item.quantity) { $item.quantity } else { 1 }
            $price = if ($item.price) { $item.price } elseif ($item.itemPrice) { $item.itemPrice } else { 0 }

            $itemNameStr = $itemName.ToString().Substring(0, [Math]::Min(20, $itemName.Length)).PadRight(20)
            $qtyStr = $qty.ToString().PadLeft(3)
            $priceStr = (Format-Currency $price).PadLeft(9)
            $lines += "$itemNameStr$qtyStr$priceStr"
        }
    }

    $lines += "-" * $MAX_WIDTH

    $lines += Pad-Line "Transaction/Reference" $referenceNo $MAX_WIDTH
    $paymentMethod = if ($receipt.paymentMethod) { $receipt.paymentMethod } else { "CASH" }
    $lines += Pad-Line "Payment Method" $paymentMethod $MAX_WIDTH
    $subtotal = if ($receipt.subtotal) { $receipt.subtotal } else { 0 }
    $lines += Pad-Line "Subtotal" (Format-Currency $subtotal) $MAX_WIDTH
    $lines += "-" * $MAX_WIDTH
    $discount = if ($receipt.discount) { $receipt.discount } else { 0 }
    $lines += Pad-Line "Discount" (Format-Currency $discount) $MAX_WIDTH
    $lines += "-" * $MAX_WIDTH
    $total = if ($receipt.total) { $receipt.total } else { 0 }
    $lines += Pad-Line "Total" (Format-Currency $total) $MAX_WIDTH

    if ($null -ne $receipt.cash) {
        $lines += Pad-Line "Cash" (Format-Currency $receipt.cash) $MAX_WIDTH
    }

    if ($null -ne $receipt.change) {
        $lines += Pad-Line "Change" (Format-Currency $receipt.change) $MAX_WIDTH
    }
```

```
    $lines += "-" * $MAX_WIDTH
    $lines += "This is not an official receipt"

    if ($receipt.cashier) {
        $lines += Pad-Line "Cashier" $receipt.cashier $MAX_WIDTH
    }

    $lines += ""
    $lines += ""
    $lines += ""

    return $lines
}

function Send-ToPrinter($lines) {
    $text = $lines -join "`r`n"

    $ESC = [char]27
    $GS = [char]29
    $INIT = "$ESC@"           # Initialize printer
    $CUT = "$GS" + "V" + [char]66 + [char]0  # Partial cut

    $printData = $INIT + $text + "`r`n`r`n`r`n" + $CUT

    try {

        $printers = Get-WmiObject -Query "SELECT * FROM Win32_Printer WHERE Name LIKE '%$printerName%'"

        if ($printers) {

            $text | Out-Printer -Name $printerName
            Write-Host "[$(Get-Date -Format 'HH:mm:ss')] Printed via Out-Printer" -ForegroundColor Green
            return $true
        }

        $rawPrinter = "\\.\USB001"  # Common USB printer port
        if (Test-Path $rawPrinter) {
            [System.IO.File]::WriteAllText($rawPrinter, $printData)
            Write-Host "[$(Get-Date -Format 'HH:mm:ss')] Printed via raw USB" -ForegroundColor Green
            return $true
        }

        Add-Type -AssemblyName System.Drawing
        $printDoc = New-Object System.Drawing.Printing.PrintDocument
        $printDoc.PrinterSettings.PrinterName = $printerName

        $lineIndex = 0
        $printDoc.add_PrintPage({
            param($sender, $e)
            $font = New-Object System.Drawing.Font("Courier New", 8)
            $brush = [System.Drawing.Brushes]::Black
            $y = 10
            foreach ($line in $lines) {
                $e.Graphics.DrawString($line, $font, $brush, 10, $y)
                $y += 12
            }
        })

        $printDoc.Print()
        Write-Host "[$(Get-Date -Format 'HH:mm:ss')] Printed via PrintDocument" -ForegroundColor Green
        return $true
    }
    catch {
        Write-Host "[$(Get-Date -Format 'HH:mm:ss')] Print error: $_" -ForegroundColor Red
        return $false
    }
}

try {
    while ($listener.IsListening) {
        $context = $listener.GetContext()
        $request = $context.Request
        $response = $context.Response

        $response.Headers.Add("Access-Control-Allow-Origin", "*")
        $response.Headers.Add("Access-Control-Allow-Methods", "POST, GET, OPTIONS")
        $response.Headers.Add("Access-Control-Allow-Headers", "Content-Type")
```

```
        if ($request.HttpMethod -eq "OPTIONS") {
            $response.StatusCode = 200
            $response.Close()
            continue
        }

        if ($request.HttpMethod -eq "GET" -and $request.Url.AbsolutePath -eq "/health") {
            $responseJson = '{"status":"ok"}'
            $buffer = [System.Text.Encoding]::UTF8.GetBytes($responseJson)
            $response.ContentType = "application/json"
            $response.ContentLength64 = $buffer.Length
            $response.OutputStream.Write($buffer, 0, $buffer.Length)
            $response.Close()
            continue
        }

        if ($request.HttpMethod -eq "POST" -and $request.Url.AbsolutePath -eq "/print") {
            $reader = New-Object System.IO.StreamReader($request.InputStream)
            $body = $reader.ReadToEnd()
            $reader.Close()

            Write-Host "[$(Get-Date -Format 'HH:mm:ss')] Received print request" -ForegroundColor Cyan

            try {
                $receipt = $body | ConvertFrom-Json
                $lines = Build-ReceiptLines $receipt

                Write-Host "--- Receipt Preview ---" -ForegroundColor DarkGray
                foreach ($line in $lines) {
                    Write-Host $line -ForegroundColor DarkGray
                }
                Write-Host "-----------------------" -ForegroundColor DarkGray

                $success = Send-ToPrinter $lines

                if ($success) {
                    $responseJson = '{"ok":true}'
                    $response.StatusCode = 200
                } else {
                    $responseJson = '{"ok":false,"message":"Print failed"}'
                    $response.StatusCode = 500
                }
            }
            catch {
                Write-Host "[$(Get-Date -Format 'HH:mm:ss')] Error: $_" -ForegroundColor Red
                $responseJson = "{`"ok`":false,`"message`":`"$_`"}"
                $response.StatusCode = 500
            }

            $buffer = [System.Text.Encoding]::UTF8.GetBytes($responseJson)
            $response.ContentType = "application/json"
            $response.ContentLength64 = $buffer.Length
            $response.OutputStream.Write($buffer, 0, $buffer.Length)
            $response.Close()
            continue
        }

        $response.StatusCode = 404
        $response.Close()
    }
}
finally {
    $listener.Stop()
    Write-Host "Server stopped" -ForegroundColor Yellow
}
```

FILE: pos/start-pos.bat
==========

```
@echo off
title Starting POS System...

net start MongoDB >nul 2>&1

cd backend
```

```
IF NOT EXIST "node_modules" (
    echo First time run detected. Installing backend dependencies...
    call npm install --legacy-peer-deps
)
start /min "POS Backend Server" cmd /k "npm start"
cd ..

cd frontend
IF NOT EXIST "node_modules" (
    echo First time run detected. Installing frontend dependencies...
    call npm install --legacy-peer-deps
)
start /min "POS Frontend Server" cmd /k "npm run dev"
cd ..

timeout /t 4 >nul
start http://localhost:5173

exit
```

FILE: pos/stop-pos.bat
==========

```
@echo off
echo Stopping POS System...
taskkill /F /IM node.exe /T
echo.
echo All POS servers have been stopped.
timeout /t 3
```